Low-Level Design Handbook — Study Edition

OOP · UML · SOLID · Clean Code · Refactoring · Patterns · Concurrency · 45 case studies in Java — every topic, readable length

22 parts · one place to study LLD: DesignGurus courses + classic books + GitHub problem sets, combined

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 1 — The LLD Interview & How to Approach It (Study Edition)**
   - 1.1 What this round is, and what this handbook covers
   - 1.2 How the interview runs
   - 1.3 A complete answer in five minutes: "Design a music playlist"
   - 1.4 The step-by-step answer framework
   - 1.5 From one line to a list of requirements
   - 1.6 Finding the classes
   - 1.7 Connecting the classes (preview)
   - 1.8 Keeping the design honest with SOLID
   - 1.9 The design patterns that come up in this round
   - 1.10 Code, edge cases and concurrency: how to write interview code
   - 1.11 Worked mini-example, end to end: a small parking lot
   - 1.12 Communication tips
   - 1.13 How to use this handbook, and a study plan
   - 1.14 Common mistakes
   - In the interview
   - Self-check
   - TL;DR
2. **Chapter 2 — OOP Fundamentals (Study Edition)**
   - 2.1 What object-oriented programming is
   - 2.2 Classes, objects, constructors, `this`, `static` and `final`
   - 2.3 Encapsulation
   - 2.4 Abstraction
   - 2.5 Inheritance
   - 2.6 Polymorphism (and the full bank example)
   - 2.7 Compile-time vs runtime polymorphism: overloading vs overriding
   - 2.8 Interfaces (default, static, private methods, functional interfaces)
   - 2.9 Abstract classes vs interfaces
   - 2.10 Enums with fields and behaviour
   - 2.11 Composition over inheritance
   - 2.12 Object identity vs equality: the `equals` / `hashCode` contract
   - 2.13 Immutability and records
   - 2.14 Access modifiers and packages
   - 2.15 Generics basics for LLD
   - 2.16 Coupling and cohesion (the yardsticks)
   - 2.17 Object-oriented analysis and design (OOA / OOD / OOP)
   - Chapter summary: in the interview
   - Self-check (whole chapter)
   - TL;DR
3. **Chapter 3 — Class Relationships & UML (Study Edition)**
4. **Part A — Class Relationships**
   - 3.1 The big picture: what a "relationship" really claims
   - 3.2 Association (uni-directional, bi-directional, multiplicity)
   - 3.3 Aggregation
   - 3.4 Composition
   - 3.5 Dependency
   - 3.6 Generalization (inheritance)
   - 3.7 Realization (interface implementation)
   - 3.8 Decision guide: choosing the relationship between two classes
5. **Part B — UML**
   - 3.9 What UML is (structural vs behavioural diagrams)
   - 3.10 Use-case diagrams
   - 3.11 Class diagrams
   - 3.12 Sequence diagrams
   - 3.13 Activity diagrams
   - 3.14 State machine diagrams
6. **Part C — UML in the interview**
   - 3.15 How much UML to draw in an interview
   - 3.16 Chapter self-check
   - 3.17 TL;DR
7. **Chapter 4 — SOLID Principles (Study Edition)**
   - 4.1 What SOLID is and why you should learn it
   - 4.2 Single Responsibility Principle (SRP)
   - 4.3 Cohesion
   - 4.4 Coupling
   - 4.5 Separation of Concerns (SoC)
   - 4.6 SRP vs cohesion vs coupling vs SoC — one table
   - 4.7 Open/Closed Principle (OCP)
   - 4.8 Liskov Substitution Principle (LSP)
   - 4.9 Interface Segregation Principle (ISP)
   - 4.10 Dependency Inversion Principle (DIP)
   - 4.11 Refactoring one class through all five principles
   - 4.12 SOLID and design patterns
   - 4.13 SOLID in the design interview
   - 4.14 Common questions about SOLID (with short answers)
   - 4.15 Over-engineering: the warnings in one place
   - Chapter self-check
   - TL;DR
8. **Chapter 5 — More Design Principles (Study Edition)**
   - 1. DRY — Don't Repeat Yourself
   - 2. KISS — Keep It Simple, Stupid
   - 3. YAGNI — You Aren't Gonna Need It
   - 4. Law of Demeter (Principle of Least Knowledge)
   - 5. Tell, Don't Ask
   - 6. Composition over Inheritance

- 7. Program to an Interface, Not an Implementation
- 8. Encapsulate What Varies
- 9. Strive for Loosely Coupled Designs
- 10. The Hollywood Principle (Inversion of Control)
- 11. Separation of Concerns (SoC)
- 12. Principle of Least Astonishment (POLA)
- 13. Fail Fast
- 14. Immutability
- 15. Design by Contract (DbC)
- 16. GRASP — General Responsibility Assignment Software Patterns
  - 16.1 Information Expert
  - 16.2 Creator
  - 16.3 Controller
  - 16.4 Low Coupling
  - 16.5 High Cohesion
  - 16.6 Polymorphism
  - 16.7 Pure Fabrication
  - 16.8 Indirection
  - 16.9 Protected Variations
- 17. Domain-Driven Design (DDD) Essentials
- Chapter summary — how the principles fit together

9. **Chapter 6 — Clean Code (Study Edition)**
   - What is clean code, and why bother?
   - 1. Meaningful Names
   - 2. Functions
   - 3. Comments
   - 4. Formatting
   - 5. Objects and Data Structures
   - 6. Error Handling
   - 7. Boundaries
   - 8. Unit Tests
   - 9. Classes
   - 10. Systems
   - 11. Emergence — Kent Beck's Four Rules of Simple Design
   - 12. Concurrency
   - 13. Successive Refinement and the Case Studies (Chapters 14–16: how clean code is actually written)
   - 14. Smells and Heuristics (the checklist)
   - 15. Code Review Checklist (best practices)
   - 16. Error-Handling Strategy (best practices for real services)
   - 17. Testing Strategies (unit vs integration, test doubles, mocks)
   - Chapter summary
10. **Chapter 7 — Refactoring (Study Edition)**
    - 7.1 Refactoring fundamentals
    - 7.2 Code smells
    - 7.3 The refactoring catalogue
    - 7.4 Smell → refactorings lookup table
    - 7.5 Worked example — from messy to clean in six compiling steps
    - 7.6 Refactoring toward patterns
    - 7.7 Putting it together
    - 7.8 Full source code and output
11. **Chapter 8 — Design Patterns: Introduction & Creational Patterns (Study Edition)**
12. **Part A — Design patterns: the big picture**
    - 8.1 What is a design pattern?
    - 8.2 A short history: from buildings to the "Gang of Four"
    - 8.3 The three categories and all 23 GoF patterns
    - 8.4 The OO principles behind every pattern
    - 8.5 How to pick a pattern
    - 8.6 Anti-patterns: when patterns go wrong
13. **Part B — Creational patterns**
    - 8.7 Creational patterns at a glance
    - 8.8 Singleton
    - 8.9 Simple Factory (an idiom, not a GoF pattern)
    - 8.10 Factory Method
    - 8.11 Abstract Factory
    - 8.12 Builder
    - 8.13 Prototype
    - 8.14 Bonus — Object Pool
    - 8.15 Creational patterns compared
    - 8.16 Chapter self-check
    - 8.17 Chapter TL;DR
14. **Chapter 9 — Structural Design Patterns (Study Edition)**
    - 9.0 What structural patterns are
    - 9.1 Adapter Pattern
    - 9.2 Bridge Pattern
    - 9.3 Composite Pattern
    - 9.4 Decorator Pattern
    - 9.5 Facade Pattern
    - 9.6 Flyweight Pattern
    - 9.7 Proxy Pattern
    - 9.8 Putting it all together — comparison
15. **Chapter 10 — Behavioral Design Patterns (Part 1) · Study Edition**
    - 10.0 Behavioral patterns: the big picture
    - 10.1 Chain of Responsibility
    - 10.2 Command
    - 10.3 Iterator
    - 10.4 Mediator
    - 10.5 Memento
    - 10.6 Interpreter
    - 10.7 Chapter recap: the six patterns side by side
16. **Chapter 11 — Behavioral Design Patterns (Part 2) & Beyond GoF · Study Edition**
    - 11.1 Observer
    - 11.2 State
    - 11.3 Strategy
    - 11.4 Template Method
    - 11.5 Visitor
    - 11.6 GoF behavioral patterns: quick-reference summary

- 11.7 Beyond GoF: patterns every LLD interview expects
- 11.8 Choose the right behavioral pattern

17. **Chapter 12 — Concurrency & Multithreading for LLD (Java) · Study Edition**
18. **Part A — Foundations**
    - 12.1 Concurrency vs parallelism
    - 12.2 Processes vs threads
    - 12.3 Thread lifecycle & states
    - 12.4 Creating threads: `Thread`, `Runnable`, `Callable`, `ExecutorService`, virtual threads
19. **Part B — What goes wrong**
    - 12.5 Race conditions & critical sections
    - 12.6 Java Memory Model basics: visibility, happens-before, `volatile`
20. **Part C — Locks**
    - 12.7 Mutex (mutual exclusion) and `synchronized`
    - 12.8 `ReentrantLock` and reentrancy
    - 12.9 Try-lock & timed locking
    - 12.10 Read-write locks: `ReadWriteLock` and `StampedLock`
    - 12.11 Coarse-grained vs fine-grained locking
21. **Part D — Coordination**
    - 12.12 Semaphores
    - 12.13 Condition variables: `wait/notify` and `Condition`
    - 12.14 `CountDownLatch` and `CyclicBarrier`
22. **Part E — Lock-free**
    - 12.15 Compare-and-swap (CAS) & atomics
    - 12.16 Concurrent collections
23. **Part F — Liveness failures**
    - 12.17 Deadlock
    - 12.18 Livelock
    - 12.19 Starvation (and fairness)
24. **Part G — Concurrency patterns**
    - 12.20 Signaling pattern
    - 12.21 Thread pool pattern
    - 12.22 Producer-consumer pattern
    - 12.23 Reader-writer pattern
    - 12.24 Immutability & thread confinement
25. **Part H — Concurrency in LLD interviews**
    - 12.25 How concurrency shows up in LLD interviews (parking, seat booking, inventory)
    - 12.26 Choosing the right tool (cheat sheet)
    - 12.27 Chapter self-check
    - 12.28 Chapter TL;DR
26. **Chapter 13 — Classic Concurrency Problems · Study Edition**
    - 13.1 Print FooBar Alternately (LeetCode 1115)
    - 13.2 Print Zero Even Odd (LeetCode 1116)
    - 13.3 Fizz Buzz Multithreaded (LeetCode 1195)
    - 13.4 Building H2O (LeetCode 1117)
    - 13.5 Dining Philosophers (LeetCode 1226)
    - 13.6 Thread-safe cache with TTL
    - 13.7 Design a (simplified) Concurrent HashMap — lock striping
    - 13.8 Thread-safe bounded Blocking Queue (two ways)
    - 13.9 Concurrent Bloom Filter
    - 13.10 Multi-threaded Merge Sort (Fork/Join)
    - 13.11 Rate limiter — token bucket (thread-safe)
    - 13.12 Chapter summary
    - 13.13 Chapter self-check
    - 13.14 Chapter TL;DR
27. **Chapter 14 — Case Studies: Everyday Objects (Study Edition)**
    - 14.1 Design a Parking Lot
    - 14.2 Design a Vending Machine
    - 14.3 Design a Coffee Vending Machine
    - 14.4 Design an ATM
    - 14.5 Design an Elevator System
    - 14.6 Design a Traffic Signal Control System
    - 14.7 Design a Shipping Locker System
    - Chapter summary
28. **Chapter 15 — Case Studies: Booking & Inventory (Part 1) (Study Edition)**
    - Why booking problems are interview favourites
    - 15.1 Design a Library Management System
    - 15.2 Design a Movie Ticket Booking System (BookMyShow)
    - 15.3 Design a Concert Ticket Booking System
    - 15.4 Design a Hotel Management System
    - Chapter summary
29. **Chapter 16 — Case Studies: Booking & Inventory (Part 2) — Study Edition**
    - Design a Car Rental System
    - Design a Restaurant Management System
    - Design an Airline Management System
    - Design a Course Registration System
    - Chapter summary
30. **Chapter 17 — Case Studies: Games & Rules — Study Edition**
    - Design Blackjack and a Deck of Cards
    - Design Chess
    - Design Tic-Tac-Toe
    - Design Snake and Ladder

- Design 2048
- Chapter summary

31. **Chapter 18 — Case Studies: Platforms & Feeds (Part 1) — Study Edition**
    - 18.0 What platform problems test
    - 18.1 Design Stack Overflow
    - 18.2 Design Cricinfo
    - 18.3 Design LinkedIn
    - 18.4 Design Facebook
    - 18.5 Chapter summary
32. **Chapter 19 — Case Studies: Commerce & Marketplaces (Part 2 of Platforms) — Study Edition**
    - Design Amazon — Online Shopping System
    - Design an Online Stock Brokerage System
    - Design an Online Auction System
    - Design a Music Streaming Service (Spotify)
    - Design an Online Food Delivery Service (Swiggy) + FoodKart
    - Design a Grocery Store (POS & Checkout)
    - Chapter summary
33. **Chapter 20 — Case Studies: Pattern-Driven Components (Study Edition)**
    - 20.1 Design Splitwise
    - 20.2 Design a Notification Service
    - 20.3 Design an In-Memory Cache (and an LRU Cache from scratch)
    - 20.4 Design a Ride Sharing Service (Uber)
    - 20.5 Design a Logging Framework
    - 20.6 Design a Pub-Sub System
    - 20.7 Design a Rate Limiter
    - 20.8 Chapter summary
    - 20.9 Chapter self-check
    - 20.10 Chapter TL;DR
34. **Chapter 21 — Case Studies: Machine-Coding Round Problems (Study Edition)**
    - 21.0 How the machine-coding round works
    - 21.1 Design a Task Management System
    - 21.2 Design a Digital Wallet Service
    - 21.3 Design an Event Calendar (Flipkart)
    - 21.4 Design a LeetCode-like Coding Platform — "Coding Blox" (Flipkart)
    - 21.5 Design a Property Listing Site — "PropertyHunt" (ClearTrip)
    - 21.6 Design a Loan Ledger — "Ledger Co" (Navi)
    - 21.7 Design an Inventory Management System
    - 21.8 Design a Unix File Search (`find`)
    - Chapter summary
35. **Chapter 22 — Revision: Cheat Sheets & Glossary**
    - 22.1 How to revise
    - 22.2 The LLD interview answer framework (one page)
    - 22.3 OOP & relationships cheat sheet
    - 22.4 SOLID one-pager
    - 22.5 Other design principles and Clean Code rules
    - 22.6 Code smells → refactorings (all 24)
    - 22.7 Design patterns: all 23 GoF + extras
    - 22.8 Concurrency cheat sheet
    - 22.9 Case-study map
    - 22.10 Machine-coding and edge-case checklists
    - 22.11 Glossary
    - Self-check
    - TL;DR

Chapter 1 — The LLD Interview & How to Approach It (Study Edition)

> **Study Edition.** Every topic of the full chapter, without the padding. Full code, outputs and extra examples are in the Full Edition (`README.md`).
>
> **You will learn:** what the LLD round is, a complete small answer, an 11-step framework with a time budget, how to get requirements and classes, a SOLID check, the six common patterns, how to write interview code, a worked parking lot, communication tips and a study plan.

1.1 What this round is, and what this handbook covers

What it is

The **low-level design (LLD)** round: design **one application** while an interviewer watches, in 45–60 minutes (machine coding: 90 min–2 h). You design **the classes inside the program** (parking lot, vending machine, chess, Splitwise), not servers. Analogy: system design is the **city plan**; LLD is the **floor plan of one house**.

Term	Meaning
Class / Object	A named type with fields and methods / one concrete instance of it
Responsibility	The one job a class does, in one sentence
Relationship	How two classes connect: uses, holds, owns, or extends
UML	Standard symbols for drawing classes and relationships
Design pattern	A named, reusable answer to a recurring design problem

The hard part is **explaining aloud** why each class exists and **justifying** how they connect.

What you should be able to do after this handbook

Bound one sentence into requirements plus an out-of-scope list; pick classes with **one job** each; relate every pair correctly; code the core flows until they run; name the pattern you used **and the one you rejected**.

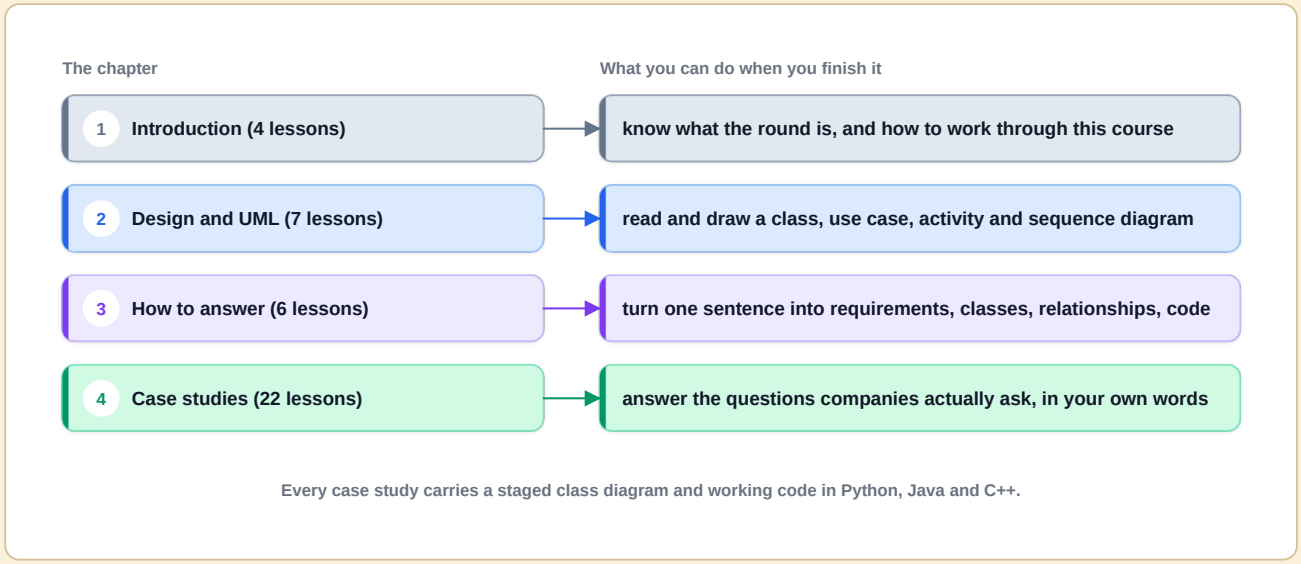


Figure — The four parts of the course material this handbook absorbs: introduction, design and UML notation, the answering method, and the case studies (course diagram)

The handbook follows this arc (round → UML → method → case studies) and adds OOP, SOLID, principles, clean code, refactoring, all 23 GoF patterns and concurrency.

What every case study in this handbook contains · Who this is for · What you need already · What this handbook will not do

- **Case-study template:** requirements with out-of-scope, actors and use cases, a staged class diagram, key flow diagrams, runnable Java with output, patterns **only where really used**, and "Extensions & follow-up questions".
- **For:** anyone facing this round. **Needed:** one OO language (examples use **Java 21**); not UML (Chapter 3), not system design (this round stays **inside one process**), not DSA beyond basic collections.
- **Won't do:** teach a language or give scripts to memorise. **The reusable part is the method.**

> **TL;DR**
>
> Design the classes of one application out loud, then code the main flows. Method beats memorisation.

1.2 How the interview runs

What it is

One sentence ("Design a parking lot"), then silence. Names for the round: **OOD interview** (US big tech), **LLD** (umbrella term), **machine-coding round** (commonly reported at Flipkart, Swiggy, PhonePe: **runnable code is mandatory**, 90 min–2 h).

	LLD	HLD / system design
Scope / vocabulary	One application: classes, interfaces, patterns	Many machines: servers, DBs, caches, queues
Output	Class diagram + code	Architecture + capacity numbers

OOD round vs machine-coding round

	OOD / LLD discussion	Machine coding
Time / output	45–60 min; requirements, diagram, code for 2–3 flows	~90 min coding; a **complete running program**
Storage / interaction	Rarely discussed; continuous conversation	**In-memory**; silent coding, then **demo + walkthrough**
Judged on	Bounding, responsibilities, relationships, trade-offs	Working code, **data models**, **service interfaces**, **extensibility**, readability, edge cases, sometimes thread safety
Common failure	Not converging, coding too early	Not finishing, or one giant class

How it works: the five core steps

1. **Clarify requirements** 2. **Actors and use cases** 3. **Find the classes** 4. **Connect them** (class diagram) 5. **Write the code** for 2–3 main flows.

Order matters more than speed: skip step 1 and step 3 invents classes nobody asked for.

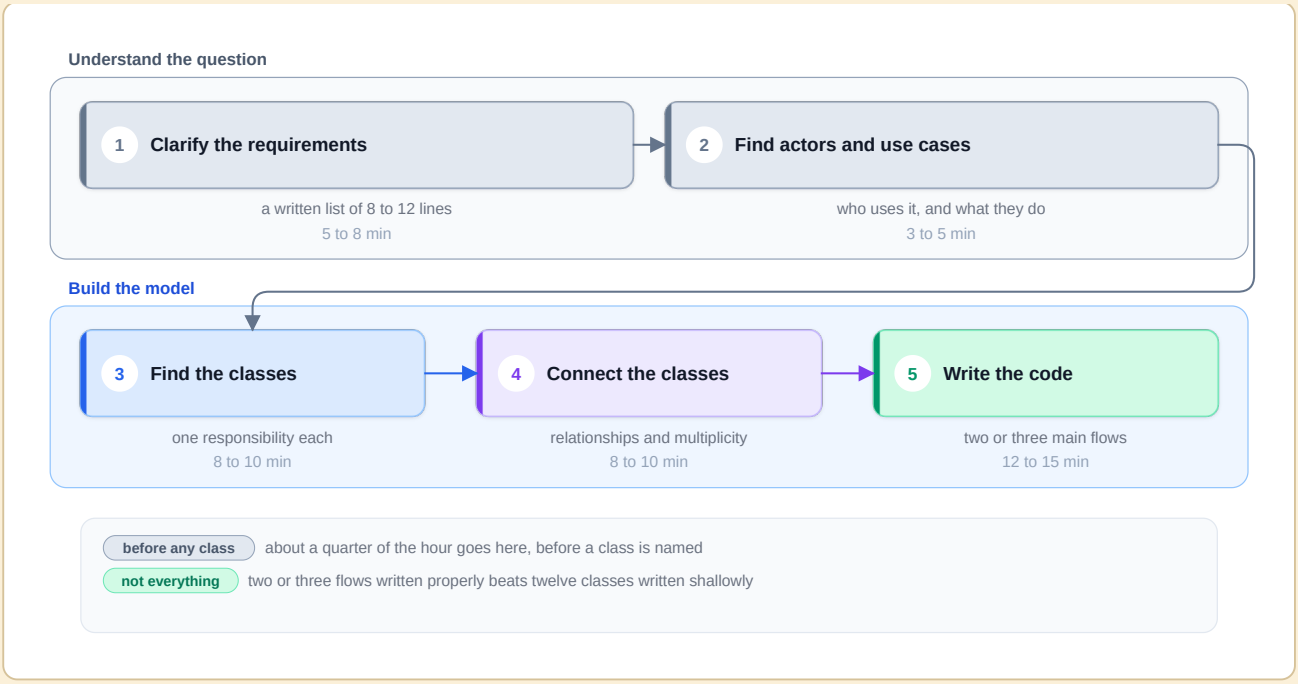


Figure — The five steps split into "understand the question" and "build the model", with a time budget for each (course diagram)

About **a quarter of the hour passes before any class is named**. **Nobody finishes the product**; you are judged on the part you built properly.

Time-boxing

45-min OOD step	Time	Output
Requirements	5–8 min	8–12 lines + out-of-scope
Actors & use cases	3–5 min	2–4 actors, 6–10 use cases
Classes	8–10 min	One responsibility each
Connect	8–10 min	Diagram with multiplicity
Code	12–15 min	2–3 main-flow methods

90-min machine coding: requirements/assumptions 5–10 · model (entities, enums, interfaces) 10–15 · core flow end to end 30–40 · remaining features + edge cases 15–20 · polish + demo 5–10 · buffer 5 (never zero). **Get something running by halfway.**

What the interviewer is actually judging

1. Did you **bound** the problem? 2. **One job** per class? 3. **Right relationships**? 4. Does **code match the diagram**? 5. Can you explain a **trade-off**?

Machine coding adds: **runs**, **modular** (new type = new class), readable, handles bad input, thread-safe (senior). Perfect UML notation is **not** graded; the model is.

A worked example of size

Parking lot: ~12 requirements → ~12 classes → ~15 relationships. Code the three that carry the main flows; name the other nine. Costliest mistake: all twelve at equal depth.

> **TL;DR**
>
> Five steps in order, a quarter of the time before naming a class, two or three flows done properly.

1.3 A complete answer in five minutes: "Design a music playlist"

Step 1: requirements

Songs are **ordered**; a listener asks for **the next song**; **shuffle** changes the order. **Out of scope (said aloud):** accounts, downloads, storage, audio.

Step 2: actors and use cases

One **actor** (someone outside the app): the listener (add song, play next, shuffle). On bigger problems **actors reveal which classes must exist**.

Step 3: classes, one sentence each

Class	Responsibility
`Song`	Title, artist, length; knows only itself
`Playlist`	Holds songs in order; stores, does not play
`PlayOrder` (interface)	Picks the next position (several rules exist)
`InOrder` / `Shuffled`	Insertion order / an order fixed up front
`Player`	Holds playlist + order, remembers the position

A rule that can vary becomes its own type. If a responsibility needs "and" to join two jobs, it is two classes.

Step 4: connect them

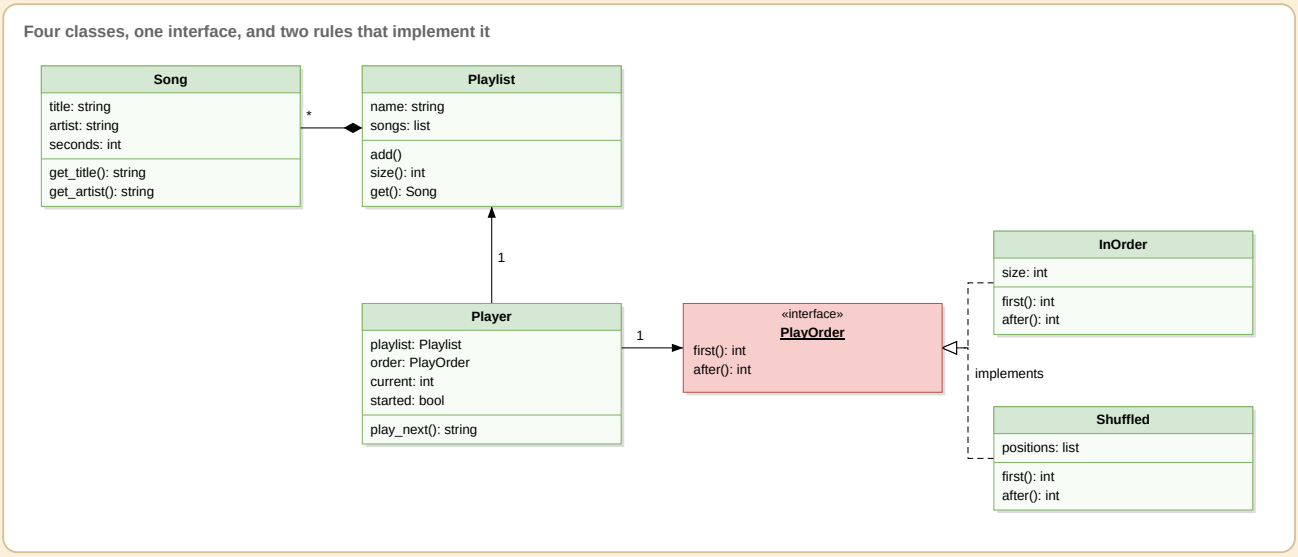
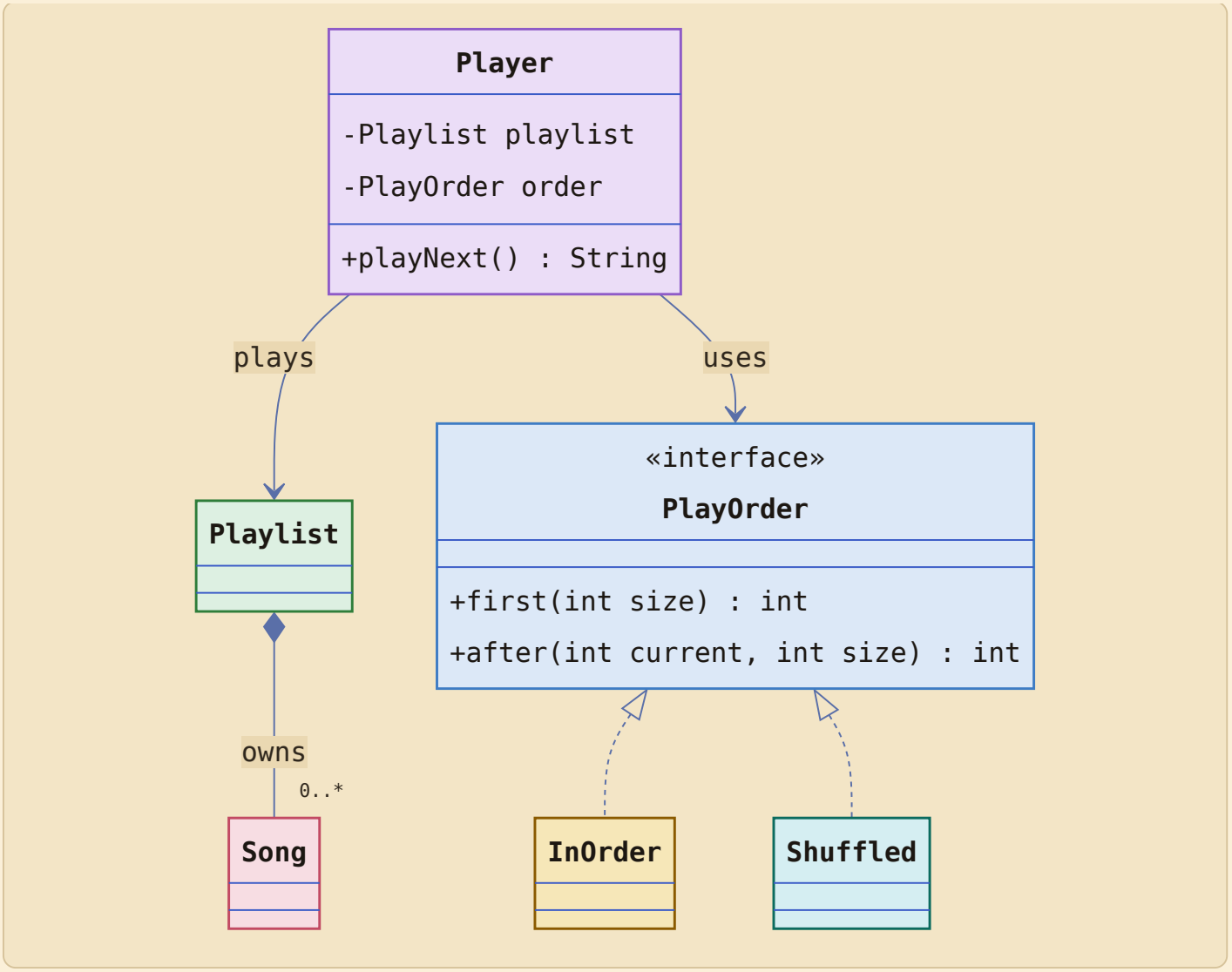


Figure — The playlist design: Playlist owns many Songs, Player uses a PlayOrder, and InOrder and Shuffled implement PlayOrder (course diagram)

Filled diamond = `Playlist` **owns** many `Song`s (composition). Plain arrow = `Player` **uses** the `PlayOrder` interface only. Dashed hollow-triangle arrows = `InOrder` / `Shuffled` **implement** it; a third rule is one more box.



Step 5: the code

```java
// File: code/lld/ch01/playlist/PlayOrder.java
public interface PlayOrder {
    int NONE = -1;                          // "there is no next position"

    int first(int size);                    // first position to play, or NONE if empty
    int after(int current, int size);       // position after 'current', or NONE at the end
}
```

```java
// File: code/lld/ch01/playlist/Player.java
public final class Player {
    private final Playlist playlist;
    private final PlayOrder order;           // depends on the interface, not on InOrder/Shuffled
    private int current = PlayOrder.NONE;
    private boolean started = false;
    // ... constructor

    public String playNext() {
        if (!started) {
            current = order.first(playlist.size());
            started = true;
        } else if (current != PlayOrder.NONE) {          // once at the end, stay at the end
            current = order.after(current, playlist.size());
        }
        if (current == PlayOrder.NONE) return "end of playlist";
        Song s = playlist.get(current);
        return "playing " + s.title() + " by " + s.artist();
    }
}
```

Full runnable code: code/lld/ch01/playlist/ (see the Full Edition).

`Song` is a validating **record**; `Playlist` keeps its list `private`; `Shuffled` is **given** its order (not `Random`), so output is the same every run: **deterministic = testable**. `Player` never checks which mode is active.

What to notice

- **Shuffle (one line) became a type**: classes come from requirements.
- **One job per class**: `Song` doesn't know playlists exist.
- **Open to change**: "repeat one" is a new `PlayOrder` (Strategy + Open/Closed).
- **Announce deliberate gaps** (pause, volume, queue).

TL;DR

Bounded requirements, one-sentence classes, one diagram, code for the main flow. A varying rule becomes an interface.

1.4 The step-by-step answer framework

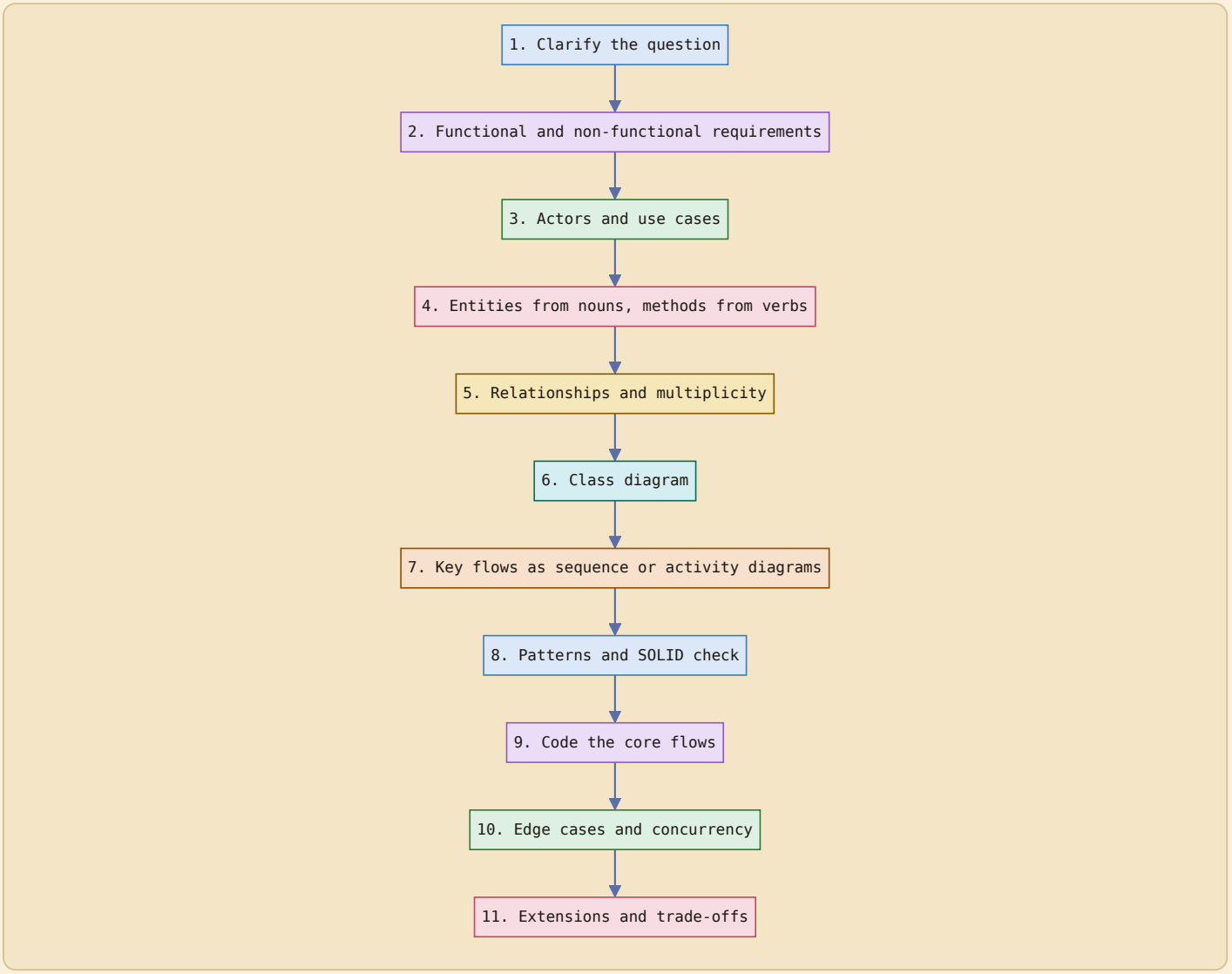
What it is

A checklist for every question, expanding the five steps into eleven.

The problem it solves

Stops freezing and premature coding; makes progress **visible** (which is scored).

How it works



Ask at step 8: varying rule (Strategy)? lifecycle (State)? many listeners (Observer)? At step 10: invalid input, full, two users on one seat?

Trade-offs

Do this	Instead of	Why
8–13 min on steps 1–3	Coding at minute 2	Unbounded scope costs more later
2–3 flows fully	12 stubs	Depth is judged
1–2 patterns with a signal	Six patterns	Over-engineering = poor judgement
Pass time in; `BigDecimal`/cents	Clock in logic; `double` money	Testable; `double` can't hold 0.10 exactly

When not to use it (rigidly)

Follow the interviewer if they steer. In machine coding, steps 5–7 are often mental; **the code is the diagram**.

TL;DR

Clarify → requirements → use cases → entities → relationships → diagram → flows → patterns/SOLID → code → edge cases → extensions. Announce each step.

1.5 From one line to a list of requirements

What it is

A **requirement** is one testable capability. **Functional** = behaviour ("customer takes a ticket"). **Non-functional** = quality (thread safety, extensibility, in-memory).

The problem it solves

Without a list the design has **no edges**, you **can't defend omissions** ("reservations were out of scope; we agreed"), and you **can't tell when you're done**.

How it works: the four questions

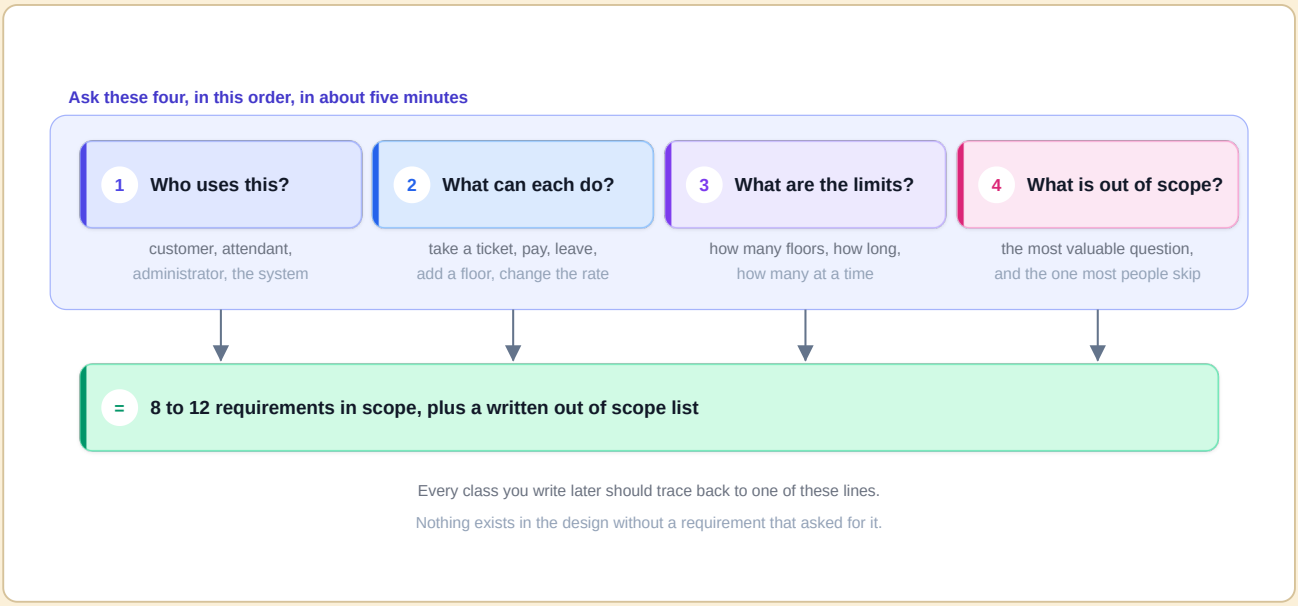


Figure — Four questions asked in order produce 8–12 in-scope requirements plus a written out-of-scope list (course diagram)

1. **Who uses it?** (actors) 2. **What can each do?** (use cases; most requirements) 3. **What are the limits?** (→ constants + validation) 4. **What is out of scope?** (the only question that makes the problem **smaller**).

Ask early: payments in scope? one site or many? users/permissions modelled? Each answer adds or removes 2–3 classes. If told "your choice": **choose, say it, write it down**.

What good looks like: the parking lot in ten lines

In scope: multiple floors of spots; spot types (compact, large, handicapped, motorbike, electric); vehicle types; ticket at entrance; pay at exit/portal/attendant; cash or card; hourly rate by stay length; free-spot display per floor; refuse entry when full; admins add floors/spots/panels and change rates. **Out of scope:** season passes, plate recognition, pre-booking, refunds, multi-site reports. **If no line asked for it, it's not in the design.**

Make each requirement checkable

Put a number on it ("at most **five** books on loan" → constant → one check) and **name the actor** ("**a member** can reserve a copy on loan" → tells you the owning class).

Use cases

Customer: take ticket, park, pay, exit. Attendant: collect cash. Admin: add floor/spot/panel, change rate. System: assign spot, update boards, refuse when full. A list is enough in 45 minutes.

Common mistakes

Saying yes to every idea (declining politely is a **strong** move); forgetting non-functional requirements; vague lines without number or actor.

In the interview

> **30-second answer:** "I ask who uses it, what each actor does, the limits, and what's out of scope. I write 8–12 numbered requirements plus an out-of-scope list, confirm them, and every class must trace back to a line."

- *"Why not just start designing?"* — No boundary means endless growth and indefensible omissions.
- *"'Your choice'?"* — I pick the simpler option and write it as a visible assumption.

Self-check

- **Q:** Which question shrinks the problem? **A:** Out of scope.
- **Q:** Make "users can book some seats" checkable. **A:** "A customer can hold at most 10 seats per show; unpaid holds release after 10 minutes."

TL;DR

Who, what, limits, out of scope → 8–12 checkable lines plus out-of-scope.

1.6 Finding the classes

What it is

Collect candidates, cut non-classes, give each survivor one job.

How it works

1. **Nouns** → candidate classes (deliberately too many). 2. **Verbs** → methods; a verb with no home = a missing class. 3. **Cut with five tests:**

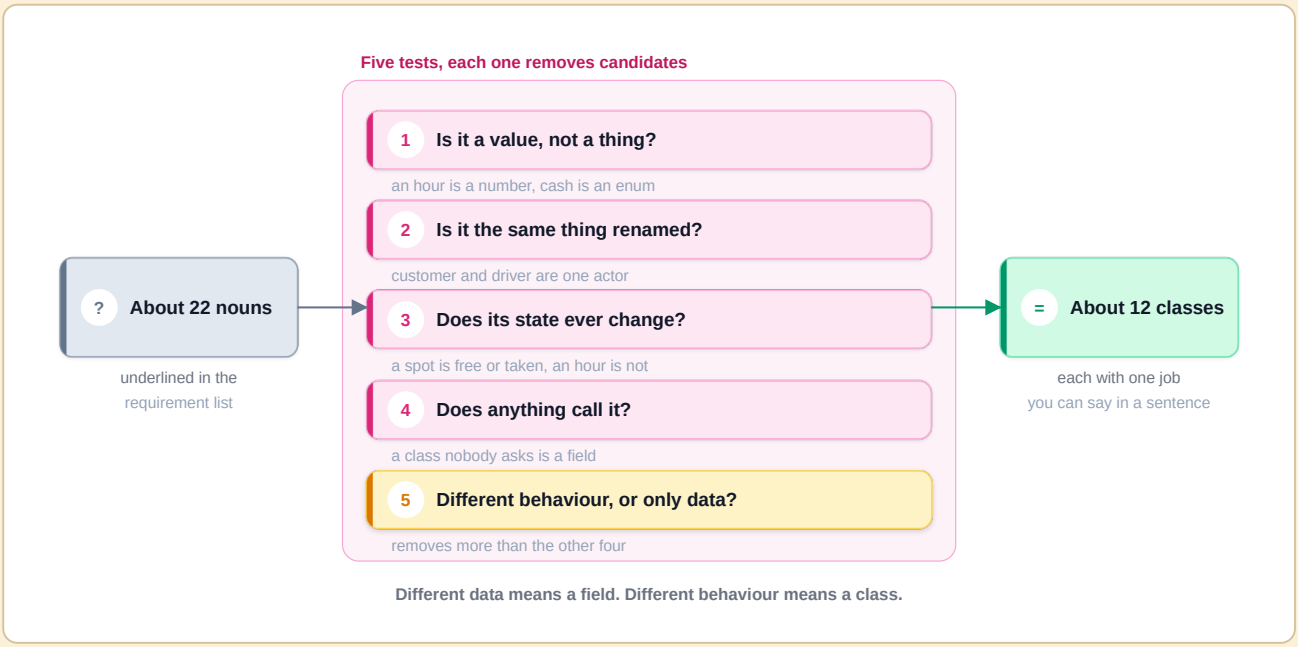


Figure — About 22 nouns pass through five tests and about 12 classes survive, each with one job (course diagram)

#	Test	Example	Result
1	A **value**?	Hour, plate, cash/card option	Field or **enum**
2	**Another name**?	Customer/driver	Merge
3	**Changing state**?	Spot free/occupied	Class
4	Anything **calls** it?	Nobody asks an `Hour`	Value
5	**Behaviour** or only **data** differs?	Compact vs large spot (data); cash vs card (behaviour)	**Data → field. Behaviour → class.**

Test 5 cuts the most: one `ParkingSpot` with a `type` field, but separate cash and card classes. Smell: subclasses that only pass a constant to the parent.

4. **One sentence per class**, e.g. `ParkingFloor` owns a level's spots and finds a free one; `ParkingSpot` tracks its occupant; `ParkingTicket` records vehicle, spot, entry time; `ParkingRate` turns duration into price; `Payment` collects money; `ParkingDisplayBoard` shows free spots. `ParkingLot` (issues tickets **and** tracks fullness) has two jobs: **name the tension aloud**.
5. **Method goes where its data is** ("find a free spot" → `ParkingFloor`). Needing three classes' data = misplaced (GRASP Information Expert; *Feature Envy*).

Common mistakes

One subclass per value; missing the **record-keeping** class (ticket/loan/booking) and the **rule** class (rate/strategy); "Manager" god classes.

Self-check

- **Q:** Car, van, truck: subclasses or enum? **A:** Enum/field unless behaviour truly differs.
- **Q:** Class with fields but no verbs? **A:** Probably a value (record/field).

TL;DR

Nouns → candidates, verbs → methods. **Different data = field; different behaviour = class.** Method where its data is.

1.7 Connecting the classes (preview)

For each pair: **uses** briefly (dependency), **holds** (association), **contains** parts that live alone (aggregation), **owns** parts that die with it (composition), or **is-a** (inheritance/realization); then **multiplicity** (1, 0..1, 1..*, *). Chapter 3 details all. Top error: inheritance where composition fits. A car is not an engine, so `Car` **has an** `Engine`.

1.8 Keeping the design honest with SOLID

What it is

The five SOLID rules (Chapter 4) used as **five quick checks** on your diagram (~5 minutes).

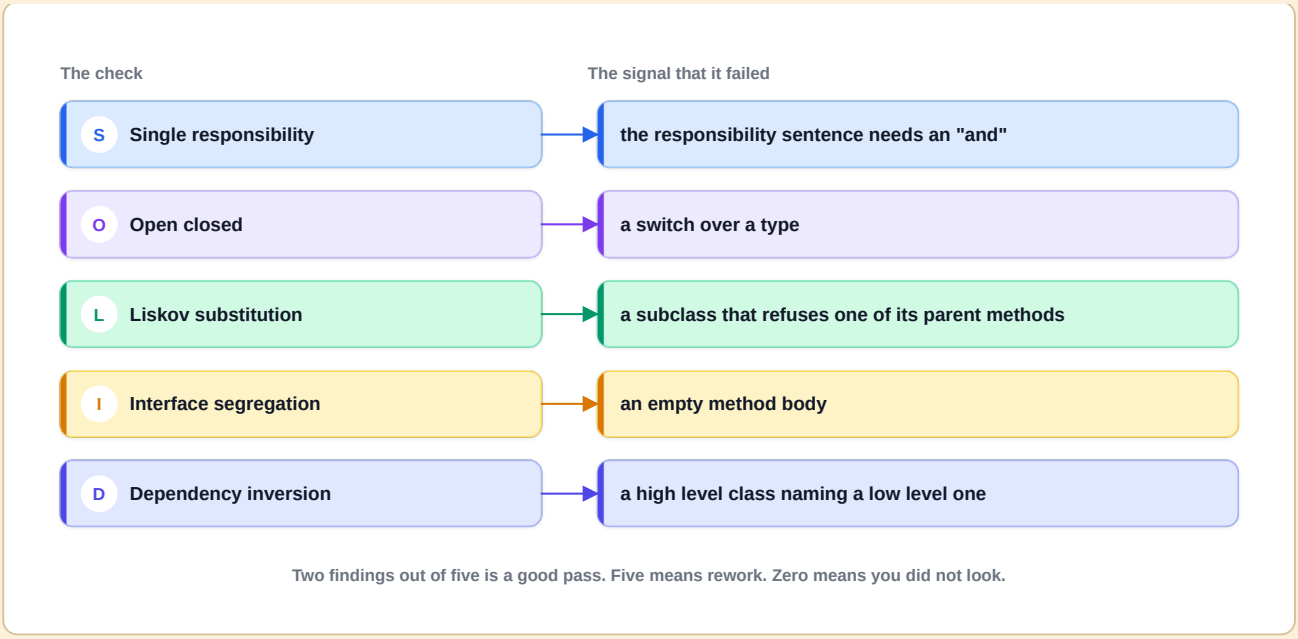


Figure — The five SOLID checks and the signal that each one has failed (course diagram)

How it works: run the five checks in this order

Check	Failure signal	Parking-lot example
S One reason to change	Sentence needs "and"	Lot that assigns spots **and** prints receipts → split
O Extend without editing	`switch` over a type	Weekend rate = new class, not new case
L Child works wherever parent does	Child throws / does nothing	`ReadOnlySpot` whose `occupy` throws
I No unused methods	Empty bodies	Split an interface half its implementers can't honour
D Depend on interfaces	High-level class does `new` of low-level	Lot depends on `Payment`, not `new CreditCardTransaction()`

The two that earn the most credit

S improves the class list; **O** is what gets probed ("trucks charged differently?" → "add a class").

When not to use it

No single-implementer interfaces "because SOLID" (unless a test double needs one); no splitting past what a reader can follow.

A five-minute pass over the parking lot

S: lot has two jobs (split or say it). O: pricing behind `ParkingRate`, passes. L: five behaviour-less spot subclasses aren't a Liskov violation, but they fail "different data → field": collapse. I, D: pass. **Two findings is good; five = rework; zero = you didn't look.**

TL;DR

Check S-O-L-I-D before the interviewer does; S and O matter most; don't over-apply.

1.9 The design patterns that come up in this round

What it is

A **pattern** is a named solution to a recurring problem (GoF has 23; Chapters 8–11). Six recur here. **A misapplied pattern hurts more than an unknown one**: learn the **signal**.

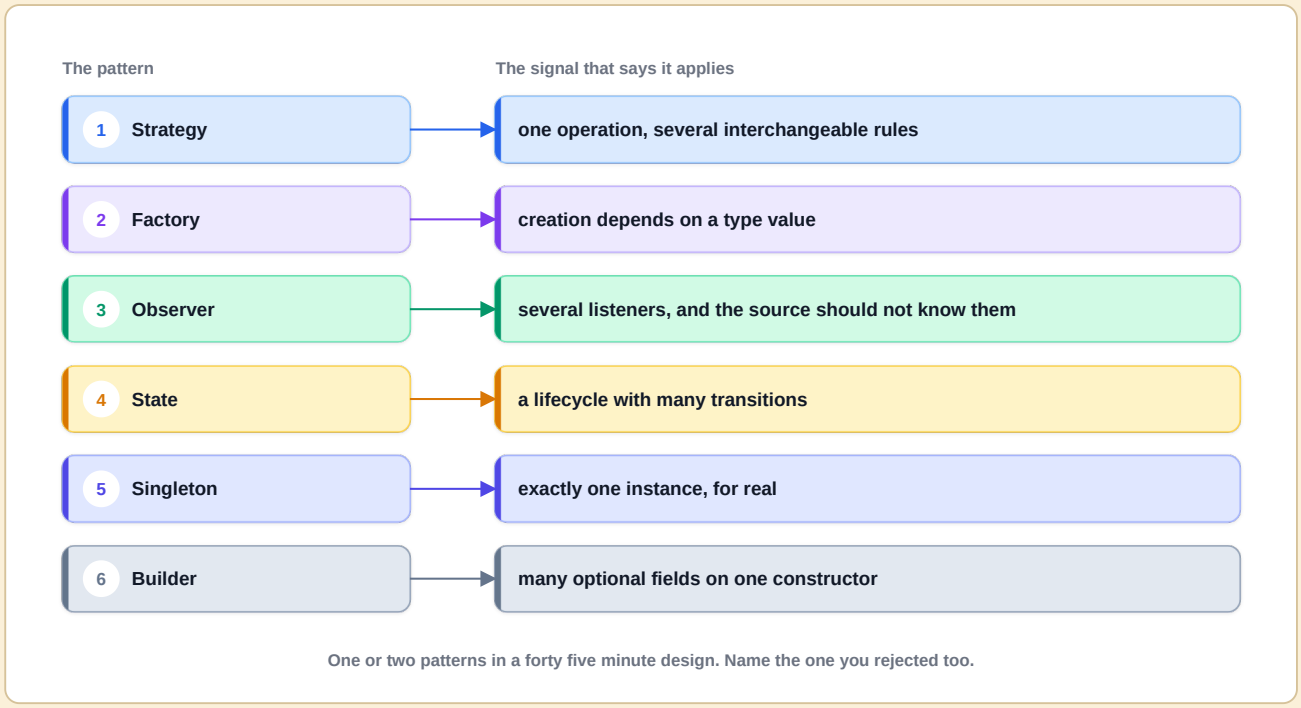


Figure — The six patterns most used in this round and the signal for each (course diagram)

What to say about each one

Pattern	Signal	What to say
Strategy	Several swappable rules for one operation	`ParkingRate` → `HourlyRate`, `WeekendRate`; a new rate is a new class, not a `switch` case
Factory	Object to build depends on a type value	Registering a new type happens in **one place**; a factory method is usually enough
Observer	One change, many listeners the source shouldn't know	The spot is the subject; boards register. One listener → direct call
State	Behaviour depends on state, fixed transitions	Illegal moves become impossible; for 3–4 states, **enum + guarded method**
Singleton	Exactly one instance	Must be **thread-safe**; **global state** hurts tests; often inject one instance instead
Builder	Many optional fields (~6+ constructor args)	Sets fields by name; a readability fix

Choosing patterns without overusing them

Skip each when its signal is absent: one rule (Strategy), one concrete class (Factory), one listener (Observer), three states and one guard (State), just wanting global access (Singleton), three required fields (Builder). **One or two patterns** per design; best is a comparison: "Rate gets Strategy; ticket doesn't get State, one guard covers three statuses."

Real-world examples

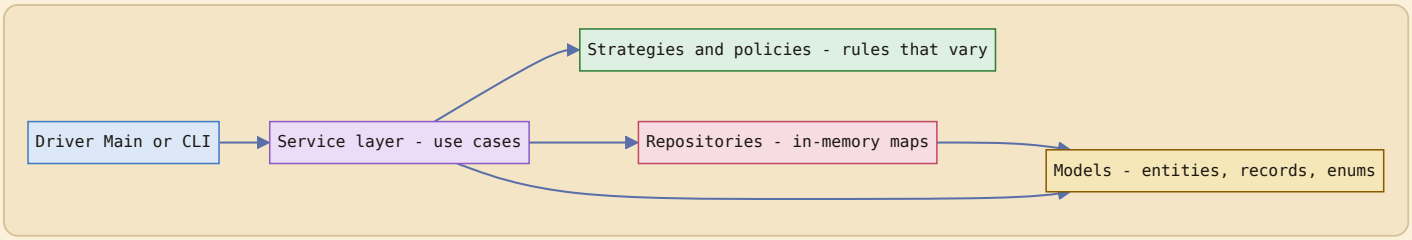
`Comparator` (Strategy); `List.of`, `Executors.newFixedThreadPool` (Factory); `ActionListener`, Spring events (Observer); order lifecycle, TCP (State); `Runtime.getRuntime()` (Singleton; Spring beans are singleton **per**

container); `StringBuilder`, `HttpRequest.newBuilder()` (Builder).

> **TL;DR**
>
> Know six; use one or two where the signal appears; name the rejected one.

1.10 Code, edge cases and concurrency: how to write interview code

How it works: a machine-coding skeleton that scales



Models (validated entities, records, enums) → **repositories** (in-memory maps behind an interface) → **strategies** (rules that vary) → **services** (use cases: the **service interface**) → **driver** (`Main` demo).

Habits: pass time in; money as `BigDecimal` (from strings) or cents, never `double`; fail loudly (`IllegalArgumentException`, `IllegalStateException`, clear custom exceptions); `Optional` for normal "not found"; `private final` fields; domain names.

Edge cases and concurrency checklist

Input (null, negative, duplicate/unknown IDs) · **Capacity** (full lot, empty playlist) · **State** (pay twice, exit without entry) · **Time** (exit before entry, round part hours) · **Concurrency** (two threads, same seat: `synchronized`, `ReentrantLock`, `ConcurrentHashMap.computeIfAbsent`; lock the **smallest** thing that keeps the invariant; Chapters 12–13).

> **TL;DR**
>
> Models → repositories → strategies → services → driver; inject time, exact money, validate, guard shared state.

1.11 Worked mini-example, end to end: a small parking lot

Problem & clarifying questions

Assumed answers: single lot, one or more floors; motorbike/car/truck; MOTORBIKE/COMPACT/LARGE spots (smaller vehicles may use bigger spots); payment out of scope (we compute the fee); tiered hourly pricing, swappable; several panels act concurrently.

Requirements

1. Floors with MOTORBIKE, COMPACT, LARGE spots. 2. Motorbike → MOTORBIKE/COMPACT; car → COMPACT/LARGE; truck → LARGE; smallest first. 3. Entry assigns a spot and issues a ticket. 4. No fit → refused. 5. Exit closes ticket, frees spot, returns fee. 6. $4.00 first hour, $3.50 hours 2–3, $2.50 after; round up; minimum 1 h; trucks double. 7. Pricing swappable at run time (flat $10 weekend). 8. Free-spot count per type.

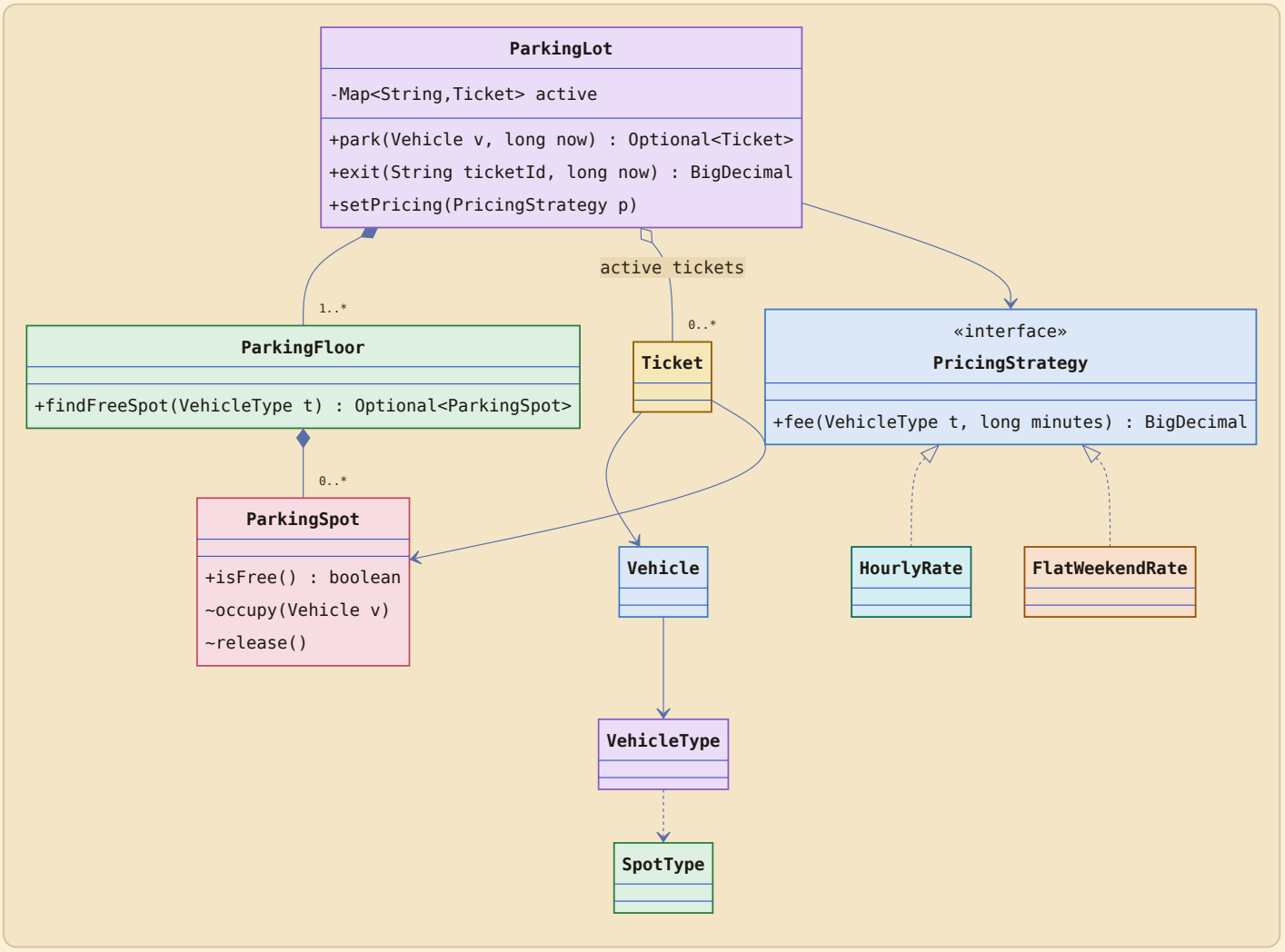
Non-functional: in-memory, thread-safe, extensible, time passed in. **Out of scope:** payment collection, reservations, boards, plate recognition, multiple lots.

Use cases · Core entities

Driver: enter/exit. Admin: add spots, change pricing. System: find spot, refuse, compute fee. Types: `SpotType`, `VehicleType` (enums; vehicle knows which spots fit), `Vehicle` and `Ticket` (records; the ticket is

the "connection object"), `ParkingSpot`, `ParkingFloor`, `PricingStrategy` + `HourlyRate`/`FlatWeekendRate`, and `ParkingLot` (the service). Spot sizes are an enum: **different data → field**.

Class diagram



Key flows

`park`: lot asks each floor `findFreeSpot(type)`; if found, `occupy` it and return a `Ticket`, else return empty (refuse). `exit`: remove the ticket, `release()` the spot, return `pricing.fee(type, minutes)`.

Design decisions & patterns used

Decision	Why	Rejected
Strategy pricing	Weekend rule required; open/closed	`switch` in the lot
Enums for types	Data only differs	Subclass per type
`Ticket` record	Fixed after issue (only *shallowly* immutable)	Mutable ticket
`BigDecimal`	`3.50 + 3.50` is exactly `7.00`	`double`
`synchronized` lot methods	Two panels never share a spot	Per-floor locks (extension)
No Singleton, no State	Injectable/testable; only active/closed	`getInstance()`; state classes

Java implementation

```java
// File: code/lld/ch01/parkinglot/HourlyRate.java
public final class HourlyRate implements PricingStrategy {
    // ... FIRST_HOUR = 4.00, HOURS_2_TO_3 = 3.50, LATER_HOURS = 2.50
    @Override
    public BigDecimal fee(VehicleType type, long minutesParked) {
        long hours = Math.max(1, (minutesParked + 59) / 60);    // ceiling, minimum one hour
        BigDecimal total = BigDecimal.ZERO;
        for (long h = 1; h <= hours; h++) {
            total = total.add(h == 1 ? FIRST_HOUR : (h <= 3 ? HOURS_2_TO_3 : LATER_HOURS));
        }
        return type == VehicleType.TRUCK ? total.multiply(BigDecimal.TWO) : total;
    }
}
```

```java
// File: code/lld/ch01/parkinglot/ParkingLot.java
public final class ParkingLot {
    private final Map<String, Ticket> active = new HashMap<>();
    private PricingStrategy pricing;
    // ... floors, nextTicket, constructor

    public synchronized Optional<Ticket> park(Vehicle v, long nowMinute) {
        for (ParkingFloor floor : floors) {
            Optional<ParkingSpot> spot = floor.findFreeSpot(v.type());
            if (spot.isPresent()) {
                spot.get().occupy(v);
                Ticket t = new Ticket("T" + nextTicket++, v, spot.get(), nowMinute);
                active.put(t.id(), t);
                return Optional.of(t);
            }
        }
        return Optional.empty();              // lot full for this vehicle type -> refuse entry
    }

    public synchronized BigDecimal exit(String ticketId, long nowMinute) {
        Ticket t = active.get(ticketId);
        if (t == null) throw new IllegalArgumentException("unknown or already closed ticket " + ticketId);
        if (nowMinute < t.entryMinute()) throw new IllegalArgumentException("exit before entry");
        active.remove(ticketId);               // validate first, then change state
        t.spot().release();
        return pricing.fee(t.vehicle().type(), nowMinute - t.entryMinute());
    }
    // ... freeSpots(type) and setPricing(p), both synchronized
}
```

Full runnable code: code/lld/ch01/parkinglot/ (see the Full Edition).

Output (excerpt): the third car is refused; `T1 exits after 150 min, pays $11.00` (4.00 + 3.50 + 3.50); the truck's 50 min → 1 h × 2 = `$8.00`; after `setPricing(new FlatWeekendRate())` the exit pays `$10.00`; a second exit of T1 prints `error: unknown or already closed ticket T1`.

Concurrency & edge cases

`synchronized park/exit` makes find-then-occupy **atomic** (no *check-then-act* race); `freeSpots` uses the same lock; `ParkingFloor` is only called under it. `exit` **validates before changing state**; `occupy` throws if taken (second defence) and is package-private.

Extensions & follow-up questions

- **EV charging:** `ELECTRIC` spot + `ELECTRIC_CAR`; if charging has behaviour, a `ChargingSpot` or composed component.
- **Nearest spot:** inject a `SpotSelectionStrategy`.
- **Display boards:** Observer on spot events.

- **Throughput:** per-floor/type locks, or a `ConcurrentLinkedQueue` of free spots with atomic `poll()`.
- **Cash/card:** `Payment` interface + implementations, collected by a `PaymentService` (DIP).

In the interview

~5 min requirements, 3 actors, 8 entities (enum vs subclass), 8 diagram, 15 coding `park`, `exit`, `HourlyRate`. Close by naming Strategy (used), Singleton and State (rejected), and per-floor locking.

Self-check

- **Q:** What race does `synchronized park` prevent? **A:** Two threads occupying the same free spot.
- **Q:** How did the weekend rate respect open/closed? **A:** A new class; nothing edited.

TL;DR

8 requirements, 10 types, one Strategy, one lock, an honest out-of-scope list.

1.12 Communication tips

Do: think out loud (name the step), write requirements/classes where visible, confirm before moving on, state assumptions, give trade-offs in pairs ("X over Y because..."), welcome hints, say what you leave out, watch the clock and announce the switch to code, use domain names (not `Manager`/`Helper`), and **when stuck, return to the requirements**. **Don't:** go silent, argue, talk `HashMap`s in minute two, or recite patterns without a reason.

1.13 How to use this handbook, and a study plan

Attempt first, then compare

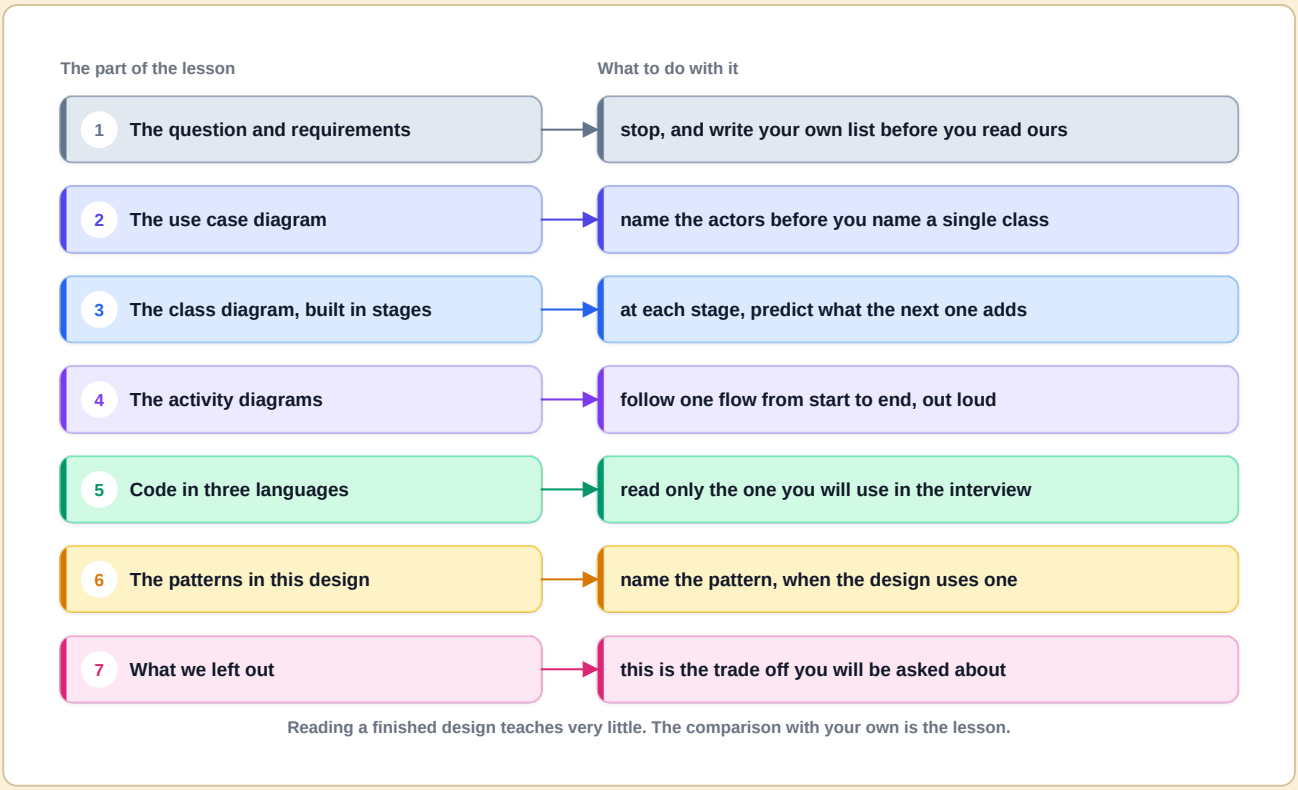


Figure — The parts of a case-study lesson and what to do at each one (course diagram)

Reading builds little skill; **attempt, then compare**. Stop (1) after the question (write requirements), (2) after the requirements (write classes; most people are **two short**: the record class and the rule class), (3) at each diagram stage (predict). Also: argue with decisions in writing, read one language, **never skip**

extensions, don't invent pattern names, switch to **timed 45-minute blank-page runs** after 6–7 case studies, run `bash tools/compile_all.sh`.

Map of this handbook

1 approach · 2 OOP · 3 relationships & UML · 4 SOLID · 5 design principles · 6 clean code · 7 refactoring · 8 creational · 9 structural · 10–11 behavioural (+MVC) · 12 concurrency concepts · 13 concurrency problems · 14 everyday objects · 15–16 booking & inventory · 17 games · 18–19 platforms · 20 components (LRU, rate limiter, Splitwise) · 21 machine coding · 22 cheat sheets.

Study plans

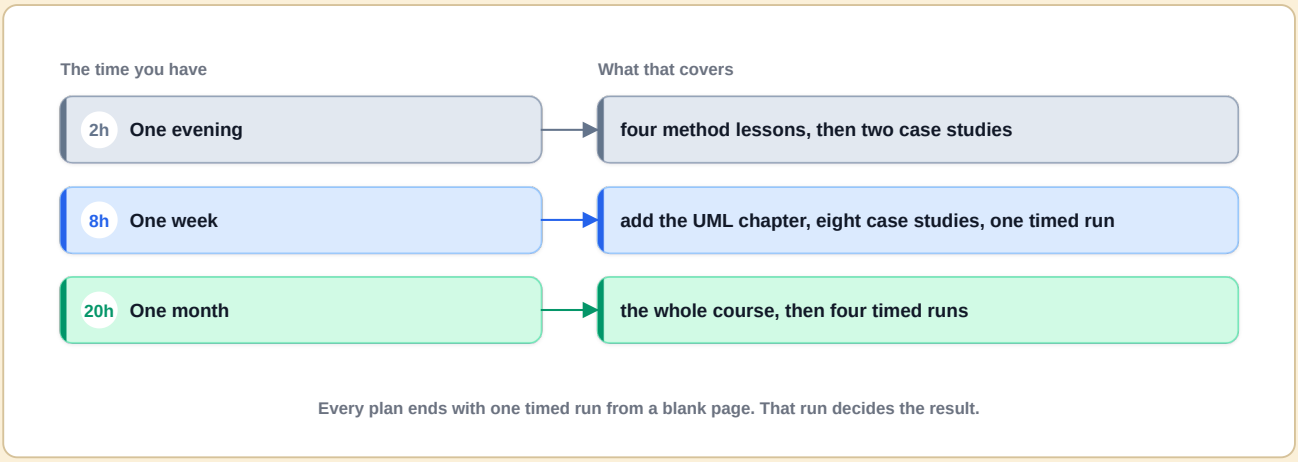


Figure — Three study plans for three amounts of time; every plan ends with one timed run from a blank page (course diagram)

- **Tomorrow (~2 h):** §1.2, §1.5, §1.6, Ch 3 relationship guide + class notation, parking lot (30 min), vending machine (25 min, a **lifecycle**).
- **One week:** Ch 1 + OOP pillars; Ch 3 diagrams; parking lot + library; ATM + vending; movie tickets + hotel; Splitwise + Amazon; **day 7: redo parking lot blank-page in 45 min**.
- **One month (~20 h):** Ch 1–4 + 3 case studies; Ch 5, 8–11 + case studies 4–10; Ch 12–13 + 11–18 + a timed run; the rest, Ch 21, three timed runs, Ch 22.
- **Beginner (6 weeks):** Ch 1–2; 3–5; 6–8; 9–11 + two case studies; 12–13 + four incl. ticket booking; Ch 21 timed runs + Ch 22.

Only six case studies? Parking lot, library, vending machine, movie tickets (concurrency), Splitwise (money), Amazon (too big: choose). Add **LRU cache** if data-structure questions are likely. **Night before:** nothing new; re-read the time budget; recite **requirements, actors and use cases, classes, relationships, code.**

1.14 Common mistakes

Mistake	Fix
Coding in minute two; no out-of-scope list; yes to every feature	8–13 min on requirements; write and agree scope
One subclass per value; god class	Data → field/enum; split on "and"
Missing record class; "Car extends Engine"	Connection with data = object; composition
Pattern stuffing; `switch` on type	1–2 patterns with signal; polymorphism
Equal depth everywhere; silent designing	2–3 flows fully; narrate
Ignoring concurrency; non-running code	Lock shared state; thin path running by halfway
`double` money; clock in logic	`BigDecimal`/cents; inject time

In the interview

30-second answer: "I clarify and write 8–12 requirements with an out-of-scope list, list actors and use cases, pick classes from the nouns with one responsibility each, connect them in a class diagram, sketch the key flow, run a SOLID check and pick one or two patterns where the signal is clear. Then I code the core flows, handle edge cases and concurrency, and finish with extensions and trade-offs."

- *"Different from system design?"* — That's many machines and scale; LLD is classes and code inside one application.
- *"You won't finish; what matters?"* — Bounded scope, clean responsibilities, correct relationships, code matching the diagram, one defended trade-off.

Self-check

- **Q:** Five core steps? **A:** Requirements, actors & use cases, classes, relationships, code.
- **Q:** Two classes beginners miss? **A:** The record-keeping class and the rule class.
- **Q:** Patterns in 45 minutes? **A:** One or two where the signal appears; name a rejected one.

TL;DR

- Design the classes of one app and code main flows; machine coding = same, running, ~90 min.
- A quarter of the time before naming a class; always an out-of-scope list.
- Different data → field; different behaviour → class; one sentence per class.
- SOLID as quick checks; one or two patterns with a signal.
- Inject time, exact money, guard shared state; attempt first, then timed blank-page runs.

Chapter 2 — OOP Fundamentals (Study Edition)

> **Study Edition.** Every topic of the full chapter, without the padding. Full programs and outputs are in the Full Edition (`README.md`).
>
> **You will learn:** the class basics, the four pillars on one bank example, and the Java OOP tools LLD relies on (sections 2.2–2.17 map one-to-one to the Full Edition).

Run all examples: `cd chapters/02-oop-fundamentals/code && javac --release 21 -d /tmp/lld_02 $(find . -name '*.java') && java -cp /tmp/lld_02 lld.ch02.<package>.Main`

2.1 What object-oriented programming is

What it is

OOP bundles data with the only code allowed to modify it (an **object**). Procedural code keeps data apart, so any function can change it. Analogy: cash on a table vs a vault with a teller (the methods) who checks every request.

The problem it solves

A rule like "balance never negative" lives **once**, next to its data, instead of in every caller; designs grow by **adding classes**.

Why this matters for the interview

LLD tests **splitting a problem into objects, giving each responsibilities and data, and justifying it**.

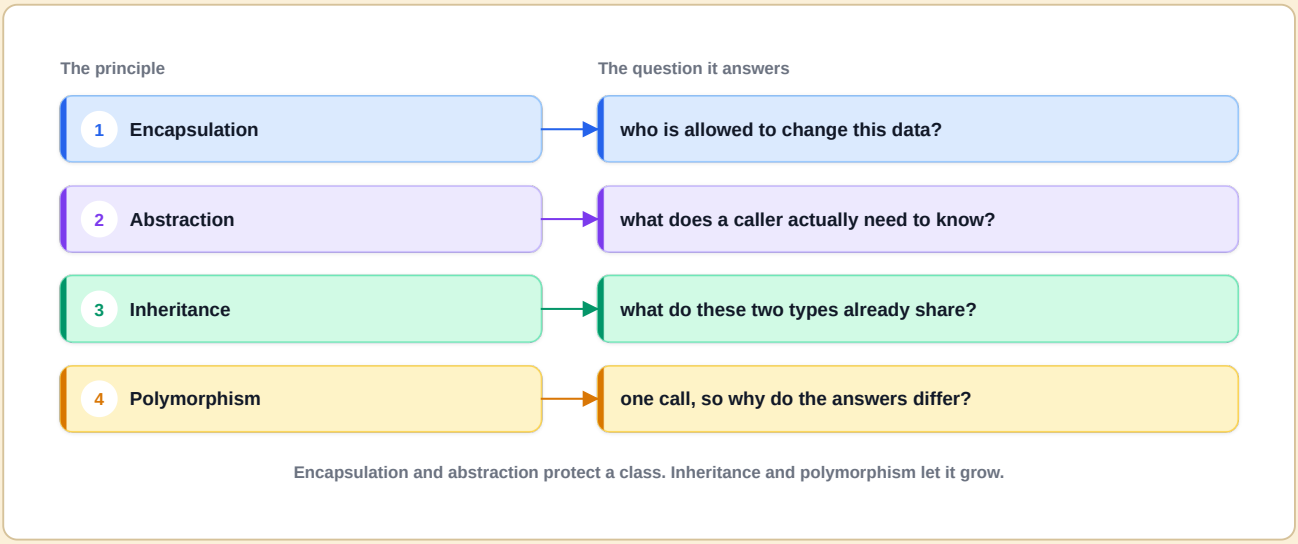


Figure — The four OOP principles and the question each one answers (course diagram)

Pillar	Question it answers
Encapsulation	Who may change this data?
Abstraction	What does a caller need to know?
Inheritance	What do these types share?
Polymorphism	One call, why do answers differ?

The first two guard one class; the last two let the design grow with new classes.

2.2 Classes, objects, constructors, `this`, `static` and `final`

What it is

Class = blueprint; **object** = one instance made with `new`. **Constructor** = named like the class, **no return type**, makes the object **valid**. `this` = current object. `static` = one per class, shared (else per-object **instance** members). `final` = variable set once / method not overridable / class not extendable.

How it works

1. `new` sets fields to defaults (0, `null`, `false`), then runs the constructor.
2. **Order:** static initialisers run **once** at first use; per `new`: parent constructor (`super(...)`) → instance initialisers in source order → constructor body.
3. No constructor written → compiler adds a public no-arg **default constructor**; write any → it doesn't.
4. **Chain** overloads to one canonical constructor with `this(...)` so validation lives once; in Java 21 it must be the **first statement** (Java 25, JEP 513, relaxes this).
5. `this`: disambiguate (`this.id = id`), pass self, chain, `return this` for **fluent** calls.
6. `static` methods have **no** `this`: utilities and factories.
7. Variables hold **references**: `Account alias = a;` is the **same** object.

Java example

```java
// File: code/lld/ch02/classes/Account.java
public class Account {
    // static = belongs to the CLASS, shared by all objects
    private static int openedCount = 0;
    // ... MIN_OPENING_CENTS constant, final id/owner, balanceCents

    public Account(String id, String owner, long openingCents) {
        if (openingCents < MIN_OPENING_CENTS) throw new IllegalArgumentException("negative opening balance");
        this.id = id;                // 'this.id' is the field, 'id' is the parameter
        this.owner = owner;
        this.balanceCents = openingCents;
        openedCount++;               // shared counter
    }

    public Account(String id, String owner) {
        this(id, owner, 0);
    }

    public Account deposit(long cents) {   // returns 'this' so calls can be chained
        if (cents <= 0) throw new IllegalArgumentException("deposit must be positive");
        this.balanceCents += cents;
        return this;
    }
    // ... accessors, static openedCount(), toString
}
```

Full runnable code: code/lld/ch02/classes/ (see the Full Edition).

A refused `new Account("X", "Bad", -5)` leaves the counter unchanged: **validate first, then change state**.

Trade-offs

Static members are shared but are global state (hard to test, thread-risky). Many constructor overloads confuse past 3–4 (use a Builder or static factories). `final` fields are safer but must be set in every constructor.

Common mistakes

Per-object data in a `static` field; heavy work or overridable calls in constructors; `id = id;`; thinking `=` copies an object.

Real-world examples

`Math.max` (static); `StringBuilder.append` returns `this`; `String` is `final`.

In the interview

> **30-second answer:** "A class is the blueprint, an object an instance with its own state. Constructors create valid objects; I chain overloads with `this(...)`. Static members are shared, so I keep them for constants, utilities and factories, never mutable state."

Self-check

- **Q:** Copies of a `static` field for 1,000 objects? **A:** One.
- **Q:** Parent constructor or child initialisers first? **A:** Parent constructor.

TL;DR

Class = blueprint, object = instance; chain with `this(...)`; `static` = per class; variables hold references.

2.3 Encapsulation

What it is

Hide data (**private**) and allow change **only through guarded methods**. Analogy: a child-proof cap.

The problem it solves

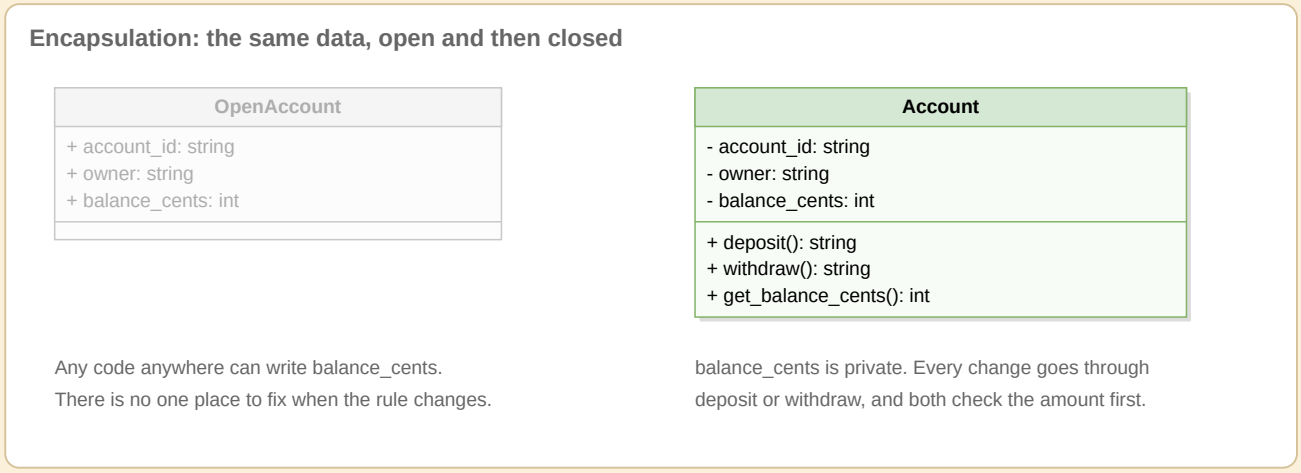


Figure — The same account data, first open (public fields anyone can write) and then closed (private fields changed only through deposit and withdraw) (course diagram)

With a private balance, **every route to the data passes a method that can say no**; public fields leave no central place for the rule.

How it works

Fields `private` (and `final`); expose **behaviour** (`withdraw`), not setters; validate there, **once**; return copies/unmodifiable views of collections; decide **who may modify each field** and say it.

Java example

`Account` in §2.6: private balance, `final deposit`/`withdraw`.

Trade-offs · When not to use it (strictly)

Rules in one place and freely changeable internals, for a little more code. Rule-free data carriers (DTOs, events): use a **record**.

Common mistakes

Getter + setter for every field (anemic, private in name only); returning the live internal list.

Real-world examples

`String`, `ArrayList`; `LocalDate.of(2026, 2, 30)` throws.

In the interview

> **30-second answer:** "State hides behind methods that hold every rule about it. If only this class may change a field, it's private and changed through `withdraw`-style methods, not setters." A rule-free setter is a public field in disguise.

Self-check

- **Q:** Where does "no negative balance" live? **A:** In `Account.withdraw`, once.

TL;DR

Private data + rule-enforcing methods; behaviour, not setters.

2.4 Abstraction

What it is

Give callers **only the operations they need**, hide **how**. Encapsulation hides **state**; abstraction hides **decisions**. Analogy: pedals work the same for petrol or electric.

The problem it solves

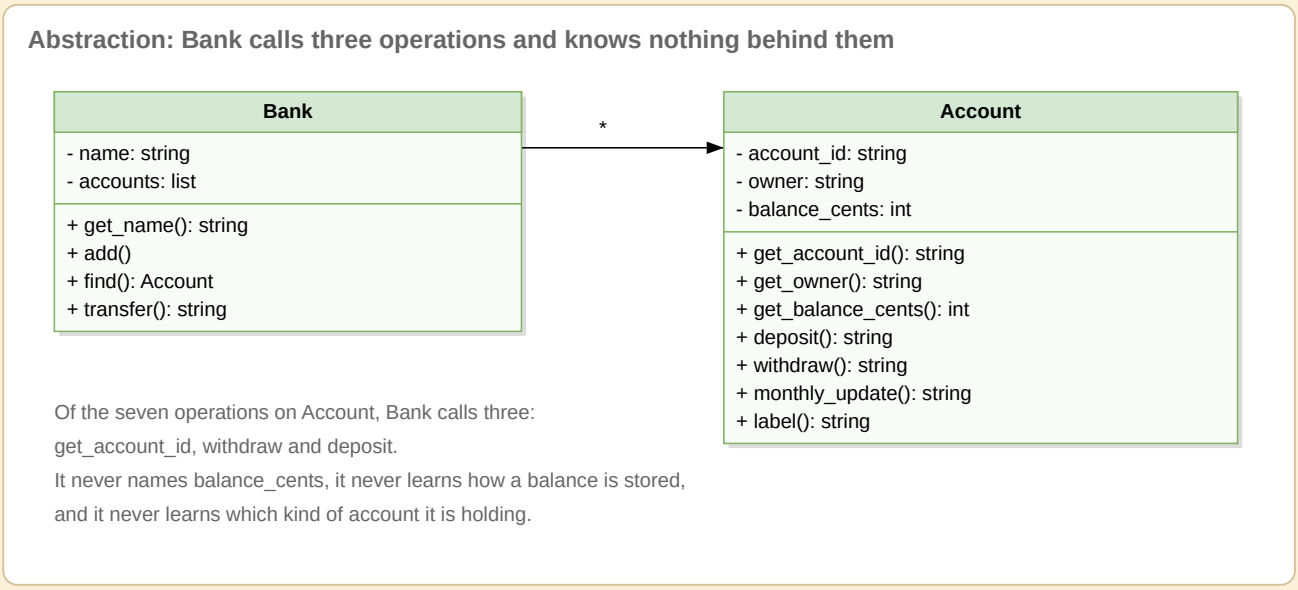


Figure — Of Account's seven public operations, Bank relies on three and sees nothing of what lies behind them (course diagram)

`Bank.transfer` uses only `id()`, `withdraw()`, `deposit()` (our Java `Account` has six public operations, the diagram seven). It doesn't know about cents or subclasses, so storage could move to a database untouched.

How it works

Smallest set of operations, named by **what** they do, on an interface/public surface; callers depend only on it.

Java example

`Bank.transfer` in §2.6.

Trade-offs

Simpler callers, enables polymorphism; a wrong abstraction is costly, too many layers confuse.

Common mistakes

Leaky abstractions (`getHashMapOfSeats()`); abstracting before two concrete cases.

Real-world examples

`List`, JDBC `Connection`, `InputStream`.

In the interview

> **30-second answer:** "Expose only what callers need; whenever 'the caller doesn't need to know that' is true, it moves behind the interface." Encapsulation protects **data**; abstraction hides **complexity**.

Self-check

- **Q:** What does `transfer` need from `Account`? **A:** `id()`, `withdraw()`, `deposit()`.

TL;DR

Minimum surface, hidden how.

2.5 Inheritance

What it is

A **child** gets everything its **parent** has, then **adds** or **overrides**: `class SavingsAccount extends Account` (a savings account **is an** account).

The problem it solves

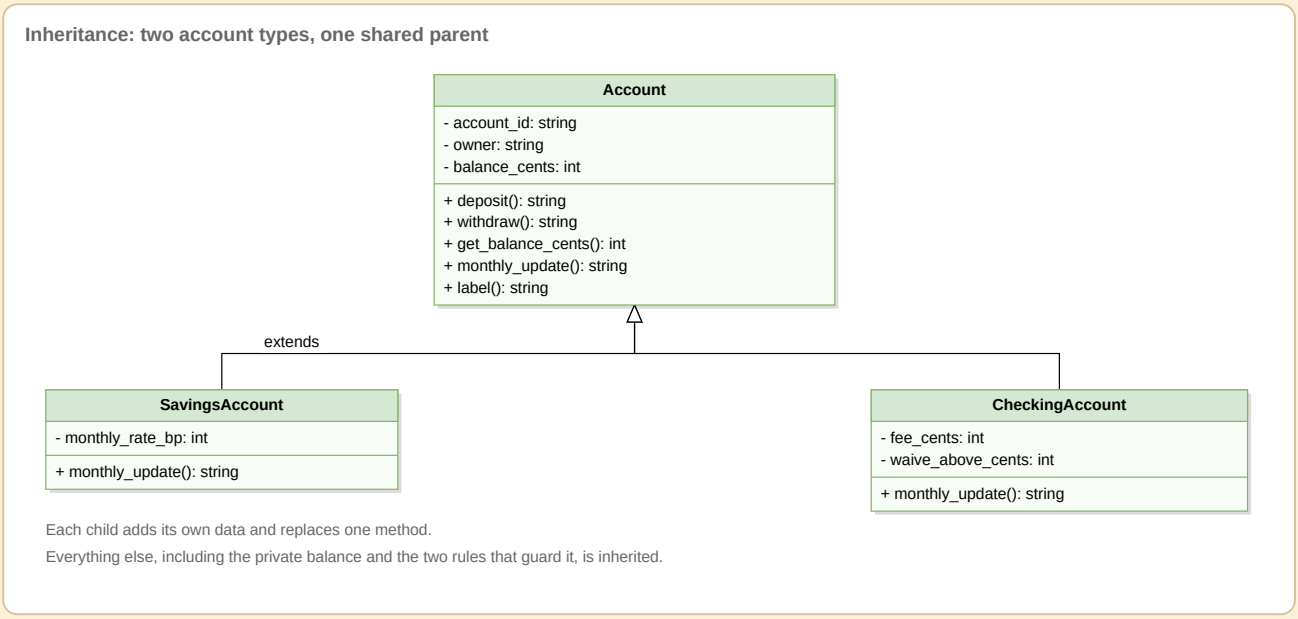


Figure — SavingsAccount and CheckingAccount extend Account; each adds its own data and replaces one method (course diagram)

Both children share the parent's balance and rules, add 1–2 fields and **override one method**. If is-a were false, the difference would be a **field**.

How it works

1. **One** parent class, many interfaces. **Constructors are not inherited**; private fields exist but are reached via parent methods.
2. Child constructor calls `super(...)` first (implicit `super()` fails if the parent lacks a no-arg constructor). `super.method()` reuses the parent's code.
3. **Overriding rules:**

Rule	Detail
Signature	Same name and parameter types
Return type	Same or a **subtype** (*covariant*); primitives exact
Access	Same or **wider** only
Checked exceptions	Fewer/narrower only
Cannot override	`final`; `static` (**hidden**); `private`
Annotation	Always `@Override`

4. Every class extends `Object` (`toString`, `equals`, `hashCode`, `getClass`). The **is-a test** must hold always.

Java example

`SavingsAccount`/`CheckingAccount` (§2.6); `Circle` (§2.7).

Trade-offs

Reuse and polymorphism for true families; but **tight coupling** to parent internals (fragile base class), compile-time fixed, single parent, hard past 2–3 levels.

When not to use it

Code reuse without is-a (use composition); a child that must **disable** a parent method (breaks **Liskov**, Chapter 4); data-only differences (field/enum).

Common mistakes

`Square extends Rectangle` with independent setters; missing `@Override` (accidental overload); overridable calls in parent constructors.

Real-world examples

Good: `FileInputStream extends InputStream`. JDK mistakes: `Stack extends Vector`, `Properties extends Hashtable`.

In the interview

> **30-second answer:** "Inheritance is right only when 'child is a kind of parent' always holds. I use `@Override`, and if a child must disable a parent method I switch to composition, since that breaks Liskov."

- *"Override a static method?"* — No, it's **hidden** (chosen by compile-time type).

Self-check

- **Q:** Override from `public` to `protected`? **A:** No.

TL;DR

True is-a only; `super(...)` first, `@Override` always.

2.6 Polymorphism (and the full bank example)

What it is

One call, different behaviour per receiving object. Code holds the **parent type**; the actual object decides. Analogy: one "start" button on a car, washer, laptop.

The problem it solves

Removes `if (a instanceof SavingsAccount) ... else if ...` chains that every new type must edit.

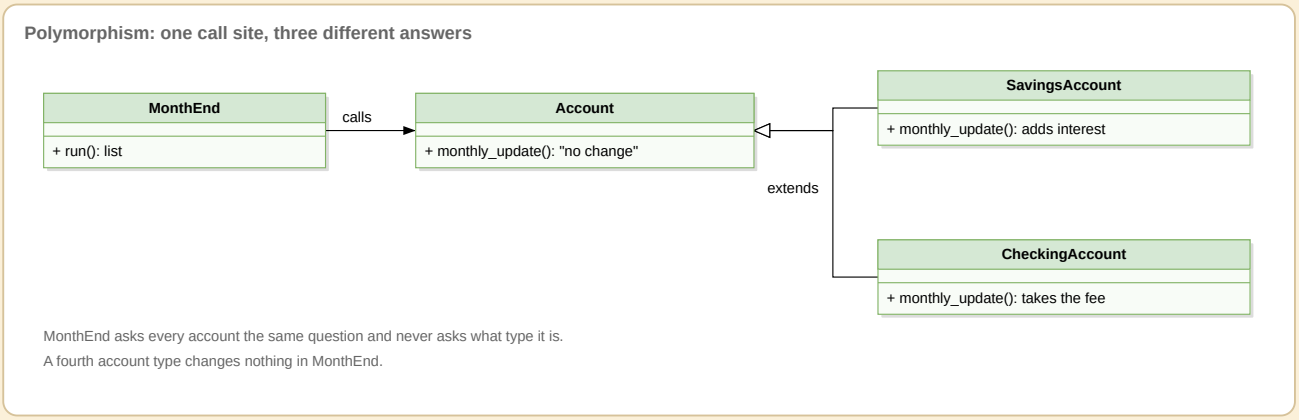


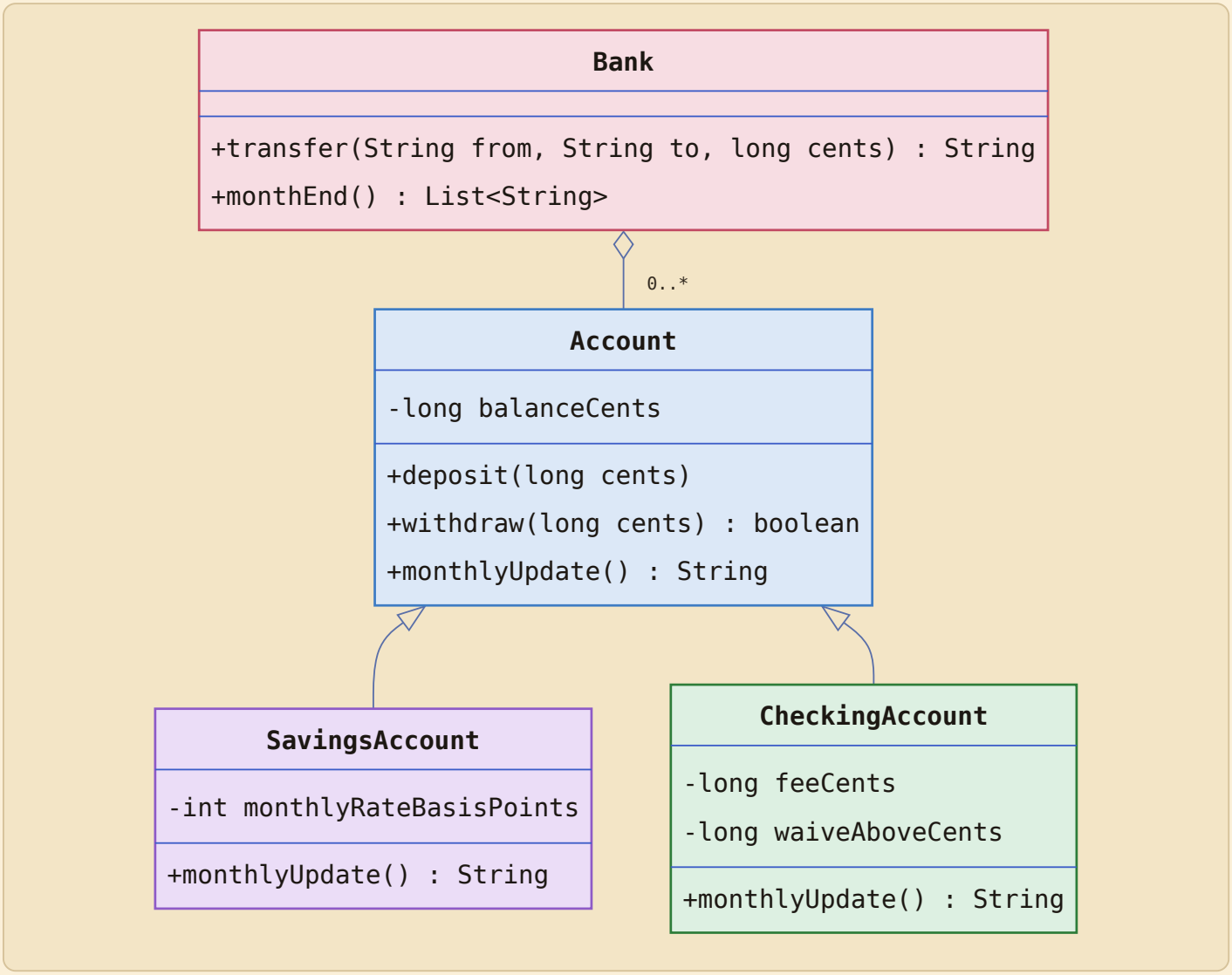
Figure — MonthEnd calls monthlyUpdate on every account and never asks which type it holds (course diagram)

The month-end loop has **no type** `if`; a fourth account type needs no edit.

How it works

Declare on the parent/interface → children override → callers hold the parent type → the JVM runs the **actual class's** version: **dynamic dispatch** (late binding).

Java example: the bank (encapsulation + abstraction + inheritance + polymorphism)




```java
// File: code/lld/ch02/bank/Account.java
public class Account {
    private long balanceCents;                      // hidden: nobody outside can set it to -5
    // ... id, owner, constructor

    public final boolean withdraw(long cents) {
        if (cents <= 0 || cents > balanceCents) return false;   // the one place the rule lives
        balanceCents -= cents;
        return true;
    }
    // ... final deposit() and accessors

    public String monthlyUpdate() { return "no change"; }
}
```

```java
// File: code/lld/ch02/bank/SavingsAccount.java
    @Override
    public String monthlyUpdate() {
        long interest = balanceCents() * monthlyRateBasisPoints / 10_000;
        if (interest > 0) deposit(interest);
        return "interest " + Money.format(interest);
    }
```

```java
// File: code/lld/ch02/bank/Bank.java
    public List<String> monthEnd() {
        List<String> lines = new ArrayList<>();
        for (Account a : accounts) {
            String what = a.monthlyUpdate();                // dynamic dispatch picks the override
            lines.add(a.id() + " (" + a.owner() + "): " + what + ", balance " +
Money.format(a.balanceCents()));
        }
        return lines;
    }
```

Full runnable code: code/lld/ch02/bank/ (see the Full Edition).

```
S-1 rejected, not enough money
month end:
  S-1 (Ada): interest $1.00, balance $401.00
  C-1 (Kai): fee $2.50, balance $177.50
  P-1 (Ravi): no change, balance $30.00
```

- The $900 transfer is refused inside `withdraw` (encapsulation); `CheckingAccount` waives its fee above a threshold; one call gives three results with no type `if` (polymorphism).
- **Why `long` cents:** exact, unlike `double`, but integer division **truncates** fractions of a cent. For interest/tax use `BigDecimal` with a `RoundingMode` (§2.13). 1 basis point = 0.01%.

How the four fit together

Encapsulation + abstraction decide **what is hidden**; inheritance + polymorphism make **new requirements into added classes**.

Trade-offs

Kills type switches and underpins most patterns; but behaviour spreads across classes (virtual-call cost rarely matters).

Common mistakes

Type `switch`/`if` chains (**pointing it out earns more credit than refactoring**); `instanceof` chains as the main dispatch.

Real-world examples

`List.add` on `ArrayList` vs `LinkedList`; `Comparator.compare`.

In the interview

30-second answer: "I code against the parent type or interface, each subtype overrides, and the JVM picks at run time, so a new type is a new class, not an edit." In a parking lot: pricing, payments, spot selection.

Self-check

- **Q:** `Account a = new SavingsAccount(...)`: whose `monthlyUpdate`? **A:** `SavingsAccount`'s, at run time.

TL;DR

Hold the parent type, let the object decide; type switches signal missing polymorphism.

2.7 Compile-time vs runtime polymorphism: overloading vs overriding

What it is

	Overloading (compile-time)	Overriding (runtime)
What	Same name, **different parameters**	Same signature in a **subclass**
Chosen by	**Compiler**, from **declared** argument types	**JVM**, from the receiver's **actual class**
Return type	May differ (not alone)	Same or covariant
Inheritance?	No	Yes

How it works: overload resolution

Phases, first match wins: (1) exact or **widening** (`int` → `long` → `double`, `Circle` → `Shape`); (2) **boxing**; (3) **varargs**. Ties → **most specific**, else ambiguous error. **Not polymorphic:** fields (**hidden**), `static` and `private` methods.

Java example

```
// File: code/lld/ch02/polymorphism/Calculator.java
    public String add(int a, int b)       { return "add(int,int) = " + (a + b); }
    public String add(long a, long b)     { return "add(long,long) = " + (a + b); }
    // ... add(double,double), add(int...)
    public String greet(Shape s)  { return "greet(Shape)"; }
    public String greet(Circle c) { return "greet(Circle)"; }
```

```
// File: code/lld/ch02/polymorphism/Circle.java
public class Circle extends Shape {
    public String kind = "circle";                          // HIDES Shape.kind, does not override it
    // ...
    @Override public double area() { return Math.PI * r * r; }  // OVERRIDE: same signature

    @Override public Circle copy() { return new Circle(r); }    // covariant return type is allowed
    // ... static describe() (hidden), who() calls super.who()
}
```

Full runnable code: code/lld/ch02/polymorphism/ (see the Full Edition).

With `Shape s = new Circle(1.0)`: `calc.add(2L, 3)` → `add(long,long)` (widening); `s.area()` → 3.14 (dispatch); `calc.greet(s)` → **`greet(Shape)`** (overloads use the declared type: a favourite question); `s.kind` → `shape` (**never hide fields**); `s.copy()` is a `Circle`. `s instanceof Circle c` is pattern matching (Java 16+).

Common mistakes

Expecting overloads to follow runtime type (needs double dispatch: Visitor, Chapter 11); `equals(Point p)` overload.

In the interview

> **30-second answer:** "Overloading is compile-time, chosen by declared types; overriding is runtime, dispatched on the actual object. Fields and statics aren't polymorphic."

Self-check

- **Q:** Differ only in return type? **A:** Compile error. **Q:** Varargs phase? **A:** Last.

TL;DR

Overloading = compiler, declared types; overriding = JVM, actual object.

2.8 Interfaces (default, static, private methods, functional interfaces)

What it is

A **contract**: methods a class promises. Analogy: a socket standard. Abstract methods are `public abstract`, fields `public static final`; a class **implements many**. **Default** (Java 8) bodies let interfaces evolve (`Collection.stream()`); **static** (Java 8) belong to the interface, not inherited; **private** (Java 9) help defaults. A **functional interface** has **exactly one abstract method** (Object/default/static don't count) → a **lambda** can implement it; `@FunctionalInterface` enforces it.

The problem it solves

Callers depend on **what**, not a class: add implementations without edits, test with fakes, multiple inheritance of **type** (not state).

How it works

Smallest method set; implement via classes or lambdas; callers use the interface type. Same default in two interfaces → class **must** override (may call `A.super.m()`); a superclass method **wins** over defaults.

Java example

```java
// File: code/lld/ch02/interfaces/Notifier.java
public interface Notifier {
    String send(String to, String message);        // abstract method: every implementer must provide it

    default String sendAll(java.util.List<String> recipients, String message) {
        StringBuilder sb = new StringBuilder();
        for (String r : recipients) sb.append(send(r, clean(message))).append('\n');
        return sb.toString().stripTrailing();
    }

    static Notifier console() { return (to, msg) -> "[console] " + to + ": " + msg; }

    private String clean(String message) { return message.strip(); }
}
```

Full runnable code: code/lld/ch02/interfaces/ (see the Full Edition).

`EmailNotifier` and a lambda both implement `Notifier`; `DiscountRule` is an `@FunctionalInterface` with a default `then` combinator. JDK ones: `Function`, `Supplier`, `Consumer`, `Predicate`.

Trade-offs

Loose coupling, testability, evolvable; but an extra type, no state, and overused defaults become mini base classes.

When not to use it

One implementation, no realistic second, no test double.

Common mistakes

Fat interfaces (ISP); constant-only interfaces.

Real-world examples

`Comparable`, `Runnable`, `AutoCloseable`, Spring `Repository`.

In the interview

> **30-second answer:** "Services depend on interfaces like `PaymentGateway` so implementations can be added and faked. Interfaces have default and static methods; one abstract method makes them lambda-friendly."

> Defaults exist to add methods like `stream` without breaking implementers.

Self-check

- **Q:** Is `Comparator` functional? **A:** Yes; only `compare` is abstract.

TL;DR

Depend on contracts; defaults evolve, statics are factories, one abstract method = lambda.

2.9 Abstract classes vs interfaces

What it is

An **abstract class** can't be instantiated; it may hold fields, constructors, finished and **abstract** methods. Analogy: a partly pre-printed form.

How it works

```java
// File: code/lld/ch02/interfaces/Report.java
public abstract class Report {
    private final String title;                    // state: interfaces cannot have instance fields

    protected Report(String title) { this.title = title; }

    public final String render() {                 // final: subclasses cannot change the skeleton
        return header() + "\n" + body() + "\n-- end of " + title + " --";
    }

    protected String header() { return "== " + title + " =="; }   // concrete, overridable
    protected abstract String body();                              // must be supplied
}
```

`SalesReport` supplies only `body()`: **Template Method** (Chapter 11).

	Abstract class	Interface
State / constructors	Yes	Constants only / no
Bodies	Any	`default`, `static`, `private`
Per class	**One**	**Many**
Expresses	"is a"	"can do"
Use	Shared state + partial implementation	Contracts, roles, strategies, test seams

Rule of thumb

Interface for the contract; abstract class beneath when implementations share state or an algorithm (`List` + `AbstractList` + `ArrayList`).

Common mistakes

Abstract class to share one helper; everything abstract "for flexibility".

In the interview

> **30-second answer:** "Interfaces define capabilities and allow many; abstract classes share state and partial implementation, one parent. Interface first; abstract base for real shared state or a template."

Self-check

- **Q:** Abstract class without abstract methods? **A:** Allowed. **Q:** Interface constructor? **A:** No.

TL;DR

Interface = contract, many, stateless; abstract class = partial implementation, one.

2.10 Enums with fields and behaviour

What it is

A **fixed set of named instances**; in Java a full class with **fields, constructors, methods, interfaces**, even per-constant bodies.

The problem it solves

String/int constants allow typos and carry no behaviour. Enums are **type-safe** and hold each value's data and rules, removing switches: LLD's home for "**different data, not behaviour**".

How it works

Implicitly extend `Enum`; private constructors; `values()`, `valueOf` (throws on unknown), `name()`, `ordinal()` (**never persist**); compare with `==`; per-constant bodies make tiny state machines; `EnumMap` / `EnumSet`; exhaustive `switch` needs no `default`; a one-element enum is the simplest safe **Singleton**.

Java example

```java
// File: code/lld/ch02/enums/OrderStatus.java
public enum OrderStatus {
    PLACED    { @Override Set<OrderStatus> next() { return Set.of(PAID, CANCELLED); } },
    PAID      { @Override Set<OrderStatus> next() { return Set.of(SHIPPED, CANCELLED); } },
    SHIPPED   { @Override Set<OrderStatus> next() { return Set.of(DELIVERED); } },
    DELIVERED { @Override Set<OrderStatus> next() { return Set.of(); } },
    CANCELLED { @Override Set<OrderStatus> next() { return Set.of(); } };

    abstract Set<OrderStatus> next();          // constant-specific method bodies

    public boolean canMoveTo(OrderStatus target) { return next().contains(target); }
    // ... isFinal()
}
```

Full runnable code: code/lld/ch02/enums/ (see the Full Edition).

`Coin` carries `cents` as a field (`PENNY(1) ... QUARTER(25)`), so summing needs no switch. `OrderStatus` refuses `PLACED -> SHIPPED`: the "**enum + guarded method**" alternative to the State pattern.

Trade-offs

Type-safe, data + behaviour; but closed (recompile to add), and large per-constant logic belongs in State/Strategy classes.

When not to use it

Values created at run time, or needing big independent behaviour.

Common mistakes

Persisting `ordinal()`; switches over the enum instead of methods on it.

Real-world examples

`DayOfWeek`, `TimeUnit.SECONDS.toMillis(5)`, `Thread.State`.

In the interview

> **30-second answer:** "Enums model fixed sets like vehicle types and statuses, carrying data like 'which spots fit' or 'which status may follow'." Enum if only data differs; classes if behaviour does.

Self-check

- **Q:** `Coin.valueOf("dime")`? **A:** `IllegalArgumentException` (case-sensitive).

TL;DR

Type-safe fixed sets with data and behaviour; `==`; no persisted ordinals.

2.11 Composition over inheritance

What it is

Hold other objects and **delegate** ("has-a") instead of inheriting ("is-a"); GoF's "favour composition over inheritance". Analogy: a phone **has** an upgradable camera module.

The problem it solves

Inheritance ties the child to the parent's **internals** (**fragile base class**), fixed at compile time. Composition stays independent and **swappable at run time**.

How it works

Put the behaviour behind an interface → hold it in a private field from the constructor → **forward** calls, adding your logic.

Java example

```java
// File: code/lld/ch02/composition/InheritingCountingSet.java
    @Override public boolean add(E e) { attempts++; return super.add(e); }

    @Override public boolean addAll(Collection<? extends E> c) { attempts += c.size(); return super.addAll(c);
}
```

```java
// File: code/lld/ch02/composition/CountingSet.java
public class CountingSet<E> {
    private final Set<E> inner;
    // ...
    public boolean addAll(Collection<? extends E> c) {
        attempts += c.size();
        boolean changed = false;
        for (E e : c) changed |= inner.add(e);     // talk to the inner set, not to our own add()
        return changed;
    }
}
```

Full runnable code: code/lld/ch02/composition/ (see the Full Edition).

```
inheritance: added 3, attempts counted = 6
composition: 4 attempts counted = 4, size = 3
```

`HashSet.addAll` calls `add` per element (parent **self-use**, not promised), so the subclass double counts. `CountingSet` works with any `Set`. `Car` **has an** `Engine`, swapped at run time.

Trade-offs

	Inheritance	Composition
Relationship / coupling	is-a / tight	has-a / loose
Run-time change	No	Yes
Boilerplate	Low	Forwarding methods

When not to use it

A genuine is-a class **designed for extension** is fine to extend.

Common mistakes

Extending concrete library classes (`HashMap`); exposing the inner object (`getInner()`).

Real-world examples

`BufferedReader` wraps a `Reader`; Spring DI; `Comparator` in `TreeMap`.

In the interview

> **30-second answer:** "I hold an interface-typed collaborator and delegate: no fragile base class, run-time swapping, and it underlies Strategy, Decorator, State. Inheritance only for true is-a." Example of breakage: the counting set.

Self-check

- **Q:** Why `inner.add`, not `this.add`? **A:** No double counting; relies only on the public contract.

TL;DR

Has-a + delegate beats is-a for reuse.

2.12 Object identity vs equality: the `equals`/`hashCode` contract

What it is

Identity `==`: same object. **Equality** `equals`: same value (default is identity; `String`, `Integer`, records override). Two copies of a book: equal, not identical.

The problem it solves

Hash collections pick a bucket by `hashCode()`, then compare with `equals()`; get either wrong and objects "disappear".

How it works: the contracts

`equals`: reflexive, symmetric, transitive, consistent, `equals(null)` false. `hashCode`: stable, **equal objects → equal hashes**, collisions allowed. **Override both on the same fields.**

Java example

```java
// File: code/lld/ch02/equality/Point.java
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;                          // reflexive fast path
        if (!(o instanceof Point p)) return false;           // also handles null
        return x == p.x && y == p.y;
    }

    @Override public int hashCode() { return Objects.hash(x, y); }
```

Full runnable code: code/lld/ch02/equality/ (see the Full Edition).

A `HashSet<Point>` holding two equal points has size 1. `BrokenPoint` (only `equals`) gives `set.contains(equal one): false`; `javac -Xlint` warns `[overrides]`: treat as an error. Compare boxed numbers with `equals`: `Integer` caches only small values (at least −128..127).

Trade-offs / design choice

Value objects (Money, Point): value equality, immutable. **Entities** (User, Order): identity or stable **ID**.

Common mistakes

`equals` without `hashCode`; `equals(Point p)`; **mutable fields in `hashCode`** then mutating a key; mixing `getClass()` / `instanceof`.

Real-world examples

`String`, `LocalDate`, `List.equals`; `new BigDecimal("1.0").equals(new BigDecimal("1.00"))` is **false** (scale; use `compareTo`).

In the interview

> **30-second answer:** "`==` is identity, `equals` value. I override `hashCode` with `equals` on the same fields, since hash collections locate the bucket by hash. Records for values, IDs for entities."

Self-check

- **Q:** Unequal objects, same hash? **A:** Allowed. **Q:** Equal objects, different hashes? **A:** Broken contract.

TL;DR

Override `equals` and `hashCode` together, on immutable fields.

2.13 Immutability and records

What it is

Immutable objects never change; "changes" return **new** objects. A **record** (Java 16+) declares an immutable data carrier in one line. Analogy: a printed receipt.

The problem it solves

Shared mutable state causes surprise changes and needs locks. Immutable objects are **thread-safe without locks** and safe hash keys.

How it works

Checklist: `final` class (or private constructors + static factories); `private final` fields; no setters; **defensive copies** in and out; don't leak `this`. **Records** generate fields, canonical constructor, accessors

(`id()`), `equals`/`hashCode`/`toString`; they're `final`, allow no extra instance fields, may implement interfaces; a **compact constructor** validates. **Only shallowly immutable**: copy mutable components.

Java example

```
// File: code/lld/ch02/immutability/Money.java
    private Money(BigDecimal amount, String currency) {
        this.currency = Objects.requireNonNull(currency);
        // UNNECESSARY = refuse (ArithmeticException) anything finer than a cent instead of silently rounding
it
        this.amount = amount.setScale(2, RoundingMode.UNNECESSARY);
    }

    public static Money of(String amount, String currency) { return new Money(new BigDecimal(amount),
currency); }

    public Money plus(Money other) {
        if (!currency.equals(other.currency)) throw new IllegalArgumentException("currency mismatch");
        return new Money(amount.add(other.amount), currency);   // new object; 'this' is untouched
    }
```

```
// File: code/lld/ch02/immutability/Order.java
public record Order(String id, List<String> items, Money total) {
    public Order {
        if (id == null || id.isBlank()) throw new IllegalArgumentException("id required");
        items = List.copyOf(items);                // unmodifiable copy: caller's list cannot leak in
    }
    // ... withItem(item, price) returns a modified copy ("wither")
}
```

Full runnable code: code/lld/ch02/immutability/ (see the Full Edition).

Scale fixed at 2 makes `Money.of("5")` equal `Money.of("5.00")`; `items()` is an unmodifiable copy.

Trade-offs

Lock-free safety, no boilerplate; but a new object per change, and records can't hide components or extend classes.

When not to use it

Entities whose state changes (occupied spot, balance) stay **mutable but encapsulated**.

Common mistakes

No defensive copies; thinking `final` freezes the referenced object; records for entities.

Real-world examples

`String`, `BigDecimal`, `java.time`, `List.of`.

In the interview

30-second answer: "Value objects like Money or Ticket are immutable records with compact-constructor validation and defensive copies: thread-safe, safe keys. Entities stay mutable but encapsulated." Records are only shallowly immutable.

Self-check

- **Q:** `final List<String> items` guarantees? **A:** The reference, not the contents.

TL;DR

Final, private final, no setters, defensive copies; records do it (shallowly).

2.14 Access modifiers and packages

What it is

A **package** groups classes (namespace + folder); **access modifiers** set visibility:

Modifier	Class	Package	Subclass elsewhere	Everyone
`private`	✔	✘	✘	✘
(none) package-private	✔	✔	✘	✘
`protected`	✔	✔	✔ (via inheritance)	✘
`public`	✔	✔	✔	✔

Top-level classes: `public` or package-private only.

The problem it solves

Encapsulation becomes **compiler-enforced**; packages hide implementation behind a small API.

How it works

Start `private`; group by **feature**; package-private helpers behind a `public` facade; `protected` only for subclass hooks (it **also** opens the package); Java 9 **modules** export chosen packages.

Java example

```
// File: code/lld/ch02/access/client/PremiumWallet.java
    public void grantBonus(long cents) {
        bonusCents += cents;                    // OK: protected, accessed through inheritance
        // auditCount++;                        // would NOT compile: package-private in another package
        // balanceCents += cents;               // would NOT compile: private
    }

    @Override public String describe() {       // may WIDEN protected -> public when overriding
        return "[premium] " + super.describe();
    }
```

Full runnable code: code/lld/ch02/access/core/ and code/lld/ch02/access/client/ (see the Full Edition).

`Wallet` (in `core`) has one field per access level; `AuditLog` is a package-private class exposed only through the public `Audits` facade.

Common mistakes

Everything `public`; `protected` fields instead of methods; one package for a whole solution.

In the interview

> **30-second answer:** "Private by default, package-private for feature collaborators, protected only for subclass hooks, public as the deliberate API; packages like model, repository, service, strategy."

Self-check

- **Q:** Non-subclass in another package reads `protected`? **A:** No.

TL;DR

private < package-private < protected < public; default private.

2.15 Generics basics for LLD

What it is

Type-safe code for **any type**: `List<String>`, `Repository<String, User>`; `T`, `ID`, `E` are **type parameters**. Analogy: a "SHOES ONLY" box checked on the way in.

The problem it solves

No `Object` + casts failing at run time: errors become **compile errors**, and one `Repository<ID, T>` serves every entity.

How it works

Generic types and methods; **bounds** (`T extends X`) let you call X's methods; wildcards `? extends T` (producer), `? super T` (consumer): **PECS**; diamond `new Repository<>()`; **erasure**: no `new T()`, no `instanceof List<String>`, no primitives; avoid **raw types**.

Java example

```java
// File: code/lld/ch02/generics/Repository.java
public class Repository<ID, T extends Identifiable<ID>> {
    private final Map<ID, T> store = new LinkedHashMap<>();

    public void save(T item)            { store.put(item.id(), item); }
    public Optional<T> findById(ID id)  { return Optional.ofNullable(store.get(id)); }
    public List<T> findAll()            { return List.copyOf(store.values()); }
}
```

Full runnable code: code/lld/ch02/generics/ (see the Full Edition).

The same class stores `User`s by `String` and `Product`s by `Integer`. `copy(List<? extends T> src, List<? super T> dst)` moves integers into a `List<Number>` (PECS). `ArrayList<String>` and `ArrayList<Integer>` share one runtime class (erasure).

Common mistakes

Raw types; thinking `List<Integer>` is a `List<Number>`; over-generic designs.

Real-world examples

Collections, `Optional<T>`, Spring Data `CrudRepository<T, ID>`.

In the interview

> **30-second answer:** "Generics give type-safe infrastructure like `Repository<ID, T>`; I know bounds, PECS, and erasure."

Self-check

- **Q:** Why no `new T()`? **A:** Erasure; pass a `Supplier<T>`.

TL;DR

Type-safe reuse; bounds, PECS, erasure.

2.16 Coupling and cohesion (the yardsticks)

Cohesion (how well a class's duties belong together) should be **high**; **coupling** (dependence on others' details) should be **low**. Encapsulation and access modifiers cut coupling, single-responsibility raises cohesion, interfaces and composition decouple. Chapters 4–5 name these as SOLID, Law of Demeter, GRASP.

2.17 Object-oriented analysis and design (OOA / OOD / OOP)

What it is

Activity	Question	Talks about
Analysis (OOA)	What is the system, what must it do?	Domain terms: books, members, loans
Design (OOD)	Which objects with which responsibilities?	`Book`, `Member`, `Loan`, who holds whom
Implementation (OOP)	How in code?	`ArrayList`, `HashMap`

The problem it solves

Choosing operations before objects are settled puts them on the wrong class.

How it works: the four steps

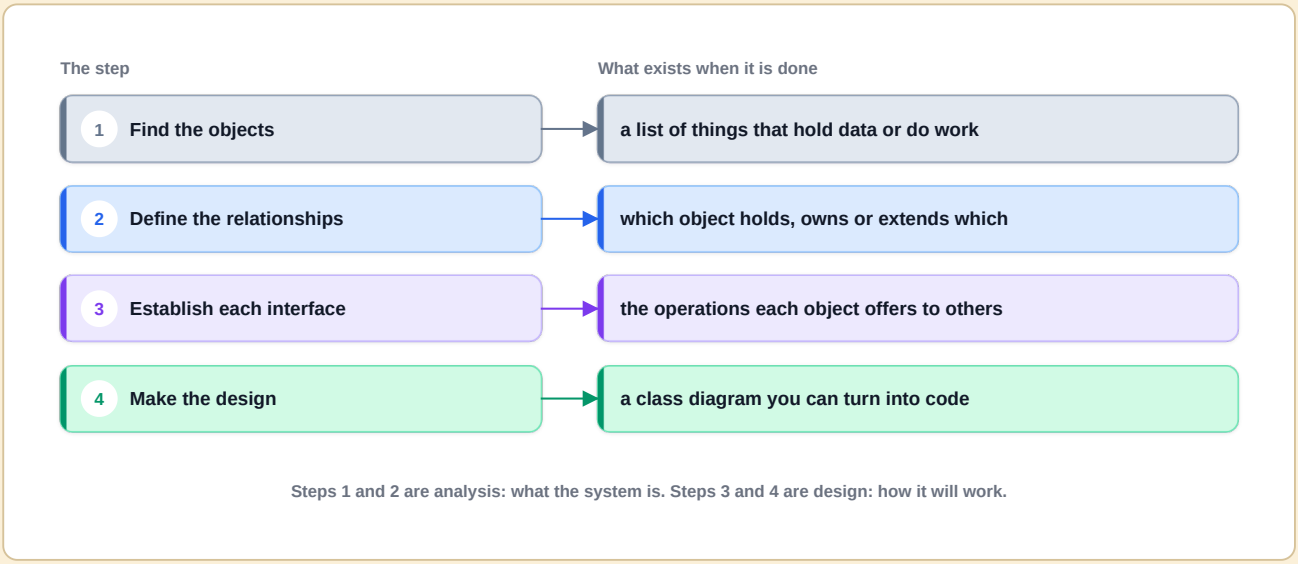


Figure — The four OOAD steps and what exists after each; steps 1–2 are analysis and steps 3–4 are design (course diagram)

1. **Find the objects** (things that store info or do work; single values become fields).
2. **Define relationships** (who references whom, ownership, how many).
3. **Establish each interface** (operations offered to others; the rest is **private**).
4. **Make the design** (one class diagram, checked against the description). Fixed order.

Worked example: "library members borrow books; each book on loan must be returned by a due date"

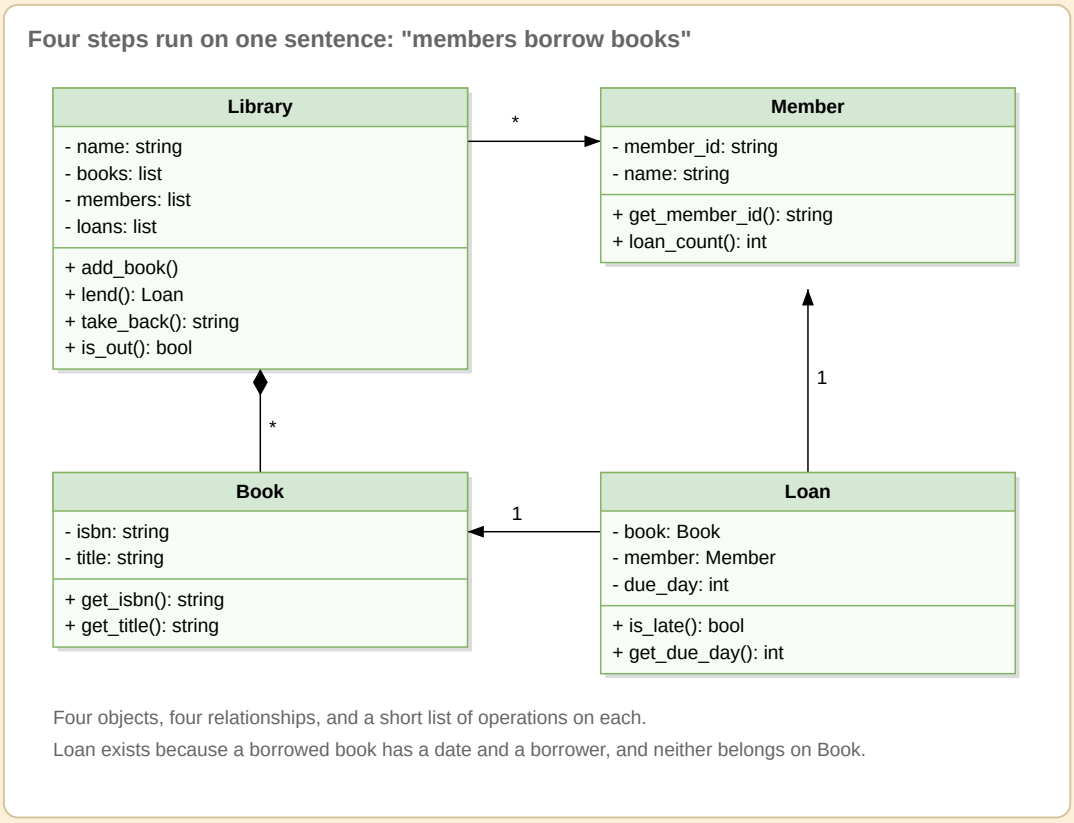
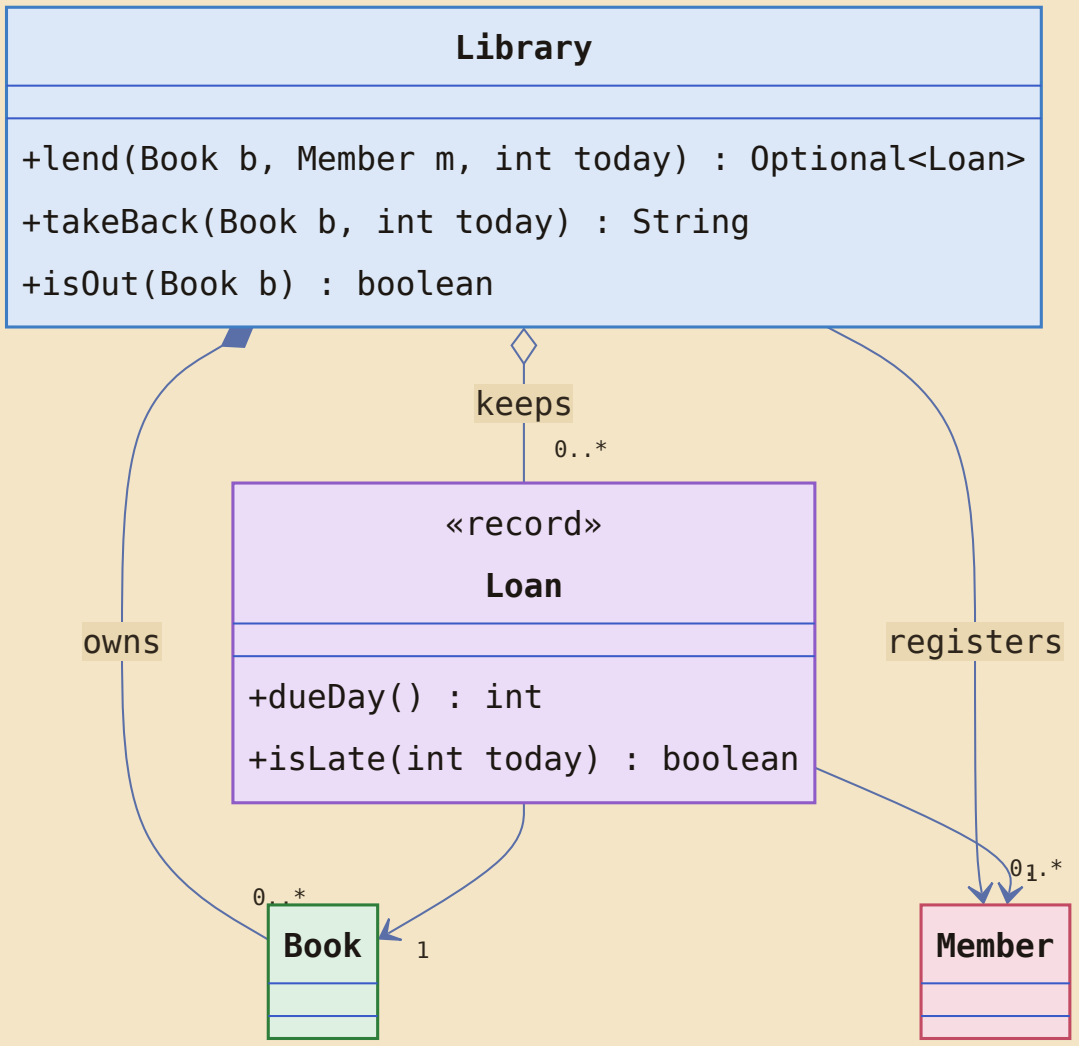


Figure — The four steps applied to one sentence: Library, Member, Book and the connection object Loan (course diagram)

Objects: `Library`, `Member`, `Book`, and **`Loan`** (most often missed); *date* is a field. **Rule: a link with data of its own becomes an object** (`Loan`, `Reservation`, `Ticket`, `Enrollment`). Library **owns** books (composition), **registers** members (association); a loan **points to** one book and one member. Only `Library` can answer "is this book out?" because it keeps the loans.




```java
// File: code/lld/ch02/ooad/Library.java
    public static final int LOAN_DAYS = 14;
    public static final int MAX_LOANS_PER_MEMBER = 5;
    // ...
    public Optional<Loan> lend(Book b, Member m, int today) {
        if (!books.contains(b) || !members.contains(m)) return Optional.empty();
        if (isOut(b) || m.loanCount() >= MAX_LOANS_PER_MEMBER) return Optional.empty();
        Loan loan = new Loan(b, m, today + LOAN_DAYS);
        loans.add(loan);
        m.loanStarted();
        return Optional.of(loan);
    }
```

Full runnable code: code/lld/ch02/ooad/ (see the Full Edition).

Limits became constants; `Loan` is a record with `isLate(today)`; `Member.loanStarted/loanEnded` are package-private so only `Library` changes counts.

Why UML

UML is the standard notation because each symbol records a decision: **filled diamond** = ownership, **plain arrow** = reference, **+ / −** = public/private (Chapter 3).

Three ways this goes wrong

Classes named after tables (`UserRow`) that do nothing; **missing the connection object** (`dueDate` on `Book`); **listing every imaginable operation**.

In the interview

> **30-second answer:** "Analysis first: objects from nouns, relationships with ownership and multiplicity. Then design: each object's public operations (which decides what's private), then the diagram. A link with its own data becomes a class."

Self-check

- **Q:** Analysis steps? **A:** Find objects, define relationships. **Q:** Parking-lot connection object? **A:** The ticket. **Q:** When is encapsulation decided? **A:** Step 3.

TL;DR

OOA = what; OOD = how with objects; OOP = code. Connections with data become objects.

Chapter summary: in the interview

> **30-second answer** ("pillars of OOP"): "Encapsulation keeps data and its rules in the owning class; abstraction exposes only what callers need; inheritance reuses a parent when is-a truly holds; polymorphism lets one call behave per object, removing type switches. I prefer interfaces and composition, enums for fixed sets, records for values, and consistent equals/hashCode."

- *"Make a class immutable?"* — Final class, private final fields, no setters, defensive copies, or a record.

Self-check (whole chapter)

- **Q:** Dynamic dispatch? **A:** The JVM choosing the override from the actual class at run time.
- **Q:** Where does "which spots fit a truck" live? **A:** On the `VehicleType` enum.
- **Q:** hashCode rule? **A:** Equal objects, equal hash codes.
- **Q:** OOAD steps? **A:** Objects, relationships, interfaces, design.

TL;DR

- OOP = data bundled with the only code allowed to change it.
- Pillars: encapsulation, abstraction, inheritance (true is-a), polymorphism (kills type switches).
- Overloading compile-time; overriding run-time; fields/statics not polymorphic.
- Interfaces = contracts; abstract classes = shared state; enums = fixed sets; composition over inheritance.
- `equals` + `hashCode` together; immutable records for values; encapsulated mutable entities.
- Private by default; generics for type-safe infrastructure; OOA → OOD → OOP.

Chapter 3 — Class Relationships & UML (Study Edition)

> **Study Edition.** Every topic of the full chapter, without the padding. Full programs and outputs are in the Full Edition (`README.md`).
>
> **You will learn:** the six class relationships and their Java shape, how to choose one, the five UML diagrams interviews use, and how much to draw.

Parts: A. Relationships (3.1–3.8) · B. UML (3.9–3.14) · C. Interview (3.15–3.17). One runnable package per relationship, plus `library` and `statemachine`.

Run: `cd chapters/03-relationships-and-uml/code && javac --release 21 -d /tmp/lld_03 $(find . -name '*.java') && java -cp /tmp/lld_03 lld.ch03.<package>.Main`

Part A — Class Relationships

3.1 The big picture: what a "relationship" really claims

What it is

A line between two classes makes two claims: about **lifetime** (if A is destroyed, is B too?) and about **substitution** (can a B be used wherever an A is expected?). Analogy: body and heart share a lifetime (composition); a car and its driver don't (association); a sports car *is a* car (generalization).

Family	Relationship	Meaning	Coupling
uses-a	**Dependency**	A uses B briefly (parameter, local, return type)	weakest
has-a	**Association**	A holds a reference to B; nobody owns anybody	↓
has-a	**Aggregation**	A is a whole of parts B; parts can outlive A	↓
has-a	**Composition**	A *owns* parts B; parts die with A	↓
is-a	**Realization**	A implements interface B	↓
is-a	**Generalization**	A extends class B	strongest

Coupling = how much one class must change when the other does; inheritance couples to every parent detail. (The order is a discussion rule of thumb, not UML.)

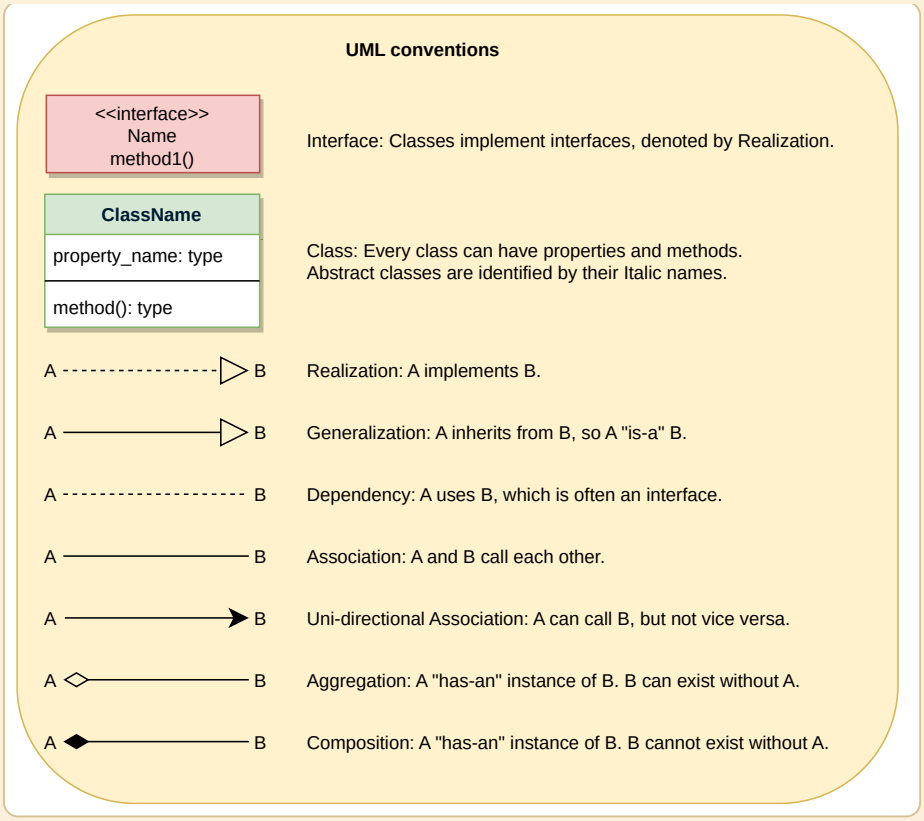


Figure — The UML conventions legend used throughout this book: class and interface boxes, and the line style for realization, generalization, dependency, association, uni-directional association, aggregation and composition (course diagram)

Line ends (triangle, diamond, arrowhead) plus solid vs dashed are the whole vocabulary. (This legend draws one-way association with a *filled* head and dependency with *no* head; standard UML and our Mermaid use an **open** arrowhead for both.)

The problem it solves

"Order has a Customer" only points at something living elsewhere; "ParkingFloor has ParkingSpots" creates, owns and deletes them. Confusing them deletes data you don't own or leaks orphans, visibly, before any code.

TL;DR

A line claims **who owns whom (lifetime)** and **who can stand in for whom (substitution)**.

3.2 Association (uni-directional, bi-directional, multiplicity)

What it is

One class keeps a **reference (field)** to another and calls it over time; **neither owns the other**. Analogy: a ticket refers to a spot; throw the ticket away and the spot remains.

- **Navigability:** **uni-directional** (one side has the field) or **bi-directional** (both do).
- **Multiplicity:** `1`, `0..1`, `*` / `0..*`, `1..*`, `2`, `2..4`.
- Optional **role/association name** (`placed by`).

The problem it solves

Objects collaborate over time (an `Order` must know its `Customer`). Association records "A knows B" without claiming ownership; it is the most common relationship, correctly.

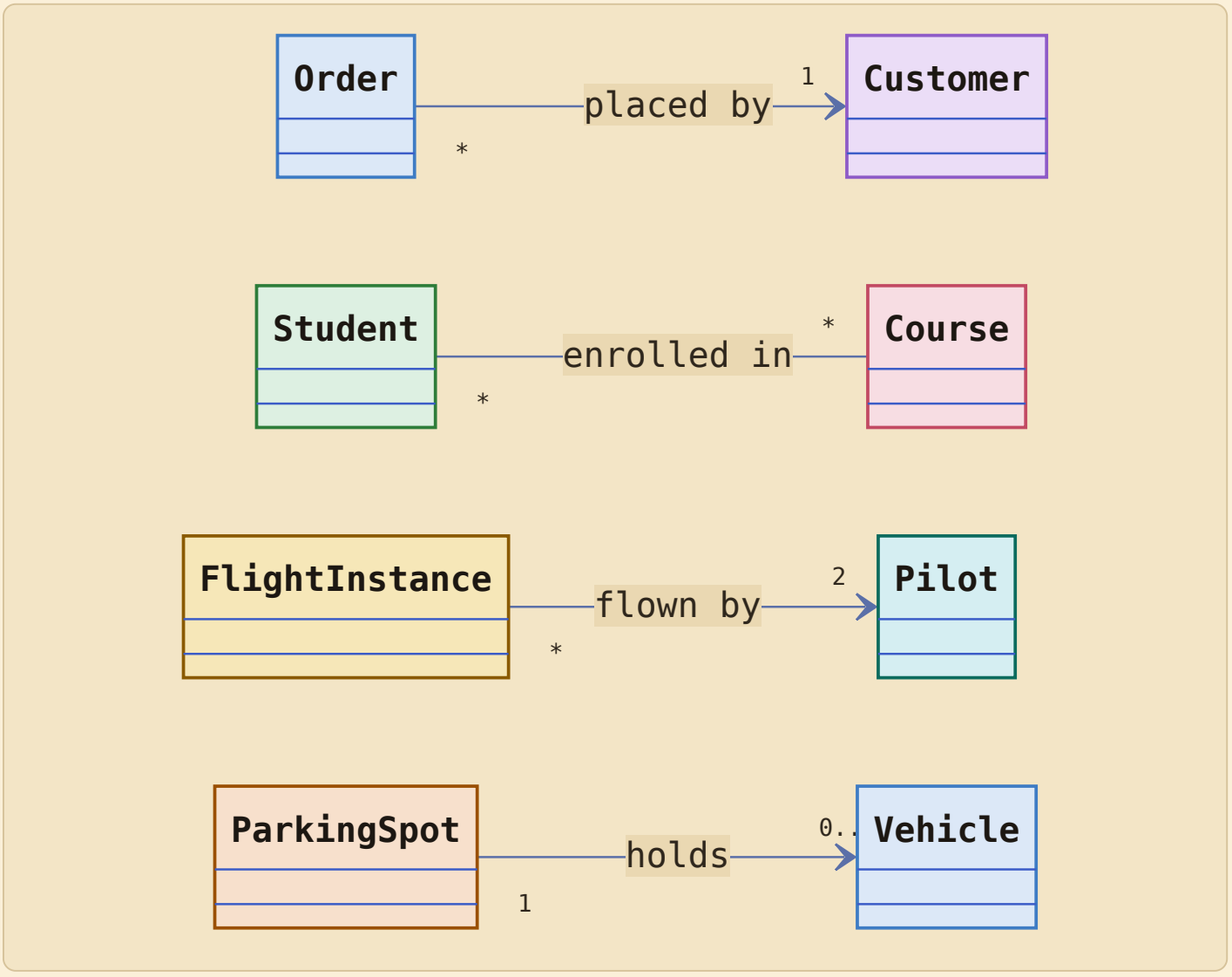
How it works

1. If only one side calls the other, make it **uni-directional**.

2. **Bi-directional** only if real use cases need both directions.
3. For bi-directional, **one side owns the update** and changes both ends in one method.
4. **Enforce multiplicity in code:** `1` → `Objects.requireNonNull`; `0..1` → `Optional<T>`; `*` → collection; `1..*` → non-empty collection; exact numbers → size check in the constructor.

UML notation

Plain solid line = association (navigability unspecified; read as "both know"). **Open arrowhead** = uni-directional (tail knows head). **×** = explicitly not navigable. Numbers sit next to the class they count: `Order "*" --> "1" Customer` = many orders, each with one customer.



Java example

```java
// File: code/lld/ch03/association/Course.java
    public void enroll(Student s) {
        if (students.contains(s)) return;
        if (students.size() == capacity) {
            throw new IllegalStateException(code + " is full (max " + capacity + ")");
        }
        students.add(s);
        s.linkCourse(this);
    }

    public void drop(Student s) {
        if (students.remove(s)) s.unlinkCourse(this);
    }
```

Full runnable code: code/lld/ch03/association/ (see the Full Edition).

- `Order` holds a `final Customer` set with `Objects.requireNonNull`; `Customer` has no field back → **uni-directional**, multiplicity `1`.
- `Student`/`Course` hold sets of each other → **bi-directional many-to-many**; only `Course.enroll/drop` change the link (`Student.linkCourse` is package-private): the **owning-side rule**.
- `FlightInstance` rejects anything but **2** pilots; `ParkingSpot` exposes **0..1** as `Optional<Vehicle>`.

Trade-offs

Uni-directional is simple and low-coupling, but the reverse question needs a query. Bi-directional navigates both ways but has two fields to sync and recursion risk in `toString`/`equals`/JSON. Strict multiplicity checks make invalid states impossible for a little constructor code.

When not to use it · Common mistakes · Real-world examples

Avoid when: Needed only inside one method → a **dependency** (3.5). Don't default to bi-directional. **Mistakes:** Every line bi-directional "just in case"; updating only one side; no multiplicity (and so no check); both sides in `toString()`/`equals()` → stack overflow. **Seen in:** JPA `@ManyToOne` (uni); a bi-directional `@OneToMany(mappedBy=...)` needs an owning side and helpers setting both ends.

In the interview

> **30-second answer:** "Association: a class holds a reference to another and calls it, with no ownership. I make it one-way unless both directions are needed, and write multiplicity on every line because it changes the code: `1` is required, `0..1` an Optional, `1..*` a non-empty collection."

- *Spot holds two motorbikes?* — `0..1` becomes `0..2`: a small collection with a size check.

Self-check

- **Q:** `Order "*" --> "1" Customer`: who has the field? **A:** `Order` (arrow points at the referenced class).

TL;DR

"A knows B", no ownership. Prefer one-way; enforce multiplicity in constructors/types.

3.3 Aggregation

What it is

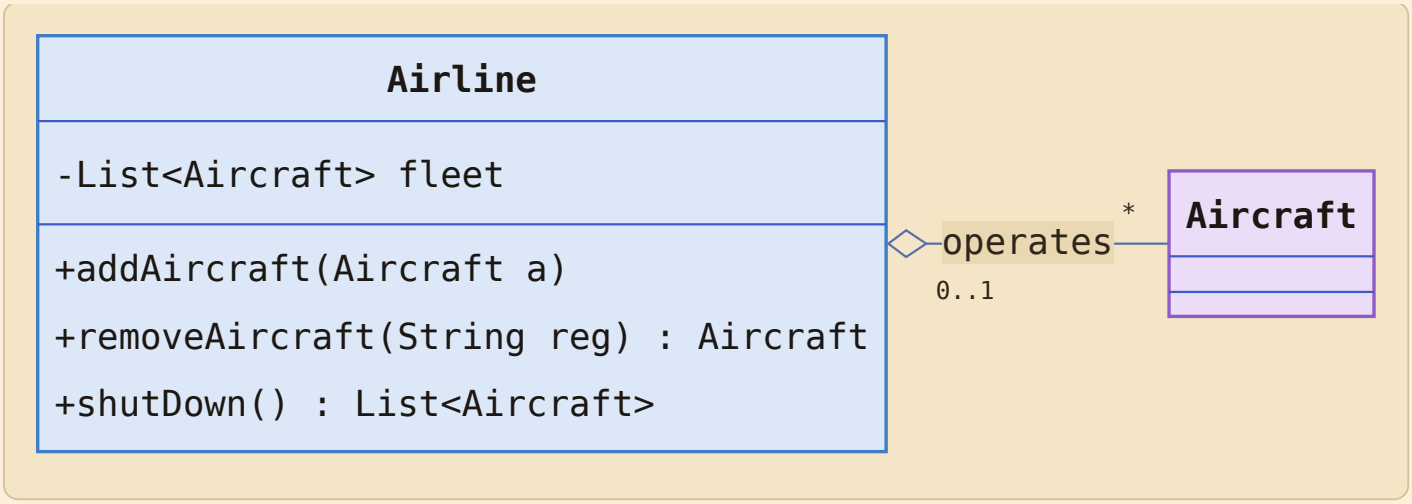
Whole–part association where **parts have their own life**: they exist before, can move, and survive the whole (*shared aggregation*). Analogy: an airline's aircraft are sold, not scrapped.

The problem it solves

It says "these form a group" while making clear the whole does **not** control their lifetime: don't delete parts with the whole.

How it works

1. Parts are **created outside** the whole. 2. They are **passed in** (`addAircraft(a)`). 3. Removing a part **returns it alive**; destroying the whole **releases** parts. 4. **Hollow diamond at the whole**.



Java example

```java
// File: code/lld/ch03/aggregation/Airline.java
    public void addAircraft(Aircraft a) { fleet.add(a); }        // part is passed in
    // ... removeAircraft(registration) hands the part back, still alive

    /** Shutting down the airline releases the parts instead of destroying them. */
    public List<Aircraft> shutDown() {
        List<Aircraft> released = List.copyOf(fleet);
        fleet.clear();
        return released;
    }
```

Full runnable code: code/lld/ch03/aggregation/ (see the Full Edition).

Aircraft are built in `Main` before any airline; one is transferred between airlines; after `skyways = null` its released aircraft join `BlueJet`.

Trade-offs

Aggregation adds whole–part **meaning** but usually **no code difference** from association (both are a field). The UML spec leaves its meaning to the modeller and authors like Fowler advise sparing use; interviews still expect it for clear whole–part groups.

When not to use it · Common mistakes · Real-world examples

Avoid when: No whole–part idea (ticket and spot) → association. Parts created by and dying with the whole → composition. **Mistakes:** Diamond at the **part's** end (it goes at the whole); choosing randomly (test: *if the whole is gone, is the part still meaningful?* Yes → aggregation); deleting parts in the whole's close method. **Seen in:** Archiving a Slack channel keeps its users; deleting a playlist keeps the songs.

In the interview

30-second answer: "Aggregation is whole–part where parts outlive the whole, hollow diamond at the whole. Shut down an airline and the aircraft remain. In Java parts are created outside, passed in, never destroyed by the whole."

- *Airline with no aircraft allowed?* — That's multiplicity (`1..*`), not aggregation vs composition.

Self-check

- **Q:** A library closes; do members vanish? **A:** No: Library ◇— Member is aggregation.

TL;DR

"Has-a, parts live on"; passed in, never destroyed; hollow diamond at the whole.

3.4 Composition

What it is

The whole **owns** its parts: each belongs to **exactly one** whole, is usually **created by** it and **dies with** it (or is explicitly removed first). Analogy: house and rooms; floor and spots.

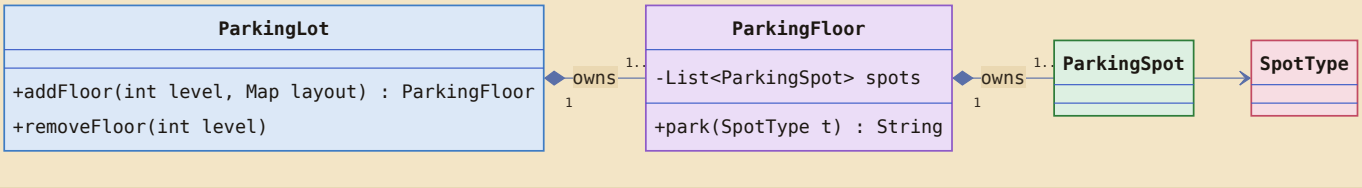
The problem it solves

Some objects make no sense alone; letting outsiders create or share them causes bugs (two floors sharing a spot, orphan spots). One owner handles the full life cycle.

How it works

1. The whole **creates** parts (constructor or `addX(data)`, not a ready-made part).
2. The part's constructor is **not public**.
3. Never hand out the internal collection: read-only views or copies.
4. Removing the whole drops the only reference → part becomes garbage.
5. **Filled diamond at the whole**.

Java has no destructors: parts die **logically** (unreachable → GC); for resources the whole's `close()` closes its parts.



Java example

```java
// File: code/lld/ch03/composition/ParkingFloor.java
    ParkingFloor(int level, Map<SpotType, Integer> layout) {
        this.level = level;
        int n = 1;
        for (var e : layout.entrySet()) {
            for (int i = 0; i < e.getValue(); i++) {
                spots.add(new ParkingSpot("L" + level + "-" + n++, e.getKey()));
            }
        }
        if (spots.isEmpty()) throw new IllegalArgumentException("a floor needs 1..* spots");
    }

    public List<ParkingSpot> spots() { return Collections.unmodifiableList(spots); }
```

Full runnable code: code/lld/ch03/composition/ (see the Full Edition).

`ParkingLot.addFloor(level, layout)` takes a *description* and builds the floor itself; `ParkingSpot`'s constructor is package-private; `spots().clear()` throws; `removeFloor(1)` takes its 4 spots with it. Spot kinds are a `SpotType` enum, not subclasses.

Trade-offs

Composition: the whole creates the part, no sharing, part doesn't outlive it; strong encapsulation, lower flexibility. Aggregation: someone else creates it, sharable, outlives the whole; weaker encapsulation, more flexibility.

When not to use it · Common mistakes · Real-world examples

Avoid when: The part has its own business identity (person, aircraft, song) or must be shared. **Mistakes:** Accepting a ready-made part (`addSpot(ParkingSpot s)`) and calling it composition; returning the internal list; confusing it with "favour composition over inheritance" (3.6), where "composition" means *holding* an

object, including association/aggregation. **Seen in:** `HashMap`'s internal `Node`s; JPA `cascade = ALL, orphanRemoval = true` (delete an `Order`, lose its `OrderLine`s).

In the interview

> **30-second answer:** "Composition: the whole creates its parts, they belong only to it and die with it; filled diamond at the whole. In Java I build parts inside the owner, hide their constructor, and expose read-only views."

- *With GC, how does a part 'die'?* — Logically: the whole held the only reference.

Self-check

- **Q:** Why is `ParkingSpot`'s constructor package-private? **A:** Only the owning floor may create spots.

TL;DR

"Owns, parts die with it"; whole creates, never shares, read-only views; filled diamond.

3.5 Dependency

What it is

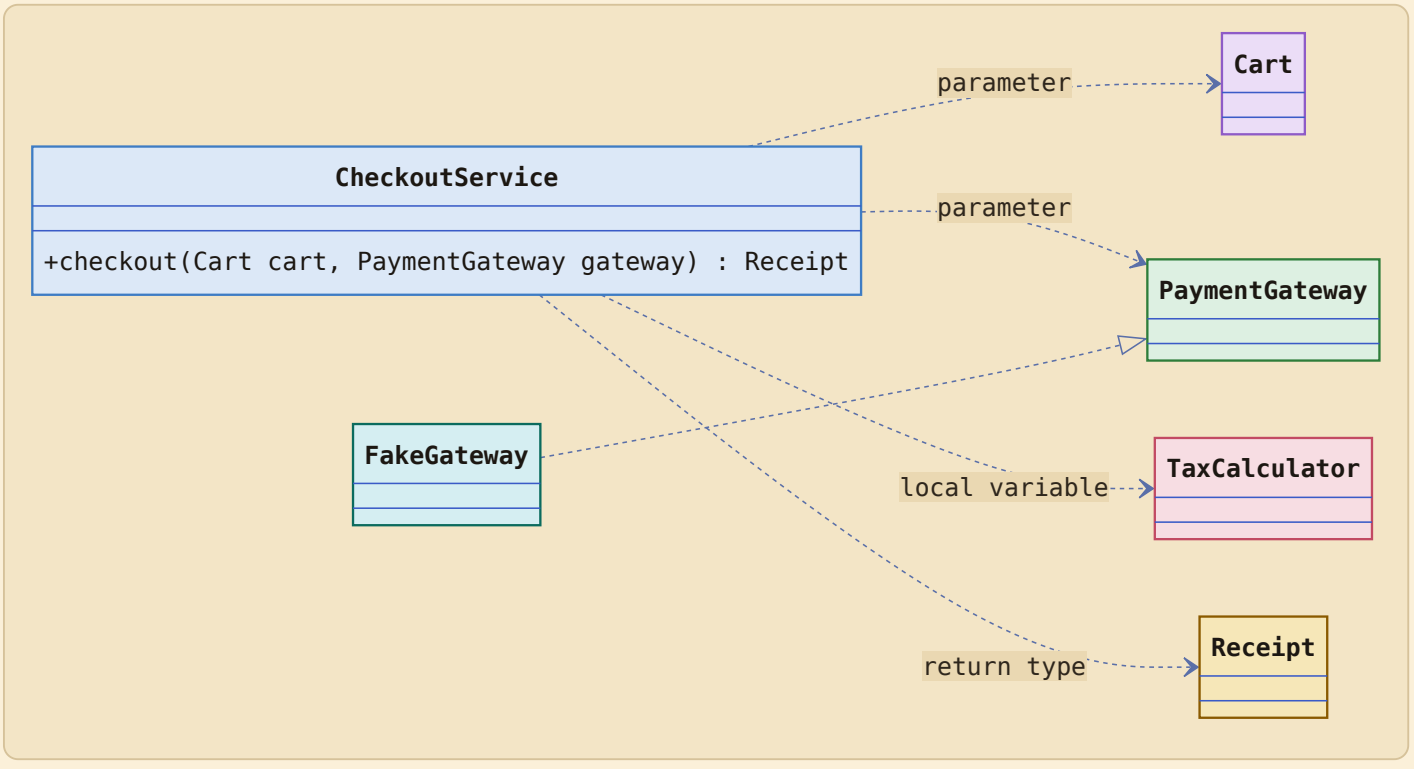
A **uses** B briefly, inside one method, with **no field**: B is a **parameter**, **local variable**, **return type** or **static call**. Analogy: a chef uses today's delivery van.

The problem it solves

Records the weakest link ("if B's interface changes, A may change"); keeps classes stateless and lets callers pass a different collaborator per call.

How it works

B as parameter / `new` local / return type / static call; no field of type B. UML: **dashed line, open arrowhead** from A to B, optionally `«use»`/`«create»`.



Java example

```java
// File: code/lld/ch03/dependency/CheckoutService.java
public final class CheckoutService {

    public Receipt checkout(Cart cart, PaymentGateway gateway) {
        TaxCalculator tax = new TaxCalculator();            // local-variable dependency
        Money subtotal = cart.subtotal();                   // parameter dependency
        Money total = subtotal.plus(tax.taxOn(subtotal));
        String txn = gateway.charge(total);                 // parameter dependency (interface)
        return new Receipt(txn, subtotal, total);           // return-type dependency
    }

    public record Receipt(String txnId, Money subtotal, Money total) { }
}
```

Full runnable code: code/lld/ch03/dependency/ (see the Full Edition).

`CheckoutService` has **no fields**; `Main` passes a `FakeGateway` and then a lambda gateway on the next call. `Money` uses `BigDecimal`.

Trade-offs

Dependency: lowest coupling, stateless, swap per call, but callers pass B every time. Association (field): higher coupling and state, swap per object, pass once.

When not to use it · Common mistakes · Real-world examples

Avoid when: When A uses B across many methods (a service and its repository), a constructor-injected field is cleaner. **Mistakes:** Solid association line for a mere parameter; drawing every dependency (`String`, `List`...); hiding `new ConcreteThing()` deep inside where the caller should decide (blocks testing). **Seen in:** `Collections.sort(list, comparator)`; `LocalDate.now(Clock clock)`.

In the interview

> **30-second answer:** "Dependency is 'uses briefly': parameter, local or return type, no field; dashed arrow. It's the loosest coupling, so a collaborator needed in one method is passed in, not stored."

- *Relation to DI?* — DI is *who supplies* the collaborator; it may be stored (association) or passed per call (dependency). DIP adds "depend on interfaces" (Chapter 4).

Self-check

- **Q:** Dashed + open arrow vs dashed + hollow triangle? **A:** Dependency vs realization.

TL;DR

"Uses for one call"; no field; dashed arrow.

3.6 Generalization (inheritance)

What it is

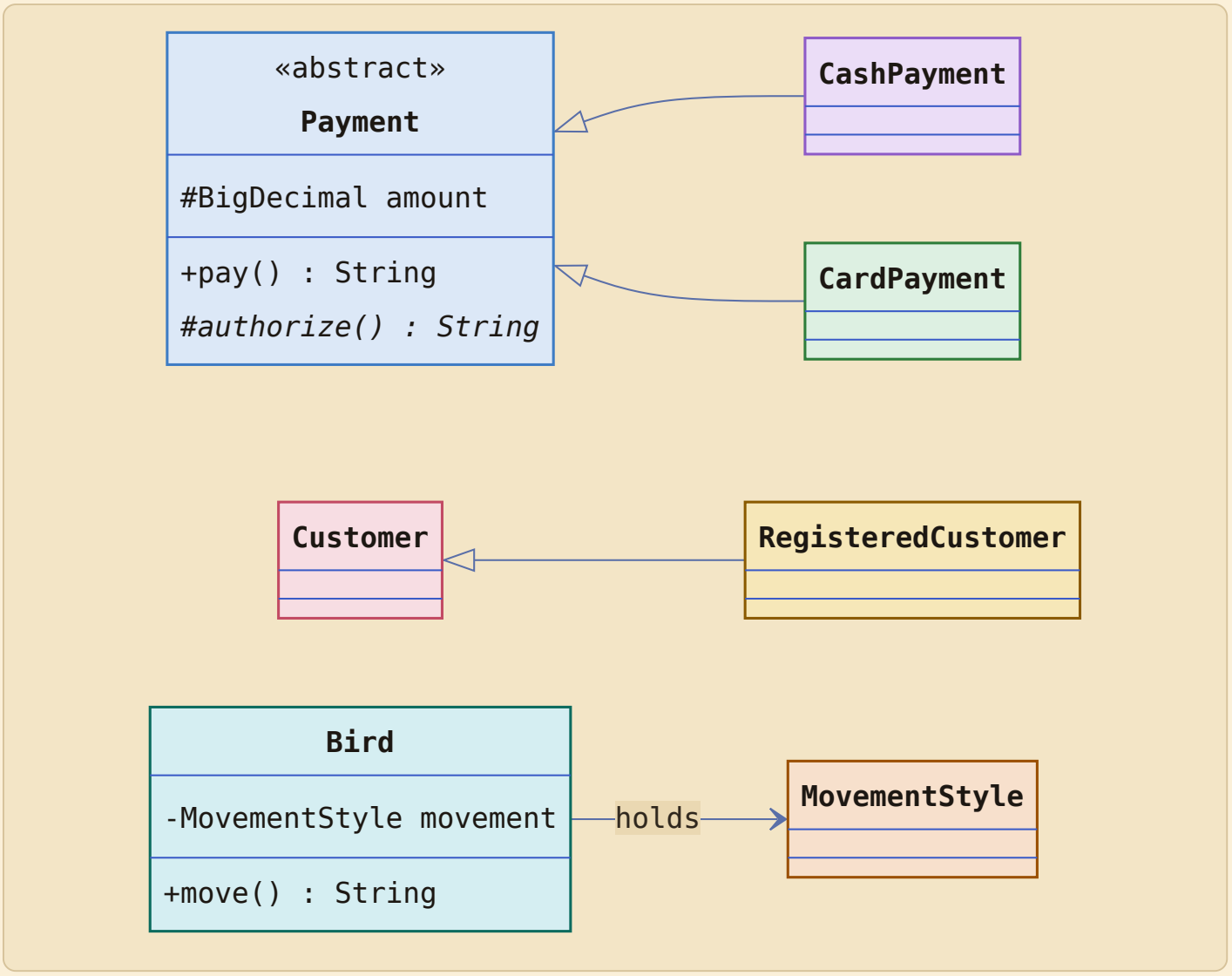
UML's word for **inheritance**: B `extends` A, so B *is a kind of* A. Analogy: a card payment *is a* payment.

The problem it solves

Classes sharing a contract **and** behaviour keep shared parts in one place, and callers use the general type (**polymorphism**).

How it works

1. Run the **substitution test** (Liskov, Chapter 4): can B go anywhere A is expected with no refused or broken method?
2. Yes → shared state/workflow in A (often `abstract`), subclasses override varying steps.
3. No → **hold instead of extend** (an interface-typed field).
4. UML: **solid line, hollow triangle at the parent**; abstract names in *italics* or `{abstract}`.



Java example

```java
// File: code/lld/ch03/generalization/Payment.java
public abstract class Payment {
    protected final BigDecimal amount;   // '#' protected: visible to subclasses
    // ... constructor rejects non-positive amounts

    public final String pay() {
        String ref = authorize();
        return getClass().getSimpleName() + " of " + amount + " -> " + ref;
    }

    protected abstract String authorize();   // abstract operation (italics in UML)
}
```

Full runnable code: code/lld/ch03/generalization/ (see the Full Edition).

- **Good hierarchy:** cash and card both truly take money, so `Main` loops over `List<Payment>`; amounts are `BigDecimal` compared with `compareTo`.

- **Fix #1, subclass that must refuse:** a `Guest extends Customer` that can't order breaks substitution → `Customer` only browses; ordering moves to `RegisteredCustomer`.
- **Fix #2, subclass that only carries a constant:** `CompactSpot` / `LargeSpot` → a `SpotType` enum field.
- **Hold, don't extend:** `Penguin` breaks a `FlyingBird` hierarchy; `Bird` holds a `MovementStyle` field (FLY, SWIM, WALK) and `setMovement` swaps it at run time.

Trade-offs

Inheritance: automatic reuse, fixed at compile time, tight coupling (fragile base class), one parent; best for true is-a with different behaviour. Holding an object: explicit delegation, swappable at run time, loose coupling, many fields/interfaces; best for varying or combinable behaviour.

When not to use it · Common mistakes · Real-world examples

Avoid when: Just sharing fields/helpers; a child would throw `UnsupportedOperationException`; data-only variation; mixable behaviours (fly + swim + walk → class explosion). **Mistakes:** Hierarchies deeper than 2–3; `Stack extends Vector` style leaks; triangle pointing at the child; inheriting when the relation holds only *sometimes* (use a role/interface). **Seen in:** `ArrayList extends AbstractList`; `FileInputStream extends InputStream`; `IOException extends Exception`. Counter-examples: `Stack extends Vector`, `Properties extends Hashtable`.

In the interview

> **30-second answer:** "Generalization is inheritance, solid line with a hollow triangle at the parent. Only when the child passes the substitution test. A subclass that only passes a constant becomes an enum field; varying behaviour I hold as an interface."

- *Why not `CompactSpot extends ParkingSpot`?* — No behaviour; a `SpotType` field does it.

Self-check

- **Q:** Guest can't order but `Customer.placeOrder()` exists. **A:** LSP broken; move `placeOrder` to `RegisteredCustomer`.

TL;DR

"Is-a", `extends`, only after the substitution test; triangle at the parent.

3.7 Realization (interface implementation)

What it is

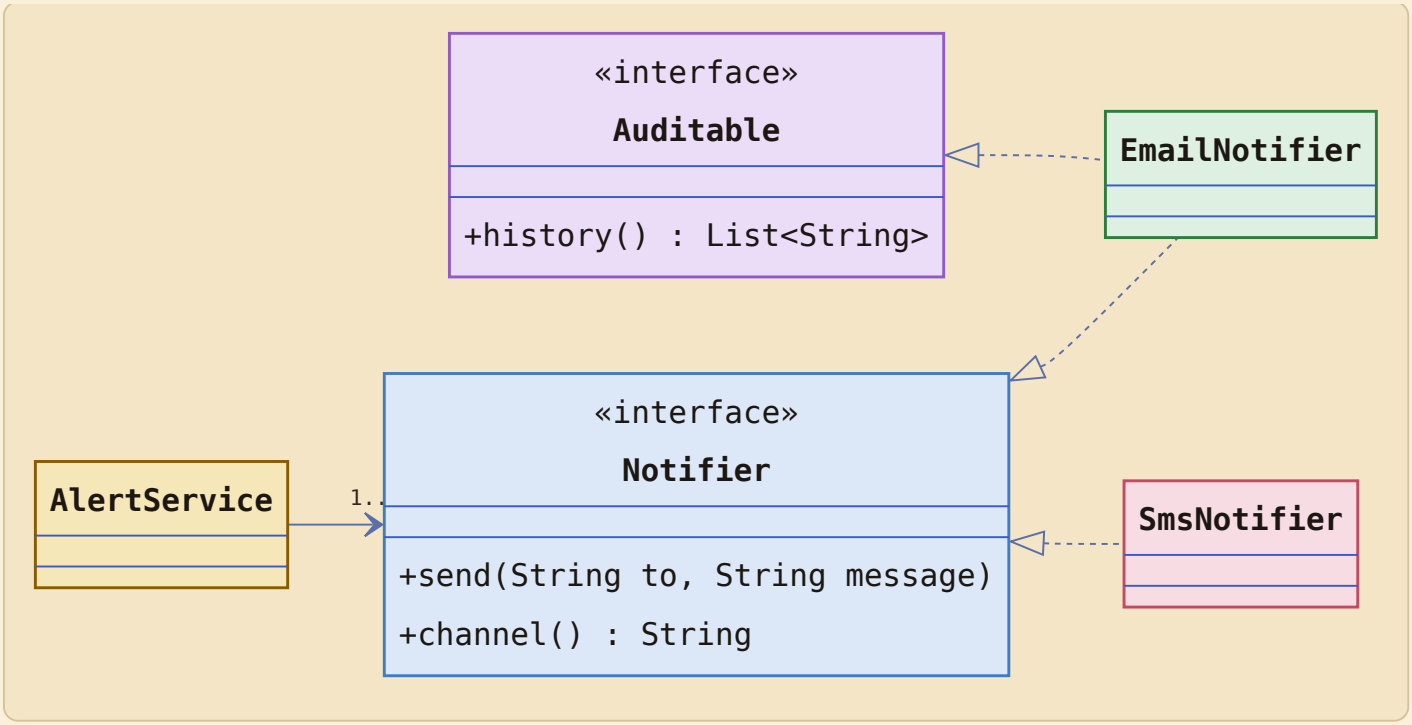
A class **implements an interface** (a stateless contract): `class EmailNotifier implements Notifier`. Analogy: a plug that realizes a socket standard.

The problem it solves

Callers depend on the **contract**: new implementations need no caller change, fakes for tests, several roles per class.

How it works

Define the operations callers need; classes `implement` them; callers hold the **interface type**; `default` / `static` methods allowed. UML: **dashed line, hollow triangle at the interface**, `«interface»` (or a "lollipop"). Same is-a idea as generalization, but only the contract is inherited.



Java example

```java
// File: code/lld/ch03/realization/AlertService.java
public final class AlertService {
    private final List<Notifier> notifiers;

    public AlertService(List<Notifier> notifiers) { this.notifiers = List.copyOf(notifiers); }

    public void alert(String to, String message) {
        for (Notifier n : notifiers) {
            System.out.println("via " + n.channel());
            n.send(to, message);
        }
    }
}
```

Full runnable code: code/lld/ch03/realization/ (see the Full Edition).

`EmailNotifier` realizes **two** interfaces; `SmsNotifier` overrides the `default channel()`; a **lambda** also realizes `Notifier` (functional interface). `AlertService` depends only on the interface, so a `PushNotifier` needs no change.

Trade-offs

Interface: many per class, no instance fields, shared code only via `default`; for roles, plug-ins, test seams. Abstract class: one parent, any fields and code; for a family sharing state and workflow.

When not to use it · Common mistakes · Real-world examples

Avoid when: A single never-varying implementation with no test need (YAGNI); empty "marker" interfaces (annotations serve better). **Mistakes:** Solid instead of dashed line; fat interfaces (ISP); variables typed with the concrete class. **Seen in:** JDBC `Connection` realized by vendor drivers; SLF4J `Logger` realized by Logback/Log4j.

In the interview

30-second answer: "Realization: a class implements an interface, dashed line with a hollow triangle at the interface. I add interfaces where behaviour varies or I need a test double (gateway, notifier, pricing), and the design depends only on them."

- *Interface or abstract class?* — Interface for capabilities (many allowed); abstract class for shared state/workflow; combine as `List` → `AbstractList` → `ArrayList`.

Self-check

- **Q:** Generalization vs realization drawn? **A:** Same hollow triangle; solid vs dashed line.

TL;DR

"Implements": dashed line + hollow triangle at `«interface»`; depend on the interface.

3.8 Decision guide: choosing the relationship between two classes

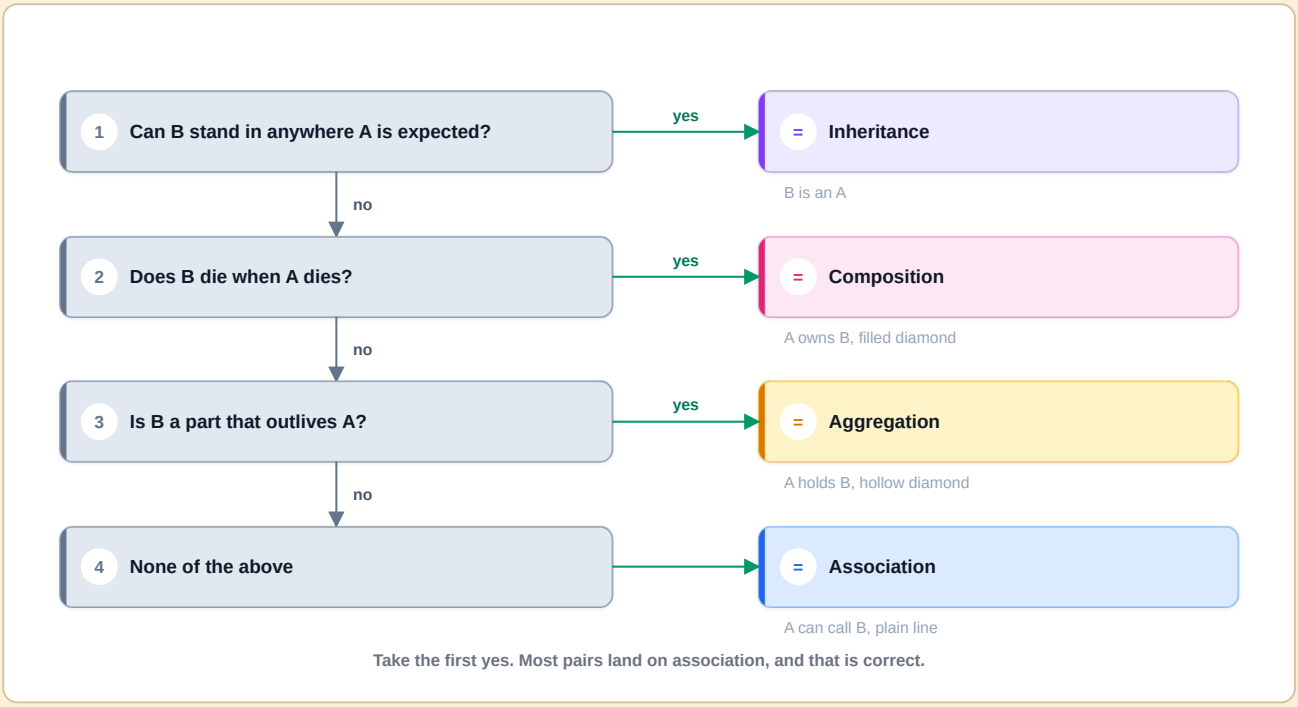
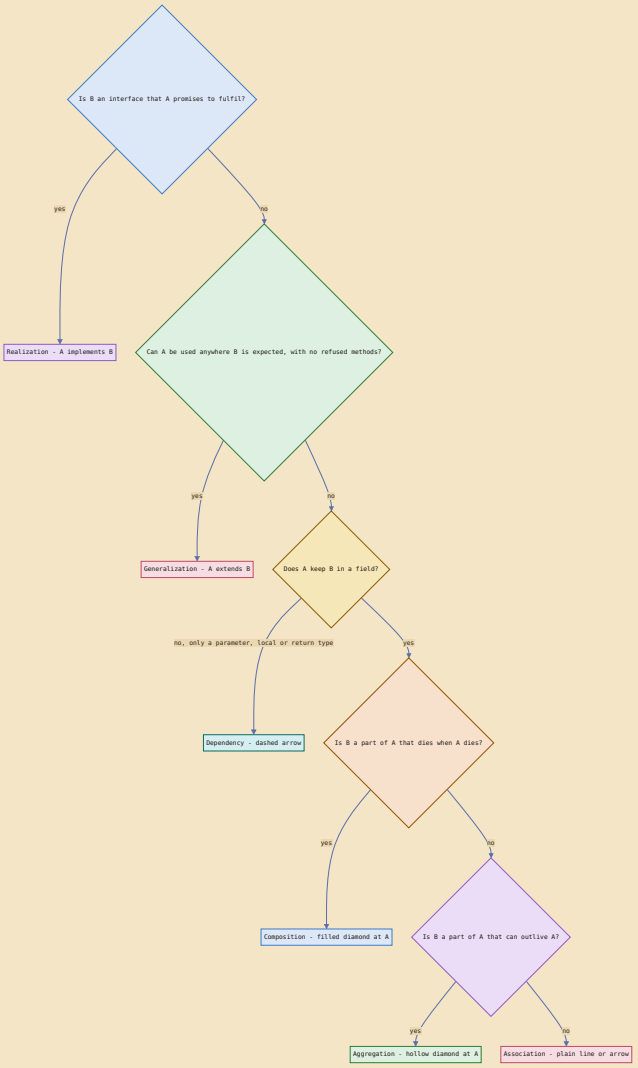


Figure — The four-question decision: inheritance, then composition, then aggregation, else association (course diagram)

Ask in order; stop at the first yes. Inheritance comes first only because it's easiest to **rule out**; most pairs end as association.



Summary table (keep this one)

Relationship	Phrase	UML line	Java shape	Lifetime	Example
Dependency	A *uses* B	dashed, open arrow → B	parameter/local/return/static call	none	`CheckoutService` uses `PaymentGateway`
Association	A *knows* B	solid (arrow if one-way)	field, B made elsewhere	independent	`Order` → `Customer`
Aggregation	A *has* B	solid, hollow ◇ at A	field/collection, parts passed in	B outlives A	`Airline` ◇— `Aircraft`
Composition	A *owns* B	solid, filled ◆ at A	A creates B, hides it	B dies with A	`ParkingFloor` ◆— `ParkingSpot`
Realization	A *implements* B	dashed, hollow △ at B	`implements`	—	`EmailNotifier` ⇢ `Notifier`
Generalization	A *is a* B	solid, hollow △ at B	`extends`	—	`CardPayment` → `Payment`

Mermaid cheat-sheet: `A ..> B` dependency · `A --> B` association A→B · `A -- B` association (both/unspecified) · `A o-- B` aggregation (A whole) · `A *-- B` composition (A whole) · `B <|.. A` A realizes B · `B <|-- A` A

extends B · `A "1" *-- "1..*" B` with multiplicities.

Common mistakes (all relationships)

Diamond not at the **whole**; triangle not at the **parent/interface**; lines without multiplicity; inheritance to share data.

In the interview

Say the test **aloud** per line ("remove the floor and its spots go: composition"); collapse constant-only subclasses yourself. Expect "why not inheritance?" (substitution test) and multiplicity changes.

Self-check

- **Q:** `Library`/`Member`? **A:** Aggregation or association. **Q:** `Flight`/`WeeklySchedule`? **A:** Composition.
- **Q:** `ParkingTicket`/`ParkingSpot`? **A:** Association. **Q:** `Report.render(Formatter f)`? **A:** Dependency.

TL;DR

Is-a only after the substitution test; composition if parts die with the whole, aggregation if they live on, association if one just knows the other, dependency if only used in a method.

Part B — UML

3.9 What UML is (structural vs behavioural diagrams)

What it is

UML is the OMG's standard design notation (UML 2.5.1): not a programming language, a whiteboard suffices. It merged Booch, OMT (Rumbaugh) and OOSE (Jacobson) in the 1990s, hence its size.

The problem it solves

Makes a design **visible**, forces **explicit decisions** (filled or hollow diamond?), and makes **disagreement cheap** (the interviewer taps a line and asks why).

How it works — the 14 diagram types

Structural (what it is made of): **Class** (always), Object, Package, Component, Composite structure, Deployment, Profile. **Behavioural** (what it does): **Use case**, **Activity**, **State machine**, and the **interaction** diagrams **Sequence**, Communication, Interaction overview, Timing. Component/deployment belong to system design rounds; profile is never used in interviews.

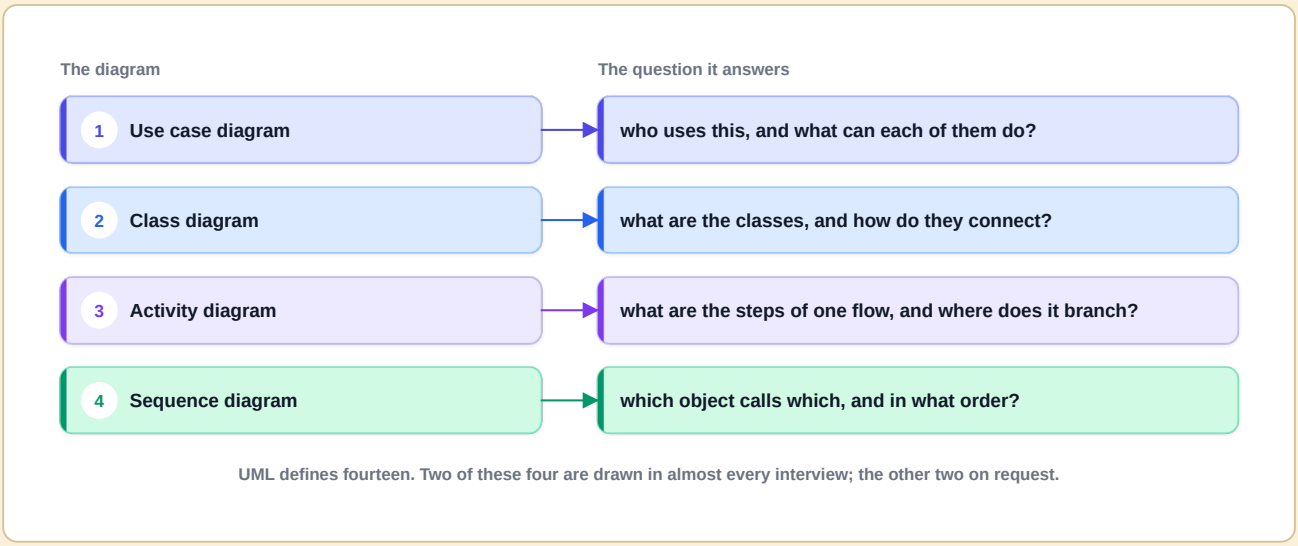


Figure — The four diagrams an OOD interview uses and the question each answers (course diagram)

Interview set: **use case** (who does what; first), **class** (the answer, maps to code), **activity** (steps and branches of one flow), **sequence** (who calls whom), plus **state machine** when an object has a lifecycle.

Trade-offs · When not to use it · Common mistakes · Real-world examples

Shared vocabulary and tools (PlantUML, Mermaid) vs a big spec, whiteboard time, stale diagrams. Avoid unmaintained formal docs; sketch UML, C4 or Mermaid beside the code is the norm, UML-lite for interviews. **Mistakes:** Thinking notation is graded: what's graded is whether **diagram and words agree** (say what a symbol means if unsure). A line you can't justify hurts more than a missing one. **Seen in:** Mermaid in GitHub/GitLab design docs; PlantUML in Confluence.

In the interview

> **30-second answer:** "UML has 14 diagrams: structural (class, object, component...) and behavioural (use case, activity, state machine, sequence...). In an OOD round I use a quick use-case list, one class diagram, and an activity or sequence diagram for the key flow."

Self-check

- **Q:** Sequence diagram family? **A:** Behavioural (interaction). **Q:** How many types? **A:** 14 (7 + 7).

TL;DR

Standard symbols, not code; structural vs behavioural; interviews use class, use case, activity, sequence, state.

3.10 Use-case diagrams

What it is

Shows **who** uses the system and **what** each can achieve; no classes, data or order.

- **Actor** — a *role* outside the system (stick figure); may be non-human (payment gateway, timer).
- **Use case** — one goal with a result the actor cares about (oval, verb phrase): "Borrow a book", not "click search".
- **System boundary** — rectangle: inside = what you build.

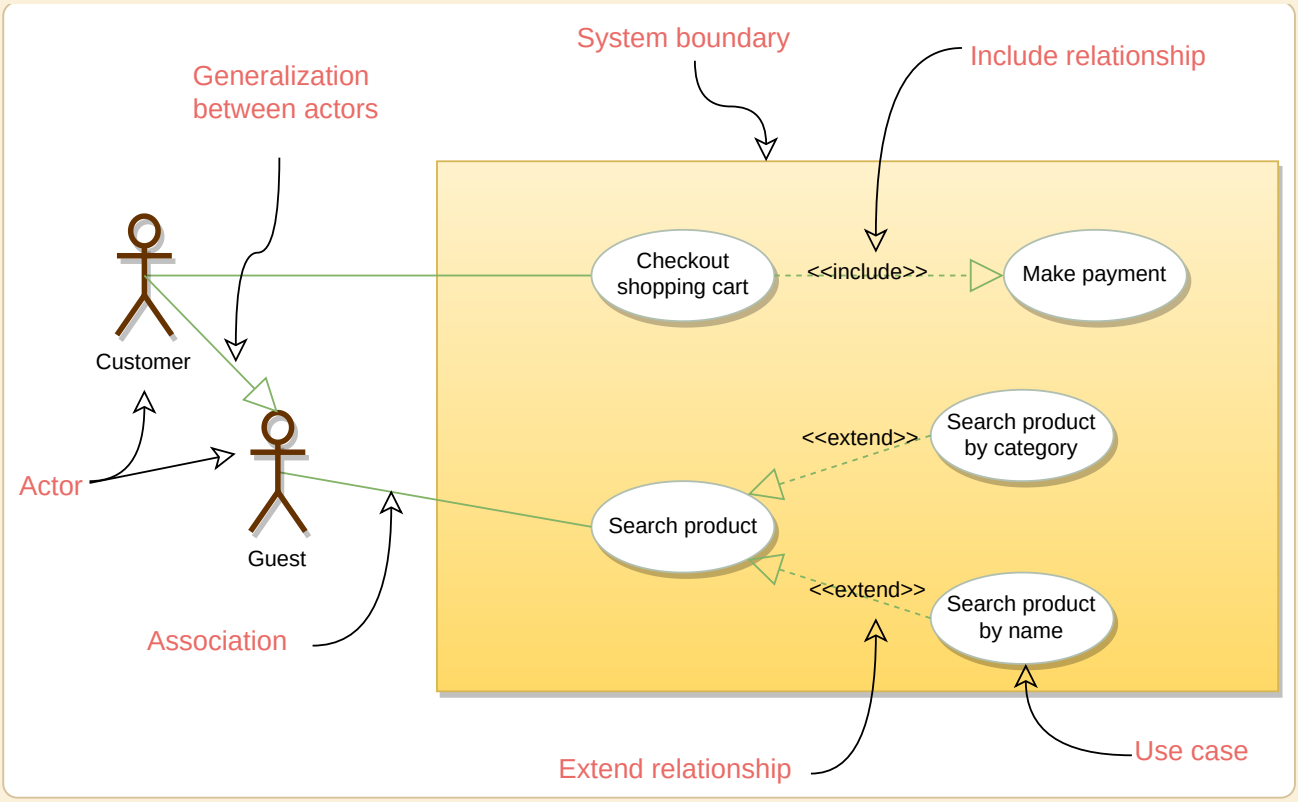


Figure — The parts of a use-case diagram for online shopping: actors (Customer, Guest), use cases, system boundary, generalization between actors, and «include» / «extend» relationships (course diagram)

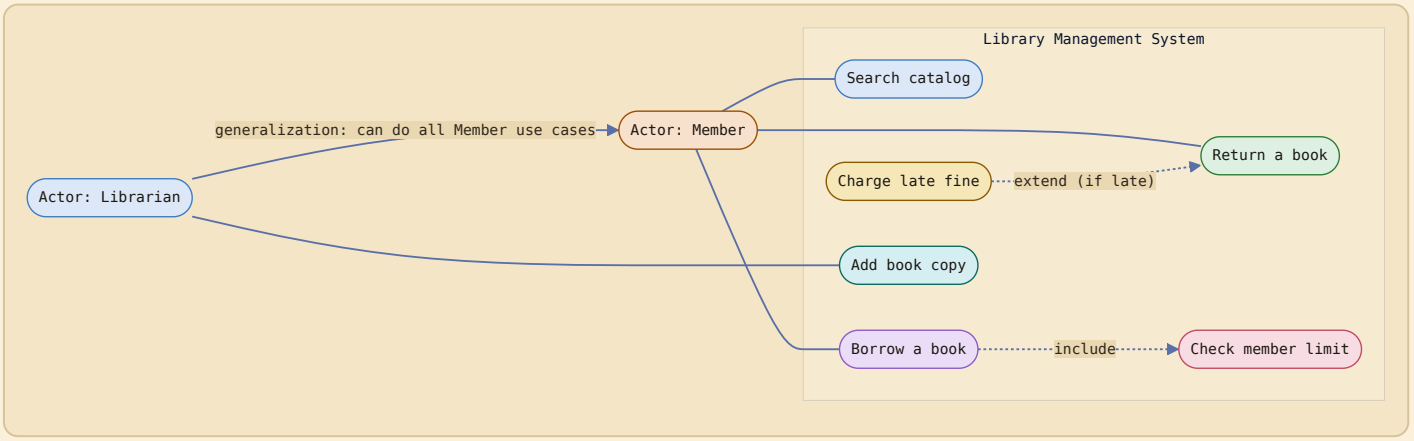
The problem it solves

Turns a one-liner into an **agreed scope** ("these nine are in; refunds and reporting are out"). **Actors usually become classes; use cases become methods.**

How it works

1. Find actors: *who starts something? who administers it* (most forgotten)? *who gets told? what acts with nobody present* (timers, jobs)?
2. Primary actors left, secondary (gateway) right; use cases inside the boundary, joined to actors by plain lines.
3. **«include»** = *always*; dashed arrow **base → included** (*Borrow* «include» *Check limit*). **«extend»** = *sometimes*, at an extension point; arrow **extension → base** (*Charge fine* «extend» *Return*). Test: can the base complete without it? Yes → extend.
4. **Generalization** (hollow triangle) between actors or use cases. Keywords are singular; old «uses» = «include».

In the course figure the triangle at Guest means a Customer can do all a Guest can, plus check out.



From use cases to code (see `lld.ch03.library`)

Use-case element	Java
Actor *Member* / *Librarian*	`class Member` / `class Librarian extends Member`
Borrow a book + «include» *Check limit*	`Member.checkout(...)` calling private `checkLimit()` **every** time
Return a book + «extend» *Charge fine*	`Member.returnBook(...)` calling `chargeFine(...)` **only if** late
Add book copy	`Librarian.addCopy(...)`

A starting point: some use-case methods belong on a service class.

Trade-offs · When not to use it · Common mistakes · Real-world examples

Fast scope agreement and finds forgotten actors, but shows no order or data and bloats with UI steps. If requirements are already precise, a spoken list suffices. **Mistakes:** UI steps as use cases; one actor per person (a borrowing librarian is two roles); reversed «extend»; «include» for something occasional; no boundary. **Seen in:** *Place order* «include» *Make payment*; *Apply gift card* «extend»s it.

In the interview

> **30-second answer:** "Before classes I list actors (roles, including admins, external services, timers) and their goals, with a boundary for scope. «include» is always, «extend» sometimes. Actors become classes, use cases methods." Takes 3–5 minutes, early.

Self-check

- **Q:** «include» arrow direction? **A:** Base → included.
- **Q:** Is a payment gateway an actor? **A:** Yes, secondary.

TL;DR

Actors outside, goals inside, boundary = scope; include always, extend sometimes.

3.11 Class diagrams

What it is

The **static structure**: classes, attributes, operations, relationships with multiplicity. The answer to an OOD question, and the only UML diagram mapping directly to code.

The problem it solves

Puts the whole design on one page to review, challenge and code; other diagrams lead to it or test it.

How it works — the class box

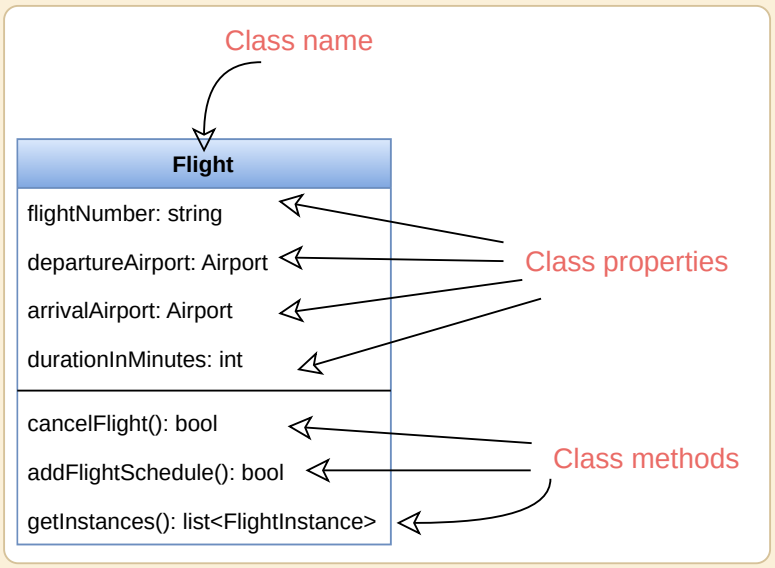


Figure — A class box: name on top, properties (attributes) in the middle, methods (operations) at the bottom, using a Flight class (course diagram)

```
visibility name : Type [multiplicity] = default      e.g.  - spots : ParkingSpot [1..*]
visibility name(param : Type, ...) : ReturnType      e.g.  + park(v : Vehicle) : Ticket
```

Visibility: `+` public, `-` private, `#` protected (Java's `protected` also opens the package), `~` package (no modifier). *Italic* or `{abstract}` = abstract; `«interface»`, `«enumeration»`; underlined = static; `{readOnly}` = `final`.

A name-only box is fine early. Mermaid: same visibility symbols, `*` = abstract, `$` = static, `List~T~` for generics.

How it works — the relationships (all arrow types)

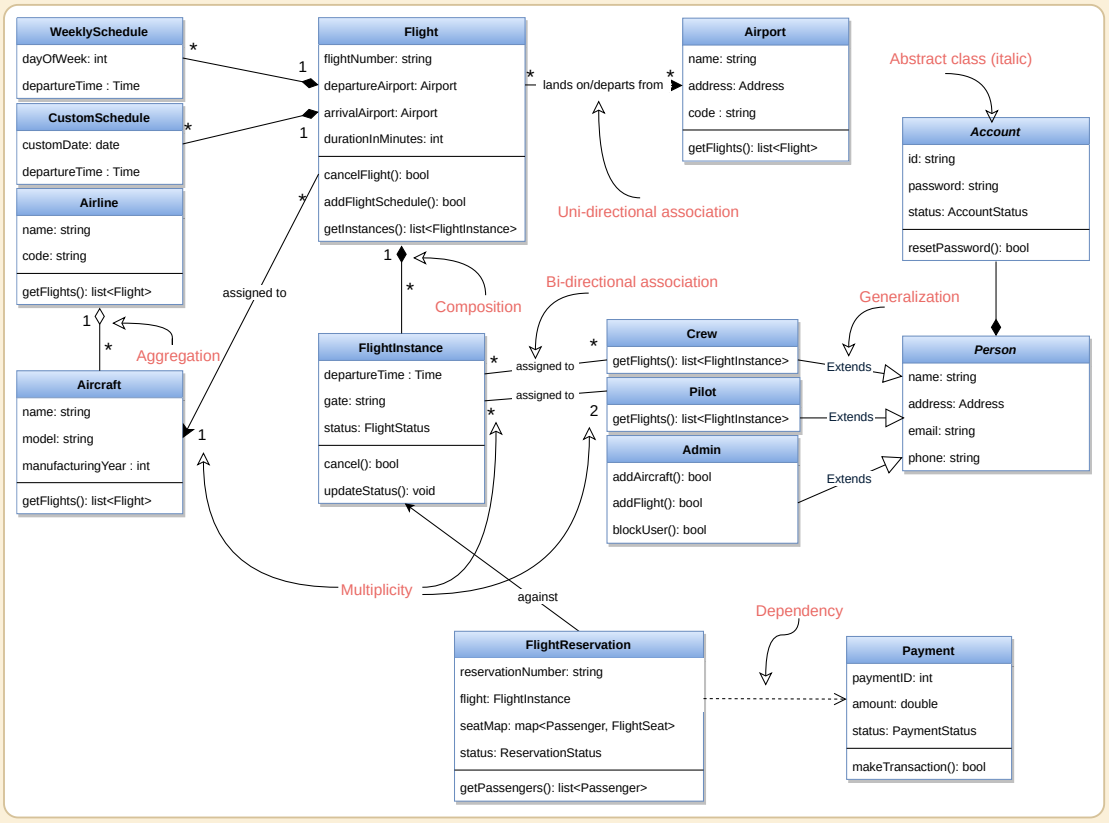
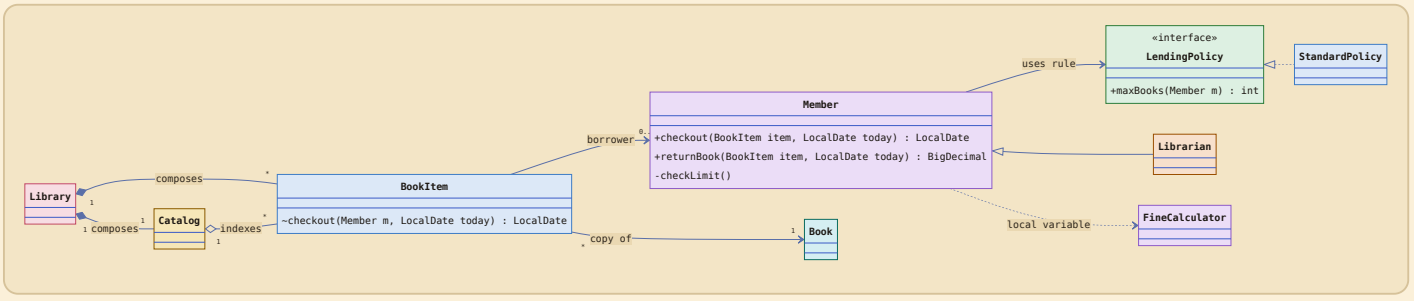


Figure — An airline-system class diagram using every relationship: associations (e.g. Flight to Aircraft, uni-directional), multiplicities (FlightInstance has 2 Pilots), aggregation (Airline–Aircraft), composition (Flight–WeeklySchedule), generalization (Person to Crew/Pilot/Admin) and dependency (course diagram)

The figure uses every line: association FlightInstance — Pilot (two-way) and Flight → Airport (one-way); multiplicity `*` — `2` Pilot; aggregation Airline ◇— Aircraft; composition Flight ◆— WeeklySchedule; generalization Pilot/Crew/Admin → Person; dependency FlightReservation ⇢ Payment. Keep arrows consistent with fields.

Also recognise: **association class** (dashed-attached class holding link data, e.g. `Enrollment{grade}`) and **qualified association** (key box, `Library [isbn] → Book` = a `Map<Isbn, Book>`).

Worked example — the library design (code in `lld.ch03.library`)



Every line type appears; `~` marks package-private methods like `BookItem.checkout`, callable only by `Member`.

How much detail to draw

10–15 named boxes; attributes/operations only where flows need them; **every line typed with multiplicity**; **drop attributes before relationships**.

Trade-offs · When not to use it · Common mistakes · Real-world examples

Detailed = near code-ready but 10+ minutes; sketch = fast and changeable. It shows no order or time: use behaviour diagrams for that. **Mistakes:** Unlabelled lines; diamonds/triangles at the wrong end; every getter and `String` field listed; lines with no matching field in code (or vice versa). **Seen in:** JPA entity diagrams are class diagrams with multiplicities.

In the interview

> **30-second answer:** "Named boxes from nouns and actors, then every line with type and multiplicity, saying the test aloud; only the attributes and methods the key flows need. `+ - # ~` for visibility, «abstract», «interface»."

- `#` *nuance in Java?* — Also visible within the package. *Static?* — Underline (Mermaid `$`).

Self-check

- **Q:** `~`? **A:** Package visibility. **Q:** Aggregation vs composition test? **A:** Is the part meaningful once the whole is gone?

TL;DR

Box = name / attributes / operations; seven line types plus multiplicity on every line.

3.12 Sequence diagrams

What it is

Which object calls which, in what order, for one scenario: objects across the top, **time down the page**.

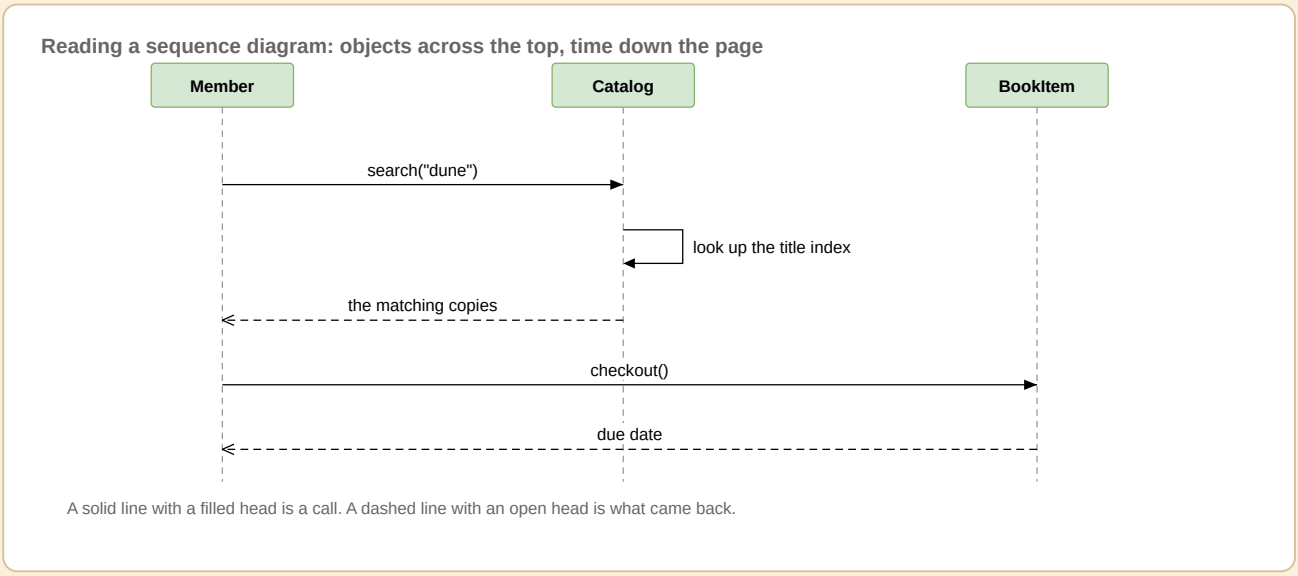


Figure — A member searches the catalog for "dune" and borrows a copy: lifelines, a self-call on Catalog, calls with filled arrowheads and dashed returns (course diagram)

The problem it solves

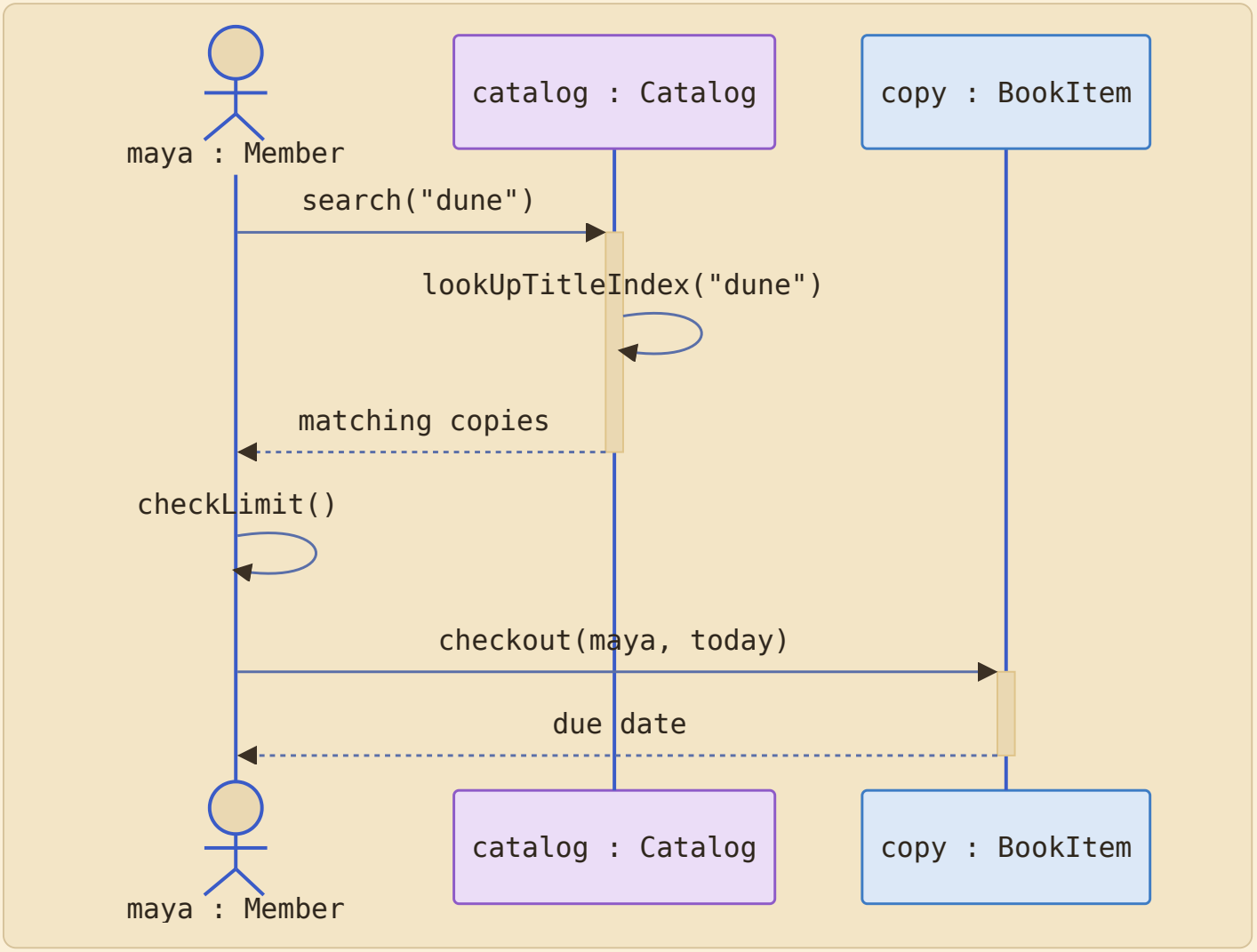
Shows what objects *actually* say and when: vital when order is easy to get wrong (reserve → pay → confirm), when several objects take part, or to explain a class diagram.

How it works — the notation

- **Lifeline:** box + dashed vertical line, one object (`catalog : Catalog`); **actor** (stick figure) starts the flow.
- **Sync message:** solid, **filled** head (caller waits). **Async:** solid, **open** head (fire-and-forget). **Return:** dashed, open head (only when it matters). **Self-message:** loops back.
- **Activation bar:** thin rectangle, object busy. **Create:** dashed «create» arrow to a new box; **destroy: X**.

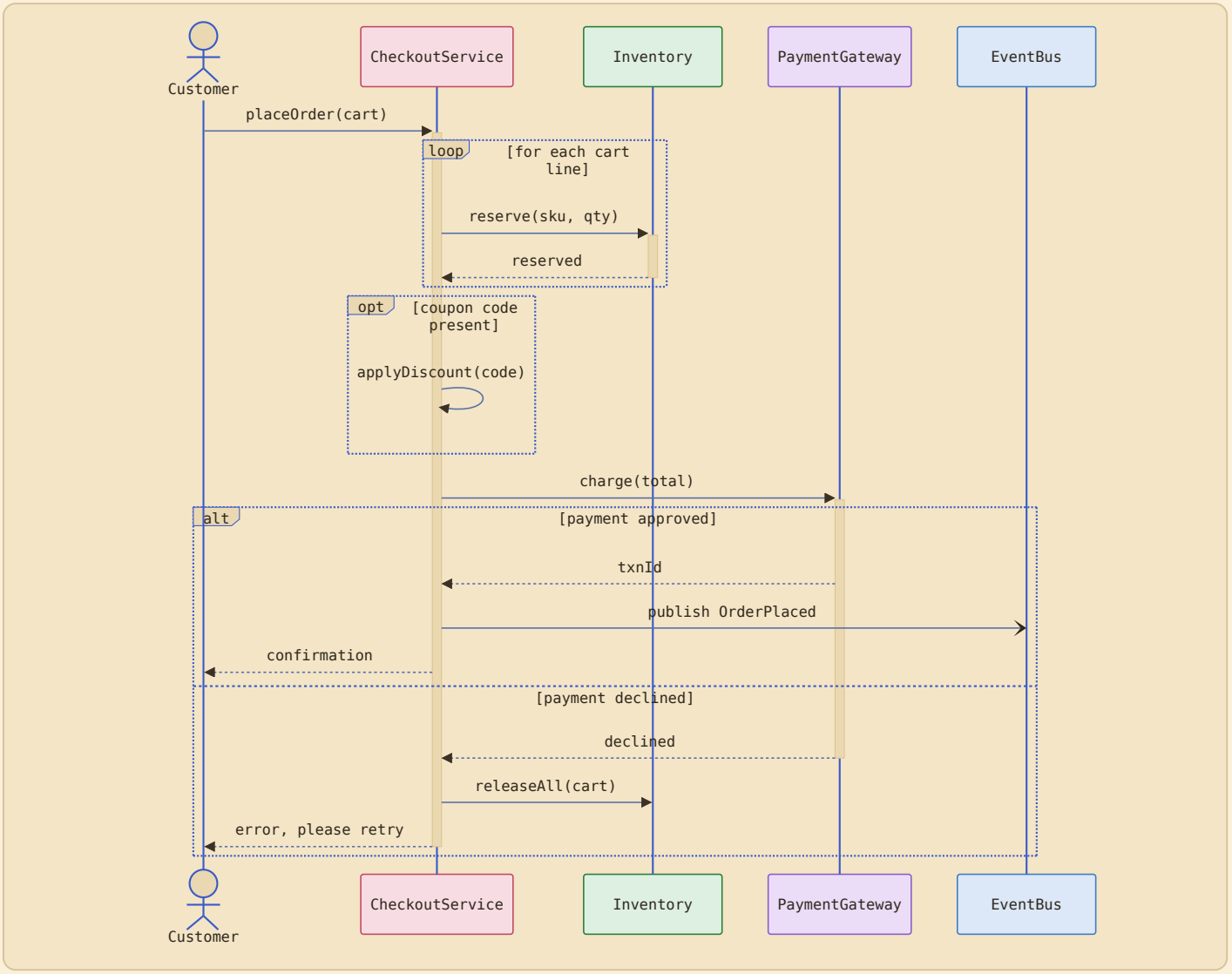
Fragments (guards in `[ ]`): `alt` if/else, `opt` if, `loop`, `par` parallel, `break` early exit, `critical` atomic, `ref` another diagram (others like `neg`, `strict` aren't needed). Whiteboards usually skip bars and fragments.

Example 1 — the library checkout, matching the Java code



The flow starts outside; the catalog **calls itself**; only two returns are drawn; the member talks **directly** to the copy, a choice the interviewer can question.

Example 2 — all fragments and async messages (checkout with payment)



`-)` is an **asynchronous** message (publishing doesn't wait). Point: reserve stock *before* charging; release it if the charge fails.

Java code for Example 1 (`lld.ch03.library`)

```
// File: code/lld/ch03/library/Member.java
    public LocalDate checkout(BookItem item, LocalDate today) {
        checkLimit();                                  // <<include>>: happens every time
        LocalDate due = item.checkout(this, today);
        onLoan.add(item);
        return due;
    }

    public BigDecimal returnBook(BookItem item, LocalDate today) {
        if (!onLoan.remove(item)) throw new IllegalStateException(name + " does not hold " + item.barcode());
        LocalDate due = item.checkIn();
        BigDecimal fine = new FineCalculator().fineFor(due, today);
        if (fine.signum() > 0) chargeFine(fine);       // <<extend>>: only when late
        return fine;
    }
```

Full runnable code: code/lld/ch03/library/ (see the Full Edition). `Catalog.search` calls its private `lookUpTitleIndex` (the self-call).

Output (excerpt): `Maya borrowed B-1, due 2026-03-15`, `Blocked: Maya already has 2 books (limit)`, `Returned B-3 3 days late, fine = 15.00`. **Every arrow is a real method; a message with no method means the class diagram is**

missing one.

Trade-offs

Good at call order, sync vs async, spotting missing methods; poor at branch-heavy flows, the whole design, and speed.

When not to use it · Common mistakes · Real-world examples

Avoid when: Three or more decisions → **activity diagram** (a second `alt`/`opt` is the warning sign). Unprompted in most OOD rounds, it steals class-diagram time. **Mistakes:** A return for every call; messages that aren't methods; mixed-up arrowheads; drawing classes when two objects of one class take part (`from : Account`, `to : Account`). **Seen in:** OAuth/OIDC login flows; payment "create order → redirect → webhook" (async webhook); sagas with compensating calls.

In the interview

> **30-second answer:** "One scenario, objects across, time down. Filled arrows sync, open async, dashed returns; `alt` / `opt` / `loop` for if/else, if, loops. Three or four objects, six to eight messages, and every arrow is a method on my class diagram."

- *Sequence vs activity?* — Sequence when *who calls whom* matters; activity when branching does.

Self-check

- **Q:** Thin rectangle on a lifeline? **A:** Activation bar. **Q:** "Retry up to 3 times"? **A:** `loop [attempt <= 3]`.

TL;DR

Objects across, time down; filled sync, open async, dashed return; keep it small.

3.13 Activity diagrams

What it is

Traces **one flow end to end** with its **branches** (and parallel steps): the flowchart of UML.

The problem it solves

Makes the flow's `if/unless/otherwise` logic checkable, and **tests the class diagram**: for every action, *which class does this?* No home = missing class.

How it works — the symbols

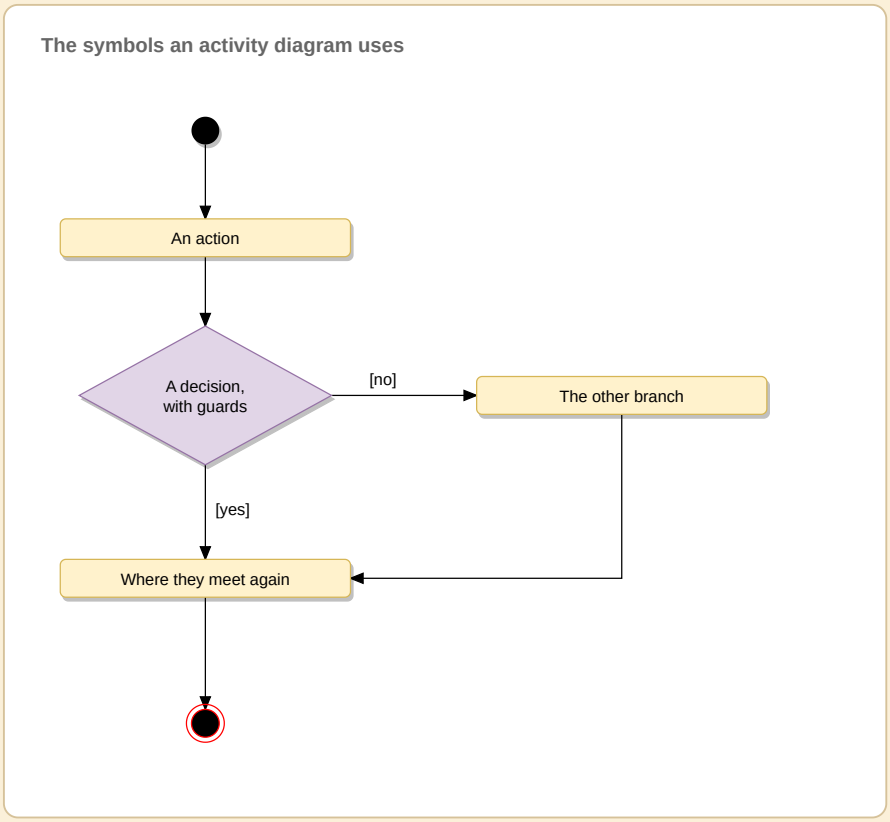


Figure — The core activity-diagram symbols: start, action, a decision with [yes]/[no] guards, the other branch, where the branches meet again, and the end (course diagram)

Initial node (filled circle, draw one) · **action** (rounded rectangle, one step) · **decision/merge** (diamond; **guards** must cover every case, use `[else]`) · **fork/join** (thick bar: start parallel actions / wait for **all**) · **activity final** (bullseye; whole activity stops) · **flow final** (circle with X; only this branch) · **swimlane** (who does each action) · object nodes and signals for data and events.

Steps: pick the flow with an "if/unless/otherwise"; actions in business language ("System calculates the fee", not `calculateFee()`); guard every outcome; fork/join only for true parallelism; swimlanes if helpful; end node(s).

Example 1 — online shopping (course)

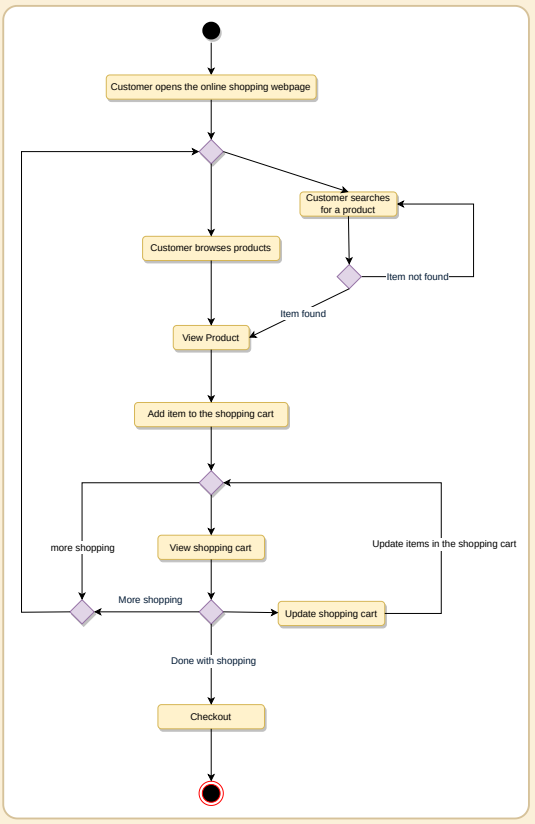
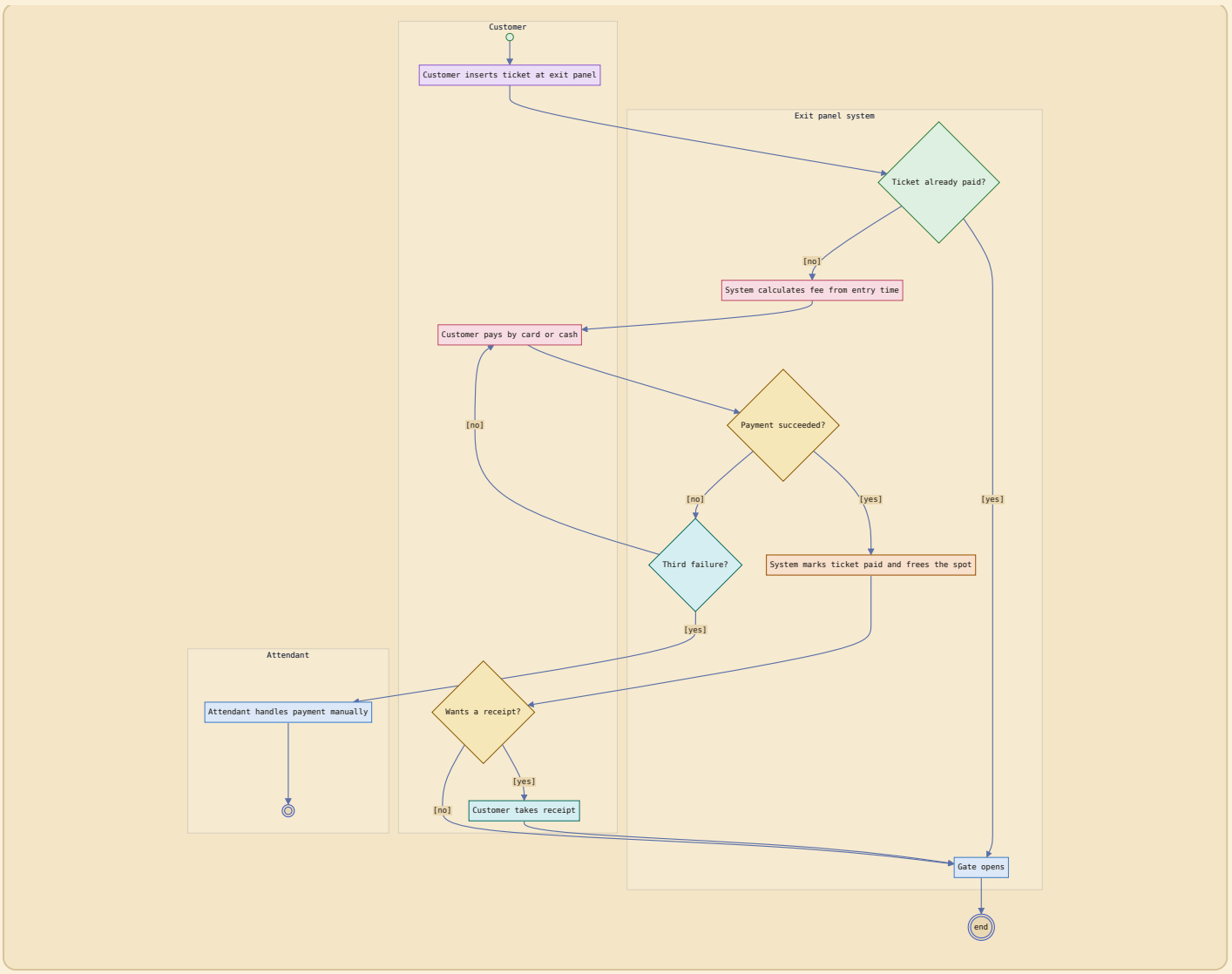


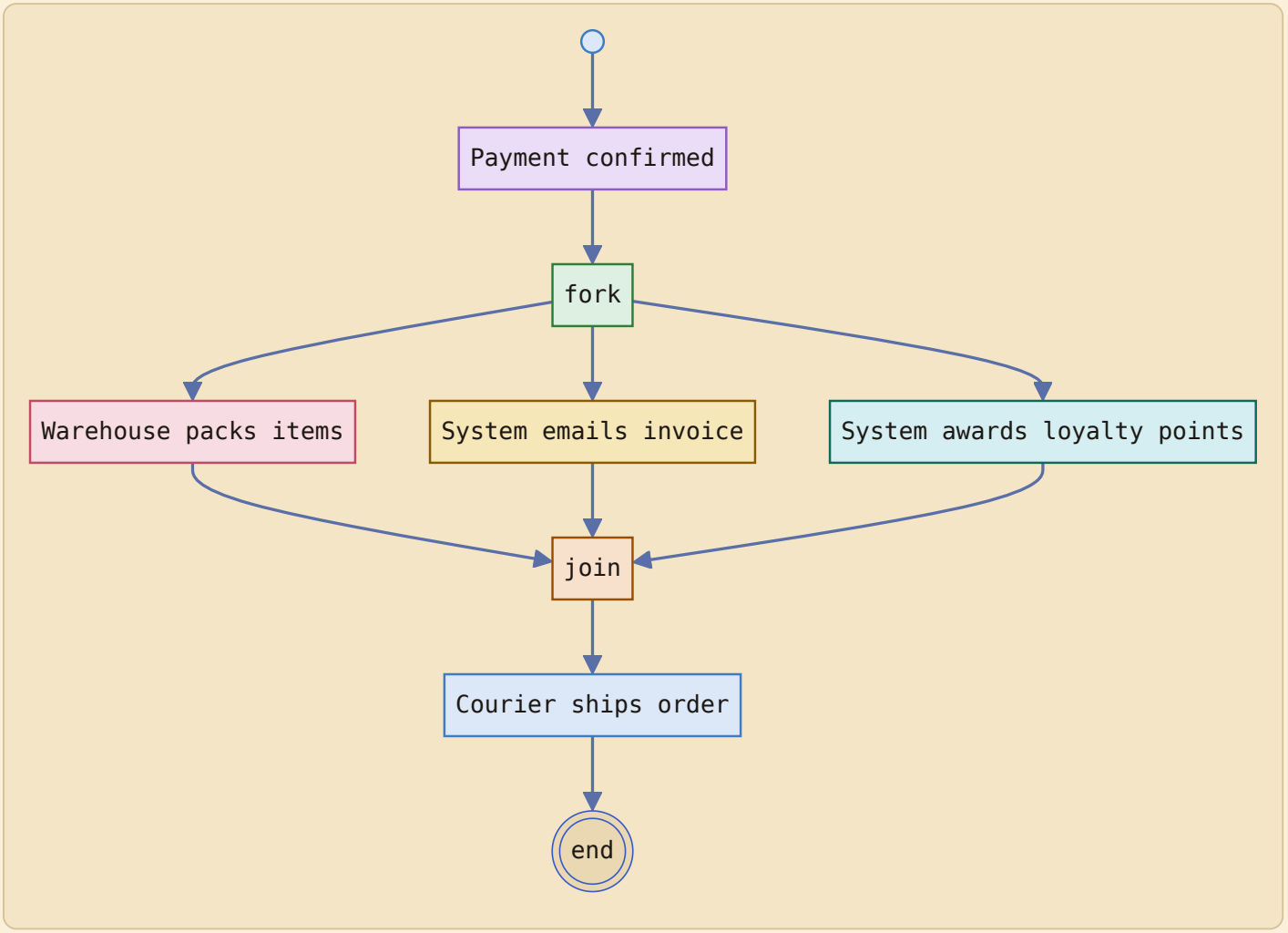
Figure — A customer shopping online: search, item found or not found, add to cart, view/update the cart, more shopping or checkout (course diagram)

Example 2 — pay at the parking exit (three decisions, with swimlanes)

Three requirement-driven decisions (ticket paid? payment succeeded? receipt?), a retry loop, and hand-off to an attendant after three failures (Parking Lot case study, Chapter 14):


Customer
Customer inserts ticket at exit panel
Exit panel system
Ticket already paid?
[no]
System calculates fee from entry time
Customer pays by card or cash
Payment succeeded?
[no]
[yes]
[yes]
[no]
Third failure?
System marks ticket paid and frees the spot
[yes]
Attendant
Attendant handles payment manually
Wants a receipt?
[yes]
[no]
Customer takes receipt
Gate opens
end

Example 3 — fork and join (order fulfilment)



After the **fork** bar the three actions run in parallel; the **join** waits for all three before shipping.

Activity vs sequence diagram

Activity shows a process as steps, handles branches naturally and answers "what happens?"; sequence shows objects messaging, handles branches awkwardly (nested `alt`) and answers "who calls whom?". Rule of thumb: **three branches** → activity; **four objects** with tricky call order → sequence.

Trade-offs · When not to use it · Common mistakes · Real-world examples

Readable by anyone, great at branches and parallelism; but no "who" without swimlanes, and big flows sprawl. Skip straight-line flows; use sequence for call order. **Mistakes:** A diamond with one exit; actions written as code; guards missing a case (`> 100` and `< 100`: what about 100?); fork/join for merely sequential steps. **Seen in:** BPMN (Camunda); CI/CD pipelines (parallel jobs = fork, `needs:` = join).

In the interview

30-second answer: "One flow: start, actions, decisions with complete guards, merges, fork/join for parallel steps, swimlanes, ends. I draw the branching flow and check every action has a class."

- *Activity final vs flow final?* — Whole activity vs one path.

Self-check

- **Q:** What must every arrow leaving a decision have? **A:** A guard; together covering all cases.

TL;DR

Start → actions → guarded decisions → merge → end; fork/join, swimlanes; use it to test the class diagram.

3.14 State machine diagrams

What it is

The **states one object** can be in and the **transitions** that events trigger.

- **State** (rounded rectangle), **initial pseudostate** (filled circle), **final state** (bullseye).
- **Transition** label `event [guard] / action`: trigger, boolean condition, effect.
- Internal `entry / exit / do` activities; **self-transition**; **choice** pseudostate (diamond); **composite state** with sub-states and **history** `(H)`; **orthogonal regions** active together.

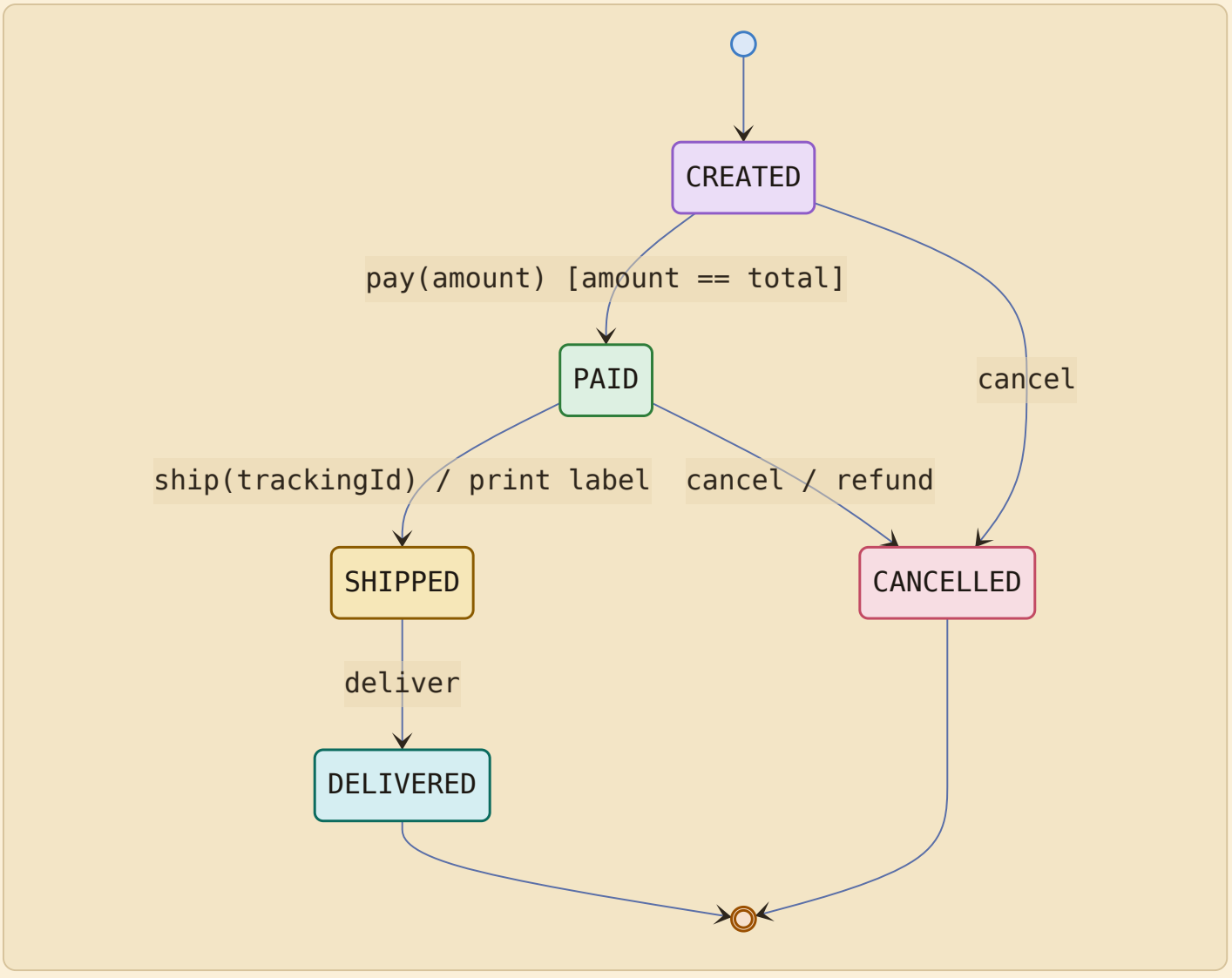
Analogy: a traffic light, always in one state, moving in a fixed order.

The problem it solves

Lifecycle bugs (shipping a cancelled order, paying twice). The diagram lists **every legal transition**, so everything else is rejected; it maps to an `enum` + transition method or the **State pattern** (Chapter 11).

How it works

Pick **one** class with a lifecycle → list states → per state: accepted events, guard, target, action → **everything else is illegal** → mark initial and final states.



For recognition, an elevator uses a **composite state** `Active` (sub-states `Moving` ⇄ `DoorsOpen`) entered from `Idle` on `request(floor) [floor != current]`, with `Active --> Maintenance : fault`.

Java example (`lld.ch03.statemachine`)

```
// File: code/lld/ch03/statemachine/Order.java
    public void handle(OrderEvent event) {
        OrderState from = state;
        OrderState to = switch (event) {
            case OrderEvent.Pay p when state == OrderState.CREATED && p.amount().compareTo(total) == 0 ->
OrderState.PAID;
            case OrderEvent.Pay p when state == OrderState.CREATED ->
                    reject(event, "guard failed: paid " + p.amount() + " but total is " + total);
            case OrderEvent.Ship s when state == OrderState.PAID -> {
                log.add("  action: print label " + s.trackingId());
                yield OrderState.SHIPPED;
            }
            case OrderEvent.Deliver d when state == OrderState.SHIPPED -> OrderState.DELIVERED;
            // ... two Cancel cases (from CREATED; from PAID with "refund" action)
            default -> reject(event, "no transition from " + state);
        };
        state = to;
        log.add(from + " --" + event.getClass().getSimpleName() + "--> " + to);
    }
```

Full runnable code: code/lld/ch03/statemachine/ (see the Full Edition).

`OrderEvent` is a **sealed** interface of records; each `case ... when` line is one diagram arrow, and `default` rejects the rest (e.g. `Ship rejected, no transition from CANCELLED`). The guard uses `compareTo` since `BigDecimal.equals` compares scale.

Trade-offs

enum + switch: all transitions in one place; best for few states with simple actions; a new state edits the switch. **State pattern (Ch. 11):** each state's behaviour in its own class; best when states behave very differently; a new state is a new class (Open/Closed).

When not to use it · Common mistakes · Real-world examples

Avoid when: A flag or two with no lifecycle; flows across many objects (activity diagram). A state machine is about **one** object. **Mistakes:** States named as actions ("Paying" vs PAID); missing or default-allowed transitions; overlapping guards; a state machine for the whole system. **Seen in:** `Thread.State`; TCP states; order tracking (cancel only before shipping); Spring Statemachine.

In the interview

> **30-second answer:** "For a lifecycle object I draw states with `event [guard] / action` transitions; anything not drawn is illegal and rejected. In Java: an enum with one transition method, or the State pattern when states carry lots of behaviour."

- *Guard?* — A boolean in `[ ]` that must hold, e.g. `[amount == total]`.
- *Thread-safe transitions?* — `synchronized` method, a lock, or CAS on an `AtomicReference<State>`.

Self-check

- **Q:** Read `PAID --> CANCELLED : cancel / refund`. **A:** In PAID, `cancel` moves to CANCELLED and refunds.

TL;DR

One object's states, `event [guard] / action` arrows; everything undrawn is illegal.

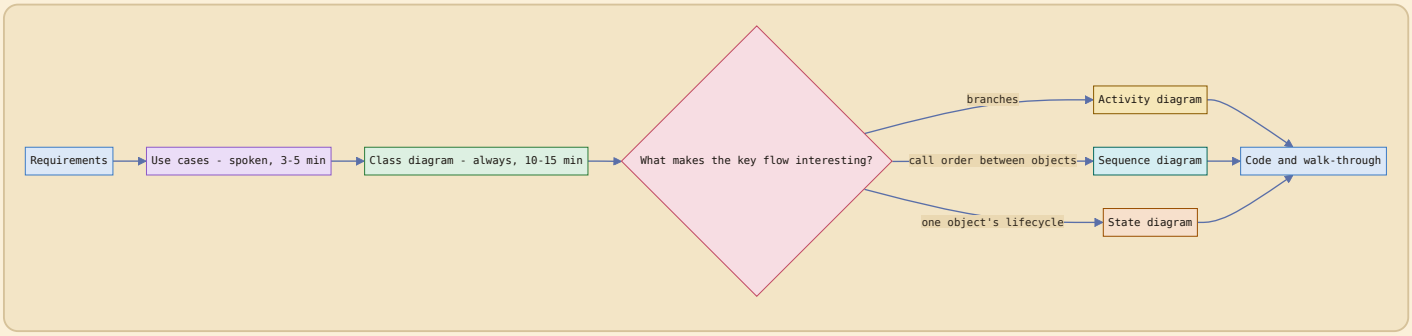
Part C — UML in the interview

3.15 How much UML to draw in an interview

Arrowheads aren't marked; what's checked is that **diagram and words agree**. A missing line costs less than an unjustifiable one.

Minute	Diagram	Depth
0–5	**Use case**	Usually spoken: actors, goals, out of scope
5–20	**Class diagram**	*Always*: 10–15 boxes, typed lines with multiplicity
20–25	**Activity**, **sequence** or **state**	One, for the key flow: branches / call order / lifecycle
25–45	Code + discussion	Walk the flow against the diagrams

Rules: one picture → the class diagram; drop attributes before relationships; label every line; say the test aloud; every activity action and sequence arrow must be a method; sequence diagrams 3–4 objects, 6–8 messages; machine coding → a 2-minute sketch; forgot a symbol → say what you mean.



In the interview

> **30-second answer:** "I list use cases verbally for scope, spend most diagram time on one class diagram with typed, multiplicity-labelled lines, then one behaviour diagram for the key flow to check every step has a class and method."

3.16 Chapter self-check

1. **Q:** Six relationships, weakest to strongest coupling? **A:** Dependency, association, aggregation, composition, realization, generalization.
2. **Q:** `CompactSpot` / `LargeSpot` subclasses passing a constant? **A:** One `ParkingSpot` with a `SpotType` enum.
3. **Q:** «include» vs «extend»? **A:** Always, base → included; sometimes, extension → base.
4. **Q:** Sequence arrowheads? **A:** Filled = sync, open = async, dashed = return.
5. **Q:** 14 UML diagrams? **A:** 7 structural (class, object, package, component, composite structure, deployment, profile), 7 behavioural (use case, activity, state machine, sequence, communication, interaction overview, timing).
6. **Q:** Only one diagram? **A:** The class diagram.

3.17 TL;DR

- **Relationships claim lifetime and substitution:** dependency (dashed arrow) · association (line/arrow) · aggregation (◇ at whole, parts live on) · composition (◆ at whole, parts die) · realization (dashed △) · generalization (solid △).
- **Choose in order:** substitution test → dies with whole? → outlives? → association; parameter-only → dependency.
- **Multiplicity changes code:** `1` required, `0..1` Optional, `1..*` non-empty, exact numbers checked.
- **UML:** 14 diagrams; interviews use use case, class (the answer), activity, sequence, state machine.

- **Draw little, label everything, say the test aloud,** and test the class diagram with behaviour diagrams.

Chapter 4 — SOLID Principles (Study Edition)

> **Study Edition.** Every topic of Chapter 4, without the padding. Code blocks are excerpts from the real files (`// ...` marks trimmed parts); full listings, outputs and longer discussion are in the Full Edition (`README.md`).
>
> **What you will learn.** The five SOLID principles, one at a time and then together, plus the ideas beneath them: **cohesion**, **coupling** and **separation of concerns**. Read the principles in order; 4.11 (one class through all five) is the best interview rehearsal.

Run the code (from `chapters/04-solid-principles/code/`):

```
javac --release 21 -d /tmp/lld_04 $(find . -name '*.java')
java -cp /tmp/lld_04 lld.ch04.srp.Main        # also: cohesion, coupling, soc, ocp, lsp, isp, dip, checkout
```

Packages: `srp` (Employee split by actor), `cohesion`, `coupling` (three steps), `soc` (rules vs storage), `ocp` (invoice formats), `lsp` (cars, contract rules, birds), `isp` (printers, over-splitting, roles), `dip` (notifications, DI, locator, plug-ins), `checkout` (one class through all five).

Money. All amounts are `BigDecimal` built from strings, and anything multiplied by a rate is rounded with `setScale(2, RoundingMode.HALF_EVEN)`. Never `double` (`0.1 + 0.2` prints `0.30000000000000004`). `BigDecimal.equals` compares scale (`2.0` ≠ `2.00`), so compare with `compareTo` or `signum()`.

4.1 What SOLID is and why you should learn it

What it is

Letter	Principle	The one-line rule
S	Single Responsibility (SRP)	A class should have one reason to change.
O	Open/Closed (OCP)	Add new behaviour by adding code, not by editing working code.
L	Liskov Substitution (LSP)	A subtype must be usable anywhere its parent type is expected.
I	Interface Segregation (ISP)	No client should be forced to depend on methods it does not use.
D	Dependency Inversion (DIP)	Depend on abstractions, not on concrete details.

Terms: a **module** is a unit bigger than a class (package, library, service); a **client** is code that uses other code; an **abstraction** is an interface or abstract class (*what*, not *how*); a **concrete class** is one you can `new`.

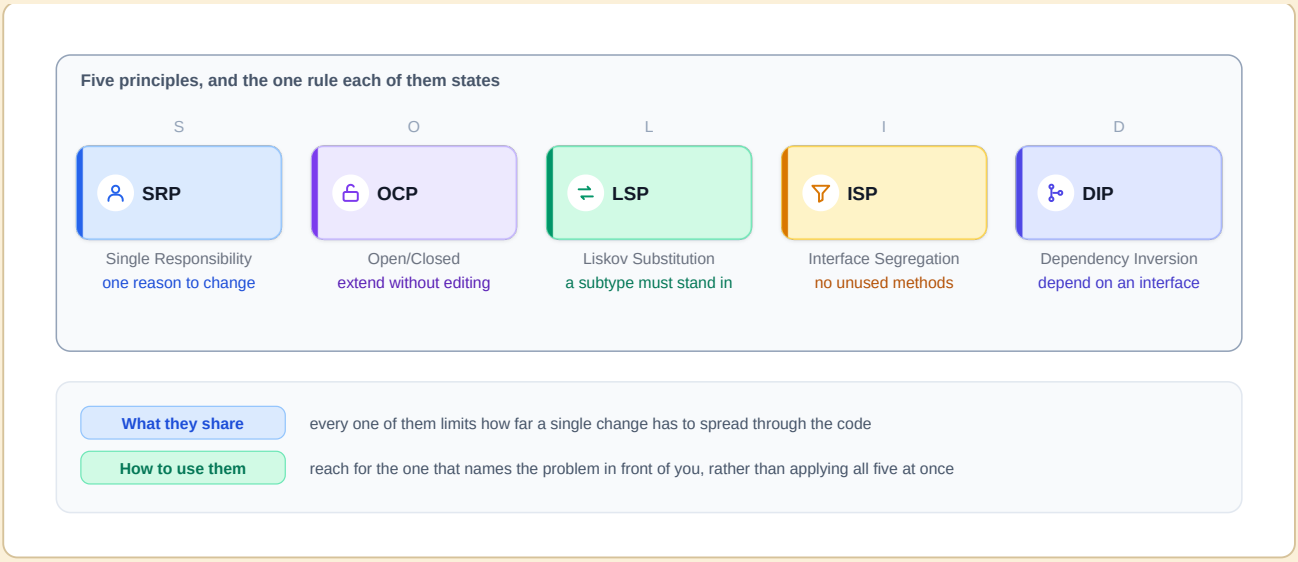


Figure — the five principles and the single rule each one states (course diagram)

The common thread: each principle **limits how far one change spreads**. Martin collected them around 2000; Feathers coined the acronym; OCP is Meyer's (1988), LSP is Liskov's (1987).

The problem it solves

Code keeps changing; without guidelines classes grow until one edit breaks three features. SOLID gives cleaner, more flexible, reusable code with fewer bugs — and **a shared vocabulary** ("this breaks ISP") so design debates are settled by argument, not taste.

How to use SOLID (the honest version)

Treat each principle as a **name for a problem**; when code hurts to change, find the name; apply the fix **when a real change asks for it**; always state the cost. Applied blindly, SOLID yields a maze of one-implementation interfaces.

TL;DR

Five guidelines that keep one change from spreading. Diagnose with them, apply on real change, state the cost.

4.2 Single Responsibility Principle (SRP)

What it is

A class should have only one reason to change — sharpened in *Clean Architecture* to **one, and only one, actor** (a person or group who can request a change). Analogy: a Swiss army knife has three jobs, so three reasons to change; a kitchen knife has one. Note: SRP is about classes/modules; "a function should do one thing" is a separate *Clean Code* rule.

The problem it solves

When one class serves several groups, every edit for one group ships inside code the others rely on: a payroll tweak can break an operations report.

What counts as one responsibility?

"One thing" has no yardstick; the **actor** test does: *who can send a change request for this code?*

Method	Who asks for the change	Example request
`calculatePay`	Finance	"Overtime starts after 38 hours, not 40."
`reportHours`	Operations	"Show the week-ending date."
`save`	Database team	"Move from SQL to a document store."

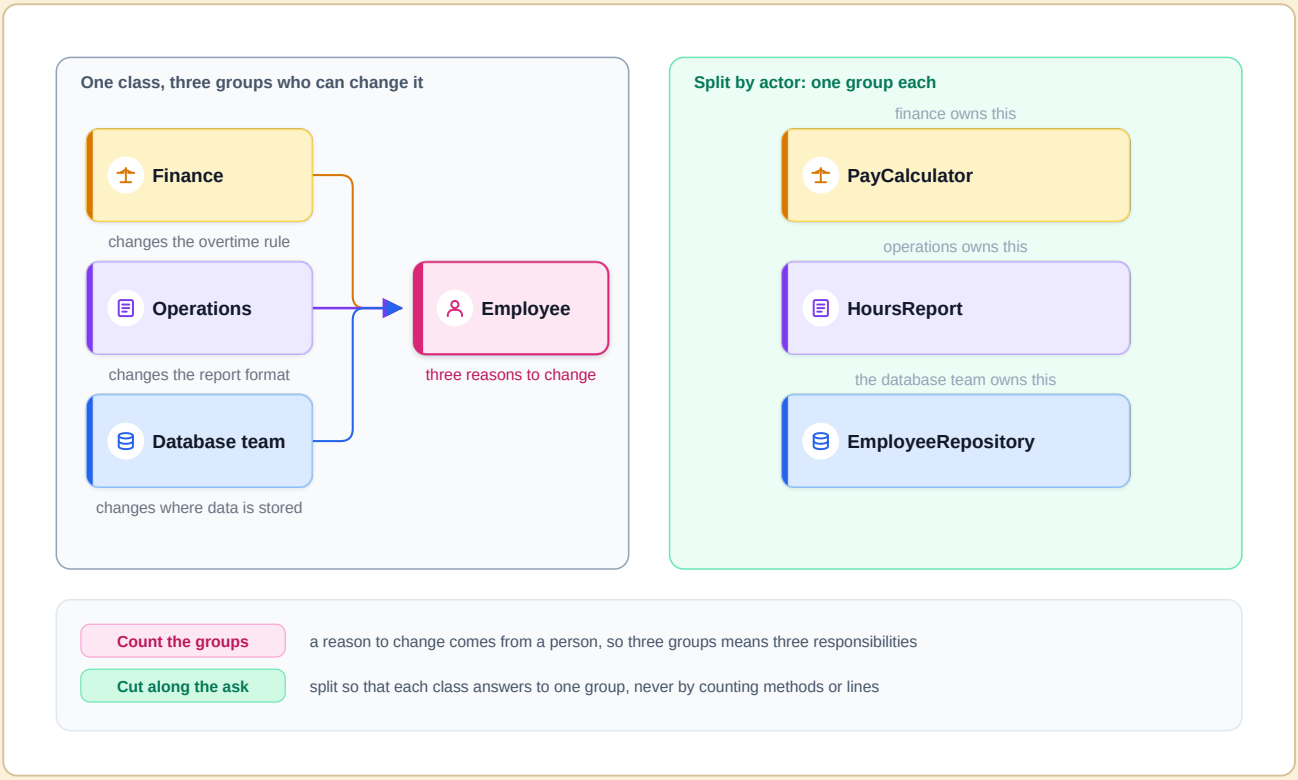


Figure — finance, operations and the database team all reach into one Employee class; splitting by actor gives each group its own class (course diagram)

Martin's trap: `calculatePay` and `reportHours` share a helper `regularHours()`; finance changes it and the operations report silently breaks. **Accidental duplication** — code that looks the same but changes for different reasons must not be merged.

How it works

List methods → name who would change each → group by actor → one class per group, named after its job → plain data in a `record` → run the three checks below.

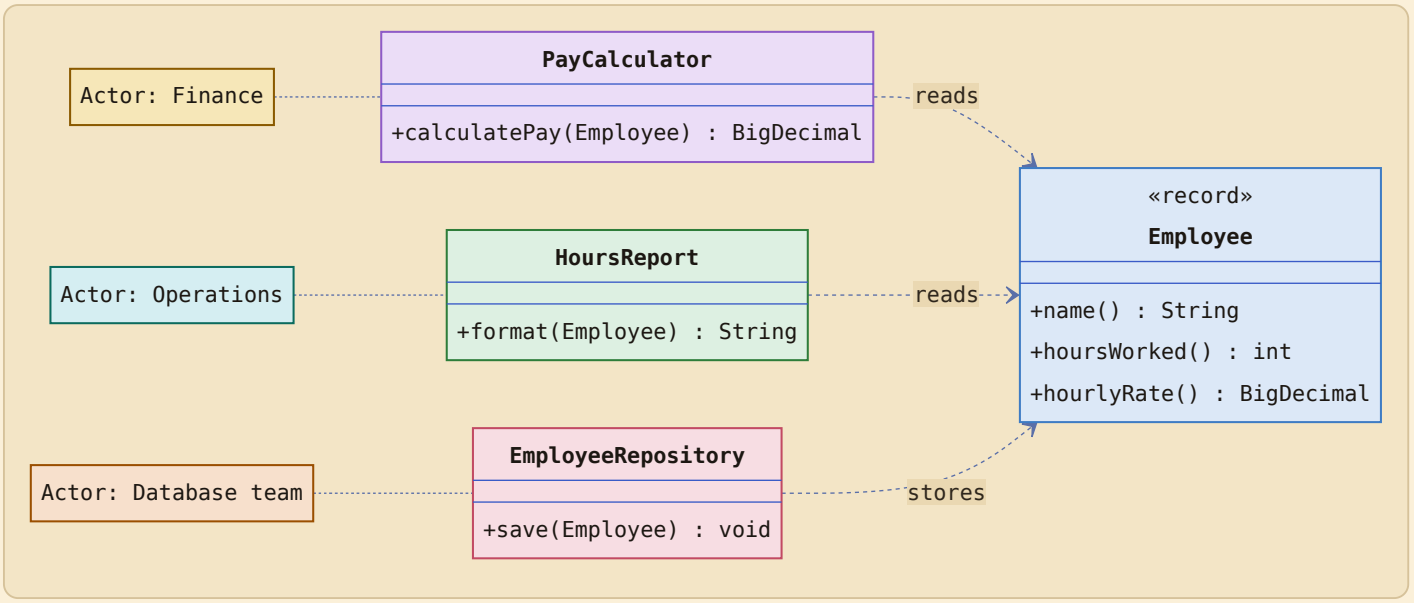
Java example: splitting `Employee` by actor

Before, `Employee` holds `calculatePay()`, `reportHours()` and `save()`. After:

```java
// File: code/lld/ch04/srp/after/Employee.java
public record Employee(String name, int hoursWorked, BigDecimal hourlyRate) { }

// File: code/lld/ch04/srp/after/PayCalculator.java
/** Only Finance asks for changes to this class. */
public class PayCalculator {
    private static final int OVERTIME_THRESHOLD = 40;
    private static final BigDecimal OVERTIME_MULTIPLIER = new BigDecimal("1.5");

    public BigDecimal calculatePay(Employee e) {
        int overtime = Math.max(0, e.hoursWorked() - OVERTIME_THRESHOLD);
        int normal = e.hoursWorked() - overtime;
        BigDecimal normalPay = e.hourlyRate().multiply(BigDecimal.valueOf(normal));
        BigDecimal overtimePay = e.hourlyRate().multiply(OVERTIME_MULTIPLIER)
                .multiply(BigDecimal.valueOf(overtime));
        return normalPay.add(overtimePay).setScale(2, RoundingMode.HALF_EVEN); // cents
    }
}
// HoursReport.format(Employee)        -- Operations
// EmployeeRepository.save(Employee)   -- Database team
```

Full runnable code: code/lld/ch04/srp/ (`before/`, `after/`) (see the Full Edition).

Output is identical before and after (`Pay: 950.00`); what changed is who must be consulted before an edit.



Where splitting goes wrong

Chopping by verb gives tiny classes always edited together — worse than no SRP. Three checks: (1) **Do the pieces change together?** (same commits = one responsibility); (2) **Does each name stand alone?** (`PayCalculator` yes, `EmployeeHelper` no); (3) **Could it be used elsewhere?** Also, **splitting doesn't automatically reduce dependencies** — deciding what each piece may know of the other is cohesion (4.3) and coupling (4.4).

Why SRP is important

Simpler maintenance, reuse, growth by adding classes, high cohesion — and a seam that *enables* low coupling (only an abstraction across it finishes the job).

Trade-offs

You gain	You pay
One group's change touches only its code	More classes and files
Small classes, easy tests, clear ownership	Wiring; risk of over-splitting

When not to use it

You can't name two actors; short-lived scripts; pieces that always change together.

Common mistakes

Splitting by method count or verbs; `Helper`/`Utils` classes; merging "duplicates" owned by different actors; thinking SRP means "one method"; confusing it with "a function does one thing".

Code smells that point to SRP (from *Refactoring*)

Divergent Change (SRP broken), **Large Class**, and **Shotgun Surgery** (often from *over*-splitting). Fix: **Extract Class**, **Move Function/Field**.

Real-world examples

Spring `@Controller`/`@Service`/`@Repository`; `DateTimeFormatter` vs `LocalDate`; Logback/Log4j 2 logger, appender and layout.

In the interview

> **30-second answer:** "One reason to change — really, one actor. I ask who would request changes to each method. If finance, operations and the DBAs can all edit `Employee`, I split it into `PayCalculator`, `HoursReport` and `EmployeeRepository` and keep `Employee` as data. It costs classes and wiring, so I only split when I can name the groups."

"Would you always split?" — No; if the halves always change together I've made two files for nothing.

Self-check

1. **Q:** What is an actor? **A:** A person or group who can request a change.
2. **Q:** The three checks for a good split? **A:** Change together? Name stands alone? Usable elsewhere?
3. **Q:** Does splitting always lower coupling? **A:** No — the dependency must point at an abstraction.

TL;DR

One class, one actor. Split along actors, keep data in a record, don't split what always changes together.

4.3 Cohesion

What it is

How closely the contents of one class belong together. **High**: every method serves one purpose. **Low**: unrelated jobs bundled.

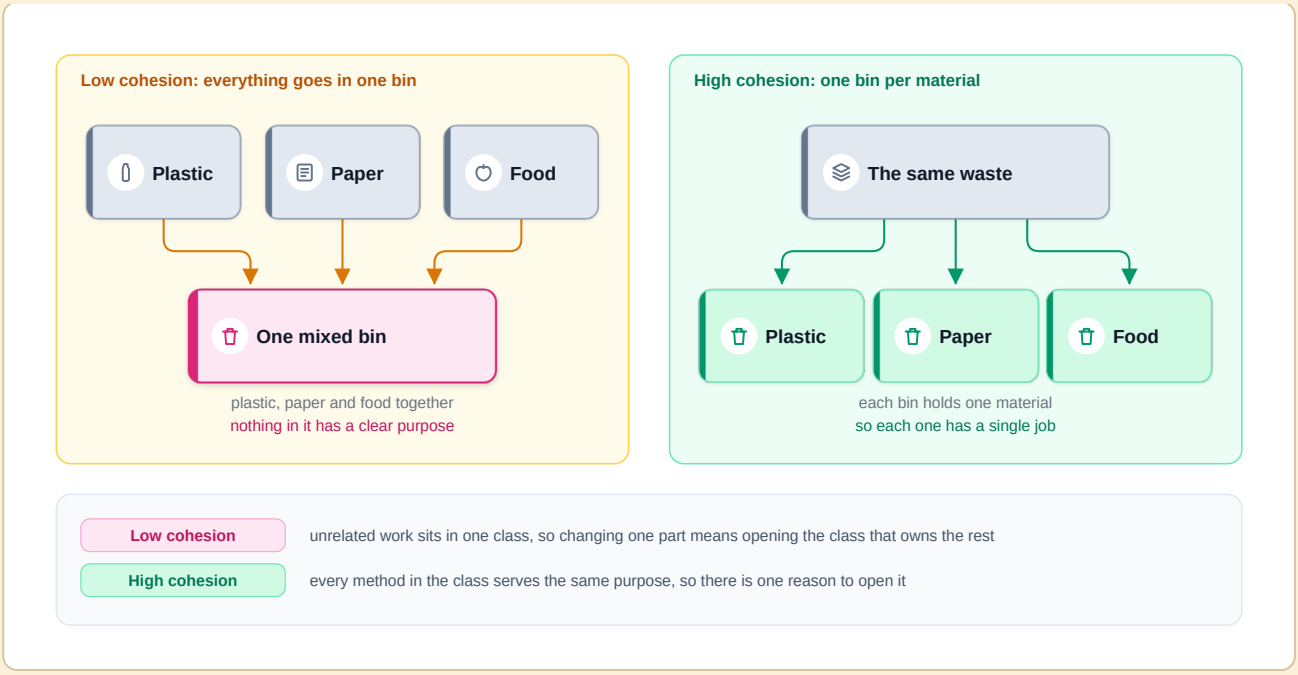


Figure — one mixed bin (low cohesion) next to one bin per material (high cohesion) (course diagram)

The problem it solves

Both directions hurt: unrelated jobs in one class (edit one, redeploy all), and one job **scattered** across classes (hunt for every piece).

The classic types of cohesion

Constantine, Yourdon, Myers (1970s), worst → best:

Type	What groups the code	Example
Coincidental	Nothing	`Utils` with `formatDate`, `sendEmail`, `parseCsv`
Logical	Category chosen by a flag	`handle(type)` with a big `switch`
Temporal	Run at the same time	`startup()` opens DB, warms caches, prints banner
Procedural	Steps in order	validate, log, retry
Communicational	Same data	`ReportBuilder` methods on one `Order` list
Sequential	Output feeds next step	`parse → validate → transform`
Functional (best)	One well-defined task	`PayCalculator`

Aim for functional; treat coincidental (the `Utils` class) as a warning.

How it works

Check which fields each method uses; disjoint groups for different purposes → split them; gather scattered pieces of one job.

Java example

`UserManager` mixed user bookkeeping (`users`) with audit-file work (`auditLines`). After the split:

```java
// File: code/lld/ch04/cohesion/after/AuditFile.java
/** HIGH cohesion: only audit-log work lives here. */
public class AuditFile {
    private final List<String> lines = new ArrayList<>();

    public void writeLine(String line) {
        // ...
    }

    public List<String> readAll() {
        // ...
    }
}
// after/UserManager keeps only users: addUser, deleteUser
```

Full runnable code: code/lld/ch04/cohesion/ (`before/`, `after/`) (see the Full Edition).

Same output; a new audit format now edits only `AuditFile`. Cost: the caller holds two objects.

Cohesion and SRP

High cohesion = one job = SRP. Aim for cohesion and SRP tends to follow.

Trade-offs

Gain: easy to name, read, test; changes land in one class. Cost: more classes to wire.

When not to use it

Don't split an already functionally cohesive class just to shrink it.

Common mistakes

Calling one mixed class a "coupling problem" (coupling needs two classes); `Utils` dumping grounds.

Real-world examples

`java.lang.Math` (functional); `java.util.Collections` (drifting toward logical).

In the interview

30-second answer: "Cohesion is how much the things inside one class belong together. Two method groups touching different fields for different reasons mean low cohesion, so I split. It's SRP seen from inside the class." A big class can still be cohesive — size is a hint, not the measure.

Self-check

1. **Q:** Worst kind? **A:** Coincidental.
2. **Q:** One job spread over three classes — cohesion problem? **A:** Yes.

TL;DR

One purpose per class; split disjoint groups; gather scattered pieces.

4.4 Coupling

What it is

How much one class depends on another — always a **pair**. **Loose**: change, replace or test one without the other. **Tight**: changing one forces changing the other. Analogy: a remote for one TV vs a universal remote using an agreed signal — the signal is the **interface**.

The problem it solves

Tight coupling blocks swapping (MySQL → Postgres), blocks unit tests without the real dependency, and spreads changes in how the dependency is constructed.

The classic types of coupling

Worst → best:

Type	What A knows about B	Example
Content	B's internals	Reflection setting a private field
Common	Shared mutable globals	Both use a `public static Map`
External	A shared external format	Both parse one hand-rolled file format
Control	A flag telling B *how*	`print(doc, boolean asPdf)`
Stamp	Whole structure for one field	`sendInvoice(Customer c)` reads `c.email()`
Data	Only needed values	`sendInvoice(String email, Money total)`
Message (lowest)	A message through an interface	`notifier.send(message)`

Goal: **depend on interfaces, pass only what's needed, never share mutable globals.**

How it works

Find where A **names** B (field type, `new B()`, `B.x()`); if A only needs *something that does the job*, describe the job as an interface, accept it in A's constructor, let the entry point choose.

Java example: three steps with `Student`

Start: `Student` holds data and talks to MySQL (a cohesion problem). **Step 1:** extract `MySqlDatabase` — cohesion fixed, coupling **tight** (`private final MySqlDatabase database = new MySqlDatabase(); // hard-wired`). **Step 2:** depend on an interface:

```java
// File: code/lld/ch04/coupling/decoupled/Student.java
/** Step 2: Student names no database at all -> LOOSE coupling. */
public class Student {
    private final String studentId;
    private final StudentRepository repository;

    public Student(String studentId, StudentRepository repository) {
        this.studentId = studentId;
        this.repository = Objects.requireNonNull(repository);
    }

    public void save() {
        repository.save(studentId);
    }
}
// StudentRepository { void save(String studentId); }
// implemented by MySqlStudentRepository and InMemoryStudentRepository (test double)
```

Full runnable code: code/lld/ch04/coupling/ (`start/`, `extracted/`, `decoupled/`) (see the Full Edition).

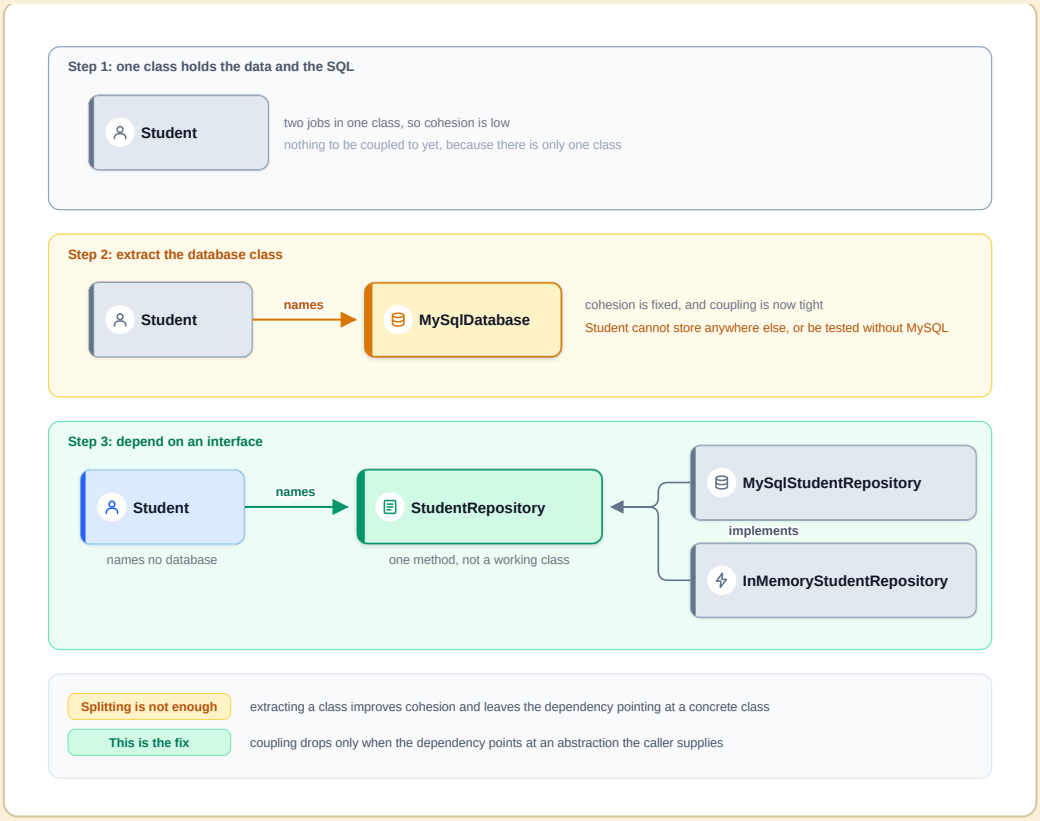


Figure — the three steps: one class with two jobs, an extracted but tightly coupled class, and a class that depends on an interface (course diagram)

Postgres is now a new class passed in; a test passes the in-memory one. This is DIP (4.10), the standard cure for tight coupling.

Coupling and SRP

SRP fixed cohesion but left `Student` welded to MySQL; pointing the dependency at an interface fixed coupling. SRP *usually* lowers coupling, never *guarantees* it.

Trade-offs

Loose coupling gains	Costs
Swap implementations; fakes in tests; no ripple	Extra interface and parameter; someone must wire it; one more hop to the real code

When not to use it

Not for **stable** classes (`String`, `LocalDate`, pure calculations); a one-implementation interface with no fake is just a file.

Common mistakes

"I extracted a class so coupling is lower"; hiding `new ConcreteThing()` in a constructor; stamp coupling.

Real-world examples

JDBC interfaces with vendor drivers; SLF4J; USB/HTTP/SQL as agreed signals.

In the interview

> **30-second answer:** "Coupling is how much one class depends on another. Where a class names a concrete collaborator but only needs the job done, I make the job an interface and pass the implementation in — swap MySQL for Postgres, test with a fake. Not for stable things like `String`." Can SRP increase coupling? Yes, if the extracted class is built around its parent's exact needs.

Self-check

1. **Q:** Can one class be tightly coupled? **A:** Not to itself; two jobs in one class is cohesion.
2. **Q:** Control coupling? **A:** A flag telling the other code how to behave.

TL;DR

Depend on an interface and pass the implementation in; extracting a class alone isn't enough.

4.5 Separation of Concerns (SoC)

What it is

Split a program so each part handles one **concern** (storage, validation, display). Dijkstra, 1974. Analogy: kitchen, waiters and cashier — a new menu doesn't change payment.

The problem it solves

Mixed concerns force edits in unrelated code. Separation gives modularity, maintainability, reusability, testability.

How it works

List concerns, one module each, small clear methods between them; **test: count the modules a likely change opens — ideally one.**

Java example: user registration

`UserStore` stores without judging; `UserService` decides and asks storage to save:

```java
// File: code/lld/ch04/soc/UserService.java
    public boolean registerUser(String username, String email) {
        if (!isValidUsername(username)) {
            System.out.println("Rejected: " + username + " is too short");
            return false;
        }
        if (!isValidEmail(email)) {
            System.out.println("Rejected: " + email + " is not an email address");
            return false;
        }
        store.addUser(username, email);
        return true;
    }
```

Full runnable code: code/lld/ch04/soc/ (see the Full Edition).

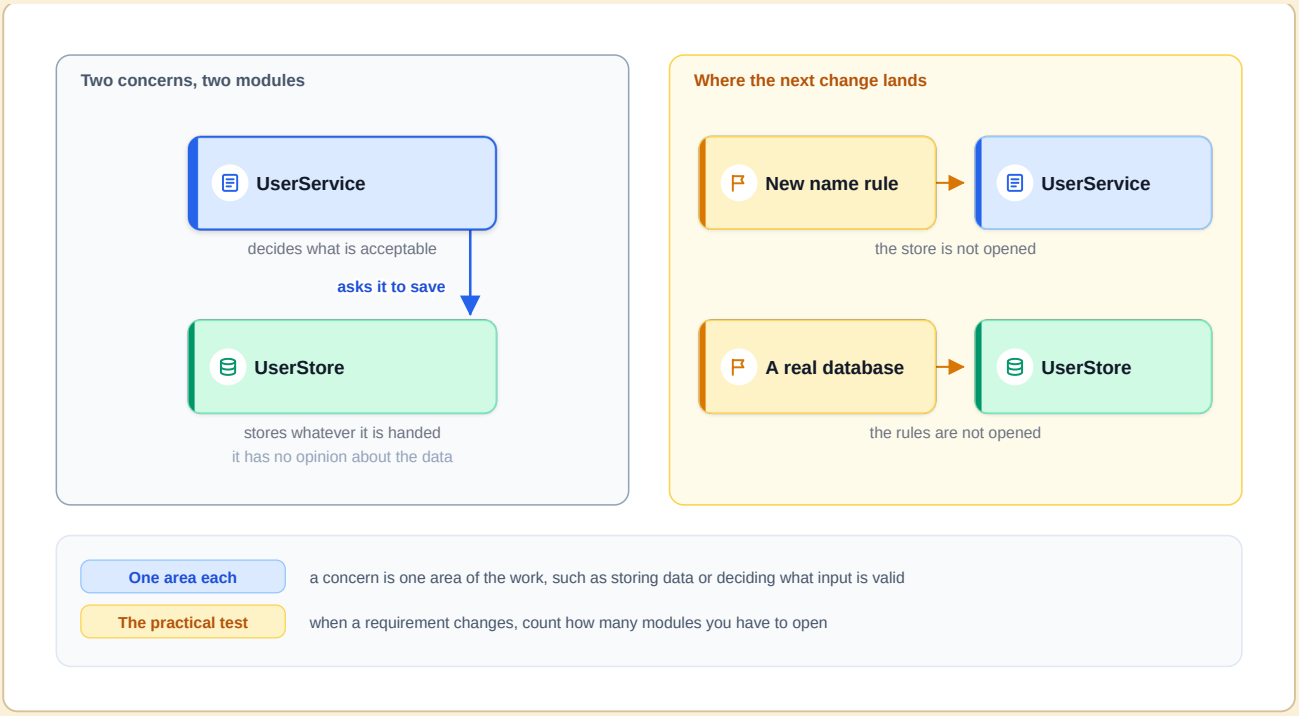


Figure — UserService decides what is acceptable and asks UserStore to save it; each change lands in one module (course diagram)

Change	Module you open
In-memory list → database	`UserStore` only
Usernames ≥ 8 characters	`UserService` only
Test rules without a DB	Give `UserService` a stand-in store

SoC and SRP

Same instinct at different scale: SRP for a class, SoC for modules and layers.

Trade-offs

Gain: local changes, isolated tests. Cost: more boundaries; cross-cutting concerns (logging, security, transactions) still span modules and need aspects or decorators.

When not to use it

A 50-line script. Separate concerns that change for different reasons or speeds.

Common mistakes

Validation hidden in the DAO; layers calling each other in every direction.

Real-world examples

MVC; HTML/CSS/JS; controller → service → repository; Spring `@Transactional` proxies.

In the interview

> **30-second answer:** "Each module handles one area of work. My test: how many modules does a requirement change open? Ideally one. It's SRP at module size."

Self-check

1. **Q:** SoC vs SRP? **A:** Mostly size — class vs modules/layers.

TL;DR

One module per area of work; measured by modules opened per change.

4.6 SRP vs cohesion vs coupling vs SoC — one table

Idea	Question it asks	Scope	Relation to SRP
SRP	How many actors can ask this class to change?	Class/module	—
Cohesion	Do the parts belong together?	Inside one unit	SRP usually raises it
Coupling	How much does A depend on B?	Between two units	SRP tends to lower it; needs abstractions
SoC	Is each area of work in its own module?	Modules, layers	SRP everywhere is one route to it

"High cohesion, low coupling": SRP gets the first half, DIP the second.

4.7 Open/Closed Principle (OCP)

What it is

Open for extension, closed for modification: add behaviour by adding code, not editing working code. **Meyer's OCP** (1988): a published class is closed and extended by **inheritance**. **Martin's polymorphic OCP** (1996): stable code depends on an **interface**; new behaviour arrives as new implementations — the version used today. Analogy: a console gains a racing wheel through a standard port without being opened.

The problem it solves

A growing `if/else` / `switch` on a type code: each case edits tested code, often in several copies. **Quick test:** *can I add the next case without opening this file?*

How it works

Wait for a real second case → identify the part that varies → make it an interface → one class per variant → the stable class **delegates** → new variants are new classes → choose the variant once, at the edge (entry point, factory, map, config).

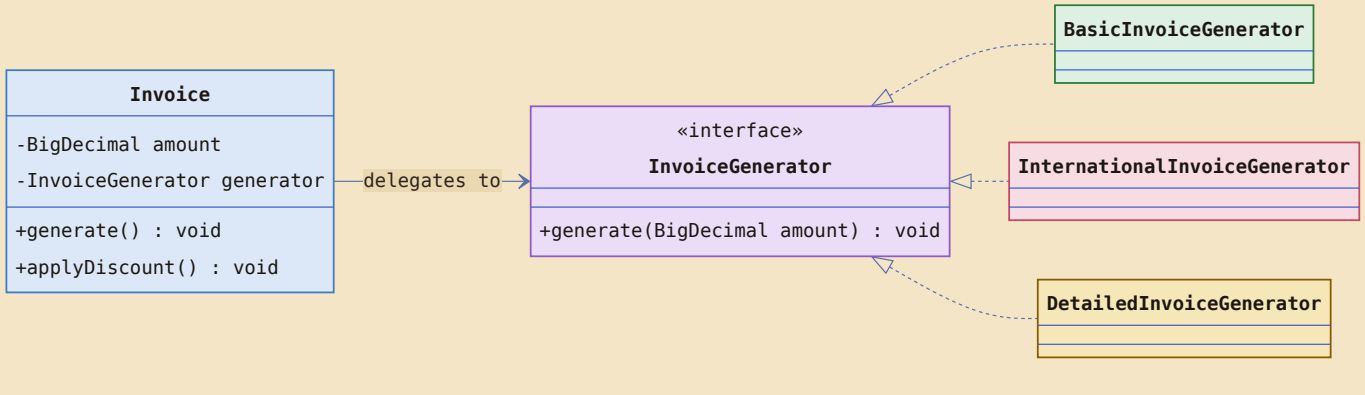
Java example: invoice formats

Before: `generate(String invoiceType)` branches on `"basic"` / `"international"` and throws for unknown types — each new type edits working code. After:

```java
// File: code/lld/ch04/ocp/after/InvoiceGenerator.java
@FunctionalInterface
public interface InvoiceGenerator {
    void generate(BigDecimal amount);
}

// File: code/lld/ch04/ocp/after/Invoice.java
public class Invoice {
    private final BigDecimal amount;
    private final InvoiceGenerator generator;
    // ...
    public void generate() {
        generator.generate(amount); // no branching
    }

    // Still closed? No -- a new discount rule would still edit this class.
    // That is fine until discounts actually start to vary.
    public void applyDiscount() {
        // ...
    }
}
// BasicInvoiceGenerator, InternationalInvoiceGenerator, and the ONLY new code
// for format three: DetailedInvoiceGenerator
```

Full runnable code: code/lld/ch04/ocp/ (`before/`, `after/`) (see the Full Edition).

The classes are named *generators* (not `BasicInvoice`, which would imply a subtype). As a `@FunctionalInterface`, a **lambda** is also an extension — OCP with zero new files.



This shape is the **Strategy pattern** — OCP for a varying algorithm.

Why not just subclass `Invoice`?

Subclassing is also OCP (Meyer's style) and fine when small. Delegation wins with **a second dimension** (3 formats × 4 currencies = 12 subclasses vs 3 + 4 = 7 classes) and **run-time choice** (pick or replace the generator once the country is known).

What is still not closed

`applyDiscount` still edits `Invoice` — fine. **A class is closed against a specific kind of change**, one you've seen happen. Martin: **strategic closure**.

Approaches to OCP

Approach	Pattern / Java feature	Good when
Interface + delegation	Strategy, `Comparator`	Algorithm varies, chosen at run time
Inheritance + overriding	Template Method	Fixed skeleton, a few steps vary
Wrapping	Decorator (`BufferedInputStream`)	Add features around behaviour
Listening	Observer	New reactions to events
Plug-ins / registry	map, `ServiceLoader`	Third parties add features
Functions	lambdas, `java.util.function`	Small one-method variations
Data / configuration	rules engines, feature flags	Variants are just numbers

The `switch` often survives — moved to **one** place at the edge (a factory or `Map<String, InvoiceGenerator>`).

Importance of OCP

Extend without touching the tested core; test only the new class; fewer regressions; adaptability.

Caution: do not over-engineer

Write the `if` first; add the interface at the second or third case (**rule of three**). Unused extension points are **Speculative Generality**.

How OCP relates to SRP

Changing parts end up in single-job classes; SRP usually comes first because the extension point stays hidden until the class has one job.

Trade-offs

You gain	You pay
New variants without editing; each tested alone; run-time choice	Interface + class per variant; flow spread over files; someone must choose the variant

When not to use it

Nothing has varied; variants are just numbers; the set is small and fixed — there Java 21 **sealed interfaces + pattern-matching** `switch` fit (compiler checks every case; easy to add operations, hard to add types).

Common mistakes

"Never edit a file" (bug fixes and refactoring still edit); plug-in systems for one implementation; moving one `switch` into five factories.

Refactorings and smells

Smell **Repeated Switches**; fixes **Replace Conditional with Polymorphism**, **Replace Type Code with Subclasses**.

Real-world examples

`list.sort(comparator)`; Java I/O decorators; servlet filters / `HandlerInterceptor`; JDBC drivers via `ServiceLoader`; IDE and browser plug-ins.

In the interview

> **30-second answer:** "I should add the next variation by adding code. When a `switch` over a type keeps growing, I put the varying behaviour behind an interface, one class per variant, and the stable class delegates — Strategy. Only once the variation is real, and closure is against a specific kind of change." Late requirement ("trucks pay a different rate"): "add a class" beats "add a branch".

Self-check

1. **Q:** Is `Invoice` closed against new discounts? **A:** No, only formats — fine until discounts vary.
2. **Q:** Which pattern is OCP for an algorithm? **A:** Strategy.
3. **Q:** Quick OCP test? **A:** "To add the next case, must I open this file?"

TL;DR

Varying part behind an interface, variants as classes, chosen at the edge — once variation is real.

4.8 Liskov Substitution Principle (LSP)

What it is

A subtype must be usable wherever its parent is expected without making the program incorrect (Liskov 1987; with Wing 1994, *behavioural subtyping*). "Substitutable" means **correct results**, not "it compiles". Analogy: promised "a car that takes petrol", handed an electric car.

The problem it solves

If a subclass secretly behaves differently, callers must check `instanceof` — polymorphism and OCP are lost.

Example 1: `Car` and `ElectricCar`

```
// File: code/lld/ch04/lsp/car/before/ElectricCar.java
/** VIOLATION: inherits a promise (refuel) it cannot keep. */
public class ElectricCar extends Car {
    @Override
    public void refuel() {
        throw new UnsupportedOperationException("Electric cars don't need refueling");
    }
    // ... plus its own recharge()
}
```

LSP is tested by code that **doesn't know** the subclass: `service(Car car)` calls `start`, `refuel`, `stop` and crashes.

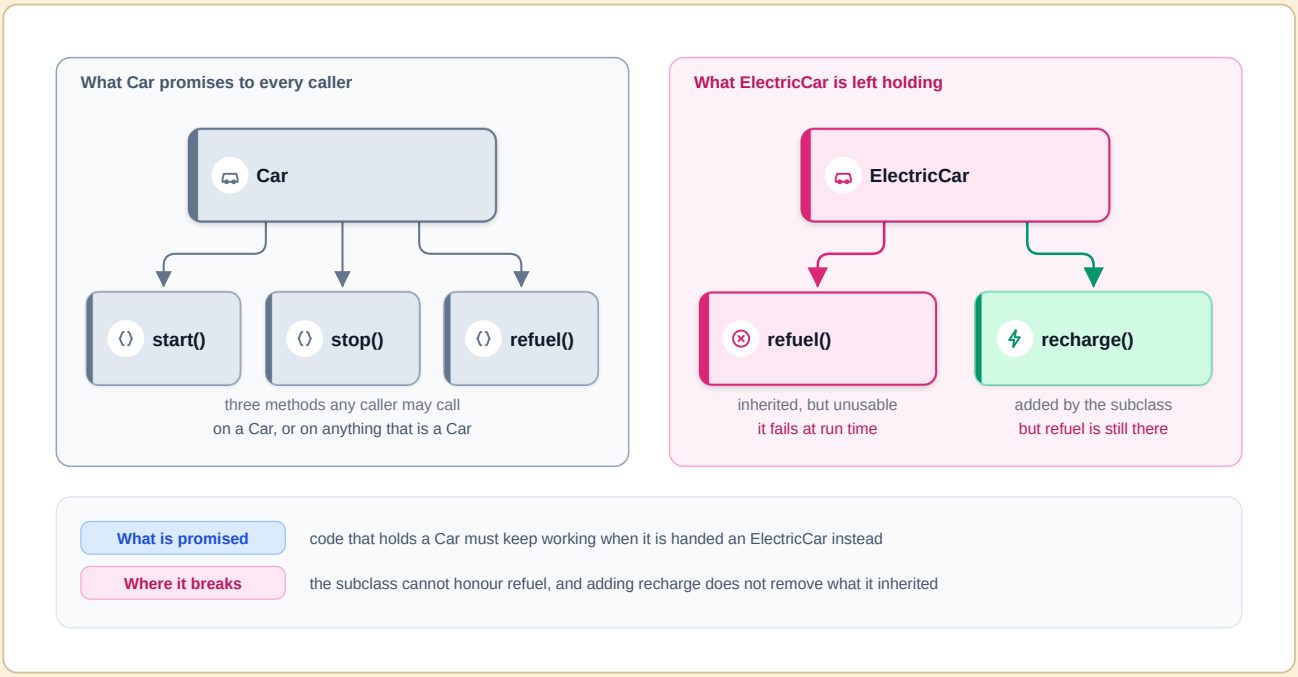


Figure — what Car promises every caller, and the unusable refuel() that ElectricCar is left holding (course diagram)

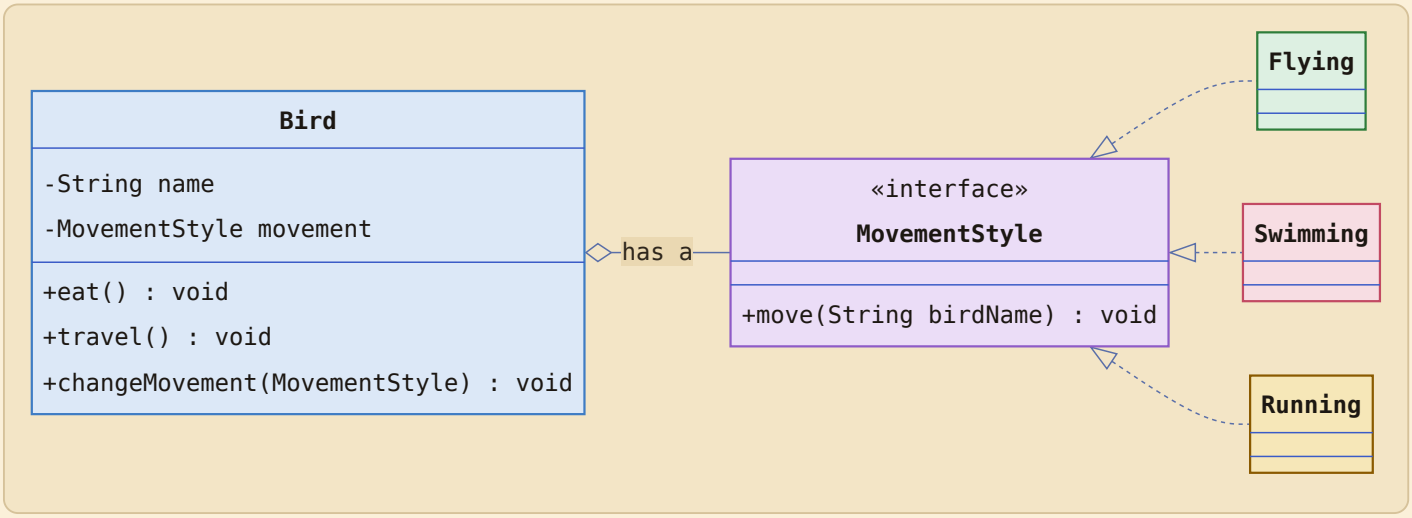
Root cause: `refuel()` names a **mechanism**, not the shared **goal**. **Raise a general parent** — `abstract class Vehicle { start(); stop(); abstract replenishEnergy(); }` — with `Car` and `ElectricCar` as **siblings**. Secretly charging inside `refuel()` would keep a lying name; adding only `recharge()` is invisible to `Vehicle` callers; and substitution is **child → parent, never between siblings**.

Break a hierarchy when a subclass can't implement a parent method, a base method names a mechanism, callers test the type, or subclasses share almost nothing. Fix: raise a parent, or move the varying behaviour out with composition.

Example 2: birds, and composition instead of inheritance

`fly()` on `Bird` forces `Penguin` to throw (**Refused Bequest**). Inheritance is **is-a** (gets every method); composition is **has-a**. Fix: no bird subclasses — each `Bird` gets a `MovementStyle`:

```java
// File: code/lld/ch04/lsp/birds/composition/Bird.java
/** A bird HAS-A movement style. No subclasses, so nothing is inherited and refused. */
public final class Bird {
    private final String name;
    private MovementStyle movement;
    // ...
    /** Tell, don't ask: callers say "travel", the bird decides how. */
    public void travel() {
        movement.move(name);
    }

    /** Composition can change behaviour at run time (e.g. an injured bird). */
    public void changeMovement(MovementStyle movement) {
        this.movement = Objects.requireNonNull(movement);
    }
}
// MovementStyle { void move(String birdName); } -> Flying, Swimming, Running
```



	Inheritance	Composition
Link	Penguin **is a** Bird	Bird **has a** MovementStyle
Change behaviour	add a subclass	supply another object
At run time?	no	yes
LSP risk	high	low

Inherit when the child truly keeps every promise; compose when behaviour varies or a subclass rejects something (throws, does nothing, "never call this on X").

Tell, Don't Ask

(Hunt & Thomas) **Tell an object what to do; don't inspect it and decide for it.** `if (bird instanceof Penguin) { swim(bird); } else { fly(bird); }` must know every subtype; `bird.travel()` works with any substitute. `instanceof` on a base type hints the base promises the wrong thing.

The rules a subclass must follow

The dangerous violations are **silent**. `Square extends Rectangle` couples the setters: set width 5, height 4, expect 20, get **16**. `Rectangle`'s unwritten promise was *width and height change independently*. **Base inheritance on the promises in code, not on real-world classification.**

A subclass may not **require more**, **give less**, or **break an assumption** (Meyer's Design by Contract: preconditions, postconditions, invariants):

1. **Preconditions may not be strengthened** — `SmallOrderDiscount` rejects amounts ≥ 100 that `Discount` accepts.
2. **Postconditions may not be weakened** — `OverdraftAccount.withdraw` lets the balance go negative where `Account` guaranteed ≥ 0.
3. **Invariants must be preserved** — `Square` (also describable as a weakened `setWidth` postcondition). Hardest: invariants are rarely written down.
4. **No new exceptions** — `ElectricCar.refuel()`. Java checks only **checked** exceptions. Loudest, least harmful.
5. **History constraint** (Liskov & Wing) — no state changes the parent forbids: `AdjustablePrice extends Price` adds `adjust()`, so a quote of 100 is charged 120. Why `String`/`Integer` are `final`.

```java
// File: code/lld/ch04/lsp/rules/Square.java
/** RULE 3 BROKEN: to stay square it couples the two setters -- a silent wrong answer. */
public class Square extends Rectangle {
    // ...
    @Override
    public void setWidth(int width) {
        this.width = width;
        this.height = width;
    }
    // ...
}
```

The fix: no shared mutable promise — immutable shapes that promise only `area()`:

```java
// File: code/lld/ch04/lsp/shapes/Shape.java
public sealed interface Shape permits Rect, Sq {
    int area();
}

// File: code/lld/ch04/lsp/shapes/Rect.java
public record Rect(int width, int height) implements Shape {
    public Rect withWidth(int w) {
        return new Rect(w, height);
    }
    // ...
}
```

Full runnable code: code/lld/ch04/lsp/ (`car/before/`, `car/after/`, `birds/inheritance/`, `birds/composition/`, `rules/`, `shapes/`) (see the Full Edition).

```
  OverdraftAccount: balance after payment: -30.00   <-- the caller never expected this
  Square:    expected area 20, got 16
  quoted 100.00, charged 120.00
```

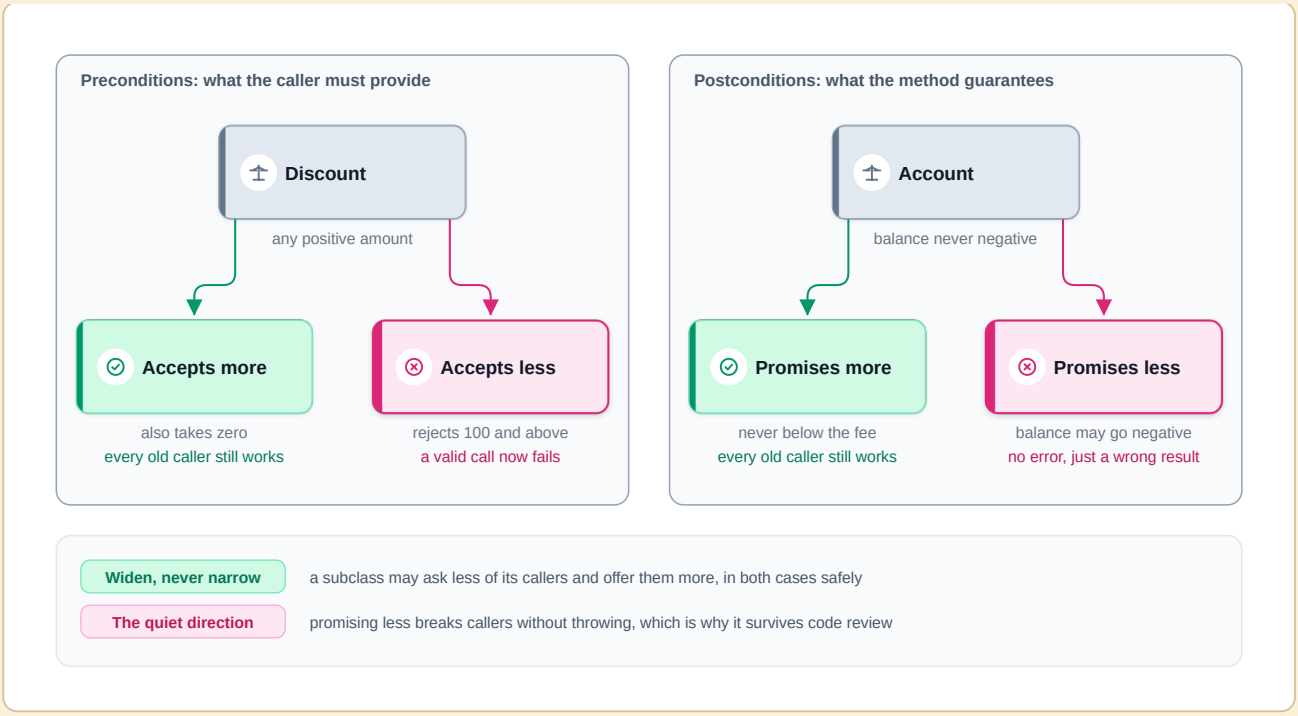


Figure — preconditions and postconditions: accepting more or promising more is safe; accepting less or promising less breaks callers (course diagram)

Rule	The subclass may not…	Usually shows up as
Preconditions	demand more	valid input rejected
Postconditions	guarantee less	a returned value breaks an assumption
Invariants	break a rule that always held	a silently wrong result
Exceptions	throw something new	crash with no matching `catch`
History	allow changes the parent forbids	a "fixed" value changes between reads

Signature rules (Java checks these): covariant returns allowed; no new/broader checked exceptions; no reduced visibility; parameter types must match exactly (else it's an **overload** — use `@Override`). Java checks **none** of the behavioural rules.

How to check a subclass

1. List the parent's **promises** (not signatures) first. 2. **Write one test against the parent type and run it for every subclass** (`resizeAndCheck(Rectangle)` fails for `Square`) — an abstract contract test in JUnit.

Why LSP is important

No fragile surprises; polymorphism without `instanceof` (what makes OCP work); correct results; safe reuse.

Trade-offs

Gain: callers trust the base type; safe new subclasses; run-time flexibility via composition. Cost: contracts to think through and document; "natural" hierarchies abandoned; more small objects.

When not to use it

LSP is never optional with polymorphism; what you can skip is inheritance — if a subclass can't keep the promises, don't make it one.

Common mistakes

Modelling the real world; "fixing" by silently doing nothing; thinking LSP is only about exceptions; expecting siblings to substitute.

Refactorings and smells

Refused Bequest → Replace Subclass/Superclass with Delegate, Extract Superclass, Push Down Method.

Real-world examples

`java.sql.Date` (`getHours()` throws); `java.sql.Timestamp` (asymmetric `equals`, "implementation inheritance"); `Stack extends Vector` (use `Deque`); `List.of(...).add` throws (an "optional operation").

In the interview

> **30-second answer:** "A subclass must work anywhere its parent is expected with correct results. It can't demand more, promise less, break invariants, throw new exceptions or allow forbidden state changes. Mutable Square/Rectangle is the classic silent break. I test with one base-type test for every subclass; when a subclass must refuse a method, I raise a better parent or compose."

"Is `ElectricCar` a substitute for `Car` after the fix?" — No, nor should it be; both substitute for `Vehicle`.

Self-check

1. **Q:** Subclass accepts fewer inputs — which rule? **A:** Preconditions strengthened.
2. **Q:** Why is Square worse than `refuel()` throwing? **A:** Silent wrong answer vs loud failure.
3. **Q:** History constraint? **A:** No state changes the parent forbids.

TL;DR

Accept at least as much, deliver at least as much, keep invariants and history, throw nothing new. Inherit by promises; otherwise raise a parent or compose.

4.9 Interface Segregation Principle (ISP)

What it is

Clients should not be forced to depend on methods they do not use. Origin: Martin at Xerox, where one `Job` class used by every task meant any change rebuilt all tasks. A **fat interface** grew feature by feature; a **role interface** describes one kind of caller's needs. Analogy: the 40-page menu for someone who wants coffee.

The problem it solves

Implementers must throw or no-op (also an LSP break); callers depend on methods they never call; test fakes grow long.

Example: a bloated `Printer` interface

```java
// File: code/lld/ch04/isp/fat/BasicPrinter.java
/** VIOLATION: forced to "implement" three methods it can only refuse. */
public class BasicPrinter implements Printer {
    // ...
    @Override
    public void scanDocument(String doc) {
        throw new UnsupportedOperationException("BasicPrinter cannot scan");
    }
    // ... faxDocument and stapleDocument throw too
}
```

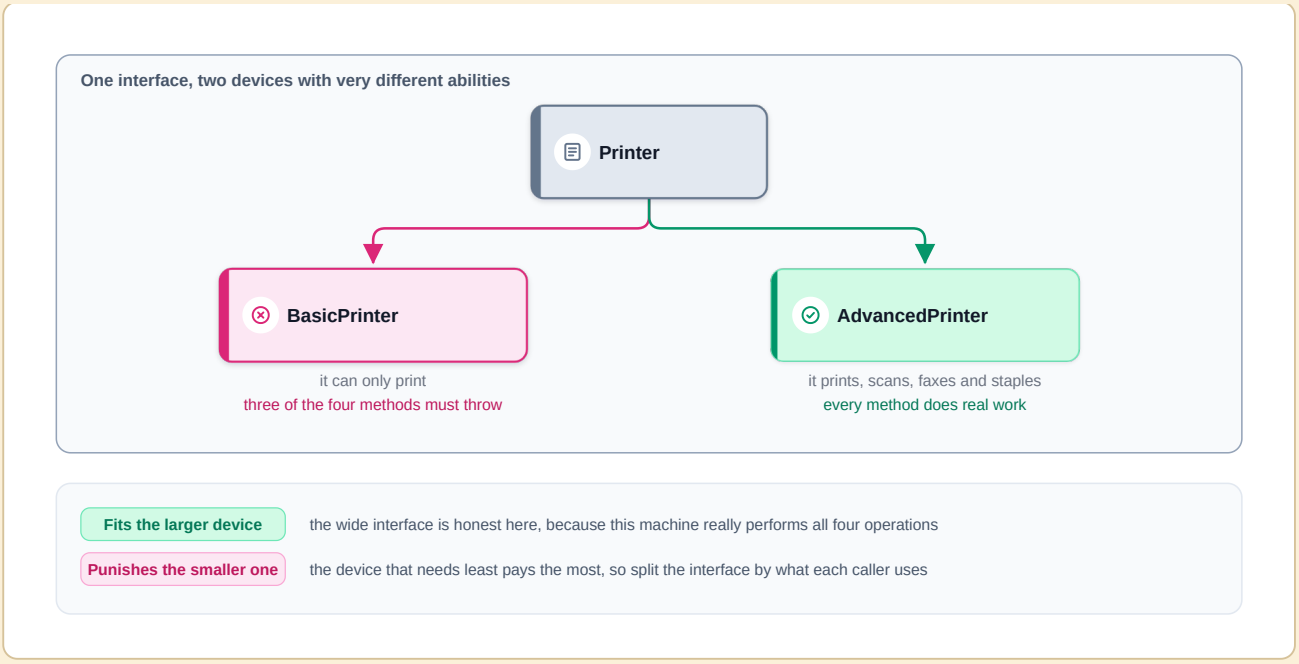


Figure — one interface, two devices: it fits the larger device and punishes the smaller one (course diagram)

Restructuring: first attempt, one interface per capability

`Printer`, `Scanner`, `Fax`, `Stapler`; `BasicPrinter` implements one, `AdvancedPrinter` all four. Valid ISP — but **one interface per method is not the goal**.

How small should an interface be?

Group by caller, not by method. Right size when both are "no": *is any implementer forced to supply what it can't honour?* and *does any caller depend on methods it never calls?*

Over-splitting cost: `OfficeWorkflow` (scan, print, staple) takes three parameters and gets **the same device three times** — `new OfficeWorkflow(adv, adv, adv)`. Grouping by role instead:

```
// File: code/lld/ch04/isp/roles/DocumentProcessor.java
/** Role interface for callers that scan, print AND staple (e.g. OfficeWorkflow). */
public interface DocumentProcessor extends Printer {
    void scanDocument(String doc);

    void stapleDocument(String doc);
}
// OfficeWorkflow(DocumentProcessor) -- one dependency
// PrintQueue(Printer)               -- a BasicPrinter is enough
```

Full runnable code: code/lld/ch04/isp/ (`fat/`, `split/`, `roles/`) (see the Full Edition).

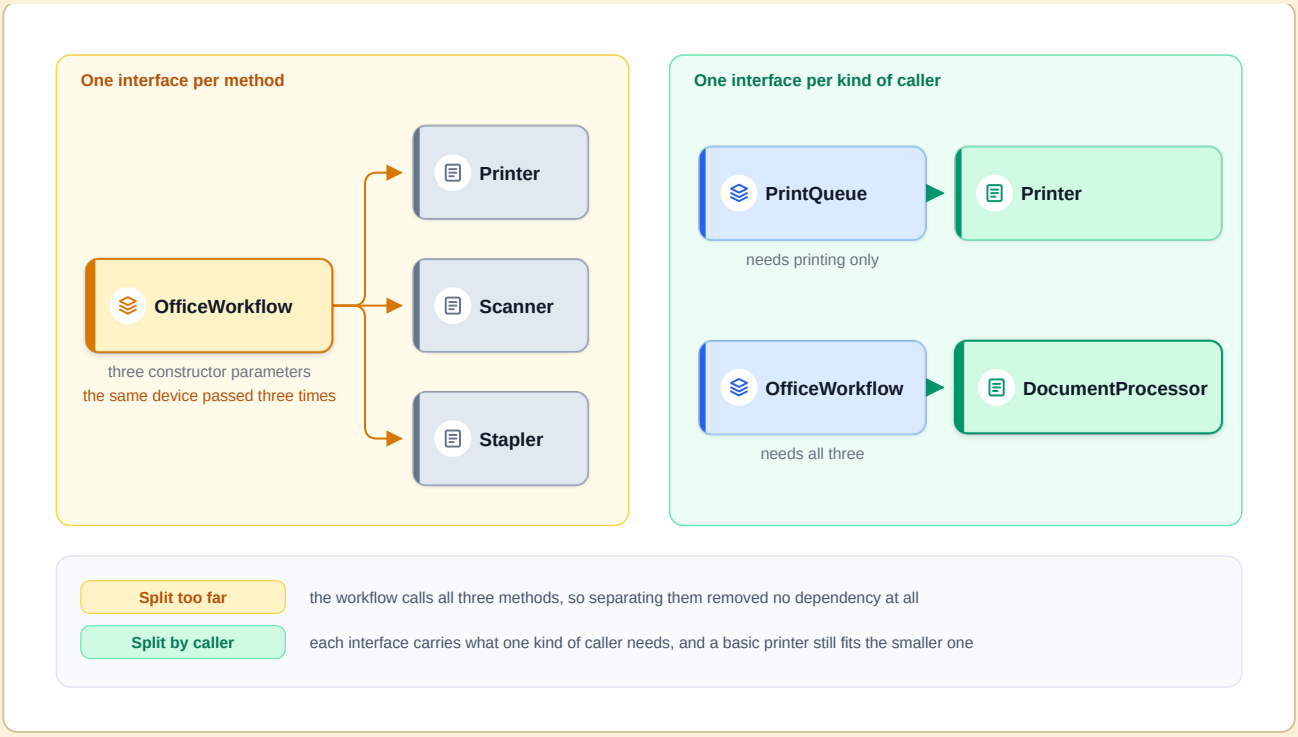


Figure — split too far (three interfaces, the same device passed three times) vs split by caller (one role per kind of caller) (course diagram)

	Four one-method interfaces	Two role interfaces
Forced/unused methods?	No	No
Dependencies for `OfficeWorkflow`	3	1
Names to learn	4	2

Rule of thumb: split only when you can **point to the caller** needing the smaller piece; merge interfaces always implemented and used together. In Python/JavaScript the interface is **the set of methods a caller actually calls** — the same needless dependency exists, just unchecked (`typing.Protocol` / `abc` can document it).

Five signals of an ISP violation

Signal	When it shows up	Example
Low cohesion in the interface	Reading it	`MediaDevice { playAudio; playVideo; recordVideo }` for a speaker
Fat interface, mechanism names	Reading it	`processCreditCard/PayPal/BankTransfer` → `PaymentMethod.pay(amount)`
Empty / throwing methods	Reading an implementer	`Vehicle { drive; fly; sail }` for a car
One interface, many roles	Comparing implementers	`Employee { calculateSalary; handleCustomerQueries; manageTeam }`
Long test stand-ins	Writing a test	dozens of empty bodies for one call

How it works (step by step)

List each caller's used methods → identical lists form a **role** → one interface per role (may extend each other) → classes implement every role they can honestly fill → each caller takes the narrowest role.

Importance of ISP

No bloated implementers, no ripple across roles, simpler tests, clearer design.

How ISP relates to the other principles

SRP for interfaces; OCP via new small interfaces; fat interfaces **force** LSP violations; DIP should depend on the smallest role.

Trade-offs

Gain: no throwing methods, insulated callers, tiny fakes (lambdas for one-method roles). Cost: more names; over-splitting adds wiring; harder to see a device's full ability in one place.

When not to use it

Every implementer supports everything and every caller uses it all; or you can't name the caller that wants less.

Common mistakes

One interface per method everywhere (most common after learning ISP); splitting by implementer features; fat interface + empty adapter class (AWT `MouseAdapter`).

Real-world examples

`Runnable`, `Comparable`, `AutoCloseable`, `Iterable`, `java.util.function`; `MouseListener`/`MouseAdapter`; Kotlin `List` vs `MutableList`; Spring Data `Repository`/`CrudRepository`/`PagingAndSortingRepository`.

In the interview

> **30-second answer:** "No caller should depend on methods it doesn't use. Symptoms: throwing implementations, fakes full of empty methods. I split by caller — one role per kind of caller, not per method — and only when I can name that caller." *SRP vs ISP?* SRP protects the class, ISP protects the caller.

Self-check

1. **Q:** The two sizing questions? **A:** Forced-to-supply? Depends-on-unused?
2. **Q:** Why didn't four interfaces help `OfficeWorkflow`? **A:** It uses all three methods; no dependency was removed.

TL;DR

One interface per kind of caller. Throwing methods = too wide; same object passed thrice = too thin.

4.10 Dependency Inversion Principle (DIP)

What it is

1. **High-level modules should not depend on low-level modules; both depend on abstractions.**
2. **Abstractions should not depend on details; details depend on abstractions.**

High level = policy (*what*); low level = detail (*how*: SMTP, MySQL). **Inversion:** the call still goes policy → detail, but the **source-code dependency is reversed** — the detail implements an interface owned by the policy. Analogy: a lamp and a wall socket.

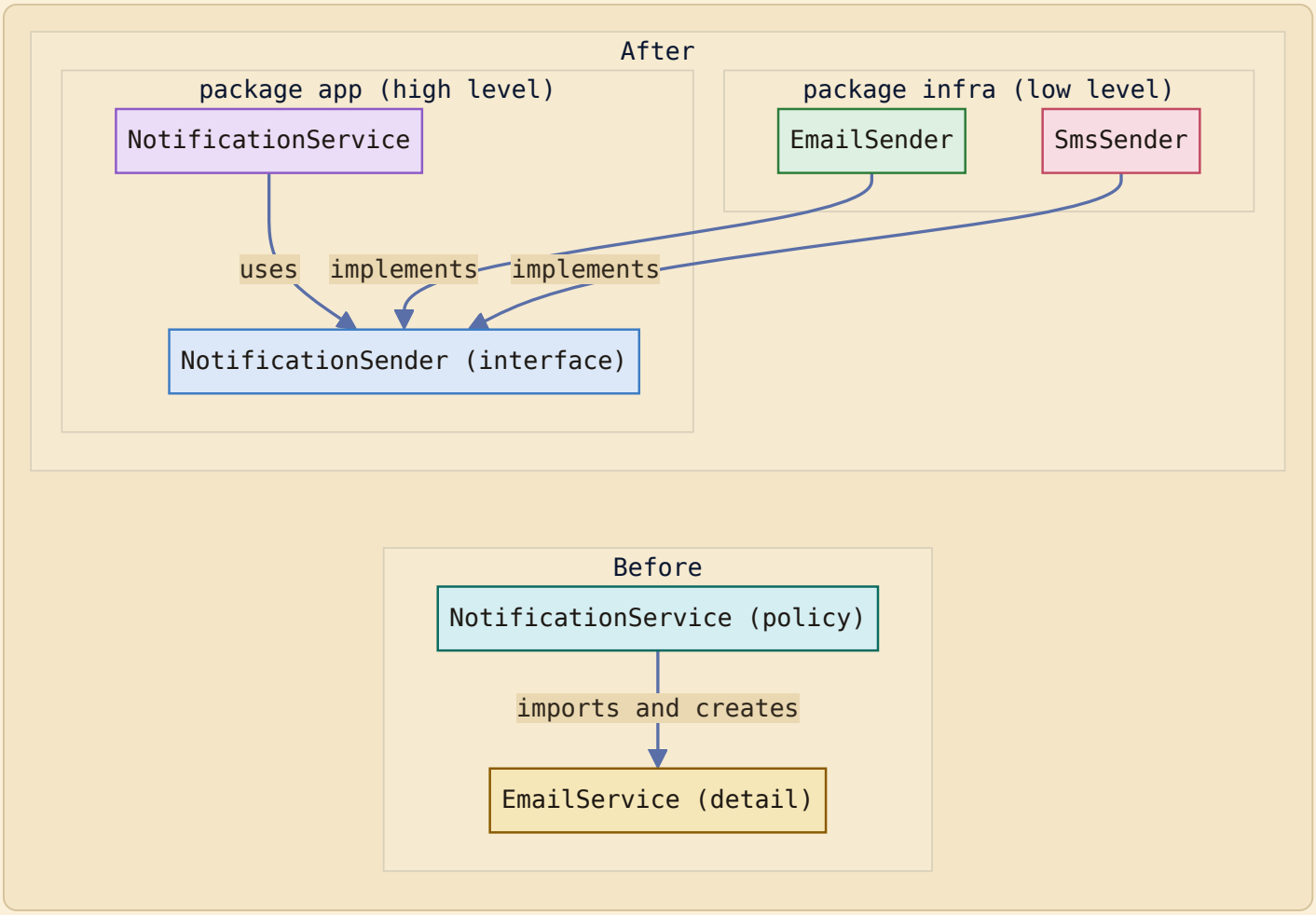
The problem it solves

Policy tied to a vendor; untestable without infrastructure; library changes ripple up.

Example: notifications

Before: `NotificationService` does `new EmailService()`. After: an interface **in the policy's package** (`app`, not `infra`) — **the high-level side owns the abstraction**:

```java
// File: code/lld/ch04/dip/app/NotificationService.java
/** High-level policy: depends only on an abstraction it owns. Imports nothing from infra. */
public class NotificationService {
    private final NotificationSender sender;

    public NotificationService(NotificationSender sender) { // constructor injection
        this.sender = Objects.requireNonNull(sender);
    }

    public void notifyUser(String userId, String message) {
        if (message.isBlank()) {
            throw new IllegalArgumentException("empty notification");
        }
        sender.send(userId, message);
    }
}
// app.NotificationSender { void send(String recipient, String message); }
// infra: EmailSender, SmsSender, RecordingSender (test double) implement it
```



DIP in practice: four real-world techniques

1. Dependency Injection — **constructor** (default; required, immutable), **setter** (optional with default), **method** (varies per call). The one place that does all the `new`-ing is the **composition root** (each `Main`).

2. Service Locator — a registry hands out services; `CheckoutFlow` depends on a `Locator` interface: `locator.paymentService(serviceName).processPayment(amount)`. It satisfies DIP, but hides real dependencies and fails late on a missing registration (Seemann: an anti-pattern). Prefer constructor injection.

3. IoC containers — Spring injects a single constructor automatically (`@Autowired` optional since 4.3). Without Spring: `new OrderService(new StripePayments())`. **The framework is a wiring convenience, not the principle.**

4. Plug-in architecture — the core owns the contract; plug-ins implement it:

```java
// File: code/lld/ch04/dip/plugin/MediaPlayer.java
/** The core. New formats arrive as plug-ins; this class is never edited for them. */
public class MediaPlayer {
    private final Map<String, AudioDecoder> decoders = new HashMap<>();

    public void install(AudioDecoder decoder) {
        decoders.put(decoder.extension(), decoder);
    }
    // ... play(fileName) looks up the decoder by extension
}
```

Full runnable code: code/lld/ch04/dip/ (`before/`, `app/`, `infra/`, `locator/`, `plugin/`) (see the Full Edition).

DI is small-scale (how one class gets collaborators); plug-ins are system-level (whole modules discovered at run time: `ServiceLoader`, OSGi).

DIP vs DI vs IoC — do not mix them up

Term	Kind	Example
DIP — depend on abstractions the policy owns	Principle	`NotificationService` → `NotificationSender`
DI — pass dependencies in	Technique	`new NotificationService(new SmsSender())`
IoC — the framework calls you (Hollywood Principle)	Broad idea; DI is one form	Spring beans, servlet `doGet`, JUnit `@Test`
IoC container	Tool	Spring `ApplicationContext`, Guice

Injection without inversion: a `MySqlDatabase`-typed parameter. Inversion without a framework: this whole chapter.

Stable vs volatile dependencies

Invert **volatile** dependencies (DBs, networks, clocks, third-party APIs, varying rules); use **stable** ones (`String`, `List`, `LocalDate`) directly.

Importance of DIP

Swappable implementations, detail changes stay in the detail, fakes instead of real infrastructure.

How DIP relates to the other principles

SRP (wiring moves to the composition root); OCP (new implementations without edits); LSP (implementations must be proper substitutes); ISP (own the smallest role).

How it works (step by step)

Find policy naming a volatile detail → write its **need** in its own words → make it an interface **in the high-level package** → detail implements it → inject (constructor) → move concrete `new`s to the composition root.

Trade-offs

Gain: technology-independent policy, easy tests, swappable parts. Cost: an interface per volatile dependency, wiring code, an extra hop.

When not to use it

Stable classes; a single implementation forever with no fake; tiny scripts.

Common mistakes

Interface in the low-level package; interface shaped by a vendor API (`sendSmtpMessage`); breaking clause 2 (returning `ResultSet`, throwing `MessagingException` — use domain types); "we use Spring so we follow DIP"; `new ConcreteThing()` "just this once".

Real-world examples

JDBC + `ServiceLoader`; SLF4J; Spring / Jakarta CDI; JPA `EntityManager`; **hexagonal architecture** (ports and adapters — DIP for a whole service).

In the interview

> **30-second answer:** "Business logic depends on an abstraction it owns, not on technical details. `NotificationService` depends on a `NotificationSender` in its own package; email and SMS implement it; I inject via the constructor and wire in `main`, so it's testable with a fake. DI is the technique, DIP the principle, Spring a wiring tool." *Who owns the interface?* The high-level module.

Self-check

1. **Q:** What is inverted? **A:** The source-code dependency.
2. **Q:** Composition root? **A:** The one place concrete classes are created and wired.
3. **Q:** `String` behind an interface? **A:** No — it's stable.

TL;DR

Policy owns the interfaces; details implement them; `main` wires them.

4.11 Refactoring one class through all five principles

The principles usually appear **in sequence**, each exposing the next. One checkout class that validates, prices, saves and emails is refactored five times.

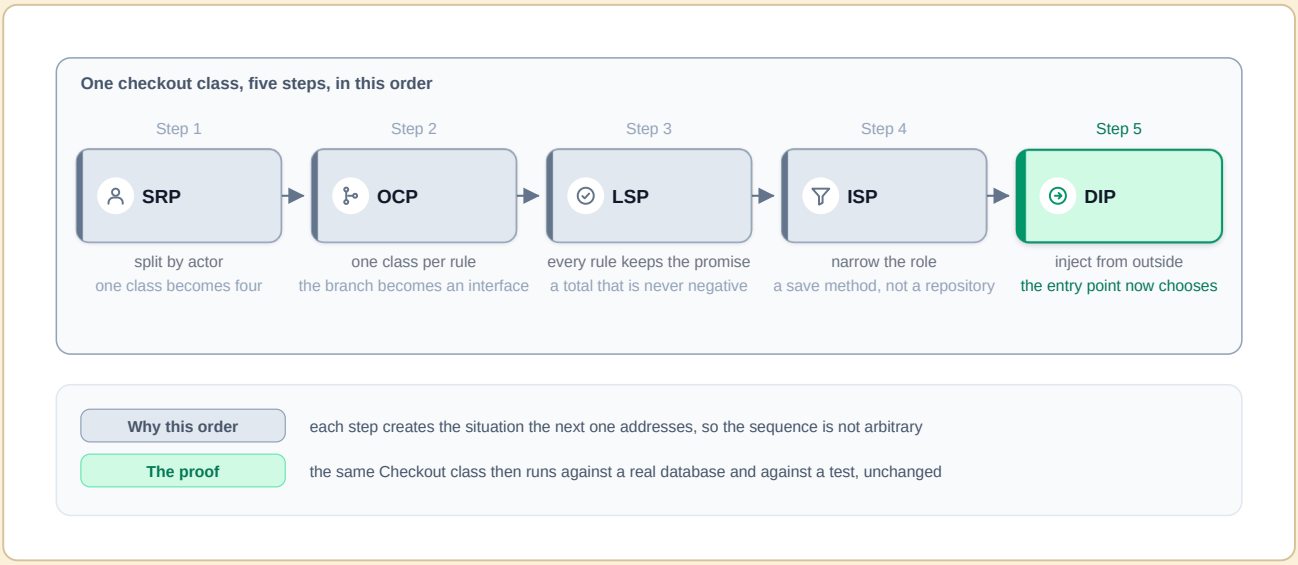


Figure — one checkout class, five steps: SRP, OCP, LSP, ISP, DIP, each creating the situation the next addresses (course diagram)

Step 0: the starting class

`OrderService.process(CustomerOrder)` does it all: rejects non-positive amounts (business-rules owner), branches on `customerType` for `"member"` (0.90) and `"staff"` (0.70) (finance), prints an `INSERT` (database team), and emails the total (marketing). It works — and every principle objects. (Code: `code/lld/ch04/checkout/step0/`.)

Step 1: Single Responsibility — split by actor

Four actors → `OrderValidator`, `PriceCalculator`, `OrderRepository`, `Notifier`, coordinated by a `Checkout` that still `new`s them itself. The customer-type `switch` survives inside `PriceCalculator` (`code/lld/ch04/checkout/step1/`).

Step 2: Open/Closed — one class per pricing rule

```
// File: code/lld/ch04/checkout/pricing/PricingRule.java
/**
 * STEP 2 (OCP): one class per rate; a new rate is a new class.
 * CONTRACT every implementation must keep (STEP 3, LSP):
 *   precondition : amount >= 0
 *   postcondition: result >= 0, rounded to 2 decimals   (a total is never negative)
 */
@FunctionalInterface
public interface PricingRule {
    BigDecimal apply(BigDecimal amount);
}
// RegularPrice, MemberDiscount, StaffDiscount; Order no longer carries a type string
```

`step2.PriceCalculator.total(order, rule)` is now just `rule.apply(order.amount())` — no branch (`code/lld/ch04/checkout/step2/`).

Step 3: Liskov Substitution — every rule keeps the promise

A careless fixed discount of 20 turns a 10 order into **−10**, silently saved. Every rule must return a total **never below zero**; one contract test (`PricingRuleContract.holds`, in `code/lld/ch04/checkout/step3/`) runs against every implementation:

```
// File: code/lld/ch04/checkout/pricing/FixedDiscount.java
    @Override
    public BigDecimal apply(BigDecimal amount) {
        return amount.subtract(off).max(BigDecimal.ZERO).setScale(2, RoundingMode.HALF_EVEN);
    }
```

Step 4: Interface Segregation — the checkout needs a save method, not a repository

The repository grows (`save`, `findById`, `listAll`) but the checkout only saves, so it depends on a one-method role, `OrderSaver`; reporting uses `OrderFinder`; `SqlOrderRepository` implements both.

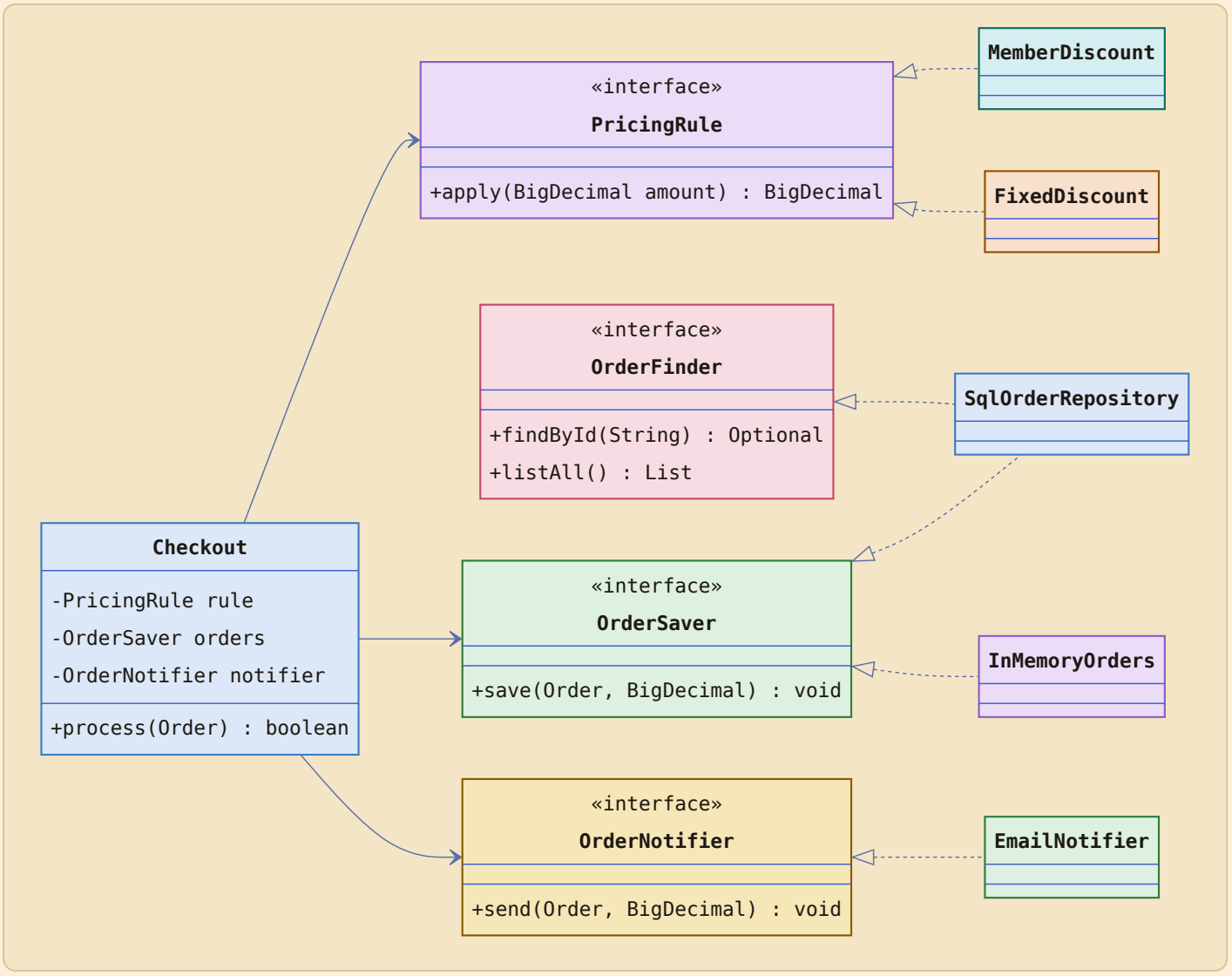
Step 5: Dependency Inversion — the entry point decides

```
// File: code/lld/ch04/checkout/fin/Checkout.java
public class Checkout {
    private final PricingRule rule;
    private final OrderSaver orders;
    private final OrderNotifier notifier;
    // ... constructor takes all three (requireNonNull)
    public boolean process(Order order) {
        // Validation has not varied yet, so it stays a plain check (see "What we did not do").
        if (order.amount().signum() <= 0) {
            System.out.println("  rejected: amount must be positive");
            return false;
        }
        BigDecimal total = rule.apply(order.amount());
        orders.save(order, total);
        notifier.send(order, total);
        return true;
    }
}
```

Full runnable code: code/lld/ch04/checkout/ (`step0/`–`step3/`, `pricing/`, `fin/`) (see the Full Edition).

```
  CarelessFixedDiscount broke the contract: apply(0.00) = -20.00
  4/5 rules are safe substitutes
Production wiring:
  INSERT INTO orders VALUES ('A-9', 180.00)
Test wiring, same Checkout class:
  kept A-9 in memory
```

One unmodified `Checkout` ran with production collaborators, test doubles (`InMemoryOrders`, `SilentNotifier`) and lambdas — **never edited or subclassed**.



Reading the five steps back

Step	The question it asked	What changed
SRP	Who might request changes?	One class → four, one per actor
OCP	What breaks when a new case appears?	Each pricing branch → a class
LSP	Does each rule work through the interface?	All rules guarantee total ≥ 0
ISP	Is the caller tied to more than it calls?	Checkout needs one save method
DIP	Who picks the concrete class?	The entry point

The order has a reason: the varying part stays hidden until each class has one job; LSP matters at the second implementation; ISP when new callers appear; DIP needs abstractions to point at.

What we did not do

`Order` stays a plain record; validation still edits `Checkout` (until a third rule is asked for); stable `BigDecimal` math isn't injected. **Every step was triggered by a real change request, not a checklist.**

Self-check

1. **Q:** Why SRP before OCP? **A:** The varying branch was buried in a four-job method.
2. **Q:** Step 3's contract? **A:** Any amount ≥ 0 gives a total ≥ 0.
3. **Q:** Proof it worked? **A:** One `Checkout`, three wirings, zero edits.

TL;DR

SRP → OCP → LSP → ISP → DIP, each triggered by a real change.

4.12 SOLID and design patterns

Principles say what good structure looks like; patterns are named recipes for it.

Pattern	Principle(s)	How
Strategy	OCP, DIP	Varying algorithm behind an interface
Template Method	OCP	Fixed skeleton, hook steps
Decorator	OCP, SRP	Behaviour added by wrapping
Adapter	DIP, ISP	Caller keeps its own interface
Factory Method / Abstract Factory	DIP, OCP	Concrete constructors in one place
Observer	OCP, DIP	New reactions = new subscribers
Composite	LSP, OCP	Leaves and groups share one interface
Proxy	SRP, OCP	Access control/caching in a separate class
Command	SRP, OCP	Each request an object
Visitor	OCP (operations)	Easy to add operations, hard to add types
Null Object	LSP	Do-nothing implementation that keeps the contract

Head First states the same ideas (encapsulate what varies, composition over inheritance, program to an interface, …); *Clean Code*'s "isolating from change" is DIP.

4.13 SOLID in the design interview

Keeping the design honest: five checks over your class diagram

Run five quick checks (~5 minutes) over the diagram you drew: **SRP** (one-sentence job; an "and" means two), **OCP** (a `switch` over a type?), **LSP** (would any inherited method throw or refuse? recheck late-added arrows), **ISP** (empty methods in big interfaces?), **DIP** (does `ParkingLot` construct a `CreditCardTransaction`? depend on `Payment`).

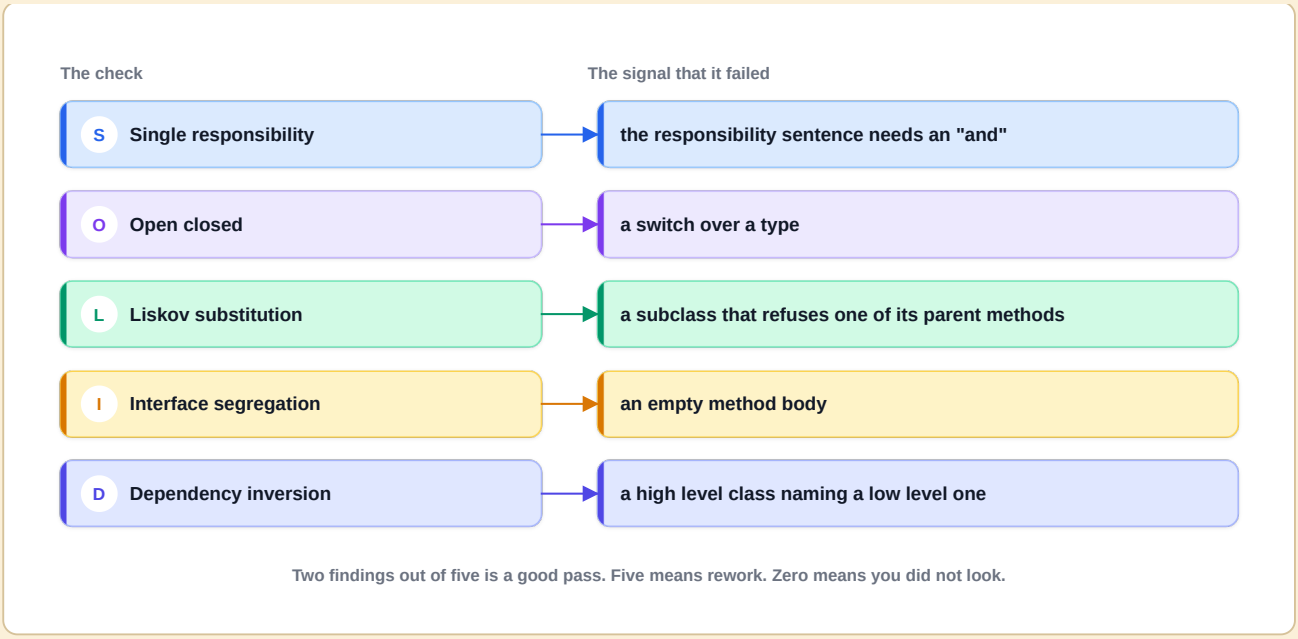


Figure — each check and the signal that it failed: an "and" in the sentence, a switch over a type, a refused parent method, an empty method body, a high-level class naming a low-level one (course diagram)

Two findings out of five is a good pass; five means rework; zero means you didn't look. SRP and OCP earn most credit. Example (parking lot): SRP — `ParkingLot` issues tickets *and* finds spots; LSP — five spot subclasses with no behaviour collapse into one class with a `type` field; OCP, ISP, DIP pass. Don't add one-implementation interfaces or split beyond what a reader can hold.

Naming a design problem out loud

Interviewers weigh: spotting and naming a problem; defending a choice ("pricing changes every quarter" beats "SOLID says so"); knowing when to hold back. **Pattern:** *what you see → what it will cost → what you'd do and what that costs.* Say the observation before the principle's name. To "would you always do that?", the honest answer is almost never "yes". Lose marks by reciting definitions, applying everything at once, or confidently naming the wrong principle. **Drill:** who can ask me to change this class? what's the next feature? which dependencies can't I fake?

4.14 Common questions about SOLID (with short answers)

1. **SRP vs ISP?** SRP guards the class (reasons to change); ISP guards the caller (unused methods).
2. **When not to apply?** Nothing has varied; throwaway code; arbitrary splits; hurts readability. Flexibility later is bought with simplicity now.
3. **Still relevant?** As diagnostic vocabulary, not absolute rules.
4. **Inversion vs injection?** Principle vs technique; a concrete parameter is injection without inversion.
5. **LSP example?** Square/Rectangle, plus the rule: demand no more, guarantee no less.
6. **OCP forbids editing?** No; closure is against a kind of change.
7. **Composition always?** No; when behaviour varies or a subclass would refuse a method.
8. **Principles vs patterns?** Patterns are principles made concrete.
9. **Most violated?** SRP (accumulation), or ISP in reverse (an interface per method).

> **Interview tip:** expand on "when not to apply" and "inversion vs injection"; strong answers always **admit a limit**.

4.15 Over-engineering: the warnings in one place

Warning sign	Misapplied	Instead
`XHelper` / `XUtils` after a split	SRP	Split by actor
Small classes always in the same commit	SRP — *Shotgun Surgery*	Merge back
Interface + factory + registry for one implementation	OCP — *Speculative Generality*	Write the `if`; rule of three
Subclass that silently does nothing	LSP	Better parent or composition
One interface per method; same object thrice	ISP	Group by caller
Interface in front of `String`-like classes	DIP	Invert only volatile deps
DI container for 200 lines	DIP	Wire in `main`

YAGNI is the counterweight: SOLID says **what** a good seam looks like; YAGNI says **when** to cut it.

Chapter self-check

1. **Q:** `switch (vehicleType)` computing fees? **A:** OCP — `ParkingRate` interface, one class per rate.
2. **Q:** `ReadOnlyList extends ArrayList` throws on `add`? **A:** LSP; compose or use a read-only interface.
3. **Q:** `Robot` implements `Worker { work(); eat(); }`? **A:** ISP; split by caller into `Workable` / `Eater`.
4. **Q:** `private final MySqlOrderDao dao = new MySqlOrderDao();`? **A:** DIP violation, untestable; inject an `OrderSaver` the service owns.
5. **Q:** DIP vs DI vs IoC? **A:** Principle; technique; broad idea (containers perform DI).

TL;DR

- **SOLID** names the ways code becomes painful to change — diagnose, don't checklist.
- **SRP** one actor per class; **OCP** varying part behind an interface; **LSP** keep every promise, compose when a subclass would refuse; **ISP** one role per kind of caller; **DIP** policy owns the abstraction, `main` wires.
- **Order**: SRP → OCP → LSP → ISP → DIP, each triggered by a real change.
- **Interview**: observation → cost → fix → price of the fix; know when *not* to apply a principle.

Chapter 5 — More Design Principles (Study Edition)

> **Study Edition.** Every principle of Chapter 5, without the padding. Code blocks are excerpts from the real files (`// ...` marks trimmed parts); full listings and outputs are in the Full Edition (`README.md`). Each topic is one `code/lld/ch05/<topic>/Main.java` with nested `Before` and `After` classes (`javac --release 21`).

SOLID is only five rules. This chapter adds rules of thumb (DRY, KISS, YAGNI), how objects talk (Demeter, Tell-Don't-Ask, loose coupling, Hollywood), the GoF/*Head First* OO principles (encapsulate what varies, composition, program to an interface), correctness (fail fast, immutability, Design by Contract), **GRASP** — nine guidelines for *which class does what* — and **Domain-Driven Design** essentials (*which classes exist and where the boundaries are*).

Key terms. **Coupling** — how much one class depends on another. **Cohesion** — how closely a class's contents belong together. **Abstraction** — a simplified view (interface/abstract class) hiding details. **Encapsulation** — private data changed only through methods. **Invariant** — a fact that must always hold ("balance never negative"). **Side effect** — anything a method changes besides its return value. **Responsibility** — something a class *knows* or *does*.

Chapter map

#	Principle	One-line idea	Comes from
1	DRY	One home per piece of knowledge	Hunt & Thomas (1999)
2	KISS	Simplest design that works	US Navy (1960), Kelly Johnson
3	YAGNI	Don't build before it's needed	Extreme Programming (Ron Jeffries)
4	Law of Demeter	Talk only to immediate friends	Holland / Lieberherr (1987)
5	Tell, Don't Ask	Tell objects what to do	Hunt & Thomas, after Alec Sharp (1997)
6	Composition over inheritance	Prefer has-a for reuse	GoF (1994), *Effective Java*
7	Program to an interface	Depend on supertypes	GoF
8	Encapsulate what varies	Isolate the changing part	GoF, *Head First*
9	Loose coupling	Interacting objects know little	*Head First*
10	Hollywood / IoC	"Don't call us, we'll call you"	*Head First*; frameworks
11	Separation of Concerns	One concern per part	Dijkstra (1974)
12	Least Astonishment	Behave as readers expect	Design folklore
13	Fail fast	Detect problems early, loudly	Jim Shore (2004)
14	Immutability	Never change after creation	*Effective Java*
15	Design by Contract	Pre/postconditions, invariants	Bertrand Meyer, Eiffel (1986)
16	GRASP (9)	Assigning responsibilities	Craig Larman (1997; nine from 2001)
17	Domain-Driven Design	Bounded contexts, entities vs value objects, aggregates, events	Eric Evans (2003); Vaughn Vernon (2013)

1. DRY — Don't Repeat Yourself

What it is

Every piece of **knowledge** (rule, formula, validation, constant) has **one authoritative representation**. Analogy: one master price list. DRY is about knowledge, not look-alike text: identical code representing *different* rules is not a violation.

The problem it solves

Copies **drift** — one is updated, the others aren't — and the system gives two answers to one question.

How it works

Ask "must all copies change together?" If yes, give the rule one home (constant, method, value object, enum, config) and reference it. **Rule of Three** (Don Roberts, via Fowler): tolerate the second copy, extract at the third.

Java example — before/after

Before, two services disagree about `bob@localhost`. After:

```
// File: code/lld/ch05/dry/Main.java
        /** The ONLY definition of "valid email" in the system. */
        record EmailAddress(String value) {
            private static final Pattern FORMAT = Pattern.compile("[^@\\s]+@[^@\\s]+\\.[a-z]{2,}");
            EmailAddress {
                if (value == null || !FORMAT.matcher(value).matches())
                    throw new IllegalArgumentException("invalid email: " + value);
            }
        }
        // ... TaxPolicy holds the 18% RATE in one private constant
```

Full runnable code: code/lld/ch05/dry/ (see the Full Edition). The compact constructor makes an invalid `EmailAddress` impossible.

Trade-offs

Gain: one change point, no drift, named rules. Cost: a shared abstraction couples its users; a wrong abstraction is worse than duplication.

When not to use it

Coincidental look-alikes ("max username 30" vs "max tag 30"); across microservices (shared libraries couple releases); tests (DAMP readability).

Common mistakes

Monster functions with flags; extracting after one repeat; forgetting duplication across code, DB constraints and docs.

Real-world examples

Spring `@Value`; database normalisation; Bean Validation `@Email`; OpenAPI/Protobuf codegen.

In the interview

30-second answer: "One authoritative home per rule — rules, not look-alike text. I extract when copies must change together, e.g. validation into a value object, using the Rule of Three." *Always a violation?* No — different rules stay apart.

Self-check

Q: What bug does duplication cause? → A: Drift.

TL;DR

One rule, one home; leave coincidental look-alikes alone.

---

2. KISS — Keep It Simple, Stupid

What it is

The plainest design that fully solves the real problem (US Navy c. 1960; Kelly Johnson's field-repairable jets). *Simple* = easy to understand and change, not fewest lines.

The problem it solves

Accidental complexity — added by us, not the problem — is the biggest source of bugs and slow delivery.

How it works

State the requirement in one sentence → most direct solution with standard features → "can a newcomer get it in two minutes?" → add structure only for a concrete need.

Java example — before/after

Three fixed tiers. Before: a priority-sorted rule engine of predicates. After:

```
// File: code/lld/ch05/kiss/Main.java
            return switch (tier) {
                case GOLD -> times(amount, GOLD_FACTOR);
                case SILVER -> times(amount, SILVER_FACTOR);
                case REGULAR -> amount;
            };
```

Full runnable code: code/lld/ch05/kiss/ (see the Full Edition). A `switch` *expression* over an enum without `default` must cover every constant, so a new tier won't compile until handled.

Trade-offs

Simple: fast to read and test, standard constructs, may need refactoring when variation arrives. "Flexible": handles hypotheticals but costs every day and needs docs.

When not to use it

Not an excuse to drop real complexity (payments *need* idempotency, retries, auditing); if users edit rules at run time, a rule engine *is* the requirement.

Common mistakes

Simple ≠ short (five nested lambdas); skipping tests or validation; copy-paste instead of simplifying.

Real-world examples

Unix philosophy; Go's small feature set; `Optional` kept small.

In the interview

30-second answer: "Simplest design that meets the requirements; patterns only when a requirement demands them." *KISS vs YAGNI? How* you build vs *what* you build.

Self-check

Q: Accidental complexity? → A: Complexity from our solution, not the problem.

TL;DR

Solve the real problem plainly.

---

3. YAGNI — You Aren't Gonna Need It

What it is

Don't build a capability until you need it (XP, Ron Jeffries). The opposite is **speculative generality**.

The problem it solves

Early features cost four times: building, maintaining, surrounding complexity, and being wrong.

How it works

Build today's requirements; keep code clean and tested so later change is cheap; extend when the need arrives; delete unused options.

Java example — before/after

Requirement: CSV export. Before: `export(orders, format, encoding, compress)` where four of five formats throw `UnsupportedOperationException`. After:

```
// File: code/lld/ch05/yagni/Main.java
        static String toCsv(List<Order> orders) {
            return orders.stream()
                    .map(o -> String.join(",", o.id(), o.customer(), o.total().toPlainString()))
                    .collect(Collectors.joining("\n", "id,customer,total\n", ""));
        }
```

Full runnable code: code/lld/ch05/yagni/ (see the Full Edition).

Trade-offs

YAGNI: less code, design fits real needs, requires test and refactoring discipline. Building ahead: pays only if the guess is right; leaves dead options.

When not to use it

Hard-to-reverse decisions (public APIs, big schemas, security, retention laws); known quality requirements are requirements.

Common mistakes

Excuse for messy code; ignoring a stated requirement; `interface` + `Impl` everywhere.

Real-world examples

MVPs; feature flags (LaunchDarkly, Unleash); iteration planning.

In the interview

30-second answer: "I build today's needs and keep the design easy to extend; I add `PaymentStrategy` only if variants were hinted." *Conflict with OCP?* No — OCP where variation is known; YAGNI stops invented variation.

Self-check

Q: When design ahead anyway? → A: Expensive-to-reverse decisions.

TL;DR

Build for today with clean, changeable code.

---

4. Law of Demeter (Principle of Least Knowledge)

What it is

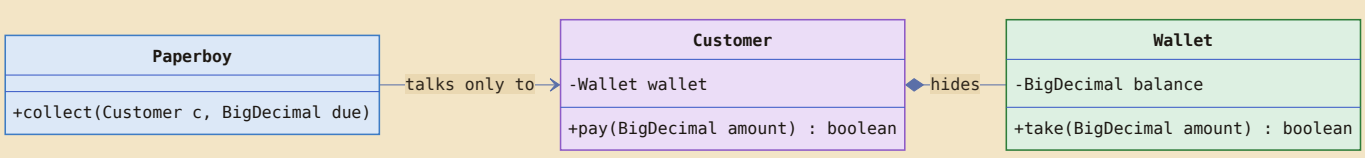
A method `m` of `O` calls only: `O` itself, `m`'s parameters, objects `m` creates, `O`'s own fields (Holland & Lieberherr, 1987; originally also globals). **"Use only one dot"** — `a.getB().getC().doIt()` is a **train wreck**. Analogy: the paperboy asks the customer to pay rather than taking their wallet.

The problem it solves

Chains couple the caller to the whole structure: switch the customer from wallet to UPI and the paperboy breaks.

How it works

Ask what the caller really wants → add that method on the first object (`customer.pay(amount)`) → keep internals private.



Java example — before/after

Before: `c.getWallet().setBalance(c.getWallet().getBalance().subtract(due))`. After:

```
// File: code/lld/ch05/demeter/Main.java
            /** Customer decides HOW to pay. */
            boolean pay(BigDecimal amount) { return wallet.take(amount); }
        // ...
                if (c.pay(due)) {                    // talk only to a parameter: allowed
```

Full runnable code: code/lld/ch05/demeter/ (see the Full Edition). Stream chains aren't violations — each call returns the same kind of thing.

Trade-offs

Following: internal changes stay local, one collaborator to mock, but wrapper methods accumulate. Ignoring: structural changes ripple and tests build object graphs.

When not to use it

Fluent APIs, builders, streams; DTOs/records (`order.shippingAddress().city()`); not dozens of pass-throughs (**Middle Man**).

Common mistakes

Counting dots, not knowledge (temp variables still violate it); getters for everything.

Real-world examples

Facade (*Head First*); Spring Data `findByCustomerId`; `Files.readString(path)`.

In the interview

> **30-second answer:** "A method calls only itself, parameters, objects it creates and its fields. For `a.getB().getC()` I add an intent method — `customer.pay()`." *Streams?* Not a violation.

Self-check

Q: The four friends? → A: `this`, parameters, created objects, own fields.

TL;DR

Only talk to friends.

---

5. Tell, Don't Ask

What it is

Don't pull data out and decide for an object — **tell** it what you want (Hunt & Thomas, after Sharp). Analogy: tell the taxi "airport".

The problem it solves

Ask-then-act spreads rules across callers; one forgets part ("frozen accounts can't withdraw"), the invariant breaks, and the object becomes an **Anemic Domain Model**.

How it works

Find read-decide-write code → move the decision into a method → reject illegal requests → remove setters.

Java example — before/after

Before, the ATM checks `isFrozen()`; the app forgets. After:

```
// File: code/lld/ch05/telldontask/Main.java
            void withdraw(long amt) {
                if (amt <= 0) throw new IllegalArgumentException("amount must be positive");
                if (frozen) throw new IllegalStateException("account is frozen");
                if (amt > balancePaise) throw new IllegalStateException("insufficient funds");
                balancePaise -= amt;
            }
```

Full runnable code: code/lld/ch05/telldontask/ (see the Full Edition).

Trade-offs

Tell: invariants in one place, richer objects, risk of bloated objects. Ask: duplicated rules; fine for pure data.

When not to use it

Display/reporting queries; decisions combining several objects' data (Information Expert elsewhere).

Common mistakes

A getter "just for this check"; emails and DB calls inside entities.

Real-world examples

`List.removeIf`; `Map.computeIfAbsent`/`merge`; `AtomicInteger.incrementAndGet()`.

In the interview

> **30-second answer:** "`account.withdraw(100)`, not read-then-set the balance — invariants stay in one place."
> *Concurrency?* Ask-then-act is a check-then-act race; one method can be atomic.

Self-check

Q: Anti-pattern it fights? → A: Anemic Domain Model.

TL;DR

Tell the object what you want.

---

6. Composition over Inheritance

What it is

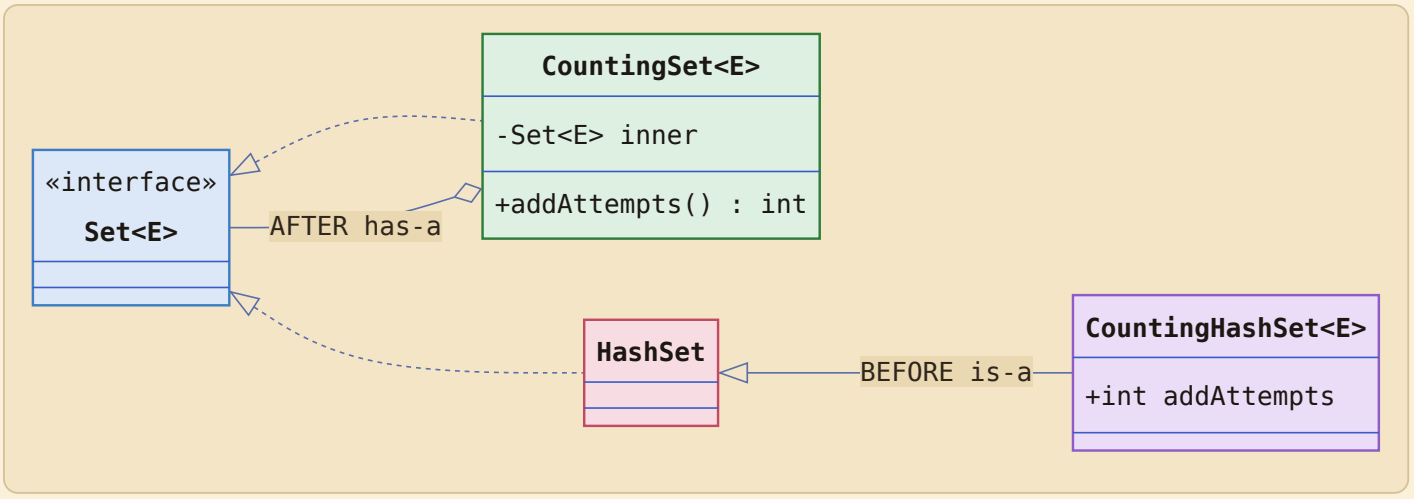
Favour composition over class inheritance (GoF). Is-a reuses by subclassing; has-a holds a reference and **delegates**. Analogy: a car *has* a swappable engine.

The problem it solves

Inheritance is the tightest coupling: **fragile base class** (breaks when parent internals change), fixed at compile time, **class explosion**, every parent method exposed.

How it works

Replace `extends A` with a private interface-typed field, forward calls, inject via constructor; inherit only for substitutable is-a with parents **designed for extension**.



Java example — before/after

The *Effective Java* trap:

```
// File: code/lld/ch05/composition/Main.java
            @Override public boolean addAll(Collection<? extends E> c) {
                addAttempts += c.size();
                return super.addAll(c);   // HashSet's inherited addAll calls add() for each element ->
counted twice!
            }
    // ...
        static final class CountingSet<E> extends AbstractSet<E> {
            private final Set<E> inner;   // "has-a", not "is-a"
            // ...
            @Override public boolean addAll(Collection<? extends E> c) { addAttempts += c.size(); return
inner.addAll(c); }
```

Full runnable code: code/lld/ch05/composition/ (see the Full Edition).

```
BEFORE: added 3 items, counter says 6
AFTER : added 3 items, counter says 3
```

`HashSet` inherits `AbstractCollection.addAll`, which calls our overridden `add` — an unpromised "self-use" detail. The wrapper also forwards `contains`/`remove` (else `AbstractSet` scans, O(n)) and works over a `TreeSet` chosen at run time.

Trade-offs

Composition	Inheritance
Coupled to public API; swap at run time; combine freely	Coupled to internals; fixed; class explosion
Forwarding boilerplate; needs an interface for polymorphism	Free reuse; automatically a subtype

When not to use it

True is-a with a base designed for extension (`AbstractList`, Template Method); sealed closed variants; classes that evolve together.

Common mistakes

`OrderService extends BaseDao`; composition without an interface; exposing the wrapped object.

Real-world examples

`Stack extends Vector`, `Properties extends Hashtable` (admitted JDK mistakes); `BufferedInputStream`; `Collections.unmodifiableList`; Unity's Entity–Component model.

In the interview

30-second answer: "Inheritance couples to parent internals, is fixed and explodes combinatorially; composition delegates to an interface, swappable at run time. Inherit only for true is-a designed for extension." *Composition-based patterns?* Strategy, Decorator, Composite, State, Bridge, Proxy, object Adapter.

Self-check

Q: Why 6? → A: Inherited `addAll` calls the overridden `add`.

TL;DR

Reuse by has-a; is-a only for substitutable subtypes designed for extension.

---

7. Program to an Interface, Not an Implementation

What it is

Type variables, fields, parameters and returns by the **most general supertype** you need ("interface" = any supertype). Analogy: a charger built for the USB-C standard.

The problem it solves

Code typed on `ArrayList` or `MySqlUserStore` is untestable without the real thing and not swappable.

How it works

Name the role → interface with only needed methods → type by it → create concrete objects at the edges and inject them (**DI**) → widest parameter type (`Collection` > `List` > `ArrayList`).

Java example — before/after

Before, `GreetingService` does `new MySqlUserStore()` (can't run on a laptop) and `countLongNames` takes `ArrayList`. After:

```
// File: code/lld/ch05/interfaces/Main.java
        interface UserRepository { Optional<User> findById(String id); }
        // ...
            private final UserRepository users;                 // any implementation
            GreetingService(UserRepository users) { this.users = users; }
        // ...
        static int countLongNames(Collection<String> names) {   // works for List, Set, Queue...
```

Full runnable code: code/lld/ch05/interfaces/ (see the Full Edition).

Trade-offs

Interface-typed: swappable, testable, but no implementation-only methods and one more type. Concrete-typed: simple, but changes ripple.

When not to use it

Value types; one implementation with no test seam (`FooServiceImpl` ceremony); genuinely specific needs (`LinkedHashMap` access order for LRU).

Common mistakes

30-method mirror interfaces (ISP); declaring `List` then casting; still `new`-ing inside.

Real-world examples

Collections Framework; JDBC; SLF4J; Spring injection by interface.

In the interview

> **30-second answer:** "I type dependencies by roles and inject implementations — in-memory for tests. It's DIP in everyday form." *Every class an interface?* No.

Self-check

Q: Where does `new ConcreteClass()` go? → A: The edges: `main`, factories, DI config.

TL;DR

Depend on roles; pick the concrete class once, at the edge.

---

8. Encapsulate What Varies

What it is

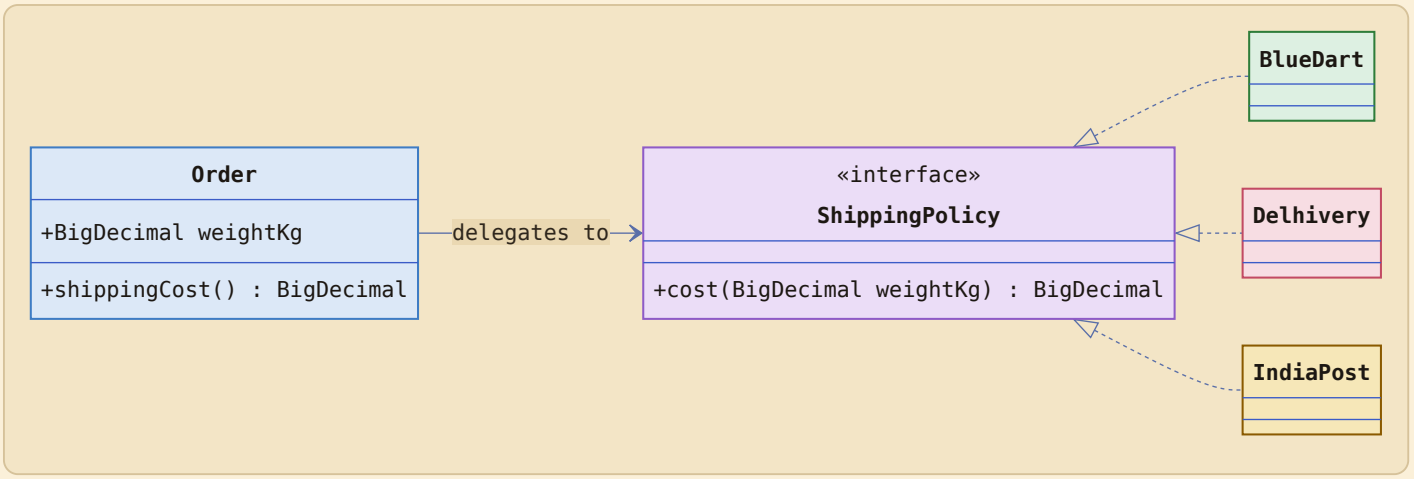
Separate what **changes** from what stays the same; put the varying part behind an abstraction (*Head First*'s first principle). Analogy: camera body, swappable lenses.

The problem it solves

Volatile rules inline force edits to stable classes; type `switch`es grow forever (OCP broken).

How it works

Find what changed recently or differs per customer/region → extract behind an interface/lambda → delegate → add variants. Behind Strategy, State, Factory, Observer, Template Method.



Java example — before/after

Before, `Order.shippingCost()` branches on carrier strings. After:

```
// File: code/lld/ch05/encapsulatevaries/Main.java
        interface ShippingPolicy { BigDecimal cost(BigDecimal weightKg); }

        /** Stable catalogue of today's policies. Adding one = adding one entry; Order never changes. */
        static final Map<String, ShippingPolicy> POLICIES = Map.of(
                "BLUEDART", w -> linear("50", "20", w),
                "DELHIVERY", w -> linear("40", "25", w),
                "INDIA_POST", w -> linear("30", "10", w));

        record Order(BigDecimal weightKg, ShippingPolicy shipping) {
            BigDecimal shippingCost() { return shipping.cost(weightKg); }
        }
```

Full runnable code: code/lld/ch05/encapsulatevaries/ (see the Full Edition). A promo policy was added with zero edits to `Order`.

Trade-offs

Encapsulated: new variants without touching stable code, each testable; more indirection. Inline: all visible, fine for two fixed variants, grows into long conditionals.

When not to use it

It has never varied (YAGNI); extract at the third variant.

Common mistakes

Wrong axis of variation; leaving the type `switch` in the client.

Real-world examples

`Comparator`; `ThreadPoolExecutor`'s `RejectedExecutionHandler`/`ThreadFactory`; payment gateways.

In the interview

> **30-second answer:** "I hide what's likely to change — pricing, payment method — behind an interface; it's behind Strategy and State." *How do you know what varies?* Requirements, history, the interviewer's hints.

Self-check

Q: Smell of missing encapsulation? → A: A growing type `switch`.

TL;DR

Put the changing part behind an interface.

---

9. Strive for Loosely Coupled Designs

What it is

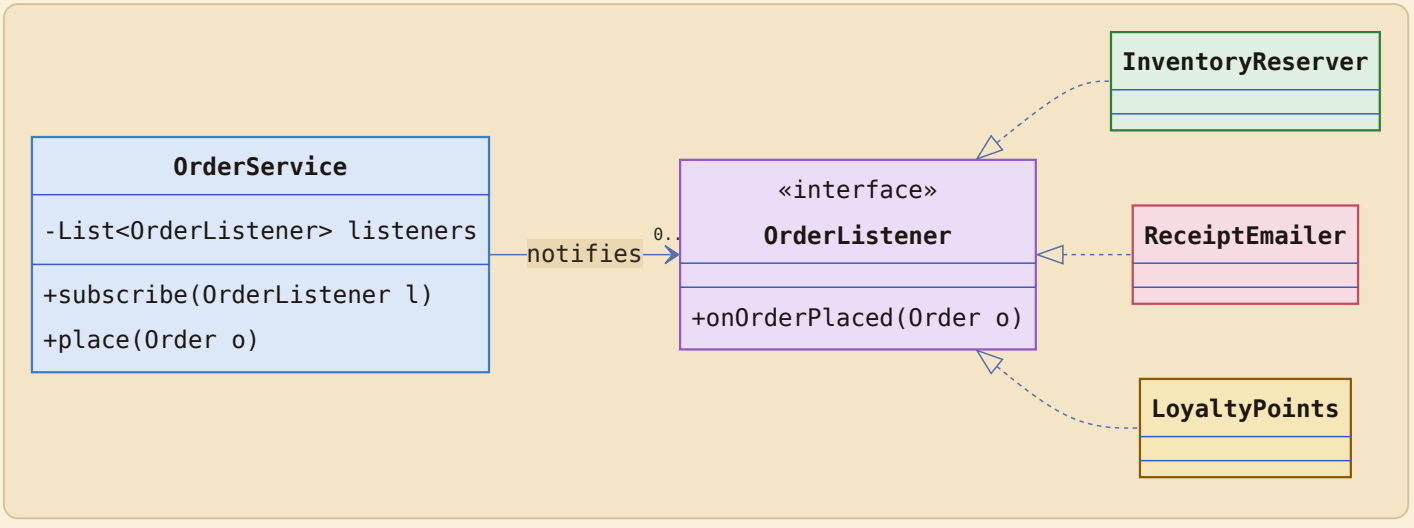
Interacting objects know as little as possible about each other — ideally a small interface (*Head First*). Analogy: a radio station doesn't know its listeners.

The problem it solves

A class that creates and calls many concrete collaborators is edited for every change, not reusable, not testable alone.

How it works

Interfaces; injection; **events** when the sender shouldn't know receivers (Observer); narrow interfaces; immutable values. Coupling, tightest → loosest: content → common → external → control → stamp → data → message.



Java example — before/after

Before, `OrderService` creates `EmailSender`, `Inventory`, `Analytics`. After:

```java
// File: code/lld/ch05/loosecoupling/Main.java
            private final List<OrderListener> listeners = new CopyOnWriteArrayList<>();
            void subscribe(OrderListener l) { listeners.add(l); }
            void place(Order o) {
                System.out.println("placed " + o.id());
                listeners.forEach(l -> l.onOrderPlaced(o));   // doesn't know who listens or what they do
            }
```

Full runnable code: code/lld/ch05/loosecoupling/ (see the Full Edition). Loyalty points became one more `subscribe`; `CopyOnWriteArrayList` allows safe subscription during notification.

Trade-offs

Loose	Tight
Independent change and tests; features = listeners	Flow easy to follow and debug
"Who handles this?"; listener order and errors need design	Every change edits the central class

When not to use it

Small cohesive modules; ordered, atomic steps (debit then credit) — orchestrate explicitly.

Common mistakes

A global event bus for everything; shared static state; one throwing listener stopping the rest.

Real-world examples

UI listeners; Spring `@EventListener`; Kafka (`OrderPlaced` consumed independently by inventory, notifications, analytics).

In the interview

30-second answer: "Collaborators share only small interfaces — interfaces, DI, events; the order service publishes 'order placed' and features subscribe." *Downside?* Harder to trace flow, ordering, failures.

Self-check

Q: Classic one-to-many pattern? → A: Observer.

TL;DR

Small interfaces, injection and events.

---

10. The Hollywood Principle (Inversion of Control)

What it is

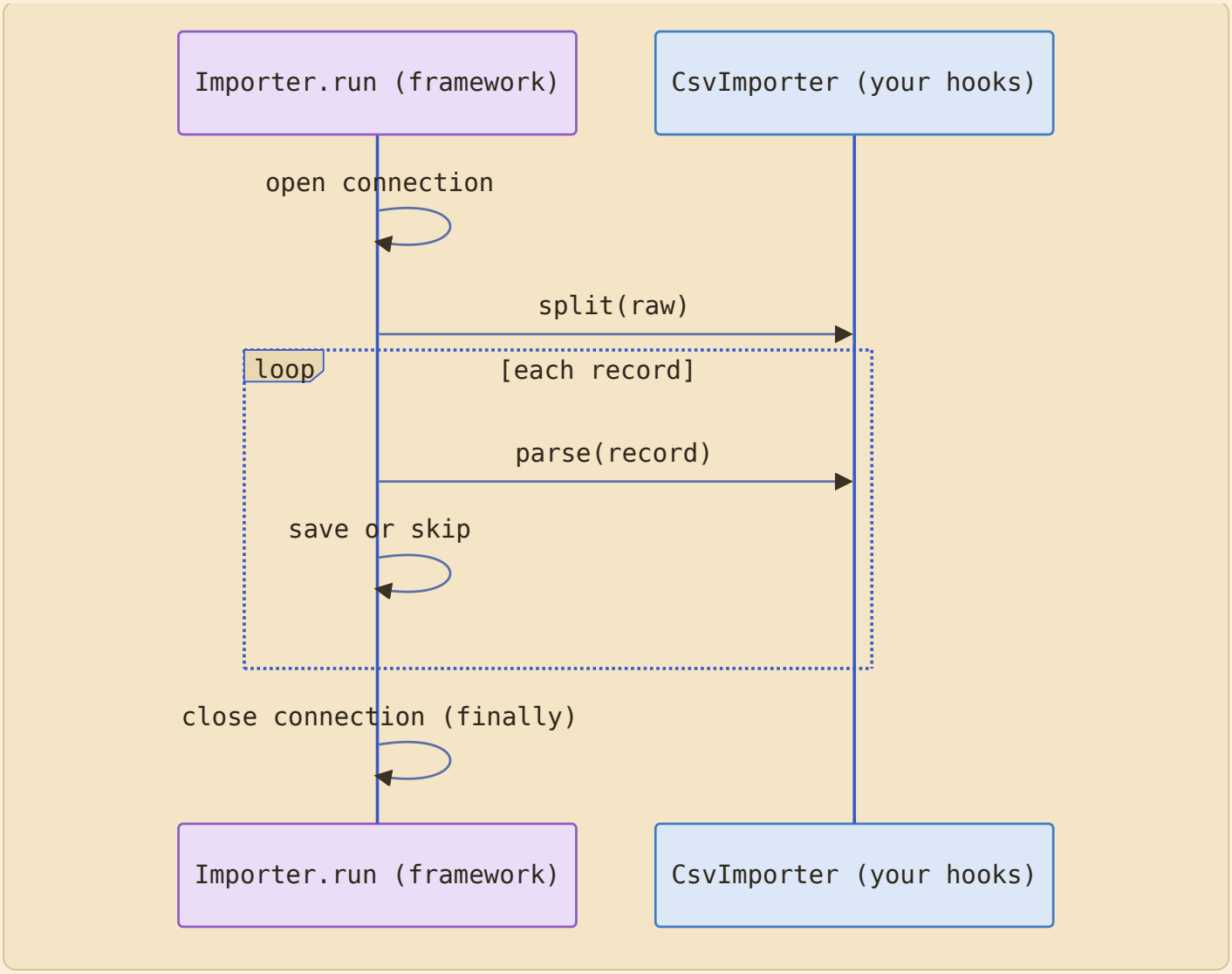
"Don't call us, we'll call you." High-level components own the flow and call low-level hooks, which never call up — **Inversion of Control**. **You call a library; a framework calls you.**

The problem it solves

Each component re-implements the skeleton, forgets steps (closing connections), and levels depend on each other (**dependency rot**).

How it works

Template Method (`final` algorithm, abstract hooks); **callbacks**; **DI**; **lifecycle management** (`@PostConstruct`, servlet `init`/`destroy`).



Java example — before/after

Before, `CsvImporter.run` drives everything and forgets to close. After:

```java
// File: code/lld/ch05/hollywood/Main.java
            /** The fixed algorithm. final = subclasses cannot reorder or skip steps. */
            public final void run(String raw) {
                System.out.println("  open connection");
                try {
                    // ... for each record from split(raw): parse, then save or skip
                } finally {
                    System.out.println("  close connection");       // guaranteed by the framework
                }
            }
            protected abstract List<String> split(String raw);       // hooks: WE call YOU
            protected abstract String[] parse(String record);
```

Full runnable code: code/lld/ch05/hollywood/ (see the Full Edition). A tiny `Container` also starts `Lifecycle` components in order and stops them in reverse (as Spring does).

Trade-offs

IoC: guaranteed flow, ordering and cleanup, no duplicated skeletons; but "magic", harder to debug, tied to hooks. Direct control: visible flow, duplicated skeletons.

When not to use it

Small scripts with no repeated skeleton.

Common mistakes

Hooks calling back up; template method not `final`; confusing IoC (idea) with DI (one form).

Real-world examples

JUnit `@Test`; servlets/Spring MVC; Spring lifecycles; Android `onCreate`; `AbstractList`; `JdbcTemplate` + `RowMapper`.

In the interview

> **30-second answer:** "The high-level component controls flow and calls hooks — what separates a framework from a library; Template Method, callbacks and DI are forms." *vs DIP?* DIP: depend on abstractions; Hollywood: who calls whom.

Self-check

Q: Why `final` template method? → A: Subclasses can't reorder or skip steps.

TL;DR

The framework owns the flow and calls your hooks.

---

11. Separation of Concerns (SoC)

What it is

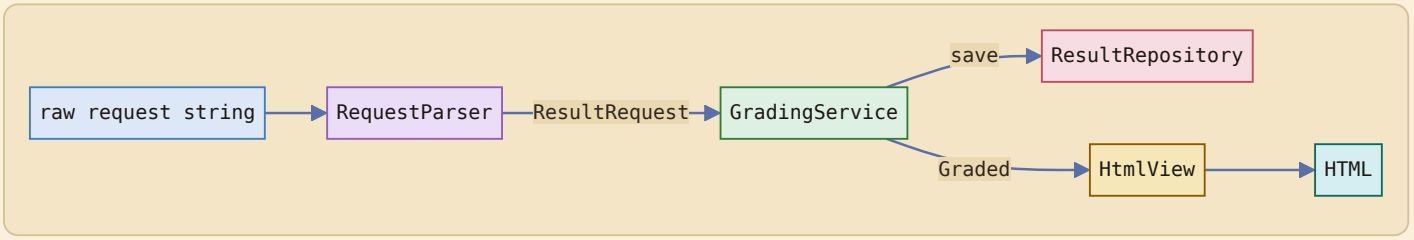
A **concern** is one aspect (input, rules, storage, presentation, security, logging); each part handles one (Dijkstra, 1974, EWD 447). Big-picture SRP, at every level: functions, classes, layers (MVC), services, languages (HTML/CSS/JS).

The problem it solves

Spaghetti: can't change HTML without risking the grading rule; can't test the rule without parsing strings.

How it works

One unit per concern with typed inputs/outputs; data flows one way (parse → rules → store → present); lower layers don't know higher ones; **cross-cutting concerns** via filters, decorators, proxies, aspects.



Java example — before/after

Before, one `handle(String request)` parses, grades, stores and returns HTML. After, the rules class:

```java
// File: code/lld/ch05/soc/Main.java
        /** Concern 2: business rules only. No HTML, no storage. */
        static final class GradingService {
            // ...
            Graded record(ResultRequest r) {
                if (r.marks() < 0 || r.marks() > 100) throw new IllegalArgumentException("marks must be
0..100");
                String grade = r.marks() >= 90 ? "A" : r.marks() >= 75 ? "B" : r.marks() >= 50 ? "C" : "F";
                repo.save(r.name(), r.marks());
                return new Graded(r.name(), r.marks(), grade);
            }
        }
```

Full runnable code: code/lld/ch05/soc/ (see the Full Edition). `RequestParser`, `ResultRepository` and `HtmlView` hold the other concerns; a JSON view would not touch rules.

Trade-offs

Separated: safe changes, parallel team work, more types. Mixed: quick one-offs, tangled and hard to test.

When not to use it

Throwaway scripts; avoid "lasagna code" (layers that only pass data through).

Common mistakes

Rules in controllers or SQL; separating only by technical type while a feature is scattered.

Real-world examples

MVC; HTML/CSS/JS; TCP/IP layers; Spring AOP, servlet filters.

In the interview

30-second answer: "Each module handles one aspect so each changes and tests alone — controller, service, repository, domain in the middle." *Cross-cutting?* Logging, auth, transactions via AOP/proxies/filters.

Self-check

Q: Four concerns of a web request? → A: Parsing, rules, persistence, presentation.

TL;DR

Each concern its own home.

---

12. Principle of Least Astonishment (POLA)

What it is

A component behaves as users expect from its name, signature and context; if not, change the behaviour or the name. Analogy: a light switch that also locks the door.

The problem it solves

Surprises cause bugs even when code "works": mutating queries, mutate-and-return-`this` aliasing, inconsistent names, 0/1-based confusion, swallowed errors.

How it works

Truthful names (`find` doesn't modify; `getOrCreate` if it creates); platform conventions; consistency; immutable results; `Optional` for absence; document unavoidable surprises.

Java example — before/after

Before, `Money.add` mutates `this` and `findUser` silently creates users. After:

```
// File: code/lld/ch05/leastastonishment/Main.java
            Money plus(Money other) { return new Money(paise + other.paise); }   // returns a NEW value
        // ...
            Optional<String> findUser(String id) { return Optional.ofNullable(users.get(id)); }
            String getOrCreateUser(String id) { return users.computeIfAbsent(id, k -> "new-user-" + k); }
```

Full runnable code: code/lld/ch05/leastastonishment/ (see the Full Edition).

```
new BigDecimal("2.0").equals(new BigDecimal("2.00")) = false
new BigDecimal("2.0").compareTo(new BigDecimal("2.00")) = 0
Arrays.asList(...).add -> UnsupportedOperationException (fixed-size list)
```

JDK gotchas: use `compareTo` for numeric equality; `Arrays.asList` is fixed-size.

Trade-offs

A longer name or extra method is cheap next to surprise bugs; "expected" depends on your platform's conventions.

When not to use it

Never surprise on purpose, but fix a name rather than bend to a wrong expectation.

Common mistakes

Getters with side effects; overloading `equals(Money)` instead of overriding `equals(Object)`; hidden global flags.

Real-world examples

`java.time` fixing `Date`/`Calendar`; `URL.equals` doing DNS (prefer `URI`); `rm` without undo; "Cancel" never saves.

In the interview

30-second answer: "Queries don't mutate, creation is named, absence is `Optional`, values immutable."
CQS? Command-Query Separation (Chapter 6) makes state changes obvious.

Self-check

Q: `findUser` on a missing user? → A: Empty `Optional`, never silent creation.

TL;DR

Do what the name promises and nothing more.

---

13. Fail Fast

What it is

Detect invalid state or input **where it arises** and stop with a clear error. Analogy: a smoke alarm at the first smoke.

The problem it solves

A `null` accepted in a constructor crashes hours later, pointing at the symptom; bad data may be saved first.

How it works

Validate at boundaries (constructors, public entry, API input, startup config); guard clauses (`requireNonNull`, `IllegalArgumentException`); specific messages; invalid objects **unconstructable**; never swallow exceptions.

Fail fast vs fail safe: fail-*safe* = failure leaves a safe state. Interview literature (not the JDK, which says "snapshot"/"weakly consistent") calls `CopyOnWriteArrayList`/`ConcurrentHashMap` iterators "fail-safe" (never throw `ConcurrentModificationException`); **fail-fast** iterators (`ArrayList`) throw on structural modification during iteration — **best-effort** only.

Java example — before/after

Before, `new RetryConfig(null, 0, null)` succeeds and an NPE appears far away. After:

```
// File: code/lld/ch05/failfast/Main.java
        record RetryConfig(String endpoint, int maxAttempts, Duration backoff) {
            RetryConfig {
                Objects.requireNonNull(endpoint, "endpoint");
                Objects.requireNonNull(backoff, "backoff");
                if (maxAttempts < 1) throw new IllegalArgumentException("maxAttempts must be >= 1, was " +
maxAttempts);
                if (backoff.isNegative()) throw new IllegalArgumentException("backoff must not be negative");
            }
        }
```

Full runnable code: code/lld/ch05/failfast/ (see the Full Edition). The demo also triggers an `ArrayList` `ConcurrentModificationException` and shows the safe `removeIf`.

Trade-offs

Fail fast: bugs near their cause, clear diagnostics, upfront validation, may stop a process for a minor issue. Fail slow: "keeps running", corruption, confusing symptoms.

When not to use it

Fail fast inside components, but be resilient at edges (a widget failure shouldn't crash the UI; a batch job may skip and log). Detect early, then handle deliberately.

Common mistakes

Swallowing exceptions or returning `null`/`-1`; validating deep inside; `assert` for public arguments (disabled by default).

Real-world examples

`List.of(null)`; Spring Boot startup checks; circuit breakers (Resilience4j); Erlang "let it crash"; CI stopping at first failure.

In the interview

30-second answer: "Validate at the boundary and throw clearly, near the cause — guard clauses so invalid objects can't exist; at system level, circuit breakers." *Bad for users?* Fail fast inside; degrade gracefully at the edge.

Self-check

Q: Why not `assert` for public checks? → A: Disabled by default.

TL;DR

Check early, throw clearly, never swallow.

---

14. Immutability

What it is

An object whose observable state can't change after construction; "changing" creates a new one (`String`, `LocalDate`, `BigDecimal`, records of immutables). Analogy: reprint a receipt, don't edit it.

The problem it solves

Aliasing bugs, **broken invariants**, **thread-safety problems**, **hash-collection bugs**. Immutables are inherently thread-safe, cacheable, safe keys.

How it works

Effective Java: no mutators; not extendable (`final`; records are); all fields `final` and `private`; **defensive copies** of mutable components in and out. Plus "withers". `Collections.unmodifiableList` is a live *view*; `List.copyOf`/`List.of` are independent. Records are **shallowly** immutable.

Java example — before/after

Before, `Itinerary` keeps and returns the caller's list — outsiders clear it. After:

```
// File: code/lld/ch05/immutability/Main.java
        record Itinerary(List<String> cities, LocalDate start) {
            Itinerary {
                cities = List.copyOf(cities);   // copy in: caller's later edits cannot reach us
            }
            Itinerary withStart(LocalDate newStart) { return new Itinerary(cities, newStart); }
            // ... plusCity(c) copies, adds, returns a new Itinerary
        }
```

Full runnable code: code/lld/ch05/immutability/ (see the Full Edition). `t1.cities().clear()` throws; `plusCity(...).withStart(...)` yields a new `t2`. `LocalDate.of(2026,1,31).plusMonths(1)` returns Feb 28.

Trade-offs

Immutable: thread-safe, safe keys, cacheable; an allocation per change (usually cheap), copying large objects. Mutable: in-place updates, needs synchronisation, aliasing bugs.

When not to use it

Hot loops (local `StringBuilder`, expose the immutable result); entities with changing state (encapsulate; make their values immutable); JPA entities.

Common mistakes

`final` field ≠ immutable; returning the internal list; `unmodifiableList` over a caller-held list; `this` escaping the constructor.

Real-world examples

`List.of`, `Map.of`, records; Git commits; Kafka records; Redux/React state.

In the interview

> **30-second answer:** "Final class, private final fields, no setters, defensive copies — thread-safe without locks, safe as keys. Value objects immutable; entity state encapsulated." *Records?* Shallowly immutable.

Self-check

Q: `unmodifiableList` vs `List.copyOf`? → A: Live view vs independent copy.

TL;DR

No setters, final everything, copy in and out.

---

15. Design by Contract (DbC)

What it is

(Meyer, Eiffel) **Precondition** — caller's duty (`pop` needs a non-empty stack); **postcondition** — method's guarantee (after `push(x)` size +1, top `x`); **class invariant** — always true between calls (`0 ≤ size ≤ capacity`).

The problem it solves

Without contracts, duplicated defensive checks or none, ambiguous returns (`-1`), lost data.

How it works

Document the contract → check preconditions at entry (`IllegalArgumentException` / `IllegalStateException`) → check postconditions/invariants in dev/tests → offer queries (`isEmpty()`, `isFull()`) → subclasses may **weaken** preconditions and **strengthen** postconditions, keeping the invariant (LSP).

Java example — before/after

Before, a full stack drops pushes and an empty one returns `-1`. After:

```java
// File: code/lld/ch05/contract/Main.java
            /** pre: !isFull(). post: size() == old size + 1 and peek() == x. */
            void push(int x) {
                require(!isFull(), "push requires !isFull()");
                int oldSize = size;
                items[size++] = x;
                ensure(size == oldSize + 1 && peek() == x, "push postcondition");
                checkInvariant();
            }
```

Full runnable code: code/lld/ch05/contract/ (see the Full Edition). Output: `precondition failed: push requires !isFull()`.

Trade-offs

Explicit: clear duties, precise errors, executable docs; runtime cost (postcondition/invariant checks can be off in production). Implicit: less code, ambiguity, silent corruption.

When not to use it

Full checks in hot loops — keep preconditions on public APIs, move expensive checks to tests or `assert`.

Common mistakes

`assert` for public preconditions; subclasses strengthening preconditions (`SavingsAccount.withdraw`; `Square.setWidth` is the postcondition mirror); special return values.

Real-world examples

Java API docs (`List.get`, `equals`/`hashCode`); Guava `Preconditions`, Spring `Assert`; Kotlin `require()`/`check()`; Eiffel, Ada 2012.

In the interview

> **30-second answer:** "Preconditions for the caller, postconditions guaranteed, invariants always true; guard clauses and query methods; subclasses weaken pre, strengthen post — LSP." *vs defensive*

> *programming?* DbC assigns each check to one party.

Self-check

Q: Whose duty is a precondition? → A: The caller's.

TL;DR

State and enforce who promises what.

---

16. GRASP — General Responsibility Assignment Software Patterns

What it is

Craig Larman's nine patterns for **"which class should be responsible for this?"** (knowing or doing) — a vocabulary for justifying decisions, not code templates. (The 1997 edition's ninth was "Don't Talk to Strangers"; 2001 replaced it with **Protected Variations**.)

#	Guideline	Question	Short answer
1	Information Expert	Who does this job?	The class with the information
2	Creator	Who creates A?	One that contains, records, closely uses or initialises A
3	Controller	Who first handles a UI event?	A non-UI use-case or system object
4	Low Coupling	Reduce impact of change?	Few dependencies
5	High Cohesion	Keep classes focused?	Related work per class
6	Polymorphism	Behaviour varies by type?	Each type implements the operation
7	Pure Fabrication	No domain class fits?	Invent a helper (repository)
8	Indirection	Avoid direct coupling?	An intermediate object
9	Protected Variations	Shield from change?	Stable interface around instability

The problem it solves

A god UI/`Manager` class plus anemic data, or logic scattered at random; GRASP gives explainable reasons ("`total()` is on `Sale` — Expert").

How it works

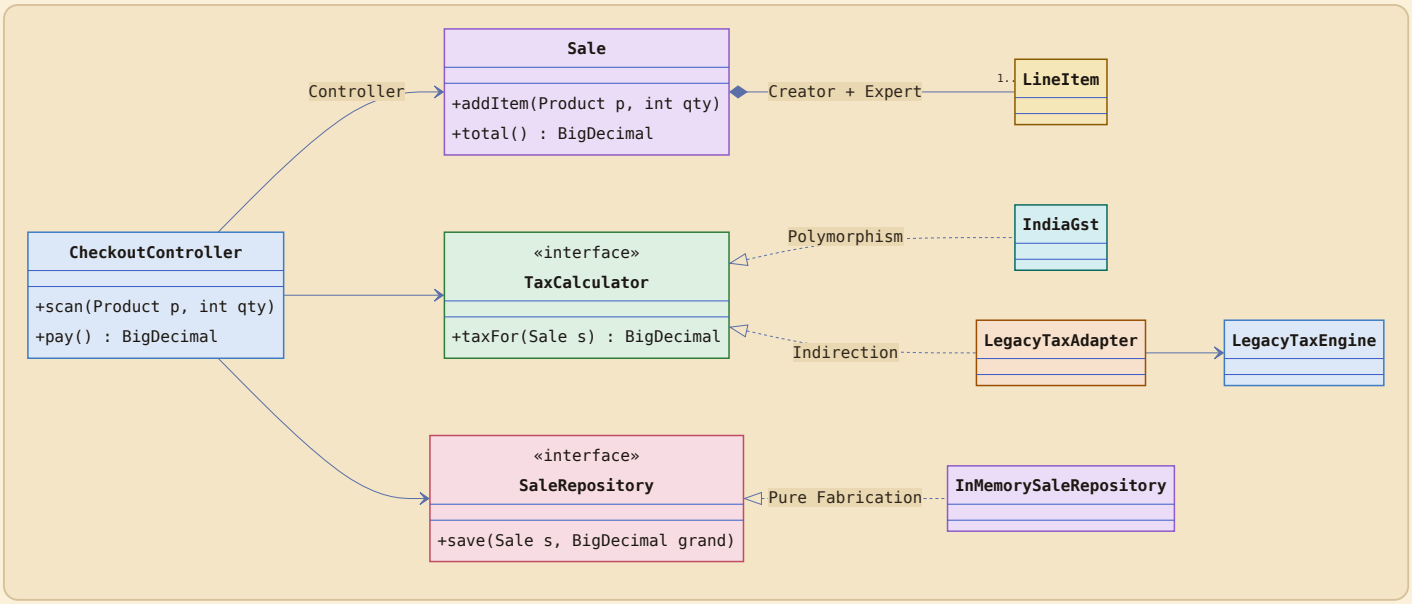
List system events → **Controller** each → **Expert** for what's known/computed → **Creator** for each object → judge by **coupling/cohesion** → **Polymorphism** for type variation, **Protected Variations** via **Indirection** for instability → **Pure Fabrication** where no domain object fits.

Java example — before/after (all nine in one checkout)

Before, `CheckoutScreen` creates items, totals them, hard-codes regional tax and builds SQL. After (each Javadoc names its guideline):

```java
// File: code/lld/ch05/grasp/Main.java
        /** INFORMATION EXPERT: a line item knows its product and quantity, so it computes its own subtotal.
*/
        record LineItem(Product product, int qty) {
            BigDecimal subtotal() { return product.price().multiply(BigDecimal.valueOf(qty)); }
        }
        // ...
            void addItem(Product p, int qty) { items.add(new LineItem(p, qty)); }        // Creator
        // ...
        /** PROTECTED VARIATIONS: tax rules vary by region/vendor -> hide them behind a stable interface. */
        interface TaxCalculator { BigDecimal taxFor(Sale sale); }
        // ... IndiaGst, FlatUsSalesTax (POLYMORPHISM); LegacyTaxAdapter (INDIRECTION); SaleRepository (PURE
FABRICATION)
            BigDecimal pay() {
                BigDecimal grand = current.total().add(tax.taxFor(current)).setScale(2, RoundingMode.HALF_UP);
                repo.save(current, grand);
                return grand;
            }
```

Full runnable code: code/lld/ch05/grasp/ (see the Full Edition). Grand totals: `IndiaGst` 377.60, `FlatUsSalesTax` 342.40, `LegacyTaxAdapter` 384.00.



16.1 Information Expert

What it is

The class that **has the information** does the job: `LineItem.subtotal()`, `Sale.total()`.

The problem it solves

Managers pulling data out (*Feature Envy*).

How it works

Job → needed information → who holds it → partial experts cooperate.

Trade-offs

Behaviour with data; blindly applied, puts SQL in `Sale`.

When not to use it

When it adds persistence, UI or network work to domain objects.

Common mistakes

Everything in services via getters; entities that save or email themselves.

Real-world examples

`String.length()`, `LocalDate.isLeapYear()`; a `Cart` totalling itself.

In the interview

"`total()` is on `Order` because it owns the line items."

Self-check

Q: Who computes a line's subtotal? → A: `LineItem`.

TL;DR

The one with the data does the work.

16.2 Creator

What it is

B creates A if B *contains*, *records*, *closely uses* or *has initialising data* for A: `Sale` creates `LineItem`s.

The problem it solves

Random `new` spreads coupling to A's constructor.

How it works

Prefer the container; complex or varying creation → Factory/Builder (Chapter 8).

Trade-offs

Low-coupling for plain objects; unsuitable for complex creation.

When not to use it

Complex creation; DI-managed objects.

Common mistakes

UI code doing `new LineItem(...)`.

Real-world examples

`HashMap` nodes; `Document` → `Paragraph`; `ExecutorService` via `ThreadFactory`.

In the interview

"`Sale.addItem` creates the item because the sale contains it." *vs Factory?* A Factory is a Pure Fabrication for complex creation.

Self-check

Q: Strongest reason to create? → A: Containment.

TL;DR

The container creates.

16.3 Controller

What it is

The first **non-UI** object receiving a **system operation**: **facade** (`Register`, `ParkingLotSystem`) or **use-case** (`CheckoutController`) controller.

The problem it solves

Keeps logic out of UI so web, mobile and tests share it.

How it works

Assign each system event to a controller that **delegates**.

Trade-offs

Clean split; a controller doing real work gets **bloated** — split by use case.

When not to use it

Trivial programs.

Common mistakes

Fat controllers with pricing and SQL; a `JFrame` computing totals.

Real-world examples

`@RestController` → `@Service`; `DispatcherServlet`; `TripService.requestRide()`.

In the interview

"`ParkingLotController.park()` receives the event and delegates."

Self-check

Q: Should it calculate tax? → A: No; it delegates.

TL;DR

A thin coordinator per system event.

16.4 Low Coupling

What it is

Evaluative: prefer fewer dependencies; the controller knows only `TaxCalculator` and `SaleRepository`.

The problem it solves

Rippling changes, no reuse.

How it works

Compare alternatives: interfaces, fewer collaborators, one direction, no cycles, DI.

Trade-offs

Stable dependencies (JDK) are harmless; chasing zero bloats indirection.

When not to use it

Don't decouple from standard libraries.

Common mistakes

Package cycles; UI knowing the DB.

Real-world examples

Microservices over APIs; JDBC drivers.

In the interview

"I depend on `PaymentGateway`, not Stripe's SDK." *Zero coupling?* Impossible.

Self-check

Q: Is `java.util.List` bad coupling? → A: No; it's stable.

TL;DR

Fewer, more stable dependencies.

16.5 High Cohesion

What it is

Evaluative: each class's responsibilities focused and related.

The problem it solves

`Utils`/`Manager` classes changing for many reasons.

How it works

"Is this related to what the class does?" Larman's levels: *very low* (many different areas) → *low* (one big complex job) → *moderate* (light unrelated jobs) → *high* (related jobs, collaborating for the rest — the goal).

Trade-offs

More, simpler files; improves alongside coupling.

When not to use it

Rarely — coarse-grained remote APIs, on purpose.

Common mistakes

Utils dumping grounds; entities sending emails.

Real-world examples

`LocalDate`, `Pattern`, Spring Data repositories.

In the interview

"I split `BookingService` into seat locking, pricing, payment."

Self-check

Q: A low-cohesion smell? → A: A `Utils`/`Manager` class.

TL;DR

One related set of things per class.

16.6 Polymorphism

What it is

When behaviour **varies by type**, each type implements a shared operation (`taxFor`), no type `switch`.

The problem it solves

Repeated type checks; new types edit every branch.

How it works

Declare the varying operation on an interface; implement per variant.

Trade-offs

Easy to add types, hard to add operations (Visitor, Chapter 11, flips it).

When not to use it

Closed type sets gaining operations — `sealed` + pattern-matching `switch`.

Common mistakes

Variation that doesn't exist; leftover `instanceof` chains.

Real-world examples

`Comparator`, `InputStream`, `Runnable`/`Callable`, card/UPI/wallet payments.

In the interview

"Each `Vehicle` answers `requiredSpotSize()`."

Self-check

Q: Which change is easy? → A: Adding a type.

TL;DR

Each type implements the varying behaviour.

16.7 Pure Fabrication

What it is

A **non-domain** class invented to keep cohesion high and coupling low: `SaleRepository`.

The problem it solves

Persistence, logging, IDs fit no domain class cleanly.

How it works

When Expert drags technical work into a domain class, create a focused service (`IdGenerator`).

Trade-offs

Good cohesion and reuse; overuse makes an anemic domain.

When not to use it

A domain object is a natural home — Expert first.

Common mistakes

Domain rules in "Manager" services.

Real-world examples

DAOs, Jackson `ObjectMapper`, `Clock`, Strategy/Adapter/Factory objects.

In the interview

"`TicketRepository` keeps persistence out of `Ticket`."

Self-check

Q: Is `Sale` a fabrication? → A: No; `SaleRepository` is.

TL;DR

Invent a helper when no domain class fits.

16.8 Indirection

What it is

An **intermediate object** so two components aren't directly coupled: `LegacyTaxAdapter` before `LegacyTaxEngine`.

The problem it solves

Direct dependencies on external or volatile parts.

How it works

Interface plus adapter, facade, mediator or proxy.

Trade-offs

Decoupling vs an extra hop (Wheeler's "another level of indirection" — except too many levels).

When not to use it

Stable classes that change together.

Common mistakes

Pure pass-through layers.

Real-world examples

Adapter/Facade/Mediator/Proxy; `DriverManager`; API gateways; brokers.

In the interview

"`SmsSender` with a `TwilioSmsSender` adapter." *vs PV?* Indirection is the mechanism, PV the goal.

Self-check

Q: GoF forms? → A: Adapter, Facade, Mediator, Proxy (and Bridge).

TL;DR

A middleman between strangers.

16.9 Protected Variations

What it is

Wrap **predicted variation** in a **stable interface** (`TaxCalculator`). Larman: Parnas's *information hiding* and OCP as a responsibility rule; Demeter is one mechanism.

The problem it solves

One vendor/regulation/format change forcing edits everywhere.

How it works

Find variation points → stable interface → variants behind it.

Trade-offs

Protection costs; protect only predicted variation.

When not to use it

Stable code (YAGNI).

Common mistakes

Leaky interfaces (`StripeCharge` from `PaymentGateway`).

Real-world examples

JDBC, JPA, SLF4J, `ServiceLoader`, `FileSystem`.

In the interview

"Payment providers vary, so `PaymentGateway` is my protected variation."

Self-check

Q: Closest SOLID principle? → A: OCP (also DIP).

TL;DR

Wrap what's unstable behind a stable interface.

GRASP in the interview (whole set)

> **30-second answer:** "Expert and Creator place most behaviour, a thin Controller takes system events, Polymorphism and Protected Variations handle variation, Indirection and Pure Fabrication keep technical concerns out of the domain, and Low Coupling/High Cohesion judge every choice." *vs GoF?* Strategy = Polymorphism + PV; Adapter = Indirection; Repository = Pure Fabrication.

Self-check (GRASP)

1. Q: Which two are evaluative? → A: Low Coupling, High Cohesion.
2. Q: Guideline for inventing `IdGenerator`? → A: Pure Fabrication.

TL;DR

Work where the data is, containers create, thin controllers, polymorphism behind stable interfaces, fabricate when needed.

---

17. Domain-Driven Design (DDD) Essentials

What it is

Eric Evans's approach (*Domain-Driven Design*, 2003) for making code mirror the business (the **domain**). **Strategic design** splits a large business into **bounded contexts**, each with its own model and **ubiquitous language**; **tactical design** gives the building blocks inside one context: **entities**, **value objects**, **aggregates**, **domain events**, **repositories** (plus domain services). The other principles shape classes; DDD decides *which classes exist and where the boundaries are*. Analogy: "patient" means different things at a hospital's reception, operating theatre and billing — each department is a bounded context with its own file.

Term	Meaning
Ubiquitous language	One shared vocabulary for experts and code: `place()`, not `setStatus(2)`.
Bounded context	Boundary where one model and language hold (Sales "order" vs Shipping "consignment"); often one team/service.
Context map / ACL	How contexts relate; an **anti-corruption layer** translates another (or legacy) model into yours.
Entity	Defined by **identity** that survives changes; equality by id.
Value object	Defined by **values**; immutable; equality by value (`Money`, `Address`).
Aggregate / root	Cluster changed as one unit; only the **root** is reachable from outside and guards invariants.
Domain event	Past-tense business fact (`OrderPlaced`) others react to.
Repository	Collection-like load/save **per aggregate root** (Chapter 11, §11.7.3).

The problem it solves

One model for every department (big ball of mud); developer vs business vocabulary drift; **anemic models** (data bags + rules in services — Tell, Don't Ask violated at scale); no consistency boundary, so invariants are checked in some paths only; teams coupled through each other's tables.

How it works

Strategic: learn the language with domain experts → draw contexts where words change meaning (Catalog, Sales, Payments, Shipping) → choose relationships (published events/API, shared kernel sparingly, conformist, **ACL** for legacy/third party) → map contexts to teams and services (see the Strangler Fig notes for splitting a monolith).

Tactical: prefer **value objects** over primitives (immutable, self-validating); entities get an id value object and intention-revealing methods. **Aggregate rules** (Vaughn Vernon, 2013): protect true invariants inside; keep aggregates **small**; **reference other aggregates by id**; **one aggregate per transaction**, others follow via events (eventual consistency). Publish events **after commit** via a transactional outbox (see Saga, Event-Driven). Repositories only for roots. Application services load → call one method → save. Separate read models = **CQRS** (notes); events as the source of truth = Event Sourcing (§11.7.6).

Java example — before/after

Before: public fields, `double` total, string status — a line with qty −5 is added after "SHIPPED". After:

```java
// File: code/lld/ch05/ddd/Main.java
        record Money(long paise) {
            Money { if (paise < 0) throw new IllegalArgumentException("Money cannot be negative"); }
            // ... rupees(), plus(), times()
        }
        static final class Order {                                  // aggregate root
            // ... id, customer (CustomerId, not Customer), lines, events, status
            void addItem(Sku sku, Money unitPrice, int quantity) {
                requireDraft("add items");
                // ... at most 10 units per SKU, then add an OrderLine entity
            }
            void place() {
                requireDraft("place");
                if (lines.isEmpty()) throw new IllegalStateException("Cannot place an empty order");
                status = OrderStatus.PLACED;
                // ... events.add(new OrderPlaced(id, customer, quantities, total()))
            }
            @Override public boolean equals(Object o) { return o instanceof Order other &&
other.id.equals(id); } // entity: by identity
        }
        // ... InMemoryOrderRepository.save(order): store, then publish order.pullEvents()
        // ... ShippingTranslator (ACL): OrderPlaced -> Consignment(reference, parcels, weightGrams)
```

Full runnable code: code/lld/ch05/ddd/ (see the Full Edition).

```
total=0.30000000000000004 status=SHIPPED lines=3 (a line was added after shipping, with qty -5)
rejected: At most 10 units of TEA-250 per order
   shipping context: created Consignment[reference=SO-1001, parcels=1, weightGrams=920]
rejected: Cannot add items: order is PLACED
two Order objects with id 1001 equal? true   (entity: identity decides)
```

Trade-offs

Business-shaped code, clear team/service boundaries, clear transaction unit, testable rich models — versus expert time, more types, eventual consistency to design, translation at borders, ORM friction.

When not to use it

CRUD/admin apps, one small team with one small model, algorithm-heavy technical problems. Value objects and ubiquitous names are still worth borrowing.

Common mistakes

Anemic aggregates with setters; huge aggregates; one repository per table; publishing events before commit; one shared model across services; DDD as folder names; primitive obsession.

Real-world examples

Microservices split by bounded context (Sam Newman's *Building Microservices*; Microsoft's eShopOnContainers uses aggregates and domain events); `java.time` and `BigDecimal` as value objects; Axon aggregates (CQRS notes); in this book, the grocery POS (Ch 19) and shipping locker (§14.7).

In the interview

30-second answer: "Bounded contexts with their own language, connected by events or APIs and translated at the border; inside, entities for identity, immutable value objects like Money, small aggregates whose root guards invariants, one aggregate per transaction, domain events for the rest, one repository per root." *Keep two aggregates consistent?* Event via outbox → handler or saga.

Self-check

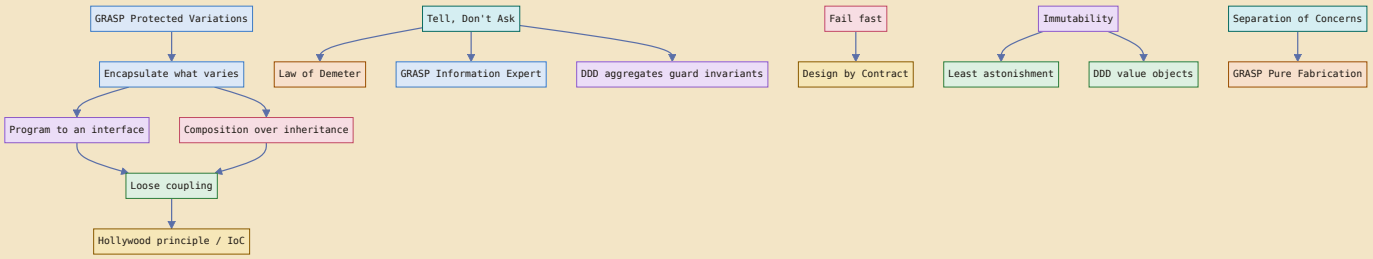
1. Q: Entity or value object? → A: Tracked over time by identity → entity; replaced when changed → value object.

2. Q: Why `CustomerId` inside `Order`? → A: Small, independent aggregates; no loading/locking the customer.
3. Q: ACL? → A: Translates a foreign model into ours at the border.

> **TL;DR**

Speak the business's language; bounded contexts with their own models; entities, value objects, small aggregates guarding invariants, domain events, repositories per root.

Chapter summary — how the principles fit together



Principle	Main goal	Smell it fixes	Key tool
DRY	One home per rule	Drifting duplicates	Value objects
KISS	Simplicity	Over-engineering	`switch` expressions
YAGNI	Only what's needed	Speculative generality	Delete unused code
Demeter	Hide structure	Train wrecks	Intent methods
Tell, Don't Ask	Behaviour with data	Anemic model	Command methods
Composition	Flexible reuse	Fragile base class	Forwarding
Program to interface	Substitutability	Hard-wired deps	Interfaces + DI
Encapsulate varies	Isolate change	Type switches	Strategy, lambdas
Loose coupling	Independent change	God orchestrators	Observer, events
Hollywood / IoC	Framework owns flow	Duplicated skeletons	Template Method
SoC	One concern per unit	Spaghetti	Layers, MVC
POLA	Predictability	Hidden side effects	Honest names, `Optional`
Fail fast	Early errors	Distant NPEs	Guard clauses
Immutability	Safe sharing	Aliasing, races	Records, `List.copyOf`
DbC	Clear obligations	Magic error values	Pre/postconditions
GRASP	Place responsibilities	God classes	Expert, Creator…
DDD	Model the business	Anemic model, big ball of mud	Value objects, aggregate roots, events

Tensions: DRY vs loose coupling; YAGNI vs encapsulate-what-varies/OCP; Demeter vs cohesion (Middle Man); Information Expert vs SoC (domain computes, fabrications persist); DRY vs bounded contexts (share events and ids, not models).

Chapter self-check

1. Q: `order.getCustomer().getAddress().getCity()` in a shipping service? → A: Demeter (and TDA); ask `order.shippingZone()`.
2. Q: A `find` that inserts a row? → A: Least Astonishment (and CQS).
3. Q: A record holding a list — truly immutable how? → A: `List.copyOf` in the compact constructor.

4. Q: DDD: why is `Money` a value object but `Order` an entity? → A: Money is defined by its values and replaced; an order keeps its identity while it changes.

TL;DR (chapter)

One home per rule, keep it simple, build only what's needed, let objects own behaviour, depend on small interfaces and compose, let frameworks call you, separate concerns, don't surprise, fail early, prefer immutable values, state contracts, use GRASP to decide who does what — and DDD to decide which classes exist.

Chapter 6 — Clean Code (Study Edition)

> **Study Edition.** Every topic of Chapter 6, without the padding. Code blocks are excerpts from the real files (`// ...` marks trimmed parts); full listings and outputs are in the Full Edition (`README.md`). Code lives in `code/lld/ch06/<topic>/`.

Design principles decide *which classes exist*; **clean code** is what happens *inside* them. In machine-coding rounds interviewers read your code line by line, so clean code often separates "hire" from "no hire". This chapter follows Robert C. Martin's *Clean Code* (2008) in order (the case studies are summarised in §13), then adds a **code review checklist**, an **error-handling strategy** and **testing strategies**.

Key terms. **Code smell** — a symptom that *usually* signals a deeper problem. **Refactoring** — changing structure, not behaviour (Chapter 7). **Level of abstraction** — `processPayroll()` high, `sb.append(',')` low. **Side effect** — any change besides the return value. **Test double** — a stand-in dependency (dummy, stub, spy, mock, fake). **TDD** — failing test, pass, refactor, in short cycles.

What is clean code, and why bother?

What it is

Code that reads like well-written prose, does one thing, has tests, has no duplication, expresses intent and looks written by someone who **cares**. Working definition: *intent obvious to the next reader, easy to change safely.* Analogy: an organised kitchen anyone can cook in.

The problem it solves

Mess slows teams more and more until productivity nears zero and a "grand redesign" reproduces the mess. We read code far more than we write it (often quoted >10:1). "Clean it later" rarely happens — *later equals never* (LeBlanc's law).

How it works

Treat readability as a requirement; follow the **Boy Scout Rule** — *leave the code a little cleaner than you found it*; write tests so you can clean without fear; make it work, then make it right, in small steps.

Trade-offs

Slightly slower the first time; faster on every later read and change. Throwaway scripts differ.

When not to use it

Unreadable code is never a goal, but avoid **gold-plating** clear, stable code.

Common mistakes

"Working" = "done"; blaming deadlines (the mess slows you immediately); big-bang rewrites instead of continuous small improvements.

Real-world examples

Google's readability reviews and certification; linters/formatters in CI (Checkstyle, SpotBugs, SonarQube, google-java-format).

In the interview

> **30-second answer:** "Obvious intent, cheap to change: intention-revealing names, small functions, exceptions not codes, immutable values, a few tests — and time to refactor once it works."

Self-check

Q: Boy Scout Rule? → A: Leave the code a little cleaner than you found it.

TL;DR

Obvious intent + safe to change; clean a little every time.

---

1. Meaningful Names

What it is

A good name answers **why it exists, what it does, how it's used**. If it needs a comment, it isn't good enough. Analogy: "Kitchen — glasses — fragile" vs "Box 7".

The problem it solves

Bad names force a **mental map** ("`d` is elapsed days, 4 means done") that every reader pays for, and misreadings cause bugs.

How it works — the rules

Rule	Bad	Good
Reveal intention	`int d;`	`int elapsedTimeInDays;`
Avoid disinformation	`accountList` for a `Set`; `l` / `O`	`accounts`
Meaningful distinctions (no number series, noise words)	`a1, a2`; `ProductInfo` vs `ProductData`	`source, destination`
Pronounceable	`crtdTs`	`createdAt`
Searchable	`7`, `e`	`MAX_LOGIN_ATTEMPTS`
No encodings (Hungarian, `m_`, `I` prefix)	`strName`, `IShapeFactory`	`name`, `ShapeFactory`
No mental mapping	`r`	`normalizedPath`
Classes = nouns	`Manager`, `Data`, `Info`	`Customer`, `AddressParser`
Methods = verbs; `get`/`is`/`has`	`data()`	`postPayment()`, `isPosted()`
Named static factories	`new Money(23)`	`Money.ofRupees(23)`
Don't be cute	`nuke()`	`deleteAll()`
One word per concept	`fetch` / `retrieve` / `get`	pick one
Don't pun	`add` = sum and append	`append`, `insert`
Solution-domain names (technical)	—	`OrderRepository`, `RetryPolicy`
Problem-domain names (business)	—	`Premium`, `Claim`
Add meaningful context	lone `state`	`shippingState` or an `Address`
No gratuitous context	`AcmeShopCustomerAddress`	`Address`

The longer the scope, the longer the name — `i` suits a 3-line loop.

Java example — before/after

Before: `class DtaRcrd102 { String strNm; int i1; int i2; boolean flg; }` and `Mgr.getThem()` filtering `x.i1 == 4 && x.i2 > 18000 && !x.flg`. After:

```java
// File: code/lld/ch06/names/Main.java
        record Task(String title, TaskStatus status, Duration timeSpent, boolean archived) {
            boolean isCompleted() { return status == TaskStatus.DONE; }          // boolean method reads as a question
            boolean tookLongerThan(Duration limit) { return timeSpent.compareTo(limit) > 0; }
        }
        // ...
            private static final Duration LONG_RUNNING = Duration.ofHours(5);  // named constant, not a magic number
            // ...
            List<Task> findCompletedLongRunningTasks() {                        // says exactly what it returns
```

Full runnable code: code/lld/ch06/names/ (see the Full Edition). The rule now reads aloud; a status code became an enum; `Temperature.ofCelsius(25)` is unambiguous.

Trade-offs

A few keystrokes (IDEs autocomplete). A very long name may mean the function does too much — split, don't abbreviate.

When not to use it

Conventional short names: `i`/`j` in tiny loops, `e` for exceptions, `x`/`y`, `T`/`K`/`V`.

Common mistakes

Not renaming ("everyone knows `tmp2`"); author-only abbreviations (`calcAmtPrc`); `customer`/`client`/`user` for one concept.

Real-world examples

JDK: `isEmpty()`, `computeIfAbsent()`, `List.of()`, `Duration.ofMinutes()`; Google Java Style casing.

In the interview

30-second answer: "Code should read like the requirement: nouns for classes, verbs for methods, booleans as questions, named constants, enums instead of status codes, one word per concept." *`I` prefix on interfaces?* Not in Java; name the role (`PaymentGateway`), implementations specifically (`RazorpayGateway`).

Self-check

1. Q: `ProductInfo` and `ProductData` together? → A: Noise words, no real distinction.
2. Q: A naming pun? → A: One word for two meanings.

TL;DR

Reveal intent, pronounceable, searchable, one word per concept.

---

2. Functions

What it is

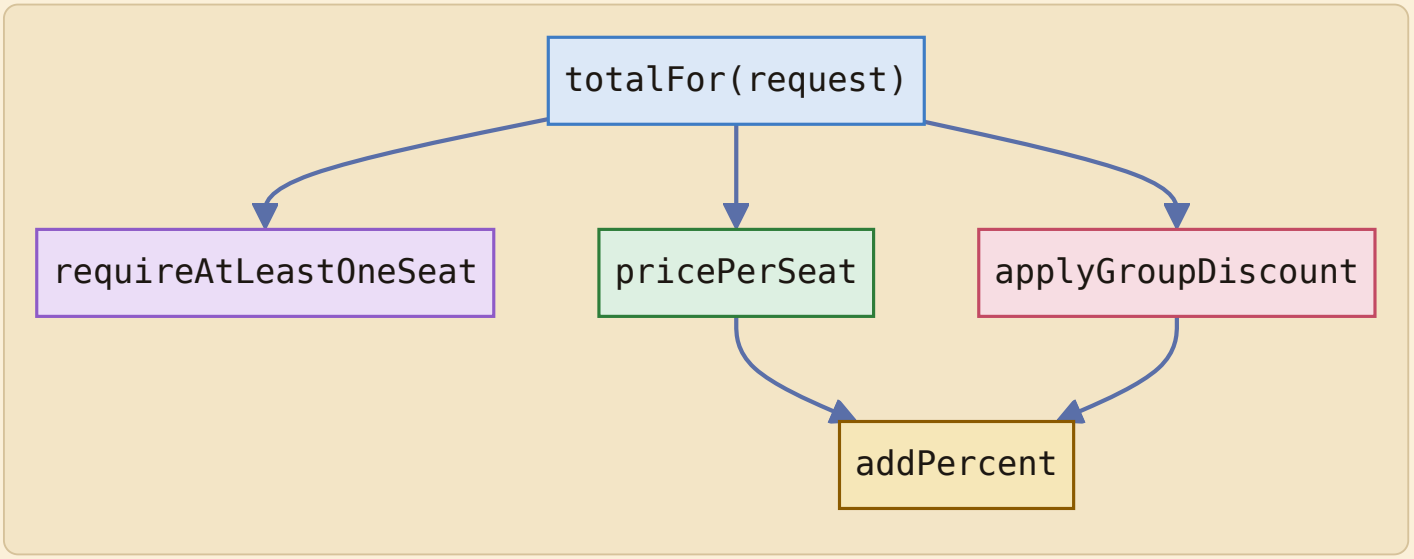
Clean functions are **small**, **do one thing**, stay at **one level of abstraction**, take **few arguments**, have **no hidden side effects**, and either **do** or **answer** — never both. Analogy: a recipe of named steps, each with its own sub-recipe.

The problem it solves

Long functions mix detail levels, hide bugs, can't be named, reused or tested in pieces; flags and output arguments confuse; hidden side effects surprise; error codes clutter callers.

How it works — the rules

1. **Small, then smaller** — rarely 20 lines; block bodies ideally one call; indentation ≤ 1–2 levels.
2. **Do one thing** — nothing meaningful left to extract; "sections" mean several things.
3. **One level of abstraction** per function. 4. **Stepdown Rule** — read top-down, each function followed by its callees.
4. **Switch statements** — only **once**, buried low (a factory creating polymorphic objects); else polymorphism or enum data.
5. **Descriptive names** beat enigmatic ones and comments.
6. **Few arguments** — zero, one, two; avoid three; more needs justification. One-arg forms: question, transform, event. **Flag arguments** mean two things — split or use an enum. **Argument objects** (`book(BookingRequest)`); varargs count as one; **verbs and keywords** (`copyFromTo(source, target)`).
7. **No side effects** — hidden ones create **temporal coupling**. 9. **No output arguments** — `letter.addSignature()`.
8. **Command-Query Separation** — change state *or* answer (`if (enable("darkMode"))` is ambiguous).
9. **Exceptions, not error codes**. 12. **Extract try/catch** — error handling is one thing. 13. **No "error code magnet"** enum.
10. **DRY**. 15. **Structured programming** matters for big functions; small ones may return early (never `goto`).
11. **How to get there** — rough draft with tests, then refine, keeping tests green.



Java example — before/after

Before: `static int book(String seatType, int count, boolean weekend, List<String> log)` — returns `-1` on error, switches on seat strings, mutates the caller's `log`, writes a global `SESSION`. After:

```java
// File: code/lld/ch06/functions/Main.java
        /** Argument object: three related arguments become one concept. */
        record BookingRequest(SeatType seat, int seats, DayType day) {}
        // ...
            int totalFor(BookingRequest request) {                      // level 1: the story
                requireAtLeastOneSeat(request);
                int total = pricePerSeat(request) * request.seats();
                return applyGroupDiscount(total, request.seats());
            }
        // ...
            void book(BookingRequest request) {
                // ... price it and append to history (a command: returns nothing)
            }
            List<String> history() { return List.copyOf(history); }
```

Full runnable code: code/lld/ch06/functions/ (see the Full Edition).

Flag → `DayType`; type switch → `SeatType` enum carrying its price, parsed once at the boundary; `-1` → `InvalidBookingException` with a message; output argument and `SESSION` → state owned by `BookingService`; **CQS**:

`book` is a void command, `history` a query (returning a created ID is an accepted exception); `bookFromForm` does only error translation.

Trade-offs

Many small functions	Few large functions
Nameable, testable, reusable	Everything in one place
Need good names; over-splitting scatters logic	Hard to understand and test; mixed levels

When not to use it

Don't split a clear 8-line function into 8 one-liners; size is rarely a performance excuse (the JIT inlines small methods).

Common mistakes

Boolean parameters in public APIs; state-changing getters; catching deep and returning `null`; names that say one thing while the body does three.

Real-world examples

`list.sort(comparator)` returns `void`; `Iterator.next()`/`Queue.poll()` are known CQS exceptions; `Files.readString`.

In the interview

> **30-second answer:** "Small, single-purpose functions at one level, stepdown order; enums instead of flags, parameter objects, CQS, and descriptive exceptions instead of error codes." *How small?* One clear name and level — often 3–10 lines, rarely over ~20.

Self-check

1. Q: When is a function "one thing"? → A: No meaningful function can be extracted from it.
2. Q: When is a `switch` acceptable? → A: Once, buried in a factory, never repeated.

TL;DR

Small, one thing, one level, top-down, ≤2–3 args, no flags, CQS, exceptions not codes.

---

3. Comments

What it is

At best a necessary evil — compensation for failing to express ourselves in code. Code changes; comments drift and **lie**. **Explain yourself in code first**. Analogy: a "Bridge closed" sign long after it reopened.

The problem it solves

Comments as a crutch for unclear code, redundant comments, outdated misleading ones.

How it works

`// free shipping if big and domestic` + `if (order.total() >= 499 && !order.isInternational())` becomes `if (order.qualifiesForFreeShipping())`.

Good comments: legal (licence header); informative (what a regex matches); explanation of intent (*why*); clarification (obscure library values); warning of consequences ("not thread-safe", "billed per call"); TODO (with an owner, reviewed); amplification ("the trim is critical"); Javadoc on public APIs.

Bad comments: mumbling; redundant (`i++; // increment i`); misleading; mandated (`@param a the a`); journal; noise; scary noise; comment instead of a function/variable; position markers; closing-brace comments;

attributions; **commented-out code** (delete — git remembers); HTML; nonlocal information; too much information; inobvious connection; function headers; Javadoc on non-public code.

Java example — before/after

Before: journal comment, `@param a the a`, a comment instead of a name over a five-part condition, `return true; // return true`, commented-out code, `} // end eligible`. After:

```
// File: code/lld/ch06/comments/Main.java
        static boolean isEligibleForPremiumLoan(Applicant a) {
            return isWorkingAge(a) && hasGoodCredit(a) && earnsEnough(a);
        }
        // ...
        // GOOD: informative comment -- clarifies what a regex matches (format: 10-character PAN, e.g.
ABCDE1234F)
        private static final Pattern PAN_FORMAT = Pattern.compile("[A-Z]{5}[0-9]{4}[A-Z]");
```

Full runnable code: code/lld/ch06/comments/ (see the Full Edition). The kept comments are the good kinds: legal header, informative, intent, public Javadoc, a billing warning and an owned `TODO(loans-team)`.

Trade-offs

Cheap to write, expensive to maintain: a few precise "why" comments help; many "what" comments hurt.

When not to use it

"No comments" isn't the goal: public libraries need Javadoc; complex algorithms deserve an explanation of approach and why.

Common mistakes

Updating code but not its comment; keeping commented-out code; TODO lists nobody reviews.

Real-world examples

JDK Javadoc (`HashMap` iteration order, `ConcurrentHashMap` weakly consistent iterators); SonarQube flags commented-out code; linters reject TODOs without tickets.

In the interview

> **30-second answer:** "Expressive code over comments — named functions and constants replace most 'what' comments. I keep why, warnings, legal, owned TODOs and public contracts, and delete commented-out code."

Self-check

1. Q: Commented-out code? → A: Delete it.
2. Q: Why do comments lie? → A: Code changes, comments don't.

TL;DR

Say it in code; comment only why, warnings, legal and public contracts.

---

4. Formatting

What it is

Layout communicates what belongs together and what matters. One team style, applied automatically.

The problem it solves

Random layout makes readers decode structure before logic, and produces noisy diffs.

How it works

Vertical: small files (many Java projects ~200 lines, few >500); **newspaper metaphor**; **openness** (blank lines between concepts); **density** (related lines together); **distance** (locals near first use, instance variables at the top, **caller above callee**, conceptual affinity); **ordering** high-level first. **Horizontal:** short lines (Martin 120, Google 100); spaces around assignments, none between name and `(`; **no column alignment**; indentation shows scope; make **dummy scopes** (empty loop bodies) visible. **Team rules:** one style, enforced by a formatter in CI.

Java example — before/after

Before, `Playlist` is crammed onto a few lines with fields scattered and aligned in columns. After (newspaper order):

```
// File: code/lld/ch06/formatting/Main.java
            // 1. Fields at the top, together (vertical density for closely related lines).
            // ...
            public String next() {
                if (songs.isEmpty()) {
                    return null;
                }
                String song = displayTitle(cursor);    // caller ABOVE callee (dependent functions close
together)
                cursor = (cursor + 1) % songs.size();
                return song;
            }

            // 4. Private helpers just below the functions that use them.
            private String displayTitle(int index) {
```

Full runnable code: code/lld/ch06/formatting/ (see the Full Edition). Behaviour is identical; only reading speed changes.

Trade-offs

Almost none once automated; agree the rules once.

When not to use it

Generated code (e.g. Protobuf) — exclude it from formatters.

Common mistakes

Mixing formatting and logic changes in one commit; personal styles; column alignment.

Real-world examples

google-java-format, Spotless, Prettier, gofmt, Black; EditorConfig.

In the interview

30-second answer: "Related lines together, concepts separated, callers above callees, fields at the top, one automated team style."

Self-check

Q: Where does a private helper go? → A: Just below its caller.

TL;DR

Headline first, related things close, one automated style.

---

5. Objects and Data Structures

What it is

An **object** hides data and exposes behaviour (`Order.isMetroDelivery()`); a **data structure** exposes data with no meaningful behaviour (record `Address`, DTO). **Data abstraction** isn't getters/setters on private fields — it's exposing the *essence* (`fuelPercentRemaining()`, not tank capacity and litres).

The problem it solves

Hybrids (public data *and* behaviour) make both new functions and new types hard. Knowing the difference tells you which change is easy.

How it works — the data/object anti-symmetry

	Procedural (data structures + functions)	OO (objects)
Add a **function** (`perimeter`)	**Easy** — one function	**Hard** — every class
Add a **type** (`Triangle`)	**Hard** — every function	**Easy** — one class

"Everything is an object" is a myth: expect new types → objects; new operations → data + functions (or Visitor). Java 21 `sealed` + records + pattern-matching `switch` make the procedural style compiler-checked. **Demeter** applies to objects (no `order.getCustomer().getAddress().city()`), not strictly to data structures. **DTOs** carry pure data across boundaries (bean form is framework ceremony, not encapsulation). **Active Record** — a DTO with `save()`/`find()`; keep business rules out.

Java example — before/after

```
// File: code/lld/ch06/objects/Main.java
            static double perimeter(Shape s) {                // NEW FUNCTION: easy -- no shape class changes
                return switch (s) {
                    case Square sq -> 4 * sq.side();
                    case Circle c -> 2 * Math.PI * c.radius();
                };
            }
        // ...
        record Triangle(double base, double height) implements Shape {   // NEW TYPE: easy -- nothing else
changes
        // ...
            boolean isMetroDelivery() { return shipTo.isMetro(); }        // tell the order what you need
```

Full runnable code: code/lld/ch06/objects/ (see the Full Edition). The train wreck became `order.isMetroDelivery()` while the `Address` record freely exposes `city`; `OrderDto` is a pure data carrier.

Trade-offs

See the anti-symmetry table: choose by which change you expect more.

When not to use it

Don't push behaviour into DTOs crossing process boundaries — keep them dumb and versioned.

Common mistakes

Reflexive getters/setters; hybrids; business logic in JPA entities that are also API bodies.

Real-world examples

Records; `java.awt.Point` (public fields); `LocalDate` (object); Spring `@RequestBody` DTOs; ActiveRecord / JPA entities.

In the interview

> **30-second answer:** "Objects hide data and expose behaviour; data structures expose data. OO makes new types easy, procedural makes new functions easy. Records for DTOs, rich objects for domain logic, no hybrids or train wrecks." *Pattern for adding operations in OO?* Visitor.

Self-check

1. Q: Which style makes new shape types easy? → A: OO.
2. Q: A hybrid? → A: Exposed data plus significant behaviour.

TL;DR

Pick objects or data structures by the change you expect; no hybrids.

---

6. Error Handling

What it is

Important — but if it **obscures the logic** it's wrong. Keep the happy path readable; make failures explicit, informative and hard to ignore.

The problem it solves

Error codes clutter callers and get forgotten; `null` returns become distant NPEs; checked exceptions leak into every signature; vague messages slow debugging.

How it works — the rules

1. **Exceptions, not codes.** 2. **Write the `try-catch-finally` first** (a transaction); prefer **try-with-resources**.
2. **Unchecked exceptions** — checked ones break OCP by rippling through signatures; checked only when callers must recover.
3. **Context** — what, which values, why; keep the **cause**. 5. **Define exceptions by the caller's needs**; **wrap third-party APIs**.
4. **Special Case pattern** (Fowler) — an object for expected absence (`NoMembership`). 7. **Don't return `null`**. 8. **Don't pass `null`**.

Java example — before/after

Before: `transfer` returns `E_NO_ACCOUNT` (which account?), `history` returns `null`, `loyaltyPoints` returns a `null` `Integer` that crashes on unboxing. After:

```java
// File: code/lld/ch06/errors/Main.java
        static final class InsufficientFundsException extends BankingException {
            InsufficientFundsException(String id, long requested, long available) {
                super("account '" + id + "' has " + available + " but " + requested + " was requested");
            }
        }
        // ...
        enum NoMembership implements Loyalty { INSTANCE; public int points() { return 0; } public String
tier() { return "NONE"; } }
        // ...
            void send(String phone, String text) {
                try { client.push(phone, text); }
                catch (IOException e) { throw new BankingException("could not send SMS to " + phone, e); }
            }
        // ...
            List<String> history(String id) { return List.of(); }                          // empty, never
null
            Optional<Long> findBalance(String id) { return Optional.ofNullable(balances.get(id)); }
```

Full runnable code: code/lld/ch06/errors/ (see the Full Edition).

Output includes `account 'B' has 100 but 900 was requested` and `could not send SMS to 000123 (cause: socket timeout to sms-gw-3)`. An unchecked `BankingException` hierarchy with context; the vendor's checked `IOException` wrapped with its cause; a try-with-resources `AuditFile` is closed even when an exception is thrown.

Trade-offs

Choice	Pro	Con
Unchecked exceptions	Clean signatures, OCP-friendly	No compiler reminder; document them
Checked exceptions	Compiler-enforced	Signature ripple; often swallowed
`Optional` return	Absence in the type	Not for fields/parameters
Special Case	No checks in callers	Can hide absence that *should* be noticed
Result/Either	Expected failures as values	Verbose in Java (§16)

When not to use it

Exceptions aren't control flow (don't throw to exit a loop); expected, frequent outcomes (row validation) may be cleaner as result values (§16).

Common mistakes

`catch (Exception e) {}`; log-and-rethrow at every layer; losing the cause (`new MyException(e.getMessage())`); `null` from collection methods.

Real-world examples

Spring `DataAccessException` translating `SQLException`s (e.g. `DuplicateKeyException`); `Collections.emptyList()`, `Optional.empty()`; `requireNonNull`; `UncheckedIOException`.

In the interview

30-second answer: "Unchecked domain exceptions with context, not codes or null; wrap vendor exceptions; empty collections, `Optional` or special cases for absence; reject nulls at entry; try-with-resources." *Checked?* Only for conditions callers must recover from.

Self-check

1. Q: Why are error codes bad? → A: Clutter, easily ignored, no context.
2. Q: Why keep the cause? → A: The original stack trace and message.

TL;DR

Exceptions not codes; unchecked with context; wrap vendors; no null in or out; always clean up.

---

7. Boundaries

What it is

Where your code meets code you don't control (libraries, frameworks, other teams, even general JDK types like `Map`). Keep it at the edge behind interfaces shaped for *your* needs.

The problem it solves

Providers want broad APIs; users want focused ones. A raw `Map<String, Session>` passed around lets anyone `clear()` it and forces everyone to know the key format; library changes break every usage site.

How it works

1. **Wrap boundary types** — expose only what the app needs (`SessionRegistry.open/activeSession/close`); keep the boundary type private.
2. **Learning tests** — small tests of the library as you'll use it; free documentation, re-run on upgrades.
3. **Use code that doesn't exist yet** — define the interface you wish you had (`FraudCheck`), use a fake, later write an **Adapter**.
4. Keep boundary knowledge in few places.

Java example — before/after

Before, an admin tool calls `sessions.clear()` on the shared map. After:

```
// File: code/lld/ch06/boundaries/Main.java
        static final class SessionRegistry {
            private final Map<String, Session> byToken = new ConcurrentHashMap<>();   // boundary type stays
private

            void open(String token, Session session) { byToken.put(token, session); }
            Optional<Session> activeSession(String token, Instant now) {
                return Optional.ofNullable(byToken.get(token)).filter(s -> now.isBefore(s.expiresAt()));
            }
            void close(String token) { byToken.remove(token); }
        // ...
        interface FraudCheck { boolean isSuspicious(String userId, long amountPaise); }
```

Full runnable code: code/lld/ch06/boundaries/ (see the Full Edition). `JdkLearningTest` pins real facts: `HashMap` allows one `null` key, `ConcurrentHashMap` rejects it, `Map.of` rejects duplicates, `"a,b,,".split(",")` drops trailing empties (use limit `-1`).

Trade-offs

A class and some delegation — worth it once the boundary type is used in more than one place or may change.

When not to use it

A `Map` used inside one method is fine; the rule is about *passing* boundary types around.

Common mistakes

Returning vendor types (`StripeCharge` from `PaymentGateway`); discovering library behaviour in production.

Real-world examples

DDD **Anti-Corruption Layer**; SLF4J; Spring `RestClient`/`WebClient`; Pact contract tests.

In the interview

> **30-second answer:** "Third-party code stays at the edge behind my interfaces; learning tests when adopting a library; for a missing API, my own interface, a fake, and an adapter later."

Self-check

Q: A learning test? → A: A test that pins down how a third-party API behaves.

TL;DR

Foreign code at the edge behind your interfaces; learn it with tests.

---

8. Unit Tests

What it is

Small automated tests of one unit of behaviour in isolation. Test code is **as important as production code** — it's what lets you change production code without fear.

The problem it solves

No tests → fear of change → rot. *Dirty* tests → break constantly, get abandoned → rot anyway.

How it works

Three laws of TDD (a ~30-second cycle): (1) no production code without a failing test; (2) write only enough test to fail (not compiling counts); (3) write only enough code to pass. Beck's coarser **Red → Green → Refactor**. **Clean tests** enable the "-ilities". Readability: **Build–Operate–Check** (Arrange–Act–Assert, Given–When–Then); a **domain-specific testing language** (`cart("TEA", 3)`); a **dual standard** (clean, not necessarily efficient). **One concept per test** (several asserts are fine for one concept).

F.I.R.S.T.	Meaning	In our code
Fast	Milliseconds	In-memory doubles
Independent	No order dependence	Fresh instance per test
Repeatable	Same result anywhere	Fixed `Clock`
Self-validating	Pass/fail automatically	Assertions, not `println`
Timely	Just before the code	TDD

Java example — a JUnit-free runner, clean tests and a dirty one

`MiniTest` (~60 lines) finds `@Test` methods by reflection, runs each on a **new instance** and reports PASS/FAIL. `Checkout` has four collaborators behind interfaces (`PriceCatalog`, `CouponRepository`, `ReceiptSender`, `AuditLog`) plus a `Clock`.

```
// File: code/lld/ch06/testing/CheckoutTest.java
    private static final Clock FIXED_CLOCK = Clock.fixed(Instant.parse("2026-09-28T10:00:00Z"),
ZoneOffset.UTC); // Repeatable
    // ...
    @Test void expired_coupon_is_ignored() {
        var coupons = new FakeCoupons().add(new Checkout.Coupon("OLD", 50, TODAY.minusDays(1)));
        long total = checkoutWith(coupons, new DummyAuditLog()).quote(cart("TEA", 1), "OLD");
        assertEquals(12_000L, total);
    }
```

Full runnable code: code/lld/ch06/testing/ (`Checkout`, `CheckoutTest`, `CheckoutIntegrationTest`, `BeforeStyleTest`, `MiniTest`, `Test`, `Main`) (see the Full Edition).

`test1` "passes" but proves nothing: three concepts, real clock, printing, meaningless name. `CheckoutTest` methods test one concept each, are named as specifications, use Given/When/Then and a fixed clock; `CheckoutIntegrationTest` wires real in-memory components. Output: `TOTAL: 8 passed, 0 failed`.

Trade-offs

Well-tested: fearless refactoring, executable documentation, but test code to maintain. Untested: faster first write, fear of change and knowledge in heads.

When not to use it

TDD suits spikes and pure UI layout less; don't unit-test trivial getters or code you don't own (learning tests instead).

Common mistakes

Testing implementation details; shared mutable fixtures; `Thread.sleep` or real clock/network; names like `test1`.

Real-world examples

JUnit 5 (new instance per test method by default), AssertJ, Mockito; `java.time.Clock` with `Clock.fixed`.

In the interview

> **30-second answer:** "Small, fast, independent tests — one concept each, Given/When/Then, doubles behind interfaces, an injected `Clock`. In a machine-coding round at least a `main` exercising key flows plus a few unit tests."

Self-check

1. Q: What's wrong with printing in a test? → A: Not self-validating.
2. Q: Repeatable time-based tests? → A: Inject a `Clock`; `Clock.fixed`.

TL;DR

Tests are first-class: TDD, one concept per test, Given/When/Then, F.I.R.S.T.

---

9. Classes

What it is

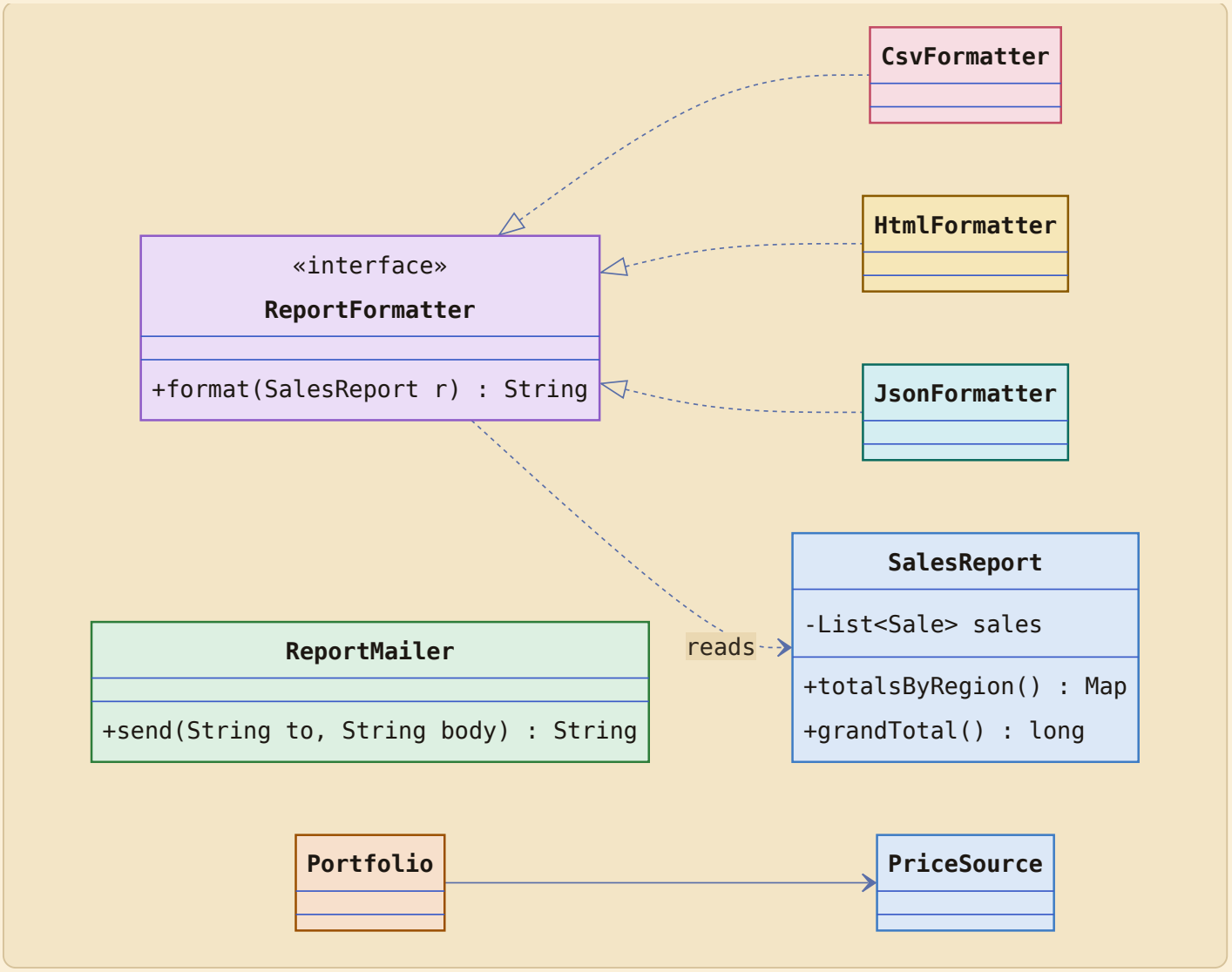
Standard **organisation**; **small** (by responsibilities); **one reason to change**; **cohesive**; **organised for change** so new features are new classes.

The problem it solves

God classes change for many reasons; growing `if/else` over formats reopens the same class repeatedly.

How it works

1. **Organisation**: public static constants → private static variables → private instance variables → public functions → private helpers after their caller.
2. **Encapsulation**: loosen for tests only as a last resort.
3. **Small** by responsibilities: if the name needs *Manager/Processor/Super* or "and/or/but", it does too much; describable in ~25 words.
4. **SRP**: many small classes beat a few large ones — same code, better organised.
5. **Cohesion**: maximal when every method uses every field; a subset of methods using a subset of fields wants its own class.
6. **Organise for change**: new features as new classes (OCP) — one `ReportFormatter` per format.
7. **Isolate from change**: depend on abstractions (`Portfolio` → `PriceSource`, DIP).



Java example — before/after

Before, `ReportManager` totals, renders CSV/HTML in a growing `if/else` and emails — three reasons to change. After:

```
// File: code/lld/ch06/classes/Main.java
        /** Organised for change: a new format is a NEW class; existing formatters are never reopened (OCP).
*/
        interface ReportFormatter { String format(SalesReport report); }
        // ...
        static final class JsonFormatter implements ReportFormatter {   // added later, nothing else edited
        // ...
        /** Isolating from change: Portfolio depends on an abstraction, not on a live stock-exchange client.
*/
        interface PriceSource { long pricePaise(String symbol); }
```

Full runnable code: code/lld/ch06/classes/ (see the Full Edition). `JsonFormatter` was added with no edits; `Portfolio` is tested with fixed prices.

Trade-offs

Many small classes: easy to understand and test, localised changes, need good names and packages. Few big: fewer files, every change touches the same class, merge conflicts.

When not to use it

Don't fragment a small cohesive class into micro-classes that always change together.

Common mistakes

`*Manager` / `*Helper` / `*Utils`; splitting only by technical layer; everything `public` for tests.

Real-world examples

`java.time` (`LocalDate`, `Duration`, `DateTimeFormatter`…) vs old `Date` / `Calendar`; `JdbcTemplate` delegating to `RowMapper`.

In the interview

> **30-second answer:** "Small by responsibility, highly cohesive; when a feature would add a branch I split so it's a new class; interfaces isolate volatile details."

Self-check

1. Q: How is class size measured? → A: Responsibilities.
2. Q: Maximal cohesion? → A: Every method uses every field.

TL;DR

Small, cohesive, open for extension, depending on abstractions.

---

10. Systems

What it is

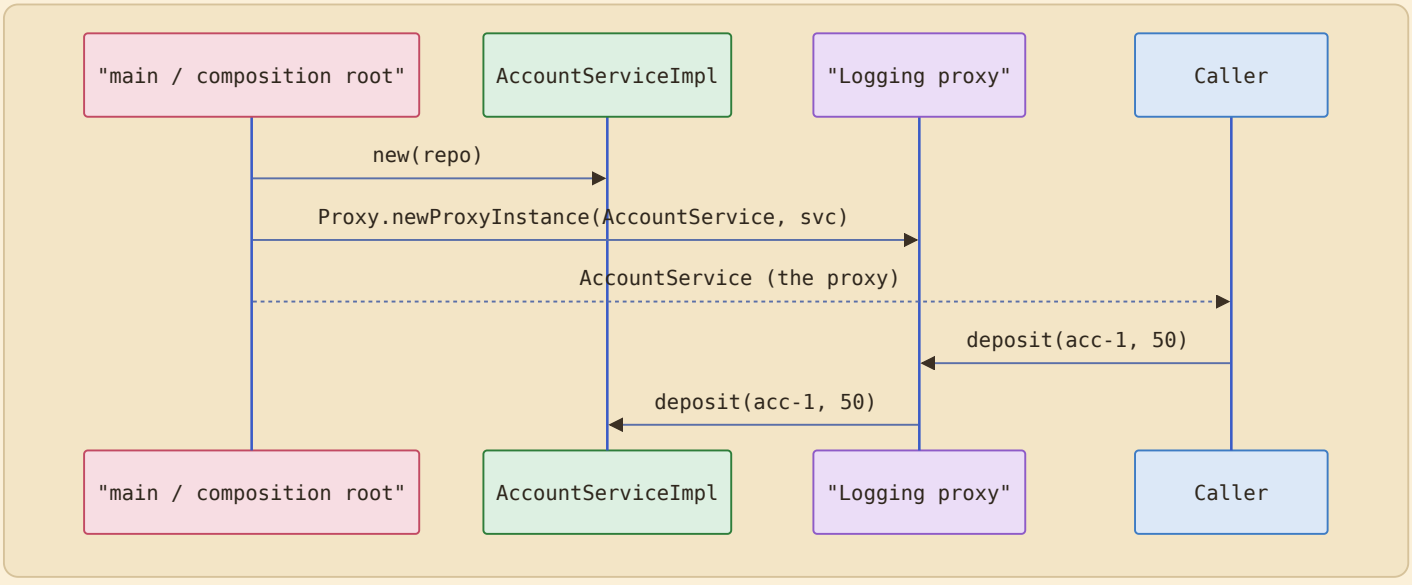
Separate startup (construction and wiring) from runtime logic, use **Dependency Injection**, handle **cross-cutting concerns** in one place, grow the architecture **incrementally**.

The problem it solves

Lazy init in business code (`if (service == null) service = new MyServiceImpl()`) hard-wires classes and blocks testing; logging copied into every method is noisy.

How it works

1. **Separate construction from use**: the **composition root** in `main` builds everything; **Abstract Factory** when the app decides *when* to create; **DI** (IoC for dependencies; constructor injection preferred).
2. **Scale up incrementally**. 3. **Cross-cutting concerns** via **dynamic proxies**, **AOP** (Spring `@Transactional`, `@Cacheable`), **AspectJ**, decorators, filters.
3. **Test-drive the architecture** with POJOs; 5. **postpone decisions** to the last responsible moment; 6. **use standards wisely**; 7. **DSLs** make code read like the domain.



Java example — before/after

Before, `AccountServiceImpl` lazily builds its repository and repeats `LOG enter/exit` in every method. After:

```
// File: code/lld/ch06/systems/Main.java
            private final AccountRepository repo;
            AccountServiceImpl(AccountRepository repo) { this.repo = repo; }
        // ...
        static <T> T withLogging(Class<T> type, T target) {
            // ... an InvocationHandler logs, calls method.invoke(target, args), and on failure
            // rethrows e.getCause() (the real exception, not the reflection wrapper)
            return (T) Proxy.newProxyInstance(type.getClassLoader(), new Class<?>[]{type}, handler);
        }

        /** Composition root: the ONLY place that knows concrete classes and wires them together (the "main"
side). */
        static AccountService buildApplication() {
            AccountRepository repo = new InMemoryAccountRepository();
            AccountService core = new AccountServiceImpl(repo);
            return withLogging(AccountService.class, core);
        }
```

Full runnable code: code/lld/ch06/systems/ (see the Full Edition). Logging is written once; tests wire the core with another repository.

Trade-offs

DI + composition root + proxies	Self-constructing objects
Testable, swappable, one wiring place; concerns in one place	Simple for tiny programs
Framework "magic"; proxies only intercept calls *through* the proxy	Duplicated, inconsistent concerns

Spring gotcha: self-invocation of a `@Transactional` method bypasses the proxy — no transaction.

When not to use it

Small programs and interviews need no DI framework — manual constructor injection in `main` is expected.

Common mistakes

Service Locator calls in business code; `@Autowired` field injection (hidden, blocks immutability); framework annotations everywhere.

Real-world examples

Spring/Spring Boot, Guice, Dagger, Jakarta CDI; Spring AOP via JDK proxies or CGLIB; servlet filters, `HandlerInterceptor`.

In the interview

30-second answer: "A composition root in `main` builds and wires everything; classes get dependencies through constructors; logging and transactions live once in a decorator, proxy or aspect." *DI vs Service Locator?* DI pushes (explicit); a locator pulls (hidden).

Self-check

1. Q: Composition root? → A: The one place objects are constructed and wired.
2. Q: Three ways to handle cross-cutting concerns? → A: Dynamic proxies, AOP/AspectJ, decorators (also filters).

TL;DR

Build in one place and inject; cross-cutting concerns in proxies/aspects; grow incrementally.

---

11. Emergence — Kent Beck's Four Rules of Simple Design

What it is

Good design **emerges** from four rules. *Clean Code* (Jeff Langr's chapter) orders them: **1. runs all the tests, 2. no duplication, 3. expresses intent, 4. minimises classes and methods. Beck's own order** (per Fowler's "BeckDesignRules"): **passes the tests, reveals intention, no duplication, fewest elements** — the middle two get swapped and reinforce each other. In interviews, quote Beck's order and mention the swap.

The problem it solves

An ordered checklist that pushes code toward SRP, DIP and good structure.

How it works

1. **Runs all the tests** — testability itself drives small, decoupled classes (DI, interfaces).
2. **Refactor with rules 2–4**: remove duplicate lines and implementations (`isEmpty()` as `size() == 0`), higher-level duplication via **Template Method**; be **expressive** (names, small units, pattern names, tests as docs, *care*); **minimise** classes and methods — lowest priority, loses to SRP and no-duplication.

Java example — before/after

Before, `calcIN` and `calcUS` copy one algorithm with magic `18`/`45` and `15`/`30`, plus a purposeless `LeaveCalcStrategyProvider`. After:

```
// File: code/lld/ch06/emergence/Main.java
        abstract static class LeavePolicy {
            /** The shared algorithm, written once (rule 2: no duplication). */
            final int earnedLeaveDays(Employee e) {
                if (e.onProbation()) return 0;
                return Math.min(daysPerYear() * e.monthsWorked() / 12, maxCarryForwardDays());
            }
            protected abstract int daysPerYear();              // what varies, with intention-revealing names
(rule 3)
            protected abstract int maxCarryForwardDays();
        }
```

Full runnable code: code/lld/ch06/emergence/ (see the Full Edition). `LeavePolicyTest` proves old and new behaviour match (rule 1); the useless interface is gone (rule 4).

Trade-offs

Rules 2–4 can conflict; resolve by priority or find a design satisfying both.

When not to use it

Always applicable — a mindset, not a process.

Common mistakes

Refactoring without tests; minimising classes at SRP's expense.

Real-world examples

XP teams; `AbstractList` removing duplication like our Template Method.

In the interview

30-second answer: "Beck's rules in priority order: passes the tests, reveals intention, no duplication, fewest elements (*Clean Code* swaps the middle two). Get it working with a quick test, then refactor duplication and names, and add no class that doesn't earn its place."

Self-check

Q: Lowest-priority rule? → A: Minimal classes and methods.

TL;DR

Tests pass → no duplication → clear intent → nothing extra.

---

12. Concurrency

What it is

A **decoupling strategy** separating *what* is done from *when*. Clean concurrency keeps concurrency code small, separate and built on library tools.

The problem it solves

Myths: it always speeds things up (only with waiting or spare cores); design doesn't change; containers make it irrelevant. Reality: overhead, non-repeatable bugs, a different design.

How it works — defence principles

1. **SRP** — concurrency code separate. 2. **Limit shared data**. 3. **Use copies**. 4. **Independent threads**. 5. **Know your library** (`ConcurrentHashMap`, `ExecutorService`, atomics, `CountDownLatch`/`Semaphore`) — not `wait/notify`. 6. **Execution models** (Chapter 13): producer–consumer, readers–writers, dining philosophers. 7. **Dependencies between synchronised methods** — client- or server-based locking. 8. **Small synchronised sections**. 9. **Plan shutdown** (`shutdown()`, `awaitTermination()`, `shutdownNow()`). 10. **Testing**: one-off failures are bugs; non-threaded code first; pluggable, tunable thread counts; more threads than cores; several platforms; jiggling (jcstress).

Vocabulary: **bound resources**, **mutual exclusion**, **starvation**, **deadlock**, **livelock** (Chapters 12–13; *Clean Code* Appendix A goes deeper).

Java example — before/after

Before, four threads do `theCount++` on a shared `int` — a read-modify-write race that loses updates. After:

```
// File: code/lld/ch06/concurrency/Main.java
                    tasks.add(() -> {
                        Map<String, Integer> local = new HashMap<>();       // thread-confined: no sharing, no
locks
                        for (int r = 0; r < repeats; r++)
                            for (String page : PAGES) countWords(page).forEach((w, c) -> local.merge(w, c,
Integer::sum));
                        return local;
                    });
                // ...
                for (Future<Map<String, Integer>> f : pool.invokeAll(tasks))
                    f.get().forEach((w, c) -> total.merge(w, c, Integer::sum));   // Future.get() safely hands
over each result
                return Map.copyOf(total);                                          // publish an immutable
copy
            } finally {
                pool.shutdown();                                                   // clean shutdown is part
of the design
                if (!pool.awaitTermination(5, TimeUnit.SECONDS)) pool.shutdownNow();
            }
```

Full runnable code: code/lld/ch06/concurrency/ (see the Full Edition).

Before: `count(the) = 182526 expected 320000 -> lost 137474 updates`; after, every thread count matches exactly. The racy count varies per run (and is occasionally right — why such bugs escape tests). The fix separates

pure `countWords` from threading, confines data per task, merges once, and is tested with 1, 4 and 16 threads.

Trade-offs

Confinement + merge: no locks or races, scales, but extra memory and a merge step. Shared state + locks: simple for small cases, but contention, deadlock risk, every access must be synchronised.

When not to use it

No concurrency without a measured need; prefer parallel streams (pure CPU) or Java 21 virtual threads (blocking I/O) over manual threads.

Common mistakes

Unsynchronised shared `HashMap`/`ArrayList`; check-then-act (`containsKey` then `put` — use `putIfAbsent`/`computeIfAbsent`); unshut executors; holding locks while calling unknown code.

Real-world examples

`java.util.concurrent` (Doug Lea), `ForkJoinPool`; MapReduce/Spark; Node.js and Redis event loops.

In the interview

> **30-second answer:** "Concurrency separate from business logic; minimal shared mutable state — confinement, immutability, merging results; `java.util.concurrent` over raw threads; planned shutdown; tests with more threads than cores." *Why isn't `count++` safe?* Read, add, write — updates get lost; use `AtomicInteger`/`LongAdder` or a lock.

Self-check

1. Q: What does concurrency decouple? → A: What from when.
2. Q: A one-off failure in concurrent code? → A: Treat it as a real threading bug.

TL;DR

Separate it, share little, use the library, small locks, plan shutdown, test hard.

---

13. Successive Refinement and the Case Studies (Chapters 14–16: how clean code is actually written)

These are case studies showing the *process*, not new rules.

Chapter 14 — Successive Refinement (an `Args` parser): write it dirty, then clean it — never stop at dirty; stop adding features once the code rots; refactor in tiny tested steps; absorb variation by design (one class per argument type behind a "marshaler" interface — OCP); gather error handling into one exception class.

Chapter 15 — JUnit Internals (`ComparisonCompactor`, by Beck and Gamma — already good): drop encoding prefixes (`fExpected` → `expected`, N6); encapsulate and prefer positive conditionals (`canBeCompacted()`, G28/G29); rename functions hiding a second job (`formatCompactedComparison`, N7/G30); make temporal coupling explicit (G31); name `+1`/`-1` boundary arithmetic (G33). Even good code can be left cleaner.

Chapter 16 — Refactoring SerialDate (JCommon): first make it work — coverage rose from ~50% to ~92% and exposed real bugs; then make it right — rename (`DayDate`), enums for int constants (`Month`, J3), move methods to their data (G14/G17), an **Abstract Factory** so the base class doesn't know its subclass (G7), delete dead code, duplication and noise (G9, G5, C3); tests green throughout.

TL;DR: make it work, then make it right, in small tested steps.

---

14. Smells and Heuristics (the checklist)

What it is

Clean Code's catalogue of coded **smells and heuristics** (C1, G5, N1…), extending Fowler's smells (Chapter 7). Hints, not laws.

The problem it solves

You must *recognise* problems fast; codes give a shared review vocabulary ("G28 — encapsulate that conditional").

How it works — the catalogue, grouped (names listed; meaning in our words)

Comments (C): C1 inappropriate information (history, authors → VCS); C2 obsolete comment; C3 redundant comment; C4 poorly written comment; C5 commented-out code (delete).

Environment (E): E1 build requires more than one step; E2 tests require more than one step.

Functions (F): F1 too many arguments; F2 output arguments; F3 flag arguments; F4 dead function.

General (G):

Theme	Heuristics
Correctness and safety	G2 obvious behaviour unimplemented (least surprise); G3 incorrect behaviour at the boundaries (test edges); G4 overridden safeties (warnings off, skipped tests); G21 understand the algorithm; G26 be precise (no floating-point currency; nulls, concurrency, ties); G31 hidden temporal couplings (make order explicit); G33 encapsulate boundary conditions (`lastIndex = size - 1`)
Duplication and dead weight	**G5 duplication** ("one of the most important rules" — use methods, polymorphism, Template Method, Strategy); G9 dead code; G12 clutter
Structure and placement	G1 multiple languages in one file; G6 code at wrong level of abstraction; G7 base classes depending on derivatives; G8 too much information (small interfaces); G10 vertical separation; G13 artificial coupling; G14 feature envy; G17 misplaced responsibility; G18 inappropriate static; G22 make logical dependencies physical; G35 configurable data at high levels; G36 avoid transitive navigation (Demeter)
Readability and intent	G11 inconsistency; G15 selector arguments; G16 obscured intent; G19 explanatory variables; G20 names say what functions do (`applyDiscountPercent(5)`); G23 polymorphism over if/switch ("one switch"); G24 conventions; G25 named constants; G27 structure over convention; **G28 encapsulate conditionals** (`isRefundable(order)`); G29 avoid negative conditionals; G30 one thing; G32 don't be arbitrary; G34 one level of abstraction

Java (J): J1 wildcard imports to shorten lists (dated — Google style forbids them; follow your team); J2 don't inherit constants (static imports); J3 enums over `int` constants.

Names (N): N1 descriptive; N2 right level of abstraction (`notify(recipient)`, not `sendSms(phone)`); N3 standard nomenclature (`...Decorator`); N4 unambiguous; N5 long names for long scopes; N6 no encodings; N7 names describe side effects (`getOrCreateUser`).

Tests (T): T1 insufficient tests; T2 use a coverage tool (JaCoCo); T3 don't skip trivial tests; T4 an ignored test is a question about an ambiguity; T5 test boundary conditions; T6 exhaustively test near bugs; T7 patterns of failure are revealing; T8 coverage patterns are revealing; T9 tests should be fast.

Java example — several heuristics on one function

Before: `static double calc(Parcel p, boolean express)` with `double` money, a double negative `!(p.membership() != Membership.PRIME)`, a tangled `&& !p.fragile() == false` condition, magic numbers and a flag. After:

```java
// File: code/lld/ch06/smells/Main.java
        static BigDecimal shippingCost(Parcel p, Speed speed) {
            BigDecimal baseFee = isPrime(p) ? BigDecimal.ZERO : BASE_FEE;
            BigDecimal handlingFee = needsSpecialHandling(p) ? SPECIAL_HANDLING_FEE : BigDecimal.ZERO;
            BigDecimal weightCharge = BigDecimal.valueOf(p.weightKg()).multiply(RATE_PER_KG);   // explanatory
variables
            BigDecimal cost = baseFee.add(handlingFee).add(weightCharge).multiply(speed.multiplier);
            if (isPrime(p) && speed == Speed.EXPRESS) cost = cost.multiply(PRIME_EXPRESS_DISCOUNT);
            return cost.setScale(2, RoundingMode.HALF_UP);                          // round once,
explicitly, to paise
        }

        private static boolean isPrime(Parcel p) { return p.membership() == Membership.PRIME; }   // positive
conditional
        private static boolean needsSpecialHandling(Parcel p) {                                  //
encapsulated conditional
            boolean heavy = p.weightKg() > HEAVY_KG;
            boolean fragileLongHaul = p.distanceKm() > LONG_HAUL_KM && p.fragile();
            return heavy || fragileLongHaul;
        }
```

Full runnable code: code/lld/ch06/smells/ (see the Full Edition). A "same" column over eight parcel/speed cases proves behaviour unchanged. Fixed: G29, G16/G28, G25, F3 (→ `Speed` enum), G19, N1/G20 (`calc` → `shippingCost`), G26 (`BigDecimal`, one `HALF_UP`).

Trade-offs

A checklist can be applied dogmatically; when items conflict, favour readability and correctness.

When not to use it

Don't block a critical fix on nits — log follow-ups.

Common mistakes

Gaming smell counts; following dated notes like J1 over your team's guide.

Real-world examples

SonarQube, PMD, Checkstyle, SpotBugs, Error Prone.

In the interview

30-second answer: "I look for duplication, long functions, flag/output arguments, magic numbers, negative or complex conditionals, feature envy, train wrecks, misplaced responsibilities, dead code and weak tests — fixed with small refactorings: extract method, introduce constant, replace conditional with polymorphism."

Self-check

1. Q: What does G28 ask? → A: Encapsulate conditionals in a named function.
2. Q: What does T4 say about a disabled test? → A: It marks an unresolved requirement ambiguity.

TL;DR

C/E/F/G/J/N/T as shared review vocabulary; fix with small tested refactorings.

---

15. Code Review Checklist (best practices)

What it is

A peer reading a change before merge to catch defects, share knowledge and keep consistency — and, in interviews, what you run over your own code before saying "done".

The problem it solves

Unreviewed changes bring bugs, security holes and inconsistency; unfocused reviews chase nits and miss real issues.

How it works — the checklist (from most to least important)

1. **Correctness** — meets the requirement; edge cases (empty, null, zero, max, duplicates, time zones, rounding); errors handled, resources closed; concurrency (shared state, check-then-act, thread-safe collections).
2. **Design** — fits the architecture; right class/layer (SRP, GRASP); DIP, OCP; no over-engineering (YAGNI) or duplication (DRY).
3. **Readability** — names, small functions, no magic numbers or flags; *why* comments; no commented-out code.
4. **Tests** — new behaviour covered; regression test per bug fix; F.I.R.S.T., one concept each.
5. **Security** — validation; no injection (parameterised queries); output encoding; no secrets in code/logs, PII masked; authorisation on sensitive operations.
6. **Performance and resources** — N+1 queries, repeated work, unbounded collections, missing pagination; timeouts; closing.
7. **Operability** — useful logs and metrics; feature flags; backwards-compatible APIs, messages, migrations.
8. **Style** — automated.

Reviewing well: small changes; the author describes what/why/how-tested and self-reviews; the reviewer is specific and kind, explains why, marks "nit:"; automate the mechanical.

Java example

No separate program: reviewing `Before.book` in §2 would flag F3, error codes, F2, the hidden `SESSION` side effect, G23 and G25 — exactly its *after* fixes.

Trade-offs

Slightly slower merges; defects caught early, knowledge spread. Too many reviewers or nit-heavy reviews slow teams without quality gains.

When not to use it

Emergency hotfixes may be reviewed after merge — but still reviewed.

Common mistakes

Style-only reviews; huge PRs; personal preferences as rules.

Real-world examples

GitHub PRs, GitLab MRs, Gerrit; Google's published review guidance; CODEOWNERS.

In the interview

> **30-second answer:** "Correctness first — requirements, edge cases, errors, concurrency — then design, readability, tests, security, performance; formatting left to tools; specific feedback, nits marked optional."

Self-check

Q: Why small PRs? → A: Large diffs get shallow reviews.

TL;DR

Correctness → design → readability → tests → security → performance → operability; automate style.

---

16. Error-Handling Strategy (best practices for real services)

What it is

§6 covered rules in code; a **strategy** is the system-wide policy: which exception types exist, where errors are translated, which are retried, how they're logged, what callers see.

The problem it solves

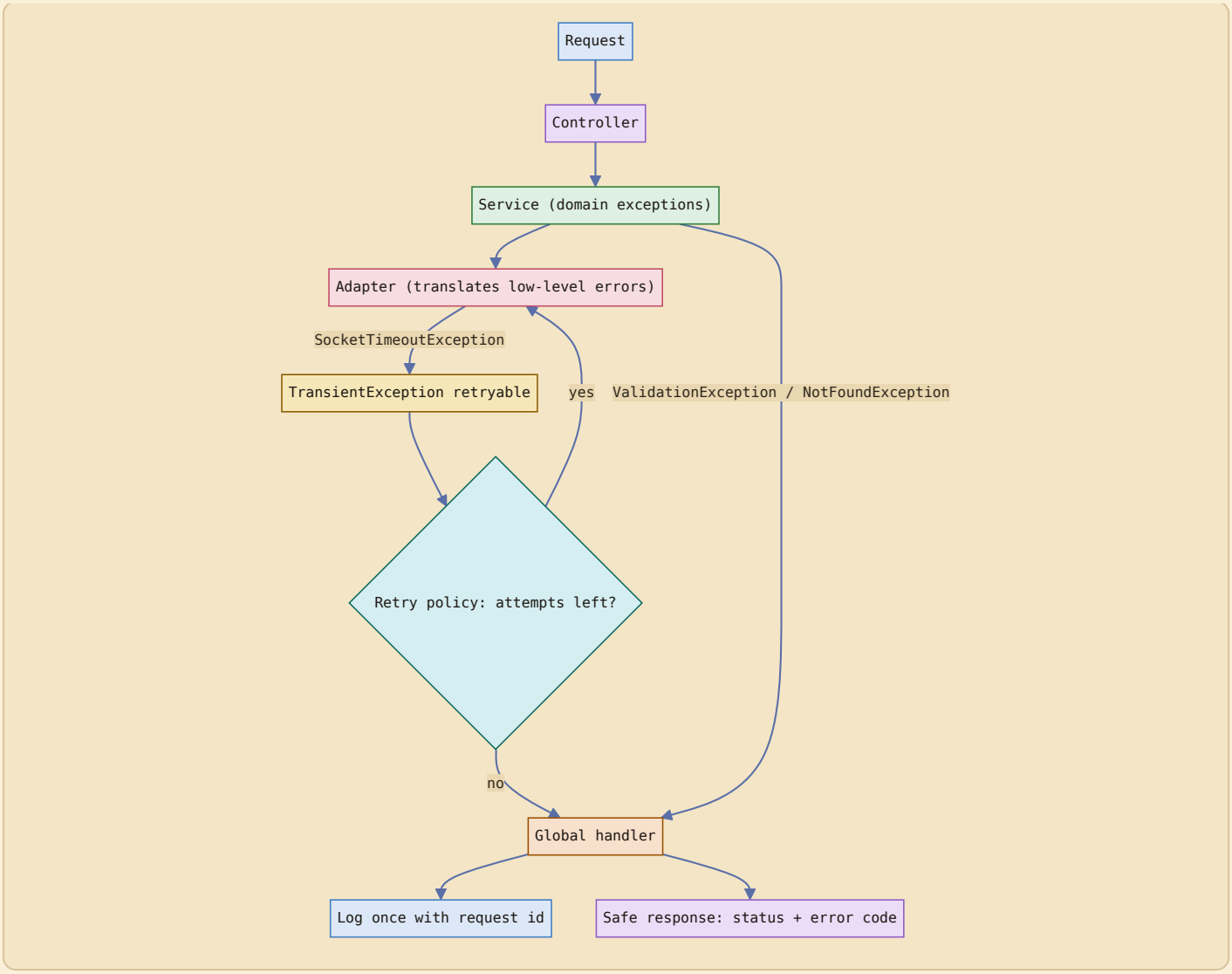
Without one: nulls, log-and-rethrow everywhere, stack traces to users, retried permanent failures overloading services.

How it works

1. **Classify errors:**

Class	Examples	Handling
Validation / client	bad input	Reject, 4xx, no retry
Business rule	insufficient funds, seat taken	Domain exception or result, 4xx (409)
Not found	unknown id	404 / empty `Optional`
Transient infrastructure	timeouts, 503, deadlock victim	Retry with backoff — only if **idempotent**
Permanent infrastructure	DB auth failure, misconfiguration	Fail fast, alert
Programming bugs	NPE, index out of bounds	Don't catch locally; generic 500, log, fix

2. **One exception hierarchy** with an error code and a *retryable* flag. 3. **Translate at boundaries** (adapters), keeping the cause. 4. **Retry only transient** errors: max attempts, **exponential backoff**, **jitter**, plus timeouts and circuit breakers. 5. **Result values for expected per-item failures** (collect all row errors). 6. **Global handler at the edge**: status, stable code, safe message, **log once** with a request id; never leak internals. 7. **Stay consistent**: transactions or compensation; idempotent handlers.



Java example

```
// File: code/lld/ch06/errorstrategy/Main.java
    static <T> T withRetry(Supplier<T> action, int maxAttempts, long initialBackoffMs) {
        long backoff = initialBackoffMs;
        for (int attempt = 1; ; attempt++) {
            try {
                return action.get();
            } catch (AppException e) {
                if (!e.retryable || attempt == maxAttempts) throw e;
                // ...
                sleep(backoff);
                backoff *= 2;                                   // exponential backoff (add jitter in
production)
            }
        }
    }
```

Full runnable code: code/lld/ch06/errorstrategy/ (see the Full Edition). req-1: a timeout is retried (10 ms, 20 ms) then succeeds; req-2: `NotFoundException` isn't retryable → 404; req-3: gives up after 3 attempts → 503; req-4: a bug → generic 500 with only a request id; a sealed `Result` (`Ok`/`Err`) collects all bulk-import row errors, then the request is rejected with 400.

Trade-offs

Choice	Benefit	Cost
Retries	Survive blips	Duplicate side effects unless idempotent; amplify load
Global handler	Consistent responses, log once	Must not swallow context
Result types	Explicit expected failures	Verbose in Java; needs a clear rule vs exceptions

When not to use it

Don't retry non-idempotent operations ("charge card") without an **idempotency key**; never retry validation or authorisation failures.

Common mistakes

Log-and-rethrow; catching `Exception` in business code; tight retry loops without backoff; 200 with an error body.

Real-world examples

Spring `@ControllerAdvice` / `@ExceptionHandler`; RFC 9457 Problem Details (`ProblemDetail`); Resilience4j; AWS SDK backoff with jitter; Stripe idempotency keys.

In the interview

> **30-second answer:** "Classify errors; one hierarchy with codes and a retryable flag; translate in adapters; retry only transient, idempotent calls with backoff and jitter; one global handler returns safe responses and logs once." *Why jitter?* To avoid a thundering herd.

Self-check

1. Q: Which errors are retried? → A: Transient ones, for idempotent operations.
2. Q: Where are errors logged? → A: Once, at the global handler, with a correlation id.

TL;DR

Classify, translate, retry transient + idempotent with backoff, results for expected failures, one global handler.

---

17. Testing Strategies (unit vs integration, test doubles, mocks)

What it is

Deciding *which kinds* of tests, *how many*, and *how to isolate* code, building on §8.

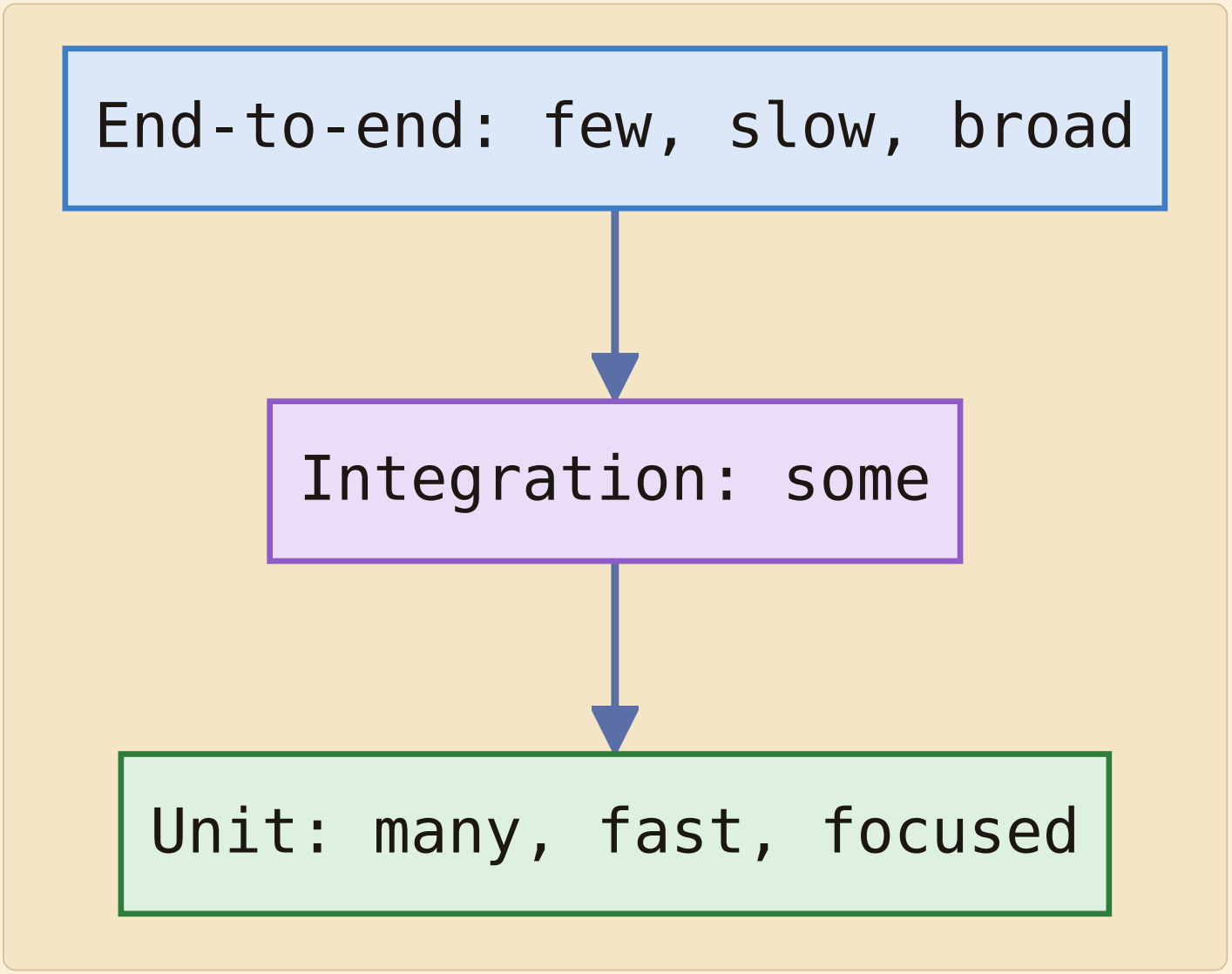
The problem it solves

Too few tests → production bugs; mostly slow E2E tests → flaky pipelines; over-mocking → tests pass while the system is broken.

How it works

Kind	Scope	Speed	Catches
Unit	One class, doubles for collaborators	ms	Logic errors
Integration	Real components together (DB, broker)	s	Wiring, queries, serialisation, config
Contract	Consumer–provider agreement	Fast	API mismatches
End-to-end	Whole system via UI/API	Slow	User-journey breakage
Others	Performance, security, exploratory, property-based, mutation	Varies	Non-functional issues, weak tests

Test Pyramid (Mike Cohn): many unit, some integration, few E2E — not the "ice-cream cone".



Test doubles (Meszaros, via Fowler):

Double	What it does	In our code
Dummy	Fills a parameter; never used	`DummyAuditLog` (throws if called)
Stub	Canned answers	`StubCatalog`
Spy	Stub that records calls	`SpySender.sent`
Mock	Pre-programmed expectations, verifies itself	`MockAuditLog.expect(...).verify()`
Fake	Working simplified implementation	`FakeCoupons`

State vs behaviour verification: prefer state (stubs/fakes); mocks for unobservable outgoing side effects. **Test** public behaviour, edge cases, error paths, every bug fix — not private methods or the framework. **Mocking:** don't mock types you don't own (wrap them) or value objects; many mocks means too many dependencies. **Coverage** (JaCoCo) is necessary, not sufficient; mutation testing (PIT) checks the tests.

Java example

The code is in §8 (`lld.ch06.testing`): `CheckoutTest` uses all five doubles, `CheckoutIntegrationTest` wires real in-memory parts, `BeforeStyleTest` shows what to avoid. The mock records calls in `actual` and its `verify()` asserts them against `expected`.

Trade-offs

More unit tests: fast, precise, may miss wiring bugs; over-mocking couples to implementation. More integration/E2E: higher confidence, slow, flaky, need environments.

When not to use it

Thin framework glue is better covered by an integration test; push edge cases down from E2E to unit tests.

Common mistakes

Mocking everything (even value objects or the class under test); leaving flaky tests unfixed; testing internal calls instead of outcomes; no error-path tests.

Real-world examples

JUnit 5, Mockito, AssertJ, Testcontainers, WireMock, Pact, PIT, JaCoCo; Spring Boot `@WebMvcTest`, `@DataJpaTest`.

In the interview

> **30-second answer:** "The test pyramid: many fast unit tests with doubles, fewer integration tests with real components (Testcontainers), a handful of E2E. State verification with stubs and fakes; mocks only for outgoing side effects; never mock types I don't own." *Mock vs stub?* A stub supplies answers for state checks; a mock verifies the calls itself.

Self-check

1. Q: The five doubles? → A: Dummy, stub, spy, mock, fake.
2. Q: Is 100% coverage proof? → A: No — mutation testing checks assertions catch bugs.

TL;DR

Many unit, some integration, few E2E; the right double; state over interactions; don't mock what you don't own.

Chapter summary

Topic	Remember this
Names	Reveal intent, pronounceable, searchable, one word per concept
Functions	Small, one thing, one level, stepdown, ≤2–3 args, no flags, CQS, exceptions
Comments	Explain in code; comment why/warnings/legal/public API
Formatting	Newspaper order, related things close, automated style
Objects vs data	Objects hide data; data structures expose it; anti-symmetry decides
Error handling	Unchecked with context, wrap vendors, no null in or out
Boundaries	Wrap third-party code; learning tests; interfaces you need
Unit tests	TDD laws, one concept per test, F.I.R.S.T.
Classes	Small by responsibility, cohesive, organised for change
Systems	Composition root + DI; cross-cutting via proxies/aspects
Emergence	Beck: tests → intent → no duplication → fewest elements (*Clean Code* swaps the middle two)
Concurrency	Separate it, share little, use the library, test hard
Smells	C/E/F/G/J/N/T checklist
Code review	Correctness → design → readability → tests → security → performance
Error strategy	Classify, translate, retry transient, handle globally, log once
Testing strategy	Pyramid; right double; state over behaviour verification

Chapter self-check

1. Q: `boolean saveAndValidate(Order o, boolean sendEmail)` breaks what? → A: Does two things, flag argument (F3), likely breaks CQS.
2. Q: `findOrders(customerId)` with no results? → A: An empty list, never `null`.
3. Q: Testing code that calls `LocalDate.now()`? → A: Inject a `Clock`; `Clock.fixed` in tests.

TL;DR (chapter)

Write for the next reader: honest names, tiny single-purpose functions, few "why" comments, consistent layout, exceptions instead of codes or nulls, foreign code behind your interfaces, clean fast tests, small cohesive classes wired from one place, simple design through refactoring, careful concurrency — and review against a shared checklist.

Chapter 7 — Refactoring (Study Edition)

> **Study Edition.** A shorter version of Chapter 7: every topic, all 24 smells and all 61 refactorings are here, but in compact form. The full chapter (README.md) has longer explanations and the complete source code.
>
> **What you will learn.** What refactoring is, when (and when not) to do it, how tests make it safe, the 24 **code smells** that tell you *where* to refactor, the 61 **refactorings** of Fowler's *Refactoring* (2nd ed., 2018) that tell you *how*, a smell → refactoring lookup table, a worked Java example in six compiling steps, and how refactoring leads to design patterns.
>
> **Why it matters for LLD interviews.** Machine-coding rounds often end with "now add feature X" or "what would you clean up?". You want to say: "this switch is repeated in three places — Repeated Switches — so I'd Replace Conditional with Polymorphism, which gives a Strategy."

7.1 Refactoring fundamentals

What it is

Refactoring (noun): a change to the internal structure of software that makes it easier to understand and cheaper to modify, *without changing its observable behaviour*. **Refactoring (verb):** applying a *series* of such small changes.

- **Behaviour-preserving** — same outputs, side effects and errors for the same inputs. Only internals (names, where code lives, how it is split) change.
- **Small steps** — each step compiles and passes the tests. If your code is broken for an hour, you are rewriting, not refactoring.

> **Term — observable behaviour.** What callers or users can see: return values, output, data written, exceptions. Call stacks, class structure and private method names are *not* observable behaviour.

Activity	Refactoring?	Why
Renaming a method with the IDE	Yes	Behaviour unchanged, tiny step
Fixing a bug / adding a feature	No	Behaviour changes on purpose
Performance tuning	No	Goal is speed, not clarity
Two days of "nothing works until the end"	No	That is restructuring/rewriting

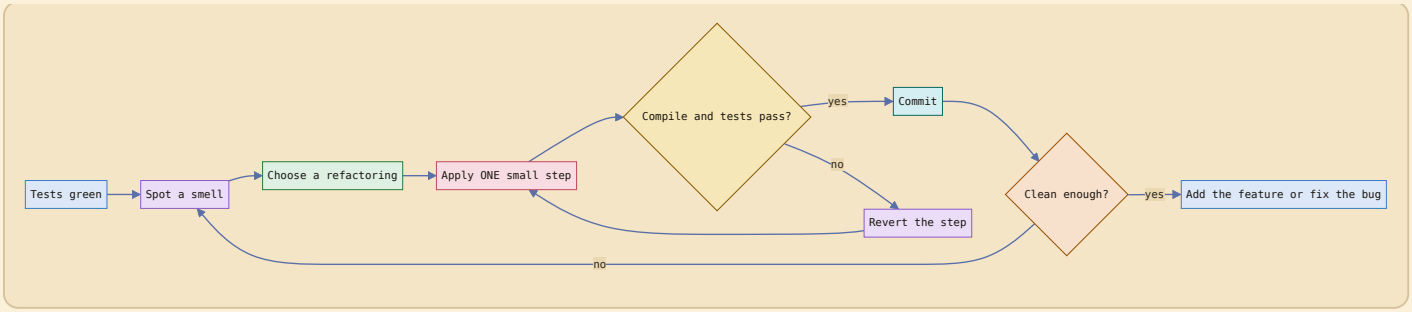
> **Term — restructuring.** The general word for reorganising code; refactoring is the small-step, behaviour-preserving kind.

The problem it solves

Code decays: every quick fix and copy-paste makes the next change harder. This **technical debt** is the extra cost on every future change. Refactoring pays it down continuously, instead of trying to get the design right up front (which never fully works, because requirements change).

How it works

1. Have tests that pin current behaviour — green. 2. Spot a smell (7.2). 3. Pick a named refactoring (7.3, 7.4). 4. Apply **one** small step (ideally IDE-automated). 5. Compile and test: green → commit locally; red → **revert** (don't debug) and take a smaller step. 6. Repeat until clean enough for the job at hand.



7.1.1 The two hats

Kent Beck: at any moment you wear one hat. **Adding function** — add behaviour and tests, don't restructure. **Refactoring** — restructure, don't add behaviour (tests for *existing* behaviour are fine). Swap often, but never mix: if a test fails while you did both, you can't tell which change broke it.

7.1.2 Why refactor

- **Improves the design** — stops decay; removes duplication so each fact lives in one place.
- **Makes code easier to understand** — moves understanding out of your head into the code.
- **Helps find bugs** — clear structure leaves bugs fewer places to hide.
- **Helps you program faster** — Fowler's **Design Stamina Hypothesis**: teams that keep internal quality high soon add features *faster*. Speed is the economic argument, not "clean code for its own sake".

7.1.3 When to refactor

Trigger	Meaning
Rule of Three (Don Roberts)	1st time do it, 2nd time wince and duplicate, 3rd time refactor
Preparatory	Just before a feature: "make the change easy, then make the easy change" (7.5 step 3)
Comprehension	Rename/extract as you come to understand code
Litter-pickup	Boy Scout Rule — fix small mess you pass; note bigger mess for later
Planned vs opportunistic	Mostly opportunistic, inside feature work; a quarterly "refactoring sprint" is a warning sign
Long-term	Gradual big changes, e.g. **Branch by Abstraction** (abstraction → move callers → swap impl → delete old)
Code review	Suggest or pair on refactorings, ideally with the author

> **Term — Rule of Three.** Don't abstract on the 1st or 2nd occurrence (you don't know what varies yet); abstract on the 3rd.

Ask your manager? Fowler: no more than for writing tests; discuss only large planned efforts.

7.1.4 When NOT to refactor

- Messy code behind a clean API that you never touch.
- When a rewrite is cheaper (7.1.10).
- Without a safety net — add tests first (7.1.11).
- An immovable deadline may postpone a *big* refactoring (skipping small ones usually slows you).
- **Published interfaces** — possible, but keep the old version working and deprecate it (Change Function Declaration, migration mechanics).

7.1.5 Problems and costs of refactoring

Problem	Mitigation
"Slows down features"	Refactor *because* it speeds features; focus on code you're about to change
Code ownership (callers in another repo)	Keep old name as a delegating `@Deprecated` method; migrate later
Long-lived feature branches	Continuous Integration / trunk-based — integrate at least daily
Missing tests	Add tests first; only IDE-automated refactorings until then
Databases	Evolutionary database design; small versioned migrations; **parallel change**

> **Term — parallel change (expand / contract).** *Expand* (add new beside old), *migrate* (move users), *contract* (remove old). Nothing is ever broken.

7.1.6 Refactoring, architecture and YAGNI

Because you can reshape code safely later, build only what today's requirements need — **YAGNI** ("You Aren't Gonna Need It"). This does not mean "no architecture": think hard up front only about things truly hard to change later (public APIs, database schemas).

7.1.7 Tests as the safety net

Self-testing code = automated tests that check themselves and run in seconds.

- Run tests after **every** step; on failure, **revert** and redo smaller.
- Test **behaviour** (public API), not private structure — otherwise every refactoring breaks tests.
- **Characterisation tests** (Feathers): record what code *currently* does, even if wrong. 7.5 uses a byte-for-byte output checker.
- **Mutation testing** (e.g. PIT) checks that your tests actually catch injected bugs.

> **Term — safety net.** Anything that tells you quickly you broke something: tests, compiler type checks, IDE-automated refactorings.

7.1.8 IDE automation

IDEs refactor using the **syntax tree and symbol resolution**, not text search — Rename changes only that symbol and its real references. (History: Opdyke's 1992 thesis → the Smalltalk *Refactoring Browser* → Java IDEs.)

Refactoring	IntelliJ (Win/Linux · macOS)	Eclipse
Rename	Shift+F6 · Shift+F6	Alt+Shift+R
Extract Method	Ctrl+Alt+M · Cmd+Opt+M	Alt+Shift+M
Extract Variable	Ctrl+Alt+V · Cmd+Opt+V	Alt+Shift+L
Inline	Ctrl+Alt+N · Cmd+Opt+N	Alt+Shift+I
Change Signature	Ctrl+F6 · Cmd+F6	Alt+Shift+C
Move	F6 · F6	Alt+Shift+V

Large-scale tools: **OpenRewrite** recipes, Google's **Refaster** templates in Error Prone.

> **Caution.** Automated refactorings can miss **reflection**, names in **config files**, **serialised field names** (JSON/Jackson, JPA) and other repositories. Renaming a serialised field changes your API's JSON — an observable behaviour change.

7.1.9 Performance and refactoring

1. Most refactorings have negligible cost — the JIT inlines small methods.
2. Programs spend most time in a small fraction of code (Chrysler payroll story: profiling found a hot spot nobody guessed).
3. Well-factored code is **easier to tune** — the hot spot is small and isolated.

Order: **make it work → make it clean → make it fast (profile, optimise hot spots)**.

Care is still needed: see the `DoubleStream.sum()` trap in 7.5 step 2.

7.1.10 Refactoring vs rewriting

	Refactoring	Rewriting
Unit of change	Many tiny safe steps	One big replacement
System works during change?	Always	Usually not until the end
Risk	Low	High — old bug fixes lost; old system keeps evolving
Delivers value	Continuously	Only at the end
Good when	Code mostly works	Code doesn't work at all, or technology must change

Even replacements go piece by piece: the **Strangler Fig** builds the new system around the old, routing one feature at a time until the old one can be switched off.

7.1.11 Legacy code

Feathers (*Working Effectively with Legacy Code*, 2004): **legacy code = code without tests**. The loop: identify change points → find test points → break dependencies at a **seam** (a place to change behaviour without editing there, e.g. inject an interface instead of `new Database()`, or override in a test subclass) → write characterisation tests → change and refactor.

Also: **Sprout Method/Class** (new tested code, called by a one-line change), **Wrap Method/Class** (rename old; new method with the old name calls old + new behaviour); automated refactorings only until tests exist; clean only what you touch.

7.2 Code smells

What it is

A **code smell** (Kent Beck's term) is a *surface sign* that usually points to a deeper design problem. It is not a bug — it tells you *where to look*. Smells are **heuristics, not rules**.

The problem it solves

You must know *when* and *where* to refactor, not just how. Fowler and Beck give no metrics ("max 10 lines"); they give **named** smells with their usual cures. Names help reviews: "Feature Envy" says in two words what would take a paragraph.

How it works

Notice discomfort → match it to a named smell → look up its cures (7.4) → apply in small steps under tests.

Smell groups (refactoring.guru classification)

Group	Idea	Smells (2nd-ed name; *guru name*)
Bloaters	Grown too big	Long Function *(Long Method)*, Large Class, Primitive Obsession, Long Parameter List, Data Clumps
OO Abusers	Wrong/incomplete OO	Repeated Switches *(Switch Statements)*, Temporary Field, Refused Bequest, Alternative Classes with Different Interfaces
Change Preventers	One change forces many	Divergent Change, Shotgun Surgery *(+ Parallel Inheritance Hierarchies, 1st ed.)*
Dispensables	Pointless; removing helps	Comments, Duplicated Code, Lazy Element *(Lazy Class)*, Data Class, Speculative Generality *(+ Dead Code)*
Couplers	Too much coupling/delegation	Feature Envy, Insider Trading *(Inappropriate Intimacy)*, Message Chains, Middle Man *(+ Incomplete Library Class)*
2nd-edition extras		Mysterious Name, Global Data, Mutable Data, Loops

> **Term — coupling / cohesion.** Coupling: how much one piece depends on another. Cohesion: how strongly the things inside one unit belong together. Aim for low coupling, high cohesion.

Below, all 24 smells in the book's order: what it is, why it hurts, a tiny smelly → cured sketch, and the cures. (Snippets use `double` for money to stay short; real money needs `BigDecimal` or a `Money` type — see `code/smells/Money.java`.)

7.2.1 Mysterious Name

What. Names that don't say what a thing does or holds: `data`, `tmp`, `proc()`, `x2`, `Manager`. **Hurts.** Every reader must decode the body; if you *can't* find a good name, the thing may do too much. `calc(l, d)` → `averageDailyRevenue(dailyRevenues, daysInPeriod)`. **Cures:** Change Function Declaration, Rename Variable, Rename Field.

7.2.2 Duplicated Code

What. The same code structure in more than one place. **Hurts.** Every change must find every copy; one gets missed. `net + net * 0.18` in two methods → one `withTax(net)` using `GST_RATE`. **Cures:** Extract Function; Slide Statements; Pull Up Method.

7.2.3 Long Function

What. A function too long to grasp at a glance. Heuristic: when you feel a block needs a **comment**, extract it and name it after its *intent*. **Hurts.** Hard to understand, reuse and test. `printOwing` → `printBanner(); outstandingOf(inv); recordDueDate(inv); printDetails(...)`. **Cures:** Extract Function (~99% of the time), plus Replace Temp with Query, Decompose Conditional, Split Loop and others (7.4).

7.2.4 Long Parameter List

What. Many parameters, often same-typed, or boolean flags. **Hurts.** Callers mix up order; each new parameter touches every caller; hides a missing concept. `book(hotelId, from, to, adults, children, breakfast)` → `book(hotel, Stay, Guests, MealPlan)`. **Cures:** Introduce Parameter Object, Preserve Whole Object, Remove Flag Argument (+ 7.4).

7.2.5 Global Data

What. Data readable and writable from anywhere: globals, public static mutable fields, mutable singletons. **Hurts.** No way to find who changed it; bugs appear far from their cause. One of the most pungent smells. `public static double discountRate` → private field + `discountRate()` / `setDiscountRate()`; better still, inject the value. **Cures:** Encapsulate Variable first, always; then narrow its scope.

7.2.6 Mutable Data

What. Data changed after creation, especially when shared or reused for different purposes. **Hurts.** A change in one place breaks code elsewhere that assumed the value stayed put — worse with threads. `public List<Item> items; public double total;` → private list with `add()`, `items()` returning `List.copyOf`, and `total()` computed on demand. **Cures:** Encapsulate Variable, Replace Derived Variable with Query, Remove Setting Method, Change Reference to Value (+ 7.4).

7.2.7 Divergent Change

What. One class changed for many different reasons (new database *and* new instrument *and* new report format). **Hurts.** Mixed responsibilities break SRP; changes interfere. `ProductService` (save + price + CSV) → `ProductRepository`, `PricingService`, `ProductCsvWriter`. **Cures:** Split Phase, Extract Class (+ Move/Extract Function).

7.2.8 Shotgun Surgery

What. The opposite: **one change forces small edits in many classes**. **Hurts.** Easy to miss one place; the concept is scattered. The "free shipping at 500" rule in four classes → one `ShippingPolicy` with `FREE_SHIPPING_THRESHOLD`, `isFree()`, `charge()`. **Cures:** Move Function, Move Field (+ 7.4); sometimes Inline first, then re-extract.

	Divergent Change	Shotgun Surgery
Shape	One class, many reasons to change	One reason, many classes
Fix	Split the class	Gather the scattered code

7.2.9 Feature Envy

What. A method more interested in another class's data than its own (calls many getters). **Hurts.** Behaviour lives away from its data. Rule: **put behaviour next to the data it uses** (Strategy/Visitor separate them on purpose). `order.lineTotal(line)` → `line.total()`. **Cures:** Move Function (Extract Function first if only part is envious).

7.2.10 Data Clumps

What. The same group of values always travels together: `(street, city, zip)`, `(start, end)`, `(amount, currency)`. **Hurts.** An unnamed concept; its behaviour gets duplicated. **Test:** delete one item — if the rest makes no sense alone, make an object. `(LocalDate start, LocalDate end)` → `record DateRange` with `contains()` (see `code/smells/DateRange.java`). **Cures:** Extract Class, Introduce Parameter Object, Preserve Whole Object — then watch for Feature Envy moving onto the new class.

7.2.11 Primitive Obsession

What. Primitives for domain concepts: money as `double`, phone as `String`, status as `"ACTIVE"`. **Hurts.** No validation, duplicated formatting, easy mix-ups (`transfer(amount, fee)` swapped compiles), and `0.1 + 0.2 = 0.30000000000000004`. `charge(double, String)` → `charge(Money)`. **Cures:** Replace Primitive with Object (+ Replace Type Code with Subclasses, 7.4).

7.2.12 Repeated Switches

What. The *same* switch on the same type code in several places. (1st ed. "Switch Statements" flagged any switch; the 2nd ed. flags only the repetition.) **Hurts.** A new case means finding every switch — Shotgun Surgery. Three `switch (b.type)` methods → `sealed interface Bird` with `speed()`, `plumage()`. **Cures:** Replace Conditional with Polymorphism. (With a `sealed` type or `enum`, switch *expressions* are exhaustiveness-checked, so a few are acceptable.)

7.2.13 Loops

What. A `for` loop that filters/maps/aggregates where a pipeline says it better (new in 2nd ed.). **Hurts.** The reader must simulate the loop. Loop adding IT names → `employees.stream().filter(...).map(Employee::name).toList()`. **Cures:** Replace Loop with Pipeline. Careful: `toList()` is **unmodifiable**; use `Collectors.toCollection(ArrayList::new)` if callers mutate.

7.2.14 Lazy Element

What. A class/function/subclass that doesn't pull its weight (1st ed. "Lazy Class"). **Hurts.** Every element costs a name, a file, an indirection. `NameFormatter.format(s)` that only trims → `rawName.trim()`. **Cures:** Inline Function, Inline Class, Collapse Hierarchy.

7.2.15 Speculative Generality

What. Hooks and abstractions "we might need someday": abstract class with one subclass, unused parameters, methods used only by tests. **Hurts.** Cost with no benefit; violates **YAGNI**. `PaymentProcessorFactoryProvider` with one implementation, `futureOptions` never used → `pay(Money m)`. **Cures:** Collapse Hierarchy, Inline Function, Inline Class, Change Function Declaration, Remove Dead Code (delete the tests too).

7.2.16 Temporary Field

What. A field set and used only in some situations, null the rest of the time. **Hurts.** Readers expect every field to matter — "is this null a bug?". `Planner.tmpDistances`, `tmpTarget` → a short-lived `RouteSearch` object holding that state. **Cures:** Extract Class, Move Function, Introduce Special Case.

7.2.17 Message Chains

What. `order.getCustomer().getAddress().getCity()`. **Hurts.** Client coupled to the whole navigation path; violates the **Law of Demeter**. → `order.shippingCity()` (see `code/smells/Order.java`). Don't hide every delegate (creates Middle Men). **Cures:** Hide Delegate, Extract Function, Move Function.

7.2.18 Middle Man

What. A class whose methods *mostly* just forward to another class. **Hurts.** A layer with no value; every delegate feature needs a new forwarder. `Person` with ten `dept.getX()` forwarders → `department()` accessor. **Cures:** Remove Middle Man, Inline Function, Replace Superclass/Subclass with Delegate. Message Chains and Middle Man pull in opposite directions — balance them.

7.2.19 Insider Trading

What. Modules poking at each other's internals; subclasses knowing too much about parents (1st ed. "Inappropriate Intimacy"). **Hurts.** Can't change one without the other. `customer.internalLoyaltyTable.get(customer.tierCode)` → `customer.loyaltyDiscount()`. **Cures:** Move Function, Move Field, Hide Delegate, Replace Subclass/Superclass with Delegate.

7.2.20 Large Class

What. A class doing too much — a **God Class**. **Hurts.** Magnet for duplication, bottleneck for changes, breaks SRP. `Employee` with identity + address + payroll + performance fields → `Address`, `Compensation`, `PerformanceRecord`. **Cures:** Extract Class, Extract Superclass, Replace Type Code with Subclasses.

7.2.21 Alternative Classes with Different Interfaces

What. Classes doing the same job with different signatures: `EmailSender.sendMail(to, body)` vs `SmsGateway.dispatch(msg, phone)`. **Hurts.** No polymorphism → callers write `if email ... else sms`. → `interface Notifier { void send(String recipient, String message); }`. **Cures:** Change Function Declaration, Move Function, Extract Superclass; for third-party classes, an **Adapter** (Ch. 9).

7.2.22 Data Class

What. Only fields and getters/setters; others manipulate its data. **Hurts.** Behaviour lives elsewhere (Feature Envy), duplicated; public setters break invariants. **Exceptions:** immutable records — Split Phase results, DTOs, value objects — are fine. `if (acc.getBalance() >= amt) acc.setBalance(...)` → `account.withdraw(amount)`. **Cures:** Encapsulate Record, Remove Setting Method, Move Function (+ 7.4).

7.2.23 Refused Bequest

What. A subclass that doesn't want what it inherits — or refuses the *interface* (throws `UnsupportedOperationException`). **Hurts.** Refusing implementation is mild; refusing the interface breaks **Liskov** (Ch. 4). `Stack<T> extends ArrayList<T>` (cf. `java.util.Stack extends Vector`) → `Stack` holding a `Deque`.

(Check: empty `pop()` now throws a different exception and `ArrayDeque` rejects nulls.) **Cures:** Push Down Method/Field, Replace Subclass with Delegate, Replace Superclass with Delegate.

7.2.24 Comments

What. Comments used as **deodorant** — explaining *what* confusing code does (Fowler: a "sweet" smell). **Hurts.** Comments go stale. **Good comments:** *why*, warnings, algorithm/ticket references, uncertainty notes, Javadoc. `// check senior discount` + complex `if` → `if (isEligibleForSeniorDiscount(customer))`; `// rate never negative` → `assert rate >= 0`. **Cures:** Extract Function, Change Function Declaration, Introduce Assertion.

1st-edition smell names (renamed or dropped in the 2nd edition)

1st-edition smell	Where it went
Long Method	**Long Function**
Switch Statements	Narrowed to **Repeated Switches**
Lazy Class	**Lazy Element**
Inappropriate Intimacy	**Insider Trading**
Parallel Inheritance Hierarchies	Special case of Shotgun Surgery; Move Function/Field so one hierarchy disappears
Incomplete Library Class	Wrapper/utility (1st ed. Introduce Foreign Method / Local Extension) or Adapter
Dead Code	The refactoring **Remove Dead Code**

7.3 The refactoring catalogue

What it is

A **named refactoring** is a small, well-defined transformation with a name, a motivation, safe mechanics and examples. Fowler's 2nd edition has **61**, in book chapters 6–12 (online at refactoring.com/catalog). Old 1st-edition names appear as aliases. Many come in **inverse pairs** (Extract ↔ Inline Function) — go whichever way makes the code clearer for the change you're about to make.

How to read each entry

Each row: **name** (*alias*; ⇄ inverse), **when/why**, and a tiny **before → after** sketch (illustrative; the fully compiling examples are in `code/`).

Book chapter	Count
6 A First Set · 7 Encapsulation · 8 Moving Features · 9 Organizing Data · 10 Simplifying Conditionals · 11 Refactoring APIs · 12 Inheritance	11 + 9 + 9 + 5 + 6 + 10 + 11 = **61**
refactoring.com extras	5 more

7.3.1 A First Set of Refactorings (book chapter 6)

The everyday ones.

Refactoring	When / why	Before → After
Extract Function (*Extract Method*; ⇄ Inline Function)	Separate *intention* from implementation: if you must study a fragment to see *what* it does, extract it and name it that. Even one line is worth it.	inline print lines → `printDetails(invoice, outstanding)`
Inline Function (*Inline Method*; ⇄ Extract Function)	Body as clear as the name, or a tangle of bad small functions (inline, then re-extract), or pure delegation	`moreThanFiveLateDeliveries(d) ? 2 : 1` → `d.lateDeliveries() > 5 ? 2 : 1`
Extract Variable (*Introduce Explaining Variable*; ⇄ Inline Variable)	Name parts of a complex expression; if the name matters beyond the function, extract a function instead	long price formula → `basePrice - quantityDiscount + shipping`
Inline Variable (*Inline Temp*; ⇄ Extract Variable)	Name says no more than the expression, or blocks another refactoring	`double basePrice = order.basePrice(); return basePrice > 1000;` → `return order.basePrice() > 1000;`
Change Function Declaration (*Rename Function/Method, Add/Remove Parameter, Change Signature*)	Names and parameters are the joints of the system. **Simple mechanics**: change all callers at once. **Migration mechanics**: new function, old one delegates + `@Deprecated`, migrate, delete.	`circum(r)` → `circumference(r)` + deprecated `circum` delegating
Encapsulate Variable (*Encapsulate Field, Self-Encapsulate Field*)	Turns "reorganise data" into "reorganise functions"; one place for validation/logging. First step for Global Data	`public static Owner defaultOwner` → private + `defaultOwner()` / `setDefaultOwner()`
Rename Variable	The wider the scope, the more the name matters	`double a = height * width` → `double area = ...`
Introduce Parameter Object	A Data Clump of parameters becomes one object — often the seed of a domain class	`(station, min, max)` → `(station, NumberRange range)` with `contains(v)`
Combine Functions into Class	Several functions share the same data argument → a class holding it as fields	`base(r)`, `taxableCharge(r)` → `ReadingCalculator(reading).base()`
Combine Functions into Transform	Read-only source data: one function returns an enriched copy with all derived values (use a class if the source can change — the copy would go stale)	scattered `base`, `taxable` → `enrichReading(r)` returning `EnrichedReading`
Split Phase	Code doing two things in sequence (parse then compute; compute then format) → two phases joined by an intermediate data structure. Cures Divergent Change; see 7.5 step 3	`priceOrder(line)` → `price(parse(line))` via `record OrderRequest`

7.3.2 Encapsulation (book chapter 7)

Deciding what each module *hides*: data structures, temps, relationships.

Refactoring	When / why	Before → After
Encapsulate Record (*Replace Record with Data Class*)	Raw maps/arrays/public-field structs expose layout	`Map.of("name", "Acme", ...)` → `record Organization(String name, String country)`
Encapsulate Collection	Getter returning the internal collection lets clients change it behind the owner's back	`getCourses()` returning the list → `courses()` unmodifiable + `addCourse` / `removeCourse`
Replace Primitive with Object (*Replace Data Value with Object, Replace Type Code with Class*)	A simple value soon needs validation/formatting/comparison. Cures Primitive Obsession	`priority().equals("high") || ...("rush")` → `enum Priority` with `higherThan(NORMAL)`
Replace Temp with Query	A temp is visible only in its function; a method is usable class-wide and eases extraction. Best if assigned once and side-effect free	`double basePrice = quantity * itemPrice` → `basePrice()` method
Extract Class (⇄ Inline Class)	A subset of fields/methods that go (and change) together. Cures Large Class, Divergent Change	`Person.officeAreaCode, officeNumber` → `record TelephoneNumber`
Inline Class (⇄ Extract Class)	Class no longer does enough; or merge-then-re-split	`Shipment.trackingInfo` → fields directly in `Shipment`
Hide Delegate (⇄ Remove Middle Man)	`a.getB().do()` couples client to B; add `a.do()`. Cures Message Chains	`person.getDepartment().getManager()` → `person.manager()`
Remove Middle Man (⇄ Hide Delegate)	Too many forwarders; let clients call the delegate	`person.manager()` → `person.department().getManager()`
Substitute Algorithm	Replace a whole body with a clearer algorithm or a library call (with tests proving they agree)	loop of `if equals "Don"/"John"/"Kent"` → `Set.of(...)` + `stream().filter(candidates::contains).findFirst()`

7.3.3 Moving Features (book chapter 8)

Moving code between classes, between functions, and within a function.

Refactoring	When / why	Before → After
Move Function (*Move Method*)	Put a function where the data it uses most lives. Cures Feature Envy, Shotgun Surgery	`Account.overdraftCharge()` logic → `AccountType.overdraftCharge(daysOverdrawn)`; `Account` delegates
Move Field	A field always passed alongside another object, or changed with it, belongs there (encapsulate first)	`Customer.discountRate` → `Plan.discountRate`; `Customer.discountRate()` delegates
Move Statements into Function (⇄ Move Statements to Callers)	Same statements run with every call → move them inside	title line at every caller → inside `photoData(p)`
Move Statements to Callers (⇄ Move Statements into Function)	A shared function needs different behaviour per caller → move the varying lines out (identical at first; vary later as a separate change)	`emitPhotoData` writes location → each caller writes location itself
Replace Inline Code with Function Call	Code repeats what an existing function does	flag loop over `states` → `states.contains("MA")`
Slide Statements (*Consolidate Duplicate Conditional Fragments*)	Keep related statements together (declare right before use); prepares Extract Function; a statement repeated in every branch slides out. Can't slide past interfering code	reorder so `pricingPlan` is declared next to `baseCharge`
Split Loop	A loop doing two things → two loops, each extractable. Double iteration is rarely a bottleneck	one loop for youngest + total salary → two loops
Replace Loop with Pipeline	`filter` / `map` / `reduce` name each step	skip-blank/comment loop → `lines.stream().filter(...).map(...).toList()`
Remove Dead Code	Unused code costs readers; version control remembers it. Don't comment it out	`if (false) { legacyDiscount(); }` → gone

7.3.4 Organizing Data (book chapter 9)

Refactoring	When / why	Before → After
Split Variable (*Remove Assignments to Parameters, Split Temp*)	A variable (not a loop counter/accumulator) assigned twice has two jobs; don't reuse parameters as locals	`temp` = perimeter, then area → `final double perimeter`, `final double area`
Rename Field	Field names explain the data structure; encapsulate first if widely used	`record Organization(String nm, ...)` → `(String name, ...)`
Replace Derived Variable with Query	Stored derived mutable data drifts out of sync; compute on demand (fine to store if the source is immutable)	`production += a.amount()` on every apply → `production()` sums the adjustments
Change Reference to Value (⇄ Change Value to Reference)	Inner object without identity → immutable **value object**, replaced wholesale, equal by value; safe to share across threads. Records are ideal	mutable `TelephoneNumber.setAreaCode` → `record TelephoneNumber` + `setTelephone(new ...)`
Change Value to Reference (⇄ Change Reference to Value)	Many copies of one logical entity must see updates → share one instance via a **repository**	`new Customer(id)` per order → `repo.findOrCreate(id)`

7.3.5 Simplifying Conditional Logic (book chapter 10)

Refactoring	When / why	Before → After
Decompose Conditional	Extract the condition and each branch into named functions (a special Extract Function)	summer date-range `if/else` → `isSummer(date) ? summerCharge() : regularCharge()`
Consolidate Conditional Expression	Several side-effect-free checks with the same result are one condition	three `return 0` checks → `if (isNotEligibleForDisability(e)) return 0;`
Replace Nested Conditional with Guard Clauses	Handle unusual cases first and return; the normal path reads straight down. "One exit point" is not worth unclarity	nested `if/else` in `payAmount` → `if (separated) return 0; if (retired) return 0; return normalPay(e);`
Replace Conditional with Polymorphism	Repeated switches on a type, or base behaviour with variants → class per case, dynamic dispatch. Leads to **Strategy/State** (7.6); 7.5 step 5	`switch (b.type())` in `speed()` → `sealed interface Bird` + one record per bird
Introduce Special Case (*Introduce Null Object*)	Many clients check the same special value (null/"unknown") and do the same thing → an object that answers itself. `Optional` shows absence in the type but callers still decide; a special case centralises the decision	`customer == null ? "occupant" : ...` → `UnknownCustomer`
Introduce Assertion	Make an assumption explicit; program must be correct without it. Java `assert` is off by default (`-ea`) — never use it for external input	`// discountRate never negative` → `assert discountRate >= 0 : ...`

7.3.6 Refactoring APIs (book chapter 11)

Refactoring	When / why	Before → After
Separate Query from Modifier	A value-returning function should have no observable side effects — **Command–Query Separation** (Meyer)	`alertForMiscreant` returns name *and* sets alarms → `findMiscreant()` (query) + `alertForMiscreant()` (modifier)
Parameterize Function (*Parameterize Method*)	Near-identical functions differing only in literals	`tenPercentRaise`, `fivePercentRaise` → `raise(p, factor)`
Remove Flag Argument (*Replace Parameter with Explicit Methods*)	A literal that picks the code path makes calls unreadable (`deliveryDate(order, true)`)	→ `rushDeliveryDate(o)`, `regularDeliveryDate(o)`; `setDimension("height", v)` → `setHeight(v)`
Preserve Whole Object	Don't unpack values from one object just to pass them (unless you want independence from its type)	`withinRange(low, high)` → `withinRange(room.daysTempRange())`
Replace Parameter with Query (*Replace Parameter with Method*; ⇄ Replace Query with Parameter)	The function can get the value itself — don't make callers pass it (unless it adds an unwanted dependency)	`discountedPrice(base, discountLevel())` → `discountedPrice(base)`
Replace Query with Parameter (⇄ Replace Parameter with Query)	Reaching into a global/singleton hurts testing and **referential transparency** — pass the value in	`Thermostat.INSTANCE.currentTemperature()` inside → `targetTemperature(plan, currentTemperature)`
Remove Setting Method	Field shouldn't change after construction → set in constructor, `final`	`setId(id)` → constructor + `final String id`
Replace Constructor with Factory Function (*Replace Constructor with Factory Method*)	Constructors have a fixed name, must return that exact class, need `new`. Factories can be named, return subclasses or cached instances (*Effective Java* Item 1; `List.of`, `Integer.valueOf`)	`new Employee(name, "E")` → `Employee.createEngineer(name)`
Replace Function with Command (*Replace Method with Method Object*; ⇄ Replace Command with Function)	Wrap a function in an object with `execute()`: holds state, supports undo/queueing, and splits a big function whose locals become fields	60-line `score(...)` → `new Scorer(c, exam, guide).execute()`
Replace Command with Function (⇄ Replace Function with Command)	A simple command adds nothing a function couldn't	`new ChargeCalculator(customer, usage).charge()` → `charge(customer, usage)`

7.3.7 Dealing with Inheritance (book chapter 12)

Refactoring	When / why	Before → After
Pull Up Method (⇄ Push Down Method)	Identical methods in siblings → superclass (make them identical first, or use Template Method)	`Salesman.name()`, `Engineer.name()` → `Employee.name()`
Pull Up Field (⇄ Push Down Field)	Same field in siblings (rename to match first)	`protected String name` in both → in `Employee`
Pull Up Constructor Body	Repeated init in subclass constructors → superclass constructor (Java 21 needs `super(...)` first; Java 25 relaxed this, JEP 513)	`Manager` sets name/id → `super(name, id); this.grade = grade;`
Push Down Method (⇄ Pull Up Method)	Method relevant only to some subclasses (callers must know the subclass)	`Employee.quota()` → `Salesman.quota()`
Push Down Field (⇄ Pull Up Field)	Field used by only one subclass	`Employee.quota` → `Salesman.quota`
Replace Type Code with Subclasses (*Extract Subclass, Replace Type Code with State/Strategy*; ⇄ Remove Subclass)	A type code driving behaviour. **Direct** (subclass the class) or **indirect** (subclass a separate type object — needed if the type changes at runtime or the class is already subclassed)	`Employee.type` → `sealed Employee permits Engineer, Manager, Salesman` + `create(name, type)` switch
Remove Subclass (*Replace Subclass with Fields*; ⇄ Replace Type Code with Subclasses)	Subclasses differing only in a constant aren't worth it	`Male`/`Female` returning `"M"`/`"F"` → `Person.genderCode` field
Extract Superclass	Two similar classes duplicate fields/methods (alternative: Extract Class + delegation; you can switch later)	`Department`, `Employee` → `abstract class Party`
Collapse Hierarchy	Parent and child no longer different enough. Cures Lazy Element, Speculative Generality	empty `Salesman extends Employee` → just `Employee`
Replace Subclass with Delegate	Inheritance varies only **one** axis and couples tightly; a delegate can vary independently and change at runtime — "favour composition over inheritance", the core of **State/Strategy**	`PremiumBooking extends Booking` → `Booking.premiumDelegate` set by `bePremium(extras)`
Replace Superclass with Delegate (*Replace Inheritance with Delegation*)	Subclass uses only part of its parent, or parent methods don't make sense (Refused Bequest, broken Liskov) → hold an instance and forward what you need	`Scroll extends CatalogItem` → `Scroll` holding a `catalogItem` delegate

7.3.8 Extra refactorings on refactoring.com

Five more in the online catalogue (not among the 61):

Refactoring	When / why	Before → After
Replace Control Flag with Break (*Remove Control Flag*)	A `found` flag only to stop a loop	flag → `break` / `continue` / `return`
Replace Error Code with Exception	Special codes (`-1`, `null`) get ignored by callers	`return -1` → `throw new InsufficientFundsException(...)`
Replace Exception with Precheck (*Replace Exception with Test*)	Exceptions are for exceptional cases; check first if it's easy	catch `ArrayIndexOutOfBoundsException` → bounds check
Replace Magic Literal (*Replace Magic Number with Symbolic Constant*)	A special-meaning literal needs a name and one home (not for obvious `0`); 7.5 step 4	`9.81` → `STANDARD_GRAVITY`
Return Modified Value	A function secretly updating shared state → return the new value; the data flow becomes visible	`calculateAscent();` → `totalAscent = calculateAscent();`

1st-edition names you may still hear

The old (1999) names are shown as *aliases* in every row above (e.g. Extract Method → Extract Function, Introduce Null Object → Introduce Special Case, Replace Inheritance with Delegation → Replace Superclass with Delegate, Remove Control Flag → Replace Control Flag with Break). The full old → new table is in the Full Edition.

7.4 Smell → refactorings lookup table

Bold = the usual first move.

Smell	Group	Refactorings
Mysterious Name	2nd-ed extra	**Change Function Declaration**, Rename Variable, Rename Field
Duplicated Code	Dispensables	**Extract Function**, Slide Statements, Pull Up Method *(also: Replace Inline Code with Function Call, Substitute Algorithm)*
Long Function	Bloaters	**Extract Function**, Replace Temp with Query, Introduce Parameter Object, Preserve Whole Object, Replace Function with Command, Decompose Conditional, Replace Conditional with Polymorphism, Split Loop
Long Parameter List	Bloaters	Replace Parameter with Query, **Preserve Whole Object**, **Introduce Parameter Object**, Remove Flag Argument, Combine Functions into Class
Global Data	2nd-ed extra	**Encapsulate Variable** (then shrink scope)
Mutable Data	2nd-ed extra	**Encapsulate Variable**, Split Variable, Slide Statements, Extract Function, Separate Query from Modifier, Remove Setting Method, Replace Derived Variable with Query, Combine Functions into Class/Transform, Change Reference to Value
Divergent Change	Change Preventers	**Split Phase**, Move Function, Extract Function, **Extract Class**
Shotgun Surgery	Change Preventers	**Move Function**, **Move Field**, Combine Functions into Class/Transform, Split Phase, Inline Function, Inline Class
Feature Envy	Couplers	**Move Function**, Extract Function
Data Clumps	Bloaters	**Extract Class**, **Introduce Parameter Object**, Preserve Whole Object
Primitive Obsession	Bloaters	**Replace Primitive with Object**, Replace Type Code with Subclasses, Replace Conditional with Polymorphism, Extract Class, Introduce Parameter Object
Repeated Switches	OO Abusers	**Replace Conditional with Polymorphism**
Loops	2nd-ed extra	**Replace Loop with Pipeline**
Lazy Element	Dispensables	**Inline Function**, **Inline Class**, Collapse Hierarchy
Speculative Generality	Dispensables	Collapse Hierarchy, Inline Function, Inline Class, Change Function Declaration, Remove Dead Code
Temporary Field	OO Abusers	**Extract Class**, Move Function, Introduce Special Case
Message Chains	Couplers	**Hide Delegate**, Extract Function, Move Function
Middle Man	Couplers	**Remove Middle Man**, Inline Function, Replace Superclass/Subclass with Delegate
Insider Trading	Couplers	**Move Function**, **Move Field**, Hide Delegate, Replace Subclass/Superclass with Delegate
Large Class	Bloaters	**Extract Class**, Extract Superclass, Replace Type Code with Subclasses
Alternative Classes w/ Different Interfaces	OO Abusers	**Change Function Declaration**, Move Function, Extract Superclass
Data Class	Dispensables	**Encapsulate Record**, Remove Setting Method, Move Function, Extract Function, Split Phase
Refused Bequest	OO Abusers	Push Down Method/Field, **Replace Subclass with Delegate**, **Replace Superclass with Delegate**
Comments	Dispensables	**Extract Function**, Change Function Declaration, Introduce Assertion

Which principle each smell family violates (Chapters 4–6):

Smell family	Principle violated
Divergent Change, Large Class	Single Responsibility
Repeated Switches, Shotgun Surgery	Open/Closed
Refused Bequest	Liskov Substitution
Long Parameter List with flags, fat interfaces	Interface Segregation / clean functions
Global Data, Mutable Data	Encapsulation / information hiding
Hard-wired `new` of concrete dependencies	Dependency Inversion
Duplicated Code	DRY
Speculative Generality	YAGNI
Message Chains, Insider Trading	Law of Demeter
Feature Envy, Data Class	Tell, Don't Ask

7.5 Worked example — from messy to clean in six compiling steps

The starting point

A courier prints invoices. Each shipment has a **service type** (`STD`, `EXP`, `FRG`), a weight and a distance.

Type	Charge	Loyalty points
STD	5 + 1.2/kg, + 0.05/km beyond 100 km	1 per full 100 km
EXP	12 + 2.0/kg, + 0.10/km beyond 50 km	1 per full 100 km, +2 bonus
FRG	8 + 1.5/kg + 3 handling; ×1.2 if > 10 kg	1 per full 100 km

New feature request: an HTML invoice now, and an OVERNIGHT service next quarter.

Step 0 (excerpt from `code/worked/step0/Main.java`):

```
    static String print(Inv inv) {
        double tot = 0;
        int pts = 0;
        int unused = 0;                                  // dead code
        StringBuilder r = new StringBuilder("Invoice for " + inv.cust + "\n");
        for (Shp s : inv.list) {
            double amt = 0;
            switch (s.t) {
                case "STD":
                    amt = 5 + s.kg * 1.2;
                    if (s.km > 100) amt += (s.km - 100) * 0.05;
                    break;
                // ... EXP and FRG cases ...
                default:
                    throw new IllegalArgumentException("unknown type: " + s.t);
            }
            // loyalty points
            pts += s.km / 100;
            if ("EXP".equals(s.t)) pts += 2;
            // ... append the formatted line ...
            tot += amt;
        }
        // ... append "Amount owed" and "Points earned" ...
        return r.toString();
    }
```

Smell	Where
Mysterious Name	`Shp`, `Inv`, `t`, `kg`, `km`, `amt`, `tot`, `pts`, `r`
Long Function	`print` computes charges, points *and* formats
Divergent Change	HTML would copy everything; a new tariff edits the formatter
Repeated Switches (coming)	`"EXP"` checked in two places
Primitive Obsession	Service type is a `String`
Magic literals	`5`, `1.2`, `100`, `0.05`, ...
Data Class / Mutable Data	Public mutable fields in `Shp`, `Inv`
Dead code	`int unused`

So: **preparatory refactoring** first.

The safety net

`lld.ch07.worked.Main` (`code/worked/Main.java`) is a **characterisation check**: it records step 0's output as "golden" and compares every later step **byte for byte**. The fixture includes boundaries (exactly 100 km STD, exactly 10 kg FRG). Each step lives in its own package `step0`…`step5`.

Step 1 — Extract Function, Rename Variable/Field, Remove Dead Code

Rename Field (`t → type`, `kg → weightKg`, `km → distanceKm`; `Shp → Shipment`, `Inv → Invoice`), Rename Variable (`tot → totalAmount`, `pts → loyaltyPoints`, `r → result`), Extract Function ×3 (`amountFor`, `pointsFor`, `money`), Remove Dead Code. The switch was *moved*, not changed — **move before you change**.

Step 2 — Inline Variable, Split Loop, Replace Temp with Query, Replace Loop with Pipeline, Slide Statements

Inline `thisAmount`; split the three-job loop; turn total and points into queries; make them streams; slide declarations away.

```
    static double totalAmount(Invoice invoice) {
        return invoice.shipments.stream().map(Main::amountFor).reduce(0.0, Double::sum);
    }
```

Trap: `mapToDouble(...).sum()` may use compensated summation and differ in the last bit; `reduce(0.0, Double::sum)` adds in order like the loop. (`IntStream.sum()` is exact.) `amountFor` now runs twice per shipment — ignore until a profiler says otherwise.

Step 3 — Split Phase, Encapsulate Record, Encapsulate Collection, Combine Functions into Transform

The key preparatory move: `StatementCalculator.calculate(Invoice)` (phase 1, a transform) → `StatementData` record → `TextRenderer.render(StatementData)` (phase 2, knows no tariffs). `Shipment`/`Invoice` become records; `Invoice` uses `List.copyOf`.

Step 4 — Replace Primitive with Object, Replace Magic Literal

The type code becomes `enum ServiceType { STANDARD("STD"), EXPRESS("EXP"), FRAGILE("FRG") }` with `fromCode()`, so a bad code fails **at the boundary** (same exception and message; here shipments are always built and printed together, so behaviour is unchanged — check this in real code). Every tariff number gets a name (`STD_FREE_KM`, ...). The switch becomes an exhaustive **switch expression** with no `default`. (`x + 0` equals the old "if not taken" for all doubles except `-0.0`, which can't occur.)

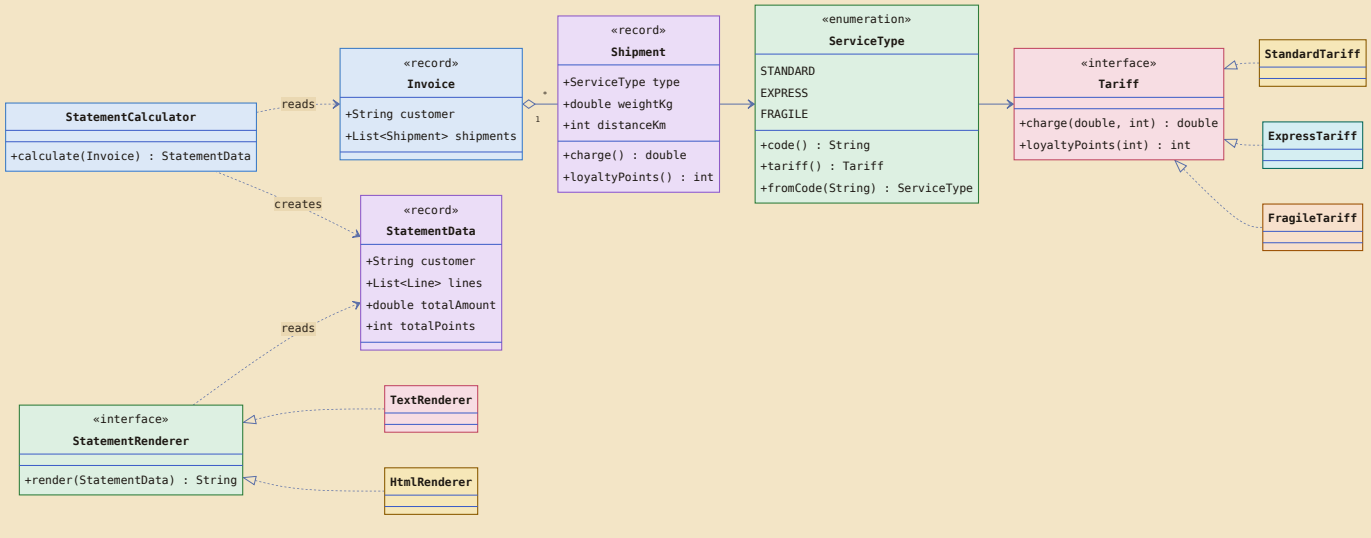
Step 5 — Replace Conditional with Polymorphism, Move Function, then the new feature

Each switch branch becomes a `Tariff` class (a **Strategy**); `ExpressTariff` overrides `loyaltyPoints`, removing the second `"EXP"` check. Move Function puts `charge()` / `loyaltyPoints()` on `Shipment` (Feature Envy cured). The calculator has **no conditionals**.

```java
public sealed interface Tariff permits StandardTariff, ExpressTariff, FragileTariff {

    double charge(double weightKg, int distanceKm);

    /** Default behaviour shared by most tariffs; ExpressTariff overrides it. */
    default int loyaltyPoints(int distanceKm) {
        return distanceKm / 100;
    }
}

public record ExpressTariff() implements Tariff {
    private static final double BASE = 12, PER_KG = 2.0, FREE_KM = 50, PER_EXTRA_KM = 0.10;
    private static final int BONUS_POINTS = 2;
    // ... charge(...) ...
    @Override
    public int loyaltyPoints(int distanceKm) {
        return Tariff.super.loyaltyPoints(distanceKm) + BONUS_POINTS;
    }
}

public record Shipment(ServiceType type, double weightKg, int distanceKm) {
    // ... compact constructor: requireNonNull(type) ...
    public double charge() { return type.tariff().charge(weightKg, distanceKm); }

    public int loyaltyPoints() { return type.tariff().loyaltyPoints(distanceKm); }
}

/** Phase 1. No switch left: adding a tariff never touches this class (Open/Closed). */
public final class StatementCalculator {

    public StatementData calculate(Invoice invoice) {
        List<StatementData.Line> lines = invoice.shipments().stream()
                .map(s -> new StatementData.Line(s.type(), s.weightKg(), s.distanceKm(), s.charge()))
                .toList();
        double total = lines.stream().map(StatementData.Line::amount).reduce(0.0, Double::sum);
        int points = invoice.shipments().stream().mapToInt(Shipment::loyaltyPoints).sum();
        return new StatementData(invoice.customer(), lines, total, points);
    }
}
```

Full runnable code: code/worked/step5/ (package `lld.ch07.worked.step5`; see the Full Edition).

Then, **adding-function hat**: extract a `StatementRenderer` interface, add a ~15-line `HtmlRenderer` reusing phase 1 unchanged. OVERNIGHT later = one `Tariff` record + one enum constant + one name in `permits`. Calculator and renderers don't change (Open/Closed).



What changed, step by step

Step	Refactorings	Smells removed
1	Extract Function ×3, Rename Variable/Field, Remove Dead Code	Mysterious Name, part of Long Function, dead code
2	Inline Variable, Split Loop, Replace Temp with Query, Replace Loop with Pipeline, Slide Statements	Long Function, Loops
3	Split Phase, Combine Functions into Transform, Encapsulate Record/Collection	Divergent Change, Data Class, Mutable Data
4	Replace Primitive with Object, Replace Magic Literal	Primitive Obsession, magic numbers
5	Replace Conditional with Polymorphism, Move Function (+ extract renderer interface)	Repeated Switches, Feature Envy

The checker prints `SAME output` for step0 … step5 and ends with `All steps preserve behaviour.`

Deliberately not done: money stays `double`. `BigDecimal` would change rounding — a behaviour change for the other hat, with its own tests and a product decision.

7.6 Refactoring toward patterns

What it is

You often **arrive at a pattern by refactoring** instead of designing it up front. Kerievsky's *Refactoring to Patterns* (2004) catalogues these moves, mostly compositions of Fowler's refactorings. Don't force patterns early (Speculative Generality); refactor *toward* one when a smell appears — and *away* when it stops earning its keep.

How it works

Smell	Toward	Via
Switch choosing an algorithm	**Strategy**	Extract Function per branch → Replace Conditional with Polymorphism (*Replace Conditional Logic with Strategy*)
Methods checking and changing a status field	**State**	Replace Type Code with Subclasses (indirect) → Replace Conditional with Polymorphism (*Replace State-Altering Conditionals with State*)
`if (x == null)` everywhere	**Null Object**	Introduce Special Case
Siblings with same skeleton, different steps	**Template Method**	Extract Function → Pull Up Method (*Form Template Method*)
Confusing constructors	**Creation Methods**	Replace Constructor with Factory Function
Scattered `new ConcreteX()`	**Factory**	+ Move Function to a factory class (*Encapsulate Classes with Factory*)
Complex (Composite) object built by many ordered calls	**Builder**	*Encapsulate Composite with Builder*
Huge function with shared locals; undo/queue	**Command**	Replace Function with Command
Logging/caching mixed into a core class	**Decorator**	*Move Embellishment to Decorator*
Hard-coded notifications / polling	**Observer**	*Replace Hard-Coded Notifications with Observer*
Same job, different interfaces	**Adapter**	*Extract Adapter*, *Unify Interfaces with Adapter*
Subclass explosion for feature combinations	**Strategy / Decorator / Bridge**	Replace Subclass with Delegate
Pattern no longer earning its keep	*Remove it*	Inline Class, Collapse Hierarchy (*Inline Singleton*)

Package `lld.ch07.towardpatterns` (`code/towardpatterns/`) shows five of these with before and after side by side and checks they agree.

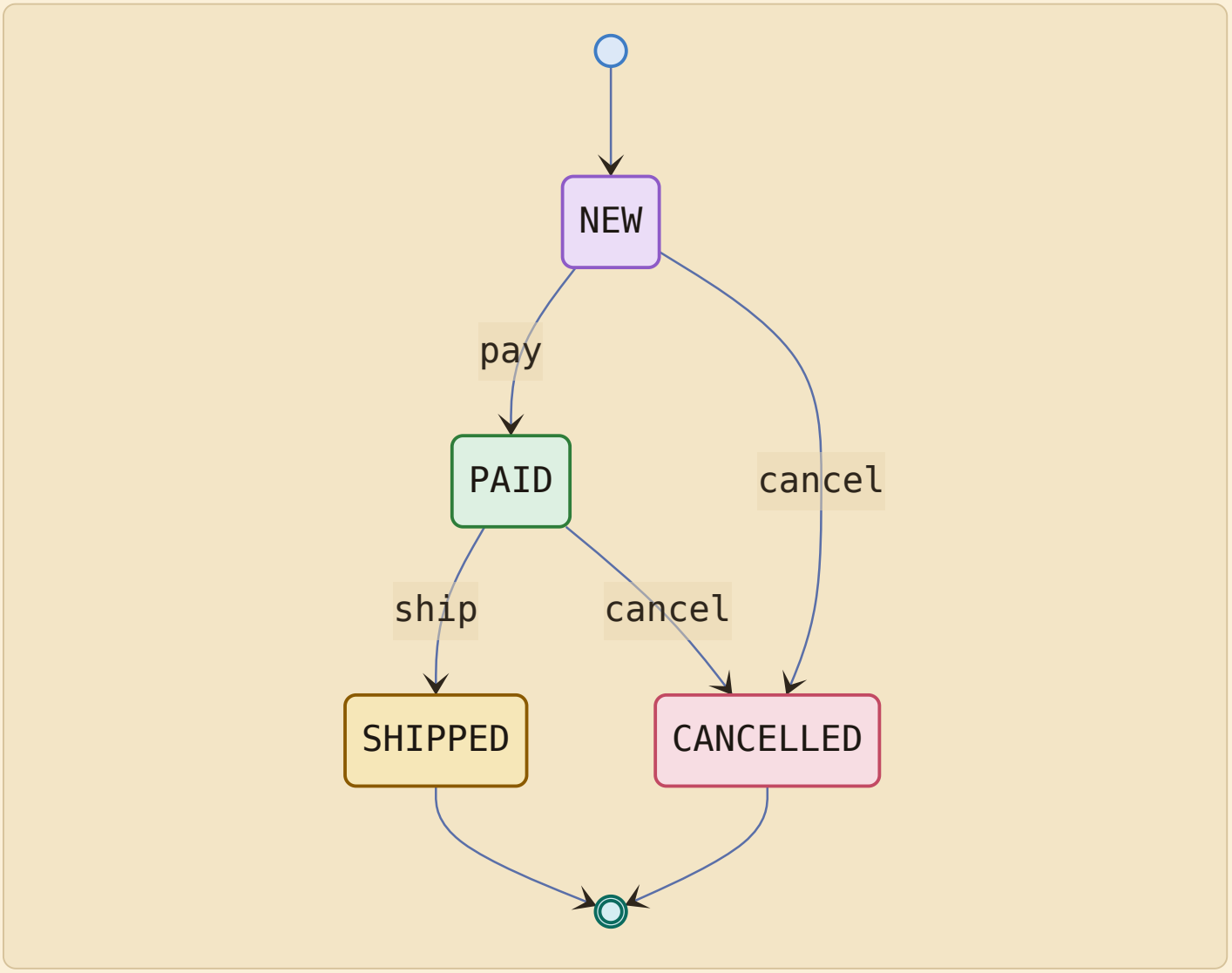
Replace Conditional Logic with Strategy

A `switch (tier)` becomes a `DiscountPolicy` per tier stored in an `EnumMap`; new tiers are **registered**, not coded.

```
    @FunctionalInterface
    interface DiscountPolicy {
        double apply(double amount);
    }
    // ...
    static final class PriceCalculator {
        private final Map<Tier, DiscountPolicy> policies = new EnumMap<>(Tier.class);
        private final DiscountPolicy fallback = amount -> amount;
        // ... constructor registers GOLD, SILVER, NEW lambdas ...
        /** New tiers are registered, not coded into a switch (Open/Closed). */
        void register(Tier tier, DiscountPolicy policy) { policies.put(tier, policy); }

        double price(Tier tier, double amount) {
            return policies.getOrDefault(tier, fallback).apply(amount);
        }
    }
```

Replace State-Altering Conditionals with State

Each state is an enum constant overriding only the moves it allows; default methods reject the rest. `Order` just does `state = state.pay(this)`.




```
    enum States implements OrderState {
        NEW {
            @Override public OrderState pay(Order o)    { o.log("paid"); return PAID; }
            @Override public OrderState cancel(Order o) { o.log("cancelled"); return CANCELLED; }
        },
        PAID {
            @Override public OrderState ship(Order o)   { o.log("shipped"); return SHIPPED; }
            @Override public OrderState cancel(Order o) { o.log("cancelled"); return CANCELLED; }
        },
        SHIPPED, CANCELLED
    }
```

Strategy vs State: same structure, different intent — a strategy is chosen by the client and stays; a state replaces *itself* as the object moves through its lifecycle.

Introduce Special Case (Null Object)

`find(id)` returns an `UnknownCustomer` answering `"occupant"` / `"basic"`, so clients have no null checks.

Form Template Method

`legacyCsv` and `legacyMarkdown` repeat header/rows/footer. After: `Exporter.export()` is `final` and calls abstract `header()`, `row()`, `footer()`; `CsvExporter` and `MarkdownExporter` fill them in.

Replace Constructors with Creation Methods

`new LegacyLoan(5000, false, 12)` — what is `false`? After: private constructor + `Loan.termLoan(5000, 12)` and `Loan.revolver(2000)`.

```
Strategy:        GOLD 250 -> 200.0, NEW 250 -> 240.0 | same as switch: true
State:           cannot ship when NEW; paid; cannot pay when PAID; shipped; cannot cancel when SHIPPED; | same as if-chains: true
All pattern refactorings preserve behaviour.
```

Full runnable code: code/towardpatterns/ (package `lld.ch07.towardpatterns`; see the Full Edition).

Small smell cures (package `lld.ch07.smells`)

`code/smells/` shows `Money` (Primitive Obsession), `DateRange` (Data Clumps), and `Order` with `shippingCity()` (Hide Delegate), `Line.total()` (Move Function) and a read-only `lines()` (Encapsulate Collection).

```
double 0.1+0.2       = 0.30000000000000004
Money  0.10+0.20     = USD 0.30
mixing currencies    -> currency mismatch: USD vs EUR
outsider tried lines().clear() -> UnsupportedOperationException (collection is encapsulated)
```

7.7 Putting it together

Trade-offs

Benefit	Cost / risk
Easier to change; features get faster (Design Stamina)	Takes time now; pay-off later, hard to measure
Bugs become visible	Without tests you can introduce bugs
Design evolves with requirements	Renames/moves cause merge conflicts on long branches
Small steps are safe, can stop anytime	Feel slow to beginners
Small, understandable units	Too much indirection → Middle Men, Lazy Elements
Removes duplication	Premature abstraction → the *wrong* abstraction, costlier than duplication

When not to use it

Not for code you don't touch; not without a safety net; not instead of a rewrite when the code doesn't work at all; not on published interfaces without a migration path; never mixed with behaviour changes in the same step.

Common mistakes

Mistake	Do this instead
Calling any change "refactoring"	Separate refactoring commits from feature/fix commits
Big-bang refactoring	Tiny steps, test after each, commit often
Refactoring without tests	Characterisation tests first; IDE-automated refactorings
Text search-and-replace renames	Symbol-aware IDE Rename; check reflection/config/serialisation
Abstracting on the first duplicate	Rule of Three
Changing rounding/ordering "while there" (`DoubleStream.sum()`, `toList()`)	Know the replacement API's exact semantics
Patterns everywhere "for flexibility"	Refactor toward a pattern when a smell demands it

Real-world examples

- **IDEs** automate catalogue entries; **OpenRewrite** (Spring Boot upgrades, JUnit 4 → 5) and Google's **Refaster** do it at scale.
- **JDK:** `Stack extends Vector`, `Properties extends Hashtable` — Refused Bequest (use `ArrayDeque`). `Date` → `java.time` replaced mutable primitive-ish types with immutable values, as a *new* API beside the published old one.
- **Strangler Fig** for monolith-to-services migrations.

In the interview

30-second answer — "What is refactoring?" "Changing the internal structure of code to make it easier to understand and modify, without changing observable behaviour. I do it in very small steps, each compiling and passing tests, so the system is never broken. Smells like Long Function or Repeated Switches tell me where; named refactorings like Extract Function or Replace Conditional with Polymorphism tell me how. Tests are the safety net, and the goal is economic: clean code lets us add features faster."

- **Q: How do you refactor code with no tests?** A: Characterisation tests first; break dependencies at a seam (inject an interface instead of `new`); until then use only IDE-automated refactorings; Sprout Method/Class for new code.
- **Q: What would you refactor here?** (machine coding) A: Name smell + cure: "`processOrder` is a Long Function → Extract Function; the `vehicleType` switch in three places is Repeated Switches → Replace Conditional with Polymorphism (Strategy); `double price` is Primitive Obsession → `Money`; `getParkingLot().getFloor(1).getSpot(3)` is a Message Chain → Hide Delegate."
- **Q: How do you rename a public API method used by other teams?** A: Migration mechanics: new method, old one delegates and is `@Deprecated`, migrate callers, then delete. For REST: versioning or expand/contract.

Self-check

1. **Q:** Fixing a bug while tidying a method — one refactoring? **A:** No; the fix changes behaviour. Two hats, separate steps and commits.
2. **Q:** `book(hotel, from, to, 2, 1, true)` — smells? **A:** Long Parameter List, Data Clumps, flag argument → Introduce Parameter Object (`Stay`, `Guests`), Remove Flag Argument / an enum like `MealPlan`.
3. **Q:** Why `reduce(0.0, Double::sum)` and not `DoubleStream.sum()`? **A:** `sum()` may use compensated summation and differ in the last bit; `reduce` adds in order like the loop.
4. **Q:** An IDE field rename broke the JSON response — was it a refactoring? **A:** It was meant to be, but the JSON name is observable behaviour, so no. Check serialisation, reflection and config.

TL;DR

- **Refactoring = small, behaviour-preserving changes**, tests after every step, never mixed with behaviour changes (two hats).
- **When:** Rule of Three, preparatory, comprehension, litter-pickup, code review — mostly opportunistic. **Not** for untouched code or without a safety net.
- **24 smells** say *where*: Mysterious Name, Duplicated Code, Long Function, Long Parameter List, Global Data, Mutable Data, Divergent Change, Shotgun Surgery, Feature Envy, Data Clumps, Primitive Obsession, Repeated Switches, Loops, Lazy Element, Speculative Generality, Temporary Field, Message Chains, Middle Man, Insider Trading, Large Class, Alternative Classes with Different Interfaces, Data Class, Refused Bequest, Comments.
- **61 refactorings** say *how*; the everyday core: Extract/Inline Function, Rename, Move Function, Extract Class, Split Phase, Replace Conditional with Polymorphism.
- **Performance:** clean first, then profile. **Rewrite** rarely, with a Strangler Fig. **Legacy code** = no tests → characterisation tests, seams, sprout/wrap.
- **Patterns** emerge: switch → Strategy/State, null checks → Null Object, duplicated skeleton → Template Method, confusing constructors → factory methods.

7.8 Full source code and output

The complete listings (checker, steps 0–5, toward-patterns, smell cures) are in the Full Edition (README.md, section 7.8). Compile and run with Java 21:

```
cd chapters/07-refactoring/code
javac --release 21 -d /tmp/lld_07 $(find . -name '*.java')
java -cp /tmp/lld_07 lld.ch07.worked.Main           # characterisation check over all six steps
java -cp /tmp/lld_07 lld.ch07.worked.step5.Main     # final design + new HTML feature
java -cp /tmp/lld_07 lld.ch07.towardpatterns.Main   # refactoring toward patterns
java -cp /tmp/lld_07 lld.ch07.smells.Main           # small smell cures
```

Packages: `code/worked/` (checker), `code/worked/step0/`, `code/worked/step1/`, `code/worked/step2/`, `code/worked/step3/`, `code/worked/step4/`, `code/worked/step5/` (steps 0–4 print the text invoice only), `code/towardpatterns/`, `code/smells/`.

Sources: Fowler, *Refactoring*, 2nd ed. (2018) and refactoring.com; refactoring.guru's smell groups; Feathers, *Working Effectively with Legacy Code* (2004); Kerievsky, *Refactoring to Patterns* (2004).

Chapter 8 — Design Patterns: Introduction & Creational Patterns (Study Edition)

> **Study Edition.** A shorter version of Chapter 8 with every topic kept and the padding removed. The full chapter (README.md) has the complete code, full outputs and more follow-ups.
>
> **What you will learn.** What a design pattern is, where the idea came from, all 23 GoF patterns, the OO principles behind them, how to pick (and not pick) a pattern, anti-patterns, and the creational patterns in depth: Singleton, Simple Factory (idiom), Factory Method, Abstract Factory, Builder, Prototype and the bonus Object Pool.
>
> **Code.** `chapters/08-patterns-creational/code/lld/ch08/<pattern>/`, one `Main` per package (Java 21: `javac --release 21`, then `java lld.ch08.<pattern>.Main`).

Part A — Design patterns: the big picture

8.1 What is a design pattern?

What it is

A **design pattern** is a named, reusable description of a good solution to a recurring design problem. It is not a library or paste-in code — more like a *recipe* you adapt. (Analogy: builders learned over hundreds of houses where the front door works best; writing that down, with reasons and costs, is a pattern.)

- **Intent** — the one-sentence purpose ("ensure a class has only one instance").
- **Participants** — the roles that take part (*Creator*, *Product*, *Client*).

The problem it solves

Without patterns every team re-solves the same problems in slightly different, slightly buggy ways. Patterns give proven solutions, readable code, built-in good principles (loose coupling, open/closed), adaptability and faster decisions — and a **shared vocabulary** ("use an Observer" replaces a paragraph), the biggest interview win.

How it works

Every GoF pattern has four parts: **Name**, **Problem** (when to apply, forces in tension), **Solution** (participants and collaborations — a template, not code), **Consequences** (costs and benefits — a pattern without trade-offs is half a pattern). You adapt the template to your language and problem.

When not to use it

Not every problem needs a pattern. Forcing one adds classes and indirection — worse than none (see 8.6).

TL;DR

Pattern = **name + problem + solution template + consequences**. Value: vocabulary and proven trade-offs. Risk: overuse.

8.2 A short history: from buildings to the "Gang of Four"

Year	Event
1977	Architect **Christopher Alexander**, *A Pattern Language* — named solutions for towns and buildings
1987	**Kent Beck** and **Ward Cunningham** apply pattern languages to Smalltalk UI design
1994	**Gamma, Helm, Johnson, Vlissides** — the **Gang of Four (GoF)** — publish *Design Patterns*: **23 patterns** in three categories
2004	*Head First Design Patterns* — the same patterns in Java, OO principles front and centre

The GoF's two central principles:

1. **Program to an interface, not an implementation.**
2. **Favour object composition over class inheritance** ("has-a" black-box reuse over "is-a").

Creational patterns make principle 1 practical: if code must not name concrete classes, *someone* still has to create the objects.

8.3 The three categories and all 23 GoF patterns

What it is

Category	In one line
Creational	Control creation so the system doesn't depend on concrete classes or construction details
Structural	Compose classes/objects into larger, flexible structures
Behavioral	Assign responsibilities and design communication

Second GoF axis, **scope**: **class patterns** use inheritance, fixed at compile time (Factory Method, class Adapter, Template Method, Interpreter); **object patterns** use composition, changeable at run time (the rest).

The full catalogue

#	Pattern	Category	Intent
1	**Abstract Factory**	Creational	Create *families* of related objects without naming concrete classes
2	**Builder**	Creational	Build a complex object step by step, separate from its representation
3	**Factory Method**	Creational	Subclasses decide which class to instantiate via an overridable method
4	**Prototype**	Creational	Create objects by copying a configured instance
5	**Singleton**	Creational	Exactly one instance with a global access point
6	**Adapter** (Wrapper)	Structural	Convert one interface into another
7	**Bridge**	Structural	Split abstraction from implementation so both vary
8	**Composite**	Structural	Treat single objects and trees uniformly
9	**Decorator**	Structural	Add responsibilities dynamically by wrapping
10	**Facade**	Structural	Simple entry point to a complex subsystem
11	**Flyweight**	Structural	Share intrinsic state across huge numbers of objects
12	**Proxy**	Structural	Stand-in controlling access to another object
13	**Chain of Responsibility**	Behavioral	Pass a request along handlers until one handles it
14	**Command**	Behavioral	Request as an object — queue, log, undo
15	**Interpreter**	Behavioral	Grammar as classes; evaluate sentences
16	**Iterator**	Behavioral	Traverse without exposing storage
17	**Mediator**	Behavioral	Centralise communication between objects
18	**Memento**	Behavioral	Capture/restore state without breaking encapsulation
19	**Observer**	Behavioral	Notify dependents when one object changes
20	**State**	Behavioral	Behaviour changes with internal state
21	**Strategy**	Behavioral	Interchangeable algorithms behind one interface
22	**Template Method**	Behavioral	Base class fixes the skeleton, subclasses fill steps
23	**Visitor**	Behavioral	Add operations without changing the classes

Creational → this chapter, structural → Ch. 9, behavioral → Ch. 10–11. **5 + 7 + 11 = 23.** (Some course lists show ten behavioral patterns, omitting the rarely used Interpreter.) **Not GoF but common:** Simple Factory (idiom), Object Pool, Dependency Injection, Null Object, Repository, MVC (a "compound pattern" of Observer + Strategy + Composite).

Self-check

- **Q:** Which creational pattern is a *class* pattern? **A:** Factory Method — subclasses override the creation method.
- **Q:** How many behavioral GoF patterns? **A:** Eleven.

8.4 The OO principles behind every pattern

Principle	Patterns applying it
Encapsulate what varies	Factory Method, Strategy, State, Builder
Composition over inheritance	Abstract Factory (client *holds* a factory), Strategy, Decorator
Program to interfaces	Every creational pattern returns an interface
Loose coupling	Observer, Mediator, factories
Open–Closed	Factory Method, Decorator
Dependency Inversion	All factories — the client never calls `new ConcreteThing()`
Least Knowledge	Facade
Hollywood Principle	Template Method, Factory Method, frameworks/IoC
Single Responsibility	Builder, Iterator

Why creational patterns exist: DIP says don't depend on concrete classes, but `new PdfDocument()` *is* such a dependency — so creational patterns move that `new` somewhere controlled (a factory, subclass, builder, prototype or container).

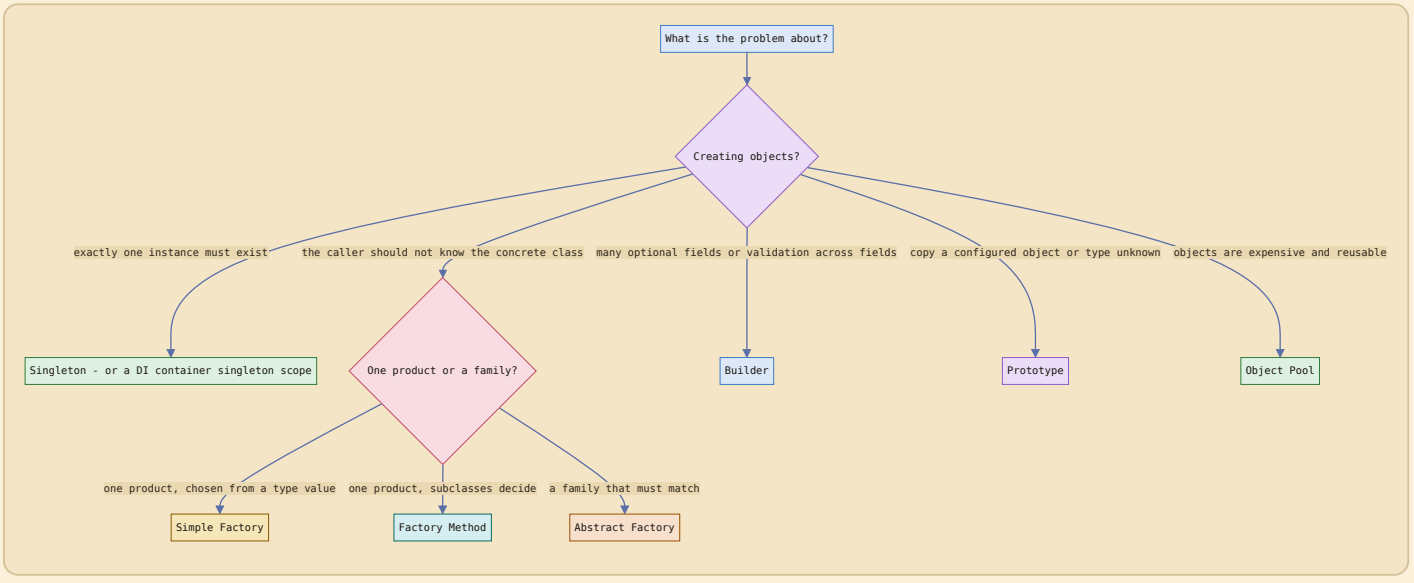
8.5 How to pick a pattern

The problem it solves

Learning 23 patterns and hunting for places to use them is backwards. Start from a **signal** in the requirements.

How it works

1. **Name the problem** in plain words ("pricing will change", "exactly one of these", "ten-argument constructor").
2. **Decide the category** — creating, structuring or behaving.
3. **Match the signal** to a pattern.
4. **Check the cost** — is there something simpler (enum, lambda, constructor, DI container)?
5. **Say what you rejected and why** — worth as much as the pick.



(For fitting objects together: Adapter, Decorator, Facade; for behaviour: Strategy, Observer, State.)

The six patterns that come up in almost every LLD interview

Learn each with its **signal** — forcing a pattern where it doesn't fit costs more than not knowing it.

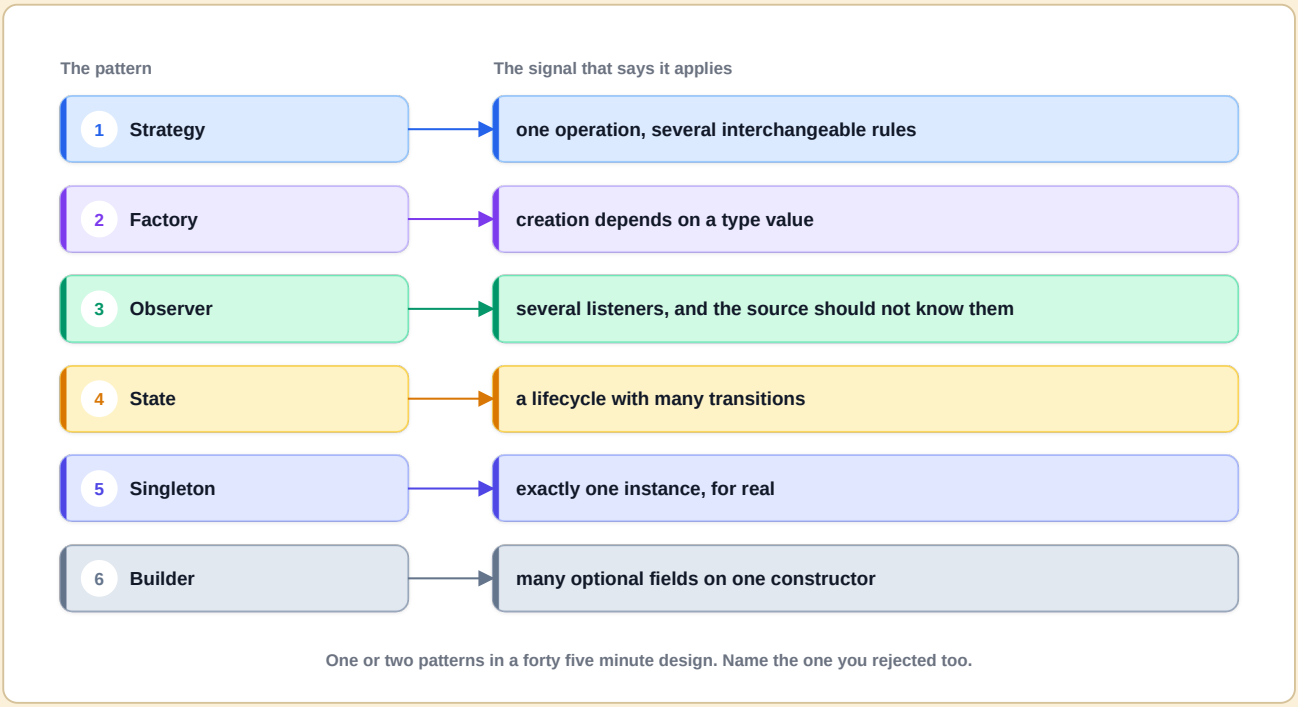


Figure — the six interview patterns and the signal that tells you each one applies (course diagram)

Pattern	Signal	Use when	Don't use when
Strategy	One operation, several swappable versions chosen at run time	Several interchangeable rules	Only one rule, no hint of a second
Factory	Something must be created; the caller shouldn't care which class	Class depends on a type value	Only one concrete class
Observer	One object changes, several others must react unknown to it	Several listeners	A single listener already referenced
State	Behaviour depends on status; allowed moves are fixed	Many states and transitions	3–4 states handled by one guarded method
Singleton	Genuinely one instance	Must be exactly one	You just want a global handle
Builder	Many optional fields; constructor keeps growing	Lots of optional fields	A few required fields

In a parking-lot / library design: **Strategy** — a `ParkingRate` interface (`HourlyRate`, `WeekendRate`, `MonthlyPass`), the answer to "what if pricing changes?"; **Factory** — type value → `Vehicle`, keep it lean (a creation method is often enough in 45 min); **Observer** — spot as subject, boards subscribe (but one reserved-book member needs only a direct call); **State** — ticket active → paid → closed (with 3–4 states an `enum` + guard is clearer); **Singleton** — one `ParkingLot`, raise thread safety and testability yourself; **Builder** — a `BookItem` past ~6 arguments.

Rules of thumb: one or two patterns per 45-minute design, placed where their signal appears; **compare out loud** — "Strategy for rates because a second tariff is coming; no State for the ticket because one guard handles three statuses."

8.6 Anti-patterns: when patterns go wrong

What it is

An **anti-pattern** is a common "solution" that looks reasonable but reliably causes more problems than it solves.

Anti-pattern	Looks like	Fix
God Object (Blob)	`SystemManager` with 80 methods — violates SRP, untestable	Extract Class (Ch. 7); move behaviour to data owners
Golden Hammer	Favourite tool/pattern applied everywhere	Start from the problem's signal (8.5)
Pattern overuse ("patternitis")	Factories for one class, interfaces "just in case" — **Speculative Generality**	YAGNI: add the pattern when the *second* variant arrives
Singletonitis	Singletons as globals — hidden deps, order-dependent tests	Dependency injection (8.8)
Cargo-cult patterns	Structure without intent (a "Builder" that validates nothing)	Know the property the pattern gives you, or drop it

TL;DR

Patterns are a vocabulary, not a checklist. God Object comes from ignoring design; Golden Hammer and overuse from over-applying it.

Part B — Creational patterns

8.7 Creational patterns at a glance

What it is

Creational patterns control *how objects are created*, hiding *which* class is instantiated, *how* it is assembled, and sometimes *whether* a new object is made. The system becomes independent of creation, hides creation complexity, and stays flexible about what/who/how/when.

Pattern	Intent
Singleton	Exactly one instance, one global way to reach it
Factory Method	Base class defines a creation method; subclasses choose the class
Abstract Factory	One factory object creates a matching *family*
Builder	Assemble one complex object step by step
Prototype	Create by cloning a prototypical instance
Simple Factory (idiom)	One method with a switch/registry picks the class
Object Pool (bonus)	Reuse a bounded set of expensive objects

8.8 Singleton

What it is

Singleton restricts a class to **exactly one instance** (per process / class loader) with **one well-known access point** — like a country's one government. **Eager** = create at class initialisation; **lazy** = on first request.

GoF intent and participants. Intent: never more than one instance; a well-known way to reach it. One participant, **Singleton**: a class-level `getInstance()` operation, usually also creating the instance. Clients just call it.

The problem it solves

Some things are one per process: settings loaded from a file, a connection pool, a metrics registry. If billing, reporting and the mailer each build their own `ConfigStore`, the file is parsed three times and one service's change never reaches the others. A plain global has two problems: tests share it (results depend on test order), and someone must assign it before use (wrong start-up order → `null`). Singleton makes **the class decide how many instances exist and create the one it allows**, on demand.

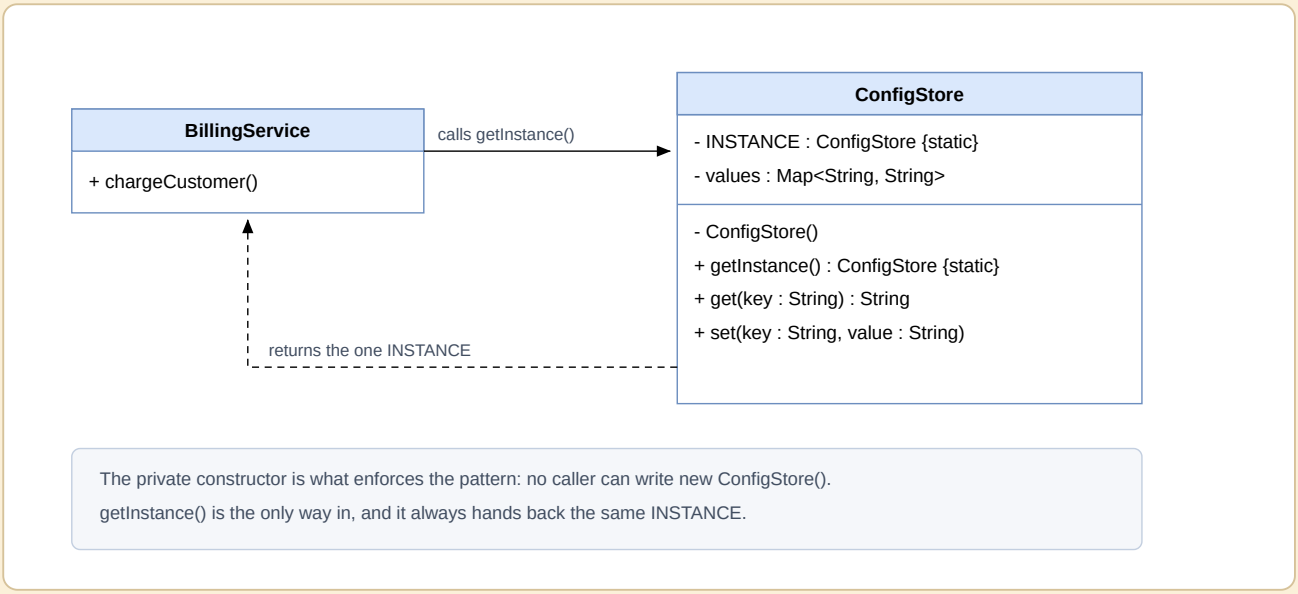
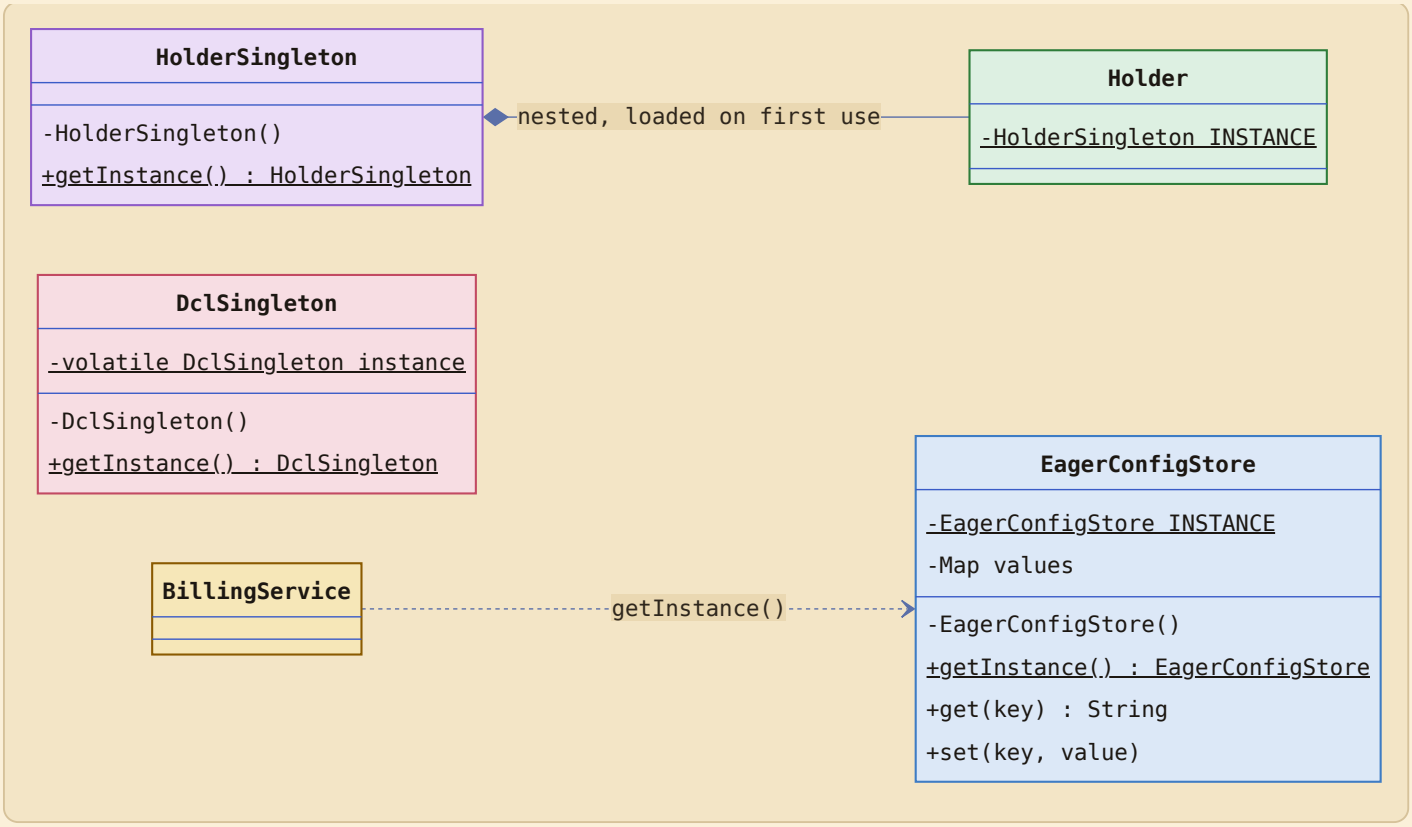


Figure — ConfigStore with a private static INSTANCE, a private constructor and a static getInstance(); BillingService only calls getInstance() (course diagram)

How it works

1. `private` `constructor` — the whole enforcement mechanism.
2. The instance in a `private static` `field` (owned by the class).
3. A `public static getInstance()` (static: callers have no instance yet).
4. Return the instance, creating it (safely) if needed.
5. Ordinary instance methods for the real work.



The six ways to write a Singleton in Java

#	Variant	Lazy?	Thread-safe?	Per-call cost	Survives serialization / reflection?	Verdict
1	**Eager** `static final`	No	Yes — class init runs once	None	No / No	Default when construction is cheap
2	**Naive lazy** `if (instance == null)`	Yes	**No**	None	No / No	Never in multi-threaded code
3	`synchronized` **getInstance()**	Yes	Yes	Lock on *every* call	No / No	Correct but slow under contention
4	**Double-checked locking** + `volatile`	Yes	Yes (Java 5+)	One volatile read	No / No	When creation needs run-time input
5	**Bill Pugh holder**	Yes	Yes — class loading	None	No / No	Best lazy option in plain Java
6	`enum`	No	Yes	None	**Yes / Yes**	Shortest, most robust (*Effective Java*)

- **Eager:** set during class *initialisation* (first active use, not JVM start); JLS §12.4.2 guarantees once-only, safely published. Cost: built even if the class is touched for another reason.
- **Naive lazy:** *check-then-act* — two threads both see `null` and both construct. Passes single-threaded tests; our 8-thread demo built **8** objects.
- `synchronized`**:** fixes the race but locks forever for a first-call problem.
- **DCL:** check without the lock, lock, **check again**. The field **must be** `volatile`: object creation is allocate → construct → store, and without `volatile` another thread may see the reference before the constructor's writes (a half-built object). `volatile` gives a **happens-before** edge. Copying to a local reads the volatile once on the fast path.
- **Holder:** `private static final class Holder { static final X INSTANCE = new X(); }` and `getInstance()` returns `Holder.INSTANCE`; `Holder` isn't initialised until first touched — lazy, thread-safe, **no locks, no** `volatile`.

- **Enum** (`enum EnumConfigStore { INSTANCE; ... }`): one instance per constant, reflective creation refused, serialization special-cased. Can't extend a class; not truly on-demand.

Which to pick: default **eager**; **holder** once you've *measured* a start-up cost; **DCL** when creation needs a run-time value (and always `volatile`); **enum** for free serialization/reflection safety.

```
/** 4. Double-checked locking: lock only on the first calls. volatile is mandatory. */
final class DclSingleton {
    static final AtomicInteger CONSTRUCTED = new AtomicInteger();
    private static volatile DclSingleton instance;
    // ... private constructor ...
    static DclSingleton getInstance() {
        DclSingleton local = instance;           // one volatile read on the fast path
        if (local == null) {                     // 1st check, no lock
            synchronized (DclSingleton.class) {
                local = instance;
                if (local == null) {             // 2nd check, under the lock
                    local = new DclSingleton();
                    instance = local;            // volatile write publishes a fully built object
                }
            }
        }
        return local;
    }
}
```

Breaking a Singleton — and the fixes

Attack	How	Fix
Reflection	`setAccessible(true).newInstance()` calls the private constructor	Throw from the constructor if `INSTANCE` exists (reliable for eager/holder, racy for lazy), or **enum**
Serialization	`readObject()` builds a new object without the constructor	`private Object readResolve() { return INSTANCE; }` (+ `transient` fields), or **enum**
Cloning	A `Cloneable` ancestor exposing `clone()`	Override `clone()` to throw (not an issue if no ancestor is `Cloneable` — `Object.clone()` is protected)
Multiple class loaders	Each loader has its own copy	Load from a shared parent loader, or let a container own it
Multiple processes	One per JVM — four servers, four instances	External coordinator (DB, Redis, ZooKeeper) for system-wide uniqueness

`HardenedSingleton` adds all three defences (constructor guard, `readResolve`, throwing `clone`). Output: the naive lazy version under 8 threads — `constructor ran 8 time(s)` (varies; anything above 1 is the bug), DCL — `1`; the hardened and enum versions print `reflection -> blocked` and `serialization -> same? true`.

Subclassing a Singleton

GoF: possible with a `protected` constructor, where `getInstance()` picks the subclass **once, at run time** — `AppLogger` chooses `PlainLogger` or `JsonLogger` from a system property. With a `private` constructor it can't be subclassed.

The dependency-injection alternative

Often the best singleton is **not a Singleton class**: a normal class, **one** instance created at the **composition root**, passed in through constructors. Dependencies become visible; tests pass a fake (`new BillingService(testConfig)`). Spring beans are **singleton-scoped by default** — one per bean definition **per container**, not per JVM, and nothing stops `new`. Spring's `prototype` **scope** creates a fresh instance per lookup.

Java implementation

The package also has `EagerConfigStore`, `EnumConfigStore`, `BreakableSingleton`/`HardenedSingleton`, `AppLogger` and `DiAlternative`.

Full runnable code: code/lld/ch08/singleton/ (see the Full Edition).

Trade-offs

Pros	Cons
Controlled access — one instance, one documented door	**Global state** — shared mutable state anyone can change
You choose when — eager or lazy	**Awkward to test** — needs a reset/injection hook that ships to production
Fits shared resources — pool, cache, device handle	**Invisible dependencies** — `getInstance()` inside isn't in the constructor
Setup paid once per process	**Uniqueness ends at the process** (class loader)
Subclassable if designed in (`protected`)	**Hard to undo**; mixes lifecycle management with real work (SRP)

Often confused with: a **static utility class** (not an object — can't implement an interface or be mocked); **DI singleton scope** (same result, better mechanism); **Multiton** (`Logger.getLogger(name)` — one per key); Prototype (opposite intent). Elsewhere: Python module object, Go `sync.Once`, C++ Meyers Singleton.

When not to use it

- **The motive is convenience** — a parameter delivers access just as well.
- **Mutable state tests touch** — tests pass alone, fail in the suite.
- **A framework manages lifecycles** — use singleton scope.
- **Multiple processes** — four servers = four "singletons"; using one for distributed locks or unique IDs is a **correctness bug**.

Common mistakes

Naive lazy in multi-threaded code; DCL without `volatile`; `Serializable` without `readResolve()`; mutable state without thread-safe structures (`ConfigStore` uses `ConcurrentHashMap`); a `public static reset()` "for tests"; assuming one per system rather than per JVM/class loader.

Real-world examples

- **JDK:** `Runtime.getRuntime()` (textbook), `Desktop.getDesktop()`; `Collections.emptyList()` (singleton-like shared constant); `Logger.getLogger(name)` (multiton).
- **Not singletons despite `getInstance()`:** `Calendar.getInstance()`, `NumberFormat.getInstance()`, `Cipher.getInstance()` return **new objects** — static factory methods (8.9).
- Spring singleton beans; connection pools, metrics registries; *Head First*'s chocolate boiler.

In the interview

> **30-second answer:** "Singleton guarantees one instance per JVM (strictly, per class loader) with a single access point: private constructor, static accessor. I'd use eager initialisation — the class loader makes it thread-safe for free — or the holder idiom if construction is expensive; an enum if it must survive serialization or reflection. But it's global state with hidden dependencies, so in a Spring app I'd make it a singleton-scoped bean and inject it."

- *Q: Why does DCL need `volatile`?* — A: Otherwise the reference can be published before the constructor's writes are visible; `volatile` gives a happens-before edge.
- *Q: Unique IDs across 10 servers?* — A: Not a Singleton (one per JVM). Use a DB sequence, Snowflake-style IDs (timestamp + machine id + counter) or UUIDs.

Self-check

- **Q:** Which keyword enforces the pattern? **A:** `private` on the constructor.
- **Q:** Lazy, lock-free, thread-safe variant? **A:** The holder idiom.
- **Q:** What does `readResolve` do? **A:** Replaces the deserialized object with the real instance.

TL;DR

Private constructor + static accessor = one per JVM. Prefer **eager**, then **holder**, then **enum**; **DCL needs** `volatile`; never naive lazy. Know the ways to break it. Mention the costs; DI singleton scope is usually better.

8.9 Simple Factory (an idiom, not a GoF pattern)

What it is

One class (usually one static method) takes a type value and returns the matching object as a common interface. Very common, often right, but **not** GoF — *Head First* calls it an idiom. Analogy: a vending machine — press "B3", get the snack.

The problem it solves

The `switch` turning `"sms"` into `new SmsNotification()` gets copied everywhere. A simple factory **centralises** the decision — it doesn't **remove** it: adding a channel still edits the factory. A registry softens this (types are *registered* at start-up).

How it works

1. Product interface (`Notification`). 2. Concrete classes. 3. One `create(type)` — an exhaustive `switch` over an enum, or a `Map<String, Supplier<Notification>>` registry. 4. Callers hold the interface, never `new`. 5. Reject unknown types loudly.

Java implementation

```java
/** Simple Factory (an idiom, not a GoF pattern): one static method owns the "which class?" decision. */
public final class NotificationFactory {

    public enum Channel { EMAIL, SMS, PUSH }

    private NotificationFactory() { }

    /** Version 1: a switch. Adding a channel means editing this method (and the enum). */
    public static Notification create(Channel channel) {
        return switch (channel) {                    // exhaustive switch: the compiler checks every enum value
            case EMAIL -> new EmailNotification();
            case SMS   -> new SmsNotification();
            case PUSH  -> new PushNotification();
        };
    }

    /** Version 2: a registry of suppliers. New types are registered, not coded into a switch. */
    private static final Map<String, Supplier<? extends Notification>> REGISTRY = new ConcurrentHashMap<>(Map.of(
            "email", EmailNotification::new,
            "sms", SmsNotification::new,
            "push", PushNotification::new));

    public static void register(String type, Supplier<? extends Notification> supplier) {
        REGISTRY.put(type.toLowerCase(Locale.ROOT), supplier);
    }

    public static Notification create(String type) {
        Supplier<? extends Notification> s = REGISTRY.get(type.toLowerCase(Locale.ROOT));
        if (s == null) throw new IllegalArgumentException("Unknown notification type: " + type);
        return s.get();
    }
}
```

- No `default` in the enum switch — the compiler flags an unhandled constant.
- The registry maps names to constructor references; `register("whatsapp", ...)` adds a channel without editing the factory (a lambda works because `Notification` has one abstract method).
- `Locale.ROOT` avoids the Turkish dotless-"i" lower-casing surprise.

Full runnable code: code/lld/ch08/simplefactory/ (see the Full Edition).

Trade-offs

Simple Factory	Factory Method
One method, usually a switch	Decision spread across subclasses
New type = edit the factory	New type = add a class
No inheritance	Creator subclass per product
Not GoF	GoF

Short, stable, config-selected type list → Simple Factory is perfectly respectable. Other developers/modules adding types without editing your class → Factory Method.

When not to use it

Only **one** concrete class (call the constructor); objects differ only in **data** (one class with a field).

Common mistakes

Calling a switch-based factory a "Factory Method"; a `default` that returns `null` or a silent default; letting the factory grow into a God class.

Real-world examples

Static factory methods (*Effective Java* item 1) — not GoF Factory Method, usually not singletons: `Calendar.getInstance()` (new object per call), `EnumSet.noneOf` (hidden `RegularEnumSet`/`JumboEnumSet`), `List.of`, `Integer.valueOf`, `Executors.newFixedThreadPool`; Spring `@Bean` methods.

In the interview

> **30-second answer:** "A simple factory is one method that maps a type value to a concrete class and returns it as an interface. It centralises the switch so callers don't depend on concrete classes. It's an idiom, not GoF — adding a type still edits it. For new types without editing, I'd use a registry of suppliers or Factory Method."

- *Q: Is it Open–Closed?* — A: Not strictly; a registry makes it open via registration.
- *Q: Static factory vs constructor?* — A: Names, caching (`Integer.valueOf`), can return subtypes.

Self-check

- **Q:** What does it centralise but not remove? **A:** The switch — knowledge of every concrete type.
- **Q:** Why no `default` in the enum switch? **A:** So the compiler flags unhandled constants.

TL;DR

One `create(type)` returning an interface. Great for small stable sets. Not GoF. `Supplier` registry = extensible variant.

8.10 Factory Method

What it is

A base class defines a creation method but **subclasses decide the concrete class**; the base class uses the product without naming its type. Analogy: a franchise manual — take order, *prepare the dish* (each branch's own), serve, bill (*Head First*'s `PizzaStore.orderPizza()` with regional `createPizza()`).

GoF intent and participants. Intent: declare an operation that creates an object but let subclasses choose the class — instantiation is deferred to them. **Product** (`Document`), **ConcreteProduct** (`PdfDocument`), **Creator** (`ReportGenerator` — declares the factory method and *calls* it from its own operations), **ConcreteCreator** (`PdfReportGenerator` — overrides it).

> **Not a "static factory method".** `List.of`, `Calendar.getInstance` and Simple Factories are **one static method that decides** — not overridable, no Creator hierarchy. GoF Factory Method is an **instance method overridden in subclasses** and called by the base class. No inheritance + overriding → not this pattern.

The problem it solves

`new PdfDocument()` inside `ReportGenerator` hard-wires PDF. The generator gathers rows, **creates a document**, renders, announces — only creation depends on format. A `switch` works until a second and third method need the same switch. Factory Method leaves a deliberate **gap**: the full procedure in the base class calls an abstract method each subclass fills.

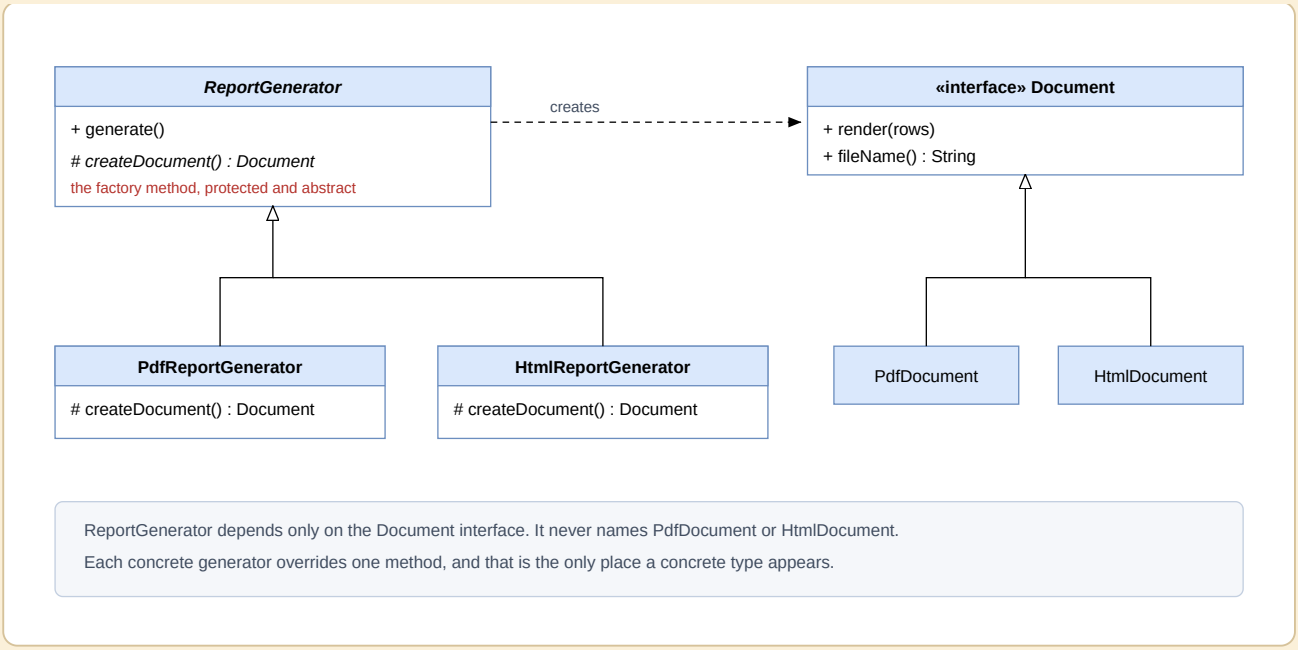
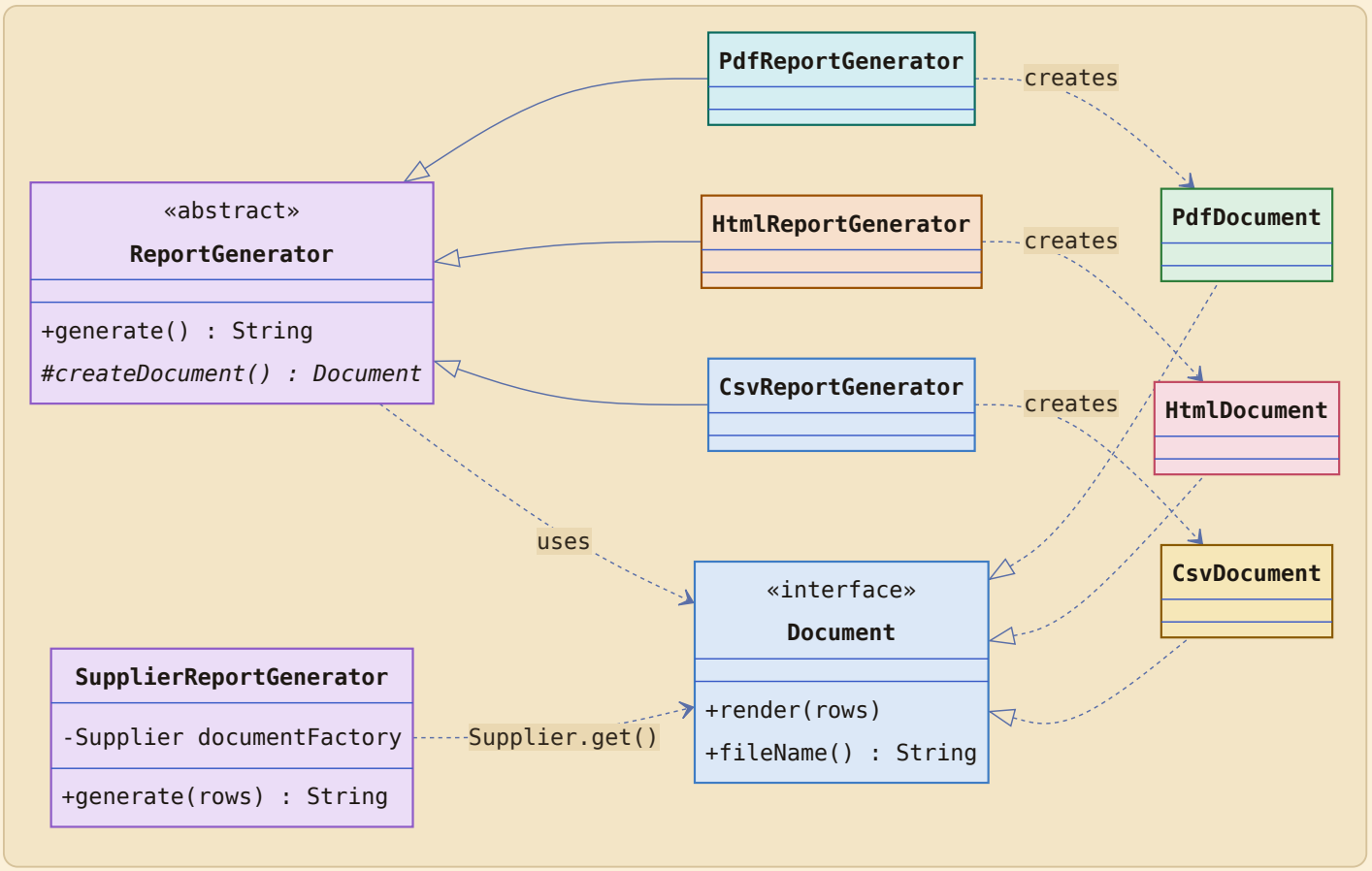


Figure — ReportGenerator declares the protected factory method createDocument(); each concrete generator overrides it to return its own Document (course diagram)

How it works

1. Product interface (`Document`: `render(rows)`, `fileName()`). 2. Concrete products. 3. Abstract creator with an **`abstract`** **`protected`** `createDocument()` (an extension point, not API). 4. Shared logic in the creator (`generate()`) calling it. 5. One concrete creator per product — the *only* place a concrete type is named. 6. The client picks a creator once, then uses `ReportGenerator` and `Document`.



Java implementation

```java
/** Creator: writes the whole procedure once and leaves one hole - the factory method. */
public abstract class ReportGenerator {

    /** The factory method. Protected: a hook for subclasses, not public API. */
    protected abstract Document createDocument();

    /** The template that USES the factory method. It never names a concrete document. */
    public final String generate() {
        System.out.println("  Collecting rows for the monthly sales report...");
        List<String> rows = List.of("north,120", "south,95", "east,140");
        Document document = createDocument();
        document.render(rows);
        System.out.println("  Report ready: " + document.fileName());
        return document.fileName();
    }
}

final class PdfReportGenerator extends ReportGenerator {
    @Override protected Document createDocument() { return new PdfDocument(); }
}
// ... HtmlReportGenerator, CsvReportGenerator (CSV added later - no existing class edited) ...

/** The modern, composition-based variant: pass the "factory method" in as a Supplier. */
public final class SupplierReportGenerator {
    private final Supplier<? extends Document> documentFactory;
    // ... constructor ...
    public String generate(List<String> rows) {
        Document d = documentFactory.get();
        d.render(rows);
        return d.fileName();
    }
}
```

`generate()` is `final` — subclasses change *what* is created, not the procedure. `CsvDocument` + `CsvReportGenerator` were added with no edits (Open–Closed). The `Supplier` variant composes better and a test can pass a lambda. In the output, every generator prints the same "Collecting rows…" line — that code exists once.

Full runnable code: code/lld/ch08/factorymethod/ (see the Full Edition).

Trade-offs

Pros	Cons
Extensible without edits — new product = new class	**More classes** — a creator per product
Common procedure in one place	**Indirection** — which subclass am I holding?
Test seam — subclass returns a fake	**Relies on inheritance** (one superclass)
Nobody holds the full type list	**Tempting too early** — all cost with one product

Often confused with: Simple Factory (one static switch); Abstract Factory (a *family* via a passed-in factory); **Template Method** (same hook mechanism but varies a *step* — `generate()` *is* one); Builder (many calls). In languages with first-class functions, passing a creation function is usually the better default.

When not to use it

One product type and no second coming (most frequent over-design); the base class has no shared work; types differ by data (use `Map<String, Supplier<Document>>`); you need matching objects (Abstract Factory).

Common mistakes

A **public** factory method (callers bypass the procedure); calling it from the base **constructor** (subclass fields not yet initialised); calling a switch-in-one-method a Factory Method.

Real-world examples

- **JDK:** `Collection.iterator()` — each collection (ConcreteCreator) returns its own `Iterator` (ConcreteProduct); `AbstractList.iterator()` built on abstract `get(int)`.
- **JDK XML:** `DocumentBuilderFactory.newDocumentBuilder()`, `TransformerFactory.newTransformer()` — factory methods inside an Abstract Factory (`newInstance()` itself is a *static* factory).
- Spring `FactoryBean<T>.getObject()`; Android `Fragment.onCreateView()`.

In the interview

> **30-second answer:** "Factory Method puts a creation hook in a base class. The base class writes the procedure once against a product interface; each subclass overrides the protected factory method to return its product. New types are new classes — Open–Closed. It pays off only with real shared logic in the base class; otherwise a constructor or simple factory is simpler. In modern Java I'd often pass a `Supplier`."

- *Q: vs Simple Factory?* — A: One static switch that knows every type vs subclasses overriding a hook the base calls. Raise this yourself — calling a switch a Factory Method is a classic slip.
- *Q: How do you test `generate()`?* — A: Subclass the creator (or pass a `Supplier`) returning a fake `Document`.

Self-check

- **Q:** The only place a concrete product is named? **A:** Each concrete creator's override.
- **Q:** What must the base class have for this to be worthwhile? **A:** Shared logic using the product.
- **Q:** Adding `CsvReportGenerator` shows which principle? **A:** Open–Closed.

TL;DR

Base class = procedure + abstract `protected createX()`; subclasses pick the type. One product, varied by inheritance. `Supplier` is the light modern form.

8.11 Abstract Factory

What it is

An interface for creating **families of related objects** that must be used together, without naming concrete classes. Choosing a family = choosing a factory. Analogy: a furniture catalogue — order from "Victorian" and every piece matches. (Course: one factory per OS theme; *Head First*: a regional pizza-ingredient factory.)

GoF intent and participants. Intent: one interface to create a set of related objects without naming their classes. **AbstractFactory** (`StorageFactory` — one create method per product kind), **ConcreteFactory** (`PostgresFactory`), **AbstractProduct** (`Connection`, `QueryBuilder`), **ConcreteProduct** (`PostgresQueryBuilder`), **Client** (`ReportService` — only abstract types). GoF notes one ConcreteFactory per family is typical, hence often a Singleton (passing it in is better).

The problem it solves

Some objects are only correct with partners from the same variant: Postgres quotes identifiers with `"`, MySQL with backticks; macOS buttons with macOS checkboxes; fake payments + fake mailer + fake clock, never a mix. Pairing `PostgresConnection` with `MySqlQueryBuilder` compiles and fails later at the database. Factory Method varies **one** product; Abstract Factory keeps a **set** consistent. *It doesn't help you choose the right combination — it removes the ability to build a wrong one.*

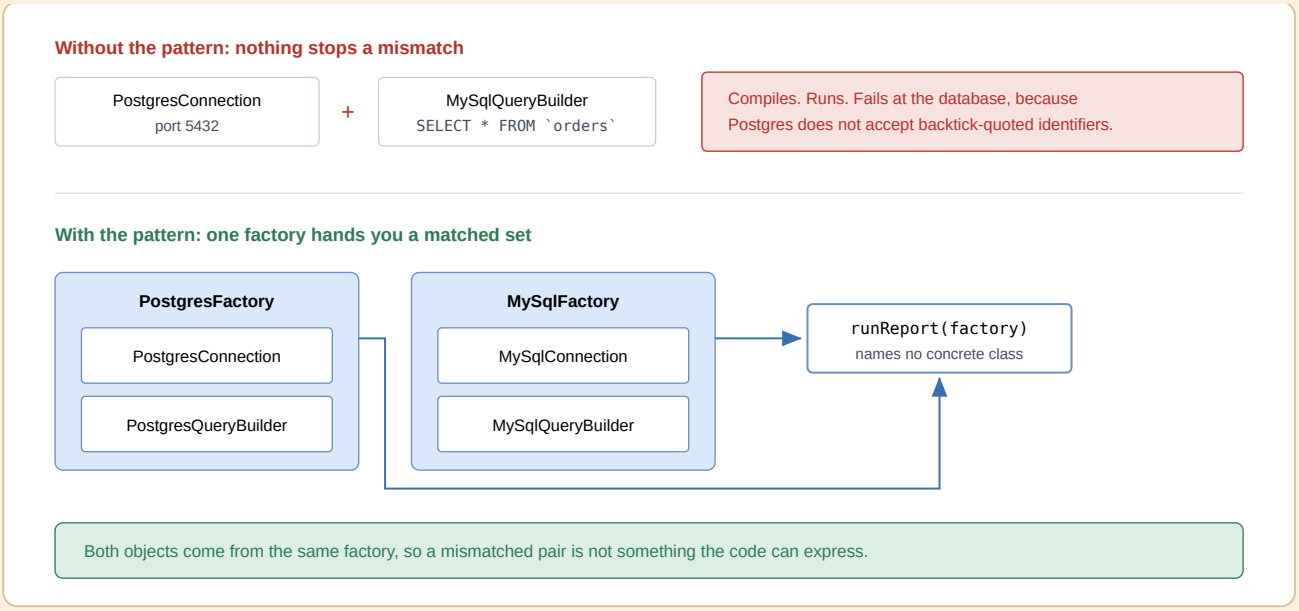
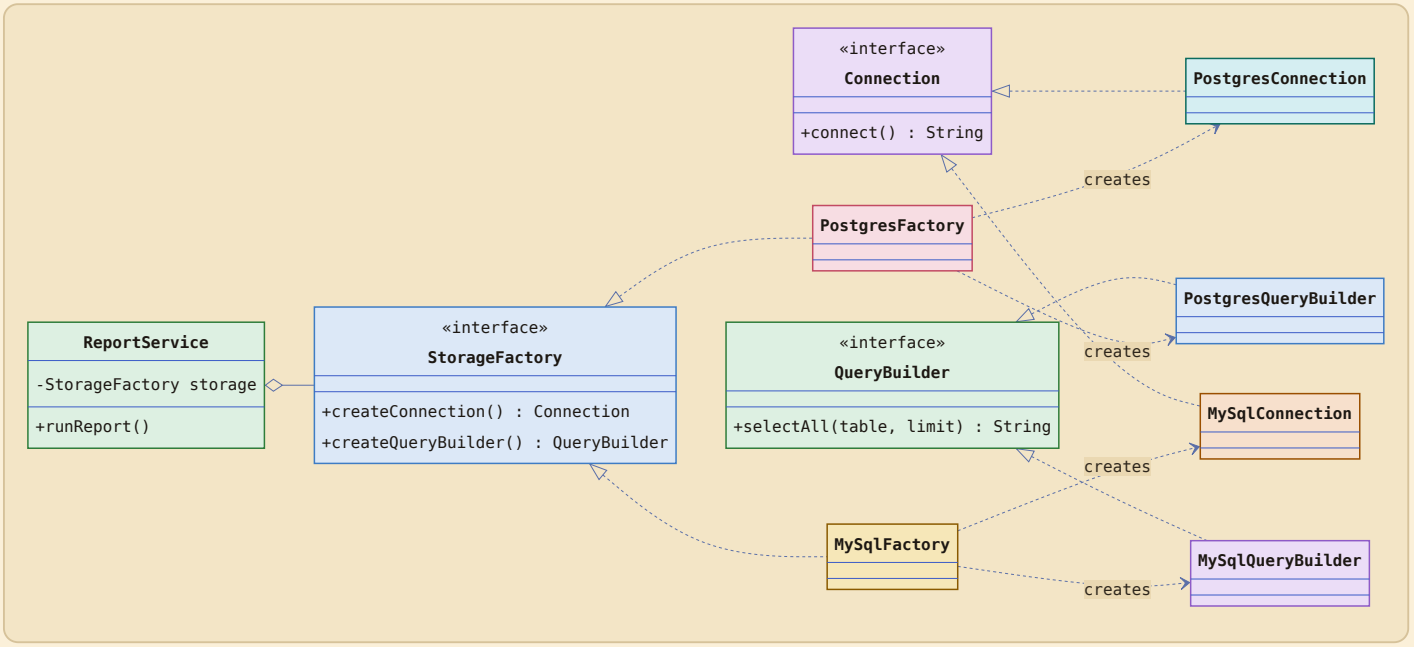


Figure — without the pattern a PostgresConnection can be paired with a MySqlQueryBuilder and fail at the database; with it, each factory hands out a matched set (course diagram)

How it works

1. Interface per **product kind**. 2. **Abstract factory** with one create method per kind. 3. Per variant, concrete products and a **concrete factory implementing every method** (covariant returns document the family). 4. The **client** receives a factory (constructor) and uses only abstract types. 5. Choose the factory **once at the composition root** (near `main`) and pass it down. (A correct Abstract Factory returns **all** products of its family — not one each.)



Java implementation

```java
/** Abstract factory: one creation method per product in the family. */
public interface StorageFactory {
    Connection createConnection();
    QueryBuilder createQueryBuilder();
}

/** Concrete factories implement EVERY method and return only their own family (covariant return types). */
final class PostgresFactory implements StorageFactory {
    @Override public PostgresConnection createConnection()     { return new PostgresConnection(); }
    @Override public PostgresQueryBuilder createQueryBuilder() { return new PostgresQueryBuilder(); }
}
// ... MySqlFactory, FakeStorageFactory ...

/** Client: holds only the abstract factory and abstract products. It cannot mix families. */
public final class ReportService {
    private final StorageFactory storage;

    public ReportService(StorageFactory storage) { this.storage = storage; }

    public void runReport() {
        Connection connection = storage.createConnection();
        QueryBuilder queries = storage.createQueryBuilder();
        System.out.println("  " + connection.connect());
        System.out.println("  " + queries.selectAll("orders", 10));
    }
}
```

`ReportService` has **no branch** on the database. `Main.chooseFactory(db)` — the composition root — holds the only `switch`. `FakeStorageFactory` swaps the **whole environment** for tests in one line.

Full runnable code: code/lld/ch08/abstractfactory/ (see the Full Edition).

Trade-offs

Pros	Cons
Wrong combinations can't be written	**Many types** — interface per kind + class per product per family
One switch for a whole environment (real↔fake, Postgres↔MySQL, themes)	**New product kinds are expensive** — edit every factory
Client never names a concrete class (DIP)	**Another layer to trace**
Test seam — one fake factory	**Pointless with one family**

The asymmetry: new **family** (`SqliteFactory`) = easy, Open–Closed; new **product kind** (`createMigrationRunner()`) = edit every factory.

Abstract Factory vs Factory Method

	Factory Method	Abstract Factory
Creates	**One** product	A **family**
Mechanism	**Inheritance** — subclass overrides	**Composition** — client holds a factory
Who chooses	Creator subclass	Factory passed in
Guarantees	New types without editing the creator	Products from one factory are compatible
New variant	New creator subclass + product	New factory + one product per kind
New product kind	n/a	Edit the interface and **every** factory
Example	`Collection.iterator()`	`DocumentBuilderFactory`, UI toolkits, DB dialects

Often confused with: Builder (one product, many steps); a DI container (same outcome via profiles — prefer it if you have one); Facade (simplifies, no consistency guarantee).

When not to use it

Objects that don't belong together (`createLogger()` + `createInvoice()` is a grab-bag); one variant only; a product-kind list that keeps growing (> 5–6); your DI container already does it (`@Profile`, Guice modules).

Common mistakes

A factory implementing only some methods or mixing families; choosing the factory deep in the call stack; making it a Singleton by reflex.

Real-world examples

- **JDK:** `DocumentBuilderFactory`, `TransformerFactory` (creates `Transformer` and `Templates`), `XPathFactory` — `newInstance()` locates an implementation whose products all match. JDBC `Connection` creates driver-specific `Statement`, `PreparedStatement`, `Blob`.
- UI toolkits (GoF's example), Hibernate `Dialect`, Spring `@Profile` bean sets.

In the interview

> **30-second answer:** "Abstract Factory is an interface with one create method per product in a family; each concrete factory returns a matched set. The client holds only the factory and abstract products, so it can't mix a Postgres connection with a MySQL query builder. I pick the factory once at the composition root. The cost: a new product kind means editing every factory."

- *Q: Add SQLite?* — A: `SqliteConnection`, `SqliteQueryBuilder`, `SqliteFactory`, one line in the composition root.
- *Q: Add a `MigrationRunner`?* — A: New method on the interface and every factory — the weak spot.

Self-check

- **Q:** What does it guarantee? **A:** Products from one factory are always the same family.
- **Q:** What is the composition root? **A:** The one place near start-up where concrete choices are made and passed down.
- **Q:** Easy vs hard change? **A:** New family easy; new product kind hard.

TL;DR

One factory → a matched family. Chosen once, passed down. Easy families, hard product kinds. Prefer DI profiles if you have a container.

8.12 Builder

What it is

Builder separates the *construction* of a complex object from the object. A builder collects values **one named step at a time** (a **fluent interface** — each returns `this`); `build()` produces the finished, usually **immutable**, product. Analogy: ordering at Subway, step by step.

	GoF Builder (1994)	**Bloch's Builder (*Effective Java* item 2)**
Targets	One construction process → **different representations** (RTF reader → ASCII, TeX, text widget)	Many optional parameters; wish for an immutable, validated object
Participants	**Builder** (abstract, one method per part), **ConcreteBuilder** (one representation, `getResult()`), **Director** (runs the steps), **Product**	Product + static nested `Builder`: fluent setters, validating `build()`; no Director or abstract Builder
What varies	Which ConcreteBuilder is plugged in	Only the values

Most real code is Bloch's; interviews may expect the GoF participant names too.

The problem it solves

`new HttpRequest(url, "GET", headers, null, 30, 3, true, false, null)` — which `null` is the body? Swap two booleans and it still compiles.

Approach	**What goes wrong**
Telescoping constructors	Overloads explode; still positional; same-typed params swap silently
JavaBeans (empty constructor + setters)	Half-built objects; can't be immutable; nowhere to validate cross-field rules

Builder: **named** values, **defaults**, **immutable** product, and `build()` as **one gate**. Our rule "a GET must not carry a body" (no defined meaning per RFC 9110 §9.3.1) spans two fields, so only `build()` can enforce it.

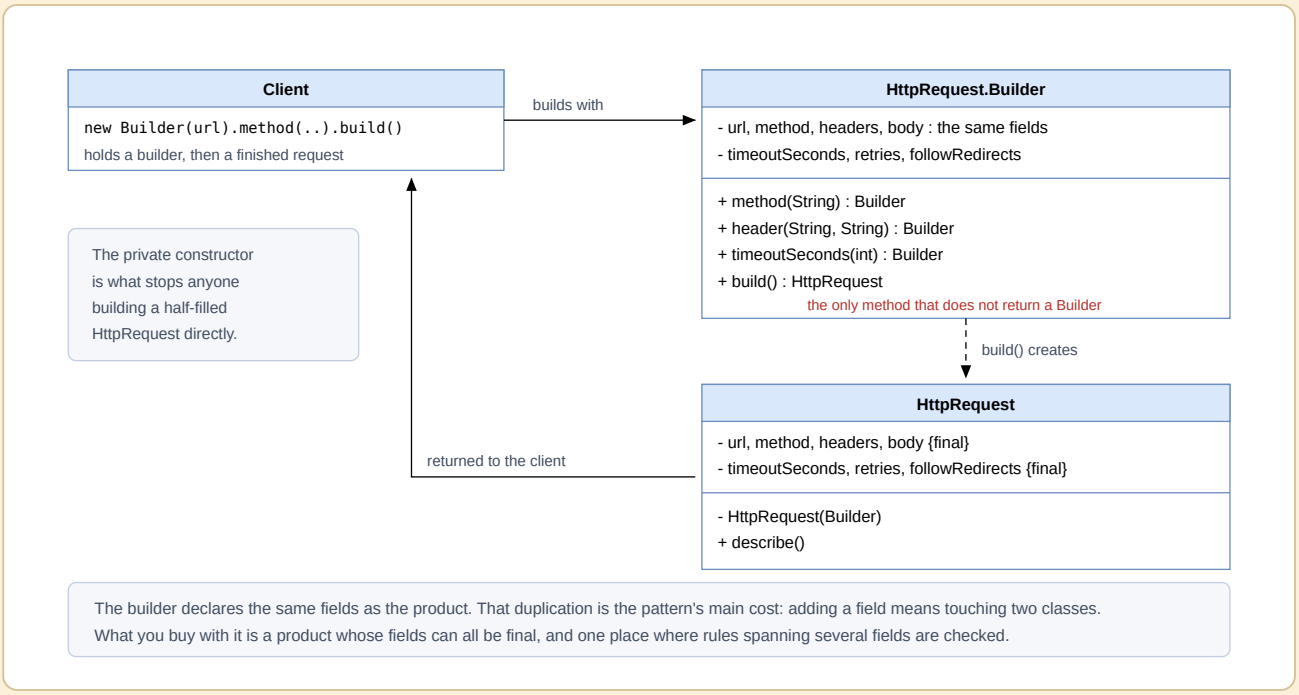
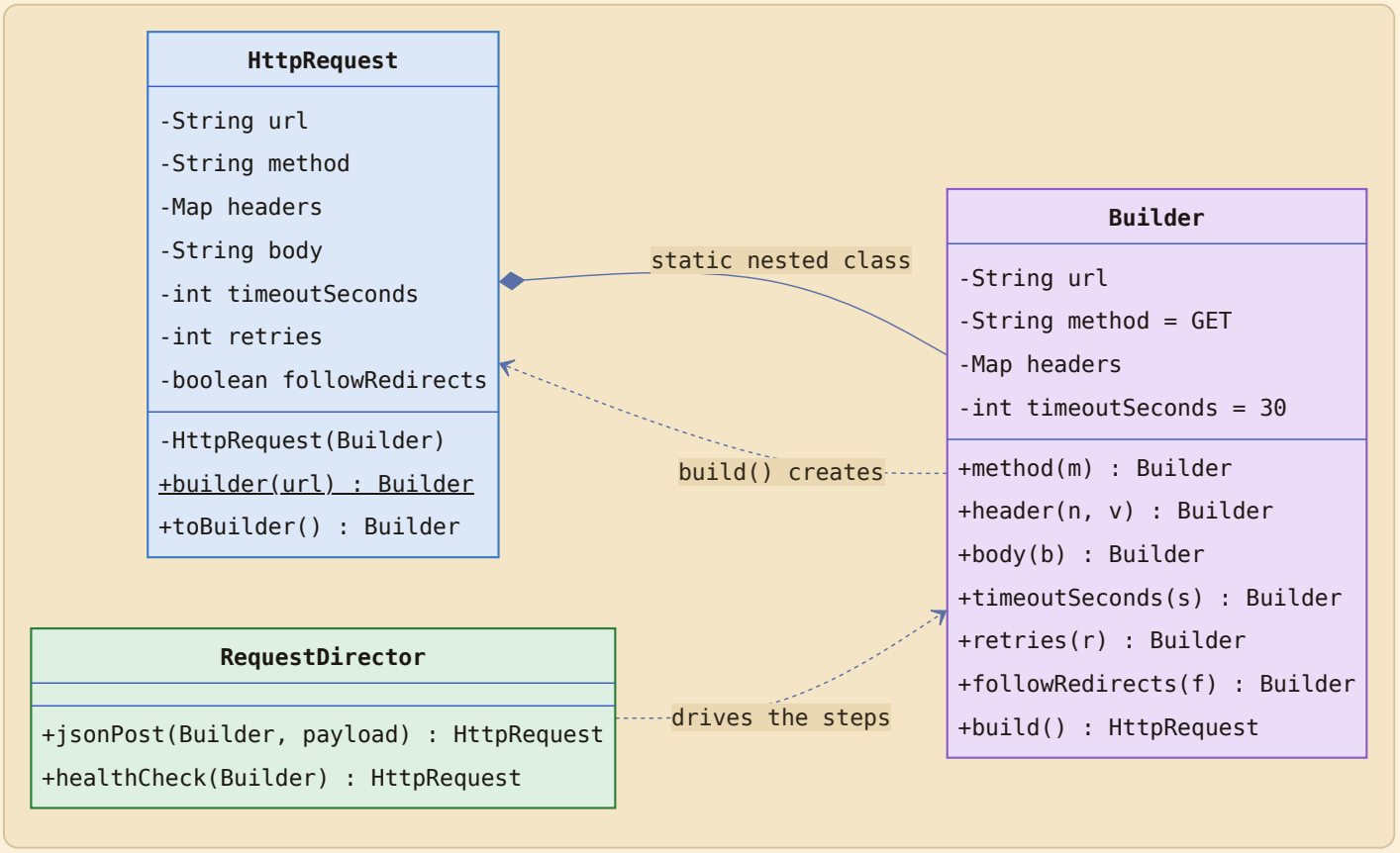


Figure — HttpRequest with final fields and a private constructor; HttpRequest.Builder with the same mutable fields, setters returning Builder, and build() returning HttpRequest (course diagram)

How it works

1. Product fields `final`, constructor `private` taking the builder. 2. `static` **nested** `Builder` with mutable fields and **defaults**. 3. **Required** values in the builder's constructor / `builder(url)`. 4. Named step

methods that `return this`. 5. `build()`: validate cross-field rules, **copy mutable collections**, call the private constructor — the only method returning the product. 6. Optional: **Director**, `toBuilder()`, **staged builder**. Price: every field is written twice.



The Director — and why you rarely see one

The GoF **Director** knows the **order of steps** and runs them on whatever builder it gets — a home for a reusable recipe. Day to day, only *values* vary, so callers chain the calls themselves. **Test:** will the Director ever get a *different kind* of builder? If not, drop it. (Our `RequestDirector` only takes `HttpRequest.Builder` — it shows just the "named recipe" half.)

The staged (step) builder

Encode required/ordered steps in **types**: `Email.builder()` returns `ToStep` (only `to()`), then `SubjectStep` (only `subject()`), then `OptionalStep` with `build()`. `Email.builder().build()` **does not compile** — the answer to "how do you stop callers forgetting a required field?".

Lombok-style builders and records

- **Lombok** `@Builder` generates it all (`toBuilder = true`, `@Singular`) but does **not** validate by default. **Records** give the immutable product — validate in the compact constructor. **Immutability checklist:** final class/private constructor, `final` fields, no setters, defensive copies, no leaking getters.

Java implementation

```
// ... HttpRequest: final fields; private HttpRequest(Builder b) copies them, with headers =
Map.copyOf(b.headers)
    /** The builder: same fields, mutable, with defaults. Every setter returns this. */
    public static final class Builder {
        private final String url;                                  // required -> constructor
        private String method = "GET";                             // optional -> defaults
        private final Map<String, String> headers = new LinkedHashMap<>();
        private String body;
        private int timeoutSeconds = 30;
        private int retries = 3;
        private boolean followRedirects = true;

        private Builder(String url) { this.url = Objects.requireNonNull(url, "url"); }

        public Builder method(String m)             { this.method = m; return this; }
        public Builder header(String n, String v)   { headers.put(n, v); return this; }
        public Builder body(String b)               { this.body = b; return this; }
        // ... timeoutSeconds, retries, followRedirects ...

        /** The single gate: cross-field validation happens here, before any product exists. */
        public HttpRequest build() {
            if (url.isBlank()) throw new IllegalStateException("url is required");
            if (method.equals("GET") && body != null) throw new IllegalStateException("a GET request cannot
have a body");
            if (retries < 0) throw new IllegalStateException("retries cannot be negative");
            if (timeoutSeconds <= 0) throw new IllegalStateException("timeout must be positive");
            return new HttpRequest(this);
        }
    }
}
```

- `TelescopingPizza` is the anti-example (five constructors; `("large", "thin", true, false, true)`).
- `Map.copyOf` matters: without it, two products built from one builder share one map, and a later header leaks into the earlier product. It's also unmodifiable, so `headers().put(...)` throws.
- `toBuilder()` gives "same as this, but…" copies; `RequestDirector.jsonPost/healthCheck` are named recipes; `Email` is the staged builder.

Full runnable code: code/lld/ch08/builder/ (see the Full Edition).

Trade-offs

Pros	Cons
Self-describing call sites	**Duplicated fields** — two edits per new field
Immutable products, thread-safe to share	**Forgettable `build()`**
Home for validation	**Overused** for 3-field objects
Free optional fields via defaults	**Reuse pitfall** — shared collections unless copied
Different representations (GoF Director)	One extra short-lived object (rarely measurable)

Often confused with: Factory Method (one call picks a type); Abstract Factory (several products at once); a **fluent interface** (a calling style — no separate product); Prototype (starts from a finished object). Named/default arguments (Python, Kotlin) usually replace builders.

When not to use it

A few required fields (the most common misuse); named/default arguments available; a builder that validates nothing for a mutable product; a mutable object living inside one method.

Common mistakes

Setters not returning the builder; no validation in `build()`; sharing mutable collections with products; a public product constructor (bypasses validation); a Director when the order never varies.

Real-world examples

- **JDK:** `StringBuilder`, `HttpClient.newBuilder()`, `HttpRequest.newBuilder()`, `Locale.Builder`, `Calendar.Builder`, `Stream.builder()`, `Thread.ofVirtual().name("w").start(task)`, `ProcessBuilder`.
- **Spring/libraries:** `UriComponentsBuilder`, `ResponseEntity.ok()...`, `WebClient.builder()`, OkHttp, Protobuf, Guava `ImmutableList.builder()`.

In the interview

> **30-second answer:** "With many optional parameters I use a Builder: required values in the builder's constructor, one named method per optional field returning the builder, and `build()` that validates cross-field rules and creates an immutable product via a private constructor. I'd mention the GoF Director but usually skip it since only values vary. For tiny objects or languages with named arguments, I wouldn't bother."

- *Q: How do you enforce required fields?* — A: Builder constructor, validation in `build()`, or a staged builder.
- *Q: Is the built object thread-safe?* — A: The immutable product is; don't share the builder across threads.

Self-check

- **Q:** The only builder method not returning the builder? **A:** `build()`.
- **Q:** What bug does copying the headers map prevent? **A:** Two products sharing and mutating one map.
- **Q:** When is a Director worth it? **A:** When the same recipe is reused, possibly with different builders.

TL;DR

Named steps + defaults + one validating `build()` → immutable product. Fixes telescoping constructors and half-built JavaBeans. Director optional; staged builder for compile-time required steps; Lombok/records cut boilerplate.

8.13 Prototype

What it is

Create objects by **copying an existing, configured object** via its own `copy()` — the caller needn't know the concrete class. Analogy: copying a contract template and changing the name and date.

GoF intent and participants. Intent: use a configured instance as the model and make new objects by copying it. **Prototype** (declares the copy operation — `Shape.copy()`), **ConcretePrototype** (`Circle`, `Rectangle`, `Group`; `NotificationTemplate` plays both roles), **Client** (asks a prototype to copy itself — `Main`, or `ShapeRegistry`, a *prototype manager*).

The problem it solves

1. **Expensive construction** — a `NotificationTemplate` parsed from disk; each welcome email differs only in subject and one channel. Re-parsing per recipient repeats the slowest work.
2. **Unknown concrete class** (the more important case) — you hold a `Shape` and need one more of the same; you can't write `new`. The object knows its own class, so it copies itself — no factory or `switch`, and the copy starts from correct values.

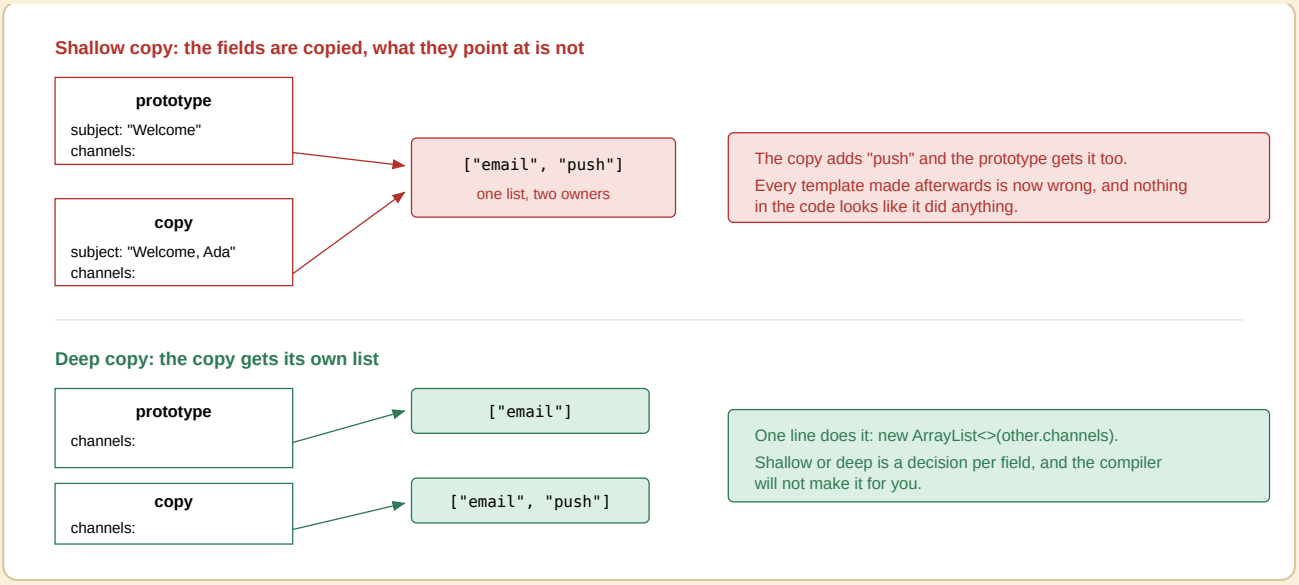
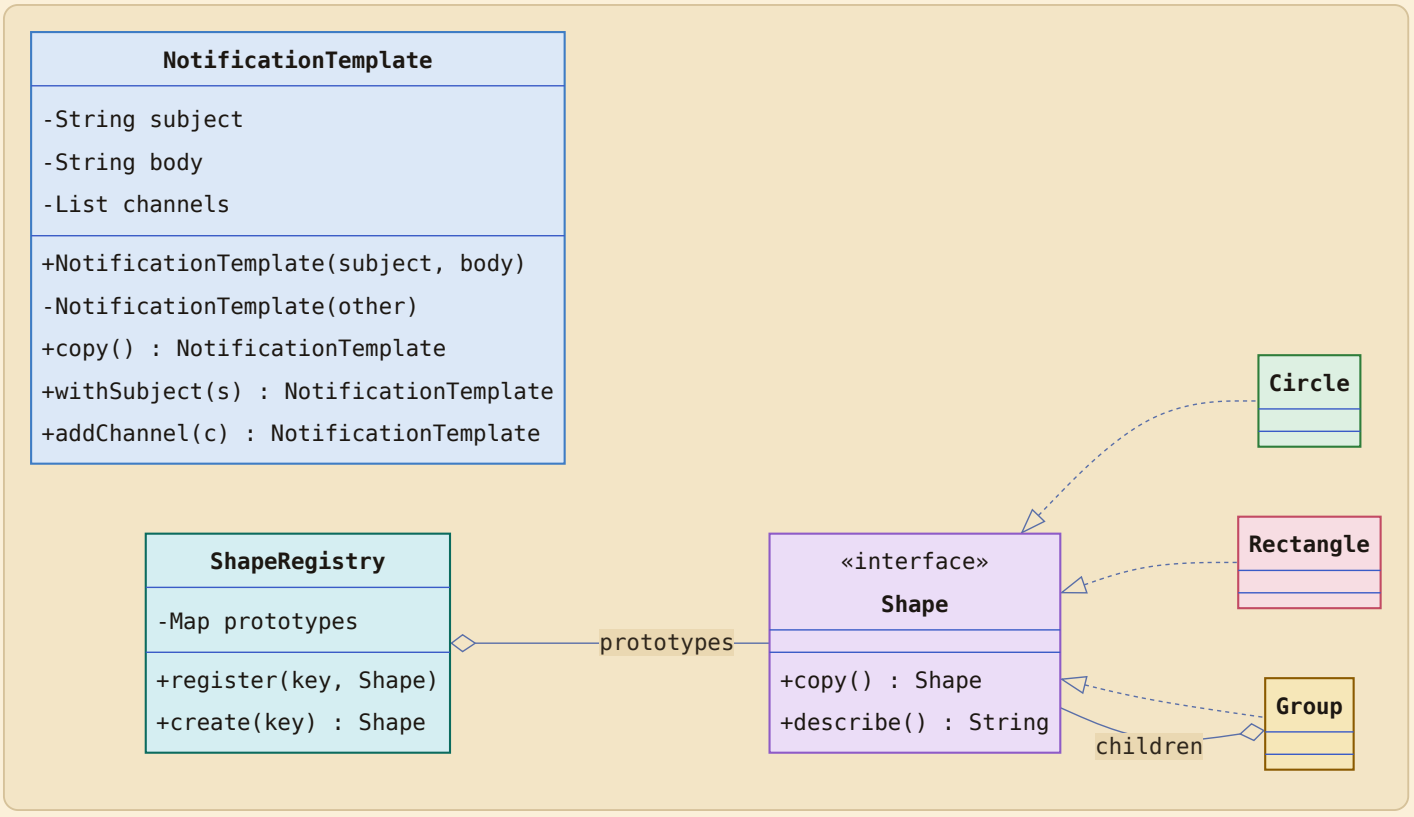


Figure — a shallow copy shares the channels list with the prototype, so adding "push" to the copy corrupts the prototype; a deep copy gets its own list (course diagram)

How it works

1. Two entrances: the **normal constructor** (expensive) and a **copy path** (none of it). 2. Implement the copy as a **private copy constructor**. 3. Per field: immutable → share; mutable → copy (`new ArrayList<>(other.channels)`), recursing for nested mutables. 4. Public `copy()`. 5. A **prototype interface** (`Shape { Shape copy(); }`) only when clients hold several unrelated types. 6. Optional **registry** that always hands out copies. (No separate creator class — the product copies itself.)



Shallow vs deep copy

	Shallow	Deep
Primitives, `String`, immutables	Copied / safely shared	Copied / safely shared
Mutable fields	**Shared**	**Duplicated**
Cost	Cheap	Proportional to the object graph
Risk	Editing the copy changes the original	Cycles; copying caches or resources
Java default	`Object.clone()`	You write it per field

With `this.channels = other.channels`, adding "push" to a copy adds it to the prototype — and **every later copy** inherits it. The fix is one line per mutable field. **Copy depth is a per-field design choice.** Related traps: a **new field not added to the copy** (silently dropped); copying something that **owns a resource** (both try to close it); **cycles** (naive recursive copy never ends — track visited objects).

Why Java's `Cloneable` is a trap

(*Effective Java* item 13.) `Cloneable` is a **marker with no methods**; `Object.clone()` is **`protected`**; it's **shallow**; it **skips constructors** (and their validation); deep-copying needs reassignment — **impossible for `final` fields**; it throws a **checked** exception. **Prefer a copy constructor or static `copyOf(other)`.** Exception: `array.clone()` for primitive arrays is fine.

Java implementation

```java
/** Prototype via a copy constructor: one expensive path, one cheap copy path. */
public final class NotificationTemplate {
    static int parses = 0;

    private String subject;
    private final String body;
    private final List<String> channels;

    /** The expensive path (imagine parsing a template file). */
    public NotificationTemplate(String subject, String body) {
        parses++;
        System.out.println("  Parsing template from disk... (expensive)");
        this.subject = subject;
        this.body = body;
        this.channels = new ArrayList<>(List.of("email"));
    }

    /** The copy path: no parsing. Deep-copies the mutable list. */
    private NotificationTemplate(NotificationTemplate other) {
        this.subject = other.subject;              // String is immutable -> sharing is safe
        this.body = other.body;
        this.channels = new ArrayList<>(other.channels);  // NEW list -> the copy owns its own
    }

    public NotificationTemplate copy() { return new NotificationTemplate(this); }
    // ... withSubject, addChannel, toString ...
}
```

Also in the package: `Group.copy()` (recursive deep copy of children), `Circle.copy()` with a covariant return, `ShapeRegistry.create()` returning `p.copy()` (never the original), and `CloneablePitfall.java` (`ShallowPlaylist` bug vs `DeepPlaylist`, whose `songs` can't be `final`).

Output: "Parsing template from disk" prints **once** for three templates, and the prototype keeps `channels=[email]` while copies add `push`/`sms`.

Full runnable code: code/lld/ch08/prototype/ (see the Full Edition).

Trade-offs

Pros	Cons
Costly setup paid once	**Copy code in every class** — drifts when fields are added
Create without naming a class	**Shallow vs deep per field** — wrong call shares state
New kinds at run time via registered prototypes	**Cyclic graphs** need care
Saved starting points (templates, layouts)	**Hazardous for identity** (keys, handles, connections); Java's `Cloneable` is flawed

Often confused with: Factory Method (fresh object); Builder (from parts); **Object Pool** (lends *existing* objects); **Spring** `prototype` **scope** (constructs anew — no cloning); JS prototypes (a language feature). JS spread is *shallow*; `structuredClone()` and Python `deepcopy` are deep.

When not to use it

Cheap construction; objects with **identity** (two rows sharing an ID is worse than a slow constructor); large graphs; immutable objects (just share them).

Common mistakes

Shallow-copying a mutable collection; trusting `super.clone()`; forgetting to update the copy constructor; handing out the registry's original.

Real-world examples

`ArrayList.clone()` / `HashMap.clone()` (**shallow**), copy constructors (`new ArrayList<>(c)`), `List.copyOf`; Unity `Instantiate(prefab)`; "Duplicate" in Docs/Figma; templates in Notion/Jira.

In the interview

> **30-second answer:** "Prototype creates objects by copying a configured instance via `copy()`. It saves expensive setup and — the deeper reason — lets me create an object when I only hold an interface. The main decision is shallow vs deep per field; a shared mutable list is the classic bug. In Java I'd write a copy constructor, not `Cloneable`, and never copy things with identity like a DB id or connection."

- *Q: Deep copy with cycles?* — A: An identity map original → copy; reuse the copy if already made.
- *Q: Serialization-based deep copy?* — A: Possible but slow, needs `Serializable`, bypasses constructors — last resort.

Self-check

- **Q:** Shallow vs deep? **A:** Shallow copies references; deep also copies the referenced mutable objects.
- **Q:** Why can't `DeepPlaylist.songs` be `final`? **A:** `clone()` must reassign it after `super.clone()`.
- **Q:** What should a registry return? **A:** A copy, never the original.

TL;DR

Copy a configured object — saves setup and works with only an interface. Decide shallow vs deep per field. Copy constructors, not `Cloneable`. Never copy identity.

8.14 Bonus — Object Pool

What it is

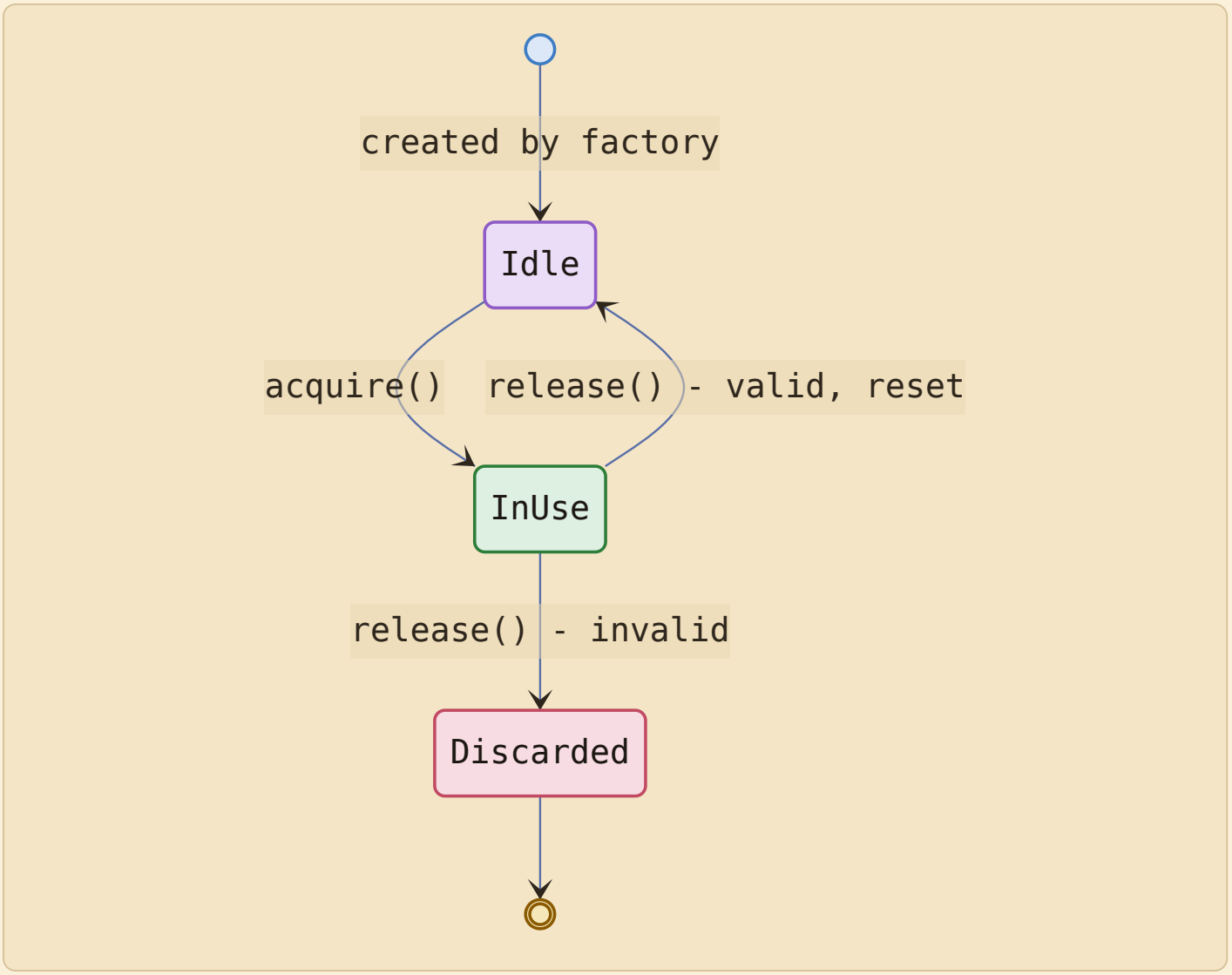
A **bounded set of reusable objects**: clients **acquire**, use, and **release** so others can reuse them. Not GoF, but a known creational pattern. Analogy: a bike-sharing dock — take, ride, return; if none, wait (or give up).

The problem it solves

Some objects are **expensive** (DB connection: TCP + TLS + auth) and/or **limited** (the DB caps connections). One per request is slow and can overwhelm the server. A pool pays creation a few times and **caps** the total.

How it works

1. An **idle queue**, a live count, and a cap (a `Semaphore` with one permit per lendable object).
2. **acquire()**: wait (with timeout) for a permit; take an idle object or create one; fail on timeout.
3. Client **always releases in** `finally` — a leak is lost forever.
4. **release()**: **validate** (discard broken ones), **reset** per-borrower state, return it; always free the permit.
5. Everything **thread-safe**.



Java implementation

```
// ... ObjectPool<T> fields: Semaphore permits (maxSize, fair), ConcurrentLinkedQueue<T> idle,
// ...   Supplier<T> factory, Consumer<T> reset, Predicate<T> isValid, AtomicInteger created
    /** Borrow an object, waiting up to the timeout if all are in use. */
    public T acquire(long timeout, TimeUnit unit) throws InterruptedException {
        if (!permits.tryAcquire(timeout, unit)) {
            throw new IllegalStateException("pool exhausted: timed out after " + timeout + " " + unit);
        }
        // Holding a permit guarantees we are within the cap: reuse an idle object or create one.
        T obj = idle.poll();
        if (obj != null) return obj;
        try {
            obj = factory.get();
            created.incrementAndGet();
            return obj;
        } catch (RuntimeException e) {
            permits.release();                                  // creation failed: give the permit back
            throw e;
        }
    }

    /** Give an object back. Broken objects are discarded so a fresh one can be created later. */
    public void release(T obj) {
        try {
            if (isValid.test(obj)) {
                reset.accept(obj);
                idle.offer(obj);
            } else {
                created.decrementAndGet();                      // discard it; its permit still comes back
            }
        } finally {
            permits.release();                                  // wakes one waiting borrower, if any
        }
    }
```

- The `Semaphore` caps borrowed objects however many threads race; holding a permit means "reuse idle, else safe to create".
- `release()` frees the permit **even when discarding**, so a waiter can create a replacement; a failed factory call also returns its permit.
- Handing objects through a concurrent queue + semaphore makes the previous borrower's writes visible to the next.
- `Main`: 10 tasks on 6 threads with a pool of 3 → `connections ever created: 3`; exhaustion prints `pool exhausted: timed out after 100 MILLISECONDS`.

Full runnable code: code/lld/ch08/objectpool/ (see the Full Edition).

Trade-offs

Pros	Cons
Avoids repeated expensive creation	Complexity: sizing, validation, timeouts, leak detection
Caps resource usage (protects the DB)	Leaks shrink the pool until it deadlocks
Predictable latency after warm-up	Incomplete reset leaks one user's data to the next
	Slower than `new` for cheap objects

When not to use it

Cheap objects (JVM allocation is faster); **virtual threads** (Java 21 — create one per task); when a **library** exists (HikariCP, Commons Pool 2).

Common mistakes

Not releasing in `finally`; not resetting state; unbounded waits; **double release** (here it would queue one object twice *and* add a permit, silently raising the cap — real pools track borrowed objects).

Real-world examples

HikariCP (Spring Boot default), DBCP, c3p0; `ThreadPoolExecutor`; Commons Pool 2 `GenericObjectPool` (Jedis); Netty `PooledByteBufAllocator`. *Not* pools: the `String` intern pool and `Integer` cache (-128..127) share immutables — that's **Flyweight** (Ch. 9).

In the interview

> **30-second answer:** "An object pool reuses a bounded set of expensive objects like DB connections. acquire takes an idle one, creates one if under the cap, or waits with a timeout; release validates, resets and returns it. It must be thread-safe and clients release in finally. For cheap objects it's counter-productive; in production I'd use HikariCP."

- *Q: Sizing?* — A: From measured concurrency and downstream limits; small pools (≈ core-count scale) often win.
- *Q: Leaks?* — A: Record borrower and time; log long holds (HikariCP `leakDetectionThreshold`).

Self-check

- **Q:** Pool vs Flyweight? **A:** Pool lends mutable objects exclusively and takes them back; Flyweight shares immutable objects with everyone at once.
- **Q:** Why validate on release? **A:** So dead objects aren't handed to the next borrower.

TL;DR

Borrow → use → return (in `finally`). Bounded, thread-safe, validated, reset. Only for expensive/limited resources; use a mature library.

8.15 Creational patterns compared

At a glance

Pattern	Distinctive feature	Use when	JDK / Spring
Singleton	One instance	One shared resource	`Runtime.getRuntime()`, singleton beans
Simple Factory	One method maps type → class	Small, stable, config-chosen set	`Calendar.getInstance()`, `EnumSet.noneOf`, `@Bean`
Factory Method	Subclasses choose via overridable method	Shared procedure, varying product	`Collection.iterator()`, `FactoryBean`
Abstract Factory	Families without concrete classes	Products must match	`DocumentBuilderFactory`, JDBC `Connection`, profiles
Builder	Construction separate from representation	Optional fields, validation, immutability	`StringBuilder`, `HttpClient.newBuilder()`
Prototype	Copy an existing object	Expensive setup, or only interface known	Copy constructors, `ArrayList.clone()`, Unity
Object Pool	Reuse a bounded set	Expensive/limited resources	HikariCP, `ThreadPoolExecutor`

Pick by the question you are asking

You are asking…	Pattern
"Only one?"	Singleton (or DI singleton scope)
"Avoid scattered `new ConcreteClass()`?"	Simple Factory
"Let subclasses/plug-ins decide?"	Factory Method
"Make sure these always match?"	Abstract Factory
"Make a 10-argument constructor readable and safe?"	Builder
"Another one of *this*, cheaply or without knowing its class?"	Prototype
"Stop creating and destroying this expensive thing?"	Object Pool

How they combine

- **Abstract Factory** is often built from **Factory Methods**, may use **Prototypes**, and is often a **Singleton** (passing it in is better).
- A **Builder** can use a **Factory** to pick parts; a **Director** can be a stateless shared service.
- Designs often *start* with Factory Method and *evolve* toward Abstract Factory, Prototype or Builder.

8.16 Chapter self-check

1. **Q:** The three GoF categories and counts? **A:** Creational 5, Structural 7, Behavioral 11 = 23.
2. **Q:** The two GoF central principles? **A:** Program to an interface; favour composition over inheritance.
3. **Q:** Why must the DCL field be `volatile`? **A:** So no thread sees a reference to a not-yet-constructed object (happens-before).
4. **Q:** Factory Method vs Abstract Factory? **A:** One product by inheritance vs a matching family by composition.
5. **Q:** How many patterns in a 45-minute design? **A:** One or two — and name the one you rejected.

8.17 Chapter TL;DR

- **Patterns** = named, proven templates with trade-offs; GoF (1994) catalogued 23 in three categories.
- **Pick from the signal**, one or two per design; state cost and alternative.
- **Singleton** — one per JVM; eager / holder / enum; DCL needs `volatile`; prefer DI.
- **Simple Factory** — one `create(type)`; idiom, not GoF.
- **Factory Method** — base-class procedure + subclass-chosen product.
- **Abstract Factory** — one factory, one matching family, chosen at the composition root.
- **Builder** — named steps, one validating `build()`, immutable product.
- **Prototype** — copy a configured object; shallow vs deep; avoid `Cloneable`.
- **Object Pool** — borrow/return expensive objects; use a real library.

Chapter 9 — Structural Design Patterns (Study Edition)

> **Study Edition.** A shorter version of Chapter 9: every pattern and sub-topic is kept, the padding is removed. The full chapter (README.md) has every source file and full outputs.
>
> **Where this fits.** Chapter 8 was about *creating* objects; this chapter is about *putting objects together* — connecting, wrapping, nesting and sharing them. Chapters 10–11 cover how objects *talk* (behavioral patterns).

#	Pattern	One-line intent	Analogy
9.1	Adapter	Translate a class's interface into the one your code uses	Travel plug adapter
9.2	Bridge	Split two independently varying dimensions into two hierarchies joined by a reference	TV remote and TV sold separately
9.3	Composite	Treat one object and a tree of objects the same way	Folders hold files and folders
9.4	Decorator	Add behaviour to one object at run time by wrapping it	Coffee toppings
9.5	Facade	One simple entry point to a complicated subsystem	Hotel concierge
9.6	Flyweight	Share the repeating part of many objects to save memory	One glyph "e" reused on every page
9.7	Proxy	Same-interface stand-in that controls access	Receptionist guarding the manager
9.8	Comparison	Adapter vs Decorator vs Proxy vs Facade vs Bridge	—

Run: `cd chapters/09-patterns-structural/code && javac --release 21 -d /tmp/lld_09 $(find . -name '*.java') && java -cp /tmp/lld_09 lld.ch09.adapter.Main` (or `bridge`, `composite`, `decorator`, `facade`, `flyweight`, `proxy`). All seven compile with zero warnings under `-Xlint:all`; each top-level type has its own file and is package-private.

9.0 What structural patterns are

What it is

The GoF's 23 patterns fall into three families: **Creational** (Ch. 8 — how to create objects), **Structural** (this chapter — how to combine classes and objects into bigger structures) and **Behavioral** (Ch. 10–11 — how objects share work and talk). Structural patterns **simplify complex structures** (Facade, Composite), **promote reuse** (Adapter), **improve flexibility by decoupling** (Bridge, Decorator, Proxy) and **improve maintainability** (clear relationships, local changes). The seven GoF structural patterns: **Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy**.

Three words you need first

- **Composition** ("has-a") — an object holds and uses another; almost every structural pattern is built on it.
- **Delegation** — forwarding a call to the held object (`wrapper.pay()` calls `inner.charge()`).
- **Interface** (design sense) — the methods a caller can use on an object.

> **Key insight.** Adapter, Decorator, Proxy and Facade have the *same physical shape* — one object holds another and forwards calls. What separates them is **intent**: what changes as the call passes through (9.8).

9.1 Adapter Pattern

What it is

Adapter lets your code use a class whose methods have the "wrong" shape — one you can't change — through a small translator. GoF intent: *convert the interface of a class into another interface clients expect.* Also called **Wrapper**. Analogy: a travel plug adapter.

Role	Meaning	Ours
Target	Interface your code wants	`PaymentGateway`
Adaptee	Existing class with the wrong interface (usually not yours)	`StripeClient`
Adapter	Implements Target, translates to Adaptee	`StripeGateway`
Client	Uses the Target	`checkout`

The problem it solves

Checkout wants `pay(Money)`; the SDK offers `charge(amountInCents, currencyCode)` (or XML vs JSON, `track(String, Map)` vs your `Event`). **The other side isn't yours to change** — a vendor, another team, many existing callers. Stripe wants integer **cents** + **lowercase** currency; PayPal wants a **decimal string** "42.50" + **uppercase**; your code wants to think in `Money`.

How it works

1. **Target interface in your vocabulary** (`PaymentGateway.pay(Money) -> Receipt`), no vendor names.
2. **One adapter per adaptee**, implementing the Target and **holding** the vendor object.
3. **Translate inputs**, 4. **call the adaptee**, 5. **translate outputs and errors back** (vendor exception → your exception).
4. **Clients depend only on the Target.**

> **The rule:** the adapter's public methods mention **only your types** — parameters, returns *and* exceptions. Let a `StripeCardException` escape and callers depend on Stripe again.

Structure

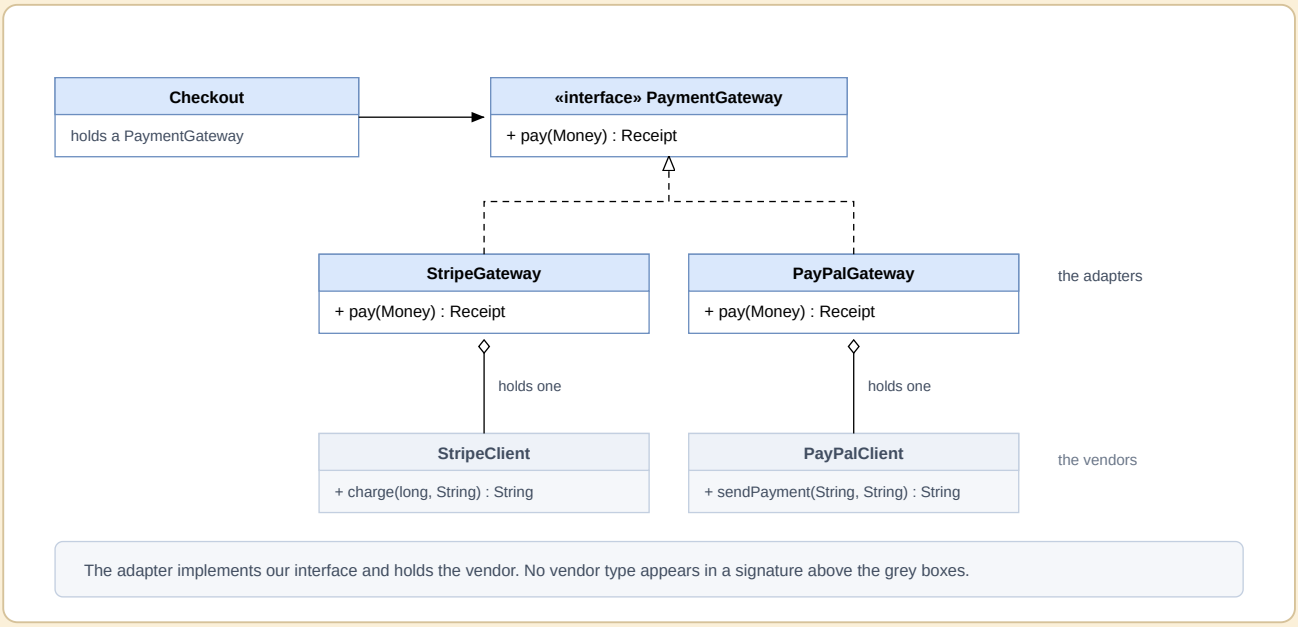
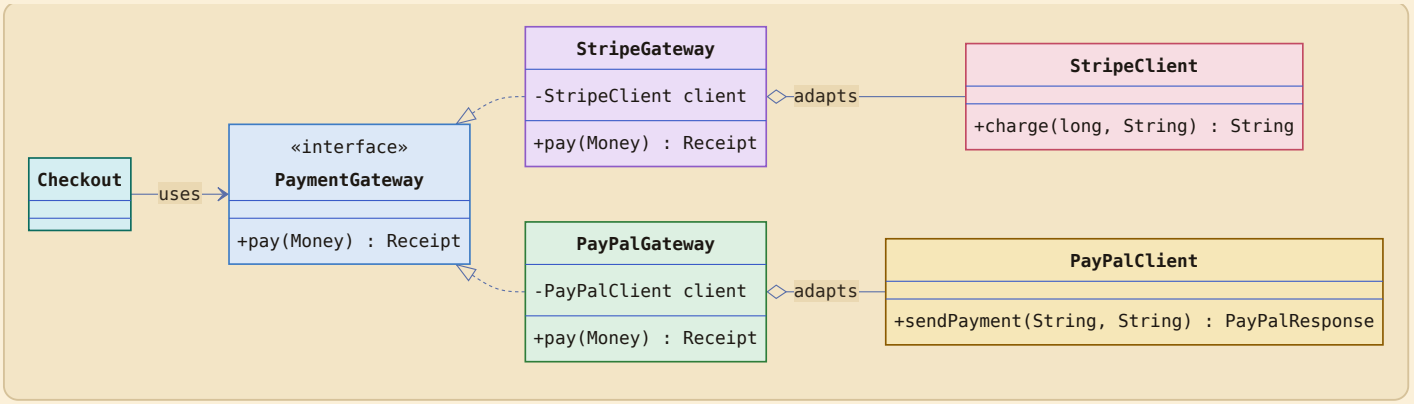


Figure — Adapter class diagram: `Checkout` holds a `PaymentGateway`; `StripeGateway` and `PayPalGateway` implement it and each wrap a vendor client, shown in grey because we do not own them (course diagram)



Java example

```java
/** TARGET interface: the shape our application wants to talk to. */
interface PaymentGateway {
    Receipt pay(Money amount);
}

/** OBJECT ADAPTER: implements our interface, HOLDS the vendor object (composition). */
final class StripeGateway implements PaymentGateway {
    private final StripeClient client;

    StripeGateway(StripeClient client) { this.client = client; }

    @Override
    public Receipt pay(Money amount) {
        try {
            String chargeId = client.charge(amount.cents(), amount.currency().toLowerCase(Locale.ROOT));
            return new Receipt(chargeId, "SUCCEEDED");
        } catch (StripeCardException e) {               // translate the vendor's error too
            throw new PaymentFailedException("Stripe declined: " + e.getMessage(), e);
        }
    }
}
```

`Money` (a record that upper-cases the currency once), `Receipt` and `PaymentFailedException` are *our* vocabulary. `PayPalGateway` does a different translation (`Money.asDecimal()` → "42.50"; `"approved"` → `"SUCCEEDED"`). `checkout(gateway, amount)` has no `if (stripe)` branch. The output shows `charge(4250, "usd")`, `sendPayment("42.50", "USD")`, and a declined Stripe charge reaching the client as `payment failed -> Stripe declined: card_declined` — *our* error type.

Full runnable code: code/lld/ch09/adapter/ (see the Full Edition).

Object adapter vs class adapter

	Object adapter (composition)	**Class adapter (inheritance)**
How	*Holds* an adaptee, implements Target	*Extends* the adaptee, implements Target
Ours	`StripeGateway`, `PayPalGateway`	`ThermometerClassAdapter extends LegacyFahrenheitThermometer implements CelsiusSensor`
Adaptee `final`?	Works	**Fails** (our `StripeClient` is final)
Adapts adaptee + subclasses?	Yes	No — bound at compile time
Override adaptee behaviour?	Harder	Yes
Leaks adaptee API?	No	**Yes** (`readFahrenheit()` stays public)
Java constraint	none	Target must be an **interface** (no multiple class inheritance)

Prefer the object adapter (composition, works with `final`, no unwanted methods).

Two-way adapter

Implements **both** interfaces so clients of either side can use one object — `TwoWayTemperatureAdapter implements CelsiusSensor, FahrenheitSensor`: new code sees `100.0C`, old code `212.0F`. Handy during **migrations**.

Adapter vs Facade vs Decorator vs Proxy (quick view)

Pattern	**Interface**	**Behaviour**
Adapter	**Translated**	Same work, re-expressed
Decorator	Same	**Extended**
Proxy	Same	Same work, **gated** (defer, cache, refuse)
Facade	**Replaced** by one simpler one over many	Same work, fewer calls

Test: *adapters exist because someone else picked the interface* — if the vendor matched yours tomorrow, the adapter would be pointless. **Bridge** looks the same but is planned *before* the classes exist; an Adapter is bolted on *after*.

Trade-offs

Pros	**Cons**
Foreign format contained in one class	Extra class and call frame
Vendor change is local (Open/Closed)	**Conversions can drop information** silently
Easy to test — fake your one-method interface	Two data models must stay aligned
Your code keeps its language (SRP)	Logic (retries, pricing) creeps in

When not to use it

Both sides are yours (change one); the models truly disagree (redraw the boundary); you want extra behaviour (Decorator); the conversion happens in exactly one place.

Common mistakes

Leaking vendor types/exceptions; business logic in the adapter; class adapter by default; one adapter for many vendors with `if (vendor == STRIPE)`; silently mapping "pending" to "SUCCEEDED".

Real-world examples

`Arrays.asList` (array → `List`, no copy; writes go through, `add` throws); `InputStreamReader`/`OutputStreamWriter` (bytes ↔ chars); `Collections.enumeration`; SLF4J bindings; JDBC drivers; Spring MVC `HandlerAdapter`; every in-house "client" wrapping a vendor SDK.

In modern languages

A one-method Target makes a **lambda** the whole adapter (`FahrenheitSensor oldSensor = () -> 212.0;`); duck/structural typing (Python, TS, Go) makes adapters tiny.

In the interview

> **30-second answer:** "An Adapter converts one interface into another the client expects. I use it at boundaries I don't control — a vendor SDK, a legacy service. It implements *my* interface, holds the vendor object and translates inputs, outputs and exceptions, so no vendor type leaks in. Swapping vendors is one new class. I prefer the object adapter: composition, works with final classes."

- *Q: Adapter vs Decorator?* — A: Adapter changes the interface, keeps behaviour; Decorator keeps the interface, adds behaviour.
- *Q: Why not change the other side?* — A: Say it up front: it's a vendor library / another team / many callers. That constraint justifies the pattern.

Self-check

1. **Q:** The four roles? **A:** Target, Adaptee, Adapter, Client.
2. **Q:** Why can't `StripeClient` be class-adapted? **A:** It's `final`.
3. **Q:** `Arrays.asList(arr).add(x)`? **A:** `UnsupportedOperationException` — a fixed-size view over the array.

TL;DR

Translator between the interface you have and the one you need; implements **yours**, holds the vendor's, translates **in, out and errors**. Prefer object adapters. Only at boundaries you don't control.

9.2 Bridge Pattern

What it is

Bridge splits a class that varies in *two independent ways* into **two hierarchies** joined by a reference (the "bridge"). GoF intent: *decouple an abstraction from its implementation so the two can vary independently* (a.k.a. **Handle/Body**). Analogy: any remote works any TV via a common protocol.

Term	Meaning	Ours
Abstraction	High-level side, holds an Implementor	`Notification`
Refined Abstraction	Its subclasses	`AlertNotification`, `ReminderNotification`, `DigestNotification`
Implementor	Interface of the low-level physical work	`Channel`
Concrete Implementor	Real low-level work	`EmailChannel`, `SmsChannel`, `PushChannel`

> **Naming trap.** "Implementation" here isn't "a class implementing an interface" — it's the **low-level half** of the split (sending bytes, drawing pixels).

The problem it solves — the class explosion

Three notification types × three channels in one hierarchy = **9** classes (`EmailAlert`, `SmsDigest`…); add one of each → 16. With a bridge they **add**: 3 + 3 = 6, then 8. **Smell:** class names made of two nouns, each duplicating build or send code. Bonus: the pairing becomes a **run-time** choice (SMS in tests, email in production). The hard part is choosing which side is the abstraction.

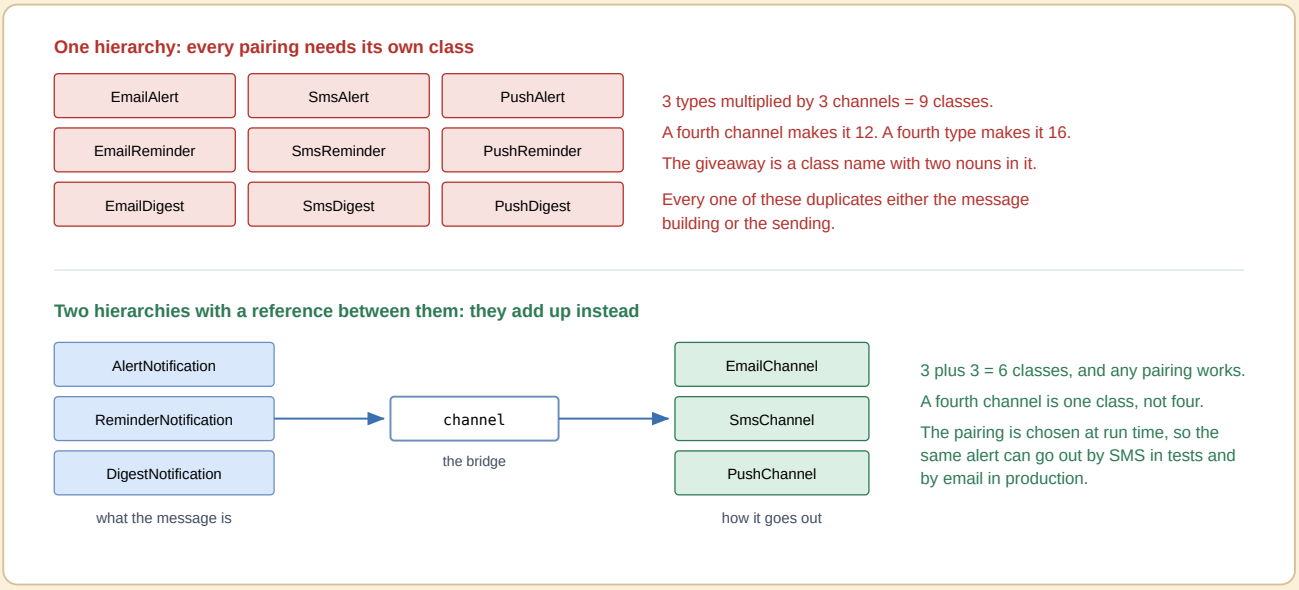
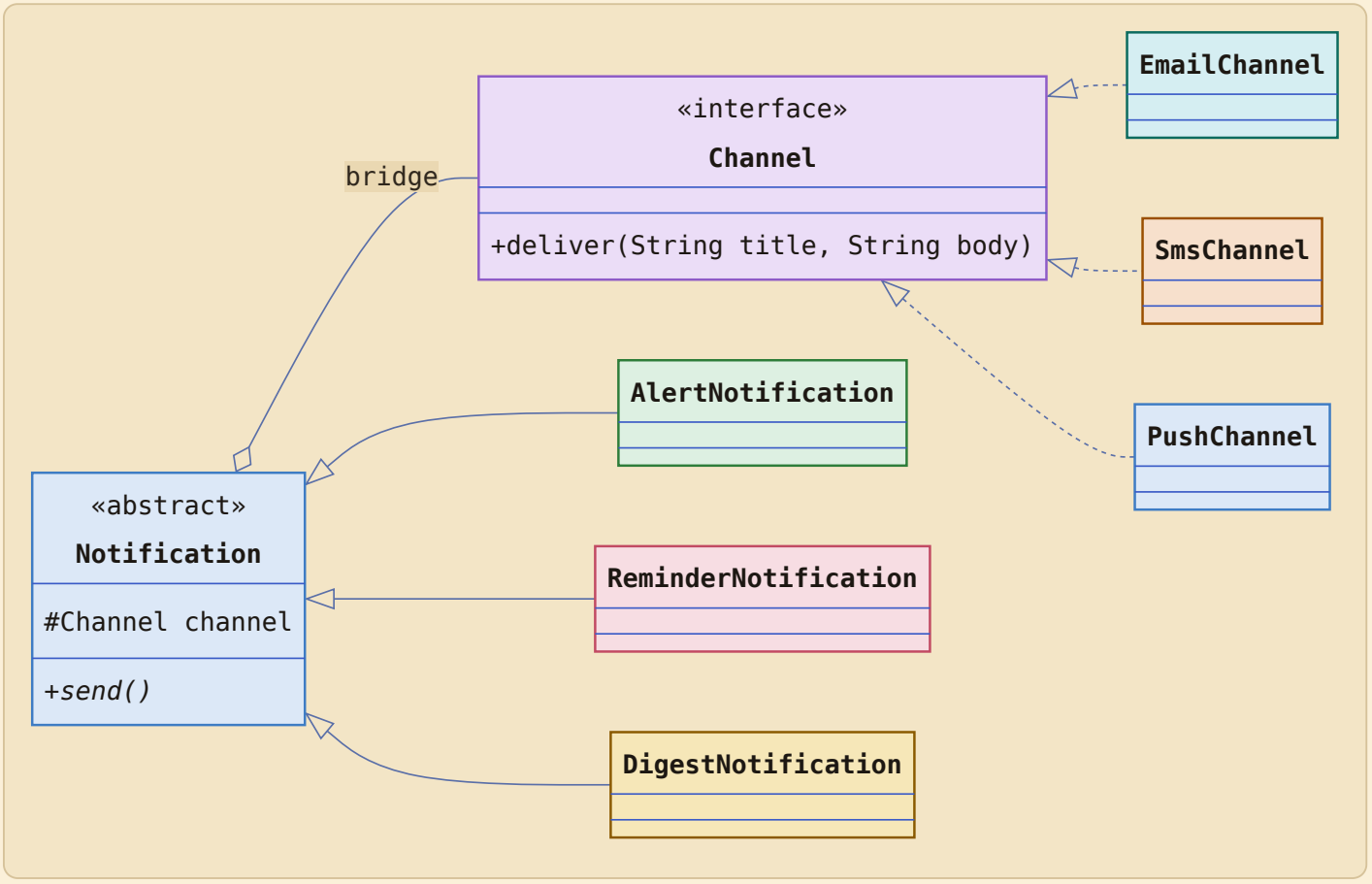


Figure — Top: one hierarchy needs 3 × 3 = 9 classes such as EmailAlert and SmsDigest. Bottom: two hierarchies joined by one `channel` reference need 3 + 3 = 6 (course diagram)

How it works

1. Find the two independent dimensions (what the message *is* vs how it *goes out*).
2. The lower-level one becomes a small, general **Implementor interface** (`Channel.deliver(title, body)`).
3. The higher-level one becomes an **abstract class holding a reference** to it — that field is the bridge.
4. Refined abstractions build content and **delegate** across the bridge, never checking the implementor type.
5. Pair at run time: `new AlertNotification(new SmsChannel(...), ...)`.

Structure



Java example

```java
interface Channel {
    void deliver(String title, String body);
}

abstract class Notification {
    protected final Channel channel;

    protected Notification(Channel channel) { this.channel = channel; }

    /** Builds the message, then hands it across the bridge. Never branches on channel type. */
    abstract void send();
}

/** REFINED ABSTRACTIONS */
final class AlertNotification extends Notification {
    private final String system;
    private final String problem;

    AlertNotification(Channel channel, String system, String problem) {
        super(channel);
        this.system = system;
        this.problem = problem;
    }

    @Override
    void send() { channel.deliver("ALERT " + system, problem + " - act now"); }
}
```

`SmsChannel` truncates long text — an SMS rule living inside SMS only. Adding `SlackChannel` or `PromoNotification` = one class each.

Full runnable code: code/lld/ch09/bridge/ (see the Full Edition).

Bridge vs Strategy

Code is **identical** (hold an interface, delegate); the *reason* differs.

	Bridge	Strategy
What varies	A whole **dimension**	**One algorithm**
When chosen	Once, at construction	Often swapped, even per call
Why	Stop two hierarchies multiplying	Make one behaviour replaceable
Decided	Up front, structurally	Later, when a 2nd algorithm appears
Example	`Notification` → `Channel`	`list.sort(comparator)`

Test: a new implementation that is *a new kind of thing the system supports* → Bridge; *one more way to do one step* → Strategy.

Getting the split wrong

- **Low-level details leaking up** — `if (channel instanceof SmsChannel)` in `AlertNotification`; move it into `SmsChannel`.
- **An implementor shaped around one implementation** — `setEmailReplyTo(...)` on `Channel` forces others to throw. Keep it minimal (two plain strings).

Trade-offs

Pros	Cons
Classes add, not multiply	Two hierarchies to understand
Each side extends independently (Open/Closed)	More indirection
Pairing chosen at run time	**Hard to undo** once both sides grow
Platform code isolated (SRP)	Pointless with one implementor

When not to use it

An axis has one member (add it when the second lands); axes aren't independent (forbidden combos become run-time checks); you can't say in one sentence which side is the abstraction; the second axis is a behaviour (Strategy).

Common mistakes

Bridge "just in case"; abstraction branching on implementor type; implementor mirroring one implementation; confusing it with Adapter (retrofitted vs designed in).

Real-world examples

JDBC (`Connection`/`Statement` + drivers — also called Adapter); SLF4J API vs backend; GoF's `Window`/`WindowImp` per platform and graphics drivers; `java.awt` peers; `NotificationService` with SES/Twilio/FCM providers; DI binding per environment.

In modern languages

A one-method implementor can be a plain function; DI containers wire the pairing per environment.

Other patterns it is confused with

Looks like	Difference
Strategy	Same shape; permanent dimension vs swappable algorithm
Adapter	Retrofitted after vs planned before
Abstract Factory	Decides *which family is created*; often supplies the implementor a bridge receives
Decorator	Shares the wrapped interface and adds behaviour; Bridge sides have unrelated interfaces

In the interview

> **30-second answer:** "Bridge separates an abstraction from its implementation so both vary independently. Notifications by type and by channel in one hierarchy need 3 × 3 = 9 classes, 16 with a fourth of each. I split it: a `Notification` hierarchy holding a `Channel` interface — 6, then 8 — and the pairing is chosen at run time. I wouldn't build it with only one channel."

- *Q: vs Strategy?* — A: Same code; Bridge splits a permanent dimension up front, Strategy makes one algorithm replaceable.
- *Q: Who picks the implementor?* — A: Whoever builds it — often a factory or DI container, per environment.

Self-check

1. **Q:** 5 types × 4 channels, without/with Bridge? **A:** 20 vs 9.
2. **Q:** Which field is the bridge? **A:** `Notification`'s `Channel channel`.
3. **Q:** Bridge added before or after classes exist? **A:** Before (Adapter is after).

TL;DR

Two varying dimensions → two hierarchies + one reference; multiplication becomes addition. Keep the implementor small. Same code as Strategy, different intent.

9.3 Composite Pattern

What it is

Composite gives a single item and a nested group **one common interface**, so clients never check whether they hold a **leaf** or a **tree**. GoF intent: compose objects into **part-whole hierarchies** and treat individual objects and compositions **uniformly**. Analogy: ask any manager or individual contributor "your team's total salary?" — same question for both.

Term	Meaning	Ours
Component	Shared interface	`FileSystemNode`, `MenuComponent`
Leaf	No children; answers itself	`FileNode`, `MenuItem`
Composite	Holds children *typed as Component*, delegates	`FolderNode`, `Menu`
Client	Uses Components only	`report(...)`

The problem it solves

Nested data (folders, invoice bundles, UI panels) makes every operation start with `if (node instanceof Folder) ... else if (node instanceof File) ...`. That test is copied into `size`, `print`, `search`…; a new node type (symlink) must find every copy.

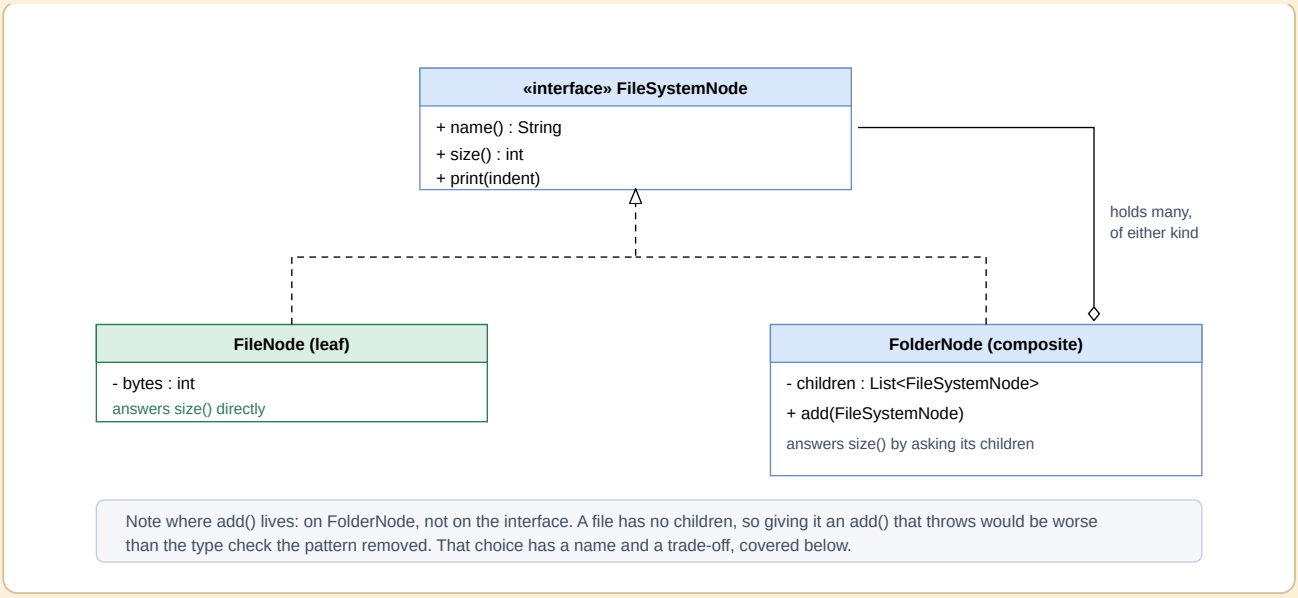
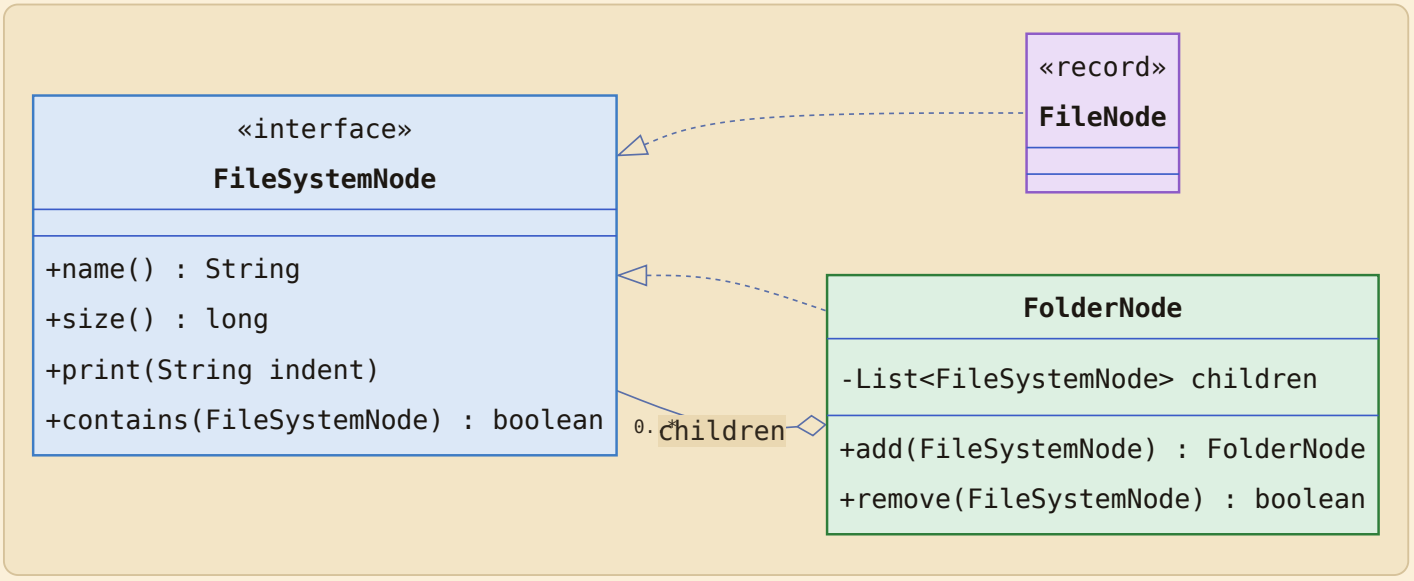


Figure — Composite class diagram: `FileNode` (leaf) and `FolderNode` (composite) implement `FileSystemNode`; the folder holds a list of `FileSystemNode` and has `add()` (course diagram)

How it works

1. A **Component** interface with operations valid for both (`name()`, `size()`, `print()`).
2. **Leaf** implements them directly.
3. **Composite** stores `List<Component>` — the **interface type** — and implements each operation by **asking children and combining**.
4. The recursion is written **once** and works to any depth.
5. **Clients** never type-check.

Structure



Java example

```java
interface FileSystemNode {
    String name();
    long size();
    void print(String indent);

    /** True if 'target' is this node or somewhere below it. Used for the cycle check. */
    default boolean contains(FileSystemNode target) { return this == target; }
}

/** LEAF: no children, answers directly. */
record FileNode(String name, long size) implements FileSystemNode {
    // ... print(indent) ...
}

/** COMPOSITE: holds children typed as the INTERFACE, so folders can hold folders. */
final class FolderNode implements FileSystemNode {
    private final String name;
    private final List<FileSystemNode> children = new ArrayList<>();
    // ... constructor ...
    /** Returns this so trees can be built fluently. Rejects cycles at insert time. */
    FolderNode add(FileSystemNode child) {
        if (child.contains(this)) {
            throw new IllegalArgumentException("cycle: " + child.name() + " already contains " + name);
        }
        children.add(child);
        return this;
    }
    // ... remove, children() returns Collections.unmodifiableList, name(), print, contains ...
    /** The recursion lives in ONE place. */
    @Override
    public long size() {
        long total = 0;
        for (FileSystemNode child : children) total += child.size();
        return total;
    }
}
```

`MenuComponent` / `Menu` / `MenuItem` is the *transparent* variant (*Head First*'s restaurant menu, prices in `BigDecimal`). Output: `report(project)` → 2150 bytes and `report(readme)` → 1200 with the same call; `src.add(project)` is rejected as a cycle; `Tea.add(...)` throws "Tea is a leaf".

Full runnable code: code/lld/ch09/composite/ (see the Full Edition).

Transparent vs safe composite

	Transparent	**Safe**
`add/remove` live	On the shared Component	Only on the Composite
Uniformity	Full, no casts	Builders must hold a `FolderNode`
Type safety	`leaf.add(...)` compiles, **throws at run time**	Compiler rejects `file.add(...)`
Ours	`MenuComponent` ("Tea is a leaf")	`FileSystemNode` / `FolderNode`
Choose when	Callers add children to arbitrary nodes (UI editor)	Tree built in one place, read in many (most cases)

Neither is wrong — check the choice was **deliberate**. *Head First* notes transparency knowingly bends SRP.

Two traps worth knowing

- **Cycles** — a folder inside its own descendant makes `size()` recurse forever (`StackOverflowError`). **Validate on insert.**
- **Repeated work** — `print()` calling `size()` at every level is ~quadratic; for big trees **cache subtree totals** and invalidate upward (needs parent pointers).

Trade-offs

Pros	Cons
No type checks in callers	Shared interface **bloats**
Recursion in one place	Weaker type safety (transparent)
New node types don't touch callers (Open/Closed)	Deep trees → deep stacks; cycles → endless recursion
Nesting for free	Totals recomputed unless cached

When not to use it

Nothing nests; leaves and containers share little (methods that throw are worse than the type test); deep trees on a hot path; trees modified during traversal.

Common mistakes

Child list typed as the leaf (`List<FileNode>`); returning the mutable child list (bypasses `add()`'s cycle check); `add()` on the interface by accident; no cycle check; using it where one recursive function with one type check would do.

Real-world examples

HTML DOM; `java.awt.Container`, Android `ViewGroup`; file systems; compiler ASTs; `java.io.File` (`listFiles()` returns `null` on a file — the transparent cost); JUnit 3 `TestSuite`; Spring `CompositeCacheManager`.

In modern languages

Sealed types + exhaustive `switch` (Java 21, Kotlin, Rust) are a real alternative — the compiler catches a forgotten node type; prefer them when the set of node types is closed. Functional languages: recursive ADTs + `fold`.

Other patterns it is confused with

Looks like	Difference
Decorator	Wraps **one** and adds behaviour; Composite holds **many** and combines
Iterator	Walks the tree without exposing it; often used together
Visitor	Adds new operations over the structure without editing nodes
Plain recursive function	Enough when only one operation needs the type check

In the interview

> **30-second answer:** "Composite lets a client treat a single object and a tree the same way. Leaf and container share one interface; the container stores children *typed as that interface* — which allows nesting — and delegates and combines. Its value is deleting the `instanceof` in every operation. I choose transparent or safe depending on whether callers need `add` on arbitrary nodes, reject cycles on insert, and cache totals for big trees."

- *Q: Transparent or safe?* — A: Safe if the tree is built in one place; transparent if callers add to arbitrary nodes.
- *Q: New operation without touching every node?* — A: Visitor.

Self-check

1. **Q:** Why `List<FileSystemNode>`, not `List<FileNode>`? **A:** So folders can contain folders.
2. **Q:** Transparent `add` on a leaf? **A:** Compiles, throws `UnsupportedOperationException`.
3. **Q:** Where to detect cycles? **A:** On insert.

TL;DR

Part-whole trees, one interface; leaf answers, composite asks children and combines. Child list holds the **interface**. Safe vs transparent; guard **cycles**; cache big totals.

9.4 Decorator Pattern

What it is

Decorator extends **one object** at run time by wrapping it in an object with **the same interface**; the original class is never edited. GoF intent: *attach additional responsibilities to an object dynamically — a flexible alternative to subclassing.* Also nicknamed **Wrapper**. Analogy: espresso + milk + mocha + whip — still a drink.

Term	Ours
Component	`UserRepository`, `Beverage`, `InputStream`
Concrete Component	`DatabaseUserRepository`, `Espresso`
Decorator (base) — holds a Component, forwards everything	`RepositoryDecorator`, `FilterInputStream`
Concrete Decorator — adds one behaviour	`LoggingRepository`, `CachingRepository`, `Mocha`, `UpperCaseInputStream`

Key property: **a decorator IS-A and HAS-A Component**, so it can wrap the real object, another decorator or a test double.

The problem it solves

Logging, caching, retries, metrics — each caller wants its own mix. **Subclassing**: 4 optional concerns → 2^4 = **16** classes, doubling per concern. **Flags**: `new Repo(true, false, true, true)`. Decorator: **one class per concern** (4), stacked at run time — Open/Closed.

How it works

1. **Component interface**. 2. **Concrete component** doing the real work. 3. **Abstract base decorator** holding an `inner` of the **interface type**, **forwarding every method**. 4. **Concrete decorators** override only what they change, working before/after `inner`. 5. **Compose**: `new Logging(new Caching(new Database()))`.

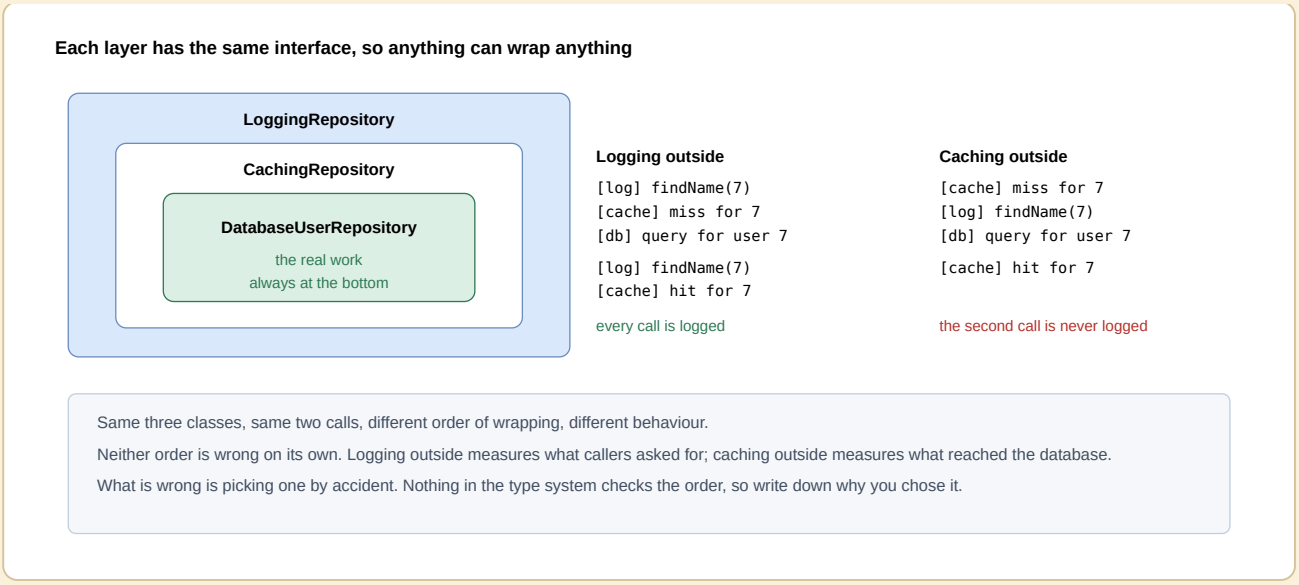
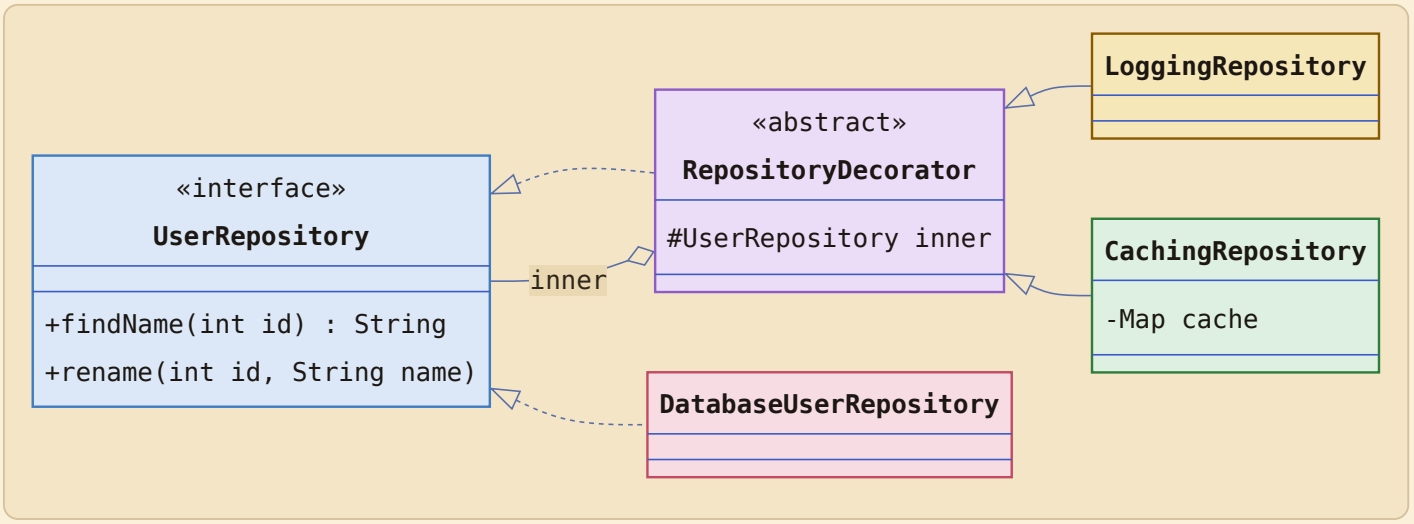


Figure — The same three classes stacked in two orders. Logging outside caching logs every call; caching outside logging means the second call (a cache hit) is never logged (course diagram)

Structure

The bug that bites is **forgotten delegation** — a decorator that overrides `findName` but swallows `rename`. The abstract base decorator forwarding everything is the Java guard.



Java example

```
abstract class RepositoryDecorator implements UserRepository {
    protected final UserRepository inner;

    protected RepositoryDecorator(UserRepository inner) { this.inner = inner; }

    @Override public String findName(int id)          { return inner.findName(id); }
    @Override public void rename(int id, String name) { inner.rename(id, name); }
}

/** CONCRETE DECORATOR 2 - note it must also invalidate on writes. */
final class CachingRepository extends RepositoryDecorator {
    private final Map<Integer, String> cache = new HashMap<>();

    CachingRepository(UserRepository inner) { super(inner); }

    @Override
    public String findName(int id) {
        if (cache.containsKey(id)) {
            System.out.println("    [cache] hit for " + id);
            return cache.get(id);
        }
        System.out.println("    [cache] miss for " + id);
        String name = super.findName(id);
        cache.put(id, name);
        return name;
    }

    @Override
    public void rename(int id, String name) {
        cache.remove(id);
        super.rename(id, name);
    }
}

/** Add-ons are decorators: they ARE a Beverage and HAVE a Beverage. */
record Milk(Beverage inner) implements Beverage {
    public String description() { return inner.description() + ", Milk"; }
    public int costCents() { return inner.costCents() + 10; }
}
```

`LoggingRepository` prints then delegates. `new Whip(new Mocha(new Mocha(new Espresso())))` = 1.99 + 0.20 + 0.20 + 0.10 = **$2.49** (double mocha = wrap twice; prices in int cents). `UpperCaseInputStream extends FilterInputStream` is our own `java.io` decorator (ASCII only).

Full runnable code: code/lld/ch09/decorator/ (see the Full Edition).

Layer order changes behaviour

- **Logger outside, cache inside** — every request logged → the log shows *what callers asked*.
- **Cache outside, logger inside** — only misses logged → the log shows *what hit the database*.

Pair	Outside	Inside
Retry vs cache	Repeats the whole lookup	Repeats only the DB call; success fills the cache. Put retry **next to what fails**
Metrics vs cache	Caller-perceived time	DB-only time (what on-call is paged on); often record both
Authorization vs cache	**Correct**	**Security bug** — a cache hit skips the check

The compiler accepts every order — **comment the reason where the stack is built.**

Two related traps

- **Stateful decorators applied twice** (a counter counts every call twice).
- **Lost identity** — `==`, `equals`, `instanceof` on the wrapper differ from the wrapped object.

Decorator vs inheritance

	Subclassing	Decorator
Chosen	Compile time	Run time
How many	One subclass	Any number, any order
Classes for N features	Up to 2^N	N
`final` / not-owned classes	No	Yes (needs only the interface)
Private state	Protected members	No — public interface only

Java I/O streams: the decorator you already use

`new BufferedReader(new InputStreamReader(new FileInputStream("data.txt"), UTF_8))` — `FileInputStream` is the concrete component, `InputStreamReader` is strictly an **adapter** (bytes → chars), `BufferedReader` a **decorator**. `FilterInputStream` is the byte-side base decorator; `BufferedInputStream`, `DataInputStream`, `PushbackInputStream`, `GZIPInputStream`, `DigestInputStream` are concrete decorators. Downside (*Head First*): many small classes.

Trade-offs

Pros	Cons
Any mix without a class per mix (4, not 16)	Deeper stack traces
Concerns tested in isolation (SRP)	**Identity surprises**
Assembled at run time	**Order matters**, unchecked
Wrapped class never edited (Open/Closed)	Over-layering

When not to use it

Exactly one permanent concern; the concern needs private state; the framework already offers it (middleware, interceptors, Spring AOP); the stack would get deep.

Common mistakes

Holding the concrete class instead of the interface (can't stack); forgetting to forward methods; a cache that doesn't invalidate on writes; authorization inside a cache; relying on `instanceof` of the outer object.

Real-world examples

`java.io`; `Collections.unmodifiableList/synchronizedList/checkedList`; `HttpServletRequestWrapper`; HTTP middleware; Resilience4j/Polly retry/circuit-breaker stacks (order documented); Python `@lru_cache`.

In modern languages

Python function decorators, JS higher-order functions, Go struct embedding, C# `DelegatingHandler`.

In the interview

> **30-second answer:** "Decorator adds responsibilities at run time by wrapping an object in another with the same interface. Four optional concerns would be sixteen subclasses; with decorators each concern is one class and I compose the stack. The decorator holds the interface, which lets layers stack. Order matters: retry closest to what fails, authorization outside any cache."

- *Q: Decorator vs Proxy?* — A: Same structure; Decorator **adds behaviour** to an object it's handed (often stacked), Proxy **controls access** and often creates the real object itself. The boundary is fuzzy — a caching wrapper can be either.
- *Q: vs Chain of Responsibility?* — A: In Decorator every layer normally works and forwards; in CoR typically one handler handles the request.

Self-check

1. **Q:** 5 optional features: decorators vs subclasses? **A:** 5 vs up to 32.
2. **Q:** Why is "authorization inside cache" a bug? **A:** A cache hit returns data without the check.
3. **Q:** Is `InputStreamReader` a decorator? **A:** No — an adapter; `BufferedReader` is the decorator.

TL;DR

Wrap to add behaviour, keep the interface (IS-A + HAS-A). N features = N classes, any order — **order matters**. Java I/O, `synchronizedList`, middleware, Resilience4j. A decorator that short-circuits drifts toward Proxy.

9.5 Facade Pattern

What it is

Facade gives a complicated subsystem **one simple front door**. GoF intent: *provide a unified interface to a set of interfaces in a subsystem; a higher-level interface that makes it easier to use.* Analogy: a hotel concierge — you can still call the restaurant yourself.

Term	Ours
Facade — knows which classes to call, in what order, and the cleanup	`OrderFacade`
Subsystem classes — do the work, unaware of the facade	`InventoryService`, `PaymentService`, `ShippingService`, `EmailService`
Client	`Main` / checkout page

The problem it solves

Placing an order = **reserve stock** → **charge card** → **create shipment** → **send confirmation**, repeated by every caller — and each must also undo the reservation if the card is declined. The **undo** is where bugs live: forget it and units stay locked. The subsystem is fine (flexible = many knobs); callers just keep rewriting the standard path.

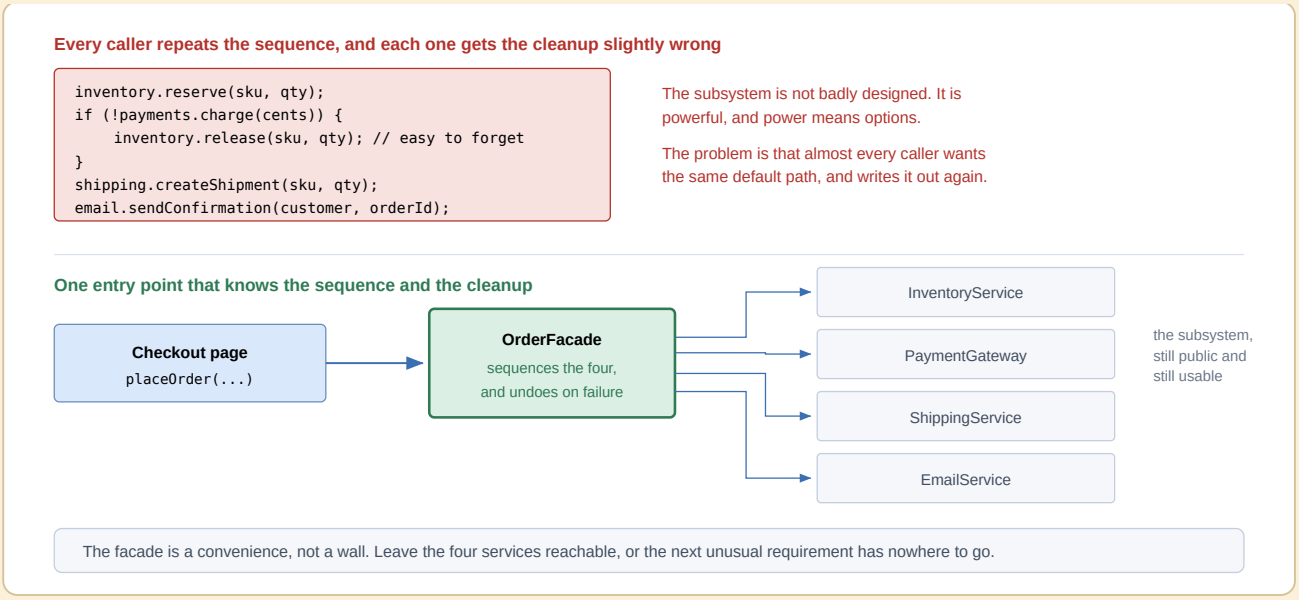


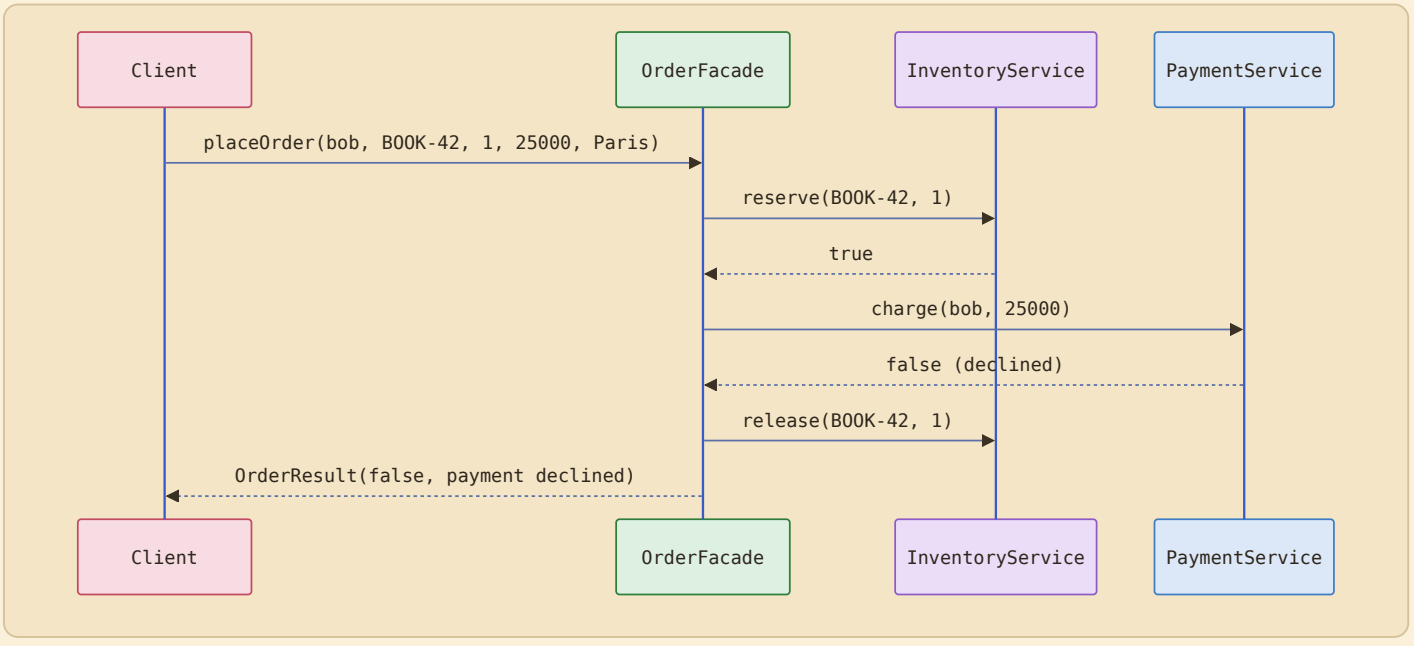
Figure — Top: each caller repeats the reserve/charge/release/ship/email sequence and may forget the release. Bottom: the checkout page calls `OrderFacade.placeOrder(...)`, which sequences the four services and undoes on failure; the services stay public (course diagram)

How it works

1. Identify the **common path**. 2. A **facade class** holding the subsystem objects. 3. One **intention-revealing method** (`placeOrder`) calling them in order. 4. **Own the failure handling** (a *compensating action*: release on decline). 5. **Keep the subsystem public**. 6. **No business rules** — it *sequences*, not *decides*.

Structure

Dependencies run **one way** — no service knows the facade. Usually a concrete class, no interface (unless you'll swap the whole subsystem).



Java example

```
final class OrderFacade {
    private final InventoryService inventory;
    private final PaymentService payments;
    private final ShippingService shipping;
    private final EmailService email;
    private int orderSeq = 100;
    // ... constructor takes the four services ...
    OrderResult placeOrder(String customer, String sku, int qty, long cents, String address) {
        String orderId = "ORD-" + orderSeq++;
        if (!inventory.reserve(sku, qty)) {
            return new OrderResult(false, orderId, "out of stock");
        }
        if (!payments.charge(customer, cents)) {
            inventory.release(sku, qty);            // compensating step callers used to forget
            return new OrderResult(false, orderId, "payment declined");
        }
        String shipmentId = shipping.createShipment(sku, qty, address);
        email.sendConfirmation(customer, orderId);
        return new OrderResult(true, orderId, "shipped as " + shipmentId);
    }
}
```

`OrderResult` is our own small result type (no subsystem types leak). `Main` still calls `inventory.reserve(...)` directly — **a convenience, not a wall**.

Full runnable code: code/lld/ch09/facade/ (see the Full Edition).

Facade vs Adapter vs Mediator

	Facade	**Adapter**	**Proxy**	**Mediator (Ch. 10–11)**
Intent	Simplify a subsystem	Convert an interface	Control access	Coordinate peers
Objects wrapped	**Several**	One	One	Several colleagues
Interface	New, smaller	Client's Target	**Same** as real	Mediator's own
Who chose it	**You**	**Someone else**	Fixed by real object	You
Wrapped objects know it?	**No**	No	No	**Yes** — talk through it
Direction	Client → facade → subsystem	Client → adapter → adaptee	Client → proxy → subject	Colleague ↔ mediator ↔ colleague
Example	`OrderFacade.placeOrder`	`StripeGateway.pay`	`LazyReport`	Air-traffic control; chat room

Fastest test: Adapter/Decorator/Proxy stand in front of **one** object; a Facade in front of **several**. If the subsystem's API became perfect tomorrow, a Facade would still simplify; an Adapter would be pointless. **Facade vs service layer:** same thing while it only sequences; with business rules it has become a domain/application service.

The failure mode to watch for

Facades **pull in features** — a discount rule, then a fraud check, then a **god object**. Rule: **a facade orchestrates; it does not decide.** Related principle (*Head First*): **Least Knowledge / Law of Demeter** — clients talk to the facade, not chains of subsystem objects.

Trade-offs

Pros	Cons
One call replaces a sequence	One more class to maintain
Order and undo in one place	**Feature magnet** → god object
One seam for swapping the subsystem	Opinionated — some caller needs an omitted option
Smaller public surface, lower coupling	Can mask problems; single point of failure

When not to use it

The subsystem is already simple; callers need the full range (every caller bypasses it); covering a mess instead of fixing it; wanting it to be the *only* entrance.

Common mistakes

Business rules in the facade; making the subsystem inaccessible; returning subsystem types; forgetting the failure path; confusing it with Mediator.

Real-world examples

Application service layers (`OrderService`); `java.net.URL.openStream()` (constructors deprecated since Java 20 — use `URI.create(s).toURL()`); SLF4J "Simple Logging Facade"; `FacesContext`; jQuery's `$`; Python `requests` over `urllib3`; e-commerce "Place order" orchestrators / backend-for-frontend.

In modern languages

Often just a module or package exporting one function (or a JPMS module exporting only the facade package).

In the interview

> **30-second answer:** "A Facade gives one simple entry point to a complex subsystem. `OrderFacade.placeOrder` reserves, charges, ships and emails in order, and releases the reservation if payment fails, so no caller has to remember the cleanup. It wraps several objects, unlike Adapter, Decorator and Proxy. I keep the subsystem public and business rules out — the risk is a god object."

- *Q: vs Mediator?* — A: Subsystem classes don't know the facade; colleagues know the mediator and talk through it both ways.
- *Q: Several facades?* — A: Yes — one per use case, or layered.

Self-check

1. **Q:** Fastest structural tell vs Adapter/Decorator/Proxy? **A:** It wraps several objects.
2. **Q:** What happens on a declined payment? **A:** It releases the reservation and returns a failed `OrderResult`.
3. **Q:** Add a 10% discount inside `placeOrder`? **A:** No — the facade sequences, a service decides.

TL;DR

One door into a complex subsystem; names the common path and owns the cleanup. Wraps **several**; subsystem stays **public**; **sequences, never decides**.

9.6 Flyweight Pattern

What it is

Flyweight shares the part of each object's state that is the same across many objects so huge numbers fit in memory. GoF intent: *use sharing to support large numbers of fine-grained objects efficiently.* Analogy: one type design for every "e", placed at different positions.

Term	Meaning	Map example
Intrinsic state	Identical for every holder; lives **inside** the shared flyweight	icon bitmap, colour, size
Extrinsic state	Varies per context; stays **outside**, **passed in** as arguments	latitude, longitude

Role	Ours
Flyweight (shared, immutable, intrinsic only)	`MarkerStyle`
Flyweight factory (owns the pool)	`MarkerStyleFactory`
Context (extrinsic state + reference)	`Marker`
Client	`Main`

GoF's own list: *Flyweight* (interface accepting extrinsic state), *ConcreteFlyweight* (`MarkerStyle` plays both), *UnsharedConcreteFlyweight* (same interface, not shared), *FlyweightFactory*, *Client*; "Context" is a later name (refactoring.guru).

The problem it solves

100,000 map markers, each with its own position plus a 2 MB icon bitmap → ~195 GB: the program **runs out of memory**. Same for a million game trees sharing one mesh, or font data per character.

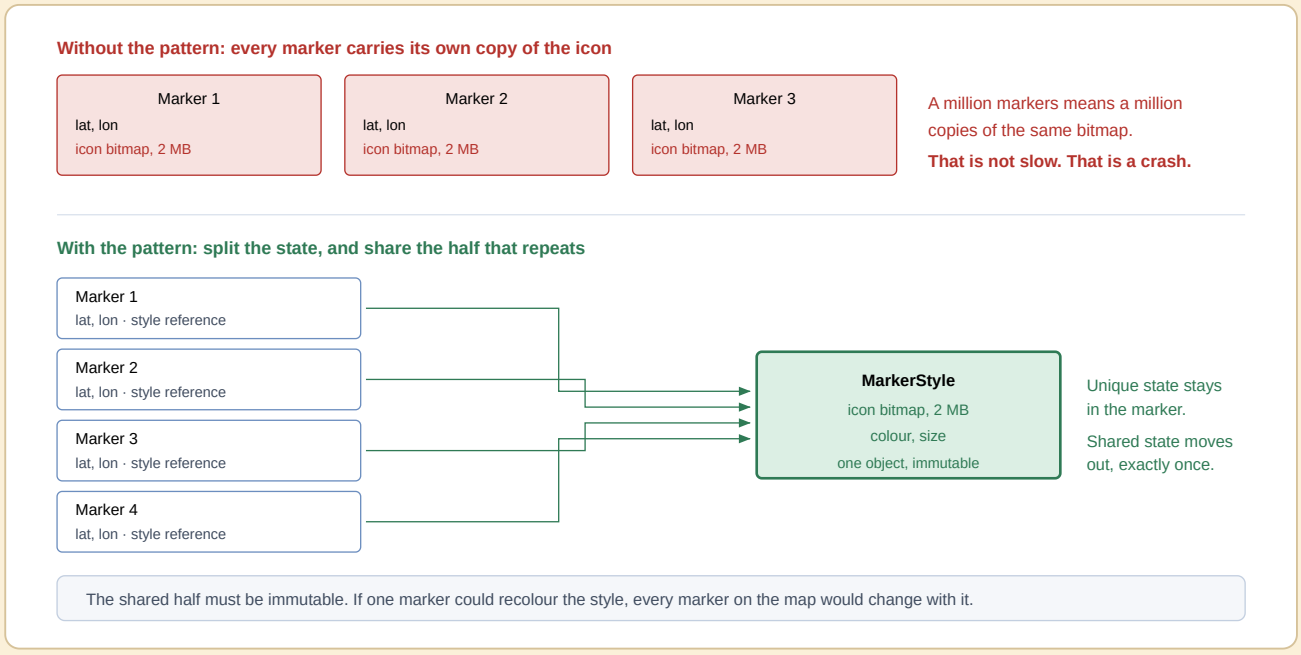


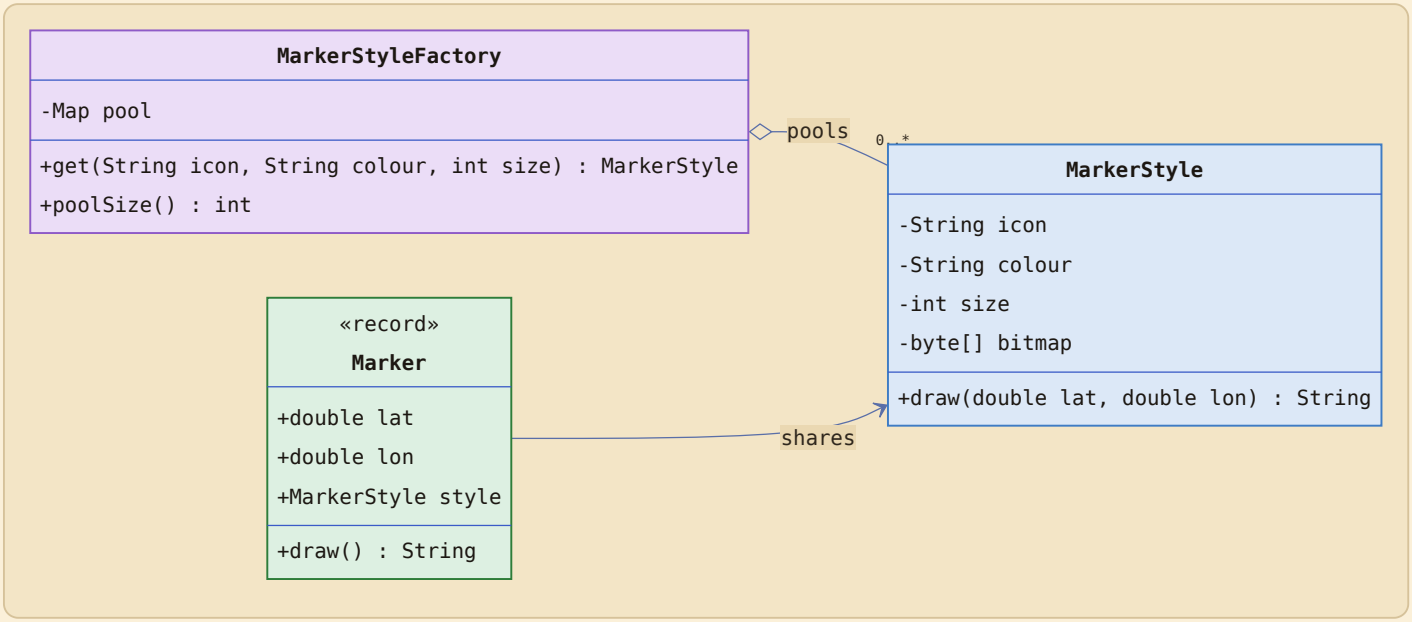
Figure — Top: every marker carries its own 2 MB icon. Bottom: each marker keeps only lat/lon and a style reference; one immutable `MarkerStyle` holds the icon, colour and size (course diagram)

How it works

1. **Split** intrinsic vs extrinsic. 2. Intrinsic → an **immutable** flyweight (`final`, no setters). 3. Extrinsic → **method arguments** (`draw(lat, lon)`). 4. A **factory owns the pool**, keyed by intrinsic values, returning existing or creating once. 5. Clients **only use the factory** (non-public constructor). 6. Contexts hold a **reference**, not a copy.

> **Two rules — break either and the pattern silently fails:** (1) **flyweights never change**; (2) **only the factory creates them**.

Structure



Java example

```
final class MarkerStyle {
    static final int ICON_BYTES = 2 * 1024 * 1024;   // pretend every icon bitmap is 2 MB

    private final String icon;
    private final String colour;
    private final int size;
    private final byte[] bitmap;   // the heavy part we want exactly one copy of
    // ... package-private constructor loads the bitmap ...
    /** EXTRINSIC state (the position) is passed in, never stored. */
    String draw(double lat, double lon) {
        return String.format("%s %s marker at (%.4f, %.4f)", colour, icon, lat, lon);
    }
    // ... bitmapBytes() ...
}

/** FLYWEIGHT FACTORY: owns the pool. Thread-safe via ConcurrentHashMap.computeIfAbsent. */
final class MarkerStyleFactory {
    /** The pool key is the combination of intrinsic values. */
    private record Key(String icon, String colour, int size) { }

    private final Map<Key, MarkerStyle> pool = new ConcurrentHashMap<>();

    MarkerStyle get(String icon, String colour, int size) {
        return pool.computeIfAbsent(new Key(icon, colour, size),
                k -> new MarkerStyle(k.icon(), k.colour(), k.size()));
    }
    // ... poolSize() ...
}

/** CONTEXT: one per marker. Holds extrinsic state + a REFERENCE to a shared style. */
record Marker(double lat, double lon, MarkerStyle style) {
    String draw() { return style.draw(lat, lon); }
}
```

`computeIfAbsent` makes get-or-create **atomic and thread-safe**; the `Key` record gives correct `equals` / `hashCode`.

```
markers: 100000, styles in pool: 3
icon memory without flyweight: ~200000 MB
icon memory with flyweight:    ~6 MB
Integer 127 == 127 -> true  (cached -128..127)
Integer 128 == 128 -> false (outside default cache)
```

Full runnable code: code/lld/ch09/flyweight/ (see the Full Edition).

Flyweights already in Java

- **`Integer.valueOf`** (autoboxing) caches **`−128..127`** (JLS-required); 128 may differ (`-XX:AutoBoxCacheMax` can raise the bound). **Compare boxed numbers with `equals`.** `Short`, `Byte`, `Long` (same range), `Character` (0–127), `Boolean.TRUE/FALSE` likewise.
- **`String` pool** — literals and compile-time constants (`"hel" + "lo"`) are interned; `new String("hello")` is new; `intern()` returns the pooled one.
- **`Enum`s** — one instance per constant.

Flyweight is not a cache

	Flyweight	Cache
Purpose	Make data *fit in memory*	Make repeated work *faster*
Contents	One copy of immutable state shared by many	Stored results of costly work
Remove entries?	Not while referenced	Any time
Effect of removal	Lost sharing / duplicates	Only a slower next call

Two mistakes that undo the pattern

- **A mutable flyweight** — one caller changes a colour and every marker changes.
- **An ever-growing pool keyed by open-ended input** — a leak disguised as an optimisation. Safe only for a **small, bounded** set of intrinsic values; otherwise you've built a cache (weak refs, eviction) — call it one.

Measure first

It trades readability for memory — without profiling it's premature optimisation. Answer with numbers: *how many objects at once?* and *how much of each is identical?*

Trade-offs

Pros	Cons
Memory can fall by orders of magnitude	State split across two places
Costly construction once per distinct value	The pool holds strong references (can leak)
CPU-cache friendly	**Thread safety is on you**
"Load once" logic in one home	Tempting too early; extrinsic state passed on every call

When not to use it

You haven't measured; the shared part can't be immutable; counts are small; unique state dominates; the key space is unbounded (that's a cache).

Common mistakes

Extrinsic state inside the flyweight; a public constructor; a non-thread-safe pool (`HashMap` check-then-put); comparing boxed `Integer`s or `String`s with `==`.

Other patterns it is confused with

Looks like	Difference
Cache	See table above
Singleton	One instance total vs one **per distinct intrinsic value**
Object pool	Lends **mutable** objects and takes them back; flyweights are immutable, shared, never returned
Prototype	Makes many objects by copying; Flyweight avoids making them

Real-world examples

`Integer.valueOf`, `Boolean.valueOf`, `String.intern()`, enums (Python interns small ints/strings too); glyphs in word processors (GoF's example); game meshes/textures with GPU instancing; map pins; `ZoneOffset.ofHours(5)` (cached 15-minute multiples).

In modern languages

Any immutable value object + memoising factory (Java record + `computeIfAbsent`, JS `Object.freeze` + `Map`, C++ `shared_ptr<const Style>`).

In the interview

> **30-second answer:** "Flyweight saves memory with huge numbers of similar objects. I split state into intrinsic — shared and invariant, like an icon — and extrinsic — per object, like a position, passed into methods. The intrinsic part is an immutable flyweight from a factory that pools one instance per distinct value. `Integer.valueOf` and the String pool are JDK examples. I'd only do it after measuring."

- *Q: Why is `Integer 128 == 128` false?* — A: `valueOf` only guarantees caching for −128..127; use `equals`.
- *Q: Thread-safe factory?* — A: `ConcurrentHashMap.computeIfAbsent` — creation runs at most once per key.

Self-check

1. **Q:** Chess piece: intrinsic vs extrinsic? **A:** Type, colour, image vs its square.
2. **Q:** What if `MarkerStyle` had `setColour`? **A:** Changing one marker changes every marker sharing it.
3. **Q:** When does a pool leak? **A:** When keys are unbounded (e.g. user input) and nothing evicts.

TL;DR

Share what repeats, pass in what varies. **Immutable** flyweight + **factory-owned**, thread-safe pool. `Integer.valueOf` (−128..127), String pool, enums. **Measure first**; not a cache.

9.7 Proxy Pattern

What it is

Proxy puts a **stand-in with the identical interface** in front of a real object to control **who** may use it, **when** it's created, or **how much** each use costs. GoF intent: *provide a surrogate or placeholder for another object to control access to it* (a.k.a. **Surrogate**). Analogy: a credit card standing in for your bank account.

Term	Ours
Subject	`Report`, `RateService`, `Device`
Real Subject	`RealReport`, `LiveRateService`, `RealDevice`
Proxy	`LazyReport`, `AuditedReport`, `CachingRateProxy`, `RemoteRateProxy`, `SharedDevice` handles

The problem it solves

Callers shouldn't touch the real object directly (or yet): it's **costly and usually unneeded** (a 100 MB report when most screens show only its title); **only some callers are permitted**; it's **on another machine**; **each call must be logged/counted** but the class isn't yours. Same interface → insert it **without editing callers** — which is also the danger: a slow call **looks cheap**.

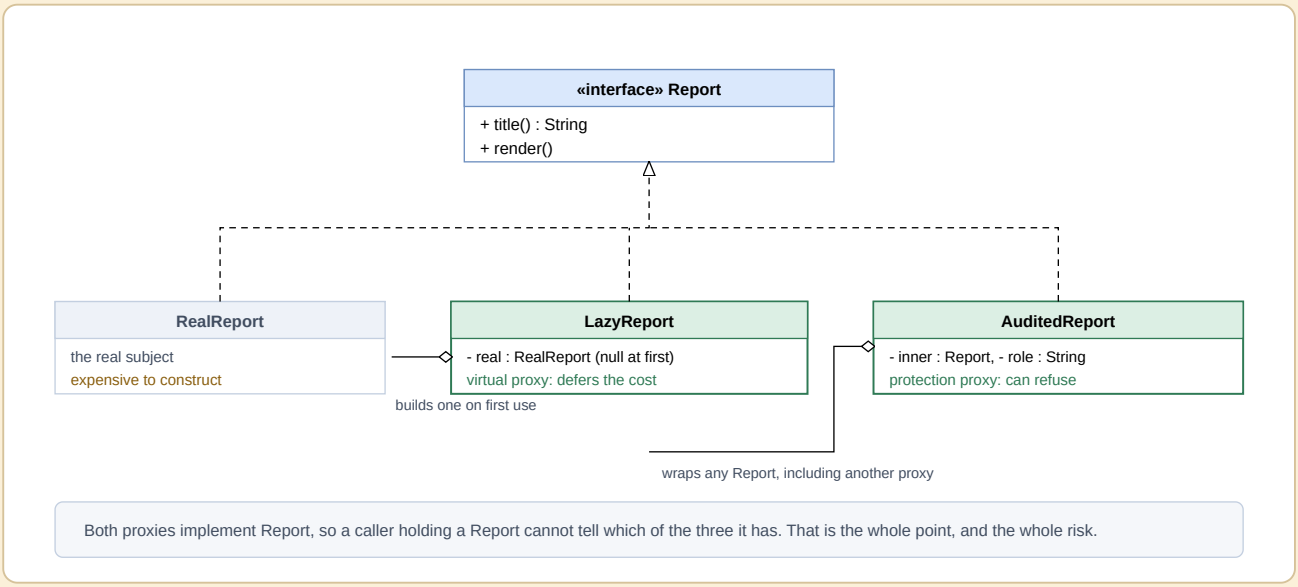


Figure — Proxy class diagram: `RealReport`, `LazyReport` (virtual proxy) and `AuditedReport` (protection proxy) all implement `Report`; `LazyReport` creates the `RealReport` itself, `AuditedReport` wraps any `Report` (course diagram)

How it works

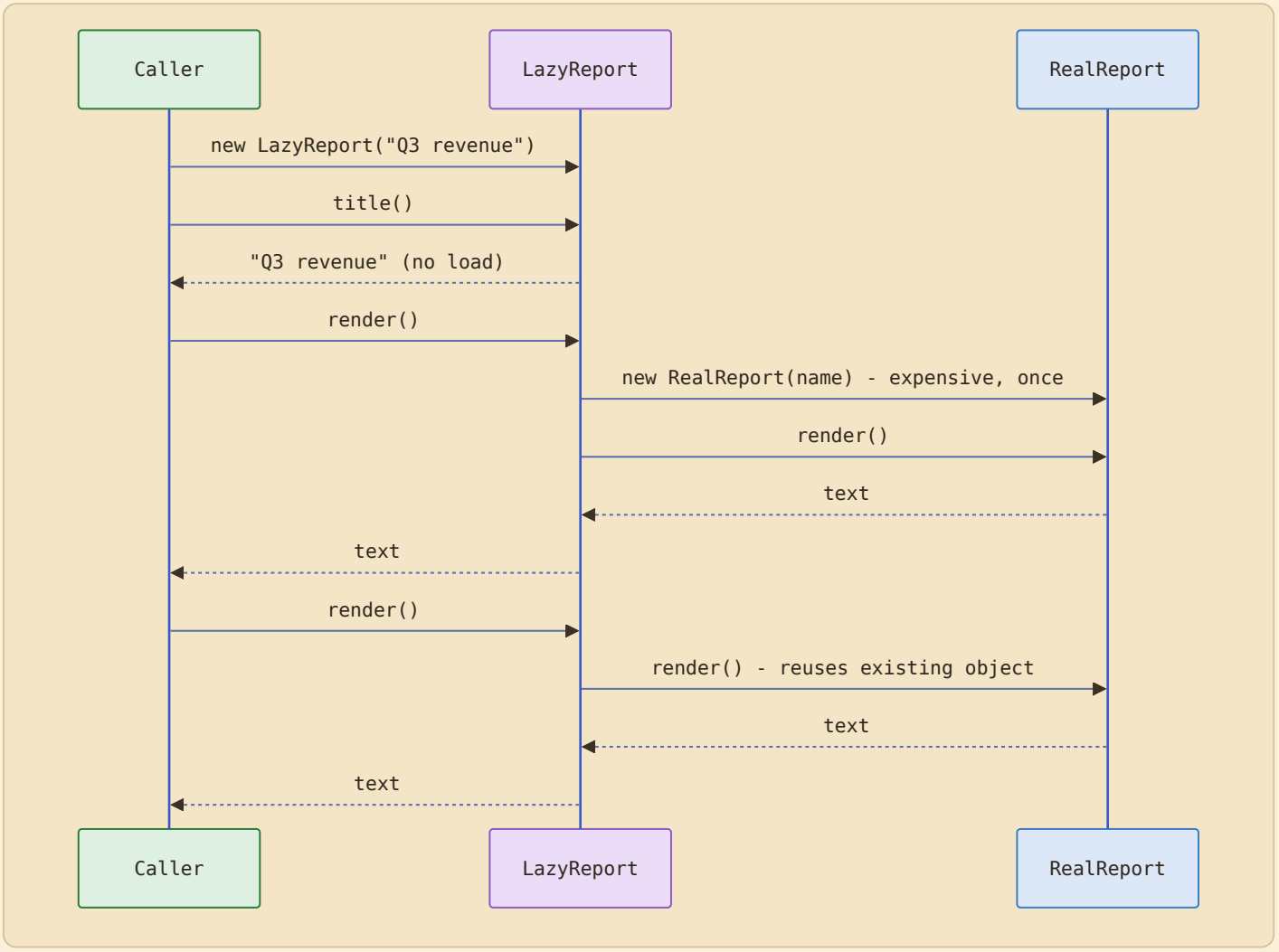
1. Define/reuse the **Subject** interface. 2. A proxy implementing it, holding the real subject **or what's needed to create it**. 3. In each method do the proxy's job (create lazily, check permission, check cache, marshal, count), then **delegate or refuse**. 4. Give callers the proxy (often via a factory or DI).

Kinds of proxy

Kind	Controls	Ours / real
Virtual	*When* it's created (lazy)	`LazyReport`; Hibernate lazy entities
Protection	*Who* may call	`AuditedReport`; `Collections.unmodifiableList`
Remote	*Where* it lives	`RemoteRateProxy` (stub) + `RateServer` (skeleton); RMI/gRPC
Caching	*Whether* the call is needed	`CachingRateProxy` (LRU + TTL)
Logging / monitoring	Records calls	`LoggingHandler` (dynamic proxy)
Smart reference	Bookkeeping: ref counting, locking, load on first use	`SharedDevice` closes on last handle

GoF names four (remote, virtual, protection, smart reference); caching and logging are later variants (GoF would file logging under smart reference).

Structure


Caller
LazyReport
RealReport
new LazyReport("Q3 revenue")
title()
"Q3 revenue" (no load)
render()
new RealReport(name) - expensive, once
render()
text
text
render()
render() - reuses existing object
text
text
Caller
LazyReport
RealReport

Java example

```
final class LazyReport implements Report {
    private final String name;
    private volatile RealReport real;      // created by the proxy itself, not handed in

    LazyReport(String name) { this.name = name; }

    @Override public String title() { return name; }   // no load needed

    @Override
    public String render() { return realReport().render(); }

    private RealReport realReport() {
        RealReport r = real;
        if (r == null) {
            synchronized (this) {
                r = real;
                if (r == null) real = r = new RealReport(name);
            }
        }
        return r;
    }
}

/** PROTECTION PROXY: checks permission, may REFUSE the call. */
final class AuditedReport implements Report {
    private final Report inner;
    private final Role callerRole;
    private final Set<Role> allowed;
    // ... constructor copies allowed with Set.copyOf ...
    @Override public String title() { return inner.title(); }  // titles are public

    @Override
    public String render() {
        if (!allowed.contains(callerRole)) {
            throw new SecurityException(callerRole + " may not render " + inner.title());
        }
        return inner.render();
    }
}
```

- **Thread-safe lazy init** — double-checked locking; the field **must be** `volatile`, or two threads may build twice (a real bug if construction opens a connection or charges a card).
- `AuditedReport` wraps a `LazyReport`: the viewer is refused **before anything loads**.
- `CachingRateProxy` is **bounded** (LRU via access-ordered `LinkedHashMap` + `removeEldestEntry`) and **time-limited** (TTL): 5 calls → 4 lookups. Rates are `BigDecimal`.
- `RemoteRateProxy` marshals `"rate|USD|INR"` over a simulated network to `RateServer`.
- `SharedDevice` hands out handles; the device opens on the first and closes on the last, all under one lock, `close()` idempotent.

Full runnable code: code/lld/ch09/proxy/ (see the Full Edition).

Static vs dynamic proxies

	Static	Dynamic
Made by	Hand-written per interface	Generated at **run time**
Code	One class each, every method	One `InvocationHandler` for **any** interface
Type safety	Full	Reflective (`Method` + `Object[]`)
Good for	Type-specific logic (lazy load, domain access rules)	**Cross-cutting**: logging, timing, transactions, security
API	—	`Proxy.newProxyInstance(loader, interfaces, handler)`
Limit	Boilerplate	JDK proxies are **interface-only**; classes need CGLIB/ByteBuddy

JDK mechanics: `newProxyInstance` generates `$Proxy0` implementing the interfaces → every call goes to `handler.invoke(proxy, method, args)` → the handler works and calls `method.invoke(target, args)` → **unwrap** `InvocationTargetException` (`e.getCause()`) so callers see the real exception.

The self-invocation trap (and Spring AOP)

In the demo, `convert` called `rate(...)` on `this`, bypassing the logging proxy. **Spring AOP** works the same way: `@Transactional`, `@Cacheable`, `@Async`, `@PreAuthorize` and `@Aspect` advice wrap your bean in a proxy — a **JDK dynamic proxy** if it implements an interface, else **CGLIB** (Spring Boot 2.0+ defaults to CGLIB). **Self-calls aren't intercepted** (`this.saveAudit()` with `REQUIRES_NEW` is silently ignored); with CGLIB, `final` classes can't be proxied and `final`/`private` methods can't be advised. Fixes: move the method to another bean; inject the bean's own proxy (lazy self-reference); or (discouraged) `AopContext.currentProxy()` with `exposeProxy = true`. AspectJ weaving avoids proxies altogether.

Proxy vs Decorator

GoF: similar implementations, different **purpose** — Decorator *adds responsibilities*, Proxy *controls access*.

	Proxy	Decorator
Purpose	**Controls access**	**Adds behaviour**
Call gets through?	Not necessarily — defer, cache, refuse	Normally yes
Who creates the real object?	**Often the proxy**	**The caller** hands it in
Client knows?	Usually not (factory/DI supplies it)	Usually yes (builds the stack)
Layers	Usually one	Often several

Creation test: `LazyReport(String name)` builds its subject; `LoggingRepository(UserRepository inner)` is given one. `AuditedReport` *is* handed its subject — only intent separates it from a decorator. A caching wrapper is honestly both; ask instead whether its cache is bounded and invalidated.

Where proxies go wrong

Unsynchronised lazy creation (duplicates); a caching proxy with no bound or expiry (leak + stale data); **concealed cost** — a remote proxy used in a loop (fix by making cost visible: batch APIs, `fetch...` names); **ORM N+1** — touching `order.getCustomer().getName()` for 100 orders fires 100 queries.

Trade-offs

Pros	Cons
Pay later for costly construction	Extra indirection in stack traces
No caller changes (Open/Closed)	**Latency and failures disguised** as local calls
Home for cross-cutting rules on classes you can't edit	Lazy creation raises thread-safety questions
Permission checks kept out of the object	Unbounded caches leak and go stale

When not to use it

The real object is cheap; callers need to see the cost (`fetchOverNetwork()`); the framework already generates proxies (ORM, DI, service mesh); you want to add behaviour (Decorator).

Common mistakes

Non-`volatile` DCL field; not unwrapping `InvocationTargetException`; expecting Spring annotations on self-calls, `private` or `final` methods; unbounded caching proxies; remote proxies in tight loops.

Real-world examples

ORM lazy loading (Hibernate `getReference`, lazy associations — the source of N+1 and `LazyInitializationException`); **Spring AOP** (and CGLIB-enhanced `@Configuration` classes unless `proxyBeanMethods = false`); RMI/gRPC/Thrift/SOAP stubs; `java.lang.reflect.Proxy` (Spring Data repositories, MyBatis mappers); `Collections.unmodifiableList`; Envoy/Istio sidecars, Nginx/Cloudflare reverse proxies; C++ `shared_ptr`.

In modern languages

JS built-in `Proxy`; Python `__getattr__`; C# `DispatchProxy` and `Lazy<T>`; C++ smart pointers.

Other patterns it is confused with

Looks like	Difference
Decorator	Adds behaviour to a handed-in object; Proxy controls access, often creates the subject
Adapter	**Different** interface on purpose; Proxy has the **same** one
Facade	New simpler interface over **several**; Proxy the same interface over **one**
A cache	A caching proxy *is* a cache — still needs bounds and eviction

In the interview

> **30-second answer:** "A Proxy is a same-interface stand-in that controls access. I name the kind: virtual defers creation, protection checks permissions, remote hides the network, caching reuses results, smart reference does bookkeeping like ref counting. Callers don't change — which is also the danger, since an expensive call looks cheap. Lazy creation needs thread safety, and ORM lazy proxies cause N+1. In Java I'd use `java.lang.reflect.Proxy` for cross-cutting concerns — that's how Spring AOP does `@Transactional`."

- *Q: Why doesn't `@Transactional` work on a same-class call?* — A: `this.method()` never goes through the proxy.
- *Q: JDK proxy vs CGLIB?* — A: JDK needs interfaces; CGLIB subclasses, so no `final` classes/methods.
- *Q: N+1 fix?* — A: Fetch joins, entity graphs or batch fetching.

Self-check

1. **Q:** Kinds of `LazyReport` and `AuditedReport`? **A:** Virtual; protection.
2. **Q:** Why wasn't the inner `rate()` logged? **A:** Self-invocation — `convert()` called `rate()` on `this`.
3. **Q:** Two must-haves for a caching proxy? **A:** A size bound and an expiry/invalidation policy.

TL;DR

Same interface, controlled access: virtual, protection, remote, caching, logging, smart reference. Static = hand-written; dynamic = `Proxy` + `InvocationHandler`. Spring AOP = proxies → beware self-invocation, `final`, `private`; watch thread safety and hidden costs.

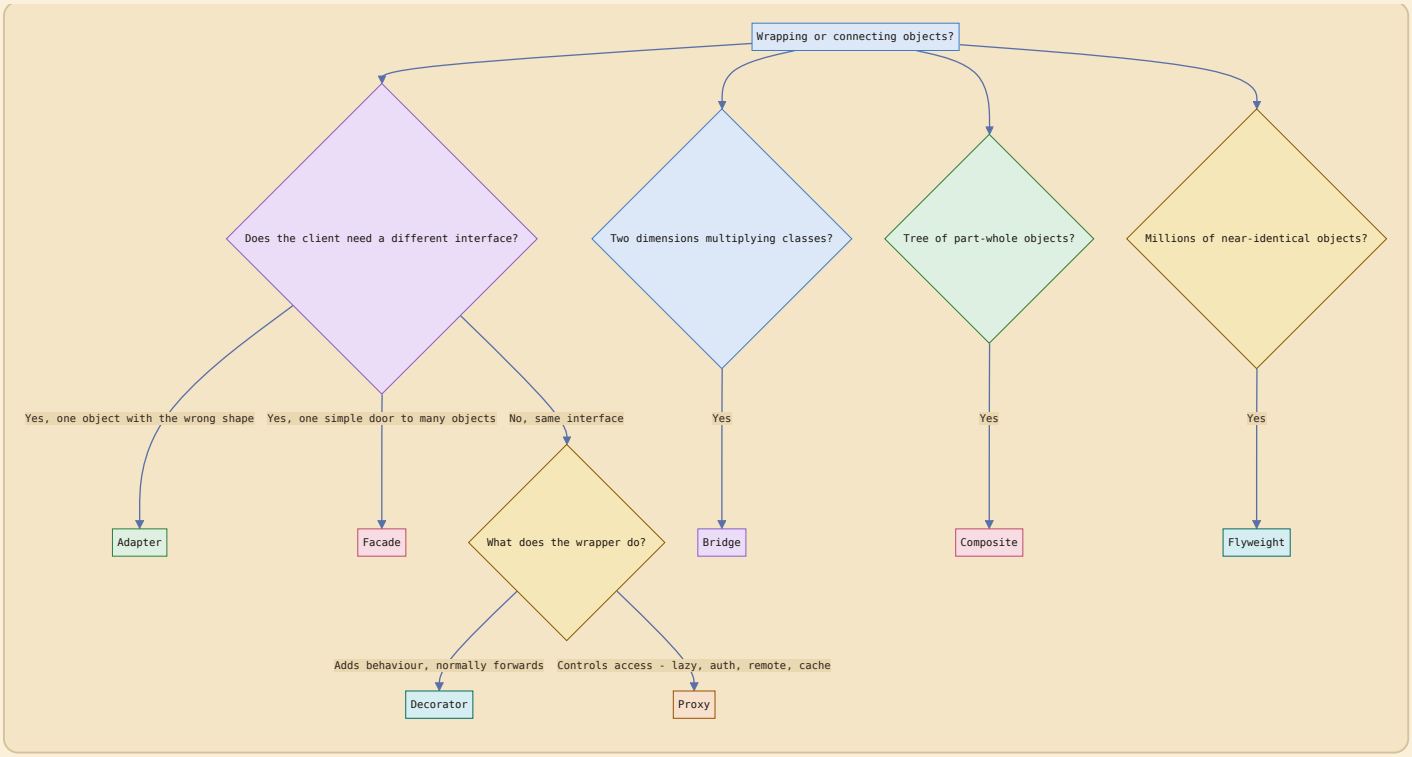
9.8 Putting it all together — comparison

Summary of all seven structural patterns

Pattern	Distinctive feature	Use it when	Example
Adapter	Bridges incompatible interfaces	An existing class doesn't fit	Plug adapter; `StripeGateway`
Bridge	Separates abstraction from implementation	Two dimensions vary independently	Shapes × renderers; notifications × channels
Composite	Part-whole tree, one interface	Treat single and group uniformly	Drawing groups; file system
Decorator	Adds responsibilities dynamically	Run-time combinations of behaviour	Scroll/border on a view; Java I/O
Facade	Simple interface to a subsystem	Callers repeat one common path	"Watch movie" button; `OrderFacade`
Flyweight	Shares common state	Huge numbers of similar objects	Word-processor glyphs; map markers
Proxy	Placeholder controlling access	Lazy, auth, remote, cache, logging, ref counting	Proxy server; Hibernate lazy entity

Adapter vs Decorator vs Proxy vs Facade vs Bridge

	Adapter	Decorator	Proxy	Facade	Bridge
Intent	Convert an interface	Add behaviour, same interface	Control access	Simplify a subsystem	Vary abstraction and implementation independently
Client sees	**Different** (yours)	**Same**	**Same**	**New, simpler**	Abstraction's own
Behaviour	Unchanged (translated)	**Extended**	**Controlled**	Unchanged, fewer calls	Delegated to implementor
Wraps	One	One (stackable)	One	**Several**	One implementor
Creates wrapped?	Passed in	Passed in by client	**Often itself**	Passed in / owned	Passed in
Introduced	**After the fact**	When a concern is added	When access needs control	When callers repeat a sequence	**Up front**
Trigger	"Vendor API doesn't match"	"Sometimes logging, sometimes caching"	"Expensive / remote / restricted"	"Every caller copies 5 steps"	"Two-noun class names"
Example	`Arrays.asList`, `InputStreamReader`	`BufferedInputStream`, `synchronizedList`	`reflect.Proxy`, `@Transactional`, Hibernate	SLF4J, service layers	JDBC + drivers, AWT peers



Memory hooks

Adapter — make it fit · **Decorator** — add to it · **Proxy** — guard it · **Facade** — simplify it · **Bridge** — split it · **Composite** — nest it · **Flyweight** — share it.

Chapter self-check

1. **Q:** Which wrap exactly one object? **A:** Adapter, Decorator, Proxy (Bridge holds one implementor by design). Facade wraps several; Composite holds many.
2. **Q:** Same code shape, different intent/timing? **A:** Adapter vs Bridge (after vs up front); Bridge vs Strategy; Decorator vs Proxy.
3. **Q:** `WindowsButton`, `MacButton`, `WindowsCheckbox`… and Linux coming? **A:** Bridge (widget × platform), often with Abstract Factory choosing the implementor.
4. **Q:** Why is `Integer.valueOf(100) == Integer.valueOf(100)` true? **A:** Flyweight (Integer cache).

Chapter TL;DR

- Structural patterns **compose objects into bigger structures** while staying flexible.
- **Adapter / Decorator / Proxy / Facade** share a shape — tell them apart by **what changes as the call passes** and **how many objects are wrapped**.
- **Bridge** stops two-dimension class explosion; **Composite** makes trees uniform; **Flyweight** shares immutable state.
- In interviews: name the pattern, the **constraint** justifying it, its **trap** (vendor leaks, wrong split, cycles, decorator order, god facade, mutable flyweight, self-invocation) and a **real JDK/Spring example**.

Chapter 10 — Behavioral Design Patterns (Part 1) · Study Edition

> **Study edition.** Every topic of the full chapter, with the padding removed. For complete code listings, full outputs and extra follow-ups, see the Full Edition (`README.md`).
>
> **What you will learn.** What behavioral patterns are, and six of them: **Chain of Responsibility, Command, Iterator, Mediator, Memento** and **Interpreter** — definition, problem, class diagram, core Java 21 code, real uses and interview answers. Chapter 11 covers Observer, State, Strategy, Template Method, Visitor and some non-GoF patterns.

How to run the code

```
cd chapters/10-patterns-behavioral-1/code
javac --release 21 -d /tmp/lld_10 $(find . -name '*.java')
java -cp /tmp/lld_10 lld.ch10.chain.Main        # also: command, iterator, mediator, memento, interpreter
```

10.0 Behavioral patterns: the big picture

What it is

Creational patterns (Ch 8) are about **how objects are made**; structural (Ch 9) about **how they are put together**. **Behavioral patterns** are about **how objects share work and talk** — who is responsible for what, and who calls whom (in a kitchen: who hears about an order first, who passes it on, who can cancel it).

They cut coupling, untangle many-to-many communication and let behaviour change **at run time**.

The eleven GoF behavioral patterns

The GoF book (*Design Patterns*, Gamma, Helm, Johnson, Vlissides, 1994) lists eleven:

Pattern	One-line intent	Ch
Chain of Responsibility	Pass a request down handlers; each handles, stops or forwards it.	10
Command	Wrap a request in an object to store, queue, log or reverse it.	10
Interpreter	A small language's grammar as classes; evaluate sentences.	10
Iterator	Walk a collection without seeing how it is stored.	10
Mediator	Route communication through one coordinator.	10
Memento	Snapshot state and restore it, keeping it private.	10
Observer	Notify all dependants when one object changes.	11
State	Change behaviour when internal state changes.	11
Strategy	Pick an interchangeable algorithm at run time.	11
Template Method	Base class fixes the skeleton; subclasses fill steps.	11
Visitor	Add an operation without changing the classes.	11

Class vs object scope. *Class* behavioral patterns use **inheritance**: only **Template Method** and **Interpreter**. All others are *object* patterns using **composition**, so they can be rewired at run time.

The common idea: take behaviour hidden in a big method (an `if`-chain, a hard-coded call, a loop) and turn it into an **object** you can name, pass, swap, store and test.

10.1 Chain of Responsibility

What it is

Chain of Responsibility (CoR) sends a request through a row of **handlers**. Each handler either handles it, rejects it, or forwards it to its successor. The sender talks only to the first handler and never learns which one handled it.

GoF intent: *avoid coupling sender to receiver by giving several objects a chance to handle the request; chain them and forward until one does.* Participants: **Handler**, **ConcreteHandler**, **Client**. Analogy: support escalation — level 1, level 2, engineer; you dialled one number.

The problem it solves

Before creating an order a request must pass authentication, rate limiting and validation. In one method: the method grows with each rule; the **order of checks is hidden** yet matters; routes needing only some checks lead to flags or copies. CoR makes **each check a small object** linked into a line, so **the line is configuration, not logic**: a new rule = one class + one wiring line; reordering touches only wiring; each route can have its own chain.

How it works

A **Handler** type has `handle(request)` and a **next** reference; each **concrete handler** checks, then **stops** or **passes on**; a **terminal handler** does the real work; the chain is **wired once** at start-up (`auth.setNext(rateLimit).setNext(validate).setNext(createOrder)`) and the client calls only the **first** handler.

Structure (UML)

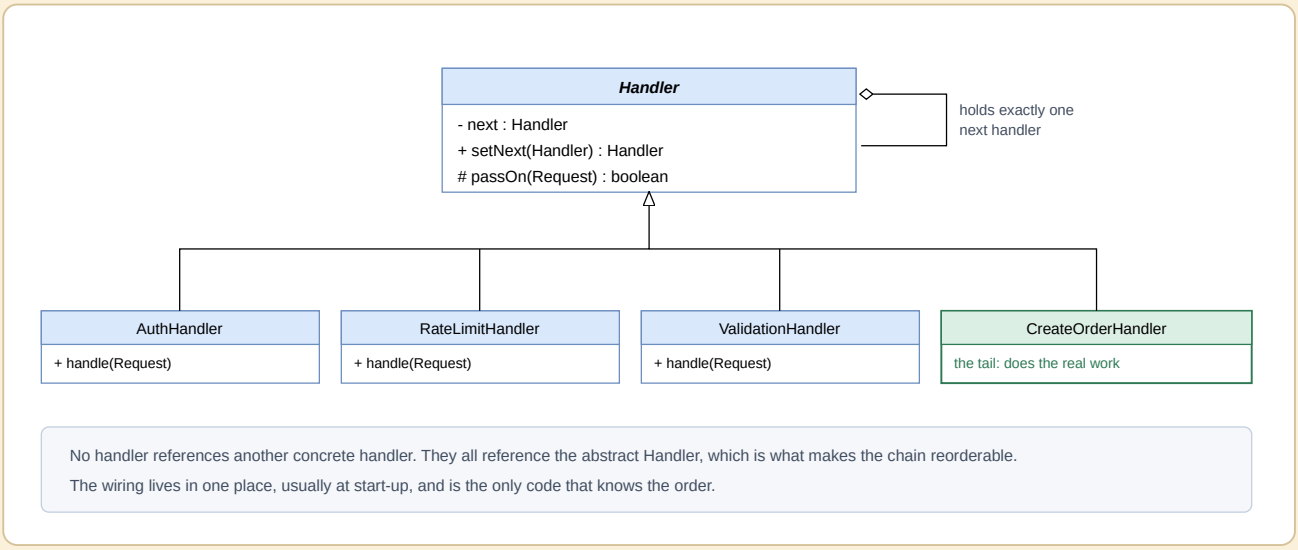
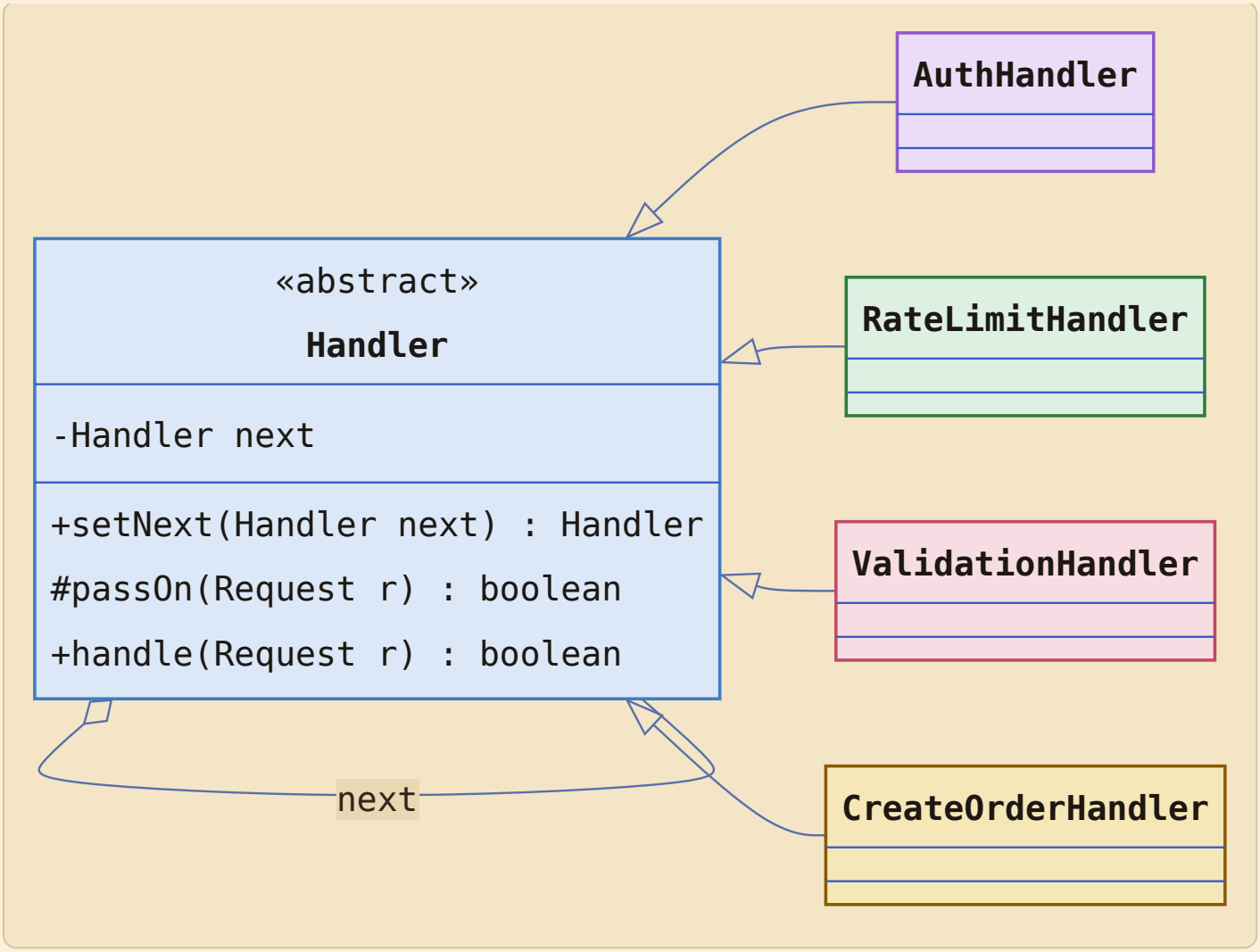


Figure — Abstract Handler holds `next`; concrete handlers extend it; CreateOrderHandler is the tail (course diagram)

> **Our code vs the course.** The course's `passOn` returns `true` when there is no next handler; ours **throws**, because silent "success" when the chain runs out is exactly the invisible failure this pattern is prone to.



Java example

`lld.ch10.chain` shows four flavours: (1) **classic linked handlers** with explicit outcomes (`false` = rejected, `passOn` = continue; `passOn` throws if there's no terminal handler); (2) an **approval chain** (`Approver`, `BigDecimal` limits) where the first capable approver wins and the chain is just data; (3) **servlet-style filters** where `next` is an **argument** (`chain.doFilter(...)`), so a filter can act before *and after* the rest, or stop by not calling it — a `FilterChain` holds a position, so one per request; (4) **logging levels**, where every handler acts if severe enough and always passes on — a pipeline-like variant.

```java
// File: code/lld/ch10/chain/Handler.java
public abstract class Handler {
    private Handler next;

    public Handler setNext(Handler next) {
        this.next = next;
        return next;
    }

    /** Forwards the request. Falling off the end means the chain has no terminal handler: a wiring bug. */
    protected boolean passOn(Request request) {
        if (next == null) {
            throw new IllegalStateException(
                getClass().getSimpleName() + " tried to pass on, but the chain has no terminal handler");
        }
        return next.handle(request);
    }

    public abstract boolean handle(Request request);
}

// File: code/lld/ch10/chain/AuthHandler.java
final class AuthHandler extends Handler {
    @Override
    public boolean handle(Request r) {
        if (r.token() == null || r.token().isBlank()) {
            System.out.println("  [auth] rejected: missing token");
            return false;                    // explicit outcome: the chain stops here
        }
        System.out.println("  [auth] ok for " + r.user());
        return passOn(r);                    // explicit outcome: continue
    }
}

// File: code/lld/ch10/chain/Main.java (wiring)
        // Order is a design decision: auth BEFORE rate limiting so the limiter can key on the user,
        // and so anonymous traffic cannot burn a real user's quota.
        Handler pipeline = new AuthHandler();
        pipeline.setNext(new RateLimitHandler(2))
                .setNext(new ValidationHandler())
                .setNext(new CreateOrderHandler());
        // ...
```

In the output, a tokenless request stops at auth (later stages never run); an expense above every limit gets an explicit `REJECTED`; a `DEBUG` log reaches only the console while an `ERROR` reaches all three handlers.

Full runnable code: code/lld/ch10/chain/ (see the Full Edition).

Trade-offs

Pros	Cons
New stage = one class + one wiring line (Open/Closed).	A request's route is spread over the wiring and several files.
Each stage testable against a stub successor; per-route chains.	Nothing verifies the chain is complete or correctly ordered.
Sender doesn't know who handles.	A handler that neither handles nor forwards makes the request vanish.

Order is a design decision no tool verifies: auth first lets the limiter count per *user*; rate-limit first keys on IP but shields expensive auth from floods. **Comment the reason beside the wiring.**

When not to use it

- Every request always goes through every stage in fixed order — four calls in one method are clearer.
- Handlers need each other's results — that's a *pipeline*; pass a context object step by step.
- No terminal handler exists (add one first).
- Long chain on a very hot path.

Common mistakes

1. **The swallowing bug:** a handler returns without `passOn` — the request silently vanishes. Give every path an explicit outcome; test with a stub successor.
2. **No terminal handler**; 3. **wrong, undocumented order**; 4. **handlers that construct each other**; 5. **sharing a stateful** `FilterChain` across requests.

Real-world examples

Servlet filters and Spring Security's `SecurityFilterChain`; Spring `HandlerInterceptor.preHandle` (return `false` to stop); HTTP middleware (Express `next()`, Django, ASP.NET Core, Go handler wrappers), OkHttp interceptors, Netty `ChannelPipeline`; `java.util.logging` parent handlers; exception propagation; DOM event bubbling (`stopPropagation()`); approval and support-escalation workflows.

In the interview

> **30-second answer:** "Chain of Responsibility splits a series of checks into separate handler objects, each linked only to its successor. The request moves down the line until some handler completes or refuses it. Only the wiring knows the order, so adding or reordering rules is cheap. Servlet filters and HTTP middleware are the everyday examples. The classic bug is a handler that forgets to pass on, so I give every path an explicit outcome and put a terminal handler at the end."

- *vs Decorator?* Decorator wrappers all run and always delegate inward to *add* behaviour; a CoR handler may **end** the request.
- *vs pipeline?* A pipeline runs **all** steps; if no stage can stop the flow, it's a pipeline, not CoR.

Self-check

1. **Q:** What does each handler reference? **A:** Only the next handler, via the abstract type.
2. **Q:** Most common CoR bug? **A:** A handler that neither handles nor passes on — the request disappears.
3. **Q:** Why do frameworks pass `next` as a parameter? **A:** A missing `next(...)` is visible in the handler body, and the handler can work before and after the rest.

TL;DR

A request walks a line of handlers; each handles, rejects, or passes on. The chain is data — but **order matters**, **always have a terminal handler**, and **never swallow a request**.

10.2 Command

What it is

Command packages a request ("add 2 widgets to the cart") as an **object** holding everything needed to perform it later. Then you can **store it, queue it, log it, retry it, run it on a worker thread** — and, if it knows its inverse, **undo** it.

GoF intent: *encapsulate a request as an object, so clients can be parameterised with requests, requests can be queued or logged, and operations can be undone.* Analogy: a waiter (invoker) writes an order slip (command) for the cook (receiver) to act on later.

The problem it solves

`cart.setDiscount(10)` runs and is forgotten. That fails once you need **undo**, **a queue**, **a log of intent**, **retry**, or **one action with many triggers** (menu, hot-key, script). Command gives the request a body that outlives the call.

How it works

Name the **three roles** — invoker, command, receiver — plus the client. GoF participants: Command, ConcreteCommand, Client, Invoker, Receiver.

Role	Job	In our code
Command	Interface with `execute()` (optionally `undo()`).	`Command`, `UndoableCommand`
Concrete command	Stores receiver + arguments; performs (and reverses) one operation.	`AddItemCommand`, `ApplyDiscountCommand`, `MacroCommand`
Receiver	Object whose state really changes; unaware of commands.	`Cart`
Invoker	Triggers commands and records them; unaware of carts.	`CommandHistory`, `ExecutorService`
Client	Builds a command and gives it its receiver.	`Main`

Invoker: `execute()` then push on an **undo stack**; **undo** pops, calls `undo()`, pushes on a **redo stack**; **redo** reverses that; **a fresh command clears redo**.

Structure (UML)

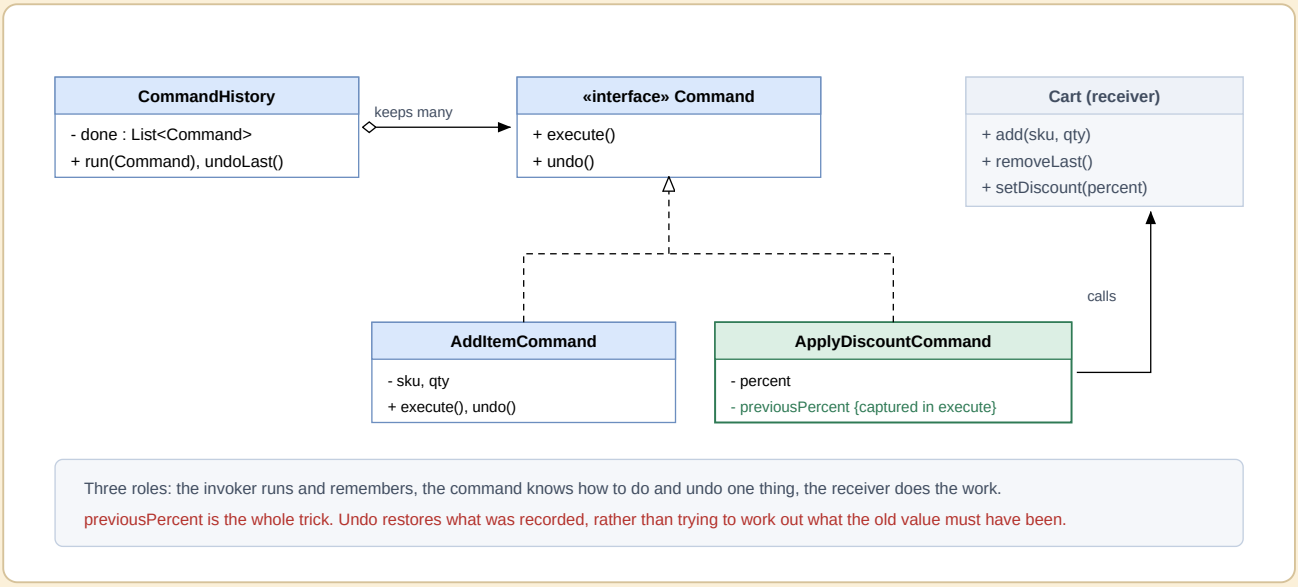
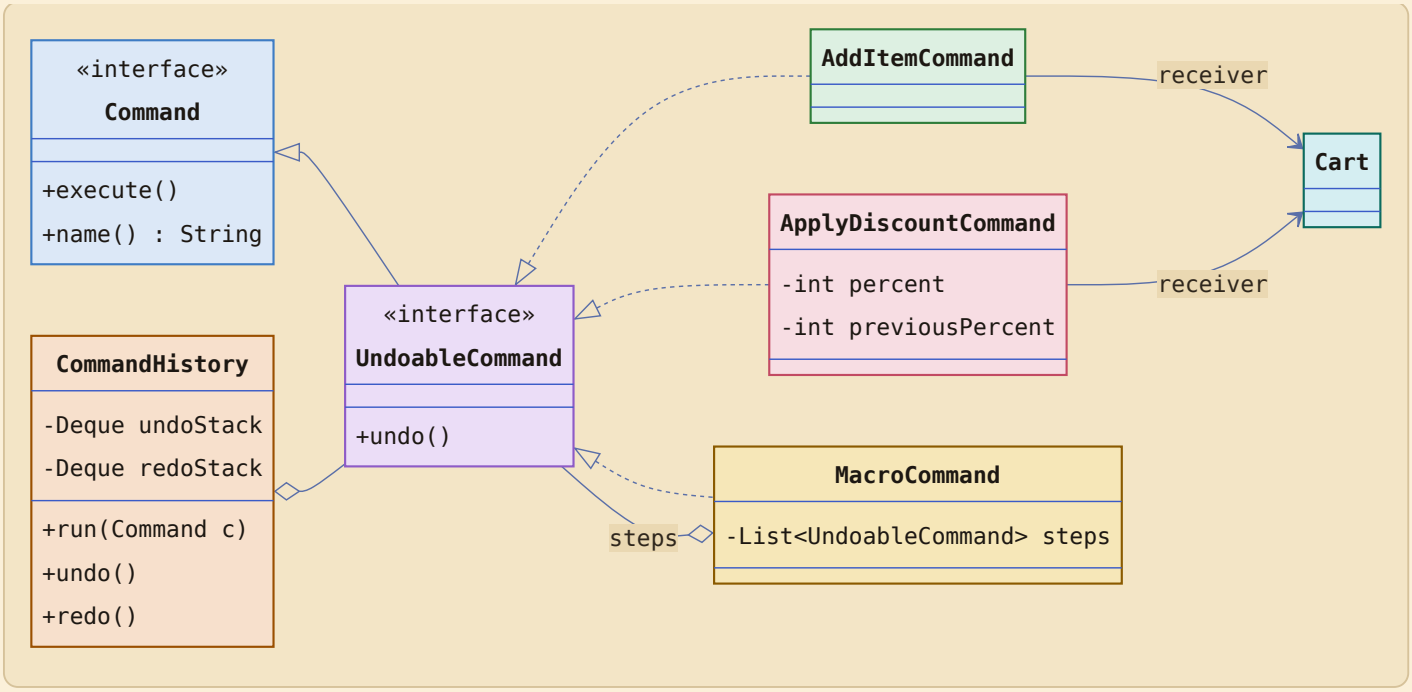


Figure — Command interface with execute/undo, two concrete commands, Cart as receiver, CommandHistory as invoker (course diagram; its `undoLast` / `removeLast` are `undo` / `remove` in our code)

`previousPercent` is the one field that makes undo correct; `CommandHistory` never mentions `Cart`, so one history serves the whole app.



Java example

- `Command` has **one abstract method**, so a lambda is a command; only honestly reversible commands implement `UndoableCommand`, and `CommandHistory` keeps irreversible ones out of the undo stack.
- `ApplyDiscountCommand` **captures the old value in** `execute()`; the cart starts at 5% to show "undo = set to 0" would be wrong.
- `MacroCommand` is a **Composite** of commands: executes in order, undoes in **reverse**.
- `Runnable` + `ExecutorService` is the JDK's version (command + invoker on a worker thread).

```java
// File: code/lld/ch10/command/Command.java
@FunctionalInterface
public interface Command {
    void execute();

    default String name() { return getClass().getSimpleName(); }
}

// File: code/lld/ch10/command/UndoableCommand.java
public interface UndoableCommand extends Command {
    void undo();
}

// File: code/lld/ch10/command/ApplyDiscountCommand.java
final class ApplyDiscountCommand implements UndoableCommand {
    private final Cart cart;
    private final int percent;
    private int previousPercent;          // filled in by execute()
    // ... constructor

    @Override
    public void execute() {
        previousPercent = cart.discountPercent();   // capture BEFORE changing
        cart.setDiscount(percent);
    }

    @Override public void undo()   { cart.setDiscount(previousPercent); }
    @Override public String name() { return "ApplyDiscount(" + percent + "%)"; }
}

// File: code/lld/ch10/command/CommandHistory.java
public final class CommandHistory {
    private final Deque<UndoableCommand> undoStack = new ArrayDeque<>();
    private final Deque<UndoableCommand> redoStack = new ArrayDeque<>();

    public void run(Command command) {
        command.execute();
        if (command instanceof UndoableCommand u) {
            undoStack.push(u);
            redoStack.clear();     // a new action destroys the "future" you could have redone
            System.out.println("  ran  " + command.name());
        } else {
            System.out.println("  ran  " + command.name() + " (irreversible: kept out of history)");
        }
    }

    public void undo() {
        if (undoStack.isEmpty()) { System.out.println("  nothing to undo"); return; }
        UndoableCommand c = undoStack.pop();
        c.undo();
        redoStack.push(c);
        System.out.println("  undo " + c.name());
    }
    // ... redo() mirrors undo(): pop from redo, execute(), push on undo
}
```

Output (excerpt): after undoing the discount the cart shows `discount=5% <- discount restored to the recorded 5%, not 0%`; running a new command then makes `redo` print `nothing to redo`; `SendReceipt` runs but is `kept out of history`.

Full runnable code: code/lld/ch10/command/ (see the Full Edition).

Undo must restore, not recompute

"Reset to 0" is a **guess**. Rule: `execute` **records what** `undo` **needs** (the fields it overwrites). History is **LIFO** — a stack. **When undo is a lie** (email sent, card charged): keep it out of history, or use **compensation** (a **refund** — both events stay on record).

Trade-offs

Pros	Cons
Request outlives the call: undo/redo, queues, audit, retry.	One class per action; an extra hop when reading code.
Generic invoker — one history for all commands; macros compose.	Each command must capture enough to run later and reverse.
One action backs menu, hot-key, script and API.	One faulty `undo` corrupts the whole history.

When not to use it

Simple immediate calls nobody undoes, queues or logs; undo that isn't really possible (use compensation); plain deferral (a lambda/`Runnable` already is a command); commands that must outlive the process (use a durable queue or event log).

Common mistakes

1. Recomputing in `undo` instead of restoring. 2. Not clearing redo on a new command. 3. Business logic in the invoker. 4. Capturing a mutable object instead of values. 5. Unbounded history.

Real-world examples

Editor undo and `javax.swing.undo.UndoManager`; `Runnable` / `Callable` with `ExecutorService` / `Thread` as invokers; Swing `Action`, .NET `ICommand`; Redux actions; job queues (Celery, Sidekiq, Kafka messages), outbox tables; DB migrations with `up` / `down`; event sourcing (Ch 11).

In the interview

30-second answer: "Command wraps a request, with its receiver and arguments, in an object — so it can be kept, queued, logged, retried or reversed. The invoker triggers and records commands without knowing what they touch; each command performs and reverses one operation; the receiver holds the state. For undo, `execute` captures the old value and `undo` restores it; history is a stack and a new command clears redo. Irreversible actions get compensation. In Java, `Runnable` plus an `ExecutorService` is the everyday form."

- *vs Strategy?* Strategy = *how* a job is done, swapped by the caller. Command = *which* job, with arguments, kept to run later or reverse.
- *vs Memento for undo?* Command stores the **action** + a little state (cheap); Memento stores the **whole state** (always correct). Real systems combine them.

Self-check

1. **Q:** What must a command carry? **A:** Its receiver, all arguments, and any state needed for undo.
2. **Q:** Why store `previousPercent`? **A:** So undo restores the real previous value instead of guessing.
3. **Q:** What happens to redo when a new command runs? **A:** It is cleared.

TL;DR

Request as object: **invoker** runs it, **command** knows do/undo, **receiver** changes. Undo **restores captured state**, history is a **stack**, irreversible actions get **compensation**. A lambda is a command when you only need deferral.

10.3 Iterator

What it is

Iterator lets you visit a collection's elements **one at a time without knowing how they are stored** — array, list, tree, or a paginated remote API.

GoF intent: *provide sequential access to an aggregate's elements without exposing its representation.* Participants: **Iterator**, **ConcreteIterator** (tracks position), **Aggregate** (creates iterators), **ConcreteAggregate**. Analogy: a TV remote's "next channel".

The problem it solves

- `get(i)` loops assume an array; move the data into a tree or remote service and every loop breaks.
- Many sources **can't be indexed**: trees (several visit orders), graphs (must track visited), paged APIs. Each caller re-implementing traversal re-implements paging, some wrongly.

Iterator moves walking into a separate object answering `hasNext` and `next`. **The position lives in the iterator, never in the collection** — so two traversals can run at once, and `iterator()` must return a **new** iterator each time.

How it works

The **aggregate** implements `Iterable<T>` (`iterator()`); the **iterator** implements `Iterator<T>` (`hasNext()`, `next()`, optional `remove()`) and keeps its own **cursor**; the client loops `while (it.hasNext()) use(it.next());` — which is what `for (T x : collection)` compiles to.

Structure (UML)

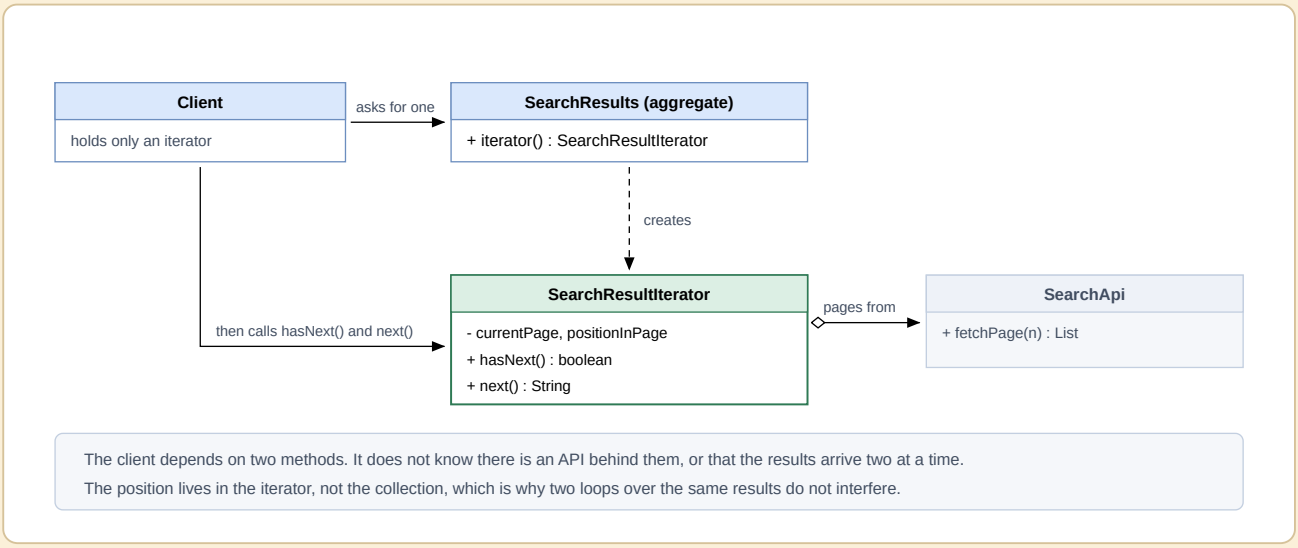
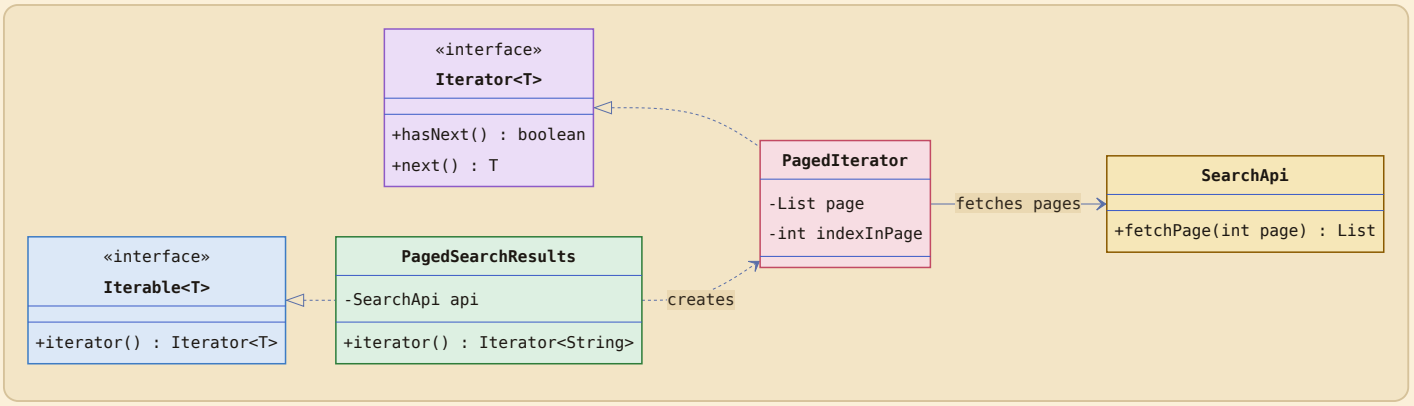


Figure — SearchResults hands out a SearchResultIterator, which calls SearchApi page by page (course diagram)

Here `hasNext` may fetch a page — never assume it is cheap or side-effect-free.



Java example

`lld.ch10.iterator` shows: (1) a **paged remote source behind a for-each**; (2) a **tree iterator** (explicit stack; two iterators independent); (3) **external** (client drives, can `break`) vs **internal** iteration (`forEach`, streams); (4) **fail-fast vs fail-safe** — removing from an `ArrayList` mid-loop *usually* throws `ConcurrentModificationException`, but removing the second-to-last element silently skips the last; use `Iterator.remove()`/`removeIf`; (5) `Spliterator` with `trySplit` for parallel streams.

```
// File: code/lld/ch10/iterator/PagedSearchResults.java
final class PagedSearchResults implements Iterable<String> {
    private final SearchApi api;

    PagedSearchResults(SearchApi api) { this.api = api; }

    @Override
    public Iterator<String> iterator() { return new PagedIterator(); }

    private final class PagedIterator implements Iterator<String> {
        private List<String> page = List.of();
        private int indexInPage = 0;
        private int pageNumber = 0;
        private boolean exhausted = false;

        @Override
        public boolean hasNext() {                // NOT a passive question: may hit the network
            if (indexInPage < page.size()) return true;
            if (exhausted) return false;
            page = api.fetchPage(++pageNumber);
            indexInPage = 0;
            if (page.isEmpty()) {
                exhausted = true;
                return false;
            }
            return true;
        }

        @Override
        public String next() {
            if (!hasNext()) throw new NoSuchElementException();   // the Iterator contract
            return page.get(indexInPage++);
        }
    }
}
```

The tree iterator (`BinarySearchTree`) keeps a `Deque` of nodes still to visit — that stack *is* its position.

Output (excerpt):

```
  5 items, 4 network calls (the last one only discovers the end)
  b: 20   <- b starts from the beginning: position lives in the iterator
  best-effort only: no exception, visited [java, tmp], 'lld' silently skipped
```

Full runnable code: code/lld/ch10/iterator/ (see the Full Edition).

Hide the shape, not the cost

Five results took **four** calls (no total or next-page token, so the last call only discovers the end). **A tidy loop conceals per-step cost** — the shape of the ORM **N+1 problem**. Name the type honestly, offer a batch API, return a total/cursor, count fetches.

Java's iterator family (quick reference)

Type	What it adds
`Iterable<T>`	`iterator()`; enables for-each; also `forEach`, `spliterator()`.
`Iterator<T>`	`hasNext()`, `next()`, optional `remove()`, `forEachRemaining()`.
`ListIterator<T>`	Bidirectional: `hasPrevious()`, `previous()`, indexes, `set()`, `add()`.
`Spliterator<T>`	`tryAdvance`, `trySplit`, `characteristics()` (`ORDERED`, `SIZED`, `DISTINCT`, `IMMUTABLE`).
`Enumeration<E>`	Java 1.0 version: `hasMoreElements()`, `nextElement()`. Prefer `Iterator`.

Fail-fast (`ArrayList`, `HashMap`): throw `ConcurrentModificationException` — best-effort, usually caused by **one thread** modifying its own loop. **Weakly consistent** (`ConcurrentHashMap`): never throw. **Snapshot** (`CopyOnWriteArrayList`): never throw; `remove()` unsupported.

Trade-offs

Pros	Cons
Client doesn't care if data is an array, tree or API; several visit orders.	Mid-loop modification is a bug unless via `Iterator.remove`.
Lazy: huge or infinite sequences work.	A lazy iterator holding a file/DB cursor leaks on early `break`.
Paging hidden behind a plain loop.	Per-step cost invisible → N+1 problems.

When not to use it

Data already an in-memory list (hand it out); callers need random access; a step has side effects the caller must decide on (loading a page is fine, charging a customer is not).

Common mistakes

1. Modifying the collection mid-loop (use `Iterator.remove`/`removeIf`; Python may silently skip elements). 2. Leaking resources on early `break` (try-with-resources). 3. Reusing a one-shot iterator/stream. 4. `next()` returning `null` instead of throwing `NoSuchElementException`. 5. Storing the cursor in the collection.

Real-world examples

Every for-each loop; JDBC `ResultSet`, `Scanner`; AWS SDK v2 paginators and Stripe auto-pagination; `Stream`s on `Spliterator`s; **generators** (Python `yield`, JS `function*`) — the pattern as a function; infinite-scroll feeds.

In the interview

> **30-second answer:** "Iterator pulls traversal into a separate object with `hasNext` and `next`, so callers walk data without caring if it's an array, tree or paginated API. Each iterator keeps its own position, so two loops are independent. I'd implement `Iterable<T>` so for-each works. My example would be the lazy paged case, and I'd flag the risk: when every step can hit the network, the loop hides its cost — that's how N+1 problems start."

- *Internal vs external?* External: client steps and can stop or interleave. Internal: collection loops and calls your function — simpler, parallelisable, harder to stop early.
- *vs Visitor / generator / stream?* Iterator = **how you move**; Visitor = **what you do** at each element; a generator is Iterator as a function; streams add `map`/`filter` on top.

Self-check

1. **Q:** Where does the cursor live? **A:** In the iterator object.

2. **Q:** What does for-each need from a class? **A:** It implements `Iterable<T>`.
3. **Q:** Why 4 calls for 5 results? **A:** With no total or cursor, the end is found only by requesting one more page.

TL;DR

A separate object walks the collection with `hasNext/next`; **position lives in the iterator**. Implement `Iterable`. Know **fail-fast vs snapshot**, **internal vs external**, and don't let a lazy loop **hide its cost**.

10.4 Mediator

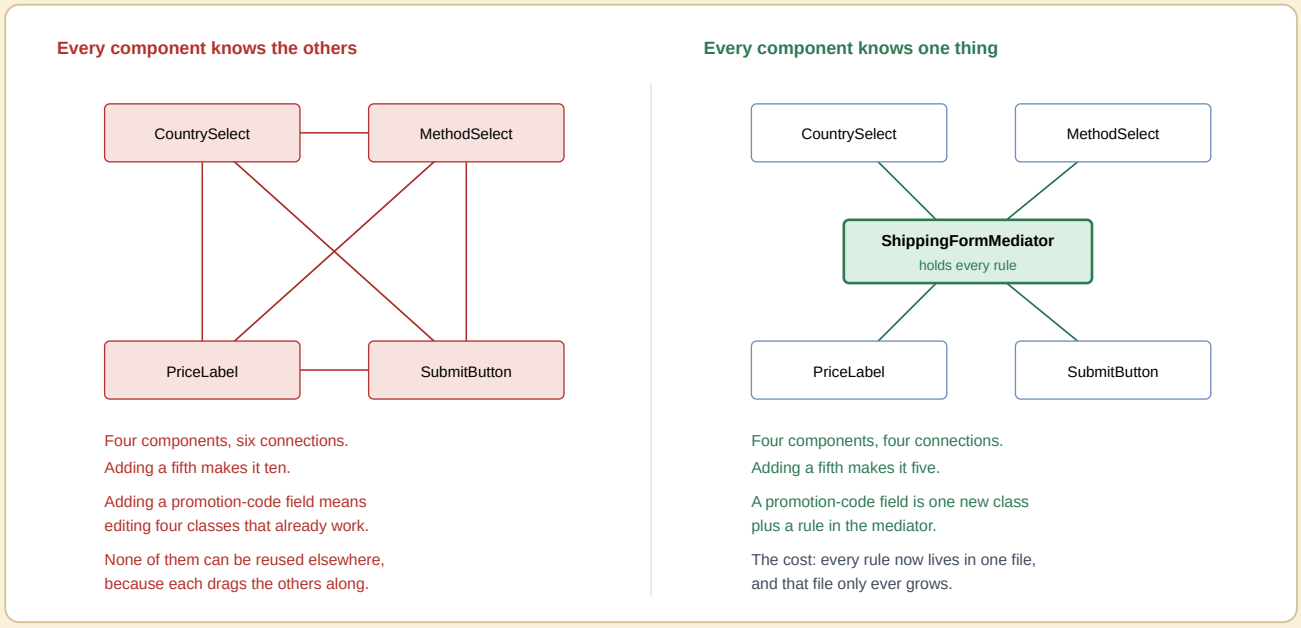


Figure — Left: four components all know each other (six links). Right: each knows only the mediator (four links) (course diagram)

What it is

Mediator makes a group of objects talk through **one coordinator** instead of referencing each other. Each object reports "this happened"; the mediator decides how others respond.

GoF intent: *define an object that encapsulates how a set of objects interact, so they don't refer to each other explicitly and their interaction can vary independently.* Participants: **Mediator**, **ConcreteMediator** (knows colleagues, implements coordination), **Colleague** (knows only its mediator). Sometimes called a *controller*. Analogy: **air traffic control** — planes talk only to the tower.

The problem it solves

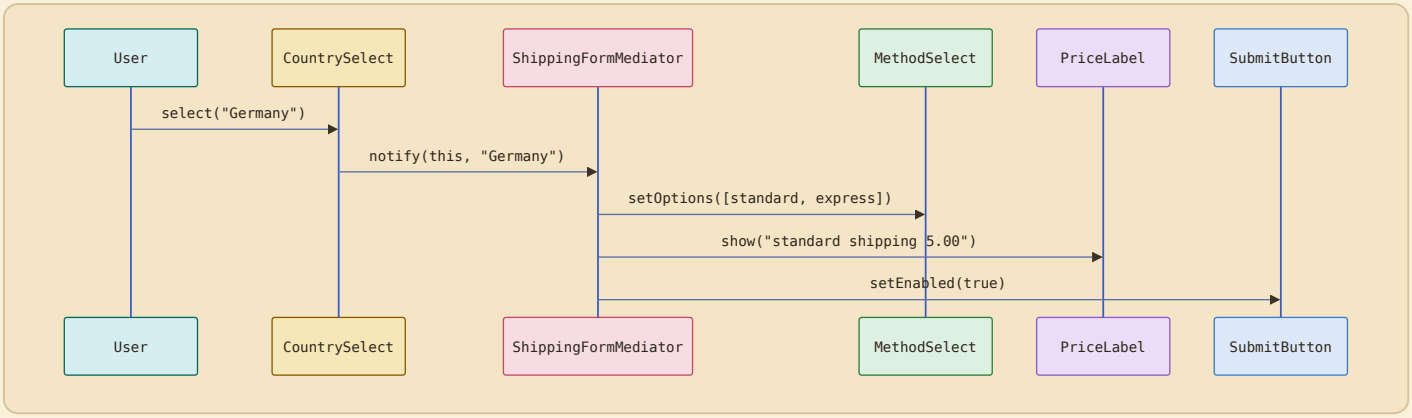
A checkout form: **country**, **delivery method**, **price** label, **submit** button — country refreshes methods, method changes price, price decides submit. Wired directly, four widgets need **six** links, five need **ten** (n(n−1)/2); a new field edits four tested classes; no widget is reusable alone. With a mediator: **one link per widget**, and widgets are reusable.

The honest trade: dependencies aren't removed — **they are relocated** into the mediator, which tends to grow.

How it works

Each **component** holds only the **Mediator** and calls `mediator.notify(this, event)`; the **concrete mediator** holds all components and the rules ("country changed → reload methods, then price, then submit") and calls **down** into them — traffic is **two-way** through one hub.

Structure (UML)



Java example

`lld.ch10.mediator` has three mediators: the **shipping form** (senders identified by reference, rates in `BigDecimal`); a **chat room** (`ChatRoom` routes broadcasts and private messages and applies moderation); **air traffic control** (`ControlTower` owns the runway and holding queue).

```
// File: code/lld/ch10/mediator/Mediator.java
public interface Mediator {
    void notify(Component sender, String event);
}

// File: code/lld/ch10/mediator/CountrySelect.java
    void select(String country) {
        selected = country;
        System.out.println("  [country] user picked " + country);
        mediator.notify(this, country);          // report up; never call a sibling
    }

// File: code/lld/ch10/mediator/ShippingFormMediator.java
    @Override
    public void notify(Component sender, String event) {
        if (sender == country) {
            Map<String, BigDecimal> available = RATES.getOrDefault(event, Map.of());
            methods.setOptions(new ArrayList<>(available.keySet()));
            if (available.isEmpty()) {
                price.show("shipping not available");
                submit.setEnabled(false);
            } else {
                showPrice();                             // ORDER matters: reload methods first, then price
                submit.setEnabled(true);
            }
        } else if (sender == methods) {
            showPrice();
        } else if (sender == submit) {
            System.out.println("  [mediator] order placed: " + country.selected() + " / " +
methods.selected());
        }
    }
```

In the output, picking "Germany" reloads methods, shows `standard shipping 5.00` and enables submit — three components updated and **no component called another**.

Full runnable code: code/lld/ch10/mediator/ (see the Full Edition).

Mediator or Observer?

	Mediator	Observer
What travels	An event **and** the decision on what follows	Only the event
Reaction logic	In one place: the mediator	Spread across observers
Parties know each other?	No	No; subject knows only a subscriber list
Flow	Two-way	One-way
Order of reactions	Explicit in the mediator	Not part of the contract

Test: does something need **coordinating in sequence** ("refresh methods, *then* price, *then* submit")? Then Mediator; otherwise the lighter Observer. **With Observer the listeners decide; with Mediator the hub decides.**

Trade-offs

Pros	Cons
Links grow linearly, not quadratically.	Every new rule (and merge conflict) lands in the mediator.
All interaction rules in one testable class.	To learn why a widget changed, read the mediator.
Widgets reusable; new widget = one class + one rule.	Can become a **god object**.

Keep it small: one mediator per screen area/workflow; coordination only (pricing in its own classes); no widget-to-widget shortcuts; dispatch via an event→handler map.

When not to use it

Only two objects interact; interactions are few and stable; nothing needs coordinating (use Observer); the mediator is already a god object (split it first).

Common mistakes

1. Components calling each other "just this once".
2. Computation inside the mediator.
3. Confusing with **Facade**: Facade sits *in front of* a subsystem that doesn't know it (one-way); Mediator sits *among* peers that know it (two-way).
4. Stringly-typed events — use sender reference, an enum or typed events.

Real-world examples

UI controllers/view-models (GoF's dialog box); chat servers; **saga orchestrators** (Temporal, AWS Step Functions), ride-hailing dispatch; Redux store, .NET MediatR; message brokers. `java.util.Timer` / `Executor` are often cited as JDK mediators — a *loose* fit.

In the interview

> **30-second answer:** "Mediator swaps peer-to-peer references for a single hub. Widgets report events; the hub owns the rules and instructs particular widgets. Four widgets go from six links to four; a fifth gives five instead of ten. The dependencies aren't gone — they're concentrated in one class I can read and test. The risk is a god object, so I split mediators per screen area or workflow and push computation into its own classes."

- *vs Facade?* Facade: one-way, subsystem unaware. Mediator: two-way among peers.
- *Microservices?* Orchestration is mediator-style; choreography is observer-style.

Self-check

1. **Q:** Links for 5 fully connected components vs with a mediator? **A:** 10 vs 5.
2. **Q:** Does a mediator remove coupling? **A:** No, it moves it into one class.
3. **Q:** Main failure mode? **A:** God-object mediator.

TL;DR

Peers talk through **one coordinator** holding the interaction rules. Links become linear and components reusable, but coupling **moves** into the mediator — keep it small and split it early.

10.5 Memento

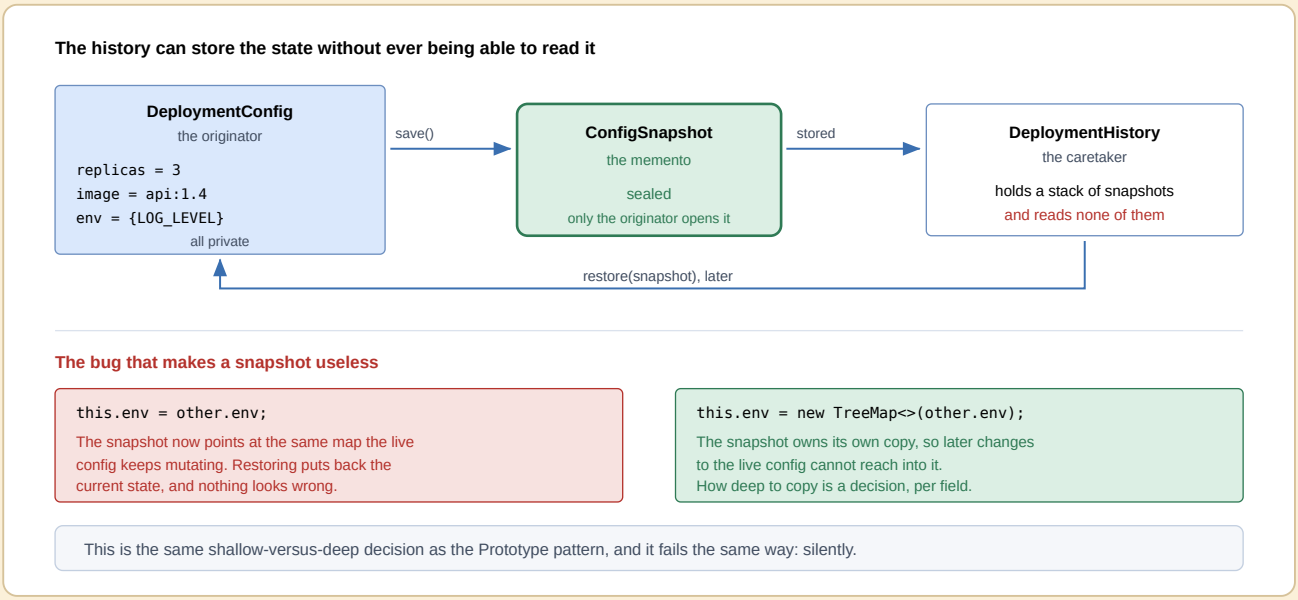


Figure — The originator (DeploymentConfig) saves a sealed snapshot; the caretaker (DeploymentHistory) stores it without reading it; below, the shared-map bug (course diagram)

What it is

Memento lets an object record its internal state in a **snapshot** and later **return** to it — without anyone else **seeing** inside.

GoF intent: *without violating encapsulation, capture and externalize an object's internal state so it can be restored later.* Participants: **Memento** (narrow interface to caretaker, wide to originator), **Originator** (creates and uses mementos), **Caretaker** (keeps mementos, never examines them). Analogy: a **save game** — the menu lists saves but never reads their contents.

The problem it solves

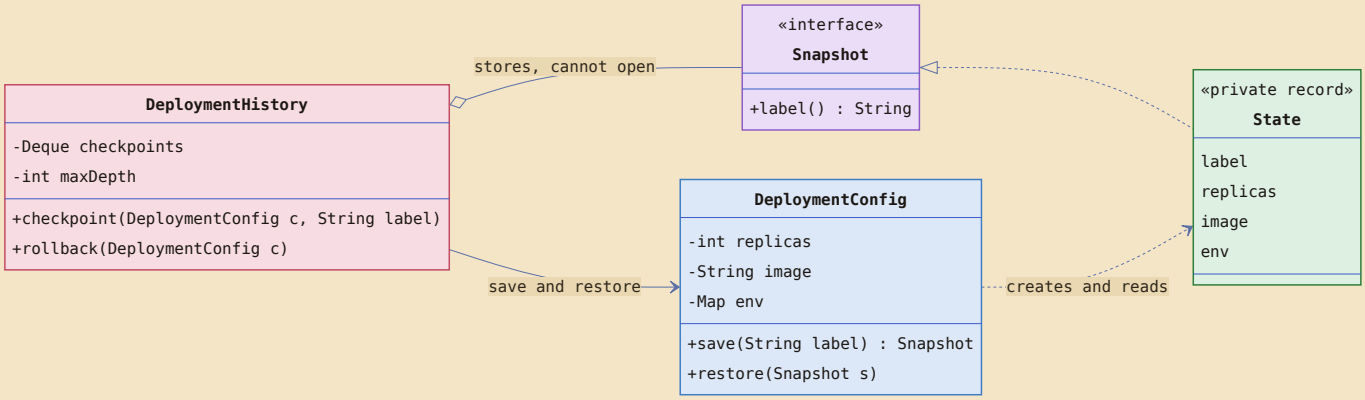
Rolling back a deployment config after a risky change. If an outside class copies every field, **encapsulation** breaks (it must know every field and change with each new one) and **atomicity** breaks (a field-by-field restore can stop half-way). With Memento the object **snapshots itself** into a sealed token **only it can open**, and restore is **one call** it controls.

How it works

Role	Job	In our code
Originator	Owns state; `save` and `restore`.	`DeploymentConfig`
Memento	Carries state; opaque to others.	`Snapshot` (narrow) / private `State` record (wide)
Caretaker	Decides **when** to save/restore; never looks **inside**.	`DeploymentHistory`

Enforcing opacity in Java (GoF narrow vs wide interface): public `Snapshot` exposes only `label()`; the data is a **private nested record** `State`; `restore` checks `instanceof State`, so a forged snapshot is refused. (Reflection can still peek.) C++ uses `friend`; Python/JS rely on convention.

Structure (UML)



Java example

`State`'s compact constructor does `env = Map.copyOf(env)`, so the snapshot **owns its copy** (changing the live map after saving doesn't affect it). `DeploymentHistory` is **bounded** (`maxDepth`) and drops the oldest.

```
// File: code/lld/ch10/memento/DeploymentConfig.java
public final class DeploymentConfig {

    public interface Snapshot {
        String label();
    }

    private record State(String label, int replicas, String image, Map<String, String> env)
            implements Snapshot {
        State {
            env = Map.copyOf(env);   // DEEP enough: the snapshot owns its own copy, not the live map
        }
    }

    private int replicas;
    private String image;
    private final Map<String, String> env = new TreeMap<>();
    // ... constructor, apply(...), setEnv(...)

    public Snapshot save(String label) {
        return new State(label, replicas, image, env);
    }

    public void restore(Snapshot snapshot) {
        if (!(snapshot instanceof State s)) {
            throw new IllegalArgumentException("not a snapshot produced by DeploymentConfig");
        }
        apply(s.replicas(), s.image(), s.env());
    }
}
```

Output (excerpt):

```
  live: replicas=3 image=api:1.4 env={LOG_LEVEL=info}   <- snapshot was a real copy
  bad snapshot  = {LOG_LEVEL=debug}  <- changed with the live object
```

Full runnable code: code/lld/ch10/memento/ (see the Full Edition).

The bug that makes a snapshot useless

If the memento keeps a **reference to the live mutable map**, the snapshot drifts with the object and restore silently does nothing. Copy mutable parts (`Map.copyOf`, deep copies — the Prototype decision, Ch 8). Also

bound history depth and don't snapshot **uncapturable state** (sockets, files, other systems).

Memento or Command undo?

	Memento	Command undo
Saved per step	Full copy of state	The operation + minimal state for reversal
Undo by	Re-installing a copy	Running the inverse
Memory per step	∝ state size	∝ one operation
Correctness risk	Low	Each inverse must be right
Prefer when	Reversal is tricky or state small	Reversal simple or state big

Drawing app (huge canvas) → Command; wizard step changing six interacting fields → Memento. **Real systems combine them:** occasional snapshots + a command log; restore nearest snapshot, replay forward (databases, event sourcing).

Trade-offs

Pros	Cons
Rollback with internals private; new fields need no caretaker change.	Memory grows with each snapshot.
Originator controls restore: all-or-nothing, to a real state.	Mutable parts must be deep-copied.
One mechanism for checkpoints, wizards, saves.	Some state can't be captured; some languages hide only by convention.

When not to use it

Large, frequently changing state (use Command or snapshots + replay); uncapturable state; **immutable objects** (an old instance already is a snapshot); operations with easy exact inverses.

Common mistakes

1. Referencing mutable internals (shallow copy). 2. Caretaker reading/editing fields. 3. Restoring field by field from outside. 4. Unbounded history. 5. Snapshotting connections or locks.

Real-world examples

DB `ROLLBACK` (undo logs); Git commits + `checkout`; **Kubernetes** keeps old ReplicaSets (`revisionHistoryLimit`, default 10) for `kubectl rollout undo`; VM snapshots; game saves, wizard "back". Python `copy.deepcopy`, JS `structuredClone` make quick mementos.

In the interview

30-second answer: "With Memento an object packs its state into a token only it can unpack. The originator makes and consumes snapshots, the memento holds the data, and the caretaker chooses when to save or roll back without inspecting contents. Restore is one call the originator controls, so it's all-or-nothing, and new fields never touch the history code. I'd deep-copy mutable parts, bound the history, and compare with Command undo: Memento stores state, Command stores actions; real systems combine snapshots with a command log."

- *Undo for a large document?* Command per edit + periodic snapshots.

Self-check

1. **Q:** Three roles? **A:** Originator, memento, caretaker.
2. **Q:** Who may read a memento? **A:** Only the originator.
3. **Q:** Why copy `env`? **A:** Otherwise the snapshot shares and changes with the live map.

TL;DR

The object **snapshots itself**; the caretaker **stores but can't read**; restore is **one originator-controlled call**. **Deep-copy**, **bound** history, and pick Memento (state) vs Command (actions) by which is smaller.

10.6 Interpreter

(Covered in the course only by its one-line table entry; this section is from GoF and standard practice.)

What it is

Interpreter represents a small language's **grammar** as classes — **one class per rule** — and evaluates a sentence by building a **tree** of these objects and calling `interpret` on the root. Analogy: a calculator reading `2 + 3 * 4` as `2 + (3 * 4)`.

Key terms: **grammar** (rules of valid sentences); **DSL** (small domain-specific language); **AST** (abstract syntax tree); **terminal** (leaf: number, variable) vs **nonterminal** expression (built from others); **context** (e.g. variable values).

The problem it solves

Rules like "age > 18 AND country = IN" keep appearing; hard-coding each means a redeploy. A tiny interpreted language turns rules into **data** — Interpreter is the simplest OO way to evaluate a **small** grammar.

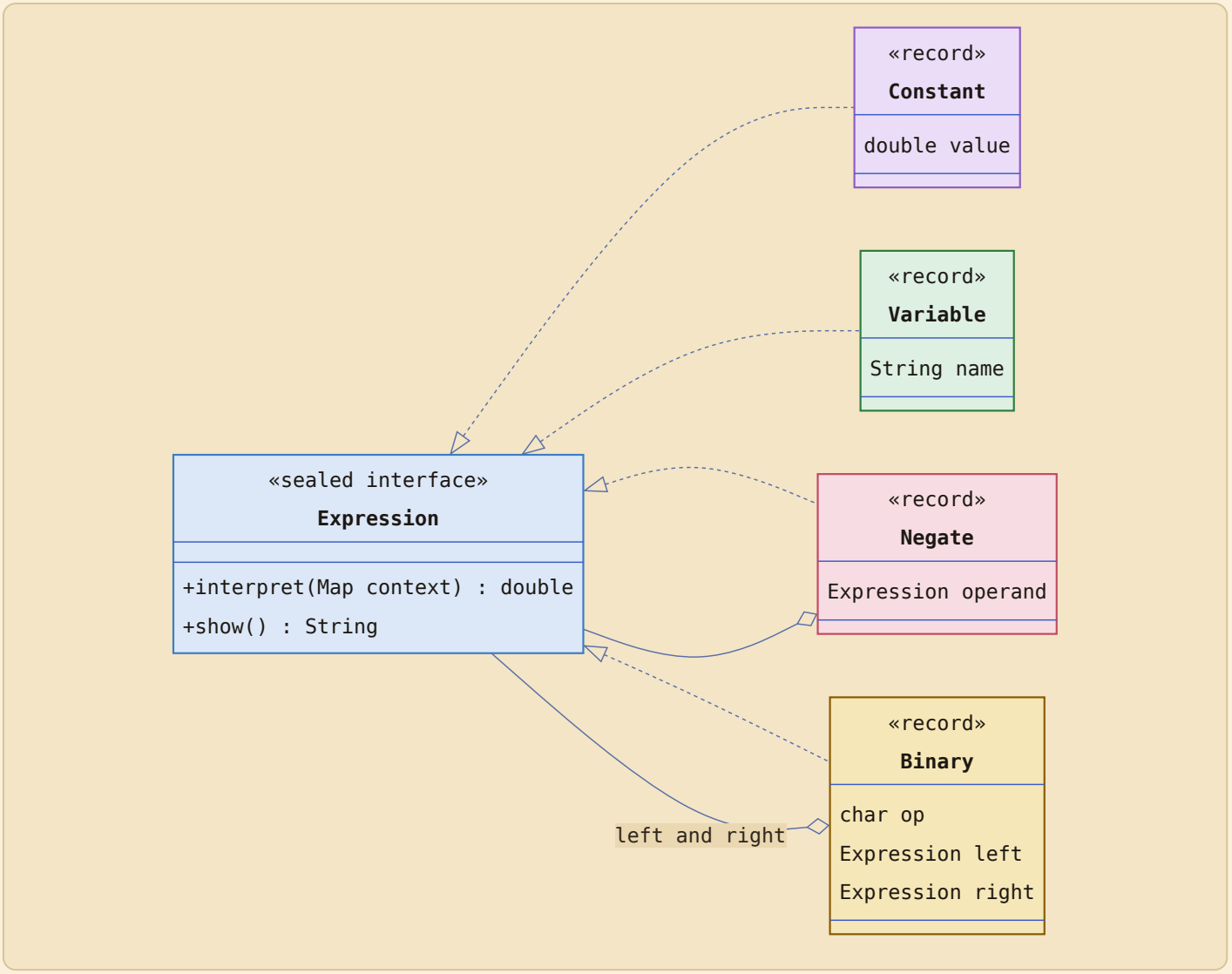
How it works

1. **Write the grammar** (EBNF):

   ```
   expr   := term   (('+' | '-') term)*
   term   := factor (('*' | '/') factor)*
   factor := NUMBER | NAME | '(' expr ')' | '-' factor
   ```

2. **One class per rule**: `Constant`, `Variable` (terminals), `Negate`, `Binary` (nonterminals), all implementing `Expression.interpret(context)`.
3. **Build the AST** — GoF leaves **parsing** out; we use a small *recursive-descent* parser (one method per rule).
4. **Interpret** recursively from the root.
5. **Parse once, interpret many times** with different contexts.

Structure (UML)



The structure is a **Composite** (Ch 9): Interpreter ≈ "Composite + `interpret`". The AST for `price * qty - discount` is `Binary -` with children `Binary *` (price, qty) and `Variable discount`.

Java example

`Expression` is a **sealed interface** permitting four records (one per rule); `Parser`'s `expr`, `term`, `factor` mirror the grammar. Errors (unknown variable, division by zero, bad syntax) report clearly.

> **Why `double`?** This is a teaching calculator; a real pricing DSL must use `BigDecimal` with an explicit `RoundingMode`.

```java
// File: code/lld/ch10/interpreter/Expression.java
public sealed interface Expression
        permits Expression.Constant, Expression.Variable, Expression.Negate, Expression.Binary {

    double interpret(Map<String, Double> context);
    // ... show()

    record Binary(char op, Expression left, Expression right) implements Expression {
        public double interpret(Map<String, Double> ctx) {
            double l = left.interpret(ctx), r = right.interpret(ctx);
            return switch (op) {
                case '+' -> l + r;
                case '-' -> l - r;
                case '*' -> l * r;
                case '/' -> {
                    if (r == 0) throw new ArithmeticException("division by zero in " + show());
                    yield l / r;
                }
                default -> throw new IllegalStateException("unknown operator " + op);
            };
        }
        // ... show()
    }
    // ... Constant, Variable (looks up the context; unknown name -> error), Negate
}

// File: code/lld/ch10/interpreter/Parser.java
    private Expression expr() {
        Expression left = term();
        while (peek('+') || peek('-')) {
            char op = src.charAt(pos++);
            left = new Expression.Binary(op, left, term());   // left-associative
        }
        return left;
    }
    // ... term() is the same loop over '*' '/' calling factor()
    // ... factor(): '-' factor | '(' expr ')' | NUMBER | NAME
```

Output (excerpt):

```
  price * qty - discount   AST ((price * qty) - discount)       = 700.0
  2 + 3 * 4                AST (2 + (3 * 4))                    = 14.0
  (2 + 3) * 4              AST ((2 + 3) * 4)                    = 20.0
  price * tax              ERROR: unknown variable 'tax'
  2 + * 3                  ERROR: unexpected '*' at position 4 in "2 + * 3"
```

Precedence comes from the **grammar**: `term` handles `*`/`/` below `expr`, so they bind tighter.

Full runnable code: code/lld/ch10/interpreter/ (see the Full Edition).

Trade-offs

Pros	Cons
New rule = new class; each rule small and testable.	Large grammars → class jungle; tree-walking is slow.
Rules become data; parse once, evaluate many.	New *operations* touch every class → **Visitor** (Ch 11); you still need a parser.

When not to use it

Large grammars (use **ANTLR** + visitor design); performance-critical evaluation (compile it); an existing language fits (regex, SpEL, JSONPath, CEL); **security** — never let users run a general-purpose script engine;

a restricted DSL is safer.

Common mistakes

1. Mixing parsing and evaluation (re-parse each time). 2. Ignoring precedence/associativity (`8 - 3 - 2` = `(8 - 3) - 2`). 3. Error messages without position. 4. Piling many operations on nodes instead of Visitor/pattern matching. 5. Growing a "tiny" DSL into an accidental programming language.

Real-world examples

`java.util.regex.Pattern`; **SpEL** (`SpelNode` AST, `getValue()`); SQL `WHERE` trees, spreadsheet formulas; promotion and feature-flag rule engines, Google **CEL**; cron expressions.

In the interview

> **30-second answer:** "Interpreter models a small language's grammar as classes, one per rule — terminals like numbers and variables, nonterminals like addition. A sentence is parsed into an AST and evaluated by calling `interpret` on the root with a context. It's Composite plus an evaluate operation. Great for small DSLs; for big grammars I'd use ANTLR, and for many operations over the tree I'd switch to Visitor or sealed types with pattern matching."

- *vs Visitor?* Interpreter puts evaluation **in** the nodes; Visitor moves operations **out**.

Self-check

1. **Q:** Terminal vs nonterminal? **A:** Leaves (numbers, variables) vs combinations (operators).
2. **Q:** What is the context? **A:** External info for evaluation, e.g. variable values.
3. **Q:** Why is `2 + 3 * 4` = 14? **A:** `*` lives in `term`, below `expr`, so it binds tighter.

TL;DR

One class per grammar rule; parse to an **AST**; evaluate recursively with a **context**. Perfect for **tiny DSLs**; big grammars need a parser generator and Visitor.

10.7 Chapter recap: the six patterns side by side

Pattern	Object you create	Question it answers	JDK / real example	Often confused with
Chain of Responsibility	A handler per check	*Who* handles this request?	Servlet filters, logging handlers	Decorator
Command	An object per request	How do I store, queue or undo it?	`Runnable`, `UndoManager`	Strategy
Iterator	A cursor object	How do I walk this without knowing its shape?	`Iterator`, `Spliterator`, `ResultSet`	Visitor
Mediator	One coordinator	How do many objects cooperate without knowing each other?	UI controllers, chat servers, saga orchestrators	Observer, Facade
Memento	A sealed snapshot	How do I safely go back?	DB rollback, Git, `kubectl rollout undo`	Command undo, Prototype
Interpreter	A node per grammar rule	How do I evaluate a small language?	`Pattern`, SpEL	Composite, Visitor

Combo facts: the request in a **CoR** is often a **Command**; a **Mediator** often dispatches **Commands**; **Memento + Command** give scalable undo; **Iterator + Composite** walk trees; **Interpreter = Composite + evaluate**, and when operations multiply move to **Visitor**.

Self-check (whole chapter)

1. **Q:** Which two GoF behavioral patterns are class-scoped? **A:** Template Method and Interpreter.
2. **Q:** Auth → rate limit → validation, any step may reject? **A:** Chain of Responsibility.
3. **Q:** Undo/redo for a text editor? **A:** Command per edit, optionally Memento snapshots.
4. **Q:** Five form widgets depend on each other? **A:** Mediator.
5. **Q:** Business users write discount rules? **A:** Interpreter (small DSL), possibly with Specification (Ch 11).

TL;DR

Behavioral patterns turn **hidden control flow** into **named objects**: handlers, requests, cursors, coordinators, snapshots, grammar nodes. Each buys flexibility with indirection — use it when the flexibility is a real requirement.

Chapter 11 — Behavioral Design Patterns (Part 2) & Beyond GoF · Study Edition

> **Study edition.** Every topic of the full chapter, with the padding removed. For complete code listings, full outputs and extra follow-ups, see the Full Edition (`README.md`).
>
> **What you will learn.** The remaining five GoF behavioral patterns — **Observer, State, Strategy, Template Method, Visitor** — and how to tell apart the three that share one diagram (Strategy, State, Bridge). Then six non-GoF patterns interviews expect: **Null Object, Dependency Injection, Repository, Specification, MVC** and **Event Sourcing (lite)**, ending with a "which pattern do I need?" table.

How to run the code

```
cd chapters/11-patterns-behavioral-2/code
javac --release 21 -Xlint:all -d /tmp/lld_11 $(find . -name '*.java')   # compiles with 0 warnings
java -cp /tmp/lld_11 lld.ch11.observer.Main
# also: state, strategy, templatemethod, visitor, nullobject, di, repository, specification, mvc,
eventsourcing
```

The idea running through this chapter: most of these patterns turn a conditional (`if` / `switch`) or a hard-coded call into an **object** that can be swapped, subscribed, or visited.

11.1 Observer

What it is

Observer lets one object (the **subject** / publisher) notify many dependants (the **observers** / subscribers / listeners) when something happens — **without knowing who they are or what they do**. Analogy: subscribers to a YouTube channel get notified of each new video; the channel doesn't care what they do with it.

GoF intent: a one-to-many dependency so that when one object changes, all its dependants are notified and update automatically.

Participant	Role	In `lld.ch11.observer`
Subject	Knows its observers; attach/detach	`BuildPipeline` (`addListener` returns an unsubscribe handle)
Observer	The update interface	`BuildListener`
ConcreteSubject	Holds the state; notifies on change	`BuildPipeline` (one class plays both in modern Java)
ConcreteObserver	Implements the update	`SlackNotifier`, `DashboardUpdater`, `MetricsRecorder`, `DeployTrigger`

The problem it solves

A build pipeline calling Slack, dashboard, metrics and deployer directly depends on a chat service, must be edited for each new reaction, needs four fakes to test, and needs flags to disable deploy in staging. Observer replaces four references with **one list of one interface**. **The source only announces; every listener chooses its own reaction.**

How it works

An **observer interface** with one method; the **subject** keeps a list with subscribe/unsubscribe; on change it **finishes updating its state first**, then calls each observer; observers come and go at run time.

Push vs pull:

	Push	Pull
Observer receives	All the data (e.g. a `Reading`)	"I changed" + a subject reference
Pros	Simple; no dependence on subject getters	Takes only what it needs; interface stable as data grows
Cons	Sends unneeded data; interface grows with data	Couples observers to the subject; must work out *what* changed

Structure (UML)

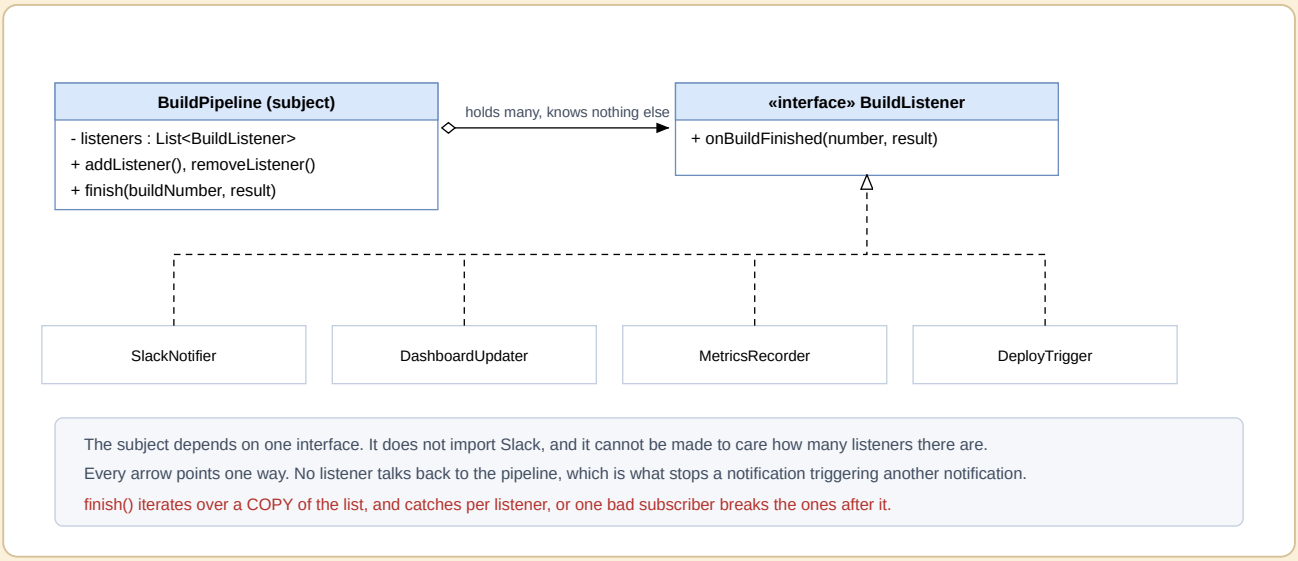
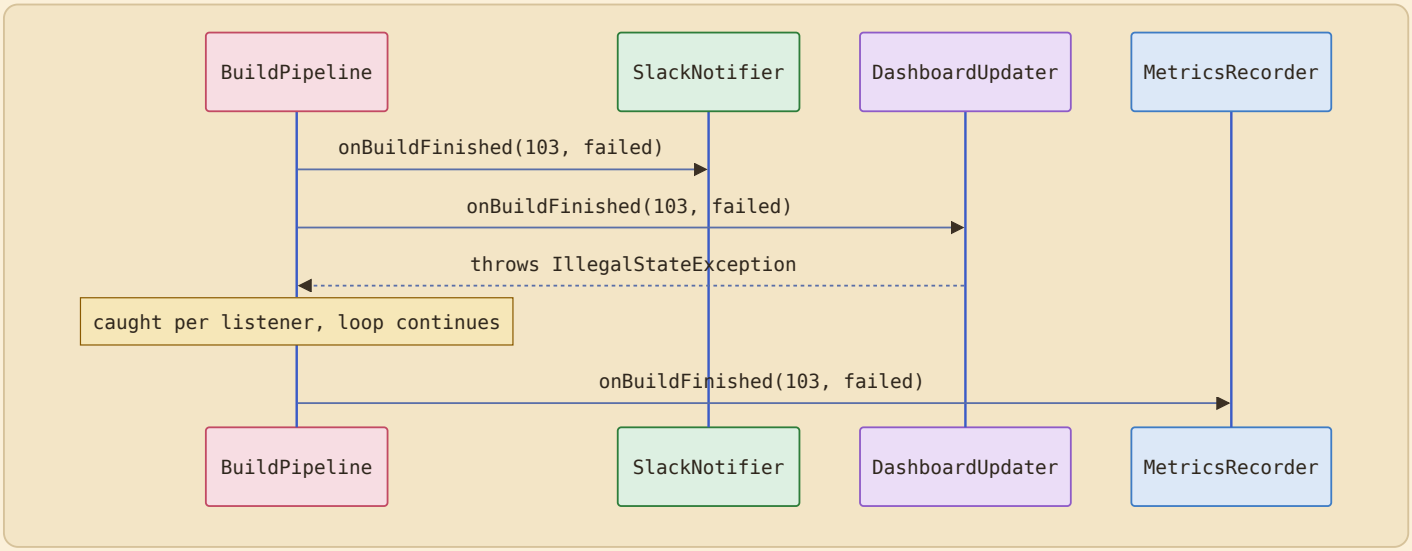


Figure — BuildPipeline holds a list of BuildListener; four concrete listeners implement it (course diagram)

Dependencies run one way: listeners never call the pipeline back.



Java example

`lld.ch11.observer` shows the **`build pipeline`** (a `CopyOnWriteArrayList` iterates a snapshot, so unsubscribing mid-loop is safe; exceptions caught per listener; `addListener` returns an unsubscribe handle), **`push vs pull`** (`WeatherStation`), a tiny typed **`EventBus`** (synchronous, stores nothing — not a queue), and JDK **`PropertyChangeSupport`**, which doesn't fire when old and new values are equal.

```java
// File: code/lld/ch11/observer/BuildListener.java
@FunctionalInterface
public interface BuildListener {
    void onBuildFinished(int buildNumber, String result);
}

// File: code/lld/ch11/observer/BuildPipeline.java
public final class BuildPipeline {
    // CopyOnWriteArrayList iterates over a snapshot: a listener that unsubscribes
    // itself mid-notification cannot break the loop, and it is safe across threads.
    private final List<BuildListener> listeners = new CopyOnWriteArrayList<>();

    /** Returns a handle: whoever subscribes owns the matching unsubscribe (prevents leaks). */
    public Runnable addListener(BuildListener listener) {
        listeners.add(listener);
        return () -> listeners.remove(listener);
    }

    public void finish(int buildNumber, String result) {
        System.out.println("Build " + buildNumber + " finished: " + result);
        for (BuildListener l : listeners) {
            try {
                l.onBuildFinished(buildNumber, result);
            } catch (RuntimeException e) {          // one bad listener must not starve the rest
                System.out.println("  [pipeline] a listener failed: " + e.getMessage());
            }
        }
    }
}
```

In the output, build 103's dashboard listener throws (`a listener failed: dashboard is offline`) and metrics still records it.

Full runnable code: code/lld/ch11/observer/ (see the Full Edition).

Why `java.util.Observer` / `Observable` were deprecated

Deprecated since **Java 9**: limited event model, **unspecified order**, notifications not one-for-one with changes; plus `Observable` is a **class** (uses up inheritance), `setChanged()` is protected, `update` is untyped, thread safety is partial. Use instead: `java.beans.PropertyChangeSupport`, your own typed listener interface, `java.util.concurrent` queues, or **`java.util.concurrent.Flow`** (reactive streams with back-pressure).

Trade-offs

Pros	Cons
Source doesn't depend on reactions; add/remove at run time.	Hard to trace; **memory leaks** if never unsubscribed.
Each reaction testable (Open/Closed).	**No ordering/delivery guarantee**; as slow as all listeners combined.

Observer vs Mediator, Chain of Responsibility, callbacks

Looks like	The actual difference
Mediator	Observer broadcasts one way and each listener decides; a mediator decides what happens next, calls components back (two-way) and owns ordering.
Pub-sub with a broker	Same idea plus persistence, redelivery, ordering and a network hop.
Chain of Responsibility	Observer informs **all**; a chain stops at the first handler that accepts.
A callback	One callback is one observer; it becomes the pattern when listeners vary at run time.

Outside Java (JS `EventEmitter`, Python callables, C# `event`), **weak references** (e.g. `weakref.WeakSet`) fix leaks at the cost of listeners vanishing when unreferenced.

When not to use it

One listener forever (call it); you need ordering or guaranteed delivery (use Kafka/RabbitMQ/SQS); slow reactions (notify asynchronously); the flow must be easily traceable.

Common mistakes (the four classic ways observers go wrong, plus one)

1. **No unsubscribe → memory leak** — every `add` needs an owner for its `remove`.
2. **A throwing listener kills the loop** — catch per listener.
3. **Changing the list during notification** → `ConcurrentModificationException`; iterate a copy.
4. **Listeners calling back into the subject** → cycles/recursion.
5. **Notifying before state is updated** → pull observers read stale data.
6. **Relying on notification order** — if order matters, that's a Mediator.

Real-world examples

UI listeners (Swing, DOM `addEventListener`); JDK `PropertyChangeListener`, `Flow`; Spring `@EventListener`; Guava `EventBus`, RxJava/Reactor; MVC model→view; Kafka/SNS pub-sub at system scale; CDC (Debezium).

In the interview

> **30-second answer:** "Observer swaps hard-coded calls for a list of objects sharing one listener interface. The subject says 'this happened' and each observer picks its own response, so reactions plug in and out and are tested alone, and the subject depends on none of them. In production I iterate over a copy of the list, catch exceptions per listener, and pair every subscribe with an unsubscribe. When I need durability, retries or ordering I use a message broker."

- *Observer vs Pub/Sub?* In Observer the subject holds observers directly; in pub/sub a broker sits between, adding durability and retries.

Self-check

1. **Q:** What does the subject know about observers? **A:** Only that they implement the interface.
2. **Q:** Classic Observer leak? **A:** Never-removed listeners stay strongly referenced.
3. **Q:** What replaces `Observable`? **A:** `PropertyChangeSupport`, a typed listener interface, or `Flow`.

TL;DR

One subject, a list of listeners, one-way notification. **Iterate a copy, catch per listener, always unsubscribe.** `Observable` is deprecated; use a broker when you need guarantees.

11.2 State

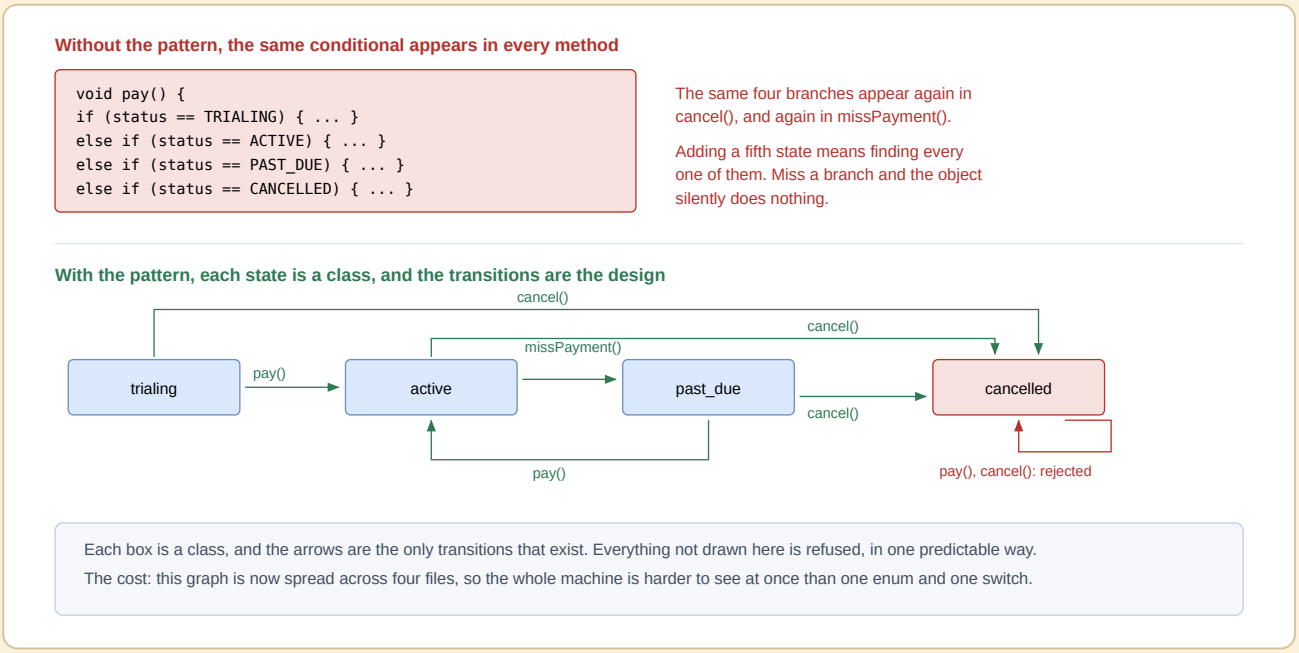


Figure — Top: the same four-branch status check in every method. Bottom: each state is a class; transitions form a graph (course diagram)

What it is

State (GoF): when an object's internal condition changes, **its behaviour changes**, as if it changed class. Instead of every method testing a status field, the object forwards each action to a **state object** and swaps it when the condition changes. Analogy: a phone's ring/vibrate/silent mode.

Participant	Role	In `lld.ch11.state`
Context	Holds current state; forwards requests	`Subscription`, `VendingMachine`
State	One method per forwarded request	`SubscriptionState`, `VendingState`
ConcreteState	Behaviour for one state	`Trialing`, `Active`, `PastDue`, `Cancelled`; `Idle`, `HasMoney`, `SoldOut`

GoF leaves open who performs transitions; letting states choose successors (as here) is flexible but makes states know each other.

The problem it solves

A subscription goes **trialing → active → past due → cancelled**; naively every method repeats the same four-way `if (status == ...)` chain — a fifth state means hunting every copy, and a **forgotten branch** silently does nothing. With State, **each state is a class**, context methods shrink to one line, and one state's rules live in one file.

How it works

A **State interface** has one method per action (every state answers every action, even "no"); the **context** delegates (`state.pay(this)`); each state rules on **legality** and the **successor** (`context.setState(new Active())`); terminal states refuse everything in one place.

States choose their own successors — the key difference from Strategy.

Structure (UML)

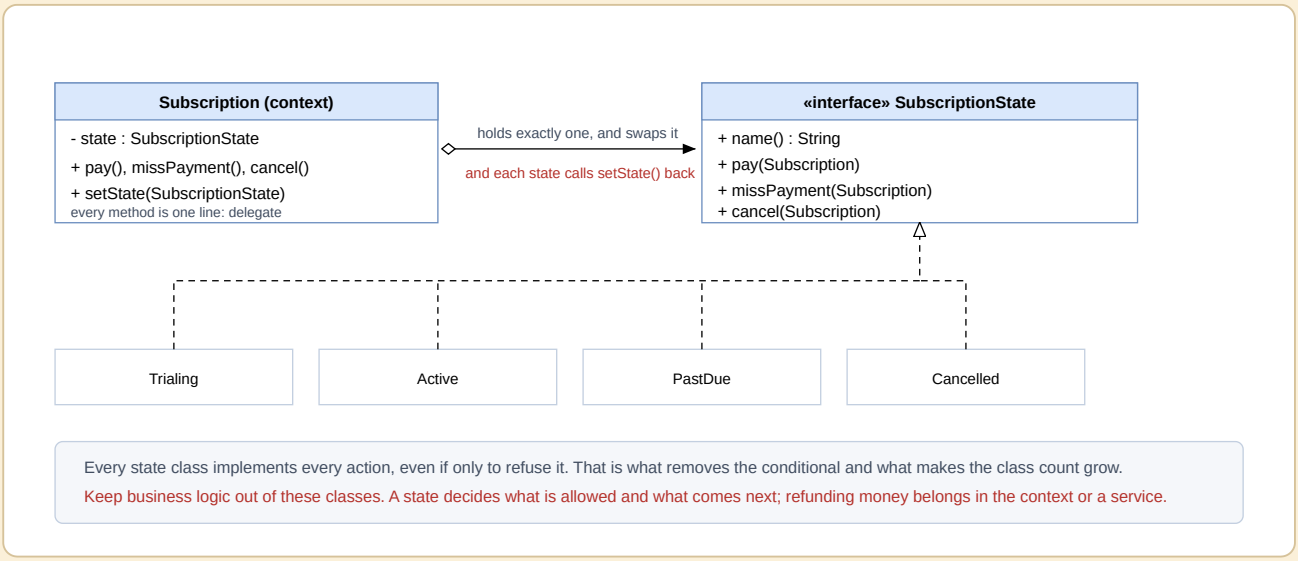
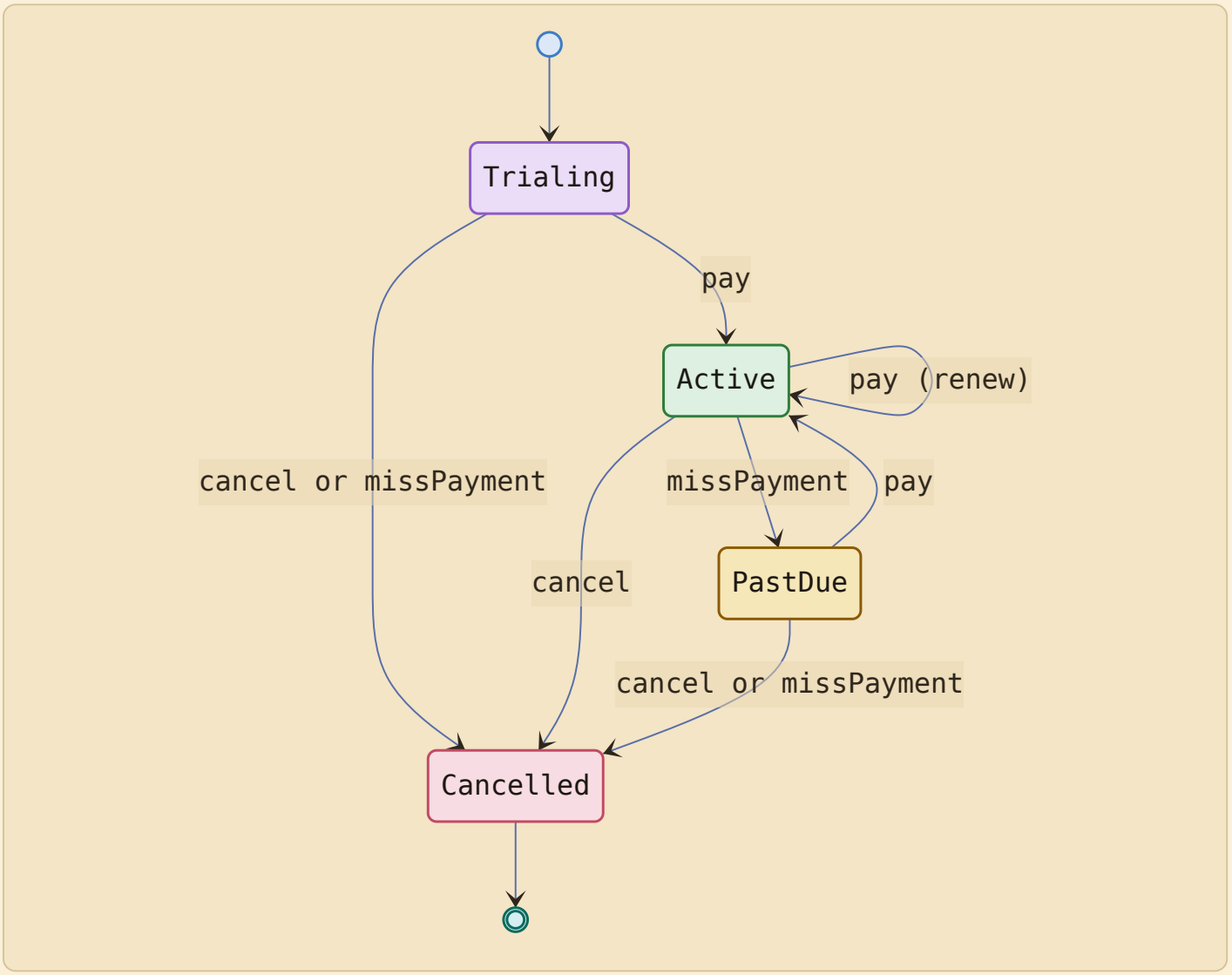


Figure — Subscription (context) holds a SubscriptionState; the four states call setState on the context (course diagram)



(The vending machine runs Idle → HasMoney → Idle or SoldOut; the traffic light RED → GREEN → YELLOW → RED.)

Java example

Three styles: (1) **class per state** — sealed interface, field-less `record` states; (2) **enum machine** (`TrafficLight`) — the idiomatic Java answer for small machines; (3) **vending machine** — **default methods that**

refuse, so each state overrides only what it allows (`SoldOut` overrides nothing), and business actions live in the **context**.

```java
// File: code/lld/ch11/state/SubscriptionState.java
public sealed interface SubscriptionState permits Trialing, Active, PastDue, Cancelled {
    String name();
    void pay(Subscription s);
    void missPayment(Subscription s);
    void cancel(Subscription s);
}

// File: code/lld/ch11/state/Trialing.java
public record Trialing() implements SubscriptionState {
    public String name() { return "trialing"; }
    public void pay(Subscription s) {
        System.out.println("  [trialing] first payment taken");
        s.setState(new Active());                    // the state chooses its successor
    }
    public void missPayment(Subscription s) {
        System.out.println("  [trialing] trial ended unpaid");
        s.setState(new Cancelled());
    }
    public void cancel(Subscription s) { s.setState(new Cancelled()); }
}

// File: code/lld/ch11/state/Subscription.java
public final class Subscription {
    private SubscriptionState state = new Trialing();

    void setState(SubscriptionState next) {
        System.out.println("    " + state.name() + " -> " + next.name());
        state = next;
    }
    // ... status()

    public void pay()         { state.pay(this); }
    public void missPayment() { state.missPayment(this); }
    public void cancel()      { state.cancel(this); }
}

// File: code/lld/ch11/state/TrafficLight.java — each constant overrides next()
    RED(30) {
        TrafficLight next() { return GREEN; }
        boolean pedestriansMayCross() { return true; }
    },
    // ... GREEN -> YELLOW, YELLOW -> RED
```

In the output, paying after cancellation prints `[cancelled] refused: no payments after cancellation` — an **illegal action** refused the same way every time, by code in one file.

Full runnable code: code/lld/ch11/state/ (see the Full Edition).

State vs Strategy (and Bridge) — same diagram, different reasons

	State	Strategy	Bridge
Who chooses the object	The **states** themselves	The **caller**	Once at construction/config
How often it changes	Repeatedly	Once, sometimes per call	Effectively never
Alternatives reference each other?	**Yes**	**No**	No
What varies	Current *condition* and allowed actions	*How* one job is done	A whole design dimension
Why it exists	Model a lifecycle	Make an algorithm replaceable	Stop two hierarchies multiplying

Naming test: do the alternatives name each other? Yes → State. No → Strategy (or Bridge, if the choice never changes).

Also: **an enum plus one** `switch` is often the better answer below three repeated switches; **state-machine libraries** (Spring Statemachine, XState) express it as data past about a dozen states.

Trade-offs

Pros	Cons
One state's rules in one file; no branch missed.	More classes; transitions spread over files.
Illegal actions rejected consistently.	Persisting needs one name↔class mapping.

When not to use it

Two states and few methods (a boolean); only data differs (a field); a big formal machine (a transition table `Map<(state,event),handler>` or Spring Statemachine). Rule of thumb: pays off once **three or more methods repeat the same status switch**. If you keep sketching the machine on paper to follow your code, make that sketch (a table) the source of truth.

Common mistakes

1. Successors hard-coded everywhere at 12+ states (move to a table).
2. Business logic in state classes (states rule on legality and successor only).
3. Illegal actions that silently do nothing.
4. Storing state objects in the DB — store a name/enum.
5. Sharing *stateful* state objects between contexts.

State in other languages

Java: enum or `sealed` + exhaustive `switch`. Kotlin/Scala: sealed + exhaustive `when`/`match`. Rust: enums and **typestate** (illegal calls don't compile). Python: a `(state, action)` → handler dict. TypeScript: discriminated unions.

Real-world examples

Order lifecycles; Stripe subscription statuses; **TCP connections** (GoF's example); approval workflows; vending machines, ATMs, elevators. (`Thread.State` is an enum of states, not the pattern.)

In the interview

> **30-second answer:** "State swaps a status field and its repeated switches for one class per state. The context forwards each action to its current state object, which decides if it's legal and which state follows. Each state's rules sit in one file and illegal actions are refused consistently. It draws like Strategy, but states name each other and advance the machine themselves, while strategies are picked from outside. For small Java machines I'd use an enum or a sealed interface."

- *Persist it?* Store the state name/enum; rebuild via one mapping.
- *Concurrency?* Racing "cancel" and "deliver": lock, or optimistic `UPDATE ... WHERE status = 'PLACED'` and check the row count.

Self-check

1. **Q:** Who changes the current state? **A:** The state objects, via the context's `setState`.
2. **Q:** The naming test? **A:** Alternatives reference each other → State; never → Strategy/Bridge.
3. **Q:** What belongs in the context, not the state? **A:** Business actions and data (dispensing, inventory).

TL;DR

A class per state; the context forwards; each state picks its successor. Same diagram as Strategy, but states **know each other**. Enums for small machines, tables/libraries for big ones.

11.3 Strategy

What it is

Strategy (GoF): define a **family** of algorithms, put each in its own class behind a **common interface**, and make them interchangeable, so the algorithm varies independently of the code using it — often chosen at run time. The **context** never branches on which algorithm it holds. Analogy: getting to the airport by car, train or taxi — same goal, different algorithm.

Participant	Role	In `lld.ch11.strategy`
Strategy	Interface all algorithms share	`RetryPolicy` (also `Comparator`)
ConcreteStrategy	One per algorithm	`FixedDelay`, `ExponentialBackoff`, the `RetryPolicy.none()` lambda (course: `NoRetry`)
Context	Configured with a strategy; delegates	`ApiClient`

The problem it solves

Retry = none, fixed delay, or exponential backoff. An `if (policy == ...)` chain puts backoff maths in the API client, needs edits per policy, and can't be tested alone. With Strategy, `ApiClient` asks "how long before the next attempt?" and **never checks which policy it holds**; the choice is made **outside** (config, feature toggle, customer setting).

How it works

A small **strategy interface** (one question: `delayBeforeAttempt(attempt)`); one **concrete strategy** per algorithm; the **context** receives one and calls it; a **factory, config or DI container** chooses — not every call site.

Structure (UML)

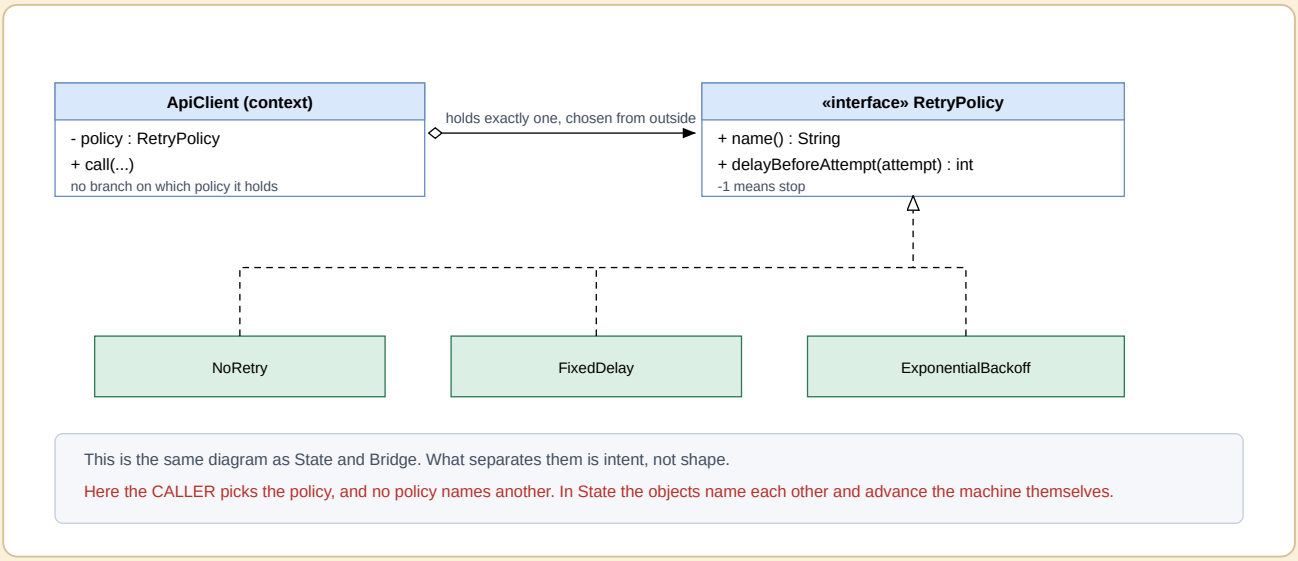
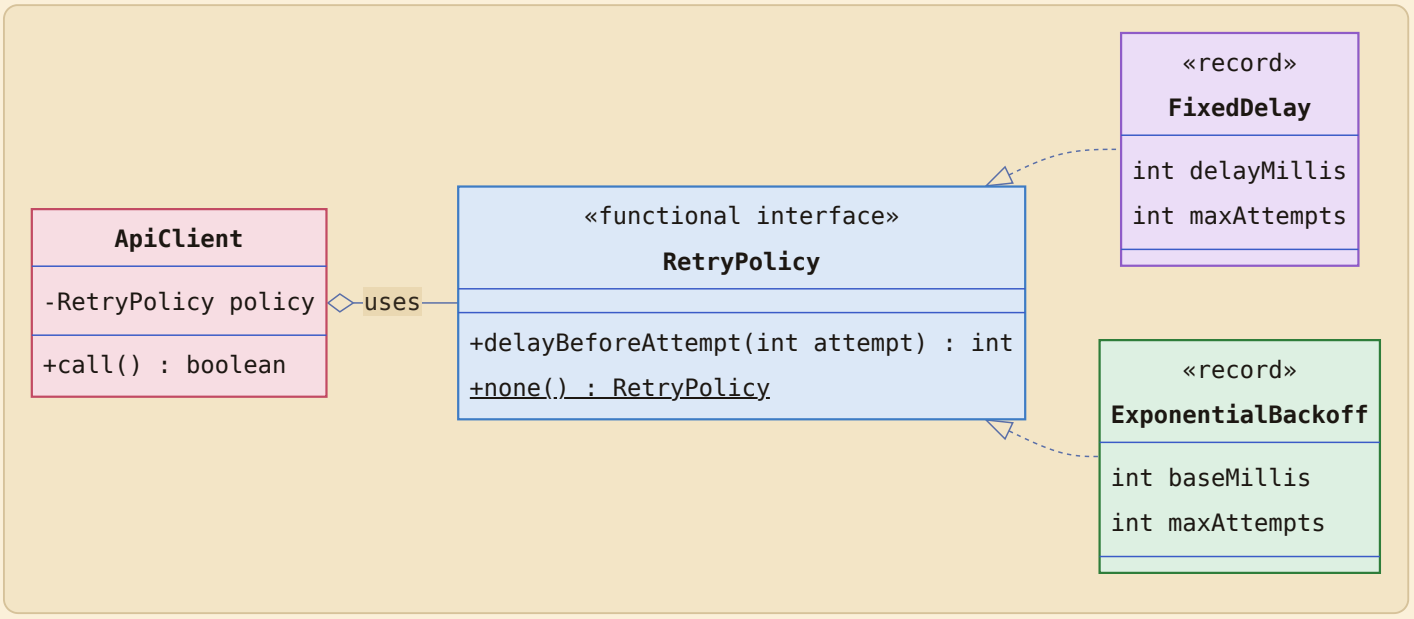


Figure — ApiClient holds a RetryPolicy; NoRetry, FixedDelay and ExponentialBackoff implement it (course diagram)

No strategy references another — that absence tells Strategy from State.



Java example

Retry policies (`none()` is a lambda; configured ones are **records**), `Comparator` composition (`comparing(...).reversed().thenComparing(...)`), and a **registry** `Map` of lambdas chosen by config key.

```java
// File: code/lld/ch11/strategy/RetryPolicy.java
@FunctionalInterface
public interface RetryPolicy {
    int STOP = -1;

    int delayBeforeAttempt(int attempt);

    static RetryPolicy none() { return attempt -> STOP; }
}

// File: code/lld/ch11/strategy/ExponentialBackoff.java
public record ExponentialBackoff(int baseMillis, int maxAttempts) implements RetryPolicy {
    public int delayBeforeAttempt(int attempt) {
        return attempt > maxAttempts ? STOP : baseMillis << (attempt - 2);
    }
}

// File: code/lld/ch11/strategy/ApiClient.java
    public boolean call() {
        for (int attempt = 1; ; attempt++) {
            if (server.test(attempt)) {
                System.out.println("  attempt " + attempt + ": ok");
                return true;
            }
            int delay = policy.delayBeforeAttempt(attempt + 1);
            if (delay == RetryPolicy.STOP) {
                System.out.println("  attempt " + attempt + ": failed, giving up");
                return false;
            }
            System.out.println("  attempt " + attempt + ": failed, waiting " + delay + "ms");  // simulated,
no sleep
        }
    }
```

Against a server that is always down, `ExponentialBackoff(100, 4)` waits 100, 200, 400 ms and then gives up. Adding jitter or `Retry-After` support is one more class or lambda; `ApiClient` doesn't change.

Full runnable code: code/lld/ch11/strategy/ (see the Full Edition).

Lambdas as strategies

Strategy is the pattern a plain function replaces most often. Any **functional interface** (one abstract method) — `Comparator`, `Function`, `Predicate`, your `RetryPolicy` — can take a lambda. Keep a named class/record when the strategy has **configuration or state**, needs a **name for logging**, or must be **discovered/registered** (Spring beans, `ServiceLoader`). Other languages: pass a function (Python dict of functions as a registry; Go one-method interface or `func` field). Money in strategies stays `BigDecimal`, never `double`.

Trade-offs

Pros	Cons
New variants without editing working code; each testable alone.	One class per variant; someone must choose.
Configurable at run time; context keeps to its job.	Interface must fit all variants; overused for one-liners.

When not to use it

Two branches with no third in sight (an `if`); one-line branches (a map of lambdas); variants needing different inputs (forced interface — rethink); a choice fixed for the program's life (decide at wiring, or Bridge).

Common mistakes

1. Every call site choosing — the branch moved instead of disappearing.
2. Bloated interface with parameters half the implementations ignore.
3. Strategies that know each other — that's State.
4. Stateful strategies shared between threads — keep them immutable.

Real-world examples

`Comparator`; `RejectedExecutionHandler` (`AbortPolicy`, `CallerRunsPolicy`, ...); Spring `PasswordEncoder`; Resilience4j/Polly retry intervals; AWT `LayoutManager`; payment methods, tier pricing, map route modes.

In the interview

> **30-second answer:** "Strategy gives every variant of one job its own class behind a common interface, and something outside — usually a factory or config — picks it at run time. The context stops branching: my API client asks its retry policy how long to wait and never which policy it is. Each algorithm is testable alone and new ones don't touch working code. In Java a lambda is often enough, like a `Comparator`; I keep a class when it has configuration. It shares its diagram with State, but strategies never reference each other."

- *vs Template Method?* Template Method swaps **one step** via **inheritance** (fixed at creation); Strategy swaps **the whole algorithm** via **composition** (run time, several axes). Prefer Strategy when both fit.
- *Selection?* Config + factory `Map<String, Strategy>`, DI (Spring can inject a `Map` of beans), or per request.

Self-check

1. **Q:** Who chooses the strategy? **A:** Something outside the context — factory/config/DI.
2. **Q:** Why can `RetryPolicy.none()` be a lambda? **A:** `RetryPolicy` is a functional interface.
3. **Q:** Two JDK strategies? **A:** `Comparator`, `RejectedExecutionHandler` (also `LayoutManager`).

TL;DR

Interchangeable algorithms behind one interface, chosen from outside. The context stops branching. Lambdas for one-liners, records when configured. Not State (states know each other), not Template Method (inheritance, one step).

11.4 Template Method

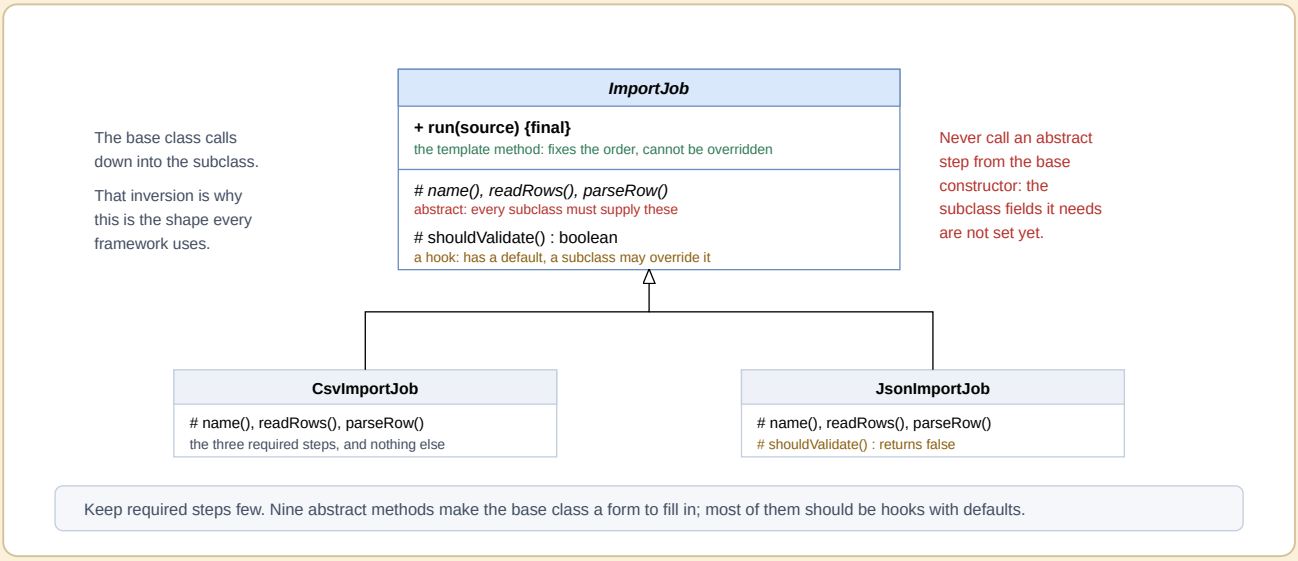


Figure — ImportJob with final run(), abstract steps, hook shouldValidate and private save; CsvImportJob and JsonImportJob override only what differs (course diagram)

What it is

Template Method (GoF): put an **algorithm's outline** in one base-class method (the *template method*) and let **subclasses** fill in certain steps without changing the sequence.

Participant	Role	In `lld.ch11.templatemethod`
AbstractClass	Declares the steps; implements the template method calling them in order	`ImportJob`, `CaffeineBeverage`
ConcreteClass	Implements the steps that differ	`CsvImportJob`, `JsonImportJob`, `Tea`, `Coffee`

Analogy: a recipe card with blanks — "boil water → brew ___ → pour → add ___ (if wanted)".

The problem it solves

An import job always reads, parses, validates, saves, reports; only reading and parsing differ for CSV vs JSON. Copying the flow per class means a change (say, an audit record) touches every copy. Template Method writes the flow once in a `final` method; varying steps are methods subclasses supply.

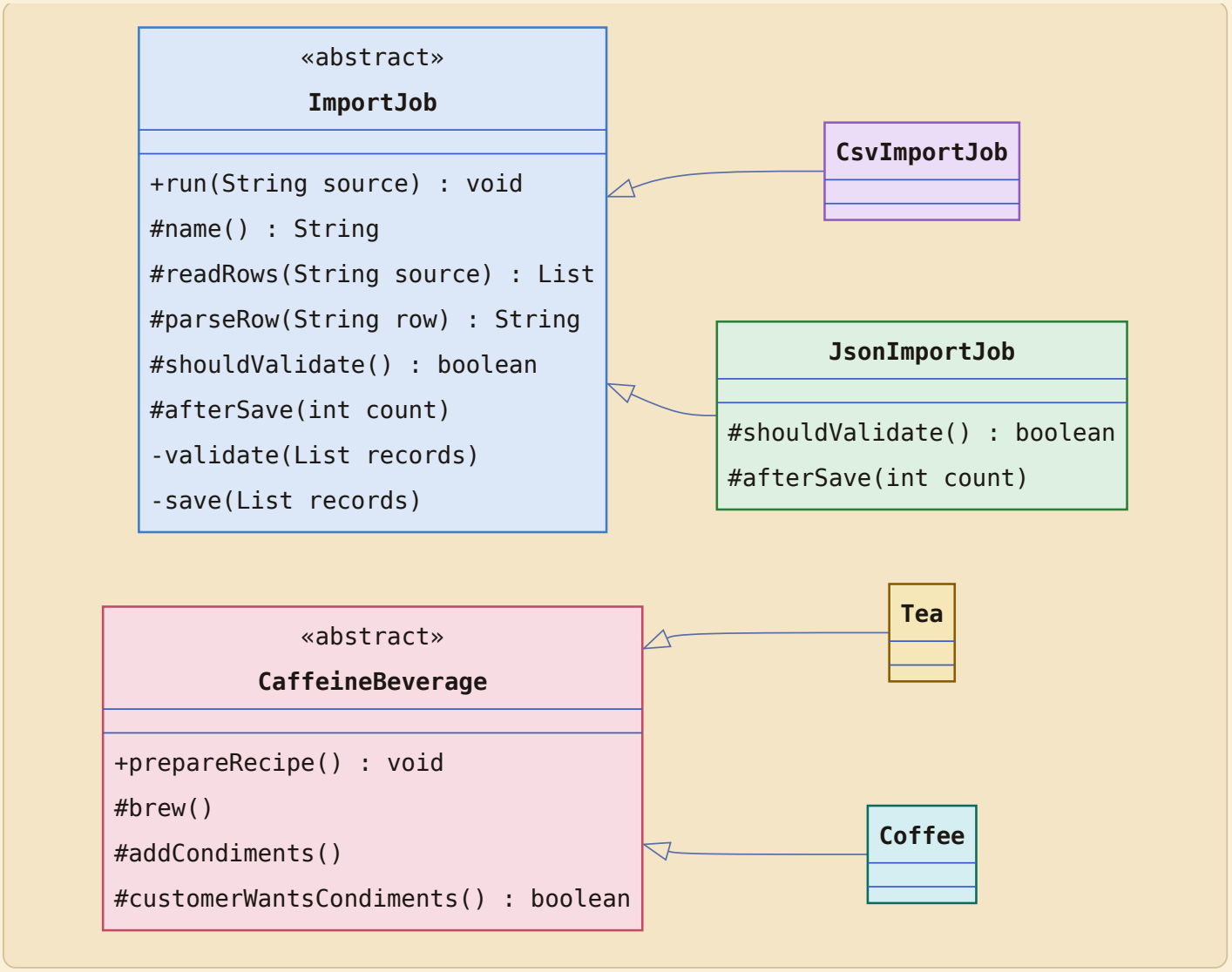
How it works

1. Write the **template method** in an abstract class, `final` so the order is guaranteed.
2. It calls three kinds of steps: **required (abstract)** — compiler-enforced; **hooks** — a default (often empty or `true`) subclasses *may* override, e.g. `shouldValidate()`; **shared** — `private`, unchangeable.
3. Each subclass implements only what differs.

Hollywood Principle — "Don't call us, we'll call you" (*Head First*): the base class decides **when** subclass methods run. This inversion of control is the backbone of frameworks — the framework runs the sequence, you supply pieces.

Structure (UML)

A subclass alone doesn't reveal the sequence it follows — only the parent does. That is the trade-off.



`run()` and `prepareRecipe()` are `final` in code (Mermaid has no marker for it).

Java example

Import jobs (`JsonImportJob` overrides both hooks) and *Head First*'s **beverages**, where `Coffee(black = true)` switches off condiments via a hook.

```java
// File: code/lld/ch11/templatemethod/ImportJob.java
public abstract class ImportJob {

    /** Template method. Marked final: subclasses cannot change the order, drop a step or insert one. */
    public final void run(String source) {
        System.out.println("Running the " + name() + " import:");
        List<String> rows = readRows(source);                // required step
        List<String> records = new ArrayList<>();
        for (String row : rows) records.add(parseRow(row));   // required step
        System.out.println("  parsed " + records.size() + " records: " + records);
        if (shouldValidate()) {                              // hook
            validate(records);
        } else {
            System.out.println("  skipping validation, source is validated upstream");
        }
        save(records);                                       // shared, fixed
        afterSave(records.size());                           // hook with an empty default
        System.out.println("  done");
    }

    // Required steps: the compiler forces every subclass to supply them.
    protected abstract String name();
    protected abstract List<String> readRows(String source);
    protected abstract String parseRow(String row);

    // Hooks: sensible defaults, overriding is optional.
    protected boolean shouldValidate() { return true; }
    protected void afterSave(int count) { }
    // ... private validate(...) and save(...): shared steps written once
}

// File: code/lld/ch11/templatemethod/JsonImportJob.java — overrides both hooks
    @Override protected boolean shouldValidate() { return false; }
```

The JSON run prints `skipping validation, source is validated upstream` and then its `afterSave` hook — the flow is identical, only steps differ.

Full runnable code: code/lld/ch11/templatemethod/ (see the Full Edition).

Template Method vs Strategy

	Template Method	Strategy
What varies	**One step**; algorithm fixed	**The entire** algorithm
Mechanism	**Inheritance**	**Composition**
When chosen	At creation (which subclass); fixed	Run time, per call
Axes that can vary	One (one parent)	Several
One file shows	Only the varying pieces	The complete algorithm

Template Method pays off when **most of the flow is common** and the compiler should **enforce the sequence**. Combined form: fixed sequence, steps passed as **callbacks** — Spring's `JdbcTemplate` owns connect/execute/close and takes a `RowMapper`.

Trade-offs

Pros	Cons
Shared flow written once; order guaranteed (`final`).	No single file tells the whole story; one parent only.
New variants small; compiler reports missing steps.	Tight coupling; base classes swell with abstract methods.

When not to use it

The flow is two lines; variants must combine (encrypted *and* CSV); behaviour must change after creation (Strategy); subclasses override every step.

Common mistakes

1. **Too many required steps** — make most of them hooks.
2. **Calling an overridable step from the base constructor** — subclass fields are still `null`/`0` (in C++ the base version runs). Call steps from the template method only.
3. **Omitting `final`** — subclasses can reorder or skip; the pattern becomes a mere agreement.
4. **Public steps** — callers could run step 3 alone; keep them `protected`.
5. **Violating Liskov** in overrides.

Template Method in other languages

Java: `final` template, `protected abstract` steps (an interface `default` method can't be `final`). Python: `abc.ABC` + `@abstractmethod`, no enforced `final`. TypeScript: `abstract` classes. C++: the **non-virtual interface (NVI)** idiom. Go/Rust: no inheritance — pass an interface/trait of steps, i.e. the Strategy shape.

Real-world examples

`InputStream.read(byte[],int,int)`; `AbstractList` (implement `get`/`size`); `HttpServlet.service()` → `doGet`/`doPost` (else 405); AQS `acquire` → `tryAcquire`; JUnit and Android `Activity` lifecycles; Maven phases; Spring `JdbcTemplate`/`TransactionTemplate`.

In the interview

> **30-second answer:** "Template Method puts an algorithm's outline in one final base-class method; subclasses supply only the steps that differ. Required steps are abstract, optional ones are hooks with a default body, fixed ones are private. It's the Hollywood principle — the base class calls the subclass — which is how servlets, JUnit and Android lifecycles work. It varies one step via inheritance; for run-time choice or several axes I prefer Strategy, or callbacks like JdbcTemplate."

- *vs Factory Method?* A factory method is often one step of a template method — the step whose job is creation (GoF notes the link; not a strict rule).

Self-check

1. **Q:** Which method is `final`? **A:** The template method (`run`, `prepareRecipe`).
2. **Q:** Abstract step vs hook? **A:** Abstract must be implemented; hooks have defaults.
3. **Q:** Why is `JdbcTemplate` more flexible? **A:** It takes the varying step as a callback (composition), not a subclass.

TL;DR

Fixed order in a `final` base method; subclasses fill steps; hooks are optional steps. Hollywood principle. Inheritance-based — prefer Strategy/callbacks for flexibility.

11.5 Visitor

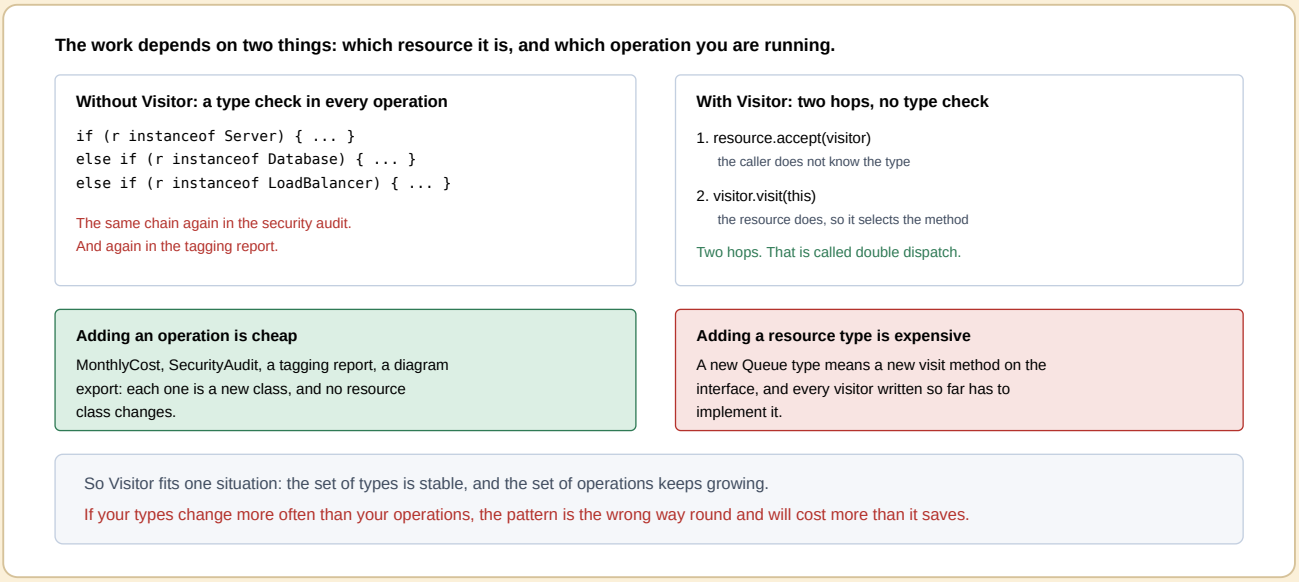


Figure — Without Visitor every operation repeats an instanceof chain; with Visitor two calls (accept, then visit) pick the right code (course diagram)

What it is

Visitor lets you **add new operations** over a set of classes **without changing them**. Each operation is a **visitor** class with one method per element kind; each element keeps a one-line `accept(visitor)` whose body is `visitor.visit(this)`.

GoF intent: represent an operation on the elements of an object structure as a separate object, so new operations don't change the element classes.

Participant	Role	In `lld.ch11.visitor`
Visitor	One `visit` per ConcreteElement	`PlanVisitor`
ConcreteVisitor	One class per operation, often accumulating	`MonthlyCost`, `SecurityAudit`
Element	Declares `accept(visitor)`	`Resource`
ConcreteElement	`accept` = `visitor.visit(this)`	`Server`, `Database`, `LoadBalancer`, `ResourceGroup`
ObjectStructure	Lets a visitor reach every element	the `ResourceGroup` tree in `Main`

Analogy: a tax inspector brings tax rules to different businesses; next year a fire inspector brings different rules — the businesses don't change.

The problem it solves

An infrastructure plan has a **stable** set of kinds (server, database, load balancer) but **operations keep arriving** (billing, security review, export). Inside the classes, unrelated teams edit `Server`; outside with `instanceof` chains, a new kind means finding every chain unaided. Visitor gives each operation a class and uses **double dispatch** instead of type checks.

How it works — double dispatch

Java picks the **override** by the receiver's run-time type, but the **overload** (`visit(Server)` vs `visit(Database)`) at **compile time** from the argument's static type — so `visitor.visit(resource)` with a `Resource` can't compile to the right overload. Visitor makes **two calls**:

1. `resource.accept(visitor)` — dispatch on the **element's** run-time type (`Server.accept` runs).
2. Inside it, `visitor.visit(this)` — `this` is statically `Server`, so `visit(Server)` is chosen; dispatch then happens on the **visitor's** run-time type.

The code that runs depends on **both** types: **double dispatch**.

Structure (UML)

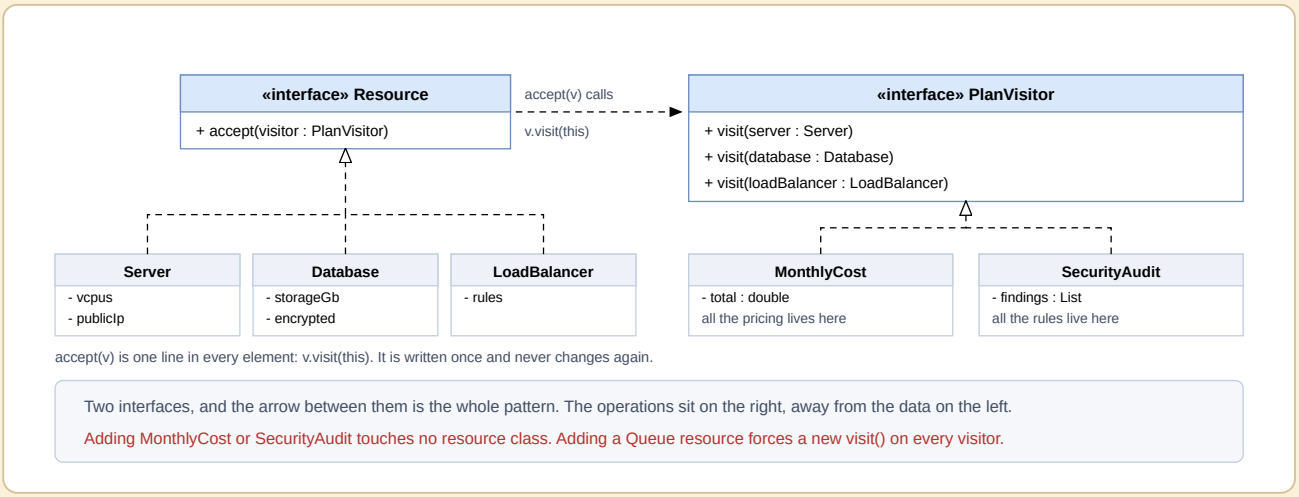
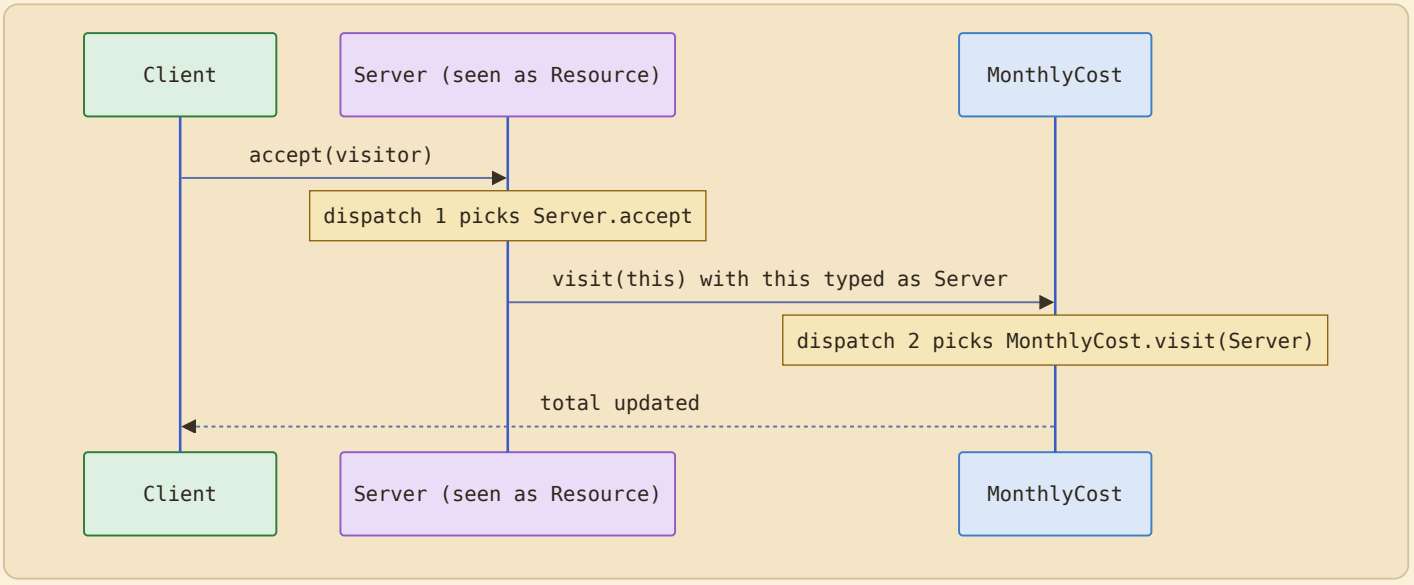


Figure — Resource with accept; PlanVisitor has one visit per resource type; MonthlyCost and SecurityAudit implement it (course diagram; it shows `total : double`, our code uses `BigDecimal`)

Each visitor depends on **every** element type — that's why **adding an element type is expensive**.



Java example

(1) **Classic Visitor** — record elements; `ResourceGroup` is a **Composite** visiting itself then its children; a new visitor per run (they accumulate). (2) **Java 21** `ModernPlan` — a sealed `Infra`, `switch` with **no** **`default`** (compiler-checked exhaustive), **record patterns** and **guards**; guarded cases don't count toward exhaustiveness, hence the unguarded fallbacks.

```java
// File: code/lld/ch11/visitor/PlanVisitor.java
public interface PlanVisitor {
    void visit(Server server);
    void visit(Database database);
    void visit(LoadBalancer loadBalancer);
    void visit(ResourceGroup group);
}

// File: code/lld/ch11/visitor/Server.java
public record Server(String name, int vcpus, boolean publicIp) implements Resource {
    public void accept(PlanVisitor v) { v.visit(this); }   // 2nd dispatch: overload picked by static type
Server
}

// File: code/lld/ch11/visitor/ResourceGroup.java
public record ResourceGroup(String name, List<Resource> children) implements Resource {
    public void accept(PlanVisitor v) {
        v.visit(this);
        for (Resource child : children) child.accept(v);
    }
}

// File: code/lld/ch11/visitor/ModernPlan.java
    public sealed interface Infra permits Vm, Sql, Lb {}
    // ... records Vm, Sql, Lb; monthlyCost(...) is the same switch without guards

    public static Optional<String> audit(Infra r) {
        return switch (r) {
            case Vm(String n, int cpus, boolean pub) when pub -> Optional.of(n + ": server has a public IP");
            case Sql(String n, int gb, boolean enc) when !enc -> Optional.of(n + ": storage not encrypted");
            case Lb(String n, int rules) when rules == 0 -> Optional.of(n + ": load balancer has no rules");
            case Vm vm -> Optional.empty();     // no finding
            case Sql db -> Optional.empty();
            case Lb lb -> Optional.empty();
        };
    }
```

Both styles print `monthly cost: 258.00` and the same findings; the modern one needs no `accept` and no visitor interface.

Full runnable code: code/lld/ch11/visitor/ (see the Full Edition).

The one question that decides it: the expression problem

	Add a new operation	**Add a new type**
Methods on the classes (plain OO)	Edit every class	One new class
Visitor (or sealed + switch)	One new visitor/function	Edit the visitor interface **and every visitor**

Ask: **which changes more often — types or operations?** Stable types + growing operations → Visitor/sealed types. Growing types → methods on classes. No plain OO design makes both cheap (the **expression problem**). Visitor is **open for new operations, closed to new element types**.

Sealed interfaces + pattern matching (Java 17 sealed; Java 21 pattern `switch` and record patterns) give Visitor's exhaustiveness check with far less code. Rules: pattern switches must be exhaustive; **unguarded** cases over every permitted subtype need no `default`; a non-sealed type needs a `default` (check lost); an unrecompiled new subtype → `MatchException`; **dominance** — guarded `case Vm(...) when pub` must precede `case Vm vm`; no `case null` → `NullPointerException` on `null`.

Use a classic Visitor when the hierarchy **can't be sealed** (library classes exposing `accept`, e.g. ASM, javac trees), for per-node callbacks over big trees, or for injectable per-operation classes. Elsewhere: Kotlin `when`, Python `singledispatch`, C++ `std::visit`, Rust `match`.

Trade-offs

Pros	Cons
New operation = one class; related logic together.	New element type changes every visitor; indirect dispatch.
Compiler reports unhandled types; elements stay small.	Elements must expose data; ceremony for small sets.

When not to use it

New types arrive often; a tiny model (three types, one operation); an operation for one type only (a method); your language checks switch exhaustiveness (use that).

Common mistakes

1. Visitor for two element types. 2. Public element fields (use accessors/records). 3. Reusing a stateful visitor without reset. 4. A `default` in a sealed-type switch — **turns off** the exhaustiveness check. 5. A composite `accept` that forgets its children (decide where traversal lives). 6. A visitor where a sealed `switch` would do.

Real-world examples

Compilers/linters (javac `TreeVisitor`, ANTLR visitors); ASM `ClassVisitor`; `FileVisitor` + `Files.walkFileTree` (really a tree-walk callback — no double dispatch); DOM serialisers; query planners.

In the interview

> **30-second answer:** "Visitor pulls an operation out of the element classes into a class with one method per element type. Each element has a one-line `accept` calling `visitor.visit(this)`, which gives double dispatch: the method chosen depends on both element and visitor types, so no `instanceof` chains. Adding operations is cheap and adding types expensive — the expression problem — so it fits stable type sets like ASTs. In Java 21 I'd usually use a sealed interface with an exhaustive pattern-matching switch instead."

- *Why not just call `visitor.visit(resource)`?* With static type `Resource` the compiler can't choose `visit(Server)`.
- *vs Iterator / Interpreter?* Iterator decides the route, Visitor the work at each stop; Interpreter keeps evaluation **in** nodes, Visitor moves it **out**.

Self-check

1. **Q:** Body of every `accept`? **A:** `visitor.visit(this)`.
2. **Q:** What's cheap and what's expensive? **A:** New operations cheap; new element types expensive.
3. **Q:** Why no `default` in `monthlyCost`? **A:** It would hide missing cases; without it the compiler forces every permitted type.

TL;DR

Operations become visitor classes; elements just `accept`; double dispatch picks the method. Cheap operations, expensive types. In Java 21 prefer **sealed interfaces + exhaustive pattern matching**.

11.6 GoF behavioral patterns: quick-reference summary

Pattern	Apply when	Example	Pros	Cons
Chain of Responsibility	Handler not known in advance	HTTP middleware	Decouples sender/receiver	May go unhandled; hard to debug
Command	Queue, log, retry, undo	GUI actions, job queues	Undo/redo; decoupled invoker	Many small classes
Interpreter	Small language to evaluate	Regex, rule DSLs	Easy to extend grammar	Unmanageable for big grammars
Iterator	Standard traversal	Playlist browsing	Simple client code	Hidden costs
Mediator	Tangled peer interactions	Chat, air traffic control	Fewer dependencies	God-object risk
Memento	Undo, checkpoints	Editor undo, save games	Atomic restore	Memory-heavy
Observer	One change must reach others	Live data feeds	Loose coupling	Leaks, no ordering
State	Lifecycle-dependent behaviour	Order lifecycles	Localised state behaviour	More classes; graph spread out
Strategy	Several ways to do one task	Retry policies	Swappable, isolated	Someone must choose
Template Method	Fixed structure, varying steps	Framework lifecycles	Reuse; enforced order	Inheritance rigidity
Visitor	Many operations, stable types	Compiler passes	Easy new operations	Hard new element types

---

11.7 Beyond GoF: patterns every LLD interview expects

From Fowler's *PoEAA* (2002), Evans's *DDD* (2003) and *Head First*; used in almost every case study.

11.7.1 Null Object

What it is

A real object implementing the interface with **neutral, do-nothing behaviour**, used instead of `null`, so callers need no `if (x != null)`. Analogy: a guest badge that opens no special doors.

The problem it solves

Returning `null` for "not found" forces every caller to remember a check; one forgotten check is a production NPE, and the checks clutter logic.

How it works

An interface (`Customer`), a real class (`RegisteredCustomer`) and a **null class** (`GuestCustomer`) returning neutral values (0% discount) or doing nothing; return it wherever you'd return `null`. Stateless null objects can be an `enum` **singleton**. Offer `isGuest()` sparingly.

Java example

```java
// File: code/lld/ch11/nullobject/GuestCustomer.java
public enum GuestCustomer implements Customer {
    INSTANCE;

    public String displayName() { return "guest"; }
    public int discountPercent() { return 0; }            // neutral value
    public void notify(String message) { }                // does nothing, on purpose
    @Override public boolean isGuest() { return true; }   // for the rare caller that must know
}

// File: code/lld/ch11/nullobject/CustomerDirectory.java
    /** Style 1: never return null; return the Null Object instead. */
    public Customer find(String id) { return byId.getOrDefault(id, GuestCustomer.INSTANCE); }

    /** Style 2: make absence explicit in the type; the caller chooses the fallback. */
    public Optional<Customer> lookup(String id) { return Optional.ofNullable(byId.get(id)); }
```

Full runnable code: code/lld/ch11/nullobject/ (see the Full Edition).

Trade-offs, when not to use it, mistakes

Pros: no null checks, polymorphism over conditionals. Cons: can **hide real errors**. **Don't use it** when absence is an **error** (throw) or the caller must decide (return `Optional`). **Null Object vs `Optional`:** `Optional` makes absence visible in the type — use it for lookups; use Null Object for legitimately optional **collaborators** (logger, listener, discount policy). Mistakes: side effects or non-neutral values, a mutable singleton, overusing `isNull()`.

Real-world examples

`Collections.emptyList()`; `OutputStream.nullOutputStream()` (Java 11); SLF4J `NOPLogger`.

In the interview

30-second answer: "Null Object replaces `null` with an implementation that does nothing or returns neutral values, so callers never need null checks — like returning an empty list. I use it for optional collaborators like loggers, and `Optional` for lookups where the caller decides. I avoid it when absence is an error, because it would hide the bug." Not GoF — Bobby Woolf (*PLoPD 3*); Fowler's "Introduce Special Case".

TL;DR: Harmless "nothing" object instead of `null`; `Optional` when the caller chooses; throw when absence is a bug.

11.7.2 Dependency Injection (DI)

What it is

A class **receives** its dependencies from outside instead of **creating** them with `new`. It is the usual way to implement **Inversion of Control** — a composition root or container wires objects. (Name settled by Fowler, 2004.) Analogy: a kitchen gets deliveries from suppliers; change supplier, the recipes don't change.

The problem it solves

`private final EmailSender sender = new EmailSender();` can never send SMS, can't be unit-tested without sending email, and hides its dependencies. Fix: depend on an **interface** (`MessageSender`), receive it via the **constructor**.

How it works

Dependencies are **constructor parameters typed by interfaces** (DIP, Ch 4); a **composition root** (`main` or a container) builds and passes them; tests pass **fakes**.

Style	How	Use for
Constructor	Constructor params → `final` fields	Required dependencies — the default
Setter	`setX(...)` later	Optional/changeable dependencies
Field	Framework writes fields (`@Autowired`)	Avoid: hidden, harder to test

Java example

`OrderService` receives a `MessageSender` **and a** `Clock` (`Clock.fixed` makes time testable). `Container` is a toy IoC container: register a recipe per type; `get` builds on demand and caches singletons.

```java
// File: code/lld/ch11/di/OrderService.java
public final class OrderService {
    private final MessageSender sender;
    private final Clock clock;

    public OrderService(MessageSender sender, Clock clock) {
        this.sender = sender;
        this.clock = clock;
    }

    public void placeOrder(String customer, String orderId) {
        LocalTime now = LocalTime.now(clock);
        sender.send(customer, "order " + orderId + " placed at " + now);
    }
}
```

A test injects `RecordingSender` instead. Full runnable code: code/lld/ch11/di/ (see the Full Edition).

Trade-offs, when not to use it, mistakes

Pros: swappable implementations, easy fakes, explicit dependencies. Cons: indirection, container "magic", misconfiguration found late. Don't inject value objects or add single-implementation interfaces "just for DI". Mistakes: **Service Locator** (classes call `container.get` — dependencies hidden again), field injection, **8+ constructor dependencies** (split the class), circular dependencies.

Real-world examples

Spring, Guice, Dagger/Hilt, Jakarta CDI, Micronaut/Quarkus. Spring can inject a `Map<String, PaymentStrategy>` — DI + Strategy.

In the interview

30-second answer: "DI means a class receives its collaborators through its constructor, typed by interfaces, instead of creating them — Dependency Inversion plus Inversion of Control: a composition root or a container like Spring wires the graph. It gives swappable implementations and easy testing with fakes — I even inject the Clock. I prefer constructor injection, avoid field injection and service locators, and treat a huge constructor as a smell." *DI vs IoC?* IoC is the general idea; DI is one way to do it. *DI vs Factory?* A factory creates on request; DI hands in ready-made dependencies.

TL;DR: Don't `new` collaborators — **receive them**; wire at the composition root.

11.7.3 Repository

What it is

A **collection-like interface for loading and saving domain objects**, in domain language (`findByEmail`, `findActive`), that **hides storage** (Fowler *PoEAA*, Evans *DDD*). Analogy: a library front desk — you ask for books by author, not shelf numbers.

The problem it solves

SQL, connections and mapping scattered in business classes make rules unreadable, untestable without a DB, and tied to one storage technology.

How it works

The interface lives in the **domain layer** (return `Optional` for "maybe"); implementations in **infrastructure** (`InMemoryUserRepository` for tests, JDBC/JPA for production); services depend on the interface via DI.

Java example

```java
// File: code/lld/ch11/repository/UserRepository.java
public interface UserRepository {
    User save(User user);
    Optional<User> findById(String id);
    Optional<User> findByEmail(String email);
    List<User> findActive();
    boolean deleteById(String id);
}

// File: code/lld/ch11/repository/SignupService.java
    public String signUp(String id, String email) {
        if (users.findByEmail(email).isPresent()) return "rejected: " + email + " already registered";
        users.save(new User(id, email, true));
        return "created " + id;
    }
```

Full runnable code: code/lld/ch11/repository/ (see the Full Edition).

Trade-offs, when not to use it, mistakes

Pros: storage-agnostic, in-memory testable logic; swappable storage. Cons: an extra layer for trivial CRUD; storage concerns can leak. One repository per **aggregate root**, not per table. **Repository vs DAO:** DAO is table-oriented; repository is domain-oriented. Mistakes: returning `null`, exposing `ResultSet`, business rules in the repository, `findByXAndYOrZ` explosions (use Specification). **Concurrency:** `signUp` is **check-then-act** — two concurrent sign-ups can both pass; enforce with a DB **unique constraint** (or `putIfAbsent` by email).

Real-world examples

Spring Data `JpaRepository` (queries derived from method names), Micronaut Data, Android Room.

In the interview

30-second answer: "A repository is a collection-like interface for aggregates in domain language — save, findById, findByEmail — that hides storage. Services depend on the interface, so logic is testable with an in-memory implementation and storage can change. In machine-coding rounds I always put an in-memory repository behind an interface."

TL;DR: A **domain-language collection interface** over storage; start in-memory, swap later.

11.7.4 Specification

What it is

A **business rule as an object** with `isSatisfiedBy(candidate)`, **combinable** with `and`, `or`, `not` (Evans & Fowler). Analogy: a shopping site's filter panel.

The problem it solves

Rules like "bargain electronics = electronics AND price < 50.00 AND in stock" get duplicated across queries, validation and UI, or explode into `findByCategoryAndPriceLessThan...` methods. Specification names each rule, makes it reusable and composable.

How it works

`Specification<T>` with default `and` / `or` / `not` returning new specs (Composite + Interpreter-like evaluation); small named factories (`inStock()`); combine and pass to a repository or stream filter.

Java example

```
// File: code/lld/ch11/specification/Specification.java
@FunctionalInterface
public interface Specification<T> {
    boolean isSatisfiedBy(T candidate);

    default Specification<T> and(Specification<T> other) { return c -> isSatisfiedBy(c) &&
other.isSatisfiedBy(c); }
    default Specification<T> or(Specification<T> other)  { return c -> isSatisfiedBy(c) ||
other.isSatisfiedBy(c); }
    default Specification<T> not()                       { return c -> !isSatisfiedBy(c); }
}
```

`inCategory("electronics").and(cheaperThan(5_000)).and(inStock())` selects `[usb-cable, keyboard]`. Full runnable code: code/lld/ch11/specification/ (see the Full Edition).

Trade-offs, when not to use it, mistakes

Pros: named, reusable, testable rules for both querying and validation. Cons: in-memory specs don't become SQL automatically (need e.g. JPA Criteria); overkill for one or two filters. Java's `Predicate<T>` already is one; a named type adds domain meaning. Mistake: filtering millions of rows in memory — push down to the DB.

Real-world examples

Spring Data JPA `Specification<T>`, search filters, eligibility rules, access policies.

In the interview

30-second answer: "A specification is a named business rule object with `isSatisfiedBy`, combinable with and/or/not. It removes duplicated conditions and finder-method explosions, and the same rule can filter a query and validate an object. `Predicate` is the building block; Spring Data JPA's `Specification` translates to SQL."

TL;DR: Rules as small combinable objects, reused for querying and validation.

11.7.5 MVC — Model–View–Controller (the Head First "compound pattern")

What it is

Model (data + rules, no UI knowledge), **View** (displays, receives input), **Controller** (interprets input, updates the model). Reenskaug, Smalltalk, Xerox PARC, late 1970s. *Head First* calls it a **compound pattern**:

Pattern inside MVC	Where
Observer	Model notifies views on change; views re-render.
Strategy	View delegates input handling to a swappable controller (e.g. read-only mode).
Composite	View is a tree of nested widgets.

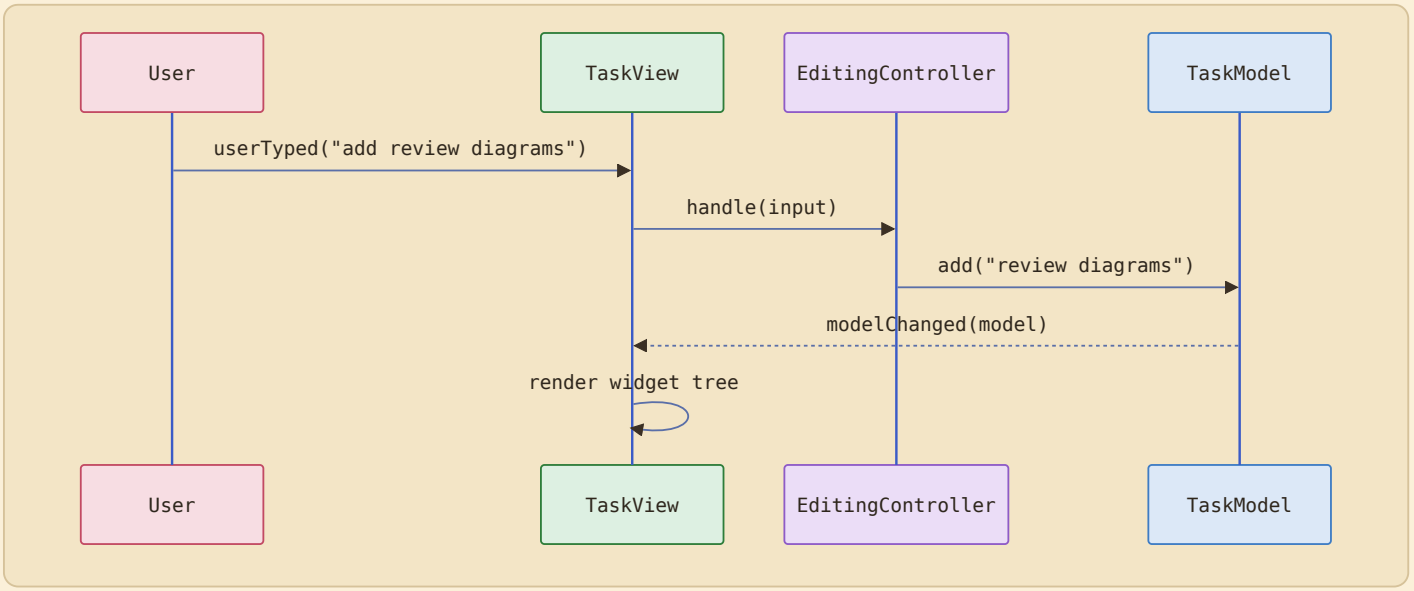
Analogy: kitchen = model, menu and plates = view, waiter = controller.

The problem it solves

Mixed UI, input and rules: can't change the screen safely, show data two ways, or test rules without a UI.

How it works

User acts on the **view** → view hands input to its **controller** → controller calls the **model** (never redraws) → model **notifies listeners** → view re-renders its **composite** tree.



Java example

A console to-do app: `TaskModel` knows only `ModelListener`s; `TaskView` implements `ModelListener`, renders `Widget.Panel`/`Widget.Label`, and forwards input to a swappable `TaskController`; switching to `ReadOnlyController` changes behaviour without touching view or model.

```
// File: code/lld/ch11/mvc/TaskModel.java
    public void add(String title) {
        if (title.isBlank()) throw new IllegalArgumentException("title must not be blank");
        tasks.add(new Task(nextId++, title, false));
        fireChanged();
    }
    // ...
    private void fireChanged() { listeners.forEach(l -> l.modelChanged(this)); }
```

Full runnable code: code/lld/ch11/mvc/ (see the Full Edition).

Trade-offs, variants and mistakes

Pros: rules testable without UI, several views per model. Cons: more classes; observer flows hard to trace; "fat" controllers. **Web MVC differs:** Spring MVC ("Model 2") has a **front controller** (`DispatcherServlet`) routing requests to `@Controller` methods; the view renders once per request and does **not** observe the model. Relatives: **MVP** (passive view) and **MVVM** (view binds to an observable view-model — Android `ViewModel` + `LiveData`/`StateFlow`, WPF; the view-model also acts as a mediator). Mistakes: fat controllers, views changing the model directly, models importing UI classes.

Real-world examples

Spring MVC, Rails, Django (MTV), Swing (`JTable` + `TableModel`), iOS UIKit, Android MVVM.

In the interview

30-second answer: "MVC separates the model — data and rules — from the view that shows it and the controller that interprets input. Head First calls it a compound pattern: the model is an Observer subject for views, the controller is a Strategy the view delegates to, and the view is a Composite of widgets. On the web, like Spring MVC, it's request-driven with a front controller, and the view doesn't observe the model."

> **TL;DR: Model** (observable state + rules) → **View** (composite, observes) → **Controller** (strategy interpreting input).

11.7.6 Event Sourcing (lite)

What it is

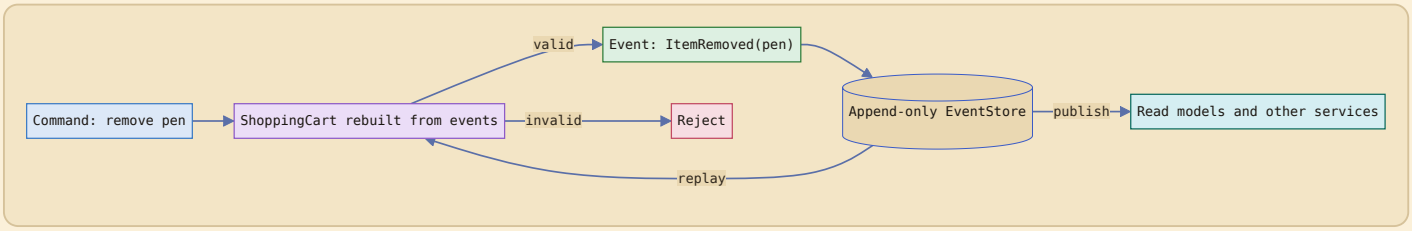
Store **every change as an immutable event** (`ItemAdded`, `ItemRemoved`, `CheckedOut`) in an **append-only log** and derive current state by **replaying** — instead of overwriting the latest state. It borrows Command (events ≈ recorded commands), Memento (snapshots) and Observer (projections) at system level. Analogy: a bank passbook — the transactions are the truth, the balance is derived.

The problem it solves

Update-in-place loses history: no "what did the cart look like before checkout?", no audit, no rebuilding a read model after a bug. Events give **time travel**, **audit**, and **new views** by replay.

How it works

1. **Command side:** replay the aggregate, **validate** the command, **emit a past-tense event**.
2. **Append** it — never update or delete events.
3. **Apply:** a pure state update, no validation (events are facts).
4. **Read side:** replay all or the first *n* events; publish events to observers building read models (**CQRS**).
5. Long streams: periodic **snapshots** (Memento) + replay of newer events.



Java example

A **sealed** `CartEvent` hierarchy; `apply` is an exhaustive pattern-matching `switch` (a forgotten event type is a compile error); `EventStore` is an append-only, `synchronized` in-memory log; prices `BigDecimal`.

```
// File: code/lld/ch11/eventsourcing/ShoppingCart.java
    public static ShoppingCart replay(List<CartEvent> history, int n) {
        ShoppingCart cart = new ShoppingCart();
        history.stream().limit(n).forEach(cart::apply);
        return cart;
    }

    /** Pure state transition: no validation, no side effects. Events are facts. */
    private void apply(CartEvent e) {
        switch (e) {
            case CartEvent.ItemAdded a -> lineTotals.merge(a.sku(),
a.unitPrice().multiply(BigDecimal.valueOf(a.qty())), BigDecimal::add);
            case CartEvent.ItemRemoved r -> lineTotals.remove(r.sku());
            case CartEvent.CheckedOut c -> checkedOut = true;
        }
    }

    /** Command side: validate against current state, THEN emit an event. */
    public CartEvent decideRemove(String sku) {
        if (checkedOut) throw new IllegalStateException("cart already checked out");
        if (!lineTotals.containsKey(sku)) throw new IllegalArgumentException(sku + " not in cart");
        return new CartEvent.ItemRemoved(sku);
    }
```

Replaying the first *n* events shows the cart after each one ("time travel"); removing from a checked-out cart prints `command rejected: cart already checked out`.

Full runnable code: code/lld/ch11/eventsourcing/ (see the Full Edition).

Trade-offs, when not to use it, mistakes

Pros: audit, time travel, new read models anytime. Cons: complexity, **event schema evolution**, eventual consistency. Not for plain CRUD. Mistakes: validation in `apply`; command-named events (`RemoveItem` vs `ItemRemoved`); mutable events; derived data in events; no snapshots; **no expected-version check on append** — real stores reject a second writer decided on stale state (optimistic concurrency); our toy store omits it.

Real-world examples

Bank ledgers; Git; Axon, EventStoreDB; Kafka with long retention (not *compacted* topics, which discard history).

In the interview

> **30-second answer:** "Event sourcing stores every change as an immutable, past-tense event in an append-only log and derives state by replaying. Commands are validated against replayed state and then produce events; `apply` just folds events into state. You get audit, time travel and new read models by replay; the costs are complexity, schema evolution and eventual consistency. Snapshots keep replay fast — Memento plus a command log."

> **TL;DR: Store facts, not state**; replay to rebuild; validate commands, never events; snapshot long streams.

11.8 Choose the right behavioral pattern

Start from the **symptom**, not the pattern name.

If you see / need...	Use	Not to be confused with
Independent checks, any may reject; per-route checks	**Chain of Responsibility**	Decorator
Undo/redo, queue, retry, log requested actions	**Command**	Strategy
Evaluate user-written rules in a small language	**Interpreter** (+ Specification)	Visitor
Walk a collection/tree/paged API	**Iterator**	Visitor
Interdependent objects where the *order* of reactions matters	**Mediator**	Observer
Roll back without exposing internals	**Memento**	Command undo
Notify an open-ended set of independent listeners	**Observer** (or a broker)	Mediator
3+ methods repeat a status `switch`; legality depends on status	**State**	Strategy
Interchangeable algorithms chosen by config/run time	**Strategy** (often a lambda)	State, Template Method
Long shared flow, a few differing steps	**Template Method** (or + callbacks)	Strategy
Many new operations over stable types	**Visitor** / sealed types + pattern matching	Iterator, Strategy
`null` checks for an optional collaborator	**Null Object** (`Optional` for lookups)	—
Classes create their own collaborators	**Dependency Injection**	Service Locator
Business logic mixed with storage	**Repository**	DAO
Same rule duplicated; finder-method explosion	**Specification**	—
UI, input and rules tangled	**MVC / MVVM**	—
Full history, audit, time travel	**Event Sourcing** (+ snapshots)	Plain audit log

Three questions for the look-alikes: (1) **Do the alternatives reference each other?** Yes → State; no → Strategy (or Bridge). (2) **Can one participant stop the flow?** Yes → CoR; all always run → Decorator/pipeline; everyone told, nobody stops → Observer. (3) **Does a central object decide what's next?** Yes → Mediator; each listener decides → Observer.

In the interview (chapter-level)

"Which patterns have you used?" Name concrete ones with their trade-off (e.g. Strategy for payment methods via a Spring-injected map; Observer via Spring events, moved to Kafka for retries; State for orders with `WHERE status = ?` guarding races). **"When would you avoid patterns?"** Two branches with no third coming, or when a lambda/enum does the job.

Self-check (whole chapter)

1. **Q:** Order placed → accepted → preparing → delivered; cancel only early. **A:** State.
2. **Q:** Card, UPI, wallet, more next quarter. **A:** Strategy via a factory/DI map.
3. **Q:** After an order: email, analytics, notify restaurant, more to come. **A:** Observer (or broker pub/sub).
4. **Q:** Type-check, optimise, pretty-print a fixed AST. **A:** Visitor, or sealed + pattern matching.
5. **Q:** Three patterns inside MVC? **A:** Observer, Strategy, Composite.

TL;DR

- **Observer** broadcasts; **Mediator** coordinates; **CoR** lets someone stop the flow.
- **State** vs **Strategy**: states know each other; strategies are chosen from outside.
- **Template Method** = inheritance, one step varies; **Strategy** = composition, whole algorithm.
- **Visitor** = cheap operations, expensive types; Java 21 **sealed + switch** usually replaces it.

- Beyond GoF: **Null Object, DI, Repository, Specification, MVC, Event Sourcing** — the glue of real systems.

Chapter 12 — Concurrency & Multithreading for LLD (Java) · Study Edition

> **Study edition.** Every topic of the full chapter, with the padding removed. For complete code listings, full outputs and extra follow-ups, see the Full Edition (`README.md`).
>
> **Why this chapter exists.** In an LLD interview you will hear: *"Two users click **Book** on the same seat at the same moment. What happens?"* A design with no answer is incomplete. This chapter teaches the ideas and Java 21 tools to answer it; Chapter 13 applies them to classic problems, and chapters 14–21 use them in case studies.

Every concept follows: *What it is → The problem it solves → How it works → Java example → Trade-offs → When not to use it → Common mistakes → Real-world examples → In the interview → Self-check → TL;DR.* Code lives in `code/lld/ch12/<topic>/Main.java`.

```
javac --release 21 -d /tmp/lld_12 $(find . -name '*.java')
java -cp /tmp/lld_12 lld.ch12.threads.Main
```

Part	Sections	Code package
A. Foundations	12.1–12.4	`threads`
B. What goes wrong	12.5 Races · 12.6 Java Memory Model	`race`
C. Locks	12.7–12.11	`locks`
D. Coordination	12.12 Semaphores · 12.13 Conditions · 12.14 Latch & barrier	`coordination`
E. Lock-free	12.15 CAS & atomics · 12.16 Concurrent collections	`atomics`, `collections`
F. Liveness failures	12.17 Deadlock · 12.18 Livelock · 12.19 Starvation	`deadlock`
G. Patterns	12.20–12.24	`patterns`
H. Interviews	12.25 Parking, seat booking, inventory	`lldcases`

Term	Meaning in one line
Thread	One independent sequence of instructions inside a program.
Shared state	Data more than one thread can read or write.
Thread-safe	Correct under any interleaving, with no extra locking by callers.
Atomic	One indivisible step: others see "before" or "after", never "halfway".
Lock / mutex	A token only one thread can hold at a time.
Critical section	Code touching shared state that must not run in two threads at once.
Contention	Many threads wanting the same lock at once.
Blocking	A thread waits (lock, I/O, signal) instead of using the CPU.
Liveness	"Something good eventually happens" — no deadlock, livelock, starvation.
Safety	"Nothing bad ever happens" — no corrupted data, no double booking.

Part A — Foundations

12.1 Concurrency vs parallelism

What it is

Concurrency = *dealing with* many things at once (tasks with overlapping lifetimes, maybe taking turns on one core). **Parallelism** = *doing* many things at the same instant on different cores.

The problem it solves

Programs wait a lot (disk, network, DB, users): concurrency does useful work meanwhile (responsiveness, throughput). Parallelism finishes CPU-heavy work sooner using all cores (speed-up).

How it works

The OS scheduler gives each runnable thread a time slice; when it ends or the thread blocks, a **context switch** saves one thread's registers and loads another's. On one core that is the *illusion* of simultaneity; on many cores threads really run at once.

	Concurrency	Parallelism
Core idea	Structure: many tasks in progress	Execution: many at the same instant
Needs multiple cores?	No	Yes
Main benefit	Overlapping waiting (I/O-bound)	Speed on CPU-bound work
Java tools	Threads, executors, virtual threads, `CompletableFuture`	Fork/Join, parallel streams, `Arrays.parallelSort`
Example	Web server with 10,000 connections	Sorting 100 million numbers

Java example

12.4's `Main` runs 10,000 sleeping tasks on virtual threads in well under a second (concurrency); Chapter 13's merge sort uses Fork/Join (parallelism).

Trade-offs

Concurrency adds complexity and context-switch overhead. **Amdahl's law:** with sequential fraction s, speed-up on N cores $\le 1 / (s + (1 - s)/N)$ — 10% sequential caps it at 10×.

When not to use it

Small, fast CPU-light tasks (splitting costs more than it saves). Single-threaded designs are valid when they remove all locking (Redis's main thread, Node.js's event loop).

Common mistakes

Mixing up the two words; expecting more threads to always be faster (past the core count, CPU-bound work slows down).

Real-world examples

Concurrency: Tomcat/Jetty thread pools, Netty/Node.js event loops. Parallelism: `Arrays.parallelSort`, parallel streams, Spark/Hadoop.

In the interview

> **30-second answer:** "Concurrency is about structure — several tasks in progress, possibly interleaved on one core. Parallelism is about execution — tasks running at the same instant on several cores. Concurrency helps I/O-bound work; parallelism helps CPU-bound work."

Self-check

- **Q:** 50,000 idle chat connections — concurrency or parallelism? **A:** Concurrency: they mostly wait.

TL;DR

Concurrency = structure; parallelism = simultaneous execution. I/O-bound → concurrency; CPU-bound → parallelism.

12.2 Processes vs threads

What it is

A **process** is a running program with its **own memory**. A **thread** runs *inside* a process, **sharing the heap** but owning its **stack** and program counter.

The problem it solves

Threads share data cheaply within one program; processes give strong isolation (one crash doesn't corrupt others).

How it works

The OS gives a process a private address space and one thread (`main`); more threads share heap objects with separate stacks. Threads communicate through shared objects (with synchronization); processes via pipes, sockets, shared memory, message queues.

	Process	Thread
Memory	Private address space	Shared heap; own stack
Creation cost	High	Lower (platform thread ≈ OS thread, ~1 MB stack reserved); virtual threads much cheaper
Communication	IPC	Shared objects — fast, needs sync
Failure isolation	Strong	Weak (e.g. `OutOfMemoryError` kills all)
Context switch	Heavier	Lighter
Java	`ProcessBuilder`	`Thread`, executors

Java example

```java
// A new PROCESS (separate memory):
Process p = new ProcessBuilder("ls", "-l").inheritIO().start();
p.waitFor();

// A new THREAD (shares this JVM's heap):
Thread t = new Thread(() -> System.out.println("same heap, different stack"));
t.start();
t.join();
```

Trade-offs

Threads: fast sharing, but every shared mutable field is a potential bug. Processes: safe isolation, slower serialized communication.

When not to use it

When isolation matters more than speed (untrusted plugins, crash-prone native code) — use processes or containers.

Common mistakes

Thinking locals are shared (each thread has its own stack); one unbounded platform thread per request (use a pool or virtual threads).

Real-world examples

Chrome tabs as processes; PostgreSQL process-per-connection vs MySQL thread-per-connection.

In the interview

> **30-second answer:** "A process is an isolated program with its own memory; a thread is a path of execution inside a process that shares its heap but has its own stack. Threads are cheaper and share data easily but need synchronization; processes are isolated and safer but communicate through IPC."

Self-check

- **Q:** Two threads increment their own local `int x`. Race? **A:** No — locals live on each stack.

TL;DR

Process = isolated house; thread = worker sharing the heap, owning only its stack. Share cheaply, synchronize carefully.

12.3 Thread lifecycle & states

What it is

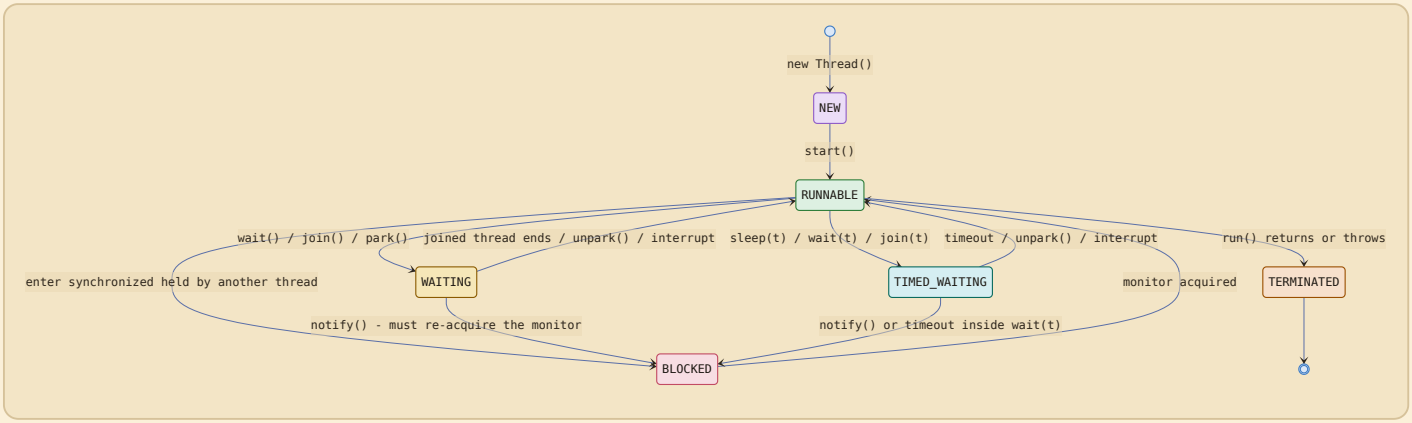
`Thread.getState()` returns one of six `Thread.State` values:

State	Meaning
`NEW`	Created, `start()` not called.
`RUNNABLE`	Running or ready (Java doesn't separate them; most blocking I/O also shows here).
`BLOCKED`	Waiting to *enter* a `synchronized` block held by another thread.
`WAITING`	Waiting indefinitely for another thread: `wait()`, `join()`, `LockSupport.park()` (used by `ReentrantLock`, `Condition`, `CountDownLatch`).
`TIMED_WAITING`	Same with a timeout: `sleep`, `wait(ms)`, `join(ms)`, `parkNanos`, `tryLock(timeout)`.
`TERMINATED`	`run()` returned or threw; cannot be restarted.

The problem it solves

Reading thread dumps (`jstack`, `jcmd <pid> Thread.print`): many `BLOCKED` on one monitor → contention or deadlock; many `WAITING` on a queue → idle pool.

How it works



Calling `start()` twice throws `IllegalThreadStateException`. A thread woken from `wait()` must re-acquire the monitor, so it may pass through `BLOCKED`. **Interruption** is cooperative: `t.interrupt()` sets a flag; `sleep` / `wait` / `join` throw `InterruptedException` and clear it. `Thread.stop()` is unsafe and no longer works.

Java example

`threadStates()` in 12.4's `Main` drives one thread into each state and prints it.

Trade-offs

A model, not a choice — but `RUNNABLE` hides whether a thread is really on a CPU or stuck in native I/O.

When not to use it

Never use `getState()` for program logic (it changes between check and act); monitoring only.

Common mistakes

Calling `run()` instead of `start()` (runs on the current thread); swallowing `InterruptedException` (rethrow or `Thread.currentThread().interrupt()`); confusing `BLOCKED` (monitor) with `WAITING` (signal).

Real-world examples

A `jstack` showing 200 Tomcat workers `BLOCKED` on one `synchronized` logger or pool method = coarse-lock bottleneck.

In the interview

30-second answer: "Java threads have six states: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED. BLOCKED means waiting for a monitor lock; WAITING/TIMED_WAITING mean waiting for another thread's action, optionally with a timeout."

Self-check

- **Q:** Inside `Thread.sleep(1000)`? **A:** `TIMED_WAITING`.

TL;DR

NEW → RUNNABLE ↔ (BLOCKED | WAITING | TIMED_WAITING) → TERMINATED. BLOCKED = monitor; WAITING = signal. `start()`, never `run()`; respect interrupts.

12.4 Creating threads: `Thread`, `Runnable`, `Callable`, `ExecutorService`, virtual threads

What it is

Way	What you write	Result?	Use when
Extend `Thread`	`class T extends Thread { run() }`	No	Almost never — mixes task and worker
`Runnable`	`() -> { ... }`	No	Fire-and-forget task
`Callable<V>`	`() -> value` (may throw checked)	Yes, via `Future<V>`	Need a result or exception back
`ExecutorService`	`pool.submit(task)`	`Future<V>`	Default: reuse threads, limit concurrency
Virtual threads (Java 21)	`Thread.ofVirtual().start(r)` / `newVirtualThreadPerTaskExecutor()`	via `Future`	Very many *blocking* tasks

The problem it solves

An OS thread per task is expensive and unbounded. Executors separate **what** runs from **how** (pooling, queueing). Virtual threads make "one thread per request" cheap again for blocking code.

How it works

Write a `Runnable`/`Callable`, `submit` it, get a `Future`; `get()` blocks (task exceptions come wrapped in `ExecutionException`); `shutdown()` + `awaitTermination`, or try-with-resources (`close()` waits, Java 19+). **Virtual threads** (JEP 444) run on a small `ForkJoinPool` of carrier threads; blocking I/O, `sleep` or a `java.util.concurrent` lock **unmounts** them. **Pinning** keeps the carrier busy: on Java 21–23 when blocking inside `synchronized` (including `wait()`) or a native method; JDK 24 (JEP 491) removed the `synchronized` case.

Java example

```
// File: code/lld/ch12/threads/Main.java
            Future<Integer> future = pool.submit(sumTask);   // returns immediately
            System.out.println("3) Callable+Future  -> sum 1..100 = " + future.get()); // get() blocks until
done
        // ...
        try (ExecutorService vexec = Executors.newVirtualThreadPerTaskExecutor()) {
            List<Future<Integer>> fs = new ArrayList<>();
            for (int i = 0; i < 10_000; i++) {
                fs.add(vexec.submit(() -> { Thread.sleep(50); return 1; }));
            }
            // ... sum the futures
        } // close() waits for all tasks
```

The 10,000 sleeping tasks finish far under 500 s; the file also tours every `Thread.State`. `t.join()` creates a happens-before edge (12.6). Full runnable code: code/lld/ch12/threads/ (see the Full Edition).

Trade-offs

	Platform thread pool	Virtual threads
Cost per thread	High (OS thread + big stack)	Very low (growable heap stack)
How many	Hundreds–few thousand	Millions
Best for	CPU-bound (size ≈ cores)	Blocking I/O
Limits concurrency?	Yes, pool size	No — add a `Semaphore`
Pooling	Yes	Never pool them

When not to use it

Virtual threads for CPU-heavy work (no extra cores); blocking in `synchronized` on Java 21–23 (pinning — prefer `ReentrantLock`); `newCachedThreadPool()` for unbounded traffic.

Common mistakes

Forgetting `shutdown()` (JVM stays alive); ignoring a `Future` (exceptions silently lost); `newFixedThreadPool` for untrusted load — its queue is **unbounded** (use a bounded `ThreadPoolExecutor`, 12.21).

Real-world examples

`Executors` factory methods (Factory pattern); Spring `@Async` on a `ThreadPoolTaskExecutor`; Spring Boot 3.2+ `spring.threads.virtual.enabled=true`.

In the interview

> **30-second answer:** "I never manage raw threads in business code. I describe work as `Runnable`/`Callable` and submit it to an `ExecutorService` sized for the workload: about cores for CPU-bound work, bounded queue with a rejection policy for incoming traffic. For many blocking I/O tasks on Java 21, I use virtual threads — one per task — and protect scarce resources with a semaphore."

Self-check

- **Q:** What does `thread.run()` do? **A:** Runs synchronously on the current thread.

TL;DR

Tasks = `Runnable`/`Callable`; run with an `ExecutorService`; results via `Future`. Virtual threads make one-thread-per-blocking-task cheap. Always shut pools down.

Part B — What goes wrong

12.5 Race conditions & critical sections

What it is

A **race condition**: the result depends on unlucky thread timing. A **critical section**: code touching shared mutable state that must run in one thread at a time. A **data race** (JMM term): two threads access one variable, at least one writes, with no happens-before order.

The problem it solves

Tells you *which* code needs protection and *why*:

Shape	Example	Why it breaks
Read-modify-write	`count++`	Really read, add, write — two threads read the same old value; one update is lost.
Check-then-act	`if (seat.isFree()) seat.book(me);`	The condition can change between check and act.

How it works

1. Find all **shared mutable state**. 2. List its **invariants** ("a seat has at most one owner"). 3. Mark read-then-update code as **critical sections**. 4. Make each **atomic** — a lock, one atomic operation (`incrementAndGet`, `putIfAbsent`), or no sharing (immutability, confinement).

Java example

```java
// File: code/lld/ch12/race/Main.java
    /** BROKEN: count++ is read-modify-write (3 steps), so concurrent increments get lost. */
    static class UnsafeCounter implements Counter {
        private int count;
        public void increment() { count++; }
        public int get() { return count; }
    }

    // ... Fix 1: SynchronizedCounter — synchronized increment() AND get() (reads need the lock for visibility)

    /** Fix 2: a lock-free atomic built on compare-and-swap. */
    static class AtomicCounter implements Counter {
        private final AtomicInteger count = new AtomicInteger();
        public void increment() { count.incrementAndGet(); }
        public int get() { return count.get(); }
    }
    // ... check-then-act fixed with one atomic step: seatOwner.compareAndSet(0, id)
```

Output: of 1,000,000 increments the unsafe counter lost e.g. `680044` (varies every run — that unpredictability *is* the race); both fixes give exactly 1000000.

Full runnable code: code/lld/ch12/race/ (see the Full Edition).

Trade-offs

Fix	Pros	Cons
Lock	Protects any multi-step, multi-variable invariant	Waiting; deadlock risk if nested
Atomic variable / map method	Fast, non-blocking	One variable or one map entry only
Immutability / confinement	No synchronization	Needs redesign

When not to use it

Don't lock data that is never shared (locals, confined objects) or never changed after safe construction.

Common mistakes

Protecting writes but not reads; combining two thread-safe calls and assuming atomicity (`containsKey` then `put` on a `ConcurrentHashMap` — use `putIfAbsent`/`computeIfAbsent`); locking on different objects.

Real-world examples

Double-booked tickets, oversold flash-sale stock, double payments on a double-click — check-then-act races, fixed with atomics, locks or DB constraints (unique index, `SELECT ... FOR UPDATE`, version columns).

In the interview

30-second answer: "A race condition is when correctness depends on thread timing. The two common shapes are read-modify-write, like `count++`, and check-then-act, like 'if seat is free then book it'. I find the shared mutable state and its invariants, and make each critical section atomic — with a lock, an atomic operation like `putIfAbsent` or `compareAndSet`, or by removing sharing."

Self-check

- **Q:** Why does `count++` lose updates? **A:** Two threads read the same value before either writes.

TL;DR

Race = result depends on timing. Watch read-modify-write and check-then-act. Make every critical section atomic, including reads.

12.6 Java Memory Model basics: visibility, happens-before, `volatile`

What it is

The **JMM** (JLS ch. 17) defines when one thread's write is guaranteed visible to another's read. **Visibility**: will B see A's write? **Reordering**: compilers, JIT and CPUs reorder when a single thread can't tell. **Happens-before (HB)**: if X HB Y, X's effects are visible to and ordered before Y. `volatile`: reads/writes of that field are visible to all threads and ordered.

The problem it solves

`while (running) iterations++;` with a non-volatile `running` may never stop — the JIT may read it once and loop forever.

How it works

Rule	Meaning
Program order	Within a thread, each action HB the next.
Monitor lock	Unlock HB every later lock of the *same* monitor.
Volatile	A volatile write HB every later read of that field.
Thread start	`t.start()` HB everything in `t`.
Thread join	Everything in `t` HB a return from `t.join()`.
Transitivity	A HB B and B HB C ⇒ A HB C.
Final fields	Separate guarantee (JLS §17.5): after a constructor that doesn't leak `this`, every thread sees correct `final` field values, even without sync.
`java.util.concurrent`	Queue put HB take; `countDown()` HB `await()` return; `Future` completion HB `get()`.

Apply: is every write-then-read pair linked by HB? Same lock → yes. Single independent flag/reference → `volatile`. Compound update → lock or atomic.

Safe publication: share a reference via a `volatile`/`AtomicReference` field, a `final` field, a static initializer, a concurrent collection, or a lock the reader also takes. **Double-checked locking** needs `volatile`:

```
class Config {
    private static volatile Config instance;          // volatile is essential
    static Config get() {
        Config c = instance;
        if (c == null) {
            synchronized (Config.class) {
                c = instance;
                if (c == null) instance = c = new Config();
            }
        }
        return c;
    }
}
```

(Simpler: the **holder idiom** — `private static class Holder { static final Config I = new Config(); }`.)

Java example

`Worker` in 12.5 uses `private volatile boolean running`; the output `volatile flag stopped worker: true` confirms the write became visible.

Trade-offs

Tool	Visibility	Atomic compound ops	Blocking	Cost
`volatile`	Yes	No	No	Low
`synchronized` / `Lock`	Yes (same lock)	Yes	Yes	Medium
`Atomic*`	Yes	One variable	No	Low–medium
`final`	Yes, after construction	N/A	No	Free

When not to use it

`volatile` for anything combining several steps or variables.

Common mistakes

"Works on my laptop" — x86 reorders less than ARM (Apple Silicon, Graviton); leaking `this` from a constructor; synchronizing only the writer.

Real-world examples

`volatile` shutdown flags; `ConcurrentHashMap`, `AtomicInteger`, `CopyOnWriteArrayList` use volatile + CAS internally; immutable `String`.

In the interview

> **30-second answer:** "The Java Memory Model says a thread is only guaranteed to see another thread's write if there's a happens-before edge — unlock-then-lock of the same monitor, a volatile write-then-read, thread start/join, or a concurrent-collection handoff. `volatile` gives visibility and ordering for a single variable but not atomicity; for compound actions I use locks or atomics."

Self-check

- **Q:** Non-volatile `ready = true` after preparing data; B spins on it. Two bugs? **A:** B may never see it, or see it with stale data.

TL;DR

Cross-thread visibility only via happens-before. `volatile` = visibility + ordering, not atomicity. Locks = both. Publish objects safely.

Part C — Locks

12.7 Mutex (mutual exclusion) and `synchronized`

What it is

A **mutex** is a lock at most one thread can hold. Every Java object has a built-in one — its **intrinsic lock** or **monitor** — taken by `synchronized` (instance method → `this`; static method → the `Class` object; block → the given object).

The problem it solves

Makes a critical section **atomic** and **visible** (the next lock holder sees everything done before the unlock).

How it works

Free monitor → take it; held → `BLOCKED` until free; released automatically on exit, **even by exception**; **reentrant** (the holder can re-enter).

Java example

`SynchronizedCounter` (12.5). Prefer a **`private final`** lock object so outside code can't take your lock:

```
private final Object lock = new Object();
void add(Item i) { synchronized (lock) { items.add(i); } }
```

Trade-offs

Pros	Cons
Simplest; auto-release; reentrant; cheap when uncontended	No timeout, `tryLock` or interruptible wait; one wait-set per object; no fairness; pins virtual threads on Java 21–23

When not to use it

When you need a timeout, interruptibility, fairness, several conditions, or read/write separation — use `ReentrantLock` / `ReentrantReadWriteLock`.

Common mistakes

Synchronizing on a *changing* reference; on string literals or boxed values (shared JVM-wide); doing slow I/O while holding the lock.

Real-world examples

`Vector`, `Hashtable`, `StringBuffer`, `Collections.synchronizedList` — safe per call but slow under contention, hence replaced by unsynchronized classes plus concurrent collections.

In the interview

30-second answer: "A mutex lets one thread at a time into a critical section. In Java, `synchronized` uses the object's monitor: it's reentrant, released automatically, and gives visibility. I lock on a private final object and keep the section short."

Self-check

- **Q:** Synchronized methods on *different* objects block each other? **A:** No — different monitors.

TL;DR

One holder at a time; `synchronized` is reentrant and auto-releasing. Private final lock object; short sections.

12.8 `ReentrantLock` and reentrancy

What it is

`ReentrantLock` is an explicit mutex with `synchronized`'s meaning plus extras. **Reentrant**: the holder can acquire it again (hold count rises) and must release it as many times.

The problem it solves

Without reentrancy, a locked method calling another locked method on the same object would **deadlock with itself**. Extras over `synchronized`: `tryLock()` / `tryLock(timeout)`, `lockInterruptibly()`, optional **fairness**, multiple `Condition`s, `isLocked()` / `getHoldCount()`.

How it works

`lock()` takes it (count 0 → 1) or waits; the same thread locking again → count 2; each `unlock()` decrements, releasing at 0. **Always** `lock(); try { ... } finally { unlock(); }`.

Java example

```java
// File: code/lld/ch12/locks/Main.java
        void withdraw(int amount) {
            lock.lock();
            try {
                if (canWithdraw(amount)) balance -= amount;   // calls another locked method
            } finally {
                lock.unlock();                                // ALWAYS unlock in finally
            }
        }
        boolean canWithdraw(int amount) {
            lock.lock();                                      // same thread re-acquires: no self-deadlock
            try {
                System.out.println("hold count inside nested call = " + lock.getHoldCount());
                return balance >= amount;
            } finally {
                lock.unlock();
            }
        }
```

Output: `hold count inside nested call = 2`, `balance = 70`. The same file also demos `tryLock` (12.9), `ReadWriteLock` and `StampedLock` (12.10). Full runnable code: code/lld/ch12/locks/ (see the Full Edition).

Trade-offs

Feature	`synchronized`	`ReentrantLock`
Release	Automatic	Manual (`finally`)
Try / timeout	No	`tryLock()`, `tryLock(t, unit)`
Interruptible wait	No	`lockInterruptibly()`
Fairness	No	Optional (slower)
Conditions	One wait set	Many
Reentrant	Yes	Yes

When not to use it

If you need none of the extras, `synchronized` is shorter and can't be left locked.

Common mistakes

`lock()` *inside* `try` (a failed lock would `unlock()` → `IllegalMonitorStateException`); missing `unlock()` on an early return; fairness "to be safe" (much slower).

Real-world examples

`ArrayBlockingQueue`, `LinkedBlockingQueue`, `ThreadPoolExecutor` use it; `ReentrantLock`, `ReentrantReadWriteLock`, `Semaphore`, `CountDownLatch` are built on **AQS**; `CyclicBarrier` on `ReentrantLock` + `Condition`.

In the interview

> **30-second answer:** "`ReentrantLock` is an explicit mutex with the same semantics as `synchronized` plus try-lock with timeout, interruptible acquisition, optional fairness, and multiple conditions. Reentrant means the owner can re-acquire it; the hold count must drop to zero to release. I always unlock in `finally`."

Self-check

- **Q:** `lock()` ×3, `unlock()` ×2 — free? **A:** No; hold count 1.

TL;DR

`synchronized` + tryLock/timeout/interruptible/fairness/conditions. `lock(); try { … } finally { unlock(); }` always.

12.9 Try-lock & timed locking

What it is

`tryLock()` takes the lock only if free *now*; `tryLock(timeout, unit)` waits up to the timeout (interruptibly), returning `false` on failure.

The problem it solves

Plain `lock()` waits forever (hung requests, permanent deadlocks). Try-lock lets a thread **give up** — fail fast, retry, return "busy" — and breaks deadlock's *hold-and-wait* condition (12.17).

How it works

`true` → `try { work } finally { unlock(); }`; `false` → fallback. With several locks: if any fails, **release all held**, back off **randomly**, retry.

Java example

In `locks/Main.java`, `printer.tryLock(100, TimeUnit.MILLISECONDS)` fails while `main` holds it and prints `worker: printer busy for 100 ms, doing something else instead`. `transferTryLock` in 12.17 uses try-lock + random back-off.

Trade-offs

Pros	Cons
No infinite waits; breaks deadlock; predictable latency	Caller handles failure; retries without random back-off can **livelock**; timeout choice is a judgement call

When not to use it

When failing is unacceptable and waiting is fine (background batch job) — plain `lock()`.

Common mistakes

`unlock()` after a `false` → `IllegalMonitorStateException`; tight retry loops; expecting untimed `tryLock()` to be fair — it *barges*; `tryLock(0, unit)` respects fairness.

Real-world examples

Distributed locks (Redis `SET key value NX PX 30000`, Redisson `tryLock(wait, lease, unit)`, ZooKeeper); ticket seat holds that fail fast.

In the interview

> **30-second answer:** "`tryLock` with a timeout lets a thread give up instead of waiting forever. I use it when latency matters or when I need several locks: if I can't get all of them, I release what I hold, back off randomly and retry — which prevents deadlock."

Self-check

- **Q:** `tryLock(0, SECONDS)`? **A:** Tries once without waiting, but respects a fair queue.

TL;DR

Acquire only if available (within a timeout). Handle `false`, release everything on failure, back off randomly.

12.10 Read-write locks: `ReadWriteLock` and `StampedLock`

What it is

A shared **read lock** (many readers) plus an exclusive **write lock** (one writer, no readers): `ReentrantReadWriteLock`, and since Java 8 `StampedLock` with cheap *optimistic reads*.

The problem it solves

A mutex makes readers block each other; for **read-heavy** data (config, caches, catalogues) reads can run in parallel.

How it works

Readers share `readLock()` when no writer holds it; a writer waits for **no readers and no writer**. `ReentrantReadWriteLock` allows **downgrade** (write → read) but **not upgrade** (read → write deadlocks on itself). `StampedLock`: `tryOptimisticRead()` → copy fields to locals → `validate(stamp)`; on failure retry with `readLock()`.

Java example

`ConfigStore` and `Point` (optimistic read) in `locks/Main.java`; three readers inside the read lock at once print `3 readers held the read lock at the same time: true`.

Trade-offs

	Mutex	ReentrantReadWriteLock	StampedLock
Parallel readers	No	Yes	Yes (optimistic reads take no lock)
Reentrant	Yes	Yes	**No**
Conditions	Yes	Write lock only	No
Overhead	Lowest	Higher bookkeeping	Low for optimistic reads
Best when	Short or write-heavy	Reads ≫ writes, non-trivial reads	Very read-heavy, tiny sections

When not to use it

Write-heavy or tiny sections (bookkeeping makes it slower); when a **copy-on-write snapshot** is simpler (immutable map in a `volatile` field, replaced on write — 12.23).

Common mistakes

Upgrading read → write; writing under the read lock; using optimistic data before `validate` or without copying to locals.

Real-world examples

Caches, config holders, routing tables; `ConcurrentHashMap` goes further — lock-free reads.

In the interview

> **30-second answer:** "A read-write lock lets many readers in together but gives writers exclusive access. It helps when reads vastly outnumber writes. `ReentrantReadWriteLock` allows downgrade but not upgrade; `StampedLock` adds optimistic reads that take no lock and are validated afterwards, but it's not reentrant."

Self-check

- **Q:** Two readers and a writer arrive — who runs together? **A:** The two readers; the writer alone.

TL;DR

Many readers OR one writer, for read-heavy data. No upgrade; `StampedLock` = optimistic, non-reentrant.

12.11 Coarse-grained vs fine-grained locking

What it is

Coarse: one lock for a lot of state (the whole parking lot). **Fine**: many locks each guarding a small piece (per floor, spot, hash bucket).

The problem it solves

Balances **simplicity and safety** against **throughput under contention**.

How it works

Start coarse and measure; if contended, split **independent** state (**lock splitting**); for big collections use **lock striping** (lock `hash(i) % N`; Java 7 `ConcurrentHashMap` had 16 segments); take multiple fine locks in a **global order**; do slow work outside locks.

	Coarse	Fine
Simplicity	High	Lower
Concurrency	Low	High
Deadlock risk	Low	Higher — needs ordering
Overhead	One lock	Many locks
Cross-part invariants	Easy	Hard

Java example

Coarse: `SynchronizedCounter` (12.5). Fine: `Show` in 12.25 (one lock per seat, sorted order); `StripedHashMap` in Chapter 13.

Trade-offs

Correct first; go fine-grained only where profiling shows contention.

When not to use it

Don't split when an invariant spans the parts ("total booked ≤ capacity") unless you handle the cross-part check.

Common mistakes

Premature fine-graining; inconsistent lock order; fine locks that still funnel through one shared counter (use `LongAdder`).

Real-world examples

Java 8 `ConcurrentHashMap` locks one bin head; DB row vs table locks.

In the interview

> **30-second answer:** "I start with one lock for correctness, then refine where contention matters: per-floor or per-spot locks for a parking lot, per-show or per-seat locks for booking, striping for maps. Finer locks raise throughput but need a consistent lock order to avoid deadlock."

Self-check

- **Q:** Lock striping? **A:** N locks; each item maps to one by hash — fine-grained with a bounded lock count.

TL;DR

Coarse = simple, low throughput; fine = parallel, complex, deadlock-prone. Start coarse, measure, order locks.

Part D — Coordination

12.12 Semaphores

What it is

A **semaphore** holds **permits**: `acquire()` takes one (waiting if none), `release()` returns one. One permit ≈ a mutex, but with **no owner** — any thread may release.

The problem it solves

Limiting concurrency (≤ N DB connections, API calls) and **signaling** (0 permits = "wait until someone signals", used heavily in Chapter 13).

How it works

`new Semaphore(n)` is **non-fair** (callers may barge); `new Semaphore(n, true)` is FIFO (untimed `tryAcquire()` still barges; `tryAcquire(0, SECONDS)` doesn't). Also `tryAcquire(timeout)`, `acquire(k)`. Always `release()` in `finally` after a successful `acquire()`.

Java example

```
// File: code/lld/ch12/coordination/Main.java
        Semaphore connections = new Semaphore(3);
        // ... 10 tasks on a pool of 10 threads, each doing:
                try {
                    connections.acquire();                      // blocks when 0 permits left
                    try {
                        int now = inUse.incrementAndGet();
                        maxSeen.accumulateAndGet(now, Math::max);
                        Thread.sleep(30);                       // "run a query"
                        inUse.decrementAndGet();
                    } finally {
                        connections.release();                  // give the permit back
                    }
                } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
```

Output: `10 queries done, never more than 3 at once: true`. The same file demos wait/notify, `Condition`, latch and barrier (12.13–12.14). Full runnable code: code/lld/ch12/coordination/ (see the Full Edition).

Trade-offs

Pros	Cons
Simple concurrency cap; doubles as a signal	No owner — buggy extra releases grow permits; protects no data by itself

When not to use it

As a mutex around shared data when a lock would do (locks have ownership and reentrancy).

Common mistakes

Releasing when `acquire` never succeeded (e.g. interrupted) → permit leak upward; not releasing on exceptions → permits vanish, system stalls.

Real-world examples

Bulkheads (Resilience4j `SemaphoreBulkhead`, Hystrix semaphore isolation); HikariCP's max pool size.

In the interview

> **30-second answer:** "A semaphore is a permit counter: acquire takes a permit or waits, release returns one. I use it to cap concurrent access to a limited resource — a bulkhead — or with zero initial permits as a signal from one thread to another. Unlike a lock it has no owner."

Self-check

- **Q:** `new Semaphore(0).acquire()`? **A:** Waits until another thread calls `release()` — a signal.

TL;DR

N permits: cap concurrency or signal. No ownership — release carefully in `finally`.

12.13 Condition variables: `wait/notify` and `Condition`

What it is

A **condition variable** lets a thread *sleep until a condition on shared state is true*, releasing the lock meanwhile, and lets another thread *wake it* after changing that state. Built-in: `wait()` / `notify()` / `notifyAll()` inside `synchronized`; explicit: `lock.newCondition()` with `await()` / `signal()` / `signalAll()`.

The problem it solves

Waiting for a state change without **busy-waiting** and without missing the moment.

How it works

The canonical pattern — memorize it:

```
synchronized (lock) {                 // or: lock.lock(); try {
    while (!conditionHolds()) {       // ALWAYS a while loop, never if
        lock.wait();                  //   cond.await();  releases the lock while waiting
    }
    doAction();                       // condition is true AND we hold the lock
    lock.notifyAll();                 //   cond.signal()/signalAll() if we changed state others wait for
}                                     // } finally { lock.unlock(); }
```

`wait()` **atomically releases the lock** and sleeps; it **re-acquires** it before returning. Why `while`? **Spurious wakeups** are allowed; another thread may grab the state first; with `notifyAll` only one waiter can win. `notify()` wakes one arbitrary waiter — with producers *and* consumers on one monitor it can wake the wrong kind and everyone sleeps. Use `notifyAll()`, or separate `Condition`s (`notFull`, `notEmpty`) with targeted `signal()`.

Java example

`Mailbox` (monitor, `while (item != null) wait();`) and `TurnTable` (Lock + `Condition`) in `coordination/Main.java`. Output: `received in order: [m1, m2, m3]` and `PING PONG PING PONG PING PONG`.

Trade-offs

	`wait/notify`	`Lock` + `Condition`
Conditions per lock	One wait set	Many
Targeted wake-up	No — use `notifyAll`	Yes
Timed wait	`wait(ms)`	`await(t, unit)`, `awaitNanos`
Verbosity	Less	More

When not to use it

Usually `BlockingQueue`, `Semaphore`, `CountDownLatch` or `CompletableFuture` already solve it; hand-roll only for custom coordination.

Common mistakes

`if` instead of `while`; `wait`/`notify` without holding that monitor → `IllegalMonitorStateException`; changing state without notifying (**lost wakeup**); waiting on one object while synchronized on another.

Real-world examples

`ArrayBlockingQueue`: one `ReentrantLock`, two `Condition`s (`notEmpty`, `notFull`) — built by hand in Chapter 13.

In the interview

> **30-second answer:** "A condition variable lets a thread release a lock and sleep until another thread changes the state and signals it. I always wait in a while loop that re-checks the condition, because of spurious wakeups and competing threads. With one monitor I use notifyAll; with ReentrantLock I use separate Conditions and targeted signal."

Self-check

- **Q:** What does `wait()` do with the monitor? **A:** Releases it while sleeping; re-acquires before returning.

TL;DR

Release lock, sleep until signalled, re-acquire, re-check. Always `while`; `notifyAll` or separate `Condition`s.

12.14 `CountDownLatch` and `CyclicBarrier`

What it is

`CountDownLatch(n)`: a one-time gate — `await()` returns after *n* `countDown()` calls; can't be reset.
`CyclicBarrier(n, action)`: a reusable meeting point — when the *n*th thread calls `await()`, the action runs once, all are released, and it resets.

The problem it solves

Coordinating phases: start after all services are up (latch); finish step 1 before step 2 (barrier); release all test threads at once (latch of 1 — used throughout this book to maximise contention).

How it works

	CountDownLatch	CyclicBarrier
Who waits	Anyone calling `await()`	The *n* parties themselves
Who counts	Anyone calling `countDown()`	Arrival at `await()`
Reusable	No	Yes
Extra	Reaching 0 is permanent	Action once per trip; a failed party **breaks** it (`BrokenBarrierException`; `reset()` repairs)
Happens-before	Before `countDown()` HB after `await()`	Before `await()` HB the action HB after release

`Phaser` (Java 7) generalises both with dynamic parties across many phases.

Java example

`latchDemo`/`barrierDemo` in `coordination/Main.java` print `all services ready: true (count now 0)` and `phase 1 complete…`, `phase 2 complete…`.

Trade-offs

Latch: simple, one-shot. Barrier: reusable but fragile — one failure breaks it for all.

When not to use it

To wait for N tasks and collect results, `invokeAll` or `CompletableFuture.allOf` is clearer.

Common mistakes

A worker throws before `countDown()` → waiting forever (put it in `finally`; use `await(timeout)`); fewer threads than barrier parties.

Real-world examples

Test harnesses releasing N threads together; start-up waiting for caches, DB pools and consumers.

In the interview

> **30-second answer:** "`CountDownLatch` is a one-shot gate that opens after N countDowns; `CyclicBarrier` is a reusable rendezvous where N threads wait for each other each phase, optionally running an action when all arrive."

Self-check

- **Q:** 3 workers syncing at the end of every simulation step? **A:** `CyclicBarrier(3)`.

TL;DR

Latch = one-shot "wait for N events"; barrier = reusable "wait for N threads". `countDown` in `finally`; use timeouts.

Part E — Lock-free

12.15 Compare-and-swap (CAS) & atomics

What it is

CAS is one CPU instruction (`LOCK CMPXCHG` on x86; LL/SC on ARM): "if the value is still `expected`, set it to `new` and succeed; else fail." Java exposes it via `AtomicInteger`, `AtomicLong`, `AtomicBoolean`,

`AtomicReference`, `AtomicIntegerArray`, `LongAdder` and `VarHandle`.

The problem it solves

Atomic updates **without locks**: no blocking, no deadlock, usually faster under low–moderate contention; the basis of the JDK's concurrency utilities.

How it works

Retry loop: read → compute → `compareAndSet(current, next)` → on failure retry. Helpers: `incrementAndGet`, `updateAndGet(fn)`, `accumulateAndGet(x, fn)` (the function may run several times — keep it side-effect-free). `LongAdder` spreads contended updates over cells and sums them (fast hot counters; `sum()` isn't an atomic snapshot). **ABA problem:** A → B → A makes a stale CAS succeed; fix with a version — `AtomicStampedReference` / `AtomicMarkableReference` (GC prevents classic node-reuse ABA for fresh nodes in Java).

Java example

```java
// File: code/lld/ch12/atomics/Main.java
    static int casIncrement(AtomicInteger a) {
        while (true) {
            int current = a.get();                  // 1. read
            int next = current + 1;                 // 2. compute
            if (a.compareAndSet(current, next)) {   // 3. write only if nobody changed it meanwhile
                return next;
            }                                       // 4. someone won the race: retry
        }
    }
```

The file also has a lock-free **Treiber stack** (push/pop swing one `AtomicReference` head with CAS), `LongAdder`, and an ABA demo.

Output: counter `400000`, stack popped `40000`; after A→B→A a plain CAS **succeeds** (`true`) while the stamped CAS **fails** (`false`).

Full runnable code: code/lld/ch12/atomics/ (see the Full Edition).

Trade-offs

	Locks	CAS / atomics
Blocking	Yes	No (retry)
Deadlock	Possible	Impossible
Scope	Any number of variables	One variable per CAS
Heavy contention	Threads sleep	Wasted retries — use `LongAdder`/striping
Difficulty	Moderate	Hard beyond counters/flags/one reference

When not to use it

Multi-variable invariants (moving money between two accounts) — use a lock, or put all fields in one immutable object and CAS its reference (Chapter 13's token bucket).

Common mistakes

`get()` then `set()` (still a race); side effects in `updateAndGet` lambdas; `AtomicLong` for very hot metrics (use `LongAdder`).

Real-world examples

`ConcurrentHashMap` CAS into empty bins; `ConcurrentLinkedQueue` (Michael–Scott); AQS state; JPA `@Version` / `UPDATE ... WHERE version = ?` (CAS in the DB); LMAX Disruptor.

In the interview

> **30-second answer:** "CAS atomically updates a value only if it still equals what I read; otherwise I retry. Java's atomic classes wrap it, so counters, flags and single references can be updated without locks and without deadlock. It covers one variable at a time, can suffer ABA — fixed with a stamped reference — and under heavy contention I prefer `LongAdder`."

Self-check

- **Q:** A failed `compareAndSet` means? **A:** Someone changed the value since you read it; re-read and retry.

TL;DR

CAS = update only if unchanged, retried. Lock-free, deadlock-free, one variable. Watch ABA; `LongAdder` for hot counters.

12.16 Concurrent collections

What it is

Need	Use	Key behaviour
Map	`ConcurrentHashMap`	Lock-free reads; writes lock one bin; atomic `putIfAbsent`, `computeIfAbsent`, `compute`, `merge`; **no nulls**; weakly consistent iterators
Sorted map/set	`ConcurrentSkipListMap` / `Set`	Lock-free, sorted, O(log n)
List read ≫ write	`CopyOnWriteArrayList` / `Set`	Writes copy the array; iteration uses a snapshot
Non-blocking queue	`ConcurrentLinkedQueue` / `Deque`	Lock-free; `poll()` returns null when empty
Blocking queue	`ArrayBlockingQueue`, `LinkedBlockingQueue`, `PriorityBlockingQueue`, `DelayQueue`, `SynchronousQueue`, `LinkedTransferQueue`	`put` waits when full, `take` when empty

Two `ConcurrentHashMap` guarantees to quote: **`computeIfAbsent` / `compute` / `merge` are atomic** — the mapping function runs **exactly once if absent, else not at all** (other updates to that bin block meanwhile, so keep it short and don't touch the same map; `ConcurrentSkipListMap`'s default may call it more than once). **Iterators are weakly consistent** — never throw CME, may or may not show later updates; `size()` is an estimate.

The problem it solves

One lock around a `HashMap` serialises everything; concurrent collections allow parallel access plus **atomic compound operations**.

How it works

Pick by access pattern; replace check-then-act with one call: `putIfAbsent`, `merge(k, 1, Integer::sum)`, `computeIfAbsent(k, ...)`; hand off work with a `BlockingQueue` (12.22). Sequences of calls are still not atomic.

Java example

`collections/Main.java` counts words with `merge(w, 1, Integer::sum)` from 4 threads, creates per-user lists with `computeIfAbsent`, and claims a seat with `putIfAbsent`:

```
word counts (4 threads x 1000 reps): {apple=12000, banana=8000, cherry=4000}
first putIfAbsent returns : null
second putIfAbsent returns: bob  (seat already bob's)
```

Full runnable code: code/lld/ch12/collections/ (see the Full Edition).

Trade-offs

Collection	Great for	Bad for
`ConcurrentHashMap`	Shared maps, caches, counters	Multi-key atomicity; `size()` estimate
`CopyOnWriteArrayList`	Listeners, rarely changed config	Frequent writes (O(n) copy)
`Collections.synchronizedMap`	Legacy, low contention	High contention; manual iteration locking
`ArrayBlockingQueue`	Bounded hand-off, back-pressure	Very high throughput (one lock)
`LinkedBlockingQueue`	Higher throughput (two locks)	Default is **unbounded**

When not to use it

Thread-confined objects (plain collections are faster); multi-key transactions (one lock or a DB).

Common mistakes

Slow work or same-map calls inside `computeIfAbsent` (bin locked; can throw `IllegalStateException` or deadlock); `null` in a `ConcurrentHashMap` → NPE.

Real-world examples

Caffeine on `ConcurrentHashMap`; Guava `EventBus` subscribers in a `CopyOnWriteArraySet`.

In the interview

30-second answer: "I use `ConcurrentHashMap` for shared maps and its atomic methods — `putIfAbsent`, `computeIfAbsent`, `merge` — instead of check-then-act. `CopyOnWriteArrayList` for read-mostly lists like listeners. `BlockingQueue`, preferably bounded, to hand work between threads."

Self-check

- **Q:** Atomic word count? **A:** `map.merge(word, 1, Integer::sum)`.

TL;DR

CHM + atomic methods; COW for read-mostly lists; bounded `BlockingQueue` for hand-offs. Single calls atomic, sequences not.

Part F — Liveness failures

12.17 Deadlock

What it is

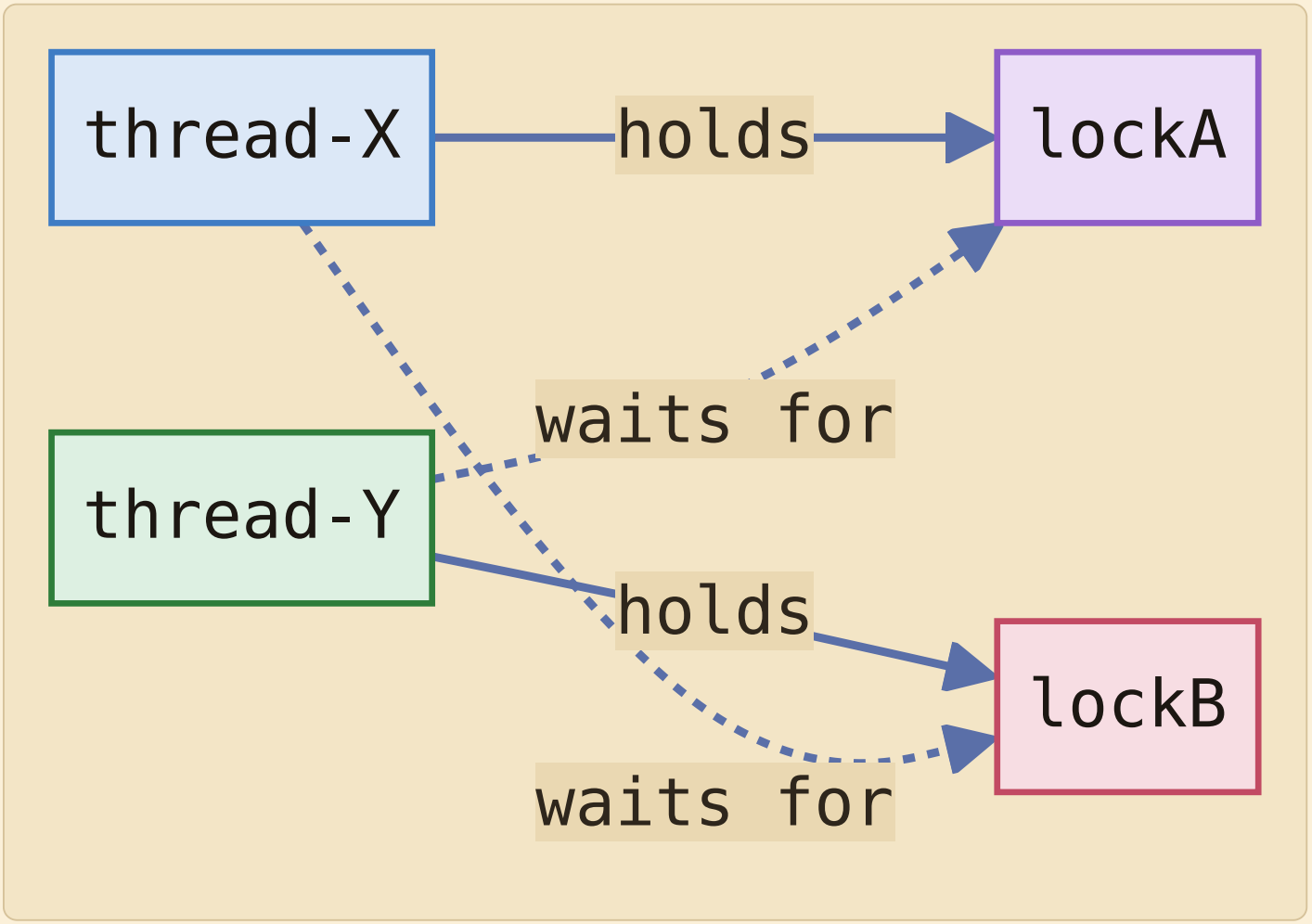
Two or more threads each wait forever for a resource another holds.

The problem it solves

Understanding it lets you *design it out*. All four **Coffman conditions** (1971) are required:

#	Condition	Meaning	How to break it
1	**Mutual exclusion**	One holder per resource	Lock-free/immutable data, shareable read locks
2	**Hold and wait**	Holding one while waiting for another	Acquire all at once, or `tryLock` and release on failure
3	**No preemption**	Only the holder releases	Timeouts, expiring leases
4	**Circular wait**	T1 waits for T2, T2 for T1	**Global lock ordering**

Break **any one** and deadlock is impossible.



How it works

1. **Lock ordering** (most common): a total order (account id, seat id, `identityHashCode` + tie-breaker lock).
2. **Try-lock**: `tryLock(timeout)`; on failure release all, random back-off, retry.
3. **Avoid nested locks**: call alien code (listeners) outside your lock.
4. **Detection**: `jstack` / `jcmd` print "Found one Java-level deadlock"; `ThreadMXBean.findDeadlockedThreads()`.
5. **Avoidance** (Banker's algorithm) — OS theory, rarely used in apps.

Java example

```java
// File: code/lld/ch12/deadlock/Main.java
    static void transferOrdered(Account from, Account to, int amount) {
        Account first = from.id < to.id ? from : to;
        Account second = from.id < to.id ? to : from;
        first.lock.lock();
        try {
            second.lock.lock();
            try {
                if (from.balance >= amount) { from.balance -= amount; to.balance += amount; }
            } finally { second.lock.unlock(); }
        } finally { first.lock.unlock(); }
    }
```

`transferTryLock` instead `tryLock`s both with 1 ms timeouts, releases on failure, and sleeps a random back-off.

`demonstrateDeadlock` forces opposite-order locking with a barrier and detects it:

```
deadlock detected between: [thread-X, thread-Y]  (states BLOCKED, BLOCKED)
lock ordering: finished, total money = 2000
tryLock + back-off: finished, total money = 2000
```

Full runnable code: code/lld/ch12/deadlock/ (see the Full Edition).

Trade-offs

Strategy	Pros	Cons
Lock ordering	Simple, zero cost	Needs a stable global order
Try-lock + back-off	No order needed; bounded waits	Retry logic; livelock without randomness
Single coarse lock	No deadlock among your locks	Lower throughput
Lock-free / immutable	No locks	Harder design

When not to use it

N/A — always prevent deadlock; prefer simple lock ordering to detection/recovery in app code.

Common mistakes

Calling listeners or another object's synchronized method under your lock; ordering by `hashCode()` without tie handling; **thread-starvation deadlock** — pool tasks that submit sub-tasks to the *same* pool and wait.

Real-world examples

InnoDB/PostgreSQL detect deadlocks and abort a victim; bank transfers lock accounts in id order.

In the interview

30-second answer: "Deadlock needs four conditions: mutual exclusion, hold-and-wait, no preemption, and circular wait. I break circular wait with a global lock order — e.g. lock seats or accounts in ascending id — or break hold-and-wait with tryLock with timeout and release-and-retry. I avoid calling external code while holding a lock, and I'd detect it in production with thread dumps."

Self-check

- **Q:** What does lock ordering break? **A:** Circular wait.

TL;DR

Deadlock = cycle of waiting; needs all 4 Coffman conditions. Lock ordering or tryLock + release. No alien code under a lock.

12.18 Livelock

What it is

Threads keep running and reacting to each other but make **no progress**.

The problem it solves

Recognising it — it often comes *from* deadlock avoidance: two threads `tryLock`, fail, release and retry at the same rhythm.

How it works (and how to fix it)

T1 takes A, fails on B, releases; T2 takes B, fails on A, releases; both retry → forever. **Fixes:** random or exponential back-off, a consistent lock order (no retries needed), or a tie-breaker (older transaction wins).

Java example

`transferTryLock` sleeps a **random** number of nanoseconds after failure; remove it and the threads could fail in lock-step.

Trade-offs

Random back-off makes progress very likely but adds latency; exponential back-off adapts to contention.

When not to use it

Never design retry loops with no randomness or tie-breaker.

Common mistakes

Fixed retry delays for everyone; retrying forever with no limit or alert.

Real-world examples

Ethernet CSMA/CD binary exponential back-off; AWS SDK and Resilience4j retries with **jitter**.

In the interview

> **30-second answer:** "Livelock is when threads keep reacting to each other without progress — like two people side-stepping in a corridor. It often comes from symmetric retry logic; I fix it with randomized or exponential back-off, or remove retries by using a lock order."

Self-check

- **Q:** Thread states in deadlock vs livelock? **A:** Deadlocked threads are blocked/waiting; livelocked ones are `RUNNABLE` but achieve nothing.

TL;DR

Busy but no progress. Break symmetry with jittered back-off or a tie-breaker.

12.19 Starvation (and fairness)

What it is

A thread waits indefinitely because others keep winning, though the system progresses.

The problem it solves

Recognising the causes: barging locks under contention, writers behind endless readers, low priorities, a long task hogging a single-thread executor.

How it works (fixes)

Fair locks/semaphores where it matters (`new ReentrantLock(true)`, `new Semaphore(n, true)`, `new ReentrantReadWriteLock(true)`); short critical sections; separate pools for slow and fast work (bulkheads); don't rely on `Thread.setPriority` (OS-dependent).

Java example

Chapter 13's Dining Philosophers `Waiter` uses `new Semaphore(n - 1, true)`.

Trade-offs

Fairness prevents starvation but reduces throughput.

When not to use it

Don't make every lock fair "just in case".

Common mistakes

Confusing it with deadlock: in starvation *someone* progresses; in deadlock nobody does.

Real-world examples

OS schedulers' *aging*; Linux CFS and EEVDF (kernel 6.6+) fair CPU shares.

In the interview

> **30-second answer:** "Starvation is when a thread never gets a resource because others always win — e.g. writers behind endless readers. Fair locks, short critical sections and separate pools fix it, at some throughput cost."

Self-check

- **Q:** Fix for writer starvation in `ReentrantReadWriteLock`? **A:** Fair mode (`new ReentrantReadWriteLock(true)`).

TL;DR

Some thread never gets a turn. Fairness, short sections, separate pools.

Part G — Concurrency patterns

The next five patterns share one runnable file, `patterns/Main.java`. Full runnable code: code/lld/ch12/patterns/ (see the Full Edition).

12.20 Signaling pattern

What it is

One thread **notifies** another that an event happened: "B must run after A".

The problem it solves

Ordering between threads without busy-waiting on a flag.

How it works

Create a signal in the "not yet" state; the waiter blocks on it; the signaller releases it; the waiter continues and (via happens-before) sees everything A did.

Tool	Reusable	Carries a value	Good for
`Semaphore(0)`	Yes	No	Ping-pong (FooBar, Ch 13)
`CountDownLatch(1)`	No	No	One-time "go"
`CompletableFuture`	No	Yes	"Here is the result"
Condition variable	Yes	Via shared state	Complex conditions

Java example

`signaling()`: B starts first but blocks on `aDone.acquire()` (a `Semaphore(0)`); A logs `charge payment` and calls `release()`, so A's line always prints first.

Trade-offs

Semaphores are compact but easy to mis-count; latches can't reset; futures are clean but single-use.

When not to use it

If the signal is really "here's a work item", use a `BlockingQueue`.

Common mistakes

A tool that *forgets* signals (bare `notify()` before the waiter waits = lost wakeup); semaphores and latches remember.

Real-world examples

`CompletableFuture` chains in WebFlux/Netty callbacks; start-up gates; "order paid → send confirmation".

In the interview

"Signaling means one thread waits for an event raised by another. I use a zero-permit semaphore for repeated hand-offs, a latch for one-time events, or a `CompletableFuture` when a result travels with the signal."

Self-check

Q: Why is bare `notify()` risky? **A:** If it happens before the wait, it's lost; pair it with a state variable checked in a loop.

TL;DR

"You may go now": Semaphore(0), latch, or future — make signals unlosable.

12.21 Thread pool pattern

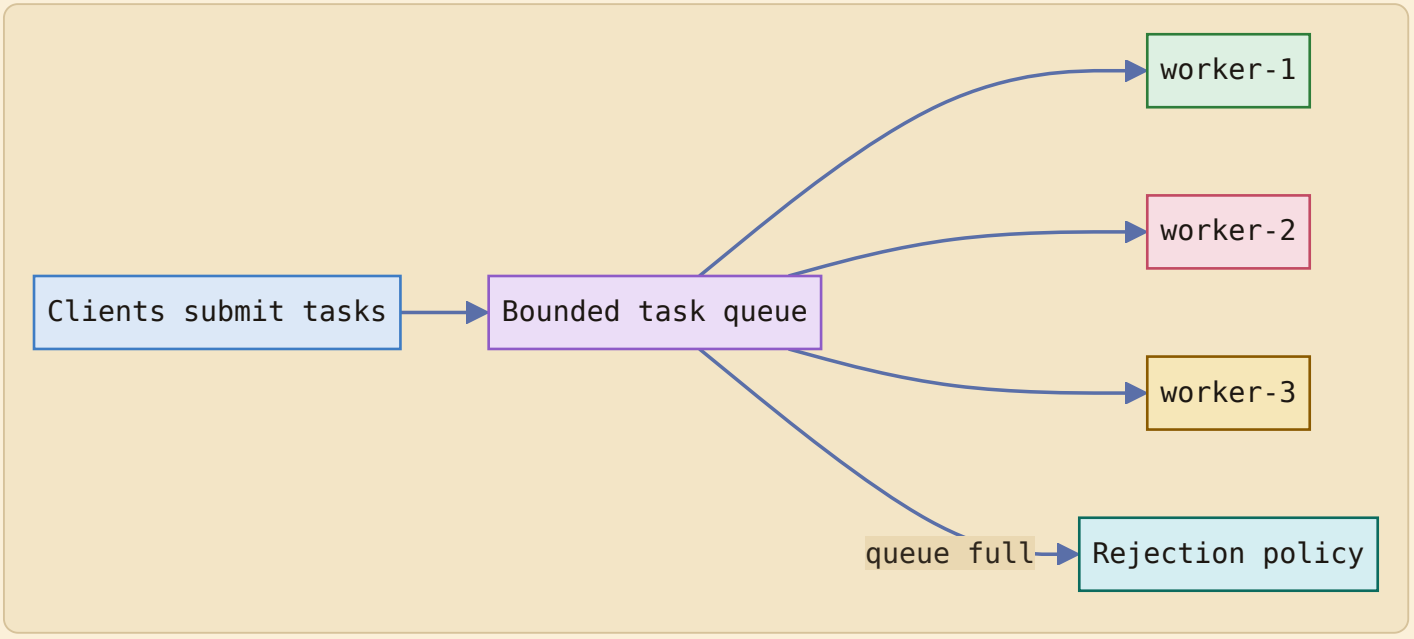
What it is

A fixed set of **worker threads** takes tasks from a shared **queue**, reusing threads.

The problem it solves

Per-task thread creation is expensive, and unbounded concurrency overwhelms CPU, memory and downstreams.

How it works



Workers loop `queue.take(); task.run()`. **`ThreadPoolExecutor` rules:** below **core** threads → new thread per task; then the **queue**; queue full → extra threads up to **max**; still full → **rejection policy** (`AbortPolicy` default throws, `CallerRunsPolicy` runs on the submitter, `DiscardPolicy`, `DiscardOldestPolicy`). Shutdown drains the queue (one **poison pill** per worker in the hand-made pool); `submit`'s "shut down? else enqueue" is check-then-act, so it shares a lock with pill-adding or a task could land behind the pills and never run. **Sizing:** CPU-bound ≈ cores; I/O-bound ≈ cores × (1 + wait/compute), or virtual threads.

Java example

```
// File: code/lld/ch12/patterns/Main.java
        ThreadPoolExecutor exec = new ThreadPoolExecutor(
                2, 4,                                   // core and max threads
                30, TimeUnit.SECONDS,                   // idle time before extra threads die
                new ArrayBlockingQueue<>(100),          // bounded: never an OOM from a huge backlog
                new ThreadPoolExecutor.CallerRunsPolicy()); // when full, the submitter runs it (back-
pressure)
```

(`SimpleThreadPool` in the same file is a hand-made pool showing the mechanics.)

Trade-offs

Choice	Effect
Bounded queue	Protects memory; needs a rejection policy
Unbounded (`newFixedThreadPool`)	Never rejects, may OOM; max size never used
`CallerRunsPolicy`	Natural back-pressure
Many pools (bulkheads)	Isolation; more threads overall

When not to use it

Huge numbers of blocking tasks on Java 21 (virtual-thread-per-task); recursive CPU work (`ForkJoinPool`, work stealing).

Common mistakes

Tasks waiting on tasks in the same bounded pool (thread-starvation deadlock); swallowed exceptions; no `shutdown()`.

Real-world examples

Tomcat `maxThreads` (default 200), Spring `ThreadPoolTaskExecutor`, DB connection pools, `ForkJoinPool.commonPool()`.

In the interview

"A thread pool reuses a bounded set of workers fed by a queue. I size it by workload type, use a bounded queue with an explicit rejection policy for back-pressure, and give slow dependencies their own pool as a bulkhead."

Self-check

Q: When are threads beyond `corePoolSize` created? **A:** Only when the queue is full (and below max).

TL;DR

Workers + queue + rejection policy. Bound everything; size by CPU vs I/O.

12.22 Producer-consumer pattern

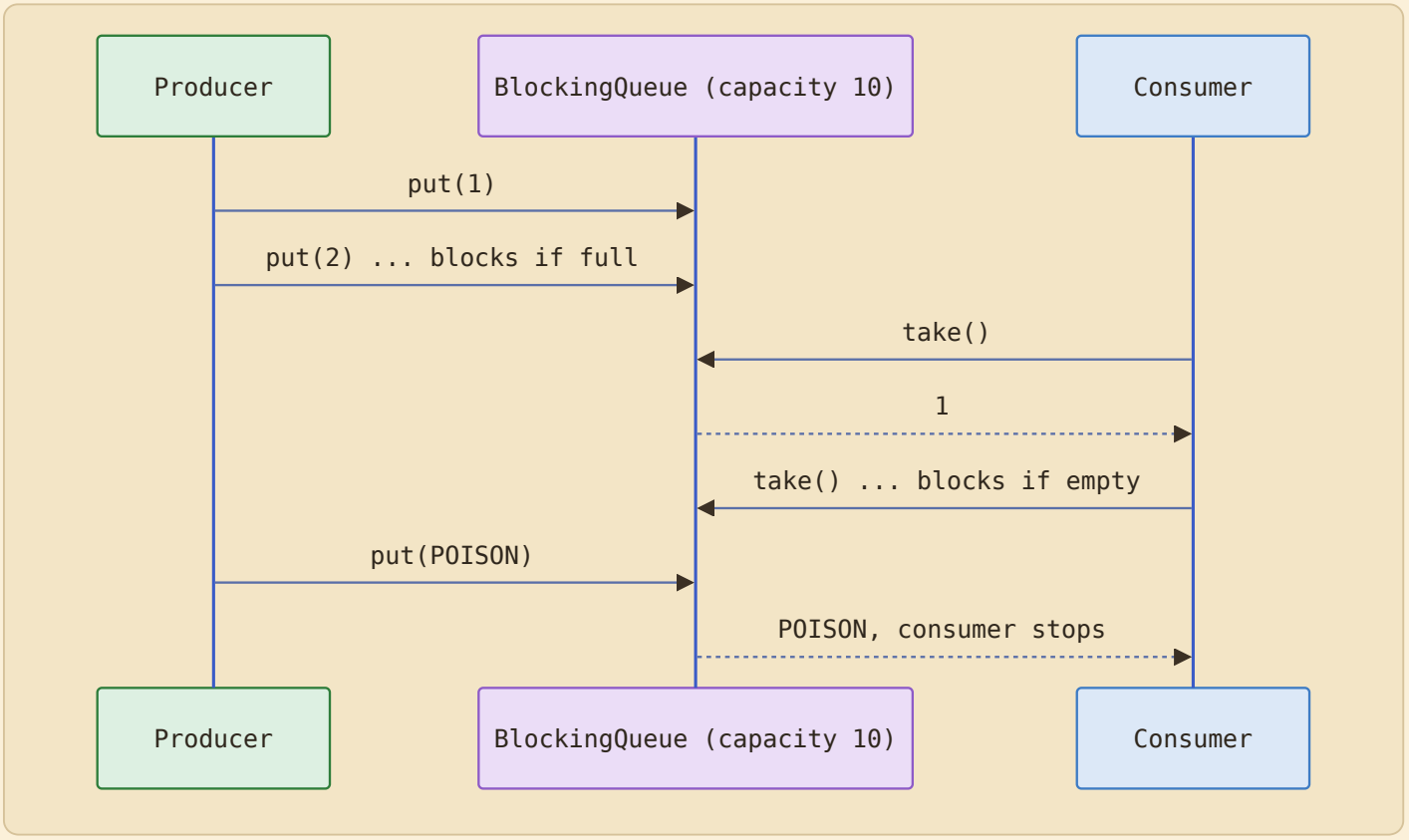
What it is

Producers put items in a shared **buffer**; **consumers** take them out; the buffer decouples their speeds.

The problem it solves

Different rates: a bounded buffer smooths bursts and applies **back-pressure**, with neither side knowing the other.

How it works



Share a bounded `BlockingQueue`; `put` waits when full, `take` when empty. Shutdown: a **poison pill per consumer**, interruption, or a done flag + `poll(timeout)`.

Java example

`producerConsumer()`: 1 producer, 2 consumers, `ArrayBlockingQueue<>(10)`, one `POISON` per consumer; the sum is exactly `500500`. Chapter 13 builds the queue itself two ways.

Trade-offs

Pros	Cons
Decouples rates; back-pressure; easy to scale consumers	Queueing latency; no ordering across consumers; needs a shutdown protocol

When not to use it

When the caller needs the result immediately — call directly.

Common mistakes

Unbounded buffer (memory blow-up); one poison pill for N consumers (N−1 hang); busy-polling `poll()` instead of `take()`.

Real-world examples

Kafka across machines (topics = buffer; consumer groups scale), and inside its Java producer (accumulator + sender thread).

In the interview

"Producer-consumer decouples producers from consumers with a bounded blocking queue: put waits when full, take waits when empty, giving natural back-pressure. I stop consumers with one poison pill each, and scale consumers independently."

Self-check

Q: Why bounded? **A:** So a slow consumer pushes back instead of the queue growing until memory runs out.

TL;DR

Bounded `BlockingQueue` between them; back-pressure for free; one poison pill per consumer.

12.23 Reader-writer pattern

What it is

Many readers may access shared data together; a writer needs exclusive access.

The problem it solves

Read-mostly data (config, catalogues, routing tables) shouldn't be serialised through one mutex.

How it works

(1) **Read-write lock** (12.10), fair if writers could starve. (2) **Copy-on-write snapshot**: an immutable object in a `volatile` field; readers just read it; a writer builds a new copy and swaps the reference.

```
private volatile Map<String, String> config = Map.of();       // immutable snapshot
String get(String k) { return config.get(k); }                  // readers: no lock
synchronized void put(String k, String v) {                      // writers: serialized
    Map<String, String> copy = new HashMap<>(config);
    copy.put(k, v);
    config = Map.copyOf(copy);                                   // publish new snapshot
}
```

	RW lock	Copy-on-write snapshot
Reader cost	Shared lock	One volatile read
Writer cost	Exclusive lock, in-place	Copy whole structure
Readers see	Latest data	Consistent, maybe slightly old snapshot
Best for	Frequent-ish writes, big data	Rare writes, small–medium data

Java example

`ConfigStore` (RW lock) in `locks/Main.java`; the snapshot above; `CopyOnWriteArrayList` in `collections/Main.java`.

Trade-offs

See the table; RW-lock readers must not upgrade.

When not to use it

Write-heavy data — a mutex or `ConcurrentHashMap`.

Common mistakes

Mutating the shared snapshot in place; read → write upgrades.

Real-world examples

Feature-flag/config clients swapping immutable snapshots; Linux RCU (read-copy-update).

In the interview

"For read-heavy data I use either a read-write lock or, if writes are rare, an immutable snapshot in a volatile field that writers replace — readers never block."

Self-check

Q: What guarantees readers see a fully built snapshot? **A:** The volatile write HB later volatile reads, and the snapshot is immutable.

TL;DR

Many readers, one writer: RW lock, or immutable snapshot + volatile swap.

12.24 Immutability & thread confinement

What it is

Thread safety **without synchronization**. **Immutability**: state can't change after construction (`String`, `LocalDate`, `BigDecimal`, records of immutables, `List.of`). **Thread confinement**: only one thread touches the data (locals, `ThreadLocal`, a single owner thread such as an event loop or actor).

The problem it solves

Races need *shared mutable* state — remove "mutable" or remove "shared".

How it works

Immutable checklist: `final` class or `record`; `private final` fields; no setters (return new objects, `withItem`); defensive-copy mutable inputs (`List.copyOf`) and never expose internals; don't leak `this`. **Confinement:** locals; `ThreadLocal.withInitial(...)` (call `remove()` in pools); a single-writer thread fed by a queue.

Java example

```
// File: code/lld/ch12/patterns/Main.java
    record Order(String id, List<String> items) {
        Order { items = List.copyOf(items); }                  // caller's list can't change ours
        Order withItem(String item) {                           // "modify" = return a new object
            List<String> copy = new ArrayList<>(items); copy.add(item);
            return new Order(id, copy);
        }
    }

    // ---------- 5. Thread confinement ----------
    private static final ThreadLocal<StringBuilder> BUFFER = ThreadLocal.withInitial(StringBuilder::new);
```

The caller's later `mutable.add("hacked")` doesn't affect `o1` (output `o1 = [pen], o2 = [pen, book]`).

Trade-offs

	Immutability	Confinement
Cost	New object per change (GC)	Per-thread copies; `ThreadLocal` leaks if not cleaned
Sharing	Share freely	Not shareable
Reasoning	Easiest	Easy, if truly enforced

When not to use it

Very large objects changed very often — use a concurrent structure.

Common mistakes

`final List` that is mutable and exposed; `ThreadLocal` in pools without `remove()` (and heavy `ThreadLocal`s with many virtual threads — **scoped values** are the alternative, preview in 21, final in Java 25 via JEP 506); sharing `SimpleDateFormat` instead of `DateTimeFormatter`.

Real-world examples

`java.time` and `DateTimeFormatter`; Netty/Vert.x event-loop confinement; Redis's single main thread; Spring `@Transactional` state in `ThreadLocal`s.

In the interview

"The easiest thread-safety is none needed: make value objects immutable — final fields, defensive copies, 'with' methods — and confine mutable state to one thread with locals, ThreadLocal, or a single owner thread fed by a queue."

Self-check

Q: Is a class with `private final Date created` immutable? **A:** Not unless it copies the mutable `Date` in and out — prefer `Instant`.

TL;DR

No shared mutable state → no races. Immutable objects share freely; confined ones aren't shared.

Part H — Concurrency in LLD interviews

12.25 How concurrency shows up in LLD interviews (parking, seat booking, inventory)

What it is

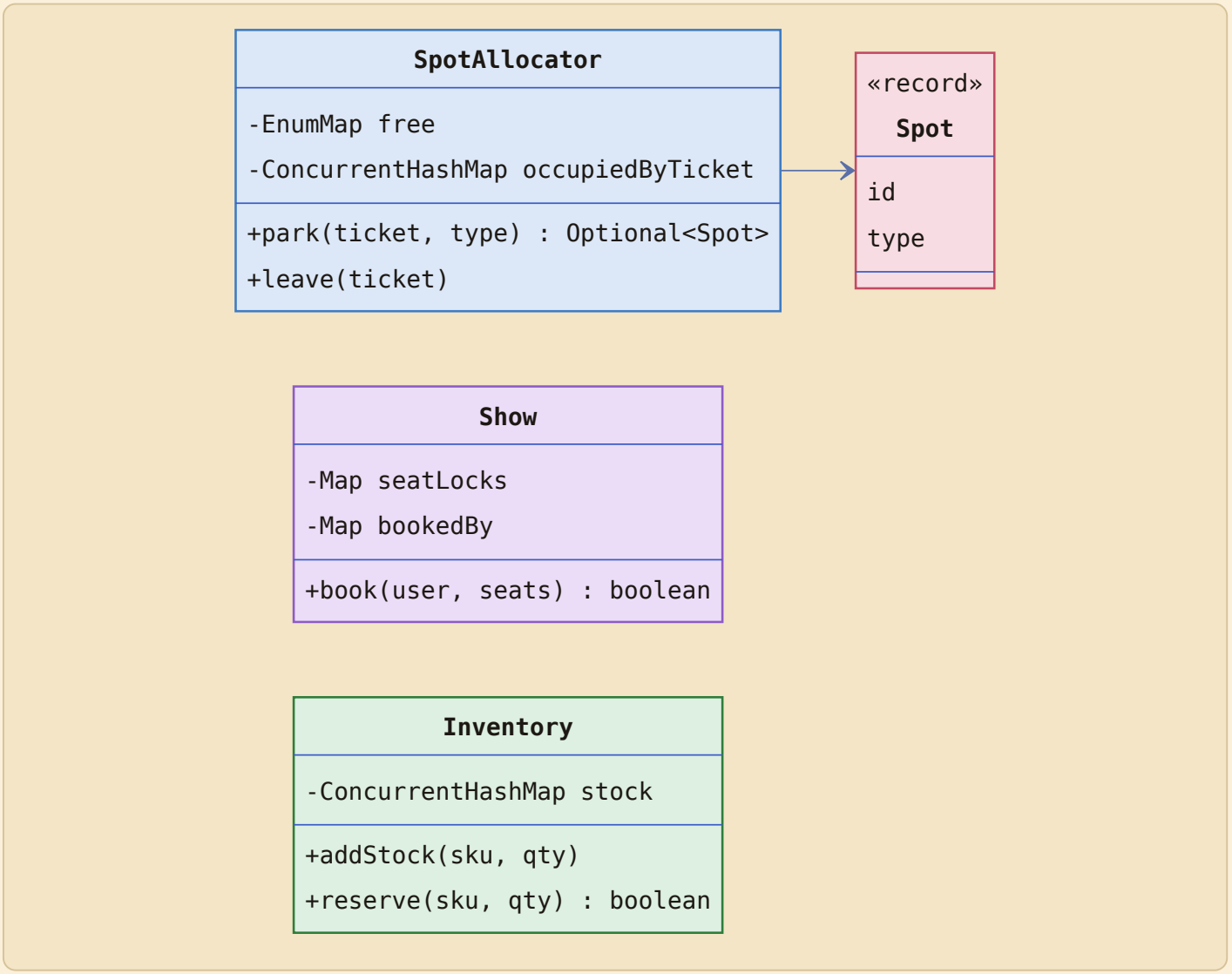
Most case studies have a **contended resource** — parking spot, seat, room, limited stock, wallet balance. The interviewer checks you prevent double allocation and still perform.

The problem it solves

Turning a single-threaded class diagram into a **thread-safe design** with the smallest correct synchronization.

How it works — a 6-step recipe

1. **Name the shared mutable state** (free spots, seat → owner, stock).
2. **State the invariant** ("a seat has at most one booking", "stock ≥ 0").
3. **Find the critical operation** (usually check-then-act: *find free → assign*).
4. **Pick the smallest atomic tool:** one item → `queue.poll()`, `putIfAbsent`, `compareAndSet`, CAS loop; several items all-or-nothing → per-item locks **in sorted order** (or a per-show/per-floor lock); across servers → DB constraints/transactions, optimistic versions, distributed locks.
5. **Choose granularity:** per floor/show/SKU, not global.
6. **Handle time:** holds with expiry (seat held 10 min during payment), timeouts, idempotent retries.



Java example

```java
// File: code/lld/ch12/lldcases/Main.java
        /** Locks seats in sorted order (prevents deadlock), checks all, then books all. */
        boolean book(String user, Set<String> seats) {
            List<ReentrantLock> held = new ArrayList<>();
            try {
                for (String id : new TreeSet<>(seats)) {         // global order: A1 < A2 < A3
                    ReentrantLock l = seatLocks.get(id);
                    if (l == null) return false;                  // unknown seat
                    l.lock();
                    held.add(l);
                }
                for (String id : seats) if (bookedBy.containsKey(id)) return false;   // someone got one first
                for (String id : seats) bookedBy.put(id, user);
                return true;
            } finally {
                for (int i = held.size() - 1; i >= 0; i--) held.get(i).unlock();
            }
        }
```

Parking keeps free spots in a `ConcurrentLinkedQueue` per type — `poll()` is an atomic claim, and `leave` uses `ConcurrentHashMap.remove` so a double exit is harmless. **Inventory** uses an `AtomicInteger` per SKU with a CAS loop that decrements only if `cur >= qty`. `raceAll` releases all tasks via a `CountDownLatch`. Output:

```
parking: 100 cars raced for 10 spots -> parked 10, distinct spots 10, free left 0
booking: 50 users wanted seat A2 -> successful bookings 1
booking: alice {A3,A4} vs bob {A4,A3} (opposite order) -> winners 1, A3 and A4 same owner: true
inventory: 200 buyers, 50 in stock -> sold 50, left 0
```

Full runnable code: code/lld/ch12/lldcases/ (see the Full Edition).

Trade-offs

Resource	In-memory	Across many servers
Parking spot	Lock-free queue per type/floor, or per-floor lock	`UPDATE ... WHERE status='FREE'` + check affected rows
Seat	Per-show/per-seat locks, sorted; hold with TTL	Unique constraint (show, seat); Redis holds (`SET NX PX`)
Inventory	`AtomicInteger` CAS / `compute`	`UPDATE stock SET qty = qty - 1 WHERE sku = ? AND qty >= 1`; expiring reservations

When not to use it

Don't bolt `synchronized` onto every method — protect only operations on contended state.

Common mistakes

A global `synchronized park()` (serialises all gates); availability check and booking as separate calls; locking seats in click order (deadlock).

Real-world examples

BookMyShow/Ticketmaster seat holds that expire (a lock with a lease); flash-sale stock decremented atomically (Redis `DECR`/Lua, conditional SQL).

In the interview

30-second answer: "The critical section is 'find a free X and assign it'. Inside one JVM I make that a single atomic step — a lock-free queue poll for spots, per-seat locks taken in sorted order for multi-

seat booking, a CAS decrement for stock. Across servers I'd push it into the database with a conditional update or unique constraint, and add holds with expiry for seats during payment."

Self-check

- **Q:** Which Coffman condition does sorting seat ids break? **A:** Circular wait.

TL;DR

Find the contended resource and invariant; make "check + claim" one atomic step with the smallest tool; lock fine-grained and in order; use DB/distributed mechanisms across servers.

12.26 Choosing the right tool (cheat sheet)

I need to…	Use
Increment a shared counter	`AtomicInteger` / `AtomicLong`; `LongAdder` if very hot
Stop a loop from another thread	`volatile boolean` or `interrupt()`
Make a multi-step update atomic	`synchronized` or `ReentrantLock`
Give up if the lock is busy	`tryLock(timeout)`
Many readers, few writers	RW lock, `StampedLock`, or volatile immutable snapshot
Limit concurrent access to N	`Semaphore(N)`
Wait for a condition	`Lock` + `Condition` (or `wait/notifyAll`) in a `while`
Wait for N events once	`CountDownLatch`
N threads meet each phase	`CyclicBarrier` / `Phaser`
Hand work between threads	Bounded `BlockingQueue`
Shared map	`ConcurrentHashMap` + `putIfAbsent` / `compute` / `merge`
Read-mostly list	`CopyOnWriteArrayList`
Run tasks	Bounded `ExecutorService`; virtual threads for blocking I/O
Split CPU work	`ForkJoinPool` / parallel streams
Combine async results	`CompletableFuture`
Avoid synchronization	Immutability, thread confinement

12.27 Chapter self-check

1. **Q:** Six `Thread.State` values? **A:** NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED.
2. **Q:** Why isn't `volatile` enough for `count++`? **A:** Visibility, not atomicity.
3. **Q:** Why `while` around `wait()`? **A:** Spurious wakeups and competing threads.
4. **Q:** Four Coffman conditions? **A:** Mutual exclusion, hold and wait, no preemption, circular wait.
5. **Q:** Deadlock vs livelock vs starvation? **A:** All wait / all run but none progress / some progress, one never does.
6. **Q:** Lock-free spot allocation across gates? **A:** Free spots in a concurrent queue; `poll()` atomically.

12.28 Chapter TL;DR

- Races come from **shared mutable state**: make critical sections atomic, or remove sharing/mutability.

- Visibility needs **happens-before**: locks, `volatile`, start/join, concurrent utilities.
- Toolbox: `synchronized` → `ReentrantLock` → RW locks → semaphores → latches/barriers → atomics/CAS → concurrent collections.
- Liveness: **lock ordering** or **tryLock + back-off** against deadlock; jitter against livelock; fairness against starvation.
- Patterns: signaling, thread pool, producer-consumer, reader-writer, immutability/confinement.
- Interviews: find the contended resource, keep "check + claim" atomic, lock fine-grained and in order, explain the multi-server version.

Chapter 13 — Classic Concurrency Problems · Study Edition

> **Study edition.** All 11 problems with their core solution code, padding removed. Demo `main` methods, helpers and full outputs are in the Full Edition (`README.md`).
>
> **Goal.** Chapter 12 gave you the toolbox; this chapter uses it on the problems interviewers ask — LeetCode's concurrency set (FooBar, Zero-Even-Odd, FizzBuzz, H2O, Dining Philosophers) and "design a thread-safe X" rounds (TTL cache, concurrent hash map, blocking queue, Bloom filter, parallel merge sort, rate limiter).

Layout per problem: *Problem → Approach → Java solution → Why it's correct → Pitfalls → Real-world examples → In the interview → Self-check → TL;DR.* Code: Java 21 in `code/lld/ch13/<problem>/Main.java`. Where event order varies, demos *verify* the result and print `true/false`.

```
javac --release 21 -d /tmp/lld_13 $(find . -name '*.java')
java -cp /tmp/lld_13 lld.ch13.foobar.Main
```

Problem map — which tool solves what

#	Problem	Core idea	Main tools
13.1	Print FooBar Alternately	Pass a "baton" between 2 threads	`Semaphore` ×2 (or monitor + turn flag)
13.2	Print Zero Even Odd	A dispatcher thread decides who goes next	`Semaphore` ×3
13.3	Fizz Buzz Multithreaded	4 threads, one counter, each waits for "its" numbers	`synchronized` + `wait/notifyAll`
13.4	Building H2O	Group threads into 2 H + 1 O	`Semaphore` (slots) + `CyclicBarrier`
13.5	Dining Philosophers	Avoid deadlock over shared forks	Lock ordering, or a "waiter" semaphore
13.6	Thread-safe cache with TTL	Expiring entries, no stampede	`ConcurrentHashMap`, `compute`, scheduled sweeper
13.7	Concurrent HashMap (simplified)	Lock striping	`ReentrantLock[]`, all-stripe resize
13.8	Bounded Blocking Queue	Wait when full / empty	`wait/notifyAll` **and** `Lock` + 2 `Condition`s
13.9	Concurrent Bloom Filter	Lock-free bit set	`AtomicLongArray` + atomic OR
13.10	Multi-threaded Merge Sort	Divide and conquer on cores	`ForkJoinPool`, `RecursiveAction`
13.11	Rate limiter (token bucket)	Atomic refill-and-take	`synchronized`, or CAS on immutable state

Checklist for any concurrency problem: (1) What is the **shared state**? (2) What **ordering or invariant** must hold? (3) Who **waits**, for **what**, and who **signals**? (4) Pick the primitive: turn-taking → semaphores; "wait until predicate" → condition variable; groups → barrier; mutual exclusion → lock; one variable → atomic. (5) Check **liveness**: can anyone wait forever, does every thread exit?

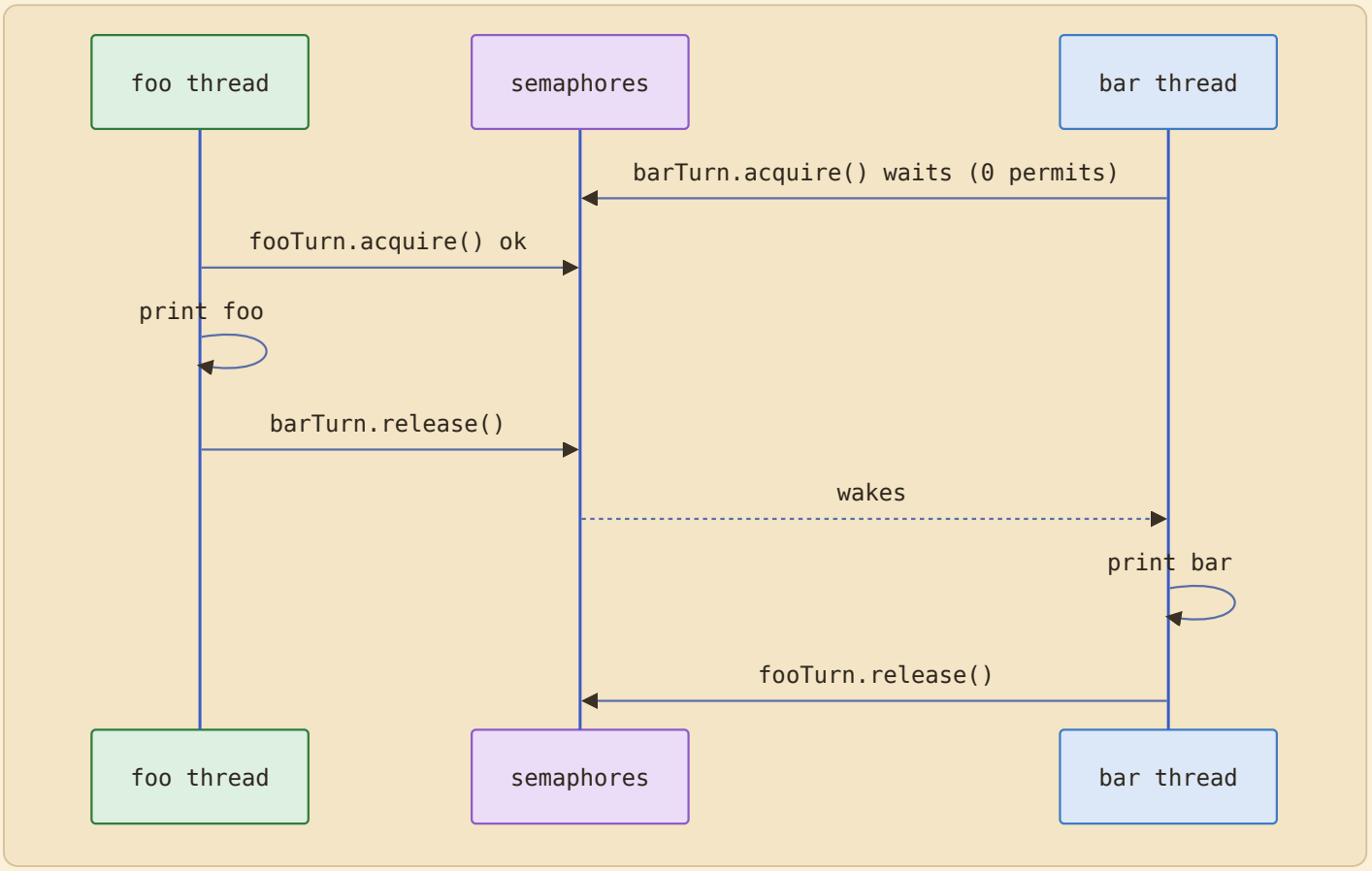
13.1 Print FooBar Alternately (LeetCode 1115)

Problem

Thread A calls `foo()` (prints "foo" *n* times), thread B calls `bar()` (prints "bar" *n* times). Output must be "foobar" × *n*, whichever starts first.

Approach

Pure **signaling** (12.20) — a baton: `fooTurn = Semaphore(1)`, `barTurn = Semaphore(0)`; each thread acquires its own, prints, releases the other's. Alternative: a monitor with a `fooTurn` flag, `while (!fooTurn) wait();`, flip, `notifyAll()`.



Java solution

```java
// File: code/lld/ch13/foobar/Main.java
    /** Solution 1: two semaphores used as a baton that is passed back and forth. */
    static class FooBarSemaphore {
        private final int n;
        private final Semaphore fooTurn = new Semaphore(1);   // foo may go first
        private final Semaphore barTurn = new Semaphore(0);   // bar must wait

        FooBarSemaphore(int n) { this.n = n; }

        void foo(Runnable printFoo) throws InterruptedException {
            for (int i = 0; i < n; i++) {
                fooTurn.acquire();       // wait for my turn
                printFoo.run();
                barTurn.release();       // hand the baton to bar
            }
        }
        void bar(Runnable printBar) throws InterruptedException {
            for (int i = 0; i < n; i++) {
                barTurn.acquire();
                printBar.run();
                fooTurn.release();
            }
        }
    }
    // ... Solution 2 (FooBarMonitor): synchronized + while (!fooTurn) wait(); flip; notifyAll()
```

Output (bar thread started first on purpose): `semaphores : foobarfoobarfoobarfoobar`, `both correct: true`. Full runnable code: code/lld/ch13/foobar/ (see the Full Edition).

Why it's correct

Invariant: the two semaphores hold exactly 1 permit in total, so turns strictly alternate; `release()` happens-before the `acquire()` it enables; matched loops terminate.

Pitfalls

One shared semaphore (a thread re-takes its own release); `if` instead of `while`; busy-waiting on a `volatile` flag.

Real-world examples

Ping-pong protocols, request/acknowledge stages, double-buffering.

In the interview

> **30-second answer:** "Two semaphores as a baton: foo's starts at 1, bar's at 0. Each thread acquires its own, prints, and releases the other's. The total permits are always one, so they alternate."

Self-check

- **Q:** What if `barTurn` starts at 1? **A:** bar may print first.

TL;DR

Two semaphores, one permit total, passed back and forth.

13.2 Print Zero Even Odd (LeetCode 1116)

Problem

Three threads: `zero()` prints only 0, `even()` only evens, `odd()` only odds; together `0 1 0 2 … 0 n` (n = 5 → `0102030405`).

Approach

The zero thread is a **dispatcher**: after the 0 for position *i* it releases `oddTurn` (odd *i*) or `evenTurn`; they print and release `zeroTurn`. Initial permits: zero 1, odd 0, even 0.

Java solution

```java
// File: code/lld/ch13/zeroevenodd/Main.java
        private final Semaphore zeroTurn = new Semaphore(1);
        private final Semaphore oddTurn = new Semaphore(0);
        private final Semaphore evenTurn = new Semaphore(0);
        // ...
        void zero(IntConsumer print) throws InterruptedException {
            for (int i = 1; i <= n; i++) {
                zeroTurn.acquire();
                print.accept(0);
                // The zero thread is the "dispatcher": it decides who prints next.
                if (i % 2 == 1) oddTurn.release(); else evenTurn.release();
            }
        }
        void odd(IntConsumer print) throws InterruptedException {
            for (int i = 1; i <= n; i += 2) {
                oddTurn.acquire();
                print.accept(i);
                zeroTurn.release();
            }
        }
        // ... even() is the same loop from 2 with evenTurn
```

Output: `n=5 -> 0102030405`, `n=10 -> 01020304050607080910`. Full runnable code: code/lld/ch13/zeroevenodd/ (see the Full Edition).

Why it's correct

One permit in the system → one printer at a time; loop counts match permits (zero n, odd $\lceil n/2 \rceil$, even $\lfloor n/2 \rfloor$), so everyone exits.

Pitfalls

Unsynchronized shared counters; loop bounds that don't match permits (a thread hangs).

Real-world examples

Dispatcher/worker designs (opcode routing, alternating schedulers).

In the interview

30-second answer: "Three semaphores. Zero starts with one permit. After each 0 it releases odd or even depending on the index, and they release zero back. One permit total in the system, so the order is forced."

Self-check

- **Q:** For n = 4, how many times does `odd()` acquire? **A:** Twice (1 and 3).

TL;DR

Zero is the dispatcher; one permit circulates among three semaphores.

13.3 Fizz Buzz Multithreaded (LeetCode 1195)

Problem

Four threads output 1..n: multiples of 15 → `fizzbuzz`, of 3 only → `fizz`, of 5 only → `buzz`, else the number — each kind printed by its own thread.

Approach

One shared `current` guarded by the monitor; each thread loops with its own predicate: `while (current <= n && !mine(current)) wait();`, exit (with `notifyAll()`) if `current > n`, else print, `current++`, `notifyAll()`. The **condition-variable** pattern (12.13); `runWhile(predicate, action)` avoids writing the loop four times.

Java solution

```java
// File: code/lld/ch13/fizzbuzz/Main.java
        private int current = 1;                              // guarded by 'this'
        // ...
        /** Shared loop: wait until 'current' is mine (or we're done), act, advance, wake everyone. */
        private void runWhile(IntPredicate mine, IntConsumer action) throws InterruptedException {
            while (true) {
                synchronized (this) {
                    while (current <= n && !mine.test(current)) wait();
                    if (current > n) { notifyAll(); return; }     // done: wake others so they can exit too
                    action.accept(current);
                    current++;
                    notifyAll();                                  // the next number may belong to anyone
                }
            }
        }

        void fizz(Runnable printFizz) throws InterruptedException {
            runWhile(i -> i % 3 == 0 && i % 5 != 0, i -> printFizz.run());
        }
        // ... buzz: i % 5 == 0 && i % 3 != 0; fizzbuzz: i % 15 == 0; number: neither
```

Output: `1, 2, fizz, 4, buzz, fizz, 7, 8, fizz, buzz, 11, fizz, 13, 14, fizzbuzz`. Full runnable code: code/lld/ch13/fizzbuzz/ (see the Full Edition).

Why it's correct

Predicates **partition** 1..n; `current` is only touched under the monitor; `notifyAll()` is required because four predicates share one wait set; the final `notifyAll` lets everyone see `current > n` and exit.

Pitfalls

`notify()` (lost wake-ups); no `current > n` exit check; overlapping predicates (two threads claim 15).

Real-world examples

Pipeline stages waiting for a job to reach their stage.

In the interview

30-second answer: "One monitor, one shared counter. Each thread waits in a while loop until the counter matches its predicate or passes n, prints, increments and notifies all. The predicates partition the numbers, so exactly one thread acts per number."

Self-check

- **Q:** Why `notifyAll`? **A:** Threads wait for *different* conditions on one monitor; only waking all guarantees the right one wakes.

TL;DR

Shared counter + per-thread predicate + `while`-wait + `notifyAll`.

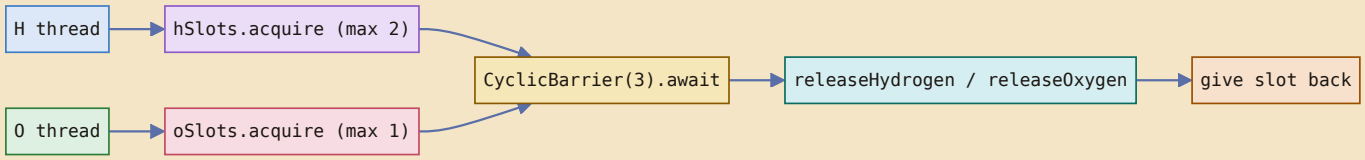
13.4 Building H2O (LeetCode 1117)

Problem

H and O threads arrive in any order and must leave in groups of exactly **2 H + 1 O**; one molecule's threads all pass before the next molecule's.

Approach

Slots — `hSlots = Semaphore(2)`, `oSlots = Semaphore(1)` cap the current molecule. **Group** — `CyclicBarrier(3)` makes the three atoms wait for each other. A thread returns its slot **after** it has been released, so the next molecule forms only when this one is done.



Java solution

```java
// File: code/lld/ch13/h2o/Main.java
    static class H2O {
        private final Semaphore hSlots = new Semaphore(2);    // at most 2 H in the current molecule
        private final Semaphore oSlots = new Semaphore(1);    // at most 1 O in the current molecule
        private final CyclicBarrier bond = new CyclicBarrier(3);   // wait until all 3 atoms are present

        void hydrogen(Runnable releaseHydrogen) throws InterruptedException {
            hSlots.acquire();
            try {
                bond.await();
            } catch (BrokenBarrierException e) { throw new IllegalStateException(e); }
            releaseHydrogen.run();
            hSlots.release();                                 // free my slot only after I have "left"
        }
        // ... oxygen() is identical with oSlots
    }
```

Output: `molecules formed: 8, every group of 3 has 2 H + 1 O: true` (orders `HHO`, `HOH`, `OHH` are all valid, so the demo verifies instead of printing). Full runnable code: code/lld/ch13/h2o/ (see the Full Edition).

Why it's correct

Slots admit ≤2 H and ≤1 O per round and the barrier needs 3 → each trip is H, H, O; slots are freed only after printing, so groups never mix; the barrier resets itself.

Pitfalls

Releasing the slot **before** printing; no barrier; H:O not exactly 2:1 (last threads hang).

Real-world examples

Batching N items of given types (matchmaking, order assembly).

In the interview

30-second answer: "Two semaphores cap the molecule at 2 H and 1 O; a CyclicBarrier of 3 makes the three wait for each other; each atom returns its slot only after it's released, so molecules never interleave."

Self-check

- **Q:** Why is `hSlots` 2, not 1? **A:** A molecule needs two hydrogens inside the barrier at once.

TL;DR

Slots decide *who* can join; the barrier decides *when* they leave; free slots only after leaving.

13.5 Dining Philosophers (LeetCode 1226)

Problem

Five philosophers, five forks, each needs **both** adjacent forks. If everyone takes the left fork first, all may wait forever for the right — **deadlock** (all four Coffman conditions, 12.17).

Approach

Break **circular wait**: (1) **resource ordering** — always take the **lower-numbered** fork first (philosopher 4 takes fork 0 first); (2) **waiter** — a **fair** semaphore lets at most **n − 1** try at once, so someone always gets both forks. (Also: `tryLock` both with random back-off; Chandy–Misra for distributed settings.)

Java solution

```java
// File: code/lld/ch13/philosophers/Main.java
    /** Fix 1 - resource ordering: always pick up the LOWER-numbered fork first (breaks circular wait). */
    static class OrderedForks implements Table {
        private final ReentrantLock[] forks;
        // ... constructor creates n ReentrantLocks

        public void eat(int p, Runnable eatAction) {
            int left = p, right = (p + 1) % forks.length;
            ReentrantLock first = forks[Math.min(left, right)], second = forks[Math.max(left, right)];
            first.lock();
            try {
                second.lock();
                try { eatAction.run(); } finally { second.unlock(); }
            } finally { first.unlock(); }
        }
    }

    /** Fix 2 - a "waiter" lets at most n-1 philosophers try at once (breaks circular wait too). */
    static class Waiter implements Table {
        // ... forks as above, plus:
            seats = new Semaphore(n - 1, true);                // fair: nobody starves waiting for a seat
        // ... eat(): seats.acquire(); lock left, then right; eat; unlock both; seats.release() in finally
    }
```

Output: both strategies finish `[10000, 10000, 10000, 10000, 10000]` meals. Full runnable code: code/lld/ch13/philosophers/ (see the Full Edition).

Why it's correct

Ordering: increasing-index acquisition makes a waiting cycle impossible. **Waiter:** 4 diners, 5 forks → by pigeonhole someone eats. Forks are released in `finally`.

Pitfalls

Left-then-right for all (deadlock); `tryLock` retries without random back-off (livelock); the fair semaphore only makes *seats* FIFO — fair fork locks are needed for strict no-starvation.

Real-world examples

Anything needing two locks: bank transfers, multi-row DB transactions.

In the interview

> **30-second answer:** "Naively everyone takes the left fork and waits for the right — circular wait, deadlock. I break circular wait by always taking the lower-numbered fork first, or by a waiter semaphore allowing only n−1 philosophers to try; fairness prevents starvation."

Self-check

- **Q:** With ordering, who behaves "differently"? **A:** The philosopher between forks 4 and 0 — takes fork 0 (right) first.

TL;DR

Break circular wait: global fork order, or only n−1 diners.

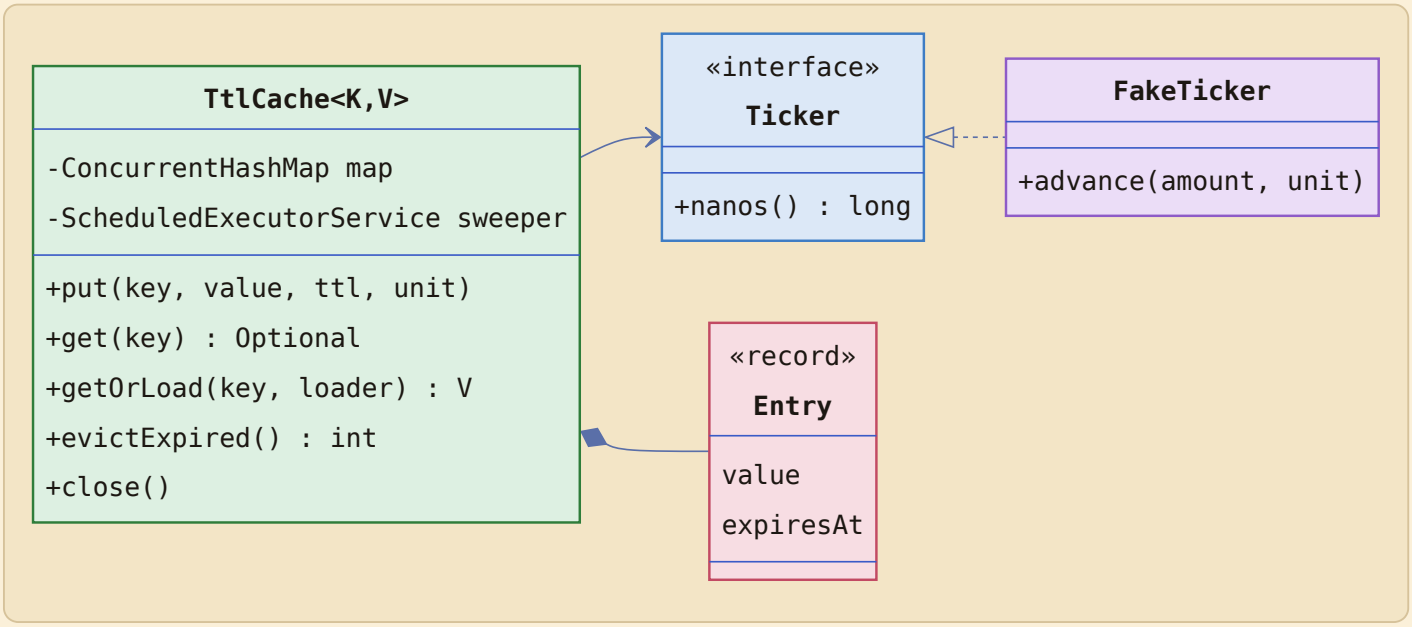
13.6 Thread-safe cache with TTL

Problem

A shared cache whose entries expire after a **TTL**: `put` (default or per-entry TTL), `get` hides expired entries, `getOrLoad` loads a miss **without a cache stampede**, expired entries leave memory, testable without sleeping.

Approach

Immutable `Entry(value, expiresAt)` in a `ConcurrentHashMap`; **lazy expiry** in `get` via conditional `remove(key, entry)`; **active expiry** by a daemon scheduled sweeper; **no stampede** via `map.compute` (loader runs once per key); an injected `Ticker` for tests.



Java solution

```
// File: code/lld/ch13/ttlcache/Main.java
        private record Entry<V>(V value, long expiresAt) { }

        private final ConcurrentHashMap<K, Entry<V>> map = new ConcurrentHashMap<>();
        // ... defaultTtlNanos, ticker, daemon sweeper scheduled at a fixed rate to call evictExpired()

        private boolean expired(Entry<V> e) { return ticker.nanos() - e.expiresAt() >= 0; }   // overflow-safe
compare

        Optional<V> get(K key) {
            Entry<V> e = map.get(key);
            if (e == null) return Optional.empty();
            if (expired(e)) {
                map.remove(key, e);   // conditional remove: never deletes a FRESH value another thread just
put
                return Optional.empty();
            }
            return Optional.of(e.value());
        }

        V getOrLoad(K key, Function<? super K, ? extends V> loader) {
            return map.compute(key, (k, old) ->
                    old != null && !expired(old) ? old : new Entry<>(loader.apply(k), ticker.nanos() +
defaultTtlNanos)
            ).value();
        }
        // ... evictExpired(): for each entry, if expired, map.remove(key, entry) — the same conditional
remove
```

Output: `16 concurrent misses -> loader ran 1 time(s), all got same value: true`; with the real clock the sweeper empties 1,000 entries within 200 ms.

Full runnable code: code/lld/ch13/ttlcache/ (see the Full Edition).

Why it's correct

Immutable entries in one CHM (never half-written); `remove(key, e)` removes only that exact entry; `compute` is atomic per key (one load); `now - expiresAt >= 0` survives `nanoTime` overflow; the sweeper is a daemon closed in `close()`. (Its constructor leaks `this` to the sweeper — safe only because fields are set and executor submission is a happens-before edge; prefer a static factory.)

Pitfalls

Wall-clock `currentTimeMillis()` (use `nanoTime`); slow loaders inside `compute` block the bin (store a `CompletableFuture`); lazy-only expiry leaks, active-only serves stale — use both; no size limit (add LRU/LFU).

Real-world examples

Caffeine (and Guava `CacheBuilder`); **Redis** `EXPIRE` (lazy + periodic); sessions, OTPs, seat holds.

In the interview

30-second answer: "A ConcurrentHashMap of immutable entries holding value and expiry time. Get checks expiry lazily and removes with a conditional remove; a daemon scheduled sweeper removes expired keys nobody reads. getOrLoad uses compute so concurrent misses load once. Time is injected for testing."

Self-check

- **Q:** Why `remove(key, e)` not `remove(key)`? **A:** So a value written between our read and remove isn't deleted.

TL;DR

CHM + immutable (value, expiry) entries; lazy + scheduled expiry; `compute` for single-flight loads; injected clock.

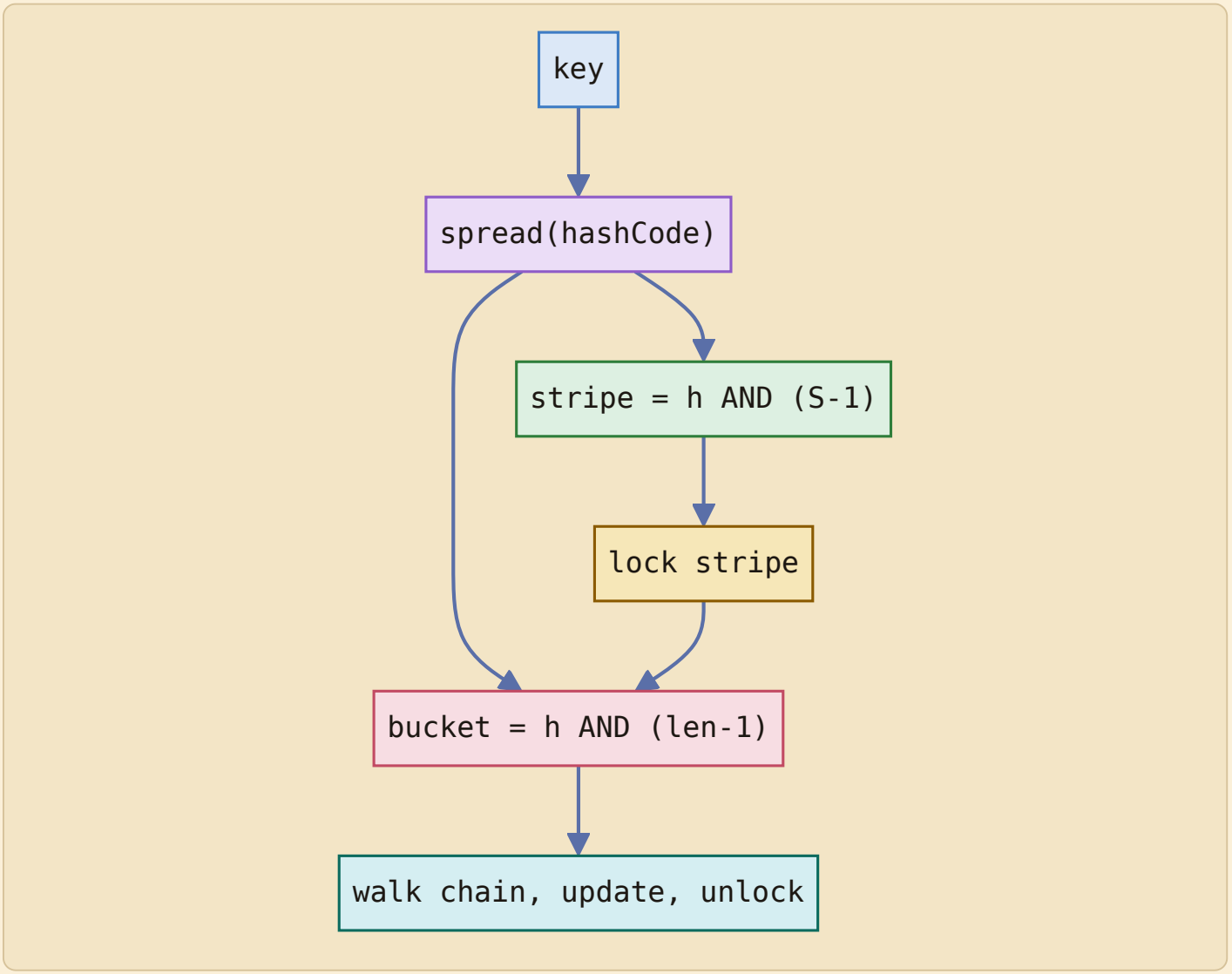
13.7 Design a (simplified) Concurrent HashMap — lock striping

Problem

A hash map with `get`, `put`, `remove`, `compute` and resizing, usable by many threads and faster than one global lock.

Approach

Lock striping (12.11): buckets of linked chains plus `S` locks (power of two). Key hash *h* → stripe `h & (S − 1)`, bucket `h & (len − 1)`; the stripe depends only on the hash, so it **never changes on resize**. Operations lock only their stripe. **Resize** (size > 0.75 × buckets): lock **all** stripes in index order, re-check, rehash into a 2× table, publish via `volatile`, unlock in reverse. Size in an `AtomicInteger`.



Java solution

```
// File: code/lld/ch13/stripedmap/Main.java
        private volatile Node<K, V>[] table;               // replaced (never mutated in place) only while
ALL locks are held
        private final ReentrantLock[] stripes;
        private final AtomicInteger size = new AtomicInteger();
        // ... constructor requires power-of-two sizes and stripes <= buckets

        // A key's stripe depends only on its hash, NOT on the table size, so it never changes on resize.
        private ReentrantLock stripeFor(int hash) { return stripes[hash & (stripes.length - 1)]; }
        // ... get/remove: lock stripeFor(h), walk t[indexFor(h, t.length)], unlock
        // ... put/compute share doCompute(): under the stripe lock find the node, apply remap,
        // ... update / unlink / insert at head; then, AFTER unlocking:
            if (grew) resize();                                    // never resize while holding one stripe

        /** Stop-the-world resize: take every stripe lock IN ORDER (no deadlock), rehash, publish. */
        private void resize() {
            for (ReentrantLock l : stripes) l.lock();
            try {
                Node<K, V>[] old = table;
                if (size.get() <= old.length * LOAD_FACTOR) return;  // another thread already resized
                Node<K, V>[] nt = (Node<K, V>[]) new Node[old.length * 2];
                // ... move every node to nt[indexFor(n.hash, nt.length)]
                table = nt;
            } finally {
                for (int i = stripes.length - 1; i >= 0; i--) stripes[i].unlock();
            }
        }
```

Output: 8 threads × 20,000 puts → `size=160000, missing=0`, table grew 16 → 262144 buckets; 80,000 concurrent `compute` increments are exact.

Full runnable code: code/lld/ch13/stripedmap/ (see the Full Edition).

Why it's correct

Each bucket has exactly one stripe (powers of two, $S \le$ length → stripe index = low bits of bucket index). **Resize** holds all locks, re-checks, publishes via `volatile`. **No deadlock:** one lock normally; resize takes all in order after releasing its own. **Compute** runs wholly under the stripe lock.

Pitfalls

Stripe from the current bucket index (changes on resize); resizing while holding a stripe; unsynchronized `table` reads; `null` keys/values.

How the real `ConcurrentHashMap` goes further

Aspect	This design	JDK 8+ `ConcurrentHashMap`
Reads	Lock the stripe	Lock-free (volatile reads)
Write locking	One of S stripe locks	CAS into empty bins; `synchronized` on the bin head otherwise
Long chains	Linked list	Red-black tree beyond 8 nodes (table ≥ 64)
Resize	Stop-the-world	Incremental; threads help transfer
Size	`AtomicInteger`	`LongAdder`-style counter cells

Real-world examples

Java 7's `ConcurrentHashMap` **segments** (16 by default); Guava `Striped<Lock>`; Caffeine's striped read buffers.

In the interview

> **30-second answer:** "An array of buckets plus a small array of locks; each key maps to a stripe by its hash bits, so different stripes work in parallel. Every bucket belongs to exactly one stripe. Resize takes all stripe locks in order, rehashes and publishes the new table through a volatile field."

Self-check

- **Q:** Why must the stripe count divide the table length? **A:** So every bucket maps to exactly one stripe.

TL;DR

N locks for M buckets, stripe by hash bits, resize under all locks in order, publish via volatile.

13.8 Thread-safe bounded Blocking Queue (two ways)

Problem

A fixed-capacity FIFO for producer-consumer: `put` blocks while full, `take` while empty, plus timed `offer(item, timeout)` and `poll(timeout)`.

Approach

A **circular array** (`head`, `tail`, `count`) and two synchronisation designs:

	Way 1: monitor	Way 2: `ReentrantLock` + 2 `Condition`s
Lock	`synchronized` methods	`lock.lockInterruptibly()`
Wait	`while (full) wait();`	`while (full) notFull.await();`
Wake	`notifyAll()` — one shared wait set	`notEmpty.signal()` / `notFull.signal()`
Timed wait	`TimeUnit.timedWait` + deadline loop	`awaitNanos` returns time left
Efficiency	Wakes everyone (thundering herd)	Wakes one thread of the right kind

Java solution

```java
// File: code/lld/ch13/blockingqueue/Main.java
    // ... Way 1 (MonitorQueue): producers AND consumers share one monitor, so notifyAll():
        public synchronized void put(T item) throws InterruptedException {
            while (count == items.length) wait();
            enqueue(item);
            notifyAll();
        }
    // ...
    // ... Way 2 (ConditionQueue): ReentrantLock with TWO condition queues (how ArrayBlockingQueue is built).
        private final ReentrantLock lock = new ReentrantLock();
        private final Condition notFull = lock.newCondition();
        private final Condition notEmpty = lock.newCondition();

        public void put(T item) throws InterruptedException {
            if (item == null) throw new NullPointerException();
            lock.lockInterruptibly();
            try {
                while (count == items.length) notFull.await();
                enqueue(item);
            } finally { lock.unlock(); }
        }
        // ... take() mirrors put(); timed poll/offer loop on nanos = notEmpty.awaitNanos(nanos) (returns time
left)
        private void enqueue(T item) {
            items[tail] = item;
            tail = (tail + 1) % items.length;
            count++;
            notEmpty.signal();                                  // wake ONE waiting consumer
        }
        @SuppressWarnings("unchecked")
        private T dequeue() {
            T x = (T) items[head];
            items[head] = null;
            head = (head + 1) % items.length;
            count--;
            notFull.signal();                                   // wake ONE waiting producer
            return x;
        }
```

Both pass a stress test (3 producers × 20,000, 3 consumers, capacity 2): `consumed 60000 items, sum ok=true, left in queue=0`; `poll(30ms)` on empty → `null`, `offer(30ms)` on full → `false`. Full runnable code: code/lld/ch13/blockingqueue/ (see the Full Edition).

Why it's correct

One lock guards all fields; `while` loops handle spurious wakeups; Way 1 needs `notifyAll()` (one shared wait set); Way 2 can `signal()` (separate queues per kind); timed waits track *remaining* time.

Pitfalls

`if` instead of `while`; `notify()` on a shared monitor; not nulling `items[head]` (leak); `null` items when `null` means timeout.

Real-world examples

`ArrayBlockingQueue` = Way 2; `LinkedBlockingQueue` uses **two** locks (`putLock`, `takeLock`).

In the interview

30-second answer: "A circular array with head, tail and count guarded by one lock. put waits while count == capacity, take waits while count == 0, both in while loops. With synchronized I must use notifyAll

because both sides share one wait set; with ReentrantLock I use two Conditions, notFull and notEmpty, and signal just the right side."

Self-check

- **Q:** Why is `signal()` safe in Way 2 but `notify()` unsafe in Way 1? **A:** Separate condition queues guarantee the woken thread is the right kind.

TL;DR

Circular buffer + one lock + wait-while-full/empty loops. Monitor → `notifyAll`; Lock → `notFull`/`notEmpty` + `signal`.

13.9 Concurrent Bloom Filter

Problem

A thread-safe **Bloom filter** — a compact probabilistic set answering *"definitely not present"* or *"probably present"* — that many threads add to and query at once.

Approach

m bits; **add** sets *k* hashed positions; **query**: any 0 → **definitely absent**, all 1 → **probably** present (*false positive* possible). No false negatives, no deletion (a *counting* filter allows it).

Sizing for *n* items and false-positive rate *p*: $m = \lceil -n \cdot \ln p / (\ln 2)^2 \rceil$, $k = \text{round}((m / n) \cdot \ln 2)$. For $n = 80{,}000$, $p = 1\% \rightarrow m \approx 766{,}805$ bits (≈ 93.6 KiB), $k = 7$ — about **9.6 bits per item**, whatever the item size.

Double hashing (Kirsch–Mitzenmacher): positions $h1 + i \cdot h2$. **Lock-free:** an `AtomicLongArray` with **atomic OR** (`getAndAccumulate`); queries are volatile reads — bits only go 0 → 1, so a query can at most miss an unfinished `add`.

Java solution

```
// File: code/lld/ch13/bloomfilter/Main.java
        private final AtomicLongArray words;     // m bits packed into 64-bit words
        // ... numBits (m) and numHashes (k) computed from expectedItems and falsePositiveRate

        /** Adds the item. Returns true if at least one bit changed (i.e. it was definitely new). */
        boolean add(String item) {
            long base = fnv1a64(item);
            long h1 = fmix64(base), h2 = fmix64(base ^ 0x9E3779B97F4A7C15L) | 1;   // h2 odd
            boolean changed = false;
            for (int i = 0; i < numHashes; i++) {
                int bit = (int) Math.floorMod(h1 + i * h2, (long) numBits);        // double hashing
                long mask = 1L << (bit & 63);
                long before = words.getAndAccumulate(bit >>> 6, mask, (cur, m) -> cur | m);  // atomic OR (CAS
loop)
                if ((before & mask) == 0) changed = true;
            }
            return changed;
        }
        // ... mightContain(): same k positions; any bit == 0 -> return false (definitely absent)
        // ... fnv1a64 (FNV-1a) and fmix64 (MurmurHash3 finalizer) mix the bits
```

Output: `m=766805 bits (93 KiB), k=7`; after an 8-thread insert, `false negatives: 0` and a measured false-positive rate of `1.01%` (target 1%).

Full runnable code: code/lld/ch13/bloomfilter/ (see the Full Edition).

Why it's correct

A plain `|=` can lose bits (false negatives); atomic OR retries. Bits are **monotonic** (no ABA, no locks) and OR is commutative, so the result is interleaving-independent.

Pitfalls

Non-atomic bit setting; weak hashes (clustered bits); far more than *n* items; negative indices (use `Math.floorMod`).

Real-world examples

Cassandra, HBase, RocksDB (skip disk reads); Guava `BloomFilter`; RedisBloom; "URL already crawled?" checks.

In the interview

> **30-second answer:** "A bit array of size m with k hash positions per item from double hashing; add sets k bits, query checks them — any zero means definitely absent. For concurrency I store bits in an AtomicLongArray and set them with an atomic OR; since bits only go from 0 to 1, reads need no lock. m and k come from n and the target false-positive rate."

Self-check

- **Q:** Bits per element for 1% FPR? **A:** About 9.6.

TL;DR

Bits + k hashes; "no" is certain, "yes" is probable. `AtomicLongArray` + atomic OR = lock-free and correct.

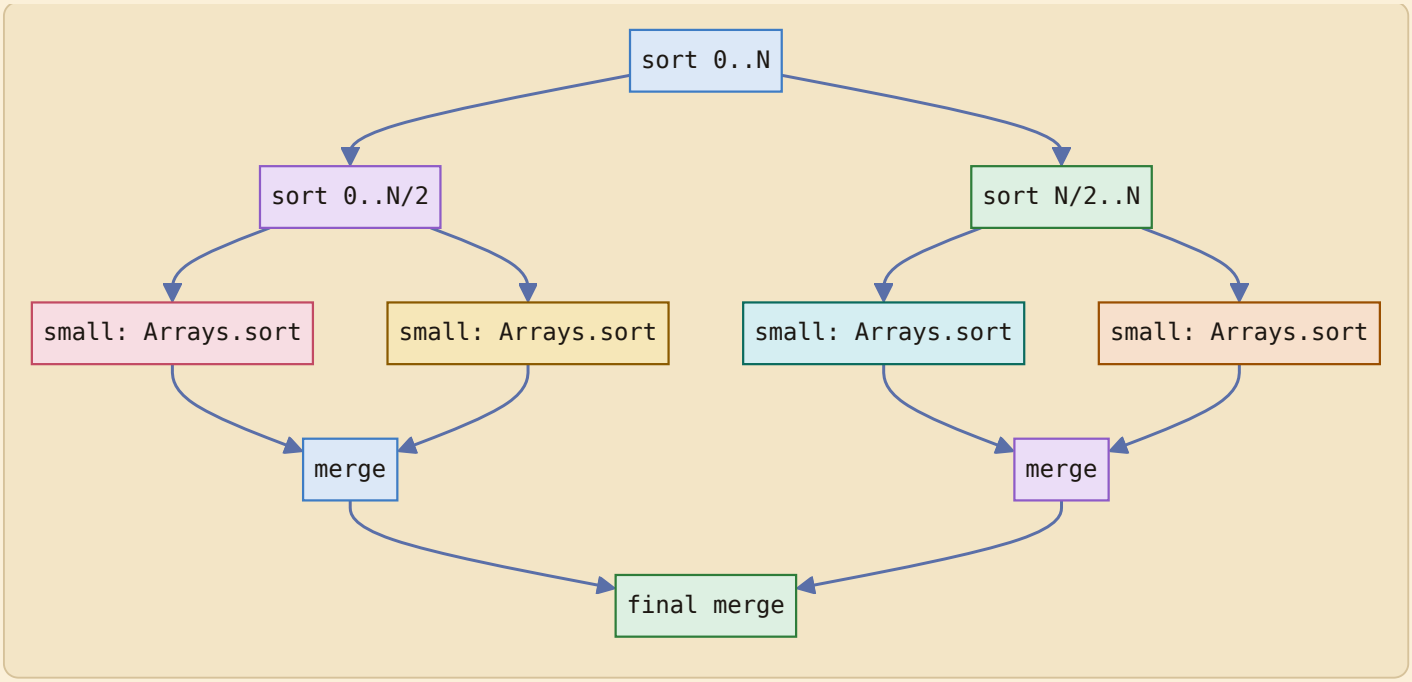
13.10 Multi-threaded Merge Sort (Fork/Join)

Problem

Sort a large `int[]` using all CPU cores.

Approach

Merge sort is **divide and conquer**; Java's **Fork/Join** (`ForkJoinPool`, `RecursiveAction`/`RecursiveTask`) fits it: ranges ≤ 8,192 → `Arrays.sort` sequentially; otherwise split at `(lo + hi) >>> 1` and `invokeAll(left, right)`; merge with **one shared scratch buffer** (tasks use disjoint `[lo, hi)` slices). **Work stealing:** idle pool threads steal tasks from the other end of busy threads' deques.



Java solution

```
// File: code/lld/ch13/mergesort/Main.java
    static final class ParallelMergeSort extends RecursiveAction {
        private static final long serialVersionUID = 1L;
        static final int THRESHOLD = 1 << 13;   // below ~8K elements, splitting costs more than it saves
        private final int[] a, buf;             // buf = one shared scratch array; tasks use disjoint ranges
        private final int lo, hi;               // sorts a[lo, hi)

        ParallelMergeSort(int[] a, int[] buf, int lo, int hi) { this.a = a; this.buf = buf; this.lo = lo;
this.hi = hi; }

        @Override protected void compute() {
            if (hi - lo <= THRESHOLD) {
                Arrays.sort(a, lo, hi);                         // sequential base case
                return;
            }
            int mid = (lo + hi) >>> 1;                          // overflow-safe midpoint
            invokeAll(new ParallelMergeSort(a, buf, lo, mid),   // fork both halves, wait for both
                      new ParallelMergeSort(a, buf, mid, hi));
            merge(mid);
        }

        private void merge(int mid) {
            if (a[mid - 1] <= a[mid]) return;                   // already in order: skip
            System.arraycopy(a, lo, buf, lo, hi - lo);
            int i = lo, j = mid, k = lo;
            while (i < mid && j < hi) a[k++] = buf[i] <= buf[j] ? buf[i++] : buf[j++];   // <= keeps it stable
            while (i < mid) a[k++] = buf[i++];
            // right-half leftovers are already in place
        }
    }

    static void parallelSort(int[] a) {
        ForkJoinPool.commonPool().invoke(new ParallelMergeSort(a, new int[a.length], 0, a.length));
    }
```

Output: `4000000 random ints: matches Arrays.sort = true`, `Arrays.parallelSort agrees = true`. Full runnable code: code/lld/ch13/mergesort/ (see the Full Edition).

Why it's correct

Disjoint ranges → no races; `invokeAll`'s join gives happens-before for the merge; `<=` keeps it **stable**.

Pitfalls

No threshold; a new buffer per merge; blocking I/O in Fork/Join tasks; `(lo + hi) / 2` overflow; expecting speed-up on tiny arrays (Amdahl).

Real-world examples

`Arrays.parallelSort`; parallel streams; Hadoop/Spark sorts across machines.

In the interview

> **30-second answer:** "A RecursiveAction splits the range in half until it's below a threshold, sorts small pieces sequentially, then merges using one preallocated buffer. invokeAll runs both halves in parallel on the ForkJoinPool, whose work stealing keeps cores busy. Subtasks touch disjoint ranges, so no locks are needed."

Self-check

- **Q:** Why is a shared `buf` safe? **A:** Concurrent tasks use non-overlapping ranges of it.

TL;DR

Split until small, sort sequentially, merge; Fork/Join + work stealing; disjoint ranges = no locks.

13.11 Rate limiter — token bucket (thread-safe)

Problem

Limit API calls per client (user id / API key / IP): bursts up to *B*, sustained rate *R*/s, correct under concurrency.

Approach

Token bucket: holds up to *capacity* tokens (the burst), starts full, refills at *refillRate*/s (never above capacity); each request takes one token or is rejected (HTTP 429). Refill **lazily** from `tokens` and `lastRefill` on each request — no background thread. **"Refill + check + take" must be one atomic step**, else two threads spend the same token: (1) `synchronized tryAcquire()`; (2) **lock-free** — `(tokens, lastRefill)` in one immutable `State` record in an `AtomicReference`, swapped by CAS (12.15's trick). Per client: `computeIfAbsent(clientId, …)` makes exactly one bucket. Time comes from an injected `Ticker`.

Algorithm	Bursts	Memory	Notes
Token bucket	Up to capacity	2 numbers	Most common for APIs
Leaky bucket	Smoothed constant output	Queue	Traffic shaping
Fixed window counter	2× burst at window edges	1 counter	Simplest
Sliding window log	Exact	One timestamp per request	Memory-heavy
Sliding window counter	Approximate, smooth	2 counters	Good compromise

Java solution

```
// File: code/lld/ch13/ratelimiter/Main.java
        public synchronized boolean tryAcquire() {
            long now = ticker.nanos();
            tokens = Math.min(capacity, tokens + (now - lastRefill) * tokensPerNano);   // refill + acquire
are ONE atomic step
            lastRefill = now;
            if (tokens >= 1) { tokens -= 1; return true; }
            return false;
        }
    // ...
    /** Same algorithm, lock-free: the whole state is one immutable record swapped with CAS. */
    static final class LockFreeTokenBucket implements RateLimiter {
        private record State(double tokens, long lastRefill) { }
        // ... capacity, tokensPerNano, ticker
        private final AtomicReference<State> state;
        // ...
        public boolean tryAcquire() {
            while (true) {
                State cur = state.get();
                long now = Math.max(ticker.nanos(), cur.lastRefill());   // never move time backwards
                double refilled = Math.min(capacity, cur.tokens() + (now - cur.lastRefill()) * tokensPerNano);
                boolean ok = refilled >= 1;
                State next = new State(ok ? refilled - 1 : refilled, now);
                if (state.compareAndSet(cur, next)) return ok;            // lost the race? recompute and
retry
            }
        }
    }

    /** One bucket per client key (user id, API key, IP). computeIfAbsent creates each bucket exactly once. */
        boolean allow(String clientId) { return buckets.computeIfAbsent(clientId, k ->
factory.get()).tryAcquire(); }
```

Output (both versions, capacity 5, refill 2/s):

```
  t=0.0s  7 requests : YYYYYnn
  t=1.0s  3 requests : YYn
  t=1.25s 1 request  : n  (only 0.5 token so far)
  after 1 idle hour  : allowed 5 of 100 concurrent requests (burst capped at 5)
```

Full runnable code: code/lld/ch13/ratelimiter/ (see the Full Edition).

Why it's correct

The decision is atomic (lock, or snapshot + CAS retry) → each token granted once; tokens clamp to `capacity`; `computeIfAbsent` prevents two buckets per client; `Math.max(..., lastRefill)` stops time going backwards.

Pitfalls

Separate refill/take calls; a thread per bucket; `currentTimeMillis()`; unbounded bucket maps (evict idle ones with 13.6's TTL cache); in-memory limits are per server — use **Redis** + Lua for global limits.

Real-world examples

Stripe (Redis token buckets); API Gateway *rate* + *burst*; Guava `RateLimiter`, Bucket4j; NGINX `limit_req` (leaky bucket).

In the interview

30-second answer: "Token bucket: capacity for bursts, refill rate for the average. I refill lazily from elapsed nanoTime on each request, and make refill-check-take one atomic step — synchronized, or lock-

free by CASing an immutable state record. A ConcurrentHashMap gives one bucket per client via computeIfAbsent; across servers I'd move the bucket into Redis with a Lua script."

Self-check

- **Q:** Why one immutable `State` in the lock-free version? **A:** `tokens` and `lastRefill` must change together; one CAS on one reference updates both.

TL;DR

Capacity = burst, rate = average. Lazy refill; refill+take atomic (lock or CAS on immutable state); one bucket per client; Redis for multi-server.

13.12 Chapter summary

Problem	Key primitive	The one invariant to state in the interview
FooBar	2 semaphores	Exactly one permit exists across both
Zero Even Odd	3 semaphores	Zero dispatches; one permit circulates
FizzBuzz	monitor + `notifyAll`	Predicates partition 1..n
H2O	2 semaphores + barrier	≤2 H and ≤1 O per round; slot freed after release
Dining Philosophers	ordered locks / n−1 semaphore	No circular wait
TTL cache	CHM + `compute` + conditional remove	Never return or keep expired data; load once per key
Striped map	lock array	Each bucket belongs to exactly one stripe
Blocking queue	lock + 2 conditions	0 ≤ count ≤ capacity; wait in loops
Bloom filter	atomic OR	Bits only go 0 → 1; no false negatives
Merge sort	Fork/Join	Subtasks touch disjoint ranges
Rate limiter	lock or CAS on immutable state	Refill + take is one atomic step

13.13 Chapter self-check

1. **Q:** In FizzBuzz, what bug appears with `notify()`? **A:** The wrong thread wakes and sleeps again; the right one never wakes → hang.
2. **Q:** In H2O, why release the slot after printing? **A:** So the next molecule can't form until this one has fully left.
3. **Q:** Why must the striped map's stripe depend only on the hash? **A:** So a key's lock doesn't change on resize.
4. **Q:** When is `signal()` safe instead of `signalAll()`? **A:** When each condition queue holds only threads waiting for the same thing and one change enables exactly one waiter.
5. **Q:** How do you make a token bucket global across 10 servers? **A:** Store it in Redis and update it atomically with a Lua script.

13.14 Chapter TL;DR

- **Ordering puzzles** (FooBar, ZeroEvenOdd, H2O): semaphores as batons, barriers for groups; "exactly one permit" invariants.
- **Predicate waiting** (FizzBuzz, blocking queue): lock + `while (!condition) wait/await` + `notifyAll` or targeted `signal`.
- **Multi-lock problems** (philosophers): break circular wait with ordering or limiting.

- **Thread-safe data structures** (cache, map, queue, Bloom filter, rate limiter): immutable entries, atomic compound ops, striping, CAS on immutable state, injected clocks.
- **Parallel algorithms** (merge sort): Fork/Join with thresholds and disjoint data.

Chapter 14 — Case Studies: Everyday Objects (Study Edition)

> **Where this fits.** Chapters 1–13 gave you the tools (OOP, UML, SOLID, clean code, the GoF patterns, concurrency). Chapters 14–21 apply them to real interview questions. This chapter starts with **machines you have used yourself** — parking lot, vending machine, coffee machine, ATM, elevator, traffic light, parcel locker — so all your attention can go to the design. This is the study edition: every topic stays, the padding goes. The Full Edition (README.md) has the complete code, outputs and stage-by-stage course diagrams.

#	Problem	The one idea the interviewer is really testing	Main patterns
14.1	Parking Lot	**Resource allocation** under concurrency: two cars must never get the same spot	Strategy (spot finding, pricing), Factory, Observer (display boards)
14.2	Vending Machine	A **state machine**: the order in which actions are allowed	State
14.3	Coffee Vending Machine	**Atomic, all-or-nothing** deduction from a shared inventory; recipes as data	Observer (low-stock alerts), data-driven recipes, sealed result types
14.4	ATM	**Security + money correctness**: authenticate, never lose money, never double-spend	State, Chain of Responsibility (cash dispenser), Command (transactions)
14.5	Elevator System	**Scheduling** (SCAN/LOOK) and **dispatching** many cars	Strategy (dispatch), State-like modes
14.6	Traffic Signal Control	**Timed state machine** with a hard safety invariant and emergency override	State, Strategy (timing), Observer (safety monitor)
14.7	Shipping Locker System	**Best-fit allocation + deadlines + a secret**: reserve a door atomically, expire holds and uncollected parcels, single-use hashed pickup codes	Strategy (door choice), Observer (SMS), enum state machines, sealed results

What these problems test

1. **State machines** — a fixed set of states plus transition rules: no dispensing before payment, no withdrawal before the PIN, never green on two crossing roads. The **State pattern** (Chapter 11) gives every button an answer in every state.
2. **Resource allocation** — spots, notes, milk, cars, locker doors handed out never twice, never below zero, fairly.
3. **Concurrency** — several gates, two ATMs on one account, a timer plus an emergency thread; the design must stay correct when calls overlap (Chapters 12–13).

> **Key insight.** The hardest part of each problem is **one method**: pick a spot, make change, deduct ingredients, pick the next stop. Find it early, name it out loud, spend your time there.

How each case study is laid out

Problem & clarifying questions → Requirements → Use cases → Core entities → Class diagram → Key flows → Design decisions & patterns → Java implementation → Concurrency & edge cases → Extensions & follow-ups → In the interview → Self-check → TL;DR. This is also a good order to speak in during an interview.

How to read the course diagrams

The course draws class diagrams in stages (earlier classes in grey, new ones in colour). This study edition keeps only the **finished** course diagram per case plus a mermaid diagram of our Java code.

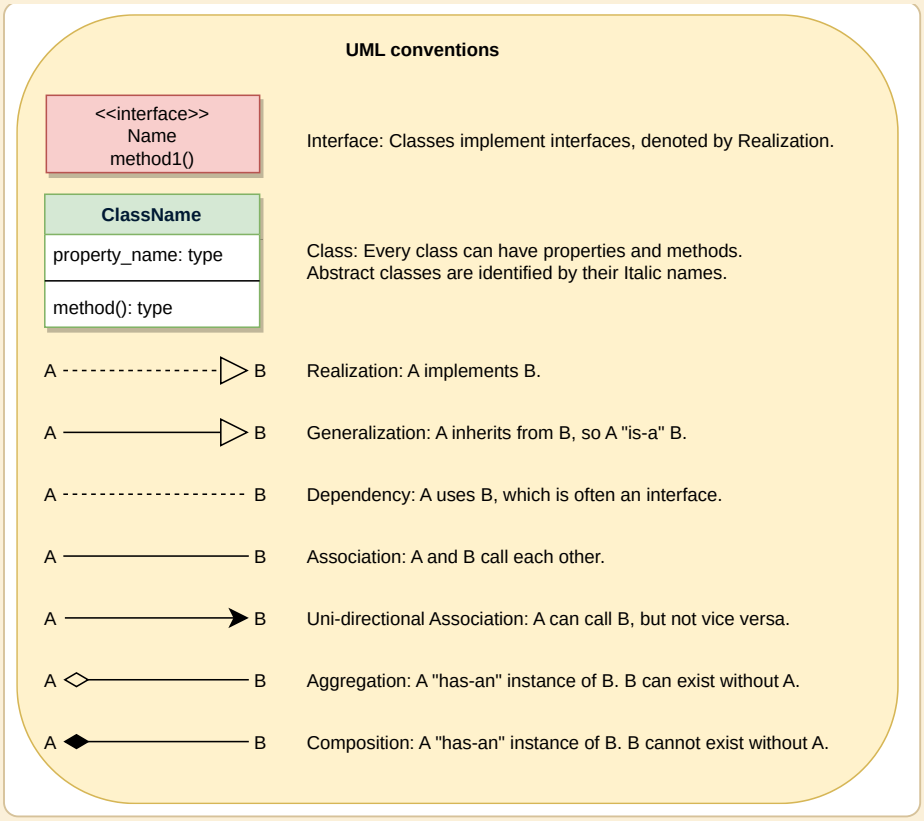


Figure — Legend for the course class diagrams: interface, class, realization, association, inheritance, composition and the other arrow kinds (course diagram)

Hollow triangle = inheritance ("is a"); **dashed line + hollow triangle** = implements an interface; **filled diamond** = composition (the part dies with the whole); **plain arrow** = association (refers to, does not own). See Chapter 3.

How to run every example in this chapter

```
cd chapters/14-case-everyday-objects/code
javac --release 21 -d /tmp/lld_14 $(find . -name '*.java')
java -cp /tmp/lld_14 lld.ch14.parkinglot.Main      # or vendingmachine, coffeemachine, atm, elevator, trafficsignal, locker
```

Java 21, standard library only. **Code-style note:** some files hold several small package-private types (e.g. `Vehicles.java`); fine except for the `auxiliaryclass` lint — in production give each type its own file.

14.1 Design a Parking Lot

A **parking lot** (car park) with many floors, many spot sizes and several ways in and out. It is the **most-asked LLD question**. It looks easy, so the interviewer can watch how you **scope** it, how you model the **fit rule** (which vehicle may use which spot), and whether you notice that **two gates can hand out the same spot** at the same moment.

Problem & clarifying questions

Question	Assumed answer
Scope?	One lot, many floors; several entrances and exits working at once.
Vehicles / spots?	Car, truck, van, motorcycle, EV / compact, large, handicapped, motorcycle, electric.
Payment?	Cash or card at the exit panel, an attendant, or a per-floor info portal.
Fee?	$4 first hour, $3.50 hours 2–3, $2.50 after; a started hour counts as full.
EV charging?	Electric spots have a pay-and-charge panel.
Reservations, passes, plate recognition?	Out of scope (extensions).

Requirements

Functional: (1) multiple floors; (2) multiple entries/exits; (3) ticket at entry, pay at exit; (4) pay at the exit panel or an attendant; (5) cash or card; (6) pay early at a per-floor **info portal** (not charged again); (7) **never exceed capacity** — "full" on the entrance panel and ground-floor board; (8) spot types; (9) **electric** spots with a pay-and-charge panel; (10) vehicle types; (11) a **display board per floor**; (12) tiered hourly fees.

Non-functional: correct under concurrency (no shared spot, no double payment); fast gate (O(log n) from a sorted free set); extensible; testable (injectable clock).

Use cases

Actors: Admin (floors, spots, attendants, rates), Customer, Parking attendant (a customer proxy who takes cash), System (messages, assigns/frees spots). **Top use cases: take ticket**, **scan ticket**, card/cash payment, admin edits, set rate.

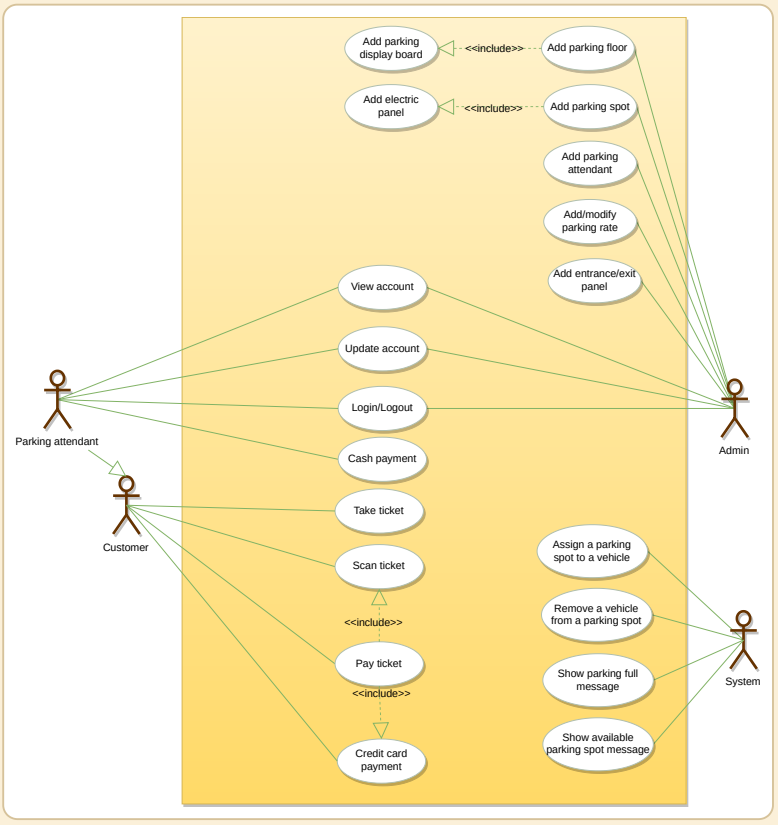


Figure — Use case diagram of the parking lot: Admin, Customer, Parking attendant and System, with their use cases (course diagram)

Core entities

Class	Responsibility
`VehicleType` (enum)	Kinds of vehicle and, for each, the spot types it fits, **best fit first**.
`SpotType` (enum)	Motorcycle, compact, large, handicapped, electric.
`Vehicle` (record)	Plate, type, accessibility permit; knows its preferred spot types.
`ParkingSpot`	One spot: id, floor, type, occupant (atomic).
`ChargingPanel`	Pay-and-charge panel owned by an electric spot.
`SpotFactory`	Builds spots; the only place that knows an electric spot needs a panel.
`ParkingFloor`	Free spots per type in a thread-safe sorted set; notifies boards.
`DisplayBoard`	Free spots per type on one floor (an observer).
`SpotAllocationStrategy`	*How* to choose a spot.
`PricingStrategy`	*How much* a stay costs.
`Ticket`	Vehicle, the **actual spot**, entry time, status ACTIVE → PAID → EXITED.
`PaymentMethod`	Cash or card.
`ParkingLot`	Ties it together: admit, price, pay, exit.
`EntrancePanel`, `ExitPanel`, `CustomerInfoPortal`	Devices customers touch; **no rules**, they call the lot.

The course also has `Account` with `Admin` and `ParkingAttendant` subclasses; they carry no parking logic, so the code omits them.

Class diagram

The course grows the design in **five stages**, each leaving something that runs (a good interview habit: if time runs out, what you drew already works):

Stage	Adds	Key decision
1	Vehicle and spot types	Spot holds **0..1** vehicle; the **fit rule lives on the vehicle** (truck/van → large; EV → electric, compact, large).
2	Floors, boards	Floor owns spots and board; board stores **counts**, not a stale "sample free spot"; one map keyed by type.
3	Ticket, rate, payment	**Ticket stores the exact spot**; abstract `Payment` with cash/card (LSP holds).
4	`ParkingLot`	Owns floors and rate; course makes it a **Singleton** and serialises assignment.
5	Panels, portal, electric panel	Panels are **thin**, forwarding to the lot; electric panel owned by its spot.

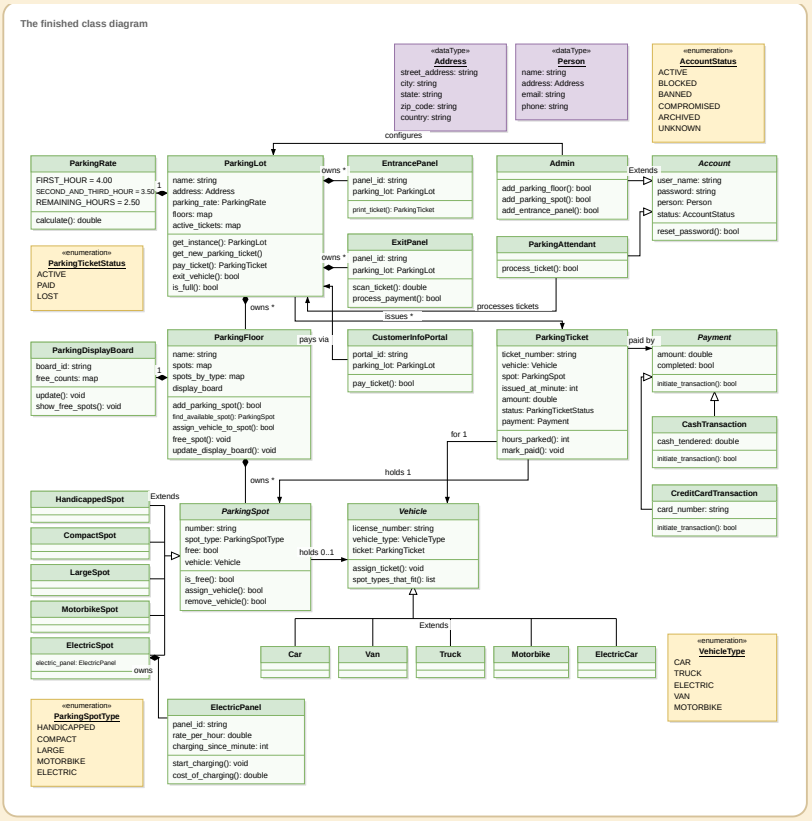
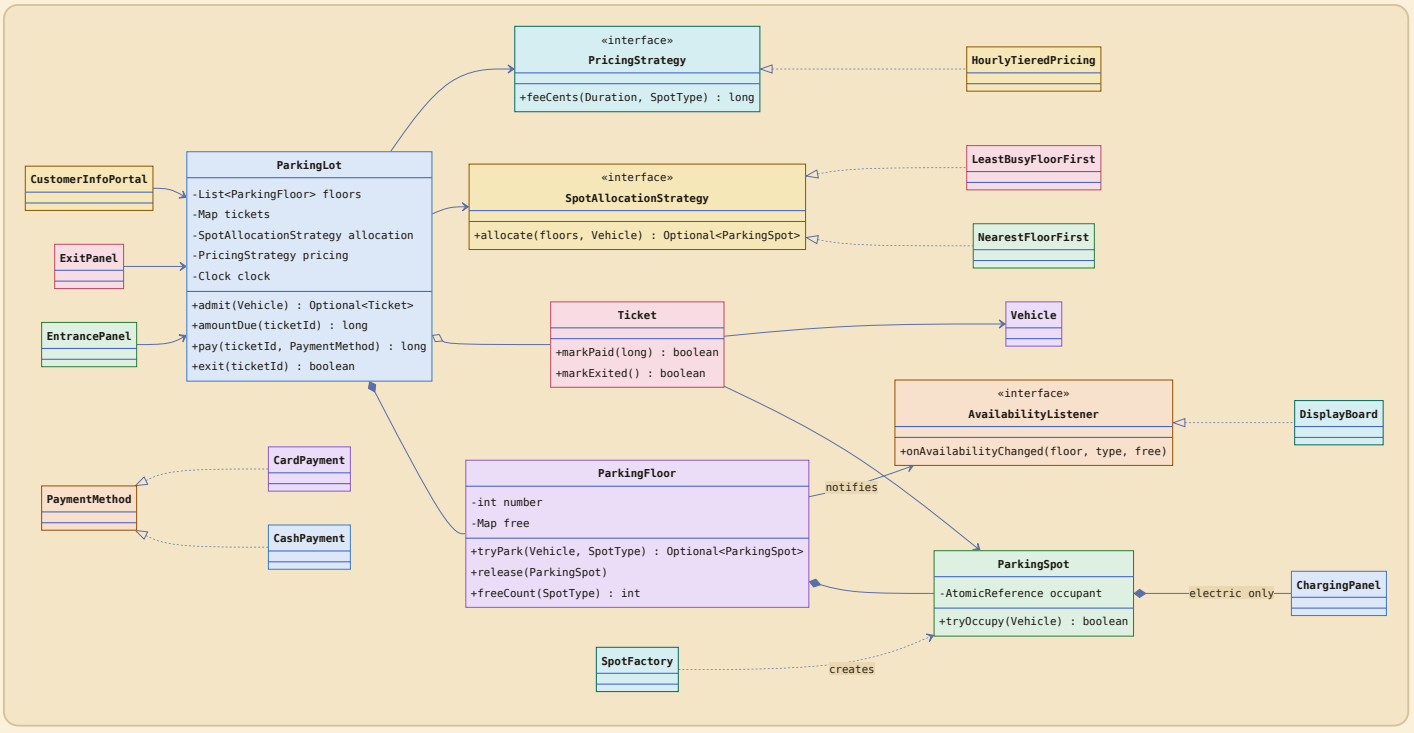


Figure — The finished class diagram of the parking lot (course diagram)

Our Java code keeps the same shape with three changes interviewers push for: spot types are an **enum** (not five empty subclasses), pricing and spot-finding are **Strategies**, and the display board is an **Observer**.



Key flows

- **Entry:** `EntrancePanel` → `lot.admit(vehicle)` → strategy tries each preferred spot type on each floor → `floor.tryPark` does `pollFirst()` on the free set (**atomic**) and notifies the board → ticket printed, or "LOT FULL".
- **Exit:** `ExitPanel` → `lot.pay(ticketId, card)` **locks the ticket**; if ACTIVE, price it and charge, then mark PAID; if already PAID at the portal, charge 0 → `lot.exit` moves PAID → EXITED and releases the spot → barrier opens.
- **Ticket life cycle:** `ACTIVE → PAID → EXITED`.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
Fit rule on `VehicleType`	Information Expert, OCP	New vehicle type = one enum constant	`switch(vehicleType)` in the lot
Spot type is an **enum field**	Composition over inheritance	The 5 course subclasses had no behaviour	`CompactSpot`, `LargeSpot`…
`SpotAllocationStrategy`	**Strategy**	Nearest floor, least busy, VIP… swap freely	`if (mode == NEAREST)`
`PricingStrategy`	**Strategy**	"What if pricing changes?"; admin swaps at run time	Tiers hard-coded in `Ticket`
`SpotFactory` attaches `ChargingPanel`	**Factory**	One place knows "electric ⇒ has a panel"	Every caller remembers
Boards subscribe to floors	**Observer**	Boards, apps, bay lights update without the floor knowing them	Lot pushing into boards
Ticket holds the **spot**	—	Exit frees the exact spot	Counting occupied spots
Lot is **not** a Singleton	Dependency injection	A Singleton is a hidden global; tests can't build two lots	`ParkingLot.getInstance()`
`Clock` injected	Testability	Demo jumps hours instantly	`Instant.now()` inside

> **Honest note on "Strategy".** A cash/card subclass pair is just an abstract base class; Strategy means an object **holds** a swappable behaviour (`setPricing(...)`). The course admits the lot does two jobs; spot-finding moved to strategy and floors, and tickets could move to a `TicketRepository`.

Java implementation

Package `lld.ch14.parkinglot`. Amounts are **cents** (`long`) to avoid floating-point rounding.

```
enum VehicleType {
    MOTORCYCLE(List.of(SpotType.MOTORCYCLE)),
    CAR(List.of(SpotType.COMPACT, SpotType.LARGE)),
    ELECTRIC_CAR(List.of(SpotType.ELECTRIC, SpotType.COMPACT, SpotType.LARGE)),
    VAN(List.of(SpotType.LARGE)),
    TRUCK(List.of(SpotType.LARGE));
    // ...
}

class ParkingFloor {
    private final Map<SpotType, ConcurrentSkipListSet<ParkingSpot>> free = new EnumMap<>(SpotType.class);
    // ...
    /** pollFirst() is atomic: two threads can never receive the same spot. Lowest id = nearest. */
    Optional<ParkingSpot> tryPark(Vehicle v, SpotType type) {
        ParkingSpot spot = free.get(type).pollFirst();
        if (spot == null) return Optional.empty();
        if (!spot.tryOccupy(v)) throw new IllegalStateException("free-set out of sync for " + spot);
        notifyListeners(type);
        return Optional.of(spot);
    }
    // ...
}

final class ParkingLot {
    // ...
    Optional<Ticket> admit(Vehicle v) {
        String placeholder = "pending";
        if (ticketByPlate.putIfAbsent(v.plate(), placeholder) != null) {
            throw new IllegalStateException(v.plate() + " is already inside");
        }
        Optional<ParkingSpot> spot = allocation.allocate(floors, v);
        if (spot.isEmpty()) {
            ticketByPlate.remove(v.plate());
            return Optional.empty();
        }
        Ticket t = new Ticket("T" + nextTicket.getAndIncrement(), v, spot.get(), clock.instant());
        tickets.put(t.id(), t);
        ticketByPlate.put(v.plate(), t.id());
        return Optional.of(t);
    }
    // ...
    long pay(String ticketId, PaymentMethod method) {
        Ticket t = ticket(ticketId);
        synchronized (t) {                // per-ticket lock: portal and exit panel cannot both charge
            long due = amountDue(ticketId);
            if (due == 0) return 0;
            method.pay(due);              // throws if the payment fails: ticket stays ACTIVE
            t.markPaid(due);
            return due;
        }
    }
    // ... exit(): markExited() (PAID -> EXITED), then floor.release(spot)
}
```

Full runnable code: code/lld/ch14/parkinglot/ (see the Full Edition).

How it works: `Vehicle.preferredSpots()` puts `HANDICAPPED` first for permit holders only. `ParkingSpot.tryOccupy` (`compareAndSet(null, v)`) is a second safety net. `NearestFloorFirst` and `LeastBusyFloorFirst` are interchangeable strategies.

Output (excerpt):

```
[entry-A] TRUCK-2 -> SORRY, LOT FULL for TRUCK
[portal] T2 due $16.00, charged $16.00
[exit-A] T2: already paid, barrier OPEN
Unpaid exit attempt: barrier opens? false
== 20 cars race for the remaining CAR spots ==
cars admitted: 5, distinct spots: 5
```

4 h 50 min rounds to 5 hours = $4 + $3.50 + $3.50 + $2.50 + $2.50 = **$16.00**; 20 racing threads get exactly 5 distinct spots.

Concurrency & edge cases

Situation	Risk	Handling
Two gates, last spot	Double booking	Atomic `pollFirst()` + CAS `tryOccupy`
Portal and exit pay the same ticket	Charged twice	`synchronized (ticket)`; a PAID ticket costs 0
Same plate enters twice	Two tickets	`ticketByPlate.putIfAbsent`
Card declined	Marked paid anyway	`method.pay` throws **before** `markPaid`
Leaves without paying	Fee lost	`exit` only from PAID; barrier stays closed
Pays at portal, stays 3 more hours	Free parking	**Grace period** (e.g. 15 min); add `paidAt`, re-price after
Board shows an old number	Out-of-order notifications	OK for display; board can **pull** `freeCount` on render
Lost ticket / 0-minute stay	No entry time / $0	Daily maximum or plate lookup / minimum one hour
Lock scope	Global lock serialises gates	Per-spot CAS and per-ticket lock only

`ConcurrentSkipListSet.size()` is not constant-time and only an estimate under change — fine for a board, never for deciding a spot is free (use `pollFirst()`).

Extensions & follow-up questions

1. **Pricing changes** (weekend, per spot type, surge) → new `PricingStrategy` (or a decorator), `setPricing`. Nothing else changes.
2. **Reservations** → `Reservation(plate, spotType, from, to)` + reserved set per floor; hold with a timeout (e.g. 30 min).
3. **Monthly passes** → pricing returning 0 for pass plates, or a `PassHolderPricing` decorator.
4. **Plate recognition** → the camera is another entrance panel calling `admit`.
5. **Database** → tables `floor`, `spot`, `ticket`, `payment`; claim with `UPDATE spot SET occupied_by=? WHERE id=? AND occupied_by IS NULL` (the DB equivalent of CAS). Show **cardinalities** (floor 1..* spots, spot 0..1 vehicle, ticket 1 spot).
6. **Many lots in a city** → a `ParkingNetwork` routing to lots — another reason for no Singleton.
7. **EV idle fees** → a charging session on `ChargingPanel` with a per-minute fee after charging completes.

Real-world example

Parking guidance systems (per-bay sensors, free counts per level) are our `DisplayBoard` observers; airports use **plate recognition**; SpotHero/ParkMobile pre-booking = reservations; Tesla **idle fees** = the EV extension.

In the interview

45 minutes: 0–5 questions/requirements → 5–10 use cases → 10–25 stages 1–4 ("fit rule on the vehicle", ticket holds the spot) → 25–38 code `admit`, `tryPark`, `pay`, `exit` → 38–45 concurrency and extensions.

30-second answer: "Vehicles and spot types are enums with the fit rule on the vehicle; floors keep a concurrent sorted set of free spots per type; the lot admits, prices and releases. Tickets point at the

real spot. Pricing and spot-finding are Strategies. Handing out a spot is atomic — `pollFirst` plus a CAS on the spot — so two gates never get the same spot, and payment locks only the one ticket."

- *"Why not a Singleton?"* — It is a hidden global that makes testing two lots impossible; I create one and inject it. If pressed, a lazy holder-class Singleton is thread-safe.
- *"How do you make finding a spot fast?"* — Keep only free spots, indexed by type, in a sorted set: O(log n), no scan.

Self-check

1. **Q:** Why must the ticket store the spot? **A:** So the exit frees exactly that spot; a count can't say which.
2. **Q:** What makes assignment thread-safe? **A:** `pollFirst()` gives a spot to exactly one caller; `compareAndSet` double-checks.
3. **Q:** Where does a weekend price go? **A:** A new `PricingStrategy` via `setPricing`.
4. **Q:** Paid at the portal — what does the exit charge? **A:** Nothing: `amountDue` is 0 for PAID; `exit` moves PAID → EXITED.

TL;DR

- Fit rule on the **vehicle**, spot type as an **enum**, free spots per type in a **concurrent sorted set**.
- **Ticket → spot** reference; status ACTIVE → PAID → EXITED.
- **Strategy** (pricing, spot-finding), **Factory** (electric spots), **Observer** (boards); inject the lot, no Singleton.
- Atomic `pollFirst` + CAS for spots, per-ticket lock for payment, `putIfAbsent` for plates.

14.2 Design a Vending Machine

A **vending machine** is an unattended shop: key in a slot code, pay, take the item from the tray. The interviewer skips the domain and checks one thing: **which actions are legal at which moment**. Accepting a coin with nothing chosen, or two items for one payment, are bugs no tidy code elsewhere can excuse.

Problem & clarifying questions

Question	Assumed answer
Select or pay first?	Select a slot code first, then pay.
Which money?	Fixed set: 5¢, 10¢, 25¢, $1, $5. Anything else is returned.
Card / contactless?	Out of scope (extension).
Change?	Yes. If **exact** change is impossible, refuse the sale and return everything.
Cancel?	Any time before dispensing, full refund.
Concurrency?	One customer at a time (one physical machine).
Restocking?	An operator restocks, loads coins, collects takings.

Requirements

Functional (the course's twelve): (1) many **slots**, each with a code, a **single** product and a **count**; (2) a product has a name and a **price in cents**; (3) **choose, then pay**; (4) only known coins/notes, others returned; (5) the **display** says what to do after every action; (6) when paid enough, release **item and change**; (7) if **exact change is impossible**, refuse and return every cent; (8) **cancel** with full refund until the item drops; (9) an empty slot **cannot be chosen**; (10) one payment = **exactly one** item; (11) an **operator** refills, adds change, removes takings; (12) item and cash counts are **exactly right** after every sale, cancel and refusal.

Non-functional: no illegal transitions (every button answered in every state); integer cents only; extensible (new payment device or state without editing every class).

Use cases

Actors: Customer, Operator, System. **Use cases:** select · insert money · dispense · collect · cancel · restock · load change / collect takings. "Dispense" is a **System** use case — the customer only causes it.

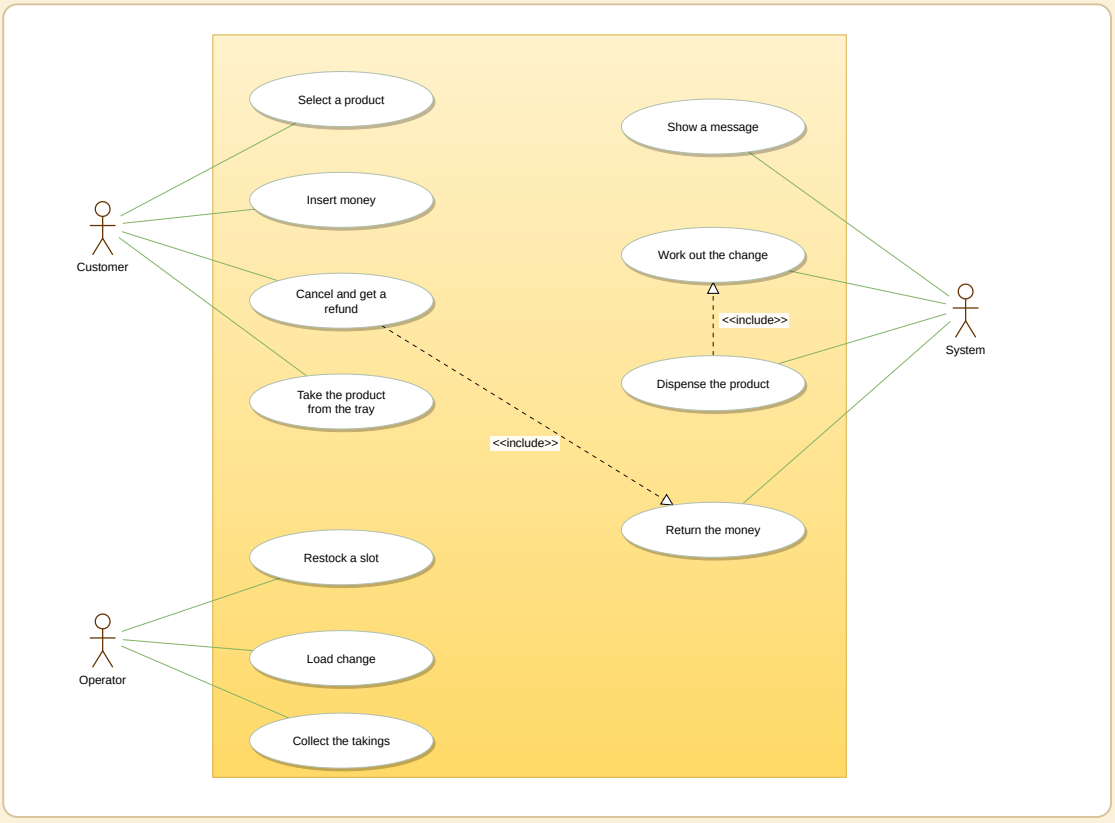


Figure — Use case diagram of the vending machine: Customer, Operator and System (course diagram)

Core entities

Class	Responsibility
`Product` (record)	Name and price in cents — *what* is sold.
`Slot`	One column: code, one product, **count** left.
`Inventory`	All slots by code.
`Denomination` (enum)	Accepted coins/notes and their cents.
`CashBox`	Counts pieces by value; plans **exact** change or reports it can't.
`VendingState` (interface)	One method per customer button: `select`, `insert`, `cancel`, `take`.
`IdleState`, `SelectedState`, `HasMoneyState`, `DispensingState`, `OutOfStockState`	The situations; each answers every button and names the next state (course calls `HasMoneyState` `PaidState`).
`Display`, `Keypad`, `CashSlot`, `Dispenser`, `Operator`	Course hardware/actor classes; in our code folded into `VendingMachine.show(...)` and its public/operator methods.
`VendingMachine`	Parts, current state, selection, balance; forwards each button to the state.

Why is the count on the slot, not the product? `Product` answers "what is it?"; `Slot` is the physical column. The same product can sit in two columns (even at different prices), and emptying one must not affect the other.

Class diagram

Course stages: (1) `Product`, `Slot` (association — a product can be in many slots), `Inventory` (composition); (2) `Coin`, `Note`, `CashBox` — **a piece goes into the box on insert**, so a refund is "pay exact change equal to

the balance"; the planner returns an exact plan or "impossible"; (3) **the heart: one class per state**, each button a method on one interface — a state that forgets a button does not compile; think of a **grid** of buttons × states where every cell needs an answer; (4) `Display` and `VendingMachine` — no `if` on state; (5) `Keypad`, `CashSlot`, `Dispenser`, `Operator` point **to** the machine; `take` finishes the sale; operator actions stay **off** the state interface.

The course draws four states; our code adds `OutOfStockState` so an empty machine refuses customer buttons until restocked. Cancel and "no exact change" end identically, so there is a **single** refund method.

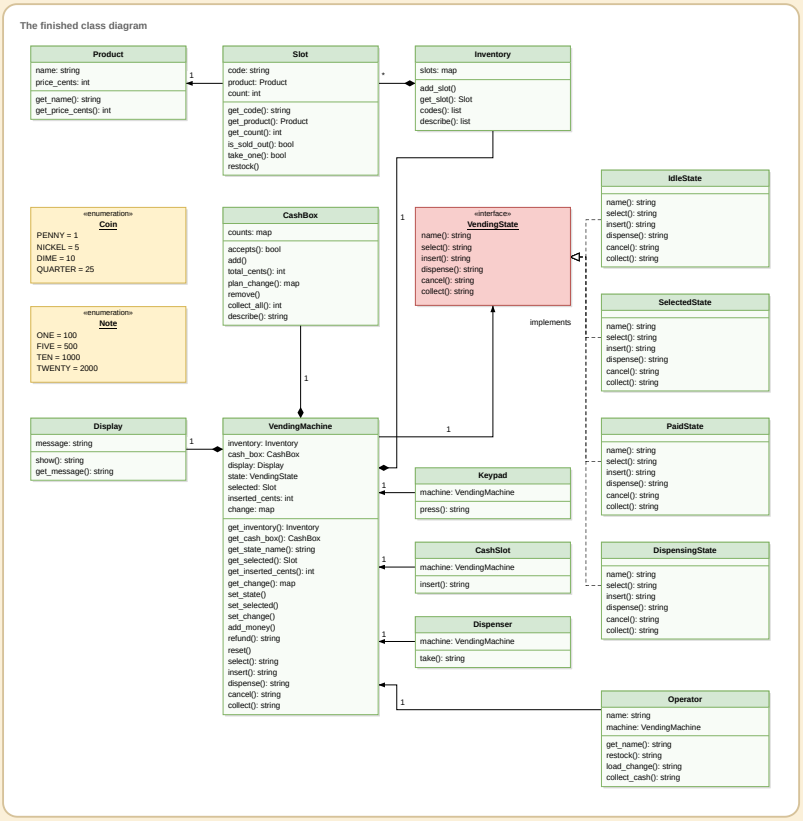
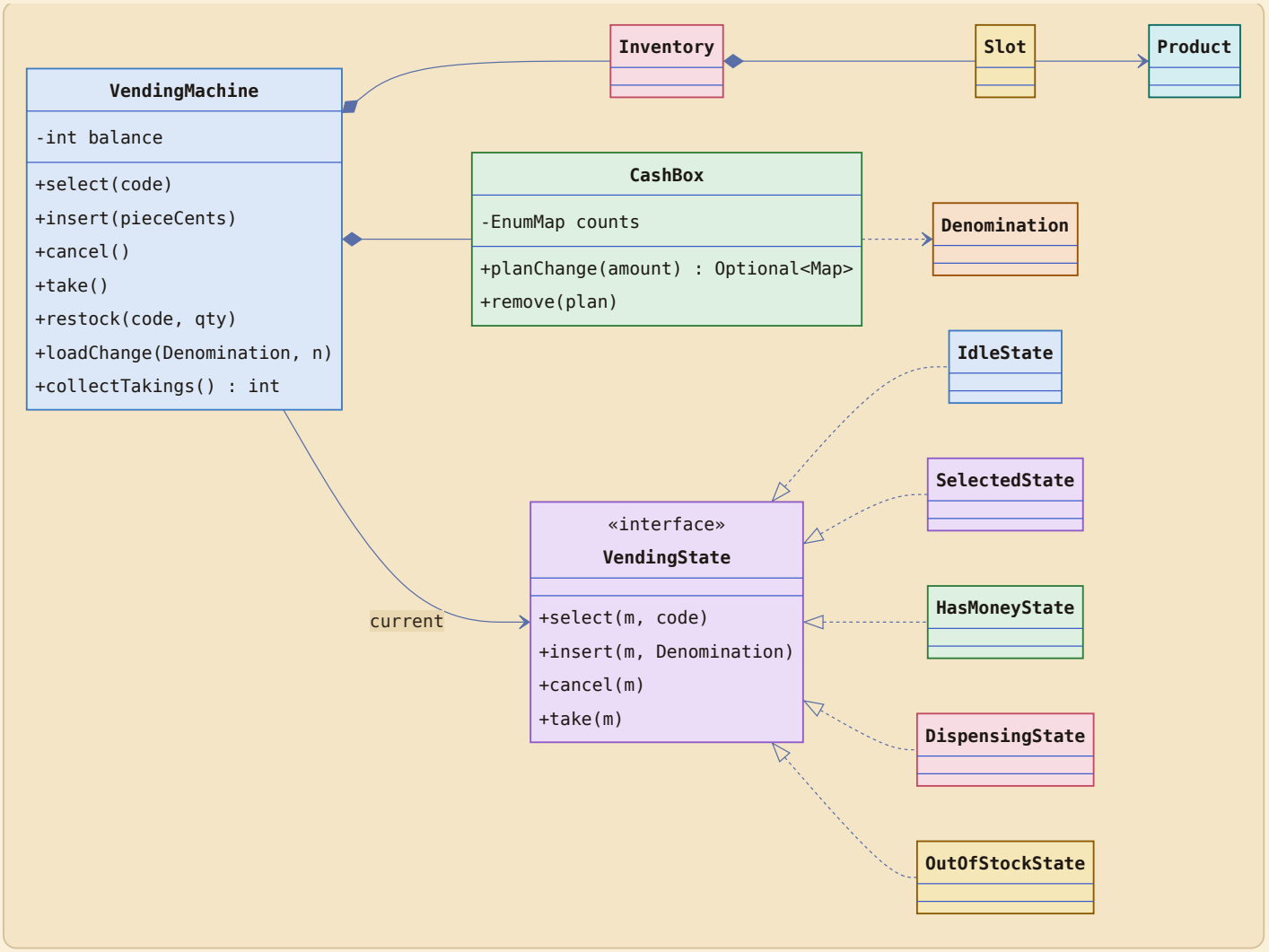
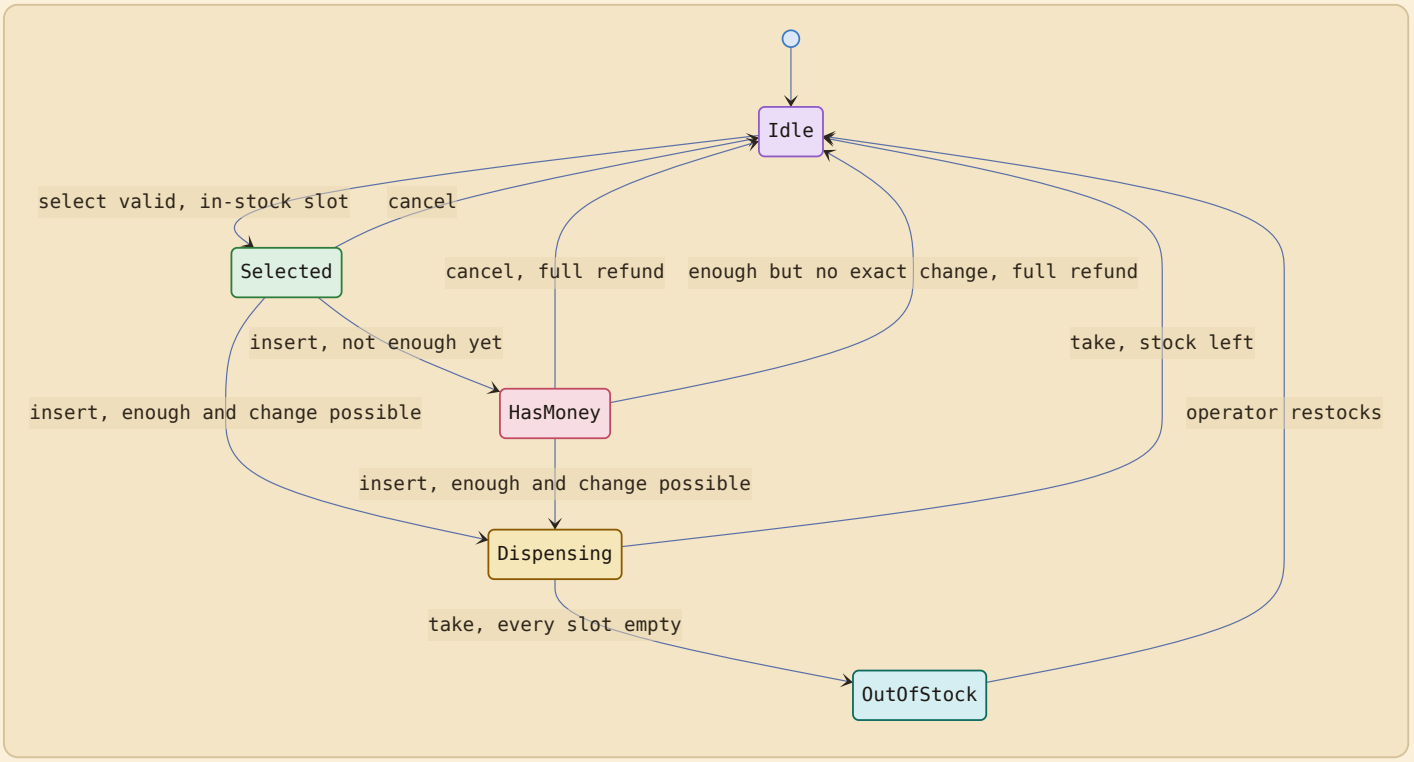


Figure — The finished class diagram of the vending machine (course diagram)



Key flows

The state machine is the design — draw it before coding.



State table (every cell answered):

Button ↓ / State →	Idle	Selected	HasMoney	Dispensing	OutOfStock
`select`	choose slot (reject empty/unknown)	change choice	"cancel first"	"take your product first"	"out of stock"
`insert`	return piece	accept; check paid	accept; check paid	return piece	return piece
`cancel`	"nothing to cancel"	clear choice → Idle	**refund all** → Idle	"too late"	"nothing to cancel"
`take`	"tray empty"	"pay first"	"pay the rest"	give product + change → Idle/OutOfStock	"tray empty"

Paying enough: insert → `acceptPiece` (into box, balance += value) → `checkPaid` → `planChange(balance − price)`; if a plan exists, remove it, `dispenseOne`, go to Dispensing; otherwise refund all and go Idle.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
One class per situation	**State**	Every cell answered; compiler enforces completeness	Flags + `if/else` (double-dispense bugs)
States are stateless singletons (`INSTANCE`)	Flyweight-style sharing	No data, so one instance each	`new IdleState()` each transition
Operator actions off the state interface	Interface Segregation	Restock is valid in every state	`restock` on `VendingState`
Money counted by value in an `EnumMap`	—	Which physical quarter comes back doesn't matter	List of coin objects
Inserted money straight into the box	—	Refund = make change for the balance	Separate escrow pile
Exact change by dynamic programming	Correctness	Greedy fails when short of one coin	Greedy only
Refuse if exact change impossible	Req. 7	Never short-change	"Close enough" change

> **About the change algorithm.** The course's planner is **greedy** (largest piece first) and says so: correct for standard coin sets when well stocked, but it can fail when the box is short. The fix is **dynamic programming** — "coin change with limited coins". Example: 30¢ from {1 quarter, 3 dimes}: greedy takes the quarter and is stuck at 5¢; DP returns three dimes.

Java implementation

Package `lld.ch14.vendingmachine`.

```java
/** State pattern: every button is on the interface, so every state must answer every button. */
interface VendingState {
    void select(VendingMachine m, String code);
    void insert(VendingMachine m, Denomination d);
    void cancel(VendingMachine m);
    void take(VendingMachine m);
    String name();
}

/** Some money is in, but not enough yet. */
final class HasMoneyState implements VendingState {
    static final HasMoneyState INSTANCE = new HasMoneyState();

    @Override public void select(VendingMachine m, String code) { m.show("Cancel first to change your
choice."); }
    @Override public void insert(VendingMachine m, Denomination d) { m.acceptPiece(d); m.checkPaid(); }
    @Override public void cancel(VendingMachine m) { m.refundAll("Cancelled."); }
    @Override public void take(VendingMachine m) { m.show("Please pay the rest first."); }
    @Override public String name() { return "HAS_MONEY"; }
}
// ... IdleState, SelectedState, DispensingState, OutOfStockState

final class VendingMachine {
    // ...
    /** Called after every accepted piece in SELECTED / HAS_MONEY. */
    void checkPaid() {
        int price = selected.product().priceCents();
        if (balance < price) {
            show("Paid %d c, %d c to go.".formatted(balance, price - balance));
            setState(HasMoneyState.INSTANCE);
            return;
        }
        var change = cashBox.planChange(balance - price);
        if (change.isEmpty()) {                              // refuse the sale rather than short-change
            refundAll("Cannot make exact change.");
            return;
        }
        cashBox.remove(change.get());
        tray.add(selected.dispenseOne().name());
        if (!change.get().isEmpty()) tray.add("change " + change.get());
        show("Dispensed. Please take: " + tray);
        balance = 0;
        selected = null;
        setState(DispensingState.INSTANCE);
    }

    /** One refund path for "cancel" and for "cannot make change". */
    void refundAll(String reason) {
        Map<Denomination, Integer> back = cashBox.planChange(balance)
                .orElseThrow(() -> new IllegalStateException("inserted pieces must be refundable"));
        cashBox.remove(back);
        show(reason + " Refunded " + balance + " c as " + back);
        balance = 0;
        selected = null;
        setState(IdleState.INSTANCE);
    }
    // ...
}
```

Full runnable code: code/lld/ch14/vendingmachine/ (see the Full Edition).

How it works: unknown pieces are returned before reaching a state. `CashBox.planChange` is bounded coin-change DP (fewest pieces); `greedyPlan` is only for comparison. `refundAll` cannot fail — the inserted pieces are in the box. Public methods are `synchronized`.

Output (excerpt):

```
  > insert 500c   (SELECTED)
    [display] Cannot make exact change. Refunded 500 c as {FIVE_DOLLAR=1}
30c from {1 quarter, 3 dimes}: greedy=Optional.empty dp=Optional[{DIME=3}]
```

Concurrency & edge cases

Edge case	Handling
Money before selecting	`IdleState.insert` returns it
Unknown coin / foreign note	Returned before reaching the state
Empty slot selected	Refused; stays Idle
Change choice after paying	"Cancel first" (avoids price confusion)
Overpayment	Change planned **before** dispensing; refuse if impossible
Cancel after dispensing	"Too late"
Two products for one payment	Impossible: balance reset to 0, Dispensing accepts no money
Last item sold	`take` → `OutOfStock`; only a restock leaves it
Two threads pressing buttons	`synchronized` public methods
Machines sharing one stock / power failure	Slot count needs a lock or atomic decrement / non-volatile transaction log (out of scope)

Extensions & follow-up questions

1. **Contactless card** → a new input device calling the same machine (e.g. `payByCard(amount)` setting the balance); states unchanged.
2. **Maintenance state** → a new `MaintenanceState` class; existing states untouched (OCP).
3. **Different price per slot** → move price from `Product` to `Slot`.
4. **Report empty slots** → `Inventory` notifies an observer; a telemetry client sends it.
5. **Walk-away timeout** → a timer calls `cancel()` after ~60 s through the same state machine.
6. **What did you leave out?** Card payment, network reporting, refrigeration, discounts — say they are scope decisions.

Real-world example

Cashless readers (**Nayax**, **Cantaloupe ePort**) plug in as another payment device; coin mechanisms light **"exact change only"** when a tube runs low — our planner saying it cannot make change.

In the interview

45 minutes: 5 questions → 5 entities → **10 draw the state diagram and table** → 15 code the interface, two or three states and `checkPaid`/`refundAll` → 10 edge cases.

30-second answer: "The question is the order of events, so I use the State pattern: Idle, Selected, HasMoney, Dispensing, OutOfStock, all implementing an interface listing every button. The machine forwards to its state. Money goes into the box on insert, so a refund is 'make exact change for the balance'; if exact change is impossible the sale is refused and everything returned. Integer cents."

- *"Why not an enum with a switch?"* — Every new state means editing every switch; a State class is forced by the compiler to answer every button.
- *"Is greedy change correct?"* — Only for well-stocked canonical coin sets; use bounded coin-change DP.

Self-check

1. **Q:** What makes the design complete? **A:** Every state implements every button, so every (state, button) cell has an answer.
2. **Q:** Why is `restock` not on `VendingState`? **A:** Not a customer action, valid in every state (ISP).
3. **Q:** A greedy failure? **A:** 30¢ with one quarter and three dimes.
4. **Q:** How is "two items for one payment" prevented? **A:** Balance reset to 0 and Dispensing returns any coins until `take`.

TL;DR

- **State pattern is the design**: five states, one interface listing every button, no `if` on state.
- Count on the **slot**; **integer cents**; money into the **box immediately**.
- Change **exact or refuse**; greedy can fail, **DP** is correct.
- One refund path; operator actions off the state interface.

14.3 Design a Coffee Vending Machine

A **coffee machine** makes drinks (espresso, americano, latte, cappuccino, mocha, hot chocolate) from **shared ingredients** (beans, water, milk, chocolate, sugar). Unlike 14.2, a drink **consumes several ingredients at once**, and many drinks draw on the same milk. The interviewer wants: (1) **recipes as data** — a new drink is a data change; (2) **atomic, all-or-nothing deduction** — a latte takes beans, water *and* milk together or nothing, never negative even with ten concurrent orders; (3) **low-stock alerts** via Observer before an ingredient runs out.

Problem & clarifying questions

Question	Assumed answer
Drinks?	Six, loaded from a menu file.
Customisation?	Extra espresso shot (+50¢), sugar in grams.
Payment?	Amount in cents; change returned (coins covered in 14.2).
Concurrent orders?	Yes — several touch screens.
Refills?	Operator, alerted at a threshold.
Alert frequency?	**Once per crossing** of the threshold, re-armed after refill.

Requirements

Functional: (1) drinks with a **price** and a **recipe**; (2) **menu** with prices and availability; (3) selection and **payment**, **change**, full refund if the drink can't be made; (4) track inventory, deduct exactly the recipe; (5) **low-stock notifications**; (6) **refills**; (7) **customisation** without changing the menu.

Non-functional: thread-safe (no over-draw, no half recipe); data-driven; observable.

Use cases

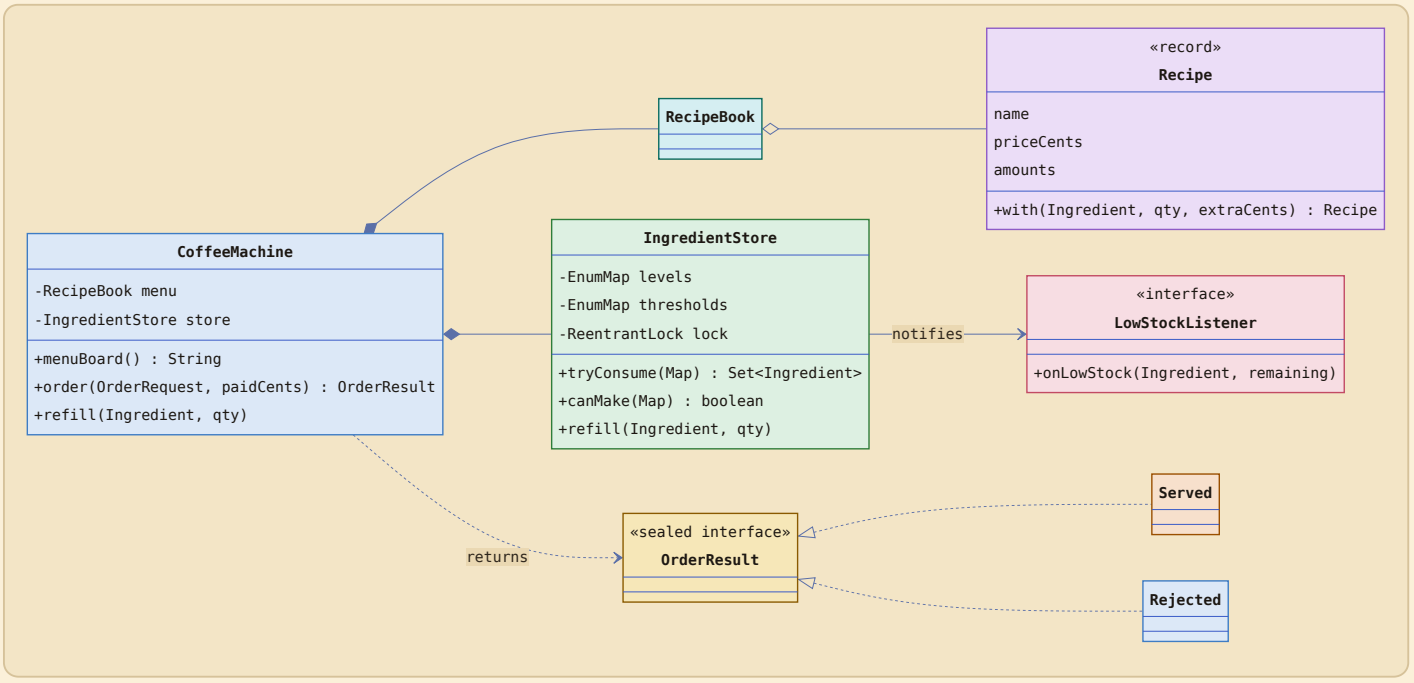
Actors: Customer (view menu, order, pay, take drink/change), Operator (refill, receive alerts), System (check stock, deduct, brew, alert). **Use cases:** view menu · order (with customisation) · pay / change / refund · refill · receive low-stock alert.

Core entities

Class	Responsibility
`Ingredient` (enum)	Beans, water, milk, chocolate, sugar — with unit (g/ml).
`Recipe` (record)	**Data**: name, price, `Map<Ingredient,Integer>`. Immutable; `with(...)` returns a customised copy.
`RecipeBook`	Parses menu text into recipes; lookup by name.
`IngredientStore`	Levels, thresholds, **one lock**; `tryConsume` = check all + deduct all atomically; alerts.
`LowStockListener`	Observer called once per threshold crossing.
`OrderRequest` (record)	Drink + extra shots + sugar grams.
`OrderResult` (sealed)	`Served(drink, price, change)` or `Rejected(reason, refund)`.
`CoffeeMachine`	Facade: menu board, `order(...)`, `refill(...)`.

Mapping to the common problem statement (*awesome-low-level-design*): `Coffee` → `Recipe`; `Ingredient` with `synchronized` updates → enum + levels in `IngredientStore`; `Payment` → `paidCents` and `OrderResult`; Singleton → injected machine. We lock the **whole recipe**, not each ingredient (per-ingredient locks can't make a multi-ingredient deduction all-or-nothing).

Class diagram



Key flows

Ordering: find recipe → apply customisation (`recipe.with(...)`) → if underpaid, `Rejected` with full refund → `tryConsume(amounts)`: lock, check **every** ingredient; if any shortage, nothing is deducted and the order is `Rejected`; else deduct all, note thresholds crossed, **unlock, then** notify listeners → `Served` with change.

Order life cycle: `Selected → Reserved (ingredients deducted atomically) → Brewing → Served`, or `Selected → Rejected` (unknown drink, underpaid, or short) with full refund.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
Recipes are **data** from menu text	Data-driven, OCP	New drink = new line	`Latte extends Coffee` (class explosion)
Check-and-deduct under **one lock**	Atomicity (Ch. 12)	Stops two orders both seeing "enough milk" (check-then-act)	Separate check and deduct; per-ingredient locks
All-or-nothing	Transaction semantics	No half-made recipe	Deduct one by one and roll back
`LowStockListener`	**Observer**	Operator app, dashboard, telemetry subscribe	Store calling an SMS class
Alert **once per crossing**	—	No alert storm	Alert on every order below threshold
Listeners **outside** the lock	Concurrency hygiene	Slow listener can't block orders; no deadlock on callback	Notifying inside the lock
Customisation returns a **new** `Recipe`	Immutability	Menu recipe never corrupted	Mutating it; a Decorator chain (heavier)
Sealed `OrderResult`	Algebraic data type (Java 17+/21 switch)	Callers must handle both; exhaustive `switch`	`null` or exceptions for normal refusals
No Singleton	DI	Tests build fresh machines (demo builds two)	`getInstance()`

> **Decorator or data?** *Head First Design Patterns* models add-ons as Decorators (`new Whip(new Mocha(new DarkRoast()))`). Good when add-ons have **behaviour**. Here they only change **quantities and price**, so `recipe.with(...)` is simpler. Say both and pick one for a reason.

Java implementation

Package `lld.ch14.coffeemachine`.

```java
/** A recipe is DATA: a name, a price and the amount of each ingredient. */
record Recipe(String name, int priceCents, Map<Ingredient, Integer> amounts) {
    // ...
    /** Customisation returns a NEW recipe; the menu entry is never modified. */
    Recipe with(Ingredient extra, int qty, int extraCents) {
        if (qty == 0) return this;
        Map<Ingredient, Integer> m = new EnumMap<>(Ingredient.class);
        m.putAll(amounts);
        m.merge(extra, qty, Integer::sum);
        return new Recipe(name, priceCents + extraCents, m);
    }
}

final class IngredientStore {
    // ...
    /** Returns the ingredients that are short. Empty set = success, everything was deducted. */
    Set<Ingredient> tryConsume(Map<Ingredient, Integer> need) {
        Set<Ingredient> shortages = EnumSet.noneOf(Ingredient.class);
        List<Runnable> alerts = new ArrayList<>();
        lock.lock();
        try {
            need.forEach((i, q) -> { if (levels.get(i) < q) shortages.add(i); });
            if (!shortages.isEmpty()) return shortages;               // nothing touched
            need.forEach((i, q) -> levels.merge(i, -q, Integer::sum));
            for (Ingredient i : need.keySet()) {
                int left = levels.get(i);
                if (left <= thresholds.get(i) && alerted.add(i)) {    // alert once per crossing
                    alerts.add(() -> listeners.forEach(l -> l.onLowStock(i, left)));
                }
            }
        } finally {
            lock.unlock();
        }
        alerts.forEach(Runnable::run);   // call listeners OUTSIDE the lock
        return shortages;
    }
    // ...
}
```

Full runnable code: code/lld/ch14/coffeemachine/ (see the Full Edition).

How it works: the menu is a Java text block (`Latte, 400, COFFEE_BEANS=18, WATER=40, MILK=180`) parsed by `RecipeBook.parse`. `Recipe`'s compact constructor stores an unmodifiable `EnumMap`. `refill` clears the `alerted` flag when the level goes back above the threshold. `CoffeeMachine.order` validates drink → customises → checks payment → reserves ingredients.

Output (excerpt):

```
REJECTED out of [MILK], refund 400 c
   beans left 410g (unchanged by the rejected latte)
== 10 customers order a Latte at the same instant (milk for exactly 4) ==
   ALERT to operator: MILK low, 80ml left
served=4 rejected=6 milk left=80ml (never negative)
```

Concurrency & edge cases

Situation	Risk	Handling
Ten orders, milk for four	Negative stock (check-then-act)	Check + deduct under one lock
One ingredient short	Beans taken, milk missing	Shortages found before any deduction
Slow listener (SMS)	Orders wait on the network	Listeners after `unlock()`
Many orders below threshold	Alert storm	One alert per crossing; refill re-arms
Underpayment	Drink made, money short	Payment checked before reserving
Brew fails after deduction	Pays, gets nothing	Extension: refund, optionally refill the amounts (compensation)
Menu says available, order fails	Display/order race	Menu is advisory; `tryConsume` is the truth

One lock is right: the critical section is tiny and the machine brews one drink at a time. For a cloud inventory shared by many machines, use optimistic concurrency (version / CAS on a snapshot) or a DB transaction.

Extensions & follow-up questions

1. **New drink** → one menu line. 2. **Sizes** → `recipe.scaled(size)`. 3. **Several brewers** → a `BlockingQueue` consumed by N `Brewer` threads, one store. 4. **Remote monitoring** → another `LowStockListener`. 5. **Different brewing steps** → Template Method or `Step` objects (Command). 6. **Cash** → reuse 14.2's `CashBox`.

Real-world example

The Thermoplan Mastrena (Starbucks) stores **recipes as configuration**; Starbucks and Microsoft (2019, Azure Sphere) push recipes and collect telemetry from the cloud; connected kiosks report stock remotely — the low-stock Observer.

In the interview

40 minutes: 5 questions → 5 entities → 5 "recipes are data" and the menu format → 15 code `tryConsume` and `order` → 10 concurrency and extensions.

> **30-second answer:** "Recipes are data — ingredient-to-amount maps loaded from a menu, so a new drink is a new row. The store checks and deducts **every** ingredient of a recipe under one lock: all-or-nothing, never negative. Crossing a threshold is noted and observers are notified once, after the lock is released. Orders return a sealed Served/Rejected result; rejected orders are refunded in full."

- *"Why not an `AtomicInteger` per ingredient?"* — Each is safe alone, but a recipe updates several at once; without a common lock you get partial deductions.
- *"Why notify outside the lock?"* — A slow or re-entrant listener would block all orders or deadlock.

Self-check

1. **Q:** All-or-nothing for `tryConsume`? **A:** Every ingredient deducted, or none (shortages detected first).
2. **Q:** Which race does the lock prevent? **A:** Check-then-act: two threads see enough milk and both deduct.
3. **Q:** How is an alert storm prevented? **A:** An `EnumSet` of alerted ingredients, re-armed by refill.
4. **Q:** When use Decorator for add-ons? **A:** When they carry behaviour, not just quantities.

TL;DR

> - **Recipes are data**; customisation makes a new immutable recipe.
> - **One lock, check-then-deduct, all-or-nothing**.
> - **Observer** alerts, **once per crossing**, **outside** the lock.

- **Sealed** `OrderResult`; full refund on rejection.

14.4 Design an ATM

An **ATM** is a cashier-less machine where a bank's customers do everyday banking outside branch hours. The interviewer checks that you keep **money correct** (never dispense without debiting, never debit without dispensing, never let two machines overdraw one account), **authenticate first**, and model the **session states** cleanly.

Problem & clarifying questions

Question	Assumed answer
Accounts / identification?	Checking and savings (daily cap); card + PIN, checked by the bank on every transaction.
Deposits available at once?	Checks **not** — they wait. Keep **total** and **available** balances.
Notes?	$100, $50, $20, $10 cassettes.
Wrong PIN?	3 wrong attempts block the card and the ATM keeps it (course leaves this out; we add it plus a journal).
Customers at a time?	One; the ATM must not shut down mid-session.

Requirements

Parts: card reader, keypad, screen, cash dispenser, deposit slot (cash/check), printer, network link to the bank.

Functional: (1) card + PIN authenticated **by the bank** first; (2) five transactions: balance inquiry, cash deposit, check deposit, withdraw, transfer; (3) the ATM **keeps the card** all session; cancel ejects it; (4) a **journal** of transactions and hardware faults; (5) a **check is held** until verified (the course holds all deposits; we credit counted cash at once); (6) **bank manager** reports.

Actors' operations: Operator — on/off key switch, refill cash, paper, ink, empty deposits. Customer — the transactions. Bank manager — total deposits/withdrawals reports, print them, check remaining cash.

Non-functional: security (no clear PINs on the ATM; limited attempts); consistency (debit and dispense together or compensate); concurrency (many ATMs, one account, no overdraft, no deadlock); auditability.

Use cases

Only Customer use cases need the PIN (Operator uses a key; Manager reads bank data). The course's activity diagrams (authenticate, withdraw, deposit check, transfer) all end by ejecting the card — **every** exit path, including declined retries, must return it.

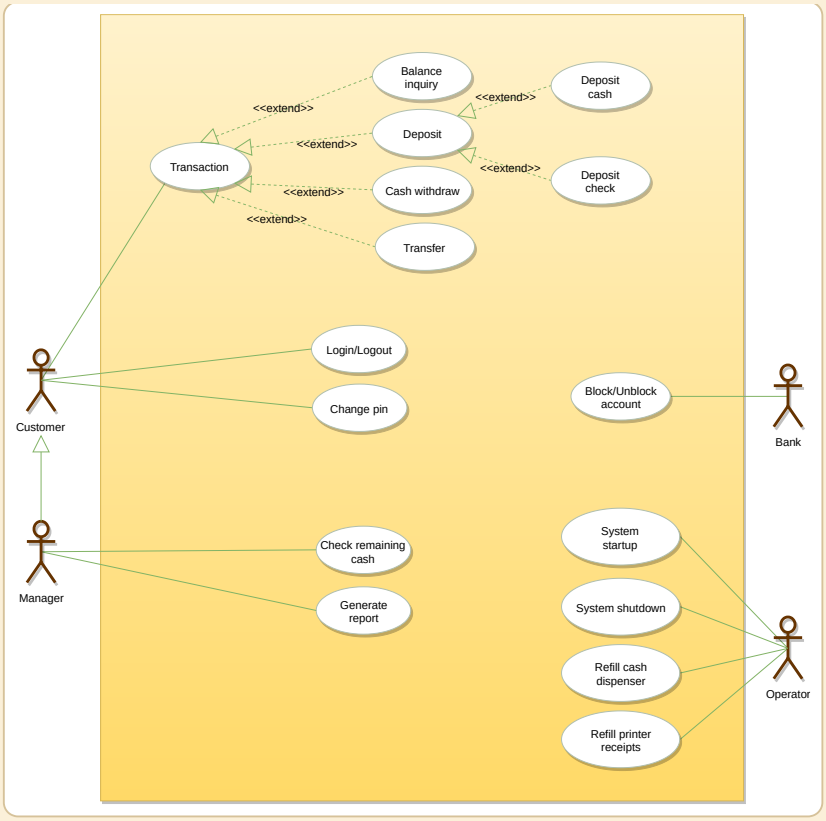


Figure — ATM use case diagram: Customer (balance, withdraw, deposit, transfer), Operator (on/off, refill cash, paper, ink, empty deposits) and Bank Manager (reports, check remaining cash) (course diagram)

Core entities

Class	Responsibility
`Card` (record)	Number and holder (course also has `Customer` with one card).
`Account` (abstract)	Id, **total** and **available** balance, a lock, withdrawal-rule hook.
`CheckingAccount`, `SavingsAccount`	Savings adds a **daily cap** — a behaviour difference, hence subclasses.
`BankService` (interface)	Authenticate, balances, withdraw, reverse, deposits, transfer.
`InMemoryBank`	Stand-in bank: PIN hashes, failed attempts, account locks, ordered-lock transfers.
`AuthResult` (sealed)	`Ok(accounts)`, `WrongPin(attemptsLeft)`, `Blocked`.
`NoteHandler` + 4 subclasses	**Chain of Responsibility** $100 → $50 → $20 → $10.
`CashDispenser`	Plans (dry run), then dispenses; can jam (fault injection).
`Transaction` (sealed) + 5 records	**Command**: one `execute(ctx)` each.
`AtmState` + 4 states	**State**: Idle → CardInserted → Authenticated, plus OutOfService.
`Atm`	Id; owns parts, session, journal; forwards buttons to state. Course hardware (`CardReader`, `Keypad`, `Screen`, `Printer`, `DepositSlot`) is folded in.

Class diagram

Course stages: (1) **money and card** — two balances (a check raises *total* now, *available* after clearing); savings overrides one method (behaviour ⇒ inheritance); (2) **the parts** — none knows what a transaction is; whether $85 can be made from loaded notes is the **dispenser's** job; the reader **keeps the card** all session; `DepositSlot` (the opening) ≠ the deposit *transaction*; (3) **five transactions**, one `execute` each; the course's

`Withdraw` checks **denominations** → **account** → **dispenser** and debits only after notes are counted; (4) **the machine** owns its parts and holds the session (one customer at a time); the `Bank` owns its ATMs.

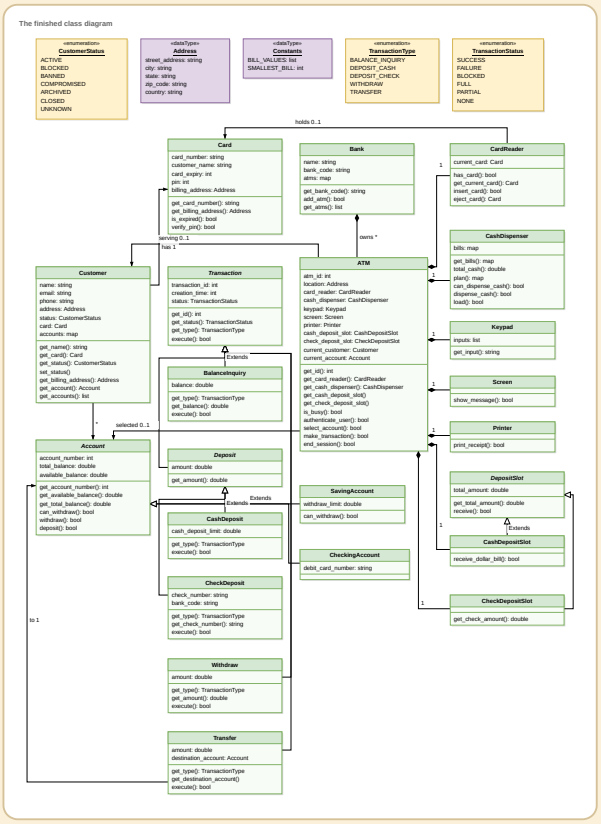
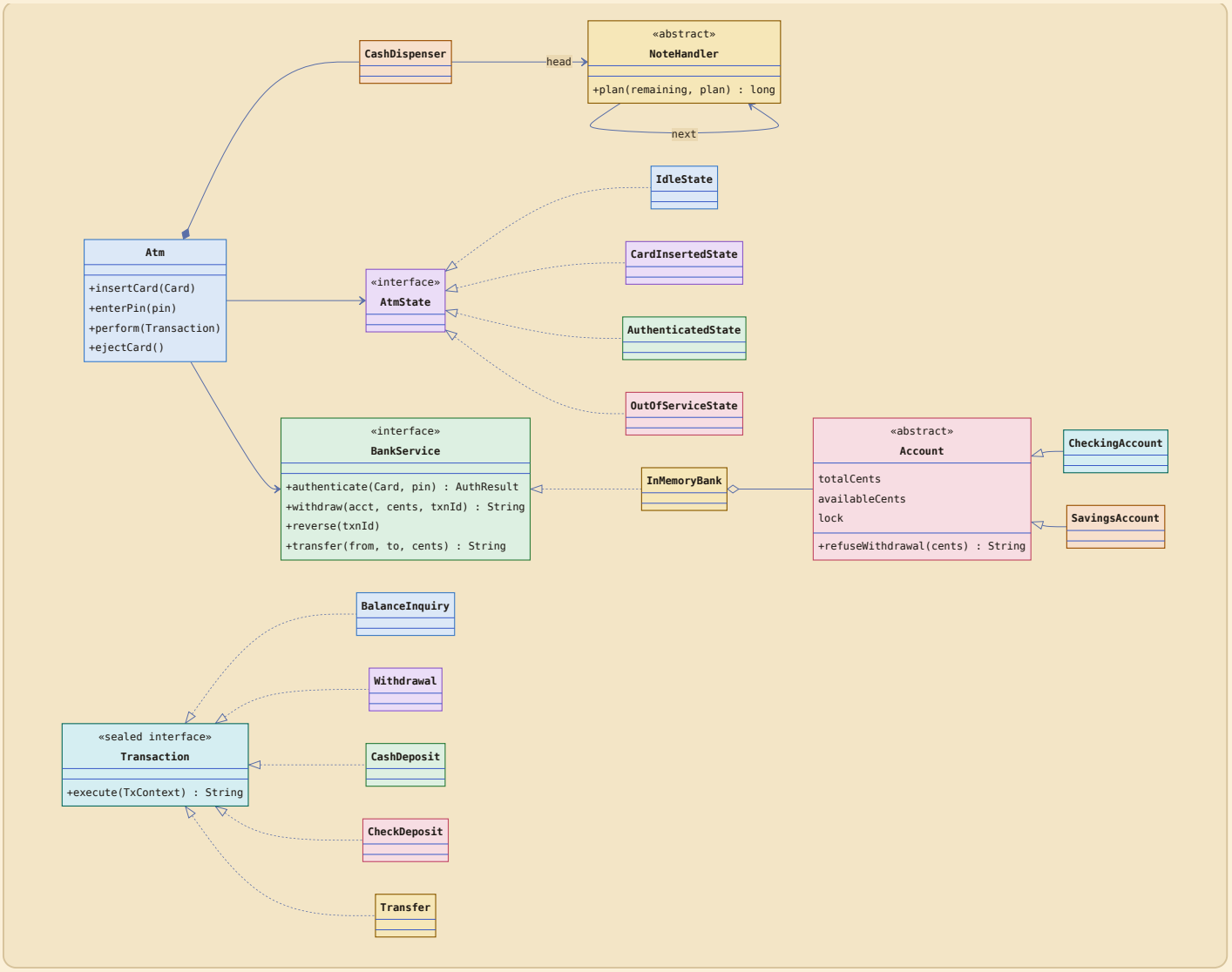


Figure — The finished class diagram of the ATM system (course diagram)



Key flows

Session states: Idle → CardInserted → Authenticated (PIN OK) → Idle (finish/cancel, card ejected). **3rd wrong PIN → Idle, card retained.** Session ends with no cash → OutOfService (card out first); key switch only when Idle; refill → Idle. **Balance inquiry:** the ATM creates a transaction, which asks the account; the result goes to the screen.

Withdrawal with compensation (our code): multiple of $10? → `plan(280)` dry run on the note chain (2×100, 1×50, 1×20, 1×10) → bank `withdraw` (lock account, check rule, debit) → `dispense(plan)` → on **jam**, `reverse(txnId)` and report "account NOT charged" → journal the result.

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Session as state classes	**State**	Each button means something different per state	Booleans + nested `if`s
Dispenser as chain of note handlers	**Chain of Responsibility**	Each takes what it can, passes the rest; a $5 cassette is one new handler	Hard-coded note list
Plan first, then dispense	—	Dry run touches nothing	Removing notes while searching
Each transaction = one class	**Command**	Rules in one place; can be journaled, queued, replayed	`switch (type)` in the ATM
`Transaction`, `AuthResult` sealed	Sealed types	Exhaustive `switch`	`instanceof` chains
ATM → `BankService` interface	Dependency Inversion	In-memory, ISO 8583, or a test fake	ATM changes balances
Total vs available	Domain modelling	Check holds correct	Single balance
Savings subclass	Inheritance for **behaviour**	Overrides one hook	`if (type == SAVINGS)`

> **Debit before or after dispensing?** The course debits **after** counting notes (safer for the customer) and admits a crash in between leaves the bank short. Our code debits then dispenses, with a **compensating** `reverse(txnId)` on a jam. Real systems make the pair atomic (**hold/authorise, dispense, confirm**) and **reconcile** from the journal. This is the "power fails halfway" follow-up.

> **Cash deposits.** Many banks make machine-counted cash available immediately and hold checks. `depositCash` credits both balances, `depositCheck` only the total — a one-line policy change in the bank, not the ATM.

Java implementation

Package `lld.ch14.atm`. Amounts are cents (`long`).

```
abstract class NoteHandler {
    // ...
    /** Dry run: fills 'plan', returns the amount that nobody in the chain could cover. */
    long plan(long remaining, Map<Long, Integer> plan) {
        int use = (int) Math.min(count, remaining / noteCents);
        if (use > 0) plan.put(noteCents, use);
        long left = remaining - use * noteCents;
        return (left == 0 || next == null) ? left : next.plan(left, plan);
    }
    // ...
}

record Withdrawal(String account, long cents) implements Transaction {
    static final long SMALLEST_NOTE = 10_00;

    public String execute(TxContext c) {
        if (cents <= 0 || cents % SMALLEST_NOTE != 0) return "Withdrawal refused: amount must be a multiple of
$10";
        var plan = c.dispenser().plan(cents);                                   // 1. can the notes make it?
        if (plan.isEmpty()) return "Withdrawal refused: ATM cannot make " + Money.fmt(cents) + " from its
notes";
        String refusal = c.bank().withdraw(account, cents, c.txnId());          // 2. atomic debit at the
bank
        if (refusal != null) return "Withdrawal refused: " + refusal;
        try {
            c.dispenser().dispense(plan.get());                                 // 3. hand out the cash
        } catch (IllegalStateException hardwareFault) {
            c.bank().reverse(c.txnId());                                        // compensate: undo the debit
            return "Withdrawal failed (" + hardwareFault.getMessage() + "), account NOT charged";
        }
        return "Dispensed " + Money.fmt(cents) + " as " + describe(plan.get());
    }
    // ...
}

final class InMemoryBank implements BankService {
    // ...
    /** Locks both accounts in a fixed (id) order, so two opposite transfers cannot deadlock. */
    @Override public String transfer(String from, String to, long cents) {
        if (cents <= 0) return "amount must be positive";
        if (from.equals(to)) return "same account";
        Account a = acc(from), b = acc(to);
        Account first = from.compareTo(to) < 0 ? a : b;
        Account second = first == a ? b : a;
        first.lock.lock();
        try {
            second.lock.lock();
            try {
                if (cents > a.availableCents) return "insufficient funds";
                a.availableCents -= cents;
                a.totalCents -= cents;
                b.availableCents += cents;
                b.totalCents += cents;
                return null;
            } finally {
                second.lock.unlock();
            }
        } finally {
            first.lock.unlock();
        }
    }
    // ...
}
```

Full runnable code: code/lld/ch14/atm/ (see the Full Edition).

How it works: the bank stores a SHA-256 hash of card+PIN and counts failures; the 3rd returns `Blocked` and the ATM **retains** the card (demo only — real banks verify an encrypted PIN block in an HSM). `withdraw` checks and debits under the account's `ReentrantLock`. `finishSession` picks Idle or OutOfService only after the card is out.

Output (excerpt):

```
   [ATM-7 screen] Dispensed $280.00 as 2x$100 + 1x$50 + 1x$20 + 1x$10
   [ATM-7 screen] Balance CHK-1: available $720.00, total $1220.00
   [ATM-7 screen] Withdrawal failed (note jam), account NOT charged
== Two ATMs, one account, same instant: $300 each from $500 ==
   cash handed out $300.00, JOINT left $200.00
== 2,000 opposite transfers in parallel (ordered locking: no deadlock) ==
   successful=2000, A+B=$2000.00
```

Concurrency & edge cases

Situation	Risk	Handling
Two ATMs, one account	Overdraft (check-then-act)	Check + debit under the **account lock**
A→B and B→A at once	**Deadlock**	Lock in a global order (account id)
Jam after debit	Charged, no cash	`reverse(txnId)` + journal
Amount not makeable	Partial dispense	Dry-run `plan` before touching the bank
Greedy chain misses a combination	$60 with one $50, three $20, no $10	OK for standard cassettes; DP planner (14.2) if asked
Many wrong PINs	Brute force	3 tries, block **at the bank**, retain card
Walks away / unlinked account	Misuse	Session timeout (extension) / `sessionAccounts` check
Cash runs out	Card trapped	Finish session, then `OutOfService`; refill → Idle
Key switch mid-session	Shuts down while serving	Refused unless idle

Extensions & follow-up questions

1. **Can't make the amount** → plan fails, refused before the bank is called.
2. **Power fails mid-withdrawal** → hold at the bank, dispense, confirm; on restart read the journal and confirm or reverse; bank reconciles.
3. **Daily limit for all accounts** → in `Account` or a policy object; ATM unchanged.
4. **Other banks' cards** → a routing `BankService` (Proxy/Adapter to the card network).
5. **Cardless** → a new state path `enterOneTimeCode`; transactions unchanged.
6. **Manager reports** → aggregate the journal; cash left = `dispenser.total()`. (Receipts / PIN change → more `Transaction` records.)

Real-world example

ATMs (NCR, Diebold Nixdorf) drive devices through **CEN/XFS** interfaces, talk **ISO 8583**, and use **ISO 9564** PIN blocks verified in HSMs; after a dispense error they reverse and reconcile from the journal.

In the interview

45 minutes: 5 parts, actors, transactions → 5 total vs available → 10 state diagram → 15 code `Withdrawal.execute` and the chain → 10 concurrency and follow-ups.

> **30-second answer:** "Parts behind interfaces, the session as a State machine, five transactions as Commands with one `execute`. The ATM never touches balances; it calls a `BankService`. Accounts have total and available balances so checks can be held. Withdrawal plans notes through a Chain of Responsibility,

debits atomically at the bank, dispenses, and reverses on hardware failure. The bank locks per account; transfers lock in id order to avoid deadlock."

- *"Where is the PIN checked?"* — At the bank (really an HSM, from an encrypted PIN block).
- *"Two ATMs, joint account?"* — Check and debit together under the account's lock (or one DB transaction).

Self-check

1. **Q:** Why two balances? **A:** A check raises the total at once but must not be spendable until it clears.
2. **Q:** Safe order of debit and dispense? **A:** Neither alone; use hold + confirm (or debit + compensating reverse) and reconcile from the journal.
3. **Q:** How are transfer deadlocks avoided? **A:** Always lock both accounts in the same global (id) order.
4. **Q:** What happens after the 3rd wrong PIN? **A:** The bank blocks the card (every ATM refuses it) and this ATM retains it.

TL;DR

- **State** (session), **Command** (transactions), **Chain of Responsibility** (cassettes), `BankService` interface.
- **Total vs available** balance.
- Withdrawal = **plan → atomic debit → dispense → reverse on failure**, journaled.
- **Per-account lock**, transfers in **id order**, PIN attempts limited at the bank.

14.5 Design an Elevator System

A building's cars are run as a **group** by a shared controller. Two decisions carry the interview: **which car answers a hall call** (dispatching) and **in what order one car visits its stops** (scheduling).

Problem & clarifying questions

Question	Assumed answer
Floors and cars?	Floors 0 … N-1, several cars serving all.
Buttons?	Hall UP/DOWN per floor (no UP on top, no DOWN on ground); car panel per floor plus door open/close.
Displays?	In car: floor, direction, load. Per floor: one display per car.
Scheduling?	Serve every stop ahead before turning — **LOOK**.
Dispatching?	Nearest suitable car; swappable.
Capacity?	Overloaded car doesn't move and says so.
Maintenance, emergencies?	Out-of-service cars; fire alarm recalls all to the lobby.
Real time?	Simulated: one `step()` = one action (testable without a clock).

Requirements

Functional: (1) many floors and cars; (2–3) hall and car panels; (4–5) car and floor displays; (6) a hall call **picks one car**; (7) a car button **adds a stop**; (8) **serve stops ahead** before turning; (9) **never move with doors open**; (10) **max load** — overloaded car stays and says so; (11) idle car waits where it finished; (12) **out of service** cars get no riders; (13) *(ours)* **fire recall**.

Non-functional: efficient (no useless trips to the shaft end); no starvation; safety overrides scheduling; thread-safe; deterministic stepping.

Use cases

Actors: Rider (call, ride, choose destination), Maintenance engineer (out of / back in service), System (choose car, move, doors, displays). **Use cases:** call · choose destination · dispatch · move one floor · open/close doors · report overload · take out of service. Dispatch and move belong to the System; the rider only triggers them.

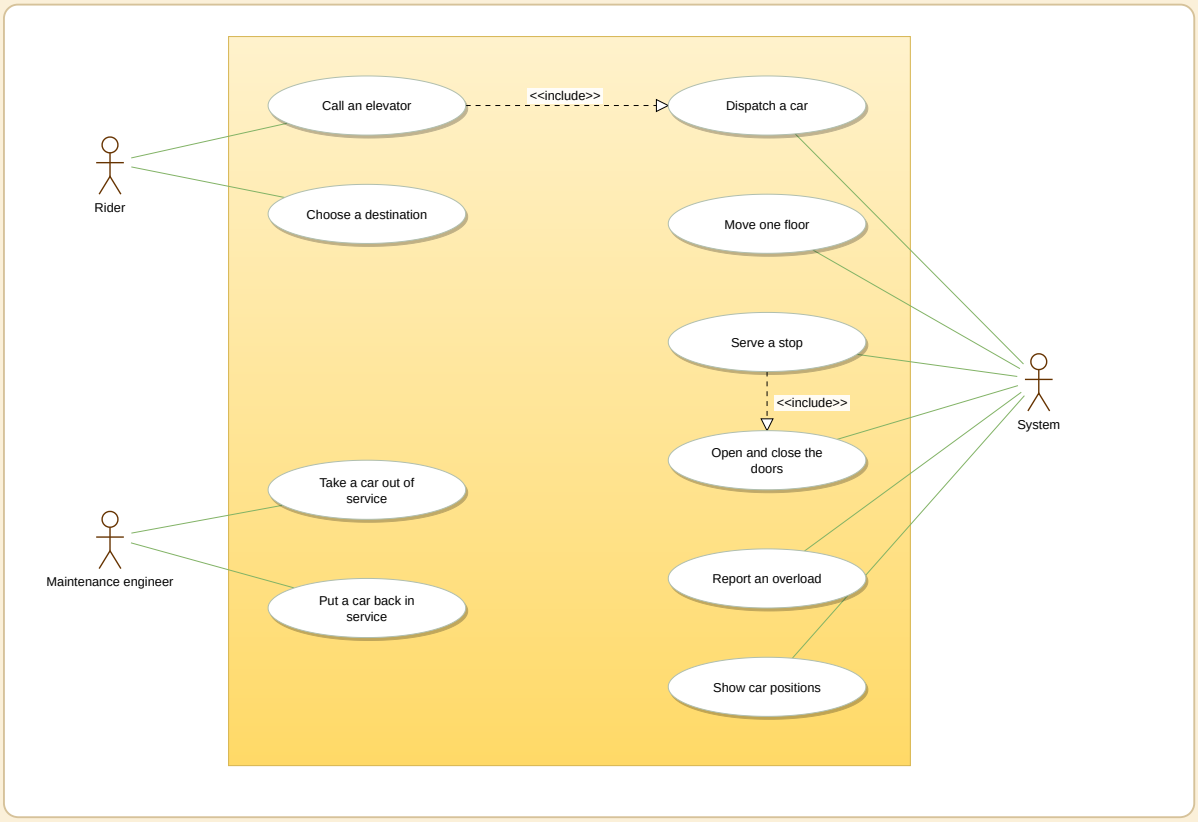


Figure — Use case diagram of the elevator system: Rider, Maintenance engineer and System (course diagram)

Core entities

Class	Responsibility
`Direction` (enum)	UP, DOWN, IDLE.
`Stop` (record)	Floor **plus** direction on leaving — "5 going up" and "5 going down" are two stops.
`StopList`	Stops in **two sorted sets** (up, down); `next(...)` implements LOOK.
`Door`	Open or closed.
`Elevator`	Floor, direction, mode, load, door, stops; `step()` does one action.
`Mode` (enum)	NORMAL, MAINTENANCE, FIRE_RECALL.
`Event` (record)	What a step did (MOVED, ARRIVED, DOORS_CLOSED, OVERLOAD, NOTHING).
`DispatchStrategy` / `NearestCarStrategy`	*Which car* answers a hall call.
`Dispatcher`	Cars + one strategy; queues calls no car can take.
`Building`	Hall lights, cars, dispatcher; ticks cars; clears lights on arrival; fire alarm.

The course also models `Button`, `HallPanel`, `ElevatorPanel`, `Display` and `Floor` one-to-one; our code folds them — say which you chose.

Class diagram

Course stages: (1) `Button` family (shared light/reset behaviour) and `Door`; (2) panels, displays, `Floor` — all **composition**; (3) **most important:** `Stop` **and** `StopList` — two sets, not one list; the next stop is reachable **without reversing**, and with nothing ahead the car heads to the **farthest** request; `Elevator` **moves**, `StopList` **chooses**; (4) `Elevator` — `step` does **one** of: close doors, refuse (overload), move one floor, stop and open; it never touches a `Floor` or another car; (5) `DispatchStrategy`, `Dispatcher`, `Building` — the only class knowing cars and floors, so it clears hall lights.

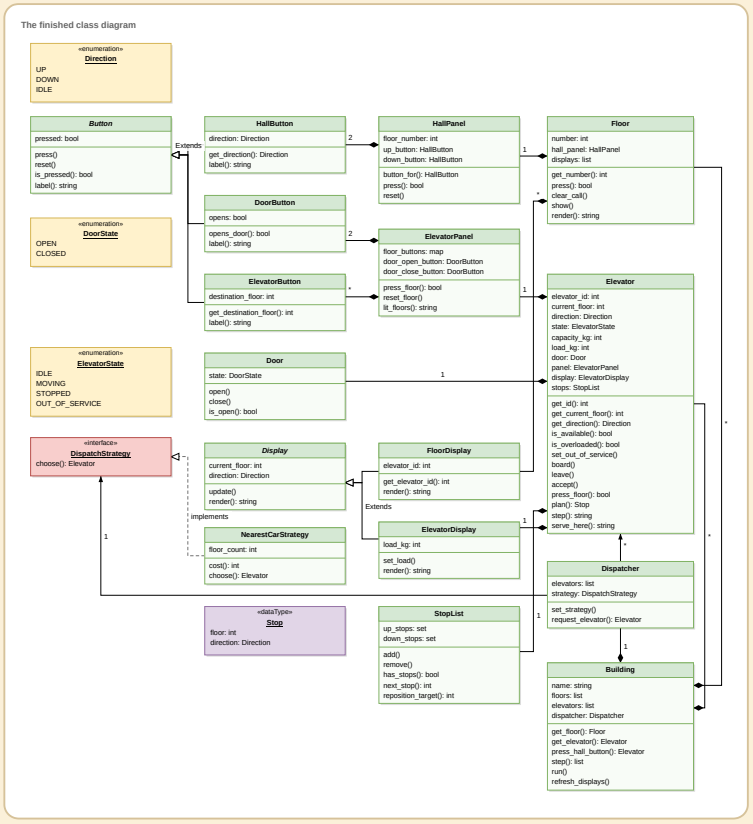
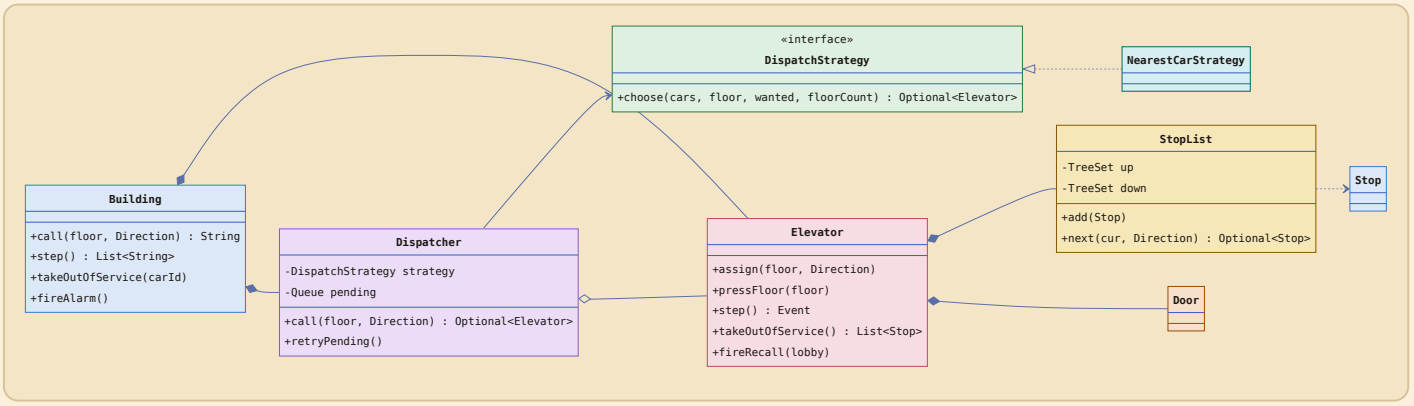


Figure — The finished class diagram of the elevator system (course diagram)



Key flows

Scheduling algorithms compared (borrowed from disk-arm scheduling — same shape: a head moving along a line of requests):

Algorithm	Rule	Good	Bad
FCFS	Arrival order	Simple	Zig-zags; very inefficient
SSTF	Nearest stop first	Short trips	**Starvation** of far floors
SCAN	Sweep to the **end of the shaft**, reverse	No starvation	Wasted travel to the ends
LOOK	Sweep only **to the last request**, reverse	No starvation, no waste	Slightly more bookkeeping
C-SCAN / C-LOOK	One direction; jump back to start	Uniform waits	Empty return trips

Our `StopList.next` (LOOK, direction-aware), going UP: (1) nearest up-stop at or above (`up.ceiling(cur)`); (2) else, if someone *above* wants DOWN, go to the **highest** such floor, then turn; (3) else turn: nearest down-stop at or below (`down.floor(cur)`); (4) else go down to the **lowest** up-stop and serve upward. DOWN mirrors it; an IDLE car picks the side of the nearest stop.

Car states: Idle → Moving → DoorsOpen (overload: stay) → Moving/Idle; Idle ⇄ Maintenance; any → FireRecall. **Hall call:** light button → strategy picks a car → `assign` → `step()` each tick → on ARRIVED, clear the light.

Dispatch cost (`NearestCarStrategy`, from the course): idle car = distance; car already coming this way **in the same direction** = distance; any other car = distance **+ two building heights**. Out-of-service cars are skipped.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
`DispatchStrategy`	**Strategy**	Office rush vs quiet hotel; the *requirement* to swap justifies the interface	`if (officeMode)`
Sweep rule in `StopList`	SRP	Car drives, stop list decides	Next-stop logic in `step()`
Two sets (up/down)	Data modelling	"5 up" ≠ "5 down"	One list
`step()` = one action	Deterministic simulation	Tests = calls + expected events, no clock	Thread per car with `sleep`
Modes as an enum	Lightweight State	Only 3 modes; switch to State classes if modes grow	State classes from day one
Re-dispatch when a car leaves service	Robustness	Accepted calls never lost	Silently clearing stops
Queue calls when no car is free	—	"Not now" ≠ "rejected"	Dropping the call

Course's honest weak points: (1) **no threads** — a real controller locks the stop list (ours is `synchronized`); (2) the dispatcher **decides once**; (3) `Elevator` nearly does two jobs, hence `StopList`. Only **one** pattern (Strategy); out of scope: emergency calls, fire service, express cars, door sensors.

Java implementation

Package `lld.ch14.elevator`.

```java
final class StopList {
    private final TreeSet<Integer> up = new TreeSet<>();
    private final TreeSet<Integer> down = new TreeSet<>();
    // ...
    Optional<Stop> next(int cur, Direction dir) {
        if (isEmpty()) return Optional.empty();
        if (dir == Direction.IDLE) dir = nearestSide(cur);
        return Optional.of(dir == Direction.UP ? goingUp(cur) : goingDown(cur));
    }

    private Stop goingUp(int cur) {
        Integer f = up.ceiling(cur);
        if (f != null) return new Stop(f, Direction.UP);                          // work ahead
        if (!down.isEmpty() && down.last() > cur) return new Stop(down.last(), Direction.DOWN); // go up, then
turn
        f = down.floor(cur);
        if (f != null) return new Stop(f, Direction.DOWN);                        // turn around now
        return new Stop(up.first(), Direction.UP);                                // go down to lowest up-
call
    }
    // ... goingDown is the mirror image
}

final class Elevator {
    // ...
    synchronized Event step() {
        if (mode == Mode.MAINTENANCE) return nothing();   // parked: never moves
        if (door.isOpen()) {
            if (mode == Mode.FIRE_RECALL && stops.isEmpty()) return nothing(); // parked at lobby, doors open
            if (loadKg > capacityKg) {
                return new Event(Event.Kind.OVERLOAD, id, floor, direction,
                        "OVERLOAD " + loadKg + "kg > " + capacityKg + "kg, doors stay open");
            }
            door.close();
            return new Event(Event.Kind.DOORS_CLOSED, id, floor, direction, "doors close");
        }
        var next = stops.next(floor, direction);
        if (next.isEmpty()) {
            direction = Direction.IDLE;
            return nothing();
        }
        Stop target = next.get();
        if (target.floor() == floor) {                       // arrived
            stops.remove(target);
            direction = stops.isEmpty() ? Direction.IDLE : target.direction();
            door.open();
            return new Event(Event.Kind.ARRIVED, id, floor, target.direction(), "doors open");
        }
        direction = target.floor() > floor ? Direction.UP : Direction.DOWN;   // never moves with doors open
        floor += direction == Direction.UP ? 1 : -1;
        return new Event(Event.Kind.MOVED, id, floor, direction, "");
    }
    // ...
}
```

Full runnable code: code/lld/ch14/elevator/ (see the Full Edition).

How it works: `TreeSet` `ceiling` / `floor` / `first` / `last` give O(log n) "nearest stop" answers. `takeOutOfService` returns the car's stops for re-dispatch; `Dispatcher.retryPending` runs every tick.

Output (excerpt):

```
== 1. LOOK on one car: at floor 4 going up; stops 6(up) 9(car) 2(down) 1(up) ==
service order [6, 9, 2, 1]  (turns at 9 and at 1: never runs to 0 or to a floor above 9)
  t15: car2@9 OVERLOAD 775kg > 680kg, doors stay open
re-dispatch hall 4 DOWN -> car2
hall 2 UP -> queued (no car available)
```

Concurrency & edge cases

Situation	Handling
Buttons pressed while the controller steps	`synchronized` car and dispatcher methods; concurrent set of hall lights
Same hall button twice	`litHallButtons.add` false: "already lit", no second dispatch
Up and down on the same floor	Two `Stop`s in two sets
Car call for current floor, doors open	Ignored
Overload	Doors stay open, OVERLOAD shown, no movement
Car out of service with accepted calls	Re-dispatched or queued, never lost
No car available	Queued; retried every tick
Fire alarm	Stops dropped, lobby added, car buttons ignored, doors open at lobby
Starvation	LOOK serves every stop within one round trip
UP on the top floor	`IllegalArgumentException`

Extensions & follow-up questions

1. **Choosing a car** → `DispatchStrategy`: zoning, **up-peak** (idle cars to the lobby at 9 a.m.), energy saving.
2. **Simultaneous presses** → synchronized dispatcher + per-car lock (the course omits it).
3. **Testing** → presses, N steps, expected events; no sleeps.
4. **Destination dispatch** → a strategy taking (from, to), grouping nearby destinations.
5. **Re-evaluate / real time** → move unserved calls to a better car each tick; a `ScheduledExecutorService` calls `step()` every second.
6. **Emergency stop / more modes** → refactor modes into State classes.
7. **Express cars / door sensor** → served-floor sets the strategy checks; an obstruction makes `step()` reopen (same slot as overload).

Real-world example

Destination dispatch (Schindler PORT, Otis CompassPlus, KONE) = extension 4; controllers switch to **up-peak** mode; fire recall follows **Phase I emergency recall** (ASME A17.1).

In the interview

45 minutes: 5 requirements/scope → 5 entities → **15 stop list and LOOK** → 10 `step()` and dispatcher → 10 concurrency, testing, follow-ups.

30-second answer: "Each car keeps stops in two sorted sets, up and down, and uses LOOK: serve stops ahead, turn at the last request, never run to the shaft end for nothing. `step()` does one action — close doors, refuse if overloaded, move one floor, or stop and open — so the system is testable without a clock. A dispatcher assigns hall calls via a `DispatchStrategy` (nearest suitable car) and queues calls when no car is free. Doors, load, maintenance and fire recall override scheduling."

- *"Why LOOK, not SSTF?"* — SSTF can starve a far floor; LOOK serves everything within one round trip.
- *"Where's the lock?"* — Per car (its monitor) and around dispatch; no global lock across cars.

Self-check

1. **Q:** SCAN vs LOOK? **A:** SCAN runs to the shaft end; LOOK reverses at the last request.

2. **Q:** Car at 4 going up; stops 6(up), 9(car), 2(down), 1(up) — order? **A:** 6, 9, 2, 1.
3. **Q:** Why is scheduling in `StopList`? **A:** SRP: the car drives, the stop list decides; each testable alone.
4. **Q:** Every car in maintenance — what happens to a call? **A:** Queued, retried every tick.
5. **Q:** When would you move modes to State classes? **A:** When many modes behave differently per action.

TL;DR

- **Two sorted sets** + **LOOK** = efficient, starvation-free scheduling.
- **`step()` = one action**; doors, load, maintenance, fire recall override scheduling.
- **Strategy** for dispatch, pending queue, re-dispatch when a car leaves service.
- Per-car lock, synchronized dispatcher, concurrent hall-light set.

14.6 Design a Traffic Signal Control System

A **traffic signal controller** decides which **approaches** (roads entering: N, S, E, W) get green, for how long, and in what order. A **phase** is a set of approaches that may be green together because their paths do not cross (e.g. N and S straight). The controller cycles phases with **yellow** (warning) and **all-red** (clearance) between them. It tests a **timed state machine** with a **hard safety rule** — *two crossing flows are never green together* — plus an **emergency override** that interrupts the cycle *safely* (never cutting a yellow short).

Problem & clarifying questions

Question	Assumed answer
Roads?	Four-way (N, S, E, W); phases configurable.
Signals?	Red, yellow, green; flashing red in fault mode.
Timing?	Fixed, or adaptive from queue sensors.
Pedestrians, turn arrows?	Out of scope (extensions).
Emergency vehicles?	Green as soon as *safely* possible, held until they pass.
Many intersections?	One controller each; central coordination is an extension.
Real time?	1 tick = 1 s; tick-driven so it can be simulated.

Requirements

Functional: (1) multiple roads; (2) red/yellow/green; (3) **configurable, traffic-adaptive** durations; (4) **safe transitions**: green → yellow → all-red → next green; (5) **emergency** priority; (6) scalable/extensible; (7) fault → **all flashing red** (all-way stop).

Non-functional: safety invariant (never conflicting greens, not even for an instant); deterministic 1 s ticks; thread safety (timer vs sensor and emergency threads); observability (listeners for light changes).

Use cases

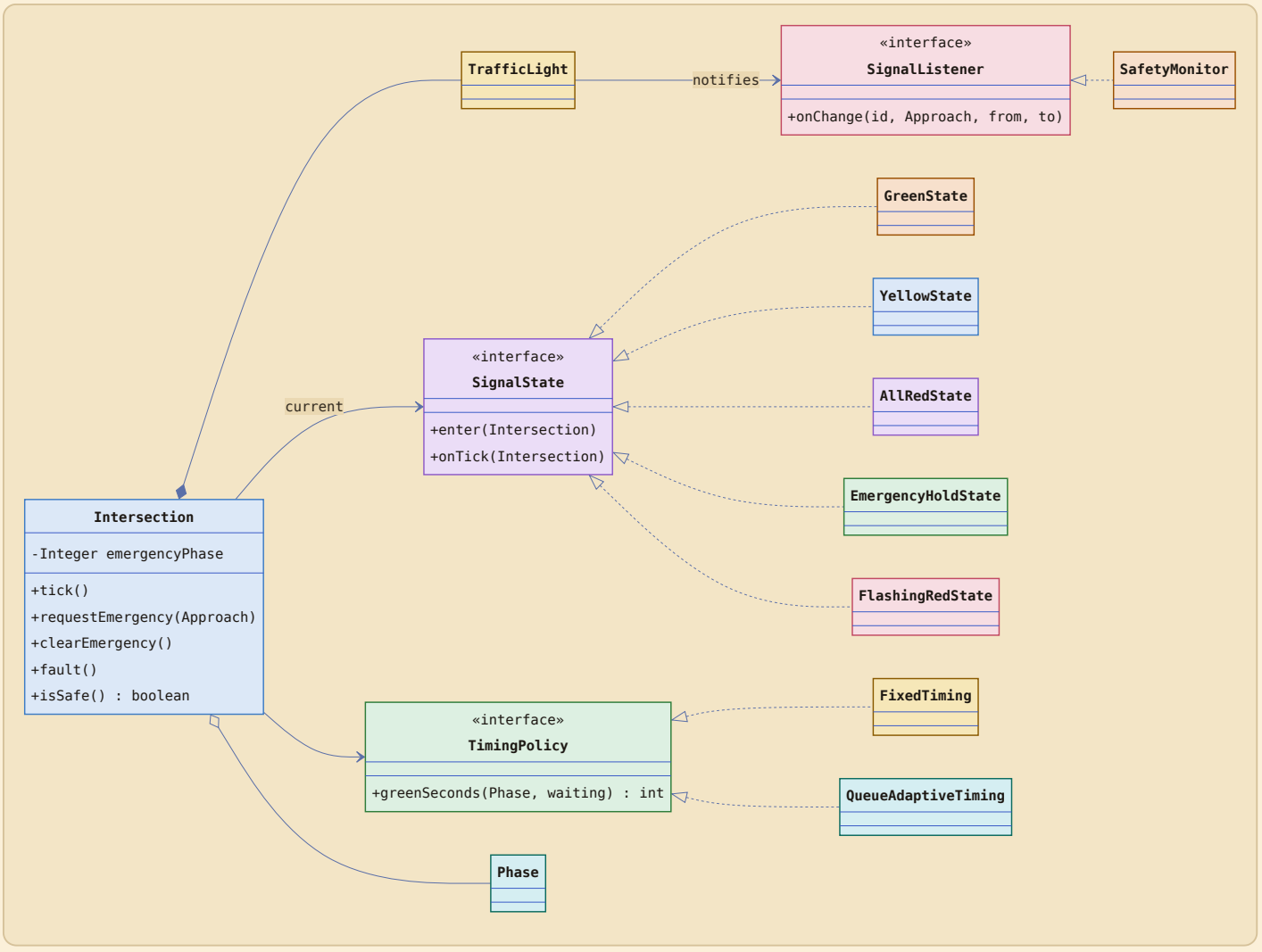
Actors → use cases: Timer → advance the cycle; Vehicle sensor → report waiting cars; Emergency vehicle → request/clear pre-emption; Traffic engineer → configure phases/timing, fault mode; Drivers see the lights.

Core entities

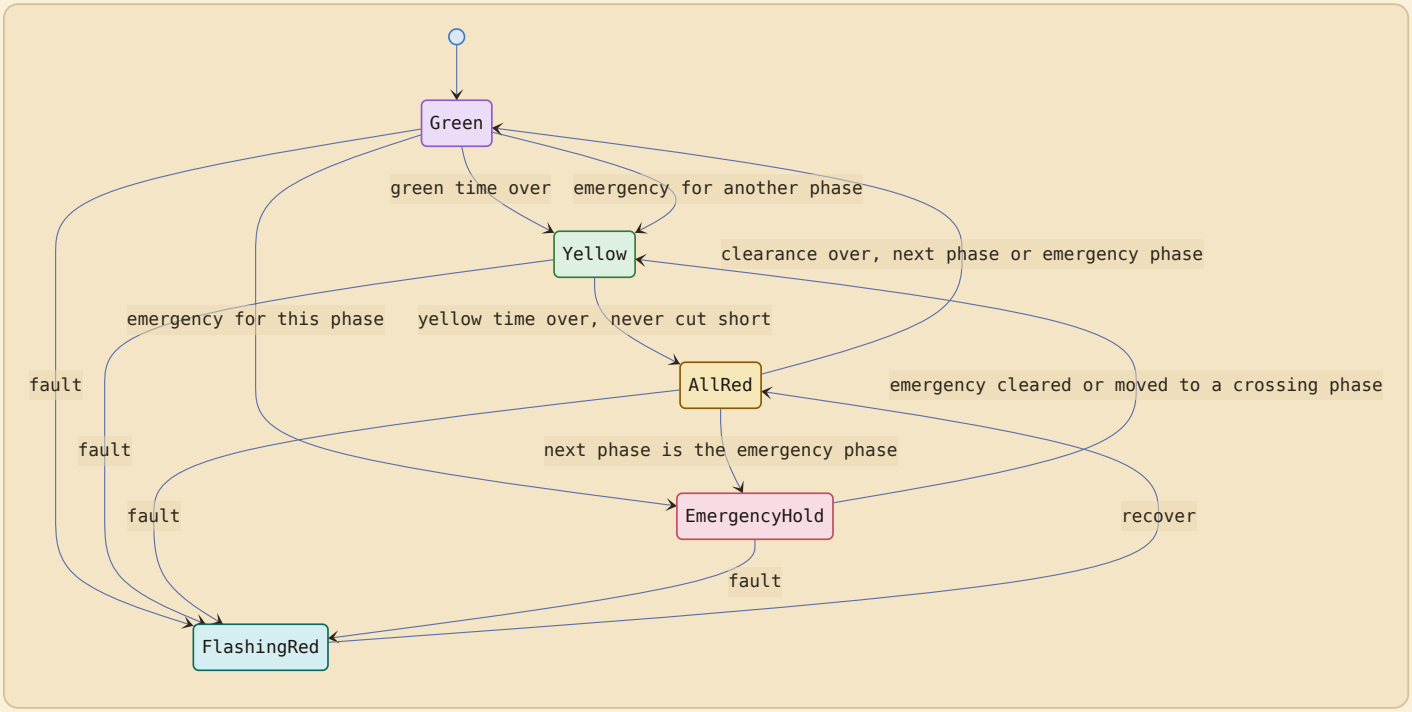
Class	Responsibility
`Signal` (enum)	RED, YELLOW, GREEN, FLASHING_RED.
`Approach` (enum)	NORTH, SOUTH, EAST, WEST.
`Phase` (record)	Name + set of approaches green together.
`TrafficLight`	One head for one approach; notifies listeners.
`SignalListener`	**Observer** for light changes.
`SafetyMonitor`	Listener checking the invariant after every change to green.
`TimingPolicy`	**Strategy**: `FixedTiming`, `QueueAdaptiveTiming`.
`SignalState` + 5 states	**State**: Green, Yellow, AllRed, EmergencyHold, FlashingRed.
`Intersection`	State context: phases, lights, timing, sensors, emergency flag; `tick()`.

Mapping to the common problem statement (*awesome-low-level-design*): `Road` → `Approach` + its `TrafficLight`; durations move into `TimingPolicy`; Singleton `TrafficController` → our `Intersection` (**not** a Singleton). We add what the reference leaves implicit: phases, all-red and the safety check.

Class diagram



Key flows



Emergency pre-emption: ambulance from WEST → `requestEmergency(WEST)` sets `emergencyPhase = East-West` → next tick N-S green ends **early** into yellow → yellow runs in full → all-red → `EmergencyHold(East-West)` → `clearEmergency()` → yellow, all-red, normal cycle.

One fixed cycle (green 5, yellow 2, all-red 1): N-S green 5 → N-S yellow 2 → all red 1 → E-W green 5 → E-W yellow 2 → all red 1 — **16 s**.

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Each interval a state class (`enter`, `onTick`)	**State**	Transitions and "yellow never cut short" live in one small class each	One `switch` with counters
States **carry data** (remaining seconds), new per transition	State with per-instance data	Unlike vending's stateless states, they hold a countdown	Shared singletons with counters on the context
Green time from `TimingPolicy`	**Strategy**	Fixed, adaptive, time-of-day swap at run time	Hard-coded in `GreenState`
Light changes published	**Observer**	Displays, logs, central monitoring, safety monitor	Calling each consumer
`showPhaseGreen`: others **red first**, then green	Safety by construction	Never an instant of conflicting greens	Arbitrary order
Separate `SafetyMonitor`	Defence in depth	Like hardware conflict monitors forcing flash	Trusting controller logic alone
Emergency is a **flag** states read	Safety	Only *shortens* green; yellow and all-red run in full	Jumping straight to emergency green
No Singleton controller	Rejected	Many intersections; a `TrafficControlCenter` can own them	`getInstance()`

Java implementation

Package `lld.ch14.trafficsignal`.

```java
/** State pattern: each state knows what one second of time means for it, and which state is next. */
interface SignalState {
    default void enter(Intersection x) {}
    void onTick(Intersection x);
    String name();
}

final class GreenState implements SignalState {
    // ...
    @Override public void onTick(Intersection x) {
        Integer em = x.emergencyPhase();
        if (em != null) {                                   // pre-emption: end this green early
            x.setState(em == phase ? new EmergencyHoldState(phase) : new YellowState(phase));
            return;
        }
        if (--remaining <= 0) x.setState(new YellowState(phase));
    }
    // ...
}

final class YellowState implements SignalState {
    // ...
    @Override public void onTick(Intersection x) {          // yellow is never cut short: that is unsafe
        if (--remaining <= 0) x.setState(new AllRedState(x.nextPhaseAfter(phase)));
    }
    // ...
}

final class Intersection {
    // ...
    synchronized void tick() { clock++; state.onTick(this); }
    // ...
    void setState(SignalState s) { state = s; s.enter(this); }
    // ...
    /** Turns red first, then green: never two crossing flows green, even for an instant. */
    void showPhaseGreen(int p) {
        Phase phase = phases.get(p);
        lights.forEach((a, l) -> { if (!phase.greens().contains(a)) l.set(Signal.RED); });
        phase.greens().forEach(a -> lights.get(a).set(Signal.GREEN));
    }
    // ...
    /** Safe = every approach that is green or yellow belongs to ONE phase. */
    synchronized boolean isSafe() {
        return phases.stream().anyMatch(p -> lights.entrySet().stream()
                .filter(e -> e.getValue().signal() == Signal.GREEN || e.getValue().signal() == Signal.YELLOW)
                .allMatch(e -> p.greens().contains(e.getKey())));
    }
    // ...
}
```

Full runnable code: code/lld/ch14/trafficsignal/ (see the Full Edition).

How it works: `QueueAdaptiveTiming` gives `secondsPerCar × waiting cars`, clamped to [min, max]. `setState` calls `enter` at once (set lights, load timer). `AllRedState` picks the emergency phase if pending, else the next. `SafetyMonitor` checks `isSafe()` on every change to GREEN; a real `ScheduledExecutorService` run with 20 concurrent emergency requests shows zero violations.

Output (excerpt):

```
t=37  N:G S:G E:R W:R  GREEN(0)
t=38  emergency request: WEST
t=39  N:Y S:Y E:R W:R  YELLOW(0)
t=41  N:R S:R E:R W:R  ALL_RED
t=42  N:R S:R E:G W:G  EMERGENCY_HOLD(1)
t=50  N:F* S:F* E:F* W:F*  FLASHING_RED
light changes seen by observer: 48, safety violations: 0
```

With 12 cars waiting E-W, the adaptive policy gives a 10 s green (the maximum).

Concurrency & edge cases

Situation	Handling
Emergency between ticks	Stored under the lock; next tick acts
Emergency during yellow	Yellow completes; all-red then picks the emergency phase
Emergency for the phase already green	Straight to `EmergencyHold` — no needless yellow
Two emergencies from crossing directions	Latest wins; the hold exits via yellow + all-red. Real systems queue by priority and arrival
Emergency never cleared	Maximum hold time
Timer and sensor threads overlap	`synchronized` methods; `CopyOnWriteArrayList` listeners
Adaptive timing starves a quiet road	Minimum green; every phase still gets its turn
Bug causes conflicting greens	`SafetyMonitor` detects; real controllers force flashing red
Timer drift	`scheduleAtFixedRate`; real deployments use the hardware clock

Extensions & follow-up questions

1. **Pedestrians** → WALK during the phase's green, flashing DON'T WALK before yellow, push-button requests.
2. **Left-turn arrows** → more phases; phases are data, state machine unchanged.
3. **Green wave** → a `TrafficControlCenter` sets cycle length and **offset** per intersection.
4. **Emergency corridor** → the centre pre-empts intersections along the route.
5. **Night mode** → time-of-day `TimingPolicy`, or flashing yellow/red states.
6. **Testing** → call `tick()` N times, assert snapshots.

Real-world example

SCATS and **SCOOT** adapt timing city-wide from sensors; **emergency pre-emption** emitters still time out yellow and red first; **conflict monitors / MMUs** force flash on conflicting greens — our `SafetyMonitor`.

In the interview

40 minutes: 5 requirements → 5 entities → 10 state machine → 15 code Green/Yellow/AllRed and `tick()` → 5 concurrency and follow-ups.

30-second answer: "Phases are sets of approaches that can be green together; the controller is a timed State machine — Green, Yellow, All-Red, plus Emergency-Hold and Flashing-Red. A timer calls `tick()` each second; each state counts down and names the next. Green time comes from a swappable timing Strategy. An emergency only shortens green: yellow and all-red always run in full, then the emergency phase is held until cleared. Lights publish to observers, including a safety monitor that checks no crossing flows are ever green together."

- *"Why not jump straight to emergency green?"* — Cars in the junction need yellow and all-red to clear it.
- *"Why not a Singleton?"* — One per intersection; a city has thousands.

Self-check

1. **Q:** What is a phase? **A:** Approaches that may be green together because their paths don't cross.
2. **Q:** Why all-red? **A:** To let vehicles still in the junction clear it.
3. **Q:** Emergency during yellow? **A:** Yellow finishes; all-red then picks the emergency phase.
4. **Q:** Why do these states hold data but vending states don't? **A:** Each interval has its own countdown.

TL;DR

- **Timed State machine**: Green → Yellow → All-Red → next Green; plus Emergency-Hold and Flashing-Red.
- **Safety first**: red before green, yellow never cut short, conflict-monitor observer.
- **Strategy** for timing, **Observer** for lights, emergency as a flag.
- Tick-driven and synchronized: deterministic tests.

14.7 Design a Shipping Locker System

A **parcel locker** is a wall of numbered doors in a shop or lobby: a courier puts a parcel behind one door, the customer types a **pickup code** and that door opens (Amazon Hub Locker, InPost Paczkomat). It looks like the parking lot — hand out limited spaces of several sizes — plus three extras interviewers probe: **deadlines** (a reservation that times out, a parcel that expires), **a secret** (the code, hard to guess, never shown to the courier) and **a network** (station full → suggest another).

Problem & clarifying questions

Question	Assumed answer
Door sizes?	Small, medium, large; a parcel may be upgraded to a bigger free door.
When is the door chosen?	In advance: the carrier **reserves** it; the hold lasts 2 h, then the door is released.
How does the customer open it?	6-digit **one-time code** by SMS/app on deposit; the courier never sees it.
How long can a parcel stay?	3 days; then it **expires** and the courier takes it back.
Code guessing?	Random, single-use; 5 wrong codes in a row lock the keypad for 15 min.
Many stations?	Yes; offer the nearest one with space. Returns and cash on delivery are extensions.

Requirements

Functional: (1) numbered doors of fixed sizes; (2) **reserve** the best-fitting free door, all-or-nothing, with a hold deadline; (3) **deposit** → one-time code **sent to the customer only**; (4) **pickup** opens one door once; (5) **expiry** → customer told, courier collects for return; (6) keypad lockout; (7) nearest station with space; (8) doors out of service.

Non-functional: never one door for two parcels under concurrency; codes unpredictable, single-use, stored as a hash; deadlines correct even if the timer is late; clock and randomness injected.

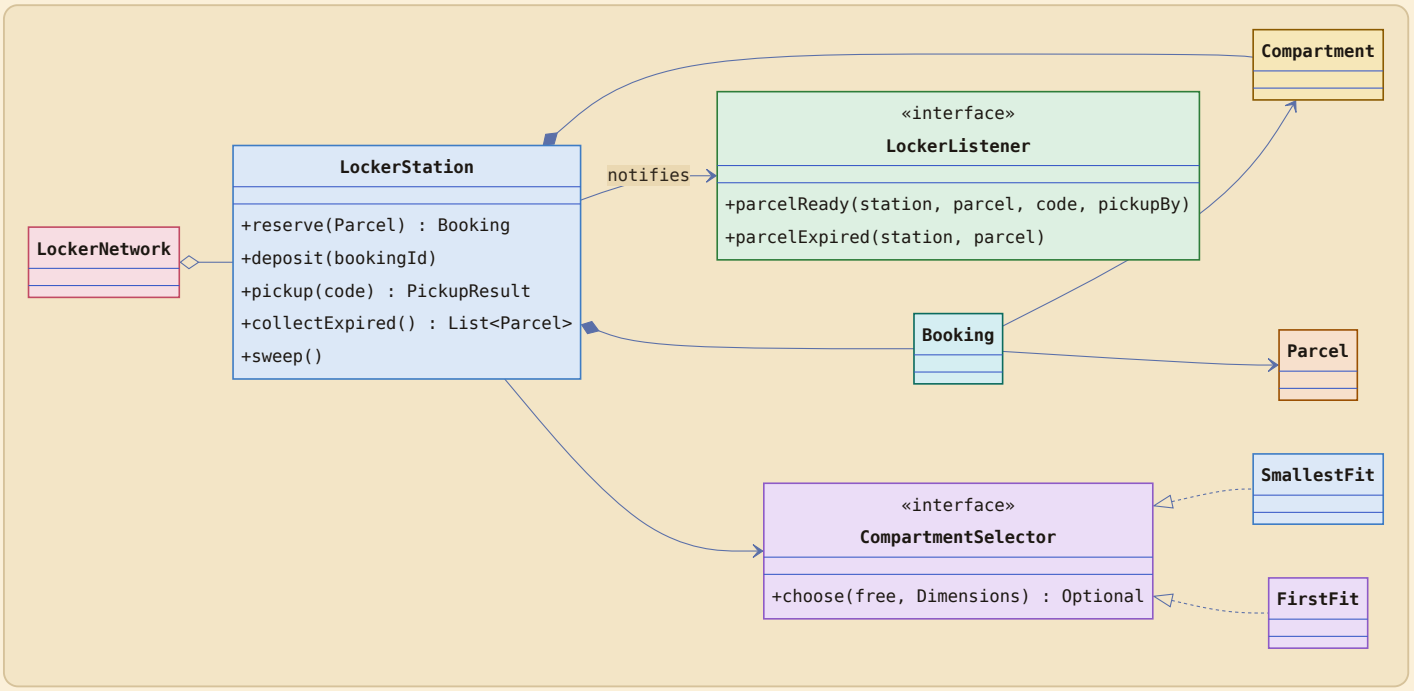
Use cases

Actors → use cases: Carrier system → reserve, find a station with space; Courier → deposit, collect expired; Customer → pick up; Technician → door out of service; Scheduler → sweep deadlines and notify.

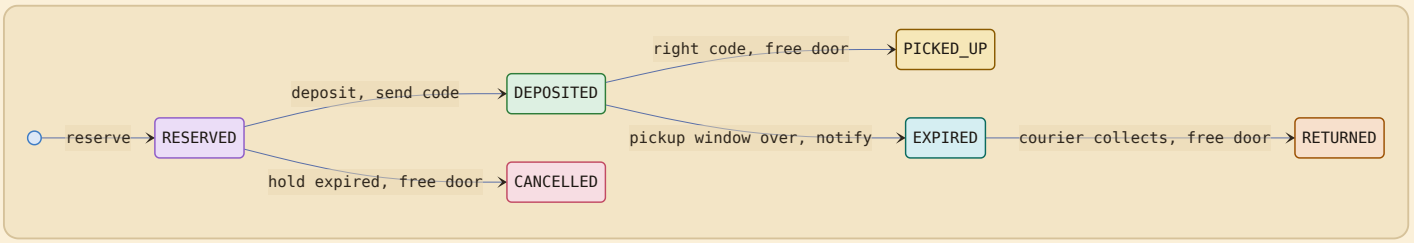
Core entities

Class	Responsibility
`Size` (enum), `Dimensions`, `Parcel` (records)	Door sizes with `fits()` (sort both boxes' sides, compare in order); parcel data.
`Compartment` + `CompartmentState`	One door: FREE, RESERVED, OCCUPIED, OUT_OF_SERVICE; legal moves in `moveTo`.
`Booking` + `BookingStatus`	One parcel's stay: RESERVED → DEPOSITED → PICKED_UP / EXPIRED → RETURNED / CANCELLED; deadlines and code **hash**.
`PickupResult` (sealed)	`DoorOpened`, `WrongCode`, `KeypadLocked`, `ParcelExpired`.
`CompartmentSelector`	**Strategy**: `SmallestFit`, `FirstFit`.
`LockerListener`	**Observer**: parcel ready (with code), parcel expired.
`LockerStation`, `LockerNetwork`	Doors, bookings, code index, lockout under one lock; nearest station with space.

Class diagram



Key flows



Delivery: carrier `reserve(parcel)` → `SmallestFit` picks the smallest free door that fits → door RESERVED, hold 2 h → courier `deposit` → door OCCUPIED, random code, only its SHA-256 hash stored → listener sends the code to the customer **after the lock is released** → customer `pickup(code)` → hash matches → door FREE, code deleted. The door has its own small machine: FREE ↔ RESERVED → OCCUPIED → FREE, and FREE ↔ OUT_OF_SERVICE.

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Door choice as `CompartmentSelector`	**Strategy**	Smallest-fit saves big doors; first-fit, eye-level-first swap in	`if (mode == …)` in `reserve`
Door state separate from booking status	Two enum state machines	Doors are reused; bookings keep deadlines and history	One field mixing both
Sealed `PickupResult`	Sealed result types	Every keypad outcome handled, compiler-checked	`null` / magic strings
Random, single-use, hashed codes	Security by design	A DB leak reveals no live code; courier never sees it	Plain codes; codes derived from the parcel id
Listeners called outside the lock	**Observer**	A slow SMS gateway can't block the keypad	SMS call inside `deposit`
Lazy expiry on every call + `sweep()` for messages	Check deadlines on read	Doors come back even if the scheduler is late	A reaper thread as the only mechanism
Injected `Clock`, `RandomGenerator`	DI	Deterministic tests	`Instant.now()`, `new Random()` inside
One lock per station	Monitor	Tens of doors, microsecond calls	Per-door CAS (needless here)

Java implementation

Package `lld.ch14.locker`.

```java
// File: lld/ch14/locker/LockerStation.java
    synchronized Booking reserve(Parcel parcel) {
        expireOverdue();
        // ... reject a parcel that already has an active booking
        List<Compartment> free = compartments.values().stream().filter(Compartment::isFree).toList();
        Compartment c = selector.choose(free, parcel.dims())
                .orElseThrow(() -> new LockerException("No free compartment fits " + parcel.dims() + " at " +
id));
        c.moveTo(CompartmentState.RESERVED);
        Booking b = new Booking(id + "-B" + nextBooking++, parcel, c, now() + reservationHold.toSeconds());
        bookings.put(b.id, b);
        return b;
    }

    void deposit(String bookingId) {
        Booking b;
        String code;
        synchronized (this) {
            // ... expireOverdue(); booking must be RESERVED
            b.compartment.moveTo(CompartmentState.OCCUPIED);
            b.status = BookingStatus.DEPOSITED;
            b.pickupBy = now() + pickupWindow.toSeconds();
            do { code = "%06d".formatted(random.nextInt(1_000_000)); }       // unique among active codes here
            while (bookingByCodeHash.containsKey(hash(code)));
            b.codeHash = hash(code);
            bookingByCodeHash.put(b.codeHash, b.id);
        }
        for (LockerListener l : listeners) l.parcelReady(id, b.parcel, code, b.pickupBy);   // outside the
lock
    }

    synchronized PickupResult pickup(String code) {
        expireOverdue();
        long t = now();
        if (t < lockedUntil) return new KeypadLocked(lockedUntil - t);
        String bookingId = bookingByCodeHash.get(hash(code));
        if (bookingId == null) {
            // ... count the wrong code; the 5th in a row sets lockedUntil = t + 15 min and returns
KeypadLocked
            return new WrongCode(MAX_WRONG_CODES - wrongCodes);
        }
        wrongCodes = 0;
        Booking b = bookings.get(bookingId);
        if (b.status == BookingStatus.EXPIRED) return new ParcelExpired(b.parcel.id());
        b.status = BookingStatus.PICKED_UP;
        b.compartment.moveTo(CompartmentState.FREE);
        bookingByCodeHash.remove(b.codeHash);                        // a code works once
        return new DoorOpened(b.compartment.id(), b.parcel.id());
    }
    // ... expireOverdue(): RESERVED past hold -> CANCELLED (door FREE); DEPOSITED past window -> EXPIRED
```

Full runnable code: code/lld/ch14/locker/ (see the Full Edition).

How it works: `Size.fits` sorts the sides of box and door and compares in order (no need to try six rotations). `sweep()` (called by a scheduler) runs the same deadline check and sends "not collected" messages outside the lock. `LockerNetwork.nearestWithSpace` sorts stations by distance and returns the first within the radius that can fit the parcel now.

Output (excerpt):

```
P3 9x12x12cm    -> MG01-03(MEDIUM)
error: Parcel P1 already has a locker
SMS to +91-98450-00001: parcel P1 is in locker MG01, code 694696, collect by 2026-10-04T09:00:00Z
right code  -> DoorOpened[compartmentId=MG01-01, parcelId=P1]
same code again -> WrongCode[attemptsLeft=4]  (a code works once)
guess 000004 -> KeypadLocked[secondsLeft=900]
booking MG01-B4 is CANCELLED, free now: {SMALL=2, MEDIUM=1, LARGE=1}
customer types the code -> ParcelExpired[parcelId=P3]
station F: small parcel -> F-01(LARGE), then large parcel -> REJECTED
station S: small parcel -> S-02(SMALL), then large parcel -> S-01(LARGE)
reserved=20 rejected=30 distinct doors=20
```

The mug was upgraded to MEDIUM because both small doors were taken; `FirstFit` wasted the large door and had to refuse a large parcel; 50 concurrent reservations on 20 doors gave exactly 20 distinct doors.

Concurrency & edge cases

Situation	Handling
Two carriers, last door	`synchronized reserve`: choose + mark is one step
Courier late past the hold	Booking CANCELLED; `deposit` refuses; carrier re-reserves
Retried reserve for the same parcel	Rejected while a booking is active (idempotent by parcel id)
Duplicate random code	Regenerate until unique among live codes
Guessing	10^6 codes, lockout after 5 wrong, codes expire
Scheduler down	Lazy expiry still frees doors; only the SMS is late
`canFit` true but `reserve` fails	`nearestWithSpace` is a hint; reserve, then try the next station
Broken door	Only a FREE door can go OUT_OF_SERVICE (`moveTo`)

Extensions & follow-up questions

1. **Returns** → drop-off booking with its own code; courier collects later.
2. **Extend pickup window** → move `pickupBy` once, only while DEPOSITED.
3. **Cash on delivery** → a payment step before the door opens.
4. **Keep large doors for large parcels** → a selector that refuses upgrades when few large doors are left.
5. **Thousands of stations** → shard by station; geo index (geohash, R-tree) for the network search.
6. **Database** → `UPDATE compartment SET state='RESERVED' WHERE id=? AND state='FREE'` and check one row changed.

Real-world example

Amazon Hub Locker sends a pickup code and returns uncollected parcels after a limited number of days (our EXPIRED → RETURNED path); InPost's Paczkomat lockers in Poland open with a code or an app and are dense enough that "nearest locker with space" matters. In both, the system — not the courier — chooses the door.

In the interview

40 minutes: 5 questions → 5 entities → 8 the two state machines → 15 code `reserve`/`deposit`/`pickup` → 7 security, concurrency, follow-ups.

30-second answer: "Doors come in a few sizes; a Strategy picks the smallest free one that fits, and choose-and-mark runs under the station lock so two carriers never get the same door. On deposit I generate a random single-use code, store only its hash and notify the customer through an observer after releasing the lock. Pickup matches the hash, frees the door and deletes the code; five wrong codes lock the keypad. Holds and pickup windows are checked on every call, so doors come back even if the scheduler is late."

- *"Why a Booking separate from the Compartment?"* — The door is reused; the booking keeps each parcel's deadlines and history.
- *"Why hash the code?"* — Anyone reading the database or logs could otherwise open doors.

Self-check

1. **Q:** How does `fits` handle a box on its side? **A:** Sort both sets of sides and compare in order.
2. **Q:** What frees a door if the scheduler never runs? **A:** Lazy expiry at the start of every call.
3. **Q:** Why notify after the `synchronized` block? **A:** A slow SMS call must not hold the station lock.
4. **Q:** Which pattern swaps smallest-fit for first-fit? **A:** Strategy (`CompartmentSelector`).

TL;DR

- **Strategy** picks the smallest free door that fits; choose-and-mark is **atomic** per station.
- Two **state machines**: door and booking (RESERVED → DEPOSITED → PICKED_UP / EXPIRED → RETURNED / CANCELLED).
- Codes **random, single-use, hashed**, sent via an **Observer** outside the lock; lockout after 5 wrong.
- Deadlines enforced **lazily**; inject clock and randomness.

Chapter summary

Problem	Core difficulty	Key data structure	Key patterns	Concurrency tool
Parking lot	Allocate spots without double booking	Concurrent sorted set of free spots per type	Strategy, Factory, Observer	`pollFirst` + CAS, per-ticket lock, `putIfAbsent`
Vending machine	Legal order of actions; exact change	`EnumMap` of coin counts; DP table	State	`synchronized` machine
Coffee machine	All-or-nothing multi-ingredient deduction	`EnumMap` levels, recipes as data	Observer, sealed results	One `ReentrantLock`; notify outside the lock
ATM	Money correctness and authentication	Total vs available balance; note chain	State, Chain of Responsibility, Command	Per-account lock; ordered locking for transfers
Elevator	Scheduling and dispatching	Two `TreeSet`s (up/down stops)	Strategy (+ modes)	Per-car lock, synchronized dispatcher
Traffic signal	Timed transitions with a safety invariant	Phases as data	State, Strategy, Observer	Synchronized tick; scheduled executor
Shipping locker	Best-fit door, holds and pickup windows, secret codes	Doors per station; bookings; code-hash index	Strategy, Observer, enum state machines, sealed results	One lock per station; lazy expiry; notify outside the lock

Chapter self-check

1. **Q:** Stateless vs data-carrying states? **A:** Vending and ATM share stateless singletons; traffic-signal states hold countdowns, new per transition.
2. **Q:** The check-then-act race in parking, coffee, ATM? **A:** Two gates one spot (atomic `pollFirst`/CAS); two orders the last milk (one lock around check+deduct); two ATMs one account (check+debit under the account lock).
3. **Q:** Where did we reject a Singleton? **A:** Parking lot, coffee machine, traffic controller — hidden global, untestable, and real deployments have many.
4. **Q:** Where does greedy fail? **A:** Change with limited coins (vending) and notes with limited cassettes (ATM); use bounded coin-change DP.
5. **Q:** Which designs are tick-driven and why? **A:** Elevator and traffic signal — deterministic, clock-free tests.

6. **Q:** Which case enforces deadlines lazily? **A:** The shipping locker — every call expires overdue holds/parcels first; the sweep only sends messages.

Chapter TL;DR

- Find the **one hard method** per problem and spend your time there.
- **State** for machines whose buttons change meaning; **Strategy** for changing rules; **Observer** for boards, alerts, monitors; **Chain of Responsibility** and **Command** for the ATM.
- Make every **check-then-act** atomic, lock at the **smallest** scope, lock in a **fixed order**, call listeners **outside** locks.
- Prefer injected instances over Singletons, integer cents over doubles, tick-driven logic over sleeps.
- Deadlines (holds, pickup windows): check on every call; secrets (pickup codes): hashed, single-use.

Chapter 15 — Case Studies: Booking & Inventory (Part 1) (Study Edition)

> **Where this fits.** Chapter 14 designed small, self-contained objects. This chapter is the first of two on **booking systems**: software that sells or lends a limited thing (a book copy, a cinema seat, a concert place, a hotel night) to many people who want it at once. Chapter 16 continues. This is the study edition: every topic stays, the padding goes; the Full Edition (README.md) has the complete code and outputs.

Why booking problems are interview favourites

A booking problem looks like simple CRUD until two people want the same thing. Then five hard questions appear, and each problem here is a different mix:

Theme	The question the interviewer is really asking	Where you will see it
Inventory	What exactly is sold or lent? A title or a copy? A seat or a seat *for one show*? A room or a room *on one night*?	`Book` vs `BookItem` (15.1), `HallSeat` vs `ShowSeat` (15.2), GA count vs numbered seat (15.3), `Room` + booked nights (15.4)
Holds / locks	How do you keep something for a customer while they pay, without keeping it forever?	Seat lock with a TTL (time-to-live) (15.2), CAS holds + reaper (15.3)
Double-booking prevention	What stops two threads (or servers) from both succeeding?	`synchronized` per item (15.1, 15.4), one monitor per show (15.2), lock-free CAS (15.3)
Payments	What if payment fails? Or succeeds but the hold was lost meanwhile?	Release on decline, refund as **compensation** (15.2, 15.3)
Cancellations	Who gets the thing next, and how much money comes back?	Waitlists (15.1, 15.3), refund policies (15.2, 15.4)

Three words used everywhere:

- **Race condition** — two threads both read "free" and both act; one result silently overwrites the other.
- **Check-then-act** — "if free, then take it". Safe only if check and act are one **atomic** step.
- **Hold** (lock, soft lock, reservation) — a temporary claim **with an expiry**, so an abandoned checkout does not block the item forever.

#	Problem	The one idea that carries the design
15.1	Library Management System	A *title* (`Book`) is not a *copy* (`BookItem`). Limits live with the member; the waitlist lives with the title.
15.2	Movie Ticket Booking (BookMyShow)	You book a `ShowSeat`, not a physical seat. Hold with TTL → pay → commit; refund if the hold was lost.
15.3	Concert Ticket Booking	Extreme contention: lock-free CAS, a GA counter, a fair virtual queue, per-user limits, a waitlist.
15.4	Hotel Management System	A status field is not availability. Each room remembers which nights are sold.

How to run every example in this chapter

```
cd chapters/15-case-booking-1/code
javac --release 21 -d /tmp/lld_15 $(find . -name '*.java')
java -cp /tmp/lld_15 lld.ch15.library.Main        # or moviebooking, concert, hotel
```

In the concurrency demos, *which* thread wins changes per run; the *number* of winners never does. Several small package-private types share a file (e.g. `Model.java`); `-Xlint:all` flags this as `[auxiliaryclass]` — a style warning, not an error.

> **Money.** Every amount is a `BigDecimal` built from a string (`new BigDecimal("0.50")`), never a `double` (0.10 is not exact in binary, so sums drift). Compare with `compareTo` / `signum()`, not `equals` (which also compares scale), and name the `RoundingMode` at every rounding step.

> **Time trick.** Every service takes a `java.time.Clock`; demos pass a `SimClock` whose `advance(Duration)` moves time forward, so "10 days later" needs no `sleep`. Production passes `Clock.systemUTC()`.

15.1 Design a Library Management System

Problem & clarifying questions

A **Library Management System** (LMS) tracks every title and physical copy, who borrowed which copy and when it is due, members and cards, reservations and fines.

Question	Answer we assume
Copies?	Many per title — each a **book item** with barcode and rack.
Limits?	**5 books** per member, **10 days** per loan, then a fine per day.
Reserve a book that is out?	Yes, with a notification when a copy returns.
Fines?	Calculated and recorded; payment out of scope. E-books, inter-library loans out of scope.

Requirements

Functional: (1) search by **title, author, subject, publication date**; (2) each book has an ISBN; each copy a **rack number**; (3) a title has **many copies**; (4) answer **"who took this copy?"** and **"which books does this member have?"**; (5) **max 5 books** per member; (6) **max 10 days** per loan; (7) **fines** for late returns; (8) **reserve** unavailable books; (9) **notify** when a reserved book is available and when a book is overdue; (10) **barcodes** on items and cards.

Non-functional: correct under concurrency (no double-lent last copy, never over 5); index-based search; swappable fine rules and channels.

Use cases

Actors: Librarian (add/edit/remove books, items, members; issue, reserve, return for a member), **Member** (search, check out, reserve, renew, return), **System** (overdue, reserved-book-back, cancelled-reservation notifications). **Top use cases:** manage books/items; search; register/cancel account; **check out**; **reserve**; **renew**; **return**.

In the course's use-case diagram, `<<include>>` means *always* performs another (checking out always clears the member's reservation), and `<<extend>>` is an *optional variation* (search by author extends search catalog).

Core entities

Class	Responsibility
`Book` (record)	The **title**: ISBN, title, subject, publisher, date, authors. Immutable.
`BookItem`	One **physical copy**: barcode, format, rack, price, status (`AVAILABLE`, `RESERVED`, `LOANED`, `LOST`), who it is held for.
`Author`, `Rack` (records)	Author name; rack number and location.
`Account` (sealed, abstract) → `Member`, `Librarian`	People; an account **owns** its `LibraryCard` (composition); a member carries the loan count.
`LibraryCard` (record)	Card number, barcode, issue date.
`BookLending`	One loan: barcode, member id, created, **due date**, return date.
`Fine` (record)	Amount owed for one late return.
`Search` / `Catalog`	Four searches; one index per search; knows every copy of every title.
`LendingService`	Rules: check out, return, renew, reserve, overdue notices.
`FinePolicy` / `PerDayFine`	Cost of a late return (Strategy).
`Notifier` / `ConsoleNotifier`	Where messages go.
`Library`	Facade: owns catalog and service; only librarians add stock.

All course classes are kept; `BookReservation` becomes a per-title waitlist and `Notification` the `Notifier` interface.

Class diagram

The course builds five stages: (1) the "what happened" records — `BookReservation`, `BookLending`, `Fine`, each with barcode and member id (answers requirements 4, 7, 8); (2) **`Book` vs `BookItem`** — the split that carries the design (the course has `BookItem` *inherit* `Book`; we use **composition**); (3) abstract `Account` with `Member`/`Librarian` — limits (req. 5, 6) belong to people; the card is **composition** (dies with the account); (4) `Search` interface implemented by `Catalog`; (5) `Library` owns catalog, racks, items; abstract `Notification` per channel.

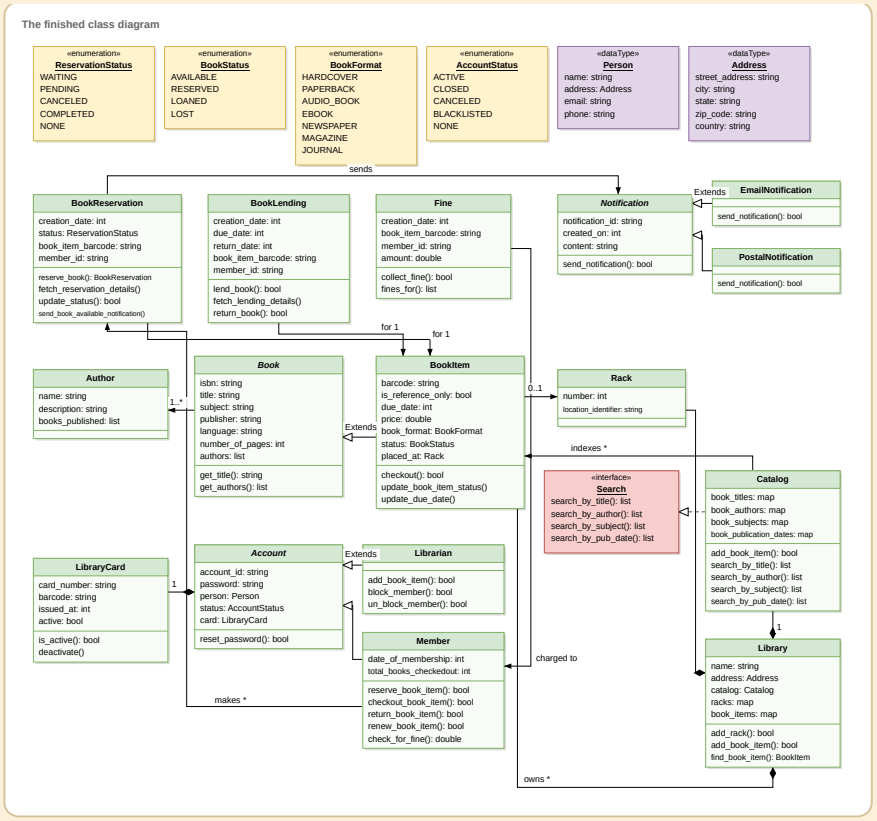
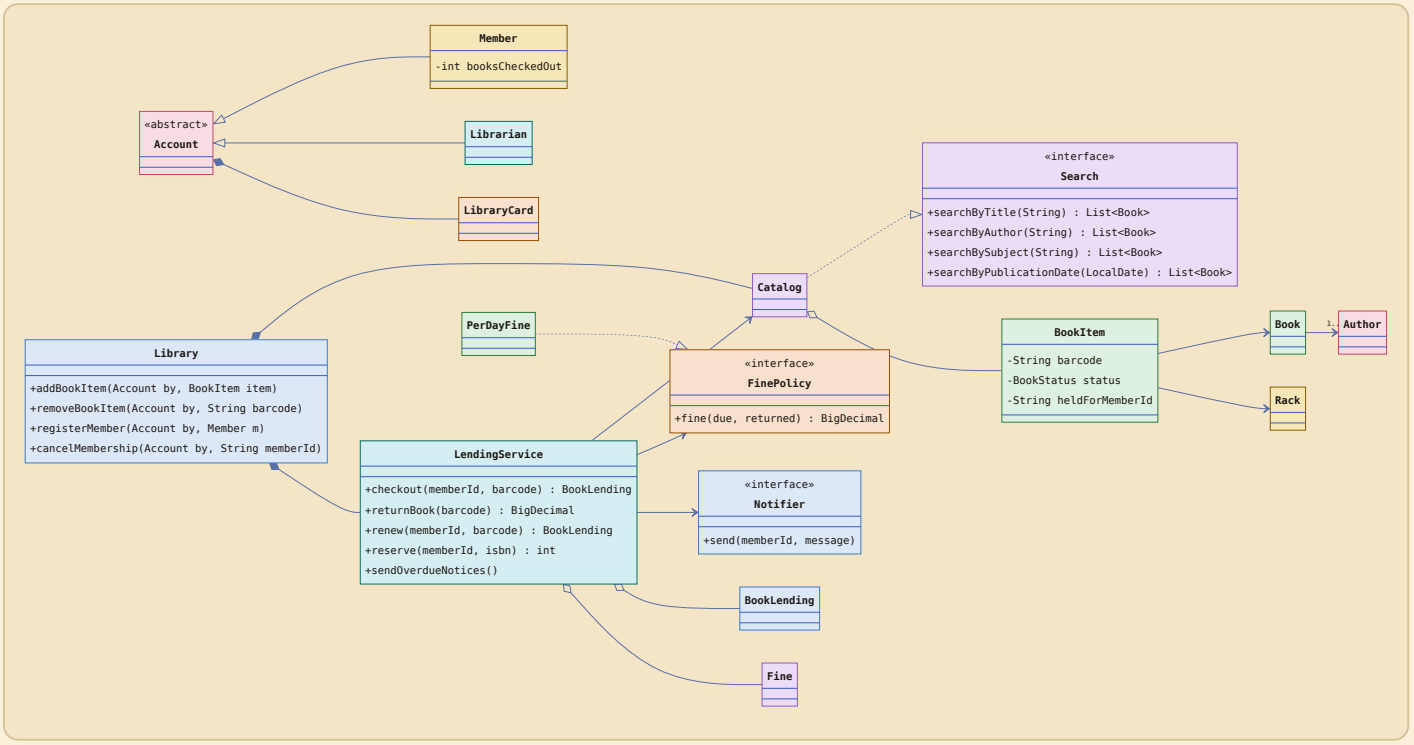


Figure — The finished library class diagram: every class from the five stages in full colour (course diagram)



Key flows

The course's three activity diagrams:

- **Check out:** scan book → can it be issued (not reserved for someone else)? → member under quota? → create the loan, mark "Loaned", increment the member's count, complete this member's reservation.
- **Return:** scan → late? charge fine → decrement count → reserved? notify the first waiting member and mark it held; else make it available.
- **Renew:** scan card and book → charge a fine if already late → someone reserved it? refuse; else new due date.

Copy state machine: `AVAILABLE → LOANED` (checkout); `LOANED → AVAILABLE` (return, nobody waiting) or `→ RESERVED` (return, waitlist non-empty); `RESERVED → LOANED` (checkout by the member it is held for); `LOANED → LOST`.

Design decisions & patterns used

Decision	Why	Alternative rejected
`Book` vs `BookItem`	Members borrow *copies*; search works on *titles*. Say it early.	One `Book` with a `copies` counter — can't say *which* copy is late or where.
Composition (`BookItem` has a `Book`)	Many copies share one immutable title; inheritance would duplicate title fields per copy; a title has no barcode, so "copy *is a* title" fails strict substitution.	`BookItem extends Book` (course) — valid for a small model; mention it.
Rules in `LendingService`, not `Member`	The course itself flags `Member` doing checkout/return/renew as two reasons to change (SRP).	Methods on `Member` (fine for a first sketch).
Reservation = FIFO waitlist per title	A member wants "a copy of Dune"; any returned copy goes to the first in line.	Per-copy reservation — wait on copy #3 while #1 sits on the shelf.
Strategy for fines (`FinePolicy`)	Caps, discounts, holidays change.	`if/else` in `returnBook`.
Observer-style `Notifier`	Lending announces; delivery is someone else's job (course's `Notification` subclasses = same idea).	Emailing from `LendingService`.
Facade (`Library`)	One entry point; role checks live there.	Exposing catalog and service to the UI.
One index per search	Every lookup is a map read; cost: a new search adds a map (a single `(field, value)` index would stay one structure).	Scanning all books.

Java implementation

Package `lld.ch15.library` (`Model`, `Catalog`, `Support`, `LendingService`, `Library`, `Main`).

```java
/**
 * The three core flows (check out, return, renew) plus reservations.
 * Lives apart from Member so a Member stays "a person", and the rules live in one place.
 *
 * Locking rule (prevents deadlock): always lock the MEMBER first, then the BOOK ITEM.
 */
public final class LendingService {
    static final int MAX_BOOKS_PER_MEMBER = 5;
    static final int MAX_LENDING_DAYS = 10;
    // ...
    BookLending checkout(String memberId, String barcode) {
        Member m = member(memberId);
        BookItem item = catalog.item(barcode);
        synchronized (m) {
            synchronized (item) {
                if (m.status() != AccountStatus.ACTIVE)
                    throw new LibraryException(memberId + " account is " + m.status());
                if (m.booksCheckedOut() >= MAX_BOOKS_PER_MEMBER)
                    throw new LibraryException(memberId + " already has " + MAX_BOOKS_PER_MEMBER + " books");
                switch (item.status()) {
                    case AVAILABLE -> { /* ok */ }
                    case RESERVED -> {
                        if (!memberId.equals(item.heldForMemberId()))
                            throw new LibraryException(barcode + " is held for another member");
                    }
                    case LOANED, LOST -> throw new LibraryException(barcode + " is " + item.status());
                }
                item.markLoaned();
                m.incrementLoans();
                LocalDate today = today();
                BookLending loan = new BookLending(barcode, memberId, today,
today.plusDays(MAX_LENDING_DAYS));
                activeLoans.put(barcode, loan);
                removeFromWaitlist(item.book().isbn(), memberId);   // their reservation is now fulfilled
                return loan;
            }
        }
    }
    // ... returnBook(): same lock order, close loan, chargeFineIfLate, handToNextInLine
    /** Caller holds the item's monitor. Give the copy to the first waiting member, or shelve it. */
    private void handToNextInLine(BookItem item) {
        Deque<String> q = waitlist(item.book().isbn());
        String next;
        synchronized (q) { next = q.pollFirst(); }
        if (next == null) { item.markAvailable(); return; }
        item.holdFor(next);
        notifier.send(next, "'" + item.book().title() + "' copy " + item.barcode() + " is held for you");
    }
    // ...
}

/** Strategy: how much a late return costs. Swappable without touching LendingService. */
interface FinePolicy {
    BigDecimal fine(LocalDate dueDate, LocalDate returnDate);
}
```

Full runnable code: code/lld/ch15/library/ (see the Full Edition).

Output (excerpt):

```
  REFUSED: M1 already has 5 books
== 3. 8 members race for the ONLY copy of Dune ==
  winners=1 losers=7 (exactly one copy was lent)
  REFUSED: cannot renew B-DUNE-1: another member reserved this title
  [notify M9] fine of $1.50 for late B-DUNE-1
  [notify M2] 'Dune' copy B-DUNE-1 is held for you
  M4 REFUSED: B-DUNE-1 is held for another member
```

A sixth book is refused; one of eight racing members wins the only copy; renewal is refused while someone waits; a 3-day-late return is fined 3 × $0.50 and the copy is **held** for the first in line. Librarian housekeeping: a copy on loan can't be removed, a member with a book out can't be cancelled, a cancelled member can't borrow.

Concurrency & edge cases

- **Last copy:** `synchronized (item)` makes "available? → mark loaned" atomic; the 8-thread race always has one winner.
- **Two librarians, one member:** quota check and increment under `synchronized (member)` — never 6 books.
- **Deadlock avoidance:** always **member first, then item** (fixed global order). `returnBook` reads the loan to find the member, then locks in that order and re-checks the loan.
- **Renew vs reserve:** renewal refused if the waitlist is non-empty — checked *before* charging a fine, so no double fine.
- **Known small window:** `reserve` checks "no free copy" then queues in two steps; a copy returned in between goes to the shelf and the *next* return reaches the member. Accepted, because taking the queue lock before copy locks would reverse the order returns use — the recipe for deadlock.
- **Edge cases:** unknown barcode/member, duplicate barcodes, inactive/blacklisted accounts, removing a loaned/held copy, cancelling with books out, reserving a title with a free copy ("just check it out"), joining a waitlist twice (keeps place), returning a book not out.
- **Across servers:** use a conditional update — `UPDATE book_item SET status='LOANED' WHERE barcode=? AND status='AVAILABLE'` and check exactly one row changed.

Extensions & follow-up questions

- **"Two members reserve at the same moment?"** — The course calls it unguarded; ours appends to a FIFO deque under a lock, so they queue in a definite order.
- **"Where is the available-copies count?"** — Derive it (`availableCopies(isbn)` counts AVAILABLE copies); if slow, cache it and update under the same lock as the status.
- **"How is the fine paid?"** — Add a `PaymentService`; block checkouts while unpaid fines exceed a threshold.
- **"Hold expiry?"** — `heldUntil` on `BookItem` plus a daily job passing it on — the TTL idea of 15.2.
- **"E-books?"** — `EBOOK` items with no rack, a fixed number of concurrent licences.

Real-world example

Koha and **Evergreen** use this model (records + barcoded items, circulation rules, holds queue per title); **Libby** applies the waitlist to e-book licences.

In the interview

30–45 minutes: 3 min clarify (copies, 5 books, 10 days, fines, reservations) → 5 min use cases → 10 min entities, **starting with** `Book` / `BookItem` → 10 min code `checkout` / `returnBook` with lock ordering → 5 min reservation hand-over and fines → follow-ups.

30-second answer: "The key split is `Book` (title) vs `BookItem` (a copy with barcode and rack). Members borrow copies; search works on titles via a `Catalog` with one index per search. A `LendingService` enforces 5 books and 10 days, records `BookLending`s, charges fines via a `FinePolicy` strategy, and hands returned copies to a FIFO waitlist per title. Checkout locks member then copy so the last copy can't be double-lent."

Self-check

- **Q:** Why a fixed lock order? **A:** Otherwise A holds member→waits item while B holds item→waits member: deadlock.
- **Q:** What stops a 6th book with two desks scanning? **A:** Check and increment inside `synchronized (member)`.
- **Q:** When can't a book be renewed? **A:** When another member is waiting for that title.
- **Q:** Which pattern changes fine rules without touching lending? **A:** Strategy (`FinePolicy`).

TL;DR

Title vs copy. Limits on the member, state on the copy, history in `BookLending`. Rules in a `LendingService`; fines as a Strategy; notifications behind an interface. Lock member-then-copy. Reservations are a FIFO queue per title.

15.2 Design a Movie Ticket Booking System (BookMyShow)

Problem & clarifying questions

An online movie booking system (**BookMyShow**, **Fandango**, **AMC**) lets customers find movies, pick a show, choose seats on a map, pay and get a ticket. The classic booking question: a place hierarchy (city → cinema → hall → seat), a time dimension (shows), and a genuine race (two people tapping one seat).

Question	Answer we assume
Venues?	Many cities; several halls per cinema; a hall runs **one show at a time**.
Seats?	Several per booking from a seat map; **held 5 minutes** while paying.
Payment, refunds?	Card or cash, optional **coupon**; refund up to 2 hours before the show.
Users?	Customers, admins, front-desk officers (walk-ins), guests (search only).

Requirements

Functional (course order): (1) list **cities**; (2) cinemas have **halls**, one show at a time; (3) movies have many **shows**; (4) **search** by title, language, genre, release date, city; (5) show **cinemas and shows** for a movie; (6) **book** a show; (7) **seating plan**, pick several seats; (8) distinguish **available** from booked; (9) **notify** on new movie, booking, cancellation; (10) pay by **card or cash**; (11) **no two customers reserve the same seat**; (12) **discount coupon**.

Non-functional: strong seat consistency; high read, bursty write; **hold expiry** (a gap the course names; we implement it); never pay for a seat you don't get.

Use cases

Actors: Admin (add/cancel movies and shows, block customers), FrontDeskOfficer (book/cancel for walk-ins), Customer (view, book, cancel), Guest (search only — must register), System (notifications). **Use cases:** search; create/modify/view booking; pay; apply coupon; assign seats; refund on cancellation within the allowed time. In the course diagram a **Customer is a specialised Guest** — enforced in code because `BookingService` takes a `Customer`.

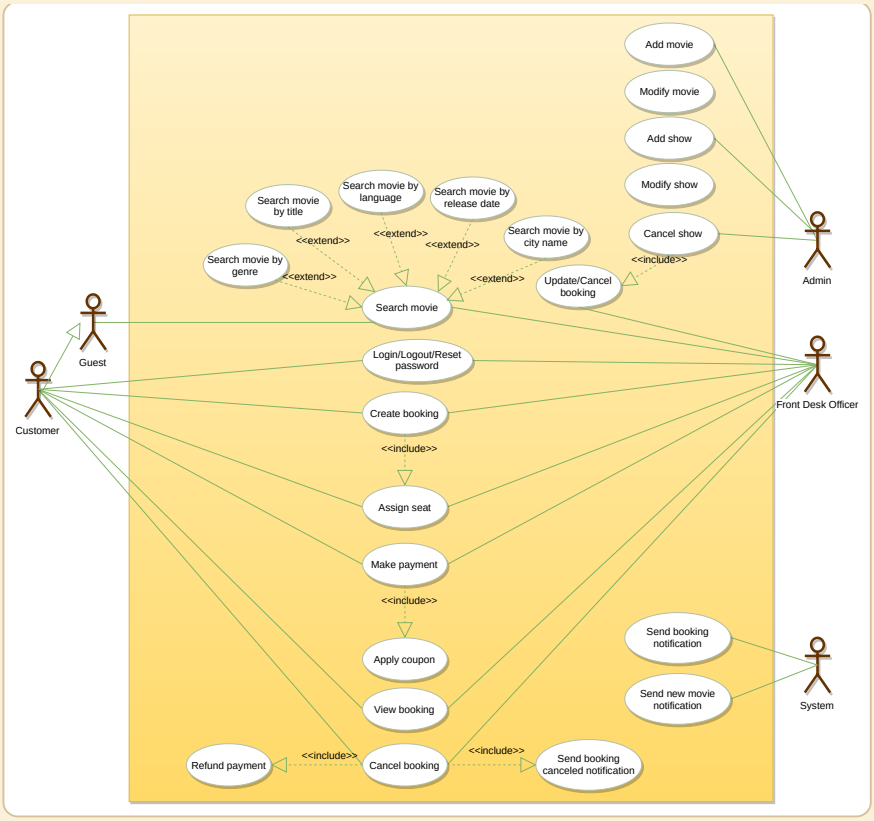


Figure — Movie booking use cases. The hollow arrow from Customer to Guest says a customer can do everything a guest can; the five "search by" use cases extend "Search movie"; "Create booking" includes "Assign seat"; "Make payment" includes "Apply coupon"; "Cancel booking" includes "Refund payment" and the cancellation notification; the admin's "Cancel show" includes updating/cancelling its bookings (course diagram)

Core entities

Class	Responsibility
`City`, `Cinema`, `CinemaHall`	Venue hierarchy; a cinema **owns** halls, refers to its city; a hall refuses an overlapping show.
`HallSeat` (record)	The bolted-down seat: id, `SeatType`. **No booking state.**
`Movie` (record)	Title, language, genre, release date, duration.
`Show`	One screening; **owns one** `ShowSeat` **per hall seat**.
`ShowSeat`	A seat *for this show*: price, `booked`. **This is what gets booked.**
`SeatLockProvider` / `InMemorySeatLockProvider`	Expiring, all-or-nothing **holds**; commit; release.
`Booking`	Customer, show, seat ids, status (`PENDING`, `CONFIRMED`, `CHECKED_IN`, `CANCELED`, `ABANDONED`), payment, coupon.
`BookingService`	hold → pay → commit; cancel; seat map.
`Payment`, `PaymentMethod` (`CardPayment`, `CashPayment`), `PaymentGateway`	Money; the gateway is the boundary to the provider.
`Coupon`	Percent off the booking amount before payment.
`CancellationPolicy` / `CutoffRefundPolicy`	Refund on cancel (Strategy).
`MovieSearch` / `Catalog`	Five searches; "by city" comes from shows.
`Customer`, `Guest`, `FrontDeskOfficer`, `Admin`; `Notifier`	Actors (`Guest` can't book); messages.

Class diagram

Six course stages: (1) city, cinema, hall, physical seat; (2) movie, show, **show seat — where the design succeeds or fails**: one physical seat is free at 3 pm and sold at 9 pm, so you reserve a `ShowSeat` (which also allows per-show prices); (3) coupon, payment types, notification types — the coupon reduces **the booking's amount**, not the payment; (4) `Booking` and `BookingService` — one seat never goes to two customers; (5) search interface and catalog; (6) people — booking restricted to customers.

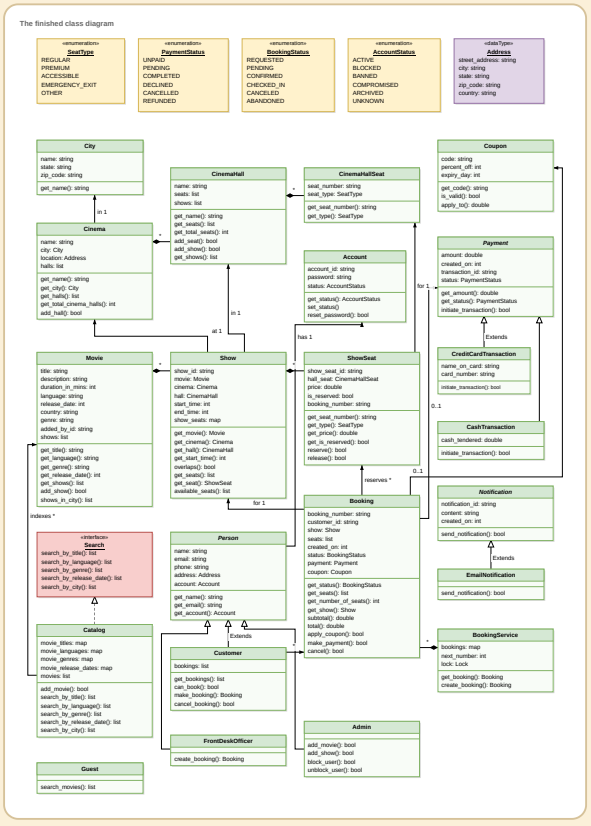
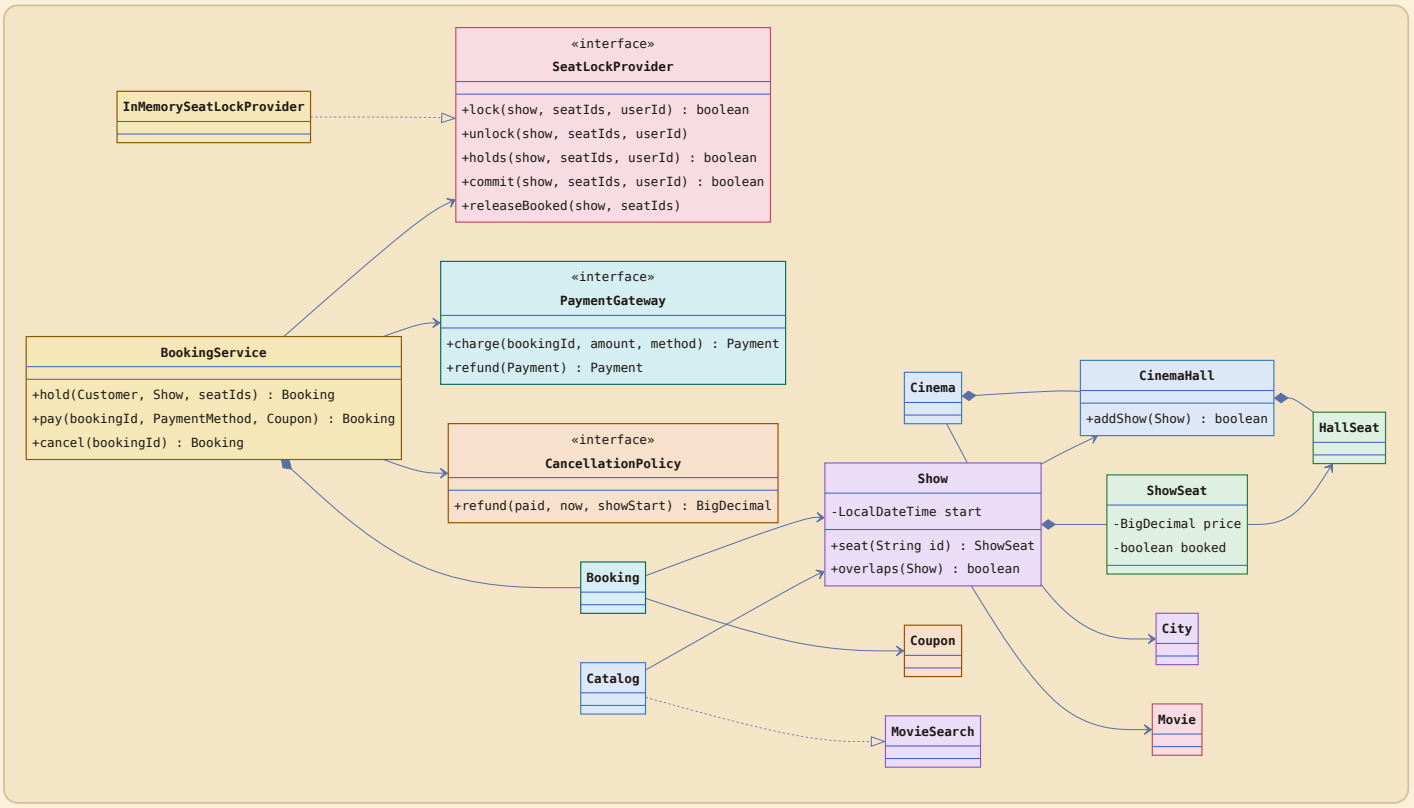


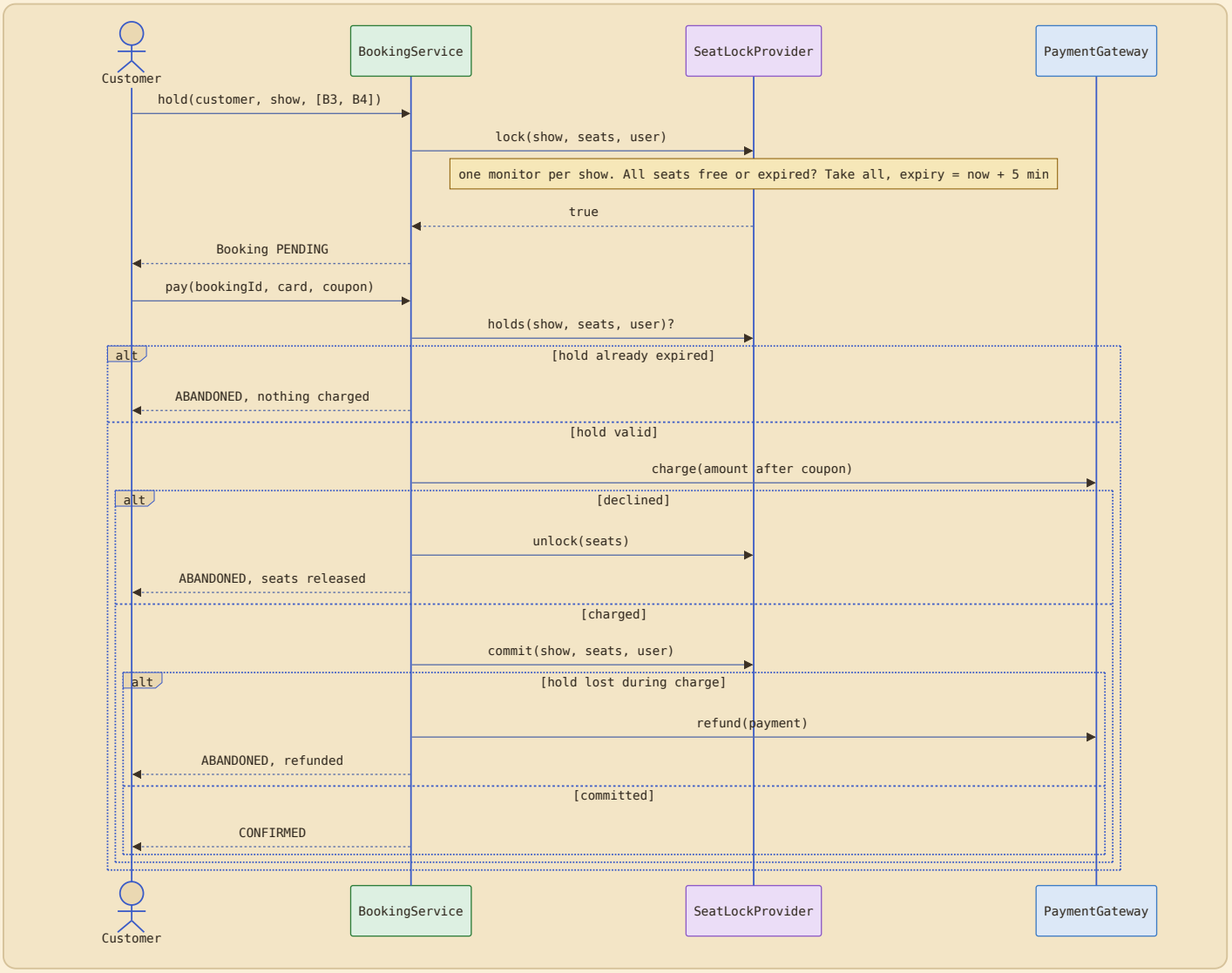
Figure — The finished movie booking class diagram (course diagram)

Our main difference from the course: a separate `SeatLockProvider` with **TTL holds** (the course's biggest self-named omission).



Key flows

The course's activity diagrams cover **make a booking** and **cancel a booking** (refund only within the allowed time). The heart is **hold → pay → commit**:



Why `commit` re-checks inside the lock: payment can take seconds; if the hold expires and someone else grabs the seat, a naive "mark booked" double-books. `commit` verifies "still held by me, not expired" **and** marks booked atomically. If that fails after money was taken, run a **compensating action** (refund) — a card charge can't be rolled back. This is the **Saga** idea: each step has an undo.

Booking lifecycle: `PENDING → CONFIRMED` (paid and committed) `→ CHECKED_IN` (scanned) or `→ CANCELED` (refund per policy); `PENDING → ABANDONED` (expired, declined, or lost and refunded) or `→ CANCELED`. On the seat map a seat is free (`.`), **held** (`L`) or booked (`X`); "held" lives only in the lock provider and disappears on expiry.

Design decisions & patterns used

Decision	Why	Alternative rejected
`ShowSeat` per show	Same physical seat free in one show, taken in another; per-show prices.	Booking `HallSeat` — breaks with two shows a day.
`SeatLockProvider` with TTL holds	Abandoned checkouts free themselves; swap in **Redis** (`SET key value NX PX 300000`) or a DB `expires_at` column.	Holding until cancel (course's gap).
All-or-nothing lock	4 seats together, never 2: check all, then take all.	Locking one by one without rollback.
One monitor per show	Shows never block each other; within a show every decision is serialized.	One global lock (course) — correct but serializes the country.
Commit re-validates + compensation	Never pay for a seat you don't get.	Trusting the earlier check.
Idempotent `pay`	Double-click/retry can't charge twice: non-PENDING returns unchanged. Real systems add an **idempotency key**.	Charging on every call.
Strategy refunds, **Strategy-like** `PaymentMethod`	Rules and payment types vary independently.	`if (method == CARD)`.
Adapter `PaymentGateway`	Wraps Stripe/Razorpay; tests use `FakeGateway`.	Provider SDK in `BookingService`.
Coupon on the booking	Amount correct before payment.	Each payment type applying coupons.
Guest has no booking method	"Register to book" enforced by *types*: `hold` takes a `Customer`.	`Guest.book()` that throws.
"Search by city" from shows	A movie is in a city only when a show is scheduled there.	A city on `Movie`.

Java implementation

Package `lld.ch15.moviebooking` (`Venue`, `SeatLockProvider`, `Payments`, `Catalog`, `BookingService`, `Main`).

```
final class InMemorySeatLockProvider implements SeatLockProvider {
    // ...
    @Override public boolean lock(Show show, List<String> seatIds, String userId) {
        Map<String, SeatLock> t = table(show);
        synchronized (t) {
            Instant now = clock.instant();
            for (String id : seatIds) {                          // 1. check every seat first
                if (show.seat(id).isBooked()) return false;
                SeatLock l = t.get(id);
                if (l != null && !l.expired(now) && !l.userId().equals(userId)) return false;
            }
            SeatLock mine = new SeatLock(userId, now.plus(ttl));
            for (String id : seatIds) t.put(id, mine);           // 2. then take them all
            return true;
        }
    }
    // ...
    @Override public boolean commit(Show show, List<String> seatIds, String userId) {
        Map<String, SeatLock> t = table(show);
        synchronized (t) {
            if (!holdsLocked(t, seatIds, userId, clock.instant())) return false;
            for (String id : seatIds) { show.seat(id).setBooked(true); t.remove(id); }
            return true;
        }
    }
    // ...
}

final class BookingService {
    // ...
    Booking pay(String bookingId, PaymentMethod method, Coupon coupon) {
        Booking b = get(bookingId);
        synchronized (b) {
            if (b.status() != BookingStatus.PENDING) return b;
            if (!locks.holds(b.show(), b.seatIds(), b.customerId())) {       // fast fail, no charge
                b.setStatus(BookingStatus.ABANDONED);
                notifier.send(b.customerId(), b.id() + " hold expired before payment, nothing charged");
                return b;
            }
            b.setCoupon(coupon);
            BigDecimal amount = coupon == null ? b.baseAmount() : coupon.apply(b.baseAmount());
            Payment p = gateway.charge(b.id(), amount, method);
            b.setPayment(p);
            if (p.status() != PaymentStatus.COMPLETED) {
                locks.unlock(b.show(), b.seatIds(), b.customerId());           // free seats at once
                b.setStatus(BookingStatus.ABANDONED);
                notifier.send(b.customerId(), b.id() + " payment " + p.status() + ", seats released");
                return b;
            }
            if (!locks.commit(b.show(), b.seatIds(), b.customerId())) {       // hold lost during charge
                b.setPayment(gateway.refund(p));                              // COMPENSATION
                b.setStatus(BookingStatus.ABANDONED);
                notifier.send(b.customerId(), b.id() + " hold expired during payment, " + Money.fmt(amount) +
" refunded");
                return b;
            }
            b.setStatus(BookingStatus.CONFIRMED);
            // ... notify CONFIRMED
            return b;
        }
    }
    // ...
}
```

Full runnable code: code/lld/ch15/moviebooking/ (see the Full Edition).

Output (excerpt):

```
  add SH2 RRR 19:00 same hall -> false
== 3. 10 customers click B3+B4 at the same instant ==
  holds granted = 1 (the other 9 saw 'not available')
  paying twice is harmless -> CONFIRMED
  [notify ASHA] BK-2 hold expired before payment, nothing charged
  [notify MEENA] BK-4 hold expired during payment, Rs.200.00 refunded
  [notify NEHA] BK-6 payment DECLINED, seats released
```

The 10% coupon turns Rs.700 into Rs.630; an overlapping show in the same hall is refused; Meena's hold expired while the bank was charging and Kiran took the seat, so she is refunded.

Concurrency & edge cases

- **Atomic check-then-act:** `lock`, `holds`, `commit`, `unlock`, `releaseBooked` for one show all run `synchronized` on that show's lock table.
- **Lazy expiry:** no background thread; an expired `SeatLock` is treated as absent the next time anyone looks (Redis TTL does the same).
- **Payment-window race:** commit re-validation + refund.
- **Lock order:** booking monitor (serialises `pay`/`cancel` — no double charge, no pay-while-cancel) → show monitor, never reversed.
- **Across many servers**, from simplest: (1) **conditional update** (course): `UPDATE show_seat SET status='HELD', held_by=?, held_until=? WHERE show_id=? AND seat_id IN (...) AND (status='FREE' OR held_until < now())`, check the row count; (2) **pessimistic** `SELECT ... FOR UPDATE`; (3) **serializable** transaction (course) — simplest to reason about, slowest under collision; (4) **optimistic** `version` column, retry on conflict (rare conflicts); (5) **unique constraint** on `(show_id, seat_id)` as the final net; (6) **Redis TTL** keys for holds, DB as truth for confirmed seats. The in-process lock only makes the common case quick; with two servers, correctness comes from the database.
- **Edge cases:** empty or duplicate seat ids (de-duplicated), unknown seat, blocked customer, overlapping show, cancelling an abandoned booking, cancelling too late (no refund).

Extensions & follow-up questions

- **Live seat map** → push seat events over WebSockets/SSE (Observer). **Extend the hold** → refresh `expiresAt` only if still held. **Dynamic pricing** → a `PricingStrategy`.
- **Paid but crashed before commit** → store PENDING with the gateway transaction id first; a reconciliation job (plus provider **webhooks**) commits or refunds.
- **Cancel a whole show** → cancel and fully refund every confirmed booking of that show.
- **Seat-gap rule** → a `SeatSelectionRule` chain (Chain of Responsibility) before locking; loyalty points → another `PaymentMethod`.

Real-world example

BookMyShow's payment-screen **countdown timer** is the hold's TTL; its **automatic refund** when the bank debits but booking fails is the compensation path.

In the interview

30–45 minutes: 4 min requirements (hold time, payment, refunds) → 4 min actors → 8 min entities, saying **why** `ShowSeat` → 12 min `SeatLockProvider` (`lock`, `commit`) and `BookingService.hold/pay/cancel` → 5 min payment failure and refund → multi-server concurrency.

> **30-second answer:** "City → Cinema → Hall → HallSeat. A Show owns a ShowSeat per hall seat — that's what's booked. Booking is hold → pay → commit: a SeatLockProvider takes all requested seats atomically with a 5-minute TTL; payment goes through a gateway interface; commit re-checks the hold and marks seats booked atomically, refunding if the hold was lost. In one process a per-show monitor prevents double booking; across servers a conditional update or unique constraint is the real guarantee, with Redis TTL keys for holds."

Follow-ups: *Why TTL?* Otherwise abandoned checkouts block seats forever. *Optimistic or pessimistic?* Pessimistic/conditional update for hot shows; optimistic when conflicts are rare.

Self-check

- **Q:** Why is `ShowSeat` separate from `HallSeat`? **A:** State and price are per show; the physical seat is shared by every show.
- **Q:** Payment succeeded but `commit` returned false? **A:** Refund (compensation), mark abandoned.
- **Q:** How are expired holds cleaned up? **A:** Lazily — any check treats an expired lock as absent.
- **Q:** What makes `pay` safe to call twice? **A:** It runs under the booking's monitor and returns immediately unless PENDING.

TL;DR

Book `ShowSeat`s. Hold (all-or-nothing, TTL) → pay → commit (re-validate atomically) → refund if lost. Coupon on the booking, gateway behind an interface, refund rule as a Strategy. Per-show lock in one JVM; conditional updates, unique constraints and Redis TTL across servers.

15.3 Design a Concert Ticket Booking System

Problem & clarifying questions

Concert ticketing (**Ticketmaster**, **Live Nation**, **SeatGeek**) has the same objects as 15.2 — we reuse them — but a different enemy: **extreme contention**. A stadium sells 50,000 tickets in minutes to a million people arriving at 10:00:00. That changes three things: (1) **fairness** — who tries first (a *virtual waiting room*); (2) **throughput** — locks that serialise everyone become the bottleneck (use *lock-free* CAS); (3) **abuse** — scalpers and bots (*per-user limits*) and fans who miss out (*waitlist*).

Question	Answer we assume
Seat types?	**VIP** and **PREMIUM** numbered seats; **GENERAL_ADMISSION** (GA) standing = a *count*.
Search?	By artist, venue, date, time (as 15.2).
Stampede?	**Virtual waiting room**: queue numbers, batches admitted in order.
Limit?	**4 tickets** per user per concert.
Hold time?	**10 minutes**.
Sold out?	Per-tier **waitlist**; released tickets offered to it first.
Notifications?	Confirmation; "a ticket freed up for you".

Requirements

Functional: show concerts with layout and availability; search; choose seats or a GA quantity and buy; **prevent double booking** under concurrency; **fair access**; secure payments; confirmations; **waitlist** for sold-out events.

Non-functional: extreme write contention; **no overselling** ever; FCFS fairness; per-user caps; self-expiring holds.

Use cases

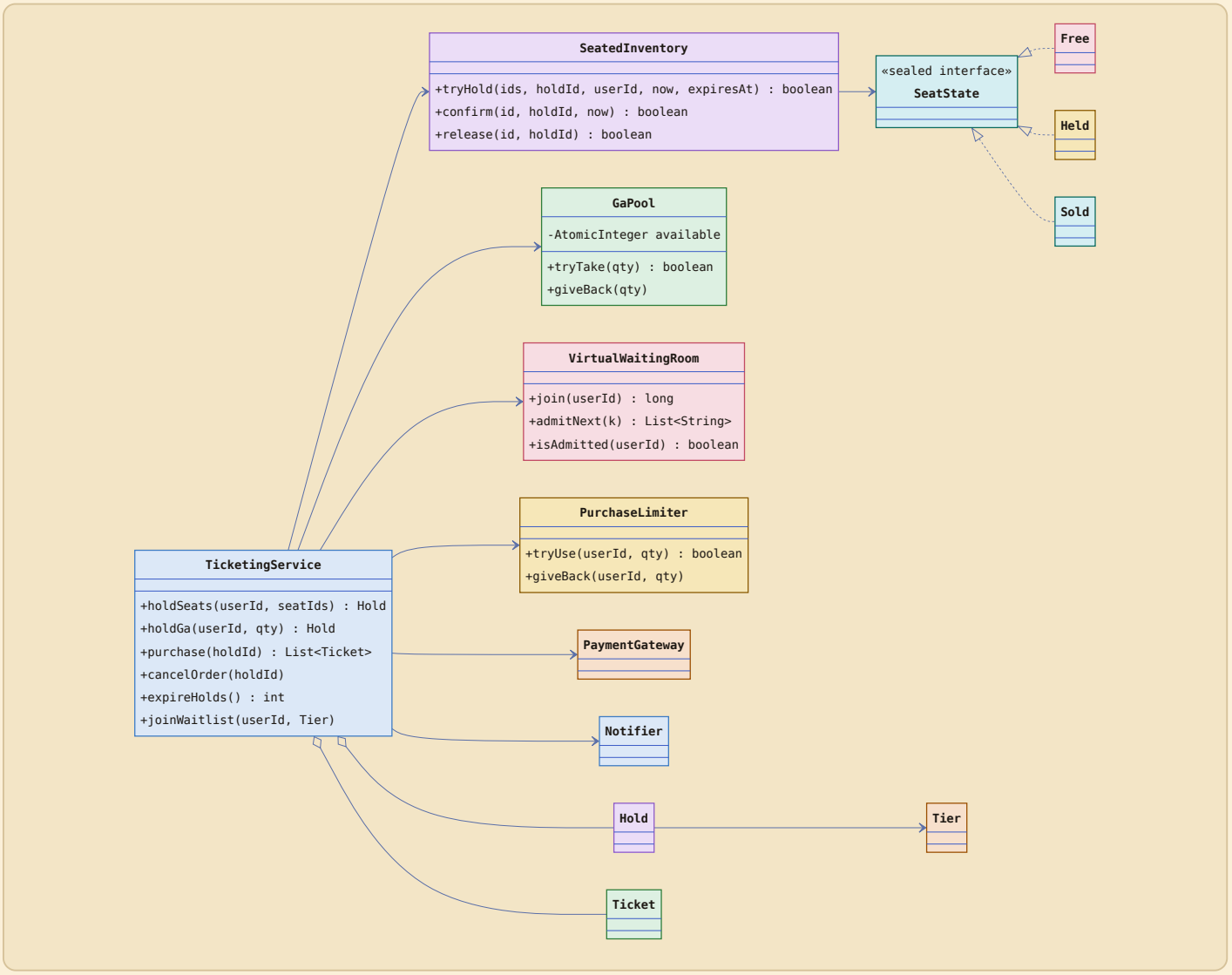
Actors: Fan (join queue, hold, buy, cancel, join waitlist), Organizer/Admin (create concert, tiers, prices, open sale), System (admit batches, expire holds, offer released tickets, notify), Payment provider. **Flow:** join waiting room → admitted → **hold** → **purchase** → tickets; cancel (refund); join waitlist; system **reaps** expired holds.

Core entities

Class	Responsibility
`Concert` (record)	Artist, venue, date/time.
`Tier` (enum)	`VIP`, `PREMIUM` (seated), `GENERAL_ADMISSION` (unseated), each with a price.
`SeatState` (sealed) → `Free`, `Held`, `Sold`	A seat's whole state as one immutable value.
`SeatedInventory`	One `AtomicReference<SeatState>` per seat; claims by **CAS**.
`GaPool`	GA capacity as an `AtomicInteger`; CAS take/give back.
`Hold` (record)	Seats or GA quantity, amount, expiry.
`Ticket` (record)	Tier and seat id or "GA".
`VirtualWaitingRoom`	Arrival-order numbers; admits FIFO batches.
`PurchaseLimiter`	Max N per user, atomic check-and-update.
`TicketingService`	hold → purchase → cancel; reaper; waitlist offers.
`PaymentGateway`, `Notifier`	External boundaries (as 15.2).

Class diagram

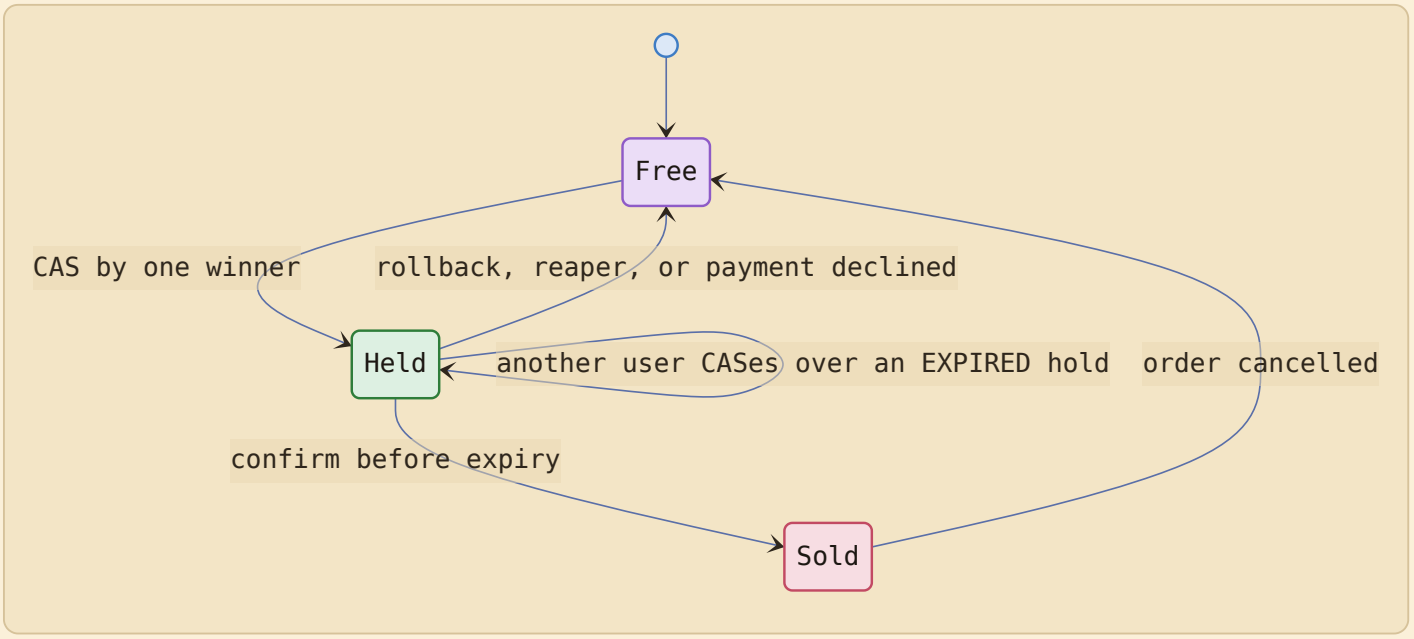
No course lesson here; the problem comes from the *awesome-low-level-design* set. Our design:



Key flows

Buying: fan joins the room (gets e.g. number 41,207; the system calls `admitNext(batch)` every few seconds) → `holdSeats`: is admitted? → `limiter.tryUse` → CAS each seat Free → Held in **sorted order**; if any CAS fails, roll back own Held seats and give back the limit (`SoldOutException`) → `purchase`: remove the hold from the active map (claims it once) → charge → CAS each seat Held → Sold → tickets.

Seat state machine — every arrow is one `compareAndSet`:



Released inventory (expired hold, cancellation) is **offered to the tier's waitlist first**: the next fan is admitted directly and given a fresh hold plus a notification; if unpaid, the reaper releases it to the next.

Design decisions & patterns used

Decision	Why	Alternative rejected
Lock-free CAS per seat	Different seats never wait; on one seat exactly one CAS wins, losers learn instantly; no thread blocks holding a lock, so no deadlock.	One monitor per concert (15.2) — at 50,000 buyers everyone queues on one lock.
Immutable sealed `SeatState`	CAS compares *references*; a fresh record per state means "what I read" is exactly "what I replace"; exhaustive pattern matching.	Mutable `status` / `heldBy` / `expiry` fields — readers see half-updated seats.
Sorted multi-seat hold + rollback	Release only seats *we* took; sorted order makes overlapping buyers contend on the same first seat, so one fails fast.	Nested locks — deadlock-prone.
GA as a counter	Standing places are interchangeable; CAS loop (`if cur < qty fail; else CAS(cur, cur-qty)`) never oversells.	40,000 `Seat` objects.
Virtual waiting room	Arrival order decides who may *try*; batches cap load; bots gain nothing. `join` is `synchronized` so number order = queue order.	"Whoever lands first" — rewards bots, melts the backend.
Per-user limit via `ConcurrentHashMap.compute`	Check-and-update for one key atomically.	`get` then `put` — a race.
TTL holds + reaper	Seats reclaimed lazily (CAS over expired `Held`); GA holds need `expireHolds()`. `holds.remove(id, h)` ensures reaper and `purchase` never both process a hold.	Holds that never expire.
Waitlist offers on release	Longest-waiting fan (FIFO), not fastest refresher.	Silent return to the pool.
Compensation on lost hold	As 15.2: release and refund.	Assuming the hold is valid.
No Singleton	The *awesome-lld* reference uses one; DI is testable and allows one service per concert.	Global Singleton.

Java implementation

Package `lld.ch15.concert` (`Model`, `Inventory`, `TicketingService`, `Main`). Demos use Java 21 **virtual threads** (`Executors.newVirtualThreadPerTaskExecutor()`), making 1,000 concurrent buyers cheap.

```java
/** A seat's whole life in one immutable value, swapped atomically with compare-and-set (CAS). */
sealed interface SeatState permits Free, Held, Sold {}
record Free() implements SeatState {}
record Held(String holdId, String userId, Instant expiresAt) implements SeatState {
    boolean expired(Instant now) { return !now.isBefore(expiresAt); }
}
record Sold(String holdId, String userId) implements SeatState {}

final class SeatedInventory {
    // ...
    private boolean tryHoldOne(String id, String holdId, String userId, Instant now, Instant expiresAt) {
        AtomicReference<SeatState> ref = seats.get(id);
        if (ref == null) throw new IllegalArgumentException("no seat " + id);
        while (true) {
            SeatState cur = ref.get();
            boolean claimable = cur instanceof Free || (cur instanceof Held h && h.expired(now));
            if (!claimable) return false;
            if (ref.compareAndSet(cur, new Held(holdId, userId, expiresAt))) return true;
            // CAS lost: someone changed the seat between get() and CAS. Re-read and decide again.
        }
    }
    // ...
}

/** General admission: no seat numbers, just a counter. A CAS loop means it can never go below zero. */
final class GaPool {
    // ...
    boolean tryTake(int qty) {
        while (true) {
            int cur = available.get();
            if (cur < qty) return false;
            if (available.compareAndSet(cur, cur - qty)) return true;
        }
    }
    // ...
}

final class PurchaseLimiter {
    // ...
    boolean tryUse(String userId, int qty) {
        boolean[] ok = {false};
        used.compute(userId, (u, cur) -> {
            int c = cur == null ? 0 : cur;
            if (c + qty > max) return cur;             // unchanged
            ok[0] = true;
            return c + qty;
        });
        return ok[0];
    }
    // ...
}

final class TicketingService {
    // ...
    List<Ticket> purchase(String holdId) {
        Hold h = holds.remove(holdId);                    // atomic claim: a hold is purchased at most once
        if (h == null) throw new IllegalStateException(holdId + " unknown, expired or already used");
        // ... expired / declined -> releaseInventory(h); then CAS each seat Held -> Sold, refund if lost
    }
    // ...
}
```

Full runnable code: code/lld/ch15/concert/ (see the Full Edition).

Output (excerpt):

```
REFUSED: eli is still in the waiting room
holds on VIP-1 = 1
sold=98 soldOut=902 gaLeft=0 (never negative)
REFUSED: ben would exceed the per-user ticket limit
[notify cai] a GA ticket freed up, held for you as H244 until 2026-06-01T09:10:00Z
expired holds reaped = 3 (VIP-1, cai's unpaid offer, dev's P-6)
```

40 concurrent CAS attempts on one seat → exactly one hold; 1,000 buyers for 98 GA places sell exactly 98; a cancellation goes straight to the waitlisted fan as a hold; money collected 85,900.00 = 98 × 800 + 3 × 2,500.

Concurrency & edge cases

- **Why CAS is correct:** `compareAndSet(expected, new)` replaces only if the value is still the object we read; otherwise re-read and decide. `tryHoldOne` retries only while the seat is still claimable.
- **ABA problem** (A → B → A fooling a CAS)? Every transition creates a *new* record and `AtomicReference` compares references, so a returned "A" is a different object — the CAS fails safely.
- **Hold claimed once:** `purchase` does `holds.remove(holdId)`, the reaper `holds.remove(id, h)` — only one wins.
- **Limit given back** on every failure path (sold out, declined, expired, lost seat).
- **Lost hold after payment:** `confirm` fails → release, give back limit, refund.
- **Fair queue scope:** fair within one process; at scale the waiting room is a separate service (often at the CDN edge) issuing signed, time-limited access tokens.
- **Across servers:** seat CAS → `UPDATE seat SET state='HELD', hold_id=?, expires_at=? WHERE id=? AND (state='FREE' OR expires_at < now())` or Redis `SET NX PX`; GA → Redis `DECRBY` in a Lua script, or `WHERE available >= ?`.

Extensions & follow-up questions

- **Best available seats** → a `SeatAllocator` strategy picks adjacent free seats, then the same CAS hold.
- **Verified fans / presale codes** → admission tokens for pre-registered fans; limiter keyed on verified identity.
- **Dynamic pricing** → a `PricingStrategy` using remaining inventory and time, frozen in the `Hold`.
- **Resale** → transfer ownership and re-issue a new barcode.
- **Waitlist for seated tiers** → same mechanism per released seat (`offerSeat`).

Real-world example

Ticketmaster uses a virtual queue ("Smart Queue"), **Verified Fan** presales and per-account limits. The 2022 *Eras Tour* presale overwhelmed the system and the general sale was cancelled — admission control matters as much as seat locking.

In the interview

30–45 minutes: say "entities like movie booking; the hard part is contention and fairness" → tiers (why GA is a counter) → TTL holds; code the seat CAS and GA CAS loop → waiting room and atomic per-user limit → waitlist and reaper → scale-out (tokens, conditional updates/Redis).

30-second answer: "Seated tiers are an `AtomicReference<SeatState>` per seat claimed by CAS — one winner, no locks. GA is an `AtomicInteger` decremented by a CAS loop, never negative. Everyone enters through a virtual waiting room admitting in arrival order, with an atomic per-user limit. Holds expire after 10 minutes, a reaper returns them, and released tickets go to a FIFO waitlist first. A hold lost during payment is refunded."

Follow-ups: *Lock-free vs locks?* Lock-free wins under high contention with tiny critical sections; locks are simpler when many things change together. *Queue service fails?* Fail closed — pause admissions. *One person, 50 accounts?* Verified identity, card fingerprinting, limits on those.

Self-check

- **Q:** Why is GA a counter? **A:** Places are interchangeable; a CAS counter is simpler and can't oversell.
- **Q:** What does `compareAndSet` guarantee? **A:** Update only if the value is still the one you read.

- **Q:** Why must `join` be synchronized? **A:** So number and queue position are assigned together, in the same order.
- **Q:** How can't a hold be both purchased and reaped? **A:** Both remove it from `holds` first; only one removal succeeds.

> **TL;DR**

Same nouns as movie booking, different enemy: contention. CAS per seat, CAS counter for GA, FIFO virtual waiting room, atomic per-user limits, TTL holds with a reaper, waitlist first for released tickets, refunds when a hold is lost.

15.4 Design a Hotel Management System

Problem & clarifying questions

A **Hotel Management System** (property management system, PMS) knows every room, takes bookings, checks guests in and out, issues key cards, records housekeeping and room service, and produces the bill. Unlike a cinema, a room is sold **night by night** — that shapes the whole design.

Question	Answer we assume
Locations, types, search?	Many `HotelLocation`s; `STANDARD`, `DELUXE`, `FAMILY_SUITE`, `BUSINESS_SUITE`; search by type and **date range**.
Cancellation?	**Full refund if more than 24 h before check-in**; otherwise none (our assumption).
Extras?	Room service, kitchen, amenities on the bill.
Payment?	**Card, check or cash**.
Keys?	Electronic cards with barcode and key id; deactivated at check-out.
Housekeeping?	Logged per room, never overwritten.
Several rooms per booking?	No — one room (a known limitation).

> **Requirements**

Functional (course): (1) book **room types**; (2) **search** and book any **available** room; (3) **who booked a room** / **which rooms a customer booked**; (4) **cancel**, full refund > 24 h before check-in; (5) **notify** before check-in and check-out; (6) **housekeeping log**; (7) **room service and food**; (8) **amenities**; (9) pay by **card, check or cash**.

Non-functional: never double-book a room-night; bill = sum of lines; actors limited to their jobs; refund rule in one place.

Use cases

Actors (course): Guest (search, book), Receptionist (rooms, bookings, check in/out), System (notifications), Manager (hires workers), Housekeeper (housekeeping records), Server (room-service records). **Top use cases:** manage rooms; search; register/cancel account; book; **check in**; **check out** (return keys); add room charge; update housekeeping.

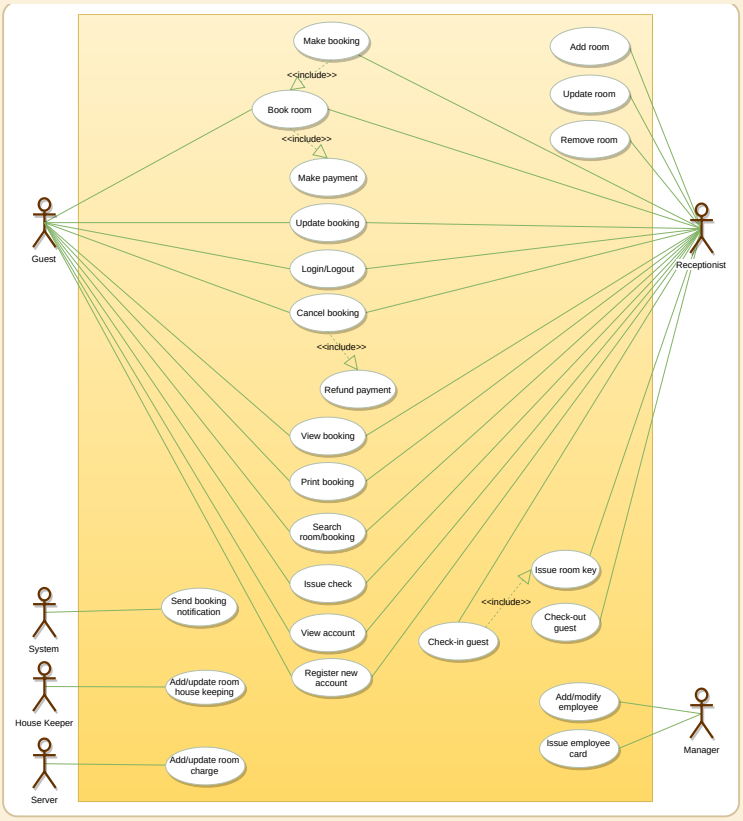


Figure — Hotel use cases: Guest and Receptionist share booking, update, cancel, view, print and search; "Make booking" includes "Book room", which includes "Make payment"; "Cancel booking" includes "Refund payment"; the receptionist's "Check-in guest" includes "Issue room key"; the System sends booking notifications; the House Keeper adds/updates housekeeping; a manager adds/modifies employees (course diagram)

Core entities

Class	Responsibility
`Hotel`	Owns locations; searches each.
`HotelLocation` implements `Search`	Owns rooms; "which rooms of this type are free for these dates?".
`Room`	Number, style, price; the **set of booked nights**; owns keys and housekeeping log; current `RoomStatus`.
`RoomStyle`, `RoomStatus`	Type; what it is doing *now* (`AVAILABLE`, `RESERVED`, `OCCUPIED`, `NOT_AVAILABLE`, `BEING_SERVICED`, `OTHER`).
`RoomKey`	Key id, barcode, issue time, master, active.
`RoomHouseKeeping` (record)	When, who, what, how long.
`RoomBooking`	Reservation number, guest id, room, dates, status, deposit; **owns its** `Invoice`; `refundDue()`.
`BookingStatus`	`REQUESTED`, `PENDING`, `CONFIRMED`, `CHECKED_IN`, `CHECKED_OUT`, `CANCELLED`, `ABANDONED`.
`Invoice`, `InvoiceItem`	Lines; total computed.
`RoomCharge` → `Amenity`, `RoomService`, `KitchenService`	Writes itself onto the invoice (Template Method).
`Payment` (sealed) → `CreditCardTransaction`, `CheckTransaction`, `CashTransaction`	Three genuinely different behaviours.
`Person` (sealed) → `Guest`, `Receptionist`, `Server`, `Housekeeper`, `Manager`	Actors; methods accept only the right type.
`FrontDesk`	Book, check in, add charge, check out, cancel, housekeeping, reminders, queries.
`Notifier`	Messages to guests.

Class diagram

Five course stages: (1) **invoice, payments, notifications** — the invoice stores **lines** and computes its total, so it can't drift from them (req. 7, 8); (2) **room, keys, housekeeping** — the key decision: **a status field is not availability** — `OCCUPIED` describes now, but guests ask about future date ranges, so each room stores its sold nights; (3) **hotel, location, search** — search sits on the **location** (a room can't list other free rooms; `Room.search()` is a common beginner mistake); (4) **booking and room charges**; (5) **people** — there is deliberately **no arrow guest ↔ booking**: the booking stores the guest's **id**, and queries filter by it.

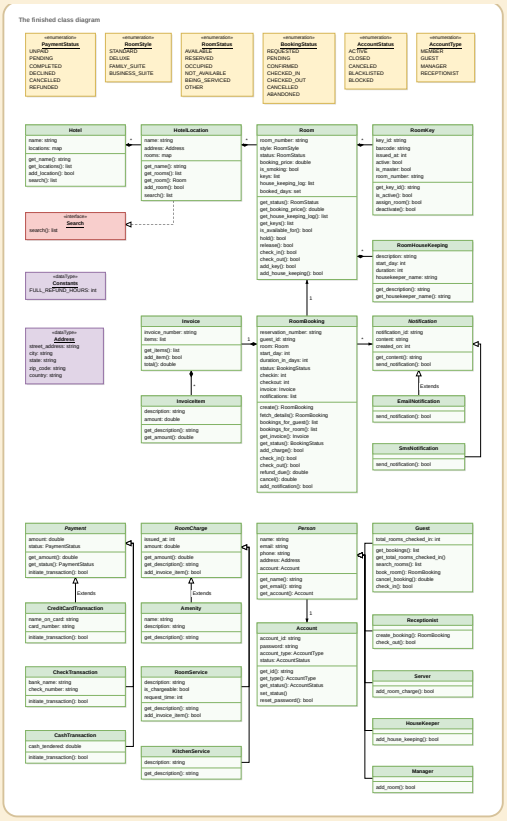
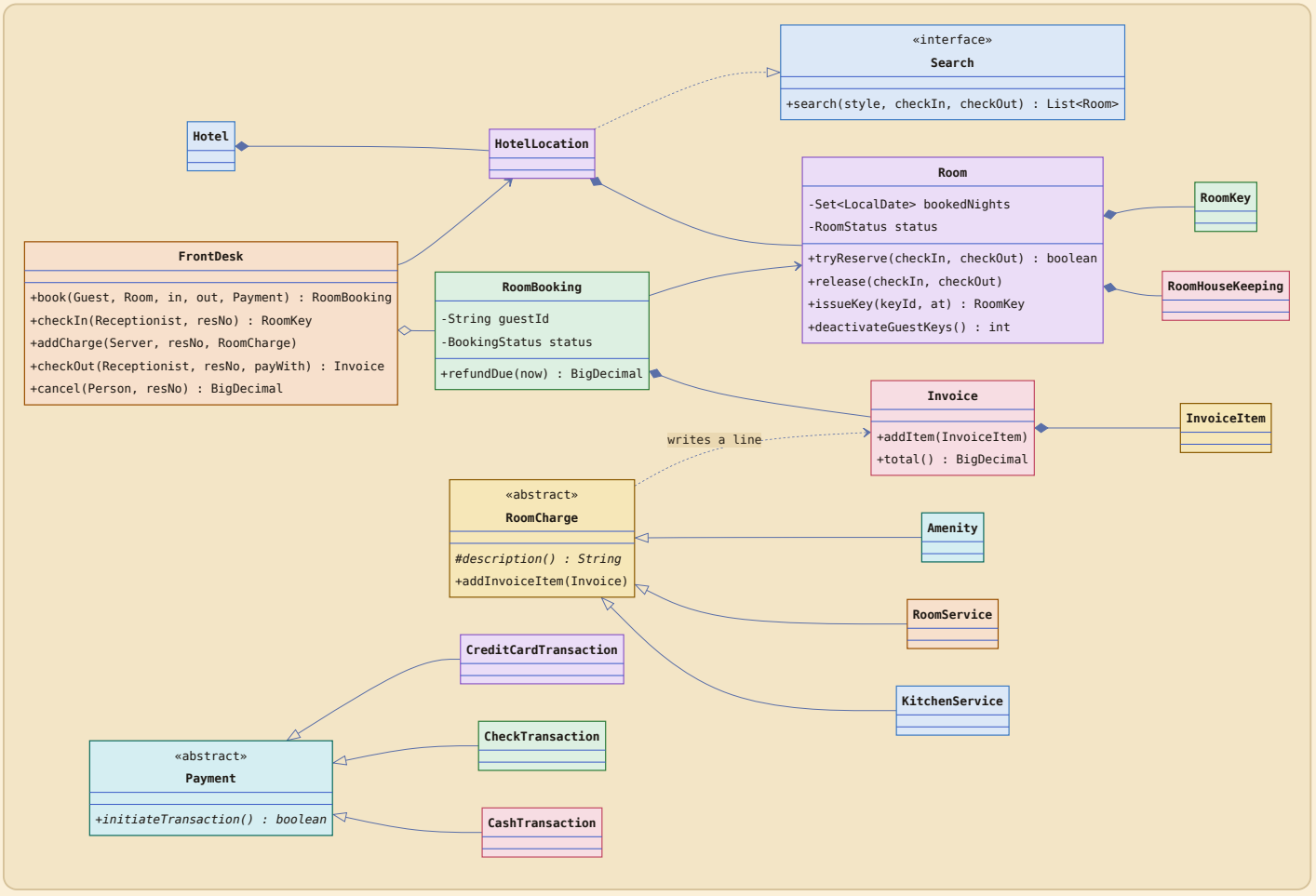


Figure — The finished hotel class diagram (course diagram)



Key flows

The course's activity diagrams: **make a booking** (search type and dates → none: tell customer; else find/register customer, book, pay, deliver details), **check in** (find booking; room not ready → wait; else

issue key, status "Checked-in"), **cancel** (find booking; before the deadline → refund, else none; print/email).

A whole stay: guest `book(room 102, Jul 10–13, deposit)` → `tryReserve` under the room monitor (nights 10, 11, 12 taken) → CONFIRMED → receptionist `checkIn` (room OCCUPIED, key issued) → server `addCharge(KitchenService)` writes an invoice line → `checkOut(pay by card)` adds the nights line and subtracts the deposit, deactivates keys, room BEING_SERVICED → invoice total = sum of lines.

Booking lifecycle: `CONFIRMED → CHECKED_IN → CHECKED_OUT`, or `CONFIRMED → CANCELLED` (refund if > 24 h ahead). The room's own status cycles independently: `AVAILABLE → OCCUPIED → BEING_SERVICED → AVAILABLE` (after housekeeping).

Design decisions & patterns used

Decision	Why	Alternative rejected
Availability = set of booked nights	Answers date ranges; nights are half-open `[checkIn, checkOut)`, so leaving and arriving on the 13th don't clash.	`RoomStatus` — only knows now.
`tryReserve` `synchronized` on the room	Check-then-act made atomic; rooms don't block each other.	Hotel-wide lock.
Search on `HotelLocation`	A room can't answer "which rooms are free?".	`Room.search()`.
Invoice = lines, total computed	Total can't disagree with lines.	Stored running total.
Template Method `RoomCharge.addInvoiceItem`	`final` parent algorithm (describe → price → write line); subclasses supply only `description()`; new charge = new subclass (OCP). The course calls it its clearest Template Method; Strategy swaps a *whole* method instead.	`switch` on charge type in the invoice.
Three `Payment` subclasses	Card completes at once; **check stays `PENDING`**; cash declines if short. Each passes LSP.	One `Payment` with a `type` field.
Refund rule in one method (`refundDue`)	One place to edit; promote to a `CancellationPolicy` strategy (15.2) for several policies.	Scattered refund arithmetic.
Actor types in signatures	`addCharge(Server, …)`, `logHousekeeping(Housekeeper, …)`, `addRoom(Manager, …)` — compiler-enforced roles; a receptionist booking for a guest calls the *same* method.	Anyone calls anything.
Booking stores guest id	No two-way dependency; "rooms of this guest" is a query.	Bidirectional links.
Nights released on failed deposit / cancel	Reserve-then-pay needs an undo.	Phantom reservations.

Java implementation

Package `lld.ch15.hotel` (`Billing`, `Rooms`, `Stay`, `Main`).

```
final class Room {
    // ...
    /** Nights are [checkIn, checkOut): the checkout day itself is free for the next guest. */
    synchronized boolean isFree(LocalDate checkIn, LocalDate checkOut) {
        for (LocalDate d = checkIn; d.isBefore(checkOut); d = d.plusDays(1)) if (bookedNights.contains(d))
return false;
        return true;
    }
    synchronized boolean tryReserve(LocalDate checkIn, LocalDate checkOut) {
        if (!isFree(checkIn, checkOut)) return false;              // re-entrant monitor: same lock
        for (LocalDate d = checkIn; d.isBefore(checkOut); d = d.plusDays(1)) bookedNights.add(d);
        return true;
    }
    // ...
}

abstract class RoomCharge {
    private final BigDecimal amount;
    RoomCharge(BigDecimal amount) { this.amount = amount; }
    protected abstract String description();                // the one step that varies
    final void addInvoiceItem(Invoice invoice) {            // the template: final, so the order can't change
        invoice.addItem(new InvoiceItem(description(), amount));
    }
}

final class RoomBooking {
    // ...
    /** The refund rule lives in exactly ONE method: full refund if cancelled more than 24h before check-in.
*/
    BigDecimal refundDue(LocalDateTime now) {
        Duration notice = Duration.between(now, checkIn.atTime(CHECK_IN_TIME));
        // compareTo, not toHours(): toHours() truncates, so 24h30m would wrongly count as "24, not more than
24".
        return notice.compareTo(FULL_REFUND_NOTICE) > 0 ? deposit() : BigDecimal.ZERO;
    }
}

final class FrontDesk {
    // ...
    /** Guest or receptionist books: same call, two actors. The room's monitor makes it race-free. */
    RoomBooking book(Guest guest, Room room, LocalDate checkIn, LocalDate checkOut, Payment deposit) {
        if (!checkIn.isBefore(checkOut)) throw new IllegalArgumentException("check-out must be after check-
in");
        if (!room.tryReserve(checkIn, checkOut))
            throw new IllegalStateException("room " + room.number() + " is not free " + checkIn + ".." +
checkOut);
        RoomBooking b = new RoomBooking("R" + seq.incrementAndGet(), guest.id(), room, checkIn, checkOut);
        if (!deposit.initiateTransaction()) {                  // payment failed: give the nights back
            room.release(checkIn, checkOut);
            throw new IllegalStateException("deposit " + deposit.status());
        }
        // ... store booking, notify
    }
    // ...
}
```

Full runnable code: code/lld/ch15/hotel/ (see the Full Edition).

Output (excerpt):

```
  successful bookings = 1 of 6
  [notify BEN] R3 confirmed: room 102 2026-07-13..2026-07-15
  [notify CARL] R5 cancelled by Carl, refund $0.00
  [notify ASHA] R4 cancelled by Asha, refund $250.00
  cash short: payment DECLINED, guest still checked in
    TOTAL                          $299.00
  key active after checkout? false, room 102 is BEING_SERVICED
```

Six concurrent overlapping bookings of room 101 → one wins; Ben's stay from the 13th fits after Asha's ending the 13th; Carl cancels 19 h ahead (no refund), Asha 8 days ahead (full); the bill is $5 + $24 + $30 + 3 × $120 − $120 deposit = **$299**, and a declined cash payment doesn't duplicate lines on retry.

Concurrency & edge cases

- **Double-booking a night:** `Room.tryReserve` is `synchronized` — check and add of every night are one step; the 6-thread race has one winner.
- **Search is advisory:** the room may be gone a millisecond later; `tryReserve` is the real check.
- **Booking transitions** (`checkIn`, `checkOut`, `cancel`) run under the booking's monitor; lock order booking → room.
- **The 24-hour rule, exactly:** compare `Duration`s; `toHours() > 24` truncates minutes, so 24 h 30 min would wrongly lose its refund.
- **Retry-safe checkout:** nights and deposit lines are written only after payment succeeds; `addCharge` takes the same booking monitor, so no charge slips in after totalling.
- **Edge cases:** check-out not after check-in; checking in too early; charges when not in house; cancelling after check-in; short cash (declined); a check stays `PENDING`.
- **Chain-wide inventory:** per-room nights don't scale across channels; real systems use an **inventory table** keyed by `(room_type, date)` with `UPDATE ... SET sold = sold + 1 WHERE sold < capacity`.

Extensions & follow-up questions

- **Two rooms in one booking** (course flags it) → a `Reservation` holding several `RoomStay`s; all-or-nothing across rooms by locking in room-number order, or one DB transaction.
- **Book by type, assign at check-in** → per-type-per-night counts for selling; pick the physical room at check-in.
- **Seasonal rates** → a `RatePlan`/`PricingStrategy`, one line per night.
- **Overbooking** → sell capacity × (1 + factor); "walk" extras to a partner hotel — a strategy on the availability check.
- **Booker vs occupants** (course names it missing) → `Occupant`s on the stay. **Lost key** → `deactivateGuestKeys()` + `issueKey()`.

Real-world example

Oracle OPERA sells room types by date, assigns rooms at check-in, and posts charges to the folio; key cards (VingCard, dormakaba) expire at check-out; **Booking.com** shows per-rate policies like "free cancellation until 24 hours before arrival".

In the interview

30–45 minutes: clarify (types, date search, 24 h refund, charges, payments, keys, housekeeping) → actors → entities, saying early **"a status field is not availability"** → code `tryReserve`, `book`, `checkIn`, `checkOut` → Template Method charges, payment subclasses, refund rule → follow-ups (two rooms, chain inventory, overbooking).

30-second answer: "A `Hotel` has `HotelLocation`s that own `Room`s and search by type and date range. Each room keeps its booked nights, and reserving nights is a synchronized check-then-act. A `RoomBooking` owns an `Invoice` of lines; room service, kitchen and amenities are `RoomCharge` subclasses adding a line via Template Method. Payment is card, check or cash subclasses. Cancellation refunds in full if more than 24 hours before check-in — one method. Check-in issues a key; check-out totals the invoice, takes payment, kills the keys and hands the room to housekeeping."

Self-check

- **Q:** Why isn't `RoomStatus` enough? **A:** It describes now; a booking asks about future nights.
- **Q:** Why half-open `[checkIn, checkOut)`? **A:** The checkout day's night isn't used, so the next guest can arrive that day.
- **Q:** Which pattern adds a charge type without touching `Invoice`? **A:** Template Method.
- **Q:** How is "only a server adds room service" guaranteed? **A:** `addCharge` takes a `Server`; other types don't compile.

TL;DR

Rooms are sold night by night: each room keeps booked nights and reserves them atomically. Search on the location. The bill is lines written by `RoomCharge` Template Methods; three payment subclasses; one 24-hour refund method. Keys issued at check-in, killed at check-out; housekeeping is append-only.

Chapter summary

Problem	Inventory unit	How a hold/booking is made safe	Payment failure handling	Cancellation
Library	`BookItem` (copy)	`synchronized` member → item (fixed order)	— (fines recorded)	Return → FIFO waitlist per title gets the copy
Movie tickets	`ShowSeat` (seat × show)	Per-show monitor; all-or-nothing lock with TTL; atomic commit	Declined → release; lost hold → refund (compensation)	Refund per `CancellationPolicy`; seats back on sale
Concert tickets	Numbered seat state / GA counter	Lock-free CAS; waiting room; per-user limit	Same compensation as movies	Released tickets offered to waitlist first
Hotel	Room × night	`synchronized` `Room.tryReserve` over a date range	Failed deposit → nights released	Full refund if > 24h before check-in

Five sentences to remember for any booking interview:

1. Name the *real* unit of inventory first (copy, show seat, room-night, GA count).
2. Every "if free then take" must be one atomic step — a monitor, a CAS, or a conditional `UPDATE`.
3. Temporary holds need an expiry, or abandoned checkouts block inventory forever.
4. Money can't be rolled back — failures after payment need a compensating refund.
5. In-process locks protect one server; with many servers, the database (constraints, conditional updates) or a TTL store like Redis is the real guarantee.

Chapter 16 — Case Studies: Booking & Inventory (Part 2) — Study Edition

> **Where this fits.** Chapter 15 covered the first group of booking problems. This chapter finishes the family with four more: car rental, restaurant, airline and course registration. They are one problem underneath: **a limited resource, a calendar, and many people trying to claim it at once.** Spot that skeleton and every new booking problem is quick to set up.

The shared skeleton of every booking problem

Idea	Car rental	Restaurant	Airline	Course registration
The thing being booked	a vehicle	a table	a seat on one flight instance	a seat in one section
The "calendar"	days	hourly time slots	the flight instance (route + date)	the term
The atomic check-then-claim	`Vehicle.tryBook`	`Table.tryHold`	`FlightInstance.holdSeats`	`Section.enroll`
When it is full	search another car	try the next table	another seat or flight	waitlist
Money	bill lines + late fee	check split by seat + tax	fare per seat class, paid per itinerary	none (fees are an extension)

A **resource** is anything in limited supply. A **race condition** is a bug where the result depends on which thread runs first — here, two people both "win" one seat because both checked "free?" before either wrote "taken".

How to run: `cd chapters/16-case-booking-2/code && javac --release 21 -d /tmp/lld_16 $(find . -name '*.java') && java -cp /tmp/lld_16 lld.ch16.carrental.Main` (or `restaurant`, `airline`, `courseregistration`). Where threads race, the *counts* are always the same; *which* racer wins varies. One file sometimes holds several small package-private types (legal Java; production code would split them).

> **Money.** All prices use `BigDecimal`, never `double` — a `double` cannot store 0.1 exactly, so sums drift by fractions of a cent.

Design a Car Rental System

Problem & clarifying questions

A **car rental system** rents vehicles for hours to weeks from many branches; a car may be returned to a different branch.

Question	Assumed answer
Vehicle kinds?	Car, truck, SUV, van, motorcycle.
Branches?	Many, each with its own vehicles; drop-off branch may differ from pick-up.
Price?	Daily rate per vehicle type + add-ons + a late fee per day late.
Add-ons?	Insurance, equipment (navigation, child seat, ski rack), services (roadside assistance, wifi), additional drivers.
Identification?	Unique barcode, scanned at pick-up and return.
History?	A vehicle log of every event (reserved, picked up, returned, fuelling, cleaning, damage).
Concurrency?	Two members must never get the same car on overlapping days.

Requirements

Functional: (1) rent car/truck/SUV/van/motorcycle; (2) add each vehicle with a unique barcode and details incl. parking stall; (3) answer "who took this vehicle?" and "what has this member rented?"; (4) late fee after the due date; (5) search and reserve any available vehicle; (6) notify when pick-up is near, due date is near, and when overdue; (7) read barcodes; (8) cancel reservations; (9) vehicle log; (10) rental insurance; (11) extra equipment; (12) services.

Non-functional: no double booking for overlapping days; new add-ons or pricing rules without changing reservation code (Open/Closed); auditability — every charge is a bill line, every event a log entry.

Use cases

Actors: **Receptionist** (adds/edits vehicles and workers, can reserve for a customer), **Member** (searches, reserves, picks up, returns), **Worker** (handles returns, updates the vehicle log), **System** (reminders, overdue alerts, cancellations). Top use cases: add/edit vehicle; search by type and availability; register/cancel membership; reserve; check out; return; add equipment/services; update log. Reserving is shared by member and receptionist; the log belongs to the worker.

Core entities

Class	Responsibility
`CarRentalSystem`	Owns the branches and the reservation service.
`CarRentalLocation`	One branch with its own `VehicleInventory`.
`VehicleInventory`	Indexes by barcode, type and model; "which SUVs are free on these days?".
`Vehicle`	Barcode, plate, type, parking stall, mileage, **the set of booked days**, its log.
`VehicleLogEntry`	One event (date, type, note).
`DateRange`	Half-open `[start, end)`; `end` is the due date.
`Reservation`	Member, vehicle, dates, pick-up/return branch, add-ons, drivers, bill, status; holds the business rules.
`ReservationAddOn` (+ `RentalInsurance`, `Equipment`, `Service`), `AdditionalDriver`	Things that add a charge; a named extra driver (no charge).
`Bill`, `BillItem`	The one place that knows what a rental costs.
`PricingStrategy`, `LateFeePolicy`	Pluggable base-charge and late-fee rules.
`ReservationService`	Creates reservations atomically, answers "who rented what", runs reminders.
`Notifier`, `PaymentMethod`	Ports for messages and money.

Class diagram

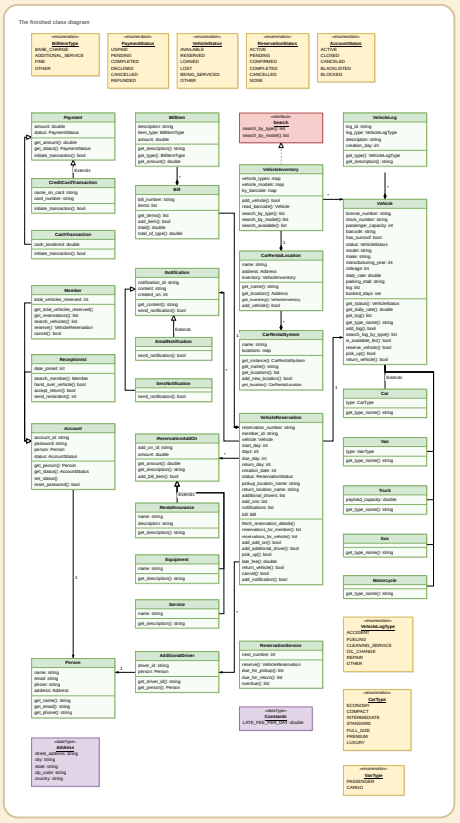
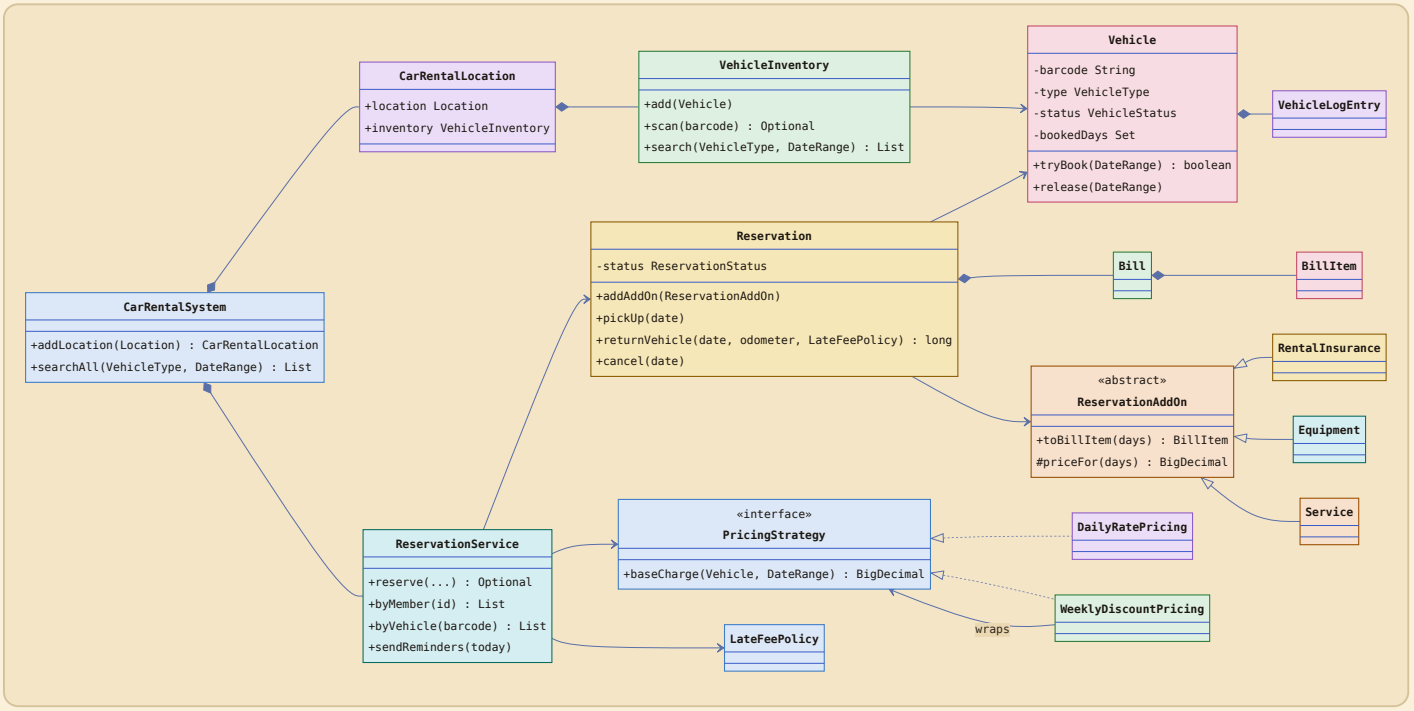


Figure — The finished class diagram of the car rental system (course diagram)

The system owns its branches and each branch owns its inventory (**composition** — the part dies with the whole); the inventory only *refers to* vehicles (**association**). A reservation owns its bill but refers to the vehicle. Our Java design differs in two ways: vehicle kinds are an enum (the five subclasses added no behaviour), and pricing is a strategy object.



Key flows

- **Pick up:** scan licence (or search by name) → check reservation → scan vehicle barcode → hand over keys; no reservation → reserve a free car on the spot.
- **Return:** worker scans the barcode → past due date adds a late fee (one line in `returnVehicle`) → check damage and fuel → settle the bill.

- **Reserve:** `ReservationService.reserve` calls `vehicle.tryBook(dates)`; if false the member picks another car; if true it prices via `PricingStrategy`, creates the reservation (bill gets a BASE line) and notifies.
- **Lifecycle (state machine):** `CONFIRMED →pickUp→ ACTIVE →returnVehicle→ COMPLETED`; `CONFIRMED →cancel→ CANCELLED` (days released on cancel and return).

Design decisions & patterns used

Decision	Why	Alternative rejected
Availability = booked days per vehicle, not a status	Members ask about future dates; a car loaned now may be free next week, a car on the lot may be promised next week.	A `RESERVED` status (can't answer date questions; one reservation per car).
Keep `VehicleStatus` too	Staff need "where is it now" (lot, loaned, in service, lost).	Dropping it.
`tryBook` checks **and** books in one `synchronized` method	Closes the check-then-act race.	`isFree()` then `book()`.
Template Method on `ReservationAddOn`	Billing written once in `final toBillItem`; subclasses give `kind` and may override the `priceFor` hook.	Type enum + `switch`.
Strategy for pricing and late fee; **Decorator** `WeeklyDiscountPricing`	Swap/stack rules without touching the reservation.	Constants; a subclass per combination.
`Bill` is the only place charges live; methods `synchronized`, `settle` idempotent	One total; no line added mid-payment; a double-click pays once.	Per-add-on running totals.
`ReservationService` separate from `Reservation`	Creating/indexing vs one reservation's rules (SRP).	Static factory.
Actors call the rules, never compute them	Member calls the service; receptionist calls the reservation.	`Receptionist.computeLateFee()`.
Course's `CarRentalSystem` **Singleton** → we pass one in	Same "one per app", but testable (dependency injection).	`getInstance()`.

Java implementation

Package `lld.ch16.carrental`. Full runnable code: code/lld/ch16/carrental/ (see the Full Edition).

```java
// Vehicle.java — check AND book inside one lock
synchronized boolean isFreeFor(DateRange r) {
    return r.dates().noneMatch(bookedDays::contains);
}

synchronized boolean tryBook(DateRange r) {
    if (!isFreeFor(r)) return false;
    r.dates().forEach(bookedDays::add);
    return true;
}

synchronized void release(DateRange r) { r.dates().forEach(bookedDays::remove); }

// Billing.java — Template Method
abstract class ReservationAddOn {
    // ...
    final BillItem toBillItem(long rentalDays) {
        return new BillItem(kind() + ": " + name, BillItemType.ADD_ON, priceFor(rentalDays));
    }

    protected abstract String kind();

    /** Hook: by default an add-on is charged per rental day. */
    protected BigDecimal priceFor(long rentalDays) { return price.multiply(BigDecimal.valueOf(rentalDays)); }
}

/** A service such as roadside assistance or wifi is a flat fee per rental. */
final class Service extends ReservationAddOn {
    Service(String name, BigDecimal flatFee) { super(name, flatFee); }
    protected String kind() { return "Service"; }
    @Override protected BigDecimal priceFor(long rentalDays) { return price; }
}

// Pricing.java — Decorator over any pricing: 10% off rentals of 7 days or more.
final class WeeklyDiscountPricing implements PricingStrategy {
    private final PricingStrategy inner;
    WeeklyDiscountPricing(PricingStrategy inner) { this.inner = inner; }
    public BigDecimal baseCharge(Vehicle v, DateRange p) {
        BigDecimal base = inner.baseCharge(v, p);
        return p.days() >= 7 ? base.multiply(new BigDecimal("0.90")).setScale(2, RoundingMode.HALF_UP) : base;
    }
}

// Reservation.java — the rules live here
synchronized long returnVehicle(LocalDate on, int odometer, LateFeePolicy lateFee) {
    require(ReservationStatus.ACTIVE, "return");
    long daysLate = Math.max(0, ChronoUnit.DAYS.between(period.end(), on));
    if (daysLate > 0) {
        bill.add(new BillItem("Late fee: " + daysLate + " day(s)", BillItemType.LATE_FEE,
                lateFee.fee(vehicle, daysLate)));
    }
    vehicle.setMileage(odometer);
    vehicle.setStatus(VehicleStatus.AVAILABLE);
    vehicle.release(period);
    // ... log RETURNED, status = COMPLETED
}

// RentalSystem.java — ReservationService
Optional<Reservation> reserve(Member m, Vehicle v, DateRange period, Location pickup, Location dropoff,
LocalDate today) {
    if (!v.tryBook(period)) return Optional.empty();
    var r = new Reservation("R" + seq.incrementAndGet(), m, v, period, pickup, dropoff, pricing.baseCharge(v,
period));
    reservations.put(r.id(), r);
    v.addLog(today, VehicleLogType.RESERVED, r.id() + " for " + period);
    notifier.send(m, "Reservation " + r.id() + " confirmed: " + v.model() + " " + period);
```

```
        return Optional.of(r);
    }
```

Output excerpt:

```
3) 8 members race for VH-002 on the same dates
   successful reservations for VH-002: 1
6) Alice's bill
   Base: RAV4 x 3 days                      BASE_CHARGE   240.00
   Insurance: Collision damage waiver       ADD_ON        45.00
   Equipment: Child seat                    ADD_ON        24.00
   Service: Roadside assistance             ADD_ON        25.00
   Late fee: 2 day(s)                       LATE_FEE      80.00
   TOTAL 414.00
   paid: true, settle again (double click) -> true, no second charge
10) A 7-day Civic rental gets the weekly discount
   base charge 315.00 (7 x 50 = 350, minus 10%)
```

Eight threads race for one car and exactly one wins; the late fee is 2 × 40.00; a second `settle` does not charge again.

Concurrency & edge cases

- **Double booking** (a gap the course admits): `tryBook` is `synchronized` per vehicle, so check and mark happen as one step; per-vehicle locks let different cars book in parallel. In a database: unique key on `(vehicle_id, day)` or `SELECT … FOR UPDATE`.
- **Search is advisory** — `reserve` may still fail; show "no longer available".
- **Late return collides with the next rental** — keep a buffer day; reassign the next booking to another car of the same type.
- **Early return** releases the whole range; refunds are policy. **Cancel after pick-up**, **add-ons after the rental**, **invalid ranges**, **odometer going backwards** are all rejected.
- **Different return branch** — recorded, but the car stays in the pick-up inventory: a known gap (transfers).

Extensions & follow-up questions

- **One-way rentals?** On return at another branch move the vehicle between inventories, log a `TRANSFER`, add a one-way fee line.
- **Seasonal/dynamic pricing?** A `SeasonalPricing` strategy or another decorator; reservation unchanged.
- **Deposits?** `authorize` at pick-up, `capture` at return. **Mileage/fuel/damage?** `FINE` bill lines. **Reminders?** A daily scheduler calls `sendReminders(today)`. **Scale?** Partition by branch, `(vehicle_id, day)` unique key, search on a replica/index.

In the interview

30-second answer: "Branches own inventories indexed by barcode, type and model. Availability is a question about *dates*, so each vehicle keeps its booked days and `tryBook` checks and books atomically. A reservation owns a bill; every charge is a line. Add-ons share a template-method base; pricing and late fees are strategies."

Plan (40 min): 0–5 clarify; 5–10 actors and entities; 10–20 class diagram ("dates, not status"); 20–35 code `tryBook`, `Reservation`, `reserve`; 35–40 concurrency, late collisions, transfers. Likely follow-ups: book next month while loaned now? (yes — dates); two clicks at once? (atomic `tryBook`); new add-on? (one subclass).

Self-check

1. **Q:** Why isn't a `RESERVED` status enough for "free next week?" **A:** Status describes *now*; only per-day bookings answer date questions.
2. **Q:** Pattern and hook in `toBillItem`? **A:** Template Method; `priceFor` is the hook (`Service` makes it flat).
3. **Q:** Why one `tryBook` not `isFree` + `book`? **A:** Atomic check-then-act under the vehicle lock.

4. **Q:** How is the late fee computed? **A:** Days from `period.end` to return × `LateFeePolicy` rate, as a `LATE_FEE` line.

TL;DR

- Availability = booked **days** per vehicle; status = where it is now.
- `tryBook` = atomic check-and-book per vehicle.
- One bill per reservation; late fee = days late × rate.
- Template Method (add-ons), Strategy + Decorator (pricing). Gaps: branch transfers, buffer between rentals.

Real-world example. Hertz or Enterprise sell a *car class* ("intermediate SUV or similar"), counting availability per type per day and binding a specific vehicle at the counter — the same design one level up.

Design a Restaurant Management System

Problem & clarifying questions

A **restaurant management system** runs tables, reservations, orders, the kitchen, and the bill.

Question	Assumed answer
One restaurant or chain?	Several branches, each with its own menu, tables, staff.
Menu?	Menu → sections → items with prices.
Orders?	Waiter creates an order per table and adds meals **per seat**; meal items point at menu items.
Reservations / walk-ins?	Search tables by time and party size and reserve; cancel; reminders; show tables free now.
Layouts?	Several seating arrangements per branch ("Weekday", "Banquet").
Payment?	Card, cheque or cash.
Kitchen?	Orders go to a chef queue, cooked in order; waiter told when ready.

Requirements

Functional: (1) several branches; (2) menu per branch; (3) sections with items; (4) waiter creates an order per table, meals per seat; (5) meal items ↔ menu items; (6) free tables for walk-ins; (7) table reservations; (8) search by date/time and reserve; (9) cancel; (10) reminder notification; (11) pay by card, cheque or cash; (12) several seating arrangements; (13, added) kitchen queue, waiter notified when ready.

Non-functional: no two parties on one table for overlapping time; prices live in one place (the menu item); easy to add a payment type or chef-assignment rule.

Use cases

Actors: **Receptionist** (tables, layouts, reservations, check-in), **Waiter** (orders), **Manager** (staff, menu), **Chef** (cooks), **Cashier** (check and payment), **System** (reminders). Top use cases: add/modify tables, search tables, place/update order, create/cancel reservation, check in, pay. Each role has a small separate set — each role class gets only its own methods.

Core entities

Class	Responsibility
`Restaurant`	Owns branches; closing it deactivates every employee account.
`Branch`	Owns menu, kitchen, seating charts, reservation service; employs staff.
`Menu`, `MenuSection`, `MenuItem`	Menu tree; **price only on `MenuItem`**.
`Table`, `TableSeat`	Capacity and seats; **current status** and **held time slots**.
`TableChart`	Named arrangement; refers to tables (a table can be in several charts).
`Reservation`	Customer, party size, time, table, status; `updatePeopleCount` refuses more than the table seats.
`ReservationService`	Best-fit search, atomic reserve, cancel, check-in, reminders, walk-ins.
`Order`, `Meal`, `MealItem`	One order per visit; one meal per seat; item × quantity; status machine + listeners.
`Kitchen`, `Chef`	Kitchen picks a chef by a pluggable rule; each chef has a FIFO queue.
`Employee` + roles	`Receptionist`, `Waiter`, `Manager`, `Chef`, `Cashier`.
`Check`	Subtotal, tax, total — computed from the order, never stored.
`Payment` (`CardPayment`, `CashPayment`, `CheckPayment`)	Three behaviours.

Class diagram

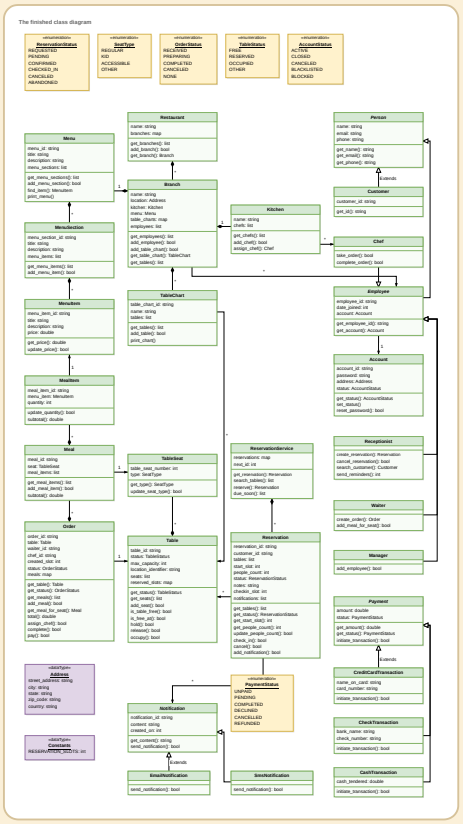
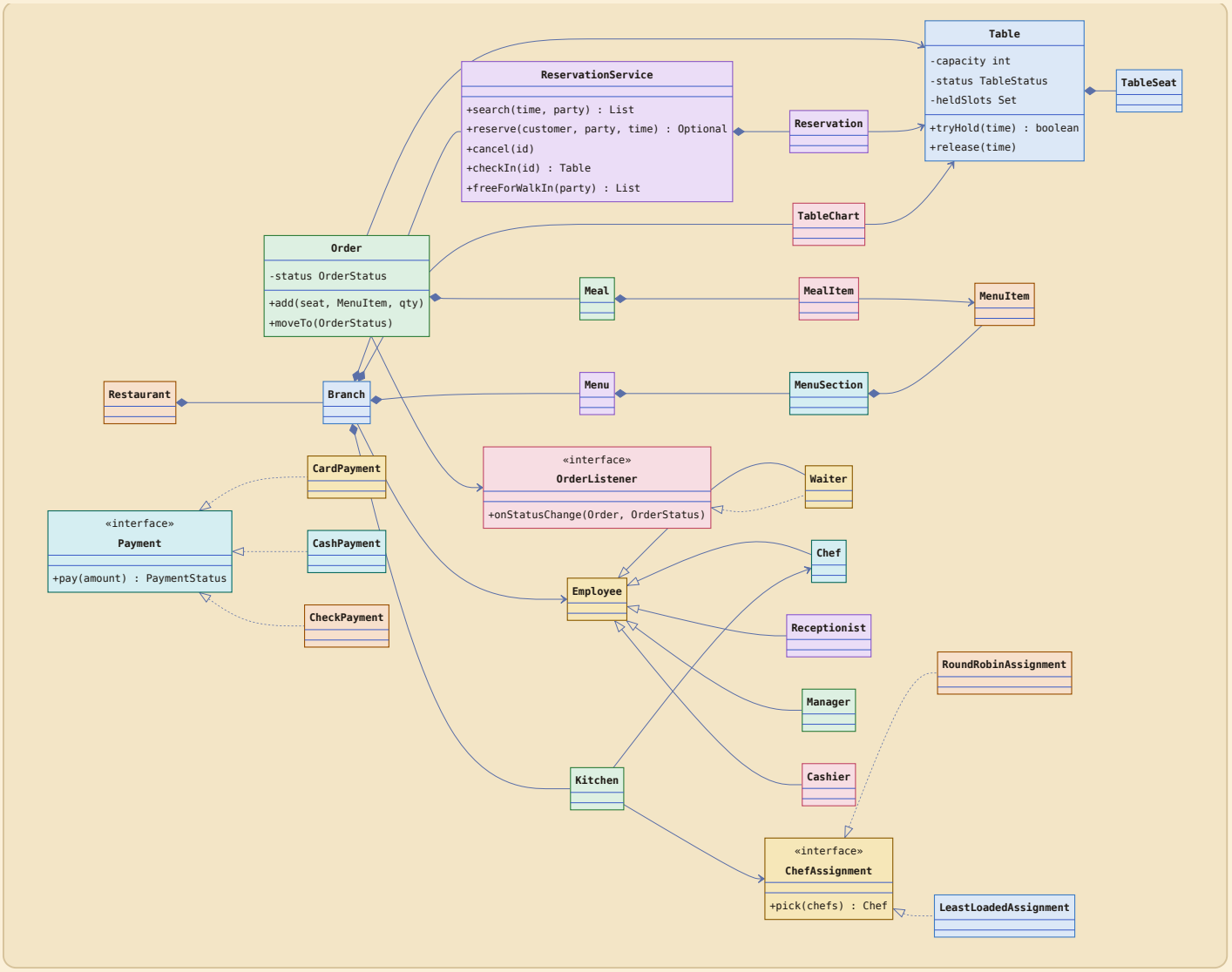


Figure — The finished class diagram of the restaurant management system (course diagram)

The branch is the hub: menu, kitchen and charts are composition; staff are associated (they can move branches). There is **no stored `Bill`** — the order already holds every priced line, so a stored total could disagree. Our `Check` is computed on demand.



Key flows

- **Place an order:** waiter opens an order for the table, adds a meal per seat, submits it to the kitchen.
- **Make a reservation:** search by time and party size → reserve and confirm, or "none available". **Cancel:** look up, cancel, release table, notify.
- **Order lifecycle:** `RECEIVED → PREPARING` (no more items) `→ READY` (waiter paged) `→ SERVED` (cashier settles); `RECEIVED → CANCELLED`.
- **Order to payment:** `Kitchen.submit` → `ChefAssignment.pick` → chef queue → `PREPARING` → `READY` → Observer pages the waiter → `Check.of(order)` → `payment.pay(total)` → `SERVED`, table `FREE`.

Design decisions & patterns used

Decision	Why	Alternative rejected
One meal per seat (`Map<seat, Meal>`)	Plate per guest; split bills are trivial.	Flat list of dishes.
Table keeps held slots as well as status	Status = now (walk-ins); slots = future bookings (a booking holds two hourly slots).	Status only.
Search on `ReservationService`, not `Table`	One table knows only itself (an old course version had an unworkable static `Table.search`).	Static search.
Best fit	Smallest table that fits keeps big tables for big parties.	First free.
Observer (`OrderListener`)	Waiter, screen, delivery app can all subscribe to "ready".	Chef calls waiter directly.
Strategy (`ChefAssignment`)	Course rotates chefs (fair, not load-aware); `LeastLoadedAssignment` is one class.	Hard-coded round-robin.
State machine in `Order.moveTo`	Illegal moves fail loudly.	Free `setStatus`.
`sealed` `Payment` with three classes	Card completes, cash may be short/gives change, cheque is pending.	One class + `if`s.
Check computed; `Customer` not an `Employee`	No second total; only staff have accounts.	Stored bill; one `Person` with a role flag.

Java implementation

Package `lld.ch16.restaurant`. Full runnable code: code/lld/ch16/restaurant/ (see the Full Edition).

```java
// Tables.java — Atomic check-then-hold of two hourly slots
synchronized boolean isFreeAt(LocalDateTime start) { return
slots(start).stream().noneMatch(heldSlots::contains); }

synchronized boolean tryHold(LocalDateTime start) {
    if (!isFreeAt(start)) return false;
    heldSlots.addAll(slots(start));
    return true;
}

// Reservations.java — Best fit first
List<Table> search(LocalDateTime time, int partySize) {
    return tables.stream().filter(t -> t.capacity() >= partySize && t.isFreeAt(time))
            .sorted(Comparator.comparingInt(Table::capacity)).toList();
}

Optional<Reservation> reserve(Customer c, int partySize, LocalDateTime time) {
    for (Table t : search(time, partySize)) {
        if (t.tryHold(time)) {     // another receptionist may have taken it since search(): try the next one
            var r = new Reservation("RES-" + seq.incrementAndGet(), c, partySize, time, t);
            reservations.put(r.id(), r);
            notifier.send(c, "Table " + t.id() + " booked for " + partySize + " at " + time);
            return Optional.of(r);
        }
    }
    return Optional.empty();
}

// Orders.java — state machine + Observer
void moveTo(OrderStatus next) {
    synchronized (this) {
        boolean ok = switch (status) {
            case RECEIVED -> next == OrderStatus.PREPARING || next == OrderStatus.CANCELLED;
            case PREPARING -> next == OrderStatus.READY;
            case READY -> next == OrderStatus.SERVED;
            case SERVED, CANCELLED -> false;
        };
        if (!ok) throw new IllegalStateException("order " + id + ": " + status + " -> " + next + " not
allowed");
        status = next;
    }
    listeners.forEach(l -> l.onStatusChange(this, next));   // notify outside the lock
}

// Kitchen.java — Strategy: which chef gets the next order
interface ChefAssignment { Chef pick(List<Chef> chefs); }

final class LeastLoadedAssignment implements ChefAssignment {
    public Chef pick(List<Chef> chefs) { return
chefs.stream().min(Comparator.comparingInt(Chef::load)).orElseThrow(); }
}

// Staff.java — Cashier
PaymentStatus settle(Check check, Payment payment) {
    // Check the order state BEFORE taking money: a refused state change after payment would leave the guest
charged.
    if (check.order().status() != OrderStatus.READY)
        throw new IllegalStateException("order " + check.order().id() + " is " + check.order().status() + ",
not READY");
    var status = payment.pay(check.total());
    if (status != PaymentStatus.DECLINED) {
        check.order().moveTo(OrderStatus.SERVED);
        check.order().table().setStatus(TableStatus.FREE);
    }
```

```
    return status;
}
```

Output excerpt:

```
2) Ravi searches tables for 4 at 19:00: [T2, T3, T4]
   Asha -> T2, Ben -> T3 (T2 is held 19:00-21:00)
   Asha's party grows to 6: rejected, table T2 seats 4, not 6
6) Kitchen works the queues
   rejected: order O1 is already PREPARING
    [Wendy's pager] order O1 for table T2 is READY
7) Check for O1 (split by seat)
   seat 1: [1x Tomato soup, 1x Margherita pizza] = 18.50
   subtotal 43.00, tax 3.44, total 46.44
   O2 total 25.92 paid by check: PENDING
```

Concurrency & edge cases

- **Two receptionists, one table:** search is advisory; `tryHold` checks and holds both slots atomically; on failure try the next best fit.
- **Notify outside the lock:** calling unknown listener code while locked risks deadlock and long waits.
- **Reminder spam:** `markReminded` sends once per reservation.
- **Party larger than any table:** "no single table fits" (combining tables is an extension). **No-shows:** `NO_SHOW` status; a job frees tables 15 min after start.
- **Party grows** beyond the table → rejected. **Pay before ready** → `settle` checks `READY` before taking money. Unknown id → clear `IllegalArgumentException`; bad seat number rejected.
- **Cash short** → `DECLINED`, order stays `READY`. **Cheque** → `PENDING`.

Extensions & follow-up questions

- **Large party, two tables?** Reservation holds `List<Table>`; lock tables in a fixed order (by id) to avoid deadlock.
- **Split bill?** Already easy — per-seat `Meal.subtotal()`. **Tips?** Extra `Check` lines. **Walk-in waitlist?** FIFO per party size.
- **Kitchen stock?** Ingredient `Inventory`, recipes, mark items "86'd". **Real kitchen threads?** Each chef takes from a `BlockingQueue<Order>`. **Shifts?** A `Shift` entity; only on-shift chefs are picked.

In the interview

30-second answer: "Branch owns a menu tree, kitchen, charts and reservation service. Tables keep status for walk-ins and held slots for bookings; the service searches best-fit and holds atomically. Orders have a meal per seat, a strict state machine and an observer for 'ready'. Chefs are assigned by a strategy. The check is computed; payment is card, cash or cheque strategies."

Plan: 0–5 clarify; 5–10 six actors; 10–20 diagram ("meal per seat", "status vs slots"); 20–35 code `tryHold`, `reserve`, `Order`, `Kitchen.submit`; 35–40 races, large parties, load balancing.

Self-check

1. **Q:** Why both status and held slots? **A:** Status = now (walk-ins); slots = future times (reservations).
2. **Q:** Why is search on the service? **A:** One table only knows itself.
3. **Q:** Which pattern pages the waiter, and why notify outside the lock? **A:** Observer; foreign code under a lock risks deadlock.
4. **Q:** Why is a cheque `PENDING`? **A:** It must clear at the bank.

TL;DR

- Menu → section → item; price only on the item.
- Table = status (now) + held slots (later); best-fit, atomic hold.

- Order → meal per seat; state machine; Observer for "ready"; Strategy for chefs.
- Check is computed; card/cash/cheque strategies.

Real-world example. POS systems like Toast send orders as tickets to kitchen display screens in order (FIFO + Observer); OpenTable models inventory by time slot and party size, like `heldSlots`.

Design an Airline Management System

Problem & clarifying questions

An **airline management system** schedules flights, reserves tickets, cancels flights, and manages staff and crew.

Question	Assumed answer
Search by?	Date, source and destination airport.
Trips?	An **itinerary** of several flights, several passengers (no accounts needed).
Seat classes/types?	Economy, economy plus, preferred economy, business, first; regular, accessible, emergency exit, extra legroom.
Who manages flights?	Only the admin adds aircraft/flights/schedules and cancels flights; everyone affected is notified.
Crew?	Pilots and crew assigned to flights, can view their schedule.
Payment / notifications?	Paid per itinerary; notify on booking changes and flight status changes.

Requirements

Functional: (1) search by date, source, destination; (2) reserve tickets, build itineraries; (3) view schedules and available seats; (4) several passengers per itinerary; (5) only the admin adds aircraft, flights, schedules and cancels flights (all notified); (6) customers cancel a reservation or itinerary; (7) assign pilots and crew; (8) take payment; (9) notify on booking create/modify and flight status change.

Non-functional: a seat on an instance is sold at most once; a multi-passenger booking gets all seats or none (**atomic**); fare rules change without touching booking code.

Use cases

Actors: **Admin**, **Front desk officer** (books for walk-ins), **Customer**, **Pilot/Crew** (view assignments), **System** (notifications). Top use cases: search; create/modify/view reservation; assign seats; pay; update schedule; assign crew. The officer's booking is the same operation as the customer's, triggered by another actor.

Core entities

The key is **four "seat-ish" and "flight-ish" ideas**:

Idea	Class	Example
The route	`Flight`	GA212 London → Tokyo, 14 h, base fare 500
The route on one date	`FlightInstance`	GA212 on 8 Oct 2026 at 10:00
The physical seat	`Seat`	12A, extra legroom, economy plus
That seat on that instance	`FlightSeat`	12A on GA212 8 Oct — booked by PNR147

Others: `Airport`; `Aircraft` (registration, model, seats); `WeeklySchedule` ("every Tuesday 10:00"); `Passenger` (a name on a ticket — **not** an account); `Customer` (books and pays); `Admin`, `FrontDeskOfficer` (admin-only methods need an `Admin` argument); `CrewMember`; `FlightReservation` (one leg: instance, passenger → seat,

amount, status); `Itinerary` (legs, connection rule, the unit paid); `FareStrategy`; `Airline` (Facade, keeps the index instance → reservations).

Class diagram

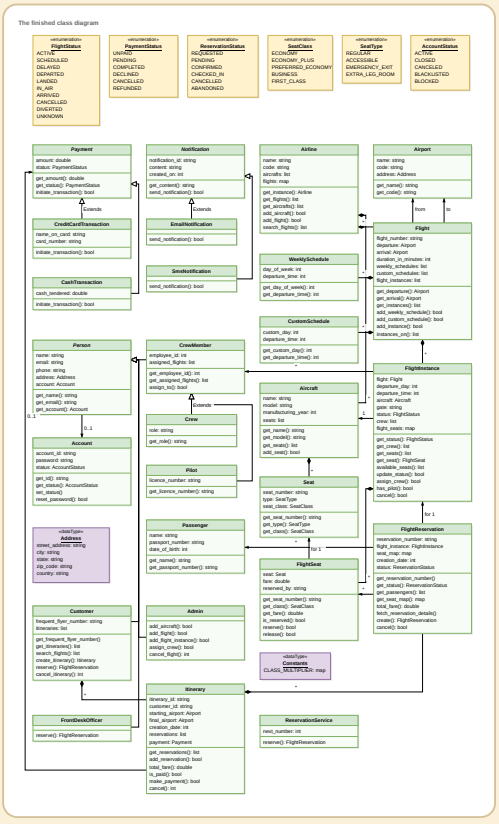
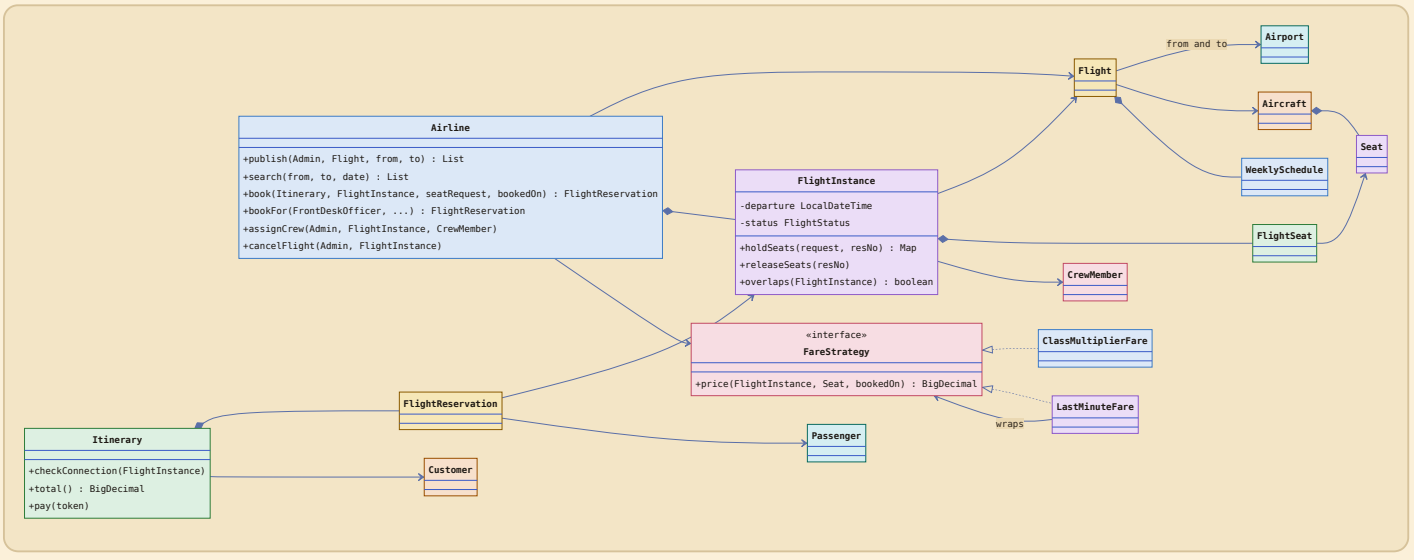


Figure — The finished class diagram of the airline management system (course diagram)

Ownership runs one way: schedules and instances belong to their flight, flight seats to their instance; airports, aircraft and crew are only referenced. `Passenger` sits outside the account tree — a passenger may never log in.



Key flows

- **Reserve:** search → choose seats and passengers → pay → confirmation. In code: `checkConnection` (same airport, ≥ 45 min after the previous leg lands) → `holdSeats` all-or-nothing → price each seat → add leg → index instance → reservation → notify.
- **Cancel reservation:** validate → cancel → release seats → notify.
- **Admin cancels a flight:** under the instance lock set `CANCELLED` and copy the index → cancel each reservation (releases seats) → notify each customer with the refund → notify every crew member.

Design decisions & patterns used

Decision	Why	Alternative rejected
Flight / FlightInstance / Seat / FlightSeat	12A is free Tuesday, taken Thursday: booking and fare belong to the seat *on an instance*.	`booked` flag on `Seat`.
`holdSeats` all-or-nothing, `synchronized` per instance	A family gets all seats or none; flights sell in parallel.	One-by-one holds; a global lock.
`Passenger` not an account	Customers book for others.	`Passenger extends Person`.
Itinerary is paid; **connection rule on `Itinerary`**	Course gap: no connecting-time check.	Pay per leg; no check.
Index instance → reservations	Course gap: cancelling notified crew but not passengers.	Scanning every itinerary.
Strategy + Decorator fares	Multipliers 1.0/1.3/1.5/2.5/4.0; `LastMinuteFare` +25% within 3 days.	Price on the seat.
Crew overlap check	No pilot on overlapping instances.	Trust the admin.
Facade `Airline`; admin ops take `Admin`	Compiler enforces "only the admin"; `bookFor` = `book` by another actor.	Rules in actors.
Course Singleton → construct and pass in	Testable.	`getInstance()`.

Java implementation

Package `lld.ch16.airline`. Full runnable code: code/lld/ch16/airline/ (see the Full Edition).

```java
// FlightInstance.java — All-or-nothing: a family of three either gets all three seats or none.
synchronized Map<Passenger, Seat> holdSeats(Map<Passenger, String> request, String reservationNumber) {
    if (status != FlightStatus.SCHEDULED && status != FlightStatus.DELAYED)
        throw new IllegalStateException(key() + " is " + status);
    for (var seatNo : request.values()) {
        var fs = seats.get(seatNo);
        if (fs == null) throw new IllegalArgumentException("no seat " + seatNo + " on " + key());
        if (!fs.isFree()) return Map.of();
    }
    if (request.values().stream().distinct().count() != request.size())
        throw new IllegalArgumentException("two passengers asked for the same seat");
    Map<Passenger, Seat> out = new LinkedHashMap<>();
    request.forEach((p, no) -> { var fs = seats.get(no); fs.assign(reservationNumber); out.put(p, fs.seat());
});
    return out;
}

// Booking.java — Decorator on any fare: +25% when booked less than 3 days before departure.
public BigDecimal price(FlightInstance fi, Seat seat, LocalDate bookedOn) {
    var p = inner.price(fi, seat, bookedOn);
    long daysBefore = ChronoUnit.DAYS.between(bookedOn, fi.departure().toLocalDate());
    return daysBefore < 3 ? p.multiply(new BigDecimal("1.25")).setScale(2, RoundingMode.HALF_UP) : p;
}

// Booking.java — Itinerary: the next leg must leave from where the last one landed, with time to change.
synchronized void checkConnection(FlightInstance next) {
    if (legs.isEmpty()) return;
    var last = legs.getLast().instance();
    if (!last.flight().to().equals(next.flight().from()))
        throw new IllegalArgumentException("leg must depart from " + last.flight().to().code());
    if (next.departure().isBefore(last.arrival().plus(MIN_CONNECTION)))
        throw new IllegalArgumentException("connection too short: " + next.key() + " leaves before " +
last.key() + " lands + 45 min");
}

// Airline.java
FlightReservation book(Itinerary it, FlightInstance fi, Map<Passenger, String> seatRequest, LocalDate
bookedOn) {
    it.checkConnection(fi);
    String resNo = "PNR" + seq.incrementAndGet();
    FlightReservation r;
    synchronized (fi) {        // hold the seats AND index the reservation as one step (see cancelFlight)
        var seatMap = fi.holdSeats(seatRequest, resNo);
        if (seatMap.isEmpty()) return null;
        var amount = seatMap.values().stream().map(s -> fares.price(fi, s, bookedOn)).reduce(BigDecimal.ZERO,
BigDecimal::add);
        r = new FlightReservation(resNo, fi, seatMap, amount);
        owner.put(r, it);
        byInstance.computeIfAbsent(fi, k -> new CopyOnWriteArrayList<>()).add(r);
    }
    it.addLeg(r);
    // ... notify customer, return r
}

void cancelFlight(Admin by, FlightInstance fi) {
    List<FlightReservation> affected;
    synchronized (fi) {        // same lock as the hold+index step in book(): no booking can slip past the
cancel
        fi.setStatus(FlightStatus.CANCELLED);
        affected = List.copyOf(byInstance.getOrDefault(fi, List.of()));
    }
    for (var r : affected) {
        if (r.cancel()) {
            // ... notify the itinerary's customer with the refund
        }
    }
```

```
        fi.crew().forEach(c -> notifier.send(c.id() + "@crew", "Stand down: " + fi.key() + " cancelled"));
    }
```

Output excerpt:

```
   itinerary total 3300.00 (2 x 400 economy + 2 x 1250 business)
   rejected: connection too short: GA212@2026-10-06 leaves before GA100@2026-10-07 lands + 45 min
   successful bookings of 10C: 1, free seats left: 3
   economy-plus fare 500 x 1.3 = 650.00, charged 812.50
   rejected: Capt. Kim already flies GA212@2026-10-08 at that time
    [email priya@example.com] GA212@2026-10-08 is cancelled. Reservation PNR103 refunded 2500.00. Itinerary
IT-101 now totals 800.00
```

20 threads race for seat 10C and exactly one wins; 650 + 25% last-minute = 812.50; the cancellation reaches every passenger through the index.

Concurrency & edge cases

- **Selling one seat twice:** `holdSeats` checks every seat, then assigns all, under the instance lock.
- **Booking while the admin cancels:** hold + index and set-`CANCELLED` + copy use the same `synchronized (fi)` — every reservation is either in the copy (cancelled and notified) or refused.
- **Double payment:** `Itinerary.pay` is idempotent. **Idempotent cancel:** `FlightReservation.cancel()` returns `false` if already cancelled — no double refund.
- **Duplicate seat in one request**, **booking a cancelled/departed flight**, **crew on overlapping instances** — all rejected.
- **Time zones:** real systems use `ZonedDateTime`/`Instant` per airport. **Holds during payment:** 10–15 min `heldUntil` on `FlightSeat` plus a sweeper.

Extensions & follow-up questions

- **Overbooking?** Per-class `overbookLimit`; assign physical seats at check-in.
- **Check-in/boarding passes, baggage, change fees, codeshares (`MarketingFlight`)** — small new entities.
- **Delays?** `DELAYED` + new departure; notify through the index; re-check connections.
- **Scale?** One row per seat with `UPDATE … WHERE reservation IS NULL` or optimistic locking; cache search.

In the interview

30-second answer: "`Flight` is the route, `FlightInstance` one departure; `Seat` is the physical chair, `FlightSeat` that chair on that departure, where booking and fare live. Seats are held all-or-nothing per instance. An itinerary groups legs, checks connections and is paid. Fares are a strategy with a last-minute decorator. An instance → reservations index lets a cancelled flight notify every passenger."

Plan: 0–5 clarify; 5–10 actors; 10–20 the four-way distinction and diagram; 20–35 code `holdSeats`, `book`, `checkConnection`, `cancelFlight`; 35–40 holds, overbooking, time zones. Expected follow-up: "How do passengers learn of a cancellation?" — "Through the instance → reservations index; without it you only reach the crew."

Self-check

1. **Q:** Where does "booked" live? **A:** On `FlightSeat` — per instance, not per aircraft.
2. **Q:** All seats or none? **A:** `holdSeats` checks all, then assigns all, in one synchronized block.
3. **Q:** `LastMinuteFare`? **A:** +25% within 3 days; a Decorator over any `FareStrategy`.
4. **Q:** Two course gaps closed? **A:** Passenger notification on cancel (index) and connection-time check (`Itinerary.checkConnection`).

TL;DR

- Flight ≠ FlightInstance; Seat ≠ FlightSeat.
- Hold seats all-or-nothing under the instance's lock.
- Itinerary = legs + connection rule + payment.

- Fares: Strategy + Decorator; instance → reservations index for cancellations.

Real-world example. Airlines separate a flight number from a leg on a date and control inventory per leg and booking class; a PNR groups passengers and segments — like `FlightReservation` / `Itinerary`. On cancellation they find every PNR on the leg, which is why the index matters.

Design a Course Registration System

Problem & clarifying questions

A **course registration system** lets students sign up for classes each term — a harsh concurrency problem: at 09:00 thousands click "register" for a few popular sections.

Terms: a **course** is a catalogue entry (CS201, 4 credits); a **section** is one offering with instructor, time and capacity (CS201-A, Mon/Wed 10:00–11:30, 30 seats) — students register for **sections** (like Flight vs FlightInstance); a **prerequisite** must be completed first; a **waitlist** is a FIFO queue for a full section; a **registration window** is when a student may register (seniors often first).

Question	Assumed answer
Courses or sections?	Sections.
Prerequisites?	Must be completed.
Capacity?	Per section; when full, a bounded waitlist.
Promotion?	Automatic on drop, FIFO, after re-checking the timetable.
Limits?	Credit limit; no time clashes; no two sections of one course.
Windows?	Open by class year; close together.
Concurrency?	Never exceed capacity; no double registration from two tabs.

Requirements

Functional: (1) register for sections and view them; (2) course has code, name, credits; section has instructor, time slot, limit; (3) search by code or name; (4) never exceed capacity, else waitlist (if room); (5) enforce prerequisites, credit limit, time conflicts, one section per course, window; (6) on drop, promote the waitlist head and notify.

Non-functional: safe concurrent registration; rules easy to add/remove ("majors only"); extensible (fees, holds, cross-listing).

Use cases

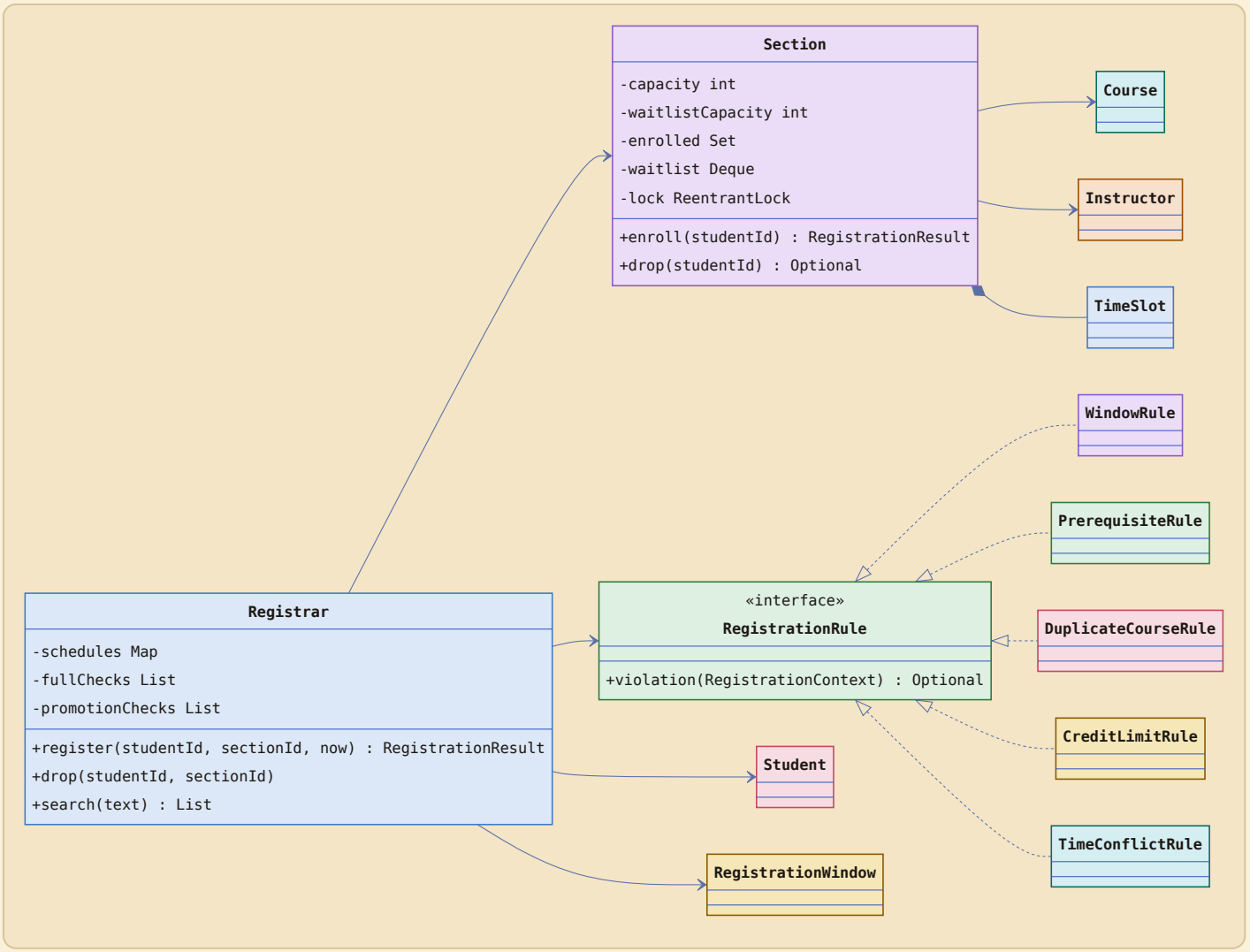
Student — search, register, drop, view schedule, join waitlist. **Registrar/Admin** — create courses/sections, set capacity and windows. **Instructor** — view roster. **System** — enforce rules, promote from waitlist, notify (a drop triggers promotion).

Core entities

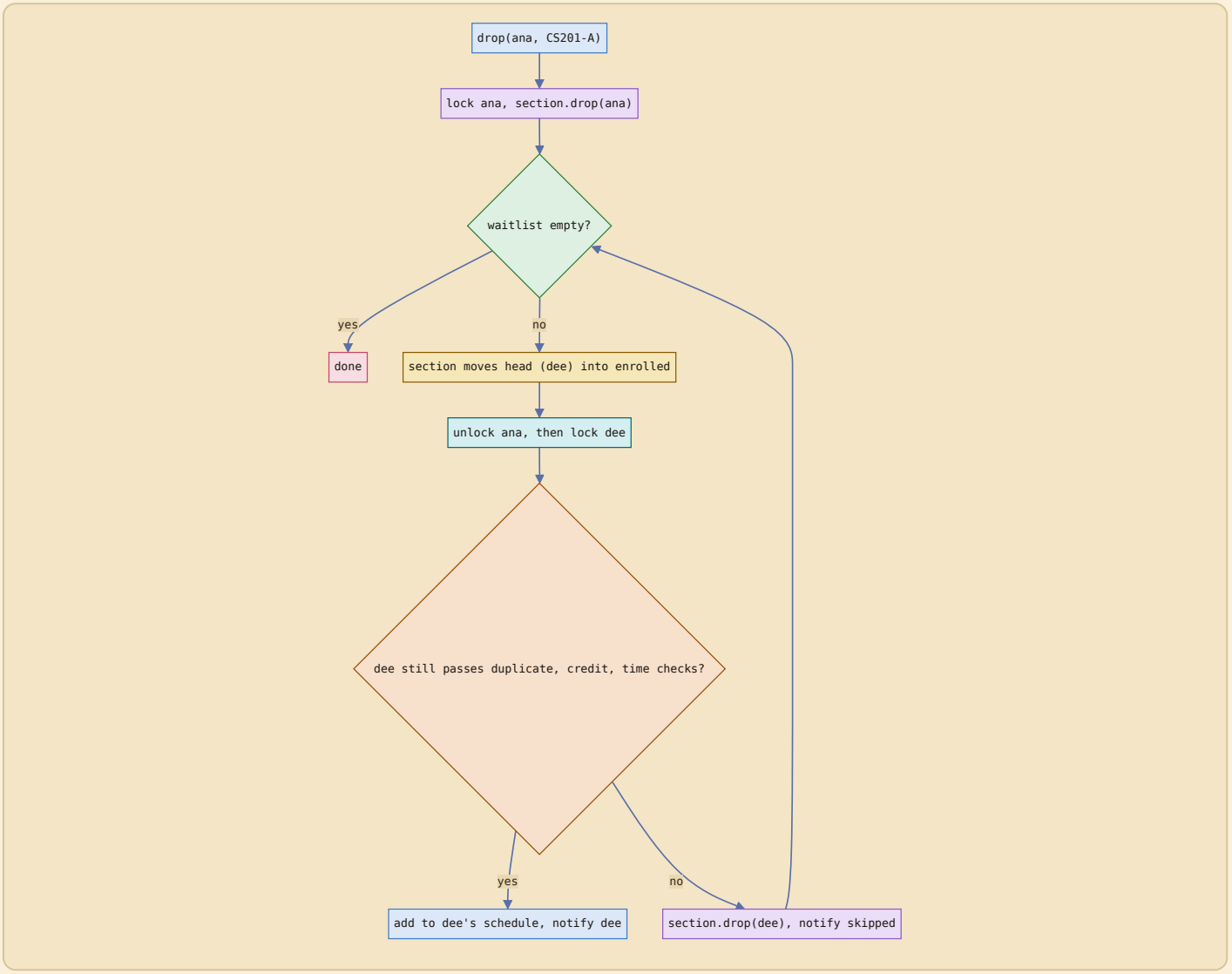
Class	Responsibility
`Course`	Code, title, credits, prerequisite codes.
`Section`	Instructor, slot, capacity, enrolled set, waitlist. **The only place a seat is taken**, under its own lock.
`TimeSlot`	Days + start/end; knows overlap.
`Instructor`, `Student`	Who teaches; year, completed courses, credit limit.
`RegistrationWindow`	Opening per class year, common close.
`RegistrationRule` (+ `WindowRule`, `PrerequisiteRule`, `DuplicateCourseRule`, `CreditLimitRule`, `TimeConflictRule`)	One rule each; returns a violation reason.
`RegistrationContext`	Everything a rule may look at.
`RegistrationResult`, `Outcome`	`ENROLLED` / `WAITLISTED` / `REJECTED` + message.
`Registrar`	Runs the rule chain, takes the seat, drops and promotes, keeps schedules, defines the lock order.

Class diagram

There is no course lesson for this problem; it comes from the *awesome-low-level-design* problem set (register, view, search, capacity, concurrency, extensibility) — all covered, with instructor and capacity on `Section`.



Key flows



- **Register:** lock the student's schedule → run window, prerequisites, duplicate, credits, time conflict → first failure = `REJECTED` + reason; else `section.enroll` (section lock) → `ENROLLED` (add to schedule), `WAITLISTED at position n`, or `REJECTED` (both full).
- **Student status in a section:** `ENROLLED`, `WAITLISTED` (→ `ENROLLED` on promotion, or `SKIPPED` if the re-check fails), `REJECTED`; drop leaves.
- **Drop and promote:** as in the flowchart above.

Design decisions & patterns used

Decision	Why	Alternative rejected
Course vs Section	Capacity, time, instructor belong to an offering.	Capacity on `Course`.
Seat taken only in `Section.enroll`, `ReentrantLock` per section	Atomic check-capacity-then-add; sections don't block each other.	`synchronized` registrar (the whole university in one line).
Per-student lock around rules + enroll	Credit/time checks read the schedule; two tabs would both pass without it.	Section lock only.
Lock order: student, then section	Deadlock needs opposite orders; promotion takes the next student's lock only *after* releasing the dropper's.	Promote while holding the dropper's lock.
Rules as objects (Specification / Chain of Responsibility)	A new rule = new class + list entry (Open/Closed); promotion reuses a subset.	One long `if` method.
Bounded FIFO `Deque` waitlist	Fair, bounded.	Unbounded list.
Re-check on promotion	Timetable may have changed while waiting.	Promote blindly.
`now` passed in	Testable windows.	`LocalDateTime.now()` in rules.
GitHub problem's **Singleton** → one `Registrar` passed in	Testable.	`getInstance()`.

Java implementation

Package `lld.ch16.courseregistration`. Full runnable code: code/lld/ch16/courseregistration/ (see the Full Edition).

```
// Section.java — The only place a seat is taken: check-and-add in one critical section.
RegistrationResult enroll(String studentId) {
    lock.lock();
    try {
        if (enrolled.contains(studentId)) return RegistrationResult.rejected("already enrolled in " + id);
        if (waitlist.contains(studentId)) return RegistrationResult.rejected("already on the waitlist of " +
id);
        if (enrolled.size() < capacity) {
            enrolled.add(studentId);
            return new RegistrationResult(Outcome.ENROLLED, "enrolled in " + id + " (" + enrolled.size() + "/"
+ capacity + ")");
        }
        if (waitlist.size() < waitlistCapacity) {
            waitlist.addLast(studentId);
            return new RegistrationResult(Outcome.WAITLISTED, "waitlisted for " + id + " at position " +
waitlist.size());
        }
        return RegistrationResult.rejected(id + " and its waitlist are full");
    } finally {
        lock.unlock();
    }
}

// Rules.java
interface RegistrationRule { Optional<String> violation(RegistrationContext c); }

final class CreditLimitRule implements RegistrationRule {
    public Optional<String> violation(RegistrationContext c) {
        int current = c.currentSchedule().stream().mapToInt(s -> s.course().credits()).sum();
        int after = current + c.section().course().credits();
        return after <= c.student().maxCredits() ? Optional.empty()
                : Optional.of("credit limit " + c.student().maxCredits() + " exceeded (" + after + ")");
    }
}

// Registrar.java — lock order is ALWAYS student-then-section
RegistrationResult register(String studentId, String sectionId, LocalDateTime now) {
    // ... look up student, section (reject if unknown)
    var schedule = schedules.get(studentId);
    synchronized (schedule) {                                   // 1st lock: the student
        var ctx = new RegistrationContext(student, section, Set.copyOf(schedule), now, window);
        var v = firstViolation(fullChecks, ctx);
        if (v.isPresent()) return RegistrationResult.rejected(v.get());
        var result = section.enroll(studentId);                 // 2nd lock: the section
        if (result.outcome() == Outcome.ENROLLED) schedule.add(section);
        return result;
    }
}

void drop(String studentId, String sectionId) {
    // ...
    synchronized (schedule) {
        promoted = section.drop(studentId);
        schedule.remove(section);
    }
    notifier.accept(studentId + " dropped " + sectionId);
    // Finish promotions OUTSIDE the dropper's lock, one student lock at a time (no nested student locks).
    while (promoted.isPresent()) promoted = finishPromotion(promoted.get(), section);
}
```

`finishPromotion` locks the promoted student, re-runs `promotionChecks` (duplicate, credit, time), and either adds the section to their schedule or drops them and returns the next promotion.

Output excerpt:

```
   ana  MATH201-A         -> REJECTED: time conflict with CS201-A [MONDAY, WEDNESDAY] 10:00-11:30
   ana  CS301-A           -> REJECTED: missing prerequisites [CS201, MATH201]
   fay  CS201-A           -> REJECTED: CS201-A and its waitlist are full
   ben  MATH201-A         -> REJECTED: credit limit 4 exceeded (7)
    [notify] dee promoted from waitlist into CS201-A
5) 50 first-years grab 30 seats (+10 waitlist) in CS101-A at 09:00 sharp
   ENROLLED=30 WAITLISTED=10 REJECTED=10, roster size 30
6) One student double-clicks: two threads, same request
   {ENROLLED=1, REJECTED=1}, gus schedule [CS201-B]
```

The 09:00 rush counts are exact every run; the double-click succeeds exactly once.

Concurrency & edge cases

- **Overselling** — section lock around "size < capacity → add". **Double-click / two tabs** — per-student lock; the second request hits `DuplicateCourseRule` or "already enrolled".
- **Deadlock** — fixed order student → section; never two student locks at once.
- **Waitlisted in one section, enrolled in another of the same course** — caught by the promotion re-check, which skips to the next student.
- **Window closes while waitlisted** — promotion doesn't re-check the window; add `WindowRule` to `promotionChecks` if that is the policy.
- Dropping a section you're not in → `IllegalArgumentException`; windows are half-open (`opens ≤ now < closes`).
- **Beyond one JVM:** `UPDATE section SET enrolled = enrolled + 1 WHERE id = ? AND enrolled < capacity` (atomic conditional update) or optimistic locking; a queue in front of hot sections.

Extensions & follow-up questions

- **Co-requisites / "prereq OR permission"?** Composite expression tree (`And`, `Or`, `CourseCompleted`, `Permission`) in `PrerequisiteRule`.
- **Holds?** A `HoldRule`. **Priority waitlist?** `PriorityQueue` by (year, time joined). **Major seat buckets?** Split capacity in `Section.enroll`.
- **Lecture + lab?** Lock both sections by id order, check both, enrol both. **Notifications/fees?** Swap the `Consumer<String>` notifier; a `Bill` per student per term.

In the interview

30-second answer: "Students register for sections, not courses. A section owns capacity and a bounded FIFO waitlist and takes seats only inside its own lock. The registrar runs a chain of rule objects under the student's lock, then asks the section for a seat. Lock order is always student then section, so no deadlocks. On drop the waitlist head is promoted after a re-check."

Plan: 0–5 clarify; 5–12 entities and diagram; 12–30 code `Section.enroll/drop`, the rule interface and two rules, `register`; 30–40 concurrency, lock order, DB conditional update.

Self-check

1. **Q:** Which two locks does `register` take, in what order? **A:** Student's schedule, then the section — always.
2. **Q:** Why not `synchronized` on the whole registrar? **A:** Correct but serialises the whole university.
3. **Q:** Why re-check on promotion? **A:** The timetable may have changed while waiting.
4. **Q:** Capacity with many servers? **A:** Atomic conditional update (`… WHERE enrolled < capacity`) or optimistic locking.

TL;DR

- Course (catalogue) vs Section (capacity, slot, waitlist).
- Rules are small objects run as a chain.
- Seats taken only in `Section.enroll`; student lock first, section lock second.

- Drop → promote head → re-check → notify. At scale: DB conditional update.

Real-world example. Student-information systems such as Ellucian Banner run priority windows by class standing, check prerequisites and clashes, and keep section waitlists; registration-day slowdowns are the 09:00 "thundering herd" this design handles.

Chapter summary

Problem	Key insight	Atomic claim	Patterns
Car rental	Availability is about **dates**, not status	`Vehicle.tryBook(range)`	Template Method, Strategy + Decorator, state machine
Restaurant	**Meal per seat**; table = status + slots	`Table.tryHold(time)`	Observer, Strategy (chefs, payment), state machine
Airline	**Flight ≠ FlightInstance, Seat ≠ FlightSeat**	`FlightInstance.holdSeats(map)` (all or nothing)	Strategy + Decorator, Facade, cancellation index
Course registration	**Course ≠ Section**; rules as objects	`Section.enroll` inside a student lock	Specification / Chain of rules, lock ordering

Find the calendar, find the atomic check-then-claim, and put the rules on the object that owns the data.

Chapter 17 — Case Studies: Games & Rules — Study Edition

> **Where this fits.** Chapters 14–16 modelled the real world. In games the "world" is a set of exact **rules** — every rule must live *somewhere*, and the interviewer will ask where. That is why games are a favourite interview topic.

What games test in an LLD interview

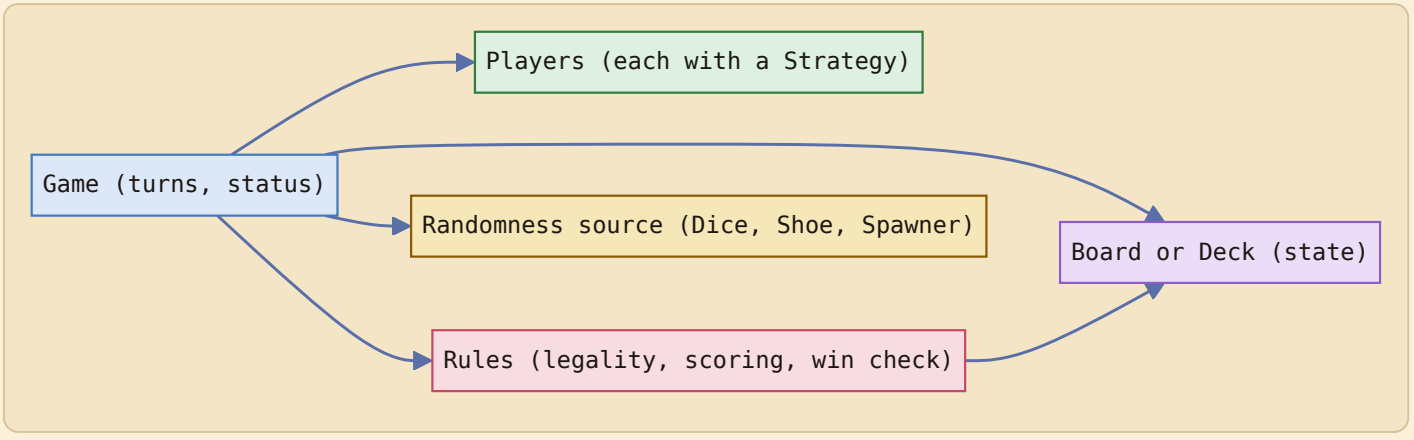
Skill	What it means	Where you see it
Rule encapsulation	Each rule lives in one obvious class.	`BlackjackScoring`; chess `Rules`; 2048 `LineMerger`.
Turn management	Whose turn, what may they do, what happens at the end of a turn.	Turn index (tic-tac-toe), player queue (snake and ladder), `turn` colour (chess), rounds (blackjack).
Polymorphic pieces	One method call, many behaviours (*polymorphism*).	Chess `Piece.targets(...)`.
Pluggable strategies	Varying behaviour behind an interface (**Strategy**, Ch. 10).	`PlayerStrategy`, `DealerPolicy`, `MoveStrategy`, `Dice`, `OverflowRule`, `WinStrategy`, `TileSpawner`.
Deterministic testing	Randomness is *injected*: `new Random(seed)` or a scripted source (stacked shoe, loaded dice).	Every `Main` prints the same output every run.
State and status	Games move through states; illegal actions in the wrong state are rejected.	`GameStatus`, `GameState` enums.

> **Key idea: seeded randomness.** `java.util.Random` is *pseudo-random*: the same **seed** always gives the same sequence, and `Collections.shuffle(list, rnd)` is deterministic for a given `Random`. Rule: **never call `new Random()` or `Math.random()` inside the game — accept a `Random` (or a `Dice`/`Spawner`) from outside.** Production passes an unseeded (or `SecureRandom`) source; tests pass a seed. This is plain Dependency Injection (Ch. 5).

Problems: 17.1 Blackjack and a deck of cards · 17.2 Chess · 17.3 Tic-Tac-Toe · 17.4 Snake and Ladder · 17.5 2048.

How to run: `cd chapters/17-case-games/code && javac --release 21 -d /tmp/lld_17 $(find . -name '*.java') && java -cp /tmp/lld_17 lld.ch17.blackjack.Main` (or `chess`, `tictactoe`, `snakeladder`, `game2048`). All randomness is seeded or scripted, so output is exact.

A reusable skeleton for any board/card game — name these five parts early:



Part	Question it answers	Typical class names
State	What does the table look like now?	`Board`, `Deck`, `Shoe`, `Hand`, `Cell`
Actors	Who acts, how do they decide?	`Player` + `PlayerStrategy` / `MoveStrategy`
Rules	Allowed? What does it do? Who won?	`Rules`, `DealerPolicy`, `OverflowRule`, `WinStrategy`
Controller	Whose turn? Which state?	`Game`, `GameStatus`
Randomness	Where does chance come from?	`Dice`, `Shoe`, `TileSpawner` — always injected

---

Design Blackjack and a Deck of Cards

Problem & clarifying questions

Blackjack: every player plays only against the **dealer** (the casino employee playing for the house); everyone tries to get close to **21** without going over (**busting** loses at once). Often asked as "design a deck of cards, then blackjack" — the first half checks the deck stays **generic**, the second checks rule placement.

Question	Assumed answer
Decks?	A **shoe** of several 52-card decks (casinos use 6–8); one deck = shoe of 1.
Players?	One or more, each against the dealer.
Options?	Hit, stand, double down, split (once), insurance. No surrender/side bets.
Dealer rule?	Hit ≤ 16, stand ≥ 17; "hit soft 17" pluggable.
Payouts?	Win 1:1, blackjack 3:2, push returns the bet, insurance 2:1.
Money?	Whole **chips** (`int`), even ante. Never `double`.
UI, accounts?	Out of scope (the course's `Account`/`Person` would attach to `Player`).

Requirements

Background rules: 13 ranks × 4 suits; two cards each; the dealer shows the **up card** and hides the **hole card**. J/Q/K = 10, **ace = 1 or 11**. **Soft** total = an ace counted as 11; **hard** = all aces as 1 (A+5 = soft 16 / hard 6). **Blackjack** (natural) = two cards totalling 21.

Functional:

1. Place an ante to play.
2. Deal two cards each (dealer: one up, one hole).
3. Dealer shows an **ace** → offer **insurance** (half the ante, pays **2:1** on dealer blackjack); the dealer then **peeks**; dealer blackjack ends the round (insurance wins, main bets lose, player blackjack pushes).
4. Below 21 the player chooses **hit**, **stand**, **double down** (double the bet, exactly one card, stop) or **split** (equal ranks only, second bet = ante, two hands, one card each, no re-split).
5. Over 21 → immediate loss.
6. Dealer plays by the fixed rule.
7. Dealer bust → every live hand wins; else higher total wins; equal = **push**. Blackjack pays **3:2**, other wins 1:1.
8. **Even money**: blackjack vs a dealer ace may take 1:1 at once (= insurance + blackjack; explained, not built).

Non-functional: generic `Card`/`Deck`; deterministic tests; pluggable dealer and player rules; whole-chip `int` money — deliberately not `BigDecimal`: an even ante keeps every payout whole (3:2 on 10 = 15). If fractions appear, use `BigDecimal` + `RoundingMode`.

Use cases

Player — bets; accepts/declines insurance, split, double, even money; hits or stands (log in / join game belong to a lobby service). **Dealer** — deals, plays by the fixed rule, collects and pays. Use cases: create hands, place bet, player plays, dealer plays, insurance / even money / examine hole card (ace up only), split, resolution; dealing and payout are `<<include>>`d in every round.

Core entities

Class	Responsibility
`Suit`, `Rank`	Fixed values; `Rank` stores *face value* 1–13 (ace 1, K 13).
`Card` (record)	Rank + suit; knows **nothing** about blackjack.
`Deck<T>`	Generic shuffleable stack; `standard52()` factory.
`Shoe`	Several decks; deals; reshuffles at the **cut card**; `Shoe.stacked(...)` for tests.
`BlackjackScoring`	Blackjack value of a rank (K = 10, ace = 1 hard) — kept off `Card`.
`Hand`	Cards + **its own bet**; hard/soft/best total, bust, blackjack, split, double.
`Action`, `PlayerStrategy`	HIT/STAND/DOUBLE/SPLIT; `BasicStrategy`, `MimicDealerStrategy` (a UI human would be a third).
`DealerPolicy`	`standOnAll17()`, `hitSoft17()`.
`Player`	Name, chip balance, strategy.
The dealer	No class here: a `Hand` in `BlackjackGame` + the `DealerPolicy` (the course has `Dealer` beside `Player` under `BasePlayer`).
`BlackjackGame`	Runs a round: bets, deal, insurance + peek, player turns, dealer turn, settlement.

Class diagram

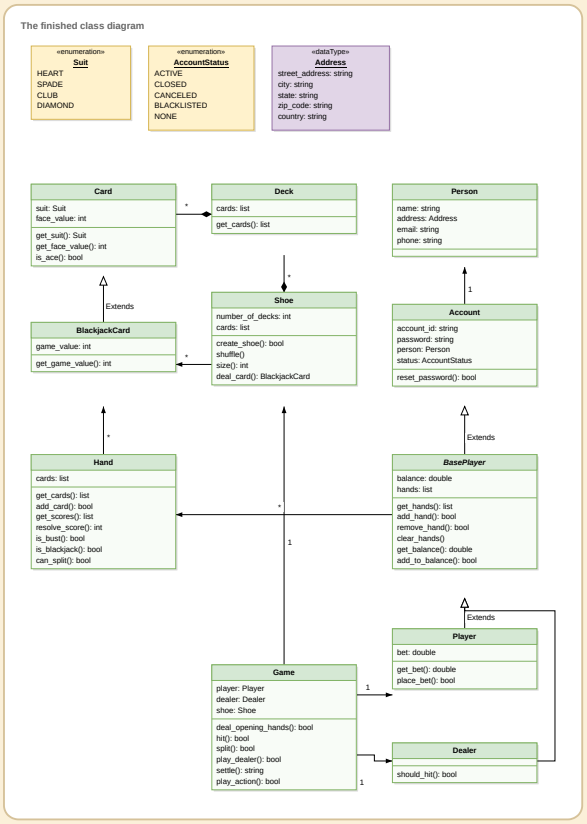
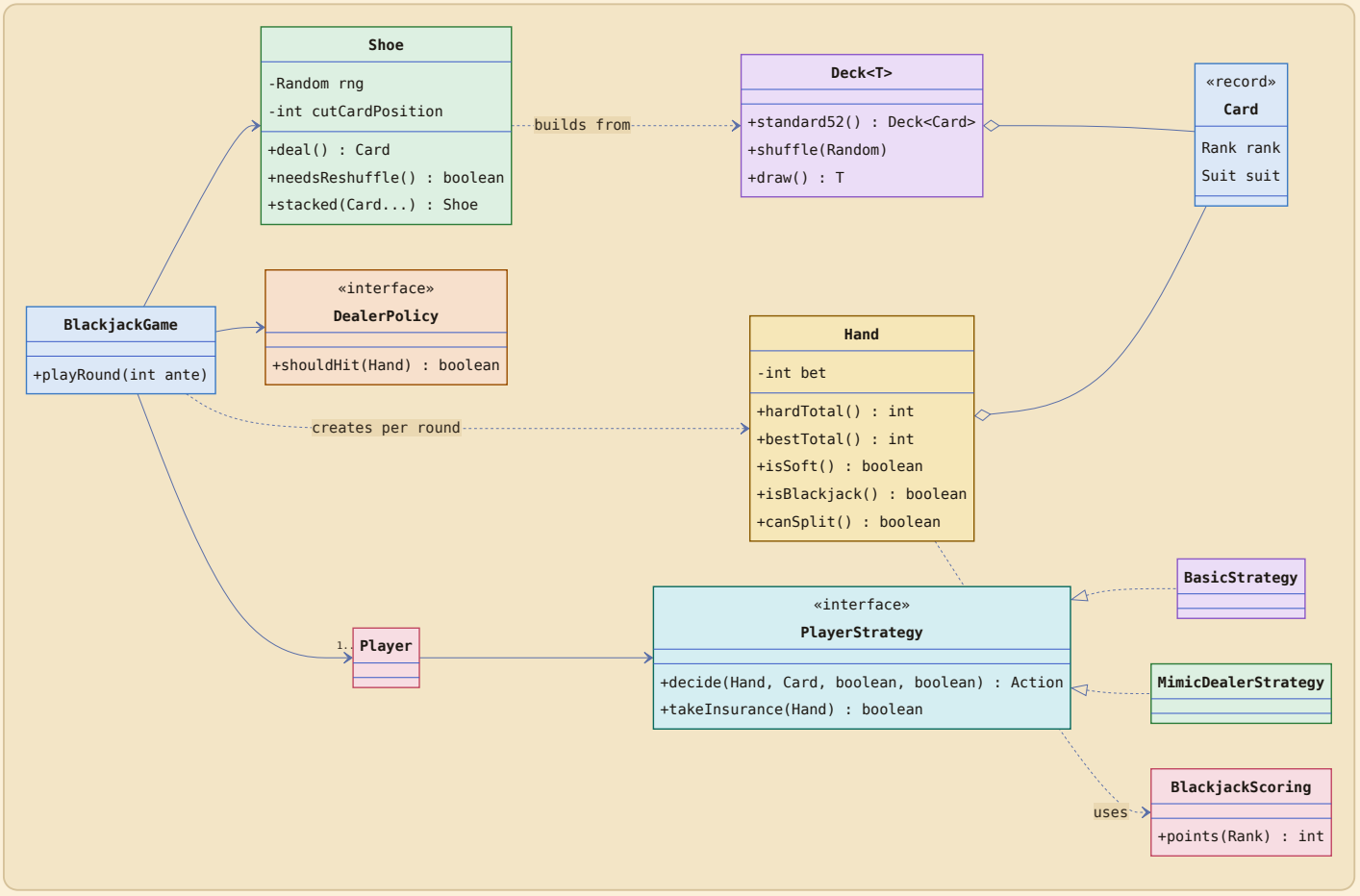


Figure — Course class diagram: Card, BlackjackCard, Deck, Shoe, Hand, BasePlayer, Player, Dealer, Game, Account (course diagram)

In the course, `BlackjackCard extends Card` adds the *game value* (K: face 13, worth 10); a `Shoe` is composed of `Deck`s; `Player` and `Dealer` share `BasePlayer` (balance, `*` hands because of splits). We keep the core idea — **the game value must not live on the generic card** — using a scoring function, and make player and dealer rules strategies:



Key flows

One round: debit antes → deal two each → if up card is an ace, offer insurance → if up card is ace or ten and dealer has blackjack, settle and stop → else each hand plays (`decide` until stand, bust, 21 or double) → dealer draws while `shouldHit` (skipped if all hands bust) → settle. States: `Betting → Dealing → (Insurance) → Peek → PlayerTurns → DealerTurn → Settled`.

Scoring a hand (the ace problem). At most one ace can ever count as 11 (two would be 22). So: (1) sum all cards with aces as 1 = **hard total**; (2) if there is an ace and `hard + 10 ≤ 21`, the hand is **soft** and best = `hard + 10`; (3) else best = hard. O(n), no search, and the soft flag comes free (needed for "hit soft 17").

Design decisions & patterns used

Decision	Why	Alternative rejected
`Card` is a game-agnostic **record**.	Poker, bridge, solitaire reuse it; K = 10 is a rule of this game.	`getValue()` returning 10 for K on `Card`.
Value in `BlackjackScoring`, not a `BlackjackCard` subclass.	Records can't be extended; one card type in the shoe.	The course's subclass — also valid; say both.
Generic `Deck<T>`.	Uno/tarot reuse shuffle/draw.	Deck hard-wired to `Card`.
Strategy for `PlayerStrategy`, `DealerPolicy`.	UI, bots, scripts plug in; S17 vs H17 is one line (Open/Closed).	`if (isHuman)`, `if (casino == X)`.
Factory methods (`standard52`, `stacked`, `standOnAll17`); injected `Random`.	Named constructors; exact tests.	Flag constructors; hidden randomness.
`hardTotal`/`bestTotal`/`isSoft`.	The course returns every possible total; hard + soft flag carries the same information.	List of totals to filter.
Hand stores its **own bet**.	Split and double change bets per hand.	One bet per player.
Game **validates** the strategy's choice.	A buggy client can't double without money.	Trusting the client.

Honest weakness. `playRound` owns the whole sequence *and* the payouts; each new option adds a branch. A larger design uses a **State** machine (one class per round state, like the Ch. 14 vending machine) and a `PayoutTable`. Say it out loud.

Java implementation

Package `lld.ch17.blackjack`. Full runnable code: code/lld/ch17/blackjack/ (see the Full Edition).

```java
/** A game-agnostic playing card: just a rank and a suit. Immutable, so it is safe to share. */
public record Card(Rank rank, Suit suit) {
    @Override public String toString() { return rank.symbol() + suit.letter(); }
}

// BlackjackScoring.java — Points with an ace counted as 1 (the "hard" value).
public static int points(Rank rank) {
    return switch (rank) {
        case ACE -> 1;
        case JACK, QUEEN, KING -> 10;
        default -> rank.faceValue();
    };
}

// Hand.java
public int hardTotal() {
    int sum = 0;
    for (Card c : cards) sum += BlackjackScoring.points(c.rank());
    return sum;
}
public boolean isSoft() { return hasAce() && hardTotal() + 10 <= 21; }
public int bestTotal() { return isSoft() ? hardTotal() + 10 : hardTotal(); }
public boolean isBust() { return bestTotal() > 21; }
public boolean isBlackjack() { return !fromSplit && cards.size() == 2 && bestTotal() == 21; }

// DealerPolicy.java
@FunctionalInterface
public interface DealerPolicy {
    boolean shouldHit(Hand dealerHand);

    /** Classic rule: draw on 16 or less, stand on every 17 ("S17"). */
    static DealerPolicy standOnAll17() { return h -> h.bestTotal() <= 16; }

    /** Variant: also draw on a soft 17 such as A+6 ("H17"), which favours the house. */
    static DealerPolicy hitSoft17() { return h -> h.bestTotal() <= 16 || (h.bestTotal() == 17 && h.isSoft()); }
}
}

// BlackjackGame.java — validate the strategy's choice, then settle
Action a = p.strategy().decide(hand, up, canDouble, canSplit);
if ((a == Action.DOUBLE && !canDouble) || (a == Action.SPLIT && !canSplit)) {
    throw new IllegalStateException(label + " chose a move that is not allowed: " + a);
}
// ...
private void settle(Player p, Hand h, Hand dealer) {
    String who = "  " + p.name() + " " + h + " vs dealer " + dealer.bestTotal() + ": ";
    if (h.isBust()) { log.accept(who + "LOSE " + h.bet()); return; }
    if (h.isBlackjack()) {                                  // dealer blackjack was handled by the peek
        p.credit(h.bet() + h.bet() * 3 / 2);
        log.accept(who + "BLACKJACK pays 3:2 (+" + h.bet() * 3 / 2 + ")");
        return;
    }
    int mine = h.bestTotal(), theirs = dealer.bestTotal();
    if (dealer.isBust() || mine > theirs) { p.credit(h.bet() * 2); log.accept(who + "WIN +" + h.bet()); }
    else if (mine == theirs) { p.credit(h.bet()); log.accept(who + "PUSH"); }
    else log.accept(who + "LOSE " + h.bet());
}
```

Output excerpt:

```
  [AS 6H] = soft 17   (hard total 7)
  [AS 6H 10C] = 17   (hard total 17)
  [AS AH 9C] = soft 21   (hard total 11)
== 2. Stacked shoe: split 8s, then double both hands ==
  Alice splits -> [8H 3C] = 11 and [8D 2S] = 10
  Alice hand 1 doubles (bet now 20): [8H 3C 9H] = 20
  Dealer reveals and plays: [6S 10C 7S] = bust 23
  Balances: Alice=140
== 3. Stacked shoe: dealer shows an ace, Bob buys insurance ==
  Dealer peeks: [AS KD] = BLACKJACK -> round over
  Bob insurance pays 2:1 (+10)
  Bob: LOSE 10
  Balances: Bob=100
```

A stacked shoe forces exact scenarios; Bob's insurance hedges his loss to break even.

Concurrency & edge cases

- **One table = one thread** (or actor); tables share only the wallet, which needs atomic debit/credit — debit *before* dealing.
- **Odd antes** (3:2 on 5 = 7.5) → we require even antes; casinos round down or use smaller chips — decide and say it.
- **Split aces**: 21 after a split is *not* blackjack (`fromSplit`). **Shoe exhaustion**: cut card reshuffles between rounds; a stacked shoe throws instead.
- **Can't afford double/split** → `canDouble=false`, and the action is rejected if chosen. **Peek** only on ace or ten.

Extensions & follow-up questions

- **Even money?** Same as insurance with a blackjack (always nets +1× ante); add `takeEvenMoney(hand)` and settle 1:1 before the peek.
- **Surrender?** `Action.SURRENDER` as first decision; refund half.
- **Testing the dealer rule?** Stacked shoe, deal A+6, assert `hitSoft17()` draws and `standOnAll17()` stands.
- **Uno deck?** `Deck<UnoCard>`. **Security?** Cut card limits counting; never send the hole card early.

Real-world example

Online casino tables split a certified **RNG/shuffle service** (regulators such as the UK Gambling Commission require independent testing), a **game engine** enforcing table rules (S17/H17, 3:2 or 6:5), and a **wallet** debiting atomically — rule variants are configuration.

In the interview

30-second answer: "A generic `Card` and `Deck<T>`. A `Shoe` combines decks with an injected `Random`, plus a stacked constructor for tests. The blackjack value lives in the game's scoring, not on `Card`. `Hand` scores itself: hard total with aces as 1, soft if one ace can be 11. Player decisions and the dealer rule are strategies. `BlackjackGame` runs bets, deal, insurance and peek, player turns, dealer turn, and settles 3:2, 1:1 or push."

Likely follow-ups: *Why not `int value` on `Card`?* — value is game-specific. *Best hand value?* — hard + 10 if an ace fits. *Testable?* — seeded `Random`, stacked shoe.

Self-check

1. **Q:** Why does `Hand` store the bet? **A:** After a split each hand has its own bet; a double changes only that hand.
2. **Q:** A+A+9? **A:** Hard 11; one ace as 11 → soft 21.
3. **Q:** When does the dealer peek? **A:** Only on an ace or 10-value up card.
4. **Q:** Is 21 after a split a blackjack? **A:** No — only the original two cards.

TL;DR

- Keep `Card` / `Deck` generic; the value rule lives in the game.
- Hard total + 10 if an ace fits → soft. Blackjack = original two cards totalling 21.
- Strategies for player and dealer; inject randomness; stacked shoe for tests.
- Round = bet → deal → (insurance + peek) → player turns → dealer → settle (3:2, 1:1, push).

Design Chess

Problem & clarifying questions

Chess: two players, 8×8 board, 16 pieces each (**8 pawns, 2 rooks, 2 knights, 2 bishops, 1 queen, 1 king**); goal is **checkmate**. We design an **online** two-player game.

Question	Assumed answer
Full rules?	Yes: all moves, check, checkmate, stalemate, castling, en passant, promotion.
Sides?	Random (`Game.withRandomSides`, injected `Random`); **white first**; alternate.
Take-backs?	Not in rated games (course); `undo()` for analysis and tests.
Endings?	Checkmate, stalemate (draw), resignation, forfeit.
Move log?	Yes.
Timers, rating, AI?	Out of scope.

Requirements

Functional:

1. Two players, a colour each, white first, alternating.
2. Move only your **own** piece; reject out-of-turn moves, impossible moves, and moves leaving your king in check.
3. All special moves:
   - **Castling** — king moves two squares toward a rook, rook jumps over; only if neither moved, squares between empty, king not in check and not passing through or landing on an attacked square.
   - **En passant** — a pawn that advanced two squares beside an enemy pawn may be captured as if it moved one, only on the very next move.
   - **Promotion** — a pawn on the last rank becomes Q, R, B or N.
4. Detect **check**, **checkmate** (in check, no legal move — attacker wins) and **stalemate** (not in check, no legal move — draw).
5. **Resignation** and **forfeit** (losing without choosing, e.g. leaving or timeout).
6. **Log** every move.

Non-functional: new piece types without touching existing ones (Open/Closed); correctness before speed; safe when two player connections call concurrently.

Use cases

Player — create/join games, **make moves**, resign/forfeit, manage account. **Admin** — add, modify, block members. "Make move" is extended per piece type — our `Piece` polymorphism. The move activity `Game.move` follows: valid for the piece? → own king safe? → capture → promote → castling moves the rook → checkmate/stalemate ends the game.

Core entities

Class	Responsibility
`Color`	WHITE/BLACK, `opposite()`.
`Position` (record)	Square a–h/1–8; validates; parses "e4" (course: *Box*).
`Piece` (abstract, sealed)	Colour + `targets(board, from)`; `attacks(...)` for check detection.
`King`, `Knight`, `Pawn`	Step pieces and the pawn rule.
`SlidingPiece` → `Rook`, `Bishop`, `Queen`	One "slide until blocked" loop; subclasses list directions.
`Board`	8×8 grid; `copy()`, `kingOf`, `isAttacked`, `render`; no turns or history.
`Move` + `MoveType`	from, to, type (NORMAL, CASTLE_KINGSIDE, CASTLE_QUEENSIDE, EN_PASSANT, PROMOTION), promotion piece.
`CastlingRights` (record)	Four permissions and how a move changes them.
`Rules`	Stateless: pseudo-legal, legal (king-safety filter), special moves, `apply`, `isInCheck`.
`Game`	Turn, status, winner, en passant square, castling rights, log, undo (Command + Memento), resign, forfeit, random sides. Thread-safe.
`Player`, `GameStatus`, `IllegalMoveException`	Supporting types.

Class diagram

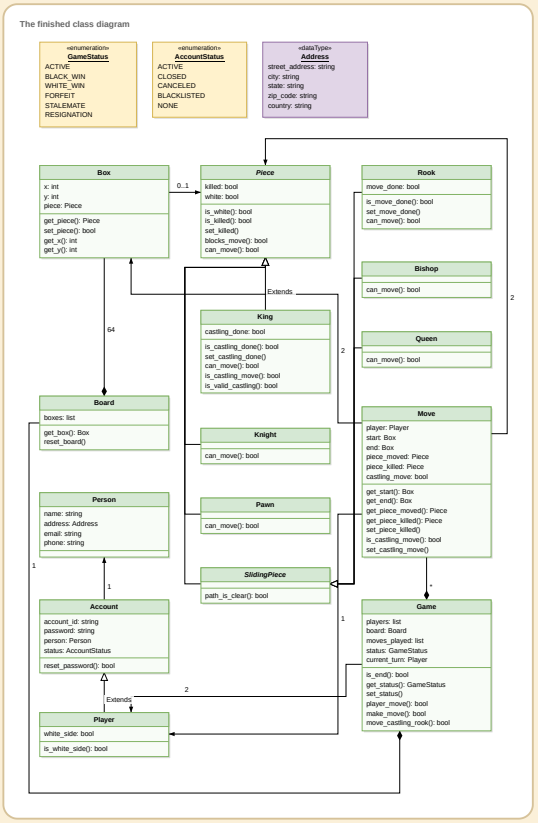
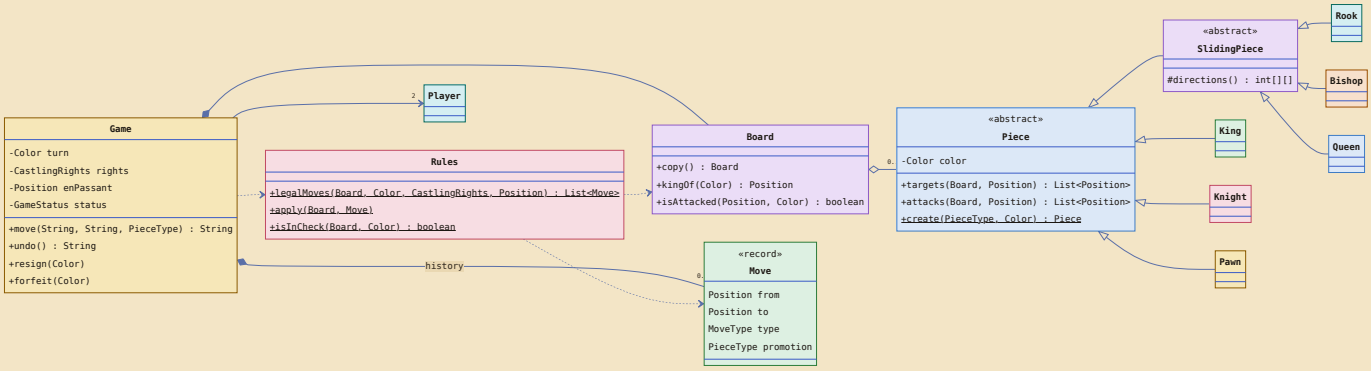


Figure — Course class diagram: Box, Board, Piece hierarchy with SlidingPiece, Move, Player, Account, Game, GameStatus (course diagram)

In the course, `Board` is **composed** of 64 `Box`es; a box refers to a piece by **association** (a captured piece isn't destroyed, so the log is replayable). `SlidingPiece` writes the "path is clear" check once; `Move` records piece moved and captured; course `GameStatus`: ACTIVE, BLACK_WIN, WHITE_WIN, FORFEIT, STALEMATE,

RESIGNATION. We fix two limits the course admits — history rules (castling, en passant) had nowhere to live, and checkmate/stalemate weren't detected — by adding `Rules` and a `MoveType` on `Move`.



Key flows

Making a move: check status, piece and colour → `Rules.legalMoves` → match or throw `IllegalMoveException` → save a `Snapshot` (memento) → `Rules.apply` → update rights, en passant, turn → check the opponent → notation (`Qh5xf7#`). **Status:** `ACTIVE` → `CHECKMATE` / `STALEMATE` / `RESIGNATION` / `FORFEIT` (undo can reactivate).

Detecting check, checkmate, stalemate:

1. **Pseudo-legal moves** — the pieces' movement rules + castling, en passant, promotions.
2. **Legal moves** — apply each to a *copy* of the board, keep it only if the mover's king is **not attacked**. This one filter handles pins, moving into check and ignoring check.
3. **King attacked?** — every enemy piece's `attacks(...)`; pawns attack diagonally even onto empty squares.
4. After a move, if the **opponent** has **zero legal moves**: in check = **checkmate**, else **stalemate**.

This one-ply search is all the *rules* need (deeper search is an *engine's* job). Cost: ~200 pseudo-legal moves × an attack scan over ≤ 16 pieces — trivial; engines use bitboards.

Design decisions & patterns used

Decision	Why	Alternative rejected
Polymorphic pieces, template-like `SlidingPiece`.	Each rule in its own class; path loop written once.	Giant `switch (pieceType)`.
Sealed hierarchy.	Closed set; exhaustive `switch`.	Open hierarchy — fine for variants.
Immutable pieces; `Board.copy()` copies references.	Cheap, safe trial moves and snapshots.	Mutable `hasMoved` flags undo must restore.
History rules in `Rules` / `Game`.	Castling and en passant depend on earlier moves.	`King` reading the game log.
`Move` carries a **`MoveType`**.	Castling moves two pieces; en passant captures off-square.	Special cases in `Game` (course's `move_castling_rook`).
Command + Memento undo.	Restoring a snapshot is always correct for special moves.	Hand-written inverse per move type (bug source).
Factory method `Piece.create`.	Set-up and promotion.	Scattered `new`.
`synchronized` `Game`.	A move and a resignation can't interleave.	No locking.

Memento vs incremental undo. A snapshot is 64 references (~0.5 KB) — nothing for a human game. An engine searching millions of positions stores only what changed (captured piece, old rights, old en passant) and reverses it. Know both.

Java implementation

Package `lld.ch17.chess`. Full runnable code: code/lld/ch17/chess/ (see the Full Edition).

```java
public abstract sealed class Piece permits King, Knight, Pawn, SlidingPiece {
    // ...
    /** Squares this piece may move to by its ordinary rules (captures included, own-king safety NOT checked).
*/
    public abstract List<Position> targets(Board board, Position from);

    /** Squares this piece attacks. Same as targets for every piece except the pawn. */
    public List<Position> attacks(Board board, Position from) { return targets(board, from); }
    // ...
}

public abstract sealed class SlidingPiece extends Piece permits Rook, Bishop, Queen {
    // ...
    protected abstract int[][] directions();

    @Override public final List<Position> targets(Board board, Position from) {
        List<Position> out = new ArrayList<>();
        for (int[] d : directions()) {
            int f = from.file() + d[0], r = from.rank() + d[1];
            while (Position.onBoard(f, r)) {
                Position p = new Position(f, r);
                Piece occupant = board.get(p);
                if (occupant == null) {
                    out.add(p);
                } else {
                    if (occupant.color() != color()) out.add(p);   // capture, then stop
                    break;
                }
                f += d[0];
                r += d[1];
            }
        }
        return out;
    }
}

// Rules.java
public static List<Move> legalMoves(Board board, Color side, CastlingRights rights, Position enPassant) {
    List<Move> legal = new ArrayList<>();
    for (Move m : pseudoLegalMoves(board, side, rights, enPassant)) {
        Board trial = board.copy();
        apply(trial, m);
        if (!isInCheck(trial, side)) legal.add(m);
    }
    return legal;
}

public static void apply(Board board, Move m) {
    Piece piece = board.get(m.from());
    board.set(m.from(), null);
    board.set(m.to(), piece);
    switch (m.type()) {
        case EN_PASSANT -> board.set(new Position(m.to().file(), m.from().rank()), null);   // the passed pawn
        case PROMOTION -> board.set(m.to(), Piece.create(m.promotion(), piece.color()));
        case CASTLE_KINGSIDE -> moveRook(board, m.from().rank(), 7, 5);
        case CASTLE_QUEENSIDE -> moveRook(board, m.from().rank(), 0, 3);
        case NORMAL -> { }
    }
}

// Game.java — inside play(): end-of-game detection
boolean check = Rules.isInCheck(board, turn);
boolean noMoves = Rules.legalMoves(board, turn, rights, enPassant).isEmpty();
if (noMoves) {
    status = check ? GameStatus.CHECKMATE : GameStatus.STALEMATE;
    winner = check ? mover.color() : null;
```

```
}

public synchronized String undo() {
    if (history.isEmpty()) throw new IllegalStateException("nothing to undo");
    HistoryEntry last = history.pop();
    Snapshot s = last.before();
    board = s.board(); turn = s.turn(); rights = s.rights(); enPassant = s.enPassant();
    status = s.status(); winner = s.winner();
    return last.notation();
}
```

`Pawn` overrides `attacks` to return both forward diagonals; `Rules.addCastling` checks rights, empty squares and attacked squares; `CastlingRights.after` clears rights when the king or a rook moves or a corner rook is captured.

Output excerpt:

```
  e7-e5 rejected: it is WHITE's turn
  a1-a4 rejected: ROOK on a1 cannot move to a4
  h5-f7 played as Qh5xf7#
  Status: CHECKMATE, winner WHITE
  undo Qh5xf7# -> status ACTIVE, WHITE to move, 43 legal moves
  c7-d8 played as c7xd8=Q+
  b2-a1 rejected: b2-a1 would leave the BLACK king in check
  Status: STALEMATE, black in check? false, black legal moves: 0
  e1-g1 rejected: cannot castle e1-g1: the king would pass through the attacked square f1
  e8-g8 rejected: cannot castle e8-g8: the king is in check
```

Validation, Scholar's mate and undo, a checking promotion, Sam Loyd's 10-move stalemate, and castling refusals with reasons (`Rules.whyCannotCastle`).

Concurrency & edge cases

- **Two players, one game:** `synchronized` public methods; each game has its own monitor, so games don't block each other.
- **Out of turn / game over / empty square** rejected before rule work. **Pins, discovered checks** handled by the copy-and-test filter.
- **Castling through check** tested for the king's crossing and landing squares (the queenside rook's crossing square b1/b8 may be attacked — legal). **Rights** lost when king or rook moves or the rook is **captured** on its corner.
- **En passant expires** after one move. **Promotion** defaults to a queen; underpromotion supported.
- **Not implemented, but name them:** threefold repetition, 50-move rule, insufficient material, clocks, draw offers.

Extensions & follow-up questions

- **Threefold repetition?** `Map<PositionKey, Integer>` keyed on (board, side, rights, en passant); engines use Zobrist hashing.
- **50-move rule?** Half-move clock, reset on pawn moves/captures. **Clocks?** A `Clock` per player (Observer on flag-fall).
- **AI?** A `MoveStrategy` over `legalMoves()`; minimax + alpha-beta. **Chess960/new piece?** New `Piece` subclass + start factory.
- **Spectators/UI?** Observer `GameListener.onMove`; the course's `GameController`/`GameView` is MVC (Ch. 11).

Real-world example

Lichess and Chess.com treat the **server** as the source of truth. Games export as PGN (a move log) and positions as FEN (board, side to move, castling rights, en passant — our `Snapshot` fields); takebacks are a server-side undo.

In the interview

> **30-second answer:** "`Board` is an 8×8 grid of immutable `Piece`s. Each piece overrides `targets`; rook/bishop/queen share a `SlidingPiece` loop. A stateless `Rules` class generates pseudo-legal moves, adds castling, en passant and promotion using history kept in `Game`, and filters out moves that leave your king attacked. If the opponent then has no legal move, it's checkmate if in check, else stalemate. `Game` handles turns, status, the log, and undo via Command plus Memento."

Likely follow-ups: *Checkmate?* — no legal moves + in check. *Where does castling live?* — `Rules`; `MoveType.CASTLE_*` tells `apply` to move the rook. *Why is Box→Piece not composition?* — a captured piece isn't destroyed. *Undo with promotion?* — the memento holds the old board with the pawn.

Self-check

1. **Q:** Why does `Pawn` override `attacks()`? **A:** It moves straight but attacks diagonally, even onto empty squares.
2. **Q:** Checkmate vs stalemate? **A:** Both = no legal move; in check = loss, not in check = draw.
3. **Q:** Three castling conditions? **A:** King and rook unmoved; squares between empty; king not in, through, or into check.
4. **Q:** Why immutable pieces? **A:** Board copies and snapshots can share them safely.

TL;DR

- Polymorphic `targets()`; `SlidingPiece` shares the path loop; pawn overrides `attacks()`.
- `Rules`: pseudo-legal → add specials → drop moves leaving own king attacked.
- No legal moves: in check = checkmate, else stalemate.
- History rules live in `Game` / `Rules`. Undo = Command + Memento. `synchronized` game.

Design Tic-Tac-Toe

Problem & clarifying questions

Players take turns placing X or O; first to fill a **row, column or diagonal** wins; a full board with no winner is a **draw**. Interviewers generalise to **N×N** and ask for **fast win detection**.

Question	Assumed answer
Board size?	Any N ≥ 3.
Win?	A full line of N marks: row, column, or one of the two main diagonals.
Players?	Two by default; more allowed, each with its own mark.
Human or computer?	Both, via a pluggable **move strategy** (human, random, rule-based, scripted).
Invalid moves?	Rejected; same player tries again.

Requirements

Functional: (1) N×N board and 2+ players with unique marks; (2) turns, current player picks an empty cell; (3) validate inside and empty; (4) detect **win** or **draw** after each move; (5) announce result and show board.

Non-functional: win check **O(1)** per move (not O(N²)); new player types without touching `Game`; seeded random bots and scripted players; games share no state.

Use cases

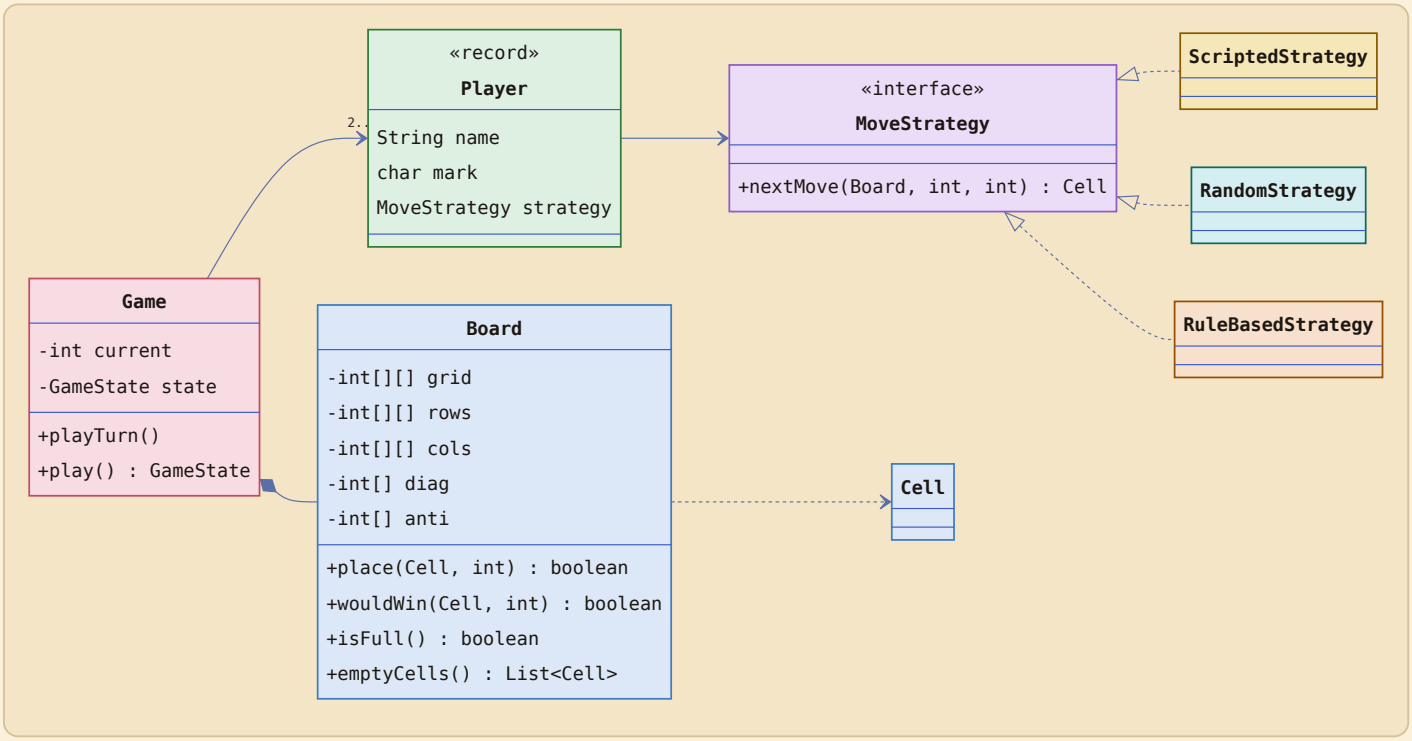
Player (human or bot): start game (choose N and players), make move, view board, see result. The `Game` validates moves, switches turns, declares the result.

Core entities

Class	Responsibility
`Cell` (record)	Row + column.
`Board`	Grid + **per-player line counters**; `place` returns "won?" in O(1); `wouldWin`, `isFull`, `emptyCells`, `render`.
`MoveStrategy`	`ScriptedStrategy`, `RandomStrategy`, `RuleBasedStrategy`.
`Player` (record), `GameState`	Name, mark, strategy; IN_PROGRESS, WON, DRAW.
`Game`	Turn order, validation, state, winner.

Class diagram

No course lesson; from the *awesome-low-level-design* problem set (3×3, X/O turns, line wins, full board draws, validate, announce) — covered and generalised to N×N.



Key flows

O(1) win detection. A move at `(r, c)` can only complete **row r, column c, and maybe the two diagonals**. Keep `rows[p][r]`, `cols[p][c]`, `diag[p]` (`r == c`) and `anti[p]` (`r + c == N-1`); increment the ones the cell belongs to; if any reaches **N**, player `p` wins (only `p`'s marks bump `p`'s counters). Four increments: **O(1) time, O(P·N) memory**.

> **The 2-player trick:** one counter per line, +1 for X and −1 for O, `|count| == N` wins — half the memory but only for two players.

The same counters give **"would this move win?"**: a line already holding `N-1` of `p`'s marks. The rule-based bot uses it to win, else block.

One turn: `nextMove` → `place` → occupied/off-board throws, `Game` logs and the same player goes again; else `WON`, `DRAW` if full, or next player. States: `IN_PROGRESS → WON | DRAW`.

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for moves.	Human, random, rule-based, network, minimax plug in.	`if (player.isComputer())`.
Counters inside `Board`.	O(1), next to the data.	Scanning — O(N²), duplicated for "would win".
`place` returns "won?".	A win can only appear right after a move.	A `checkWinner()` callers forget.
Board throws, Game catches.	Board guards its invariant; game sets policy (retry).	Silently ignoring bad moves.
`current = (current+1) % size`.	Any number of players.	X/O toggle.

Java implementation

Package `lld.ch17.tictactoe`. Full runnable code: code/lld/ch17/tictactoe/ (see the Full Edition).

```
// Board.java — Places a mark and returns true if this move completed a line. O(1).
boolean place(Cell c, int player) {
    if (!inside(c)) throw new IllegalArgumentException("cell " + c + " is off the board");
    if (grid[c.row()][c.col()] != -1) throw new IllegalArgumentException("cell " + c + " is already taken");
    grid[c.row()][c.col()] = player;
    filled++;
    int r = c.row(), k = c.col();
    boolean win = ++rows[player][r] == n;
    win |= ++cols[player][k] == n;
    if (r == k) win |= ++diag[player] == n;
    if (r + k == n - 1) win |= ++anti[player] == n;
    return win;
}

public boolean wouldWin(Cell c, int player) {
    int r = c.row(), k = c.col();
    return rows[player][r] == n - 1 || cols[player][k] == n - 1
            || (r == k && diag[player] == n - 1) || (r + k == n - 1 && anti[player] == n - 1);
}

public boolean isFull() { return filled == n * n; }

/** Win if you can, else block an opponent's win, else take the centre, else the first corner, else anything.
*/
public final class RuleBasedStrategy implements MoveStrategy {
    @Override public Cell nextMove(Board board, int me, int playerCount) {
        List<Cell> empty = board.emptyCells();
        for (Cell c : empty) if (board.wouldWin(c, me)) return c;
        for (int other = 0; other < playerCount; other++) {
            if (other == me) continue;
            for (Cell c : empty) if (board.wouldWin(c, other)) return c;
        }
        // ... centre, then corners, then empty.get(0)
    }
}

// Game.java
public void playTurn() {
    if (state != GameState.IN_PROGRESS) throw new IllegalStateException("game is over");
    Player p = players.get(current);
    Cell cell = p.strategy().nextMove(board, current, players.size());
    boolean won;
    try {
        won = board.place(cell, current);
    } catch (IllegalArgumentException e) {
        log.accept("  " + p.name() + " tried " + cell + ": rejected (" + e.getMessage() + "), try again");
        return;
    }
    log.accept("  " + p.name() + " (" + p.mark() + ") -> " + cell);
    if (won) { state = GameState.WON; winner = p; }
    else if (board.isFull()) state = GameState.DRAW;
    else current = (current + 1) % players.size();
}
```

Output excerpt:

```
== 1. 3x3 scripted game (with one invalid move) ==
  Xena (X) -> (0,0)
  Omar tried (0,0): rejected (cell (0,0) is already taken), try again
  ...
    X O O
    . X .
    . . X
  Result: Xena wins
== 3. 4x4 rule-based vs rule-based ==
  Result: draw
```

Concurrency & edge cases

- **Many games:** each `Game` owns its `Board`; keep them in a `ConcurrentHashMap<gameId, Game>`, one move at a time per game.
- **Two clicks at once:** `synchronized` `makeMove(player, cell)` that checks the caller is the current player.
- Move after game over → `IllegalStateException`; off-board/occupied → retry; scripted strategy out of moves → `null`, fail fast; N ≥ 3.
- **Centre of an odd board** is on both diagonals → two independent `if`s.

Extensions & follow-up questions

- **K in a row on N×N (Gomoku)?** Counters fail; walk out from the new cell in 4 directions counting own marks — O(K).
- **Unbeatable AI?** Minimax (3×3 has ≤ 9! orders), alpha-beta or memoisation for bigger boards — another `MoveStrategy`.
- **Undo?** Move stack; clear the cell and **decrement** counters. **Online?** Server holds the `Game`, broadcasts state (Observer). **Replay?** Move list + `ScriptedStrategy`.

Real-world example

Google Search's built-in tic-tac-toe offers difficulty levels — different players behind the same rules, the Strategy split here.

In the interview

30-second answer: "`Board` is N×N plus per-player counters for each row, column and both diagonals. A move bumps at most four; if one hits N the player wins — O(1). `Game` holds a player list and turn index, validates moves, and ends on a win or full board. Each `Player` has a `MoveStrategy` — human, seeded random, or rule-based that wins if it can and blocks otherwise."

Time plan: spend 10 minutes on the counter idea — draw it.

Self-check

1. **Q:** Why check only four lines? **A:** A cell is in one row, one column and at most two diagonals.
2. **Q:** Anti-diagonal test? **A:** `row + col == N - 1`.
3. **Q:** Why per-player counters, not +1/−1? **A:** Work for 3+ players and give "would win" for anyone.
4. **Q:** Draw detection? **A:** No win and filled cells = N^2.

TL;DR

- Per-player row/column/diagonal counters → O(1) win; `N-1` → O(1) "would win".
- `Game` = players + turn index + state; invalid moves retried.
- Players via `MoveStrategy`; K-in-a-row needs a directional walk instead.

Design Snake and Ladder

Problem & clarifying questions

A race on a numbered board (usually 1–100): roll, move forward; a **ladder foot** takes you up, a **snake head** down; first to the last square wins. No decisions — the design is about a **clean rules engine**, **turn management** and **testable randomness**.

Question	Assumed answer
Board?	Configurable (default 100, given snakes/ladders); validated.
Dice?	Configurable count and faces (default one d6).
Players?	Two or more, starting off the board (square 0).
Overshoot?	Configurable: **exact finish** or **bounce back**.
Rolling a six?	Roll again; three sixes cancels the turn (on/off).
End?	**First to finish** or **rank everyone**.
Several games?	Yes, independent sessions.

Requirements

Functional: (1) numbered board (~100) with predefined snakes and ladders; (2) several players with one token, off the board; (3) roll and move by the total; (4) snake head → tail, ladder foot → top; (5) overshoot rule; (6) optional bonus on a six with the three-sixes penalty; (7) continue until the win strategy says over; (8) many independent sessions.

Non-functional: deterministic dice (seeded or loaded); pluggable rules without editing the loop; invalid boards rejected **at construction**.

Use cases

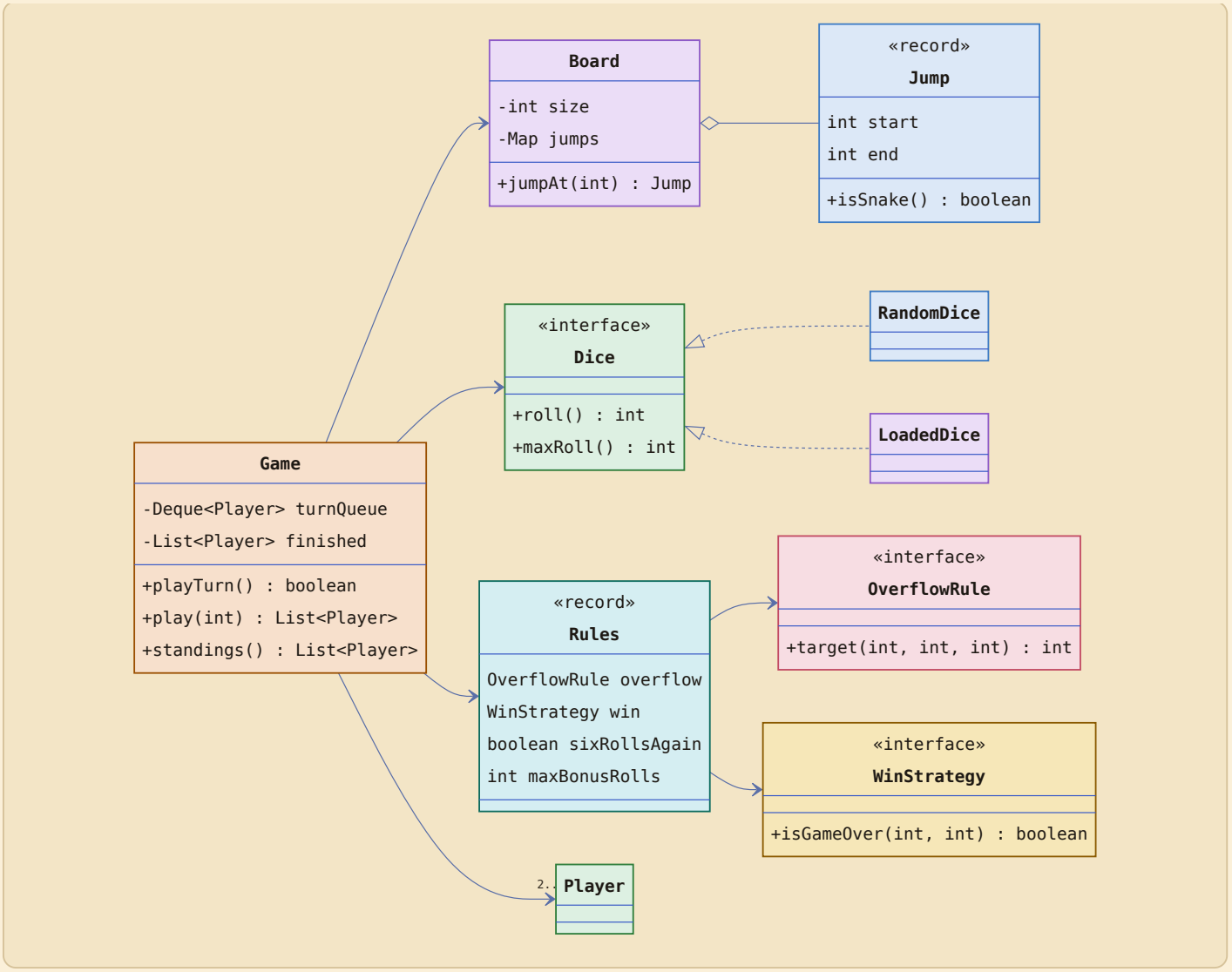
Player (rolls, or the system rolls) and **Game admin/host** (configures board, dice, rules; starts). Use cases: configure board, add players, start, take turn (roll → move → snake/ladder → maybe bonus), declare winner/standings.

Core entities

Class	Responsibility
`Jump` (record)	Snake or ladder: `start → end`; one class, `isSnake()` = `end < start`.
`Board`	Size + `Map<start, Jump>` (O(1)); validates config.
`Dice`	`roll()`, `maxRoll()`; `RandomDice` (seeded), `LoadedDice` (scripted).
`OverflowRule`	`exactFinish()` or `bounceBack()`.
`WinStrategy`	`firstToFinish()` or `rankAll()`.
`Rules` (record)	A variant: overflow, win, six-rolls-again, max bonus rolls.
`Player`, `Game`	Name + position; turn queue, `playTurn`, `play(maxTurns)`, standings, event log.

Class diagram

No course lesson; from the *awesome-low-level-design* problem set (board, snakes and ladders, tokens, turns, win at the last cell, many concurrent sessions — its reference uses a singleton `GameManager`, a thread per session). All covered.



Key flows

One turn: take the next player from the queue → roll → a max roll with bonus enabled that is the 3rd in a row voids the turn (back to the start square) → else `OverflowRule` gives the target → apply a jump starting there → bonus roll if earned and not finished → on the last square add to `finished`, else re-queue → stop when `WinStrategy` says so.

Board validation (once, in the constructor): (1) no jump to itself; (2) no jump starting on square 1 or the last square; (3) no two jumps with the same start; (4) no jump ending on another jump's start — forbids chains and therefore infinite loops.

Design decisions & patterns used

Decision	Why	Alternative rejected
One `Jump` for snakes and ladders.	Same behaviour; direction is data.	Duplicate `Snake`/`Ladder` classes.
`Map<Integer, Jump>`.	O(1); fine for sparse/huge boards.	Array of 101 (also fine).
Strategy for `Dice`, `OverflowRule`, `WinStrategy`.	House rules vary; lambdas keep them tiny.	Flags and `if`s in the loop.
`Rules` record.	One object = one variant ("classic", "kids").	Many `Game` constructor params.
Queue of players.	Finished players just aren't re-queued.	Index arithmetic skipping finishers.
`Consumer<String>` event log.	Engine doesn't print; full design uses **Observer** (`GameListener`).	`System.out` in the engine.
Validate in the constructor; `play(maxTurns)` guard.	Fail fast; a bad rule set can't hang a thread.	Mid-game loop detection; `while (true)`.

Java implementation

Package `lld.ch17.snakeladder`. Full runnable code: code/lld/ch17/snakeladder/ (see the Full Edition).

```java
// Board.java — validate once, up front
public Board(int size, Collection<Jump> all) {
    if (size < 10) throw new IllegalArgumentException("board too small");
    this.size = size;
    for (Jump j : all) {
        if (j.start() == j.end()) throw new IllegalArgumentException("jump to itself: " + j);
        if (j.start() <= 1 || j.start() >= size) throw new IllegalArgumentException("jump cannot start on
first/last square: " + j);
        if (j.end() < 1 || j.end() > size) throw new IllegalArgumentException("jump ends off board: " + j);
        if (jumps.putIfAbsent(j.start(), j) != null) throw new IllegalArgumentException("two jumps start at "
+ j.start());
    }
    for (Jump j : jumps.values()) {   // no chains: an end square may not be another jump's start (also rules
out loops)
        if (jumps.containsKey(j.end())) throw new IllegalArgumentException("jump " + j + " ends on another
jump's start");
    }
}

// OverflowRule.java
static OverflowRule exactFinish() { return (from, roll, size) -> from + roll > size ? from : from + roll; }
static OverflowRule bounceBack() { return (from, roll, size) -> from + roll > size ? size - (from + roll -
size) : from + roll; }

// WinStrategy.java
static WinStrategy firstToFinish() { return (finished, total) -> finished >= 1; }
static WinStrategy rankAll() { return (finished, total) -> finished >= total - 1; }

// Rules.java
public record Rules(OverflowRule overflow, WinStrategy win, boolean sixRollsAgain, int maxBonusRolls) {
    public static Rules classic() { return new Rules(OverflowRule.exactFinish(), WinStrategy.firstToFinish(),
true, 3); }
}

// Game.java
public boolean playTurn() {
    if (isOver()) throw new IllegalStateException("game is over");
    Player p = turnQueue.poll();
    takeTurn(p);
    if (p.position() == board.size()) {
        finished.add(p);
        log.accept("  *** " + p.name() + " finishes in place " + finished.size() + " ***");
    } else {
        turnQueue.offer(p);
    }
    return isOver();
}

private void takeTurn(Player p) {
    int start = p.position();
    int bonus = 0;
    // ...
    while (true) {
        int roll = dice.roll();
        // ...
        boolean max = rules.sixRollsAgain() && roll == dice.maxRoll();
        if (max && ++bonus == rules.maxBonusRolls()) {
            p.moveTo(start);                      // too many sixes in a row: the whole turn is cancelled
            // ...
            break;
        }
        move(p, roll, line);
        if (!max || p.position() == board.size()) break;
        // ...
    }
```

```
    // ...
}
```

`move` asks the overflow rule for the target, then looks up `board.jumpAt(target)` and moves to its end.

Output excerpt:

```
  Ann @0: rolls 4 -> 4 (ladder to 14)
  Bob @0: rolls 6 -> 6, bonus roll: rolls 3 -> 9 (ladder to 31)
  Ann @14: rolls 3 -> 17 (snake to 7)
  Bob @31: rolls 6 -> 37, bonus roll: rolls 6 -> 43, bonus roll: rolls 6 -> 3 sixes in a row, turn void, back
to 31
  rejected: jump Jump[start=10, end=30] ends on another jump's start
  winners by seed: {1=Ann, 2=Bob, 3=Cid, 4=Bob, 5=Cid, 6=Cid, 7=Ann, 8=Bob}
  same as running each session alone? yes
```

Eight seeded sessions on eight threads match their solo results — sessions share no mutable state.

Concurrency & edge cases

- **Many sessions:** each `Game` owns players, queue and **its own** `Random` (a shared one is contended and breaks per-game reproducibility).
- Rolling after the end → `IllegalStateException`. A finished player is never re-queued. A bonus roll that finishes stops immediately.
- **Three sixes** voids the turn and returns the token to the turn's start (even past a ladder). **Bounce-back onto a snake** is applied.
- **Endless games** impossible with a valid board and fair dice, but `maxTurns` guards custom rules.

Extensions & follow-up questions

- **Special cells?** Generalise `Jump` into a `CellEffect` interface (`apply(player, game)`).
- **Two dice?** `new RandomDice(2, 6, rng)`; 2d6 isn't uniform (7 most likely). **Roll a 6 to enter?** An `EntryRule`. **Pause/resume?** Serialize positions, queue, dice state.
- **UI/spectators?** Observer events (`PlayerMoved`, `SnakeBitten`, …).

Real-world example

Apps such as Ludo King (with a Snakes & Ladders mode) isolate each match, roll dice where players can't tamper (the server), and push moves as events.

In the interview

30-second answer: "`Board` holds size and a map from square to `Jump` — snake or ladder, same behaviour — validated on construction. `Dice` is a strategy: seeded for play, loaded for tests. `Rules` bundles overflow (exact or bounce), win strategy (first or rank all) and the bonus-six rule. `Game` keeps a queue of players; each turn rolls, applies overflow, then any jump, and re-queues the player unless they finished."

Self-check

1. **Q:** Why is one `Jump` class enough? **A:** Both mean "land on start, go to end"; direction is data.
2. **Q:** How is an infinite loop prevented? **A:** No jump may end on another jump's start.
3. **Q:** Testing "three sixes voids the turn"? **A:** `LoadedDice(6, 6, 6)`; assert the position is unchanged.
4. **Q:** Why a queue? **A:** Finishers simply aren't put back.

TL;DR

- Board = size + `Map<start, Jump>`; validate once (no jump on last square, no chains/loops).
- Dice, overflow, win and bonus rules are strategies; seeded or loaded dice.
- Queue-based turns with a `maxTurns` guard; events to a listener, not `System.out`.

Design 2048

Problem & clarifying questions

2048: single-player 4×4 puzzle. Each move slides all tiles one way; two **equal** tiles that collide **merge** into their sum, added to the **score**. After each move that changes the board a new tile (usually 2, sometimes 4) appears in a random empty cell. Win on a 2048 tile; **over** when no move changes the board.

No course lesson; the common machine-coding statement adds: start with **two 2s**; with **three** equal tiles the pair **nearest the wall** merges, with **four** the first two and last two; input **0 = left, 1 = right, 2 = up, 3 = down**.

Question	Assumed answer
Size?	N (default 4).
New tile?	One per successful move: 2 (p = 0.9) else 4, uniform empty cell; start with two 2s.
Merge rule?	Each tile merges **at most once per move**, from the wall you move toward.
No-change move?	Not a turn: no tile, no score.
Win?	Target tile (2048, configurable); may continue.
Over?	No empty cell **and** no equal horizontal/vertical neighbours.

Requirements

Functional: (1) N×N with two 2s; (2) four directions: slide, merge once, score; (3) spawn a 2 or 4 only if the board changed; (4) detect WON, allow continuing; (5) detect LOST; (6) accept 0/1/2/3 (`Direction.fromKey`).

Non-functional: deterministic spawns via a seeded `Random`; merge logic written **once**, not four times; pluggable spawner (e.g. an "evil" one for hard cases).

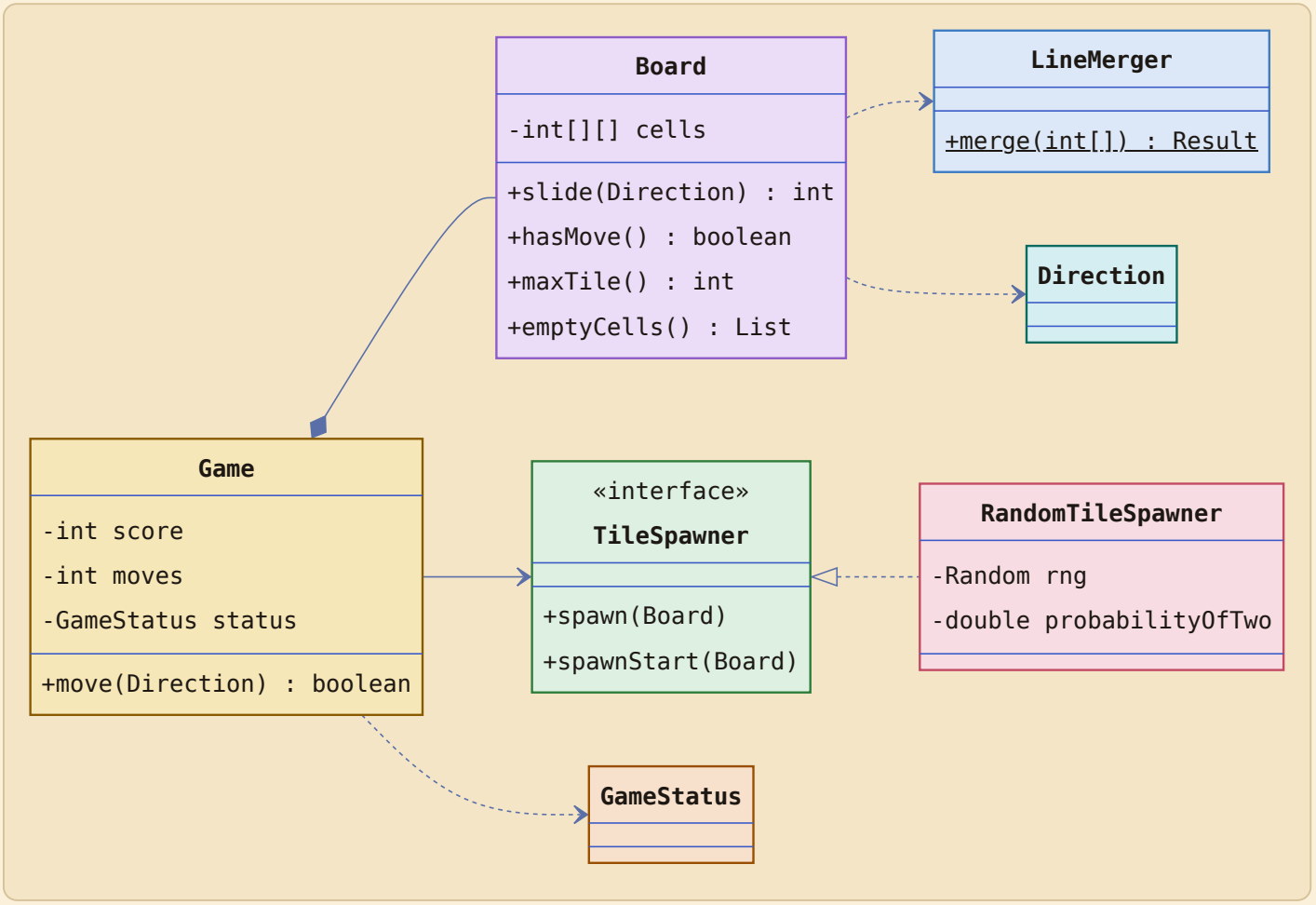
Use cases

Player: start new game, make a move (includes slide and merge, spawn tile, update score), see win, see game over, restart.

Core entities

Class	Responsibility
`Direction`	LEFT, RIGHT, UP, DOWN; `fromKey(0..3)`.
`LineMerger`	Pure function: slide and merge **one line toward index 0**; new line + points.
`Board`	N×N values; maps directions to lines; `slide`, `hasMove`, `maxTile`, `emptyCells`, `render`.
`TileSpawner` / `RandomTileSpawner`	Seeded, 90% twos; `spawnStart` places the two starting 2s.
`GameStatus`, `Game`	PLAYING, WON, LOST; score, moves, status; slide + spawn + status.

Class diagram



Key flows

Merging one line toward index 0: walk from the wall, skip zeros; keep one **pending** tile; if the next equals it, write the double, add points, clear pending (so it **can't merge again**); if different, write pending and make the new tile pending; finally write any pending tile. Examples (left): `[2,2,2,2] → [4,4,0,0]` (not one 8), `[2,2,2,0] → [4,2,0,0]` (pair nearest the wall), `[2,2,4,0] → [4,4,0,0]` (new 4 doesn't re-merge), `[4,0,4,8] → [8,8,0,0]`.

One function for four directions — `coords(direction, i, j)` maps "line i, position j from the wall" to a cell:

Direction	Line i is	j = 0 is
LEFT	row i	column 0
RIGHT	row i	column N-1
UP	column i	row 0
DOWN	column i	row N-1

A move: `slide` merges every line; nothing changed → return `false`, no spawn; else add points, spawn, update status. States: `PLAYING → WON` (target first reached) `→ PLAYING` (keep going); `PLAYING/WON → LOST` (no move left).

Game-over check: a move exists if any cell is empty or two neighbours are equal; comparing each cell with its right and lower neighbour is enough. O(N²).

Design decisions & patterns used

Decision	Why	Alternative rejected
One pure `LineMerger` + direction mapping.	Tricky rule written and tested once (DRY).	Four copies of nested loops — a bug farm.
`LineMerger` is **pure**.	Trivial unit tests.	Merge mixed with board mutation.
Strategy `TileSpawner`.	Seeded tests, evil spawner, variants.	`Math.random()` in `Board`.
No change = no turn.	Matches the real game; no spawn on a no-op.	Always spawning — unfair.
WON set once; keep playing.	Like the original game.	Ending at 2048.

Java implementation

Package `lld.ch17.game2048`. Full runnable code: code/lld/ch17/game2048/ (see the Full Edition).

```java
// LineMerger.java
public static Result merge(int[] in) {
    int n = in.length;
    int[] out = new int[n];
    int write = 0, points = 0;
    int pending = 0;                        // tile waiting for a possible merge partner
    for (int v : in) {
        if (v == 0) continue;
        if (pending == 0) {
            pending = v;
        } else if (pending == v) {
            out[write++] = v * 2;           // merge, and the new tile cannot merge again this move
            points += v * 2;
            pending = 0;
        } else {
            out[write++] = pending;
            pending = v;
        }
    }
    if (pending != 0) out[write] = pending;
    return new Result(out, points);
}

// Board.java — returns the points earned, or -1 if nothing moved
int slide(Direction d) {
    boolean changed = false;
    int points = 0;
    for (int i = 0; i < n; i++) {
        int[] line = new int[n];
        for (int j = 0; j < n; j++) line[j] = read(d, i, j);       // j = 0 is the cell next to the wall
        LineMerger.Result res = LineMerger.merge(line);
        for (int j = 0; j < n; j++) {
            if (read(d, i, j) != res.line()[j]) changed = true;
            write(d, i, j, res.line()[j]);
        }
        points += res.points();
    }
    return changed ? points : -1;
}

private int[] coords(Direction d, int i, int j) {
    return switch (d) {
        case LEFT -> new int[]{i, j};
        case RIGHT -> new int[]{i, n - 1 - j};
        case UP -> new int[]{j, i};
        case DOWN -> new int[]{n - 1 - j, i};
    };
}

public boolean hasMove() {
    for (int r = 0; r < n; r++)
        for (int c = 0; c < n; c++) {
            if (cells[r][c] == 0) return true;
            if (c + 1 < n && cells[r][c] == cells[r][c + 1]) return true;
            if (r + 1 < n && cells[r][c] == cells[r + 1][c]) return true;
        }
    return false;
}

// Game.java
public boolean move(Direction d) {
    if (status == GameStatus.LOST) throw new IllegalStateException("game over");
    int points = board.slide(d);
    if (points < 0) return false;          // nothing moved: no spawn, no turn
    score += points;
    moves++;
```

```
        spawner.spawn(board);                // at least one cell is empty, because something moved or merged
        updateStatus();
        return true;
    }
```

Output excerpt:

```
  [2, 2, 2, 2] -> [4, 4, 0, 0]  +8
  [2, 2, 2, 0] -> [4, 2, 0, 0]  +4
  [2, 2, 4, 0] -> [4, 4, 0, 0]  +4
  move LEFT changed board? false, moves=0
  checkerboard, no empty cell, no equal neighbours -> LOST
  reached 128 after 85 moves (status WON), score 852
  game over after 232 moves, score 2584, max tile 256
```

A seeded corner-strategy game reaches the 128 target and keeps playing until the board locks.

Concurrency & edge cases

- Single-player, single-threaded; if a UI thread and an input thread both call `move`, make it `synchronized` (or post to the UI event queue).
- **Key repeat:** a no-op move returns `false` — harmless.
- **Spawn needs an empty cell:** guaranteed after a change (tiles move into gaps or merge, freeing a cell); the spawner still checks.
- **Integer overflow:** unreachable on 4×4 (max tile 2^17 = 131072). **Merge order:** `[2,2,2]` left → `[4,2,0]`.

Extensions & follow-up questions

- **Undo?** Memento: push board + score before each successful move.
- **Sizes/targets?** Already parameters (`new Game(5, 4096, spawner)`).
- **AI?** Expectimax (max over moves, average over spawns) as a move-choosing strategy.
- **Animations?** `LineMerger` also returns tile origins for the view (Observer, MVC). **Leaderboard?** A separate service.

Real-world example

2048 (Gabriele Cirulli, 2014, open source) separates a grid model, a game manager and an animating view; new tiles are 2 at 90%, 4 at 10%.

In the interview

> **30-second answer:** "One pure function slides and merges a single line toward index 0: skip zeros, merge equal neighbours once, score the merges. The board maps each direction to lines in the right order, reusing the function four times. After a move that changes the board, a seeded `TileSpawner` adds a 2 (90%) or 4. Won when the target appears; over when no empty cell and no equal neighbours."

Time plan: give `LineMerger` + mapping 15 minutes; write its tests first.

Self-check

1. **Q:** `[4,4,8,8]` moving left? **A:** `[8,16,0,0]`, 24 points.
2. **Q:** Why no spawn on a no-op? **A:** It isn't a turn; spawning would punish harmless key presses.
3. **Q:** Avoiding four copies of slide code? **A:** Map directions to lines, one merge function.
4. **Q:** Repeatable tests? **A:** Seeded spawner or `Board.of(...)`.

TL;DR

- One pure `LineMerger` (merge once, from the wall) + direction→line mapping.
- Spawn only after a real change; 2 (90%) or 4; seeded spawner.
- WON once at the target; LOST with no empty cell and no equal neighbours.

Chapter summary

Game	Core rule object	Pluggable strategies	Randomness for tests	Signature technique
Blackjack	`Hand` + `BlackjackScoring`	`PlayerStrategy`, `DealerPolicy`	Seeded `Shoe`, stacked shoe	Soft/hard totals; generic `Deck<T>`
Chess	`Rules` (legal-move filter)	(bot = move strategy)	None needed	Polymorphic pieces; checkmate = in check + no legal move; Command + Memento
Tic-Tac-Toe	`Board` counters	`MoveStrategy`	Seeded `RandomStrategy`, scripted	O(1) win with counters
Snake & Ladder	`Board` + `Rules`	`Dice`, `OverflowRule`, `WinStrategy`	`RandomDice`, `LoadedDice`	Validated config; queue-based turns
2048	`LineMerger`	`TileSpawner`	Seeded spawner, `Board.of`	One merge function for 4 directions

Patterns used: Strategy (everywhere), Factory Method (`Deck.standard52`, `Piece.create`, `Board.of`), template-style base (`SlidingPiece`), Command + Memento (chess undo), State (status enums; full State pattern suggested for blackjack rounds), Observer (event-log hooks; suggested for UIs).

Three habits interviewers look for: (1) **put every rule in exactly one place** and say where; (2) **inject randomness** so the design is testable; (3) **model states explicitly** and reject actions invalid in the current state.

Chapter 18 — Case Studies: Platforms & Feeds (Part 1) — Study Edition

> **Where this fits.** Chapters 14–17 designed *things*, *bookings* and *games*. This chapter and the next design **platforms**: websites where many people create content, react to each other and build up a history. The objects are people, posts, votes, relationships and feeds. This is the study edition: every topic of the full chapter, without the padding.

#	Problem	What makes it interesting	Main patterns and ideas
18.0	What platform problems test	The toolkit reused in every social design	Facade, Observer, Strategy, services for two-sided operations, inverted index
18.1	Stack Overflow	Reputation unlocks privileges; one vote per member; moderation; bounties	Shared `Post` base, Strategy (reputation), Observer (badges), privilege table
18.2	Cricinfo (+ Udaan "Cricket Match Dashboard")	A real sport's rules; live scorecards; stats from balls	Template Method (stat queries), Observer (live feeds), "derive, don't store"
18.3	LinkedIn	Connections and degrees, invitations, messaging, jobs, search, feed	Service for two-sided operations, State (invitation), BFS, Strategy (feed ranking)
18.4	Facebook	Friends vs followers, privacy lists, pages, groups, feed, suggestions	One visibility policy for feed + search, sealed `Audience`, pull feed, suggestions

Running the code: `cd chapters/18-case-platforms-1/code && javac --release 21 -d /tmp/lld_18 $(find . -name '*.java') && java -cp /tmp/lld_18 lld.ch18.stackoverflow.Main` (or `cricinfo`, `cricketdashboard`, `linkedin`, `facebook`). Java 21, standard library only. A file may hold several small package-private types (e.g. `Posts.java` holds `Post`, `Question`, `Answer`, `Comment`, `Tag`, `Bounty`) to keep a design readable in one place.

18.0 What platform problems test

What it is

A **platform problem** asks you to design a site where *users produce the content* — Stack Overflow, Cricinfo, LinkedIn, Facebook. There is no single "machine" object; instead there is a **rich domain model**: many small classes (profile, post, vote, tag, badge, invitation, group…) each with a little data and a little behaviour. Analogy: a vending machine is a toaster; a platform is a town — people, buildings, entry rules, a notice board (feed), a post office (notifications) and a library catalogue (search).

The problem it solves

Interviewers use platform problems to test six skills at once:

Skill tested	What it means	Where you practise it
Rich domain model	Nouns → classes with clear jobs; knowing which noun is *just data* (a record) and which has *behaviour*	Cricinfo's `Over` knows a wide is not one of its six balls
User roles	Guest, member, moderator, admin, commentator may do different things	Stack Overflow privileges; Facebook admin blocks members
Feeds	A personalised, ranked list of recent content from people and pages you are linked to	LinkedIn and Facebook feed services with a ranking Strategy
Notifications	Telling a user something happened, through **channels** (in-app, email, push)	Observer + Strategy in every design
Search	Finding things by words, using an **inverted index** (word → items containing it)	Question, member and privacy-aware post search
Reputation / trust	Numbers summarising past behaviour that unlock abilities	Reputation and badges; endorsements; page ratings

How it works — the reusable toolkit

1. **A facade in front** (Chapter 9): one class offers `ask`, `vote`, `post`, `invite` and hides the rest; it checks "is this account active / allowed?" before anything changes.
2. **Shared content behaviour goes in a base class** (`Post`, `SocialContent`): voting and liking written once.
3. **Store *who*, not *how many*.** Likes and votes are a **set of member ids**: blocks double votes, allows "unlike", and the count is `set.size()`.
4. **Derive, don't store.** If a number can be computed from data you keep (follower count, batter's runs, page rating), compute it. A stored copy will one day disagree. If too slow, keep a **cache** you can always rebuild.
5. **An operation that changes two objects lives in a service** (a class whose job is to perform an operation). Accepting a connection changes *both* members, so neither member owns it.
6. **Notifications through Observer + Strategy.** The domain publishes events; observers react; the delivery channel is a Strategy, so a new channel is a new class, not a new `if`.
7. **Search is its own class** — a `SearchIndex`, not a method on the entity. Every key maps to a *collection*, even for one match, so the second "Sharma" never breaks it.
8. **Privacy and permissions are asked in one place.** Write `canSee(viewer, post)` once and call it from both feed and search.

Trade-offs

Choice	Benefit	Cost
Rich model (many small classes)	Each rule has one home; easy to test	More classes to explain in 45 minutes
Sets of voter ids instead of counters	No double votes, reversible	Memory grows; real systems use a votes table + cached count
Derive stats from raw events	Never inconsistent	Slow at scale; needs a cache
Services for two-sided operations	Symmetry guaranteed in one place	One more class; overdone, entities become "anemic"

When not to use it

For a tiny problem ("just a like button"), a set on the post is enough — no services, no observers.

Common mistakes

- Search as a static `Question.search()`, so the entity must know every other question.
- `likeCount` as a constructor argument — a post can be created "with 1,000 likes".
- Forgetting that privacy also applies inside search and the feed.
- `acceptInvitation` on the sender (or receiver), updating only one side.

Real-world examples

Kafka was built at LinkedIn to move activity events between systems (industrial Observer). **Facebook's TAO** serves the social graph at huge scale. **Stack Overflow** publishes its reputation-to-privilege table — exactly the `Privilege` enum in 18.1.

In the interview

> **30-second answer:** "Platform problems test whether I can model a domain with many small classes, put each rule in one place, and handle roles, feeds, notifications and search. I'll start with requirements and actors, then core entities, then the tricky flows — voting, accepting requests, privacy — and I'll say which operations need a service because they touch two objects."

- *Why a set instead of a counter?* — A number does not record *who* liked, so it can neither block a repeat nor undo one; the set gives the count for free.
- *Where does notification code go?* — The domain raises an event; a notification service with pluggable channels delivers it.

Self-check

- **Q:** What is an inverted index? **A:** A map from each word to the items containing it.
- **Q:** Why is "accept a friend request" not a method on `Member`? **A:** It changes two members; a service updates both together so the halves cannot drift.
- **Q:** Two notification channels and the pattern that makes adding a third easy? **A:** Email and push; Strategy (`NotificationChannel`).

TL;DR

Platform problems = rich domain model + roles + feed + notifications + search + reputation. Share behaviour in a content base class, store *who* not *how many*, derive numbers, put two-sided operations in services, notify via Observer + Strategy, keep search and privacy each in one place.

18.1 Design Stack Overflow

Stack Overflow is a Q&A community for programmers. Members **upvote/downvote** questions and answers, edit posts wiki-style, and earn **reputation** (points) and **badges** (named awards). Higher reputation unlocks **privileges**. An upvote on an answer is worth +10; the course says +5 for a question upvote, but since November 2019 it is +10 as well, and the code uses today's value.

Problem & clarifying questions

Question to ask	Answer we assume
Who uses it?	Guests, members, moderators, admins, and the system (notifications, badges)
What can a guest do?	Search and read only
What can be voted on?	Questions, answers, comments (comments: upvote only)
Reputation per vote?	Upvote +10 (question or answer); downvote −2 to the author, −1 to whoever downvotes an *answer* (downvoting a question is free); accepted answer +15 (+2 to the asker)
Privileges?	Vote up 15, comment everywhere 50, set bounty 75, vote down 125, create tag 1,500, edit 2,000, close votes 3,000, delete votes 10,000
Closing?	3 close votes from privileged members, or one moderator. Moderators also reopen, delete, undelete
Bounty?	50–500 points staked to attract answers, runs 7 days (+24 h grace on the real site), goes to the answer the sponsor picks. Deducted at once, **never refunded**; if unawarded, the site may auto-award half to a qualifying new answer
Search?	By keyword and tag, plus "most used tags"
Scale?	Read-heavy; votes must never double-count

Requirements

Functional (the course's thirteen): (1) guests search and view, must join to post or vote; (2) members ask; (3) answer **open** questions; (4) comment on questions and answers; (5) upvote questions, answers and comments; (6) **flag** posts for moderators; (7) add a **bounty** to their own question; (8) earn **badges**; (9) vote to **close**, and a moderator can close or reopen alone; (10) add **tags**; (11) vote to **delete** off-topic or low-quality questions; (12) moderators close and undelete; (13) report the **most used tags**. Added from the real site: downvotes, accepting an answer, reputation privileges, blocking accounts.

Non-functional: one vote per member per post, reversible reputation; search and top tags must not scan every question; concurrent votes each counted once; no voting on your own post and blocked accounts can do nothing; reputation, badge and channel rules change without touching voting code.

Use cases

Actors: **Guest** (search, view), **Member** (ask, answer, comment, vote, flag, bounty, tag, delete/undelete own posts), **Moderator** (close, reopen, delete, undelete *any* question), **Admin** (block/unblock), **System** (notifications, badges). In the course's use-case diagram, "Add question" `<<include>>`s "Add tag" and a bounty is an `<<extend>>`. The activity diagram for asking checks, per tag, "does it exist?" and if not "does the member have enough reputation to create it?". Our `ask()` follows the same order.

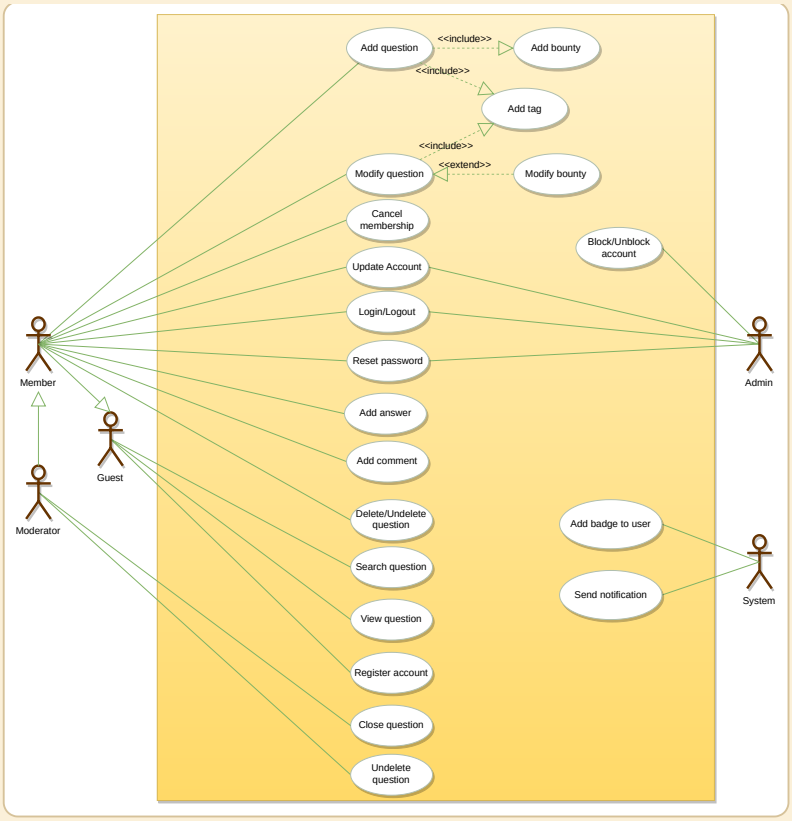


Figure — Use cases per actor: Guest, Member, Moderator, Admin, System (course diagram)

Core entities

Class	Responsibility
`Member` (+ `Moderator`, `Admin`)	Identity, atomic reputation, badges, status; `hasPrivilege(p)`
`Privilege` (enum)	Reputation needed per action — a data table, not scattered `if`s
`Post` (sealed, abstract)	Author, body, votes per voter, flags, comments
`Question`	Title, tags, answers, status, closing remark, close/delete voters, bounty, views
`Answer` / `Comment`	Answer belongs to one question and can be *accepted*; comment belongs to a question or answer (vote/flag counts derived from sets)
`Tag`	Name + description; counts its own uses
`Bounty`	Staked reputation, sponsor, expiry, winner
`Badge` (record)	Name + description
`ReputationPolicy` / `DefaultReputationPolicy`	Strategy: points per event
`ReputationListener` / `BadgeAwarder`	Observer: awards badges on reputation change
`Notifier` + `NotificationChannel` / `EmailChannel`	Pluggable delivery
`Search` / `QuestionIndex`	Inverted index by word and tag
`StackOverflow`	Facade: status + privilege checks, delegation, reputation

Class diagram

The course builds the diagram in five stages:

1. **`Badge`**, **`Tag`**, **`Notification`** (reqs 8, 10, 13). `Tag` counts its own uses, so "top tags" is a quick read. `EmailNotification` extends abstract `Notification` (passes the substitution test).

2. **`Photo`**, **`Bounty`** (req 7). The bounty owns the staked amount. The course says it can go back to the sponsor; on the real site it is **not** refundable, so our code deducts it at once and never returns it.
3. **The posts** — the heart. Voting and flagging work the *same* on three classes, so that lives in abstract `Post`; what they *contain* differs, so there are three subclasses, not one class with a "kind" field. `Post` remembers **who** voted. Filled diamonds (composition): deleting a question deletes its answers, comments, photos, bounty. `Tag` is a plain association — it outlives any question.
4. **Search** — `Search` interface + `QuestionIndex`, not a static method on `Question`.
5. **Accounts** — `Guest` ← `Member` ← `Moderator` / `Admin`. Requirement 1 is enforced by *shape*: a guest has no `createQuestion` method. `Member` owns its badges.

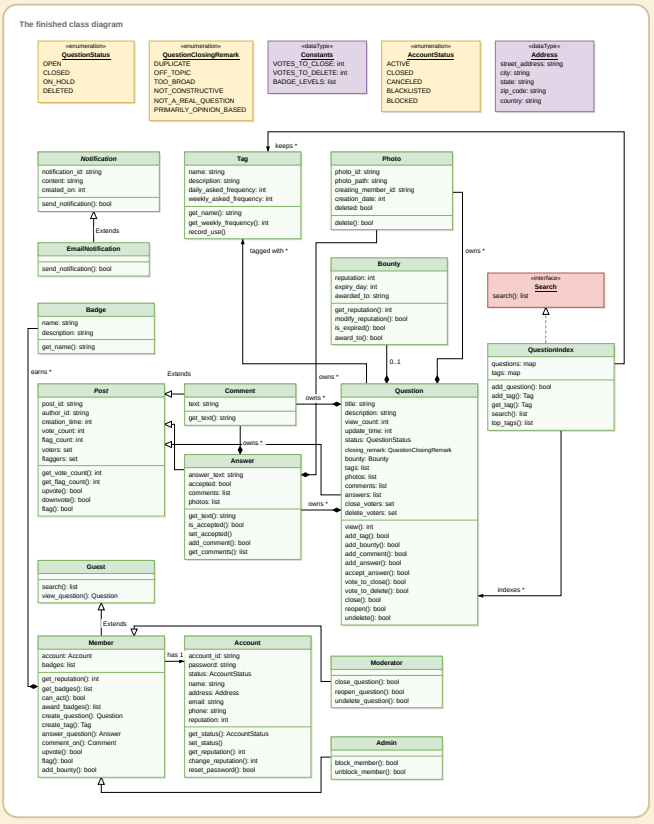
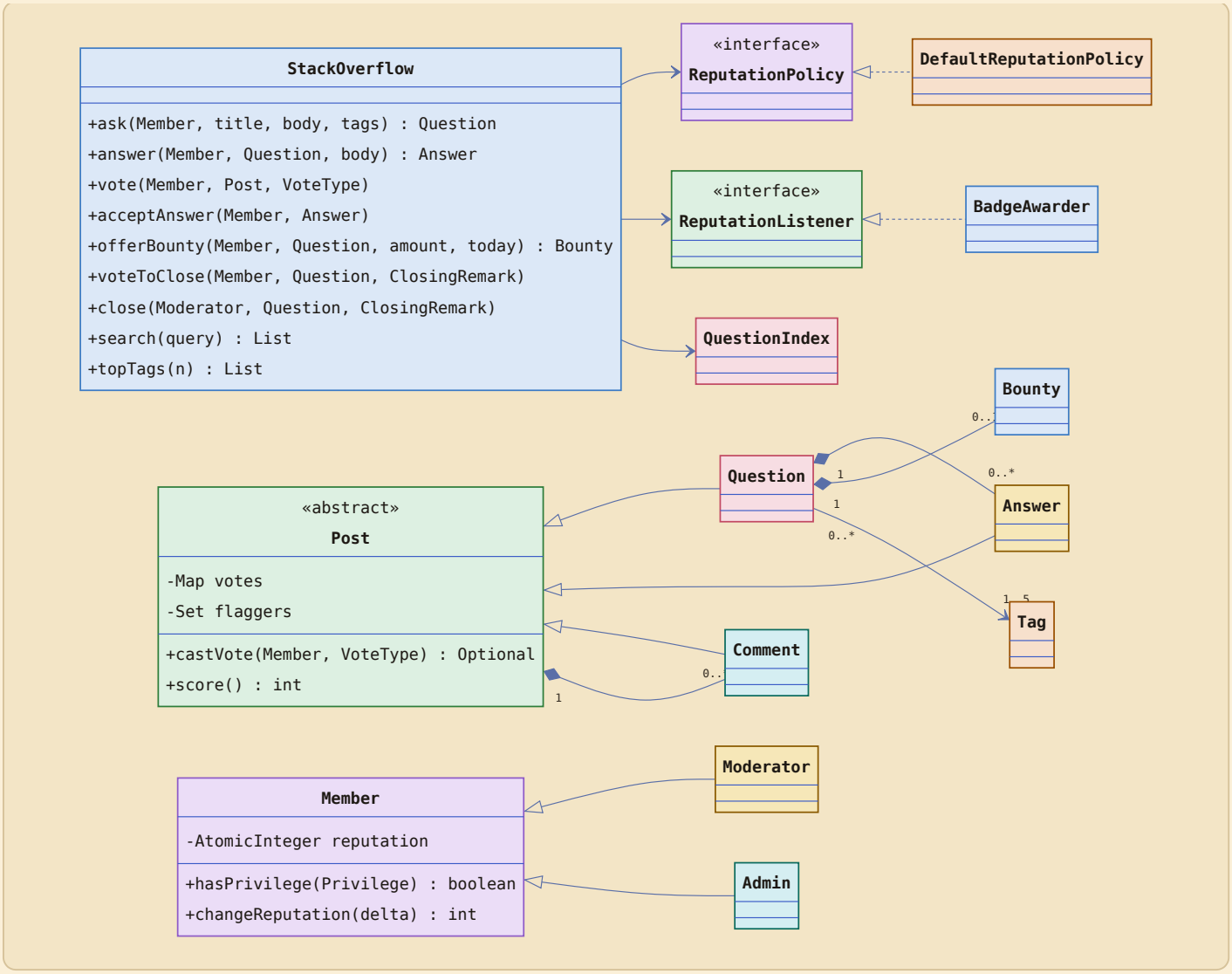


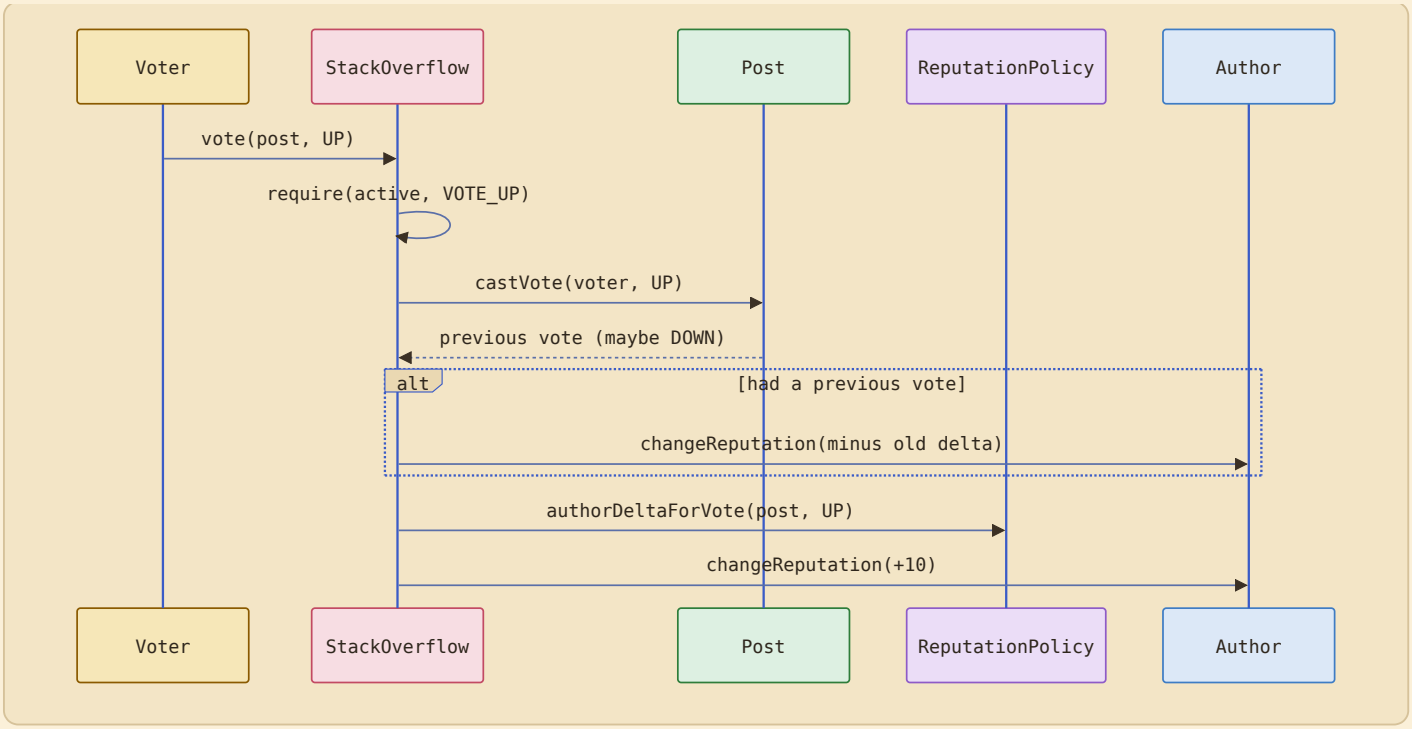
Figure — The finished Stack Overflow class diagram (course diagram)

Our Java design keeps this structure and adds a reputation **Strategy**, a badge **Observer** and a data-driven **privilege table**. A guest is simply "no `Member` object": every write needs a `Member` argument. We also add the downvote cost the course leaves out; edit history and duplicate links are extensions; chat is out of scope.



Key flows

Voting (with a changed vote). The facade checks privilege, asks the post to record the vote; the post returns the voter's *previous* vote so the facade can reverse its reputation effect before applying the new one; listeners (badges) are told.



Question lifecycle: OPEN → CLOSED (3rd close vote or moderator) → OPEN (moderator reopen); CLOSED → DELETED (3rd delete vote or moderator); OPEN → DELETED (moderator); DELETED → CLOSED (undelete). No new answers while closed.

Bounty: `offerBounty` checks `SET_BOUNTY` and the 50–500 range, attaches it to the open question and deducts the amount immediately; `awardBounty` (sponsor only, not their own answer, before expiry) credits the winner.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
`Post` base for question/answer/comment	Inheritance, `sealed`	Voting, flagging, commenting identical	One class + `type` field → `if (type == …)` everywhere
Votes as `Map<voterId, VoteType>`	Store who, not how many	Blocks double votes; allows reversal	`int voteCount`
`ReputationPolicy`	**Strategy**	Scoring rules change (they did in 2019)	Hard-coded numbers in `vote()`
`ReputationListener` + `BadgeAwarder`	**Observer**	Badges, notifications react without the facade knowing	Calling `awardBadges()` inside `vote()`
`Privilege` enum with thresholds	Table-driven	One place for rules	`if (rep >= 125)` scattered
Moderator-only methods take `Moderator`	Type-level permission	Cannot call `close(…)` without a moderator	Forgettable runtime role check
`QuestionIndex` separate	SRP	Swap for Elasticsearch behind `Search`	Static `Question.search()`
`Tag` counts its uses	Aggregate at write time	Top tags = sort a small collection	Scan all questions
`StackOverflow` facade	**Facade**	One entry that validates, delegates, logs	UI calling entities directly
Pattern-matching `switch` over sealed `Post`	Exhaustive dispatch	Compiler forces every post kind	`instanceof` chains

Java implementation

Package `lld.ch18.stackoverflow` (`Types.java`, `Accounts.java`, `Posts.java`, `Reputation.java`, `QuestionIndex.java`, `StackOverflow.java`, `Main.java`). The core: vote storage, the policy, the badge observer and the facade's `vote`.

```java
// File: lld/ch18/stackoverflow/Posts.java (excerpt)
abstract sealed class Post permits Question, Answer, Comment {
    // ...
    private final Map<Long, VoteType> votes = new HashMap<>();
    private final Set<Long> flaggers = new HashSet<>();
    // ...
    /** Records or changes a vote and returns the voter's previous vote, if any. */
    synchronized Optional<VoteType> castVote(Member voter, VoteType type) {
        if (voter.id() == author.id()) throw new IllegalStateException("you cannot vote on your own post");
        VoteType previous = votes.put(voter.id(), type);
        if (previous == type) throw new IllegalStateException(voter.name() + " already voted " + type + " on
this post");
        return Optional.ofNullable(previous);
    }

    synchronized int score() {
        return votes.values().stream().mapToInt(v -> v == VoteType.UP ? 1 : -1).sum();
    }
    // ...
}

// File: lld/ch18/stackoverflow/Reputation.java (excerpt)
final class DefaultReputationPolicy implements ReputationPolicy {
    @Override public int authorDeltaForVote(Post post, VoteType type) {
        return switch (post) {
            case Question q -> type == VoteType.UP ? 10 : -2;   // +5 before November 2019
            case Answer a -> type == VoteType.UP ? 10 : -2;
            case Comment c -> 0;                        // comment votes earn no reputation
        };
    }
    @Override public int voterDeltaForVote(Post post, VoteType type) {
        return post instanceof Answer && type == VoteType.DOWN ? -1 : 0;
    }
    @Override public int acceptedAnswerBonus() { return 15; }
    @Override public int acceptorBonus() { return 2; }
}

final class BadgeAwarder implements ReputationListener {
    // ... levels: 15 Contributor, 50 Helpful, 125 Trusted
    @Override public void onReputationChanged(Member m, int oldValue, int newValue, String reason) {
        for (Level level : levels) {
            if (oldValue < level.reputation() && newValue >= level.reputation() && m.addBadge(level.badge()))
{
                notifier.send(m, "You earned the '" + level.badge().name() + "' badge");
            }
        }
    }
}

// File: lld/ch18/stackoverflow/StackOverflow.java (excerpt)
    void vote(Member voter, Post post, VoteType type) {
        require(voter, type == VoteType.UP ? Privilege.VOTE_UP : Privilege.VOTE_DOWN);
        if (post instanceof Comment && type == VoteType.DOWN) throw new IllegalStateException("comments can
only be upvoted");
        Optional<VoteType> previous = post.castVote(voter, type);
        log(voter.name() + " voted " + type + " on " + post.label() + " -> score " + post.score());
        previous.ifPresent(p -> {                                  // undo the old vote's effect first
            adjust(post.author(), -policy.authorDeltaForVote(post, p), "vote changed on " + post.label());
            adjust(voter, -policy.voterDeltaForVote(post, p), "vote changed");
        });
        adjust(post.author(), policy.authorDeltaForVote(post, type), type + " on " + post.label());
        adjust(voter, policy.voterDeltaForVote(post, type), "downvoted an answer");
    }
```

Full runnable code: code/lld/ch18/stackoverflow/ (see the Full Edition).

Output (excerpt) — a changed vote reverses the old effect first:

```
  REJECTED: alice needs 1500 reputation for CREATE_TAG (has 200)
erin voted DOWN on answer #103 by carol -> score -1
    rep carol  -2 -> 3198 (DOWN on answer #103 by carol)
    rep erin   -1 -> 3099 (downvoted an answer)
erin voted UP on answer #103 by carol -> score 1
    rep carol  +2 -> 3200 (vote changed on answer #103 by carol)
    rep erin   +1 -> 3100 (vote changed)
    rep carol  +10 -> 3210 (UP on answer #103 by carol)
erin voted to close question #107 as DUPLICATE -> CLOSED
  REJECTED: question is CLOSED, no new answers
```

Key points: `votes.put` both records the vote and returns the previous one (voting the same way twice throws). The sealed `Post` makes the policy's `switch` exhaustive. `Member.changeReputation` uses `AtomicInteger.updateAndGet` with a floor of 1. `BadgeAwarder` fires only when a threshold is *crossed*, and `Set.add` stops duplicate badges. Alice (200 rep) cannot create the new tag `hashmap`. The third close vote closes the question; a moderator reopens it.

Concurrency & edge cases

- **Simultaneous votes:** `castVote`, `score`, `flag` are `synchronized` per post; reputation is `AtomicInteger`, so both changes apply.
- **Vote and reputation not in one lock:** a reader may briefly see the vote before the reputation — acceptable. In a database: one transaction plus a **unique constraint** on `(post_id, voter_id)`.
- **Reputation floor of 1** makes reversal not perfectly symmetric (same quirk as the real site); keep a ledger of events if exactness matters.
- **Accepting a different answer:** `Question.accept` returns the previous one so its +15 is removed; the asker's +2 is given once.
- **Self-actions** (vote own post, award own bounty) rejected; **closed** questions take no answers, **deleted** ones are hidden from search; **blocked** members fail `requireActive`; **bounty** award refused after expiry.

Extensions & follow-up questions

- **Stop double voting** — the per-post voter map; in a DB, a unique key on `(post_id, voter_id)`.
- **Edit history** — immutable `Revision`s appended per edit (close to **Memento**).
- **Daily reputation cap (200/day)** — a `CappedReputationPolicy` **decorator**; no voting code changes.
- **Duplicate link** — `duplicateOf` on `Question` set when closed as `DUPLICATE`.
- **Review queues** — flagged posts go to a `ReviewQueue` where reviewers vote keep/delete.
- **Hot questions** — ranking Strategy over recent votes/answers/views with time decay.
- **Tag frequency per day/week** — counters per time bucket rolled over by a scheduled job.

Real-world example

The numbers are real stackoverflow.com rules (privileges page: 15, 50, 75, 125, 1,500, 2,000, 3,000, 10,000). The 2019 change of question upvotes from +5 to +10 touches only the `ReputationPolicy`; lowering close votes from 5 to 3 after a 2019 experiment is one constant in our code.

In the interview

Minutes	What to do
0–5	Actors and reputation rules; requirements
5–10	Use cases; "guest has no posting method"
10–20	Class diagram: `Post` + 3 subclasses, `Member` hierarchy, `Tag`, `Bounty`, index, facade
20–30	`vote()` and `acceptAnswer()` with reversal; question states
30–40	Code `Post.castVote`, the policy, facade `vote`
40–45	Concurrency and extensions

30-second answer: "Questions, answers and comments share voting and flagging, so they extend one `Post` that stores votes per voter — that stops double voting and lets me reverse a changed vote. Reputation rules are a Strategy; badges are an Observer on reputation changes. Privileges come from a reputation table; moderator actions take a `Moderator` parameter. Search is a separate inverted index, and tags count their own uses for 'top tags'."

- *Why not store `voteCount` only?* — It cannot prevent duplicates or support changes; derive it from the voter map.
- *Is reputation awarded inside `upvote`?* — No; the facade asks the policy, so posts know nothing about scoring (the course's suggested improvement).
- *Scale search?* — Keep `Search`, replace `QuestionIndex` with a search engine fed by events.

Self-check

- **Q:** Which requirement is enforced by class shape? **A:** Guests cannot post or vote — no `Member` object, and every write needs one.
- **Q:** Why is `Post` abstract and sealed? **A:** A bare post is never created; sealed makes switches over post kinds complete.
- **Q:** Erin changes a downvote on an answer to an upvote — reputation? **A:** Old effect reversed (author +2, Erin +1), new applied (author +10).
- **Q:** How do comments differ in voting? **A:** Upvote only, and they earn no reputation.

TL;DR

Stack Overflow = `Post` base (votes per voter, flags, comments) + `Question`/`Answer`/`Comment` + reputation **Strategy** + badge **Observer** + reputation **privilege table** + moderator-typed methods + inverted-index search + a facade that checks, delegates and scores.

18.2 Design Cricinfo

Cricinfo (ESPNcricinfo) is a cricket site with **live ball-by-ball commentary**, a database of historic matches, and news. This design is graded on whether you model the *rules*, not just the nouns. Cricket primer:

Term	Meaning
Delivery / ball	One ball bowled by the **bowler** to the **batter** on strike
Over	Six **legal** deliveries by one bowler. A **wide** or **no ball** gives a penalty run and is re-bowled — it does not count toward the six
Striker / non-striker	The striker faces; they change ends on an odd number of runs (1, 3, 5) and after every over
Runs	Off the bat (batter's) or **extras** — wides, no balls, **byes**, **leg byes** (the team's, not the batter's)
Wicket	Bowled, caught, LBW, stumped, hit wicket, run out, obstructing the field. **A run out is not the bowler's**
Innings	One team's turn to bat; ends all out (10 wickets), overs used up, or target reached
Formats	**T20** (20 overs/side), **ODI** (50 overs/side), **Test** (no limit, two innings per side)
Squad / playing eleven	A team names a squad per tournament, then picks exactly **eleven** from it per match

Problem & clarifying questions

Question	Answer we assume
Formats?	T20, ODI, Test
Who enters data?	**Admin** (scorer) records each ball; **Commentator** adds text
Real rules?	Yes: six legal balls, extras, wicket credit, strike rotation, bowler limits
Live updates?	Push to subscribers after every ball
Stats?	Per player, match, tournament, plus global ("highest wicket taker ever")
Tournaments?	Yes, with squads and points table
Out of scope?	Duckworth–Lewis–Stern, follow-on and declarations in Tests, powerplays, free hits, leg/off-side wides, partnerships, net run rate, news (our code adds a simple `declare()`)

Requirements

Functional (the course's eight): (1) track every team and its matches; (2) live ball-by-ball commentary; (3) follow international rules; (4) a squad per tournament; (5) a playing eleven per match from the squad; (6) stats for players, matches, tournaments; (7) global stat questions ("most wickets ever", "most Test hundreds"); (8) ODI, Test and T20.

Non-functional: stats exactly match the ball record (a correction fixes every derived number); live updates within seconds to millions; two scorers must not interleave balls; a new stat, format or subscriber is a new class.

Use cases

Admin adds/modifies teams, players, tournaments, matches, stadiums, umpires, referees and records balls (some stats system-generated); **Commentator** adds commentary. In the use-case diagram "Add match" is *extended* by the three formats and "Add ball" *includes* its runs and wicket. Activity "record a ball": choose ball type (normal, wide, no ball, wicket) → wicket details if any → a run record per run type → commentary → save.

Core entities

Class	Responsibility
`Person` (record)	Name, country; roles *hold* a person
`Player`; `MatchOfficial` → `Umpire`, `Referee`, `Commentator`	Roles; officials share "assign to match"
`Team` ⊇ `TournamentSquad` ⊇ `Playing11`	Everyone ever picked ⊇ one tournament ⊇ exactly eleven, in batting order
`Tournament`	Owns squads, lists matches, points table
`BallOutcome` (record)	Bat runs, extra type/runs, boundary?, wicket type, who was out, commentary; validates the laws
`Ball`, `Wicket` (records) + `WicketType`	Recorded delivery; `creditedToBowler()`
`Over`	Six **legal** deliveries; runs conceded; maiden?
`Innings`	Owns overs; decides striker, over end, new batter, bowler limits, innings end
`Match` (abstract) → `T20`, `ODI`, `Test`	Formats override only overs per innings and innings per side
`Result` (sealed) → `Win`, `Tie`, `Draw`	Outcome
`MatchObserver` → `CommentaryFeed`, `LiveScorecard`	Live subscribers
`PlayerStats`, `StatsService`, `StatQuery` → `MostRuns`, `MostHundreds`, `MostWickets`, `MostSixes`	Derived stats; Template Method queries

Class diagram

The course builds it in six stages:

1. **People** — `Player` *holds* a `Person` (composition over inheritance: one human can play, then commentate). Officials share assignment in `MatchOfficial`.
2. **Teams, squads, eleven** — three nested sets. `Playing11` keeps its squad so it rejects outsiders and a twelfth pick: requirement 5 lives in code.
3. **The delivery** — three hidden rules: an over is six *legal* balls (two wides → eight deliveries); a **run out is not the bowler's wicket**; byes and leg byes are the team's. `Ball` owns runs, wicket, commentary; it only *refers* to bowler and batter.
4. **The innings** — owns the overs, so it alone decides when an over ends. `score` uses runs/wickets and overs.balls notation.
5. **Match and formats** — only two numbers differ (overs per innings, innings per match), so `Match` is abstract and each format overrides two short methods. Test returns 0 overs ("no limit") and four innings total.
6. **Stats** — **worked out from the balls**, never stored beside them (a stored counter drifts after a correction). Fifties/hundreds counted per innings (`countInnings`) beside `countBall`. Each global question is a small `StatQuery` subclass.

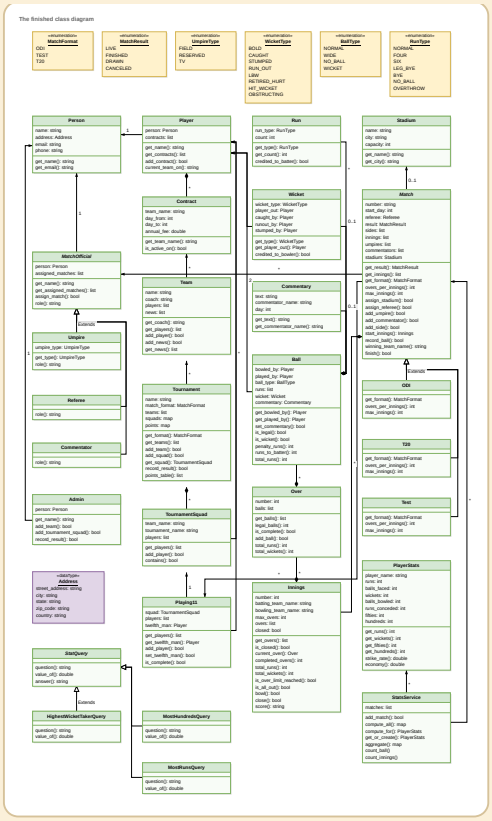
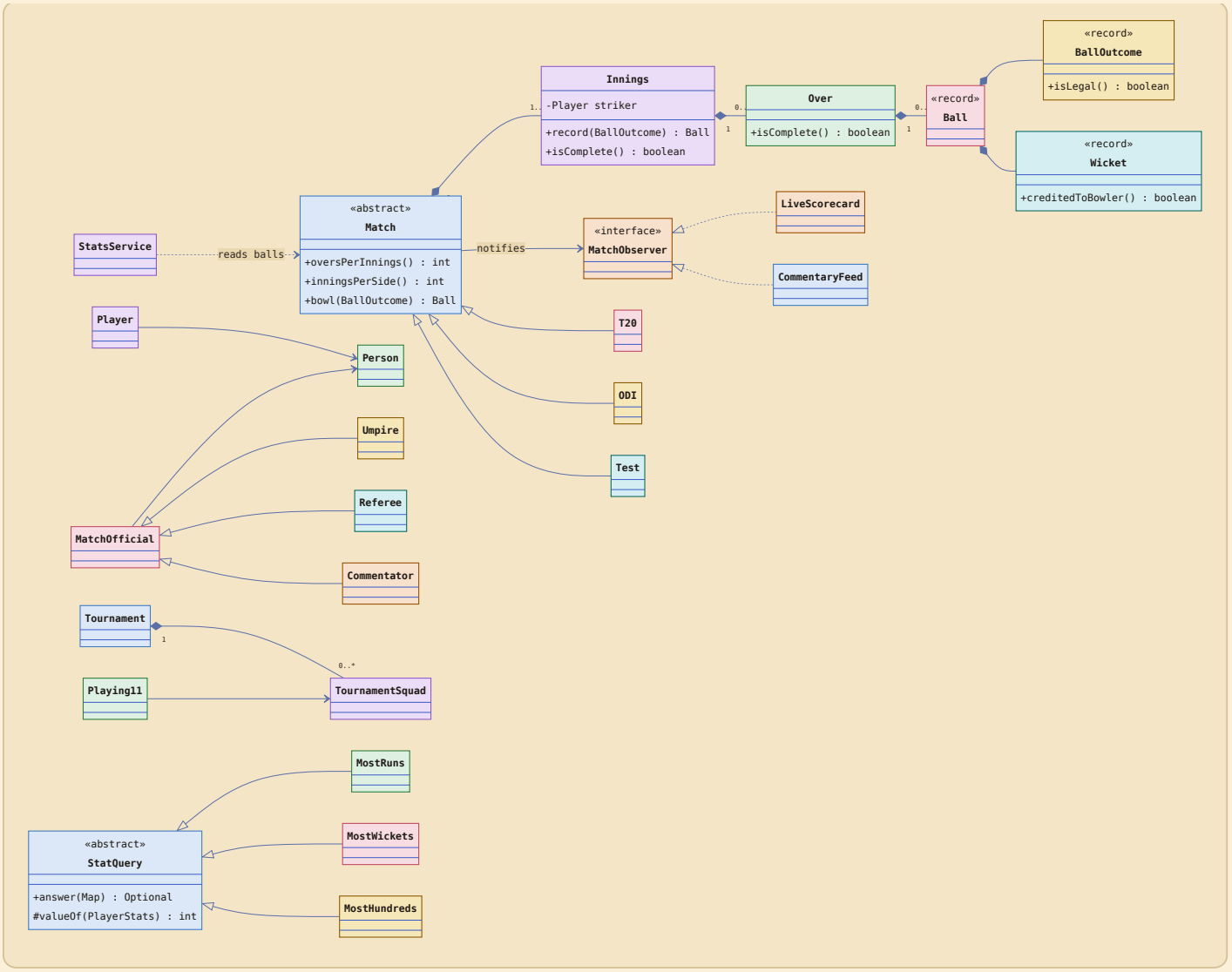


Figure — The finished Cricinfo class diagram (course diagram)

Our Java design adds what the course flags as missing: **strike rotation**, bowler rules (not two overs in a row; 4 overs per bowler in T20, 10 in ODI), which dismissals are legal off a wide/no ball, result calculation, and **Observers**. A `BallOutcome` record replaces separate `Run` objects.



Key flows

Recording a ball: scorer → `Match.bowl(outcome)` → `Innings.record`: build the `Ball` with current bowler and striker, add to the over, add runs; wicket → new batter; odd runs run → swap ends; over complete → swap ends. The match then notifies `onBall`, and `onOverEnd`/`onInningsEnd` if finished; after the last innings or target reached, `finish()` computes the `Result` and calls `onMatchEnd`.

Innings end (`Innings.isComplete`): all out (wickets = players − 1) → final innings and runs ≥ target → over limit reached and last over complete → declared.

Match status: SCHEDULED → LIVE → FINISHED; LIVE → DRAWN (Test out of time); SCHEDULED → CANCELED (washed out).

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
Roles hold a `Person`	Composition over inheritance	One human, many roles	`Player extends Person`
`Match` abstract; formats override two numbers	Template-style base (not a named pattern here)	Only what differs is overridden	Three copies of `Match`
`StatQuery.answer()` final, `valueOf()` abstract	**Template Method**	New statistic = one tiny subclass (OCP)	`switch (questionType)`
Stats derived from balls	Single source of truth	Corrections fix every number	Stored counters that drift
`Innings` owns over rule and strike	Information Expert	Class with the data decides	UI deciding when an over ends
`BallOutcome` validates in constructor	Fail fast, immutable	Bowled off a no ball never enters the record	Checking later in stats
`MatchObserver` (default methods)	**Observer**	Feeds subscribe without the match knowing	Match printing itself
`Result` sealed records	Algebraic data type	Exhaustive `switch`	Strings like "Team A won"
`Match.bowl` `synchronized`	Monitor lock per match	Scorers cannot interleave	Unsynchronised updates

Java implementation

Package `lld.ch18.cricinfo` (`People`, `Teams`, `Delivery`, `Innings`, `Match`, `Observers`, `Stats`, `Main`). The demo shows both eleven refusals, a scripted first over (a wide and a no ball make **eight deliveries**; a bowled that counts for the bowler; a run out that does not), then a seeded simulation of a T20 and an ODI.

```java
// File: lld/ch18/cricinfo/Delivery.java (excerpt)
enum WicketType {
    BOWLED, CAUGHT, LBW, STUMPED, HIT_WICKET, RUN_OUT, OBSTRUCTING_THE_FIELD;

    /** A run out (or obstructing the field) is a wicket for the fielding side, NOT for the bowler. */
    boolean creditedToBowler() { return this != RUN_OUT && this != OBSTRUCTING_THE_FIELD; }
}
// ... record BallOutcome: isLegal() { return extra != Extra.WIDE && extra != Extra.NO_BALL; }
    /** Byes and leg byes are not the bowler's fault; wides and no balls are. */
    int runsChargedToBowler() { return batRuns + (isLegal() ? 0 : extraRuns); }
// ...
/** An over is SIX LEGAL deliveries, not six balls. */
final class Over {
    // ...
    int legalBalls() { return (int) balls.stream().filter(b -> b.outcome().isLegal()).count(); }
    boolean isComplete() { return legalBalls() == LEGAL_BALLS; }
    // ...
}

// File: lld/ch18/cricinfo/Innings.java (excerpt)
    Ball record(BallOutcome o) {
        Over over = currentOver();
        if (over == null || over.isComplete()) throw new IllegalStateException("start a new over first");
        if (isComplete()) throw new IllegalStateException("innings is complete");

        int deliveryNo = over.legalBalls() + 1;          // a wide or no ball is re-bowled, so it shares the
next number
        Wicket wicket = null;
        if (o.wicket() != null) wicket = new Wicket(o.wicket(), o.nonStrikerOut() ? nonStriker : striker,
over.bowler());
        Ball ball = new Ball(over.number(), deliveryNo, over.bowler(), striker, o, wicket);
        over.add(ball);
        balls.add(ball);
        runs += o.totalRuns();

        if (o.runsRun() % 2 == 1) swapEnds();              // odd runs: batters crossed
        if (wicket != null) {
            wickets++;
            if (!allOut()) {
                Player newBatter = batting.battingOrder().get(nextBatter++);
                if (wicket.batter().equals(striker)) striker = newBatter; else nonStriker = newBatter;
            }
        }
        if (over.isComplete()) swapEnds();                  // ends change after every over
        return ball;
    }

// File: lld/ch18/cricinfo/Stats.java (excerpt)
abstract class StatQuery {
    record Leader(Player player, int value) {}

    abstract String question();
    abstract int valueOf(PlayerStats s);

    final Optional<Leader> answer(Map<Player, PlayerStats> stats) {
        Leader best = null;
        for (PlayerStats s : stats.values()) {
            int v = valueOf(s);
            if (v > 0 && (best == null || v > best.value()
                    || (v == best.value() && s.player.name().compareTo(best.player().name()) < 0))) {
                best = new Leader(s.player, v);
            }
        }
        return Optional.ofNullable(best);
    }
}
```

```java
final class MostWickets extends StatQuery {
    @Override String question() { return "Highest wicket taker"; }
    @Override int valueOf(PlayerStats s) { return s.wickets; }
}
```

Full runnable code: code/lld/ch18/cricinfo/ (see the Full Edition).

Output (excerpt) — the first over has eight lines but ends at `0.6`:

```
  0.1  Tarun to Arun, FOUR  [Ravi: driven through cover]
  0.2  Tarun to Arun, wide  [Ravi: sprayed down leg]
  0.2  Tarun to Arun, 1 run(s)  [Ravi: pushed to mid-on]
  0.3  Tarun to Bala, no ball  [Ravi: overstepped, free hit next]
  0.3  Tarun to Bala, OUT BOWLED (Bala)  [Ravi: through the gate!]
  0.4  Tarun to Chetan, OUT RUN_OUT (Chetan) - not the bowler's wicket  [Ravi: a second run was never there]
  0.5  Tarun to Arun, no run  [Ravi: defended]
  0.6  Tarun to Arun, SIX  [Ravi: launched over long-on]
  REJECTED: Tarun cannot bowl two overs in a row
```

Key points: ball numbers are "completed overs . ball"; an extra carries the number of the ball it replaces. Chetan is run out after one completed run: `record` swaps ends for the single first, then replaces the dismissed batter, so Dev arrives at the right end; Tarun's figures show only **1** wicket. Five bowlers rotate, so each bowls exactly 4 overs (`maxOversPerBowler() = 20 / 5`). The chase target is set automatically (`first-innings total + 1`). The T20 ends "Lions won by 23 runs"; the ODI "Lions won by 5 wickets". Global queries are one-line subclasses; "Most runs in ODIs" reuses the service with a format filter.

Machine-coding variant: Cricket Match Dashboard (Udaan)

A **machine-coding round** asks for a *working, runnable* program in 60–90 minutes, judged on correctness, readability and extensibility. This problem was set by Udaan (Indian B2B e-commerce) and appears in the kumaransg/LLD collection.

Problem statement (in our words)

- **Input:** players per team, number of overs, each team's batting order, then each over's balls: runs `0`–`6`, `W`, `Wd` or `Nb`.
- **After every over:** each batter's runs, 4s, 6s, balls (with `*` on the two at the crease), then total/wickets and overs.
- **Rules:** strike changes on 1s and 3s and at over end; wides and no balls add one run and are re-bowled; innings ends all out, overs done, or target passed.
- **At the end:** winner and margin (runs or wickets). **Good to have:** bowler figures, extras, strike rates.

How the full Cricinfo design shrinks to a 90-minute solution

Cricinfo class	Machine-coding version	Why it is enough
`BallOutcome`	`BallEvent` sealed interface (`Runs`, `Wide`, `NoBall`, `Wicket`) with `parse(token)` factory	Only four token kinds
`Innings`	`TeamInnings` — same responsibilities, ints instead of objects	Same rules, no bowler data
Scorecard from balls	`Batter` counters + `ScorecardPrinter`	Printer separate so another view can replace it
`Match` + `Result`	`MatchRunner`	One format, two innings

The lesson: keep the *same responsibilities in the same places* (innings owns strike and the over rule; parsing, scoring and printing separate), but use the simplest data that satisfies the input.

Code

Package `lld.ch18.cricketdashboard` (`Model.java`, `Main.java`; `Main` feeds the sample input as a text block).

```java
// File: lld/ch18/cricketdashboard/Model.java (excerpt)
sealed interface BallEvent {
    record Runs(int runs) implements BallEvent {}
    record Wide() implements BallEvent {}
    record NoBall(int batRuns) implements BallEvent {}
    record Wicket() implements BallEvent {}
    // ... static BallEvent parse(String token): "W", "Wd", "Nb..", or 0-6
    default boolean isLegal() { return !(this instanceof Wide || this instanceof NoBall); }
}
// ...
    void play(BallEvent e) {
        if (isOver()) throw new IllegalStateException("innings already finished");
        Batter s = order.get(striker);
        switch (e) {
            case BallEvent.Wide w -> { total += 1; extras += 1; }
            case BallEvent.NoBall nb -> {
                total += 1 + nb.batRuns(); extras += 1;
                s.faced(nb.batRuns());              // a no ball counts as a ball faced by the batter
                if (nb.batRuns() % 2 == 1) swap();
            }
            case BallEvent.Wicket w -> {
                legalBalls++;
                s.balls++;
                s.out = true;
                wickets++;
                if (!allOut()) striker = next++;     // new batter takes strike
            }
            case BallEvent.Runs r -> {
                legalBalls++;
                total += r.runs();
                s.faced(r.runs());
                if (r.runs() % 2 == 1) swap();       // 1s and 3s change strike
            }
        }
        if (e.isLegal() && legalBalls % 6 == 0 && !isOver()) swap();  // end of over
    }

    boolean isOver() {
        return allOut() || legalBalls == maxOvers * 6 || (target != null && total >= target);
    }
```

Full runnable code: code/lld/ch18/cricketdashboard/ (see the Full Edition).

```
Scorecard for Team 2:
Player   Score  4s  6s Balls     SR
P6          10   1   1     3  333.3
P7*          1   0   0     1  100.0
P8           0   0   0     1    0.0
P9*          1   0   0     1  100.0
P10          0   0   0     0      -
Total: 12/2   Extras: 0
Overs: 1
...
Result: Team 1 won the match by 4 runs
```

Notes on the sample and on correctness

- Our output matches the published sample except where the sample is inconsistent: after team 2's first over it prints `12/1`, but the over has **two** `W`s, so `12/2` is correct; and it later shows P6 with 2 balls after 3. Pointing this out calmly is a plus.
- Team 2's second over stops after four balls: 5 players → all out at 4 wickets ("Overs: 1.4").
- The over loop counts **legal** balls, so a wide makes an over seven tokens long.

- Extras and strike rate are printed; bowler figures would need bowler names in the input (add a `Bowler` class).

Concurrency & edge cases

- **Two scorers:** `bowl`, `startOver`, `startInnings` are `synchronized` per match; different matches never block each other.
- **Many readers:** observers in a `CopyOnWriteArrayList`. At web scale, publish each ball to a broker and push via WebSockets/SSE; the scorecard is a cached view rebuilt from balls.
- **Rejected sequences:** over while one is in progress, same bowler twice in a row, bowler over quota, bowling after the innings, bowler from batting side, twelfth player, non-squad player.
- **Impossible dismissals:** bowled/caught/LBW/stumped/hit wicket off a no ball; bowled/caught/LBW off a wide.
- **Wicket on the last ball:** new batter comes in, then ends change. **Tie** → `Result.Tie`. **Tests:** 0 = no over limit; `declare()` and `declareDraw()`.

Extensions & follow-up questions

- **Who is on strike?** — `Innings` tracks it (odd runs and over end swap).
- **Stats too slow** — a per-player running total updated by an Observer, rebuilt from balls after corrections; balls stay the truth.
- **Duckworth–Lewis–Stern** — a `TargetCalculator` Strategy (default: opponent total + 1).
- **Free hit** — innings remembers "next legal ball is a free hit"; only run-out-type dismissals allowed.
- **Partnerships / fall of wickets, net run rate** — derive from balls and innings totals.
- **Push notifications** — a new `MatchObserver` firing on wickets and 50/100 milestones.

Real-world example

ESPNcricinfo (founded 1993, bought by ESPN in 2007) is the reference for cricket stats; its **Statsguru** answers requirement-7 questions across all internationals. Live pages update ball by ball with commentary like our `CommentaryFeed`, for millions of phones — hence push-based Observers, brokers and caches.

In the interview

Minutes	What to do
0–5	Formats, actors; say early **"an over ends after six *legal* balls"**
5–10	Team ⊇ squad ⊇ eleven; people hold a `Person`
10–20	Match (abstract) → innings → over → ball; formats differ in two numbers
20–30	Ball flow, strike rotation, **no bowler credit for a run out**, byes not the batter's
30–40	Derived stats; Template Method queries; Observers
40–45	One scorer lock per match, broadcast to readers; extensions

30-second answer: "I model cricket's rules, not just its nouns. An over is six legal balls; a wide or no ball is re-bowled. The innings owns the overs, so only it decides when an over ends, who is on strike and when the innings is over. Formats differ only in overs per innings and innings per match. Stats are derived from balls so corrections propagate, each stat question is a Template Method subclass, and live pages subscribe as observers."

- *Why not store `wickets` on `Player`?* — It drifts from the ball record and cannot be filtered by format; derive it.
- *Add "most catches"?* — Fielder on `Wicket`, count it in `PlayerStats`, `MostCatches extends StatQuery`.
- *Is `Match`/formats a Template Method?* — No; only `StatQuery` is a named pattern here — formats are plain abstract-class specialisation.

Self-check

- **Q:** A wide and a no ball in an over — how many deliveries? **A:** Eight.

- **Q:** Who gets a run-out wicket? **A:** The fielding side; not in the bowler's figures.
- **Q:** Are leg byes the batter's runs? **A:** No — team extras, not charged to the bowler either.
- **Q:** What distinguishes `T20`, `ODI`, `Test`? **A:** `oversPerInnings()` (20, 50, 0) and `inningsPerSide()` (1, 1, 2).
- **Q:** Correct team 2 score after over 1 in the Udaan sample? **A:** 12/2.

> **TL;DR**
>
> Cricinfo = people hold a `Person`; team ⊇ squad ⊇ eleven; `Match` (abstract, formats override two numbers) → `Innings` (owns overs, strike, end rules) → `Over` (six **legal** balls) → `Ball`. Run outs are not the bowler's; byes are not the batter's. Stats derived from balls; stat questions are **Template Method** subclasses; live pages are **Observers**. The Udaan dashboard is the same responsibilities in a 90-minute, input-driven form.

18.3 Design LinkedIn

LinkedIn is a networking site for **working lives**: profiles with skills, jobs and schooling, and a home **feed** of updates from your network. It is like Facebook with career features: **connections** (mutual), **follows** (one-sided), **endorsements**, **recommendations**, **company pages** and **job postings**.

Problem & clarifying questions

Question	Answer we assume
Is a connection mutual?	Yes: an **invitation** is accepted or rejected. Following is one-sided
Degrees?	1st, 2nd, 3rd; beyond that "out of network"
Messaging?	Only connections (non-connections need paid *InMail*, out of scope)
Endorsements?	Only 1st-degree connections, once per skill
Recommendations?	*Requested* from a connection; only that person writes it
Profile stats?	Views, connections, search appearances
Companies/jobs?	Members create pages; page admins post jobs; apply once per job
Feed?	Posts from connections and followed members, ranked (course omits it; we add one)

> **Requirements**

Functional (the course's twelve): (1) profile with details, jobs, schooling, skills, accomplishments; (2) search members, companies (and jobs); (3) send/accept/reject connection requests; (4) request recommendations; (5) profile stats; (6) posts to connections; (7) comment on posts, like/share posts **and comments**; (8) messages; (9) notifications for new messages, invitations, comments on your posts; (10) company pages and job postings; (11) create and join groups; (12) follow members or companies.

Non-functional: a connection always appears on **both** members; fast reads (feed, search, degrees); blocked members' content disappears and no spamming strangers; new channels and ranking rules plug in.

Use cases

Actors: **Member**, **Admin** (block/unblock), **System** (notifications). Top cases: profile, search and connect, follow/unfollow, message connections, post/like/comment, notify. Activity diagrams: *add experience* (enter details, optional media, verify, save or ask for missing data) and *send message* (recipients, body, media, verify, deliver, email + push to each recipient).

Core entities

Class	Responsibility
`Member`, `Admin`	Identity, profile, connection set (changed only by the service), follows, posts, inbox, notifications
`Profile`	Owns `Experience`, `Education`, `Skill`, `Accomplishment`, `Recommendation`, `ProfileStats`
`Skill`	Name + set of endorsers
`Recommendation`	Born empty on request; only the named author writes it (REQUESTED → WRITTEN)
`ProfileStats`	Counters bumped by whoever witnesses the event
`SocialContent` → `Post`, `Comment`	Likes as a set; posts own comments and may be shares
`Message` (record)	Not social content — nobody likes a message (text only in our code)
`ConnectionInvitation` + `InvitationStatus`	PENDING → ACCEPTED / REJECTED / CANCELED
`ConnectionService`	Invite, accept, reject, withdraw; BFS degree; mutuals; people-you-may-know
`Company`, `JobPosting`, `JobApplication`, `Group`	Pages, jobs (apply once), groups (owner cannot leave)
`Notification`, `NotificationService`, channels	One class with a *type* field; delivery Strategy
`SearchIndex`	Word → set of members / companies / jobs
`FeedService` + `FeedRanker`	Pull feed with pluggable ranking
`LinkedIn`	Facade

Class diagram

Six course stages:

1. **Profile** — each part is a small class because other code uses it. `Recommendation` is created *without* text (requirement 4 is about asking). The profile never bumps its own `ProfileStats` — whoever sees the event does (an object cannot count events it never sees). All composition.
2. **Posts, comments, messages** — *comments get likes too*, so both extend `SocialContent` holding author, text and **liker ids**. `Message` stays apart.
3. **Invitations and notifications** — an invitation **changes state over time**, so it is its own object that can remember "no". Notification kind is a **field**: **subclass when behaviour differs; field when only data differs.**
4. **Companies, jobs, groups** — a page owns its postings; followers are ids; a group's creator can never leave.
5. **Actors and `ConnectionService`** — accepting edits **two** members, so it belongs to neither; one service keeps connections **symmetric**. Following is one-sided: just a set on the follower.
6. **Search** — the course's real bug: the index stored *one* member per name and broke on the second Sharma. Rule: **an index value is always a collection.** Keys are lowercase words.

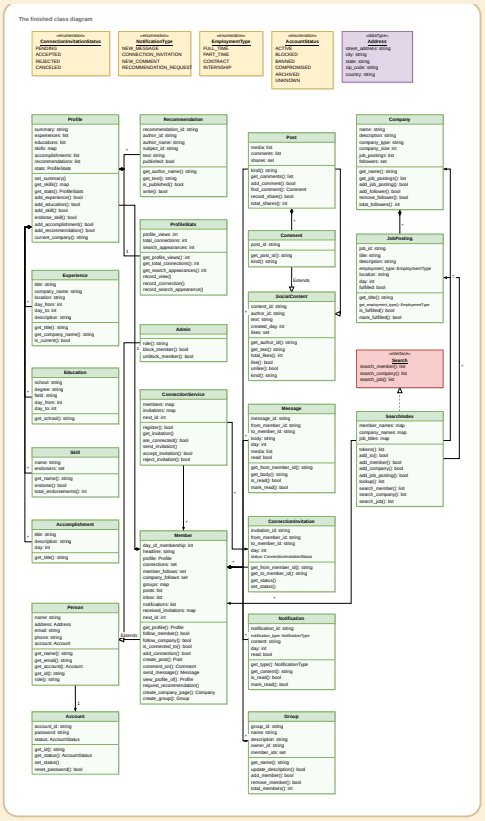
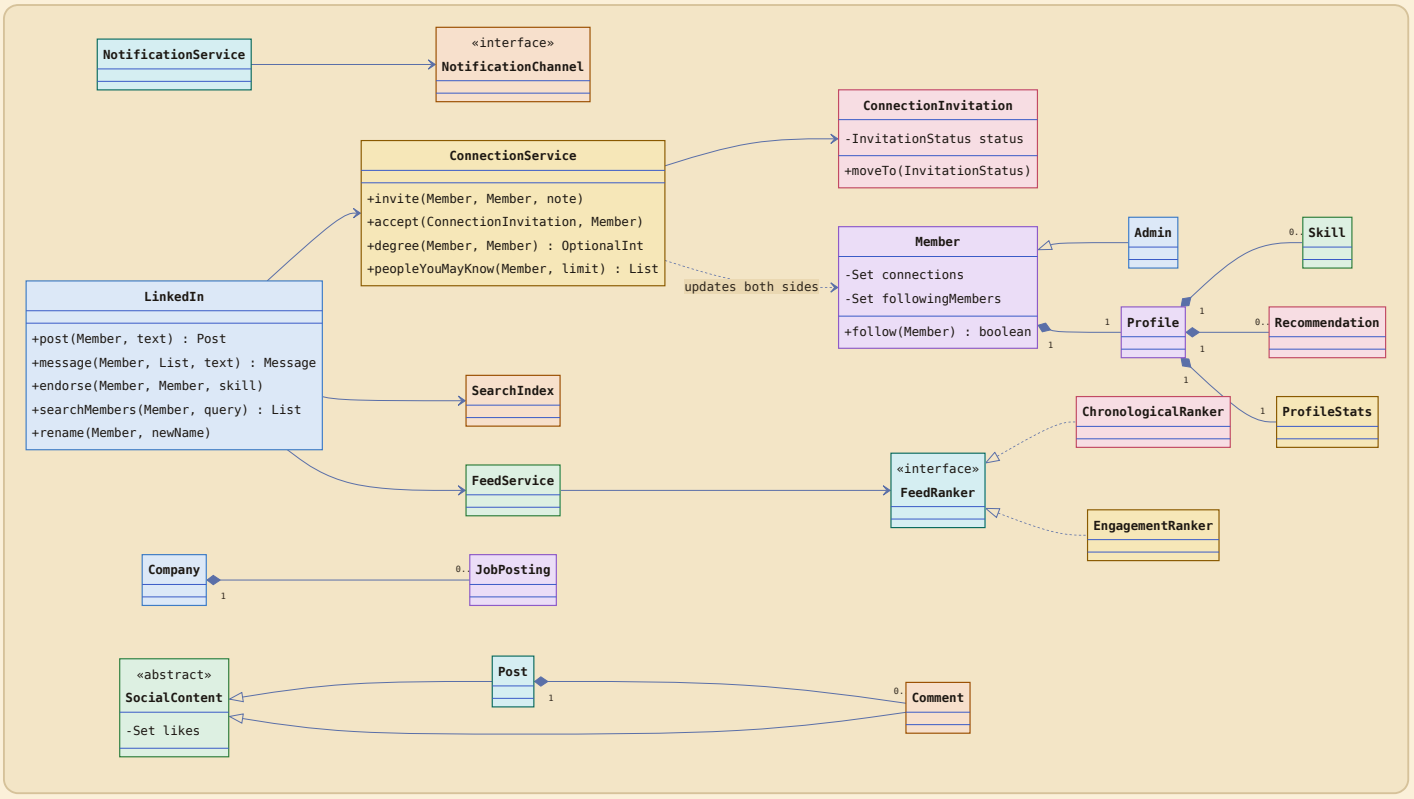


Figure — The finished LinkedIn class diagram (course diagram)

The course omits the feed and ranking, suggestions, degrees, job applications, privacy controls and member-to-member blocking, and admits two weak spots: no feed (pull at load vs copy on write — depends on typical connection count) and a search index that goes stale on rename. **Our Java design** fills these: a pull **feed** with a ranking Strategy, **BFS degrees**, **people you may know**, a **rename that updates the index**, job **applications**, and a connection count derived from the set.



Key flows

Invite and accept: `invite` checks not-self, not-connected, and nothing pending *either way*, creates a PENDING invitation and notifies the receiver. `accept` locks the graph, moves the invitation to ACCEPTED, and adds

each member to the other's set together.

Invitation states: PENDING → ACCEPTED (receiver accepts) / REJECTED (receiver rejects) / CANCELED (sender withdraws); all final.

Degree of connection: BFS (all 1-hop neighbours, then 2-hop…) from the viewer, stopping at depth 3: Asha → Meera 1st, Karan 2nd, Zoya 3rd, Ravi out of network.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
`ConnectionService.accept` updates both under one lock	Service for two-object operation	Symmetry has one guardian	`member.accept(inv)` editing the other
Following = set on follower	Simplicity	One-sided, no agreement	Reusing invitations
Invitation with explicit states	**State** (enum + allowed transitions)	Accept-twice / accept-after-reject rejected	Boolean `accepted`
`SocialContent` base	Inheritance	Comments are liked too	Likes only on `Post`
One `Notification` with `type`	Field when only data differs	Delivery identical	Subclass per event
`NotificationChannel`	**Strategy**	In-app, email, push interchangeable	`if (email) … else …`
`FeedRanker`	**Strategy**	"Recent" vs "Top" at run time	Sorting inside the service
Index key → set	Robust inverted index	Both Sharmas found	Key → single member
`rename` tells the index	Sync derived data at the write point	Search never stale	Nightly rebuild
Count = `connections().size()`	Derive, don't store	Cannot disagree	Separate counter

Java implementation

Package `lld.ch18.linkedin` (`Profile`, `Content`, `Network`, `Pages`, `Services`, `LinkedIn`, `Main`). The heart is `Network.java`:

```java
// File: lld/ch18/linkedin/Network.java (excerpt)
enum InvitationStatus {
    PENDING, ACCEPTED, REJECTED, CANCELED;
    boolean canMoveTo(InvitationStatus next) { return this == PENDING && next != PENDING; }
}
// ... ConnectionInvitation:
    synchronized void moveTo(InvitationStatus next) {
        if (!status.canMoveTo(next)) throw new IllegalStateException("invitation is already " + status);
        status = next;
    }
// ...
final class ConnectionService {
    // ...
    synchronized ConnectionInvitation invite(Member from, Member to, String note) {
        if (from.id() == to.id()) throw new IllegalArgumentException("cannot connect to yourself");
        if (from.isConnectedTo(to)) throw new IllegalStateException(from + " and " + to + " are already
connected");
        if (pendingBetween(from, to)) throw new IllegalStateException("an invitation between " + from + " and
" + to + " is already pending");
        ConnectionInvitation inv = new ConnectionInvitation(from, to, note);
        invitations.put(inv.id(), inv);
        notifications.send(to, NotificationType.CONNECTION_INVITATION, from + " wants to connect: \"" + note +
"\"");
        return inv;
    }

    /** One lock for the whole graph change: both sides are updated together or not at all. */
    synchronized void accept(ConnectionInvitation inv, Member by) {
        if (by.id() != inv.to().id()) throw new IllegalStateException("only " + inv.to() + " can accept");
        inv.moveTo(InvitationStatus.ACCEPTED);
        inv.from().addConnection(inv.to().id());
        inv.to().addConnection(inv.from().id());
    }
    // ...
    /** Degree of connection (1st, 2nd, 3rd) by breadth-first search; empty means "out of network". */
    OptionalInt degree(Member from, Member to) {
        if (from.id() == to.id()) return OptionalInt.of(0);
        Set<Long> seen = new HashSet<>(Set.of(from.id()));
        Deque<long[]> queue = new ArrayDeque<>();                // {memberId, depth}
        queue.add(new long[]{from.id(), 0});
        while (!queue.isEmpty()) {
            long[] cur = queue.poll();
            if (cur[1] == 3) continue;                             // LinkedIn stops at 3rd degree
            for (long next : lookup.apply(cur[0]).connections()) {
                if (next == to.id()) return OptionalInt.of((int) cur[1] + 1);
                if (seen.add(next)) queue.add(new long[]{next, cur[1] + 1});
            }
        }
        return OptionalInt.empty();
    }
    // ... mutualConnections, peopleYouMayKnow
}
```

Full runnable code: code/lld/ch18/linkedin/ (see the Full Edition).

```
  REJECTED: an invitation between Ravi Sharma and Asha Sharma is already pending
degree asha->meera=1, asha->karan=2, asha->zoya=3, asha->ravi=out of network
people asha may know: [Karan Mehta]
search 'sharma' (by meera): [Asha Sharma, Ravi Sharma]
search 'khan' after rename: [], search 'ali': [Zoya Ali]
feed(meera) hides blocked Karan's share: ["Tip: prefer composition over inheritance." by Asha Sharma]
```

Key points: Ravi's counter-invite is refused (pending *either* direction). A second accept fails because only PENDING can move. `peopleYouMayKnow` counts mutuals for 2nd-degree members, skips pending invitations, sorts

by count then name. `endorse` needs a 1st-degree connection and rejects repeats via the endorser set; `Recommendation.write` accepts only the named author, once. `rename` removes old words before adding new ones. `FeedService` is a **pull** feed: Karan sees Asha's tip because he *follows* her.

Concurrency & edge cases

- **Symmetric connections:** all graph changes are `synchronized` on the one service. Distributed: write both edges in one transaction, or one row keyed by the ordered id pair.
- **Double invitations** either way refused while pending.
- **Likes, endorsements, applications:** id-keyed sets/maps with `synchronized` mutators.
- `Member` **collections** are concurrent and getters return copies.
- **Stale search:** handled by `rename` — "whatever changes the data tells the index" (events in production).
- **Blocked members:** writes refused, posts vanish from feeds. **Validation:** experience ending before it starts, blank posts rejected.

Extensions & follow-up questions

- **Feed for 900 million members — fan-out on write (push)**: append post ids to each connection's precomputed feed (fast reads, costly for huge followings). **Fan-out on read (pull)**: assemble on open (ours; cheap writes, slower reads). **Hybrid**: push for ordinary members, pull for very popular accounts, merged at read.
- **People you may know at scale** — compute offline (friends-of-friends + shared companies, schools), store, read live.
- **Connection limit** — 30,000; enforce in `accept`.
- **InMail** — a premium credit balance spent by `message`.
- **Who viewed my profile** — store view events, derive the count.
- **Job recommendations** — a Strategy matching skills and title to postings.

Real-world example

LinkedIn shows 1st/2nd/3rd-degree badges in search (our BFS depth) and "People You May Know" from shared connections. Messaging non-connections usually needs paid **InMail**. Its event stream gave rise to **Apache Kafka**, open-sourced in 2011.

In the interview

Minutes	What to do
0–5	Connect vs follow, degrees, who can message and endorse
5–15	Profile composition, `SocialContent`, invitation with state
15–25	**Say it:** accept changes two objects → `ConnectionService`; follow needs no service
25–35	Search (key → list), BFS degrees, people you may know
35–45	Feed pull/push/hybrid, notification Strategy, concurrency

30-second answer: "Members own profiles made of small parts. Posts and comments share a base class because both are liked, with likes as a set. A connection request is an object with states. Accepting changes both members, so it lives in a `ConnectionService` that updates both sides under one lock; following is just a set. Search is an inverted index mapping each word to a list. Degrees come from a BFS to depth 3. The feed is pull with a pluggable ranking Strategy, moving to a push/pull hybrid at scale."

- *Why not store `connectionCount`?* — It is `connections.size()`; a copy drifts.
- *2nd-degree badge for every search result quickly?* — Cache each member's 1st-degree set; a result is 2nd-degree if its set intersects yours.

Self-check

- **Q:** Why is accepting not a `Member` method? **A:** It updates two members; a service keeps both consistent.

- **Q:** Connect vs follow? **A:** Mutual with acceptance vs one-sided and immediate.
- **Q:** Why does `Recommendation` start without text? **A:** It is requested first; the author fills it later.
- **Q:** Which feed model is `FeedService`, and its cost? **A:** Pull; cheap writes but every read scans connections' posts.

TL;DR

LinkedIn = profile composition + `SocialContent` (likes as sets) + invitation **State** + `ConnectionService` for the two-sided accept (following is a set) + BFS degrees + people-you-may-know + inverted-index search (key → list) + notification and feed-ranking **Strategies** + a facade.

18.4 Design Facebook

Facebook: **friends** (mutual), **follows** of people and **pages**, **groups**, posts. New challenges versus LinkedIn: **privacy** (who may see each post), a real **news feed**, **pages** with ratings, public/private **groups**, and **friend suggestions**.

Problem & clarifying questions

Question	Answer we assume
Friend vs follow?	Friendship mutual (request + accept); following one-sided
Who sees a post?	Author picks an **audience**: Public, Friends, custom **privacy list**, Only me
Privacy list?	A named subset of the author's friends ("Close friends")
Pages?	Created by members; followed; **recommendations** (1–5 rating + text)
Groups?	Public (join at once) or private (owner approves; posts visible only to members)
Search?	Members, groups, pages by name; posts by word — respecting privacy
Feed?	Friends, followed members/pages, joined groups — privacy-filtered, ranked
Extended?	Suggest people you may know

Requirements

Functional (twelve + extended): (1) profile — details, jobs, schooling, places; (2) search members, groups, pages; (3) friend requests; (4) follow without friendship; (5) create groups/pages, join groups, follow pages; (6) posts to friends; (7) comments; posts **and comments** liked or shared; (8) privacy lists; (9) messages; (10) page recommendations; (11) notifications for messages, friend requests, comments; (12) search posts by word; (13) **extended:** suggest connections.

Non-functional: privacy never leaked (feed, search, comments, likes, shares); fast feeds for hundreds of friends; symmetric friend graph; stable (non-reshuffling) suggestions and feeds.

Use cases

Actors: **Member**, **Admin** (block/unblock), **System** (notifications). Top cases: profile, search, friend requests, follow/unfollow, message friends, post/like/comment on visible posts, notifications, create page, recommend page. Activity *create post*: text, optional media, **choose the privacy list**, verify, publish to that audience, notify.

Core entities

Class	Responsibility
`Member`, `Admin`	Identity, profile, follows, groups, posts, privacy lists, inbox, notifications
`Profile` (+ `Work`, `Education`, `Place`)	Defaults for every field
`PrivacyList`	Owner + named set of friend ids
`Audience` (sealed) → `Public`, `Friends`, `Custom(list)`, `OnlyMe`	Who may see a post
`SocialContent` → `Post`, `Comment`	Likes as set; posts own comments; may belong to a group/page or be a share
`FriendRequest` + `FriendService`	Friend graph, `areFriends`, `pendingBetween` (both ways)
`VisibilityService`	The single `canSee(viewer, post)`
`Page` (+ `PageType`, `PageRecommendation`)	Followers, one recommendation per member, average rating
`Group` (+ `GroupPrivacy`)	Members, join requests, owner approval, posts
`NotificationService` + `PushChannel`/`EmailChannel`	Routes each type to channels
`SearchIndex`	Names; post words — `searchPosts` takes the viewer
`NewsFeedService` + `FeedRanker`; `SuggestionService` + `Suggestion`	Feed; friends-of-friends
`Facebook`	Facade

Class diagram

Five course stages plus the extended requirement:

1. **Profile** — `Work`, `Education`, `Place` in lists; every field has a **default** so a new account already works.
2. **Posts, privacy, messages, notifications** — fixes three old bugs: **store likers, not a like count** (an old constructor took the count); **comments get likes too**; **the post checks its own privacy list** — but it cannot know friendships (stage 5 supplies that). `Post` → `PrivacyList` is `0..1`; one list serves many posts. `Notification` abstract with a subclass per delivery method.
3. **Pages and groups** — *no member-count field* (an old version stored both count and members). Rule: **if a number can be computed by counting a collection, keep the collection and count it.**
4. **Actors and friend graph** — an old invitation forgot the *sender*. Acceptance changes two members → `FriendService`, which owns the graph and answers "friends?" and "pending?". Following is a set, no service.
5. **Search** — **`searchPost` takes the viewer**: privacy applies in search. Visibility needs the post's list *and* the friend graph; neither answers for the other. Keys map to collections.
6. **Extended — suggestions** — friends of friends **scored by mutual friends**; never yourself, a friend, or anyone with a request pending **in either direction**; sort by mutual count then id so equal scores don't reshuffle. Arrives as two new classes, no edits (OCP).

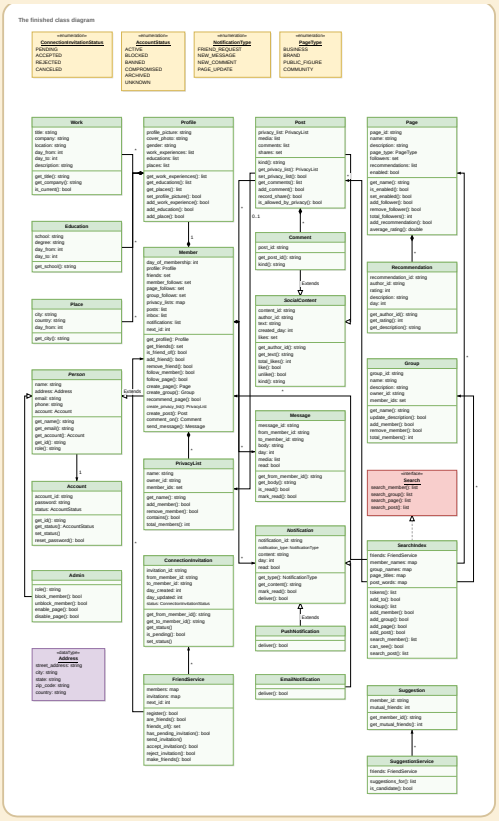
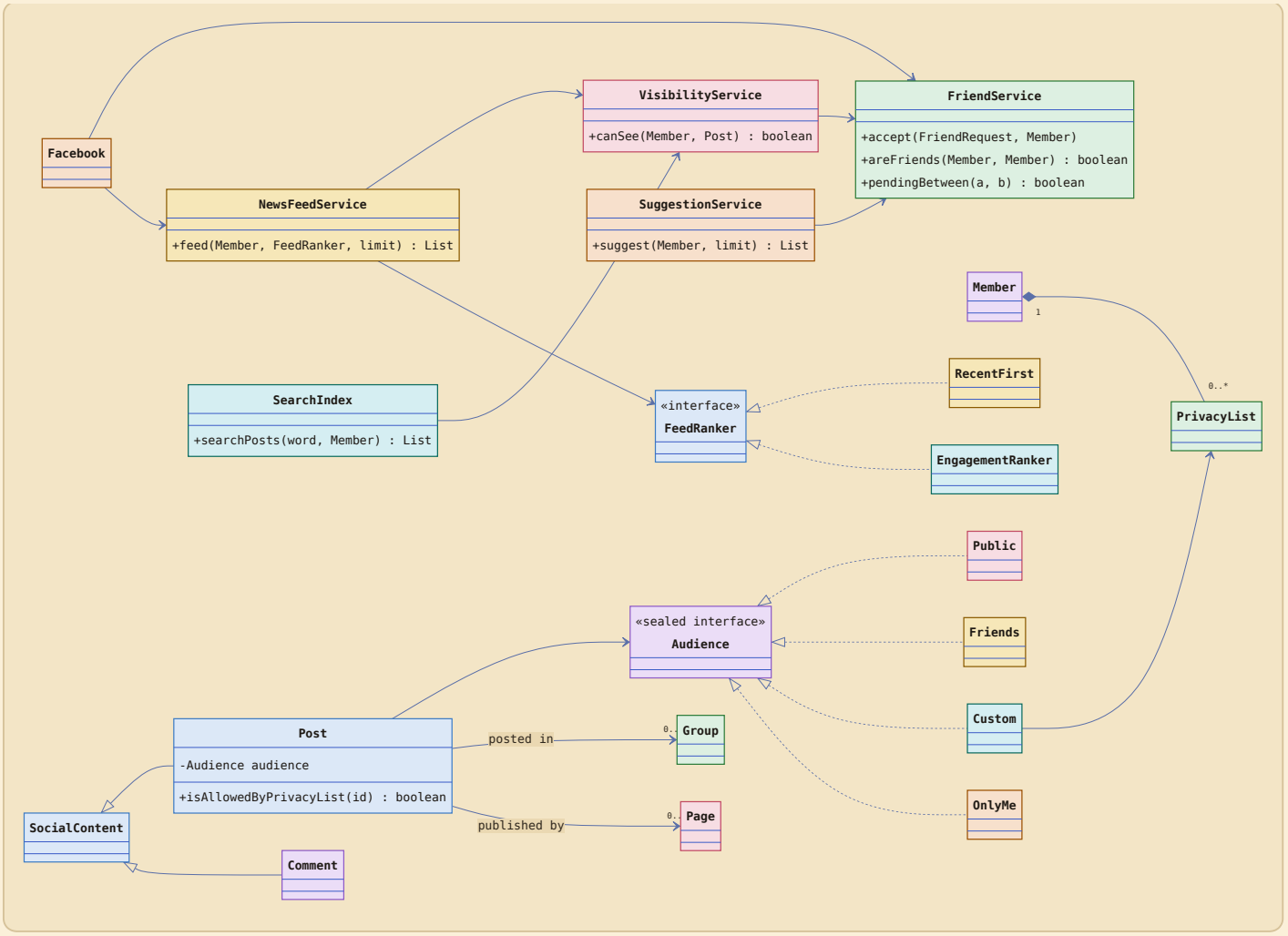


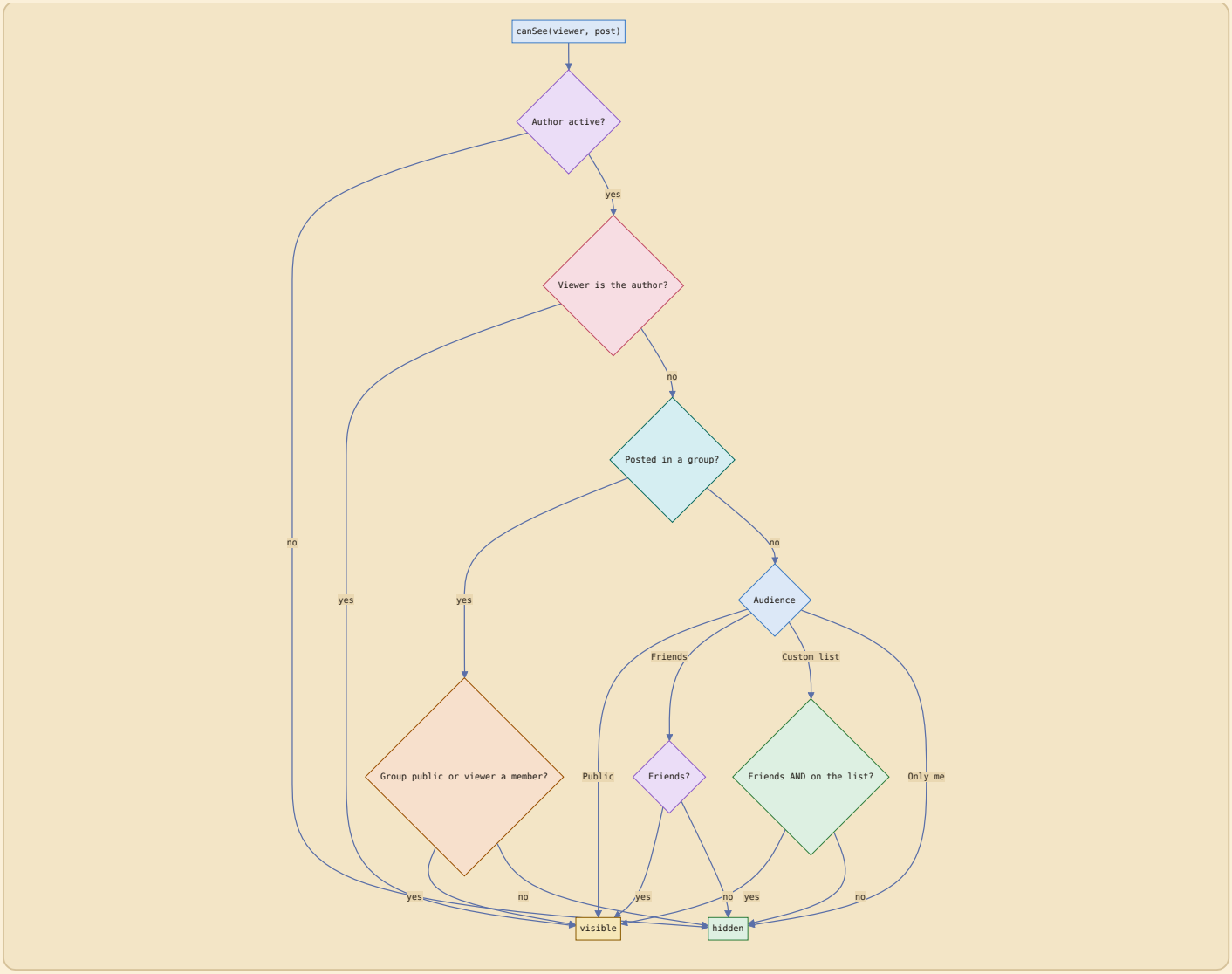
Figure — The finished Facebook class diagram (course diagram)

Our Java design adds a sealed `Audience`, one **`VisibilityService`** used by feed, search, comments, likes and shares, a **`NewsFeedService`** with ranking Strategies, private groups with approval, and page posts in the feed. The course deliberately omits the feed, member blocking, non-like reactions, tagging, events, marketplace, stories and albums with separate privacy; we add the feed and sketch the rest under Extensions.



Key flows

Can this viewer see this post? (feed, search, comment, like, share):



News feed (pull): friends + followed members + followed pages + joined groups → collect posts, drop own and duplicates → filter with `canSee` → sort with the ranker (recency, or engagement × affinity / age) → top N.

Friend suggestions: for each friend F, for each friend C of F: skip if C is me, already a friend, or pending either way; else `mutual[C] += 1`; sort by mutual desc, then id; take top N.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
`VisibilityService.canSee` everywhere	Single policy point (Specification-like)	Search and feed cannot leak	Checks per screen
Sealed `Audience` records	Algebraic data type, exhaustive `switch`	New audience forces every decision to handle it	Int/string codes
`Post.isAllowedByPrivacyList` + `FriendService.areFriends`	Information Expert	Each answers what it knows	Post holding the whole graph
`FriendService` owns the graph	Service for two-sided ops	Symmetry; "pending either way" in one place	`member.acceptFriend()`
Likes as a set; no stored counts	Store who; derive, don't store	No fake/double likes; counts can't drift	`likeCount`, constructor counts
`NotificationService` routes types → channels	**Strategy** + config map	Friend requests push + email; comments push only	`if/else` on type
`FeedRanker`	**Strategy**	Recent vs ranked	Ranking inside the feed
`SuggestionService` added without edits	OCP	Extended requirement = two new classes	Fields on `Member`
`Facebook`	**Facade**	One entry, status checks	UI calling every service

Java implementation

Package `lld.ch18.facebook` (`Model`, `PagesAndGroups`, `Services`, `Facebook`, `Main`).

```java
// File: lld/ch18/facebook/Model.java (excerpt)
sealed interface Audience {
    record Public() implements Audience {}
    record Friends() implements Audience {}
    record Custom(PrivacyList list) implements Audience {}
    record OnlyMe() implements Audience {}
}
// ... in Post:
    /** The half of the privacy rule the post CAN answer on its own. */
    boolean isAllowedByPrivacyList(long viewerId) {
        return !(audience instanceof Audience.Custom c) || c.list().contains(viewerId);
    }

// File: lld/ch18/facebook/Services.java (excerpt)
final class VisibilityService {
    private final FriendService friends;
    VisibilityService(FriendService friends) { this.friends = friends; }

    boolean canSee(Member viewer, Post post) {
        Member author = post.author();
        if (author.status() != AccountStatus.ACTIVE) return false;
        if (viewer.id() == author.id()) return true;
        if (post.group().isPresent()) {
            Group g = post.group().get();
            return g.isPublic() || g.isMember(viewer.id());
        }
        return switch (post.audience()) {
            case Audience.Public p -> true;
            case Audience.Friends f -> friends.areFriends(viewer, author);
            case Audience.Custom c -> friends.areFriends(viewer, author) &&
post.isAllowedByPrivacyList(viewer.id());
            case Audience.OnlyMe o -> false;
        };
    }
}
// ...
    List<Suggestion> suggest(Member me, int limit) {
        Set<Long> mine = friends.friendsOf(me.id());
        Map<Long, Integer> mutual = new HashMap<>();
        for (long f : mine) {
            for (long candidate : friends.friendsOf(f)) {
                if (candidate == me.id() || mine.contains(candidate) || friends.pendingBetween(me.id(),
candidate)) continue;
                mutual.merge(candidate, 1, Integer::sum);
            }
        }
        return mutual.entrySet().stream()
                .map(e -> new Suggestion(members.apply(e.getKey()), e.getValue()))
                .sorted(Comparator.comparingInt(Suggestion::mutualFriends).reversed()
                        .thenComparingLong(s -> s.member().id()))           // stable tie-break
                .limit(limit)
                .toList();
    }
```

Full runnable code: code/lld/ch18/facebook/ (see the Full Edition).

```
  Alice Rao  public=true  friends=true  closeFriends=true
  Bob Das    public=true  friends=true  closeFriends=true
  Erin Shah  public=true  friends=true  closeFriends=false
  Frank Lee  public=true  friends=false closeFriends=false
search posts 'party' as Bob:  ["Planning a surprise party for Erin" by Alice Rao]
search posts 'party' as Erin: []
before: [Dave Paul(2 mutual), Grace Lee(1 mutual)]
after inviting Grace: [Dave Paul(2 mutual)]
```

Key points: the visibility table is the course's key scenario — Bob is a friend *and* on "Close friends", Erin only a friend, Frank neither. Search respects privacy because `searchPosts` takes the viewer. `createPrivacyList` refuses non-friends. Comments, likes and shares also call `canSee` (otherwise "comment" is a side door). Pages allow one recommendation per member (`putIfAbsent`); count and average are computed. A private group's join waits for approval and its posts are hidden from non-members, even in search. Once Frank is blocked his posts vanish from feeds. Notification routing is a `Map<NotificationType, List<NotificationChannel>>`.

Concurrency & edge cases

- **Friend graph:** `sendRequest`, `accept`, `reject`, `unfriend` `synchronized`; concurrent adjacency sets. In a DB, one row keyed by `(min(id), max(id))` makes symmetry automatic.
- **Pending both ways:** `pendingBetween(a, b)` for requests and suggestions.
- **Privacy changes after posting:** visibility computed at read time, so unfriending hides posts at once (a push feed must still re-check `canSee` on read).
- **Shares:** need to see the original; production should cap the share's audience at the original's.
- **Deterministic ordering:** suggestions tie-break by id, feeds by creation time; a logical `Tick` clock makes output repeatable.
- **Blocked/deleted authors:** `canSee` returns false.

Extensions & follow-up questions

- **Scale the feed** — fan-out on write vs on read; mix (push for normal users, pull for celebrities), always re-check privacy.
- **Better ranking** — affinity, content type, recency decay behind `FeedRanker`: early **EdgeRank** (affinity × weight × time decay).
- **Suggestions at scale** — precompute offline (the original diagram's `memberSuggestions` field), add shared groups/schools/work.
- **Member blocking** — per-member block set checked both ways by `canSee` and `sendRequest`.
- **Reactions** — `Map<memberId, Reaction>`, like Stack Overflow's vote map.
- **Tagging** — tagged ids on `Post`, notify, optionally widen audience.
- **Events, stories, marketplace, albums** — new `SocialContent` types with their own `Audience`.

Real-world example

Facebook launched **News Feed** in 2006. Posts carry an audience selector, a profile is capped at **5,000 friends** (then followers or pages), and the graph is served by **TAO**, a distributed store for objects and associations.

In the interview

Minutes	What to do
0–5	Friend vs follow, audiences, pages, groups, feed
5–15	Profile, `SocialContent`, `PrivacyList`, `Audience`
15–25	`FriendService` owns the graph; follow is a set
25–35	**Strongest point:** `canSee` consults post list + friend graph; every read path calls it
35–45	Feed models, suggestions, concurrency

30-second answer: "Posts and comments share a likes set. A post has an audience — public, friends, a custom list, or only me. The post checks its own list, but only the friend service knows friendships, so one `VisibilityService.canSee` asks both, and every read path — feed, search, comment, like, share — calls it. The friend service owns the graph because accepting changes two members; following is one-sided. The feed gathers from friends, follows, pages and groups, filters by `canSee`, and ranks with a Strategy. Suggestions are friends of friends by mutual count, excluding pending requests either way, with a stable tie-break."

- *How could a private post leak?* — Any read path that forgets privacy: search, feed, comments, shares, notifications.
- *Why the second sort key?* — Equal mutual counts would swap between loads; a self-reshuffling list looks broken.

Self-check

- **Q:** Erin is a friend but not on "Close friends" — can she see a close-friends post? **A:** No; friend *and* on the list.
- **Q:** Why does `searchPosts` take a viewer? **A:** Privacy applies inside search.
- **Q:** Who is excluded from suggestions? **A:** Yourself, existing friends, anyone pending either way.
- **Q:** Push vs pull feed weaknesses? **A:** Push is expensive for huge audiences; pull reads are slower.

TL;DR

Facebook = LinkedIn's structure + **privacy**: sealed `Audience`, privacy lists of friends, one `VisibilityService.canSee` (post's list AND friend graph) called by feed, search, comments, likes and shares. `FriendService` owns the symmetric graph; following is a set. Pages and groups store collections, not counts. The feed gathers, filters, ranks with a Strategy; suggestions are friends-of-friends by mutual count, excluding pending both ways, stable tie-break.

18.5 Chapter summary

The four designs side by side

Aspect	Stack Overflow	Cricinfo	LinkedIn	Facebook
Core content	Question / answer / comment	Ball / over / innings / match	Post / comment / message	Post / comment / message / page & group posts
User relationship	None (votes, reputation)	Team membership, officials	Connection + follow	Friendship + follow
Two-sided op → service	—	—	`ConnectionService.accept`	`FriendService.accept`
Roles	Reputation privileges + Moderator/Admin	Admin scorer, Commentator	Member / Admin	Member / Admin + audiences
Derive, don't store	Score, tag counts	All stats from balls	Connection, follower counts	Member/follower counts, ratings
Strategy	Reputation policy, channels	(Target calculator, extension)	Feed ranker, channels	Feed ranker, channels
Observer	Reputation listeners	Match observers	Notifications	Notifications
Template Method	—	`StatQuery`	—	—
Search	Words and tags	Stat queries	Members, companies, jobs	Members, groups, pages, privacy-aware posts

Common mistakes across all four

- Counters instead of sets — cannot stop duplicates.
- Stored counts beside the collections they summarise — they drift.
- Two-sided operations on one entity — asymmetric graphs.
- Privacy or permission checks in one read path but not another.
- Index values that are a single object instead of a list.
- Modelling nouns but not rules (six *legal* balls; run outs; byes).

Chapter TL;DR

Platforms are rich domain models. Share behaviour in a content base class; keep votes and likes as sets of ids; derive every number you can; put operations that change two objects in a service; notify through Observer with Strategy channels; rank feeds with a Strategy; keep search in an inverted index mapping words to lists; and write each permission or privacy rule once, then call it from every read path.

Chapter 19 — Case Studies: Commerce & Marketplaces (Part 2 of Platforms) — Study Edition

> **Where this fits.** Chapter 18 designed platforms where people *talk*. This chapter designs platforms where people *buy, sell and trade*. Three pressures shape every design: **money must be exact, state must follow rules, and shared things must not be double-sold.** This is the study edition: every topic of the full chapter, without the padding.

#	Problem	The one idea that makes it work	Main patterns
19.1	Amazon — online shopping	An order is an immutable copy of the cart, moved through a **state machine**	State machine (enum), Strategy (payment), Observer, Facade
19.2	Stock brokerage + exchange matching engine	Four order types differ in only **two questions**; a book matches by **price, then time**	Template Method, Observer, Strategy (price policy)
19.3	Online auction	"Read highest bid, compare, replace" is **one atomic step**	Monitor lock, Observer, Specification
19.4	Music streaming (Spotify)	Play order is a **pluggable strategy**; plan rules live **in the plan**	Strategy, Observer, enum-with-behaviour
19.5	Food delivery (Swiggy) + FoodKart	Capacity and riders are **claimed atomically**; selection is a **strategy**	Strategy, Observer, State machine, CAS
19.6	Grocery store (POS & checkout)	A **pure pricing engine**: best offer per product, then a coupon spread over items so tax and refunds stay exact	Strategy (offers, tax, payment), Facade, Observer, enum state machine

Running the code: `cd chapters/19-case-platforms-2/code && javac --release 21 -d /tmp/lld_19 $(find . -name '*.java') && java -cp /tmp/lld_19 lld.ch19.amazon.Main` (or `brokerage`, `stockexchange`, `auction`, `music`, `fooddelivery`, `foodkart`, `grocery`). Java 21, standard library only; the stock-exchange `Main` also runs a small test suite. One file may hold several small package-private types.

Four rules that apply to every design in this chapter

1. **Money is `BigDecimal`, never `double`.** A `double` cannot store 0.1 exactly (`0.1 + 0.2` = `0.30000000000000004`). Build it from a `String` (`new BigDecimal("0.10")`) and compare with `compareTo`, not `equals` — `new BigDecimal("2.0").equals(new BigDecimal("2.00"))` is `false` because the scales differ.
2. **Status changes follow a state machine** — a fixed list of states plus allowed moves, kept *inside the enum*, so an illegal move ("cancel a delivered order") throws at once.
3. **Check-then-act on shared stock must be atomic.** "One left? Take it" is two steps; two threads can both see "yes". Do both under one lock, or with one compare-and-set (CAS: "set to B only if it is still A").
4. **Copy prices into orders.** Later catalog or menu price changes must not change old orders.

Design Amazon — Online Shopping System

Problem & clarifying questions

We design Amazon's **online retail part**: people list products, others find them, fill a cart, pay and track the parcel.

Question	Assumed answer
Buy without an account?	No. Browsing and carts are open; ordering needs a **registered member**
Who sells?	Members can list products (a marketplace)
Payment?	Credit card (clears at once) or bank transfer (clears later)
Cancel?	Yes, until shipped
Notifications?	On every order or shipment status change
Returns, wish lists, payouts, recommendations, multi-warehouse stock?	Out of scope — say so out loud

Requirements

Functional: (1) members add products to sell; (2) anyone searches by **name** or **category**; (3) anyone views, only members buy; (4) add/remove/change cart items; (5) check out the whole cart; (6) rate and review; (7) shipping address; (8) cancel an unshipped order; (9) notified on each order/shipment status change; (10) pay by card or bank transfer; (11) track shipment and order state.

Non-functional: two buyers never both get the last unit; exact `BigDecimal` money; orders and shipments keep a **history** of states; a new payment method or channel must not change checkout.

Use cases

Actors: **Admin** (block/unblock accounts, manage categories), **Guest** (search, cart, register), **Member** (guest + order + sell), **System** (notifications). In the use-case diagram, "Checkout" *includes* "Make payment" and "Add shipping address"; search by name/category *extend* "Search product".

Core entities

Class	Responsibility
`ProductCategory`	Named group; exists without products
`Product`	Price, seller, stock; **owns** reviews; guards stock (`reserve` / `release`)
`ProductReview`	Rating 1–5 + text; stores reviewer **by id** (no `Product ↔ Member` dependency)
`Search` / `Catalog`	Name-word and category indexes, both updated in `addProduct`; points to, never owns, products
`ShoppingCart` / `Item`	Mutable cart owning its lines; `Item` is an immutable record (product id, name, qty, unit price)
`Order` / `OrderLog`	Immutable **copy** of cart lines + address + status on legal edges; one log entry per change
`Payment` (`CreditCardPayment`, `BankTransferPayment`)	Card clears now; transfer returns `PENDING`
`Shipment` / `ShipmentLog`	Parcel + tracking history; last entry = "where is it now"
Notification channel (`EmailChannel`, `SmsChannel`)	One delivery way (course: abstract `Notification` with subclasses; same idea)
`Account`, `Customer` → `Guest`, `Member`, `Admin`	`Customer` has only what *anyone* may do; ordering lives on `Member`
`OrderService`	Turns a cart into an order; drives the state machine

Class diagram

The course builds it in five stages (products → catalog → payment/shipment/notification → cart/order → accounts). Filled diamonds are composition (`Product` owns reviews, cart owns items, order owns log and shipment); plain arrows are associations (order only *refers to* its payment); `Guest` and `Member` extend `Customer`.

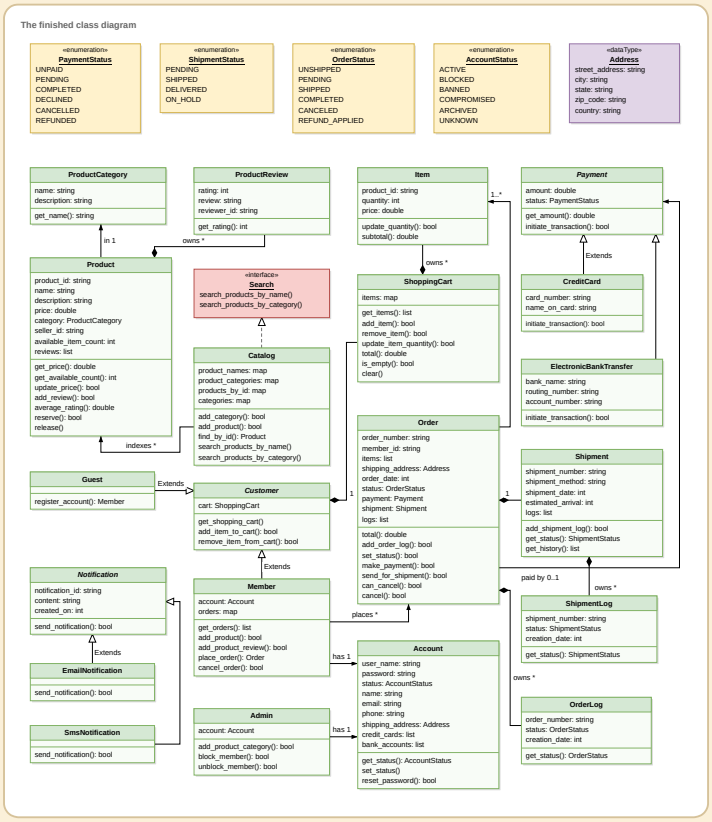
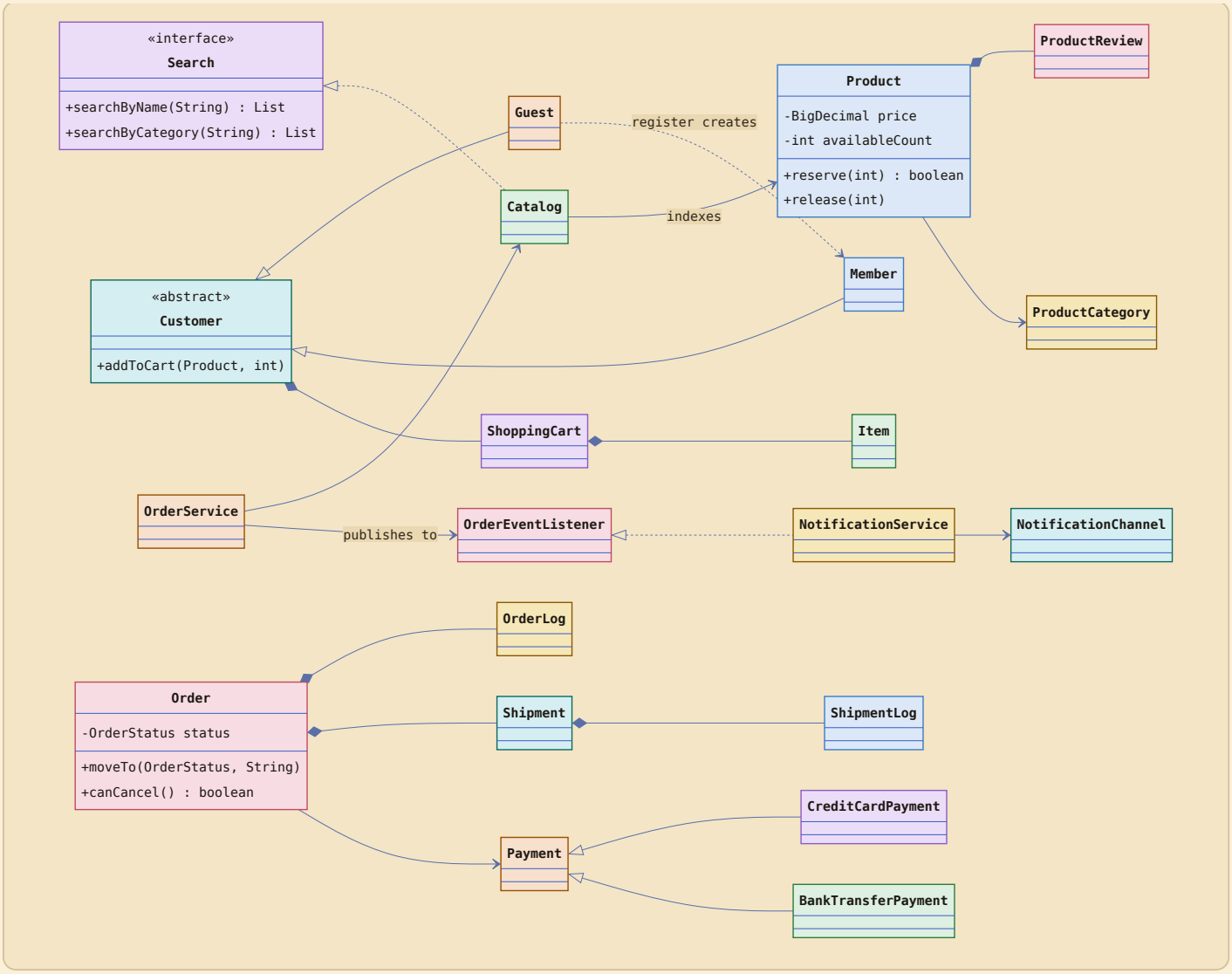


Figure — The finished class diagram of the online shopping system (course diagram)



Key flows

The course's activity diagram is the shopping loop (search, view, add, update cart, "more shopping?", checkout); its sequence diagrams cover search, add-to-cart, and **checkout**: get cart items → billing/shipping info → create order → verify items → pay → create shipment → return status (with error paths).

Checkout (`OrderService.placeOrder`) follows that order, but stock is **reserved** during verification, not just checked: member active and cart not empty → reserve each line (any failure releases what was reserved and fails) → new `Order` copying the lines → `payment.process()`: COMPLETED → `UNSHIPPED` + notify; PENDING (transfer) → notify PENDING; DECLINED → release stock, `CANCELED`.

Order state machine (`OrderStatus.next()`): PENDING → UNSHIPPED (paid) or CANCELED; UNSHIPPED → SHIPPED or CANCELED; SHIPPED → COMPLETED; CANCELED → REFUND_APPLIED. Requirement 8 is one line — `canCancel()` returns `status.canMoveTo(CANCELED)`; there is no `SHIPPED → CANCELED` edge.

Design decisions & patterns used

Decision	Why	Alternative rejected
Ordering on `Member`, not `Customer`	A guest *is a* customer; inheriting `placeOrder` it must reject breaks **LSP**	`Customer.placeOrder()` that throws for guests
`Guest.register()` returns a new `Member` carrying the cart	The cart survives registration	Mutating a guest (Java objects can't change class)
`Order` copies cart lines	A finished purchase must not change	Shared `Item` objects
Stock reserved at order time	No two buyers get the last unit (abandoned orders hold stock → timer release)	Check, pay, then decrement (race window)
State machine inside `OrderStatus`	One owner of the rules	Scattered `if`s
Sealed abstract `Payment`, a subclass per method (**Strategy**)	Methods *behave* differently; new method = new subclass (OCP); that is why it is not an enum	`enum PaymentType` + `switch`
Payment passed as `Function<BigDecimal, Payment>`	Total known only after building the order	Creating payment before the total
Observer for notifications	New channel = new `NotificationChannel`	Email/SMS code inside `OrderService`
`Shipment` keeps a log; reviews store `reviewerId`	History for tracking/disputes; no two-way dependency	Single status field; review holding `Member`
`OrderService` as **Facade**	Hides stock, payment, shipment, notification	Callers orchestrating five objects

Java implementation

Package `lld.ch19.amazon`: `Money.java`, `Catalog.java`, `Order.java`, `Payments.java`, `Accounts.java`, `OrderService.java`, `Main.java`.

```java
// File: lld/ch19/amazon/Order.java (excerpt)
enum OrderStatus {
    PENDING, UNSHIPPED, SHIPPED, COMPLETED, CANCELED, REFUND_APPLIED;

    private Set<OrderStatus> next() {
        return switch (this) {
            case PENDING -> EnumSet.of(UNSHIPPED, CANCELED);      // waiting for payment
            case UNSHIPPED -> EnumSet.of(SHIPPED, CANCELED);      // paid, not yet handed to carrier
            case SHIPPED -> EnumSet.of(COMPLETED);                // too late to cancel
            case CANCELED -> EnumSet.of(REFUND_APPLIED);          // money goes back if it was taken
            case COMPLETED, REFUND_APPLIED -> EnumSet.noneOf(OrderStatus.class);
        };
    }

    boolean canMoveTo(OrderStatus target) { return next().contains(target); }
}

// File: lld/ch19/amazon/Catalog.java (excerpt, in Product)
    synchronized boolean reserve(int quantity) {
        if (quantity <= 0 || quantity > availableCount) return false;
        availableCount -= quantity;
        return true;
    }

// File: lld/ch19/amazon/OrderService.java (excerpt)
    Order placeOrder(Member member, Address address, Function<BigDecimal, Payment> paymentMethod) {
        if (!member.canBuy()) throw new IllegalStateException("account " + member.account().status());
        List<Item> items = member.cart().snapshot();
        if (items.isEmpty()) throw new IllegalStateException("cart is empty");

        List<Item> reserved = new ArrayList<>();
        for (Item i : items) {
            Product p = catalog.find(i.productId()).orElseThrow();
            if (!p.reserve(i.quantity())) {
                reserved.forEach(r -> catalog.find(r.productId()).orElseThrow().release(r.quantity()));
                throw new IllegalStateException("out of stock: " + i.name());
            }
            reserved.add(i);
        }

        Order order = new Order("ORD-" + orderSeq.incrementAndGet(), member.id(), items, address);
        orders.put(order.orderNumber(), order);

        Payment payment = paymentMethod.apply(order.total());
        order.attachPayment(payment);
        switch (payment.process()) {
            case COMPLETED -> { transition(order, OrderStatus.UNSHIPPED, "paid by " + payment.describe());
member.cart().clear(); }
            case PENDING -> { publish(order, OrderStatus.PENDING); member.cart().clear(); }
            default -> {                                  // declined: give the stock back, keep the cart
                releaseStock(order);
                transition(order, OrderStatus.CANCELED, "payment declined");
            }
        }
        return order;
    }
```

Full runnable code: code/lld/ch19/amazon/ (see the Full Edition).

```
== 6. Illegal transition is rejected by the state machine ==
  ORD-1002: REFUND_APPLIED -> SHIPPED is not allowed
== 9. Ten members race for the last copy ==
  winners=1 losers=9 stock left=0
```

The demo also shows: name search matches every query word (`usb cable` finds "USB-C Cable"); the guest's cart survives registration; each status change goes to email + SMS; the cancel path `PENDING → UNSHIPPED → CANCELED → REFUND_APPLIED` returns stock; a blocked member is rejected; ten threads released together by a `CountDownLatch` produce exactly one winner.

Concurrency & edge cases

Risk	How the code prevents it
Last unit, two buyers	`Product.reserve` is `synchronized` (check + decrement in one step)
Multi-line cart, one line out of stock	`placeOrder` releases everything it reserved (all-or-nothing)
Cancel and ship at the same time	Both lock the `Order` around check-then-move
Payment declined	Stock released, order `CANCELED`, cart kept for retry
Bank transfer never clears	A timeout job cancels old `PENDING` orders
Price changes after adding to cart	`Item` stores unit price; production re-validates at checkout
Blocked member	`Member.canBuy()` checks `ACTIVE`
Deadlock	Each lock held briefly, never while waiting for another in opposite order

Real-world example

Amazon offers *Cancel items* only before the order enters shipping — our missing `SHIPPED → CANCELED` edge. Its 2007 **Dynamo** paper chose availability so "add to cart" always works during failures, merging conflicting cart versions later. At scale, cart, catalog and orders are separate services.

Extensions & follow-up questions

- **New payment (gift card, UPI)** — `GiftCardPayment extends Payment` (add to `permits`); `OrderService` untouched.
- **Last unit, distributed** — `UPDATE ... SET stock = stock - 1 WHERE id = ? AND stock >= 1`, or optimistic locking with a version column.
- **Abandoned orders** — reservation expiry; a job cancels `PENDING` orders after ~30 minutes.
- **Returns** — `COMPLETED → RETURN_REQUESTED → RETURNED → REFUND_APPLIED` (local enum change).
- **Multiple warehouses** — `Inventory` per warehouse; a Strategy picks one per line. **Split shipments** — one order, many `Shipment`s.
- **Search at scale** — Elasticsearch/OpenSearch fed by product events behind `Search`. **Payouts, recommendations, wish lists** — modules listening to order events.

In the interview

Plan: clarify (3 min) → actors/use cases (5) → class diagram, **saying the LSP sentence**: "Ordering is on `Member`, not `Customer`, otherwise every guest inherits a method it is not allowed to use" (10) → state machine, no cancel-after-ship edge (5) → code `placeOrder` and `cancel` (10) → concurrency and extensions (5).

> **30-second answer:** "Catalog with name and category indexes; a cart per customer; ordering only on `Member`; checkout reserves stock for every line atomically, copies the lines into an immutable order, then takes payment through a `Payment` strategy. The order moves through a state machine that forbids cancelling after shipping, and every status change is published to a notification observer."

- *Tax and discounts?* — A `PricingRule` chain (Chain of Responsibility or list of Strategies) applied when building the order; the order stores final line prices.
- *Double-click checkout?* — Client idempotency key; return the existing order for a repeated key.

Self-check

1. **Q:** Why can't `placeOrder` live on `Customer`? **A:** Guests extend `Customer` and must not order; inheriting a method you must refuse violates LSP.

2. **Q:** Two faults of "check stock, pay, then decrement"? **A:** A race lets two buyers pass the check; payment can succeed for goods no longer there.
3. **Q:** Which edges allow a cancel? **A:** `PENDING → CANCELED` and `UNSHIPPED → CANCELED`.
4. **Q:** Why `compareTo` on `BigDecimal`? **A:** `equals` also compares scale (`2.0` ≠ `2.00`).

TL;DR

- Guests browse and fill carts; **only members order** (LSP).
- Checkout = **reserve all lines atomically → copy into an immutable order → pay (Strategy) → publish (Observer)**.
- `OrderStatus` is a **state machine**; cancel after ship has no edge. Money in `BigDecimal`; histories in logs; stock guarded by a lock.

Design an Online Stock Brokerage System

Two halves, both asked in interviews:

- **Part A — the brokerage** (the course): members, money in/out, watchlists, four order types, lots and positions, statements.
- **Part B — the exchange's matching engine** (a Geektrust machine-coding problem kept in kumaransg/LLD as "Navi"): an order book matching by **price-time priority**, implemented and tested.

A **broker** holds your cash and shares and forwards orders; an **exchange** is where orders from all brokers meet and match. The code keeps them apart.

Problem & clarifying questions

An online brokerage lets members trade without a human broker, track transactions, see charts, and get alerts when a stock moves by a chosen amount — faster and cheaper, with real-time prices.

Question	Assumed answer
Order types?	Market, limit, stop-loss, stop-limit
Partial fills?	Yes, at several prices
Separate purchases tracked?	Yes — each is a **lot** (tax depends on which lot you sell)
Money in/out?	Check, wire, electronic bank transfer, each with a per-transfer limit
Reports?	Quarterly statements and a yearly tax statement
Notifications?	When an order executes
Margin, shorting, options, dividends/splits, streaming quotes, settlement delay, wash-sale rule?	Out of scope

Requirements

Functional (brokerage): (1) buy and sell; (2) **multiple watchlists**; (3) **market, limit, stop-loss, stop-limit** orders; (4) **multiple lots** per stock kept apart; (5) **quarterly** and **yearly tax** statements; (6) money via **check, wire, electronic transfer**; (7) notify on execution.

Functional (matching engine, Navi): (8) input lines `<id> <time> <symbol> <buy/sell> <price> <qty>` (e.g. `#1 09:45 BAC sell 240.12 100`) from a file given as argument, trades to console; (9) match each new order against the opposite side — lower sells and higher buys first, **earlier first** at the same price; (10) trade when buy ≥ sell, recorded at the **sell order's price**; (11) print `<buy_id> <price> <qty> <sell_id>`; unmatched quantity rests.

Non-functional: money never over-committed (ten $1,000 orders cannot all pass a $1,000 check); deterministic matching; engine safe under concurrency and conserves quantity.

Use cases

Admin (block/unblock), **Member** (search, trade, watchlists), **System** (order notifications, fetch quotes). Top cases: register/cancel membership, manage watchlists, search by symbol, place/cancel orders, deposit/withdraw.

Core entities

Class	Responsibility
`MoneyTransfer` → `CheckTransfer`, `WireTransfer`, `ElectronicBankTransfer`	Differ only in name and clearing days (5/1/3); the 100,000 limit lives once in the base
`Notifier` / `EmailNotifier`	"Order filled" messages (an `SmsNotifier` would be a second implementation)
`Order` (abstract) + `MarketOrder`, `LimitOrder`, `StopLossOrder`, `StopLimitOrder`	Shared fields; each subclass answers the **two questions**; `StopLimitOrder extends StopLossOrder`
`OrderPart`	One fill (e.g. 70 @ 10.00 + 30 @ 10.05)
`Stock`, `StockInventory`	Symbol + latest price; inventory is the **single source of truth** for prices
`StockExchange`, `OrderListener`	Holds open orders, fills on new orders/ticks, calls back `onFill` / `onCancelled` (never knows `Member`)
`StockLot`, `StockPosition`, `LotSale`	One purchase; all lots of a symbol, sold FIFO unless a lot is named; per-lot realised gain
`Watchlist`	Named set of **symbols** (never stale)
`Trade`, `Statement` → `QuarterlyStatement`, `TaxStatement`	Fills as the member sees them; statements share day-range filtering
`Member`, `Admin`	Cash, **reserved funds**, positions, watchlists, trades; implements `OrderListener`
(engine) `OrderBook`, `MatchingEngine`, `Trade`, `ExecutionPricePolicy`	Price levels with FIFO queues; routing and locking; trade price

Class diagram

Course stages: transfers and notifications → **the order hierarchy** (the part that earns most credit) → stock, inventory, exchange → lots and positions → watchlists, trades, statements → members and admins. `Member` extends `Account` and implements `OrderListener`; it owns positions, watchlists and trades.

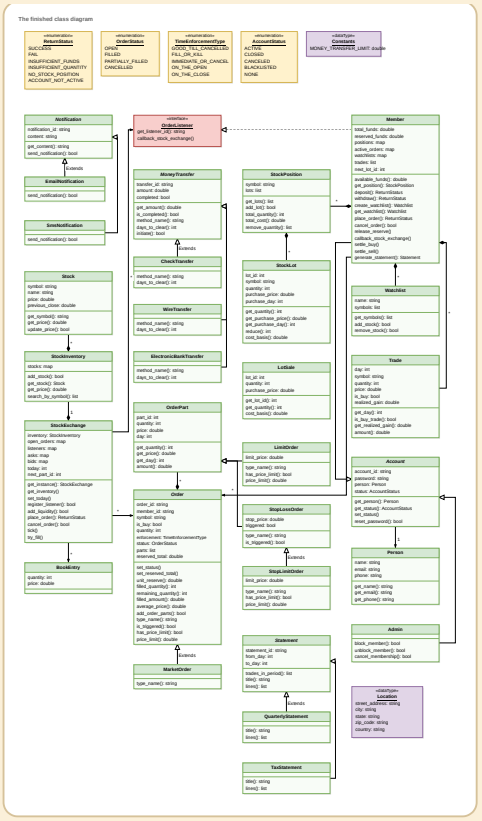
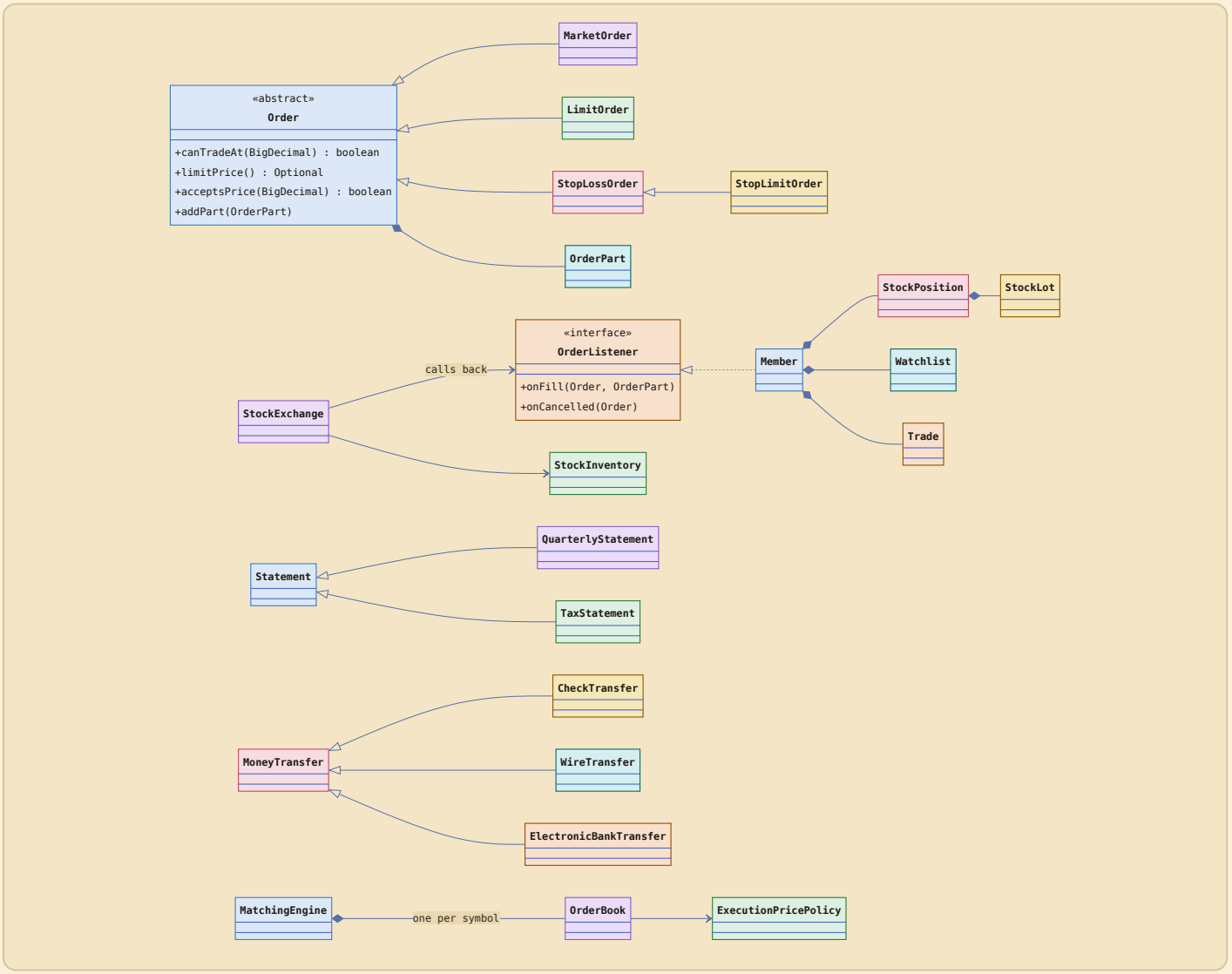


Figure — The finished class diagram of the online stock brokerage (course diagram)



Key flows

Buy order (course activity diagram): pick stock and type, enter quantity, limit and time enforcement → check funds → reserve money → send to exchange → "order placed". **Sell order** mirrors it, choosing **which lots** and checking owned shares instead of money.

The four order types share everything except two answers:

Type	Q1: may it trade at this price?	Q2: worst acceptable price	Plain words
Market	Always	None	"Now, at whatever the market gives"
Limit	Always	The limit	"Buy at 9.50 or less / sell at 10.90 or more"
Stop-loss	Only after price **reaches the stop**, then forever	None (becomes market)	"If it falls to 10.00, get me out"
Stop-limit	Only after the stop	The limit	"If it rises to 305, buy — but pay no more than 306"

A SELL stop triggers when price ≤ stop; a BUY stop when price ≥ stop; once triggered it "latches".

Fill: member reserves (funds or free shares) → exchange checks `canTradeAt`, fills level by level while `acceptsPrice` → `onFill` moves cash, releases the reservation, adds a lot or sells FIFO. Later `tick(symbol, price)` re-checks open orders (stops trigger here).

Matching engine — price-time priority. Per symbol, an `OrderBook` with two sides, each a `TreeMap` price → FIFO `ArrayDeque` (bids highest-first, asks lowest-first). For an incoming order: look at the best opposite price; stop if it does not **cross**; else fill the oldest order at that level for `min(remaining, resting)`; remove filled orders and empty levels; repeat; rest any remainder at the back of its own level.

Order	Book before	What happens
#1 sell 240.12 × 100	empty	rests
#2 sell 237.45 × 90	asks 240.12	rests; best ask 237.45
#3 buy 238.10 × 110	asks 237.45×90, 240.12×100	trade `#3 237.45 90 #2`; 20 left; 240.12 too high; rests 20 @ 238.10
#4 buy 237.80 × 10	best ask 240.12	no cross; rests
#5 buy 237.80 × 40	same	rests **behind #4**
#6 sell 236.00 × 50	bids 238.10×20, 237.80×(10, 40)	fills #3 20, #4 10, #5 20 — all at 236.00 (sell price)

Design decisions & patterns used

Decision	Why	Alternative rejected
One abstract `Order` + two abstract questions	Only "may I trade?" and "worst price?" differ. The course calls it plain polymorphism — not State (no order changes kind); its only named patterns are Observer and Singleton (Template Method/Strategy labels here are ours)	Four copied classes; one class with `type` + `switch`
`StopLimitOrder extends StopLossOrder`	Inherits the trigger, overrides only `limitPrice()`	Duplicated trigger logic
`OrderPart` per fill	Each fill has its own price and becomes a lot	Average price only (loses tax info)
Observer `OrderListener`	Exchange never imports `Member`	Exchange updating balances
Reserved funds / shares	Stops over-commitment; money actually moves only in `onFill`, called by the exchange (course's `callbackStockExchange`)	Balance check only at fill time
Lots, FIFO default	Tax depends on which shares were sold	One average-cost position
Template Method `Statement`	`render` final (shared filter); subclasses fill `body`	Two copies of the filter
`MoneyTransfer` base holds the limit	One rule, three methods	Limit copied per subclass
Singleton?	Course makes `StockExchange` a Singleton; we inject one instance (testable)	Global `getInstance()`
Engine: `TreeMap<price, ArrayDeque>` per side	Best price O(log n); FIFO gives time priority	One `PriorityQueue` (O(n) cancel, no visible levels)
Engine: lock per symbol	Symbols match in parallel; same symbol serialised	Global lock or no lock
Engine: `ExecutionPricePolicy` (**Strategy**)	Navi: sell price; real exchanges: resting order's price	Hard-coding one rule

Java implementation

Part A — Brokerage (`lld.ch19.brokerage`)

Files: `Orders.java`, `Market.java`, `Holdings.java`, `Accounts.java`, `Main.java`. The simulated exchange offers 70 shares per price level, 5 cents apart, 3 levels, so fills come in parts.

```java
// File: lld/ch19/brokerage/Orders.java (excerpt)
abstract class Order {
    // ... id, memberId, symbol, side, quantity, timeInForce, parts, status
    /** Question 1. Market and limit orders always say yes; stop orders wait for the trigger. */
    abstract boolean canTradeAt(BigDecimal marketPrice);

    /** Question 2. Empty means "any price" (market behaviour). */
    abstract Optional<BigDecimal> limitPrice();

    /** Is a fill at this price acceptable? BUY: price <= limit. SELL: price >= limit. */
    final boolean acceptsPrice(BigDecimal price) {
        return limitPrice().map(l -> side == Side.BUY ? price.compareTo(l) <= 0 : price.compareTo(l) >= 0)
                           .orElse(true);
    }
    // ...
}

final class MarketOrder extends Order {
    // ...
    @Override boolean canTradeAt(BigDecimal p) { return true; }
    @Override Optional<BigDecimal> limitPrice() { return Optional.empty(); }
    // ...
}
// ... LimitOrder: canTradeAt -> true, limitPrice -> Optional.of(limit)
class StopLossOrder extends Order {
    // ...
    @Override final synchronized boolean canTradeAt(BigDecimal p) {
        if (!triggered) {
            triggered = side() == Side.SELL ? p.compareTo(stopPrice) <= 0 : p.compareTo(stopPrice) >= 0;
        }
        return triggered;
    }
    @Override Optional<BigDecimal> limitPrice() { return Optional.empty(); }
    // ...
}

/** A stop order that becomes a LIMIT order (not a market order) once triggered. */
final class StopLimitOrder extends StopLossOrder {
    // ...
    @Override Optional<BigDecimal> limitPrice() { return Optional.of(limit); }
    // ...
}

// File: lld/ch19/brokerage/Accounts.java (excerpt, in Member)
    private synchronized ReturnStatus reserveFor(Order o) {
        if (status != AccountStatus.ACTIVE) return ReturnStatus.ACCOUNT_NOT_ACTIVE;
        if (o.side() == Side.BUY) {
            BigDecimal perShare = o.limitPrice().orElse(inventory.priceOf(o.symbol()));
            BigDecimal need = Money.times(perShare, o.quantity());
            if (need.compareTo(availableFunds()) > 0) return ReturnStatus.INSUFFICIENT_FUNDS;
            reservedFunds.put(o.id(), need);          // released part by part as fills arrive
            return ReturnStatus.SUCCESS;
        }
        // ... sells: free shares = position - shares promised to open sells of this symbol
    }
```

```
== 3. Market buy 100 AAPL fills in two parts (two lots) ==
    [email alice@mail.test] filled 70 AAPL @ 10.00 (O1 PARTIALLY_FILLED)
    [email alice@mail.test] filled 30 AAPL @ 10.05 (O1 FILLED)
Tax statement for Alice (days 1-365)
  AAPL L1: 70 sh bought @ 10.00 sold @ 11.00 gain $70.00
  AAPL L2: 30 sh bought @ 10.05 sold @ 10.95 gain $27.00
  AAPL L3: 20 sh bought @ 9.40 sold @ 10.95 gain $31.00
  AAPL L3: 30 sh bought @ 9.40 sold @ 9.95 gain $16.50
  realised gain: $144.50
```

Reading the output: a 200,000 check fails the 100,000 limit. The market order reserved $1,000.00 but cost $1,001.50 — a market order can spend more than reserved (the course's warning). A limit buy waits for the 9.40 tick; a 120-share sell goes FIFO across three lots; the stop does nothing at 10.50 and fires at 9.95; the stop-limit triggers at 310 but its 306 limit blocks the fill until 305.50; a second $2,000 order is refused by the reservation and cancelling returns the money; the tax statement shows gain **per lot**.

Full runnable code: code/lld/ch19/brokerage/ (see the Full Edition).

Part B — Matching engine (`lld.ch19.stockexchange`)

Files: `OrderBook.java`, `MatchingEngine.java` (routing, `synchronized (book)` per symbol, sequence taken inside the lock, trades published outside it), `MatchingEngineTests.java` (nine tests), `Main.java`.

```java
// File: lld/ch19/stockexchange/OrderBook.java (excerpt)
interface ExecutionPricePolicy {
    BigDecimal price(Order buy, Order sell, Order incoming);

    ExecutionPricePolicy SELL_PRICE = (buy, sell, incoming) -> sell.price();
    ExecutionPricePolicy RESTING_PRICE = (buy, sell, incoming) -> incoming == buy ? sell.price() :
buy.price();
}

final class OrderBook {
    // ...
    private final NavigableMap<BigDecimal, Deque<Order>> bids = new TreeMap<>(Collections.reverseOrder());
    private final NavigableMap<BigDecimal, Deque<Order>> asks = new TreeMap<>();
    // ...
    /** Match the incoming order against the other side, then rest whatever is left. */
    List<Trade> add(Order incoming) {
        List<Trade> trades = new ArrayList<>();
        NavigableMap<BigDecimal, Deque<Order>> opposite = incoming.side() == Side.BUY ? asks : bids;

        while (!incoming.isDone() && !opposite.isEmpty()) {
            Map.Entry<BigDecimal, Deque<Order>> best = opposite.firstEntry();
            if (!crosses(incoming, best.getKey())) break;          // best price does not meet ours: stop
            Deque<Order> queue = best.getValue();
            Iterator<Order> it = queue.iterator();
            while (!incoming.isDone() && it.hasNext()) {
                Order restingOrder = it.next();                    // oldest first = time priority
                int qty = Math.min(incoming.remainingQty(), restingOrder.remainingQty());
                Order buy = incoming.side() == Side.BUY ? incoming : restingOrder;
                Order sell = incoming.side() == Side.SELL ? incoming : restingOrder;
                trades.add(new Trade(buy.id(), pricePolicy.price(buy, sell, incoming), qty, sell.id()));
                incoming.fill(qty);
                restingOrder.fill(qty);
                if (restingOrder.isDone()) {
                    it.remove();
                    resting.remove(restingOrder.id());
                }
            }
            if (queue.isEmpty()) opposite.pollFirstEntry();
        }
        if (!incoming.isDone()) rest(incoming);
        return trades;
    }
    // ... cancel, bestBid, bestAsk, crosses, rest
}
```

```
== Trades (execution price = sell order's price) ==
#3 237.45 90 #2
#3 236.00 20 #6
#4 236.00 10 #6
#5 236.00 20 #6
== Same input, real-exchange rule (price of the resting order) ==
#3 237.45 90 #2
#3 238.10 20 #6
#4 237.80 10 #6
#5 237.80 20 #6
  9/9 passed
```

The first block matches the problem's expected output exactly; the second prices each trade at the *resting* buyer's price. Tests cover price priority, time priority, no-cross, partial fill, cancel, symbol isolation, validation, and concurrency (8 threads × 500 orders: quantity conserved, book never crossed).

Full runnable code: code/lld/ch19/stockexchange/ (see the Full Edition).

Concurrency & edge cases

Risk	Where	Prevention
Over-spending	Brokerage	`reserveFor` (synchronized on member) subtracts all reservations first
Selling shares twice	Brokerage	Free shares = position − remaining open sells of that symbol
Member/exchange deadlock	Brokerage	Member reserves under its lock, submits outside it; only exchange → member calls
Market order costs more than reserved	Brokerage	Known gap; reserve a buffer (+5%) or a price band
Time in force not enforced	Brokerage	Enum has GTC, FOK, IOC, on-the-open, on-the-close but nothing reads it; belongs in the exchange's fill loop
Same-price orders "together"	Engine	Lock acquisition order = time priority; `sequence` taken inside the lock
Cancel racing a match	Engine	Same per-symbol lock: matched or cancelled, never both
Bad input (duplicate id, zero qty, negative price)	Engine	Validated before registering
Slow trade publishing	Engine	Published after releasing the lock
Floating-point prices	Both	`BigDecimal`; `TreeMap` compares with `compareTo`, so `237.8` = `237.80`

Real-world example

Zerodha offers **SL** (stop-loss limit) and **SL-M** (stop-loss market) — our `StopLimitOrder` and `StopLossOrder`; orders go to exchanges like NSE with price-time priority books. **LMAX** (Martin Fowler's "The LMAX Architecture") runs matching on a **single thread** fed by a ring buffer (the Disruptor) — "one serialised writer per book" taken to its end. US tax rules treat shares as sold FIFO unless you name lots, hence `StockPosition.sell`'s optional lot id.

Extensions & follow-up questions

- **Trailing stop** — new subclass moving its stop with the best price; only `canTradeAt` changes.
- **IOC / FOK** — IOC cancels the remainder after one pass; FOK checks available quantity first and cancels if short.
- **Price alerts** — a `PriceAlert` observer on ticks, firing once past the threshold.
- **Modify order** — reducing quantity keeps priority; price change or increase = cancel + new.
- **Faster engine** — single writer per symbol via a queue, pre-allocated objects, `long` price ticks.
- **Market orders in the engine** — limit at +∞ / 0, never rest the remainder. **Settlement** (T+1) — `unsettledCash`.

In the interview

Brokerage plan: clarify (3) → actors (3) → **key sentence**: *"The four order types differ in exactly two things: whether they may trade yet, and the worst price they accept"* (8) → lots and tax (4) → reserved funds (3) → code the two questions and `StockPosition.sell` (10) → Observer, statements, extensions (5). **Navi, 90 minutes:** parse lines → `OrderBook` with two `TreeMap`s of `ArrayDeque` → matching loop → print trades → tests (sample, price, time, partial, cancel). Pass the sample first, then refactor.

> **30-second answer:** "One abstract `Order` with four tiny subclasses answering 'may I trade now?' and 'what is my worst price?'. Fills are `OrderPart`s and each purchase becomes a lot, sold FIFO. Money is reserved at placement so a member cannot over-commit; the exchange calls back through an `OrderListener`. The matching engine keeps, per symbol, two `TreeMap`s of price levels with FIFO queues — best price first, oldest first — under one lock per symbol."

- *Book data structure and complexity?* — `TreeMap` levels: best level and new level O(log L); FIFO append O(1), O(1) per fill; cancel O(log L) + O(k) in the level (O(1) with a linked handle).
- *Trade price?* — Navi: the sell price. Real continuous trading: the resting order's price (incoming may get "price improvement").

Self-check

1. **Q:** SELL stop at 10.00; price 10.50 → 9.95 → 10.20. When does it trade? **A:** At 9.95, and it stays triggered.
2. **Q:** What does stop-limit add? **A:** A worst price after triggering — it becomes a limit order.
3. **Q:** Why did one market order create two lots? **A:** It filled at two prices; a lot is defined by its purchase price.
4. **Q:** Bids b1 then b2 at 50.00; a sell for 7 at 49. Trades? **A:** b1 gets 5, b2 gets 2 (time priority).
5. **Q:** What is a "crossed" book? **A:** Best bid ≥ best ask — those orders should already have traded.

TL;DR

- **Brokerage:** abstract `Order`, four subclasses answering *may I trade now?* and *worst price?*; fills are `OrderPart`s; purchases are **lots**; money **reserved** at placement; **Observer** callback; statements are a **Template Method**.
- **Matching engine:** per symbol, two `TreeMap<price, FIFO queue>`; best price, then oldest; rest the remainder; one lock per symbol; trade price is a pluggable policy.

Design an Online Auction System

Problem & clarifying questions

An online auction (like eBay) lets a **seller** list an item with a starting price and end time; **bidders** outbid each other; at the end the highest bidder wins — if the hidden **reserve price** was reached.

Question	Assumed answer
Listing contents?	Name, description, category, starting price, minimum increment, optional reserve, duration
Valid bid?	First ≥ starting price; later ≥ current highest + increment
Seller bids? Leader outbids self?	No and no
When does it end?	At end time; a late bid is refused even if the closing job hasn't run
Who is told what?	Outbid bidder immediately; winner and seller at close; seller if unsold
Proxy bidding, anti-sniping, payments?	Extensions

Requirements

Functional: (1) register and log in (login is the usual auth service, not coded); (2) create listings; (3) search by name, category, price range; (4) bid on **active** listings; (5) keep the **current highest bid** and notify; (6) close at end time — **highest bidder wins** if the reserve is met. **Non-functional:** (7) many simultaneous bids stay consistent (one true highest); (8) new bid rules, channels or search criteria without rewriting the core.

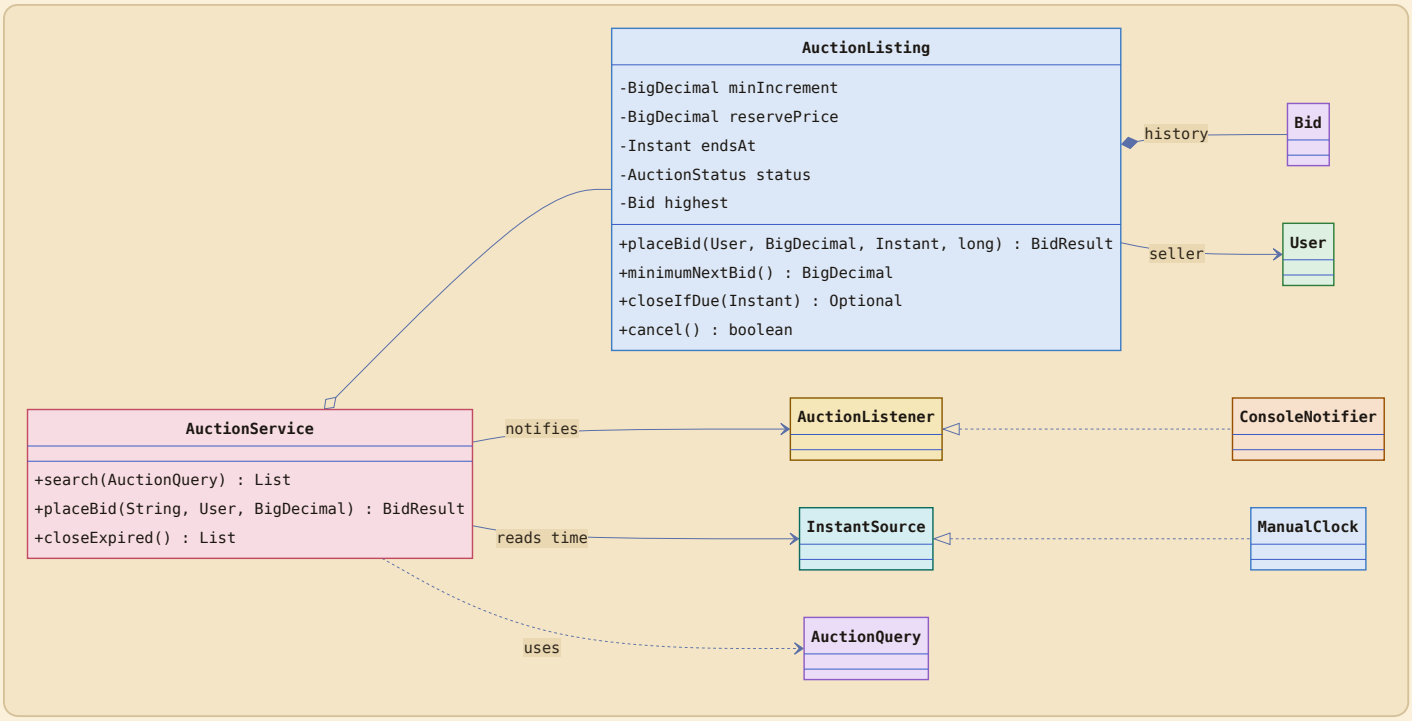
Use cases

Actor	Use cases
Seller	Create listing, cancel (only while nobody has bid), see result
Bidder	Search, view listing and history, bid, be told when outbid or won
System (scheduler)	Activate, close expired, declare winner, notify

Core entities

Class	Responsibility
`User`, `Bid`	Records; bid = bidder, `BigDecimal` amount, time, sequence
`AuctionStatus`	`SCHEDULED → ACTIVE → CLOSED`, or `CANCELLED`
`AuctionListing`	Item, seller, prices, times, history, highest bid; **owns the bidding rules** under its own lock
`BidOutcome` / `BidResult` / `CloseResult`	Why accepted/refused, next minimum, accepted bid, who was outbid; winner and reserve met
`AuctionQuery`	Fluent search; each criterion a `Predicate` (**Specification**)
`AuctionListener` / `ConsoleNotifier`	Observer: new high bid, outbid, won, unsold
`AuctionService`	Facade; calls listeners **outside** listing locks
`ManualClock`	An `InstantSource` moved forward in demos and tests

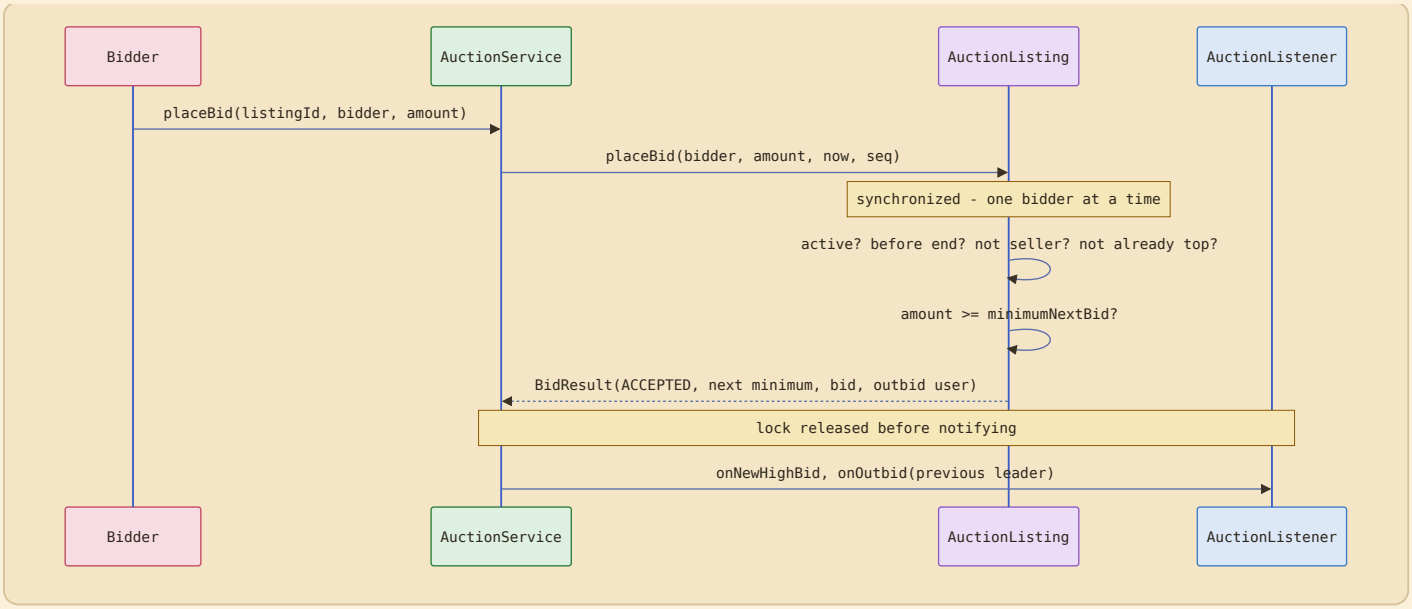
Class diagram



Key flows

Lifecycle: SCHEDULED → ACTIVE (start time) or CANCELLED (seller withdraws); ACTIVE → CANCELLED (withdraw with no bids) or CLOSED (end time). Activation is **lazy**: every method first calls `refresh(now)`, so no thread is needed to flip the state.

Placing a bid (the critical section):



Closing: a scheduler (a `ScheduledExecutorService` in production) calls `closeExpired()`; `closeIfDue(now)` atomically moves `ACTIVE → CLOSED` and returns a winner only if the reserve is met; notifications go out after locks are released.

Design decisions & patterns used

Decision	Why	Alternative rejected
Bid rules inside `AuctionListing` under its lock (a *monitor*: all methods `synchronized`)	Compare-and-replace is atomic; per-listing lock, so auctions don't wait for each other	Global lock; or `AtomicReference<Bid>` + CAS (valid, but seller/leader/time/increment rules are easier under one lock)
Refusals returned (`BidOutcome`), not thrown	A low bid is normal business; caller gets the next minimum	Exception per low bid
Observer called outside the lock	Slow notifications don't block bidders; `BidResult` carries a **snapshot** taken inside the lock	Notifying inside `synchronized` (slow, deadlock risk)
Minimum increment	No 1-cent wars; predictable next minimum	Any higher amount
Reserve price checked only at close	Seller protection without revealing it	Rejecting bids below reserve
Injected `InstantSource`	Testable time (`ManualClock`)	`Instant.now()` everywhere
Specification for search	Each criterion a `Predicate`, combined with `and`	An ever-growing `search(...)` parameter list
Not a Singleton	awesome-low-level-design uses a Singleton `AuctionSystem`; we pass one instance so tests get fresh ones	Global shared state

Java implementation

Package `lld.ch19.auction`: `Auction.java`, `AuctionService.java`, `Main.java`.

```java
// File: lld/ch19/auction/Auction.java (excerpt, in AuctionListing)
    synchronized BidResult placeBid(User bidder, BigDecimal rawAmount, Instant now, long seq) {
        refresh(now);
        BigDecimal amount = money(rawAmount);
        if (status != AuctionStatus.ACTIVE) return refuse(BidOutcome.AUCTION_NOT_ACTIVE);
        if (!now.isBefore(endsAt)) return refuse(BidOutcome.AUCTION_ENDED);  // closing job may not have run
yet
        if (bidder.id().equals(seller.id())) return refuse(BidOutcome.SELLER_CANNOT_BID);
        if (highest != null && highest.bidder().id().equals(bidder.id())) return
refuse(BidOutcome.ALREADY_HIGHEST);
        if (amount.compareTo(minimumNextBid()) < 0) return refuse(BidOutcome.TOO_LOW);

        Optional<User> outbid = Optional.ofNullable(highest).map(Bid::bidder);
        highest = new Bid(bidder, amount, now, seq);
        bids.add(highest);
        return new BidResult(BidOutcome.ACCEPTED, minimumNextBid(), Optional.of(highest), outbid);
    }

    /** First bid must reach the starting price; later bids must beat the leader by the increment. */
    synchronized BigDecimal minimumNextBid() {
        return highest == null ? startingPrice : highest.amount().add(minIncrement);
    }

    /** ACTIVE -> CLOSED once time is up. Returns empty if not yet due (or already closed). */
    synchronized Optional<CloseResult> closeIfDue(Instant now) {
        refresh(now);
        if (status != AuctionStatus.ACTIVE || now.isBefore(endsAt)) return Optional.empty();
        status = AuctionStatus.CLOSED;
        boolean reserveMet = highest != null && highest.amount().compareTo(reservePrice) >= 0;
        return Optional.of(new CloseResult(this, reserveMet ? Optional.of(highest) : Optional.empty(),
reserveMet));
    }

// File: lld/ch19/auction/AuctionService.java (excerpt)
    BidResult placeBid(String listingId, User bidder, BigDecimal amount) {
        AuctionListing a = listings.get(listingId);
        if (a == null) throw new IllegalArgumentException("no listing " + listingId);
        BidResult r = a.placeBid(bidder, amount, clock.instant(), bidSeq.incrementAndGet());
        r.accepted().ifPresent(top -> {                 // snapshot taken inside the lock
            listeners.forEach(l -> l.onNewHighBid(a, top));
            r.outbid().ifPresent(u -> listeners.forEach(l -> l.onOutbid(a, u, top.amount())));
        });
        return r;
    }
```

Full runnable code: code/lld/ch19/auction/ (see the Full Edition).

```
  Ana bids 90 (below start) -> TOO_LOW (next minimum 100.00)
  Ana bids 120 (already top) -> ALREADY_HIGHEST (next minimum 110.00)
  Bob bids 105 (increment is 10) -> TOO_LOW (next minimum 110.00)
  Sam bids on own item -> SELLER_CANNOT_BID (next minimum 120.00)
  accepted=28 (varies run to run), highest=650.00 by bidder49, history strictly increasing by >= increment:
true
  late bid -> AUCTION_ENDED (next minimum 660.00)
```

Every refusal rule fires; outbid notices print before the bidder's own result line (the service notifies before returning). The watch closes unsold (60 < 500 reserve). Of 50 concurrent bids the accepted count varies, but the highest is always 650.00 and the history always rises by at least the increment — the invariant the lock protects.

Concurrency & edge cases

Case	Handling
Same amount at the same time	First through the lock wins; the second is `TOO_LOW`
Bid exactly at end time	`!now.isBefore(endsAt)` → `AUCTION_ENDED` (end is exclusive)
Close job runs late / races a bid	Late bids refused anyway; both `synchronized`, so the bid lands before or after the close
Slow or failing listener	Outside the lock; production publishes to a retrying queue
Seller cancels after bids	Refused
Money precision	Scale 2 `BigDecimal`, `compareTo`
Very hot auction	Single writer per auction (partition by id, e.g. Kafka), or `UPDATE ... SET high = ? WHERE id = ? AND high < ?`

Real-world example

eBay uses **proxy bidding** (it bids for you up to your maximum, by the increment), hidden **reserve prices**, and fixed end times — hence "sniping". Other sites use a **soft close** (late bids extend the end) to defeat it.

Extensions & follow-up questions

- **Proxy bidding** — per-bidder `maxAmount`; after each accepted bid, inside the lock, the strongest proxy responds with `min(its max, competitor + increment)`; ties to the earlier proxy (`Bid.seq`).
- **Anti-sniping** — soft close: a bid in the last N minutes extends `endsAt` (non-final, same lock).
- **Buy It Now** — fixed price closes the auction if paid before the first bid.
- **Dutch / sealed / Vickrey** — an `AuctionRules` strategy (`validate`, `winner`, `price`); Vickrey: highest wins, pays second-highest.
- **Payment after winning** — `AWAITING_PAYMENT`, 48 hours, else offer to the second bidder.
- **Scale the close job** — a priority queue by `endsAt`.

In the interview

Plan: bid rules (4 min) → entities and lifecycle (8) → code `placeBid` under the listing lock and **say why the lock is per listing** (12) → closing and notifications outside the lock (6) → concurrency and extensions (10).

> **30-second answer:** "Each listing owns its bid rules and highest bid, and `placeBid` is synchronized on the listing, so compare-and-replace is atomic and different auctions never contend. Refusals return a reason and the next minimum. The service notifies outbid users and, at close, the winner and seller, always after releasing the lock. Time comes from an injected clock, so closing and late bids are testable."

- *Why not `AtomicReference` + CAS?* — The rule also checks seller, leader and time; one small lock is simpler and still fast per listing.
- *Close on time across servers?* — A scheduler with a time-ordered index; `closeIfDue` is idempotent, so running twice is harmless.
- *Live price for thousands?* — Push `onNewHighBid` to pub/sub and WebSockets; viewers read a cache.

Self-check

1. **Q:** When is a bid `TOO_LOW`? **A:** Below the starting price with no bids, or below highest + increment.
2. **Q:** Why notify after the listing call returns? **A:** Short lock; no slowdown or deadlock from notification code.
3. **Q:** Bid at exactly the end instant? **A:** Refused — the end is exclusive.
4. **Q:** Where does the reserve matter? **A:** Only at close.
5. **Q:** What is the Specification pattern here? **A:** Search criteria as composable `Predicate`s.

TL;DR

- One **lock per listing**; all bid rules inside it; refusals are values, not exceptions.
- `SCHEDULED → ACTIVE → CLOSED | CANCELLED` ; activation lazy, closing idempotent.
- **Observer** after the lock; **injected clock**; **Specification** for search.

Design a Music Streaming Service (Spotify)

Problem & clarifying questions

Users search a big catalog, build playlists and play songs with play, pause, skip, previous, seek, shuffle and repeat. Free and paid plans differ; the service suggests music.

Question	Assumed answer
Search?	Songs, albums, artists
Playlists?	Users create, reorder, edit
Controls?	Play, pause, resume, skip, previous, seek; shuffle; repeat off / all / one
Plans?	FREE (ads, limited skips/hour, no downloads), PREMIUM (no ads, unlimited skips, offline)
Recommendations?	A history-based stub (real ones are ML)
Audio bytes, DRM, CDNs?	Out of scope — the player models *what* plays

Requirements

Functional: (1) browse/search songs, albums, artists; (2) playlists; (3) accounts (auth is the usual service); (4) play, pause, skip, seek (+ previous, shuffle, repeat); (5) recommendations from history; (6) FREE vs PREMIUM limits. **Non-functional:** (7) concurrent users on a read-mostly, thread-safe catalog; (8) large scale (indexes in production); (9) extensible — sharing, offline, new play orders and recommenders.

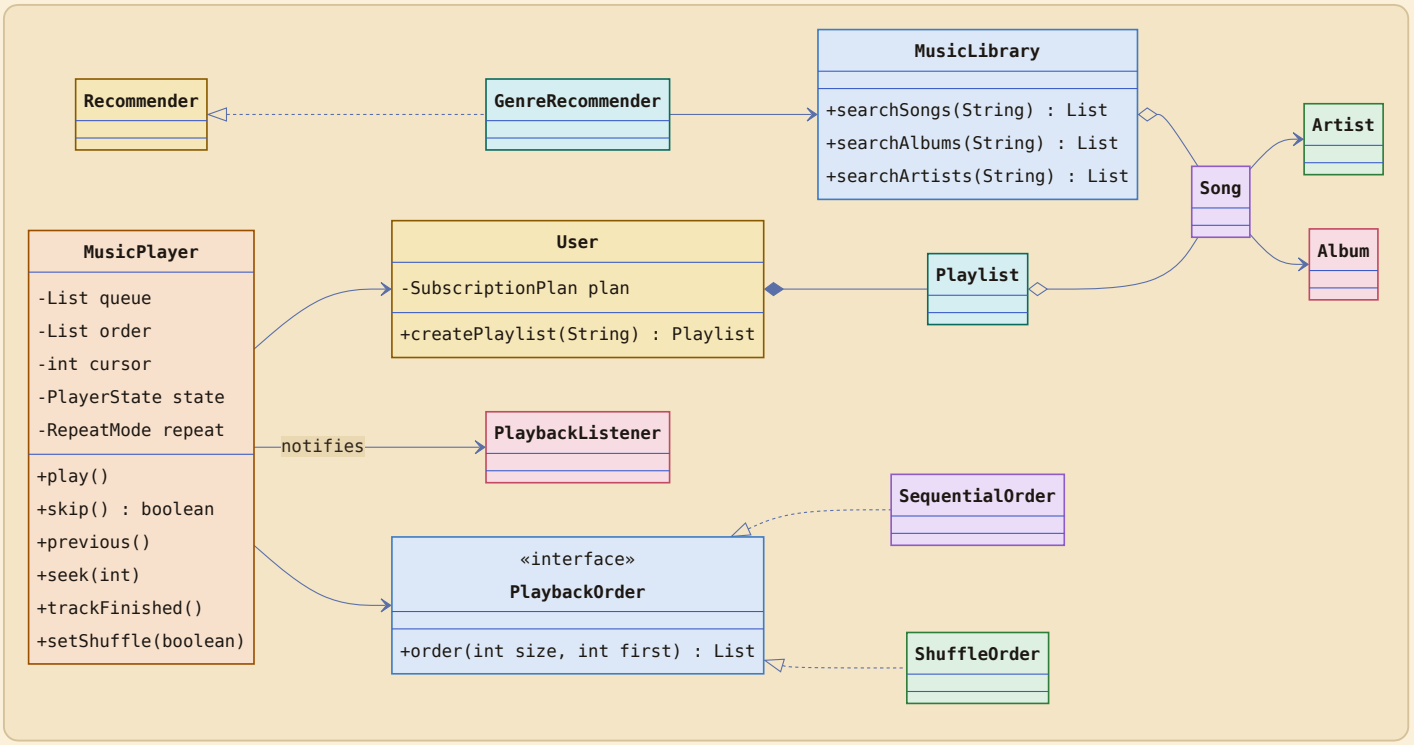
Use cases

Actor	Use cases
Listener	Search, controls, shuffle/repeat, playlists, upgrade plan
Artist / label	Add songs and albums
System	Record plays (history, royalties), ads for free users, recommend

Core entities

Class	Responsibility
`Artist`, `Album`, `Song`, `Genre`	Immutable catalog records
`MusicLibrary`	Thread-safe catalog with search by title, album, artist
`SubscriptionPlan`	Enum **carrying the rules**: price, skips/hour, ads, offline
`User`, `Playlist`	Plan, playlists, history; ordered song list (add, remove, move)
`PlaybackOrder` → `SequentialOrder`, `ShuffleOrder`	Strategy: a permutation starting from the current song
`RepeatMode`, `PlayerState`	`OFF/ALL/ONE`; `STOPPED/PLAYING/PAUSED`
`MusicPlayer`	One session: queue, order, cursor, position, state, skip window, ad counter
`PlaybackListener`	Observer on "song started"
`Recommender` → `GenreRecommender`	Strategy: unplayed songs from favourite genres

Class diagram



Key flows

Player states: STOPPED → PLAYING (play) ⇄ PAUSED (pause/resume); PLAYING → PLAYING on skip or track end with more songs; PLAYING → STOPPED at end of queue with repeat OFF.

Song ends or Next pressed: natural end with repeat ONE → replay; else more songs in the order → cursor + 1; else repeat ALL/ONE → build a new order (reshuffle if shuffling) and play the first; else STOPPED. Pressing Next on repeat-ONE moves on (repeat-one only affects natural ends). **Skip** first checks the plan's rolling one-hour limit: a deque of skip timestamps, drop those older than an hour, compare with `skipsPerHour`.

Shuffle keeps the current song: a new permutation starts with the playing song and shuffles the rest (`Collections.shuffle(list, random)`); turning it off rebuilds the sequential order from the current song. The queue never changes — only the **order of indexes** — so nothing is lost.

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for play order	Sequential, shuffle, "smart shuffle" interchangeable	`if (shuffle)` in skip, previous, wrap
Order = indexes into an unchanged queue	Shuffle off restores the original	Physically shuffling the queue
Seeded `Random`	Repeatable demos/tests	`Math.random()`
Plan rules in `SubscriptionPlan` enum	No `if (plan == FREE)`; new plan = new constant	Boolean flags on `User`
Observer for plays	History, royalties, recommendations react to "song started"	Player calling each subsystem
Strategy for recommendations	Genre today, collaborative filtering tomorrow	One algorithm in `User`
One `MusicPlayer` per session, all `synchronized`	Phone + desktop may send commands at once; library uses concurrent maps	One shared player
State as enum, not State pattern	Three trivial states; GoF State pays off only for substantial per-state behaviour	Three state classes
No Singletons	awesome-low-level-design makes `MusicLibrary`, `UserManager`, `MusicRecommender` Singletons; we inject	Global `getInstance()`

Java implementation

Package `lld.ch19.music`: `Catalog.java`, `Player.java`, `Main.java`.

```java
// File: lld/ch19/music/Catalog.java (excerpt)
enum SubscriptionPlan {
    FREE(new BigDecimal("0.00"), 6, true, false),
    PREMIUM(new BigDecimal("10.99"), Integer.MAX_VALUE, false, true);
    // ... monthlyPrice, skipsPerHour, ads, offlineDownloads
}

// File: lld/ch19/music/Player.java (excerpt)
/** Strategy: in what order should the queue be played? */
interface PlaybackOrder {
    /** A permutation of 0..size-1 that starts with {@code first}. */
    List<Integer> order(int size, int first);
}

final class ShuffleOrder implements PlaybackOrder {
    private final Random random;
    ShuffleOrder(long seed) { this.random = new Random(seed); }   // seeded: repeatable demo and tests
    @Override public List<Integer> order(int size, int first) {
        List<Integer> rest = new ArrayList<>(IntStream.range(0, size).filter(i -> i !=
first).boxed().toList());
        Collections.shuffle(rest, random);
        rest.add(0, first);                    // the song you are hearing keeps playing
        return rest;
    }
}
// ... in MusicPlayer:
    synchronized boolean skip() {
        if (state == PlayerState.STOPPED) return false;          // nothing to skip
        Instant now = clock.instant();
        while (!recentSkips.isEmpty() && recentSkips.peekFirst().isBefore(now.minus(Duration.ofHours(1))))
recentSkips.pollFirst();
        if (recentSkips.size() >= user.plan().skipsPerHour) {
            System.out.println("  [player] skip limit reached on " + user.plan() + " plan");
            return false;
        }
        recentSkips.addLast(now);
        advance(false);
        return true;
    }

    synchronized void setShuffle(boolean on) {
        int now = order.isEmpty() ? 0 : order.get(cursor);
        playbackOrder = on ? new ShuffleOrder(42) : new SequentialOrder();
        order = playbackOrder.order(queue.size(), now);
        cursor = 0;
    }
```

Full runnable code: code/lld/ch19/music/ (see the Full Edition).

```
== 4. Shuffle keeps the current song, reorders the rest ==
  now Blue in Green - Miles Davis, up next [Love of My Life - Queen, So What - Miles Davis, Bohemian Rhapsody
- Queen, Hello - Adele]
  [player] skip limit reached on FREE plan
  recommended: [You're My Best Friend - Queen, All Blues - Miles Davis, When We Were Young - Adele]
```

Repeat-ONE replays "Bohemian Rhapsody" on natural end; on FREE an ad plays every three songs, repeat-ALL wraps and reshuffles, and the seventh skip in the hour is refused; PREMIUM has no ads and with repeat OFF the queue ends; the recommender suggests *unheard* songs from the most-played genres.

Concurrency & edge cases

Case	Handling
Two devices press "next" together	`synchronized` player: applied one after the other
Catalog updated during search	`ConcurrentHashMap`, weakly consistent iteration
Playlist edited while playing	`load` copies it (`List.copyOf`) — the queue is a snapshot
Seek out of range	Clamped to `[0, duration]`
Previous after 3 seconds	Restarts the song; within 3 s goes back one
Skip when stopped	Refused, not counted
Plan change mid-session	`plan` is `volatile`
Empty queue	`play()` throws `IllegalStateException`

Real-world example

Spotify's free tier has limited skips (e.g. six per hour on mobile) and ads; Premium removes both and allows downloads — our enum's rules. In 2014 Spotify engineers wrote that truly random shuffle "felt" non-random when an artist repeated, so they spread same-artist songs apart — a new `PlaybackOrder` in our design. Discover Weekly is the industrial `Recommender`.

Extensions & follow-up questions

- **Play next / add to queue** — a separate user queue consumed before the context order.
- **Offline** — `offlineDownloads`, a `DownloadManager`, and a local/remote `AudioSource` Strategy.
- **Collaborative playlists** — collaborators + edit log, changes pushed via Observer.
- **Crossfade / gapless** — a `TransitionStrategy` around `trackFinished`.
- **Royalties** — a `PlaybackListener` counting a play after a listening threshold.
- **Search at scale** — inverted index with prefix and fuzzy matching. **Smart shuffle** — a `PlaybackOrder` spacing out artists.

In the interview

Plan: controls, plans, scope (4 min) → catalog + search (6) → user, playlist, plan (5) → **player**: queue + order + cursor, state machine, shuffle/repeat (15) → Observer, recommender, concurrency, extensions (10).

> **30-second answer:** "Immutable catalog records in a thread-safe library. Each user has a player session holding the queue, a play order produced by a `PlaybackOrder` strategy, a cursor and a repeat mode. Shuffle rebuilds the order around the current song. Plan limits live in the plan enum. A playback listener records history, which feeds a pluggable recommender."

- *How does shuffle avoid repeats?* — It is a permutation (each index once), not "random next".
- *Previous in shuffle mode?* — Walk back along the same order list.

Self-check

1. **Q:** Why store the order as indexes? **A:** Shuffle off restores the original order.
2. **Q:** Repeat-ONE and Next? **A:** Moves on; replay only on natural end.
3. **Q:** How is "6 skips per hour" enforced? **A:** Deque of skip times; drop entries older than an hour; refuse at the limit.
4. **Q:** Why an enum, not the State pattern? **A:** The states have almost no distinct behaviour.

TL;DR

- Catalog = immutable records in a concurrent library.
- Player = **queue + order (Strategy) + cursor + repeat mode + state**; shuffle keeps the current song.

- Plan rules in the **enum**; plays via **Observer**; recommendations via **Strategy**.

Design an Online Food Delivery Service (Swiggy) + FoodKart

Two shapes:

- **Part A — Swiggy-style platform** (awesome-low-level-design): restaurants and menus, cart, tracked orders, payments, **rider assignment** by a pluggable strategy. We add **kitchen capacity** (max items cooking at once), the twist of a reported Flipkart food-ordering variant (not part of FoodKart).
- **Part B — FoodKart** (Flipkart machine coding, kumaransg/LLD): command-driven — users and pincodes, one-dish restaurants, ratings, `show_restaurant` by rating or price, `place_order` — plus a **restaurant selection strategy** filtered by capacity.

Problem & clarifying questions

Question	Assumed answer
Mix restaurants in one cart?	No — one restaurant per cart
Who confirms?	The restaurant; it refuses if the kitchen is full
When is capacity freed?	When food is ready, not when delivered
Rider choice?	A strategy: nearest, or fair share (fewest deliveries today)
No rider free?	Order waits in a FIFO queue; the next freed rider takes the oldest
Customer cancel?	Only before cooking (PENDING or CONFIRMED)
Payments?	UPI, card, cash on delivery (nothing to refund)
GPS, ETA, batching, surge?	Extensions

Requirements

Functional (Part A): (1) browse restaurants, view menus, order; (2) restaurants manage menus, prices, availability; (3) agents accept and fulfil; (4) track status with notifications; (5) several payment methods; (6) real-time notifications to customers, restaurants, agents.

Functional (Part B — FoodKart): (7) `register_user(name, gender, phone, pincode)`, phone unique; (8) `login_user(phone)` switches context; (9) `register_restaurant(name, pincodes "A/B", dish, price, quantity)` by the logged-in owner — **one** dish, **many** pincodes; (10) `update_quantity`, `update_location` — owner only; (11) `rate_restaurant(name, 1..5, optional comment)`, rating = average; (12) `show_restaurant(rating|price)` — **serviceable** (my pincode **and** quantity > 0), **descending**; (13) `place_order(name, qty)`, bonus order history; (14) *variant:* order without naming a restaurant, chosen by a **selection strategy** among those delivering here **with** capacity for the whole quantity.

Non-functional: no double-booked kitchen capacity or riders under concurrency; no negative stock, no rider on two orders; in-memory, demoable driver, graceful failures.

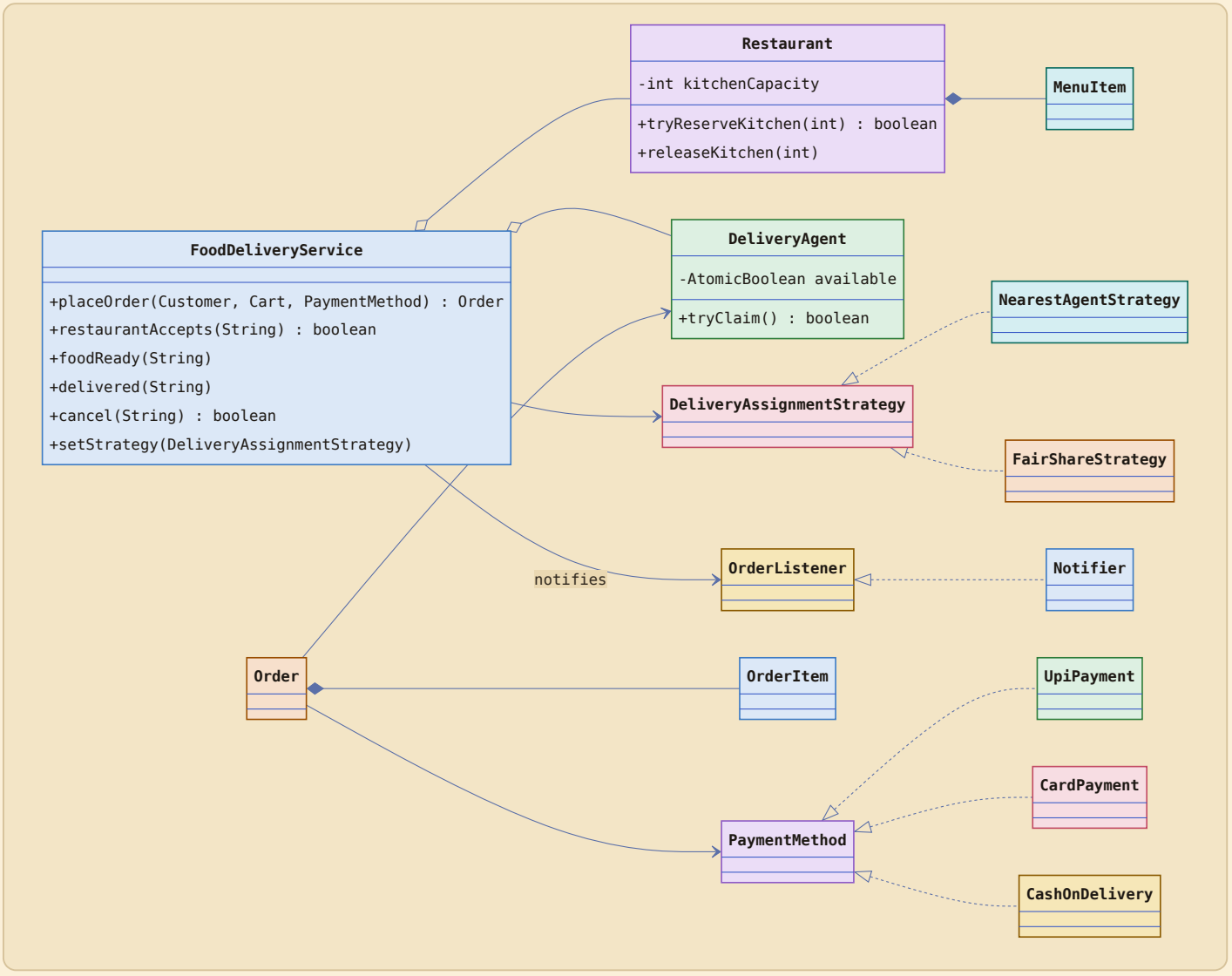
Use cases

Actor	Use cases
Customer	Browse nearby, cart, pay, order, track, cancel, rate
Restaurant	Manage menu, accept/refuse, preparing, ready
Delivery agent	Get assigned, pick up, deliver
System	Fees, rider assignment, waiting queue, notifications

Core entities

Class	Responsibility
`Location`, `Customer`, `MenuItem`	Point + `distanceTo`; customer details; name, price, available flag
`Restaurant`	Menu, open flag, **kitchen capacity**, `tryReserveKitchen` / `releaseKitchen`
`Cart`	One restaurant + quantities; refuses a second restaurant
`OrderItem`	Snapshot of name, **unit price at order time**, quantity
`OrderStatus`	PENDING → CONFIRMED → PREPARING → READY_FOR_PICKUP → OUT_FOR_DELIVERY → DELIVERED, or CANCELLED
`Order`	Customer, restaurant, items, fee, payment, history, agent
`PaymentMethod` → `UpiPayment`, `CardPayment`, `CashOnDelivery`	Sealed; COD not paid upfront
`DeliveryAgent`	Location, `AtomicBoolean available`, deliveries today; `tryClaim()` is a CAS
`DeliveryAssignmentStrategy` → `NearestAgentStrategy`, `FairShareStrategy`	Ranks free riders
`OrderListener` / `Notifier`, `FoodDeliveryService`	Observer; facade with the waiting-for-rider queue
(FoodKart) `User`, `Restaurant`, `Review`, `FoodOrder`, `SortBy`, `RestaurantSelectionStrategy`, `FoodKartService`	`Restaurant.take(q)` is the atomic capacity check

Class diagram



FoodKart (Part B): `FoodKartService` (current user; `registerUser`, `loginUser`, `registerRestaurant`, `updateQuantity`, `updateLocation`, `rateRestaurant`, `showRestaurants(SortBy)`, `placeOrder(name, qty)`, `placeOrder(qty, strategy)`, `orderHistory`) holds `User`s, `Restaurant`s (pincodes, one dish, price, quantity, `take`, `serviceableFor`, `rating`; owns `Review`s) and `FoodOrder`s; `RestaurantSelectionStrategy` → `HighestRatingStrategy`, `LowestPriceStrategy`.

Key flows

Order states: PENDING → CONFIRMED (accepted, kitchen slots reserved) or CANCELLED (kitchen full / customer); CONFIRMED → PREPARING or CANCELLED (slots released); PREPARING → READY_FOR_PICKUP (slots released) → OUT_FOR_DELIVERY (rider assigned) → DELIVERED. Real apps split pickup into "rider assigned / arrived / picked up" — a local enum change.

Food ready → rider: `foodReady` marks READY_FOR_PICKUP, releases slots, adds the order to the queue; while the queue is not empty: the strategy ranks free agents, `tryClaim()` (CAS true → false) on each in turn; claimed → poll, assign, OUT_FOR_DELIVERY, notify; none → stop, order waits. `delivered()` frees the rider and runs the same loop.

FoodKart `place_order(name, qty)`: logged in? → restaurant exists? → delivers to my pincode? → `take(qty)` atomically → record history → "Order Placed Successfully." else "Cannot place order". **Selection variant:** filter by pincode and `quantity ≥ qty` → strategy picks → `take(qty)`; if a racer took it, drop that restaurant and ask again (bounded retries).

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for riders; `rank` returns a list	Swappable at run time (`setStrategy`); the list lets us try the next rider if the first was just taken	Returning one rider
CAS on the rider (`compareAndSet(true,false)`)	Exactly one claimer wins, even from outside the `synchronized` dispatcher (another dispatcher, a "rider accepts" button)	`isAvailable()` then set
Kitchen capacity reserved on accept, released on ready	"Cannot accept while the kitchen is full"	Unlimited orders
FIFO waiting queue	No free rider is normal at peak; fair	Failing the order
State machine in `OrderStatus`	Cancel after cooking impossible	Scattered `if`s
`OrderItem` copies price	Later menu changes can't alter orders	Referencing `MenuItem`
Sealed `PaymentMethod` records	Closed set; COD overrides `paidUpfront()`	`String paymentType`
Observer `OrderListener`	Customer, restaurant, rider apps react to the same events	Service printing itself
One restaurant per cart	Single kitchen, pickup, fee	Split orders
FoodKart: `SortBy` enum holds its comparator	`sorted(sortBy.comparator)`, descending, name tie-break	`if (sort.equals("rating"))`
FoodKart: owner recorded; `take(q)` synchronized	Only the owner updates; two users can't take the last plates	Anyone updates; get + set

Java implementation

Part A — Food delivery platform (`lld.ch19.fooddelivery`)

Files: `Model.java`, `FoodDeliveryService.java`, `Main.java`.

```java
// File: lld/ch19/fooddelivery/Model.java (excerpt)
    synchronized boolean tryReserveKitchen(int items) {
        if (itemsInKitchen + items > kitchenCapacity) return false;
        itemsInKitchen += items;
        return true;
    }
// ... in DeliveryAgent:
    boolean tryClaim() { return available.compareAndSet(true, false); }

    void completeDelivery() { deliveriesToday.incrementAndGet(); available.set(true); }

// File: lld/ch19/fooddelivery/FoodDeliveryService.java (excerpt)
interface DeliveryAssignmentStrategy {
    List<DeliveryAgent> rank(Order order, List<DeliveryAgent> free);
}
// ...
/** Fewest deliveries today first (spreads earnings), nearest as tie-breaker. */
final class FairShareStrategy implements DeliveryAssignmentStrategy {
    @Override public List<DeliveryAgent> rank(Order o, List<DeliveryAgent> free) {
        Location r = o.restaurant().location();
        return free.stream().sorted(Comparator.comparingInt(DeliveryAgent::deliveriesToday)
                .thenComparingDouble(a -> a.location().distanceTo(r))).toList();
    }
}
// ...
    private synchronized void dispatchWaiting() {
        Order o;
        while ((o = awaitingRider.peek()) != null) {
            List<DeliveryAgent> free = agents.stream().filter(DeliveryAgent::isAvailable).toList();
            Optional<DeliveryAgent> got = Optional.empty();
            for (DeliveryAgent a : strategy.rank(o, free)) if (a.tryClaim()) { got = Optional.of(a); break; }
            if (got.isEmpty()) {
                System.out.println("    (" + o.id() + " waiting for a free rider)");
                return;
            }
            awaitingRider.poll();
            o.assign(got.get());
            transition(o, OrderStatus.OUT_FOR_DELIVERY);
        }
    }
```

Full runnable code: code/lld/ch19/fooddelivery/ (see the Full Edition).

```
== 4. Kitchen capacity: 4 items max at Dosa Corner ==
    (Dosa Corner kitchen full: needs 2 item slots, free 1)
    [OD103 CANCELLED] -> customer Asha
    (OD104 waiting for a free rider)
    [OD101 DELIVERED] -> customer Asha
    [OD104 OUT_FOR_DELIVERY] -> customer Vikram: Ravi is on the way
  o6 rider: Meena (fewer deliveries today)
```

The far grill (beyond 10 km) is hidden; a second restaurant in the cart is refused; fee = 20 + 8 × 1.4 km = 31.20 and a later dosa price change does not alter the order; an order out for delivery cannot be cancelled while a confirmed card order can (and is refunded); with both riders at one spot, fair share picks Meena (1 delivery) over Ravi (2).

Part B — FoodKart (`lld.ch19.foodkart`)

Files: `FoodKart.java`, `Main.java` (replays the sample test case, then extras).

```java
// File: lld/ch19/foodkart/FoodKart.java (excerpt)
    synchronized boolean take(int q) {
        if (q <= 0 || q > quantity) return false;
        quantity -= q;
        return true;
    }

    /** Serviceable = delivers to this pincode AND has non-zero quantity (from the problem statement). */
    boolean serviceableFor(String pincode) { return pincodes.contains(pincode) && quantity() > 0; }
// ...
enum SortBy {
    RATING(Comparator.comparingDouble(Restaurant::rating).reversed()),
    PRICE(Comparator.comparing(Restaurant::price).reversed());

    final Comparator<Restaurant> comparator;
    SortBy(Comparator<Restaurant> c) { comparator = c.thenComparing(Restaurant::name); }
}

/** Strategy: when the user does not name a restaurant, which one should we pick? */
interface RestaurantSelectionStrategy {
    Optional<Restaurant> select(List<Restaurant> canFulfil);
}
// ... HighestRatingStrategy, LowestPriceStrategy
    String placeOrder(int qty, RestaurantSelectionStrategy strategy) {
        User u = requireLogin();
        List<Restaurant> canFulfil = restaurants.values().stream()
                .filter(r -> r.pincodes().contains(u.pincode()) && r.quantity() >= qty).toList();
        for (int attempt = 0; attempt < 3; attempt++) {           // retry if a racer took the capacity
            Optional<Restaurant> pick = strategy.select(canFulfil);
            if (pick.isEmpty()) return "Cannot place order";
            if (pick.get().take(qty)) {
                record(u, pick.get(), qty);
                return "Order Placed Successfully from " + pick.get().name() + ".";
            }
            canFulfil = canFulfil.stream().filter(r -> r != pick.get()).toList();
        }
        return "Cannot place order";
    }
```

Full runnable code: code/lld/ch19/foodkart/ (see the Full Edition).

```
show_restaurant(price):
  Food Court-2, Burger
  Food Court-1, NI Thali
place_order(Food Court-1, 2): Order Placed Successfully.
place_order(Food Court-2, 7): Cannot place order
highest rating: Order Placed Successfully from Food Court-2.
lowest price  : Order Placed Successfully from Food Court-4.
```

The sample block reproduces every expected line (price-sorted Burger 120 before Thali 100; owner updates to `Burger - 8`). Extras show owner-only updates, rating validation, a comment-less rating and a non-serviceable pincode. The variant: the two strategies choose **different** restaurants, Food Court-1 (3 plates) is never picked for 4, and once nobody has 4 left the order is cleanly refused.

Concurrency & edge cases

Case	Handling
Two claims for one rider	`tryClaim()` CAS: one wins, the loser tries the next ranked rider
Two dispatch loops	`dispatchWaiting` is `synchronized` (FIFO kept)
Kitchen over-booked	`tryReserveKitchen` `synchronized`
Cancel races accept	Both lock the `Order`; accept re-checks PENDING; slots released only if reserved
Item sold out after adding	`placeOrder` re-checks `available`
Restaurant closes	Hidden in `browse`; `placeOrder` refuses
COD cancelled	No refund
FoodKart: no login / duplicates / rating out of 1–5	`requireLogin()` throws; `putIfAbsent` rejects; `Review` compact constructor rejects
FoodKart: capacity taken after selection	`take()` fails; restaurant dropped; strategy asked again

Real-world example

Swiggy and Zomato show the same states (placed, confirmed, preparing, rider assigned/picked up, delivered) and ask you to **reset your cart** when adding from a second restaurant. Rider dispatch uses distance, load and fairness and may batch two orders — variations of `DeliveryAssignmentStrategy`. FoodKart is a Flipkart 90-minute, in-memory, driver-class exercise rewarding clean separation over clever algorithms.

Extensions & follow-up questions

- **Batching** — a strategy accepting a rider already carrying one order when the second pickup is on the way (agent capacity 2).
- **ETA** — an `EtaEstimator`: prep time by load + travel time.
- **Surge / distance fees** — a `DeliveryFeePolicy` Strategy.
- **Rider rejects** — rider back to available, order to the queue head, rider excluded for it.
- **Multi-dish FoodKart** — `Map<String, MenuItem>`; `take` per item, all-or-nothing.
- **Order split across restaurants** — the strategy returns a set covering all items; reserve in each, roll back on failure.
- **Live tracking** — rider location events forwarded by an Observer over WebSocket.

In the interview

Design round (45 min): clarify (5) → entities + one-restaurant cart (8) → **state machine** (7) → **assignment strategy + CAS claim + waiting queue** (12) → payments/notifications (5) → concurrency and extensions (8).
FoodKart (90 min): model classes → one service method per command → driver replaying the sample → `SortBy` → owner checks and validation → selection strategy → history; runnable at every step, exact output lines.

> **30-second answer:** "Restaurants own their menus and a kitchen capacity reserved on accept and released when food is ready. Orders copy item prices and move through a state machine from PENDING to DELIVERED. When food is ready the order joins a FIFO queue; a pluggable assignment strategy ranks free riders and we claim one with a compare-and-set, so a rider can never get two orders. Every status change is published to observers for the customer, restaurant and rider apps."

- *Why release capacity on ready?* — The kitchen is free once food leaves; delivery is the rider's constraint.
- *Why does the strategy return a list?* — The top rider may be claimed between ranking and claiming; fall through without re-ranking.
- *Scale across cities?* — Partition by zone; one dispatcher (single writer) per zone.

Self-check

1. **Q:** Which states allow a customer cancel? **A:** PENDING and CONFIRMED — after that the food cost is spent.
2. **Q:** What stops two orders getting one rider? **A:** `AtomicBoolean.compareAndSet(true, false)` in `tryClaim()`.
3. **Q:** FoodKart "serviceable"? **A:** Delivers to the user's pincode and quantity > 0.
4. **Q:** Why is Burger (120) before NI Thali (100) by price? **A:** The statement asks for descending order.
5. **Q:** Patterns used? **A:** Strategy (assignment, selection, payment), Observer, state machine, Facade.

TL;DR

- One-restaurant cart; order copies prices; **state machine** PENDING → … → DELIVERED.
- **Kitchen capacity** reserved on accept, released on ready; **riders** claimed with **CAS**, chosen by a swappable **strategy**, FIFO queue when none are free.
- FoodKart: pincode serviceability, one dish per restaurant, average rating, `SortBy` enum, atomic `take`, capacity-filtered **selection strategy**.

Design a Grocery Store (POS & Checkout)

A **point-of-sale (POS) system** runs a supermarket till: scan, running total, offers, payment in one or more ways, receipt, stock down. Three things make it different from the Amazon cart: **weighed items** (produce priced per kg from the scale), **interacting offers** (buy-2-get-1, "3 for ₹80", % off produce, a basket coupon — the combining rule decides price, tax and refunds), and **the till never blocks a physical sale** (if the system says 0 but the customer holds the carton, the count is wrong). Money: rupees in `BigDecimal` (`Rs` in the output).

Problem & clarifying questions

Question	Assumed answer
Item identification?	Barcode (EAN-13) for packaged goods; 4-digit **PLU** + scale weight for produce.
Tax?	Shelf prices tax-inclusive (India/UK/EU); support US-style tax-on-top too.
Offers and stacking?	One item offer per product — the **best** for the customer; then one basket coupon on what is still owed.
Payment?	Cash (change), card, UPI; **split tender**.
Changes of mind?	Void a line; back to scanning if nothing paid; cancel (reverse card/UPI).
Returns?	With the receipt; refund the **net paid**, never more units than bought.
Stock?	Down on sale, up on return; alerts; never refuse a sale because of the count.

Requirements

Functional: lookup by barcode/PLU, weighed pricing; sale with voidable lines; item offers + basket coupons with stacking rules; tax per category (inclusive/exclusive); split payment; immutable receipt; returns against a receipt; stock updates with low/negative alerts. **Non-functional:** exact money (`BigDecimal`, `HALF_UP`, parts add up); several registers share catalog/stock/receipts safely; deterministic pricing; auditable voids and scan-time prices.

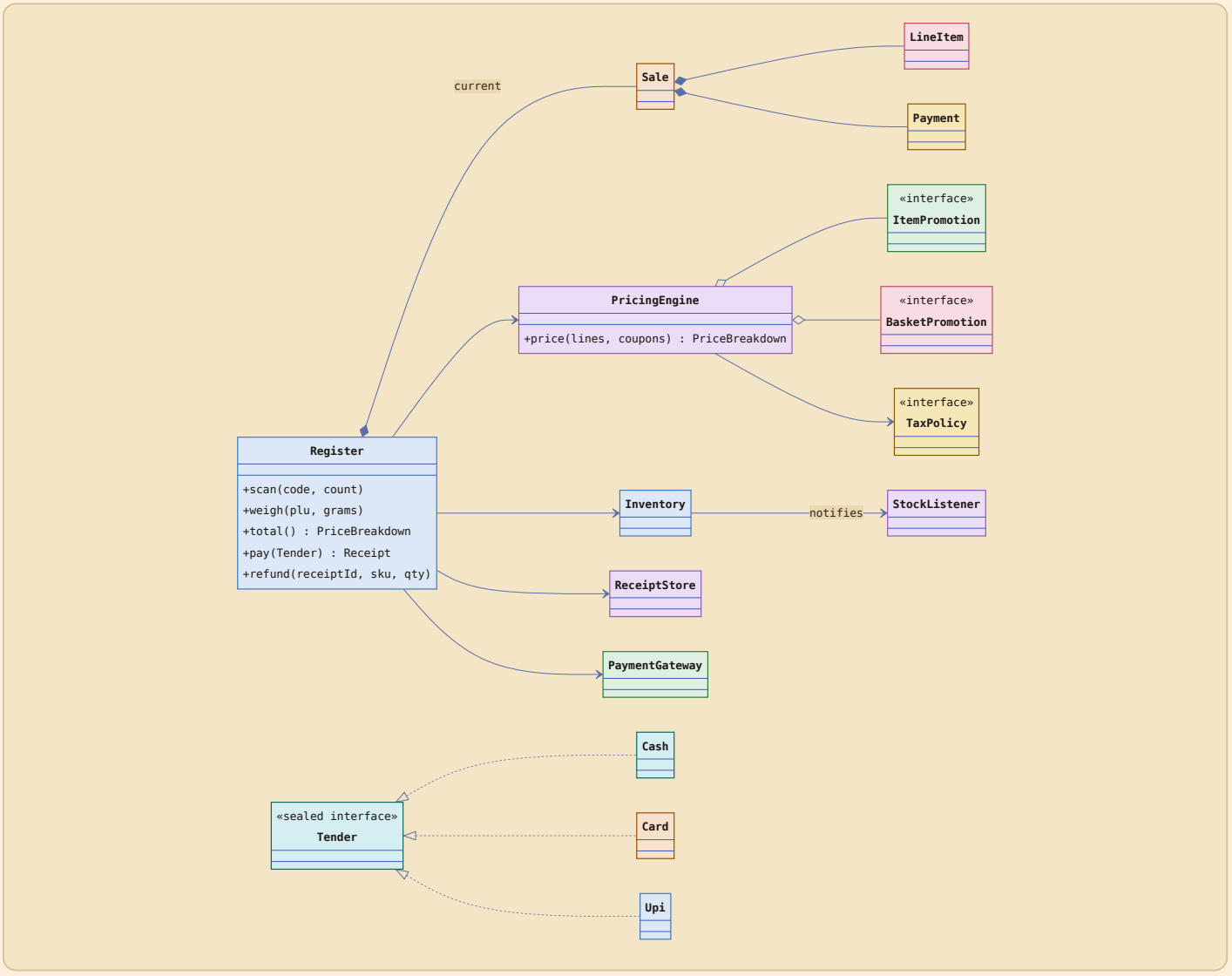
Use cases

Cashier / self-checkout (start, scan, weigh, void, coupon, total, pay, cancel), **Customer** (pay, return), **Supervisor** (approve voids/returns, reprice), **Payment network** (authorise, reverse), **Purchasing** (stock alerts).

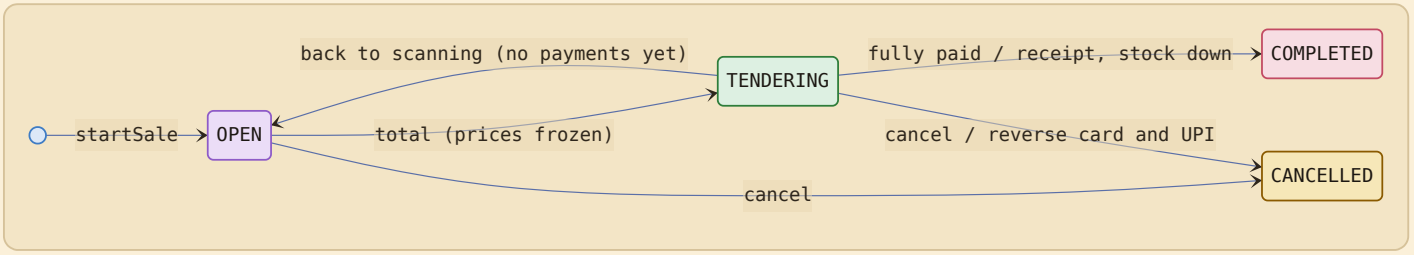
Core entities

Class	Responsibility
`Product`, `Catalog`	SKU, code, department, EACH/PER_KG, price, `TaxCategory`; lookup; reprice replaces the record.
`LineItem`, `Sale` + `SaleStatus`	One scan (price copied in); OPEN → TENDERING → COMPLETED / CANCELLED.
`ItemPromotion` (`BuyXGetYFree`, `MultiBuy`, `PercentOffDepartment`), `BasketPromotion` (`SpendThresholdCoupon`)	**Strategy** for offers.
`TaxPolicy` (`TaxInclusive`, `TaxExclusive`)	**Strategy** for tax.
`PricingEngine` → `PriceBreakdown`	Pure function: lines + coupons → discounts, net per SKU, tax, total.
`Tender` (sealed: `Cash`, `Card`, `Upi`), `Payment`, `PaymentGateway`	Tenders; applied amount; port to card/UPI networks.
`Receipt`, `ReceiptStore`	Immutable proof; atomic returns ledger.
`Inventory`, `StockListener`	Shared stock; **Observer** alerts.
`Register`	**Facade** for one lane.

Class diagram



Key flows



Pricing one basket (the hard method): skip voids → group by SKU → each SKU gets its **best** item offer → net per SKU → best basket coupon on the sum → **spread** the coupon over SKUs in proportion (remainder to the largest) → tax per category on discounted amounts → total (+ tax if added on top).

Split payment: `total()` freezes ₹588.23 → declined card throws, nothing recorded → UPI ₹400 → cash ₹500 → change ₹311.77 → receipt stored, stock reduced.

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Offers behind `ItemPromotion` / `BasketPromotion`	**Strategy**	New offer = new class	`switch` over offer types
Best offer per SKU, coupon after	Explicit stacking policy	Written once, explainable at the till	Apply all in list order (order-dependent, can go negative)
Pure `PricingEngine`	Stateless service	Same basket, same price; shared by all registers	Mutating discounts into lines while scanning
Freeze at `total()`	Snapshot	Price/offer changes mid-payment can't change the amount due	Re-price per payment
Spread the coupon over SKUs	Exact allocation	Tax per category and per-item refunds need it	One unallocated number
`TaxPolicy`	**Strategy**	Inclusive vs exclusive per store	Two engines
Sealed `Tender`, `Register` facade	Sealed types, **Facade**	Compiler-checked receipt switch; UI sees one API	Strings; UI calling every part
Stock may go negative; edge-triggered alerts	**Observer**	Shelf is the truth; one alert per crossing	Refusing the sale; alert spam

Java implementation

Package `lld.ch19.grocery`.

```java
// File: lld/ch19/grocery/Pricing.java
    PriceBreakdown price(List<LineItem> lines, Set<String> coupons) {
        // ... group non-voided lines by SKU into SkuTotal(product, quantity, gross)
        for (SkuTotal t : bySku.values()) {
            Optional<Discount> best = itemOffers.stream()
                    .map(p -> p.evaluate(t.product(), t.quantity()))
                    .flatMap(Optional::stream)
                    .max(Comparator.comparing(Discount::amount));
            BigDecimal off = best.map(d -> d.amount().min(t.gross())).orElse(Money.ZERO);
            best.ifPresent(d -> discounts.add(new Discount(d.description(), d.sku(), off)));
            net.put(t.product().sku(), t.gross().subtract(off));
        }
        BigDecimal subtotal = sum(net.values());
        Optional<Discount> basket = basketOffers.stream()
                .map(b -> b.evaluate(subtotal, coupons))
                .flatMap(Optional::stream)
                .max(Comparator.comparing(Discount::amount));
        basket.ifPresent(d -> { discounts.add(d); spread(d.amount(), net); });
        // ... tax per TaxCategory on the discounted nets via taxPolicy; total = nets (+ tax if added on top)
    }
```

```java
// File: lld/ch19/grocery/Register.java
    /** delta < 0 for a sale, > 0 for a return. merge() is atomic per key, so registers never lose updates. */
    void adjust(String sku, BigDecimal delta) {
        BigDecimal after = onHand.merge(sku, delta, BigDecimal::add);
        BigDecimal before = after.subtract(delta);
        BigDecimal point = reorderPoint.getOrDefault(sku, BigDecimal.ZERO);
        // edge-triggered: alert only when THIS change crosses the line, so each crossing alerts exactly once
        if (before.compareTo(point) > 0 && after.compareTo(point) <= 0)
            listeners.forEach(l -> l.stockAlert(sku, after, "LOW - reorder"));
        if (before.signum() >= 0 && after.signum() < 0)
            listeners.forEach(l -> l.stockAlert(sku, after, "NEGATIVE - recount the shelf"));
    }
```

Full runnable code: code/lld/ch19/grocery/ (see the Full Edition).

How it works: `scan` checks the EAN-13 check digit before lookup; `weigh` turns grams into exact kg (`movePointLeft(3)`). Cash above the amount due becomes change; card/UPI must be authorised and cannot exceed the amount due. Returns: `ReceiptStore.claimReturn` uses `compute` (atomic check-and-add), and the refund is `share(after) − share(before)` of the SKU's net paid, so partial returns add up exactly.

Output (excerpt):

```
   error: Bad barcode 8901000000018: check digit wrong, rescan
    10% off PRODUCE: Bananas                          -Rs 7.47
    Buy 2 get 1 free: Chips 50g                      -Rs 20.00
    3 for Rs 80.00: Biscuits                         -Rs 10.00
    Coupon SAVE50                                    -Rs 50.00
  TOTAL                                              Rs 588.23
    incl. tax STANDARD @18%                           Rs 40.07
  Change                                             Rs 311.77
   return 1 of 3 chips -> refund Rs 12.29
   return 2 more chips -> refund Rs 24.58  (all three add up to the Rs 36.87 paid)
   inclusive total Rs 588.23 vs exclusive total Rs 644.73 (tax {EXEMPT=0.00, REDUCED=9.22, STANDARD=47.28})
   paneer on hand: -21  (sold 120 + 1 earlier; the till never refuses a real item)
   alert: PNR LOW - reorder (on hand 20)
   alert: PNR NEGATIVE - recount the shelf (on hand -1)
```

Chips qualified for both "buy 2 get 1" (₹20) and "3 for ₹50" (₹10): only the better applied. Returns refund the net paid, not the ₹20 shelf price. Four concurrent registers took paneer below zero without refusing a sale; each alert fired once.

Concurrency & edge cases

Case	Handling
Registers sell the same SKU at once	`ConcurrentHashMap.merge` (atomic per key)
Count says 0, item in hand	Sell; stock negative; one recount alert
Two desks refund the same line	`compute` in `claimReturn`; second over-claim refused
Price/offer change mid-sale	Line keeps scan price; breakdown frozen at `total()`
Card declined / cancel after card	Nothing recorded / non-cash payments reversed
Rounding in coupon split and refunds	Remainder to the largest SKU; refunds as differences of cumulative shares
One register, one cashier	`Register` / `Sale` single-threaded; shared parts are the thread-safe ones

Real-world example

Tills key produce by **PLU** codes (4011 = yellow bananas) and read an integrated scale; EAN-13 barcodes end in a check digit, so a smudged code is rejected instead of misread; tax-inclusive receipts (GST in India, VAT in the UK) show the tax contained, US receipts add sales tax — our two `TaxPolicy` strategies; cycle counts fix negative stock.

Extensions & follow-up questions

- **Loyalty / member prices** → a `CustomerContext` for the engine; points on the net paid.
- **Pre-packed weighed labels** → variable-measure barcodes decoded into a line.
- **Bundles (pasta + sauce)** → multi-SKU promotion; best-per-SKU becomes a small optimiser.
- **Refund to original tender** → keep tender splits on the receipt.
- **Offline mode** → queue authorisations under a floor limit; cache catalog and offers per register.
- **End of day** → `Shift` with float and expected cash.

In the interview

45 minutes: clarify (5) → entities + facade (8) → sale state machine (5) → **pricing engine and stacking** (12) → split payment + receipt (7) → returns, stock, concurrency (8).

> **30-second answer:** "A Register facade drives one Sale through OPEN, TENDERING, COMPLETED or CANCELLED; lines copy the price at scan; produce is priced by weight. A pure pricing engine gives each SKU its single best offer, applies a basket coupon, spreads it over SKUs so tax and refunds are exact, and computes tax through an inclusive or exclusive policy. Prices freeze at total; tenders are sealed; cash gives change; card and UPI go through a gateway. On completion an immutable receipt is stored and shared stock drops atomically with edge-triggered alerts — the till never refuses a real item."

- *Why spread the coupon?* Tax differs per category and returns must refund what was paid for that item.
- *Why stateless pricing?* Shared by all registers; reproducible for audits and returns.

Self-check

1. **Q:** Two offers match the chips — which applies? **A:** The larger discount only.
2. **Q:** Why is a one-chip refund ₹12.29? **A:** Net paid after the offer and the coupon share.
3. **Q:** What stops a double refund? **A:** `compute` makes check-and-add atomic.
4. **Q:** Why allow negative stock? **A:** The item is physically there; the count is wrong.

TL;DR

- **Register facade** → **Sale** state machine; scan-time prices; produce by **weight**.
- **Pure pricing engine**: best offer per SKU → coupon → spread → tax via **TaxPolicy**.
- Freeze at total; **split tender**; returns refund the **net paid**, claimed atomically; stock atomic, may go negative, **edge-triggered** alerts.

Chapter summary

Problem	Hard part	The move that solves it
Amazon	Guests vs members; last-item races; cancel rules	Ordering only on `Member` (LSP); atomic all-or-nothing reservation; order state machine
Brokerage	Four order types; tax lots; over-spending	Two abstract questions on `Order`; `StockLot` + FIFO `LotSale`; reserved funds
Stock exchange	Fair matching	Per-symbol `TreeMap<price, FIFO queue>`; price then time; one lock per symbol
Auction	Concurrent bids	All bid rules in one per-listing critical section; notify after unlocking
Music streaming	Shuffle/repeat without losing order	Strategy-produced permutation of indexes; plan rules in the enum
Food delivery / FoodKart	Capacity and riders under concurrency	Atomic kitchen reservation; CAS-claimed riders; strategies for assignment and selection
Grocery POS	Weighed items, stacking offers, split tender, returns	Pure pricing engine (best offer per SKU, coupon spread, tax per category); frozen price; atomic return claims; edge-triggered stock alerts

Recurring patterns: Strategy (payments, play order, rider assignment, restaurant selection, trade price, grocery offers and tax), Observer (notifications, fills, plays), state machines in enums (orders, auctions, player), Template Method (statements), Facade (every `*Service`), and one concurrency rule — **make check-then-act atomic** (lock or compare-and-set).

Sources: DesignGurus *Grokking the Object-Oriented Design Interview* (Amazon, stock brokerage); awesome-low-level-design (auction, music streaming, food delivery); kumaransg/LLD machine-coding statements (Navi stock exchange, Flipkart FoodKart); grocery POS designed from first principles (GS1 EAN-13, IFPS PLU codes). All Java code is original.

Chapter 20 — Case Studies: Pattern-Driven Components (Study Edition)

> **Goal.** These problems are not whole products but **building blocks** most systems contain: a ledger that splits money, a notification sender, a cache, a rider–driver matcher, a logger, a message broker and a rate limiter. Each is small enough to finish in an interview and each is **anchored on one or two design patterns**. Name the pattern and the change it absorbs, and half the question is answered.

Why components get their own chapter. A component has a small public surface (a few methods) and many hidden rules. The interviewer keeps asking "what if this rule changes?"; a good design answers "one new class, nothing else touched".

#	Component	Anchor pattern(s)	The change the pattern absorbs
20.1	Splitwise (expense sharing)	**Strategy** (split rules)	A new way to split a bill
20.2	Notification service	**Template Method** (retry loop), **Strategy** (send rules), **Observer** (outcomes), Simple Factory (channels)	A new channel, a new reason not to send, a new watcher of results
20.3	In-memory cache (+ LRU from scratch)	**Strategy** (eviction), **Observer** (removal listener), Builder	A new eviction rule
20.4	Ride sharing (Uber)	**Strategy** (matching, pricing, payment), **Decorator** (surge), State machine (trip)	A new matching rule, surge on/off, a new payment method
20.5	Logging framework	**Chain of Responsibility** (logger hierarchy), **Singleton** (LogManager), Strategy (formatter), Decorator (async appender)	A new sink, a new format, async writing
20.6	Pub-Sub system	**Observer / Publish-Subscribe**, append-only log with offsets	New subscribers, replay, slow consumers
20.7	Rate limiter	**Strategy** (algorithm), Simple Factory, Composite	A new algorithm, a new tier, combined limits

Each case follows the Chapters 14–21 template (problem → requirements → entities → diagrams → flows → decisions → code → edge cases → interview).

Running the code. Java 21, standard library only, under `code/lld/ch20/<name>/`, each package with a `Main`. From `chapters/20-case-components/code`:

```
javac --release 21 -d /tmp/lld_20 $(find . -name '*.java')
java -cp /tmp/lld_20 lld.ch20.splitwise.Main      # also: notification, cache, lrucache,
                                                  # ridesharing, carpool, logging, pubsub, ratelimiter
```

Money is always `BigDecimal`. A `double` cannot hold 0.1 exactly (`0.1 + 0.2` prints `0.30000000000000004`). Two rules: compare with `compareTo` (not `equals`, which also compares scale, so `2.0` ≠ `2.00`), and always give a rounding mode when dividing (`divide(x, 2, RoundingMode.HALF_EVEN)`), or 1/3 throws `ArithmeticException`.

Reading the course diagrams. Figures marked "(course diagram)" use one arrow legend throughout:

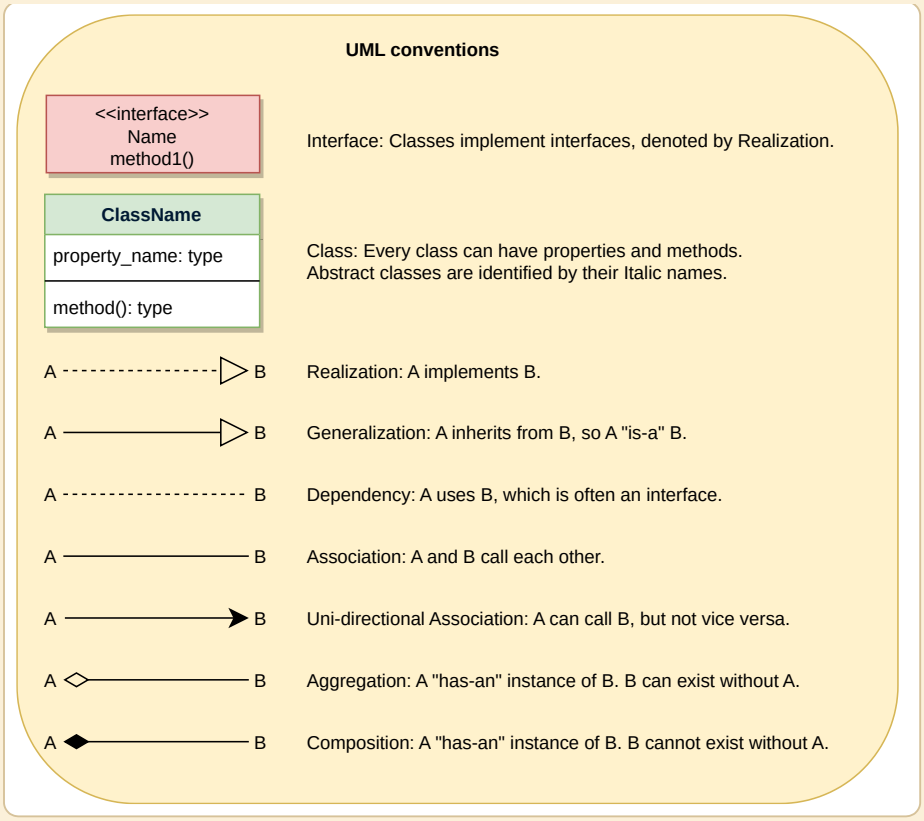


Figure — UML conventions used in the course class diagrams: interface, class, realization (implements), generalization (inherits), dependency, association, one-way association, aggregation and composition (course diagram)

The mermaid diagrams are drawn from **this chapter's Java code**.

20.1 Design Splitwise

Problem & clarifying questions

What it is. Splitwise remembers who paid for what when friends share costs and tells everyone who owes whom — the shared notebook for a weekend trip where Alice paid the hotel, Bob dinner, Carol the cab. The hard part is **not** the UI: it is deciding *which class owns the money arithmetic*, so the books balance after every operation.

Question	Assumed answer
Which split types?	Equal, exact, percentage. More later.
Must shares add up exactly?	Yes; otherwise the expense is rejected and no balance changes.
Rounding?	2-decimal money; no cent may be lost.
Pay only for yourself?	No: at least two people (payer counts) must be involved.
Settle up / simplify debts?	Yes / yes (on request, fewer transfers).
Currencies, receipts, undo?	Out of scope (say so).

Requirements

Functional: everything in the table, plus: users (name, email), groups, expense history per group, balance with one person and overall net. Invalid expense → **nothing changes**.

Non-functional: money correctness above all; a new split type changes no existing class (OCP); thread-safe.

Use cases

Actors: **Member** (adds expenses, settles up, asks what they owe), **Group creator**, **System** (computes shares, validates, updates the ledger, simplifies). *Add an expense* **includes** (always performs) *work out each share*, *check the shares add up*, *update the ledger*.

Core entities

Class	Responsibility
`User` (record)	Identity only: id, name, email.
`Group`	Name + member ids + expense ids. **Holds no balances.**
`SplitStrategy` + `EqualSplit`, `ExactSplit`, `PercentSplit`	Turn a total into one `Share` per person.
`Share` (record)	(person, amount) value object.
`Expense`	Description, total, payer, shares; **validates itself once** at creation.
`Ledger`	**One signed number per pair** of people; one mutator `adjust`.
`DebtSimplifier` / `Payment`	Greedy minimum-cash-flow → list of suggested transfers.
`Splitwise`	Facade; orders the steps, contains no arithmetic.
`Money`	2-decimal `BigDecimal` helpers.

Why the group holds no money: a person in two groups would have "Bob owes Alice" stored twice, and the copies would drift. One ledger = single source of truth. **`Group → User` is association, not composition:** deleting a group must not delete its people.

Class diagram

The course builds it in five stages, each leaving a working slice (copy this order in an interview): (1) `User`, `Group`; (2) `SplitStrategy` + rules + `Share`; (3) self-checking `Expense`; (4) `Ledger`; (5) `Splitwise` facade + `DebtSimplifier`.

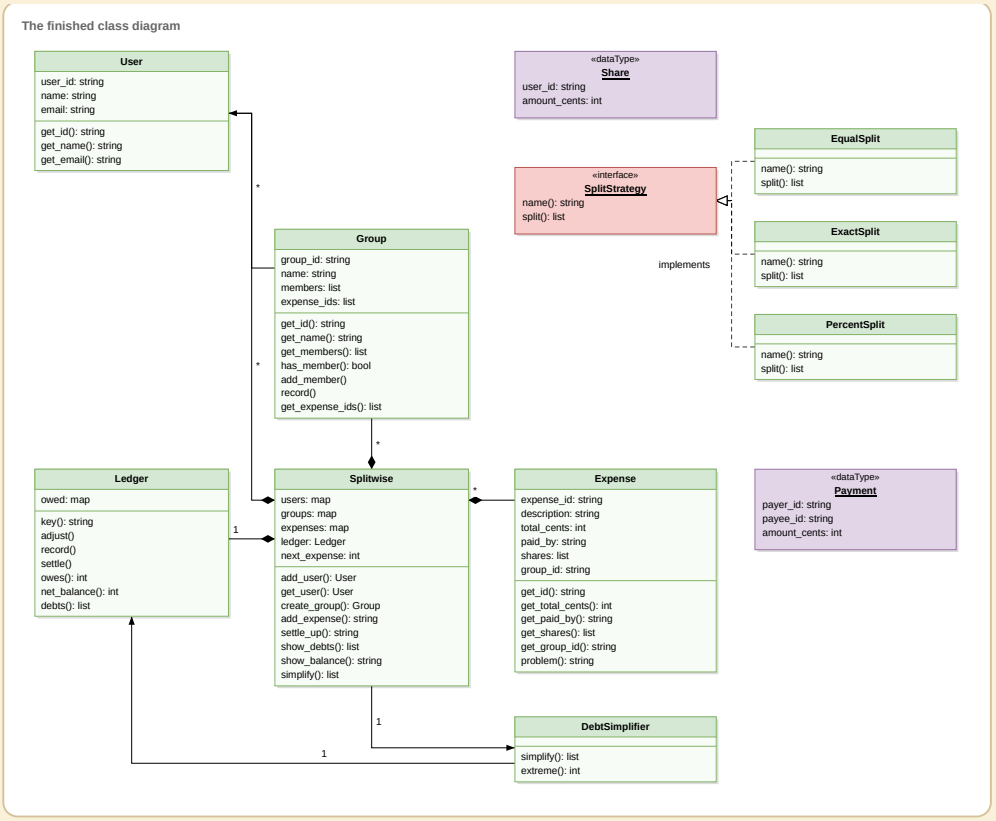
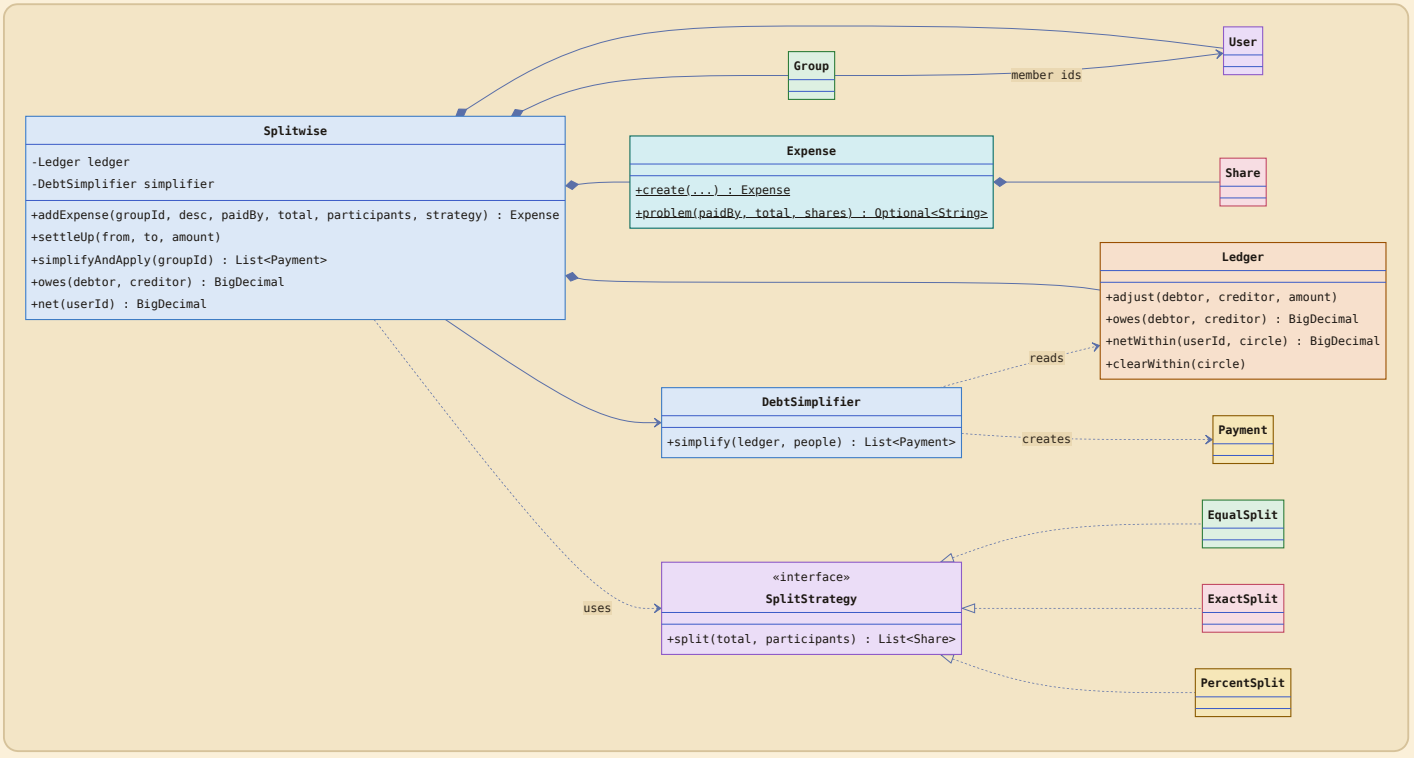


Figure — The finished Splitwise class diagram: service owning users, groups, expenses and the ledger, with the split strategies and the simplifier (course diagram)



Key flows

Adding an expense: strategy computes shares → `Expense.create` validates (≥ 2 people, sum = total) → only then, for each share, `ledger.adjust(sharer, payer, amount)`. If validation fails, an `IllegalArgumentException` with the reason is thrown and nothing is recorded.

Simplifying debts (greedy minimum cash flow):

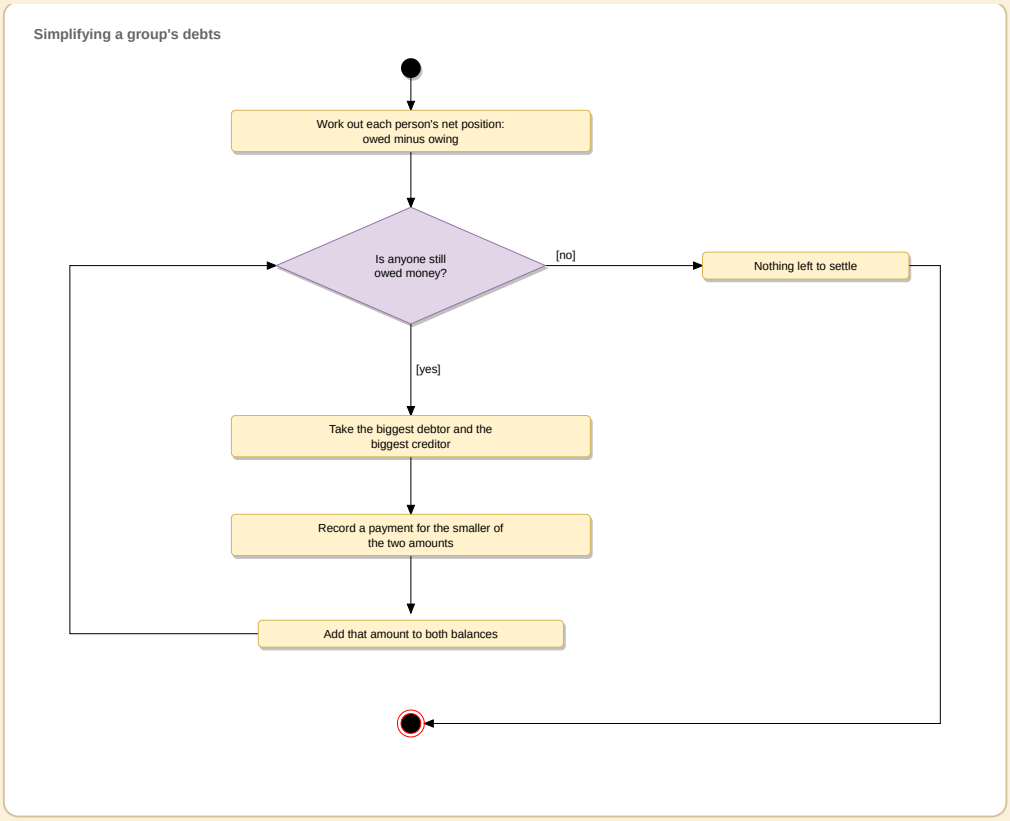


Figure — Simplifying debts: compute net positions, then the largest debtor repeatedly pays the largest creditor until every balance is zero (course diagram)

1. Net position per member = owed to them − they owe. Positive = creditor; nets sum to zero.
2. Creditors in one max-heap, debtors in another (ties by name → deterministic).
3. Pop biggest creditor C and debtor D; x = min(C, D); suggest "D pays C x".
4. Push back whichever side has a remainder; stop when a heap is empty.

Each round zeroes at least one person → **at most n − 1 payments**. It is **not always the true minimum** (that is NP-hard — it means finding the most zero-sum subgroups). Say so before you are asked.

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for split rules	A new rule = a new class; all return `List<Share>`, so nothing downstream knows which ran.	`if (type == EQUAL) … else if …` in the service.
Expense validates in a **static factory**	Checked once; an invalid expense can never exist.	Checking in the ledger on every read.
`problem()` returns the **reason**	User sees *why* ("shares add up to 15.00 but the total is 30.00").	`boolean isValid()`.
One signed number per unordered pair	A debt can't disagree with itself; reverse debts cancel automatically.	Two maps "A owes B" / "B owes A".
One mutator `adjust`	Expenses, settle-ups, simplification share one cancel rule.	Separate `addDebt`, `settle`, `reduce`.
`DebtSimplifier` separate	SRP: keeping balances vs planning settlements.	Algorithm inside the ledger.
`BigDecimal`, spare cents handed out	100.00/3 → 33.34 + 33.33 + 33.33; percent split: last person absorbs remainder.	`double` (the course's whole-cents `long` is equally correct).
`Splitwise` is a **Facade**	One entry point that orders steps and decides nothing.	UI calling ledger/strategies directly.

Only one GoF pattern (Strategy) — fine: good designs use the pattern that absorbs the likely change, not a collection.

Java implementation

Key excerpts (the split strategy, the self-validating expense, the ledger's single mutator, and the facade's ordering):

```
// File: code/lld/ch20/splitwise/SplitStrategy.java
interface SplitStrategy {
    List<Share> split(BigDecimal total, List<String> participants);
}

/** Equal split. Spare cents are handed out one at a time, in participant order. */
final class EqualSplit implements SplitStrategy {
    @Override
    public List<Share> split(BigDecimal total, List<String> participants) {
        if (participants.isEmpty()) throw new IllegalArgumentException("nobody to split between");
        BigDecimal n = BigDecimal.valueOf(participants.size());
        BigDecimal base = total.divide(n, 2, RoundingMode.DOWN);           // 100.00/3 -> 33.33
        BigDecimal leftover = total.subtract(base.multiply(n));           // 0.01
        int spareCents = leftover.divide(Money.CENT).intValueExact();     // 1
        List<Share> shares = new ArrayList<>();
        for (int i = 0; i < participants.size(); i++) {
            BigDecimal amount = i < spareCents ? base.add(Money.CENT) : base;
            shares.add(new Share(participants.get(i), amount));
        }
        return shares;
    }
}
// ... ExactSplit, PercentSplit (last person absorbs the remainder)

// File: code/lld/ch20/splitwise/Expense.java
    static Optional<String> problem(String paidBy, BigDecimal total, List<Share> shares) {
        if (total.signum() <= 0) return Optional.of("total must be positive");
        // ... negative / duplicate share checks
        Set<String> people = new HashSet<>(sharers);
        people.add(paidBy); // the payer counts as one of the two people even if they owe no share
        if (people.size() < 2) return Optional.of("an expense needs at least two people");
        BigDecimal sum = shares.stream().map(Share::amount).reduce(BigDecimal.ZERO, BigDecimal::add);
        if (sum.compareTo(total) != 0)
            return Optional.of("shares add up to " + sum + " but the total is " + total);
        return Optional.empty();
    }

// File: code/lld/ch20/splitwise/Ledger.java  (key (lo, hi), lo < hi; positive = "hi owes lo")
    /** The ONLY mutator: 'debtor' now owes 'creditor' 'amount' more. Opposite debts cancel. */
    void adjust(String debtor, String creditor, BigDecimal amount) {
        if (debtor.equals(creditor) || amount.signum() == 0) return;
        Pair key = Pair.of(debtor, creditor);
        BigDecimal delta = creditor.equals(key.lo()) ? amount : amount.negate();
        BigDecimal updated = balances.getOrDefault(key, Money.ZERO).add(delta);
        if (updated.signum() == 0) balances.remove(key); else balances.put(key, updated);
    }

// File: code/lld/ch20/splitwise/Splitwise.java
        List<Share> shares = strategy.split(total, participants);       // may throw
        Expense e = Expense.create("e" + ids.incrementAndGet(), description, paidBy, total, shares, groupId);
        for (Share s : e.shares()) ledger.adjust(s.userId(), paidBy, s.amount()); // payer's own share is a
no-op
    // ... settleUp: refuse paying more than is owed, then ledger.adjust(to, from, amount) - the reverse debt
cancels
```

Full runnable code: code/lld/ch20/splitwise/ (see the Full Edition).

Notes: `Money.of("10.005")` throws (`RoundingMode.UNNECESSARY` refuses silent rounding); `PercentSplit` rounds with `HALF_EVEN` (banker's rounding); `DebtSimplifier` uses two `PriorityQueue`s.

Output (excerpt):

```
Hotel shares: [alice=33.34, bob=33.33, carol=33.33]
  refused: shares add up to 15.00 but the total is 30.00
  suggested: dave pays alice 16.68
  suggested: carol pays alice 13.33
  suggested: bob pays alice 9.99
```

Four pairwise debts became three payments; every net (alice +40.00, bob −9.99, carol −13.33, dave −16.68) is unchanged.

Related machine-coding problem: The Ledger Co. (Navi)

The Geektrust/Navi round "Ledger Co." also keeps money records, but for **bank loans** (simple interest, monthly EMI, lump-sum prepayments). It is built in **Chapter 21 (The Ledger Co.)**. Both keep **one authoritative record** and compute every answer from it, and both use `BigDecimal` with a named rounding mode; the rules differ (*who owes whom* vs *how much is repaid by month N*).

Concurrency & edge cases

- **Concurrent expenses:** every public `Splitwise` method is `synchronized` (build → validate → adjust is atomic); at scale lock per group/pair or use a DB transaction.
- **Payer not a sharer** (Alice buys Bob's lunch) is valid; the payer's own share is a no-op `adjust`.
- **Overpaying on settle-up**, unknown users, duplicate/negative shares, zero totals: refused before anything changes.
- **Simplify touches other groups:** one number per *pair* means simplifying a group rewrites all debts between its members, even ones from another group. If that is unacceptable, keep per-group ledgers plus a derived global view.

Extensions & follow-up questions

- **Split by shares (2:1:1)?** New `ShareRatioSplit`; nothing else changes.
- **Delete/edit an expense?** Re-add the inverse, or better make the ledger an **append-only journal** (event sourcing): delete = append a reversing entry.
- **Is greedy optimal?** No. Nets A −6, B −5, C +2, D +4, E +5: greedy needs 4 payments; optimal 3 (B→E 5, A→D 4, A→C 2).
- **Is `Splitwise` doing too much?** At scale, yes: give users, groups, expenses and ledger their own repositories. Multiple currencies: `Money(amount, currency)` + one ledger per currency.

Real-world example

Splitwise really offers "simplify debts" for groups (the net-position idea). Bill-splitting in payment apps solves rounding the same way: one person absorbs the leftover cent.

In the interview

30-second answer: "Users and groups hold no money. Each split rule is a `SplitStrategy` returning shares. An `Expense` validates itself once: at least two people, shares add to the total. The `Ledger` keeps one signed number per pair and has a single `adjust`, so debts cancel automatically; settling up is an adjust the other way. Simplification: net positions + greedy biggest-debtor-pays-biggest-creditor, at most n − 1 payments, not always optimal."

Tip: say "one class per split rule" early; spend your time on the ledger.

Self-check

1. Why does `Group` hold no balances? → Each debt must live in one place; two copies drift.
2. Split 10.00 equally among 3. → 3.34, 3.33, 3.33.

3. Max payments from the greedy simplifier for n people? → n − 1.
4. Why `compareTo` for `BigDecimal`? → `equals` compares scale: `2.0` ≠ `2.00`.
5. Alice pays 20.00, Bob owes all of it — valid? → Yes, two people are involved.

TL;DR

- **Strategy** for split rules; **Expense validates itself** once.
- **Ledger = one signed number per pair**, one mutator; settle-up = reverse adjust.
- **Simplify** = net positions + greedy; ≤ n − 1 payments; optimal is NP-hard.
- **BigDecimal**, explicit rounding, no lost cents.

20.2 Design a Notification Service

Problem & clarifying questions

What it is. The component that sends emails, SMS and push alerts ("Your order has shipped", "482913 is your login code"). Analogy: a post-office counter that picks the delivery method, respects "do not disturb", retries if the courier fails, and keeps a receipt. **Why it is a design question:** it starts as one `sendEmail()` and grows fourteen `if`s (SMS, quiet hours, promo limits…). The exercise is *where each addition should have gone instead*.

Question	Assumed answer
Channels?	Email, SMS, push; more later.
Who picks the channel?	The user, **per kind** of notification, in preference order.
Wording?	Templates with placeholders: `Hi {name}, it arrives {eta}.`
Quiet hours?	Per user, may wrap midnight (22–07); urgent (OTP) ignores them.
Rate limits?	≤ N per user per kind per hour.
Failures?	Retry a channel a bounded number of times, then fall back to the next channel; all fail → FAILED.
Audit?	Every attempt recorded: SENT / FAILED / SUPPRESSED + reason.
Sync or async?	Caller must not wait for a slow gateway.

Requirements

Functional: the table above, plus: user has email, phone, device token; FAILED when no channel succeeds; **a new channel or a new reason to hold back needs no edit to the service.**

Non-functional: caller not blocked (async); thread-safe; testable without real time (injected clock); a broken listener never breaks delivery.

Use cases

Actors: **Product code** (asks), **Person** (channels, quiet hours), **Operator** (adds channels, watches outcomes), **Service** (renders, checks rules, tries channels, retries, records).

Core entities

Class	Responsibility
`User` / `Preferences`	Who someone is vs what they accept (ordered channels per kind, quiet hours with wrap logic in **one** place). Separate because they change for different reasons (SRP).
`Template` / `Message`	Subject + body with `{holes}`; `render(values)` → identical `Message` for every channel.
`NotificationRequest`	kind, user, values, priority (NORMAL / URGENT).
`Gateway`	External provider (SMTP, SMS gateway, push service); swappable and fakeable.
`NotificationChannel` (abstract) + Email/Sms/Push	**Template Method**: final `deliver()` has the retry-with-backoff loop; subclasses give `addressOf(user)`.
`ChannelFactory`	**Simple Factory**: type → channel.
`RetryPolicy`	Max attempts + initial backoff (doubles).
`SendRule` + `QuietHoursRule`, `RateLimitRule`	**Strategy**: one reason not to send, returned as text.
`DeliveryRecord` / `DeliveryListener`	Outcome record; **Observer** (`LoggingListener`, `MetricsListener`).
`TimeSource`	Where "now" comes from (manual clock in tests).
`NotificationService`	Thin coordinator: rules → preferences → render → channels in order → `finish()`.

Class diagram

Course build stages: (1) `User` + `Preferences`; (2) `Template` + `Message` + request; (3) channel hierarchy; (4) `SendRule`s; (5) service + listeners.

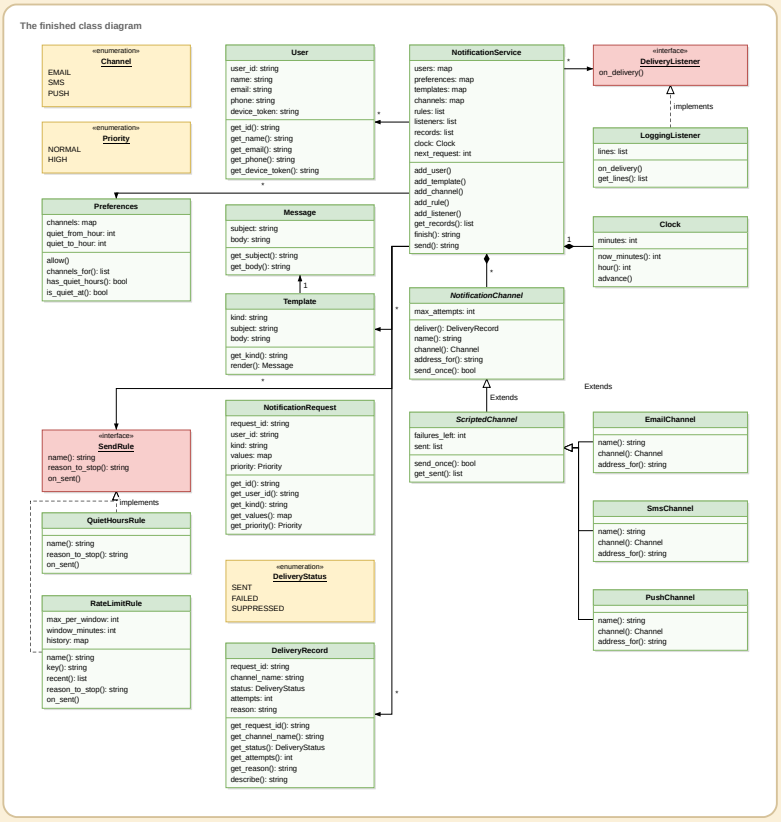
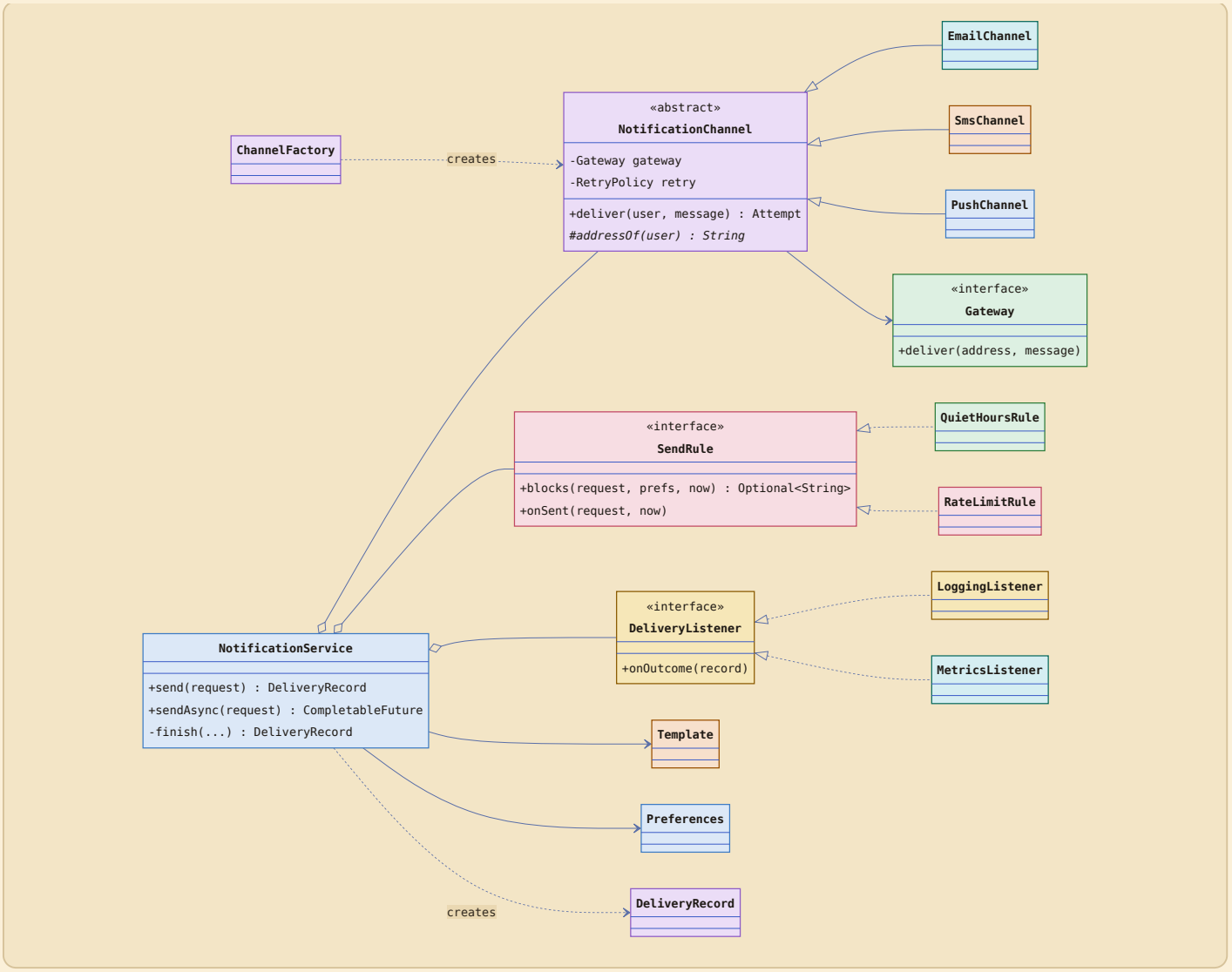


Figure — The finished notification service class diagram: the service in the middle, the channel hierarchy with the retry in the parent, the send rules, templates, preferences and the delivery listeners (course diagram)



Key flows

Sending one notification: ask every rule (any reason → SUPPRESSED); no accepted channel → SUPPRESSED; render the template **once**; try each accepted channel in the user's order (each with its own retry loop); first success → `onSent` to rules, SENT; all fail → FAILED. Every path exits through `finish()`.

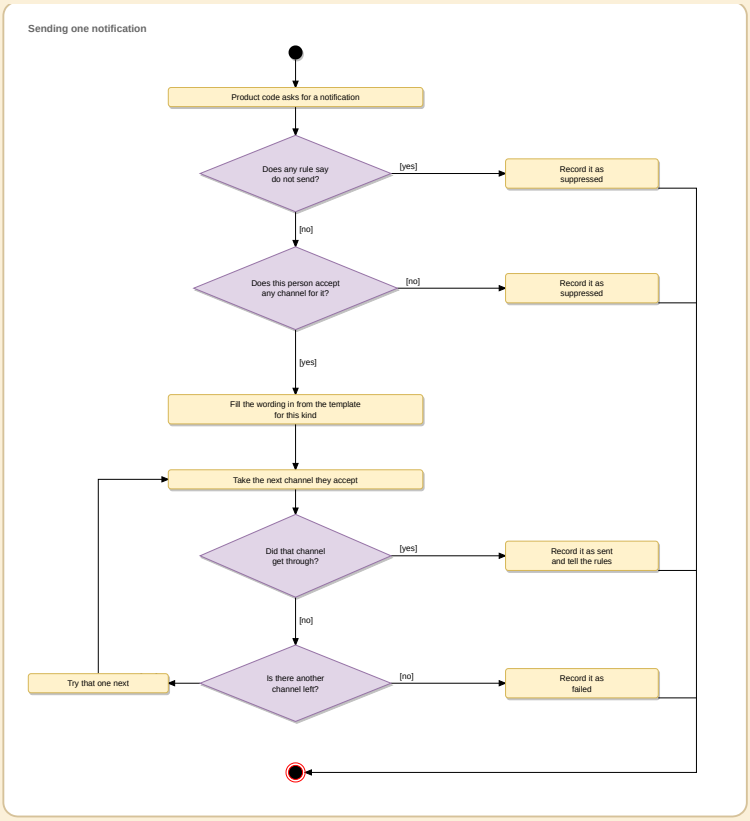


Figure — Sending one notification: ask every rule, render once, try each accepted channel in turn, and record sent / failed / suppressed (course diagram)

Design decisions & patterns used

Pattern / decision	Where	Why
Template Method	`final deliver()`; subclasses fill `addressOf()`	Retry loop is identical for all channels; only a *step* varies. Strategy would replace the whole method.
Strategy	`SendRule`	Each "reason not to send" is a class; a new rule (unsubscribed, dedupe) = new class + one wiring line.
Observer	`DeliveryListener`	Logs, dashboards, alerts want outcomes; the service doesn't know who listens.
Simple Factory	`ChannelFactory`	One place maps type → class.
Adapter (target)	`Gateway`	One adapter per vendor SDK; tests use a fake.
Rule returns **reason text**	`SendRule.blocks`	Reason is recorded.
Urgency handled **inside** the rule	`QuietHoursRule`	Service needn't know which rules urgent ignores.
Rate limit counts only **successful** sends	`onSent` hook	Suppressed/failed messages don't use up the allowance.
Single exit `finish()`	Service	Every path stores a record and notifies listeners.
Render **once**	Service	Same text everywhere; wording change = one edit.

Three patterns, three *separate* axes of change: another channel (Template Method + Factory), another reason to hold back (Strategy), another interested party (Observer).

Java implementation

```java
// File: code/lld/ch20/notification/Channels.java
abstract class NotificationChannel {
    private final Gateway gateway;
    private final RetryPolicy retry;
    // ... constructor
    abstract ChannelType type();

    /** Hook: where this channel reaches the user, or null if the user has no such address. */
    protected abstract String addressOf(User user);

    final Attempt deliver(User user, Message message) {
        String address = addressOf(user);
        if (address == null || address.isBlank()) return new Attempt(false, 0, "no " + type() + " address");
        String lastError = null;
        long backoff = retry.initialBackoff().toMillis();
        for (int attempt = 1; attempt <= retry.maxAttempts(); attempt++) {
            try {
                gateway.deliver(address, message);
                return new Attempt(true, attempt, null);
            } catch (Exception e) {
                lastError = e.getMessage();
                if (attempt < retry.maxAttempts()) sleep(backoff);
                backoff *= 2;
            }
        }
        return new Attempt(false, retry.maxAttempts(), lastError);
    }
    // ...
}

final class SmsChannel extends NotificationChannel {
    SmsChannel(Gateway g, RetryPolicy r) { super(g, r); }
    ChannelType type() { return ChannelType.SMS; }
    protected String addressOf(User u) { return u.phone(); }
}

// File: code/lld/ch20/notification/NotificationService.java  (inside send)
        for (SendRule rule : rules) {
            Optional<String> reason = rule.blocks(req, prefs, now);
            if (reason.isPresent()) return finish(req, DeliveryStatus.SUPPRESSED, null, 0, reason.get());
        }
        // ... no accepted channel -> SUPPRESSED; render the template ONCE (missing value -> FAILED via
finish)
        for (ChannelType type : accepted) {
            NotificationChannel channel = channels.get(type);
            if (channel == null) continue;
            Attempt a = channel.deliver(user, message);
            totalAttempts += a.attempts();
            if (a.success()) {
                rules.forEach(r -> r.onSent(req, now));
                return finish(req, DeliveryStatus.SENT, type, totalAttempts, null);
            }
            lastError = type + ": " + a.error();
        }
        return finish(req, DeliveryStatus.FAILED, null, totalAttempts, "all channels failed, last " +
lastError);
```

Full runnable code: code/lld/ch20/notification/ (see the Full Edition).

Notes: `Preferences.isQuiet` handles wrap-around (start ≤ end → range; else "after start **or** before end"). A missing template value is an error, not a raw `{name}` in an inbox. Backoff doubles (the course retried immediately — a named weakness). `RateLimitRule` prunes old timestamps (the course's version leaked them). A throwing listener is caught in `finish()`. `sendAsync` uses a pool; production uses a durable queue.

Output (excerpt):

```
== 2. SMS fails 3 times -> retried, then falls back to EMAIL
    SMTP -> asha@mail.com : Order A-17 shipped | Hi Asha, it arrives Tuesday.
  [log] SENT       ORDER_SHIPPED user=u1 channel=EMAIL attempts=4
  [log] SUPPRESSED PROMO         user=u1 channel=- attempts=0 reason=quiet hours
  [log] SUPPRESSED PROMO         user=u1 channel=- attempts=0 reason=rate limit 2 per 60 min
```

Concurrency & edge cases

- Shared state: `ConcurrentHashMap` for users/prefs/templates, `CopyOnWriteArrayList` for listeners/history, rate-limit deque locked per key.
- **Check-then-act gap:** two concurrent sends can both pass `blocks()` before `onSent()`. Fix: reserve a slot atomically at check time (release on failure) or serialize per user.
- **Retries block worker threads** with `sleep`; real systems re-enqueue with a delay (retry topic).
- **Idempotency:** a retry after a timeout may deliver twice → pass an idempotency key; receivers tolerate duplicates.
- Missing address → 0-attempt failure, next channel. No template → FAILED; no accepted channel → SUPPRESSED.
- **Time zones:** quiet hours in the *user's* zone (`ZoneId` in preferences).

Extensions & follow-up questions

- **Add WhatsApp:** one subclass, one enum value, one `Gateway` adapter.
- **Too many messages:** a new `SendRule` (daily cap, dedupe), not a new `if`.
- **Campaign to a million users:** fan-out stage onto a queue + many workers calling `send`; rate-limit per provider.
- Priorities (separate queues), localization (templates by kind + locale), webhooks for receipts, a **circuit breaker** per provider.

Real-world example

Push goes through **FCM** (Android) and **APNs** (iOS) to a *device token*; email via SES/SendGrid, SMS via Twilio — each its own API, hence `Gateway`. **AWS SNS** fans one message out to many channels. LinkedIn's **Air Traffic Controller** decides whether/when/how to notify each member — `SendRule` at company scale. Phone "Do Not Disturb" = quiet hours with an urgent bypass.

In the interview

30-second answer: "Channels sit behind an abstract class whose final `deliver` holds retry-with-backoff (Template Method); each channel supplies only its address. Each do-not-send condition is a `SendRule` (Strategy) returning a reason. Preferences give an ordered channel list per kind; we render once and try channels in order, falling back on failure. Every outcome goes through one `finish` that records and notifies listeners (Observer). For scale, a queue sits in front."

Follow-ups: *Why not Strategy for channels?* → Channels share an algorithm and vary in one step: Template Method (injecting `RetryPolicy` + a `Sender` strategy is a valid composition alternative). *Duplicates on retry?* → Idempotency keys; at-least-once delivery.

Self-check

1. Which pattern puts the retry loop in one place? → Template Method (`final deliver()`).
2. Why `Optional<String>` from `blocks`? → The suppression reason is recorded.
3. Why not count suppressed sends against the rate limit? → The user never got them.
4. Quiet hours 22–07: is 06:00 quiet? 12:00? → Yes; no.
5. What must every exit of `send` do? → Go through `finish()`.

TL;DR

- **Template Method** channels (retry once in the parent); **Strategy** send rules returning reasons; **Observer** outcomes; **Factory** channels; **Adapter** providers.
- Render once, try channels in order, fall back, single `finish()` exit.
- Scale with a queue; watch idempotency and the rate-limit check-then-act gap.

20.3 Design an In-Memory Cache (and an LRU Cache from scratch)

Problem & clarifying questions

`What it is.` A small, fast store of recently used values, so you don't re-fetch them from a slow place (DB, another service). Analogy: the few books on your desk vs the library. When the desk is full, one book must go back — **which one** is the eviction policy, the rule most likely to change. Expect the question: *which data structure gives O(1) get and put?* (O(1) = time doesn't grow with the number of entries.)

Question	Assumed answer
Capacity in?	Number of entries.
Eviction?	LRU by default; swappable for FIFO or LFU without changing the cache.
TTL?	Optional per entry; an expired read is a miss and removes it.
Stats / notifications?	Hits, misses, evictions, expirations; subscribers learn about removals and the cause.
Thread-safe?	Yes (the course left it out; we add it).
Load on miss / write-back?	Out of scope (extensions).

Requirements

Functional: the table above, plus: `put` overwrites; a full cache evicts before inserting; O(1) get/put; invalidate one key or clear all; **every entry leaves through one code path**.

Non-functional: O(1); thread-safe; testable TTL (injected clock); listeners can't slow down or deadlock the cache.

Use cases

Actors: **Application** (read, store, remove), **Operator** (chooses rule, reads stats), **Cache** (evicts, expires, counts, notifies).

Core entities

Class	Responsibility
`Node<K,V>`	key, value, `expiresAtMillis`, `frequency`, own `prev`/`next`.
`DoublyLinkedList<K,V>`	Order, newest at front, two **sentinel** nodes (no null checks).
`EvictionPolicy<K,V>`	**Strategy**: `onInsert`, `onAccess`, `onRemove`, `victim()`. Owns the *order*.
`LruPolicy`, `FifoPolicy`, `LfuPolicy`	The three rules; LFU is O(1) with chained frequency buckets.
`EvictionPolicies` + `PolicyType`	**Simple Factory**.
`Cache<K,V>` + `Cache.Builder`	Map + policy + clock + stats + listeners; one lock; **one removal method**. **Builder** for config.
`CacheStats`, `RemovalReason`, `EvictionListener`	`LongAdder` counters; EVICTED/EXPIRED/REMOVED/CLEARED; **Observer**.
`ShardedCache<K,V>`	N caches chosen by key hash → less lock contention.

The core idea: two structures, each answering one question.

Question	Structure	Cost
"Where is key k?"	`HashMap<K, Node>`	O(1)
"Who leaves next?"	Doubly linked list by recency	O(1) at the ends
"Move this entry to the front"	Node has own links; the map hands it over	O(1), no search

A map can't name the oldest entry; a list can't find a key without walking. **Doubly** linked because unlinking a middle node needs its predecessor.

Class diagram

Course build order: (1) `Node` + list; (2) `EvictionPolicy` + LRU; (3) FIFO + LFU with no cache change; (4) `Cache` with clock, stats, TTL; (5) listeners. **Who owns what:** the cache owns the map (and nodes); the policy owns the *order*. Neither reaches into the other — that is why FIFO/LFU need no `Cache` change.

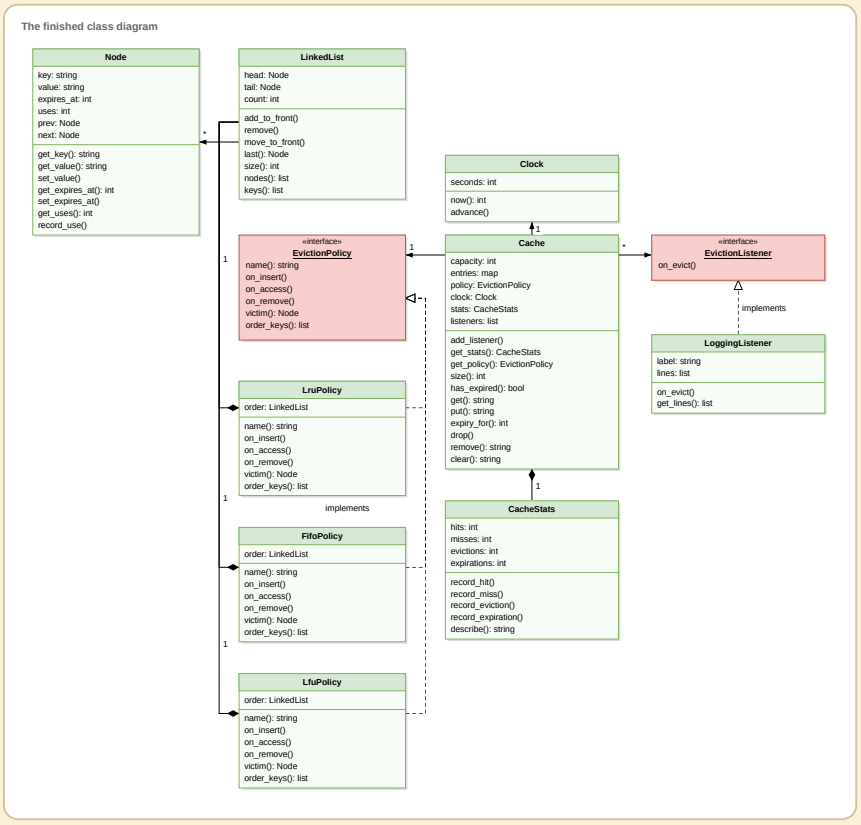
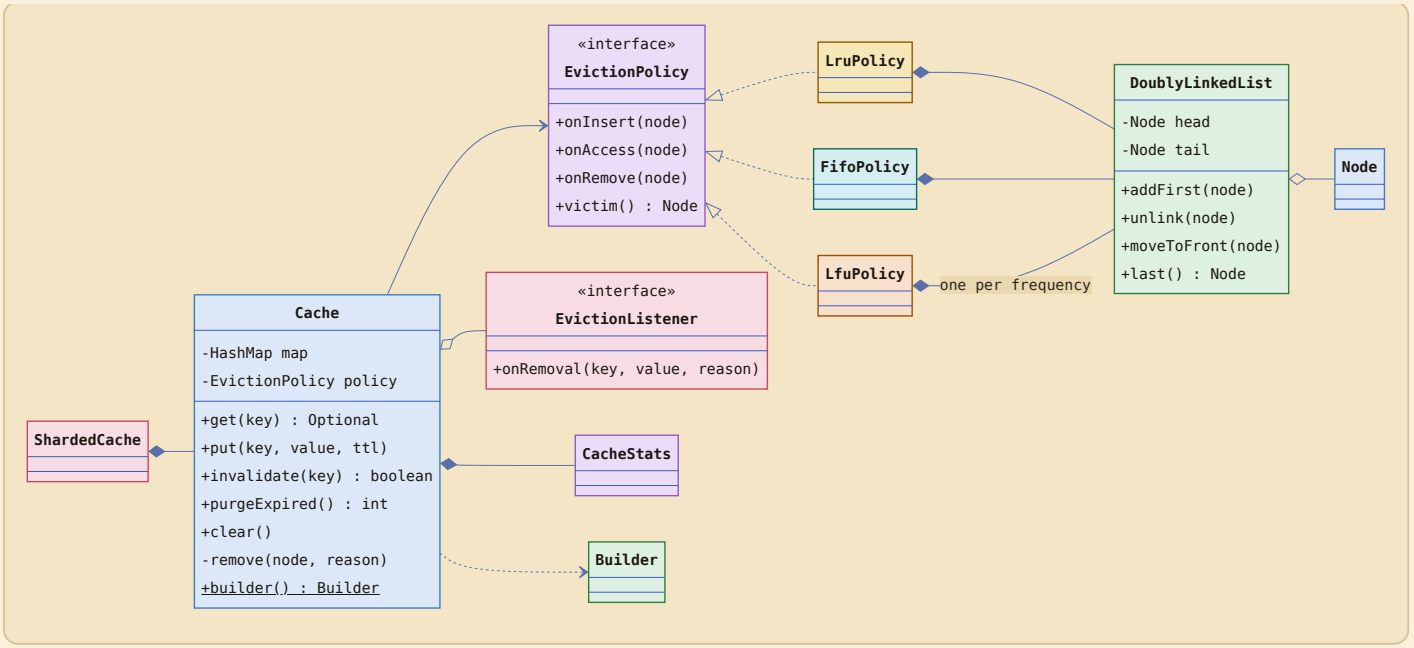


Figure — The finished cache class diagram: Cache with its map, policy, clock, stats and listeners; the three eviction policies behind one interface (course diagram)



Key flows

Read: absent → miss; present but expired → miss + removal (EXPIRED); else hit, `policy.onAccess`, return.
Store: replace if present (counts as a use); otherwise, only if full, ask `policy.victim()` and remove it through the single removal path, then insert. Listeners are called after the lock is released.

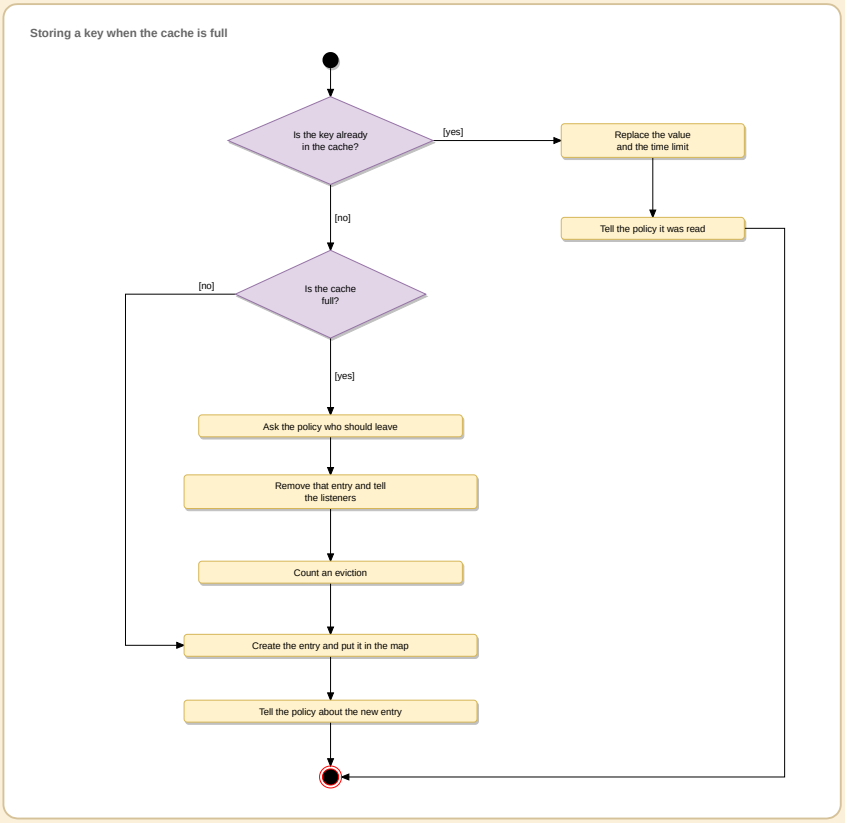


Figure — Storing a key: replace if present; otherwise, if full, ask the policy for a victim and remove it through the single removal path, then insert (course diagram)

How the three policies differ:

Event	LRU	FIFO	LFU (O(1) buckets)
`onInsert`	add to front	add to front	frequency = 1, bucket 1 (always first in chain)
`onAccess`	move to front	**nothing**	move bucket f → f+1 (created right after f); drop f if empty
`onRemove`	unlink	unlink	unlink; drop bucket if empty
`victim()`	back of list	back of list	back of the **first** bucket (LRU among least frequent)

FIFO is LRU with one empty method — evidence of good interface boundaries. The course's LFU scanned all entries (O(n)); ours keeps buckets in a **doubly linked chain in increasing frequency**, so the lowest bucket is always first and every operation is O(1) (Shah, Mitra & Matani, 2010).

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for eviction	The rule most likely to change.	`switch (policyType)` in get/put.
Observer for removals, with reason	Logging, metrics, cleanup don't belong in the cache.	Hard-coded callbacks.
One removal method `remove(node, reason)`	Eviction, expiry, removal, clear all update map, policy, stats alike.	Four copies of clean-up code.
Sentinel nodes	Add/unlink are always the same pointer writes.	Null checks everywhere.
Injected clock (`LongSupplier`)	TTL tests run instantly.	`System.currentTimeMillis()` → tests must sleep.
Lazy expiry + optional sweeper	Dropped on read; `purgeExpired()` reclaims memory.	Timer per entry.
One `ReentrantLock`, listeners **after unlock**	LRU `get` writes (moves a node), so a read-write lock doesn't help; slow/re-entrant listeners can't block or deadlock.	`synchronized` around listener calls.
Builder	Defaults + validation once.	Telescoping constructors.
Sharding	N locks; threads on different shards never wait.	One global lock.

Java implementation

```java
// File: code/lld/ch20/cache/EvictionPolicy.java
interface EvictionPolicy<K, V> {
    void onInsert(Node<K, V> node);
    void onAccess(Node<K, V> node);
    void onRemove(Node<K, V> node);
    Node<K, V> victim();
}

final class LruPolicy<K, V> implements EvictionPolicy<K, V> {
    private final DoublyLinkedList<K, V> order = new DoublyLinkedList<>();
    public void onInsert(Node<K, V> n) { order.addFirst(n); }
    public void onAccess(Node<K, V> n) { order.moveToFront(n); }
    public void onRemove(Node<K, V> n) { order.unlink(n); }
    public Node<K, V> victim() { return order.last(); }
}
// ... FifoPolicy (onAccess does nothing), LfuPolicy (chained frequency buckets)

// File: code/lld/ch20/cache/Cache.java
    void put(K key, V value, Duration ttl) {
        List<Removal<K, V>> removed = new ArrayList<>(1);
        lock.lock();
        try {
            long expiresAt = ttl == null ? 0 : clockMillis.getAsLong() + ttl.toMillis();
            Node<K, V> existing = map.get(key);
            if (existing != null) {          // replace: counts as a use
                existing.value = value;
                existing.expiresAtMillis = expiresAt;
                policy.onAccess(existing);
                return;
            }
            if (map.size() >= capacity) removed.add(remove(policy.victim(), RemovalReason.EVICTED));
            Node<K, V> n = new Node<>(key, value, expiresAt);
            map.put(key, n);
            policy.onInsert(n);
        } finally {
            lock.unlock();
            notifyListeners(removed);
        }
    }
    // ... get, invalidate, purgeExpired, clear follow the same lock / collect / notify-after-unlock shape

    /** THE single exit path for every entry, whatever the reason. Caller holds the lock. */
    private Removal<K, V> remove(Node<K, V> n, RemovalReason reason) {
        map.remove(n.key);
        policy.onRemove(n);
        if (reason == RemovalReason.EVICTED) stats.evictions.increment();
        if (reason == RemovalReason.EXPIRED) stats.expirations.increment();
        return new Removal<>(n.key, n.value, reason);
    }
```

Full runnable code: code/lld/ch20/cache/ (see the Full Edition).

Output (excerpt) — same accesses (b read 3×, c once, a twice), three different victims:

```
== LRU (capacity 3): put a,b,c; get b,b,b,c,a,a; put d
  listener: b=2 left, EVICTED
== FIFO (capacity 3): put a,b,c; get b,b,b,c,a,a; put d
  listener: a=1 left, EVICTED
== LFU (capacity 3): put a,b,c; get b,b,b,c,a,a; put d
  listener: c=3 left, EVICTED
  stats: hits=2 misses=1 evictions=0 expirations=2 hitRate=67%
```

LRU Cache from scratch (the classic interview question)

Often asked directly: **"Implement an LRU cache with O(1) get and put"** (LeetCode 146). Same two structures, inline: HashMap for lookup; doubly linked list with `head`/`tail` sentinels (most recent after `head`). `get`: unlink + add at front. `put`: if present update + move to front; else if full remove `tail.prev` from list **and** map (why the node stores its key), then add. Java shortcut: `LinkedHashMap(accessOrder = true)` + `removeEldestEntry` — mention it, but write the manual version when asked "from scratch".

```
// File: code/lld/ch20/lrucache/Main.java
        synchronized V get(K key) {
            Node n = index.get(key);
            if (n == null) return null;
            unlink(n);
            addFront(n);                // now the most recently used
            return n.value;
        }

        synchronized void put(K key, V value) {
            Node n = index.get(key);
            if (n != null) {            // update + mark as recently used
                n.value = value;
                unlink(n);
                addFront(n);
                return;
            }
            if (index.size() == capacity) {
                Node lru = tail.prev;    // least recently used lives just before the tail sentinel
                unlink(lru);
                index.remove(lru.key);
            }
            n = new Node(key, value);
            addFront(n);
            index.put(key, n);
        }
        // ... addFront / unlink: four and two pointer writes
    // ... jdkLru(capacity): new LinkedHashMap<>(16, 0.75f, true) overriding removeEldestEntry -> size() > capacity
```

Full runnable code: code/lld/ch20/lrucache/ (see the Full Edition). The demo prints LeetCode 146's expected values (1, miss, miss, 3, 4); this generic version returns `null` for a miss instead of `-1`.

Approach	get / put	Thread-safe	When to use
HashMap + own doubly linked list	O(1) / O(1)	Only with a lock	Interviews; custom behaviour
`LinkedHashMap(accessOrder=true)` + `removeEldestEntry`	O(1) / O(1)	No	Quick single-threaded caches
Caffeine / Guava	amortized O(1)	Yes	Production Java

Concurrency & edge cases

- **A read lock is not enough:** LRU `get` rewrites pointers, so it needs the exclusive lock.
- **Contention:** `ShardedCache` = 8 locks; trade-off: LRU order is only per shard.
- **Cache stampede** (many misses for one key all hit the DB): let the first thread load and others wait on its result (`computeIfAbsent` / a `CompletableFuture` per key) — Chapter 13's TTL cache does this with `ConcurrentHashMap.compute`.
- **TTL precisely:** expires *at* the deadline (`now >= expiresAt`); lazy — a dead entry holds a slot until read, swept, or evicted (then reported `EVICTED`). `put` without TTL on an existing key **clears** the old TTL.
- Capacity ≤ 0 refused by the builder; nulls unsupported (Caffeine too); use a monotonic clock (`nanoTime`) in production.

Extensions & follow-up questions

- **LFU:** one new policy; naive LFU is O(n), chained buckets O(1).
- **Size in bytes:** a `Weigher` strategy; evict until under max weight.
- **Load on miss:** *read-through* (`get(key, loader)`), *write-through* (cache + DB together), *write-back* (flush later; faster, may lose data), *cache-aside* (app does check → DB → put).
- **Distributed:** consistent hashing (high-level design). **Best policy?** Workload-dependent; Caffeine's *Window TinyLFU* mixes recency and frequency.

Real-world example

`LinkedHashMap` access order is the JDK's LRU. **Guava/Caffeine** expose the same knobs: `maximumSize`, `expireAfterWrite`/`expireAfterAccess`, `removalListener` (with causes like `EXPIRED`, `SIZE`, `EXPLICIT`, `REPLACED`), `recordStats()`. **Redis** picks eviction by config (`maxmemory-policy`: `allkeys-lru`, `allkeys-lfu`, `volatile-ttl`, `noeviction`…) and approximates LRU by sampling keys.

In the interview

> **30-second answer:** "A HashMap key → node for O(1) lookup, and a doubly linked list with sentinels for O(1) ordering; the node carries its own links so we can unlink it directly. The eviction rule sits behind `EvictionPolicy` — LRU moves on access, FIFO doesn't, LFU uses frequency buckets for O(1). Every exit goes through one `remove` that updates stats; listeners are notified after the lock is released. One lock is correct; sharding reduces contention."

Follow-up: *Why does the node store the key?* → To delete it from the map on eviction.

Self-check

1. Which two structures give O(1) LRU? → HashMap (key → node) + doubly linked list.
2. What do sentinels buy? → No null checks; add/remove always the same writes.
3. FIFO vs LRU in code? → `onAccess` does nothing.
4. Why can't a read-write lock speed up LRU `get`? → `get` modifies the list.
5. Cache stampede fix? → One thread loads; others wait on the same future.

TL;DR

> - **HashMap + doubly linked list (sentinels)** = O(1) get/put.
> - **Strategy** eviction (LRU / FIFO / O(1) LFU); **Observer** removals; **Builder** config.
> - **One removal path**; lazy TTL + sweeper; injected clock.
> - One lock (gets are writes), listeners outside it, **shard** to scale; know `LinkedHashMap`, Caffeine, Redis policies.

20.4 Design a Ride Sharing Service (Uber)

Problem & clarifying questions

What it is. Connects a rider with a nearby driver — a radio taxi dispatcher who calls the nearest free cab, then the next if refused, and the meter shows the fare. Two decisions are the whole question: **which driver is asked** (matching) and **what the trip costs** (pricing). Either should change with no edits elsewhere.

Question	Assumed answer
Vehicle types?	Economy, premium, van; rider asks for one.
Who gets a request?	Online, free drivers of the right type within range.
Offer one at a time or broadcast?	One at a time, nearest first; decline → next; none left → cancel.
Trip states?	Requested → accepted → started → ended; cancel only from requested/accepted. Illegal actions do nothing.
Fare / surge?	Base + per km + per minute, with a minimum, per type; surge multiplies the same fare.
Ratings / payment?	Each side rates once after a finished trip (average shown); charge rider's method (cash, wallet, card).
Map?	Grid with Manhattan distance (production: a routing engine giving road distance + duration).
Pool, scheduled rides, tracking, payouts?	Out of scope.

Requirements

Functional: the table above, plus: driver has one vehicle (plate, seats, type) and goes online/offline; rider cancels only before start; trip history for both sides.

Non-functional: a driver can **never** be on two trips (even under concurrency); rules swappable; deterministic tie-breaks.

Use cases

Actors: **Rider** (request, cancel, rate), **Driver** (online, accept/decline, start/end, rate), **System** (find, offer, price, surge, history).

Core entities

Class	Responsibility
`Location`, `VehicleType`, `Vehicle`	Grid point (Manhattan `distanceTo`), car type and details.
`Account` (abstract) → `Rider`, `Driver`	Shared id/name/phone, **rating as total + count**, finished trips. Rider + payment method; Driver + vehicle, location, online, **atomic busy flag** (`tryClaim`).
`TripStatus` / `Trip`	State machine table; trip holds **ranked candidate list + cursor**; every action checks state.
`PricingStrategy`, `StandardPricing` + `RateCard`	**Strategy**: base + km + min with floor, per type.
`SurgePricing`	**Decorator** wrapping any `PricingStrategy`.
`MatchingStrategy`	**Strategy**: `NearestDriverMatcher`, `BestRatedMatcher`.
`PaymentMethod`	**Strategy**: `CashPayment`, `WalletPayment`.
`TripListener`	**Observer** of trip events.
`RideSharingService`	Facade; owns accounts and trips; knows only the order of steps.

Rating as total + count: only the average is shown, and adding a rating is O(1). `Rider` / `Driver` extend `Account` because they pass the LSP "is-a" test.

Class diagram

Course stages: (1) accounts, vehicle, location; (2) `Trip` + `TripStatus`; (3) pricing + surge wrapper; (4) matching; (5) the service. In the course figure, the arrow from `SurgePricing` back to `PricingStrategy` ("has-a of my own type") is what makes it a Decorator rather than another Strategy.

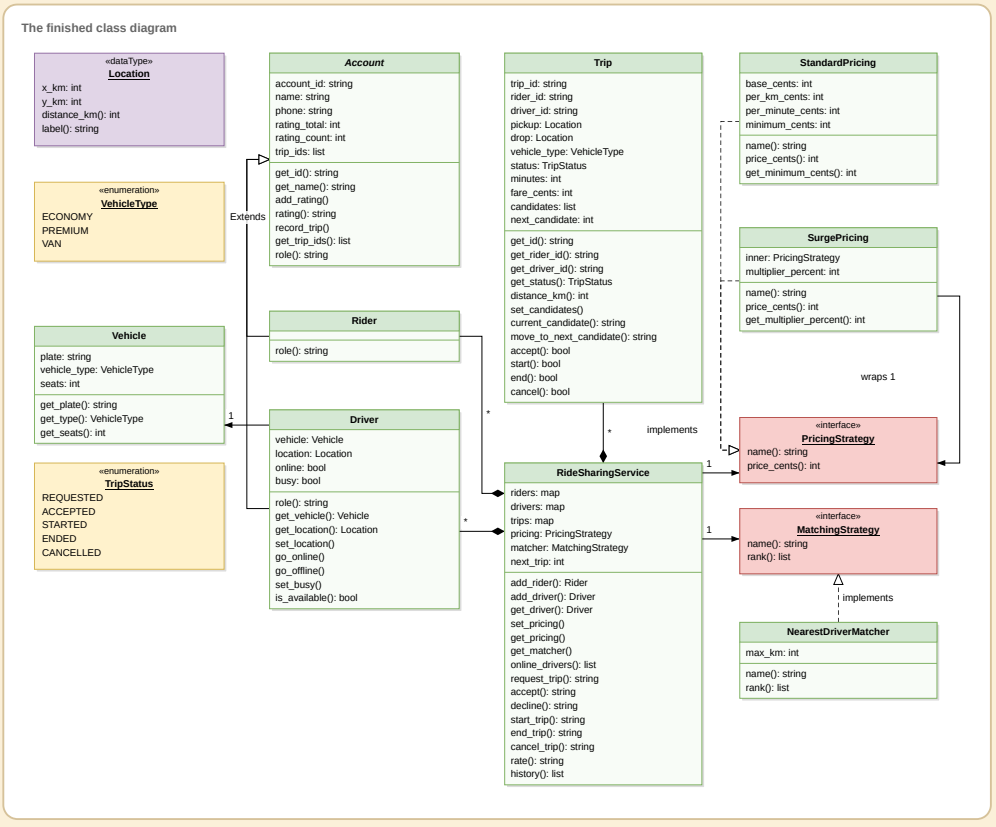
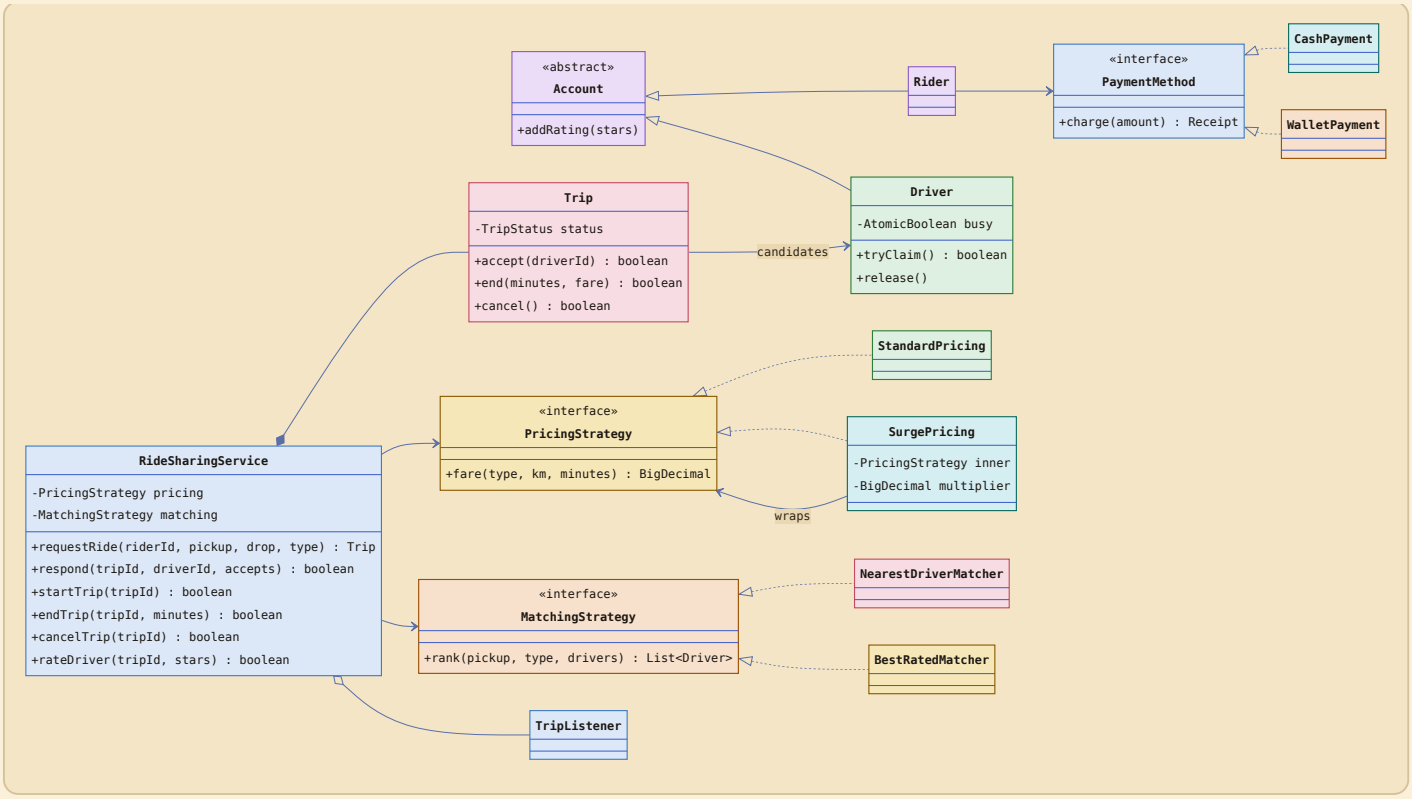
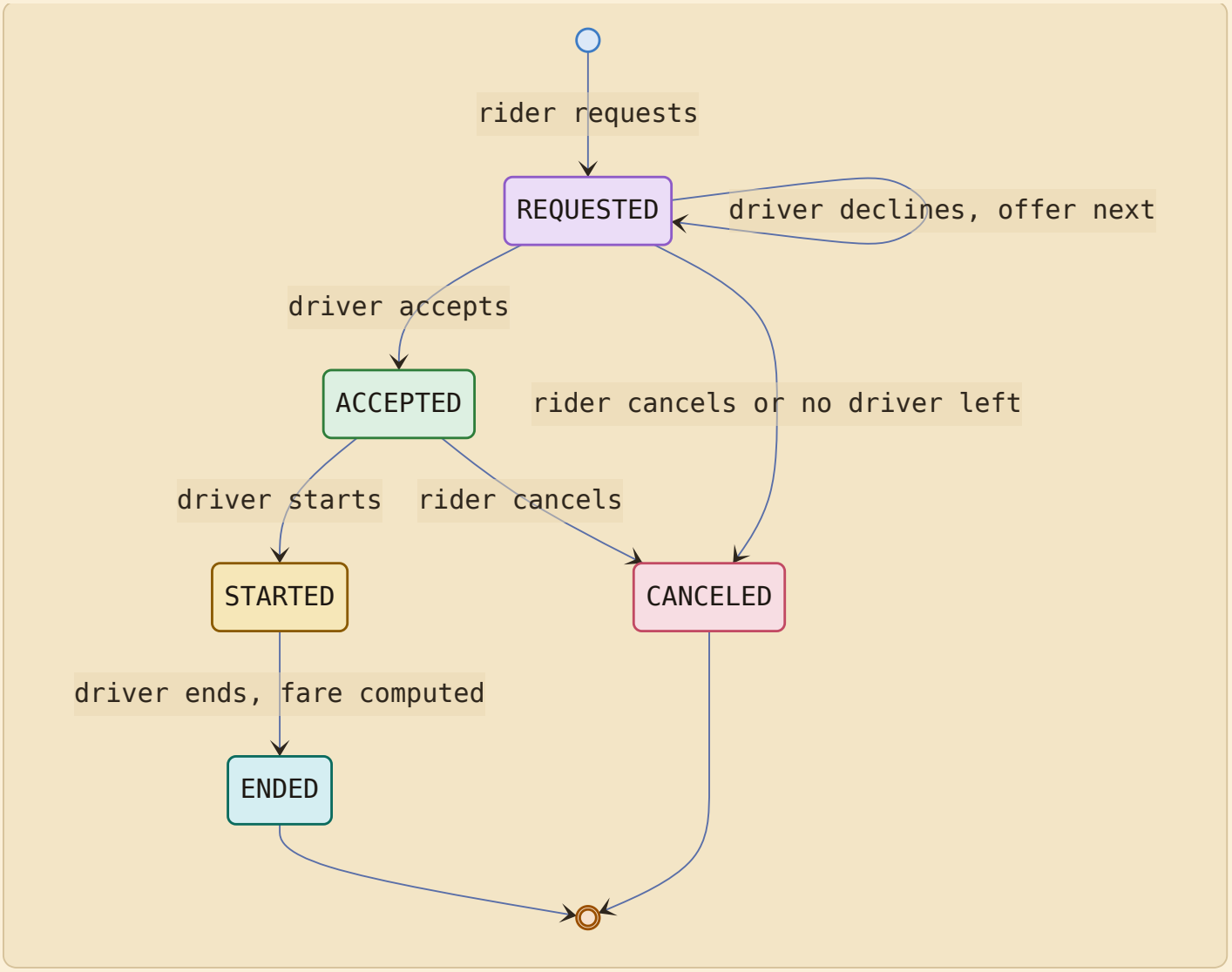


Figure — The finished ride sharing class diagram: the service owning riders, drivers and trips; PricingStrategy with StandardPricing and the SurgePricing wrapper; MatchingStrategy with the nearest-driver matcher (course diagram)



Key flows

Trip lifecycle (a state machine = fixed states + the only legal moves):



Finding a driver: rank free drivers of the right type nearest first; offer to the first; decline → move the cursor; nobody left → cancel. **Ending:** refuse if not started; else one `pricing.fare(...)` call (however many decorators wrap it), charge the rider, move the driver to the drop point, release.

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for matching	Cities rank differently (nearest, best-rated…).	Sorting inside the service.
Matcher returns a **ranked list** stored on the trip	Decline = move cursor; no re-search; remembers who refused.	Returning one driver.
Strategy pricing + **Decorator** surge	Surge wraps *any* rule; off = remove the wrapper; decorators stack (surge + airport fee + discount).	`if (surge)` in pricing.
State machine via `TripStatus.canMoveTo`	One table; illegal moves return `false`. Full State pattern only pays when per-state behaviour is large.	Scattered `if (status == …)`.
Atomic claim (`AtomicBoolean.compareAndSet`)	Two trips racing for a driver: exactly one wins; loser treats it as decline.	Check `isAvailable()` then set: race.
Move driver to **drop point** at end	Next match must use the real position.	Forgetting (real bug class).
Observer for trip events	Apps, push, analytics listen.	Service calling them directly.
Tie-break by driver id; `BigDecimal` `HALF_UP`	Reproducible tests; exact fares (course uses whole cents and surge as a percentage, 150 = ×1.5).	`double`.

Java implementation

```
// File: code/lld/ch20/ridesharing/Trip.java
enum TripStatus {
    REQUESTED, ACCEPTED, STARTED, ENDED, CANCELED;

    private static final Map<TripStatus, Set<TripStatus>> NEXT = Map.of(
            REQUESTED, EnumSet.of(ACCEPTED, CANCELED),
            ACCEPTED, EnumSet.of(STARTED, CANCELED),
            STARTED, EnumSet.of(ENDED),
            ENDED, EnumSet.noneOf(TripStatus.class),
            CANCELED, EnumSet.noneOf(TripStatus.class));

    boolean canMoveTo(TripStatus next) { return NEXT.get(this).contains(next); }
}

// File: code/lld/ch20/ridesharing/Strategies.java
/** Decorator: wraps ANY pricing rule and multiplies its answer. Remove the wrapper to switch surge off. */
final class SurgePricing implements PricingStrategy {
    private final PricingStrategy inner;
    private final BigDecimal multiplier;
    // ... constructor
    @Override
    public BigDecimal fare(VehicleType type, int km, int minutes) {
        return inner.fare(type, km, minutes).multiply(multiplier).setScale(2, RoundingMode.HALF_UP);
    }
}

// File: code/lld/ch20/ridesharing/Model.java  (Driver)
    boolean tryClaim() { return online && busy.compareAndSet(false, true); }

// File: code/lld/ch20/ridesharing/RideSharingService.java
    /** Driver answers the offer. A lost race for the driver counts as a decline. */
    boolean respond(String tripId, String driverId, boolean accepts) {
        Trip trip = trips.get(tripId);
        Driver current = trip.currentCandidate();
        if (current == null || !current.id().equals(driverId) || trip.status() != TripStatus.REQUESTED) return
false;
        if (accepts && current.tryClaim()) {
            if (trip.accept(driverId)) {
                publish(trip, "accepted by " + driverId);
                return true;
            }
            current.release(); // trip was canceled meanwhile
            return false;
        }
        publish(trip, (accepts ? "could not claim " : "declined by ") + driverId);
        offerOrCancel(trip, trip.nextCandidate());
        return false;
    }
    // ... endTrip: price -> trip.end -> charge rider -> d.moveTo(drop) -> d.release() -> history
```

Full runnable code: code/lld/ch20/ridesharing/ (see the Full Edition).

`Trip` methods are `synchronized`, so a rider's cancel and a driver's accept can't interleave. `endTrip` relocates the driver **before** releasing them, so no matcher ever sees a free driver at the old spot.

Output (excerpt):

```
  [T1] declined by D1
  [T1] accepted by D2
  [T1] ended: 7 km, 12 min, fare 13.90 (wallet debited 13.90, left 36.10)
  [T2] ended: 1 km, 3 min, fare 7.50 (collect 7.50 in cash)
  [T4] cancel refused in state STARTED
  [T5] no driver left - canceled
  trips accepted = 10 (never more than 10 drivers)
```

Trip 1: 2.50 + 7 × 1.20 + 12 × 0.25 = **13.90**. Trip 2: 4.45 → minimum 5.00 → surge × 1.5 = **7.50**. 40 concurrent requests vs 10 drivers → exactly 10 accepted.

Machine-coding variant: carpool / "offer and select a ride"

A Flipkart-style round (kumaransg/LLD collection) with **no dispatcher**: users *offer* spare seats on a route and others *select* a ride with a **selection strategy**.

- `add_user`, `add_vehicle` (several per user); `offer_ride(user, origin, seats, vehicle, destination)` **fails if that vehicle already has an active offer**.
- `select_ride(user, origin, destination, seats, strategy)` — seats 1 or 2; **Most Vacant** or **Preferred Vehicle** ("Activa").
- `end_ride`, `print_ride_stats` — taken/offered per user, counting only **ended** rides.

Design: `Ride` with an atomic `book`; `RideSelectionStrategy` (`MostVacant`, `PreferredVehicle`); `RideService` filters active rides by route and seats, excludes the user's own offers, and lets the strategy choose.

```java
// File: code/lld/ch20/carpool/Main.java
    interface RideSelectionStrategy {
        Optional<Ride> select(List<Ride> candidates);
    }

    static final class MostVacant implements RideSelectionStrategy {
        public Optional<Ride> select(List<Ride> c) { return
c.stream().max(Comparator.comparingInt(Ride::availableSeats)); }
    }
    // ... PreferredVehicle(model): first ride whose vehicle model matches
    // ... selectRide: filter ACTIVE rides on the same route, not offered by this user, with enough seats;
    // ... strategy.select(matching), then ride.book(user, seats) (synchronized check-and-book)
```

Full runnable code: code/lld/ch20/carpool/ (see the Full Edition). The demo replays the problem's sample (answers 2(c), 2(b), none, 2(d), none; then the stats).

	Uber-style (main design)	Carpool variant
Who supplies the ride	Professional drivers	Any user, per offered ride
Who chooses	System offers to drivers	Rider picks with a selection strategy
Key pattern	Strategy + Decorator + state machine	Strategy (selection)
Key invariant	A driver on ≤ 1 trip	A vehicle has ≤ 1 active offer; seats never negative

Concurrency & edge cases

- **Double booking:** prevented by `tryClaim()` CAS (the 40/10 race proves it).
- **Cancel vs accept:** both via `synchronized` `Trip` methods; if cancel wins, `accept` fails and the driver is released.
- **Stale candidate list:** computed once; a claimed/offline driver fails `tryClaim` → next. A nearer driver freed later isn't noticed (real systems re-match).
- **Offer timeouts:** no answer in N seconds = decline (scheduled task per offer).
- No drivers → canceled immediately. Illegal transitions → `false`, state unchanged. Wallet too low → `Receipt(paid=false)`. One rating per side, 1–5 stars.

Extensions & follow-up questions

- **Rank by rating:** swap in `BestRatedMatcher`.
- **Airport fee, promo, tolls:** more decorators: `new PromoDiscount(new AirportFee(new SurgePricing(standard, 1.5)))`.
- **How is surge computed?** Demand (open requests) vs supply (free drivers) per area and window → multiplier; the pricing design doesn't change.
- **Nearby drivers fast:** geospatial index — cells (geohash, S2, Uber's H3 hexagons) → map cell → drivers; search pickup cell + neighbours.
- **Pool rides:** honest answer: `Trip` becomes many riders/stops and the matcher considers detours. Live tracking: latest location per driver, published via pub-sub (20.6). Scheduled rides: a scheduler starts matching shortly before.

Real-world example

Uber shows surge ("dynamic pricing") when demand exceeds supply and gives a driver a short window to accept before the next — the ranked-list-with-cursor idea. Uber open-sourced **H3**, a hexagonal grid used for pricing and dispatch. Ola, Lyft and Grab follow the same request → match → accept → trip → rate lifecycle.

In the interview

> **30-second answer:** "Two rules behind interfaces: `MatchingStrategy` returns a ranked list of free drivers with the right car; `PricingStrategy` computes base + distance + time with a floor. Surge is a Decorator wrapping any pricing rule. The trip is a small state machine; every action checks the transition table and returns false if illegal. A decline moves a cursor along the stored ranking. Drivers are claimed with compare-and-set, so one driver never gets two trips."

Tip: name matching and pricing in the first minutes; spend real time on surge — *wrapping* the rule instead of adding a flag separates strong answers. *Strategy vs Decorator?* → Strategy replaces the rule; Decorator wraps it and stays in the call path (holds an `inner` of the same interface).

Self-check

1. Why a ranked list, not one driver? → Decline = next candidate, no re-search.
2. What makes `SurgePricing` a Decorator? → It is a `PricingStrategy` wrapping another and adjusting its result.
3. Can a rider cancel a STARTED trip? → No; `STARTED` only → `ENDED`.
4. Which Java class makes the claim atomic? → `AtomicBoolean.compareAndSet(false, true)`.
5. Carpool: when does `offer_ride` fail? → The vehicle already has an active offer.

TL;DR

> - **Strategy** ×3 (matching, pricing, payment) + **Decorator** (surge) + **state machine** (trip).
> - Ranked list + cursor; decline = next. **Atomic claim** prevents double booking; relocate driver at trip end.
> - Ratings = total + count; `BigDecimal` fares with a minimum.
> - Carpool: users offer seats; riders select with Most Vacant / Preferred Vehicle.

20.5 Design a Logging Framework

Problem & clarifying questions

`What it is.` Code writes messages ("payment 991 authorised") with a **level** (severity), and the framework sends them to **destinations** (console, file, log server) in a chosen **format** — a mini Log4j 2 / Logback /

`java.util.logging`. Analogy: a company mail room that routes notes by department and urgency stamp to inboxes, in the right envelope.

Question	Assumed answer
Levels?	TRACE < DEBUG < INFO < WARN < ERROR < FATAL (and OFF).
Per-module config?	Loggers named `app.payment`, `app.orders` form a **dot hierarchy**; a child inherits the nearest ancestor's level.
Destinations / formats?	Console, file, in-memory; pattern and JSON; both pluggable; each destination has its own minimum level.
Propagation?	Event goes to the logger's destinations then **up** to ancestors', unless additivity is off.
Async / thread-safe?	Optional async (app never waits for disk); yes.
Config files, rolling files, shipping?	Out of scope.

Requirements

Functional: the table above, plus: same name → same logger; exceptions attachable; lazy messages; async without changing appenders.

Non-functional: very cheap when disabled; thread-safe; logging never crashes the app; testable (injected clock).

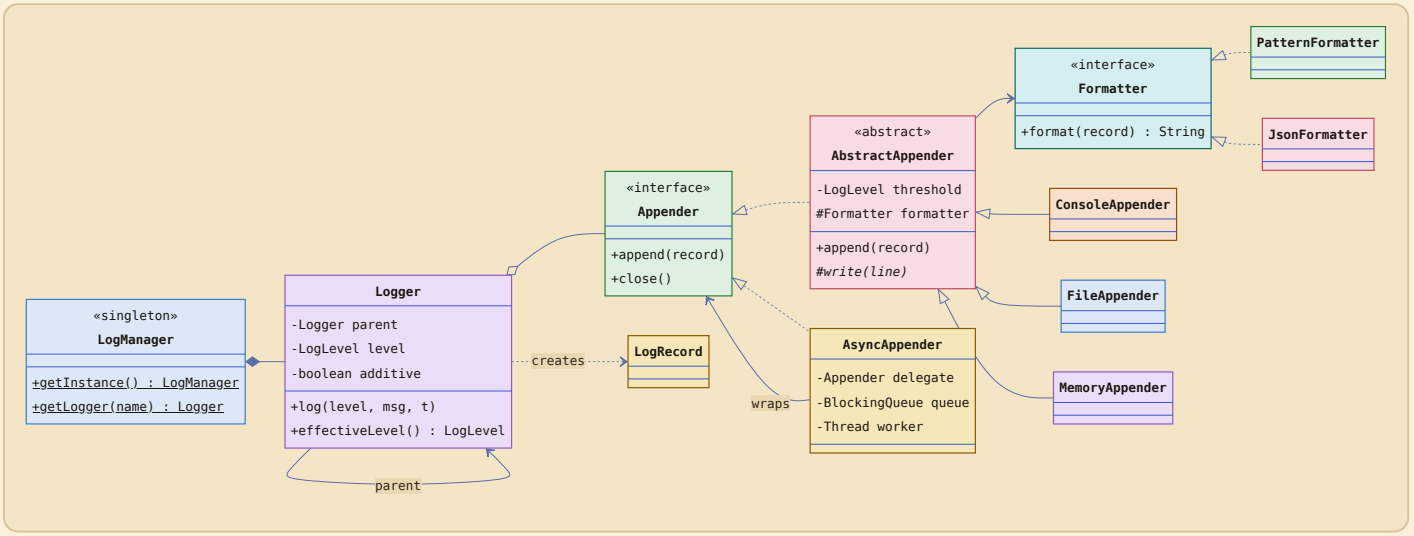
Use cases

Actors: **Developer code** (logs), **Operator** (configures levels, appenders, formats), **Framework** (filters, formats, routes, writes). Use cases: log a message, get a logger by name, set a level for a subtree, add a destination, choose a format, switch to async, stop propagation for a subtree.

Core entities

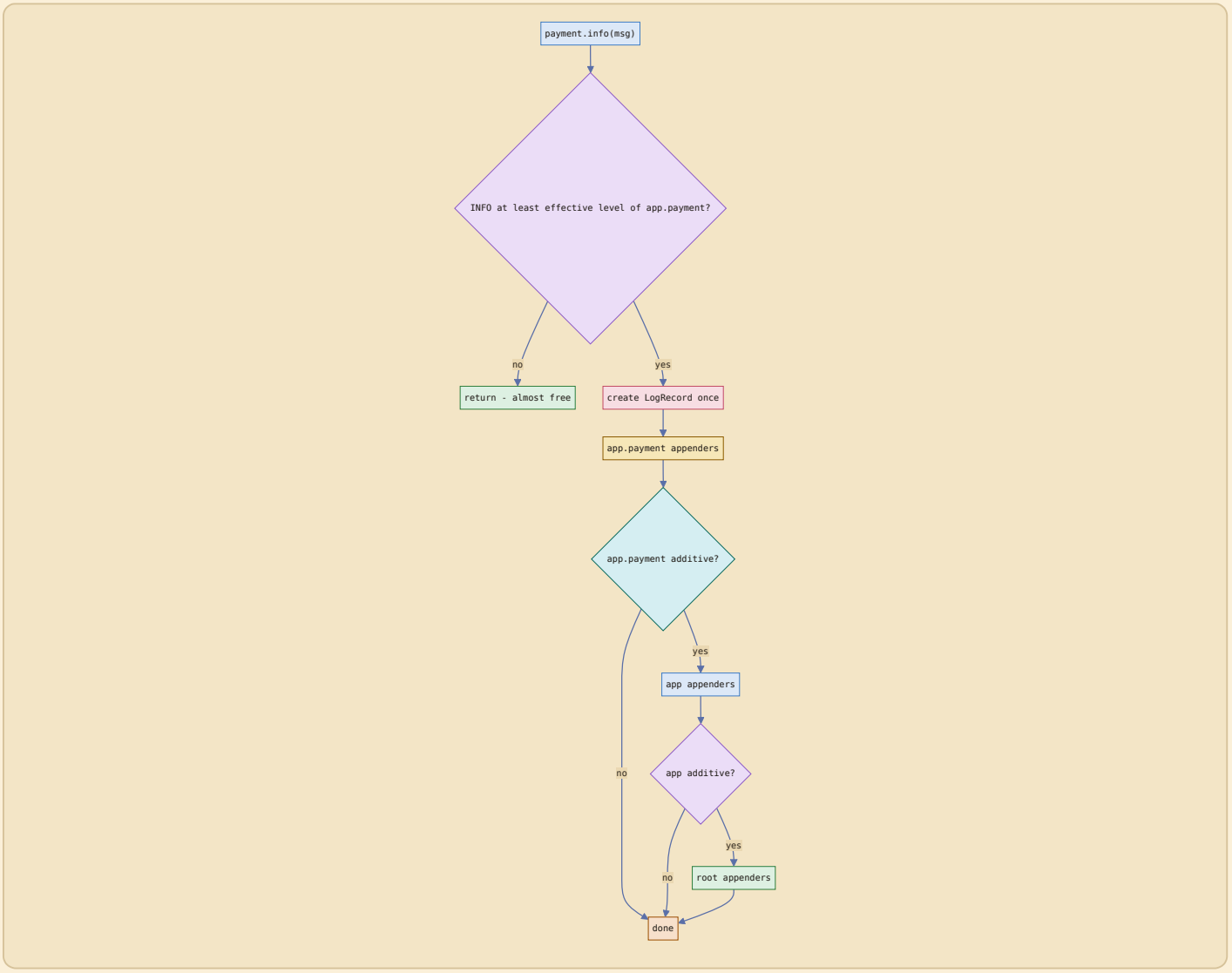
Class	Responsibility
`LogLevel`	Ordered severities; `isAtLeast(threshold)`.
`LogRecord`	One immutable event: time, level, logger, thread, message, error.
`Logger`	Named node: parent, optional level, appenders, additivity; `log()` walks up.
`LogManager`	**Singleton** registry; `getLogger(name)` creates parents first, one instance per name; holds the clock.
`Appender` / `AbstractAppender`	A sink; **Template Method** base checks threshold + formats, subclasses only `write(line)`.
`ConsoleAppender`, `FileAppender`, `MemoryAppender`	Concrete sinks.
`AsyncAppender`	**Decorator**: bounded queue + one writer thread around any appender.
`Formatter`	**Strategy**: `PatternFormatter`, `JsonFormatter`.

Class diagram



Key flows

Logging one event = Chain of Responsibility up the hierarchy (a request passes along a chain of handlers; each may handle it and/or pass it on):



The Log4j/Logback rule we copy: the **level check happens once, at the logger where the call was made**; parents' levels are not re-checked, only each **appender's own threshold** filters. So a DEBUG event from `app.payment` (set to DEBUG) still reaches root's console even though root is INFO.

Async: the app thread calls `append` → `queue.offer` (never blocks) → returns; a writer thread `take()`s and calls the slow delegate. `close()` enqueues a poison record, then `join()`s until the queue is drained.

Design decisions & patterns used

Pattern / decision	Where	Why
Chain of Responsibility	`Logger.log`: this → parent → … → root, stops at `additive == false`	Configure sinks once near the root; subtrees add or stop.
Singleton (holder idiom)	`LogManager`	One registry → same logger per name; lazy + thread-safe with no locking (JVM initializes the nested class once). Double-checked locking is easy to get wrong.
Static factory	`getLogger(name)`	Caches instances, creates parents first; `new Logger` would break the hierarchy.
Strategy	`Formatter`	Any sink × any format; no `JsonFileAppender` class explosion.
Template Method	final `append` → `write`	Threshold + formatting once.
Decorator	`AsyncAppender`	Async is an add-on, not `AsyncFileAppender`, `AsyncConsoleAppender`…
Level check first; lazy `Supplier<String>`	`isEnabled`, `debug(Supplier)`	Disabled logging = one comparison.
Appender failures caught	`Logger.log`	Logging never crashes the app.
Parent resolved **outside** `computeIfAbsent`	`LogManager.loggerFor`	A `ConcurrentHashMap` mapping function must not modify the same map (`IllegalStateException: Recursive update`).

Another common "CoR logger" answer chains *level handlers* (`ErrorHandler → WarnHandler → InfoHandler`). It works as a toy, but real frameworks put the chain on the **logger hierarchy**, because that is what operators configure. If the interviewer expects the level-handler version, explain both.

Java implementation

```
// File: code/lld/ch20/logging/Logger.java
    // ... effectiveLevel() walks up to the nearest logger with a level; isEnabled(); debug(Supplier) for lazy
messages
    void log(LogLevel lvl, String msg, Throwable t) {
        if (!isEnabled(lvl)) return;                    // cheap check first
        LogRecord r = new LogRecord(manager.now(), lvl, name, Thread.currentThread().getName(), msg, t);
        for (Logger l = this; l != null; l = l.parent) { // the chain
            for (Appender a : l.appenders) {
                try {
                    a.append(r);
                } catch (RuntimeException e) {
                    System.err.println("appender failed: " + e); // logging must never crash the app
                }
            }
            if (!l.additive) break;
        }
    }

/** Singleton registry of loggers (initialization-on-demand holder idiom: lazy and thread-safe). */
final class LogManager {
    // ...
    private static final class Holder { static final LogManager INSTANCE = new LogManager(); }

    static LogManager getInstance() { return Holder.INSTANCE; }

    private Logger loggerFor(String name) {
        if (name == null || name.isEmpty() || name.equals("root")) return root;
        Logger existing = loggers.get(name);
        if (existing != null) return existing;
        int dot = name.lastIndexOf('.');
        Logger parent = dot < 0 ? root : loggerFor(name.substring(0, dot)); // resolved OUTSIDE
computeIfAbsent
        return loggers.computeIfAbsent(name, n -> new Logger(n, parent, this));
    }
    // ...
}

// File: code/lld/ch20/logging/Appenders.java
    @Override
    public final void append(LogRecord r) {
        if (r.level().isAtLeast(threshold)) write(formatter.format(r));
    }
    // ...
    @Override
    public void append(LogRecord r) {
        if (closed || !queue.offer(r)) dropped.incrementAndGet(); // full or closed: count it, never block
    }
```

Full runnable code: code/lld/ch20/logging/ (see the Full Edition).

`LogRecord` is created once and shared (immutable, thread-safe). Sinks synchronize `write` so lines never interleave. `AsyncAppender`'s promise is precise: `written + dropped` always equals the number of `append` calls; a throwing delegate is caught inside the writer loop so the writer never dies (which would leave `close()` waiting forever).

Output (excerpt):

```
2026-09-28 10:15:30.123 DEBUG [main] app.payment - card token=tok_42 (built lazily)
== 4. Additivity off: 'app.payment' events stop at its own appenders
  (nothing printed on console above) audit sink now has 2 lines
== 6. Async appender: 4 threads x 1000 events, then close() flushes
  written=4000 dropped=0 total=4000
```

Concurrency & edge cases

- Appenders in `CopyOnWriteArrayList`; level and additivity `volatile`; each sink's `write` synchronized.
- **Async queue full:** drop-and-count (never block). Alternatives: block (back-pressure, may stall the app) or drop low levels only — Logback's `AsyncAppender` by default discards TRACE/DEBUG/INFO at 80% full.
- **Shutdown:** without `close()` queued events are lost → JVM shutdown hook.
- **Recursive logging** from an appender can loop forever → thread-local guard.
- **Security:** never interpret messages — Log4Shell (CVE-2021-44228, Dec 2021) came from Log4j 2 evaluating `${jndi:...}` in logged strings. Mask card numbers, tokens, passwords.

Extensions & follow-up questions

- **Rolling files:** `RollingFileAppender` + `RollingPolicy` (size/time) Strategy.
- **Config file:** parse into `LogManager` calls; watch it for runtime changes.
- **Filters:** a `Filter` chain per appender — another Chain of Responsibility.
- **MDC / request id:** thread-local map copied into each record. Ship logs: network appender behind `AsyncAppender`, batching. Structured logging: key-value fields rendered as JSON.

Real-world example

SLF4J is the facade; **Logback** and **Log4j 2** implement exactly this model (dot-named hierarchy, inherited levels, appenders with layouts, *additivity*). Log4j 2 async loggers use the LMAX Disruptor ring buffer. `java.util.logging`: `Logger`, `Handler` (appender), `Formatter`, `Level`, `setUseParentHandlers(false)` (additivity off).

In the interview

> **30-second answer:** "Loggers form a dot hierarchy under a root managed by a singleton `LogManager`. A log call checks the inherited effective level, builds one immutable record, and passes it up the chain of loggers to their appenders — Chain of Responsibility — until additivity stops it. Each appender has its own threshold and a pluggable formatter (Strategy). `AsyncAppender` is a Decorator with a bounded queue and a writer thread."

Follow-ups: *Why is a disabled debug call cheap?* → One comparison before any string; `Supplier` for expensive ones. *Queued logs on crash?* → Lost unless flushed; close on shutdown.

Self-check

1. `app.payment` unset, `app` WARN, root INFO → effective level? → WARN.
2. Additivity=false? → Stops the event reaching ancestors' appenders.
3. Pattern making any appender async? → Decorator.
4. Why is `Formatter` separate from `Appender`? → Formats and sinks combine freely (Strategy).
5. How does `close()` lose nothing queued? → Poison pill + `join()`; refused records are counted as dropped.

TL;DR

- Named **logger hierarchy**, inherited level, one cheap check up front.
- **Chain of Responsibility** to the root (additivity stops it); **Singleton** `LogManager`.
- **Strategy** formatters, **Template Method** appender base, **Decorator** async.
- Immutable record, thread-safe sinks, never crash, never block on a full queue.

20.6 Design a Pub-Sub System

Problem & clarifying questions

What it is. Parts of a system talk **without knowing each other**: a *publisher* sends to a named *topic*; every *subscriber* of it receives the message. Analogy: a newspaper printed once, delivered to every subscriber, new readers welcome any time. It is the **Observer pattern as infrastructure**: a **broker** in the middle decouples by topic, time and often machine. We design a simplified, Kafka-like, in-process broker.

Question	Assumed answer
Delivery to how many?	Every subscription gets every message (fan-out), in publish order.
Guarantee?	**At-least-once**: ack only after processing; failures retried; duplicates possible.
Slow subscriber?	Must not slow others or the publisher.
Keep messages?	Yes: append-only log with **offsets** → late subscribers and replay.
Poison message?	After N attempts → dead-letter topic.
Push or pull?	Push, one dispatcher per subscription.
Partitions, consumer groups, disk, retention?	Out of scope.

Requirements

Functional: create topics; publish (key, payload) → an **offset** (position in the log); subscribe from EARLIEST or LATEST; in-order delivery; commit only after the callback returns; retry, then dead-letter; **seek** to replay; unsubscribe; clean shutdown.

Non-functional: thread-safe publish; subscribers isolated; no busy-waiting; order preserved.

Use cases

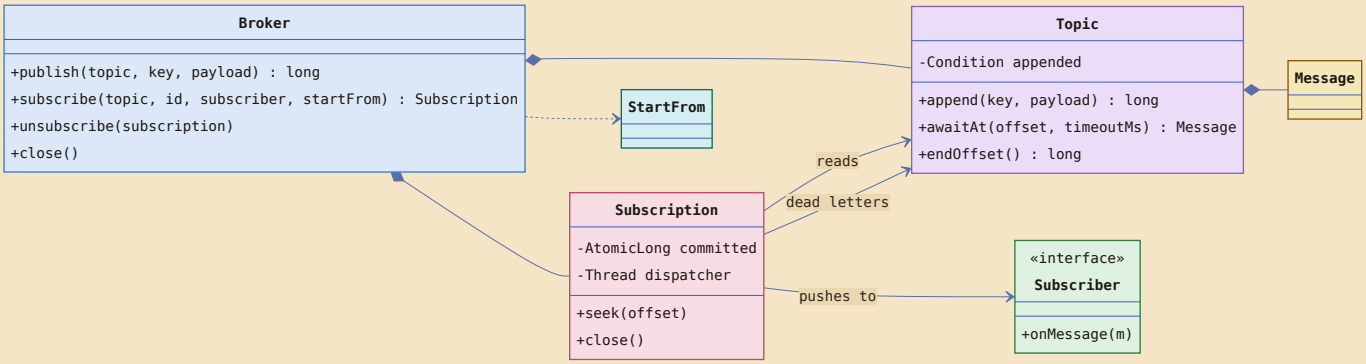
Actors: **Publisher**, **Subscriber** (sometimes fails), **Operator** (creates topics, replays, inspects dead letters), **Broker** (stores, dispatches, retries).

Core entities

Class	Responsibility
`Message`	topic, offset, key, payload; immutable.
`Topic`	Append-only `List<Message>` guarded by a `ReentrantLock` + `Condition` "appended".
`Subscriber`	Callback `onMessage`; throwing = negative ack.
`Subscription`	Committed offset (`AtomicLong`), **own dispatcher thread**, retry + dead-letter, `seek`.
`StartFrom`	EARLIEST / LATEST.
`Broker`	Registry, publish, subscribe, unsubscribe, DLQ, `awaitIdle` (tests), close.

Key idea: messages are not removed when consumed. Each subscription only remembers *how far it has read*. That gives late joiners, replay (`seek`) and independent progress for free — Kafka's model. (RabbitMQ-style queues delete on ack.)

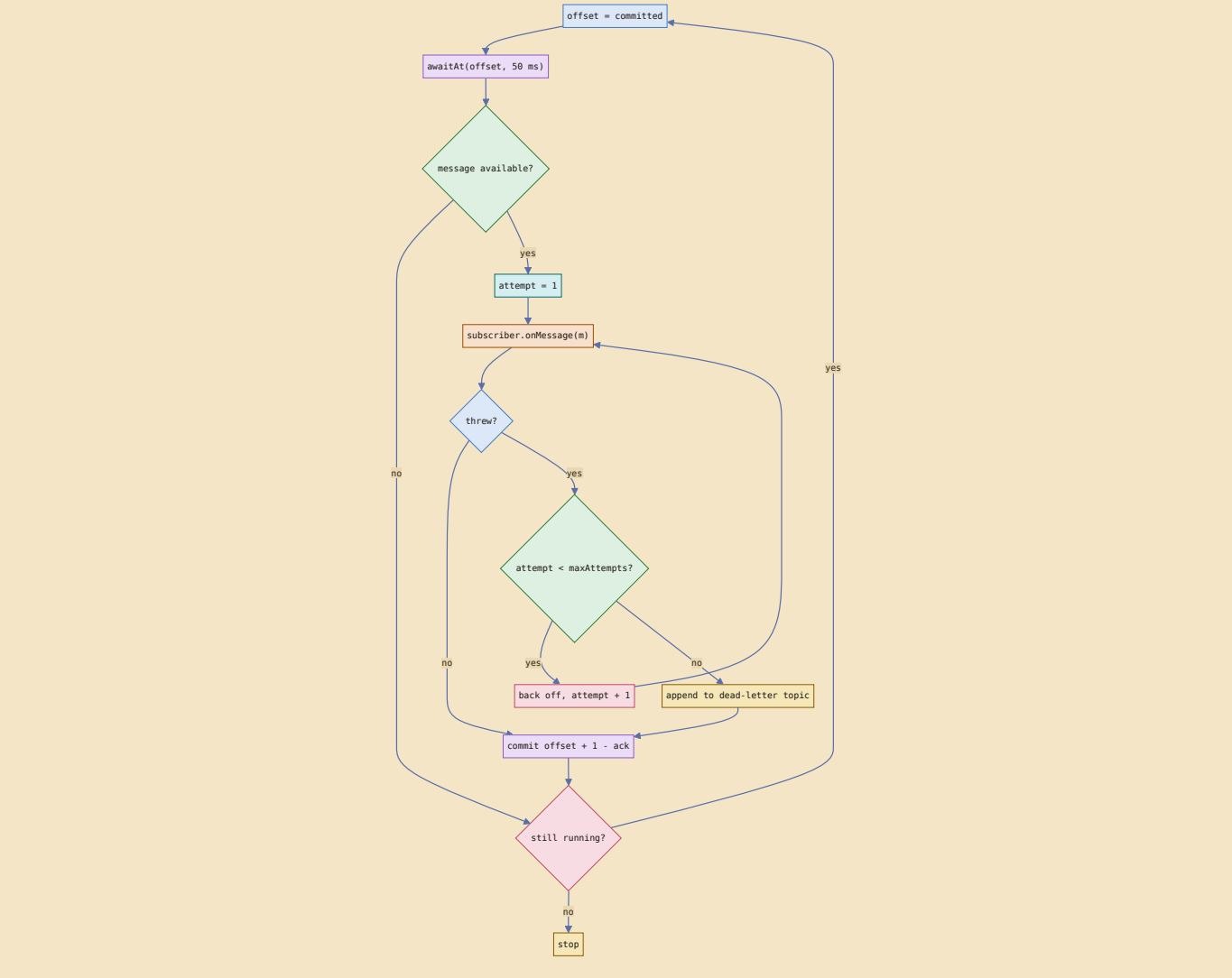
Class diagram



Key flows

Publish and fan-out: dispatchers sleep on the topic's `Condition`; `publish` appends (offset N) and `signalAll()`s; each dispatcher independently delivers message N and commits N+1 — a slow one delays only itself.

One subscription's dispatcher loop (at-least-once with dead letters):



Design decisions & patterns used

Decision	Why	Alternative rejected
Publish-Subscribe (Observer via broker)	Add subscribers with zero publisher changes.	Publisher calling consumers directly.
Append-only log + offsets	Replay, late joiners, independent progress; order = log order.	Delete on delivery; per-subscriber copies.
One dispatcher per subscription (virtual threads)	Slow subscriber delays only itself; one deliverer keeps order; virtual threads make it cheap.	One shared thread (slowest blocks all); pool per message (loses order).
`Lock` + `Condition` with timeout, in a `while`	No busy-waiting; timeout notices `close()` / `seek()`; `while` handles spurious wake-ups.	`Thread.sleep` polling.
Ack after processing	At-least-once: a crash re-delivers.	Commit first = at-most-once.
Dead-letter topic	A poison message can't block forever.	Infinite retries.
`compareAndSet` on commit	A concurrent `seek()` wins.	Plain `set`.

Delivery guarantees:

Guarantee	How	Risk
At-most-once	Commit, then process	Crash loses the message
At-least-once (ours)	Process, then commit	Duplicates → subscribers must be **idempotent**
Exactly-once	At-least-once + dedup / transactions committing output and offset together	Complex; Kafka offers it within its own transactions

Java implementation

```
// File: code/lld/ch20/pubsub/Topic.java
    // ... append(): under the lock, add to the log and appended.signalAll()
    /** Returns the message at 'offset', waiting up to the timeout for it to be published. */
    Message awaitAt(long offset, long timeoutMs) throws InterruptedException {
        lock.lock();
        try {
            long nanos = TimeUnit.MILLISECONDS.toNanos(timeoutMs);
            while (offset >= log.size()) {         // loop: guards against spurious wake-ups
                if (nanos <= 0) return null;
                nanos = appended.awaitNanos(nanos);
            }
            return log.get((int) offset);
        } finally {
            lock.unlock();
        }
    }

// File: code/lld/ch20/pubsub/Broker.java  (Subscription)
    private void loop() {
        try {
            while (running) {
                long offset = committed.get();
                Message m = topic.awaitAt(offset, 50);
                if (m == null) continue;              // timed out: re-check 'running' and any seek()
                deliver(m);
                committed.compareAndSet(offset, offset + 1); // ack; a concurrent seek() wins
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    private void deliver(Message m) throws InterruptedException {
        for (int attempt = 1; attempt <= maxAttempts; attempt++) {
            try {
                subscriber.onMessage(m);
                return;
            } catch (Exception e) {
                if (attempt == maxAttempts) {
                    deadLetters.append(id, m.topic() + "@" + m.offset() + " " + m.payload() + " -> " +
e.getMessage());
                    return;                           // skip it so one poison message cannot block the rest
                }
                Thread.sleep(5L * attempt);           // simple backoff
            }
        }
    }
    // ...
    void seek(long offset) { committed.set(offset); }
```

Full runnable code: code/lld/ch20/pubsub/ (see the Full Edition).

Output (excerpt):

```
  inventory [0:order#1, 1:order#2, 3:order#3]  (offset 1 retried, offset 2 dead-lettered)
  dead letter: orders@2 BAD-JSON -> cannot parse
  live after replay [4:order#4, 3:order#3, 4:order#4] (order#4 seen twice: at-least-once)
```

Concurrency & edge cases

- Publishers: `append` under a lock → unique, gap-free offsets; messages immutable.

- **Unbounded log:** real brokers apply **retention** (by age/size); a lagging subscriber may then fall off the end → detect "offset out of range" and reset.
- **Subscriber blocks forever:** stalls only its own subscription; add a processing timeout. Java 21 caveat: a virtual thread blocking inside `synchronized` *pins* its carrier thread — prefer `ReentrantLock` in subscriber code, or platform threads.
- **Duplicates:** subscribers must be idempotent (remember processed ids).
- **Shutdown:** `close()` flips `running` and joins; the 50 ms timeout bounds it.

Extensions & follow-up questions

- **Consumer groups:** split a topic into **partitions**; same key → same partition (`hash(key) % n`) keeps per-key order; each group member owns partitions; offset per (group, partition). That is how Kafka scales.
- **Persist:** segment files + offset index; committed offsets in a special topic (`__consumer_offsets`).
- **Pull:** `poll(id, max)` + `commit(offset)` → back-pressure for free.
- Filtering by predicate, wildcard topics (`orders.*`), delayed retry topics (`orders.retry.5s`) instead of sleeping.

Real-world example

Apache Kafka is this model at scale (partitioned append-only logs, consumer offsets, consumer groups, per-partition order, retention by time/size). **Google Cloud Pub/Sub** and **AWS SNS + SQS** offer managed fan-out with acks and **dead-letter** queues; Spring Kafka's `DeadLetterPublishingRecoverer` is our `__dead_letters`. In-JVM: Guava `EventBus`, Spring `ApplicationEventPublisher`.

In the interview

> **30-second answer:** "Each topic is an append-only log; publishing appends and returns an offset. Each subscription keeps a committed offset and its own dispatcher thread that waits on a condition, calls the subscriber, and commits only after success — at-least-once, in order. Failures retry with backoff, then go to a dead-letter topic. Messages aren't deleted on read, so late subscribers start at the beginning and anyone can replay. To scale: partitions by key, consumer groups."

Follow-ups: *Slow consumer?* → One dispatcher per subscription; the log is shared, progress isn't. *Exactly-once?* → Hard; at-least-once + idempotent consumers. *Order?* → One writer assigns offsets, one dispatcher per subscription; with partitions, order is per key.

Self-check

1. What is an offset? → A message's position in the log; subscribers remember the next one.
2. Commit before or after processing for at-least-once? → After.
3. Why a dead-letter topic? → One poison message can't block a subscription forever.
4. Why does replay work here but not in a delete-on-ack queue? → Messages stay; seek moves the offset.
5. Why `while` around `Condition.await`? → Spurious wake-ups.

TL;DR

- **Pub-sub = Observer through a broker.**
- **Append-only log + per-subscription offsets** → fan-out, late joiners, replay.
- **One dispatcher per subscription** (isolation + order); **ack after processing** = at-least-once.
- Retry, then **dead-letter**; subscribers **idempotent**; scale with partitions and consumer groups.

20.7 Design a Rate Limiter

Problem & clarifying questions

`What it is.` Decides whether a request is allowed now, so one client can't overwhelm a service. Analogy: a turnstile allowing 10 people per minute per ticket type; over the limit: "come back in 6 seconds" — HTTP **429 Too Many Requests** with `Retry-After`. **Chapter 13 (the rate-limiter / token-bucket problem) already built the concurrency core** (a `synchronized` and a lock-free CAS token bucket, algorithm comparison). Here we design the **objects around it**: pluggable algorithms, rules per API and tier, a factory, combined limits, a decision object.

Question	Assumed answer
Limit by?	(user, API); IP or API key is the same design, different key.
Different limits?	Per API and tier (FREE / PREMIUM), plus a default.
Algorithms?	Token bucket, fixed window, sliding log, sliding counter — per rule.
Several at once?	Yes: "2 per 10 s **and** 3 per minute" on `/login`.
Response?	allowed + remaining + retry-after.
Distributed?	Single process here; distributed in extensions.

Requirements

Functional: `check(userId, api)` → `Decision(allowed, remaining, retryAfter)`; rules per API + tier with a default; **one limiter per (user, api)**, created on first use; pluggable algorithms; combined rules must all allow.

Non-functional: ~O(1) per check; exactly N allowed under any concurrency; injectable clock; bounded memory.

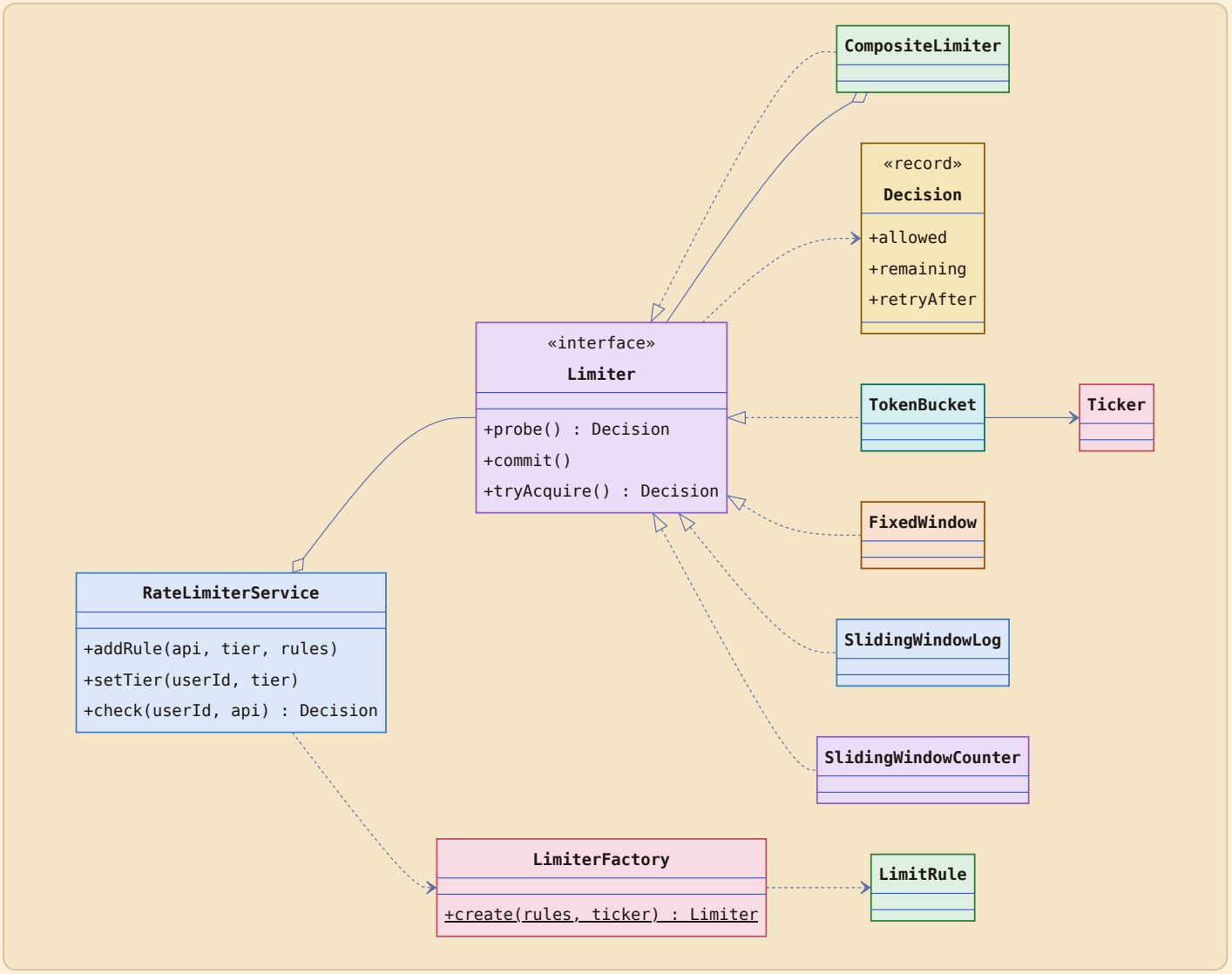
Use cases

Actors: **Client** (sends requests), **API gateway / filter** (calls `check` before the handler), **Operator** (configures rules and tiers).

Core entities

Class	Responsibility
`Limiter`	**Strategy**: `probe()` (would one more pass? consumes nothing), `commit()`, `tryAcquire()` = both atomically. One per key.
`TokenBucket`, `FixedWindow`, `SlidingWindowLog`, `SlidingWindowCounter`	The algorithms (`synchronized`).
`CompositeLimiter`	**Composite**: all children must allow; probe all, then commit all, under one lock.
`Decision`, `LimitRule`, `Algorithm`, `Tier`	Result; config (algorithm, limit, window); enums.
`LimiterFactory`	**Simple Factory**: rule(s) → limiter (single or composite).
`RateLimiterService`	**Facade**: resolves tier + rules, one limiter per (user, api) in a `ConcurrentHashMap`.
`Ticker`	Injected nanosecond clock.

Class diagram



Key flows

Gateway → `check("ann", "/search")` → `limiters.computeIfAbsent(key)` (first time: factory builds the limiter from the FREE-tier rules) → `tryAcquire()` → allowed: call the handler, 200; denied: **429 + Retry-After**.

The four algorithms (limit N per window W):

Algorithm	State per key	Bursts	Accuracy	Notes
Token bucket	tokens, last refill	Up to capacity	Good	Most common for APIs
Fixed window counter	window start, count	**Up to 2N across a boundary**	Poor at edges	Simplest
Sliding window log	one timestamp per request	None beyond N	Exact	O(N) memory per key
Sliding window counter	2 counters	Approximate	Good	previous × overlap + current
(Leaky bucket)	a queue	Constant output rate	—	Traffic shaping; not implemented

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy (`Limiter`)	Algorithms differ in accuracy/memory; new one = new class.	`switch` in `check`.
Simple Factory	One place turns config into objects.	`new TokenBucket` in the service.
Composite, **probe all then commit all**	"10/s and 1000/day" looks like one limiter; a request denied by one child uses up nothing in another.	Calling each child's `tryAcquire` in turn (a later denial leaves earlier children charged).
Facade service	Gateway asks one question.	Gateway juggling tiers/rules.
`computeIfAbsent` per key	Exactly one limiter even if the first two requests race.	`get` then `put`.
`Decision` with remaining/retryAfter	Maps to `X-RateLimit-Remaining`, `Retry-After`.	`boolean`.
Injected `Ticker`; `synchronized` probe+commit	Tests never sleep; refill + check + take is one atomic step.	`nanoTime()` inside; unsynchronized fields.

Java implementation

```java
// File: code/lld/ch20/ratelimiter/Limiters.java
interface Limiter {
    Decision probe();

    void commit();

    /** Check and take in ONE atomic step (probe + commit under this limiter's lock). */
    default Decision tryAcquire() {
        synchronized (this) {
            Decision d = probe();
            if (d.allowed()) commit();
            return d;
        }
    }
}

/** Token bucket: bursts up to 'capacity', refills at 'perSecond'. Lazy refill, no background thread. */
final class TokenBucket implements Limiter {
    // ... capacity, refillPerNano, ticker, tokens, last
    @Override
    public synchronized Decision probe() {
        long now = ticker.nanos();
        tokens = Math.min(capacity, tokens + (now - last) * refillPerNano); // refilling is safe to do early
        last = now;
        if (tokens >= 1) return Decision.allow((long) (tokens - 1));
        long waitNanos = refillPerNano == 0 ? Long.MAX_VALUE : (long) Math.ceil((1 - tokens) / refillPerNano);
        return Decision.deny(Duration.ofNanos(waitNanos));
    }

    @Override
    public synchronized void commit() { tokens -= 1; }
}
// ... FixedWindow, SlidingWindowLog, SlidingWindowCounter

final class CompositeLimiter implements Limiter {
    private final List<Limiter> children;
    // ...
    @Override
    public synchronized Decision probe() {
        long remaining = Long.MAX_VALUE;
        for (Limiter l : children) {
            Decision d;
            synchronized (l) { d = l.probe(); }
            if (!d.allowed()) return d;           // first denial wins; nothing has been consumed
            remaining = Math.min(remaining, d.remaining());
        }
        return Decision.allow(remaining);
    }

    @Override
    public synchronized void commit() {
        for (Limiter l : children) synchronized (l) { l.commit(); }
    }
}

// File: code/lld/ch20/ratelimiter/RateLimiterService.java
    Decision check(String userId, String api) {
        // computeIfAbsent creates exactly ONE limiter per (user, api), even under a race
        Limiter limiter = limiters.computeIfAbsent(new Key(userId, api), k -> {
            Tier tier = tiers.getOrDefault(userId, Tier.FREE);
            List<LimitRule> r = rules.getOrDefault(api, Map.of()).getOrDefault(tier, defaultRules);
            return LimiterFactory.create(r, ticker);
        });
```

```
        return limiter.tryAcquire();
    }
```

Full runnable code: code/lld/ch20/ratelimiter/ (see the Full Edition).

Composite children are created for it alone by the factory and never shared, so the composite's lock makes probe-all + commit-all atomic.

Output (excerpt):

```
  t=0.0 ann /search #4 -> 429 Too Many Requests (retry after 1000 ms)
== 4. Boundary burst: limit 5 per 10 s, 5 calls at t=9.9 and 5 at t=10.1
  FIXED_WINDOW     allowed 10 of 10 within 0.2 s
  SLIDING_LOG      allowed 5 of 10 within 0.2 s
  SLIDING_COUNTER  allowed 5 of 10 within 0.2 s
  allowed = 50 (exactly 50, never more)
```

The classic point: **fixed window lets 10 through in 0.2 s** across a boundary; both sliding algorithms hold at 5. 640 concurrent attempts on a 50-token bucket → exactly 50.

Concurrency & edge cases

- Contention is per key (each limiter `synchronized`; map is `ConcurrentHashMap`), so different users never block each other.
- **Naive composite bug:** child 1 allows (and consumes), child 2 denies → child 1 is charged for a request that never ran. The demo's "3 per minute first" case shows the +11s call would wrongly be refused for ~49 s. Probe-then-commit fixes it (acquire-and-refund is the harder alternative).
- **Memory:** one limiter per key forever → evict idle limiters (expire-after-access cache, as in 20.3) with idle timeout ≥ the longest window — a fresh limiter then behaves identically.
- Monotonic clock; rule changes at runtime need clearing or versioning the limiter map.

Extensions & follow-up questions

- **Distributed:** shared state in Redis with an atomic Lua script (or `INCR` + `EXPIRE` for fixed windows); costs a network hop — mitigate with local token pre-allocation or sticky routing.
- **Leaky bucket:** bounded queue drained at a constant rate (protects fragile downstreams).
- Cost per request (`tryAcquire(permits)`), soft limits (warn at 80%), key by IP when logged out; runs in an API gateway or servlet filter / Spring `HandlerInterceptor`.

Real-world example

Stripe described combining a token-bucket request limiter, a concurrent-requests limiter and load shedders on Redis — the composite idea. **GitHub** returns `X-RateLimit-Limit/Remaining/Reset` (our `Decision`). HTTP 429 is RFC 6585. **NGINX** `limit_req` uses leaky bucket. Java: Guava `RateLimiter`, Resilience4j, **Bucket4j**. Cloudflare uses the sliding-window-counter approximation at the edge.

In the interview

30-second answer: "A `Limiter` interface with interchangeable algorithms — token bucket by default, sliding window when exactness matters. A factory builds limiters from rules chosen by API and tier; a composite combines '10 per second and 1000 per day'. The service keeps one limiter per (user, api) via `computeIfAbsent` and returns remaining and retry-after for the 429. Each limiter makes refill-check-take atomic; for many servers, move state into Redis with an atomic script."

Follow-ups: *Why is fixed window risky?* → Up to 2N across a boundary. *Token bucket vs sliding log?* → Bucket: bursts, O(1) memory; log: exact, stores every timestamp.

Self-check

1. Burst up to capacity, then a steady rate? → Token bucket.
2. Fixed window of 5 across a boundary in 0.2 s? → Up to 10.

3. Pattern making two rules look like one limiter? → Composite.
4. Why must refill + check + take be atomic? → Two threads could both take the last token.
5. Why probe every child before committing? → A request denied by one rule must not use a slot in another.

TL;DR

- **Strategy** algorithms, **Factory** from rules, **Composite** (probe all, commit all), **Facade** service.
- One limiter per (user, api) via `computeIfAbsent`; `Decision` → 200 or 429 + Retry-After.
- Token bucket (bursty, O(1)), fixed window (2× edge burst), sliding log (exact, O(N)), sliding counter (approx, O(1)).
- Atomic refill-check-take (Chapter 13); Redis + Lua when distributed.

20.8 Chapter summary

Component	Anchor pattern(s)	The one sentence to remember
Splitwise	Strategy	Split rules return shares; ledger = one signed number per pair; simplify with net positions + greedy (≤ n − 1).
Ledger Co. (full case in Chapter 21)	—	A loan ledger: same "one authoritative record + `BigDecimal`" idea, different rules.
Notification service	Template Method + Strategy + Observer	Retry once in the parent, one class per reason not to send, one exit that tells listeners.
In-memory cache	Strategy + Observer	HashMap + doubly linked list; policy owns the order; one removal path.
LRU cache	— (data structure)	Map to node, move to front on use, evict before tail; or `LinkedHashMap(accessOrder=true)`.
Ride sharing	Strategy + Decorator + state machine	Ranked list for declines, surge wraps pricing, atomic driver claim.
Carpool variant	Strategy	Offer seats per vehicle; select with Most Vacant / Preferred Vehicle.
Logging framework	Chain of Responsibility + Singleton + Decorator	Hierarchy with inherited levels; propagate up until additivity stops; async is a wrapper.
Pub-sub	Observer via broker	Append-only log + offsets; ack after processing; dead letters.
Rate limiter	Strategy + Factory + Composite	One limiter per key; swappable algorithms; 429 + Retry-After.

Common thread: the part most likely to change sits behind an interface, and the service only orders the steps. "Now add X" → a new class plus a one-line wiring change.

20.9 Chapter self-check

1. Two patterns in more than half the designs? → Strategy and Observer.
2. Strategy vs Decorator? → Strategy *replaces* a rule; Decorator *wraps* one and adds to its result.
3. Strategy vs Template Method? → Strategy swaps the whole algorithm (composition); Template Method fixes it and lets subclasses fill steps (inheritance).
4. Which designs used a "single exit", and why? → Notification (`finish`) and cache (`remove`): records, stats and listeners can't be skipped.
5. Three uses of CAS / `computeIfAbsent` for correctness? → Driver claim (`AtomicBoolean`), subscription commit (`AtomicLong.compareAndSet`), limiter creation (`computeIfAbsent`).

20.10 Chapter TL;DR

- Interviews probe the **rule most likely to change** — put it behind an interface.
- **Strategy** everywhere; **Observer** for "tell someone"; **Decorator** to add behaviour (surge, async); **Template Method** for a shared algorithm with one varying step; **Chain of Responsibility** for the logger hierarchy; **Composite** for combined limits.
- `BigDecimal` money; injected clocks; the smallest correct concurrency tool (one lock, CAS, `computeIfAbsent`, per-key locks, sharding).
- Volunteer the honest limits: greedy isn't optimal, LRU needs a write lock, at-least-once duplicates, fixed windows burst at edges.

Chapter 21 — Case Studies: Machine-Coding Round Problems (Study Edition)

> **Goal.** Chapters 14–20 taught whiteboard design. This chapter trains the **machine-coding round**: a written problem, 90–120 minutes, and **running code** the interviewer feeds their own inputs into. You get the scoring rubric, a skeleton to type from memory, a time plan, and eight real problems solved end to end in Java.

Each problem follows the usual template (problem → requirements → use cases → entities → diagram → flows → decisions → code → edge cases → extensions → interview → self-check → TL;DR). Code is Java 21, standard library only, one package per problem under `code/lld/ch21/<name>/`. From `chapters/21-case-machine-coding/code`:

```
javac --release 21 -d /tmp/lld_21 $(find . -name '*.java')
java -cp /tmp/lld_21 lld.ch21.taskmgmt.Main       # or wallet, calendar, codingblox, propertyhunt, ledger, inventory, filesearch
```

#	Problem	Source	The one idea that makes it work	Patterns
21.1	Task Management System	awesome-low-level-design	Status *state machine* + composable filters	State (enum), Specification, Strategy, Observer
21.2	Digital Wallet Service	awesome-low-level-design	`BigDecimal` money, idempotency keys, lock ordering	Strategy (offers), Facade
21.3	Event Calendar	Flipkart (kumaransg/LLD)	Half-open intervals + a boundary *sweep* for free slots	Strategy, Repository
21.4	Coding Blox	Flipkart (kumaransg/LLD)	Command dispatcher + level-based scoring	Command, Strategy, Repository
21.5	PropertyHunt	ClearTrip (kumaransg/LLD)	Optional criteria → list of predicates; unit normalisation	Builder, Specification, Strategy
21.6	Ledger Co (loans & EMIs)	Navi (kumaransg/LLD, Geektrust)	Derive balances from facts, don't store them	Strategy (interest), Command
21.7	Inventory Management	Common (Amazon/Flipkart-style)	Append-only movement ledger + per-SKU lock	Strategy (allocation), Observer (reorder)
21.8	Unix File Search (`find`)	Common OOD/machine-coding question	Specification filters + Composite AND/OR/NOT; Builder for criteria	Specification, Composite, Builder

(Food ordering "FoodKart" is solved in Chapter 19.)

21.0 How the machine-coding round works

What it is

An OO design interview where **working code is the answer**: a 1–2 page statement, your IDE, 90–120 minutes, no database/UI/framework — data in maps and lists, driven by `main`, a command list or tests. Unlike system design (servers, databases, queues) it never leaves one program; its vocabulary is **classes**. Analogy: whiteboard design *describes* a house; machine coding *builds a shed* while the inspector leans on every wall. Common at Flipkart, Swiggy, PhonePe, Navi, Razorpay, CRED, Meesho, Cleartrip.

How it is judged

Criterion	What is checked → how to score well
Working, demo-able code	Compiles, runs the sample, survives *their* inputs → Thin end-to-end path in the first hour; never finish with non-compiling code
Functional completeness	Every mandatory requirement; bonus after → Implement in the statement's order
Clean models	One job per entity, private fields, invariants inside the entity → Enums for fixed sets, records for values, classes for changing things
Separation of concerns	Parsing, logic, storage apart; no `System.out` in services → Driver → Service → Repository
Extensibility	"Add a new offer/sort/rule" = add a class → Strategy / Specification for "can be X or Y"
Edge cases	Unknown ids, duplicates, invalid input, wrong user → Custom exceptions; driver catches per command and **keeps going**
Readability	Names, small methods, no copy-paste → Nouns → classes, verbs → methods
Concurrency (when asked)	Wallets, inventory, booking → Say where the lock is and why; demo with a pool
Tests (bonus)	Extra credit → An `assert`-style self-check in `main` suffices

The five-step method, and how the clock changes it

Whiteboard method (Chapter 1): **Requirements** → **Actors & use cases** → **Classes** → **Relationships** → **Code** (roughly 5–8 / 3–5 / 8–10 / 8–10 / 12–15 min in a 45-minute round, code covering only 2–3 key methods). **Machine coding flips the proportions:** steps 1–4 take ~20 minutes, code the rest. A parking lot has ~12 classes but only 3 carry the main flows — build those, name the rest. Also judged: **scope**, **one job per class**, **relationships** (hold/own/extend), **code and model agree**, **a stated trade-off** — never UML notation. Top failures: **nothing runs at the end**, and a **god class** `Manager`.

A reusable project skeleton

```
src/
  model/          entities & value objects      (Task, Wallet, Loan, Property ...)   - enums, records, small
classes
  repository/     storage contracts + in-memory (TaskRepository, InMemoryTaskRepository)
  service/        business rules, one per area  (TaskService, WalletService)          - no printing, no
parsing
  strategy/       pluggable rules               (Offer, AllocationStrategy, SortBy)
  exception/      domain exceptions             (NotFoundException, InsufficientFundsException)
  driver/         Main + CommandProcessor       (parses input, calls services, prints output)
```

Layer	Knows about	Must NOT know about	Why
model	itself, other models	services, storage, console	Entities guard invariants (`Task.changeStatus` rejects `TODO → DONE`).
repository	model	services, console	Swap `HashMap` for a DB touching only this layer (DIP).
service	model, repository & strategy *interfaces*	concrete repositories, console	The part the interviewer reads most carefully.
strategy	model	services	Each "X or Y" rule → interface + small classes (OCP).
exception	—	—	One class per failure kind, readable message.
driver	services	repository internals	Parsing/printing replaceable; logic isn't.

(This book uses one package per problem; in the round, create the sub-packages.) Reusable blocks: a **generic in-memory repository** (Coding Blox), an `AtomicInteger` **ID generator**, and a **command processor** `Map<String, Consumer<String[]>>` (Command pattern, Chapter 10).

The 90-minute time plan

Minutes	Activity	Output
0–10	Read twice, ask questions, mark mandatory vs bonus, paste sample input into the driver	Numbered requirement list
10–20	Nouns → entities, verbs → methods; pick the 1–2 patterns the statement begs for	Packages, enums, records
20–60	Mandatory requirements **in order**, running after each	Sample input → sample output
60–75	Edge cases, validation, custom exceptions	Driver prints errors and continues
75–85	Bonus (only if all mandatory work); thread-safety for money/stock	Bonus commands
85–90	Clean names, delete dead code, final run, 2-minute walkthrough	Compiling code, demo, story

Run the program at least every 15 minutes — a compile error at minute 88 is a disaster, at minute 30 a 20-second fix.

What to say in the first two minutes

1. "Everything in memory behind a repository interface, so a DB can be added later."
2. "I'll drive it from `main` with the sample commands and print errors without crashing."
3. "Mandatory is A, B, C; bonus D, E. A–C first."
4. "Where does it need to be thread-safe?" (Wallets and inventory: always.)

Common mistakes in this round

Mistake	Do this instead
20 minutes on a perfect class diagram	Enums/records first; the diagram lives in names
One `Manager` with 15 maps	One service per area + repositories
`System.out` inside services	Services return data; the driver prints
`double` for money (`0.1 + 0.2 != 0.3`)	`BigDecimal` with fixed scale (21.2) or whole units (`long` paise)
Crashing on the first bad command	Catch domain exceptions in the driver loop
Bonus before mandatory	Mandatory in order; bonus last
Every class in full detail	Core flow first; stub or name the rest
Unseeded `Random`; `LocalDate.now()`	Seed it / Strategy (21.4); inject a `Clock` (21.1, 21.2)

Real-world examples

Standard Spring Boot layering: controllers = driver, `@Service` = rules, `@Repository` = storage; in-memory repositories are the usual unit-test fakes.

In the interview

> **30-second answer:** "I read the statement twice, list mandatory and bonus requirements and clarify ambiguities. I set up model / repository / service / strategy / exception / driver packages, put the sample input in the driver, and implement requirements one by one, running after each. Services return data and throw domain exceptions; the driver prints and continues. Then edge cases, thread-safety where state is shared, and bonus features."

Why an interface for an in-memory repository? → Services depend on an abstraction; JDBC/Redis can replace it.

Self-check

- **Q:** The six packages? → **A:** model, repository, service, strategy, exception, driver.
- **Q:** Where does `System.out.println` live? → **A:** Only in the driver.
- **Q:** Minute 60, one mandatory requirement missing, bonus tempting? → **A:** Finish the mandatory one.
- **Q:** Why inject a `Clock`? → **A:** Time-based logic gives the same output every run.

TL;DR

Working code first, clean layering second, patterns where the statement asks for variation, graceful errors everywhere. **model** → **repository** → **service** → **strategy** → **exception** → **driver**. Run often.

21.1 Design a Task Management System

Problem & clarifying questions

The core of Jira/Asana/Trello: create, update, delete tasks (title, description, due date, priority, status), assign, move through statuses, comment, search/filter (priority, due date, assignee), reminders, task history, safe concurrent access.

Question	Assumed answer
Statuses and transitions?	TODO, IN_PROGRESS, BLOCKED, DONE, CANCELLED; only listed transitions (TODO can't jump to DONE; CANCELLED final; DONE can re-open).
Assignees?	One (nullable).
"Reminder" in a console app?	A method a scheduler calls: notify owners of open tasks due within N days or overdue.
Past due dates?	Not on creation.
Notifications?	A listener interface; the demo prints.
"Task history"?	Per-task audit log (built); a user's finished tasks is just a search: `assignedTo(u).and(withStatus(DONE))`.

Requirements

Functional: register users; create/update/delete tasks; assign; change status via allowed transitions; comment; search with any filter combination (assignee, status, min priority, due before, open only, title contains) + sort (due date, priority, id); reminders; per-task history. **Non-functional:** thread-safe; new filters/sorts/channels without editing code; deterministic (injected `Clock`).

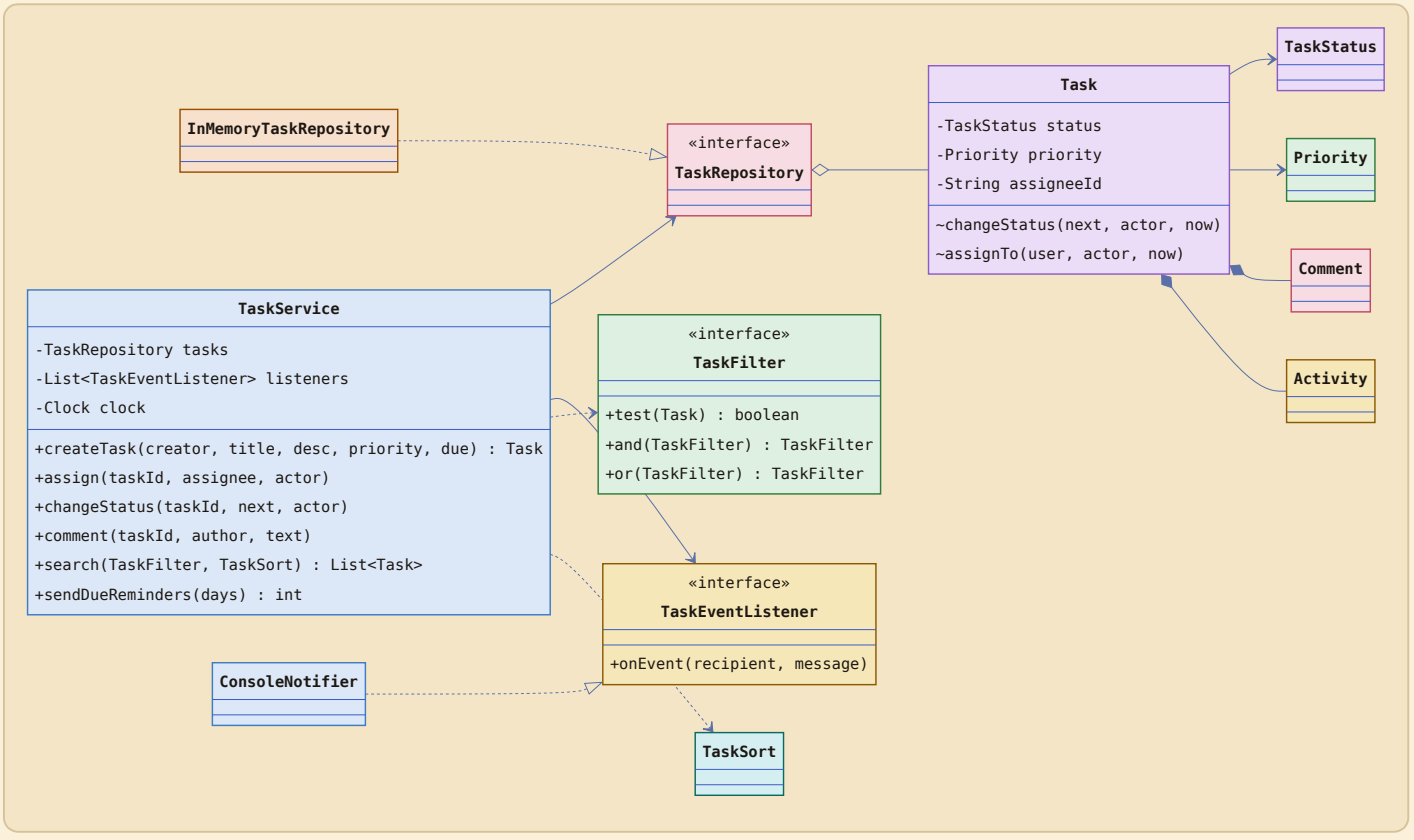
Use cases

Task creator (create, update, delete, assign, comment, search), **Assignee** (change status, comment, see own tasks), **Scheduler** (due-date reminders), **Notification channel** (receives events).

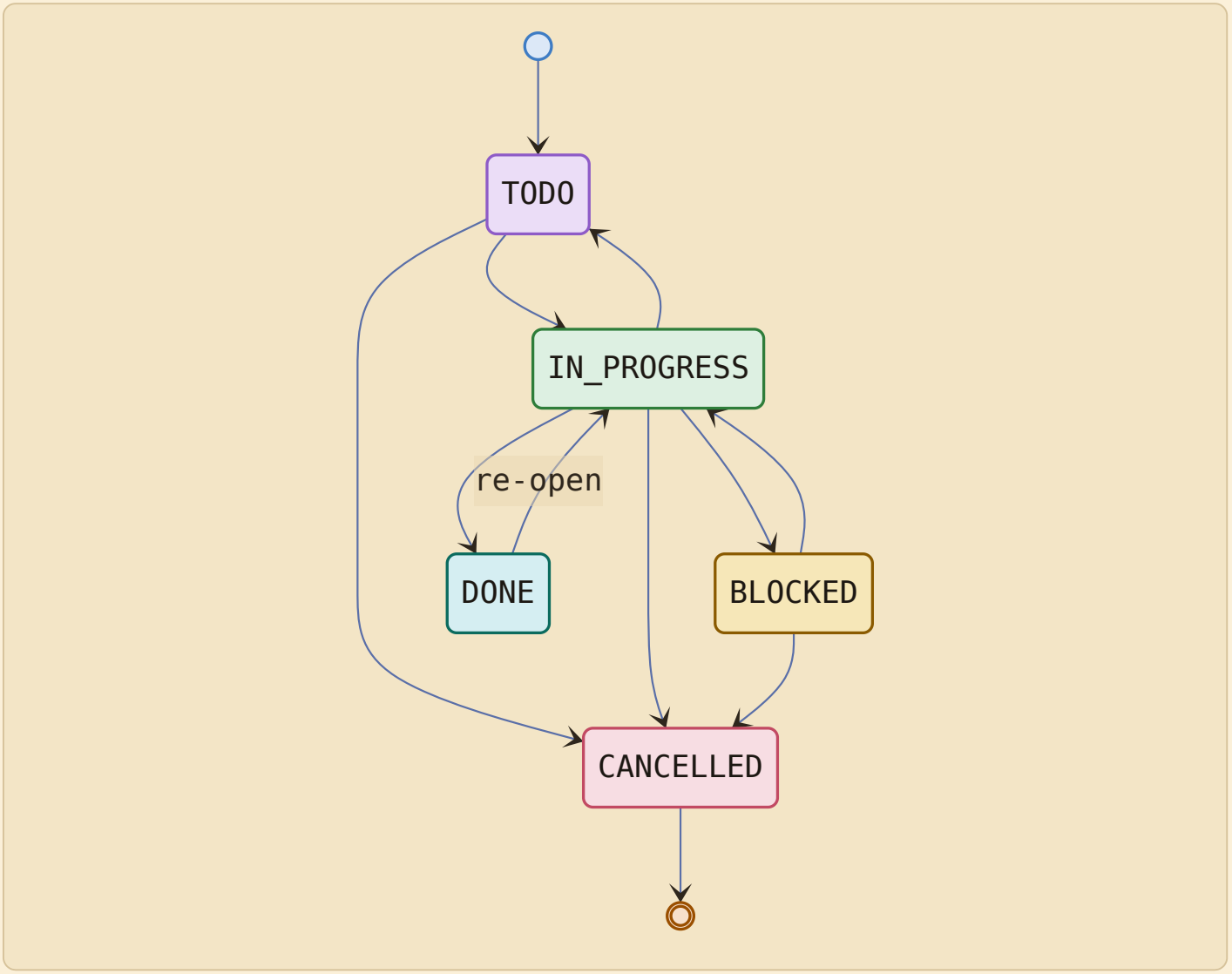
Core entities

Class	Responsibility
`Task`	Data; enforces legal status changes; records its own history; mutators `synchronized`.
`TaskStatus` (enum)	Statuses + allowed next statuses (tiny state machine).
`Priority` (enum)	LOW < MEDIUM < HIGH < CRITICAL (enum order compares).
`User`, `Comment`, `Activity`	Immutable records.
`TaskRepository` / `InMemoryTaskRepository`	Storage contract + `ConcurrentHashMap`.
`TaskFilter`	Composable predicate (**Specification**) with factories and `and` / `or`.
`TaskSort` (enum)	Named comparators (**Strategy**); `BY_ID` sorts `T2` before `T10`.
`TaskEventListener` / `ConsoleNotifier`	**Observer**.
`TaskService`	**Facade**: every use case, validation, clock, events.
`NotFoundException`, `InvalidTransitionException`	Clear failures.

Class diagram



Key flows



Change status: service finds the task → `task.changeStatus(next, actor, now)` → illegal: `InvalidTransitionException` (driver prints it); legal: log activity, set status, notify the creator.

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Transitions in the `TaskStatus` enum	State (enum)	One place for rules; `Task` asks `canMoveTo`.	Class per status — overkill with no per-state behaviour.
`TaskFilter extends Predicate<Task>` + `and/or`	Specification	Any combination; new filter = one factory method.	`search(assignee, status, priority, …)` with nullable params.
`TaskSort` enum of comparators	Strategy	Sort order is data.	`switch` on a string.
Listener list	Observer	Email/Slack/push without touching the service.	`System.out` in the service.
`TaskService` single entry	Facade	Simple driver API.	Driver using repositories.
Injected `Clock`	DI	Deterministic reminders/tests.	`LocalDate.now()` scattered.
`synchronized` `Task` mutators	Monitor per entity	Different tasks never block each other.	One global service lock.

Java implementation

```java
// File: lld/ch21/taskmgmt/TaskStatus.java
public enum TaskStatus {
    TODO, IN_PROGRESS, BLOCKED, DONE, CANCELLED;

    public Set<TaskStatus> allowedNext() {
        return switch (this) {
            case TODO -> EnumSet.of(IN_PROGRESS, CANCELLED);
            case IN_PROGRESS -> EnumSet.of(BLOCKED, DONE, TODO, CANCELLED);
            case BLOCKED -> EnumSet.of(IN_PROGRESS, CANCELLED);
            case DONE -> EnumSet.of(IN_PROGRESS);          // re-open
            case CANCELLED -> EnumSet.noneOf(TaskStatus.class);
        };
    }

    public boolean canMoveTo(TaskStatus next) { return allowedNext().contains(next); }

    public boolean isOpen() { return this != DONE && this != CANCELLED; }
}

// File: lld/ch21/taskmgmt/Task.java
    synchronized void changeStatus(TaskStatus next, String actor, LocalDateTime now) {
        if (!status.canMoveTo(next)) throw new InvalidTransitionException(id, status, next);
        log(now, actor, "status " + status + " -> " + next);
        status = next;
    }

// File: lld/ch21/taskmgmt/TaskFilter.java
@FunctionalInterface
public interface TaskFilter extends Predicate<Task> {

    static TaskFilter all() { return t -> true; }
    static TaskFilter assignedTo(String userId) { return t -> userId.equals(t.assigneeId()); }
    static TaskFilter withStatus(TaskStatus s) { return t -> t.status() == s; }
    static TaskFilter atLeast(Priority p) { return t -> t.priority().compareTo(p) >= 0; }
    // ... dueOnOrBefore, open, titleContains

    default TaskFilter and(TaskFilter other) { return t -> test(t) && other.test(t); }
    default TaskFilter or(TaskFilter other) { return t -> test(t) || other.test(t); }
}
```

Full runnable code: code/lld/ch21/taskmgmt/ (see the Full Edition).

Output (excerpt):

```
== reminders (due within 3 days)
  [notify carol] Reminder: T4 'Review PR #42' is due 2026-09-28
  [notify bob] Reminder: T3 'Upgrade to Java 21' is due 2026-10-01
== edge cases
  error: Task T2 cannot move from TODO to DONE
  error: Due date is in the past: 2026-09-27
  error: Cannot assign a CANCELLED task T4
```

Self-assignment sends no notification; reminders consider only open tasks; the illegal jump is rejected with a clear message.

Concurrency & edge cases

- Per-task `synchronized` mutators and getters; `ConcurrentHashMap` storage with `findAll()` returning a copy; listeners in `CopyOnWriteArrayList`; ids from `AtomicInteger`.
- Handled: unknown ids, blank title, past due date, illegal transition, assigning a closed task.
- **Not handled (say so):** lost updates on text fields (last writer wins) → optimistic locking with a `version` field.

Extensions & follow-up questions

- **Sub-tasks / dependencies:** `parentId` + `blockedBy`; forbid DONE while a blocker is open.
- **Custom workflows (Jira):** move the transition table into an injected `Workflow` (`Map<Status, Set<Status>>`) per project.
- Also: watchers (`Set<String>`), recurring tasks, `remindAt` + a `ScheduledExecutorService`, secondary indexes or Elasticsearch for search at scale.

Real-world example

Jira projects have *workflows* (statuses + allowed transitions that notify watchers) and JQL search like `assignee = currentUser() AND status != Done ORDER BY priority DESC` — compose filters, then sort, as `TaskFilter` + `TaskSort` do.

In the interview

Present: (1) statuses and transitions — shows you think in invariants; (2) `Task` and `TaskService` API; (3) filters as composable predicates; (4) observer; (5) run the demo. *Why not the full State pattern?* → Statuses have no behaviour besides legal transitions, so an enum table is simpler and still closed to illegal moves.

Self-check

- **Q:** Where is "TODO cannot go to DONE" coded? → **A:** `TaskStatus.allowedNext()`, enforced by `Task.changeStatus`.
- **Q:** Add a "created by X" filter? → **A:** `static TaskFilter createdBy(String u) { return t -> u.equals(t.createdBy()); }` — nothing else changes.
- **Q:** Why a lock per `Task`, not in the service? → **A:** Edits to different tasks never contend.

TL;DR

Task guards its own state machine and history; service = facade; filters = Specification, sorts = Strategy, notifications = Observer; thread-safety per task.

21.2 Design a Digital Wallet Service

Problem & clarifying questions

A wallet like Paytm, PhonePe or a PayPal balance: wallets, top-up, transfer, statement. The awesome-low-level-design statement also lists accounts, payment methods, external transfers, multi-currency, security and **concurrent transactions with consistency**. **Cashback** is added here (common in variants; shows Strategy and double-entry). The interviewer focuses on **money arithmetic**, **no double-spending under concurrency**, and **no double-charging on retries** (idempotency).

Question	Assumed answer
Money type?	`BigDecimal`, scale 2; > 2 decimals rejected, not rounded.
Negative balance?	No.
Client retries after a timeout?	Each request has an **idempotency key**; a repeat returns the original result and moves no money.
Offers?	"10% on first transfer, max 50" and "flat 25 on transfers ≥ 1000"; no stacking — best wins.
Cashback source?	A funded `PROMO` **wallet**; cashback = ordinary transfer PROMO → sender, so money is never created. Budget gone → no cashback, transfer stands.
Multi-currency?	Out of scope (extensions).

Requirements

Functional: create wallet; top up; atomic transfer; statement with running balance; best cashback from the PROMO wallet; idempotent top-up and transfer. **Non-functional:** no lost updates, negative balances or

deadlocks; **total money conserved** (only top-ups add money); exact decimals; new offers without touching transfer code.

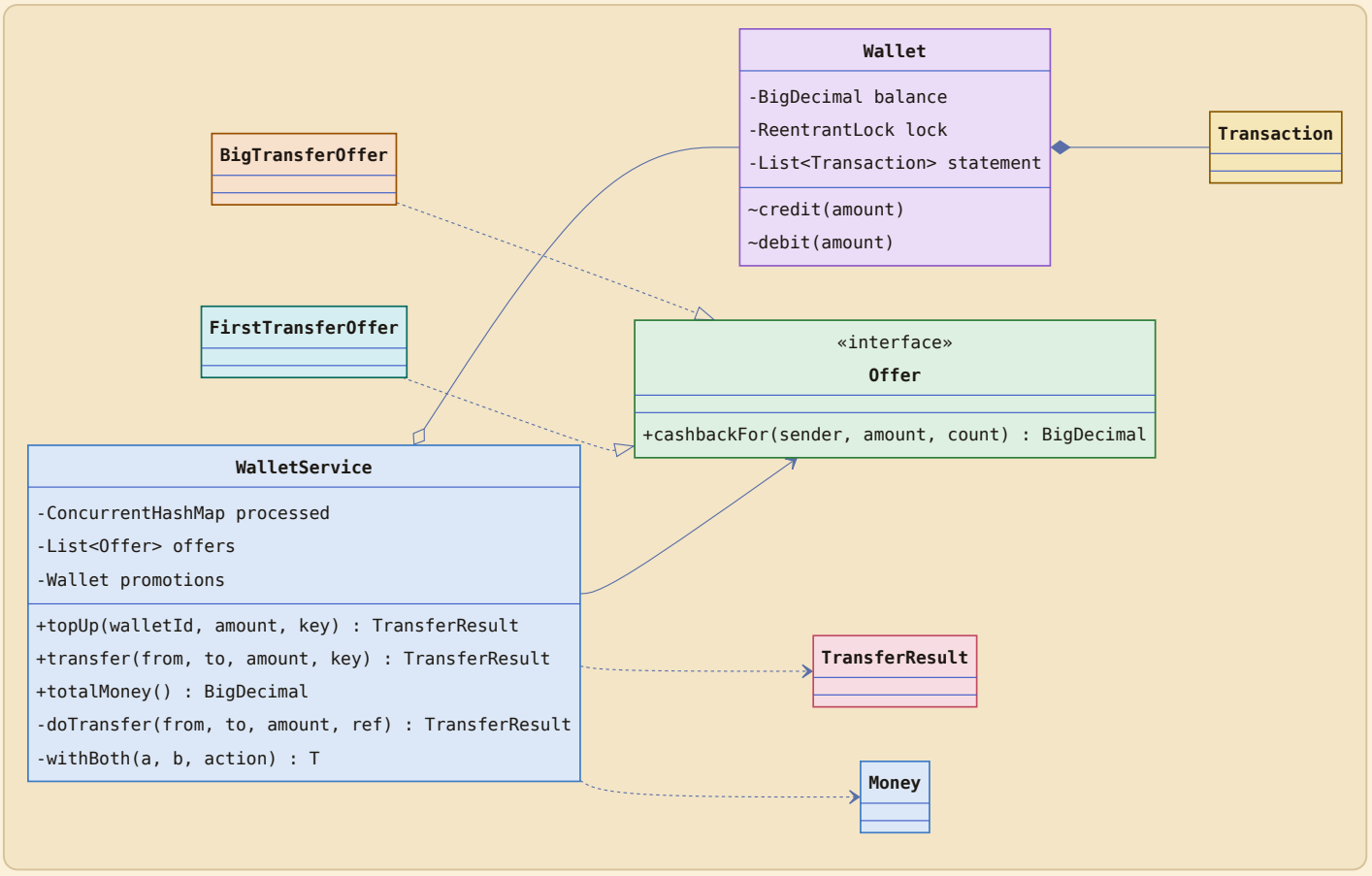
Use cases

User (create wallet, top up, transfer, statement), **Bank/card gateway** (top-up source), **Marketing** (defines offers, funds PROMO), **Client app** (retries with the same key).

Core entities

Class	Responsibility
`Wallet`	Balance + own statement + its `ReentrantLock`; `debit` refuses to go below zero.
`Transaction` (record)	Immutable line: type, amount, balance after, counterparty, reference.
`TxnType`	TOP_UP, TRANSFER_OUT/IN, CASHBACK_OUT (on PROMO)/IN.
`TransferResult` (record)	Stored for idempotent replays.
`Money`	Validates/normalises amounts.
`Offer` + `FirstTransferOffer`, `BigTransferOffer`	**Strategy**: cashback for a completed transfer.
`WalletService`	**Facade**: wallets, lock ordering, idempotency, offers.
`InsufficientFundsException`, `WalletNotFoundException`	Failures.

Class diagram



Key flows

Transfer: validate → `processed.computeIfAbsent(key, doTransfer)` (a known key returns the stored result) → lock both wallets **in id order** → debit, credit, record TRANSFER_OUT/IN → unlock → best offer → second locked step PROMO → sender (PROMO sorts before W1), recording CASHBACK_OUT/IN → store result.

Design decisions & patterns used

Decision	Why	Alternative rejected
`BigDecimal` scale 2, reject > 2 decimals	Exact money; no silent rounding of `1.005`.	`double` (wrong); `long` paise is also fine — say so.
Lock per wallet, acquired in id order	A→B and B→A would deadlock if each locked its source first; a global order makes a cycle impossible.	One global lock (serialises every transfer).
Debit before credit, under both locks	`debit` throws before any change → atomic.	Credit first + compensation.
Idempotency via `computeIfAbsent`	Check + run is one atomic step per key; if it throws, nothing is stored and the client may retry.	`containsKey` then `put` (race).
Cashback from a funded `PROMO` wallet	Double-entry: every credit has a debit; total changes only on top-ups; campaigns can't overspend.	Crediting from thin air.
Cashback as a second step, after the transfer locks	At most two locks at once; transfers don't queue on PROMO.	Locking three wallets per transfer.
`Offer` interface, best wins	New promotions = new classes; no-stacking in one method.	`if/else` on codes.

`computeIfAbsent` detail: the call is atomic, the function runs only if absent, an exception records nothing, and other updates to the *same map* may block while it runs — so keep the function short and never update the map inside it (ours only touches wallets).

Java implementation

```java
// File: lld/ch21/wallet/WalletService.java
    public TransferResult transfer(String fromId, String toId, BigDecimal amount, String idempotencyKey) {
        if (fromId.equals(toId)) throw new IllegalArgumentException("Cannot transfer to the same wallet");
        BigDecimal amt = Money.requirePositive(amount);
        Wallet from = get(fromId), to = get(toId);
        return processed.computeIfAbsent(idempotencyKey, key -> doTransfer(from, to, amt, key));
    }

    private TransferResult doTransfer(Wallet from, Wallet to, BigDecimal amt, String ref) {
        // Step 1: the transfer itself, under both wallet locks.
        int n = withBoth(from, to, () -> {
            from.debit(amt);                        // throws before any change if funds are short
            to.credit(amt);
            from.record(txn(ref, from, TxnType.TRANSFER_OUT, amt, to.id()));
            to.record(txn(ref, to, TxnType.TRANSFER_IN, amt, from.id()));
            return transferCounts.computeIfAbsent(from.id(), k -> new AtomicInteger()).incrementAndGet();
        });
        // Step 2: cashback is a second, ordinary transfer PROMO -> sender (same lock-ordering rule).
        BigDecimal offered = bestOffer(from, amt, n);
        // ... withBoth(promotions, from, ...): if the budget covers it, debit PROMO, credit sender,
        // ... record CASHBACK_OUT / CASHBACK_IN; otherwise cashback = 0
        return new TransferResult(ref, from.id(), to.id(), amt, cashback);
    }

    /** Locks two wallets, always the smaller id first, so two opposite operations can never deadlock. */
    private static <T> T withBoth(Wallet a, Wallet b, Supplier<T> action) {
        Wallet first = a.id().compareTo(b.id()) < 0 ? a : b;
        Wallet second = first == a ? b : a;
        first.lock().lock();
        try {
            second.lock().lock();
            try { return action.get(); } finally { second.lock().unlock(); }
        } finally { first.lock().unlock(); }
    }
```

Full runnable code: code/lld/ch21/wallet/ (see the Full Edition).

Output (excerpt):

```
  TransferResult[reference=req-3, from=W1, to=W2, amount=200.00, cashback=20.00]
  retry returns same result? true
  error: At most 2 decimal places: 1.005
  failed req-5 can be retried later: 10.00
== concurrency: 1000 opposite transfers on 8 threads
  total before=2400.50 after=2400.50 conserved=true
```

Retried `req-3` moved no money; failed `req-5` stored nothing, so its key worked later; the total was conserved with no deadlock.

Concurrency & edge cases

- Every read-modify-write of a balance holds the wallet's lock; all two-lock paths go through `withBoth`, so the order can't be forgotten; never more than two locks held.
- **Same key, different body** (client bug): today the stored result is returned; production also stores a request hash and rejects mismatches.
- Cashback runs after the transfer's locks are released, so briefly a transfer is visible without its cashback — fine for a reward.

Extensions & follow-up questions

- **Multi-currency:** `Currency` per wallet + `FxRateProvider`; store both amounts and the rate. **Payment methods / payouts:** `PaymentMethod.charge` before crediting; `PENDING_PAYOUT` + `PayoutGateway`. **Refunds:** post a

compensating transaction, never edit history.

- **Distributed:** row locks (`SELECT ... FOR UPDATE`) or optimistic versions; idempotency keys under a unique constraint.
- **Slow transfers blocking the map:** `putIfAbsent(key, new CompletableFuture<>())`; winner runs outside the map, duplicates `join()`; remove the key on failure.
- **Limits (daily cap, KYC):** validators in a Chain of Responsibility before `doTransfer`.
- **Why not `AtomicReference<BigDecimal>` CAS?** A transfer changes two balances together; CAS on two variables isn't atomic.

Real-world example

Stripe's `Idempotency-Key` header returns the saved result of the first attempt on a retry instead of charging twice. Paytm/PhonePe cashback rules ("X% up to a cap on your first transaction") are `FirstTransferOffer`.

In the interview

Lead with the three risks (exact money, concurrency, retries — and "cashback comes out of a budget wallet"). Show `Money`, then `doTransfer` with lock ordering (draw the A→B / B→A deadlock), then idempotency; run the concurrent demo and point at `conserved=true`. Offers are a 2-minute Strategy mention.

Self-check

- **Q:** Why `new BigDecimal("0.1")` over `new BigDecimal(0.1)`? → **A:** The double constructor captures the binary approximation; the String one is exact.
- **Q:** W1→W2 and W2→W1 at once — why no deadlock? → **A:** Both lock W1 first.
- **Q:** A transfer fails for insufficient funds; reuse the key? → **A:** Yes — `computeIfAbsent` stores nothing when the function throws.
- **Q:** Where does "max 3 transfers per minute" go? → **A:** A validator before `doTransfer`.

TL;DR

`BigDecimal` money, lock both wallets in a global order, debit before credit, idempotency key → at most once, Strategy offers paid from a funded promotions wallet; prove it with a conserved-total concurrent demo.

21.3 Design an Event Calendar (Flipkart)

Problem & clarifying questions

Flipkart statement (kumaransg/LLD `Event_calendar_flipkart`): users with **working hours**; **teams** (a user in at most one); one-day **events** whose participants are users and/or teams; a user can be in **one event at a time**. For teams the creator says how many **representatives (N)** each must send; the system blocks N *available* members, else the event isn't created. Required: create user, team, event; **a user's events in a time range** (including as representative); **suggest free slots** for users/teams on a day. Bonus: **update event time**, **recurring events**. No DB/UI; "do not make assumptions, ask".

Question	Answer used
Does 14:00–15:00 clash with 15:00–16:00?	No: **half-open** `[start, end)`.
Must events fit every attendee's working hours?	Yes.
Which members represent a team?	First N free members in team order (pluggable strategy).
Direct attendee also in an invited team?	Not picked again as representative.
Free for a team with N?	At least N members working and not busy.
Recurring series with one clash?	Reject the whole series.

The sample is inconsistent. It says A + T1 (N = 1) are free "10AM to 3PM and 4PM to 7PM", but T1's members start at 11:00 and 11:30, so by the rules the answer is `11:00-15:00, 16:00-19:00` (what our program prints). Pointing this out politely is exactly what "ask, don't assume" tests. (It also says "Onboard 5 Users" but lists six.)

Requirements

Functional: create user (working hours); create team (one team per user); create event, rejected if anyone is outside hours, busy, or a team lacks N free members; a user's events in a range; free slots; bonus: atomic update time; recurring daily/weekly. **Non-functional:** "is everyone free?" + booking must be **atomic**; representative choice replaceable.

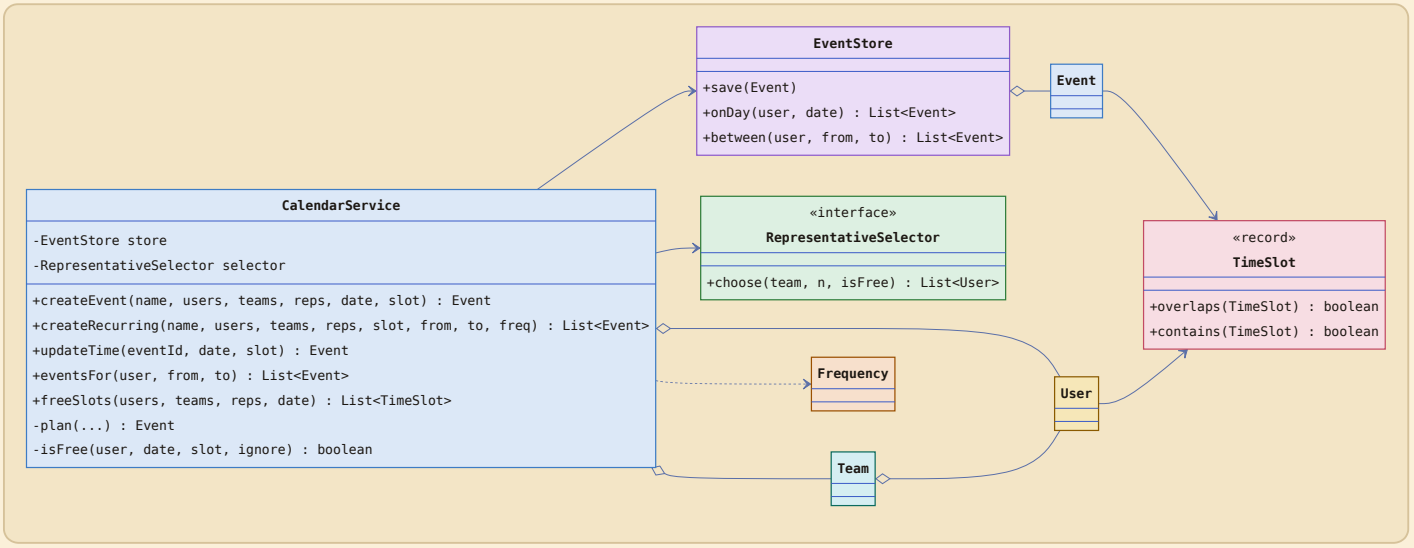
Use cases

Admin (onboard user, create team), **Organiser** (create/recurring/move event, find free slots), **Attendee** (own events in a range).

Core entities

Class	Responsibility
`TimeSlot` (record)	Half-open `[start, end)`; `overlaps`, `contains`; validates start < end.
`User`, `Team` (records)	Name + working hours; name + ordered members.
`Event` (record)	One occurrence: date, slot, request (users, teams, N), actual `attendees`, optional `seriesId`.
`Frequency`	DAILY / WEEKLY, knows the next date.
`EventStore`	Repository + index `user → date → events` (`TreeMap.subMap` range queries).
`RepresentativeSelector`	**Strategy**: pick N free members.
`CalendarService`	Use cases; planning separated from saving.
`CalendarException`	Readable rejection reason.

Class diagram



Key flows

Create event = plan, then save: reject if no participants; each direct user must be inside working hours with no overlapping event; for each team the selector picks N free members not already attending (fewer → reject "team has only k free"); then save and index per attendee.

Free-slot sweep: collect every interesting time that day — each person's working-hour start/end and each of their events' start/end — and sort. Nothing changes between two consecutive cuts, so test each piece once

(all direct users free, each team has ≥ N free) and merge neighbouring good pieces. For A + T1 the cuts are 10:00, 11:00, 11:30, 15:00, 16:00, 18:30, 19:00, 19:30 → 11:00–15:00 and 16:00–19:00.

Design decisions & patterns used

Decision	Why	Alternative rejected
Half-open intervals	Back-to-back meetings don't clash; `overlaps` = `a.start < b.end && b.start < a.end`.	Closed intervals.
`plan()` separate from `save()`	Side-effect-free plan reused by create, recurring (plan all, then save all) and update (plan ignoring the old event, then swap) → all-or-nothing for free.	Save as you go + rollback.
`RepresentativeSelector`	Fairness / least-busy pluggable.	Hard-coded loop.
Index `user → TreeMap<date, events>`	Conflict checks read one user's one day; ranges use `subMap`.	Scanning all events.
Boundary sweep	Exact, any N, no minute loops.	Checking every 15/30 min (misses 11:30).
`synchronized` service methods	Check-then-book atomic.	Per-user locks (needs lock ordering; a scale-up).

Java implementation

```
// File: lld/ch21/calendar/TimeSlot.java
/** Half-open interval [start, end) inside one day. 14:00-15:00 and 15:00-16:00 do not overlap. */
public record TimeSlot(LocalTime start, LocalTime end) {
    // ... compact constructor rejects start >= end
    public boolean overlaps(TimeSlot o) { return start.isBefore(o.end) && o.start.isBefore(end); }
    public boolean contains(TimeSlot o) { return !o.start.isBefore(start) && !o.end.isAfter(end); }
    // ...
}

// File: lld/ch21/calendar/CalendarService.java  (freeSlots: the sweep)
        TreeSet<LocalTime> cuts = new TreeSet<>();
        for (User u : everyone) {
            cuts.add(u.workingHours().start());
            cuts.add(u.workingHours().end());
            for (Event e : store.onDay(u.name(), date)) { cuts.add(e.slot().start());
cuts.add(e.slot().end()); }
        }
        List<TimeSlot> result = new ArrayList<>();
        LocalTime prev = null;
        for (LocalTime cut : cuts) {
            if (prev != null) {
                TimeSlot piece = new TimeSlot(prev, cut);
                boolean ok = direct.stream().allMatch(u -> isFree(u, date, piece, null))
                        && tms.stream().allMatch(t -> t.members().stream()
                                .filter(m -> isFree(m, date, piece, null)).count() >= reps);
                if (ok) {
                    int last = result.size() - 1;
                    if (last >= 0 && result.get(last).end().equals(piece.start()))
                        result.set(last, new TimeSlot(result.get(last).start(), piece.end()));
                    else result.add(piece);
                }
            }
            prev = cut;
        }
        return result;
    // ...
    private boolean isFree(User u, LocalDate date, TimeSlot slot, Event ignore) {
        if (!u.workingHours().contains(slot)) return false;
        return store.onDay(u.name(), date).stream()
                .filter(e -> ignore == null || !e.id().equals(ignore.id()))
                .noneMatch(e -> e.slot().overlaps(slot));
    }
```

Full runnable code: code/lld/ch21/calendar/ (see the Full Edition).

Output (excerpt):

```
  FAILED: Event2: C is not available 2026-09-29 14:00-15:00
  ok: Event3 2026-09-28 15:00-16:00 [C, E, B, D]
  FAILED: Event5: F is not available 2026-09-28 10:00-11:00
== 5. free slots today for A + T1 (1 representative)
  [11:00-15:00, 16:00-19:00]
== bonus: update event time
  FAILED: Event1: team T1 has only 1 free member(s), needs 2
  FAILED: Sync@2026-09-28: A is not available 2026-09-28 15:30-16:30
```

The refused update keeps the old booking; the "Sync" series is rejected as a whole.

Concurrency & edge cases

- All service methods `synchronized` → "is free?" + save is atomic.
- Handled: start ≥ end, unknown user/team, user already in a team, no participants, N < 1, any clash in a series, moving an event onto a slot overlapping *itself* (ignored).

- Out of scope: time zones (real calendars store UTC instants + the user's zone), multi-day events (split into per-day slots).

Extensions & follow-up questions

- **Invites:** `InviteStatus {PENDING, ACCEPTED, DECLINED}`; declined attendees don't block.
- **Fair representatives:** `LeastBusySelector`.
- **Recurring at scale:** store one rule (iCalendar RRULE) and expand lazily per query.
- **Rooms:** a room is a "user" with building hours. **Big teams:** count arrays or a segment tree over minutes.

Real-world example

Google Calendar's "Find a time" and Outlook's Scheduling Assistant intersect attendees' busy blocks the same way, and store recurring meetings as an RRULE plus exceptions.

In the interview

Draw a timeline: free time = working time minus busy time, intersected. One sentence on half-open intervals. Code `TimeSlot`, `plan()`, `createEvent` first (first three requirements), then `eventsFor`, then the sweep. Mention the sample inconsistency. Point out that the bonus features fall out of `plan()`.

Self-check

- **Q:** Why does `plan()` take an `ignore` event? → **A:** When moving an event, its own booking mustn't conflict with the new time.
- **Q:** Why is all-or-nothing recurrence easy? → **A:** Plan every occurrence first, save only if all succeed.
- **Q:** Half-open overlap test? → **A:** `a.start < b.end && b.start < a.end`.
- **Q:** Why a `TreeMap` per user? → **A:** Range query = `subMap(from, to)`.

TL;DR

Half-open `TimeSlot`s, per-user day index, side-effect-free `plan()` reused by create/recurring/update, pluggable representative selector, boundary sweep for free slots — and ask about the contradictory sample.

21.4 Design a LeetCode-like Coding Platform — "Coding Blox" (Flipkart)

Problem & clarifying questions

Flipkart SDE-2 statement (kumaransg/LLD `leetcode-lld-flipkart-coding-blox`). Users sign up, create contests and attend others'. Questions and contests have a level (LOW/MEDIUM/HIGH); a contest uses **all questions of its level**. Command-driven:

Command	Meaning
`CreateUser <name>`	Default score **1500**.
`CreateQuestion <level> <score>` / `ListQuestion [level]`	Ids auto-increment from 1.
`CreateContest <name> <level> <creator>` / `ListContest [level]`	Creator auto-attends.
`AttendContest <contestId> <user>`	Register.
`RunContest <contestId> <creator>`	Creator only. Simulate solved questions (random), sum `contestPoints`, then: LOW → `score + (points − 50)`, MEDIUM → `score + (points − 30)`, HIGH → `score + points`.
`LeaderBoard <asc/desc>`	Users sorted by score.
Bonus `ContestHistory` / `WithdrawContest`	Per-contest results; leave before the run (not the creator).

Expectations: your own sample data, driver or tests, modular OO with interfaces, graceful edge cases, DRY.

Question	Answer used
How many questions solved?	Random 1..all, behind a **seeded** `SolveSimulator`.
Run twice / attend after run?	No (errors) — ratings never applied twice.
Tie-break / empty level?	By name / can't create a contest with no questions of its level.

Requirements

Functional: all commands above including both bonus ones. **Non-functional:** commands from a built-in list; errors printed, processing continues; scoring rule and solving logic replaceable; rating updates safe if two contests finish at once.

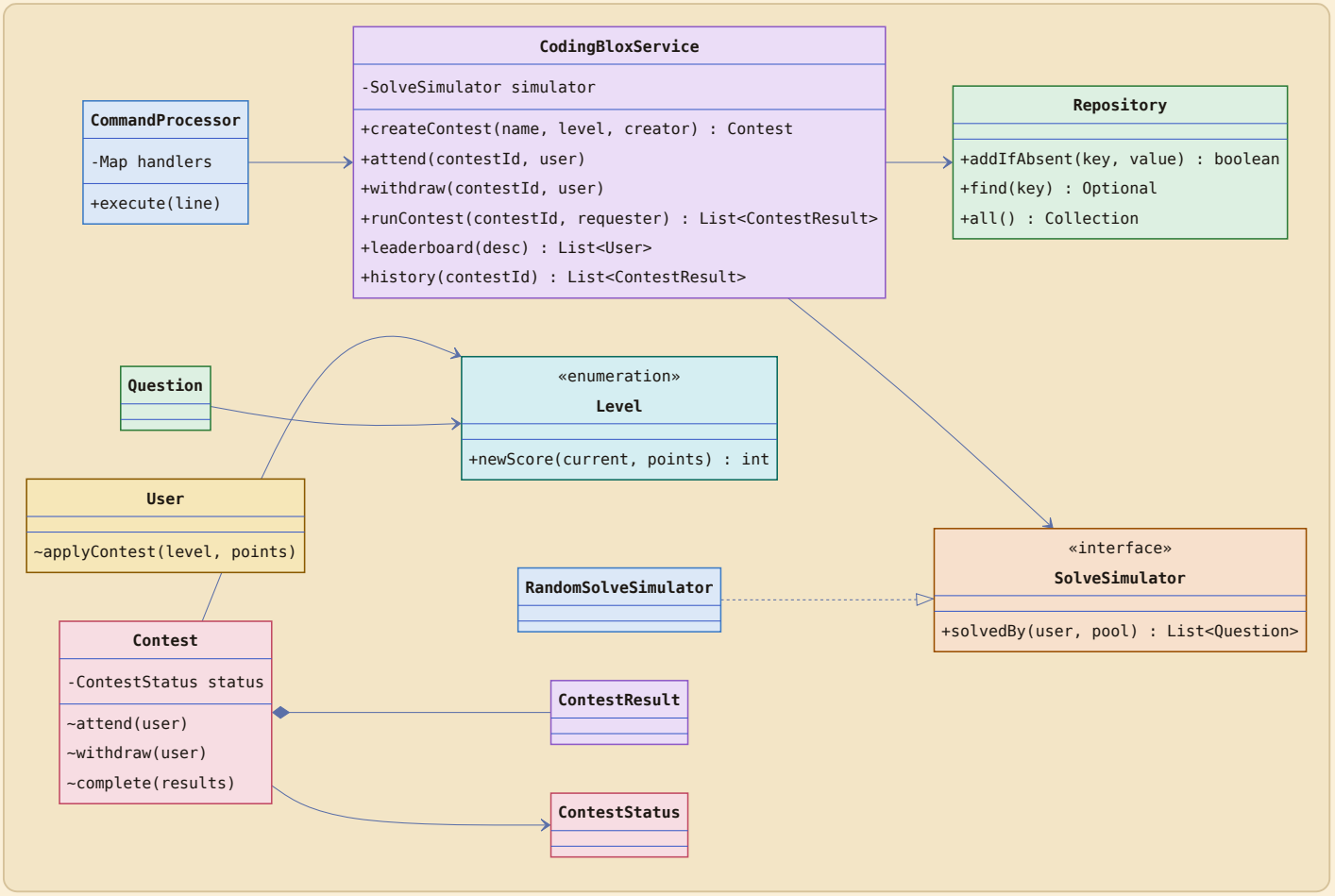
Use cases

User (sign up, attend/withdraw, leaderboard, history), **Contest creator** (create, run), **Admin / problem setter** (questions), **Judge** (simulated: who solved what).

Core entities

Class	Responsibility
`Level` (enum)	LOW/MEDIUM/HIGH with penalty; `newScore(current, points)` is the rating rule.
`User`	Name + score; `applyContest` updates atomically.
`Question`, `ContestResult` (records)	id/level/score; user/points/solved ids.
`Contest` + `ContestStatus`	Participants, OPEN/COMPLETED, results; guards "open only" rules.
`SolveSimulator` / `RandomSolveSimulator`	**Strategy** for "who solved what".
`Repository<K,V>`	Generic in-memory store reused three times.
`CodingBloxService`	Business rules per command.
`CommandProcessor`	Parses a line, dispatches via a map, prints.
`BloxException`	Readable failure.

Class diagram



Key flows

RunContest: requester must be the creator → pool = questions of the contest level → for each participant, `simulator.solvedBy` and sum points → `contest.complete(results)` (throws if already COMPLETED) → only then apply ratings. Contest lifecycle: **OPEN** (attend/withdraw allowed) → **COMPLETED** on `RunContest` by the creator.

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Command name → handler map	Command	New command = one `put`; parsing/printing out of the service.	Parsing in the service / giant `switch`.
Scoring rule on `Level`	Strategy (per-constant data)	A new level brings its own rule.	`if (level == LOW)…` in `runContest`.
`SolveSimulator` interface	Strategy	Seeded for demo, scripted for tests, real judge later.	`new Random()` in the service.
`Contest.complete` checks status	Entity guards invariant	"Run once" enforced where state lives.	Check in the driver.
Generic `Repository<K,V>`	Repository	Three stores, one class (DRY).	Three copy-pasted maps.
Results → `complete` → ratings	Safe ordering	If `complete` throws, no rating changed.	Updating ratings in the loop.

Java implementation

```java
// File: lld/ch21/codingblox/Level.java
public enum Level {
    LOW(50), MEDIUM(30), HIGH(0);

    private final int penalty;
    Level(int penalty) { this.penalty = penalty; }

    /** newScore = currentScore + (contestPoints - penalty), exactly as the statement defines. */
    public int newScore(int current, int contestPoints) { return current + contestPoints - penalty; }
    // ... parse(String)
}

// File: lld/ch21/codingblox/CodingBloxService.java
    public List<ContestResult> runContest(int contestId, String requester) {
        Contest c = contest(contestId);
        if (!c.creator().equals(requester)) throw new BloxException(requester + " did not create contest " +
contestId);
        List<Question> pool = listQuestions(c.level());
        List<ContestResult> results = new ArrayList<>();
        for (String name : c.participants()) {
            List<Question> solved = simulator.solvedBy(name, pool);
            int points = solved.stream().mapToInt(Question::score).sum();
            results.add(new ContestResult(name, points, solved.stream().map(Question::id).toList()));
        }
        c.complete(results);                              // throws if already run: scores never double-apply
        for (ContestResult r : results) user(r.user()).applyContest(c.level(), r.points());
        return results;
    }

// File: lld/ch21/codingblox/CommandProcessor.java
        handlers.put("runcontest", a -> svc.runContest(Integer.parseInt(arg(a, 1)), arg(a, 2))
                .forEach(r -> out("  " + r.user() + " solved " + r.solvedIds() + " -> " + r.points() + "
pts")));
        // ... one entry per command
```

Full runnable code: code/lld/ch21/codingblox/ (see the Full Edition).

Output (excerpt):

```
> RunContest 1 Ross
  Ross solved [2] -> 20 pts
  Monica solved [1, 2] -> 30 pts
  Joey solved [1, 2] -> 30 pts
> RunContest 1 Ross
ERROR: Contest 1 has already run
> LeaderBoard score desc
  Monica : 1540
  Chandler : 1510
  Joey : 1480
  Ross : 1470
```

Contest 1 is LOW: Ross 1500 + (20 − 50) = 1470; Joey/Monica 1480. Contest 2 is MEDIUM: Monica 1480 + (90 − 30) = 1540, Chandler 1500 + (40 − 30) = 1510.

Concurrency & edge cases

- `ConcurrentHashMap` repositories; `synchronized` `Contest` methods; `User.applyContest` under the user's lock, so two contests finishing together can't lose an update.
- **Known gap (say it):** a user attending between the participant copy and `complete` gets no result → add a `RUNNING` status switched atomically with handing back the participant list.
- Handled: duplicates, unknown ids/levels, attend/withdraw after the run, creator withdraw, non-creator run, bad args, unknown command.

Extensions & follow-up questions

- Fixed number of questions: a `QuestionSelectionStrategy` at creation. Real submissions: a judge + `Submission(user, question, verdict, time)`, ICPC-style scoring.
- Elo-style ratings: a `RatingStrategy` over all participants' results. Scheduled contests: `SCHEDULED → RUNNING → COMPLETED` + scheduler.
- Leaderboard at scale: `TreeSet` by (score, name) or a Redis sorted set updated on each change.

Real-world example

LeetCode weekly contests recompute each participant's rating from their relative rank after the contest; HackerRank and Codeforces share the shape: a problem pool, registration, ratings only after finalisation.

In the interview

Paste the sample commands first; build CreateUser → CreateQuestion → CreateContest → AttendContest → RunContest → LeaderBoard, running after each. Name the three moves: command map, scoring on the enum, seeded randomness behind an interface. Bonus commands are tiny because `Contest` stores results and guards status.

Self-check

- **Q:** Why update ratings only after `complete`? → **A:** A second run throws before any rating changes.
- **Q:** Add `VERY_HIGH` with +20 bonus? → **A:** An enum constant with its own rule (e.g. negative penalty).
- **Q:** Deterministic `RunContest` in tests? → **A:** Inject a scripted or seeded `SolveSimulator`.
- **Q:** Where is "creator always attends" enforced? → **A:** The `Contest` constructor.

TL;DR

Command map in the driver, rules in the service, invariants in `Contest`, scoring on `Level`, randomness behind a seeded `SolveSimulator`; complete first, then update ratings.

21.5 Design a Property Listing Site — "PropertyHunt" (ClearTrip)

Problem & clarifying questions

ClearTrip statement (kumaransg/LLD `PropertyHunt`; the longer screenshot version is followed). Users **register**, **list** properties to sell or rent, **view own listings**, **shortlist** others' properties, **view shortlisted** (sold ones show as SOLD), **search** available properties by any combination of **location** (case-insensitive), **price range**, **type** (sell/rent), **size range**, **rooms** (1/2/3BHK), **sort by** price or size; **mark sold** (owner only); **view sold**. Ranges `X-Y` are inclusive; a single `X` means "at least X" for size and "at most X" for price. **Bonus:** sq ft / sq m / sq yd; **nearby** locations shown last; prices like `40k`, `60L`, `2Cr`; comma-separated values for location and rooms.

Question	Answer used
Input format?	One command per line, fields split by `` ` ``
Shortlist my own property?	No (the text readme's sample does it — a conflict to ask about).
Shortlist a sold property?	No; but one sold *after* shortlisting stays, shown as SOLD.
Internal units?	Price in whole rupees (`long`), size in sq ft (`double`).
Ordering with a location?	Exact matches, then nearby, then the chosen sort.
Mark sold records buyer/price?	Status only; a `Sale` record is an extension.

The sample `bellandur` search returns a *sarjapur* property — consistent only because sarjapur declares `nearby bellandur|…` (the text readme drops that field).

Requirements

Functional: RegisterUser, ListProperty, Search, Shortlist, ViewShortlisted, ViewListed, MarkSold, ViewSold + all four bonus items. **Non-functional:** filters combine freely; new filters/units without rewriting search; clear errors; concurrent-safe.

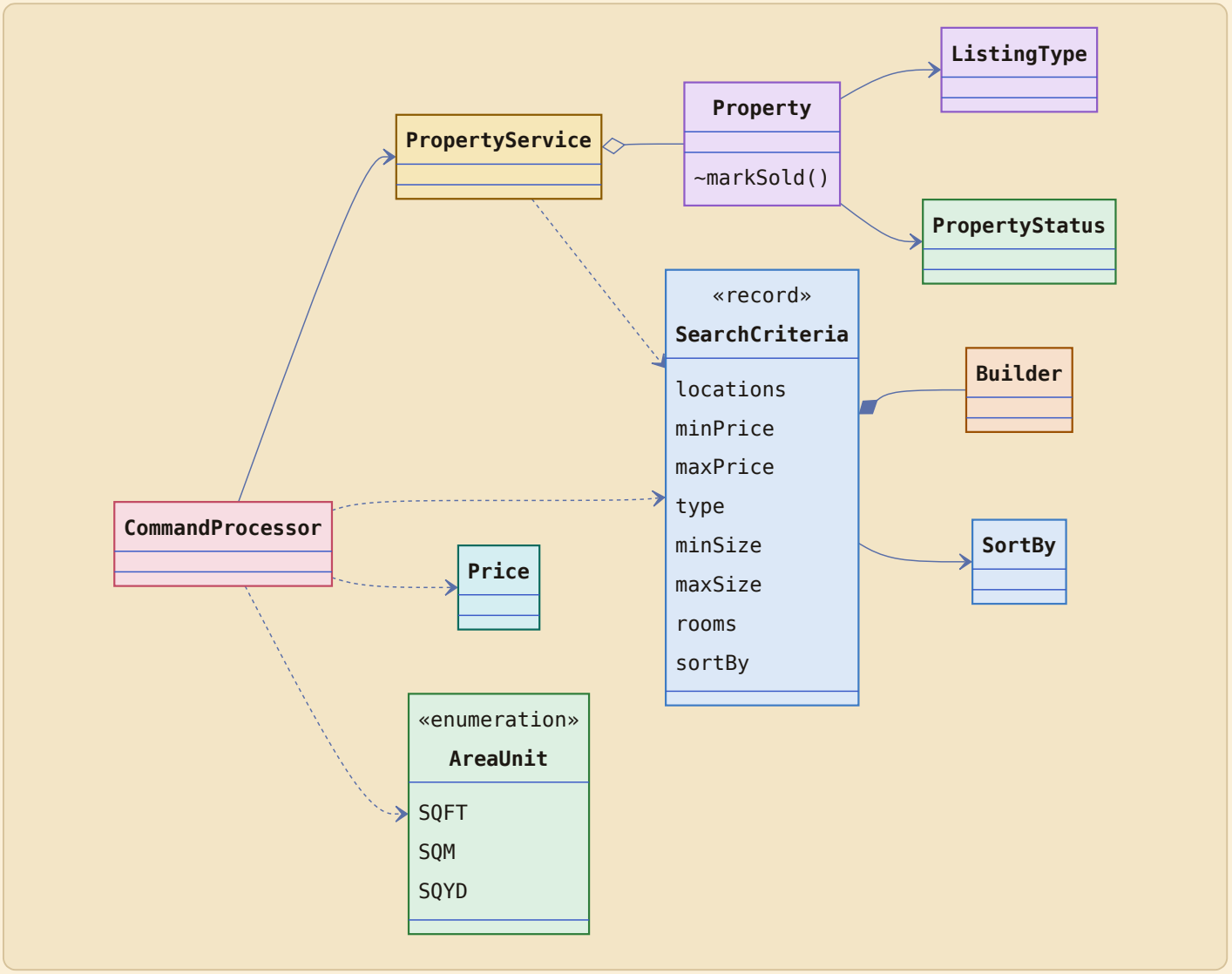
Use cases

Seller / landlord (list, view listed, mark sold, view sold), **Buyer / tenant** (search, shortlist, view shortlisted). Roles are per property, not per account.

Core entities

Class	Responsibility
`Property`	Listing data, owner, nearby locations, status; `markSold` guards double sale.
`ListingType`, `PropertyStatus`, `SortBy`	SELL/RENT, AVAILABLE/SOLD, sort comparators (**Strategy**).
`Price`	Parse `40k/60L/2Cr/1.5Cr` ↔ rupees.
`AreaUnit`	SQFT/SQM/SQYD → sq ft; parses "70 sqm".
`SearchCriteria` + `Builder`	All-optional search parameters (**Builder**).
`PropertyService`	Use cases; owner-vs-buyer rules.
`CommandProcessor`, `HuntException`	Parses `

Class diagram



Key flows

Search: parse fields → normalise with `Price.parse` / `AreaUnit.parseSize` → `SearchCriteria.Builder` → service builds a **list of predicates** (AVAILABLE, location or nearby, price, type, size, rooms) → reduce with `Predicate.and` → sort by (location rank, price or size, id) → driver prints the table. Property lifecycle: AVAILABLE → SOLD (owner only).

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
All-optional criteria with a Builder	Builder	Eight optional fields; eight nulls are unreadable.	Telescoping constructors.
Each criterion → `Predicate`, reduced with `and`	Specification	Any combination; new filter = one line.	`if` per combination.
`SortBy` enum of comparators	Strategy	"Sort by rooms" = one constant.	`switch` in search.
Normalise at the boundary (rupees, sq ft)	—	Business code compares plain numbers.	Comparing strings like "90L".
Location rank 0/1/2	—	One function filters *and* orders (exact, nearby, excluded).	Two searches merged.
Shortlist = set of ids per user	—	Status read live, so SOLD appears automatically.	Copied objects go stale.

Java implementation

```
// File: lld/ch21/propertyhunt/PropertyService.java
    public List<Property> search(String user, SearchCriteria c) {
        requireUser(user);
        Set<String> locs = c.locations() == null ? null : lower(c.locations());
        List<Predicate<Property>> filters = new ArrayList<>();
        filters.add(p -> p.status() == PropertyStatus.AVAILABLE);
        if (locs != null) filters.add(p -> locationRank(p, locs) < 2);
        if (c.minPrice() != null) filters.add(p -> p.price() >= c.minPrice());
        if (c.maxPrice() != null) filters.add(p -> p.price() <= c.maxPrice());
        if (c.type() != null) filters.add(p -> p.type() == c.type());
        if (c.minSize() != null) filters.add(p -> p.sizeSqft() >= c.minSize());
        if (c.maxSize() != null) filters.add(p -> p.sizeSqft() <= c.maxSize());
        if (c.rooms() != null) filters.add(p -> c.rooms().contains(p.rooms()));
        Predicate<Property> all = filters.stream().reduce(x -> true, Predicate::and);

        Comparator<Property> order = locs == null ? c.sortBy().comparator
                : Comparator.<Property>comparingInt(p -> locationRank(p,
locs)).thenComparing(c.sortBy().comparator);
        return properties.values().stream().filter(all).sorted(order).toList();
    }
    // ... shortlist(): rejects own, sold and duplicate properties; stores only the id
```

Full runnable code: code/lld/ch21/propertyhunt/ (see the Full Edition).

Output (excerpt):

```
-> Search|Jack|bellandur|null|null|null|null|price
<- 1   A 3BHK for sale      bellandur  90L    1800sqft  3BHK  Sale
<- 2   A 2BHK for rent      sarjapur   40k    1500sqft  2BHK  Rent
-> ViewShortlisted|Jill
<- 1   A 3BHK for sale      bellandur  90L    1800sqft  3BHK  Sale  SOLD
<- 2   A 2BHK for rent      sarjapur   40k    1500sqft  2BHK  Rent  AVAILABLE
-> ListProperty|Jill|Bad|x|abc|sell|100 sqft|1
ERROR: Bad price: abc
```

Nearby sarjapur comes after exact bellandur; a sold property leaves searches but stays (SOLD) in shortlists.

Concurrency & edge cases

- `ConcurrentHashMap`s, concurrent shortlist sets, `synchronized` `markSold` (no double sale).
- A search racing a sale may still show it — fine for listings; a purchase flow re-checks under a lock.
- Handled: unregistered user, unknown property, own/duplicate/sold shortlist, non-owner mark sold, bad price/size/unit/type.

Extensions & follow-up questions

- Descending / multi-key sort: direction flag + `.reversed()`, or chained `thenComparing`.
- Fuzzy locations: a `LocationResolver`. "Within 5 km": lat/long + haversine; at scale a geo-index (geohash, R-tree) or a search engine.
- Millions of listings: secondary indexes or Elasticsearch/OpenSearch. Price-drop alerts: Observer on `Property`.

Real-world example

99acres, MagicBricks and NoBroker filter by locality, budget, BHK, buy/rent and area, suggest nearby localities, and quote prices in lakhs and crores — hence parsing `60L` and `2Cr`.

In the interview

Start with model and units (`Price.parse("90L")`, `AreaUnit`), then register → list → search with *one* filter → add predicates → sort. Call out the Builder + predicate list as the extensibility story and "normalise at the boundary".

Self-check

- **Q:** Buyer searches in sq m, seller listed in sq yd? → **A:** Both converted to sq ft at parse time.
- **Q:** Why shortlist ids, not objects? → **A:** Status is read live.
- **Q:** A single number in the price field means? → **A:** "At most X" (size: "at least X").
- **Q:** Add a "furnished" filter? → **A:** A `Property` field, an optional criterion, one predicate line.

TL;DR

Normalise price and size at the edges, turn each optional criterion into a predicate and AND them, sort by (location rank, key, id), keep shortlists as ids.

21.6 Design a Loan Ledger — "Ledger Co" (Navi)

Problem & clarifying questions

The Geektrust "Ledger Co" challenge (kumaransg/LLD `ledger_company_navi`, used by Navi). Banks lend; borrowers repay in **EMIs** (Equated Monthly Instalments, same amount each month) and may pay a **lump sum** any time.

Command	Meaning
`LOAN BANK BORROWER P N R`	Principal `P`, `N` years, `R` % simple interest per year.
`PAYMENT BANK BORROWER LUMP_SUM EMI_NO`	Lump sum paid right after EMI number `EMI_NO`.
`BALANCE BANK BORROWER EMI_NO`	Print `BANK BORROWER AMOUNT_PAID EMIS_LEFT` after EMI `EMI_NO`.

Rules: `I = P × N × R / 100`; `A = P + I`; EMI = `A / (N × 12)` **rounded up** (86.67 → 87, 100.10 → 101); paid after EMI k = `k × EMI` + every lump sum with EMI number ≤ k; EMIs left = remaining ÷ EMI **rounded up**; the last EMI shrinks to what is owed. A lump sum reduces the *number* of EMIs, never the amount. Only `BALANCE` prints.

Question	Answer used
Paid can exceed `A`?	No — capped at `A` (EMIs left = 0).
Whole-number interest?	Assumed rounded up (say so); official samples are whole either way.
One loan per bank-borrower?	Yes; a second `LOAN` is an error.
`EMI_NO` beyond tenure?	Error; `0` is valid.

Requirements

Functional: LOAN, PAYMENT, BALANCE as above; reproduce the official samples. **Non-functional:** input from a list/file; a bad line doesn't stop the rest; interest rule replaceable; exact `BigDecimal` arithmetic.

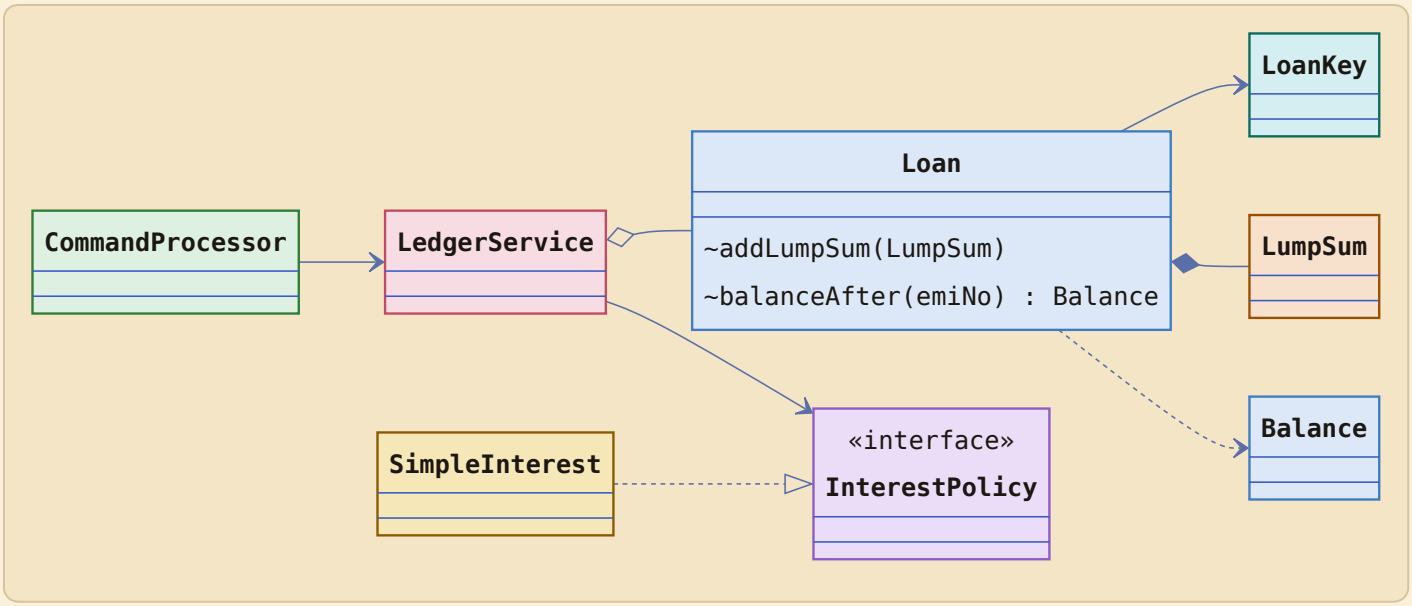
Use cases

Bank (records a loan), **Borrower** (pays lump sums), **Anyone** (balance after EMI k).

Core entities

Class	Responsibility
`LoanKey` (record)	(bank, borrower) — a perfect map key (records give `equals/hashCode`).
`Loan`	Computes `A` and EMI once; stores lump sums; derives a `Balance` for any EMI.
`LumpSum`, `Balance` (records)	amount + after-EMI; output line.
`InterestPolicy` / `SimpleInterest`	**Strategy** for total interest.
`LedgerService`, `CommandProcessor`, `LedgerException`	Loans store; parsing and validation; readable errors.

Class diagram



Key flows

Worked example (Sample 2, Shelly): P = 15000, N = 2, R = 9 → I = 2700, A = 17700, EMI = ⌈737.5⌉ = 738. After 12 EMIs + a 7000 lump sum: paid 15856, remaining 1844, EMIs left ⌈2.499⌉ = 3 → `UON Shelly 15856 3`.

Design decisions & patterns used

Decision	Why	Alternative rejected
Derive balance from facts (A, EMI, lump sums) per query	Queries may ask about *any* EMI, even before a later payment.	A mutable `remaining` field — wrong for earlier EMIs.
`InterestPolicy` Strategy	Reducing-balance or compound later without touching `Loan`.	Formula hard-coded.
`LoanKey` record as key	Value semantics for free.	`"IDIDI#Dale"` strings.
`BigDecimal` + `RoundingMode.CEILING`	Exact "always up".	`Math.ceil` on doubles (4.000000001 → 5).
Driver returns `List<String>`	Easy to compare with expected output.	Printing in the service.

Java implementation

```java
// File: lld/ch21/ledger/Loan.java
    Loan(LoanKey key, BigDecimal principal, int years, BigDecimal rate, InterestPolicy policy) {
        if (principal.signum() <= 0 || years <= 0 || rate.signum() < 0)
            throw new LedgerException("Invalid loan terms for " + key);
        this.key = key;
        this.tenureMonths = years * 12;
        this.totalRepayable = principal.add(policy.interest(principal, BigDecimal.valueOf(years), rate));
        this.emi = totalRepayable.divide(BigDecimal.valueOf(tenureMonths), 0, RoundingMode.CEILING);
    }
    // ... addLumpSum validates amount > 0 and EMI number in range

    /** Amount paid after `emiNo` EMIs (lump sums made at or before that EMI count), capped at A. */
    synchronized Balance balanceAfter(int emiNo) {
        if (emiNo < 0 || emiNo > tenureMonths) throw new LedgerException("EMI number out of range: " + emiNo);
        BigDecimal lumps = lumpSums.stream().filter(l -> l.afterEmi() <= emiNo)
                .map(LumpSum::amount).reduce(BigDecimal.ZERO, BigDecimal::add);
        BigDecimal paid = emi.multiply(BigDecimal.valueOf(emiNo)).add(lumps).min(totalRepayable);
        BigDecimal remaining = totalRepayable.subtract(paid);
        int left = remaining.divide(emi, 0, RoundingMode.CEILING).intValueExact();
        return new Balance(key.bank(), key.borrower(), paid, left);
    }
```

Full runnable code: code/lld/ch21/ledger/ (see the Full Edition).

Output (excerpt) — the Sample 1 and 2 blocks match the official Geektrust expected outputs:

```
== Sample 2
IDIDI Dale 1326 9
IDIDI Dale 3652 4
UON Shelly 15856 3
MBI Harry 9044 10
== Edge cases
SBI Asha 1120 0
ERROR EMI number out of range: 13  [BALANCE SBI Asha 13]
ERROR Not a number: ten  [PAYMENT SBI Asha ten 1]
```

Asha's 2000 lump sum overpays a 1120 loan, so paid is capped at 1120 and 0 EMIs remain.

Concurrency & edge cases

- `ConcurrentHashMap` + `putIfAbsent` makes "one loan per pair" atomic; `synchronized` `Loan` methods keep PAYMENT and BALANCE apart.
- Handled: duplicate/unknown loan, EMI out of range, invalid terms, overpayment (capped), bad arguments, unknown command.

Extensions & follow-up questions

- **Reducing-balance EMI:** new policy, `EMI = P·r·(1+r)^n / ((1+r)^n − 1)`; `Loan` API unchanged.
- **Lump sum that reduces the EMI instead:** a `PrepaymentPolicy` (REDUCE_TENURE / REDUCE_EMI).
- **Late payments:** record dated payment events and derive EMIs paid. **File input:** `Files.readAllLines` into the same `run`.

Real-world example

Navi, banks and NBFCs show an EMI schedule and allow part-prepayment, usually choosing between fewer EMIs (this puzzle's rule) or a smaller EMI. Real loans use reducing-balance interest, not flat simple interest — worth raising.

In the interview

Show the three formulas and compute one sample by hand (proves you understood the rounding); code `Loan.balanceAfter` (the heart), the service, the command loop; run both samples. Emphasise "state = facts,

balances = derived".

Self-check

- **Q:** Why ceil the EMI? → **A:** A floor leaves a shortfall; rounding up guarantees the EMIs cover `A`.
- **Q:** Does `BALANCE … 5` include a PAYMENT after EMI 10? → **A:** No; only `afterEmi ≤ 5`.
- **Q:** Why compute the balance rather than store it? → **A:** Queries arrive for any EMI in any order.
- **Q:** Compound interest? → **A:** A new `InterestPolicy` injected into `LedgerService`.

TL;DR

EMI = ⌈(P + PNR/100) / 12N⌉. Store loan facts and lump sums; derive paid and EMIs-left on demand with ceiling rounding; interest is a Strategy; the driver is a tiny command loop.

21.7 Design an Inventory Management System

Problem & clarifying questions

A common e-commerce / quick-commerce question: track **SKUs** (stock keeping units — one id per sellable item) across **warehouses**, record every **stock movement** (received, shipped, transferred, adjusted after a physical count), fulfil orders by choosing warehouses, and raise **reorder alerts**.

Question	Answer used
Stock per warehouse?	Yes: (SKU, warehouse) → quantity.
Split orders?	Yes; warehouse order is a strategy (nearest city first / most stock first).
Partial shipments?	No — all-or-nothing.
Reorder alert when?	*Total* stock ≤ reorder point; once per dip, re-armed on recovery.
Reorder how much?	`max − current`.
Negative stock?	Never.

Requirements

Functional: add products (reorder point, max level) and warehouses; receive; ship by strategy; transfer; cycle-count adjust; stock per warehouse and total; SKU movement history; reorder alerts. **Non-functional: no overselling** under concurrency; different products in parallel; full audit trail.

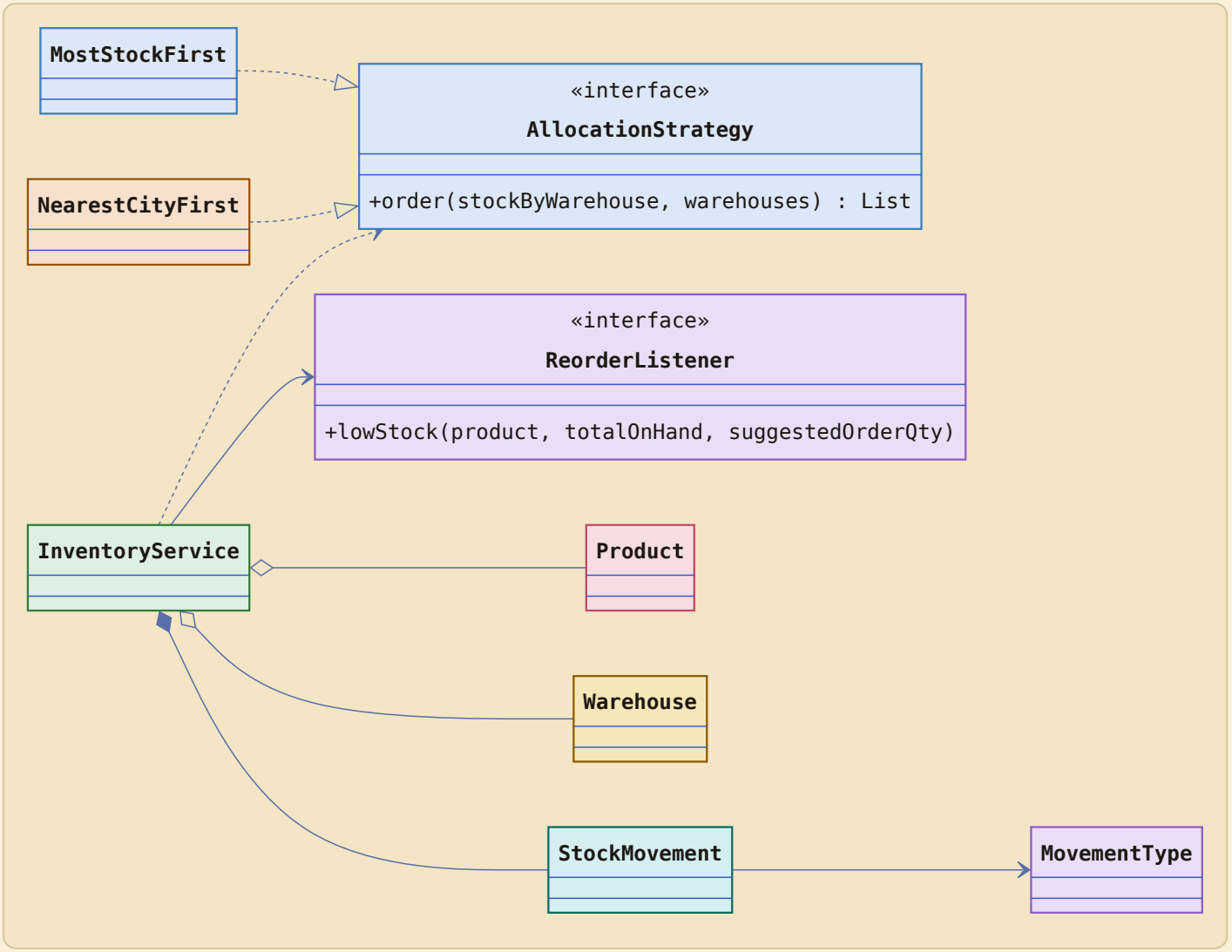
Use cases

Warehouse staff (receive, transfer, adjust), **Order service** (ship), **Purchasing** (reorder alerts), **Manager** (stock and history).

Core entities

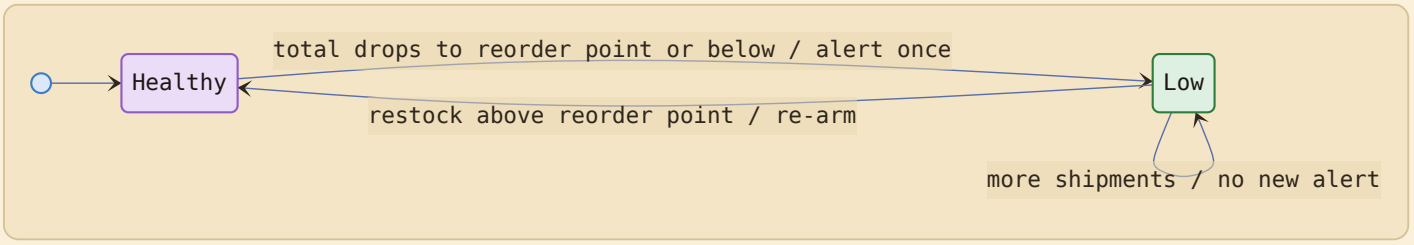
Class	Responsibility
`Product`, `Warehouse` (records)	SKU, name, reorder point, max level; id, city.
`StockMovement` + `MovementType`	Append-only ledger line: INBOUND, OUTBOUND, TRANSFER, ADJUSTMENT.
`Allocation` (record)	"take q from warehouse w".
`AllocationStrategy` + `NearestCityFirst`, `MostStockFirst`	**Strategy**: order to drain warehouses.
`ReorderListener`	**Observer** for low stock.
`InventoryService`, `InventoryException`	Levels, per-SKU locks, movements, alerts; readable errors.

Class diagram



Key flows

Ship: lock the SKU → total < qty? reject, nothing changed → strategy orders a *snapshot* of the levels → build the full plan → apply it, writing OUTBOUND movements → `checkReorder` → unlock.



Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
One lock per SKU	Lock striping by key	Covers all of a SKU's warehouses with no lock-ordering puzzle; different SKUs never contend.	Lock per (SKU, warehouse) — needs ordering; one global lock.
Plan, then apply	—	Check total and build the full plan before changing anything → all-or-nothing.	Deduct and roll back.
`AllocationStrategy`	Strategy	Speed vs fewer split parcels as small classes.	`if (mode == …)`.
Append-only movement ledger	Event log / audit	Every change explainable; levels rebuildable; adjustments record the delta.	Quantities with no history.
Once-per-dip `ReorderListener`	Observer + edge-triggering	Notified once on crossing, not every sale.	Alert on every movement (spam).
Strategy gets a snapshot	Defensive copy	Can't modify live stock.	Passing the live map.

Java implementation

```
// File: lld/ch21/inventory/InventoryService.java
    /** All-or-nothing: either the whole quantity is allocated across warehouses, or nothing moves. */
    public List<Allocation> ship(String sku, int qty, AllocationStrategy strategy, String orderRef) {
        positive(qty);
        return withSku(sku, () -> {
            Map<String, Integer> levels = stock.get(sku);
            int total = levels.values().stream().mapToInt(Integer::intValue).sum();
            if (total < qty) throw new InventoryException("Only " + total + " of " + sku + " available, order
needs " + qty);
            List<Allocation> plan = new ArrayList<>();
            int remaining = qty;
            for (String wh : strategy.order(Map.copyOf(levels), warehouses)) {
                int take = Math.min(remaining, levels.get(wh));
                if (take > 0) { plan.add(new Allocation(wh, take)); remaining -= take; }
                if (remaining == 0) break;
            }
            for (Allocation a : plan) {
                change(sku, a.warehouseId(), -a.quantity());
                record(MovementType.OUTBOUND, sku, a.warehouseId(), null, -a.quantity(), orderRef);
            }
            return plan;
        });
    }
    // ...
    /** Fires once when stock crosses down to the reorder point; re-arms when stock recovers. */
    private void checkReorder(String sku) {
        Product p = products.get(sku);
        int total = totalUnlocked(sku);
        if (total <= p.reorderPoint()) {
            if (alerted.add(sku)) listeners.forEach(l -> l.lowStock(p, total, p.maxLevel() - total));
        } else {
            alerted.remove(sku);
        }
    }
    // ... withSku(sku, action): look up the SKU's ReentrantLock, lock, run, unlock
```

Full runnable code: code/lld/ch21/inventory/ (see the Full Edition).

Output (excerpt):

```
== ship 32 phones to a Bengaluru customer (nearest city first)
  [Allocation[warehouseId=BLR-1, quantity=30], Allocation[warehouseId=BLR-2, quantity=2]]
== ship 25 more (most stock first) -> crosses reorder point
  [ALERT] PHONE low: 18 left, reorder 82 to reach 100
  ship 1 more (no duplicate alert): [Allocation[warehouseId=BLR-1, quantity=1]]
== concurrency: 200 one-unit orders race for 50 cases
  shipped=50 rejected=150 left=0
```

The alert fires once at 18 (≤ 20), not again at 17; exactly 50 of 200 concurrent orders ship — no overselling.

Concurrency & edge cases

- Check → plan → apply under the SKU's `ReentrantLock`; different SKUs in parallel.
- **Ledger** = `ConcurrentLinkedQueue` (lock-free appends, order kept, no `ConcurrentModificationException`); a `CopyOnWriteArrayList` would copy the whole log on every movement. **Listeners** = `CopyOnWriteArrayList` (read often, changed rarely); they run under the SKU lock — hand slow ones (email) to an executor.
- Handled: unknown SKU/warehouse, bad quantities, same-warehouse transfer, orders larger than stock, zero-change adjustments (not logged).

Extensions & follow-up questions

- **Reservations (cart holds):** `onHand` and `reserved`; available = onHand − reserved; holds expire; shipping converts a reservation.
- **Batches / expiry:** FEFO (first-expiry-first-out) is just another `AllocationStrategy`.
- **Auto purchase orders:** a listener creating a pending PO for `max − current`.
- **Database / many services:** optimistic updates (`UPDATE ... SET qty = qty - ? WHERE qty >= ? AND version = ?`); ledger as an event stream (Kafka).

Real-world example

Amazon and Flipkart choose the fulfilment centre by proximity and stock and may split an order (`AllocationStrategy`); Shopify has multi-location fulfilment priorities; quick-commerce dark stores (Blinkit, Zepto, Instamart) depend on accurate per-store stock and fast reorder alerts.

In the interview

Start with the data shape: `stock[sku][warehouse] = qty` + an append-only movement list. Code receive and ship (plan-then-apply, per-SKU lock) first — most of the marks; then transfer/adjust, the once-per-dip observer, allocation strategies; run the 200-thread demo.

Self-check

- **Q:** Why check the total before planning? → **A:** Reject short orders before touching any warehouse.
- **Q:** Why is a per-SKU lock enough for multi-warehouse orders? → **A:** An order has one SKU; its lock covers all its warehouses.
- **Q:** Add FEFO? → **A:** Batch-level stock + a new `AllocationStrategy`; `ship` unchanged.
- **Q:** Why log adjustments as movements? → **A:** Shrinkage is auditable and levels rebuildable.

TL;DR

Stock per (SKU, warehouse), every change a ledger movement, a lock per SKU prevents overselling, allocation is a Strategy, low-stock alerts are an edge-triggered Observer.

21.8 Design a Unix File Search (`find`)

Problem & clarifying questions

Build a small `find` (e.g. `find /var/log -type f -size +10M -mtime +7`): a file-system model, **filters**, a way to **combine** them with AND / OR / NOT, and a clean API for search options. The trap to avoid: one method with ten

optional parameters. Terms: a **predicate** is a yes/no test on one entry; a **glob** is a wildcard name pattern (`*`, `?`, `[0-9]`); **depth** = levels below the start (start = 0).

Question	Answer used
Real disk?	In-memory tree (deterministic); `java.nio` adapter is an extension.
Tests?	name / iname glob, type, size, mtime, owner, empty.
Combining?	AND, OR, NOT, parentheses; precedence NOT > AND > OR; side-by-side tests = AND.
Size semantics?	GNU: round **up** to the unit, then compare — `-size -1M` matches only empty files. Files only.
Options?	min/max depth, limit, **prune** (skip e.g. `node_modules`).
API?	Fluent **Builder** + a `find` command-line parser.
Order / huge trees?	Pre-order DFS, children by name; stream results; no recursion; parallel variant.

Requirements

Functional: model files/dirs (name, owner, mtime, size); search with any AND/OR/NOT combination; depth, limit, prune; parse `find` lines; clear errors. **Non-functional:** Open/Closed (new test = one class); streaming, deep trees; searching while files are added must not throw.

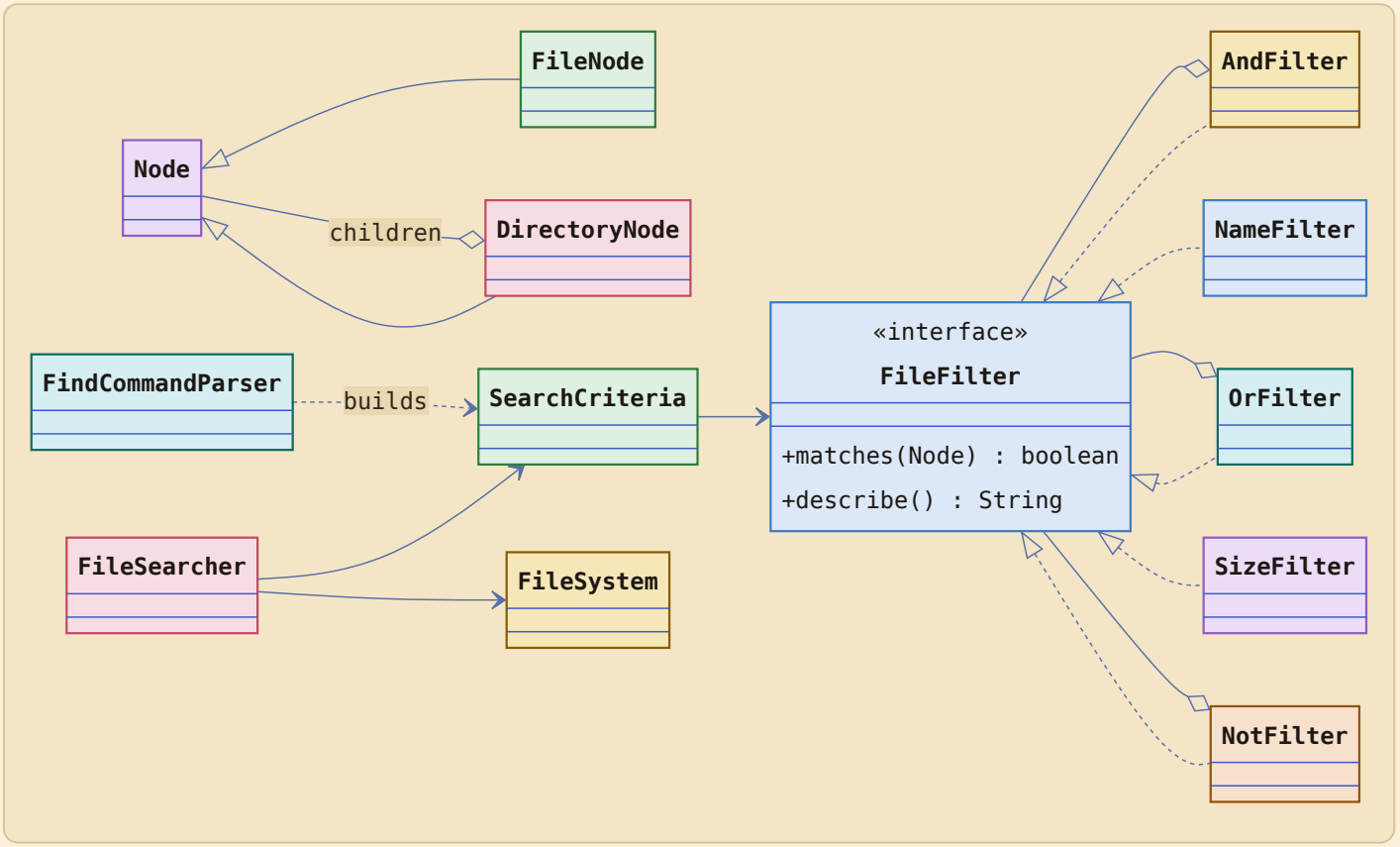
Use cases

Developer (Builder, search, stream), **Shell user** (type a command, get paths or an error), **Ops script** (big old logs, empty directories).

Core entities

Class	Responsibility
`Node` (sealed) → `FileNode`, `DirectoryNode`	**Composite** file tree; `path()`, `size()`, `totalSize()`; children in a sorted `ConcurrentSkipListMap`.
`FileSystem`	Root, `mkdirs`, `createFile`, `resolve`.
`FileFilter`	**Specification**: `matches`, `describe`, default `and` / `or` / `negate`.
`AndFilter`, `OrFilter`, `NotFilter`	**Composite** filter tree.
`Filters`	Leaf tests (name, iname, type, size, mtime, owner, empty, all).
`SearchCriteria` + `Builder`	Immutable, validated search description.
`FileSearcher`	Iterative DFS, streams to a `Consumer`; fork/join variant.
`FindCommandParser`	Recursive descent → `SearchCriteria`.

Class diagram



Two Composites: the **file tree** and the **filter tree**; the searcher walks one and asks the other one question per entry.

Key flows

`find /home ( -name '*.jpg' -o -name '*.png' ) ! -size +1M` parses to **AND(OR(name *.jpg, name *.png), NOT(size > 1M))**.

Search loop: push (start, 0) → pop → if depth ≥ minDepth and filter matches, emit → if directory, depth < maxDepth and not pruned, push children in reverse name order → repeat until empty or limit.

Precedence: `-o` < AND (`-a` or nothing) < `!` < `( )`. So `-name a -o -name b -type f` = `a OR (b AND file)`.

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
One small class per test	**Specification**	New test = one record; loop unchanged	`search(name, ext, minSize, …)` with `if`s
And/Or/Not hold filters	**Composite**	Arbitrary nesting; printable, inspectable, parser output	Lambdas only (can't print; cf. §21.1)
`Node` / `FileNode` / `DirectoryNode`	**Composite**	Uniform walk and `totalSize()`	`instanceof` everywhere
`SearchCriteria.Builder`	**Builder**	Named optional settings, validated once, immutable	Telescoping constructors
Recursive-descent parser	Interpreter-style	One method per precedence level	String splitting
Explicit stack, `Consumer` output	Streaming	No stack overflow; constant memory	Recursion; always a full list
Short-circuit AND/OR	—	Cheap tests first	Evaluate every test
`ConcurrentSkipListMap` children	Concurrent collection	Sorted, safe to iterate during writes	`TreeMap` (CME)

Java implementation

```
// File: lld/ch21/filesearch/FileFilter.java
public interface FileFilter {
    boolean matches(Node n);
    String describe();
    default FileFilter and(FileFilter other) { return AndFilter.of(this, other); }
    default FileFilter or(FileFilter other) { return OrFilter.of(this, other); }
    default FileFilter negate() { return new NotFilter(this); }
}
```

```
// File: lld/ch21/filesearch/Filters.java  (SizeFilter.matches)
            if (node.type() != FileType.FILE) return false;
            long units = (node.size() + unitBytes() - 1) / unitBytes();      // round up
            return switch (cmp) { case '+' -> units > n; case '-' -> units < n; default -> units == n; };
```

```
// File: lld/ch21/filesearch/FileSearcher.java
    public int search(SearchCriteria c, Consumer<Node> out) {
        Deque<Frame> stack = new ArrayDeque<>();
        stack.push(new Frame(fs.resolve(c.startPath()), 0));
        int found = 0;
        while (!stack.isEmpty() && found < c.limit()) {
            Frame f = stack.pop();
            Node n = f.node();
            if (f.depth() >= c.minDepth() && c.filter().matches(n)) { out.accept(n); found++; }
            if (n instanceof DirectoryNode d && f.depth() < c.maxDepth() &&
!c.pruneNames().contains(d.name())) {
                List<Node> kids = new ArrayList<>(d.children());
                Collections.reverse(kids);                    // push in reverse so we pop in name order
                for (Node k : kids) stack.push(new Frame(k, f.depth() + 1));
            }
        }
        return found;
    }
```

```
// File: lld/ch21/filesearch/FindCommandParser.java
    private FileFilter parseOr() {
        FileFilter left = parseAnd();
        while (peekIs("-o", "-or")) { pos++; left = left.or(parseAnd()); }
        return left;
    }
    // parseAnd: unary ( [-a] unary )* ; parseUnary: ! unary | ( expr ) | -name G | -size N | ...
```

Full runnable code: code/lld/ch21/filesearch/ (see the Full Edition).

How it works: globs compile once to a regex on the last path component; `-size` accepts `[+-]N[cbkMG]` (no suffix = 512-byte blocks); `-mtime` counts whole days against an injected `now`; each Builder `where` is ANDed; the parser skips `-maxdepth`/`-mindepth` at every level; `searchParallel` forks one task per sub-directory and joins in name order — identical results.

Output (excerpt):

```
$ find /home ( -name '*.jpg' -o -name '*.png' ) ! -size +1M
  parsed: from /home where ((name='*.jpg' OR name='*.png') AND NOT size+1M)
   /home/asha/photos/c.png
   /home/asha/photos/d.jpg
$ find /home -size -1M
  parsed: from /home where size-1M
   /home/ravi/todo.md
$ from /home/asha/projects where (name='*.java' OR name='*.js') prune=[node_modules]
   /home/asha/projects/app/Main.java
   /home/asha/projects/app/Util.java
   find / ( -name a                  -> Missing ')'
   find / -maxdepth 1 -mindepth 3    -> mindepth 3 > maxdepth 1
   sequential=1600 parallel=1600 same order=true first=/data/d000/sub0/f060.csv last=/data/d199/sub4/f099.csv
```

`-size -1M` found only the empty file (the rounding gotcha); pruning skipped `node_modules`; 20,000 generated files gave the same 1,600 matches sequentially and in parallel.

Concurrency & edge cases

- Weakly consistent `ConcurrentSkipListMap` iterators: no exception when files are added mid-search.
- Parallel walk only reads; each task builds its own list — no locks.
- Real disks: don't follow symlinks by default (GNU `find` needs `-L`), or track visited inodes.
- Handled: missing start path, `mindepth > maxdepth`, unknown predicate, missing argument, bad `-size`/`-mtime`/`-type`, unbalanced `(`, unclosed `[` in a glob, limit mid-walk.

Extensions & follow-up questions

- **Real disk** → adapter over `java.nio` / `Files.walkFileTree` (`SKIP_SUBTREE` for prune).
- **Actions** (`-delete`, `-exec`) → a `Consumer` per action; delete in post-order.
- **Content search** → `ContainsTextFilter`, placed last in the AND.
- **Repeated searches** → an index like `locate`, possibly stale.
- **More tests** → one record + one parser `case`.

Real-world example

GNU findutils' `find` uses this precedence, short-circuits, rounds `-size` up and does not follow symlinks by default; Java's `Files.find(start, maxDepth, predicate)` splits walking from testing the same way; IDE "Find in files" builds the same filter tree from checkboxes.

In the interview

First 10 minutes: `Node` model + `FileFilter` with two leaves → demo. Then And/Or/Not, the Builder with depths, the iterative walk; the parser last. Say it: "a new filter is one class; the searcher never changes."

Self-check

- **Q:** `-name a -o -name b -type f`? → **A:** `a OR (b AND file)`.
- **Q:** Why does `-size -1M` match only empty files? → **A:** Rounded up, any non-empty file is ≥ 1 MiB unit.
- **Q:** Prune vs filtering a directory out? → **A:** Prune never enters it; a filter only hides the entry.
- **Q:** Why records instead of lambdas for And/Or/Not? → **A:** Printable, inspectable, what the parser builds.

TL;DR

Two Composites (files, filters); Specification leaves combined by AND/OR/NOT; a Builder for criteria; an iterative streaming DFS; a recursive-descent parser with `!` > AND > OR and GNU's rounded-up `-size`.

Chapter summary

Problem	Core entities	Key pattern(s)	Concurrency point	Trap to avoid
Task Management	Task, TaskStatus, TaskService	Specification filters, enum state machine, Observer	per-task `synchronized`	TODO → DONE jumps; filters as many parameters
Digital Wallet	Wallet, Transaction, Offer	Strategy offers, idempotency, funded cashback	lock both wallets in id order	`double` money; check-then-act on keys; cashback from nothing
Event Calendar	TimeSlot, Event, EventStore	Strategy for representatives, plan-then-save	atomic check-and-book	closed intervals; trusting the sample blindly
Coding Blox	Contest, Level, User	Command map, Strategy (solver, scoring)	atomic rating update	unseeded randomness; running a contest twice
PropertyHunt	Property, SearchCriteria	Builder + predicate list, Strategy sort	synchronized mark-sold	comparing "90L" strings; stale shortlist copies
Ledger Co	Loan, LumpSum, Balance	Strategy interest, derived state	synchronized lump sums	storing a running balance; floor instead of ceil
Inventory	StockMovement, AllocationStrategy	Strategy allocation, Observer alerts	per-SKU lock	partial shipments; alert spam
Unix find	Node (Composite), FileFilter, SearchCriteria	Specification + Composite filters, Builder, recursive-descent parser	weakly consistent children map; read-only parallel walk	parameter-list search methods; `-o` vs AND precedence

The pattern behind all eight: entities guard their own rules, services orchestrate, strategies hold the "X or Y" rules, repositories hide storage, the driver parses and prints, and every error is a clear message — not a crash.

Chapter 22 — Revision: Cheat Sheets & Glossary

> **What this chapter is.** The whole handbook on a few pages: the answer framework (Ch 1); OOP, relationships and UML (Ch 2–3); SOLID, other principles and clean code (Ch 4–6); all 24 smells (Ch 7); all 23 GoF patterns + extras and a "which pattern?" table (Ch 8–11); the concurrency toolbox (Ch 12–13); a map of every case study (Ch 14–21); machine-coding and edge-case checklists; and a glossary. Nothing here is new and there is no code — each row tells you where the full explanation lives.

How to read the references. "Ch 12 · Deadlock" means the *Deadlock* topic in Chapter 12; "Ch 16 · Car Rental" means the Car Rental case study in Chapter 16. When a row or heading already names the topic, only the chapter is given ("Ch 2"). Numbers such as "22.7" refer to sections of this chapter.

22.1 How to revise

In 15 minutes (the morning of the interview)

Minutes	Read	Why
0–3	22.2 (the framework)	You will use it in the first minute of the interview
3–6	22.4 (SOLID one-pager) and the relationship table in 22.3	The two things asked about most often by name
6–10	"Which pattern?" table in 22.7	Patterns should come from a signal, not from memory
10–13	Concurrency cheat sheet and deadlock checklist in 22.8	Every booking/inventory problem has a race
13–15	The row(s) of 22.9 for the problems you expect	One sentence per problem: entities, pattern, the atomic step

The night before (about 2 hours)

1. **Read 22.2–22.8 slowly** (45 min), saying *one handbook example* per row; if you can't, open the referenced topic.
2. **Three unpractised case studies** from 22.9 (45 min): requirements, classes, class diagram and the one hard method, without looking; then compare with the chapter's TL;DR.
3. **Scan the glossary** (22.11); read up only the terms you can't explain in one sentence (20 min).
4. **Stop.** Sleep matters more than one more pattern.

Common mistakes when revising

- Re-reading chapters instead of **recalling** first and then checking. Recall is what the interview tests.
- Memorising all 23 patterns equally. Six come up in almost every round (22.7): Strategy, Factory, Observer, State, Singleton, Builder.
- Skipping the "when *not* to use it" column. Interviewers ask "would you always do that?" (Ch 4 · SOLID in interviews).

22.2 The LLD interview answer framework (one page)

Source: Chapter 1 (how the interview runs, the answer framework, requirements, classes, SOLID check, patterns, interview code, communication tips, common mistakes).

> **TL;DR**

Clarify → requirements → use cases → entities → relationships → class diagram → key flows → patterns/SOLID → code → edge cases/concurrency → extensions. Say each step out loud as you enter it.

The eleven steps

#	Step	Ask yourself	Output
1	Clarify	Who uses it? Single site or many?	Shared understanding
2	Requirements	Functional? Non-functional (thread-safe, extensible, in-memory)? **Out of scope?**	8–12 numbered lines + out-of-scope list
3	Use cases	What can each actor do?	Actor → use-case list
4	Entities	Which nouns hold state or behaviour? Which are just values?	Classes, one sentence each
5	Relationships	Uses / knows / has / owns / is-a? How many?	Typed lines with multiplicity
6	Class diagram	Fields, key methods, interfaces, enums	UML class diagram
7	Key flows	Steps of the 2–3 main use cases; lifecycle of the core object	Sequence / activity / state diagram
8	Patterns & SOLID	A rule that varies? A lifecycle? Many listeners? Two reasons to change?	1–2 named patterns + one rejected
9	Code	Interfaces, enums, models, services, driver	Running code for the main flows
10	Edge cases & concurrency	Bad input? Full? Two users, one seat?	Validation, exceptions, locks
11	Extensions	"What if X is added?"	Short answers, maybe one class

Time budgets

45-minute OOD round		**90-minute machine-coding round**	
Clarify requirements	5–8 min	Read, requirements, assumptions	5–10 min
Actors and use cases	3–5 min	Model: entities, enums, service interfaces	10–15 min
Find the classes	8–10 min	Core flow end to end	30–40 min
Connect the classes	8–10 min	Remaining features, validation, edge cases	15–20 min
Write the code	12–15 min	Polish, one extension point, demo	5–10 min
		Buffer	5 min (never zero)

Rule of thumb: roughly **a quarter of the hour goes before you name a class**, and in machine coding you **get something running by the halfway mark** (Ch 1 · The interview).

Finding classes (Ch 1 · Finding the classes)

- **Nouns → candidate classes; verbs → methods.** A verb with no class to live on means a class is missing.
- Cut candidates with five tests: is it a **value**? the **same thing** under another name? does it have **changing state**? does anything **call** it? does it differ in **behaviour or only data**?
- **Different data → field or enum. Different behaviour → class.**
- One sentence per class, **no "and"**. Put each method where its data is (Information Expert).
- Beginners most often forget the **record-keeping class** (ticket, loan, booking) and the **rule class** (rate, strategy).

What the interviewer judges (Ch 1 · The interview)

1. Did you **bound** the problem? 2. Does each class have **one job** you can say in a sentence? 3. Are the **relationships** right? 4. Does the **code match the diagram**? 5. Can you explain a **trade-off** (the choice and the rejected option)?

Machine coding adds: does it **run**, is it **modular**, **readable**, does it **handle bad input**, and (senior roles) is shared state **thread-safe**.

Code habits interviewers notice (Ch 1 · Interview code)

Habit	Instead of
Pass time in (`long nowMinute`, `Instant now`, injected `Clock`)	Reading the clock inside logic
Money as `BigDecimal` from strings, or whole cents in a `long`	`double` (0.1 + 0.2 ≠ 0.3)
`IllegalArgumentException` / `IllegalStateException` / a custom exception	Silent failure or error codes
`Optional` when "not found" is normal	Returning `null`
`private final` fields	Public mutable fields
Domain names (`ParkingTicket`)	`Manager`, `Helper`, `Data2`

Skeleton: **Models** → **Repositories** → **Strategies** → **Services** → **Driver** (Ch 1 · Interview code).

In the interview

30-second answer (from Ch 1 · Common mistakes): "I clarify and write 8 to 12 requirements with an out-of-scope list, list actors and use cases, pick classes from the nouns and give each one responsibility, connect them and draw a class diagram, sketch the key flow, run a quick SOLID check and pick one or two patterns where the signal is clear, then code the two or three core flows, handle edge cases and concurrency, and finish with extensions and trade-offs."

Common mistakes (Ch 1)

Mistake	Fix
Coding in minute two	8–13 minutes on requirements and actors first
No out-of-scope list	Write it down and agree it
One subclass per value (`CompactSpot`, `LargeSpot`)	Different data → field/enum
God class (`ParkingLotManager`)	One sentence per class; split on "and"
Inheritance for reuse ("Car extends Engine")	Composition
Pattern stuffing	One or two patterns, each with its signal
All twelve classes at equal depth	Two or three flows fully
Ignoring two users on one resource	Lock or atomic operation on the shared state
Non-running code in machine coding	Thin end-to-end path running by halfway

---

22.3 OOP & relationships cheat sheet

Source: Chapter 2 (OOP) and Chapter 3 (relationships and UML).

The four pillars (Ch 2)

Pillar	One line	Java tools	Smell when missing
Encapsulation (Ch 2)	Data is private; only the owning class's methods change it, and they enforce the rules	`private` fields, no setters, validation in constructors	Public fields; setters that break invariants
Abstraction (Ch 2)	Expose only what callers need; hide how	Interfaces, abstract classes, small public API	Callers reach into internals
Inheritance (Ch 2)	Child reuses and specialises parent — **only when "child is a parent" is truly correct**	`extends`, `super`, `@Override`	Subclass refuses or throws on parent methods
Polymorphism (Ch 2)	One call, different behaviour per object; removes type switches	Overriding + dynamic dispatch	`switch` / `instanceof` chains on type

Encapsulation vs abstraction: hiding *data* behind rule-enforcing methods vs hiding *complexity* behind a minimal set of operations (Ch 2 self-check).

Other OOP facts worth one line each

Topic	Remember	Where
Overloading vs overriding	Compile time, by declared parameter types vs run time, by the actual object. Fields and `static` methods are not polymorphic (statics are hidden, not overridden)	Ch 2 · Compile-time vs runtime polymorphism
`this`, `static`, `final`	Current object; one per class; cannot be reassigned/overridden/extended	Ch 2 · Classes & keywords
Interfaces	Contracts; can have `default`, `static`, `private` methods; a functional interface has one abstract method (lambdas)	Ch 2
Enums	Fixed sets with fields and behaviour ("which spot types fit a truck" lives on `VehicleType`)	Ch 2 · Enums with fields and behaviour
Composition over inheritance	Avoids fragile base classes; allows run-time swapping	Ch 2, Ch 5 · Composition over inheritance
`equals` / `hashCode`	Equal objects must have equal hash codes; override both or hash collections break	Ch 2 · Object identity vs equality
Immutability & records	Final class, private final fields, no setters, defensive copies. Records are only **shallowly** immutable — copy collections in the compact constructor	Ch 2 · Immutability & records
Access modifiers	Default to `private`; package-private = same package only	Ch 2 · Access modifiers and packages
Generics	Type-safe reusable infrastructure; **PECS**: producer `extends`, consumer `super`	Ch 2 · Generics basics for LLD
Coupling & cohesion	Low coupling (between classes), high cohesion (inside a class)	Ch 2 · Coupling and cohesion
OOA → OOD → OOP	What the problem is → how, with objects → code. Connections that carry data become objects	Ch 2 · Object-oriented analysis and design

Abstract class vs interface (Ch 2 · Abstract classes vs interfaces)

	Abstract class	Interface
Instantiable?	No	No
Instance fields (state)	Yes	No (only `public static final` constants)
Constructors	Yes	No
Method bodies	Any	`default`, `static`, `private` only
How many per class	**One** (`extends`)	**Many** (`implements`)
Relationship expressed	"is a" (shared identity and state)	"can do" (capability / role)
Typical use	Shared state + partial implementation for a close family (Template Method)	Contracts, roles, strategies, test seams

Rule of thumb: start with an interface; add an abstract class underneath when implementations share real state or a fixed algorithm (JDK: `List` + `AbstractList` + `ArrayList`).

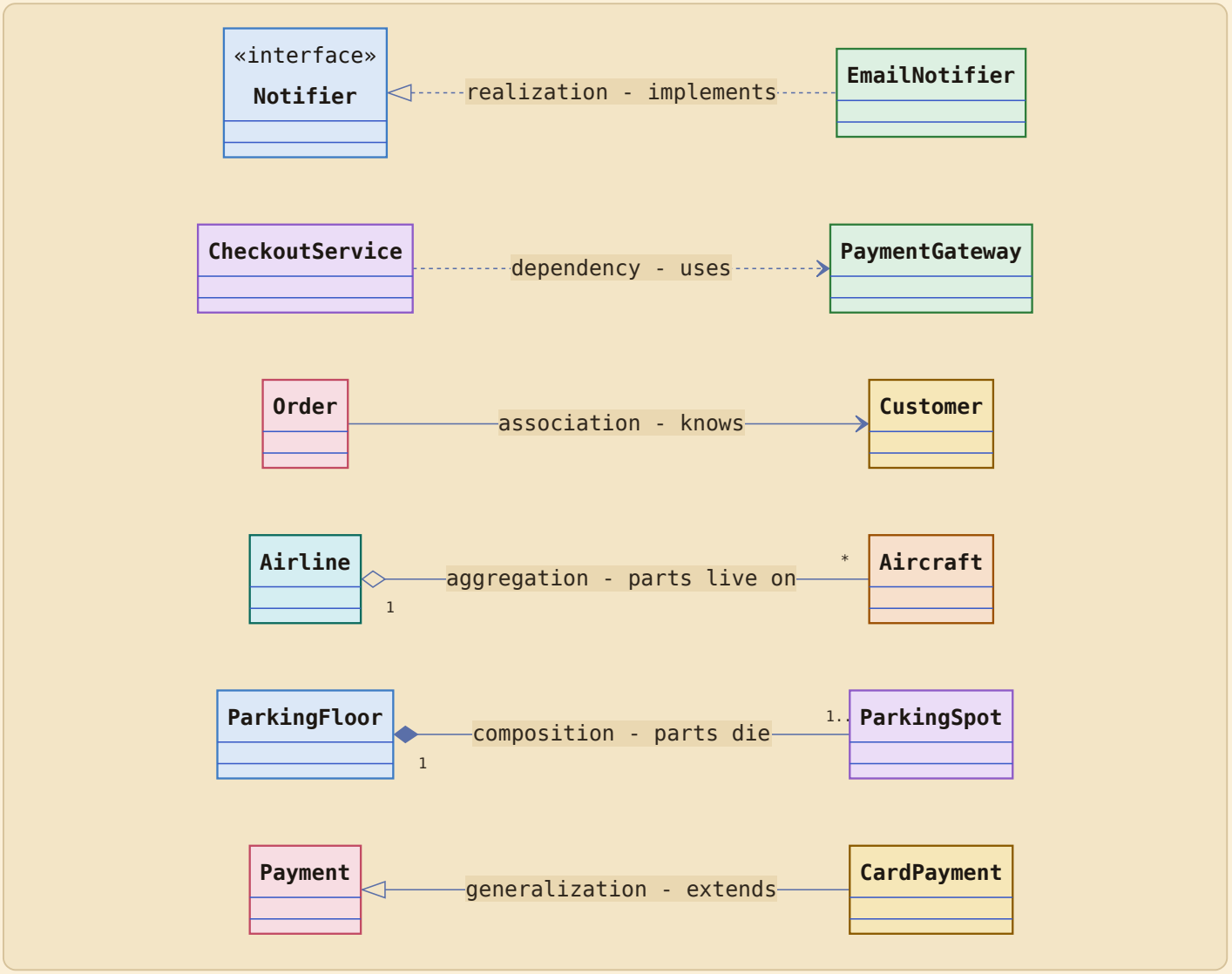
The six relationships (Ch 3)

Weakest to strongest coupling: **dependency, association, aggregation, composition, realization, generalization** (Ch 3 · self-check).

Relationship	Phrase	UML line	Java shape	Lifetime	Example
Dependency (Ch 3)	A *uses* B	dashed, open arrow → B	B as parameter / local / return / static call	none	`CheckoutService` uses `PaymentGateway`
Association (Ch 3)	A *knows* B	solid line (arrow for one-way)	field of type B, created elsewhere	independent	`Order` → `Customer`
Aggregation (Ch 3)	A *has* B, parts live on	solid, hollow ◇ at A (the whole)	field/collection, parts passed in	B outlives A	`Airline` ◇— `Aircraft`
Composition (Ch 3)	A *owns* B	solid, filled ◆ at A (the whole)	A creates B, hides it, B not shared	B dies with A	`ParkingFloor` ◆— `ParkingSpot`
Realization (Ch 3)	A *implements* B	dashed, hollow △ at B	`implements`	—	`EmailNotifier` ⇢ `Notifier`
Generalization (Ch 3)	A *is a* B	solid, hollow △ at B	`extends`	—	`CardPayment` → `Payment`

Choosing, in order (Ch 3 · Choosing the relationship): interface A promises to fulfil → realization. Substitutable with no refused methods → generalization. Not kept in a field → dependency. Part that dies with the whole → composition. Part that outlives it → aggregation. Otherwise → association.

UML arrow legend (Mermaid)



Mermaid syntax: `A ..> B` dependency · `A --> B` association (navigable A→B; `A -- B` unspecified) · `A o-- B` aggregation and `A *-- B` composition (A is the whole) · `B <|.. A` realization · `B <|-- A` generalization · `A "1" *-- "1..*" B` adds multiplicities.

Multiplicity → code (Ch 3 · TL;DR): `1` required field; `0..1` `Optional`/nullable; `1..*` non-empty collection checked in the constructor; exact numbers checked in constructors. Visibility: `+` public, `-` private, `#` protected, `~` package.

UML diagrams in the interview (Ch 3 · UML diagrams)

Diagram	Use it for	Budget in 45 min
Use case (Ch 3 · Use-case diagrams)	Scope: actors + what each does. «include» = always; «extend» = sometimes	Usually spoken, 3–5 min
Class (Ch 3 · Class diagrams)	**The answer.** 10–15 boxes, every line typed, with multiplicity	Always drawn, 10–15 min
Sequence (Ch 3 · Sequence diagrams)	Call order between objects (3–4 objects, 6–8 messages)	One behaviour diagram only
Activity (Ch 3 · Activity diagrams)	A flow that branches; fork/join for parallel steps	"
State machine (Ch 3 · State machine diagrams)	One object's lifecycle; transition label `event [guard] / action`	"

UML has 14 diagram types: 7 structural, 7 behavioural (Ch 3 · What UML is).

Common mistakes

- Diamond at the wrong end (it goes at the **whole**); triangle at the wrong end (it goes at the **parent/interface**) (Ch 3 · Choosing the relationship).
- Lines without type or multiplicity (Ch 3 · Choosing the relationship).
- Inheritance used to share data; subclasses that only pass a constant to `super` → one class with an enum field (Ch 3 · self-check).

22.4 SOLID one-pager

Source: Chapter 4 (the five principles, SOLID and patterns, SOLID in the interview, over-engineering); smells from Chapter 7 (smell lookup table).

TL;DR

SOLID = five names for the ways code becomes painful to change. **Use them to diagnose, not as a checklist**, apply a fix when a real change asks for it, and always say what the fix costs.

Principle	One-line rule	Smell that signals a violation	Fix	Patterns that help
S — Single Responsibility (Ch 4 · Single Responsibility Principle)	A class should have one reason to change (one actor asks for its changes)	The responsibility sentence needs an "and"; *Divergent Change*, *Large Class*	Split by actor (Extract Class); each new class needs a name that stands alone	Command, Proxy, Decorator (each concern in its own class)
O — Open/Closed (Ch 4 · Open/Closed Principle)	Add new behaviour by adding code, not by editing working code	A `switch` / `if` chain over a type; *Repeated Switches*, *Shotgun Surgery*	Put the part that *has* varied behind an interface; add variants as classes (Replace Conditional with Polymorphism)	Strategy, Template Method, Decorator, Factory Method, Observer, Command, Visitor
L — Liskov Substitution (Ch 4 · Liskov Substitution Principle)	A subtype must be usable anywhere its parent is expected	Subclass throws, does nothing, or refuses a parent method; *Refused Bequest*	Better parent, a narrower interface, or composition (Replace Subclass with Delegate)	Composite, Null Object
I — Interface Segregation (Ch 4 · Interface Segregation Principle)	No client should depend on methods it does not use	Empty method bodies or `UnsupportedOperationException` in implementers; fat interfaces	Split into **role interfaces**, one per kind of caller	Adapter
D — Dependency Inversion (Ch 4 · Dependency Inversion Principle)	Depend on abstractions, not concrete details; the policy owns the abstraction	A high-level class does `new` of (or names) a low-level one; cannot test without the real DB/provider	Inject an interface owned by the high-level class; `main` wires the details	Strategy, Factory Method / Abstract Factory, Adapter, Observer

LSP in detail (Ch 4 · Liskov Substitution Principle, TL;DR): a subclass must accept at least as much, promise at least as much, keep invariants and history, and throw nothing new. Square/Rectangle is worse than `Penguin.fly()` throwing because it fails **silently** (wrong area, no error).

Five-minute pass over your diagram (Ch 4 · SOLID in interviews, Ch 1 · SOLID check)

Run S → O → L → I → D over the diagram you just drew. **Two findings is a good pass; five means rework; zero means you did not look.** S and O earn the most credit: O is what interviewers probe with a late requirement ("what if trucks are charged differently?" → "add a class").

Related ideas (Ch 4 · Cohesion, Coupling, SoC, SOLID Q&A)

Idea	Question it asks
SRP	How many actors can ask this class to change?
Cohesion	Do the parts of this class belong together?
Coupling	How much does A depend on B?
Separation of Concerns	Is each area of work in its own module?
SRP vs ISP	SRP splits **classes** (guards the worker); ISP splits **interfaces** (guards the caller)
DIP vs DI vs IoC	DIP = principle (depend on abstractions); DI = technique (pass dependencies in); IoC = the broader idea that a framework calls your code; IoC containers perform DI

Common mistakes: over-engineering (Ch 4)

Warning sign	Principle misapplied	Do instead
`XHelper`, `XUtils`, `XManager2` after a split	SRP	Split by actor; names must stand alone
Six small classes that always change in the same commit	SRP over-split	Merge them back
Interface + factory + registry for one implementation	OCP (*Speculative Generality*)	Write the `if`; refactor at the second or third case (rule of three)
A subclass that "works" by silently doing nothing	LSP	Better parent or composition
One interface per method	ISP over-split	Group by caller into role interfaces
An interface in front of stable `String`-like classes	DIP	Invert only volatile dependencies
A DI container for a 200-line program	DIP	Wire by hand in `main`

YAGNI is the counterweight: SOLID says *what* a good seam looks like; YAGNI says *when* to cut it.

In the interview

Say **observation → cost → fix → price of the fix**, and describe before you label: "`Reporting` implements nine methods but calls two — an interface segregation problem" (Ch 4 · SOLID in interviews).

22.5 Other design principles and Clean Code rules

Other principles (Chapter 5)

Principle	One-line idea	Smell it fixes	Key Java tool	Where
DRY	Each piece of knowledge has one authoritative home	Duplicated rules drifting apart	Value objects, constants, methods	Ch 5 · DRY
KISS	Choose the simplest design that works	Over-engineering	Plain code, `switch` expressions	Ch 5 · KISS
YAGNI	Don't build features before they are needed	Speculative Generality	Delete unused code	Ch 5 · YAGNI
Law of Demeter	Only talk to your immediate friends	Train wrecks `a.getB().getC().do()`	Intent-revealing delegate methods	Ch 5 · Law of Demeter
Tell, Don't Ask	Tell objects what to do; don't pull their data out and decide for them	Anemic model, Feature Envy	Command methods on the owner	Ch 5 · Tell, Don't Ask
Composition over inheritance	Prefer "has-a" to "is-a" for reuse	Fragile base class	Forwarding / decorators	Ch 5 · Composition over inheritance
Program to an interface	Depend on supertypes, choose the concrete class once at the edge	Hard-wired concrete dependencies	Interfaces + DI	Ch 5 · Program to an interface
Encapsulate what varies	Isolate the part that changes behind an interface	Growing type switches	Strategy, lambdas	Ch 5 · Encapsulate what varies
Loose coupling	Interacting objects know little about each other	God orchestrators	Observer, events, DI	Ch 5 · Loose coupling
Hollywood Principle / IoC	"Don't call us, we'll call you" — the framework owns the flow	Duplicated skeletons	Template Method, callbacks, DI	Ch 5 · Hollywood Principle
Separation of Concerns	Each part handles one concern	Spaghetti code	Layers, MVC	Ch 5 · Separation of Concerns, Ch 4
Least Astonishment	Do what the name promises and nothing more	Hidden side effects	Honest names, `Optional`	Ch 5 · Least Astonishment
Fail fast	Detect problems early and loudly	Distant `NullPointerException`s	Guard clauses, `requireNonNull`	Ch 5 · Fail fast
Immutability	Objects never change after creation	Aliasing, races	Records, `List.copyOf`	Ch 5 · Immutability
Design by Contract	Preconditions (caller), postconditions (method), invariants (always)	Magic error values	Checks at method entry/exit	Ch 5 · Design by Contract
GRASP	Nine guidelines for "which class should do this?"	God classes	Expert, Creator, Controller…	Ch 5 · GRASP
Domain-Driven Design	Model the business: bounded contexts, entities vs value objects, small aggregates, domain events, repositories per root	Anemic model, big ball of mud, primitive obsession	Records as value objects, aggregate roots, events + outbox	Ch 5 · DDD

GRASP in one line each (Ch 5 · GRASP):

Guideline	Rule
Information Expert	The class with the data does the work
Creator	The container (or the one with the initial data) creates
Controller	A thin non-UI object receives each system event and delegates
Low Coupling	Choose the design with fewer, more stable dependencies
High Cohesion	Each class does one related set of things
Polymorphism	Let each type implement the varying behaviour; don't switch on types
Pure Fabrication	Invent a helper class (e.g. a repository) when no domain class is a clean home
Indirection	Put a middleman between things that shouldn't know each other
Protected Variations	Wrap what's unstable behind a stable interface

Tensions to mention (Ch 5 summary): DRY vs loose coupling (sharing code couples its users); YAGNI vs encapsulate-what-varies (abstract only real or clearly predicted variation); Law of Demeter vs cohesion (too many delegates → Middle Man); Information Expert vs SoC (domain objects compute, fabrications persist).

Clean Code rules checklist (Chapter 6)

Area	Rule of thumb	Where
Names	Reveal intent; pronounceable; searchable; one word per concept; code reads like the requirement	Ch 6 · Meaningful names
Functions	Small; do one thing; one level of abstraction; top-down (stepdown); ≤ 2–3 arguments; **no flag arguments**; no hidden side effects; commands *or* queries (CQS); exceptions, not error codes	Ch 6 · Functions
Comments	Say it in code; comment only the *why*, warnings, legal text, public contracts; delete commented-out code	Ch 6 · Comments
Formatting	Newspaper order (headline first); related things close; one automated team style	Ch 6 · Formatting
Objects vs data structures	Objects hide data and expose behaviour; data structures expose data; no hybrids; no train wrecks	Ch 6 · Objects and data structures
Error handling	Unchecked exceptions with context; wrap vendor exceptions; empty collections / `Optional` / special case instead of `null`; never pass `null`; always clean up	Ch 6 · Error handling
Boundaries	Keep third-party code behind your own interfaces; learn it with tests	Ch 6 · Boundaries
Unit tests	Three laws of TDD; one concept per test; Given–When–Then; **F.I.R.S.T.** (Fast, Independent, Repeatable, Self-validating, Timely)	Ch 6 · Unit tests
Classes	Small by responsibility; cohesive; open for extension; depend on abstractions	Ch 6 · Classes
Systems	Build objects in one place (composition root) and inject them; cross-cutting concerns in proxies/aspects	Ch 6 · Systems
Simple design	Beck: passes the tests → reveals intention → no duplication → fewest elements (*Clean Code* swaps the middle two)	Ch 6 · Simple design
Concurrency	Separate concurrency code; share as little as possible; use the library; keep locks small; plan shutdown	Ch 6 · Concurrency
Smells	The C/E/F/G/J/N/T heuristics catalogue as a review vocabulary	Ch 6 · Smells and heuristics
Code review	Correctness → design → readability → tests → security → performance → operability; automate style; small PRs	Ch 6 · Code review checklist
Error strategy	Classify, translate at boundaries, retry only transient + idempotent with backoff, one global handler that logs once	Ch 6 · Error-handling strategy
Testing strategy	Many unit, some integration, few E2E; right test double; verify state over interactions; don't mock what you don't own	Ch 6 · Testing strategies

Boy Scout Rule: leave the code a little cleaner than you found it (Ch 6 intro).

22.6 Code smells → refactorings (all 24)

Source: Chapter 7 (code smells, refactoring catalogue, smell lookup table). ***Bold*** *= the usual first move.*

#	Smell	Signal (one line)	Group	Refactorings that cure it
1	Mysterious Name	Name doesn't say what it does/holds (`data`, `proc()`, `Manager`)	2nd-ed extra	**Change Function Declaration**, Rename Variable, Rename Field
2	Duplicated Code	Same structure in more than one place	Dispensables	**Extract Function**, Slide Statements, Pull Up Method
3	Long Function	You want a comment to explain a block	Bloaters	**Extract Function**, Replace Temp with Query, Introduce Parameter Object, Preserve Whole Object, Replace Function with Command, Decompose Conditional, Replace Conditional with Polymorphism, Split Loop
4	Long Parameter List	Many parameters, same types, boolean flags	Bloaters	Replace Parameter with Query, **Preserve Whole Object**, **Introduce Parameter Object**, Remove Flag Argument, Combine Functions into Class
5	Global Data	Readable and writable from anywhere (incl. mutable singletons)	2nd-ed extra	**Encapsulate Variable** (then shrink its scope)
6	Mutable Data	Shared data changed after creation	2nd-ed extra	**Encapsulate Variable**, Split Variable, Slide Statements, Extract Function, Separate Query from Modifier, Remove Setting Method, Replace Derived Variable with Query, Combine Functions into Class/Transform, Change Reference to Value
7	Divergent Change	One class changed for many different reasons (SRP)	Change Preventers	**Split Phase**, Move Function, Extract Function, **Extract Class**
8	Shotgun Surgery	One change forces many small edits in many classes	Change Preventers	**Move Function**, **Move Field**, Combine Functions into Class/Transform, Split Phase, Inline Function, Inline Class
9	Feature Envy	Method uses another class's data more than its own	Couplers	**Move Function**, Extract Function
10	Data Clumps	Same group of values always travel together (`startDate, endDate`)	Bloaters	**Extract Class**, **Introduce Parameter Object**, Preserve Whole Object
11	Primitive Obsession	Money as `double`, status as `String`	Bloaters	**Replace Primitive with Object**, Replace Type Code with Subclasses, Replace Conditional with Polymorphism, Extract Class, Introduce Parameter Object
12	Repeated Switches	The *same* switch on a type code in several places	OO Abusers	**Replace Conditional with Polymorphism**
13	Loops	Classic loop that filters/maps/aggregates	2nd-ed extra	**Replace Loop with Pipeline**
14	Lazy Element	A class/function that doesn't pull its weight	Dispensables	**Inline Function**, **Inline Class**, Collapse Hierarchy
15	Speculative Generality	Hooks "we might need someday"; abstract class with one subclass	Dispensables	Collapse Hierarchy, Inline Function, Inline Class, Change Function Declaration, Remove Dead Code

#	Smell	Signal (one line)	Group	Refactorings that cure it
16	Temporary Field	Field set only in some situations, `null` otherwise	OO Abusers	**Extract Class**, Move Function, Introduce Special Case
17	Message Chains	`order.getCustomer().getAddress().getCity()`	Couplers	**Hide Delegate**, Extract Function, Move Function
18	Middle Man	Most methods just forward to another class	Couplers	**Remove Middle Man**, Inline Function, Replace Superclass/Subclass with Delegate
19	Insider Trading	Two modules poke at each other's internals	Couplers	**Move Function**, **Move Field**, Hide Delegate, Replace Subclass/Superclass with Delegate
20	Large Class	Too many fields/methods — a God Class	Bloaters	**Extract Class**, Extract Superclass, Replace Type Code with Subclasses
21	Alternative Classes with Different Interfaces	Same job, different method names (`sendMail` vs `dispatch`)	OO Abusers	**Change Function Declaration**, Move Function, Extract Superclass
22	Data Class	Only fields and getters/setters; others manipulate it	Dispensables	**Encapsulate Record**, Remove Setting Method, Move Function, Extract Function, Split Phase
23	Refused Bequest	Subclass doesn't want inherited methods, or throws on them (LSP)	OO Abusers	Push Down Method/Field, **Replace Subclass with Delegate**, **Replace Superclass with Delegate**
24	Comments	Comment used as deodorant for unclear code	Dispensables	**Extract Function**, Change Function Declaration, Introduce Assertion

Smell family → principle violated (Ch 7 · Smell lookup table)

Smell family	Principle
Divergent Change, Large Class	SRP
Repeated Switches, Shotgun Surgery	OCP
Refused Bequest	LSP
Long Parameter List with flags, fat interfaces	ISP / clean function design
Global Data, Mutable Data	Encapsulation
Hard-wired `new` of concrete dependencies	DIP
Duplicated Code	DRY
Speculative Generality	YAGNI
Message Chains, Insider Trading	Law of Demeter
Feature Envy, Data Class	Tell, Don't Ask

Refactoring toward patterns (Ch 7)

Smell → pattern, in short (the full symptom table is in 22.7): algorithm-choosing switch → **Strategy**; methods checking and changing a status field → **State**; `if (x == null)` everywhere → **Null Object / Special Case**; sibling subclasses with the same skeleton → **Template Method**; many confusing constructors → **Factory Method** / creation methods; object built by many ordered calls → **Builder**; undo/queueing, or a huge function with

shared locals → **Command**; logging/caching mixed into a core class → **Decorator**; polling or hard-coded notification → **Observer**; two classes, same job, different interfaces → **Adapter**; subclass explosion from feature combinations → **Strategy / Decorator / Bridge**; a pattern that no longer earns its keep → **remove it** (Inline Class, Collapse Hierarchy).

Refactoring = changing structure without changing behaviour, in small steps, with tests as the safety net. Wear one of **two hats** at a time: adding function or refactoring (Ch 7 · Refactoring fundamentals).

22.7 Design patterns: all 23 GoF + extras

Source: Chapters 8–11 (catalogue in Ch 8 · The 23 GoF patterns; comparison tables at the end of Chapters 8, 9, 10 and 11).

Count: 5 creational + 7 structural + 11 behavioral = **23**. **Class-scoped** (inheritance) patterns: Factory Method, class Adapter, Template Method, Interpreter; almost everything else is object-scoped (composition) (Ch 8 · The 23 GoF patterns).

Creational (Chapter 8)

Pattern	Intent (one line)	Use when	Java / JDK example	Commonly confused with	Where
Singleton	Exactly one instance with a global access point	There really must be exactly one (config, pool, registry)	`Runtime.getRuntime()`; Spring singleton-scoped beans	A global variable; a DI singleton scope (preferred for testing)	Ch 8
Factory Method	Subclasses decide which concrete class to create via an overridable method	Shared procedure, varying product; others will add types	`Collection.iterator()`	Simple Factory (one method with a switch)	Ch 8
Abstract Factory	Create *families* of related objects without naming concrete classes	Products must match; swap a whole set at once	`DocumentBuilderFactory`; JDBC `Connection`	Factory Method (one product by inheritance vs a family by composition)	Ch 8
Builder	Build a complex object step by step; one validating `build()`	Many optional fields, cross-field validation, immutable result	`StringBuilder`, `HttpClient.newBuilder()`	A plain constructor (fine for a handful of required fields); telescoping constructors	Ch 8
Prototype	Create new objects by copying a configured instance	Setup is expensive, or only the interface is known	Copy constructors, `ArrayList.clone()`	Memento (a snapshot to restore, not a copy to use)	Ch 8

Singleton notes (Ch 8 · TL;DR): eager, holder-class or `enum` (most robust: blocks reflection and serialization duplicates); double-checked locking needs a `volatile` field. It is global state that hurts testing — prefer injecting one shared instance. Many case studies reject it (Ch 14 self-check).

Structural (Chapter 9)

Pattern	Intent (one line)	Use when	Java / JDK example	Commonly confused with	Where
Adapter	Convert one interface into another that clients expect	"The vendor API doesn't match ours" (after the fact)	`Arrays.asList`, `InputStreamReader`	Bridge (same shape, but Bridge is designed up front); Facade	Ch 9 · Adapter Pattern
Bridge	Split abstraction from implementation so both vary independently	Class names with two nouns; two dimensions multiplying classes	JDBC `Connection` + drivers	Adapter; Strategy (same shape)	Ch 9 · Bridge Pattern
Composite	Treat single objects and trees uniformly	Part-whole trees (files, menus, UI)	`java.awt.Container` is a `Component` holding `Component`s	Decorator (also wraps a component, but adds behaviour to one)	Ch 9 · Composite Pattern
Decorator	Add responsibilities dynamically by wrapping; same interface, stackable	"Sometimes logging, sometimes caching, sometimes both"	`BufferedInputStream`, `Collections.synchronizedList`	Proxy (same shape; Proxy *controls access*); Chain of Responsibility (in CoR someone can stop the flow)	Ch 9 · Decorator Pattern
Facade	One simple entry point to a complex subsystem	Every caller repeats the same multi-step sequence	SLF4J `LoggerFactory`, service layers	Adapter (converts one object's interface); Mediator	Ch 9 · Facade Pattern
Flyweight	Share intrinsic (immutable) state across huge numbers of objects	Millions of near-identical objects	`Integer.valueOf`, `String.intern()`	Object Pool (reuses objects one borrower at a time; flyweights are shared at once)	Ch 9 · Flyweight Pattern
Proxy	A stand-in that controls access (lazy, remote, protective, caching)	It's expensive, remote or restricted	`java.lang.reflect.Proxy`, Spring `@Transactional`, Hibernate lazy load	Decorator	Ch 9 · Proxy Pattern

Memory hooks (Ch 9 · Structural comparison): Adapter *make it fit* · Decorator *add to it* · Proxy *guard it* · Facade *simplify it* · Bridge *split it* · Composite *nest it* · Flyweight *share it*.

Behavioral (Chapters 10–11)

Pattern	Intent (one line)	Use when	Java / JDK example	Commonly confused with	Where
Chain of Responsibility	Pass a request along handlers until one handles (or stops) it	Several checks, any may reject; handler not known in advance	Servlet `Filter` chain; `java.util.logging` handlers	Decorator (every layer always runs)	Ch 10
Command	Turn a request into an object: queue, log, retry, undo	Undo/redo, job queues, scheduling	`Runnable` / `Callable`; Swing `UndoManager`	Strategy (Strategy = *how*; Command = *what*)	Ch 10
Iterator	Traverse a collection without exposing how it is stored	Walk a collection/tree/paged API	`Iterator`, `Spliterator`, JDBC `ResultSet`	Visitor (what to do at each node)	Ch 10
Mediator	Centralise communication between peers in one coordinator	Many objects interact in tangled ways; order of reactions matters	Chat servers, UI controllers; `Executor` / `Timer` are a loose fit (Ch 10)	Observer (broadcast, no coordination); Facade	Ch 10
Memento	Capture and restore state without breaking encapsulation	Undo, checkpoints, rollback	A `record` snapshot; DB `ROLLBACK`, Git commits (real-world)	Command undo (store actions, not states); Prototype	Ch 10
Interpreter	One class per grammar rule; evaluate sentences of a small language	User-written rules/expressions	`java.util.regex.Pattern`, SpEL	Composite (same tree); Visitor (many operations over the AST)	Ch 10
Observer	Notify many dependents automatically when one object changes	An open-ended set of independent listeners	`PropertyChangeListener`, `java.util.concurrent.Flow`, Spring `@EventListener`	Mediator; pub-sub (Observer through a broker, Ch 20 · Pub-sub)	Ch 11
State	Behaviour changes with internal state, as if the class changed	Many states/transitions; methods repeat a `switch` on status	TCP connection (GoF's example), order lifecycles. `Thread.State` is only an enum of states, not the pattern	Strategy (same diagram; states know each other and advance themselves)	Ch 11
Strategy	Make a family of algorithms interchangeable behind one interface	Several ways to do one job, chosen at run time	`Comparator`; `RejectedExecutionHandler` policies	State; Template Method; Decorator (Strategy *replaces* a rule, Decorator *wraps* one)	Ch 11
Template Method	Fixed algorithm skeleton in a base class;	Long fixed flow, a few varying steps	`AbstractList`, `InputStream.read(byte[],…)`, `HttpServlet.service()`	Strategy (Template = inheritance, one step	Ch 11

Pattern	Intent (one line)	Use when	Java / JDK example	Commonly confused with	Where
	subclasses fill in steps			varies; Strategy = composition, whole algorithm varies)	
Visitor	Add operations to a stable set of classes without changing them (double dispatch)	Many new operations over a fixed type set (ASTs)	`javax.lang.model.element.ElementVisitor`; `FileVisitor` is named after it but is really a tree-walk callback	Iterator; in Java 21, **sealed types + pattern-matching** `switch` usually replace it	Ch 11

Beyond GoF: extras you will meet

Pattern	Intent (one line)	Use when	Java / real example	Commonly confused with	Where
Simple Factory (idiom)	One `create(type)` method maps a type value to a class	Small, stable set of types chosen by config	`Calendar.getInstance()`, `EnumSet.noneOf`	Factory Method	Ch 8
Object Pool	Borrow and return a bounded set of expensive objects	DB connections, limited resources	HikariCP, `ThreadPoolExecutor`	Flyweight	Ch 8
Null Object	A do-nothing implementation instead of `null`	`null` checks everywhere for an optional collaborator	`Collections.emptyList()`, `OutputStream.nullOutputStream()`	`Optional` (for lookups)	Ch 11 · Beyond GoF
Dependency Injection	Receive collaborators (as interfaces) through the constructor	Classes create their own collaborators; hard to test	Spring constructor injection; a hand-wired `main`	Service Locator (anti-pattern: the class asks for its dependencies)	Ch 11 · Beyond GoF
Repository	Collection-like interface over storage	Business logic mixed with storage code	Spring Data `CrudRepository`	DAO (table-oriented)	Ch 11 · Beyond GoF
Specification	A named, combinable business rule (`isSatisfiedBy`, and/or/not)	Same rule in queries and validation; finder-method explosion	`Predicate`; Spring Data JPA `Specification`	—	Ch 11 · Beyond GoF
MVC	Separate model, view and controller (Observer + Strategy + Composite)	UI, input handling and rules tangled	Spring MVC; Swing `JTable` + `TableModel`	MVP, MVVM (relatives)	Ch 11 · Beyond GoF
Event Sourcing (lite)	Store the events; derive current state by replaying them	Full history, audit, time travel	Append-only logs	Plain audit log	Ch 11 · Beyond GoF

"Which pattern?" by problem symptom

Combined from Ch 8 · How to pick a pattern, Creational patterns compared, Ch 9 · Structural comparison, Ch 11 · Choosing a behavioral pattern and Ch 7 · Refactoring toward patterns.

If you see / need…	Use	Not to be confused with
One operation, several interchangeable rules chosen at run time (pricing, matching, routing)	**Strategy**	State, Template Method
The class to create depends on a type value; caller shouldn't know the concrete class	**Factory** (Simple Factory / Factory Method)	Abstract Factory
Products must always match as a family	**Abstract Factory**	Factory Method
Constructor has ~6+ arguments, many optional	**Builder**	—
Another one of *this*, cheaply or without knowing its class	**Prototype**	Memento
Exactly one instance must exist	**Singleton** (or DI singleton scope)	A convenient global handle
Expensive objects created and destroyed repeatedly	**Object Pool**	Flyweight
Existing class has the wrong interface	**Adapter**	Facade, Bridge
Two dimensions multiplying classes (`WindowsButton`, `MacButton`, …)	**Bridge**	Adapter
Tree of part-whole objects treated the same	**Composite**	Decorator
Add behaviour in combinations without editing the class	**Decorator**	Proxy
Every caller repeats the same five steps into a subsystem	**Facade**	Mediator
Millions of near-identical objects	**Flyweight**	Object Pool
Lazy load, access control, remote, caching (same interface)	**Proxy**	Decorator
A request must pass several checks; any may reject it	**Chain of Responsibility**	Decorator (all layers run)
Undo/redo, queue, schedule, retry or log *requested actions*	**Command**	Strategy
Evaluate user-written rules in a small language	**Interpreter** (+ Specification)	Visitor
Walk a collection/tree/paged API without exposing structure	**Iterator**	Visitor
Many objects depend on each other and the **order** of reactions matters	**Mediator**	Observer
Roll back to an earlier state without exposing internals	**Memento**	Command undo
One change must reach an open-ended set of listeners	**Observer** (or a broker)	Mediator
Three or more methods repeat one `switch` over a status; legal actions depend on it	**State** (or enum + guarded method for 3–4 states)	Strategy
Several classes share a long fixed flow, differ in a few steps	**Template Method**	Strategy
Many new operations over a stable set of types	**Visitor**, or sealed types + pattern matching	Iterator
`null` checks everywhere	**Null Object** / `Optional`	—
Classes create their own collaborators	**Dependency Injection**	Service Locator
Business logic mixed with storage	**Repository**	DAO
Same rule duplicated in queries and validation	**Specification**	—

Three questions for the famous look-alikes (Ch 11 · Choosing a behavioral pattern):

1. Do the alternatives reference each other? Yes → **State**. No → **Strategy** (or Bridge if chosen once at wiring).
2. Can one participant stop the flow? Yes → **Chain of Responsibility**. Everyone always runs → **Decorator**. Everyone is told, nobody stops anyone → **Observer**.
3. Does a central object decide what happens next? Yes → **Mediator**. Each listener decides for itself → **Observer**.

The six that come up in almost every round (Ch 1 · Patterns in this round, Ch 8 · How to pick a pattern)

Pattern	Signal	Do not use when
Strategy	One operation, several swappable rules	Only one rule exists and nothing hints at a second
Factory	Creation depends on a type value	Only one concrete class exists
Observer	Several listeners the source should not know	A single listener the source already references
State	Behaviour depends on status; many transitions	3–4 states handled by one guarded method
Singleton	Genuinely exactly one	You just want a global handle
Builder	Many optional fields; long constructor	A handful of required fields

Common mistakes (Ch 8 · Anti-patterns)

Anti-pattern	Fix
God Object (one class does everything)	Split by responsibility; push behaviour to the data owners
Golden Hammer (favourite pattern everywhere)	Start from the problem's signal
Pattern overuse ("patternitis") — factories for one class, single-implementation interfaces	YAGNI: add the pattern when the second variant arrives
Singletonitis (singletons as globals)	Dependency injection
Cargo-cult patterns (structure without intent)	Name the property the pattern gives you, or drop it

In the interview

Use **one or two** patterns in a 45-minute design, placed where their signal appears, and **name one you rejected** and why (Ch 8 · How to pick a pattern, self-check).

22.8 Concurrency cheat sheet

Source: Chapter 12 (concepts) and Chapter 13 (classic problems).

TL;DR (Ch 12)

Races come from **shared mutable state**. Make the critical section atomic, or remove sharing/mutability. Visibility needs **happens-before** edges. Prevent deadlock with **lock ordering** or **tryLock + back-off**. In LLD: name the contended resource, keep **check + claim atomic**, lock fine-grained and in order, and say what changes with many servers.

Primitive | use for | gotcha

Primitive	Use for	Gotcha	Where
`synchronized`	Making a multi-step update atomic (mutex + visibility)	Don't lock on a changing reference, a boxed value or a string literal; no slow I/O while holding it; readers must use the same lock	Ch 12 · Mutex and synchronized
`ReentrantLock`	Same as `synchronized` plus try/timed/fair locking and `Condition`s	Call `lock()` **just before** `try`, `unlock()` in `finally`; fairness is noticeably slower	Ch 12 · ReentrantLock and reentrancy
`tryLock(timeout)`	Give up if the lock is busy; deadlock prevention with back-off	Never `unlock()` when `tryLock` returned `false`; don't retry in a tight loop (livelock)	Ch 12 · Try-lock & timed locking
`ReentrantReadWriteLock`	Many readers, few writers	Cannot upgrade read → write (self-deadlock); don't modify under the read lock; writer starvation in non-fair mode	Ch 12 · Read-write locks
`StampedLock`	Read-mostly data with optimistic reads	**Not reentrant**; `validate` the stamp before using values read optimistically	Ch 12 · Read-write locks
`volatile`	Visibility of a flag or a published immutable snapshot	Gives visibility, **not** atomicity: `count++` is still a race	Ch 12 · Java Memory Model
`AtomicInteger` / `AtomicLong` / CAS	Lock-free counters and single-variable claims	Separate `get()` then `set()` is still a race — use `compareAndSet`/`updateAndGet`; no side effects in the lambda (may run several times); ABA → `AtomicStampedReference`	Ch 12 · CAS & atomics
`LongAdder`	Very hot counters (metrics)	Use instead of `AtomicLong` when contention is high	Ch 12 · CAS & atomics
`Semaphore(N)`	Limit concurrent access to N; signalling between threads	Release in `finally`; don't release when `acquire` never succeeded	Ch 12 · Semaphores
`wait/notifyAll`, `Condition`	Waiting for a condition on shared state	Always `while`, never `if` (spurious wakeups); hold the monitor; `notify()` can wake the wrong thread; state change without notify = lost wakeup	Ch 12 · Condition variables
`CountDownLatch`	Wait for N events once	One-shot; `countDown()` in `finally`, `await(timeout)`	Ch 12 · CountDownLatch and CyclicBarrier
`CyclicBarrier` / `Phaser`	N threads meet at each phase (reusable)	Fewer threads than parties → everyone waits forever	Ch 12 · CountDownLatch and CyclicBarrier
`ConcurrentHashMap`	Shared map	`containsKey` then `put` is a race — use `putIfAbsent`/`computeIfAbsent`/`merge`; no slow work or recursive updates inside `compute`; no `null` keys/values	Ch 12 · Concurrent collections

Primitive	Use for	Gotcha	Where
`CopyOnWriteArrayList`	Read-mostly lists (listener lists)	Every write copies the array	Ch 12 · Concurrent collections
`BlockingQueue` (bounded)	Handing work between threads (producer–consumer)	Unbounded queue → no back-pressure → memory blow-up; one poison pill per consumer	Ch 12 · Producer-consumer
`ExecutorService` / `ThreadPoolExecutor`	Running tasks on a bounded pool	Call `shutdown()`; `newFixedThreadPool` has an **unbounded** queue; exceptions in `submit`ted tasks are lost unless you call `get()`; tasks waiting on tasks in the same pool → thread-starvation deadlock	Ch 12 · Creating threads, Thread pool pattern
Virtual threads	Many blocking I/O tasks	Pinning (Java 21–23: blocking inside `synchronized`); heavy `ThreadLocal`s waste memory	Ch 12 · Creating threads, Immutability & confinement
`ForkJoinPool` / parallel streams	Splitting CPU work	Use a threshold: tiny tasks cost more to schedule than to run; subtasks must touch disjoint data	Ch 12 · Choosing the tool, Ch 13 · Parallel merge sort
`CompletableFuture`	Combining async results (`thenApply`, `thenCombine`, `allOf`)	Handle errors with `exceptionally` rather than calling `get()` at every step	Ch 12 · Creating threads
`ThreadLocal`	Per-thread state (thread confinement)	Clear it (`remove()`) in pooled threads, or values leak into the next task	Ch 12 · Immutability & confinement
Immutability / confinement	Avoiding synchronization entirely	`final List` is not immutable if the list is mutable and exposed	Ch 12 · Immutability & confinement

Which `java.util.concurrent` class for which job (Ch 12 · Choosing the tool)

Counter → `AtomicInteger`/`AtomicLong` (`LongAdder` if very hot) · stop a loop from another thread → `volatile boolean` or `interrupt()` · atomic multi-step update → `synchronized`/`ReentrantLock` · give up if busy → `tryLock(timeout)` · many readers, few writers → `ReentrantReadWriteLock`, `StampedLock` or an immutable snapshot in a `volatile` field · limit access to N → `Semaphore(N)` · wait for a condition → `Lock` + `Condition` (or `wait/notifyAll`) in a `while` loop · wait for N events once → `CountDownLatch` · N threads meet each phase → `CyclicBarrier`/`Phaser` · hand work between threads → bounded `BlockingQueue` · shared map → `ConcurrentHashMap` with `putIfAbsent`/`compute`/`merge` · read-mostly list → `CopyOnWriteArrayList` · run tasks → bounded `ExecutorService`, virtual threads for blocking I/O · split CPU work → `ForkJoinPool`/parallel streams · combine async results → `CompletableFuture` · avoid synchronization → immutability, thread confinement.

Deadlock checklist (Ch 12 · Deadlock, Livelock, Starvation)

A deadlock needs **all four Coffman conditions**; break any one and it cannot happen.

#	Condition	How to break it
1	Mutual exclusion	Lock-free/immutable data, shareable resources (read locks)
2	Hold and wait	Acquire everything at once, or `tryLock` and release everything on failure
3	No preemption	Timeouts (`tryLock(timeout)`), leases that expire
4	Circular wait	**Global lock ordering** (always lock the lower id first)

- [ ] Do I ever hold two locks? If yes, is there **one global order** (account id, seat id), with a tie-breaker for equal keys?
- [ ] Do I call a **listener, callback or another object's synchronized method while holding my lock**? Move it outside the lock.
- [ ] Do tasks in a bounded pool **wait for other tasks in the same pool**? (thread-starvation deadlock)
- [ ] Are retries randomised (**jitter**) so two threads don't back off in lockstep? (livelock, Ch 12)
- [ ] Could one thread wait forever while others progress? Consider fairness (Ch 12 · Starvation).
- [ ] Detection: a thread dump (`jstack` / `jcmd`) prints "Found one Java-level deadlock"; `ThreadMXBean.findDeadlockedThreads()`.

Deadlock vs livelock vs starvation: nobody progresses and all wait / everybody runs but nobody progresses / some progress while one never does (Ch 12 · self-check).

The LLD concurrency recipe (Ch 12 · Concurrency in LLD interviews)

1. **Name the shared mutable state** (free spots, seat → owner, stock).
2. **State the invariant** ("a seat has at most one booking", "stock ≥ 0").
3. **Find the critical operation** — usually a check-then-act (*find free → assign*).
4. **Pick the smallest tool that makes it atomic:** one item → `queue.poll()`, `putIfAbsent`, `compareAndSet`; several items all-or-nothing → per-item locks **in sorted order**; across servers → DB constraints/transactions, optimistic versioning, or a distributed lock.
5. **Choose granularity:** per floor / show / SKU, not one global lock.
6. **Handle time:** holds with expiry, timeouts, idempotent retries.

Classic problems (Ch 13 · summary)

Problem	Key primitive	Invariant to state
FooBar (Ch 13 · Print FooBar Alternately)	2 semaphores	Exactly one permit exists across both
Zero Even Odd (Ch 13 · Print Zero Even Odd)	3 semaphores	Zero dispatches; one permit circulates
FizzBuzz (Ch 13 · Fizz Buzz Multithreaded)	monitor + `notifyAll`	Predicates partition 1..n
H2O (Ch 13 · Building H2O)	2 semaphores + barrier	≤ 2 H and ≤ 1 O per round
Dining Philosophers (Ch 13)	ordered locks / n−1 semaphore	No circular wait
TTL cache (Ch 13 · Thread-safe cache with TTL)	`ConcurrentHashMap` + `compute` + conditional remove	Never return or keep expired data
Striped map (Ch 13 · Striped concurrent map)	lock array	Each bucket belongs to exactly one stripe
Blocking queue (Ch 13 · Thread-safe bounded Blocking Queue)	lock + 2 conditions	0 ≤ count ≤ capacity; wait in loops
Bloom filter (Ch 13 · Concurrent Bloom Filter)	atomic OR	Bits only go 0 → 1; no false negatives
Merge sort (Ch 13 · Parallel merge sort)	Fork/Join	Subtasks touch disjoint ranges
Token-bucket rate limiter (Ch 13)	lock or CAS on immutable state	Refill + take is one atomic step

Common mistakes (Chapter 12)

- A global `synchronized` on `ParkingLot.park()` — correct, but serialises every gate.
- Checking availability with one call and booking with another (check-then-act).
- Locking seats in the order the user clicked them (deadlock between two multi-seat bookings).

- Forgetting that in-memory locks don't work across multiple app servers.
- Swallowing `InterruptedException` — rethrow, or restore the flag with `Thread.currentThread().interrupt()` (Ch 12 · Thread states).
- Calling `run()` instead of `start()` (Ch 12 · Thread states).

22.9 Case-study map

Source: Chapters 14–21. "Concurrency concern" is the race each chapter guards against, and "atomic step" is how.

Everyday objects (Chapter 14)

Problem	Key entities	Patterns used	Concurrency concern	Where
Parking lot	`ParkingLot`, `ParkingFloor`, `ParkingSpot`, `Vehicle`, `Ticket`, `DisplayBoard`, `SpotType` / `VehicleType` enums	Strategy (pricing, spot-finding), Factory (electric spots), Observer (boards); Singleton rejected	Two gates take one spot → atomic `pollFirst` + CAS; per-ticket lock for payment; `putIfAbsent` for plates	Ch 14
Vending machine	`VendingMachine`, `Slot`, `Inventory`, `CashBox`, `Denomination`, `VendingState` + 5 states	**State** (the whole design)	`synchronized` machine; change-making by bounded coin-change DP, not greedy	Ch 14
Coffee machine	`CoffeeMachine`, `Recipe`, `RecipeBook`, `IngredientStore`, `OrderResult` (sealed)	Observer (low stock), recipes as data	Two orders take the last milk → one `ReentrantLock` around check + deduct; notify outside the lock	Ch 14 · Coffee Vending Machine
ATM	`Atm`, `AtmState` + 4 states, `Transaction` (sealed), `NoteHandler`, `CashDispenser`, `BankService`, `Account`	State (session), Command (transactions), Chain of Responsibility (note cassettes)	Overdraw from two ATMs → per-account lock; transfers lock in id order	Ch 14
Elevator	`Elevator`, `StopList` (two sorted sets), `Door`, `Dispatcher`, `DispatchStrategy`, `Building`	Strategy (dispatch) + modes; LOOK scheduling	Per-car lock, synchronized dispatcher; tick-driven `step()`	Ch 14 · Elevator System
Traffic signal	`Intersection`, `TrafficLight`, `Phase`, `SignalState` + 5 states, `TimingPolicy`, `SafetyMonitor`	State (timed), Strategy (timing), Observer (lights)	Synchronized tick; safe with a timer thread plus emergency threads	Ch 14 · Traffic Signal Control System
Shipping locker	`LockerStation`, `Compartment`, `Booking`, `CompartmentSelector`, `PickupResult` (sealed), `LockerListener`, `LockerNetwork`	Strategy (door choice), Observer (SMS), enum state machines, sealed results	Two carriers, one door → station lock around choose-and-mark; lazy expiry of holds and pickup windows; hashed single-use codes, keypad lockout	Ch 14 · Shipping Locker System

Booking & inventory (Chapters 15–16)

Problem	Key entities	Patterns used	Concurrency concern	Where
Library	`Book` (title) vs `BookItem` (copy), `Member`, `BookLending`, `LendingService`, `Catalog`	Strategy (fines), notifier interface; FIFO reservation queue per title	Last copy lent twice → lock member then copy (fixed order)	Ch 15 · Library
Movie tickets (BookMyShow)	`Show`, `ShowSeat`, `Booking`, `SeatLockProvider`, `BookingService`, `CancellationPolicy`	Strategy (refund rule), gateway behind an interface	Book `ShowSeat`s; all-or-nothing hold with TTL → pay → atomic commit; refund if hold lost	Ch 15 · BookMyShow
Concert tickets	`SeatedInventory`, `GaPool`, `SeatState` (sealed), `Hold`, `VirtualWaitingRoom`, `PurchaseLimiter`	Sealed seat states, waitlist	Extreme contention → CAS per seat, CAS counter for GA, FIFO waiting room, per-user limits, TTL holds with a reaper	Ch 15 · Concert tickets
Hotel	`Hotel`, `Room`, `RoomBooking`, `Invoice`, `RoomCharge`, `Payment` subclasses, `FrontDesk`	Template Method (`RoomCharge`)	Room × night reserved atomically in `synchronized Room.tryReserve`	Ch 15 · Hotel
Car rental	`Vehicle`, `DateRange`, `Reservation`, `Bill`/`BillItem`, `ReservationAddOn`, `PricingStrategy`	Template Method (add-ons), Strategy + Decorator (pricing), state machine	Availability is about **dates** → `Vehicle.tryBook(range)` synchronized	Ch 16 · Car Rental
Restaurant	`Menu`/`MenuSection`/`MenuItem`, `Table`, `Reservation`, `Order`, `Meal`/`MealItem`, `Kitchen`, `Check`	Observer (order ready), Strategy (chef assignment, payment), state machine	Two receptionists, one table → `Table.tryHold(time)` synchronized; listeners after the lock	Ch 16 · Restaurant
Airline	`Flight` vs `FlightInstance`, `Seat` vs `FlightSeat`, `FlightReservation`, `Itinerary`, `FareStrategy`	Strategy + Decorator (fares), Facade	Selling one seat twice → `FlightInstance.holdSeats` all-or-nothing under the instance lock	Ch 16 · Airline
Course registration	`Course` vs `Section`, `Student`, `RegistrationRule`s, `Registrar`	Chain of rules / Specification	Overselling a section → section lock inside a student lock (fixed order)	Ch 16 · Course Registration

Games (Chapter 17)

Problem	Key entities	Patterns used	Concurrency concern	Where
Blackjack + deck	`Card`, `Deck<T>`, `Shoe`, `Hand`, `BlackjackScoring`, `PlayerStrategy`, `DealerPolicy`	Strategy (player, dealer); injected randomness	One table = one thread; shared balances need an atomic wallet service	Ch 17 · Blackjack
Chess	`Piece` (sealed) + subclasses, `SlidingPiece`, `Board`, `Move`, `Rules`, `Game`	Polymorphic pieces, Factory Method, Command + Memento (undo)	`synchronized` game so a move and a resignation don't interleave	Ch 17 · Chess
Tic-tac-toe	`Board` (row/col/diagonal counters), `Game`, `Player`, `MoveStrategy`	Strategy (players)	Many games in a `ConcurrentHashMap`; per-game lock	Ch 17 · Tic-Tac-Toe
Snake & ladder	`Board`, `Jump`, `Dice`, `Rules`, `OverflowRule`, `WinStrategy`, `Game`	Strategy (dice, overflow, win)	Each game its own `Random` and its own thread of turns	Ch 17 · Snake and Ladder
2048	`LineMerger`, `Board`, `TileSpawner`, `Game`, `Direction`	Strategy (spawner); one pure merge function	Single-threaded; `synchronized move` if UI and input threads both call it	Ch 17 · 2048

Platforms (Chapters 18–19)

Problem	Key entities	Patterns used	Concurrency concern	Where
Stack Overflow	`Post` (sealed) → `Question` / `Answer` / `Comment`, `Member`, `Tag`, `Bounty`, `QuestionIndex`	Strategy (reputation), Observer (badges), Facade; inverted index	Two votes at once → `synchronized` post; `AtomicInteger` reputation	Ch 18
Cricinfo	`Match` → `Innings` → `Over` → `Ball`, `Team`, `Playing11`, `StatQuery`	Template Method (stat queries), Observer (live pages)	Two scorers → `synchronized` match; observers in `CopyOnWriteArrayList`	Ch 18
LinkedIn	`Member`, `Profile`, `SocialContent`, `ConnectionInvitation`, `ConnectionService`, `SearchIndex`, `FeedService`	State (invitation), Strategy (feed ranking, notifications), Facade	Symmetric connections → all graph changes synchronized on the service	Ch 18
Facebook	`Member`, `Audience` (sealed), `PrivacyList`, `FriendService`, `VisibilityService`, `NewsFeedService`	Strategy (feed ranker), Observer (notifications)	Accept can't leave a one-way friendship → synchronized service, concurrent adjacency sets	Ch 18
Amazon	`Product`, `ShoppingCart`, `Order`, `OrderLog`, `Payment`, `Shipment`, `Customer` → `Guest` / `Member`	Strategy (payment), Observer (notifications), order state machine, Facade	Last unit, two buyers → `synchronized Product.reserve`; all-or-nothing multi-line reservation	Ch 19 · Amazon
Stock brokerage + exchange	`Order` → Market/Limit/StopLoss/StopLimit, `OrderPart`, `StockLot`, `OrderBook`, `MatchingEngine`	Observer (fills), Template Method (statements), trade-price policy	Over-spending → reserved funds; one lock per symbol; price-then-time matching	Ch 19 · Stock Brokerage
Auction	`AuctionListing`, `Bid`, `BidResult`, `AuctionService`, `AuctionListener`	Observer, Specification (search), status state machine	Concurrent bids → all bid rules in one per-listing critical section; notify after unlocking	Ch 19 · Auction
Music streaming (Spotify)	`Song` / `Album` / `Artist`, `MusicLibrary`, `Playlist`, `MusicPlayer`, `PlaybackOrder`, `Recommender`	Strategy (play order, recommendations), Observer (plays), plan rules in an enum	Two devices press "next" → `synchronized` player methods	Ch 19 · Music Streaming
Food delivery (Swiggy) + FoodKart	`Restaurant`, `Cart`, `Order`, `DeliveryAgent`, `DeliveryAssignmentStrategy`, `RestaurantSelectionStrategy`	Strategy (assignment, selection, payment), Observer, order state machine	Kitchen capacity reserved atomically; riders claimed with CAS	Ch 19 · Food Delivery
Grocery store (POS)	`Register`, `Sale`, `LineItem`, `PricingEngine`, `ItemPromotion` / `BasketPromotion`, `TaxPolicy`, `Tender` (sealed), `Receipt`, `Inventory`	Strategy (offers, tax, payment gateway), Facade (register), Observer (stock), sale state machine	Registers share stock → atomic `merge`, may go negative, edge-triggered alerts; returns claimed with `compute`	Ch 19 · Grocery Store

Pattern-driven components (Chapter 20)

Problem	Key entities	Patterns used	Concurrency concern	Where
Splitwise	`Expense`, `SplitStrategy` (Equal/Exact/Percent), `Share`, `Ledger`, `DebtSimplifier`	Strategy (splits)	One signed number per pair, one mutator; `BigDecimal`, no lost cents	Ch 20
Notification service	`NotificationChannel` (abstract), channels, `SendRule`, `ChannelFactory`, `DeliveryListener`	Template Method (channels), Strategy (send rules), Observer, Simple Factory, Adapter	Rate-limit check-then-act gap; idempotency	Ch 20
In-memory cache / LRU	`Cache<K,V>`, `Node`, `DoublyLinkedList`, `EvictionPolicy`, `Cache.Builder`, `ShardedCache`	Strategy (eviction), Observer (removals), Builder	One lock (a `get` is a write in LRU); listeners outside the lock; shard to scale	Ch 20
Ride sharing (Uber)	`Rider`, `Driver`, `Trip`, `MatchingStrategy`, `PricingStrategy`, `SurgePricing`	Strategy ×3, Decorator (surge), trip state machine	Double-booking a driver → atomic claim (`AtomicBoolean`)	Ch 20 · Ride sharing
Logging framework	`Logger`, `LogManager`, `LogRecord`, `Appender`, `AsyncAppender`, `Formatter`	Chain of Responsibility (hierarchy), Singleton (`LogManager`), Strategy (formatters), Template Method (appender), Decorator (async)	Thread-safe sinks; never block on a full queue; never crash the app	Ch 20
Pub-sub	`Broker`, `Topic`, `Subscription`, `Subscriber`, `Message`	Observer through a broker	One dispatcher per subscription; ack after processing (at-least-once) → idempotent subscribers	Ch 20 · Pub-sub
Rate limiter	`Limiter` (TokenBucket, FixedWindow, SlidingWindowLog/Counter), `CompositeLimiter`, `LimiterFactory`	Strategy, Simple Factory, Composite, Facade	Atomic refill-check-take; one limiter per key via `computeIfAbsent`; Redis + Lua across servers	Ch 20

Machine-coding problems (Chapter 21)

Problem	Key entities	Patterns used	Concurrency concern	Where
Task management	`Task`, `TaskStatus`, `TaskService`, `TaskRepository`, `TaskFilter`	Specification (filters), Strategy (sorts), Observer, enum state machine	Per-task `synchronized`	Ch 21 · Task management
Digital wallet	`Wallet`, `Transaction`, `Money`, `Offer`, `WalletService`	Strategy (offers), idempotency key	Lock both wallets in a global (id) order; debit before credit; total conserved	Ch 21 · Digital wallet
Event calendar (Flipkart)	`TimeSlot` (half-open), `Event`, `EventStore`, `RepresentativeSelector`, `CalendarService`	Strategy (representative), plan-then-save	Atomic check-and-book	Ch 21
Coding Blox (Flipkart)	`Contest`, `Level`, `User`, `Question`, `SolveSimulator`, `CommandProcessor`	Command map, Strategy (solver, scoring)	Atomic rating update; don't run a contest twice	Ch 21
PropertyHunt (ClearTrip)	`Property`, `SearchCriteria` (+ Builder), `PropertyService`	Builder + predicate list, Strategy (sort)	Synchronized mark-sold	Ch 21
Ledger Co (Navi)	`Loan`, `LumpSum`, `Balance`, `InterestPolicy`, `LedgerService`	Strategy (interest), derived state, command loop	Synchronized lump sums	Ch 21
Inventory management	`Product`, `Warehouse`, `StockMovement`, `AllocationStrategy`, `ReorderListener`	Strategy (allocation), Observer (edge-triggered alerts)	Overselling → lock per SKU	Ch 21 · Inventory
Unix file search (`find`)	`Node` → `FileNode` / `DirectoryNode`, `FileFilter`, `AndFilter` / `OrFilter` / `NotFilter`, `SearchCriteria` (+ Builder), `FileSearcher`, `FindCommandParser`	Specification + Composite (filters and file tree), Builder, recursive-descent parser	Weakly consistent children map; read-only fork/join walk	Ch 21 · Unix File Search

TL;DR — what every case study teaches

- **Name the real unit of inventory** (copy, show seat, room-night, vehicle-day) (Ch 15 summary).
- **Every "if free then take" is one atomic step** — a monitor, a CAS, or a conditional `UPDATE` (Ch 15 summary).
- **Put rules on the object that owns the data**; the service only orders the steps (Ch 16, Ch 20 summaries).
- **Strategy** for the rule most likely to change; **Observer** for "tell someone when it happens"; **State** (or an enum state machine) for lifecycles.
- **Inject** time and randomness; money in `BigDecimal` or integer cents; call listeners **outside** locks.

22.10 Machine-coding and edge-case checklists

Source: Ch 21 · Machine-coding round (and Ch 1 · Interview code).

Machine-coding checklist

Before coding (0–20 min)

- [ ] Read the statement twice; mark **mandatory vs bonus**; ask clarifying questions.
- [ ] Put the statement's sample input into the driver as a `List<String>` of commands.
- [ ] Ask: "Where does it need to be thread-safe?" (wallets and inventory: always yes).
- [ ] Nouns → entities, verbs → service methods. Pick the 1–2 patterns the statement begs for ("can be sorted by X or Y" → Strategy).
- [ ] Create the six packages: **model, repository, service, strategy, exception, driver**.

While coding (20–75 min)

- [ ] Implement mandatory requirements **in the statement's order**; run the driver after each one (at least every 15 minutes).
- [ ] Entities guard their own invariants (e.g. `Task.changeStatus` rejects `TODO → DONE`).
- [ ] Services return data and throw domain exceptions; **no `System.out` in services**.
- [ ] Repository = interface + `ConcurrentHashMap` implementation; IDs from an `AtomicInteger`.
- [ ] Driver = `Map<String, Consumer<String[]>>` command processor; catch domain exceptions per command, **print and continue**.

Before you stop (75–90 min)

- [ ] Edge cases and validation done (see below); custom exceptions with readable messages.
- [ ] Thread-safety where money or stock is shared; say where the lock is and why.
- [ ] Bonus only after every mandatory feature runs.
- [ ] Clean names, delete dead code, run everything once more, prepare a 2-minute walkthrough.

Rubric the interviewer uses (Ch 21 · Machine-coding round)

(1) **Working, demo-able code** — thin end-to-end path in the first hour, never finish non-compiling; (2) **functional completeness** — mandatory first, in order; (3) **clean models** — enums for fixed sets, records for values, classes for changing things; (4) **separation of concerns** — Driver → Service → Repository; (5) **extensibility** — new rule = new class; (6) **edge cases** — custom exceptions, the driver keeps going; (7) **readability** — names from the statement's nouns and verbs; (8) **concurrency (when asked)** — say where the lock is, demo with a pool; (9) **tests (bonus)** — an `assert`-style self-check in `main` is enough.

Edge-case checklist (Ch 1 · Interview code, Ch 21 · Machine-coding round, case studies)

Category	Questions to ask your code
Input	`null`, empty, negative, zero, duplicate IDs, unknown IDs, invalid numbers
Ownership	Wrong user acting on someone else's object; a player moving out of turn
Capacity	Full lot, zero seats left, empty playlist, max loans reached, party larger than any table
State	Paying twice, exiting without entering, cancelling a delivered order, moving after game over, running a contest twice
Time	Exit before entry, holds/bookings expiring, part hours (round up?), an event exactly at the end instant (half-open intervals), late returns
Money	`BigDecimal` scale and rounding; no lost cents in splits; compare with `compareTo`, not `equals` (Ch 4 money note)
Concurrency	Two threads booking the same seat/spot/rider; multi-item bookings locking in sorted order; retries that must be idempotent
Failure	Payment declined after a hold; hold lost before payment → compensating refund; listener throws
Randomness	Seeded `Random` or a scripted strategy so the demo is repeatable

Common mistakes (Ch 21 · Machine-coding round)

Besides the Chapter 1 mistakes in 22.2: 20 minutes on a perfect diagram (enums/records first); one `Manager` with 15 maps (a service per area + repositories); `System.out` in services (the driver prints); `double` money (`BigDecimal` or whole units in a `long`); crashing on a bad command (catch per command); bonus before mandatory; unseeded `Random` (seed it or use a Strategy); `LocalDate.now()` in logic (inject a `Clock`).

---|---| | 20 minutes on a perfect class diagram | Enums/records first; the diagram lives in names | | One `Manager` with 15 maps | One service per area + repositories | | `System.out.println` inside a service | Services return data; the driver prints | | `double` for money | `BigDecimal` with a fixed scale, or whole units in a `long` | | Crashing on the first bad command | Catch domain exceptions in the driver loop | | Bonus before mandatory | Mandatory in order; bonus last | | `Random` without a seed | Seed it, or put randomness behind a Strategy | | `LocalDate.now()` in logic | Inject a `java.time.Clock` |

22.11 Glossary

Alphabetical. "Where" gives the chapter and topic where the term is explained in full. **Not repeated here** (see the tables above): the 23 GoF patterns and the extras (Simple Factory, Object Pool, Null Object, Dependency Injection, Repository, Specification, MVC, Event Sourcing) → 22.7; the five SOLID principles → 22.4; DRY, KISS, YAGNI, Law of Demeter, Tell Don't Ask, Program to an interface, Encapsulate what varies, Loose coupling, Hollywood Principle, Separation of Concerns, Least Astonishment, Fail fast, Design by Contract and the nine GRASP guidelines → 22.5; the 24 code smells → 22.6; the four pillars, overloading/overriding, interfaces vs abstract classes, the six relationships and the other one-line OOP facts → 22.3; the `java.util.concurrent` primitives, Coffman conditions, deadlock, livelock and starvation → 22.8.

Term	Plain meaning	Where
ABA problem	A value changes A → B → A, so a compare-and-swap cannot notice the change; fix with a version stamp (`AtomicStampedReference`)	Ch 12 · CAS & atomics
Activity diagram	UML flowchart of a process, with decisions, swimlanes and fork/join	Ch 3 · Activity diagrams
Actor	Someone or something that uses the system (customer, admin, the system itself)	Ch 1 · Requirements
Aggregate (DDD)	A cluster of objects changed as one unit; only its root is reachable from outside, and the root guards the invariants	Ch 5 · DDD
Anemic domain model	Classes with data but no behaviour; the logic lives in services that pull the data out	Ch 5 · Tell, Don't Ask
Anti-corruption layer	Translation layer at a bounded context's border that keeps a foreign or legacy model out of yours	Ch 5 · DDD
Anti-pattern	A common "solution" that reliably causes more problems than it solves	Ch 8 · Anti-patterns
Append-only log	Messages are only ever added at the end; readers track their own offset	Ch 20 · Pub-sub
Association class	A class that holds data about a link itself (e.g. `Enrollment{grade}`)	Ch 3 · Class diagrams
At-least-once delivery	A message may be delivered more than once, never lost; consumers must be idempotent	Ch 20 · Pub-sub
Atomic variable	`AtomicInteger` / `AtomicLong` / `AtomicReference`: lock-free single-variable updates using CAS	Ch 12 · CAS & atomics
Back-pressure	Slowing producers when consumers can't keep up, e.g. with a bounded queue	Ch 12 · Producer-consumer
`BigDecimal`	Exact decimal type used for money; build from strings, compare with `compareTo`	Ch 1 · Interview code, Ch 4 (money note)
Bloom filter	Probabilistic set: may say "maybe present" wrongly, never says "absent" wrongly	Ch 13 · Concurrent Bloom Filter
Bounded context	Boundary inside which one domain model and one vocabulary apply consistently (often one team or service)	Ch 5 · DDD
Boy Scout Rule	Leave the code a little cleaner than you found it	Ch 6 (intro)
Caretaker / Originator	Memento roles: the originator makes and restores snapshots; the caretaker stores them without looking inside	Ch 10 · Memento
CAS (compare-and-swap)	Atomic "set to new value only if it still equals the expected value"	Ch 12 · CAS & atomics
Characterisation test	A test that records what existing code does now, before you refactor it	Ch 7 · Full source code and output
Check-then-act	Read a condition, then act on it, as two steps: a race unless made atomic	Ch 12 · Race conditions, Concurrency in LLD interviews
Circular wait	T1 waits for T2's lock while T2 waits for T1's; one of the four deadlock conditions	Ch 12 · Deadlock
Class	A blueprint for objects: fields plus methods	Ch 2 · Classes & keywords
Class diagram	UML diagram of classes, fields, methods and relationships — "the answer" in an LLD round	Ch 3 · Class diagrams

Term	Plain meaning	Where
Class explosion	Subclasses multiplying across two dimensions (`WindowsButton`, `MacButton`, …)	Ch 9 · Bridge Pattern
Clean code	Code whose intent is obvious to the next reader and which is easy to change safely	Ch 6 (intro)
Code review	A peer reads a change before merge: correctness first, then design, readability, tests, security, performance	Ch 6 · Code review checklist
Code smell	A surface symptom that usually signals a deeper design problem; not a bug by itself	Ch 7 · Code smells
Command–Query Separation (CQS)	A method either changes state or returns a value, not both	Ch 6 · Functions
Compensation	Undoing a completed step with a new action (e.g. a refund) because money can't be rolled back	Ch 15 · BookMyShow
Composition root	The one place (usually `main`) where objects are created and wired together	Ch 6 · Systems
Concurrency vs parallelism	Many tasks in progress (structure) vs many executing at the same instant (hardware)	Ch 12 · Concurrency vs parallelism
Condition variable	Lets a thread wait until shared state meets a condition (`wait/notify`, `Condition`)	Ch 12 · Condition variables
Constructor chaining	One constructor calling another with `this(...)`	Ch 2 · Classes & keywords
Critical section	Code that touches shared state and must run in one thread at a time	Ch 12 · Race conditions
Dead-letter queue	Where messages go after retries are exhausted, for later inspection	Ch 20 · Pub-sub
Deep vs shallow copy	Copying nested objects too vs sharing them with the original	Ch 8 · Prototype
Derive, don't store	Compute counts and totals from the underlying records so they can't drift	Ch 18 · summary
Design pattern	A named, reusable solution to a recurring design problem	Ch 8 · What is a design pattern?
Director	Optional Builder role that runs a fixed sequence of building steps	Ch 8 · Builder
Domain event	Past-tense record of a business fact (`OrderPlaced`) that other parts of the system react to	Ch 5 · DDD
Domain-Driven Design (DDD)	Modelling software on the business: bounded contexts, entities, value objects, aggregates, domain events, repositories	Ch 5 · DDD
Double dispatch	Choosing a method by both the element's and the visitor's type (Visitor's `accept`)	Ch 11 · Visitor
Double-checked locking	Lazy singleton that checks, locks, checks again; the field must be `volatile`	Ch 8 · Singleton
Dynamic dispatch	The JVM picks an overridden method from the object's actual class at run time	Ch 2 · Polymorphism, Compile-time vs runtime polymorphism
Entity (DDD)	Object defined by an identity that survives changes; two entities are equal when their ids are equal	Ch 5 · DDD
Expression problem	Visitor makes adding operations cheap but adding types expensive (and the reverse for plain polymorphism)	Ch 11 · Visitor
Fairness	Serving waiting threads roughly in order so none starves; costs throughput	Ch 12 · Starvation

Term	Plain meaning	Where
F.I.R.S.T.	Clean tests are Fast, Independent, Repeatable, Self-validating, Timely	Ch 6 · Unit tests
Flag argument	A boolean parameter that makes a function do two things; split it instead	Ch 6 · Functions
Fork/Join	Split a task recursively into subtasks run in parallel, then combine results	Ch 12 · Choosing the tool, Ch 13 · Parallel merge sort
Fragile base class	A change in a parent class silently breaks subclasses	Ch 5 · Composition over inheritance
Functional interface	An interface with one abstract method, usable with lambdas	Ch 2 · Interfaces
Functional requirement	A behaviour the system must have ("a customer takes a ticket")	Ch 1 · Requirements
Glob	Filename wildcard pattern: `*` any characters, `?` one character, `[0-9]` one character from a set	Ch 21 · Unix File Search
God class / God Object	One class that knows and does almost everything	Ch 8 · Anti-patterns, Ch 7 · Code smells
Golden Hammer	Applying a favourite tool or pattern to every problem	Ch 8 · Anti-patterns
GRASP	Larman's nine guidelines for assigning responsibilities to classes	Ch 5 · GRASP
Guard clause	An early check-and-return/throw at the top of a method instead of nested `if`s	Ch 7 · Refactoring catalogue, Ch 5 · Fail fast
Happens-before	Java Memory Model rule guaranteeing one action's effects are visible to another	Ch 12 · Java Memory Model
HLD vs LLD	Many services and machines vs classes inside one application	Ch 1 · The interview
Hold with TTL	A temporary claim (e.g. seats during payment) that expires automatically	Ch 15 · BookMyShow
Idempotency	Running an operation twice has the same effect as once; an idempotency key enforces it	Ch 21 · Digital wallet, Ch 20 · Pub-sub
Immutability	Objects that never change after creation; safe to share between threads	Ch 2 · Immutability & records, Ch 5 · Immutability
Intrinsic vs extrinsic state	Flyweight's shared, immutable part vs the per-use part passed in by the caller	Ch 9 · Flyweight Pattern
Invariant	A fact about an object that must always be true ("balance is never negative")	Ch 5 · Design by Contract
Inversion of Control (IoC)	The framework controls the flow and calls your code; DI is one way to achieve it	Ch 5 · Hollywood Principle, Ch 4 · SOLID Q&A
Inverted index	A map from each word/tag to the list of items containing it, for search	Ch 18 · Stack Overflow
Invoker / Receiver	Command roles: the invoker triggers commands; the receiver does the real work	Ch 10 · Command
Java Memory Model (JMM)	Rules for when one thread's writes become visible to another	Ch 12 · Java Memory Model
Lock ordering	Always acquire locks in one global order to prevent circular wait	Ch 12 · Deadlock

Term	Plain meaning	Where
Lock striping	Guarding a map with an array of locks, each covering some buckets	Ch 13 · Striped concurrent map
LOOK scheduling	Elevator keeps moving in one direction while stops remain ahead, then reverses	Ch 14 · Elevator System
Lost wakeup	A signal sent before the waiter started waiting, so the waiter sleeps forever	Ch 12 · Condition variables
LRU cache	Evicts the least recently used entry; hash map + doubly linked list gives O(1)	Ch 20 · Cache / LRU
Machine-coding round	LLD round where working, runnable in-memory code is the answer (90–120 min)	Ch 1 · The interview, Ch 21 · Machine-coding round
Matching engine	Per-symbol order book matching buy and sell orders by best price, then oldest first	Ch 19 · Stock Brokerage
Multiplicity	How many objects take part at each end of a relationship (`1`, `0..1`, `*`, `1..*`)	Ch 3 · Association
Mutex	A lock that lets only one thread into a critical section	Ch 12 · Mutex and synchronized
Non-functional requirement	A quality: thread safety, extensibility, in-memory storage, latency	Ch 1 · Requirements
Object	One instance of a class with real values	Ch 2 · Classes & keywords
Optimistic locking	Read with a version; the write succeeds only if the version is unchanged	Ch 12 · Read-write locks, Concurrency in LLD interviews
`Optional`	A container that may be empty; return it when "not found" is normal	Ch 1 · Interview code
Out-of-scope list	Written list of what you will not build; shrinks the problem	Ch 1 · Requirements
Pinning	A virtual thread stuck on its carrier thread while it blocks	Ch 12 · Creating threads
PLU code	Four-digit price look-up code for loose produce, keyed at the till together with the scale weight	Ch 19 · Grocery Store
Poison pill	A special message telling a consumer to stop; one per consumer	Ch 12 · Producer-consumer
Producer–consumer	Threads hand work through a (bounded) queue	Ch 12 · Producer-consumer
Pub-sub	Observer through a broker: publishers and subscribers don't know each other	Ch 20 · Pub-sub
Qualified association	An association looked up by a key (`Library [isbn] → Book`, i.e. a `Map`)	Ch 3 · Class diagrams
Race condition	The result depends on the timing of threads touching shared mutable state	Ch 12 · Race conditions
Rate limiter	Caps how many requests a key may make per time window (token bucket, windows)	Ch 13 · Rate limiter, Ch 20 · Rate Limiter
Record	Compact Java class for immutable data; only shallowly immutable	Ch 2 · Immutability & records
Reentrancy	A thread may re-acquire a lock it already holds	Ch 12 · ReentrantLock and reentrancy

Term	Plain meaning	Where
Refactoring	Changing internal structure without changing observable behaviour	Ch 7 · Refactoring fundamentals
Requirement	One thing the system must do, written so you can check it	Ch 1 · Requirements
Role interface	A small interface serving one kind of caller (the ISP fix)	Ch 4 · Interface Segregation Principle
Rule of three	Refactor to an abstraction at the second or third case, not the first	Ch 4 · Over-engineering
Sealed type	A type whose permitted subtypes are fixed, enabling exhaustive `switch`	Ch 11 · Visitor
Sequence diagram	UML diagram of messages between objects over time	Ch 3 · Sequence diagrams
Service Locator	Anti-pattern where a class asks a registry for its dependencies, hiding them	Ch 11 · Beyond GoF
Sharding	Splitting data across independent parts (each with its own lock) to scale	Ch 20 · Cache / LRU
Signaling	One thread tells another that something has happened (semaphore, latch)	Ch 12 · Signaling pattern
Sliding window	Rate-limit algorithm counting requests in a moving time window (log or counter)	Ch 20 · Rate Limiter
SOLID	SRP, OCP, LSP, ISP, DIP: five guidelines that keep one change from spreading	Ch 4 · What SOLID is
Split tender	Paying one bill with several payment methods (e.g. part UPI, part cash)	Ch 19 · Grocery Store
Spurious wakeup	A waiting thread wakes with no signal; why `wait()` sits in a `while` loop	Ch 12 · Condition variables
State machine diagram	UML diagram of one object's states and transitions (`event [guard] / action`)	Ch 3 · State machine diagrams
`static`	Belongs to the class, one per class, not to each object	Ch 2 · Classes & keywords
Test double	Anything standing in for a real dependency in a test: dummy, stub, spy, mock, fake	Ch 6 · Testing strategies
Test-Driven Development (TDD)	Failing test → make it pass → refactor, in very short cycles	Ch 6 · Unit tests
Thread	An independent path of execution inside a process, sharing its memory	Ch 12 · Processes vs threads
Thread confinement	Keeping data inside one thread so it never needs locking	Ch 12 · Immutability & confinement
Thread pool	A fixed set of reusable threads fed from a task queue	Ch 12 · Thread pool pattern
Thread states	NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED	Ch 12 · Thread states
Thread-starvation deadlock	Tasks in a bounded pool wait for tasks queued behind them in the same pool	Ch 12 · Deadlock, Thread pool pattern
Token bucket	Rate-limit algorithm: tokens refill at a fixed rate; each request takes one; allows bursts	Ch 13 · Rate limiter
Two hats	While coding you are either adding function or refactoring, never both at once	Ch 7 · Refactoring fundamentals

Term	Plain meaning	Where
Ubiquitous language	One vocabulary shared by domain experts and the code (`place()`, not `setStatus(2)`)	Ch 5 · DDD
UML	Unified Modeling Language: 14 diagram types, 7 structural and 7 behavioural	Ch 3 · What UML is
Use case	One thing an actor can do with the system	Ch 1 · Requirements, Ch 3 · Use-case diagrams
Use-case diagram	UML diagram of actors, use cases and the system boundary («include», «extend»)	Ch 3 · Use-case diagrams
Value object	A small immutable object defined by its values (money, date range), usually a record	Ch 2 · Immutability & records, Ch 5 · DRY, Ch 5 · DDD
Virtual waiting room	FIFO queue that admits buyers gradually during a ticket rush	Ch 15 · Concert tickets

Self-check

- **Q:** Name the eleven framework steps in order. **A:** Clarify, requirements, use cases, entities, relationships, class diagram, key flows, patterns & SOLID, code, edge cases & concurrency, extensions.
- **Q:** Where does the diamond go, and what does filled mean? **A:** At the whole; filled = composition (the part dies with the whole).
- **Q:** A `switch (vehicleType)` computes fees. Which principle, smell and pattern? **A:** OCP; Repeated Switches (if it repeats); Strategy.
- **Q:** Decorator vs Proxy vs Chain of Responsibility? **A:** Decorator adds behaviour and every layer runs; Proxy controls access; in CoR one handler can stop the flow.
- **Q:** State vs Strategy? **A:** States know each other and advance themselves; strategies are chosen from outside.
- **Q:** Four Coffman conditions, and the usual fix? **A:** Mutual exclusion, hold and wait, no preemption, circular wait; global lock ordering.
- **Q:** Why is `volatile` not enough for `count++`? **A:** Visibility, not atomicity; use `AtomicInteger` or a lock.
- **Q:** The six machine-coding packages? **A:** model, repository, service, strategy, exception, driver.
- **Q:** What makes every booking problem safe? **A:** Naming the real unit of inventory and making "if free then take" one atomic step (plus holds that expire).

TL;DR

- **Framework:** clarify → requirements → use cases → entities → relationships → diagram → flows → patterns/SOLID → code → edge cases/concurrency → extensions.
- **Classes:** different data → field; different behaviour → class; one sentence per class; composition over inheritance.
- **SOLID** diagnoses change pain; always state the cost of the fix, and know when not to apply it.
- **Patterns** come from signals: Strategy (rules vary), Factory (type value → class), Observer (many listeners), State (lifecycle), Singleton (exactly one), Builder (many optional fields). Use one or two, name a rejected one.
- **Concurrency:** find the shared state, state the invariant, make check-then-act atomic with the smallest tool, lock in a fixed order, notify outside locks.
- **Machine coding:** run early and often, mandatory first, services return data, the driver prints and keeps going.